

المملكة العربية السعودية

الهيئة السعودية للملكية الفكرية (SAIP)

الإدارة العامة لحق المؤلف — إدارة تسجيل حقوق مصنفات الحاسب الآلي

وثيقة إيداع وتسجيل رسمي

وثيقة الكود المصدري الكامل للمصنف البرمجي

تطبيق الهاتف المحمول (حضر — Haderedu Mobile Application v1.0.0)

بيانات المصنف البرمجي الرسمية (Official Software Metadata)

اسم المصنف الرسمي:	حضر — Haderedu
المكون البرمجي المودع:	تطبيق الهاتف المحمول الموجه للمعلمين (Mobile Application - iOS & Android)
رقم الإصدار (Version):	Version 1.0.0 (Build 1.0.1+2)
تاريخ إنشاء / إصدار المصنف:	12-08-2026
صاحب الحق المالي:	مؤسسة مواهب المملكة
فريق التطوير البرمجي (المؤلفون):	عزة خالد محمد احمد ، حمدي صلاح كامل
لغات البرمجة وأطر العمل:	Dart Language (SDK ^3.11.1), Flutter Framework (Cross-Platform)
نمط المعمارية وإدارة الحالة:	Clean Architecture (Feature-First), BLoC Pattern (flutter_bloc 9.x), GetIt (DI), Dio REST Client
المنصات والبيئات المستهدفة:	Android (Google Play / Android SDK API 21+), iOS (Apple App Store / iOS 12.0+)
إجمالي عدد الملفات البرمجية:	383 ملف برمجي أصلي (يشمل كافة طبقات /lib وملف pubspec.yaml)
إجمالي عدد الأسطر البرمجية:	53,306 سطر برمجي موثق بالكامل
نطاق الحماية والتسجيل:	تسجيل حقوق المؤلف بموجب نظام حماية حقوق المؤلف الصادر بالمرسوم الملكي في المملكة العربية السعودية ولائحته التنفيذية.

إشعار وتنويه رسمي: تحتوي هذه الوثيقة على الكود المصدري الأصلي والكامل للتطبيق المذكور أعلاه وفق متطلبات الهيئة السعودية للملكية الفكرية (SAIP). تم فحص وتطهير الكود من أي مفاتيح اعتماد سرية أو بيانات وصول حساسة لضمان أعلى معايير الأمان المعلوماتي وحماية الملكية الفكرية.

1. نبذة عن التطبيق والوظائف التعليمية المبتكرة

يعد تطبيق "حضر — Haderedu" منصة برمجية ذكية وشاملة مخصصة لخدمة المعلمين والمعلمات في الميدان التعليمي بالمملكة العربية السعودية. يهدف التطبيق إلى رقمنة وأتمتة المهام اليومية للمعلم ورفع كفاءة التخطيط الدراسي والتدريس التفاعلي، حيث يقدم حلاً متكاملاً تشمل:

📌 تحضير الدروس الذكي والمخصص

إنشاء وتوليد خطط الدروس والتأخير المتوافقة مع معايير وزارة التعليم، وتخصيص استراتيجيات التدريس والوسائل والتقويم وتصديرها بصيغ Word و PDF.

📖 توزيع المناهج وتصفح الكتب المدرسية

استعراض وتنزيل المقررات الدراسية لكافة المراحل التعليمية (الابتدائية، المتوسطة، الثانوية)، ومتابعة الخطة الدراسية وتوزيع الأسابيع تلقائياً.

📊 العروض التقديمية التفاعلية (PPTX)

استعراض وتحميل عروض البوربوينت التفاعلية لكل درس مع دعم التخصيص والعرض المباشر داخل الفصول الدراسية.

📝 بنك الأسئلة وتوليد الاختبارات الآلي

توليد نماذج الاختبارات الدورية والنهائية ونماذج الإجابة وتوزيع الدرجات آلياً عبر بنك أسئلة مصنفة حسب مستويات بلوم المعرفية.

💳 الاشتراكات والدفع الإلكتروني ونظام الإحالة

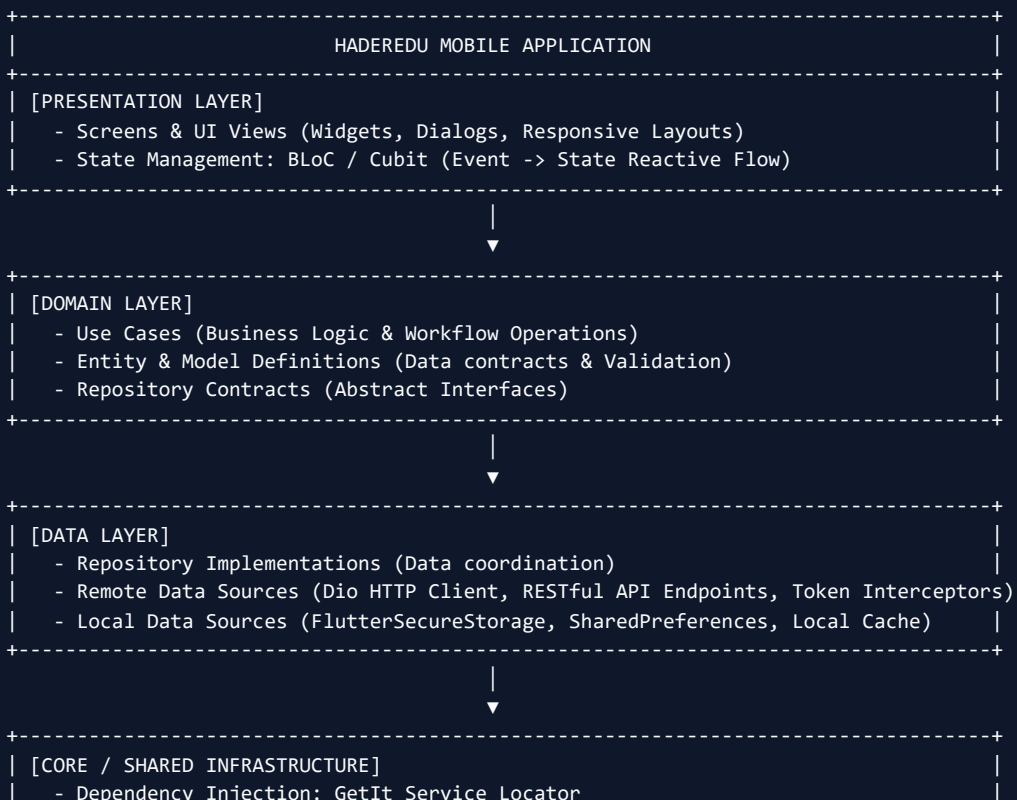
تكاملاً شاملاً مع بوابة الدفع الإلكتروني (MyFatoorah) لدعم مدى و Apple Pay، وإدارة باقات الاشتراك، ونظام إحالة متطور للمكافآت والخصومات.

📅 الجدول المدرسي وإدارة الحصص

تنظيم جدول الحصص الأسبوعي والتنبيه بمواعيد الدروس وإدارة سجلات التحضير اليومية ومتابعة المهام التعليمية.

2. المعمارية البرمجية وتصميم النظام (Software Architecture)

تم بناء التطبيق باتباع معمارية **Clean Architecture** القائمة على مبدأ فصل المسؤوليات (Separation of Concerns) وتنظيم الشيفرة بأسلوب **Feature-First** لضمان قابلية التوسع والصيانة والأمان وقابلية الاختبار:



- Error Handling: Failures, Exceptions, ApiErrorHandler	
- Theme & Design System: Colors, Typography, Dimension Constants	
- Utilities & Helpers: DateFormatter, Sanitizers, PDF/File Exporters	
+-----+	

3. إقرار وتعهد أصالة المصنف والملكية الفكرية

إقرار وتعهد أصالة العمل البرمجي:

يقر ويشهد كل من المؤلفين (فريق التطوير البرمجي) والممثل النظامي لصاحب الحق المالي (مؤسسة مواهب المملكة) بأن الشيفرة المصدرية (Source Code) المودعة في هذه الوثيقة هي ثمرة عملهم البرمجي والفكري الأصلي والمستقل، وأنه تم تطويرها بالكامل دون نقل أو اقتباس أو انتهاك لأي حقوق ملكية فكرية أو أسرار تجارية عائدة لأي طرف ثالث، وأن صاحب الحق المالي يمتلك كافة الحقوق الحصرية للاستغلال المالي والنظامي للمصنف.

صاحب الحق المالي: مؤسسة مواهب المملكة المفوض بالتوقيع والختم الرسمي:	فريق التطوير البرمجي (المؤلفون): عزة خالد محمد احمد ، حمدي صلاح كامل توقيع المؤلفين:
التاريخ: 2026-08-12	التاريخ: 2026-08-12

139	32	53,306	383
متوسط الأسطر / ملف	عدد الوحدات والنطاقات	إجمالي الأسطر البرمجية	إجمالي الملفات البرمجية

#	النطاق / الوحدة الوظيفية	مسار الملف البرمجي (File Path)	عدد الأسطر
1	Root / Package Configuration	pubspec.yaml	136
2	Application Entry Point	lib/main.dart	146
3	Core / Di	lib/core/di/injections.dart	143
4	Core / Errors	lib/core/errors/failures.dart	43
5	Core / Models	lib/core/models/admin_teacher_model.dart	186
6	Core / Models	lib/core/models/login_request.dart	15
7	Core / Models	lib/core/models/login_response.dart	41
8	Core / Models	lib/core/models/madrasati_session_data.dart	59
9	Core / Models	lib/core/models/profile_model.dart	38
10	Core / Models	lib/core/models/register_request.dart	34
11	Core / Models	lib/core/models/register_response.dart	23
12	Core / Models	lib/core/models/teacher_model.dart	112
13	Core / Models	lib/core/models/user_model.dart	72
14	Core / Network	lib/core/network/local/cache_helper.dart	32
15	Core / Network	lib/core/network/local/secure_storage_helper.dart	57
16	Core / Network	lib/core/network/remote/api_endpoints.dart	91
17	Core / Network	lib/core/network/remote/api_service.dart	1,002
18	Core / Network	lib/core/network/remote/dio_helper.dart	238
19	Core / Network	lib/core/network/remote/madrasati_session_interceptor.dart	277
20	Core / Services	lib/core/services/madrasati_session_service.dart	86
21	Core / Services	lib/core/services/referral_service.dart	109
22	Core / Theme	lib/core/theme/colors.dart	116
23	Core / Theme	lib/core/theme/text_styles.dart	99
24	Core / Theme	lib/core/theme/theme.dart	141
25	Core / Utils	lib/core/utils/constants/assets_helper.dart	42
26	Core / Utils	lib/core/utils/constants/constants.dart	59
27	Core / Utils	lib/core/utils/constants/image_helper.dart	72
28	Core / Utils	lib/core/utils/constants/my_bloc_observer.dart	29
29	Core / Utils	lib/core/utils/constants/primary/circle_indecator.dart	217
30	Core / Utils	lib/core/utils/constants/primary/conditional_builder.dart	48
31	Core / Utils	lib/core/utils/constants/primary/elvation_icon_button.dart	41
32	Core / Utils	lib/core/utils/constants/primary/no_data_widget.dart	60
33	Core / Utils	lib/core/utils/constants/primary/primary_elevated_button_label.dart	39
34	Core / Utils	lib/core/utils/constants/primary/primary_elevated_button.dart	87
35	Core / Utils	lib/core/utils/constants/primary/primary_text_field.dart	187

عدد الأسطر	مسار الملف البرمجي (File Path)	النطاق / الوحدة الوظيفية	#
127	lib/core/utils/constants/primary/primary_two_week_calendar.dart	Core / Utils	36
119	lib/core/utils/constants/primary/upgrade_prompt_bottom_sheet.dart	Core / Utils	37
273	lib/core/utils/constants/routes.dart	Core / Utils	38
8	lib/core/utils/constants/secrets.dart	Core / Utils	39
126	lib/core/utils/constants/spacing.dart	Core / Utils	40
9	lib/core/utils/constants/translations.dart	Core / Utils	41
799	lib/core/utils/cubit/auth/auth_cubit.dart	Core / Utils	42
98	lib/core/utils/cubit/auth/auth_state.dart	Core / Utils	43
71	lib/core/utils/cubit/theme/theme_cubit.dart	Core / Utils	44
18	lib/core/utils/cubit/theme/theme_state.dart	Core / Utils	45
43	lib/core/utils/extensions/context_extension.dart	Core / Utils	46
19	lib/core/utils/extensions/date_formatter.dart	Core / Utils	47
70	lib/core/utils/formatters/saudi_phone_input_formatter.dart	Core / Utils	48
77	lib/core/utils/pdf_export_helper.dart	Core / Utils	49
6	lib/core/utils/platform_url/platform_url_stub.dart	Core / Utils	50
7	lib/core/utils/platform_url/platform_url_web.dart	Core / Utils	51
3	lib/core/utils/platform_url/platform_url.dart	Core / Utils	52
52	lib/core/widgets/background_wrapper.dart	Core / Widgets	53
139	lib/core/widgets/custom_app_dialog.dart	Core / Widgets	54
36	lib/core/widgets/dot_gridPainter.dart	Core / Widgets	55
63	lib/core/widgets/logout_all_option.dart	Core / Widgets	56
144	lib/core/widgets/session_expired_dialog.dart	Core / Widgets	57
70	lib/core/widgets/session_monitor_wrapper.dart	Core / Widgets	58
213	lib/features/account_suspended/presentation/cubit/account_suspended_cubit.dart	Feature / Account Suspended	59
35	lib/features/account_suspended/presentation/cubit/account_suspended_state.dart	Feature / Account Suspended	60
108	lib/features/account_suspended/presentation/screen/account_suspended_screen.dart	Feature / Account Suspended	61
42	lib/features/account_suspended/presentation/widgets/suspended_badge_widget.dart	Feature / Account Suspended	62
95	lib/features/account_suspended/presentation/widgets/suspended_email_card_widget.dart	Feature / Account Suspended	63
26	lib/features/account_suspended/presentation/widgets/suspended_header_shield_icon_widget.dart	Feature / Account Suspended	64
78	lib/features/account_suspended/presentation/widgets/suspended_notice_card_widget.dart	Feature / Account Suspended	65
81	lib/features/account_suspended/presentation/widgets/suspended_status_check_widget.dart	Feature / Account Suspended	66
131	lib/features/account_suspended/presentation/widgets/suspended_ticket_form_card_widget.dart	Feature / Account Suspended	67
70	lib/features/account_suspended/presentation/widgets/suspended_ticket_type_dropdown_widget.dart	Feature / Account Suspended	68
91	lib/features/account_suspended/presentation/widgets/suspended_whatsapp_card_widget.dart	Feature / Account Suspended	69

#	النطاق / الوحدة الوظيفية	مسار الملف البرمجي (File Path)	عدد الأسطر
70	Feature / Account Suspended	lib/features/account_suspended/presentation/widgets/suspended_working_hou rs_card_widget.dart	92
71	Feature / Admin	lib/features/admin/contact/presentation/cubit/admin_contact_cubit.dart	126
72	Feature / Admin	lib/features/admin/contact/presentation/cubit/admin_contact_state.dart	82
73	Feature / Admin	lib/features/admin/contact/presentation/screen/admin_contact_screen.dart	47
74	Feature / Admin	lib/features/admin/contact/presentation/screen/admin_ticket_details_scre en.dart	122
75	Feature / Admin	lib/features/admin/contact/presentation/widgets/admin_contact_list_widge t.dart	160
76	Feature / Admin	lib/features/admin/contact/presentation/widgets/admin_contact_stats_widge t.dart	132
77	Feature / Admin	lib/features/admin/contact/presentation/widgets/admin_ticket_chat_histor y.dart	368
78	Feature / Admin	lib/features/admin/contact/presentation/widgets/admin_ticket_info_card.da rt	199
79	Feature / Admin	lib/features/admin/exams/data/datasources/admin_exam_remote_data_source.d art	68
80	Feature / Admin	lib/features/admin/exams/data/models/admin_exam_models.dart	90
81	Feature / Admin	lib/features/admin/exams/data/repositories/admin_exam_repository_impl.dar t	57
82	Feature / Admin	lib/features/admin/exams/domain/entities/admin_exam_entities.dart	52
83	Feature / Admin	lib/features/admin/exams/domain/repositories/admin_exam_repository.dart	19
84	Feature / Admin	lib/features/admin/exams/domain/usecases/get_pending_questions_usecase.da rt	25
85	Feature / Admin	lib/features/admin/exams/domain/usecases/review_question_usecase.dart	23
86	Feature / Admin	lib/features/admin/exams/presentation/cubit/admin_exams_cubit.dart	122
87	Feature / Admin	lib/features/admin/exams/presentation/cubit/admin_exams_state.dart	38
88	Feature / Admin	lib/features/admin/exams/presentation/screen/admin_exams_screen.dart	773
89	Feature / Admin	lib/features/admin/payments/data/repositories/admin_payments_repository.d art	82
90	Feature / Admin	lib/features/admin/payments/presentation/cubit/admin_payments_cubit.dart	145
91	Feature / Admin	lib/features/admin/payments/presentation/cubit/admin_payments_state.dart	27
92	Feature / Admin	lib/features/admin/payments/presentation/screen/admin_payments_screen.dar t	136
93	Feature / Admin	lib/features/admin/payments/presentation/widgets/admin_payments_search_fi lter_widget.dart	118
94	Feature / Admin	lib/features/admin/payments/presentation/widgets/admin_payments_table_wid get.dart	192
95	Feature / Admin	lib/features/admin/promo/data/models/admin_promo_models.dart	79
96	Feature / Admin	lib/features/admin/promo/presentation/cubit/admin_promo_cubit.dart	95
97	Feature / Admin	lib/features/admin/promo/presentation/cubit/admin_promo_state.dart	34
98	Feature / Admin	lib/features/admin/promo/presentation/screen/admin_promo_screen.dart	176
99	Feature / Admin	lib/features/admin/promo/presentation/widgets/admin_promo_body_widget.dar t	151
100	Feature / Admin	lib/features/admin/promo/presentation/widgets/admin_promo_create_dialog.d art	252

عدد الأسطر	مسار الملف البرمجي (File Path)	النطاق / الوحدة الوظيفية	#
69	lib/features/admin/promo/presentation/widgets/admin_promo_stat_card_widget.dart	Feature / Admin	101
194	lib/features/admin/promo/presentation/widgets/admin_promo_table_row_widget.dart	Feature / Admin	102
173	lib/features/admin/teachers/data/repositories/admin_teachers_repository.dart	Feature / Admin	103
224	lib/features/admin/teachers/presentation/cubit/admin_teachers_cubit.dart	Feature / Admin	104
44	lib/features/admin/teachers/presentation/cubit/admin_teachers_state.dart	Feature / Admin	105
239	lib/features/admin/teachers/presentation/screen/admin_teachers_screen.dart	Feature / Admin	106
116	lib/features/admin/teachers/presentation/widgets/admin_stats_cards_widget.dart	Feature / Admin	107
465	lib/features/admin/teachers/presentation/widgets/admin_teacher_action_dialogs.dart	Feature / Admin	108
106	lib/features/admin/teachers/presentation/widgets/admin_teachers_search_filter_widget.dart	Feature / Admin	109
800	lib/features/admin/teachers/presentation/widgets/admin_teachers_table_widget.dart	Feature / Admin	110
138	lib/features/certificates/data/models/certificate_template_model.dart	Feature / Certificates	111
27	lib/features/certificates/domain/entities/certificate_data.dart	Feature / Certificates	112
24	lib/features/certificates/domain/entities/certificate_entity.dart	Feature / Certificates	113
225	lib/features/certificates/presentation/cubit/certificate_cubit.dart	Feature / Certificates	114
45	lib/features/certificates/presentation/cubit/certificate_state.dart	Feature / Certificates	115
161	lib/features/certificates/presentation/screens/certificate_screen.dart	Feature / Certificates	116
35	lib/features/certificates/presentation/widgets/certificate_bg_widget.dart	Feature / Certificates	117
41	lib/features/certificates/presentation/widgets/certificate_dot_pattern_widget.dart	Feature / Certificates	118
178	lib/features/certificates/presentation/widgets/certificate_form_widget.dart	Feature / Certificates	119
86	lib/features/certificates/presentation/widgets/certificate_gender_tab_widget.dart	Feature / Certificates	120
114	lib/features/certificates/presentation/widgets/certificate_generate_button_widget.dart	Feature / Certificates	121
68	lib/features/certificates/presentation/widgets/certificate_logo_slot_widget.dart	Feature / Certificates	122
130	lib/features/certificates/presentation/widgets/certificate_preview_section_widget.dart	Feature / Certificates	123
520	lib/features/certificates/presentation/widgets/certificate_preview_widget.dart	Feature / Certificates	124
80	lib/features/certificates/presentation/widgets/certificate_ready_texts_widget.dart	Feature / Certificates	125
62	lib/features/certificates/presentation/widgets/certificate_signature_widget.dart	Feature / Certificates	126
309	lib/features/certificates/presentation/widgets/certificate_template_card_widget.dart	Feature / Certificates	127
93	lib/features/certificates/presentation/widgets/certificate_template_grid_widget.dart	Feature / Certificates	128
70	lib/features/certificates/presentation/widgets/templates/cert_bg_blue_academic.dart	Feature / Certificates	129

عدد الأسطر	مسار الملف البرمجي (File Path)	النطاق / الوحدة الوظيفية	#
110	lib/features/certificates/presentation/widgets/templates/cert_bg_luxury_frame.dart	Feature / Certificates	130
69	lib/features/certificates/presentation/widgets/templates/cert_bg_minimal_stamp.dart	Feature / Certificates	131
149	lib/features/certificates/presentation/widgets/templates/cert_bg_modern_corner.dart	Feature / Certificates	132
92	lib/features/certificates/presentation/widgets/templates/cert_bg_national_ribbon.dart	Feature / Certificates	133
101	lib/features/certificates/presentation/widgets/templates/cert_bg_royal_gold.dart	Feature / Certificates	134
108	lib/features/contact/presentation/cubit/contact_cubit.dart	Feature / Contact	135
69	lib/features/contact/presentation/cubit/contact_state.dart	Feature / Contact	136
158	lib/features/contact/presentation/screen/contact_support_screen.dart	Feature / Contact	137
328	lib/features/contact/presentation/screen/create_ticket_screen.dart	Feature / Contact	138
227	lib/features/contact/presentation/screen/my_tickets_screen.dart	Feature / Contact	139
98	lib/features/contact/presentation/screen/ticket_details_screen.dart	Feature / Contact	140
368	lib/features/contact/presentation/widgets/ticket_chat_history.dart	Feature / Contact	141
231	lib/features/curriculum/data/models/curriculum_distribution_models.dart	Feature / Curriculum	142
201	lib/features/curriculum/data/repositories/curriculum_repository.dart	Feature / Curriculum	143
40	lib/features/curriculum/presentation/cubit/curriculum_books_cubit.dart	Feature / Curriculum	144
31	lib/features/curriculum/presentation/cubit/curriculum_books_state.dart	Feature / Curriculum	145
98	lib/features/curriculum/presentation/cubit/curriculum_distribution_cubit.dart	Feature / Curriculum	146
41	lib/features/curriculum/presentation/cubit/curriculum_distribution_state.dart	Feature / Curriculum	147
389	lib/features/curriculum/presentation/screens/curriculum_books_screen.dart	Feature / Curriculum	148
248	lib/features/curriculum/presentation/screens/curriculum_distribution_screen.dart	Feature / Curriculum	149
62	lib/features/curriculum/presentation/widgets/curriculum_error_view.dart	Feature / Curriculum	150
126	lib/features/curriculum/presentation/widgets/export_pdf_card.dart	Feature / Curriculum	151
147	lib/features/curriculum/presentation/widgets/inline_books_section.dart	Feature / Curriculum	152
118	lib/features/curriculum/presentation/widgets/region_legend_row.dart	Feature / Curriculum	153
274	lib/features/curriculum/presentation/widgets/subject_picker_section.dart	Feature / Curriculum	154
339	lib/features/curriculum/presentation/widgets/weeks_grid.dart	Feature / Curriculum	155
82	lib/features/exam_generation/data/datasources/local_curriculum_provider.dart	Feature / Exam Generation	156
122	lib/features/exam_generation/data/models/curriculum_models.dart	Feature / Exam Generation	157
165	lib/features/exam_generation/data/models/exam_models.dart	Feature / Exam Generation	158
102	lib/features/exam_generation/data/models/exam_requests.dart	Feature / Exam Generation	159
293	lib/features/exam_generation/data/repositories/exam_repository_impl.dart	Feature / Exam Generation	160
370	lib/features/exam_generation/data/services/exam_pdf_service.dart	Feature / Exam Generation	161
124	lib/features/exam_generation/domain/entities/exam_entities.dart	Feature / Exam Generation	162
44	lib/features/exam_generation/domain/repositories/exam_repository.dart	Feature / Exam Generation	163

عدد الأسطر	مسار الملف البرمجي (File Path)	النطاق / الوحدة الوظيفية	#
15	lib/features/exam_generation/domain/usecases/add_manual_question_usecase.dart	Feature / Exam Generation	164
25	lib/features/exam_generation/domain/usecases/generate_exam_usecase.dart	Feature / Exam Generation	165
15	lib/features/exam_generation/domain/usecases/get_exam_usecase.dart	Feature / Exam Generation	166
31	lib/features/exam_generation/domain/usecases/get_my_questions_usecase.dart	Feature / Exam Generation	167
15	lib/features/exam_generation/domain/usecases/update_custom_question_usecase.dart	Feature / Exam Generation	168
15	lib/features/exam_generation/domain/usecases/update_question_points_usecase.dart	Feature / Exam Generation	169
158	lib/features/exam_generation/presentation/cubit/bank_questions_cubit.dart	Feature / Exam Generation	170
140	lib/features/exam_generation/presentation/cubit/custom_questions_cubit.dart	Feature / Exam Generation	171
55	lib/features/exam_generation/presentation/cubit/custom_questions_state.dart	Feature / Exam Generation	172
250	lib/features/exam_generation/presentation/cubit/exam_info_cubit.dart	Feature / Exam Generation	173
215	lib/features/exam_generation/presentation/cubit/exam_preview_cubit.dart	Feature / Exam Generation	174
91	lib/features/exam_generation/presentation/cubit/generate_exam_cubit.dart	Feature / Exam Generation	175
160	lib/features/exam_generation/presentation/cubit/lesson_selection_cubit.dart	Feature / Exam Generation	176
147	lib/features/exam_generation/presentation/cubit/manual_exam_cubit.dart	Feature / Exam Generation	177
25	lib/features/exam_generation/presentation/cubit/manual_exam_state.dart	Feature / Exam Generation	178
140	lib/features/exam_generation/presentation/cubit/my_exams_cubit.dart	Feature / Exam Generation	179
57	lib/features/exam_generation/presentation/cubit/my_exams_state.dart	Feature / Exam Generation	180
104	lib/features/exam_generation/presentation/cubit/question_count_cubit.dart	Feature / Exam Generation	181
399	lib/features/exam_generation/presentation/screens/bank_questions_screen.dart	Feature / Exam Generation	182
491	lib/features/exam_generation/presentation/screens/custom_questions_screen.dart	Feature / Exam Generation	183
462	lib/features/exam_generation/presentation/screens/exam_info_screen.dart	Feature / Exam Generation	184
507	lib/features/exam_generation/presentation/screens/exam_preview_screen.dart	Feature / Exam Generation	185
233	lib/features/exam_generation/presentation/screens/lesson_selection_screen.dart	Feature / Exam Generation	186
253	lib/features/exam_generation/presentation/screens/manual_exam_screen.dart	Feature / Exam Generation	187
282	lib/features/exam_generation/presentation/screens/my_exams_screen.dart	Feature / Exam Generation	188
486	lib/features/exam_generation/presentation/screens/question_counts_screen.dart	Feature / Exam Generation	189
374	lib/features/exam_generation/presentation/widgets/add_manual_question_dialog.dart	Feature / Exam Generation	190
209	lib/features/exam_generation/presentation/widgets/exam_list_item_widget.dart	Feature / Exam Generation	191
101	lib/features/exam_generation/presentation/widgets/generation_source_selector.dart	Feature / Exam Generation	192
252	lib/features/exam_generation/presentation/widgets/manual_exam_dynamic_form.dart	Feature / Exam Generation	193

عدد الأسطر	مسار الملف البرمجي (File Path)	النطاق / الوحدة الوظيفية	#
72	lib/features/exam_generation/presentation/widgets/manual_exam_type_selector.dart	Feature / Exam Generation	194
92	lib/features/exam_generation/presentation/widgets/my_exams_header_widget.dart	Feature / Exam Generation	195
89	lib/features/forgot_password/presentation/cubit/forgot_password_cubit.dart	Feature / Forgot Password	196
27	lib/features/forgot_password/presentation/cubit/forgot_password_state.dart	Feature / Forgot Password	197
93	lib/features/forgot_password/presentation/cubit/reset_password_cubit.dart	Feature / Forgot Password	198
31	lib/features/forgot_password/presentation/cubit/reset_password_state.dart	Feature / Forgot Password	199
90	lib/features/forgot_password/presentation/screen/forgot_password_screen.dart	Feature / Forgot Password	200
111	lib/features/forgot_password/presentation/screen/reset_password_screen.dart	Feature / Forgot Password	201
44	lib/features/forgot_password/presentation/widgets/forgot_password_badge_widget.dart	Feature / Forgot Password	202
79	lib/features/forgot_password/presentation/widgets/forgot_password_form_widget.dart	Feature / Forgot Password	203
35	lib/features/forgot_password/presentation/widgets/forgot_password_header_widget.dart	Feature / Forgot Password	204
104	lib/features/forgot_password/presentation/widgets/forgot_password_sent_widget.dart	Feature / Forgot Password	205
141	lib/features/forgot_password/presentation/widgets/reset_password_form_widget.dart	Feature / Forgot Password	206
40	lib/features/forgot_password/presentation/widgets/reset_password_header_widget.dart	Feature / Forgot Password	207
51	lib/features/forgot_password/presentation/widgets/reset_password_success_widget.dart	Feature / Forgot Password	208
105	lib/features/guest/presentation/screen/guest_screen.dart	Feature / Guest	209
165	lib/features/home/presentation/cubit/extension_install_cubit.dart	Feature / Home	210
26	lib/features/home/presentation/cubit/extension_install_state.dart	Feature / Home	211
68	lib/features/home/presentation/cubit/home_cubit.dart	Feature / Home	212
21	lib/features/home/presentation/cubit/home_state.dart	Feature / Home	213
322	lib/features/home/presentation/screen/home_screen.dart	Feature / Home	214
79	lib/features/home/presentation/widgets/chose_app_header.dart	Feature / Home	215
178	lib/features/home/presentation/widgets/chose_app_option_card.dart	Feature / Home	216
103	lib/features/home/presentation/widgets/chose_app_usage_card.dart	Feature / Home	217
270	lib/features/home/presentation/widgets/chose_app.dart	Feature / Home	218
263	lib/features/home/presentation/widgets/download_extention.dart	Feature / Home	219
89	lib/features/home/presentation/widgets/extension_ios_dialog.dart	Feature / Home	220
293	lib/features/home/presentation/widgets/extension_kiwi_missing_bottom_sheet.dart	Feature / Home	221
384	lib/features/home/presentation/widgets/extension_usage_slider.dart	Feature / Home	222
99	lib/features/home/presentation/widgets/home_action_chip_widget.dart	Feature / Home	223
352	lib/features/home/presentation/widgets/home_app_bar_widget.dart	Feature / Home	224
104	lib/features/home/presentation/widgets/home_feature_item_widget.dart	Feature / Home	225

عدد الأسطر	مسار الملف البرمجي (File Path)	النطاق / الوحدة الوظيفية	#
465	lib/features/home/presentation/widgets/home_features_section_widget.dart	Feature / Home	226
118	lib/features/home/presentation/widgets/home_hero_banner_widget.dart	Feature / Home	227
180	lib/features/home/presentation/widgets/home_search_section_widget.dart	Feature / Home	228
71	lib/features/home/presentation/widgets/home_tip_of_day_widget.dart	Feature / Home	229
38	lib/features/layout/presentation/cubit/layout_cubit.dart	Feature / Layout	230
6	lib/features/layout/presentation/cubit/layout_state.dart	Feature / Layout	231
105	lib/features/layout/presentation/screen/main_layout_screen.dart	Feature / Layout	232
130	lib/features/login/presentation/cubit/login_cubit.dart	Feature / Login	233
37	lib/features/login/presentation/cubit/login_state.dart	Feature / Login	234
187	lib/features/login/presentation/cubit/madrasati_cubit.dart	Feature / Login	235
116	lib/features/login/presentation/screen/login_screen.dart	Feature / Login	236
882	lib/features/login/presentation/screen/microsoft_login_screen.dart	Feature / Login	237
83	lib/features/login/presentation/widgets/login_action_buttons_widget.dart	Feature / Login	238
36	lib/features/login/presentation/widgets/login_footer_widget.dart	Feature / Login	239
109	lib/features/login/presentation/widgets/login_form_widget.dart	Feature / Login	240
82	lib/features/login/presentation/widgets/login_header_widget.dart	Feature / Login	241
70	lib/features/login/presentation/widgets/login_illustration_widget.dart	Feature / Login	242
69	lib/features/login/presentation/widgets/login_remember_forgot_widget.dart	Feature / Login	243
34	lib/features/payment/data/models/order_model.dart	Feature / Payment	244
82	lib/features/payment/data/models/payment_model.dart	Feature / Payment	245
112	lib/features/payment/data/models/subscription_current_model.dart	Feature / Payment	246
32	lib/features/payment/data/models/subscription_plan_model.dart	Feature / Payment	247
259	lib/features/payment/presentation/cubit/payment_cubit.dart	Feature / Payment	248
118	lib/features/payment/presentation/cubit/payment_state.dart	Feature / Payment	249
75	lib/features/payment/presentation/cubit/subscription_cubit.dart	Feature / Payment	250
37	lib/features/payment/presentation/cubit/subscription_state.dart	Feature / Payment	251
201	lib/features/payment/presentation/screen/bank_transfer_screen.dart	Feature / Payment	252
393	lib/features/payment/presentation/screen/checkout_screen.dart	Feature / Payment	253
255	lib/features/payment/presentation/screen/myfatoorah_payment_screen.dart	Feature / Payment	254
102	lib/features/payment/presentation/screen/payment_history_screen.dart	Feature / Payment	255
136	lib/features/payment/presentation/screen/payment_result_screen.dart	Feature / Payment	256
166	lib/features/payment/presentation/widgets/bank_info_widget.dart	Feature / Payment	257
218	lib/features/payment/presentation/widgets/moyasar_card_details_widget.dart	Feature / Payment	258
51	lib/features/payment/presentation/widgets/moyasar_card_name_widget.dart	Feature / Payment	259
32	lib/features/payment/presentation/widgets/moyasar_footer_widget.dart	Feature / Payment	260
50	lib/features/payment/presentation/widgets/moyasar_header_widget.dart	Feature / Payment	261
91	lib/features/payment/presentation/widgets/order_summary_widget.dart	Feature / Payment	262
181	lib/features/payment/presentation/widgets/payment_history_item_widget.dart	Feature / Payment	263
106	lib/features/payment/presentation/widgets/payment_method_tile_widget.dart	Feature / Payment	264

عدد الأسطر	مسار الملف البرمجي (File Path)	النطاق / الوحدة الوظيفية	#
72	lib/features/payment/presentation/widgets/plan_card_widget.dart	Feature / Payment	265
240	lib/features/payment/presentation/widgets/promo_code_section_widget.dart	Feature / Payment	266
108	lib/features/payment/presentation/widgets/receipt_upload_widget.dart	Feature / Payment	267
511	lib/features/payment/presentation/widgets/subscription_status_dialog.dart	Feature / Payment	268
188	lib/features/payment/presentation/widgets/trial_banner_widget copy.dart	Feature / Payment	269
186	lib/features/payment/presentation/widgets/trial_banner_widget.dart	Feature / Payment	270
125	lib/features/payment/presentation/widgets/visa_card_display.dart	Feature / Payment	271
79	lib/features/presentations/data/models/presentation_models.dart	Feature / Presentations	272
51	lib/features/presentations/data/repositories/presentations_repository.dart	Feature / Presentations	273
151	lib/features/presentations/domain/services/pptx_generator_service.dart	Feature / Presentations	274
304	lib/features/presentations/presentation/cubit/presentations_cubit.dart	Feature / Presentations	275
24	lib/features/presentations/presentation/cubit/presentations_state.dart	Feature / Presentations	276
1,111	lib/features/presentations/presentation/screens/presentation_preview_screen.dart	Feature / Presentations	277
532	lib/features/presentations/presentation/screens/presentations_screen.dart	Feature / Presentations	278
225	lib/features/privacy_policy/presentation/cubit/privacy_policy_cubit.dart	Feature / Privacy Policy	279
43	lib/features/privacy_policy/presentation/cubit/privacy_policy_state.dart	Feature / Privacy Policy	280
130	lib/features/privacy_policy/presentation/screen/privacy_policy_screen.dart	Feature / Privacy Policy	281
39	lib/features/privacy_policy/presentation/widgets/privacy_ai_check_item_widget.dart	Feature / Privacy Policy	282
43	lib/features/privacy_policy/presentation/widgets/privacy_cookie_bullet_widget.dart	Feature / Privacy Policy	283
63	lib/features/privacy_policy/presentation/widgets/privacy_data_collected_card_widget.dart	Feature / Privacy Policy	284
63	lib/features/privacy_policy/presentation/widgets/privacy_encryption_card_widget.dart	Feature / Privacy Policy	285
157	lib/features/privacy_policy/presentation/widgets/privacy_header_card_widget.dart	Feature / Privacy Policy	286
40	lib/features/privacy_policy/presentation/widgets/privacy_madrasati_guarantee_item_widget.dart	Feature / Privacy Policy	287
56	lib/features/privacy_policy/presentation/widgets/privacy_section_ai_usage_widget.dart	Feature / Privacy Policy	288
56	lib/features/privacy_policy/presentation/widgets/privacy_section_cookies_widget.dart	Feature / Privacy Policy	289
68	lib/features/privacy_policy/presentation/widgets/privacy_section_data_collected_widget.dart	Feature / Privacy Policy	290
62	lib/features/privacy_policy/presentation/widgets/privacy_section_encryption_widget.dart	Feature / Privacy Policy	291
78	lib/features/privacy_policy/presentation/widgets/privacy_section_header_widget.dart	Feature / Privacy Policy	292
71	lib/features/privacy_policy/presentation/widgets/privacy_section_intro_widget.dart	Feature / Privacy Policy	293
85	lib/features/privacy_policy/presentation/widgets/privacy_section_madrasati_widget.dart	Feature / Privacy Policy	294

عدد الأسطر	مسار الملف البرمجي (File Path)	النطاق / الوحدة الوظيفية	#
87	lib/features/privacy_policy/presentation/widgets/privacy_section_no_selling_widget.dart	Feature / Privacy Policy	295
59	lib/features/privacy_policy/presentation/widgets/privacy_section_teacher_rights_widget.dart	Feature / Privacy Policy	296
177	lib/features/privacy_policy/presentation/widgets/privacy_section_updates_contact_widget.dart	Feature / Privacy Policy	297
79	lib/features/privacy_policy/presentation/widgets/privacy_support_card_widget.dart	Feature / Privacy Policy	298
30	lib/features/privacy_policy/presentation/widgets/privacy_tag_badge_widget.dart	Feature / Privacy Policy	299
58	lib/features/privacy_policy/presentation/widgets/privacy_teacher_right_item_widget.dart	Feature / Privacy Policy	300
70	lib/features/privacy_policy/presentation/widgets/privacy_toc_item_widget.dart	Feature / Privacy Policy	301
133	lib/features/privacy_policy/presentation/widgets/privacy_toc_widget.dart	Feature / Privacy Policy	302
35	lib/features/profile/presentation/cubit/change_password_cubit.dart	Feature / Profile	303
14	lib/features/profile/presentation/cubit/change_password_state.dart	Feature / Profile	304
82	lib/features/profile/presentation/cubit/profile_cubit.dart	Feature / Profile	305
27	lib/features/profile/presentation/cubit/profile_state.dart	Feature / Profile	306
256	lib/features/profile/presentation/screen/change_password_screen.dart	Feature / Profile	307
212	lib/features/profile/presentation/screen/profile_screen.dart	Feature / Profile	308
181	lib/features/profile/presentation/screen/settings_screen.dart	Feature / Profile	309
62	lib/features/profile/presentation/widgets/logout_dialog.dart	Feature / Profile	310
69	lib/features/profile/presentation/widgets/profile_info_card.dart	Feature / Profile	311
129	lib/features/profile/presentation/widgets/subscription_info_card.dart	Feature / Profile	312
93	lib/features/referral/data/models/referral_dashboard_model.dart	Feature / Referral	313
31	lib/features/referral/presentation/cubit/referral_cubit.dart	Feature / Referral	314
18	lib/features/referral/presentation/cubit/referral_state.dart	Feature / Referral	315
97	lib/features/referral/presentation/screen/referral_screen.dart	Feature / Referral	316
161	lib/features/referral/presentation/widgets/referral_body_widget.dart	Feature / Referral	317
43	lib/features/referral/presentation/widgets/referral_empty_history_widget.dart	Feature / Referral	318
51	lib/features/referral/presentation/widgets/referral_footer_widget.dart	Feature / Referral	319
213	lib/features/referral/presentation/widgets/referral_hero_card_widget.dart	Feature / Referral	320
120	lib/features/referral/presentation/widgets/referral_history_item_widget.dart	Feature / Referral	321
91	lib/features/referral/presentation/widgets/referral_stat_card_widget.dart	Feature / Referral	322
116	lib/features/register/presentation/cubit/register_cubit.dart	Feature / Register	323
24	lib/features/register/presentation/cubit/register_state.dart	Feature / Register	324
129	lib/features/register/presentation/screen/register_screen.dart	Feature / Register	325
123	lib/features/register/presentation/widgets/register_account_type_widget.dart	Feature / Register	326
82	lib/features/register/presentation/widgets/register_action_buttons_widget.dart	Feature / Register	327
36	lib/features/register/presentation/widgets/register_footer_widget.dart	Feature / Register	328

عدد الأسطر	مسار الملف البرمجي (File Path)	النطاق / الوحدة الوظيفية	#
212	lib/features/register/presentation/widgets/register_form_widget.dart	Feature / Register	329
85	lib/features/register/presentation/widgets/register_header_widget.dart	Feature / Register	330
40	lib/features/register/presentation/widgets/register_illustration_widget.dart	Feature / Register	331
88	lib/features/register/presentation/widgets/register_terms_widget.dart	Feature / Register	332
551	lib/features/reports/data/report_pdf_service.dart	Feature / Reports	333
74	lib/features/reports/data/saved_report_model.dart	Feature / Reports	334
50	lib/features/reports/presentation/cubit/report_cubit.dart	Feature / Reports	335
45	lib/features/reports/presentation/cubit/report_state.dart	Feature / Reports	336
55	lib/features/reports/presentation/cubit/saved_reports_cubit.dart	Feature / Reports	337
42	lib/features/reports/presentation/cubit/saved_reports_state.dart	Feature / Reports	338
70	lib/features/reports/presentation/screen/pdf_preview_screen.dart	Feature / Reports	339
484	lib/features/reports/presentation/screen/report_screen.dart	Feature / Reports	340
187	lib/features/reports/presentation/screen/saved_report_details_screen.dart	Feature / Reports	341
153	lib/features/reports/presentation/screen/saved_reports_screen.dart	Feature / Reports	342
652	lib/features/reports/presentation/widgets/report_form_widget.dart	Feature / Reports	343
173	lib/features/reports/presentation/widgets/report_lessons_widget.dart	Feature / Reports	344
198	lib/features/reports/presentation/widgets/report_result_section_widget.dart	Feature / Reports	345
97	lib/features/reports/presentation/widgets/report_type_selector_widget.dart	Feature / Reports	346
186	lib/features/reports/presentation/widgets/saved_report_card_widget.dart	Feature / Reports	347
77	lib/features/reports/presentation/widgets/saved_reports_filter_widget.dart	Feature / Reports	348
116	lib/features/reports/presentation/widgets/saved_reports_header_widget.dart	Feature / Reports	349
263	lib/features/reports/presentation/widgets/selection_bottom_sheet.dart	Feature / Reports	350
68	lib/features/root/presentation/screen/root_screen.dart	Feature / Root	351
226	lib/features/root/presentation/widgets/custom_bottom_nav_bar.dart	Feature / Root	352
139	lib/features/schedule/data/models/schedule_models.dart	Feature / Schedule	353
437	lib/features/schedule/presentation/cubit/schedule_cubit.dart	Feature / Schedule	354
37	lib/features/schedule/presentation/cubit/schedule_state.dart	Feature / Schedule	355
209	lib/features/schedule/presentation/screen/schedule_screen.dart	Feature / Schedule	356
57	lib/features/schedule/presentation/widgets/action_button_widget.dart	Feature / Schedule	357
526	lib/features/schedule/presentation/widgets/class_card.dart	Feature / Schedule	358
81	lib/features/schedule/presentation/widgets/day_tab_item.dart	Feature / Schedule	359
89	lib/features/schedule/presentation/widgets/day_tabs_list.dart	Feature / Schedule	360
47	lib/features/schedule/presentation/widgets/schedule_app_bar.dart	Feature / Schedule	361
197	lib/features/schedule/presentation/widgets/schedule_bottom_sheet.dart	Feature / Schedule	362
32	lib/features/schedule/presentation/widgets/status_item_widget.dart	Feature / Schedule	363
77	lib/features/schedule/presentation/widgets/status_strip.dart	Feature / Schedule	364
56	lib/features/search/data/models/search_result_model.dart	Feature / Search	365

عدد الأسطر	مسار الملف البرمجي (File Path)	النطاق / الوحدة الوظيفية	#
68	lib/features/search/data/repositories/search_repository.dart	Feature / Search	366
74	lib/features/search/presentation/cubit/search_cubit.dart	Feature / Search	367
38	lib/features/search/presentation/cubit/search_state.dart	Feature / Search	368
90	lib/features/search/presentation/screen/search_screen.dart	Feature / Search	369
63	lib/features/search/presentation/widgets/search_empty_widget.dart	Feature / Search	370
99	lib/features/search/presentation/widgets/search_filter_chips_widget.dart	Feature / Search	371
97	lib/features/search/presentation/widgets/search_input_widget.dart	Feature / Search	372
182	lib/features/search/presentation/widgets/search_result_item_widget.dart	Feature / Search	373
186	lib/features/search/presentation/widgets/search_results_body_widget.dart	Feature / Search	374
162	lib/features/verify_email/presentation/cubit/verify_email_cubit.dart	Feature / Verify Email	375
25	lib/features/verify_email/presentation/cubit/verify_email_state.dart	Feature / Verify Email	376
115	lib/features/verify_email/presentation/screen/verify_email_screen.dart	Feature / Verify Email	377
137	lib/features/verify_email/presentation/widgets/email_verification_banner.dart	Feature / Verify Email	378
110	lib/features/verify_email/presentation/widgets/verify_email_actions_widget.dart	Feature / Verify Email	379
66	lib/features/verify_email/presentation/widgets/verify_email_badge_widget.dart	Feature / Verify Email	380
59	lib/features/verify_email/presentation/widgets/verify_email_header_widget.dart	Feature / Verify Email	381
68	lib/features/verify_email/presentation/widgets/verify_email_steps_widget.dart	Feature / Verify Email	382
134	lib/features/verify_email/presentation/widgets/verify_email_success_widget.dart	Feature / Verify Email	383

يتضمن هذا القسم الكود المصدري الأصلي والكامل لجميع ملفات التطبيق (383 ملفاً) بإجمالي (53,306 سطر)، مرتبة ومفهرسة بأرقام الأسطر والترويسات الرسمية.

Lines • YAML 136

Root / Package Configuration

#1 – pubspec.yaml

```
1  name: moean
2  description: "A new Flutter project."
3  # The following line prevents the package from being accidentally published to
4  # pub.dev using `flutter pub publish`. This is preferred for private packages.
5  publish_to: 'none' # Remove this line if you wish to publish to pub.dev
6
7  # The following defines the version and build number for your application.
8  # A version number is three numbers separated by dots, like 1.2.43
9  # followed by an optional build number separated by a +.
10 # Both the version and the builder number may be overridden in flutter
11 # build by specifying --build-name and --build-number, respectively.
12 # In Android, build-name is used as versionName while build-number used as versionCode.
13 # Read more about Android versioning at https://developer.android.com/studio/publish/versioning
14 # In iOS, build-name is used as CFBundleShortVersionString while build-number is used as CFBundleVersion.
15 # Read more about iOS versioning at
16 # https://developer.apple.com/library/archive/documentation/General/Reference/InfoPlistKeyReference/Articles/
CoreFoundationKeys.html
17 # In Windows, build-name is used as the major, minor, and patch parts
18 # of the product and file versions while build-number is used as the build suffix.
19 version: 1.0.1+2
20
21 environment:
22   sdk: ^3.11.1
23
24 # Dependencies specify other packages that your package needs in order to work.
25 # To automatically upgrade your package dependencies to the latest versions
26 # consider running `flutter pub upgrade --major-versions`. Alternatively,
27 # dependencies can be manually updated by changing the version numbers below to
28 # the latest version available on pub.dev. To see which dependencies have newer
29 # versions available, run `flutter pub outdated`.
30 dependencies:
31   flutter:
32     sdk: flutter
33   bloc: ^9.2.0
34   flutter_bloc: ^9.1.1
35   shared_preferences: ^2.5.4
36   get_it: ^9.2.1
37   equatable: ^2.0.8
38   flutter_svg: ^2.2.4
39   dartz: ^0.10.1
40   dio: ^5.9.2
41   flutter_launcher_icons: ^0.14.4
42   pininput: ^5.0.0
43   google_fonts: ^8.0.2
44   # The following adds the Cupertino Icons font to your application.
45   # Use with the CupertinoIcons class for iOS style icons.
46   cupertino_icons: ^1.0.8
47   lucide_icons_flutter: ^3.1.14+2
48   flutter_inappwebview: ^6.1.5
49   flutter_secure_storage: ^9.2.4
50   rounded_loading_button_plus: ^3.0.1
51   curved_navigation_bar: ^1.0.6
52   url_launcher: ^6.3.1
53   android_intent_plus: ^5.1.0
54   pdf: ^3.11.1
55   printing: ^5.11.1
56   path_provider: ^2.1.5
57   file_picker: ^8.1.7
58   screenshot: ^3.0.0
59   open_xml: ^0.3.1
```

```

60 file: ^7.0.1
61 share_plus: ^12.0.2
62 iconify_design: ^1.0.2
63 myfatoorah_flutter: ^3.3.2
64 flutter_credit_card: ^4.1.0
65 app_links: ^7.0.0
66 android_play_install_referrer: ^0.4.0
67
68 dev_dependencies:
69   flutter_test:
70     sdk: flutter
71
72   # The "flutter_lints" package below contains a set of recommended lints to
73   # encourage good coding practices. The lint set provided by the package is
74   # activated in the `analysis_options.yaml` file located at the root of your
75   # package. See that file for information about deactivating specific lint
76   # rules and activating additional ones.
77   flutter_lints: ^6.0.0
78
79 # For information on the generic Dart part of this file, see the
80 # following page: https://dart.dev/tools/pub/pubspecc
81
82 # The following section is specific to Flutter packages.
83 flutter:
84
85   # The following line ensures that the Material Icons font is
86   # included with your application, so that you can use the icons in
87   # the material Icons class.
88   uses-material-design: true
89
90   # To add assets to your application, add an assets section, like this:
91   # assets:
92   #   - images/a_dot_burr.jpeg
93   #   - images/a_dot_ham.jpeg
94
95   # An image asset can refer to one or more resolution-specific "variants", see
96   # https://flutter.dev/to/resolution-aware-images
97
98   # For details regarding adding assets from package dependencies, see
99   # https://flutter.dev/to/asset-from-package
100
101   # To add custom fonts to your application, add a fonts section here,
102   assets:
103     - assets/images/
104     - assets/translations/
105     - assets/fonts/
106     - assets/templates/
107     - assets/html/
108     - assets/js/
109
110   fonts:
111     - family: Tajawal
112       fonts:
113         - asset: assets/fonts/Tajawal-Regular.ttf
114           weight: 400
115         - asset: assets/fonts/Tajawal-Medium.ttf
116           weight: 500
117         - asset: assets/fonts/Tajawal-Bold.ttf
118           weight: 700
119
120     - family: Aldhabi
121       fonts:
122         - asset: assets/fonts/Aldhabi.ttf
123
124   # For details regarding fonts from package dependencies,
125   # see https://flutter.dev/to/font-from-package
126 flutter_launcher_icons:
127   android: "launcher_icon"
128   ios: true
129   adaptive_icon_background: "#016B5F"
130   adaptive_icon_foreground: "assets/images/logo_icon.png"
131   image_path: "assets/images/logo_icon.png"
132   min_sdk_android: 21 # android min sdk min:16, default 21
133   web:
134     generate: true
135     image_path: "assets/images/logo_icon.png"

```

```
135 background_color: "#016B5F"
136 theme_color: "#016B5F"
```

Lines • Dart 146

Application Entry Point

#2 – lib/main.dart

```
1 import 'package:flutter/material.dart';
2 import 'package:flutter/services.dart';
3 import 'package:flutter_bloc/flutter_bloc.dart';
4 import 'package:moean/core/di/injections.dart';
5 import 'package:moean/core/network/local/cache_helper.dart';
6 import 'package:moean/core/network/local/secure_storage_helper.dart';
7 import 'package:moean/core/network/remote/api_service.dart';
8 import 'package:moean/core/services/madrasati_session_service.dart';
9 import 'package:moean/core/theme/theme.dart';
10 import 'package:moean/core/utils/constants/routes.dart';
11 import 'package:moean/core/utils/constants/constants.dart';
12 import 'package:moean/core/utils/cubit/theme/theme_cubit.dart';
13 import 'package:moean/core/utils/cubit/theme/theme_state.dart';
14 import 'package:moean/core/widgets/session_monitor_wrapper.dart';
15 import 'dart:developer' as developer;
16 import 'package:moean/core/services/referral_service.dart';
17
18 final RouteObserver<ModalRoute> routeObserver = RouteObserver<ModalRoute>();
19 final GlobalKey<NavigatorState> navigatorKey = GlobalKey<NavigatorState>();
20
21 void main() async {
22   WidgetsFlutterBinding.ensureInitialized();
23
24   SystemChrome.setEnabledSystemUIMode(SystemUiMode.edgeToEdge);
25   SystemChrome.setSystemUIOverlayStyle(
26     const SystemUiOverlayStyle(
27       statusBarColor: Colors.transparent,
28       systemNavigationBarColor: Colors.transparent,
29     ),
30   );
31
32   await initInjections();
33
34   // Initialize ReferralService safely in the background
35   // We do not await it tightly so it doesn't block startup
36   ReferralService.init().catchError((e) {
37     developer.log('ReferralService init error: $e');
38   });
39
40   final bool isDark = CacheHelper.getData(key: 'isDark') ?? false;
41   final bool isArabic = CacheHelper.getData(key: 'isArabicLang') ?? true;
42
43   developer.log('Main: Loading translations...');
44   final String translation = await rootBundle.loadString(
45     'assets/translations/${isArabic ? 'ar' : 'en'}.json',
46   );
47   developer.log('Main: Translations loaded');
48
49   // — Restore token from secure storage —————
50   final secureStorage = sl<SecureStorageHelper>();
51   final String? savedToken = await secureStorage.getToken();
52
53   String initialRoute = Routes.login;
54
55   if (savedToken != null && savedToken.isNotEmpty) {
56     // Token found - verify it is still valid via /auth/me
57     token = savedToken;
58     developer.log('Main: Token found, verifying via /auth/me...');
59
60     final meResult = await ApiService.getMe();
61
62     meResult.fold(
63       (error) {
64         // 401 or network error - clear token
65         developer.log('Main: Token invalid ($error), clearing...');
66         token = null;
67         secureStorage.deleteToken();
68         initialRoute = Routes.home; // start at home
```

```

69     },
70     (user) {
71       developer.log('Main: Token valid, user=${user.name}');
72       if (user.email == 'admin@moeen.com' || user.email == 'admin@moeen.sa') {
73         initialRoute = Routes.adminTeachers;
74       } else {
75         initialRoute = Routes.home; // start at home
76       }
77       sl<MadrasatiSessionService>().notifySessionActive();
78     },
79   );
80 } else {
81   developer.log('Main: No token found, going to home');
82   initialRoute = Routes.home;
83 }
84
85 runApp(MyApp(
86   isDark: isDark,
87   isArabic: isArabic,
88   translation: translation,
89   initialRoute: initialRoute,
90 ));
91 }
92
93 class MyApp extends StatelessWidget {
94   final bool isDark;
95   final bool isArabic;
96   final String translation;
97   final String initialRoute;
98
99   const MyApp({
100     super.key,
101     required this.isDark,
102     required this.isArabic,
103     required this.translation,
104     required this.initialRoute,
105   });
106
107   @override
108   Widget build(BuildContext context) {
109     final themeCubit = sl<ThemeCubit>();
110     themeCubit
111       ..init(isDark: isDark)
112       ..initializeLanguage(isArabic: isArabic, translations: translation);
113
114     return MultiBlocProvider(
115       providers: [
116         BlocProvider(create: (context) => themeCubit),
117       ],
118       child: BlocBuilder<ThemeCubit, ThemeState>(
119         builder: (context, state) {
120           final themeCubit = context.read<ThemeCubit>();
121
122           return MaterialApp(
123             navigatorKey: navigatorKey,
124             debugShowCheckedModeBanner: false,
125             routes: Routes.routes,
126             initialRoute: initialRoute,
127             theme: ThemesManager.lightTheme,
128             darkTheme: ThemesManager.darkTheme,
129             themeMode:
130               themeCubit.isDarkMode ? ThemeMode.dark : ThemeMode.light,
131             builder: (context, child) {
132               return Directionality(
133                 textDirection: themeCubit.isArabicLang
134                   ? TextDirection.rtl
135                   : TextDirection.ltr,
136                 // SessionMonitorWrapper listens to session expiry globally
137                 child: SessionMonitorWrapper(child: child!),
138               );
139             },
140           );
141         },
142       ),
143     );

```

```
144 }
145 }
146 }
```

Lines • Dart 143

Core / Di

#3 – lib/core/di/injections.dart

```
1 import 'package:moean/core/network/local/secure_storage_helper.dart';
2 import 'package:moean/core/network/remote/api_endpoints.dart';
3 import 'package:moean/core/network/remote/madrasati_session_interceptor.dart';
4 import 'package:moean/core/services/madrasati_session_service.dart';
5 import 'package:moean/core/utils/cubit/theme/theme_cubit.dart';
6 import 'package:dio/dio.dart';
7 import 'package:flutter_secure_storage/flutter_secure_storage.dart';
8 import 'package:get_it/get_it.dart';
9 import 'package:shared_preferences/shared_preferences.dart';
10
11 import 'package:moean/features/exam_generation/domain/repositories/exam_repository.dart' as moean_exam;
12 import 'package:moean/features/exam_generation/data/repositories/exam_repository_impl.dart' as moean_exam;
13 import 'package:moean/features/exam_generation/domain/usecases/generate_exam_usecase.dart' as moean_exam;
14 import 'package:moean/features/exam_generation/domain/usecases/get_exam_usecase.dart' as moean_exam;
15 import 'package:moean/features/payment/presentation/cubit/subscription_cubit.dart';
16 import 'package:moean/features/exam_generation/domain/usecases/update_question_points_usecase.dart' as moean_exam;
17 import 'package:moean/features/exam_generation/domain/usecases/add_manual_question_usecase.dart' as moean_exam;
18 import 'package:moean/features/exam_generation/presentation/cubit/exam_info_cubit.dart' as moean_exam;
19 import 'package:moean/features/exam_generation/presentation/cubit/lesson_selection_cubit.dart' as moean_exam;
20 import 'package:moean/features/exam_generation/presentation/cubit/question_count_cubit.dart' as moean_exam;
21 import 'package:moean/features/exam_generation/presentation/cubit/generate_exam_cubit.dart' as moean_exam;
22 import 'package:moean/features/exam_generation/presentation/cubit/exam_preview_cubit.dart' as moean_exam;
23 import 'package:moean/features/exam_generation/presentation/cubit/my_exams_cubit.dart' as moean_exam;
24 import 'package:moean/features/exam_generation/presentation/cubit/manual_exam_cubit.dart' as moean_exam;
25 import 'package:moean/features/exam_generation/presentation/cubit/bank_questions_cubit.dart' as moean_exam;
26 import 'package:moean/features/exam_generation/domain/usecases/get_my_questions_usecase.dart' as moean_exam;
27 import 'package:moean/features/exam_generation/domain/usecases/update_custom_question_usecase.dart' as moean_exam;
28 import 'package:moean/features/exam_generation/presentation/cubit/custom_questions_cubit.dart' as moean_exam;
29
30 import 'package:moean/features/admin/exams/data/datasources/admin_exam_remote_data_source.dart';
31 import 'package:moean/features/admin/exams/data/repositories/admin_exam_repository_impl.dart';
32 import 'package:moean/features/admin/exams/domain/repositories/admin_exam_repository.dart';
33 import 'package:moean/features/admin/exams/domain/usecases/get_pending_questions_usecase.dart';
34 import 'package:moean/features/admin/exams/domain/usecases/review_question_usecase.dart';
35 import 'package:moean/features/admin/exams/presentation/cubit/admin_exams_cubit.dart';
36
37 import 'package:moean/features/profile/presentation/cubit/profile_cubit.dart';
38
39 final sl = GetIt.instance;
40
41 Future<void> initInjections() async {
42   // — Theme —————
43   sl.registerLazySingleton(() => ThemeCubit());
44
45   // — SharedPreferences (for non-sensitive prefs like theme) —
46   final sharedPref = await SharedPreferences.getInstance();
47   sl.registerLazySingleton(() => sharedPref);
48
49   // — Secure Storage —————
50   const androidOptions = AndroidOptions(encryptedSharedPreferences: true);
51   const secureStorage = FlutterSecureStorage(aOptions: androidOptions);
52   sl.registerLazySingleton<FlutterSecureStorage>(() => secureStorage);
53   sl.registerLazySingleton<SecureStorageHelper>(
54     () => SecureStorageHelper(sl<FlutterSecureStorage>()),
55   );
56
57   // — Session Service —————
58   sl.registerLazySingleton<MadrasatiSessionService>(
59     () => MadrasatiSessionService(),
60   );
61
62   // — Dio —————
63   // NOTE: Dio is registered first without the session interceptor,
64   // then the interceptor is added afterwards (because the interceptor
```



```

65 // itself needs the Dio instance to replay requests).
66 sl.registerLazySingleton<Dio>(() {
67     final dio = Dio(
68         BaseOptions(
69             baseUrl: baseUrl,
70             connectTimeout: const Duration(seconds: 30),
71             receiveTimeout: const Duration(seconds: 60),
72         ),
73     );
74
75 // — Language header interceptor —————
76 dio.interceptors.add(
77     InterceptorsWrapper(
78         onRequest: (options, handler) {
79             try {
80                 final isArabic = sl<ThemeCubit>().isArabicLang;
81                 options.headers['Accept-Language'] = isArabic ? 'ar' : 'en';
82             } catch (_) {}
83             return handler.next(options);
84         },
85     ),
86 );
87
88 // — Madrasati silent session refresh interceptor —————
89 // Uses QueuedInterceptor to ensure only one refresh request
90 // fires even if multiple API calls fail simultaneously.
91 dio.interceptors.add(
92     MadrasatiSessionInterceptor(
93         dio: dio,
94         secureStorage: sl<SecureStorageHelper>(),
95         sessionService: sl<MadrasatiSessionService>(),
96     ),
97 );
98
99 return dio;
100 });
101
102 // — Exam Generation —————
103 sl.registerLazySingleton<moean_exam.ExamRepository>(() => moean_exam.ExamRepositoryImpl());
104 sl.registerLazySingleton(() => moean_exam.GenerateExamUseCase(sl()));
105 sl.registerLazySingleton(() => moean_exam.GetExamUseCase(sl()));
106 sl.registerLazySingleton(() => moean_exam.UpdateQuestionPointsUseCase(sl()));
107 sl.registerLazySingleton(() => moean_exam.AddManualQuestionUseCase(sl()));
108 sl.registerLazySingleton(() => moean_exam.GetMyQuestionsUseCase(sl()));
109 sl.registerLazySingleton(() => moean_exam.UpdateCustomQuestionUseCase(sl()));
110
111 sl.registerLazySingleton(() => moean_exam.ExamInfoCubit());
112 sl.registerLazySingleton(() => moean_exam.LessonSelectionCubit());
113 sl.registerLazySingleton(() => moean_exam.QuestionCountCubit());
114 sl.registerFactory(() => moean_exam.GenerateExamCubit(sl()));
115 sl.registerFactory(() => moean_exam.ExamPreviewCubit(
116     getExamUseCase: sl(),
117     updateQuestionPointsUseCase: sl(),
118     examRepository: sl(),
119 ));
120 sl.registerFactory(() => moean_exam.MyExamsCubit(sl()));
121 sl.registerFactory(() => moean_exam.ManualExamCubit(sl()));
122 sl.registerFactory(() => moean_exam.BankQuestionsCubit());
123 sl.registerFactory(() => moean_exam.CustomQuestionsCubit(sl(), sl()));
124
125 // — Admin Exams —————
126 sl.registerLazySingleton<AdminExamRemoteDataSource>(
127     () => AdminExamRemoteDataSourceImpl());
128 sl.registerLazySingleton<AdminExamRepository>(
129     () => AdminExamRepositoryImpl(sl()));
130 sl.registerLazySingleton(() => GetPendingQuestionsUseCase(sl()));
131 sl.registerLazySingleton(() => ReviewQuestionUseCase(sl()));
132 sl.registerFactory(() => AdminExamsCubit(
133     getPendingQuestionsUseCase: sl(),
134     reviewQuestionUseCase: sl(),
135 ));
136
137 // — Subscription —————
138 sl.registerLazySingleton(() => SubscriptionCubit());
139

```

```
140 // — Profile —
141 sl.registerLazySingleton(() => ProfileCubit());
142 }
143
```

Lines • Dart 43

Core / Errors

#4 – lib/core/errors/failures.dart

```
1 abstract class Failure {
2   final String message;
3   const Failure(this.message);
4 }
5
6 class ValidationFailure extends Failure {
7   final Map<String, dynamic> errors;
8   const ValidationFailure(super.message, this.errors);
9 }
10
11 class UnauthorizedFailure extends Failure {
12   const UnauthorizedFailure(super.message);
13 }
14
15 class ForbiddenFailure extends Failure {
16   const ForbiddenFailure(super.message);
17 }
18
19 class NotFoundFailure extends Failure {
20   const NotFoundFailure(super.message);
21 }
22
23 class TimeoutFailure extends Failure {
24   const TimeoutFailure(super.message);
25 }
26
27 class ExamLockedFailure extends Failure {
28   const ExamLockedFailure(super.message);
29 }
30
31 class AiGenerationFailure extends Failure {
32   const AiGenerationFailure(super.message);
33 }
34
35 class PaymentRequiredFailure extends Failure {
36   final String code;
37   const PaymentRequiredFailure(super.message, {this.code = 'quota_exceeded'});
38 }
39
40 class ServerFailure extends Failure {
41   const ServerFailure(super.message);
42 }
43
```

Lines • Dart 186

Core / Models

#5 – lib/core/models/admin_teacher_model.dart

```
1 class AdminTeacherUserModel {
2   final int id;
3   final String name;
4   final String email;
5   final String? phone;
6   final String role;
7   final bool active;
8   final String? createdAt;
9
10   const AdminTeacherUserModel({
11     required this.id,
12     required this.name,
13     required this.email,
14     this.phone,
15     required this.role,
16     required this.active,
17     this.createdAt,
18   });
19
```

```

20     factory AdminTeacherUserModel.fromJson(Map<String, dynamic> json) {
21         return AdminTeacherUserModel(
22             id: json['id'] as int,
23             name: json['name'] as String,
24             email: json['email'] as String,
25             phone: json['phone'] as String?,
26             role: json['role'] as String? ?? 'teacher',
27             active: json['active'] == true || json['active'] == 1,
28             createdAt: json['created_at'] as String?,
29         );
30     }
31 }
32
33 class AdminTeacherSubscriptionModel {
34     final int id;
35     final String name;
36     final String slug;
37
38     const AdminTeacherSubscriptionModel({
39         required this.id,
40         required this.name,
41         required this.slug,
42     });
43
44     factory AdminTeacherSubscriptionModel.fromJson(Map<String, dynamic> json) {
45         return AdminTeacherSubscriptionModel(
46             id: json['id'] as int,
47             name: json['name'] as String,
48             slug: json['slug'] as String,
49         );
50     }
51 }
52
53 class AdminTeacherModel {
54     final int id;
55     final bool active;
56     final bool isInTrial;
57     final bool isSubscribed;
58     final String? subscriptionEndsAt;
59     final String? trialEndsAt;
60     final int trialDaysRemaining;
61     final AdminTeacherUserModel user;
62     final AdminTeacherSubscriptionModel? subscription;
63
64     const AdminTeacherModel({
65         required this.id,
66         required this.active,
67         this.isInTrial = false,
68         this.isSubscribed = false,
69         this.subscriptionEndsAt,
70         this.trialEndsAt,
71         this.trialDaysRemaining = 0,
72         required this.user,
73         this.subscription,
74     });
75
76     factory AdminTeacherModel.fromJson(Map<String, dynamic> json) {
77         return AdminTeacherModel(
78             id: json['id'] as int? ?? 0,
79             active: json['active'] == true || json['active'] == 1 || json['is_active'] == true || json['is_active']
== 1,
80             isInTrial: json['is_in_trial'] as bool? ?? false,
81             isSubscribed: json['is_subscribed'] as bool? ?? false,
82             subscriptionEndsAt: json['subscription_ends_at'] as String?,
83             trialEndsAt: json['trial_ends_at'] as String?,
84             trialDaysRemaining: json['trial_days_remaining'] as int? ?? 0,
85             user: json['user'] != null
86                 ? AdminTeacherUserModel.fromJson(json['user'] as Map<String, dynamic>)
87                 : const AdminTeacherUserModel(id: 0, name: '', email: '', role: 'teacher', active: true),
88             subscription: json['subscription'] != null
89                 ? AdminTeacherSubscriptionModel.fromJson(json['subscription'] as Map<String, dynamic>)
90                 : null,
91         );
92     }
93 }

```

```

94     DateTime? get parsedSubscriptionEndsAt =>
95         subscriptionEndsAt != null ? DateTime.TryParse(subscriptionEndsAt!) : null;
96
97     DateTime? get parsedTrialEndsAt =>
98         trialEndsAt != null ? DateTime.TryParse(trialEndsAt!) : null;
99
100    DateTime? get effectiveEndsAt {
101        if (isSubscribed) return parsedSubscriptionEndsAt;
102        if (isInTrial) return parsedTrialEndsAt;
103        return parsedSubscriptionEndsAt ?? parsedTrialEndsAt;
104    }
105
106    int get dynamicDaysRemaining {
107        final endsAt = effectiveEndsAt;
108        if (endsAt == null) return trialDaysRemaining > 0 ? trialDaysRemaining : 0;
109        final now = DateTime.now();
110        if (endsAt.isBefore(now)) return 0;
111        return (endsAt.difference(now).inHours / 24).ceil();
112    }
113
114    bool get isExpired {
115        if (isSubscribed) {
116            final ends = parsedSubscriptionEndsAt;
117            return ends != null && ends.isBefore(DateTime.now());
118        }
119        if (isInTrial) {
120            final ends = parsedTrialEndsAt;
121            if (ends != null && ends.isBefore(DateTime.now())) return true;
122            return dynamicDaysRemaining <= 0;
123        }
124        if (parsedTrialEndsAt != null && parsedTrialEndsAt!.isBefore(DateTime.now())) {
125            return true;
126        }
127        if (parsedSubscriptionEndsAt != null && parsedSubscriptionEndsAt!.isBefore(DateTime.now())) {
128            return true;
129        }
130        return false;
131    }
132
133    bool get isTrialExpired => !isSubscribed && (isExpired || (!isInTrial && parsedTrialEndsAt != null));
134
135    bool get isFreePlan =>
136        subscription == null ||
137        subscription?.slug == 'free' ||
138        subscription?.name == 'مجانى' ||
139        subscription?.name == 'الخطه المجانية';
140
141    String get planTitle {
142        if (isSubscribed && subscription != null && !isFreePlan) {
143            return subscription!.name;
144        }
145        if (isInTrial && !isExpired) {
146            return 'تجربة مجانية';
147        }
148        if (isTrialExpired) {
149            return 'انتهت التجربة';
150        }
151        if (subscription != null && !isFreePlan) {
152            return subscription!.name;
153        }
154        return 'مجانى';
155    }
156 }
157
158 class AdminTeacherPaginationModel {
159     final int currentPage;
160     final int lastPage;
161     final int perPage;
162     final int total;
163     final List<AdminTeacherModel> data;
164
165     const AdminTeacherPaginationModel({
166         required this.currentPage,
167         required this.lastPage,
168         required this.perPage,

```

```

169     required this.total,
170     required this.data,
171   });
172
173   factory AdminTeacherPagingModel.fromJson(Map<String, dynamic> json) {
174     return AdminTeacherPagingModel(
175       currentPage: json['current_page'] as int? ?? 1,
176       lastPage: json['last_page'] as int? ?? 1,
177       perPage: json['per_page'] as int? ?? 20,
178       total: json['total'] as int? ?? 0,
179       data: (json['data'] as List<dynamic>?)
180         ?.map((e) => AdminTeacherModel.fromJson(e as Map<String, dynamic>))
181         .toList() ??
182       [],
183     );
184   }
185 }
186

```

Lines • Dart 15

Core / Models

#6 – lib/core/models/login_request.dart

```

1  class LoginRequest {
2    final String email;
3    final String password;
4
5    LoginRequest({
6      required this.email,
7      required this.password,
8    });
9
10   Map<String, dynamic> toJson() => {
11     'email': email,
12     'password': password,
13   };
14 }
15

```

Lines • Dart 41

Core / Models

#7 – lib/core/models/login_response.dart

```

1  import 'package:moean/core/models/user_model.dart';
2  import 'package:moean/core/models/teacher_model.dart';
3
4  class LoginResponse {
5    final UserModel user;
6    final String token;
7    final TeacherModel? teacher;
8    final bool madrasatiConnected;
9
10   /// The Madrasati school ID extracted from the login response.
11   /// Populated when [madrasatiConnected] is true.
12   /// Needed for HeadlessWebView session refresh.
13   final String? madrasatiSchoolId;
14
15   LoginResponse({
16     required this.user,
17     required this.token,
18     this.teacher,
19     required this.madrasatiConnected,
20     this.madrasatiSchoolId,
21   });
22
23   factory LoginResponse.fromJson(Map<String, dynamic> json) {
24     // Extract school_id from the nested madrasati object
25     final madrasatiMap = json['madrasati'] as Map<String, dynamic>;
26     final schoolId = madrasatiMap['school_id'] as String? ??
27       madrasatiMap['real_school_id'] as String? ??
28       madrasatiMap['school_madrasati_id'] as String?;
29
30     return LoginResponse(
31       user: UserModel.fromJson(json['user']),
32       token: json['token'] as String,
33       teacher: json['teacher'] != null

```

```

34         ? TeacherModel.fromJson(json['teacher'])
35         : null,
36         madrasatiConnected: json['madrasati_connected'] as bool ?? false,
37         madrasatiSchoolId: schoolId,
38     );
39 }
40 }
41

```

Lines • Dart 59 Core / Models

#8 - lib/core/models/madrasati_session_data.dart

```

1  import 'dart:convert';
2
3  class MadrasatiSessionData {
4      final String sessionCookie;
5      final String schoolId;
6      final DateTime? expiresAt;
7
8      /// The Microsoft OAuth refresh_token used for silent session renewal.
9      /// Stored securely so the interceptor can use it without user interaction.
10     final String? refreshToken;
11
12     const MadrasatiSessionData({
13         required this.sessionCookie,
14         required this.schoolId,
15         this.expiresAt,
16         this.refreshToken,
17     });
18
19     Map<String, dynamic> toJson() => {
20         'session_cookie': sessionCookie,
21         'school_id': schoolId,
22         'expires_at': expiresAt?.toIso8601String(),
23         'refresh_token': refreshToken,
24     };
25
26     factory MadrasatiSessionData.fromJson(Map<String, dynamic> json) {
27         return MadrasatiSessionData(
28             sessionCookie: json['session_cookie'] as String? ?? '',
29             schoolId: json['school_id'] as String? ?? '',
30             expiresAt: json['expires_at'] != null
31                 ? DateTime.tryParse(json['expires_at'] as String)
32                 : null,
33             refreshToken: json['refresh_token'] as String?,
34         );
35     }
36
37     /// Encode to JSON string for secure storage
38     String encode() => jsonEncode(toJson());
39
40     /// Decode from JSON string
41     static MadrasatiSessionData? decode(String? raw) {
42         if (raw == null || raw.isEmpty) return null;
43         try {
44             final map = jsonDecode(raw) as Map<String, dynamic>;
45             return MadrasatiSessionData.fromJson(map);
46         } catch (_) {
47             return null;
48         }
49     }
50
51     bool get isExpired {
52         if (expiresAt == null) return false;
53         return DateTime.now().isAfter(expiresAt!);
54     }
55
56     bool get hasRefreshToken =>
57         refreshToken != null && refreshToken!.isNotEmpty;
58 }
59

```

```
1 import 'package:moean/core/models/user_model.dart';
2 import 'package:moean/core/models/teacher_model.dart';
3
4 class ProfileModel {
5   final UserModel user;
6   final bool madrasatiConnected;
7   final int? aiQuotaRemaining;
8   final String? subscriptionName;
9   final String? phone;
10  final String? role;
11  final TeacherModel? teacher;
12
13  ProfileModel({
14    required this.user,
15    required this.madrasatiConnected,
16    this.aiQuotaRemaining,
17    this.subscriptionName,
18    this.phone,
19    this.role,
20    this.teacher,
21  });
22
23  factory ProfileModel.fromJson(Map<String, dynamic> json) {
24    final userJson = json['user'] ?? json;
25    final teacherJson = json['teacher'];
26    final parsedTeacher = teacherJson != null ? TeacherModel.fromJson(teacherJson) : null;
27    return ProfileModel(
28      user: UserModel.fromJson(userJson),
29      madrasatiConnected: json['madrasati_connected'] as bool? ?? false,
30      aiQuotaRemaining: parsedTeacher?.aiQuotaRemaining ?? json['teacher']?['ai_quota_remaining'] as int?,
31      subscriptionName: parsedTeacher?.subscription?.name ?? json['teacher']?['subscription']?['name'] as Str
ing?,
32      phone: userJson['phone'] as String?,
33      role: userJson['role'] as String?,
34      teacher: parsedTeacher,
35    );
36  }
37 }
38
```

```
1 class RegisterRequest {
2   final String name;
3   final String email;
4   final String password;
5   final String passwordConfirmation;
6   final String phone;
7   final String? referralCode;
8
9   RegisterRequest({
10     required this.name,
11     required this.email,
12     required this.password,
13     required this.passwordConfirmation,
14     required this.phone,
15     this.referralCode,
16   });
17
18   Map<String, dynamic> toJson() {
19     final Map<String, dynamic> data = {
20       'name': name,
21       'email': email,
22       'password': password,
23       'password_confirmation': passwordConfirmation,
24       'phone': phone,
25     };
26
27     if (referralCode != null && referralCode!.isNotEmpty) {
28       data['referral_code'] = referralCode;
29     }
30   }
31 }
```

```

30
31     return data;
32 }
33 }
34

```

Lines • Dart 23 Core / Models

#11 – lib/core/models/register_response.dart

```

1  import 'package:moean/core/models/user_model.dart';
2  import 'package:moean/core/models/teacher_model.dart';
3
4  class RegisterResponse {
5      final UserModel user;
6      final String token;
7      final TeacherModel? teacher;
8
9      RegisterResponse({
10         required this.user,
11         required this.token,
12         this.teacher,
13     });
14
15     factory RegisterResponse.fromJson(Map<String, dynamic> json) {
16         return RegisterResponse(
17             user: UserModel.fromJson(json['user']),
18             token: json['token'] as String,
19             teacher: json['teacher'] != null ? TeacherModel.fromJson(json['teacher']) : null,
20         );
21     }
22 }
23

```

Lines • Dart 112 Core / Models

#12 – lib/core/models/teacher_model.dart

```

1  import 'package:moean/features/payment/data/models/subscription_plan_model.dart';
2
3  class TeacherModel {
4      final int id;
5      final int? userId;
6      final int? subscriptionId;
7      final bool canPrepareLesson;
8      final int aiQuotaRemaining;
9      final bool isInTrial;
10     final bool isSubscribed;
11     final DateTime? subscriptionEndsAt;
12     final DateTime? trialEndsAt;
13     final int trialDaysRemaining;
14     final SubscriptionPlanModel? subscription;
15
16     TeacherModel({
17         required this.id,
18         this.userId,
19         this.subscriptionId,
20         required this.canPrepareLesson,
21         required this.aiQuotaRemaining,
22         required this.isInTrial,
23         required this.isSubscribed,
24         this.subscriptionEndsAt,
25         this.trialEndsAt,
26         required this.trialDaysRemaining,
27         this.subscription,
28     });
29
30     factory TeacherModel.fromJson(Map<String, dynamic> json) {
31         return TeacherModel(
32             id: json['id'] as int? ?? 0,
33             userId: json['user_id'] as int?,
34             subscriptionId: json['subscription_id'] as int? ?? (json['subscription'] != null ? json['subscription']
35             ['id'] as int? : null),
36             canPrepareLesson: json['can_prepare_lesson'] as bool? ?? false,
37             aiQuotaRemaining: json['ai_quota_remaining'] as int? ?? 0,
38             isInTrial: json['is_in_trial'] as bool? ?? false,

```



```

38         isSubscribed: json['is_subscribed'] as bool? ?? false,
39         subscriptionEndsAt: json['subscription_ends_at'] != null
40             ? DateTime.tryParse(json['subscription_ends_at'])
41             : null,
42         trialEndsAt: json['trial_ends_at'] != null ? DateTime.tryParse(json['trial_ends_at']) : null,
43         trialDaysRemaining: json['trial_days_remaining'] as int? ?? 0,
44         subscription: json['subscription'] != null ? SubscriptionPlanModel.fromJson(json['subscription']) : nul
1,
45     );
46 }
47
48 /// تاريخ انتهاء الصلاحية الفعلي (سواء للاشتراك المدفوع أو التجربة)
49 DateTime? get effectiveEndsAt => isSubscribed ? subscriptionEndsAt : trialEndsAt;
50
51 /// حساب الأيام المتبقية ديناميكياً
52 int get dynamicDaysRemaining {
53     final endsAt = effectiveEndsAt;
54     if (endsAt == null) return 0;
55     final now = DateTime.now();
56     if (endsAt.isBefore(now)) return 0;
57     return (endsAt.difference(now).inHours / 24).ceil();
58 }
59
60 int get dynamicTrialDaysRemaining => dynamicDaysRemaining;
61
62 bool get isLastTrialDay => isInTrial && dynamicTrialDaysRemaining <= 1;
63
64 /// هل انتهت الصلاحية تماماً؟
65 bool get isExpired {
66     if (isSubscribed) {
67         return subscriptionEndsAt != null && subscriptionEndsAt!.isBefore(DateTime.now());
68     }
69     return !isInTrial || dynamicTrialDaysRemaining <= 0;
70 }
71
72 bool get isTrialExpired => !isSubscribed && !isInTrial;
73
74 /// عنوان الخطأ الحالي
75 String get planTitle {
76     if (isSubscribed) return subscription?.name ?? 'اشتراك مدفوع';
77     if (isInTrial) return 'تجربة مجانية (7 أيام)';
78     return 'انتهت التجربة';
79 }
80
81 /// نص موعد الانتهاء
82 String get expirationSubtitle {
83     final endsAt = effectiveEndsAt;
84     // ignore: avoid_print
85     print('[TeacherModel] expirationSubtitle → isSubscribed=$isSubscribed, isInTrial=$isInTrial, subscription
EndsAt=$subscriptionEndsAt, trialEndsAt=$trialEndsAt, effectiveEndsAt=$endsAt');
86     if (endsAt == null) return 'بدون تاريخ انتهاء';
87     final formattedDate = "${endsAt.year}/${endsAt.month.toString().padLeft(2, '0')}/${endsAt.day.toString().
padLeft(2, '0')}";
88
89     if (isSubscribed) {
90         return 'يتم يوم $dynamicDaysRemaining متبقي $formattedDate ينتهي في';
91     }
92     if (isInTrial) {
93         return 'يوم : "أب" ? 1 == $dynamicDaysRemaining متبقي $formattedDate تنتهي في'
م");';
94     }
95     return 'انتهت الصلاحية في $formattedDate';
96 }
97
98 Map<String, dynamic> toJson() => {
99     'id': id,
100     'user_id': userId,
101     'subscription_id': subscriptionId,
102     'can_prepare_lesson': canPrepareLesson,
103     'ai_quota_remaining': aiQuotaRemaining,
104     'is_in_trial': isInTrial,
105     'is_subscribed': isSubscribed,
106     'subscription_ends_at': subscriptionEndsAt?.toIso8601String(),
107     'trial_ends_at': trialEndsAt?.toIso8601String(),
108     'trial_days_remaining': trialDaysRemaining,

```

```
109     if (subscription != null) 'subscription': subscription,
110   };
111 }
112
```

Lines • Dart 72 Core / Models

#13 – lib/core/models/user_model.dart

```
1  class UserModel {
2    final int id;
3    final String name;
4    final String email;
5    final String? phone;
6    final String? role;
7    final bool isActive;
8    final bool isEmailVerified;
9    final DateTime? emailVerifiedAt;
10
11  const UserModel({
12    required this.id,
13    required this.name,
14    required this.email,
15    this.phone,
16    this.role,
17    this.isActive = true,
18    this.isEmailVerified = false,
19    this.emailVerifiedAt,
20  });
21
22  factory UserModel.fromJson(Map<String, dynamic> json) {
23    return UserModel(
24      id: json['id'] is int
25        ? json['id'] as int
26        : int.tryParse(json['id']?.toString() ?? '0') ?? 0,
27      name: json['name'] as String? ?? '',
28      email: json['email'] as String? ?? '',
29      phone: json['phone'] as String?,
30      role: json['role'] as String?,
31      isActive: json['is_active'] == true || json['is_active'] == 1 || json['is_active'] == null,
32      isEmailVerified: json['is_email_verified'] == true || json['is_email_verified'] == 1,
33      emailVerifiedAt: json['email_verified_at'] != null
34        ? DateTime.tryParse(json['email_verified_at'].toString())
35        : null,
36    );
37  }
38
39  Map<String, dynamic> toJson() => {
40    'id': id,
41    'name': name,
42    'email': email,
43    if (phone != null) 'phone': phone,
44    if (role != null) 'role': role,
45    'is_active': isActive,
46    'is_email_verified': isEmailVerified,
47    'email_verified_at': emailVerifiedAt?.toIso8601String(),
48  };
49
50  UserModel copyWith({
51    int? id,
52    String? name,
53    String? email,
54    String? phone,
55    String? role,
56    bool? isActive,
57    bool? isEmailVerified,
58    DateTime? emailVerifiedAt,
59  }) {
60    return UserModel(
61      id: id ?? this.id,
62      name: name ?? this.name,
63      email: email ?? this.email,
64      phone: phone ?? this.phone,
65      role: role ?? this.role,
66      isActive: isActive ?? this.isActive,
```

```

67     isEmailVerified: isEmailVerified ?? this.isEmailVerified,
68     emailVerifiedAt: emailVerifiedAt ?? this.emailVerifiedAt,
69   );
70 }
71 }
72

```

Lines • Dart 32 Core / Network

#14 – lib/core/network/local/cache_helper.dart

```

1  import 'package:shared_preferences/shared_preferences.dart';
2  import 'package:moean/core/di/injections.dart';
3
4  class CacheHelper {
5    static dynamic getData({required String key}) {
6      final prefs = sl<SharedPreferences>();
7      return prefs.get(key);
8    }
9
10   static Future<bool> saveData({
11     required String key,
12     required dynamic value,
13   }) async {
14     final prefs = sl<SharedPreferences>();
15
16     if (value is String) return await prefs.setString(key, value);
17     if (value is int) return await prefs.setInt(key, value);
18     if (value is bool) return await prefs.setBool(key, value);
19     if (value is double) return await prefs.setDouble(key, value);
20     if (value is List<String>) return await prefs.setStringList(key, value);
21
22     throw ArgumentError(
23       'Unsupported type for SharedPreferences: ${value.runtimeType}',
24     );
25   }
26
27   static Future<bool> removeData({required String key}) async {
28     final prefs = sl<SharedPreferences>();
29     return await prefs.remove(key);
30   }
31 }
32

```

Lines • Dart 57 Core / Network

#15 – lib/core/network/local/secure_storage_helper.dart

```

1  import 'package:flutter_secure_storage/flutter_secure_storage.dart';
2  import 'package:moean/core/models/madrasati_session_data.dart';
3
4  /// Secure storage keys
5  const _kAuthToken = 'secure_auth_token';
6  const _kMadrasatiSession = 'secure_madrasati_session';
7
8  /// A secure wrapper around [FlutterSecureStorage] for sensitive app data.
9  /// Use this for tokens and Madrasati session instead of [CacheHelper].
10 class SecureStorageHelper {
11   final FlutterSecureStorage _storage;
12
13   SecureStorageHelper(this._storage);
14
15   // ——— Auth Token ———
16
17   /// Saves the Bearer token securely.
18   Future<void> saveToken(String token) async {
19     await _storage.write(key: _kAuthToken, value: token);
20   }
21
22   /// Returns the saved Bearer token, or null if not found.
23   Future<String?> getToken() async {
24     return await _storage.read(key: _kAuthToken);
25   }
26
27   /// Deletes the saved token (used on logout / 401).
28   Future<void> deleteToken() async {

```

```

29     await _storage.delete(key: _kAuthToken);
30 }
31
32 // — Madrasati Session —————
33
34 /// Saves Madrasati school session data securely.
35 Future<void> saveMadrasatiSession(MadrasatiSessionData session) async {
36     await _storage.write(key: _kMadrasatiSession, value: session.encode());
37 }
38
39 /// Returns the saved Madrasati session, or null if not found.
40 Future<MadrasatiSessionData?> getMadrasatiSession() async {
41     final raw = await _storage.read(key: _kMadrasatiSession);
42     return MadrasatiSessionData.decode(raw);
43 }
44
45 /// Deletes the saved Madrasati session (used on disconnect).
46 Future<void> deleteMadrasatiSession() async {
47     await _storage.delete(key: _kMadrasatiSession);
48 }
49
50 // — Misc —————
51
52 /// Clears all secure data (logout).
53 Future<void> clearAll() async {
54     await _storage.deleteAll();
55 }
56 }
57

```

Lines • Dart 91

Core / Network

#16 – lib/core/network/remote/api_endpoints.dart

```

1 //const String baseUrl = 'http://192.168.1.13:8000/api';
2
3 const String baseUrl = 'https://api.haderedu.com/api';
4
5 const String registerApi = '/auth/register';
6 const String loginApi = '/auth/login';
7 const String meApi = '/auth/me';
8 const String logoutApi = '/auth/logout';
9 const String changePasswordApi = '/auth/password';
10 const String forgotPasswordApi = '/auth/forgot-password';
11 const String resetPasswordApi = '/auth/reset-password';
12 const String resendEmailVerificationApi = '/auth/email/resend-verification';
13 const String verifyEmailApi = '/auth/email/verify';
14 const String connectMadrasatiApi = '/madrasati/connect';
15 const String refreshMadrasatiSessionApi = '/madrasati/refresh-session';
16 const String scheduleApi = '/schedule';
17 const String madrasatiScheduleApi = '/madrasati/schedule';
18 const String prepareApi = '/prepare';
19
20 // Admin API
21 const String adminTeachersApi = '/admin/teachers';
22 const String subscriptionsApi = '/subscriptions';
23 const String adminContactStatsApi = '/admin/contact/stats';
24 const String adminContactApi = '/admin/contact';
25 const String adminPaymentsApi = '/admin/payments';
26 const String adminPendingQuestionsApi = '/admin/questions/pending-review';
27 String adminReviewQuestionApi(int id) => '/admin/questions/$id/review';
28
29 // Contact API
30 const String contactTypesApi = '/contact/types';
31 const String contactApi = '/contact';
32 const String contactMyApi = '/contact/my';
33
34 // Reports API
35 const String educationalReportApi = '/reports/educational/generate';
36 const String savedEducationalReportsApi = '/reports/educational';
37
38 // Payment & Subscription API
39 const String subscriptionCurrentApi = '/subscription/current';
40 const String ordersApi = '/orders';
41 const String paymentsVerifyApi = '/payments/verify';

```

```

42 const String paymentsSaveReferenceApi = '/payments/save-reference';
43 const String paymentsManualApi = '/payments/manual';
44 const String paymentsBankTransferInfoApi = '/payments/bank-transfer-info';
45 const String paymentsHistoryApi = '/payments/history';
46 const String paymentsMyfatoorahSessionApi = '/payments/myfatoorah/session';
47 const String paymentsMyfatoorahExecuteApi = '/payments/myfatoorah/execute';
48 const String paymentsMyfatoorahVerifyApi = '/payments/myfatoorah/verify';
49
50 // Exam Generation API
51 const String examsGenerateApi = '/exams/generate';
52 const String examsApi = '/exams';
53 const String questionsApi = '/questions';
54 const String questionsMyApi = '/questions/my';
55 String questionUpdateApi(int id) => '/questions/$id';
56 String examDetailsApi(int id) => '/exams/$id';
57 String examPointsApi(int id) => '/exams/$id/questions/points';
58 String examPublishApi(int id) => '/exams/$id/publish';
59
60 // Search API
61 const String searchApi = '/search';
62
63 // Curriculum API
64 const String subjectsApi = '/subjects';
65 String subjectLessonsApi(int id) => '/subjects/$id/lessons';
66 String lessonQuestionsApi(int id) => '/lessons/$id/questions';
67 String lessonPresentationApi(int id) => '/lessons/$id/presentation';
68 String lessonPresentationGenerateApi(int id) => '/lessons/$id/presentation/generate';
69
70 // Referral & Promo API
71 const String referralsMeApi = '/referrals/me';
72 const String promoValidateApi = '/promo-codes/validate';
73
74 // Admin Promo & Referral API
75 const String adminPromoCodesApi = '/admin/promo-codes';
76 const String adminReferralStatsApi = '/admin/referrals/statistics';
77 String adminPromoActivateApi(int id) => '/admin/promo-codes/$id/activate';
78 String adminPromoDeactivateApi(int id) => '/admin/promo-codes/$id/deactivate';
79 String adminPromoDeleteApi(int id) => '/admin/promo-codes/$id';
80
81 // Curriculum Distribution & Books API
82 const String curriculumPlansApi = '/curriculum/plans';
83 String curriculumPlanDetailApi(int id) => '/curriculum/plans/$id';
84 String curriculumPlanCurrentWeekApi(int id) => '/curriculum/plans/$id/weeks/current';
85 const String curriculumBooksApi = '/curriculum/books';
86 String curriculumBookDownloadApi(int id) => '/curriculum/books/$id/download';
87 const String curriculumProgressApi = '/curriculum/progress';
88 String curriculumPlanWeekPrepareApi(int planId, int weekId) =>
89     '/curriculum/plans/$planId/weeks/$weekId/prepare';
90
91

```

Lines • Dart 1002

Core / Network

#17 – lib/core/network/remote/api_service.dart

```

1 import 'dart:convert';
2 import 'package:dartz/dartz.dart';
3 import 'package:dio/dio.dart';
4 import 'package:moean/core/network/remote/api_endpoints.dart';
5 import 'package:moean/core/network/remote/dio_helper.dart';
6
7 import 'package:moean/core/models/login_request.dart';
8 import 'package:moean/features/exam_generation/data/models/curriculum_models.dart';
9 import 'package:moean/core/errors/failures.dart';
10 import 'package:moean/core/utils/constants/constants.dart';
11 import 'package:moean/core/models/register_request.dart';
12 import 'package:moean/core/models/register_response.dart';
13 import 'package:moean/core/models/login_response.dart';
14 import 'package:moean/core/models/user_model.dart';
15 import 'package:moean/core/models/profile_model.dart';
16
17 class ApiService {
18     static dynamic _decodeData(dynamic data) {
19         if (data is String) {
20             try {

```

```

21         return jsonDecode(data);
22     } catch (_) {
23         return data;
24     }
25 }
26 return data;
27 }
28
29 static Future<Either<String, RegisterResponse>> registerUser(RegisterRequest request) async {
30     final response = await DioHelper.postData(
31         url: registerApi,
32         data: request.toJson(),
33     );
34
35     return response.fold(
36         (error) => Left(error),
37         (res) => Right(RegisterResponse.fromJson(_decodeData(res.data))),
38     );
39 }
40
41 static Future<Either<String, LoginResponse>> loginUser(LoginRequest request) async {
42     final response = await DioHelper.postData(
43         url: loginApi,
44         data: request.toJson(),
45     );
46
47     return response.fold(
48         (error) => Left(error),
49         (res) => Right(LoginResponse.fromJson(_decodeData(res.data))),
50     );
51 }
52
53 static Future<Either<String, Map<String, dynamic>>> forgotPassword(String email) async {
54     final response = await DioHelper.postData(
55         url: forgotPasswordApi,
56         data: {
57             'email': email.trim(),
58         },
59     );
60
61     return response.fold(
62         (error) => Left(error),
63         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
64     );
65 }
66
67 static Future<Either<String, Map<String, dynamic>>> resetPassword({
68     required String email,
69     required String token,
70     required String password,
71     required String passwordConfirmation,
72 }) async {
73     final response = await DioHelper.postData(
74         url: resetPasswordApi,
75         data: {
76             'email': email.trim(),
77             'token': token.trim(),
78             'password': password,
79             'password_confirmation': passwordConfirmation,
80         },
81     );
82
83     return response.fold(
84         (error) => Left(error),
85         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
86     );
87 }
88
89 static Future<Either<String, UserModel>> getMe() async {
90     final response = await DioHelper.getData(
91         url: meApi,
92     );
93
94     return response.fold(
95         (error) => Left(error),

```

```

96     (res) {
97         final data = _decodeData(res.data);
98         return Right(UserModel.fromJson(data['user'] ?? data));
99     },
100 );
101 }
102
103 static Future<Either<String, Map<String, dynamic>>> resendEmailVerification({
104     String? email,
105 }) async {
106     final response = await DioHelper.postData(
107         url: resendEmailVerificationApi,
108         data: email != null && email.isNotEmpty ? {'email': email} : {},
109     );
110
111     return response.fold(
112         (error) => Left(error),
113         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
114     );
115 }
116
117 static Future<Either<String, UserModel>> verifyEmailToken({
118     required String id,
119     required String hash,
120     required String expires,
121     required String signature,
122 }) async {
123     final response = await DioHelper.postData(
124         url: verifyEmailApi,
125         data: {
126             'id': id,
127             'hash': hash,
128             'expires': expires,
129             'signature': signature,
130         },
131     );
132
133     return response.fold(
134         (error) => Left(error),
135         (res) {
136             final data = _decodeData(res.data);
137             return Right(UserModel.fromJson(data['user'] ?? data));
138         },
139     );
140 }
141
142 static Future<Either<String, ProfileModel>> getProfile() async {
143     final response = await DioHelper.getData(
144         url: meApi,
145     );
146
147     return response.fold(
148         (error) => Left(error),
149         (res) => Right(ProfileModel.fromJson(_decodeData(res.data))),
150     );
151 }
152
153 static Future<Either<String, bool>> logout() async {
154     final response = await DioHelper.postData(
155         url: logoutApi,
156     );
157
158     return response.fold(
159         (error) => Left(error),
160         (res) => const Right(true),
161     );
162 }
163
164 static Future<Either<String, bool>> changePassword({
165     required String currentPassword,
166     required String password,
167     required String passwordConfirmation,
168 }) async {
169     final response = await DioHelper.patchData(
170         url: changePasswordApi,

```

```

171     data: {
172         'current_password': currentPassword,
173         'password': password,
174         'password_confirmation': passwordConfirmation,
175     },
176 );
177
178 return response.fold(
179     (error) => Left(error),
180     (res) => const Right(true),
181 );
182 }
183
184 static Future<Either<String, Map<String, dynamic>>> getSchedule({
185     required String weekDate,
186 }) async {
187     /*
188     try {
189         // Using Dio directly to avoid base URL conflict in DioHelper for the absolute mock URL
190         final dio = Dio();
191         final response = await dio.get('https://n8n.qraura.shop/webhook/mock-schedule');
192
193         // Ensure we pass back a Map<String, dynamic>
194         final data = response.data;
195         if (data is String) {
196             return Right(jsonDecode(data) as Map<String, dynamic>);
197         }
198         return Right(data as Map<String, dynamic>);
199     } catch (e) {
200         return Left(e.toString());
201     }
202     */
203
204     final response = await DioHelper.getData(
205         url: scheduleApi,
206         query: {'week': weekDate},
207     );
208
209     return response.fold(
210         (error) => Left(error),
211         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
212     );
213 }
214
215 static Future<Either<String, Map<String, dynamic>>> syncSchedule({
216     required String weekDate,
217 }) async {
218     final response = await DioHelper.getData(
219         url: madrasatiScheduleApi,
220         query: {'week_date': weekDate},
221     );
222
223     return response.fold(
224         (error) => Left(error),
225         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
226     );
227 }
228
229 static Future<Either<String, Map<String, dynamic>>> prepareLesson({
230     required int lessonId,
231     required int subjectId,
232     required String classroomId,
233     required String schoolMadrasatiId,
234     required String timeTableId,
235     required List<String> selectedModules,
236     required String encryptedToken,
237 }) async {
238     final response = await DioHelper.postData(
239         url: prepareApi,
240         data: {
241             'lesson_id': lessonId,
242             'subject_id': subjectId,
243             'classroom_id': classroomId,
244             'school_madrasati_id': schoolMadrasatiId,
245             'time_table_id': timeTableId,

```



```

246         'selected_modules': selectedModules,
247         'encrypted_token': encryptedToken,
248     },
249 );
250
251 return response.fold(
252     (error) => Left(error),
253     (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
254 );
255 }
256
257 static Future<Either<String, Map<String, dynamic>>> getAvailableLessons({
258     required String weekDate,
259 }) async {
260     final response = await DioHelper.getData(
261         url: '$scheduleApi/available-lessons',
262         query: {'week': weekDate},
263     );
264
265     return response.fold(
266         (error) => Left(error),
267         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
268     );
269 }
270 static Future<Either<String, Map<String, dynamic>>> checkPreparationStatus({
271     required int preparationId,
272 }) async {
273     final response = await DioHelper.getData(
274         url: '$prepareApi/$preparationId/status',
275     );
276
277     return response.fold(
278         (error) => Left(error),
279         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
280     );
281 }
282
283 // --- Contact API (Teacher) ---
284
285 static Future<Either<String, Map<String, dynamic>>> getContactTypes() async {
286     final response = await DioHelper.getData(
287         url: contactTypesApi,
288     );
289
290     return response.fold(
291         (error) => Left(error),
292         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
293     );
294 }
295
296 static Future<Either<String, Map<String, dynamic>>> submitContactRequest({
297     required String name,
298     required String email,
299     String? phone,
300     required String type,
301     required String message,
302 }) async {
303     final response = await DioHelper.postData(
304         url: contactApi,
305         data: {
306             'name': name,
307             'email': email,
308             if (phone != null && phone.isNotEmpty) 'phone': phone,
309             'type': type,
310             'message': message,
311         },
312     );
313
314     return response.fold(
315         (error) => Left(error),
316         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
317     );
318 }
319
320 static Future<Either<String, Map<String, dynamic>>> getMyContactTickets() async {

```

```

321     final response = await DioHelper.getData(
322       url: contactMyApi,
323     );
324
325     return response.fold(
326       (error) => Left(error),
327       (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
328     );
329   }
330
331   static Future<Either<String, Map<String, dynamic>>> getMyContactTicketDetails({
332     required int id,
333   }) async {
334     final response = await DioHelper.getData(
335       url: '$contactMyApi/$id',
336     );
337
338     return response.fold(
339       (error) => Left(error),
340       (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
341     );
342   }
343
344   static Future<Either<String, Map<String, dynamic>>> replyContactTicket({
345     required int id,
346     required String body,
347   }) async {
348     final response = await DioHelper.postData(
349       url: '$contactMyApi/$id/reply',
350       data: {
351         'body': body,
352       },
353     );
354
355     return response.fold(
356       (error) => Left(error),
357       (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
358     );
359   }
360
361   // --- Admin Contact API ---
362
363   static Future<Either<String, Map<String, dynamic>>> getAdminContactStats() async {
364     final response = await DioHelper.getData(
365       url: adminContactStatsApi,
366     );
367
368     return response.fold(
369       (error) => Left(error),
370       (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
371     );
372   }
373
374   static Future<Either<String, Map<String, dynamic>>> getAdminContactTickets({
375     String? status,
376     String? type,
377     String? search,
378     int? hasUnread,
379     int perPage = 20,
380     int page = 1,
381   }) async {
382     final query = <String, dynamic>{
383       'per_page': perPage,
384       'page': page,
385     };
386     if (status != null && status.isNotEmpty) query['status'] = status;
387     if (type != null && type.isNotEmpty) query['type'] = type;
388     if (search != null && search.isNotEmpty) query['search'] = search;
389     if (hasUnread != null) query['has_unread'] = hasUnread;
390
391     final response = await DioHelper.getData(
392       url: adminContactApi,
393       query: query,
394     );
395

```

```

396     return response.fold(
397         (error) => Left(error),
398         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
399     );
400 }
401
402 static Future<Either<String, Map<String, dynamic>>> getAdminContactTicketDetails({
403     required int id,
404 }) async {
405     final response = await DioHelper.getData(
406         url: '$adminContactApi/$id',
407     );
408
409     return response.fold(
410         (error) => Left(error),
411         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
412     );
413 }
414
415 static Future<Either<String, Map<String, dynamic>>> updateAdminContactTicket({
416     required int id,
417     String? status,
418     String? adminNotes,
419 }) async {
420     final response = await DioHelper.patchData(
421         url: '$adminContactApi/$id',
422         data: {
423             if (status != null) 'status': status,
424             if (adminNotes != null) 'admin_notes': adminNotes,
425         },
426     );
427
428     return response.fold(
429         (error) => Left(error),
430         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
431     );
432 }
433
434 static Future<Either<String, Map<String, dynamic>>> replyAdminContactTicket({
435     required int id,
436     required String body,
437 }) async {
438     final response = await DioHelper.postData(
439         url: '$adminContactApi/$id/reply',
440         data: {
441             'body': body,
442         },
443     );
444
445     return response.fold(
446         (error) => Left(error),
447         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
448     );
449 }
450
451 static Future<Either<String, bool>> deleteAdminContactTicket({
452     required int id,
453 }) async {
454     final response = await DioHelper.deleteData(
455         url: '$adminContactApi/$id',
456     );
457
458     return response.fold(
459         (error) => Left(error),
460         (res) => const Right(true),
461     );
462 }
463
464 // --- Educational Reports API ---
465
466 static Future<Either<String, Map<String, dynamic>>> generateEducationalReport({
467     required String reportType,
468     required String grade,
469     required dynamic subject,
470     required List<String> selectedLessons,

```

```

471 }) async {
472     final response = await DioHelper.postData(
473         url: educationalReportApi,
474         data: {
475             'reportType': reportType,
476             'grade': grade,
477             'subject': subject,
478             'selectedLessons': selectedLessons,
479         },
480     );
481
482     return response.fold(
483         (error) => Left(error),
484         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
485     );
486 }
487
488 static Future<Either<String, Map<String, dynamic>>> getSavedEducationalReports({
489     String? reportType,
490     int page = 1,
491     int perPage = 15,
492 }) async {
493     final query = <String, dynamic>{
494         'page': page,
495         'per_page': perPage,
496     };
497     if (reportType != null && reportType.isNotEmpty) {
498         query['report_type'] = reportType;
499     }
500
501     final response = await DioHelper.getData(
502         url: savedEducationalReportsApi,
503         query: query,
504     );
505
506     return response.fold(
507         (error) => Left(error),
508         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
509     );
510 }
511
512 // --- Payment & Subscription API ---
513
514 static Future<Either<String, List<dynamic>>> getSubscriptions() async {
515     final response = await DioHelper.getData(url: subscriptionsApi);
516     return response.fold(
517         (error) => Left(error),
518         (res) => Right(_decodeData(res.data) as List<dynamic>),
519     );
520 }
521
522 static Future<Either<String, Map<String, dynamic>>> getCurrentSubscription() async {
523     final response = await DioHelper.getData(url: subscriptionCurrentApi);
524     return response.fold(
525         (error) => Left(error),
526         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
527     );
528 }
529
530 static Future<Either<String, Map<String, dynamic>>> upgradeSubscription({
531     required String planSlug,
532     String? promoCode,
533 }) async {
534     final data = <String, dynamic>{'plan_slug': planSlug};
535     if (promoCode != null && promoCode.isNotEmpty) {
536         data['promo_code'] = promoCode;
537     }
538     final response = await DioHelper.postData(
539         url: '$subscriptionsApi/upgrade', // POST /api/subscription/upgrade
540         data: data,
541     );
542     return response.fold(
543         (error) => Left(error),
544         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
545     );

```

```

546 }
547
548 static Future<Either<String, Map<String, dynamic>>> createOrder(
549     int serviceId, {
550     String? promoCode,
551 }) async {
552     final data = <String, dynamic>{'service_id': serviceId};
553     if (promoCode != null && promoCode.isNotEmpty) {
554         data['promo_code'] = promoCode;
555     }
556     final response = await DioHelper.postData(
557         url: ordersApi,
558         data: data,
559     );
560     return response.fold(
561         (error) => Left(error),
562         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
563     );
564 }
565
566 static Future<Either<String, Map<String, dynamic>>> getReferralDashboard() async {
567     final response = await DioHelper.getData(url: referralsMeApi);
568     return response.fold(
569         (error) => Left(error),
570         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
571     );
572 }
573
574 static Future<Either<String, Map<String, dynamic>>> validatePromoCode({
575     required String code,
576     required String planSlug,
577 }) async {
578     final response = await DioHelper.postData(
579         url: promoValidateApi,
580         data: {
581             'code': code,
582             'plan_slug': planSlug,
583         },
584     );
585     return response.fold(
586         (error) => Left(error),
587         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
588     );
589 }
590
591 static Future<Either<String, Map<String, dynamic>>> getOrderCheckout(int orderId) async {
592     final response = await DioHelper.postData(url: '$ordersApi/$orderId/pay');
593     return response.fold(
594         (error) => Left(error),
595         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
596     );
597 }
598
599 static Future<Either<String, Map<String, dynamic>>> createMyfatoorahSession(int orderId) async {
600     final response = await DioHelper.postData(
601         url: paymentsMyfatoorahSessionApi,
602         data: {'order_id': orderId},
603     );
604     return response.fold(
605         (error) => Left(error),
606         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
607     );
608 }
609
610 static Future<Either<String, Map<String, dynamic>>> executeMyfatoorahPayment({
611     required int orderId,
612     required String sessionId,
613 }) async {
614     final response = await DioHelper.postData(
615         url: paymentsMyfatoorahExecuteApi,
616         data: {
617             'order_id': orderId,
618             'session_id': sessionId,
619         },
620     );

```

```

621     return response.fold(
622         (error) => Left(error),
623         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
624     );
625 }
626
627 static Future<Either<String, Map<String, dynamic>>> verifyMyfatoorahPayment({
628     required String key,
629 }) async {
630     final response = await DioHelper.getData(
631         url: paymentsMyfatoorahVerifyApi,
632         query: {'key': key},
633     );
634     return response.fold(
635         (error) => Left(error),
636         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
637     );
638 }
639
640 static Future<Either<String, Map<String, dynamic>>> savePaymentReference({
641     required int orderId,
642     required String moyasarPaymentId,
643 }) async {
644     final response = await DioHelper.postData(
645         url: paymentsSaveReferenceApi,
646         data: {
647             'order_id': orderId,
648             'moyasar_payment_id': moyasarPaymentId,
649         },
650     );
651     return response.fold(
652         (error) => Left(error),
653         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
654     );
655 }
656
657 static Future<Either<String, Map<String, dynamic>>> verifyPayment(String paymentId) async {
658     final response = await DioHelper.getData(
659         url: paymentsVerifyApi,
660         query: {'id': paymentId},
661     );
662     return response.fold(
663         (error) => Left(error),
664         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
665     );
666 }
667
668 static Future<Either<String, Map<String, dynamic>>> getBankTransferInfo() async {
669     final response = await DioHelper.getData(url: paymentsBankTransferInfoApi);
670     return response.fold(
671         (error) => Left(error),
672         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
673     );
674 }
675
676 static Future<Either<String, Map<String, dynamic>>> uploadManualReceipt({
677     required int orderId,
678     required String filePath,
679     required String fileName,
680 }) async {
681     final formData = FormData.fromMap({
682         'order_id': orderId.toString(),
683         'receipt': await MultipartFile.fromFile(filePath, filename: fileName),
684     });
685     final response = await DioHelper.postData(
686         url: paymentsManualApi,
687         data: formData,
688     );
689     return response.fold(
690         (error) => Left(error),
691         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
692     );
693 }
694
695 static Future<Either<String, Map<String, dynamic>>> getPaymentHistory() async {

```

```

696     final response = await DioHelper.getData(url: paymentsHistoryApi);
697     return response.fold(
698       (error) => Left(error),
699       (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
700     );
701   }
702
703   // =====
704   // Curriculum APIs (Subjects & Lessons)
705   // =====
706
707   static Future<Either<Failure, List<CurriculumStageModel>>> getSubjects() async {
708     try {
709       final responseEither = await DioHelper.getData(url: subjectsApi);
710       return responseEither.fold(
711         (error) => Left(ServerFailure(error)),
712         (response) {
713           final List<dynamic> data = response.data is List ? response.data : (response.data['data'] ?? []);
714           return Right(data.map((e) => CurriculumStageModel.fromJson(e)).toList());
715         },
716       );
717     } catch (e) {
718       return Left(ServerFailure('An unexpected error occurred: $e'));
719     }
720   }
721
722   static Future<Either<Failure, SubjectDetailsModel>> getSubjectLessons(int subjectId) async {
723     try {
724       final responseEither = await DioHelper.getData(url: subjectLessonsApi(subjectId));
725       return responseEither.fold(
726         (error) => Left(ServerFailure(error)),
727         (response) {
728           final data = response.data;
729           if (data is Map<String, dynamic>) {
730             return Right(SubjectDetailsModel.fromJson(data));
731           }
732           return Left(ServerFailure('Invalid data format'));
733         },
734       );
735     } catch (e) {
736       return Left(ServerFailure('An unexpected error occurred: $e'));
737     }
738   }
739
740   // =====
741   // Exam Generation
742   // =====
743
744   static Future<Either<String, Map<String, dynamic>>> generateExam(Map<String, dynamic> request) async {
745     try {
746       final options = Options(
747         receiveTimeout: const Duration(seconds: 120),
748         sendTimeout: const Duration(seconds: 120),
749         headers: {
750           'Accept': 'application/json',
751           'Content-Type': 'application/json',
752           if (token != null && token!.isNotEmpty) 'Authorization': 'Bearer $token',
753         },
754       );
755
756       final response = await DioHelper.getDio().post(
757         examsGenerateApi,
758         data: request,
759         options: options,
760       );
761
762       return Right(_decodeData(response.data) as Map<String, dynamic>);
763     } on DioException catch (error) {
764       // Return the error so our new repository logic can parse it
765       return Left(DioHelper.parseError(error));
766       // Actually we will handle DioException in repository, but since api_service returns String, we'll let
767       api_service return error string.
768     } catch (e) {
769       return Left(e.toString());
770     }
771   }

```

```

770 }
771
772 static Future<Either<String, Map<String, dynamic>>> getExam(int id) async {
773   final response = await DioHelper.getData(
774     url: examDetailsApi(id),
775   );
776
777   return response.fold(
778     (error) => Left(error),
779     (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
780   );
781 }
782
783 static Future<Either<String, Map<String, dynamic>>> updateExamPoints(int id, Map<String, dynamic> request)
784 async {
785   final response = await DioHelper.patchData(
786     url: examPointsApi(id),
787     data: request,
788   );
789
790   return response.fold(
791     (error) => Left(error),
792     (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
793   );
794 }
795
796 static Future<Either<String, Map<String, dynamic>>> getMyExams({
797   int page = 1,
798   int perPage = 20,
799   String? status,
800   String? search,
801 }) async {
802   final query = <String, dynamic>{
803     'page': page,
804     'per_page': perPage,
805   };
806   if (status != null && status.isNotEmpty) query['status'] = status;
807   if (search != null && search.isNotEmpty) query['search'] = search;
808
809   final response = await DioHelper.getData(
810     url: examsApi,
811     query: query,
812   );
813
814   return response.fold(
815     (error) => Left(error),
816     (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
817   );
818 }
819
820 static Future<Either<String, Map<String, dynamic>>> publishExam(int id) async {
821   final response = await DioHelper.postData(
822     url: examPublishApi(id),
823   );
824
825   return response.fold(
826     (error) => Left(error),
827     (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
828   );
829 }
830
831 static Future<Either<String, bool>> deleteExam(int id) async {
832   final response = await DioHelper.deleteData(
833     url: examDetailsApi(id),
834   );
835
836   return response.fold(
837     (error) => Left(error),
838     (res) => const Right(true),
839   );
840 }
841
842 static Future<Either<String, Map<String, dynamic>>> getExams({
843   int page = 1,
844   int perPage = 20,

```



```

844     String? status,
845   }) async {
846     final query = <String, dynamic>{
847       'page': page,
848       'per_page': perPage,
849     };
850     if (status != null && status.isNotEmpty) query['status'] = status;
851
852     final response = await DioHelper.getData(
853       url: examsApi,
854       query: query,
855     );
856
857     return response.fold(
858       (error) => Left(error),
859       (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
860     );
861   }
862
863   static Future<Either<String, Map<String, dynamic>>> addManualQuestion(Map<String, dynamic> request) async {
864     final response = await DioHelper.postData(
865       url: questionsApi,
866       data: request,
867     );
868
869     return response.fold(
870       (error) => Left(error),
871       (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
872     );
873   }
874
875   static Future<Either<String, Map<String, dynamic>>> getLessonQuestions({
876     required int lessonId,
877     String? type,
878     String? difficulty,
879     int page = 1,
880     int perPage = 20,
881   }) async {
882     final query = <String, dynamic>{
883       'page': page,
884       'per_page': perPage,
885     };
886     if (type != null && type.isNotEmpty) query['type'] = type;
887     if (difficulty != null && difficulty.isNotEmpty) query['difficulty'] = difficulty;
888
889     final response = await DioHelper.getData(
890       url: lessonQuestionsApi(lessonId),
891       query: query,
892     );
893
894     return response.fold(
895       (error) => Left(error),
896       (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
897     );
898   }
899
900   static Future<Either<String, Map<String, dynamic>>> getMyQuestions({
901     int page = 1,
902     int perPage = 20,
903     String? type,
904     String? difficulty,
905     String? reviewStatus,
906     int? lessonId,
907     String? search,
908   }) async {
909     final query = <String, dynamic>{
910       'page': page,
911       'per_page': perPage,
912     };
913     if (type != null && type.isNotEmpty) query['type'] = type;
914     if (difficulty != null && difficulty.isNotEmpty) query['difficulty'] = difficulty;
915     if (reviewStatus != null && reviewStatus.isNotEmpty) query['review_status'] = reviewStatus;
916     if (lessonId != null) query['lesson_id'] = lessonId;
917     if (search != null && search.isNotEmpty) query['search'] = search;
918

```

```

919     final response = await DioHelper.getData(
920         url: questionsMyApi,
921         query: query,
922     );
923
924     return response.fold(
925         (error) => Left(error),
926         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
927     );
928 }
929
930 static Future<Either<String, Map<String, dynamic>>> updateCustomQuestion(int id, Map<String, dynamic> data)
931 async {
932     final response = await DioHelper.patchData(
933         url: questionUpdateApi(id),
934         data: data,
935     );
936
937     return response.fold(
938         (error) => Left(error),
939         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
940     );
941 }
942
943 // =====
944 // Admin Promo Codes & Referral Statistics
945 // =====
946
947 static Future<Either<String, Map<String, dynamic>>> getAdminReferralStats() async {
948     final response = await DioHelper.getData(url: adminReferralStatsApi);
949     return response.fold(
950         (error) => Left(error),
951         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
952     );
953 }
954
955 static Future<Either<String, Map<String, dynamic>>> getAdminPromoCodes({
956     int page = 1,
957 }) async {
958     final response = await DioHelper.getData(
959         url: adminPromoCodesApi,
960         query: {'page': page},
961     );
962     return response.fold(
963         (error) => Left(error),
964         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
965     );
966 }
967
968 static Future<Either<String, Map<String, dynamic>>> createAdminPromoCode(
969     Map<String, dynamic> data,
970 ) async {
971     final response = await DioHelper.postData(url: adminPromoCodesApi, data: data);
972     return response.fold(
973         (error) => Left(error),
974         (res) => Right(_decodeData(res.data) as Map<String, dynamic>),
975     );
976 }
977
978 static Future<Either<String, bool>> activateAdminPromoCode(int id) async {
979     final response = await DioHelper.postData(url: adminPromoActivateApi(id));
980     return response.fold(
981         (error) => Left(error),
982         (_) => const Right(true),
983     );
984 }
985
986 static Future<Either<String, bool>> deactivateAdminPromoCode(int id) async {
987     final response = await DioHelper.postData(url: adminPromoDeactivateApi(id));
988     return response.fold(
989         (error) => Left(error),
990         (_) => const Right(true),
991     );
992 }

```

```

993 static Future<Either<String, bool>> deleteAdminPromoCode(int id) async {
994   final response = await DioHelper.deleteData(url: adminPromoDeleteApi(id));
995   return response.fold(
996     (error) => Left(error),
997     (_) => const Right(true),
998   );
999 }
1000 }
1001
1002

```

Lines • Dart 238

Core / Network

#18 – lib/core/network/remote/dio_helper.dart

```

1 import 'package:moean/core/di/injections.dart';
2 import 'package:dartz/dartz.dart';
3 import 'package:dio/dio.dart';
4 import 'package:flutter/material.dart';
5 import 'package:moean/core/services/madrasati_session_service.dart';
6 import 'package:moean/core/utils/constants/constants.dart';
7
8 class DioHelper {
9   static Dio getDio() => sl<Dio>();
10
11   static Future<Either<String, Response>> getData({
12     required String url,
13     Map<String, dynamic>? query,
14     String? search,
15   }) async {
16     try {
17       debugPrint('🚀 GET Request: $url');
18       debugPrint('🔑 Sending Token: $token');
19
20       final Response response = await getDio().get(
21         url,
22         queryParameters: {
23           'q': ?search,
24           '...':?query,
25         },
26         options: Options(
27           headers: {
28             'Accept': 'application/json',
29             'Content-Type': 'application/json',
30             if (token?.isNotEmpty ?? false)
31               'Authorization': 'Bearer $token',
32           },
33         ),
34       );
35       return Right(response);
36     } on DioException catch (error) {
37       final msg = parseError(error);
38       if (error.response?.statusCode == 401 && (msg == 'Error 401' || msg == 'Unauthenticated')) {
39         return const Left('Unauthenticated');
40       }
41       return Left(msg);
42     } catch (e) {
43       return const Left('something went wrong');
44     }
45   }
46
47   static Future<Either<String, Response>> postData({
48     required String url,
49     dynamic data,
50     Map<String, dynamic>? query,
51   }) async {
52     try {
53       debugPrint('🚀 POST Request: $url');
54       debugPrint('🔑 Sending Token: $token');
55       final Response response = await getDio().post(
56         url,
57         data: data,
58         queryParameters: query,
59         options: Options(
60           headers: {

```

```

61         'Accept': 'application/json',
62         if (data is! FormData) 'Content-Type': 'application/json',
63         if (token != null && token!.isNotEmpty)
64             'Authorization': 'Bearer $token',
65     },
66 ),
67 );
68 return Right(response);
69 } on DioException catch (error) {
70     final msg = parseError(error);
71     if (error.response?.statusCode == 401 && (msg == 'Error 401' || msg == 'Unauthenticated')) {
72         return const Left('Unauthenticated');
73     }
74     return Left(msg);
75 } catch (e) {
76     return const Left('something went wrong');
77 }
78 }
79
80 static Future<Either<String, Response>> putData({
81     required String url,
82     dynamic data,
83     Map<String, dynamic>? query,
84 }) async {
85     try {
86         debugPrint('🚀 PUT Request: $url');
87         final Response response = await getDio().put(
88             url,
89             data: data,
90             queryParameters: query,
91             options: Options(
92                 headers: {
93                     'Accept': 'application/json',
94                     if (data is! FormData) 'Content-Type': 'application/json',
95                     if (token != null && token!.isNotEmpty)
96                         'Authorization': 'Bearer $token',
97                 },
98             ),
99         );
100     return Right(response);
101 } on DioException catch (error) {
102     final msg = parseError(error);
103     return Left(msg);
104 } catch (e) {
105     return const Left('something went wrong');
106 }
107 }
108
109 static Future<Either<String, Response>> patchData({
110     required String url,
111     dynamic data,
112     Map<String, dynamic>? query,
113 }) async {
114     try {
115         debugPrint('🚀 PATCH Request: $url');
116         final Response response = await getDio().patch(
117             url,
118             data: data,
119             queryParameters: query,
120             options: Options(
121                 headers: {
122                     'Accept': 'application/json',
123                     if (data is! FormData) 'Content-Type': 'application/json',
124                     if (token != null && token!.isNotEmpty)
125                         'Authorization': 'Bearer $token',
126                 },
127             ),
128         );
129     return Right(response);
130 } on DioException catch (error) {
131     final msg = parseError(error);
132     return Left(msg);
133 } catch (e) {
134     return const Left('something went wrong');
135 }

```

```

136 }
137
138 static Future<Either<String, Response>> deleteData({
139   required String url,
140 }) async {
141   try {
142     debugPrint('🚫 DELETE Request: $url');
143     final Response response = await getDio().delete(
144       url,
145       options: Options(
146         headers: {
147           'Accept': 'application/json',
148           'Content-Type': 'application/json',
149           if (token != null && token!.isNotEmpty)
150             'Authorization': 'Bearer $token',
151         },
152       ),
153     );
154     return Right(response);
155   } on DioException catch (error) {
156     final msg = parseError(error);
157     if (error.response?.statusCode == 401 && (msg == 'Error 401' || msg == 'Unauthenticated')) {
158       return const Left('Unauthenticated');
159     }
160     return Left(msg);
161   } catch (e) {
162     return const Left('something went wrong');
163   }
164 }
165
166 static String parseError(DioException error) {
167   final response = error.response;
168   if (response == null) return 'No response from server';
169
170   if (response.statusCode == 402 && response.data is Map) {
171     final map = response.data as Map;
172     final code = map['code']?.toString() ?? 'quota_exceeded';
173     final message = map['message']?.toString() ?? 'ترقية الحساب المطلوبة';
174     return '__402__:$code:$message';
175   }
176
177   if (response.data is Map) {
178     final map = response.data as Map;
179
180     // — Detect Madrasati session expiry from error code —————
181     final errorCode = map['code']?.toString() ?? '';
182     const sessionExpiredCodes = [
183       'madrasati_session_expired',
184       'madrasati_session_required',
185     ];
186     final fullMapStr = map.toString();
187     final isSessionExpired = sessionExpiredCodes.contains(errorCode) ||
188       fullMapStr.contains('الجلسة غير موجود في المدرسة') ||
189       fullMapStr.contains('أعد ربط حساب مدرستي');
190
191     if (isSessionExpired) {
192       debugPrint('🔴 DioHelper: detected session expired code or message');
193       try {
194         sl<MadrasatiSessionService>().notifySessionExpired();
195       } catch (_) {}
196     }
197     // —————
198
199     if (errorCode == 'account_suspended' ||
200       (response.statusCode == 403 && (errorCode == 'account_suspended' || fullMapStr.contains('تعليق') ||
201 fullMapStr.contains('موقوف')))) {
202       final message = map['message']?.toString() ?? 'account_suspended';
203       return '__403__:account_suspended:$message';
204     }
205
206     final errors = map['errors'];
207     if (errors is Map && errors.isNotEmpty) {
208       final buffer = <String>[];
209       errors.forEach((key, value) {
210         if (value is List && value.isNotEmpty) {

```

```

210         buffer.add(value.join(', '));
211     } else if (value != null) {
212         buffer.add(value.toString());
213     }
214     });
215     if (buffer.isNotEmpty) {
216         return buffer.join('\n');
217     }
218 }
219
220 final errorObj = map['error'];
221 if (errorObj is Map && errorObj['detail'] != null) {
222     final detailStr = errorObj['detail'].toString();
223     if (detailStr.isNotEmpty) {
224         return detailStr;
225     }
226 }
227
228 final message = map['message']?.toString();
229 if (message != null && message.isNotEmpty) {
230     return message;
231 }
232
233 return 'Error ${response.statusCode}: ${response.statusMessage}';
234 }
235 return 'Error: ${response.statusCode}: ${response.statusMessage}';
236 }
237 }
238

```

Lines • Dart 277

Core / Network

#19 – lib/core/network/remote/madrasati_session_interceptor.dart

```

1  import 'package:dio/dio.dart';
2  import 'package:flutter/foundation.dart';
3  import 'package:moean/core/models/madrasati_session_data.dart';
4  import 'package:moean/core/network/local/secure_storage_helper.dart';
5  import 'package:moean/core/network/remote/api_endpoints.dart';
6  import 'package:moean/core/services/madrasati_session_service.dart';
7
8  /// A [QueuedInterceptor] that silently refreshes the Madrasati session
9  /// whenever the backend returns a session-expired error.
10 ///
11 /// ## How it works
12 /// 1. Catches `DioException` with:
13 ///    - Status 401, OR
14 ///    - Status 422 with `code: "missing_madrasati_auth_cookie"`
15 /// 2. Reads the saved `refreshToken` and `schoolId` from [SecureStorageHelper].
16 /// 3. Calls `POST /api/madrasati/refresh-session` to get a new session.
17 /// 4. On success: saves the new `session_cookie` and `refresh_token`, then
18 ///    replays the original failed request transparently.
19 /// 5. On failure (refresh itself returns 401): clears stored credentials and
20 ///    fires `MadrasatiSessionService.notifySessionExpired()` to prompt the
21 ///    user to do a manual WebView login.
22 ///
23 /// Using [QueuedInterceptor] ensures that if multiple requests fail at the
24 /// same time, only ONE refresh call is made; the rest are queued and retried
25 /// once the refresh completes.
26 class MadrasatiSessionInterceptor extends QueuedInterceptor {
27     final Dio _dio;
28     final SecureStorageHelper _secureStorage;
29     final MadrasatiSessionService _sessionService;
30
31     MadrasatiSessionInterceptor({
32         required Dio dio,
33         required SecureStorageHelper secureStorage,
34         required MadrasatiSessionService sessionService,
35     }) : _dio = dio,
36         _secureStorage = secureStorage,
37         _sessionService = sessionService;
38
39     @override
40     Future<void> onError(
41         DioException err,

```

```

42     ErrorInterceptorHandler handler,
43 ) async {
44     // Skip: if this request is the refresh call itself (avoid infinite loop)
45     if (err.requestOptions.path.contains(refreshMadrasatiSessionApi)) {
46         debugPrint('🔴 [Interceptor] Skipping – request is the refresh call itself');
47         return handler.next(err);
48     }
49
50     final response = err.response;
51     final bool isMadrasatiSessionError = _isMadrasatiSessionError(response);
52
53     debugPrint('—————');
54     debugPrint('🔵 [MadrasatiSessionInterceptor] onError() triggered');
55     debugPrint('    failed request : ${err.requestOptions.method} ${err.requestOptions.path}');
56     debugPrint('    statusCode      : ${response?.statusCode}');
57     debugPrint('    isMadrasatiErr    : $isMadrasatiSessionError');
58     if (response?.data != null) {
59         debugPrint('    error body       : ${response?.data}');
60     }
61     debugPrint('—————');
62
63     if (!isMadrasatiSessionError) {
64         debugPrint('🔴 [Interceptor] Not a Madrasati session error – passing through');
65         return handler.next(err);
66     }
67
68     debugPrint('🔵 [Interceptor] Madrasati session error detected – attempting silent refresh...');
69
70     try {
71         // 1. Read saved credentials
72         debugPrint('');
73         debugPrint('📁 [Interceptor] Step 1 – Reading saved credentials from SecureStorage...');
74         final savedSession = await _secureStorage.getMadrasatiSession();
75         final refreshToken = savedSession?.refreshToken;
76         final schoolId = savedSession?.schoolId;
77
78         debugPrint('    schoolId      : ${schoolId ?? '❌ null'}');
79         debugPrint('    refreshToken : ${refreshToken != null && refreshToken.isNotEmpty ? '✅ "${refreshToken.substring(0, refreshToken.length.clamp(0, 30))}"' : '❌ null or empty'}');
80
81         if (refreshToken == null ||
82             refreshToken.isEmpty ||
83             schoolId == null ||
84             schoolId.isEmpty) {
85             debugPrint('🚨 [Interceptor] No refresh_token or schoolId – cannot silent refresh → notifySessionExpired()');
86             _sessionService.notifySessionExpired();
87             return handler.next(err);
88         }
89
90         // 2. Call the silent refresh endpoint
91         debugPrint('');
92         debugPrint('📡 [Interceptor] Step 2 – Calling POST $refreshMadrasatiSessionApi ...');
93         debugPrint('    payload: { refresh_token: "${refreshToken.substring(0, refreshToken.length.clamp(0, 20))}" , madrasati_school_id: "$schoolId" }');
94
95         final refreshResponse = await _dio.post(
96             refreshMadrasatiSessionApi,
97             data: {
98                 'refresh_token': refreshToken,
99                 'madrasati_school_id': schoolId,
100             },
101             options: Options(
102                 headers: {
103                     'Accept': 'application/json',
104                     'Content-Type': 'application/json',
105                 },
106             ),
107         );
108
109         // ————— طباعة كاملة لكل حقل راجع من —————
110         debugPrint('');
111         debugPrint('—————');

```

```

112     debugPrint('🍪 /refresh-session RESPONSE - Full Breakdown ');
113     debugPrint('_____');
114     debugPrint(' HTTP statusCode : ${refreshResponse.statusCode}');
115     debugPrint('_____');
116     debugPrint(' [top-level] ');
117     debugPrint('    success : ${refreshResponse.data['success']}');
118     debugPrint('    message : ${refreshResponse.data['message']}');
119     debugPrint('_____');
120     debugPrint(' [data] object: ');
121
122     final rawData = refreshResponse.data['data'];
123     if (rawData == null) {
124         debugPrint('⚠️ data is NULL - backend did not return a data object');
125     } else {
126         final d = rawData as Map;
127
128         final sCookie = d['session_cookie'] as String?;
129         debugPrint(' session_cookie : ${sCookie != null ? '✅ "${sCookie.substring(0, sCookie.length.clamp(0, 30))}..." : '❌ null'}');
130
131         final newRT = d['new_refresh_token'] as String?;
132         debugPrint(' new_refresh_token : ${newRT != null ? '✅ "${newRT.substring(0, newRT.length.clamp(0, 30))}..." : '❌ null - old token will be reused'}');
133
134         final rt = d['refresh_token'] as String?;
135         debugPrint(' refresh_token : ${rt != null ? '✅ "${rt.substring(0, rt.length.clamp(0, 30))}..." : '❌ null'}');
136
137         final exp = d['expires_at'];
138         debugPrint(' expires_at : ${exp ?? '❌ null - fallback: now + 30 days'}');
139
140         final knownKeys = {'session_cookie', 'new_refresh_token', 'refresh_token', 'expires_at'};
141         final extras = d.keys.where((k) => !knownKeys.contains(k)).toList();
142         if (extras.isNotEmpty) {
143             debugPrint(' [extra fields] : $extras');
144         }
145     }
146     debugPrint('_____');
147     debugPrint('');
148
149     final refreshData = refreshResponse.data;
150     final bool success = refreshData['success'] == true;
151
152     if (!success) {
153         debugPrint('❌ [Interceptor] success=false → notifySessionExpired()');
154         _sessionService.notifySessionExpired();
155         return handler.next(err);
156     }
157
158     // 3. Extract and save the new session data
159     debugPrint('📁 [Interceptor] Step 3 - Saving new session to SecureStorage...');
160     final data = refreshData['data'] as Map<String, dynamic>;
161     final newSessionCookie = data?['session_cookie'] as String?;
162     final newRefreshToken = data?['new_refresh_token'] as String?;
163     final expiresAtRaw = data?['expires_at'] as String?;
164
165     if (newSessionCookie == null || newSessionCookie.isEmpty) {
166         debugPrint('❌ [Interceptor] session_cookie missing in refresh response → notifySessionExpired()');
167         _sessionService.notifySessionExpired();
168         return handler.next(err);
169     }
170
171     final expiresAt = expiresAtRaw != null
172         ? DateTime.tryParse(expiresAtRaw.replaceAll(' ', 'T'))
173         : DateTime.now().add(const Duration(days: 30));
174
175     final tokenSaved = newRefreshToken ?? refreshToken;
176     debugPrint(' refresh_token to save : ${newRefreshToken != null ? '✅ new token from backend' : '⚠️ no new token - reusing old token'}');
177     debugPrint(' expiresAt parsed : $expiresAt');
178
179     await _secureStorage.saveMadrasatiSession(
180         MadrasatiSessionData(
181             sessionCookie: newSessionCookie,
182             schoolId: schoolId,

```



```

183         expiresAt: expiresAt,
184         refreshToken: tokenSaved,
185     ),
186 );
187 debugPrint('    SecureStorage saved ✅');
188
189 _sessionService.notifySessionActive();
190 debugPrint('🔔 [Interceptor] notifySessionActive() called ✅');
191
192 // 4. Replay the original failed request
193 debugPrint('');
194 debugPrint('🔄 [Interceptor] Step 4 – Replaying original request: ${err.requestOptions.method} ${err.re
questOptions.path}');
195 final opts = err.requestOptions;
196 final retryResponse = await _dio.request<dynamic>(
197     opts.path,
198     data: opts.data,
199     queryParameters: opts.queryParameters,
200     options: Options(
201         method: opts.method,
202         headers: opts.headers,
203         responseType: opts.responseType,
204         contentType: opts.contentType,
205     ),
206 );
207 debugPrint('✅ [Interceptor] Retry succeeded – statusCode: ${retryResponse.statusCode}');
208
209 return handler.resolve(retryResponse);
210 } on DioException catch (refreshErr) {
211     final statusCode = refreshErr.response?.statusCode;
212
213     // ————— (منتھية session) خطأ مصادقة vs نفق: خطأ شبكة (انقطاع) —————
214     // مشكلة شبكة مش مصادقة response لو مفيش
215     final bool isNetworkError = refreshErr.response == null;
216
217     debugPrint('');
218     debugPrint('❌ [Interceptor] DioException during refresh call!');
219     debugPrint('    statusCode    : ${statusCode ?? '❌ null (no response)'}');
220     debugPrint('    isNetworkErr   : $isNetworkError');
221     debugPrint('    error body     : ${refreshErr.response?.data}');
222     debugPrint('    message      : ${refreshErr.message}');
223
224     if (isNetworkError) {
225         // 🚫 انقطع الإنترنت - مش تظهر الدايلوج ومش نحذف الجلسة
226         debugPrint('🚫 [Interceptor] Network error – passing through WITHOUT showing dialog');
227         return handler.next(err);
228     }
229
230     // من أول connect خطأ مصادقة حقيقي (401/403) ... - لازم يعمل
231     if (statusCode == 401 || statusCode == 403) {
232         await _secureStorage.deleteMadrasatiSession();
233         debugPrint('🗑️ [Interceptor] refresh_token rejected ($statusCode) – cleared all session data');
234     }
235
236     _sessionService.notifySessionExpired();
237     debugPrint('❌ [Interceptor] notifySessionExpired() called → SessionExpiredDialog will show');
238     return handler.next(err);
239 } catch (e, st) {
240     debugPrint('❌ [Interceptor] Unexpected error: $e');
241     debugPrint('    StackTrace: $st');
242     // خطأ غير متوقع - نعامله كخطأ شبكة (مش تظهر الدايلوج)
243     debugPrint('⚠️ [Interceptor] Unexpected error – passing through WITHOUT showing dialog');
244     return handler.next(err);
245 }
246 }
247
248
249 /// Returns true if the response indicates a Madrasati session problem.
250 bool _isMadrasatiSessionError(Response<dynamic>? response) {
251     if (response == null) return false;
252
253     final statusCode = response.statusCode;
254     if (statusCode == null) return false;
255
256     if (response.data is Map) {

```

```

257     final data = response.data as Map;
258     final code = data['code']?.toString() ?? '';
259
260     // Standard App Authentication Error (Bearer token expired/missing)
261     // We should NOT treat this as a Madrasati session error!
262     if (statusCode == 401 && code == 'unauthenticated') {
263         return false;
264     }
265
266     // 422 with specific Madrasati session error codes
267     if (statusCode == 422 || statusCode == 401) {
268         return code == 'missing_madrasati_auth_cookie' ||
269             code == 'madrasati_session_expired' ||
270             code == 'madrasati_session_required';
271     }
272 }
273
274 return false;
275 }
276 }
277

```

Lines • Dart 86

Core / Services

#20 – lib/core/services/madrasati_session_service.dart

```

1  import 'dart:async';
2  import 'package:flutter/foundation.dart';
3
4  /// Represents the current status of the Madrasati school session.
5  enum MadrasatiSessionStatus {
6      /// Status not yet determined (app just started).
7      unknown,
8
9      /// Session is valid and active.
10     active,
11
12     /// Session has expired (detected from API error response).
13     expired,
14 }
15
16 /// Application-level service that monitors the Madrasati school session.
17 ///
18 /// Any screen or widget can listen to [onSessionExpired] to react when
19 /// the backend reports a session expiry (code: madrasati_session_expired).
20 ///
21 /// Usage:
22 /// ```dart
23 /// sl<MadrasatiSessionService>().onSessionExpired.listen((_) {
24 ///     // show dialog
25 /// });
26 /// ```
27 class MadrasatiSessionService {
28     MadrasatiSessionService();
29
30     // — State —————
31
32     MadrasatiSessionStatus _status = MadrasatiSessionStatus.unknown;
33
34     MadrasatiSessionStatus get status => _status;
35
36     bool get isExpired => _status == MadrasatiSessionStatus.expired;
37
38     // — Stream —————
39
40     final StreamController<void> _sessionExpiredController =
41         StreamController<void>.broadcast();
42
43     final StreamController<void> _sessionActiveController =
44         StreamController<void>.broadcast();
45
46     /// Fires whenever the Madrasati session is detected as expired.
47     /// The [SessionMonitorWrapper] listens to this to show the dialog.
48     Stream<void> get onSessionExpired => _sessionExpiredController.stream;
49

```

```

50  /// Fires whenever the Madrasati session is successfully refreshed.
51  Stream<void> get onSessionActive => _sessionActiveController.stream;
52
53  // — Public API —————
54
55  /// Called by [DioHelper] when it detects session-expired error codes.
56  /// Only fires the event once per expiry cycle (prevents multiple dialogs).
57  void notifySessionExpired() {
58    if (_status == MadrasatiSessionStatus.expired) return; // already notified
59    _status = MadrasatiSessionStatus.expired;
60    debugPrint('🔴 MadrasatiSessionService: session expired detected');
61    if (!_sessionExpiredController.isClosed) {
62      _sessionExpiredController.add(null);
63    }
64  }
65
66  /// Called after a successful session refresh or fresh connect.
67  void notifySessionActive() {
68    _status = MadrasatiSessionStatus.active;
69    debugPrint('🟢 MadrasatiSessionService: session is active');
70    if (!_sessionActiveController.isClosed) {
71      _sessionActiveController.add(null);
72    }
73  }
74
75  /// Reset to unknown (e.g., on logout).
76  void reset() {
77    _status = MadrasatiSessionStatus.unknown;
78    debugPrint('🟡 MadrasatiSessionService: reset');
79  }
80
81  void dispose() {
82    _sessionExpiredController.close();
83    _sessionActiveController.close();
84  }
85 }
86

```

Lines • Dart 109

Core / Services

#21 – lib/core/services/referral_service.dart

```

1  import 'dart:async';
2  import 'package:app_links/app_links.dart';
3  import 'package:android_play_install_referrer/android_play_install_referrer.dart';
4  import 'package:flutter/foundation.dart';
5  import 'package:moean/core/network/local/cache_helper.dart';
6
7  class ReferralService {
8    static final AppLinks _appLinks = AppLinks();
9    static StreamSubscription<Uri>? _linkSubscription;
10
11    static Future<void> init() async {
12      // 1. Check Google Play Install Referrer
13      await _checkInstallReferrer();
14
15      // 2. Handle Initial Deep Link (App opened via link)
16      try {
17        final initialUri = await _appLinks.getInitialLink();
18        if (initialUri != null) {
19          _handleDeepLink(initialUri);
20        }
21      } catch (e) {
22        debugPrint('Failed to get initial app link: $e');
23      }
24
25      // 3. Listen for incoming Deep Links while app is open
26      _linkSubscription?.cancel();
27      _linkSubscription = _appLinks.uriLinkStream.listen(
28        (Uri uri) {
29          _handleDeepLink(uri);
30        },
31        onError: (err) {
32          debugPrint('AppLinks Stream Error: $err');
33        },
34      );
35    }
36  }
37

```

```

34     );
35 }
36
37 static Future<void> _checkInstallReferrer() async {
38     try {
39         // Check if we already have a referral code
40         final existingCode = CacheHelper.getData(key: 'referral_code');
41         if (existingCode != null && existingCode.toString().isNotEmpty) return;
42
43         // Also check if user is already authenticated (token exists)
44         final existingToken = CacheHelper.getData(key: 'auth_token');
45         if (existingToken != null && existingToken.toString().isNotEmpty) return;
46
47         final ReferrerDetails referrerDetails = await AndroidPlayInstallReferrer.installReferrer;
48         final referrerUrl = referrerDetails.installReferrer;
49
50         if (referrerUrl != null && referrerUrl.isNotEmpty) {
51             debugPrint('Install Referrer URL: $referrerUrl');
52
53             // Parse the referrer string (e.g., utm_source=referral&utm_content=HAMDY7K2)
54             final uri = Uri.parse('http://localhost?$referrerUrl');
55             final utmContent = uri.queryParameters['utm_content'];
56
57             if (utmContent != null && utmContent.isNotEmpty) {
58                 _storeReferralCode(utmContent);
59             }
60         }
61     } catch (e) {
62         debugPrint('Failed to get install referrer: $e');
63     }
64 }
65
66 static void _handleDeepLink(Uri uri) {
67     try {
68         // Check if user is already authenticated (token exists)
69         final existingToken = CacheHelper.getData(key: 'auth_token');
70         if (existingToken != null && existingToken.toString().isNotEmpty) {
71             debugPrint('User is already authenticated. Ignoring referral deep link.');
```

```

72             return;
73         }
74
75         // Check if we already have a referral code
76         final existingCode = CacheHelper.getData(key: 'referral_code');
77         if (existingCode != null && existingCode.toString().isNotEmpty) {
78             debugPrint('Referral code already exists locally. Preserving existing code: $existingCode');
```

```

79             return;
80         }
81
82         // Handle https://haderedu.com/r/HAMDY7K2
83         if (uri.host == 'haderedu.com' && uri.pathSegments.isNotEmpty) {
84             if (uri.pathSegments[0] == 'r' && uri.pathSegments.length > 1) {
85                 final referralCode = uri.pathSegments[1];
86                 if (referralCode.isNotEmpty) {
87                     _storeReferralCode(referralCode);
88                 }
89             }
90         }
91     } catch (e) {
92         debugPrint('Error handling deep link: $e');
```

```

93     }
94 }
95
96 static void _storeReferralCode(String code) {
97     // Only store if we don't have one
98     final existingCode = CacheHelper.getData(key: 'referral_code');
99     if (existingCode == null || existingCode.toString().isEmpty) {
100         CacheHelper.saveData(key: 'referral_code', value: code);
101         debugPrint('Referral code stored locally: $code');
```

```

102     }
103 }
104
105 static void dispose() {
106     _linkSubscription?.cancel();
107 }

```

```
108 }  
109
```

Lines • Dart 116

Core / Theme

#22 – lib/core/theme/colors.dart

```
1 import 'package:moean/core/utils/cubit/theme/theme_cubit.dart';  
2 import 'package:flutter/material.dart';  
3  
4 class ColorsManager {  
5   static bool get isDark => themeCubit.isDarkMode;  
6  
7   static const Color black = Colors.black;  
8   static const Color white = Colors.white;  
9   static const Color themeDarkPrimary = Color(0xFF1E293B);  
10  static const Color themeActiveAccent = Color(0xFF0E7A5E);  
11  static const Color themeBackground = Color(0xFFFF8FFFC);  
12  static const Color themeDivider = Color(0xFFE2E8F0);  
13  static const Color textBody = Color(0xFF64748B);  
14  static const Color themePink = Color(0xFFD61F69);  
15  static const Color textPrimaryLight = themeDarkPrimary;  
16  static const Color backgroundColorLight = Color(0xFFf7fcfa);  
17  static const Color precpictioncardColor = Color(0xFFDB805B);  
18  static const Color textPrimaryDark = Color(0xFFFF4FFFC);  
19  static const Color backgroundDark = Color(0xFF071C18);  
20  static const Color surfaceDark = Color(0xFF0D2A24);  
21  static const Color primaryDark = Color(0xFF24B998);  
22  static const Color goldDark = Color(0xFFE2AD3B);  
23  static const Color mutedDark = Color(0xFF9CB8B0);  
24  static const Color textSecondaryDark = Color(0xFF075244);  
25  static Color get primaryColor => isDark ? primaryDark : themeActiveAccent;  
26  static Color get secondaryColor => isDark ? goldDark : const Color(0xFFE2AD3B);  
27  static Color get placeholder => isDark ? mutedDark : textBody;  
28  static const Color yellowBackground = Color(0xFFFF4F424);  
29  static const Color buttonBackground = Color(0xFFB8BEA6);  
30  static const Color errorColor = Color(0xffBA1A1A);  
31  static const Color successColor = Color(0xFF16A34A);  
32  
33  // UI Kit Colors for Schedule Screen  
34  static const Color statusSuccess = Color(0xFF22C55E);  
35  static const Color statusWarning = Color(0xFFFF97316);  
36  static const Color statusWaiting = Color(0xFF3B82F6);  
37  static const Color statusActivity = Color(0xFF8B5CF6);  
38  static const Color brandMint = Color(0xFFCFF3EE);  
39  static const Color brandGold = Color(0xFFFF5B25A);  
40  static Color get scheduleBackground => isDark ? backgroundDark : const Color(0xFFFF6FAF9);  
41  static Color get secondaryText => isDark ? textPrimaryDark : textSecondaryDark;  
42  
43  static Color get mainText => isDark ? textPrimaryDark : const Color(0xFF111827);  
44  static Color get secondaryText => isDark ? mutedDark : const Color(0xFF6B7280);  
45  static Color get borderLightGray => isDark ? white.withOpacity(0.1) : const Color(0xFFE5E7EB);  
46  
47  static Color get background => isDark ? backgroundDark : backgroundColorLight;  
48  static Color get textPrimary => isDark ? textPrimaryDark : textPrimaryLight;  
49  static Color get textSecondary => textPrimary;  
50  static Color get surfacePrimary => isDark ? surfaceDark : white;  
51  static Color get primaryAction => primaryColor;  
52  
53  static const Color borderLight = themeDivider;  
54  static Color get borderColor => isDark ? white.withOpacity(0.24) : borderLight;  
55  
56  // 135deg Gold Gradient Colors  
57  static const Color goldLight = Color(0xFFFF5D76E);  
58  
59  static const Color goldMedium = Color(0xFFC9A227);  
60  
61  
62  // Prescriptions Feature Colors  
63  static Color get prescriptionSearchBackground =>  
64    isDark ? const Color(0xFF1E293B) : const Color(0xFFFF1F5F9);  
65  static Color get prescriptionCardGreen => primaryColor;  
66  static Color get prescriptionCardOrange => const Color(0xFFEA580C);  
67  static Color get prescriptionCardOrangeBg =>  
68    isDark ? const Color(0xFF431407) : const Color(0xFFFFF7ED);
```

```

69 static Color get prescriptionCardBlue => const Color(0xFF0EA5E9);
70 static Color get prescriptionCardBlueBg =>
71   isDark ? const Color(0xFF0C4A6E) : const Color(0xFFFF0F9FF);
72 static Color get prescriptionListGrey =>
73   isDark ? surfaceDark : const Color(0xFFFF8FAFC);
74
75 // Email Verification Colors
76 static Color get verifyBadgeBg =>
77   isDark ? const Color(0xFF0D2E26) : const Color(0xFFE8F8F3);
78 static Color get verifySuccessDark =>
79   isDark ? textPrimaryDark : const Color(0xFF0A5C49);
80 static Color get verifyStepBorder =>
81   isDark ? const Color(0xFF163E34) : const Color(0xFFE3EFEA);
82 static Color get verifyStepText =>
83   isDark ? mutedDark : const Color(0xFF556E66);
84 static Color get verifyEmailContainerBg =>
85   isDark ? const Color(0xFF0D2E26) : const Color(0xFFEAF7F2);
86 static Color get verifyEmailContainerBorder =>
87   isDark ? const Color(0xFF163E34) : const Color(0xFFD5EAE2);
88 static Color get verifyMailIconBg =>
89   isDark ? const Color(0xFF382D13) : const Color(0xFFFFF8E7);
90 static const Color verifyMailIcon = Color(0xFFC98D14);
91 static Color get verifyMailBadgeBg =>
92   isDark ? const Color(0xFF382D13) : const Color(0xFFEF3D6);
93 static const Color verifyMailBadgeText = Color(0xFFB47D16);
94
95 // Account Suspended Colors
96 static Color get suspendedBadgeBg =>
97   isDark ? const Color(0xFF3B1212) : const Color(0xFFFFEE2E2);
98 static Color get suspendedBadgeBorder =>
99   isDark ? const Color(0xFF5C1D1D) : const Color(0xFFFFECACA);
100 static Color get suspendedBadgeText =>
101   isDark ? const Color(0xFFCA5A5) : const Color(0xFFDC2626);
102 static Color get suspendedIconBg =>
103   isDark ? const Color(0xFF451515) : const Color(0xFFFFFECCEC);
104 static Color get suspendedCardBorder =>
105   isDark ? const Color(0xFF4B1A1A) : const Color(0xFFFFFD6D6);
106 static Color get whatsappIconBg =>
107   isDark ? const Color(0xFF0F3822) : const Color(0xFFDCFCF7);
108 static const Color whatsappIconColor = Color(0xFF16A34A);
109 static Color get emailIconBg =>
110   isDark ? const Color(0xFF0B3047) : const Color(0xFFE0F2FE);
111 static const Color emailIconColor = Color(0xFF0284C7);
112 static Color get workingHoursIconBg =>
113   isDark ? const Color(0xFF38230D) : const Color(0xFFFEF3C7);
114 static const Color workingHoursIconColor = Color(0xFFD97706);
115 }
116

```

Lines • Dart 99 Core / Theme

#23 – lib/core/theme/text_styles.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:google_fonts/google_fonts.dart';
3 class TextStylesManager {
4   static TextStyle _tajawal(double size, FontWeight weight) {
5     return GoogleFonts.tajawal(
6       fontSize: size,
7       fontWeight: weight,
8       height: 1.3,
9     );
10  }
11
12  static TextStyle _aldhabi(double size, FontWeight weight) {
13    return TextStyle(
14      fontFamily: 'Aldhabi',
15      fontSize: size,
16      fontWeight: weight,
17      height: 1.3,
18    );
19  }
20
21  /// Regular text styles
22  static TextStyle get regular8 => _tajawal(8.0, FontWeight.w400);

```

```
23
24 static TextStyle get regular10 => _tajawal(10.0, FontWeight.w400);
25
26 static TextStyle get regular12 => _tajawal(12.0, FontWeight.w400);
27
28 static TextStyle get regular13 => _tajawal(13.0, FontWeight.w400);
29
30 static TextStyle get regular14 => _tajawal(14.0, FontWeight.w400);
31
32 static TextStyle get regular16 => _tajawal(16.0, FontWeight.w400);
33
34 static TextStyle get regular18 => _tajawal(18.0, FontWeight.w400);
35
36 static TextStyle get regular20 => _tajawal(20.0, FontWeight.w400);
37
38 static TextStyle get regular22 => _tajawal(22.0, FontWeight.w400);
39
40 static TextStyle get regular24 => _tajawal(24.0, FontWeight.w400);
41
42 static TextStyle get regular40 => _tajawal(40.0, FontWeight.w400);
43
44 /// Medium text styles
45 static TextStyle get medium10 => _tajawal(10.0, FontWeight.w500);
46
47 static TextStyle get medium12 => _tajawal(12.0, FontWeight.w500);
48
49 static TextStyle get medium13 => _tajawal(13.0, FontWeight.w500);
50
51 static TextStyle get medium14 => _tajawal(14.0, FontWeight.w500);
52
53 static TextStyle get medium16 => _tajawal(16.0, FontWeight.w500);
54
55 static TextStyle get medium18 => _tajawal(18.0, FontWeight.w500);
56
57 static TextStyle get medium20 => _tajawal(20.0, FontWeight.w500);
58
59 static TextStyle get medium22 => _tajawal(22.0, FontWeight.w500);
60
61 static TextStyle get medium24 => _tajawal(24.0, FontWeight.w500);
62
63 /// Bold text styles
64 static TextStyle get bold10 => _tajawal(10.0, FontWeight.w700);
65
66 static TextStyle get bold12 => _tajawal(12.0, FontWeight.w700);
67
68 static TextStyle get bold13 => _tajawal(13.0, FontWeight.w700);
69
70 static TextStyle get bold14 => _tajawal(14.0, FontWeight.w700);
71
72 static TextStyle get bold16 => _tajawal(16.0, FontWeight.w700);
73
74 static TextStyle get bold18 => _tajawal(18.0, FontWeight.w700);
75
76 static TextStyle get bold20 => _tajawal(20.0, FontWeight.w700);
77
78 static TextStyle get bold22 => _tajawal(22.0, FontWeight.w700);
79
80 static TextStyle get bold24 => _tajawal(24.0, FontWeight.w700);
81
82 static TextStyle get bold26 => _tajawal(26.0, FontWeight.w700);
83
84 static TextStyle get bold40 => _tajawal(40.0, FontWeight.w700);
85
86 static TextStyle get bold48 => _tajawal(48.0, FontWeight.w700);
87
88 /// Aldhabi display text styles (for headings / hero text)
89 static TextStyle get aldhabiRegular16 => _aldhabi(16.0, FontWeight.w400);
90 static TextStyle get aldhabiRegular20 => _aldhabi(20.0, FontWeight.w400);
91 static TextStyle get aldhabiRegular24 => _aldhabi(24.0, FontWeight.w400);
92 static TextStyle get aldhabiRegular28 => _aldhabi(28.0, FontWeight.w400);
93 static TextStyle get aldhabiRegular32 => _aldhabi(32.0, FontWeight.w400);
94 static TextStyle get aldhabiBold20 => _aldhabi(20.0, FontWeight.w700);
95 static TextStyle get aldhabiBold24 => _aldhabi(24.0, FontWeight.w700);
96 static TextStyle get aldhabiBold28 => _aldhabi(28.0, FontWeight.w700);
97 static TextStyle get aldhabiBold32 => _aldhabi(32.0, FontWeight.w700);
```

```
98 }  
99
```

Lines • Dart 141

Core / Theme

#24 – lib/core/theme/theme.dart

```
1 import 'package:moean/core/theme/colors.dart';  
2 import 'package:flutter/material.dart';  
3 import 'package:google_fonts/google_fonts.dart';  
4  
5 class ThemesManager {  
6   static ThemeData get lightTheme => ThemeData(  
7     useMaterial3: true,  
8     scaffoldBackgroundColor: ColorsManager.background,  
9     cardColor: Colors.white,  
10    cardTheme: CardThemeData(  
11      color: Colors.white,  
12      elevation: 4,  
13      shadowColor: Colors.black.withValues(alpha: 0.05),  
14      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),  
15    ),  
16    bottomNavigationBarTheme: const BottomNavigationBarThemeData(  
17      backgroundColor: Colors.white,  
18      selectedItemColor: ColorsManager.themeActiveAccent,  
19      unselectedItemColor: ColorsManager.textBody,  
20      elevation: 8,  
21      type: BottomNavigationBarType.fixed,  
22    ),  
23    textTheme: GoogleFonts.tajawalTextTheme(),  
24    colorScheme: ColorScheme.fromSeed(  
25      seedColor: ColorsManager.primaryColor,  
26      surface: ColorsManager.surfacePrimary,  
27      primary: ColorsManager.primaryAction,  
28      onPrimary: Colors.white,  
29      outline: ColorsManager.borderColor,  
30      onSurface: ColorsManager.textPrimary,  
31    ),  
32    appBarTheme: const AppBarTheme(  
33      backgroundColor: ColorsManager.backgroundColorLight,  
34      foregroundColor: ColorsManager.textPrimaryLight,  
35      elevation: 0,  
36      scrolledUnderElevation: 0,  
37    ),  
38    dividerColor: ColorsManager.borderColor,  
39    inputDecorationTheme: InputDecorationTheme(  
40      filled: true,  
41      fillColor: ColorsManager.background,  
42      contentPadding: const EdgeInsets.symmetric(horizontal: 20, vertical: 16),  
43      hintStyle: GoogleFonts.tajawal(  
44        fontSize: 14,  
45        fontWeight: FontWeight.w400,  
46        color: ColorsManager.textBody,  
47      ),  
48      border: OutlineInputBorder(  
49        borderRadius: BorderRadius.circular(24),  
50        borderSide: const BorderSide(color: ColorsManager.themeDivider),  
51      ),  
52      enabledBorder: OutlineInputBorder(  
53        borderRadius: BorderRadius.circular(24),  
54        borderSide: const BorderSide(color: ColorsManager.themeDivider),  
55      ),  
56      focusedBorder: OutlineInputBorder(  
57        borderRadius: BorderRadius.circular(24),  
58        borderSide: const BorderSide(color: ColorsManager.themeActiveAccent),  
59      ),  
60    ),  
61    elevatedButtonTheme: ElevatedButtonThemeData(  
62      style: ElevatedButton.styleFrom(  
63        backgroundColor: ColorsManager.primaryAction,  
64        foregroundColor: Colors.white,  
65        elevation: 0,  
66        padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 14),  
67        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(24)),  
68      ),
```



```

69     ),
70 );
71
72 static ThemeData get darkTheme => ThemeData(
73     useMaterial3: true,
74     scaffoldBackgroundColor: ColorsManager.backgroundDark,
75     cardColor: ColorsManager.surfaceDark,
76     cardTheme: CardThemeData(
77         color: ColorsManager.surfaceDark,
78         elevation: 4,
79         shadowColor: Colors.black.withValues(alpha: 0.2),
80         shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
81     ),
82     bottomNavigationBarTheme: const BottomNavigationBarThemeData(
83         backgroundColor: ColorsManager.surfaceDark,
84         selectedItemColor: ColorsManager.themeActiveAccent,
85         unselectedItemColor: ColorsManager.textBody,
86         elevation: 8,
87         type: BottomNavigationBarType.fixed,
88     ),
89     textTheme: GoogleFonts.tajawalTextTheme(
90         ThemeData(brightness: Brightness.dark).textTheme,
91     ),
92     colorScheme: ColorScheme.fromSeed(
93         seedColor: ColorsManager.primaryColor,
94         brightness: Brightness.dark,
95         surface: ColorsManager.surfaceDark,
96         primary: ColorsManager.primaryAction,
97         onPrimary: Colors.white,
98         outline: Colors.white24,
99         onSurface: ColorsManager.textPrimaryDark,
100     ),
101     appBarTheme: const AppBarTheme(
102         backgroundColor: ColorsManager.backgroundDark,
103         foregroundColor: ColorsManager.textPrimaryDark,
104         elevation: 0,
105         scrolledUnderElevation: 0,
106     ),
107     dividerColor: Colors.white24,
108     inputDecorationTheme: InputDecorationTheme(
109         filled: true,
110         fillColor: ColorsManager.surfaceDark,
111         contentPadding: const EdgeInsets.symmetric(horizontal: 20, vertical: 16),
112         hintStyle: GoogleFonts.tajawal(
113             fontSize: 14,
114             fontWeight: FontWeight.w400,
115             color: ColorsManager.textBody,
116         ),
117         border: OutlineInputBorder(
118             borderRadius: BorderRadius.circular(24),
119             borderSide: const BorderSide(color: Colors.white24),
120         ),
121         enabledBorder: OutlineInputBorder(
122             borderRadius: BorderRadius.circular(24),
123             borderSide: const BorderSide(color: Colors.white24),
124         ),
125         focusedBorder: OutlineInputBorder(
126             borderRadius: BorderRadius.circular(24),
127             borderSide: const BorderSide(color: ColorsManager.themeActiveAccent),
128         ),
129     ),
130     elevatedButtonTheme: ElevatedButtonThemeData(
131         style: ElevatedButton.styleFrom(
132             backgroundColor: ColorsManager.primaryAction,
133             foregroundColor: Colors.white,
134             elevation: 0,
135             padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 14),
136             shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(24)),
137         ),
138     ),
139 );
140 }
141

```

```

1 class AssetsHelper {
2   static const String logo = 'assets/images/logo-removebg-preview (1).png';
3   static const String img1 = 'assets/images/img1.jpeg';
4   static const String img2 = 'assets/images/img2.png';
5   static const String img3 = 'assets/images/img3.jpeg';
6   static const String img4 = 'assets/images/img4.jpeg';
7   static const String img5 = 'assets/images/img5.jpeg';
8   static const String img6 = 'assets/images/img6.png';
9   static const String img7 = 'assets/images/img7.jpeg';
10  static const String img8 = 'assets/images/img8.png';
11  static const String img9 = 'assets/images/img11.png';
12  static const String screen1 = 'assets/images/screen1.jpeg';
13  static const String screen2 = 'assets/images/screen2.jpeg';
14  static const String screen3 = 'assets/images/screen3.jpeg';
15  static const String minstry = 'assets/images/minstry.jpg';
16  static const String icon = 'assets/images/logo_icon-removebg-preview.png';
17  static const String tips = 'assets/images/tips.png';
18  static const String microsoft = 'assets/images/microsoft.png';
19  static const String roaa = 'assets/images/roaa.png';
20  static const String scr1 = 'assets/images/scr1.jpeg';
21  static const String scr2 = 'assets/images/scr2.jpeg';
22  static const String scr3 = 'assets/images/scr3.jpeg';
23  static const String scr4 = 'assets/images/scr4.jpeg';
24  static const String scr5 = 'assets/images/scr5.jpeg';
25  static const String scr6 = 'assets/images/scr6.jpeg';
26  static const String scr7 = 'assets/images/scr7.jpeg';
27  static const String scr8 = 'assets/images/scr8.jpeg';
28  static const String scr9 = 'assets/images/scr9.jpeg';
29  static const String scr10 = 'assets/images/scr10.jpeg';
30  static const String scr11 = 'assets/images/scr11.jpeg';
31  static const String scr12 = 'assets/images/scr12.jpeg';
32  static const String scr13 = 'assets/images/scr13.jpeg';
33  static const String scr14 = 'assets/images/scr1.jpeg';
34  static const String scr15 = 'assets/images/scr15.jpeg';
35  static const String scr16 = 'assets/images/img16.jpg';
36  static const String presentation1 = 'assets/images/presentaion1.jpeg';
37  static const String presentation2 = 'assets/images/presentation2.jpeg';
38  static const String presentation3 = 'assets/images/presentation3.jpeg';
39
40
41 }
42

```

```

1 import 'package:moean/core/theme/colors.dart';
2 import 'package:moean/core/utils/constants/translations.dart';
3 import 'package:moean/core/utils/cubit/theme/theme_cubit.dart';
4 import 'package:flutter/material.dart';
5 // import 'package:url_launcher/url_launcher.dart';
6
7 TranslationModel appTranslation() =>
8   themeCubit.translationModel ?? TranslationModel.fromJson({});
9
10 String? token;
11
12 // Future<void> launchInBrowser(Uri url) async {
13 //   if (!await launchUrl(
14 //     url,
15 //     mode: LaunchMode.externalApplication,
16 //   )) {
17 //     throw Exception('Could not launch $url');
18 //   }
19 // }
20
21 String formattedCreatedAt(DateTime createdAt) {
22   final date = createdAt.toLocal();
23   final formattedDate =
24     "${date.year.toString().padLeft(4, '0')}-${
25     "${date.month.toString().padLeft(2, '0')}-${
26     "${date.day.toString().padLeft(2, '0')}";

```

```

27
28   final formattedTime =
29     "${date.hour.toString().padLeft(2, '0')}: "
30     "${date.minute.toString().padLeft(2, '0')}";
31
32   return "$formattedDate || $formattedTime";
33 }
34
35 InputDecoration helpCenterInputDecoration({String? hint}) {
36   return InputDecoration(
37     hintText: hint,
38     hintStyle: TextStyle(
39       color: ColorsManager.textPrimary.withValues(alpha: .6),
40     ),
41     filled: true,
42     fillColor: ColorsManager.surfacePrimary,
43     isDense: true,
44     contentPadding: const EdgeInsets.symmetric(horizontal: 14, vertical: 14),
45     border: OutlineInputBorder(
46       borderRadius: BorderRadius.circular(12),
47       borderSide: BorderSide(color: ColorsManager.textPrimary),
48     ),
49     enabledBorder: OutlineInputBorder(
50       borderRadius: BorderRadius.circular(12),
51       borderSide: BorderSide(color: ColorsManager.textPrimary),
52     ),
53     focusedBorder: const OutlineInputBorder(
54       borderRadius: BorderRadius.all(Radius.circular(12)),
55       borderSide: BorderSide(color: Color(0xFF2D6CDF)),
56     ),
57   );
58 }
59

```

Lines • Dart 72 Core / Utils

#27 – lib/core/utils/constants/image_helper.dart

```

1  import 'package:moean/core/network/remote/api_endpoints.dart';
2
3  class ImageHelper {
4    /// Builds a robust image URL by prepending the base URL and handling common subfolders.
5    /// It covers cases where the backend returns relative paths like 'avatars/xxx.jpg'
6    /// and ensures Laravel's '/storage/' prefix is added if likely needed.
7    static String? getFullImageUrl(String? path) {
8      if (path == null || path.trim().isEmpty) return null;
9
10     final trimmedPath = path.trim();
11
12     // 1. If it's already a full URL, just ensure it's HTTPS if needed
13     if (trimmedPath.startsWith('http')) {
14       if (trimmedPath.startsWith('http://') && baseUrl.startsWith('https://')) {
15         return trimmedPath.replaceFirst('http://', 'https://');
16       }
17       return trimmedPath;
18     }
19
20     // 2. Prepare the base and relative path
21     final base = baseUrl.endsWith('/')
22       ? baseUrl.substring(0, baseUrl.length - 1)
23       : baseUrl;
24
25     String relativePath = trimmedPath.startsWith('/')
26       ? trimmedPath.substring(1)
27       : trimmedPath;
28
29     // 3. Heuristics for common subfolders that usually live under /storage/
30     // We check if it doesn't already have storage/ or public/
31     final List<String> storageFolders = [
32       'avatars',
33       'profiles',
34       'patients',
35       'doctors',
36       'facilities',
37       'medications',

```

```

38     'prescriptions',
39   ];
40
41   bool needsStoragePrefix = true;
42   if (relativePath.startsWith('storage/') ||
43       relativePath.startsWith('public/')) {
44     needsStoragePrefix = false;
45   } else {
46     // Check if it starts with any common storage folder
47     bool isMatch = false;
48     for (var folder in storageFolders) {
49       if (relativePath.startsWith('$folder/')) {
50         isMatch = true;
51         break;
52       }
53     }
54     if (!isMatch) {
55       // If it doesn't match a known folder, we still might need storage/
56       // but we'll be more conservative.
57       // Let's assume for now that if it's relative, it needs storage/
58       // UNLESS the path starts with something like 'api/'
59       if (relativePath.startsWith('api/')) {
60         needsStoragePrefix = false;
61       }
62     }
63   }
64
65   if (needsStoragePrefix) {
66     relativePath = 'storage/$relativePath';
67   }
68
69   return '$base/$relativePath';
70 }
71 }
72

```

Lines • Dart 29

Core / Utils

#28 – lib/core/utils/constants/my_bloc_observer.dart

```

1  import 'package:bloc/bloc.dart';
2  import 'package:flutter/material.dart';
3
4  class MyBlocObserver extends BlocObserver {
5    @override
6    void onCreate(BlocBase bloc) {
7      super.onCreate(bloc);
8      debugPrint('onCreate -- ${bloc.runtimeType}');
9    }
10
11    @override
12    void onChange(BlocBase bloc, Change change) {
13      super.onChange(bloc, change);
14      debugPrint('onChange -- ${bloc.runtimeType}, $change');
15    }
16
17    @override
18    void onError(BlocBase bloc, Object error, StackTrace stackTrace) {
19      debugPrint('onError -- ${bloc.runtimeType}, $error');
20      super.onError(bloc, error, stackTrace);
21    }
22
23    @override
24    void onClose(BlocBase bloc) {
25      super.onClose(bloc);
26      debugPrint('onClose -- ${bloc.runtimeType}');
27    }
28  }
29

```

Lines • Dart 217

Core / Utils

#29 – lib/core/utils/constants/primary/circle_indecator.dart

```

1  import 'package:flutter/material.dart';
2  import 'dart:math' as math;

```

```

3 class DiscreteCircle extends StatefulWidget {
4   final double size;
5   final Color color;
6   final Color secondCircleColor;
7   final Color thirdCircleColor;
8
9
10  const DiscreteCircle({
11    super.key,
12    required this.color,
13    required this.size,
14    required this.secondCircleColor,
15    required this.thirdCircleColor,
16  });
17
18  @override
19  State<DiscreteCircle> createState() => _DiscreteCircleState();
20 }
21
22 class _DiscreteCircleState extends State<DiscreteCircle>
23   with SingleTickerProviderStateMixin {
24   late AnimationController _controller;
25
26   @override
27   void initState() {
28     super.initState();
29     _controller = AnimationController(
30       vsync: this,
31       duration: const Duration(milliseconds: 2000),
32     )..repeat();
33   }
34
35   @override
36   void dispose() {
37     _controller.dispose();
38     super.dispose();
39   }
40
41   // دالة بديلة للمكتبة الخارجية لعمل حسابات القيم بناءً على الوقت والمنحنى
42   double _evalDouble({
43     double from = 0.0,
44     required double to,
45     required double begin,
46     required double end,
47     Curve curve = Curves.linear,
48   }) {
49     final interval = Interval(begin, end, curve: curve);
50     final progress = interval.transform(_controller.value);
51     return from + (to - from) * progress;
52   }
53
54   @override
55   Widget build(BuildContext context) {
56     final double size = widget.size;
57     final double strokeWidth = size / 8;
58
59     return AnimatedBuilder(
60       animation: _controller,
61       builder: (context, child) {
62         final bool isFirstHalf = _controller.value <= 0.5;
63         final bool isSecondHalf = _controller.value >= 0.5;
64
65         return SizedBox(
66           width: size,
67           height: size,
68           child: Stack(
69             children: <Widget>[
70               // الدائرة الثالثة
71               if (isSecondHalf)
72                 Transform.rotate(
73                   angle: _evalDouble(
74                     to: 2 * math.pi,
75                     begin: 0.68,
76                     end: 0.95,
77                     curve: Curves.easeOut,

```

```

78         ),
79         child: _Arc(
80             color: widget.thirdCircleColor,
81             strokeWidth: strokeWidth,
82             startAngle: -math.pi / 2,
83             sweepAngle: _evalDouble(
84                 from: math.pi / 2,
85                 to: math.pi / (size * size),
86                 begin: 0.7,
87                 end: 0.95,
88                 curve: Curves.easeOutSine,
89             ),
90         ),
91     ),
92
93     // الدائرة الثانية
94     if (isSecondHalf)
95         _Arc(
96             color: widget.secondCircleColor,
97             strokeWidth: strokeWidth,
98             startAngle: -math.pi / 2,
99             sweepAngle: _evalDouble(
100                 from: -2 * math.pi,
101                 to: math.pi / (size * size),
102                 begin: 0.6,
103                 end: 0.95,
104                 curve: Curves.easeOutSine,
105             ),
106         ),
107
108     // الدائرة الأساسية (النصف الأول من الحركة)
109     if (isFirstHalf)
110         Transform.rotate(
111             angle: _evalDouble(
112                 to: math.pi * 0.06,
113                 begin: 0.48,
114                 end: 0.5,
115             ),
116             child: _Arc(
117                 color: widget.color,
118                 strokeWidth: strokeWidth,
119                 startAngle: -math.pi / 2,
120                 sweepAngle: _evalDouble(
121                     from: math.pi / (size * size),
122                     to: 1.94 * math.pi,
123                     begin: 0.05,
124                     end: 0.48,
125                     curve: Curves.easeOutSine,
126                 ),
127             ),
128         ),
129
130     // الدائرة الأساسية (النصف الثاني من الحركة)
131     if (isSecondHalf)
132         _Arc(
133             color: widget.color,
134             strokeWidth: strokeWidth,
135             startAngle: -math.pi / 2,
136             sweepAngle: _evalDouble(
137                 from: -1.94 * math.pi,
138                 to: math.pi / (size * size),
139                 begin: 0.5,
140                 end: 0.95,
141                 curve: Curves.easeOutSine,
142             ),
143         ),
144     ],
145     ),
146 );
147 },
148 );
149 }
150 }
151
152 // =====

```

```

153 // Private Helper Widgets for Drawing (Standalone)
154 // =====
155 class _Arc extends StatelessWidget {
156   final Color color;
157   final double strokeWidth;
158   final double startAngle;
159   final double sweepAngle;
160
161   const _Arc({
162     required this.color,
163     required this.strokeWidth,
164     required this.startAngle,
165     required this.sweepAngle,
166   });
167
168   @override
169   Widget build(BuildContext context) {
170     return CustomPaint(
171       size: Size.infinite,
172       painter: _ArcPainter(
173         color: color,
174         strokeWidth: strokeWidth,
175         startAngle: startAngle,
176         sweepAngle: sweepAngle,
177       ),
178     );
179   }
180 }
181
182 class _ArcPainter extends CustomPainter {
183   final Color color;
184   final double strokeWidth;
185   final double startAngle;
186   final double sweepAngle;
187
188   _ArcPainter({
189     required this.color,
190     required this.strokeWidth,
191     required this.startAngle,
192     required this.sweepAngle,
193   });
194
195   @override
196   void paint(Canvas canvas, Size size) {
197     final center = Offset(size.width / 2, size.height / 2);
198     final radius = (math.min(size.width, size.height) - strokeWidth) / 2;
199     final rect = Rect.fromCircle(center: center, radius: radius);
200
201     final paint = Paint()
202       ..color = color
203       ..style = PaintingStyle.stroke
204       ..strokeWidth = strokeWidth
205       ..strokeCap = StrokeCap.round;
206
207     canvas.drawArc(rect, startAngle, sweepAngle, false, paint);
208   }
209
210   @override
211   bool shouldRepaint(covariant _ArcPainter oldDelegate) {
212     return oldDelegate.color != color ||
213       oldDelegate.strokeWidth != strokeWidth ||
214       oldDelegate.startAngle != startAngle ||
215       oldDelegate.sweepAngle != sweepAngle;
216   }
217 }

```

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/utils/constants/primary/circle_indecator.dart';
3
4 class ConditionalBuilder extends StatelessWidget {
5   final bool loadingState;

```

```

6   final bool errorState;
7   final bool emptyState;
8
9   final WidgetBuilder successBuilder;
10  final WidgetBuilder? loadingBuilder;
11  final WidgetBuilder? errorBuilder;
12  final WidgetBuilder? emptyBuilder;
13
14  const ConditionalBuilder({
15    super.key,
16    required this.loadingState,
17    this.errorState = false,
18    this.emptyState = false,
19    required this.successBuilder,
20    this.loadingBuilder,
21    this.errorBuilder,
22    this.emptyBuilder,
23  });
24
25  @override
26  Widget build(BuildContext context) {
27    if (loadingState) {
28      return loadingBuilder?.call(context) ??
29        const Center(child: DiscreteCircle(
30          size: 60,
31          color:Color(0xFFD61F69),
32          secondCircleColor:Color(0xFFE2AD3B),
33          thirdCircleColor:Color(0xFF0E7A5E)
34        ),);
35    }
36
37    if (errorState) {
38      return errorBuilder?.call(context) ?? const SizedBox.shrink();
39    }
40
41    if (emptyState) {
42      return emptyBuilder?.call(context) ?? const SizedBox.shrink();
43    }
44
45    return successBuilder(context);
46  }
47 }
48

```

Lines • Dart 41

Core / Utils

#31 – lib/core/utils/constants/primary/elevation_icon_button.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4
5  class ElevationIconButton extends StatelessWidget {
6    final String text;
7    final IconData icon;
8    final VoidCallback onPressed;
9
10   const ElevationIconButton({
11     super.key,
12     required this.text,
13     required this.icon,
14     required this.onPressed,
15   });
16
17   @override
18   Widget build(BuildContext context) {
19     return ElevatedButton.icon(
20       onPressed: onPressed,
21       icon: Icon(
22         icon,
23         color: Colors.white,
24         size: 18,
25       ),
26       label: Text(
27         text,

```



```

28         style: TextStylesManager.bold14.copyWith(color: Colors.white),
29     ),
30     style: ElevatedButton.styleFrom(
31         backgroundColor: ColorsManager.primaryColor,
32         padding: const EdgeInsets.symmetric(vertical: 12),
33         elevation: 0,
34         shape: RoundedRectangleBorder(
35             borderRadius: BorderRadius.circular(24),
36         ),
37     ),
38 );
39 }
40 }
41

```

Lines • Dart 60

Core / Utils

#32 – lib/core/utils/constants/primary/no_data_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/spacing.dart';
5
6  class NoDataWidget extends StatelessWidget {
7      final String title;
8      final String? subtitle;
9      final IconData? icon;
10
11      const NoDataWidget({
12          super.key,
13          required this.title,
14          this.subtitle,
15          this.icon,
16      });
17
18      @override
19      Widget build(BuildContext context) {
20          return Center(
21              child: Column(
22                  mainAxisAlignment: MainAxisAlignment.center,
23                  children: [
24                      Container(
25                          padding: const EdgeInsets.all(24),
26                          decoration: BoxDecoration(
27                              color: ColorsManager.primaryColor.withValues(alpha: 0.05),
28                              shape: BoxShape.circle,
29                          ),
30                      child: Icon(
31                          icon ?? Icons.search_off_rounded,
32                          size: 64,
33                          color: ColorsManager.primaryColor.withValues(alpha: 0.5),
34                      ),
35                  ],
36                  verticalSpace24,
37                  Text(
38                      title,
39                      style: TextStylesManager.bold18,
40                      textAlign: TextAlign.center,
41                  ),
42                  if (subtitle != null) ...[
43                      verticalSpace8,
44                      Padding(
45                          padding: const EdgeInsets.symmetric(horizontal: 40),
46                          child: Text(
47                              subtitle!,
48                              style: TextStylesManager.regular14.copyWith(
49                                  color: ColorsManager.textPrimary,
50                              ),
51                          textAlign: TextAlign.center,
52                      ),
53                  ],
54              ],
55          ),
56

```

```
57     );
58   }
59 }
60
```

Lines • Dart 39

Core / Utils

#33 – lib/core/utils/constants/primary/primary_elevated_button_label.dart

```
1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/text_styles.dart';
3 import 'package:moean/core/utils/constants/primary/conditional_builder.dart';
4
5 class PrimaryElevatedButtonLabel extends StatelessWidget {
6   final bool isLoading;
7   final String text;
8   final TextStyle? textStyle;
9
10  const PrimaryElevatedButtonLabel({
11    super.key,
12    required this.isLoading,
13    required this.text,
14    this.textStyle,
15  });
16
17  @override
18  Widget build(BuildContext context) {
19    return ConditionalBuilder(
20      loadingState: isLoading,
21      loadingBuilder: (context) => const SizedBox(
22        width: 24,
23        height: 24,
24        child: CircularProgressIndicator(
25          color: Colors.white,
26          strokeWidth: 2.5,
27        ),
28      ),
29      successBuilder: (context) => Text(
30        text,
31        style: textStyle ??
32          TextStylesManager.bold14.copyWith(
33            color: Colors.white,
34          ),
35      ),
36    );
37  }
38 }
39
```

Lines • Dart 87

Core / Utils

#34 – lib/core/utils/constants/primary/primary_elevated_button.dart

```
1 import 'package:moean/core/theme/colors.dart';
2 import 'package:flutter/material.dart';
3 import 'package:moean/core/utils/constants/primary/primary_elevated_button_label.dart';
4
5 class PrimaryElevatedButton extends StatelessWidget {
6   final String text;
7   final VoidCallback? onPressed;
8   final bool isLoading;
9   final double height;
10  final double radius;
11  final Color? backgroundColor;
12  final TextStyle? textStyle;
13  final Widget? icon;
14  final double? width;
15  final BorderSide? borderSide;
16
17  const PrimaryElevatedButton({
18    super.key,
19    required this.text,
20    required this.onPressed,
21    this.isLoading = false,
22    this.height = 52,
23    this.radius = 24,
```

```

24     this.backgroundColor,
25     this.textStyle,
26     this.icon,
27     this.width,
28     this.borderSide,
29   });
30
31   @override
32   Widget build(BuildContext context) {
33     return SizedBox(
34       width: width ?? double.infinity,
35       height: height,
36       child: icon == null
37         ? ElevatedButton(
38           onPressed: isLoading ? null : onPressed,
39           style: ElevatedButton.styleFrom(
40             backgroundColor: backgroundColor ?? ColorsManager.primaryColor,
41             foregroundColor: Colors.white,
42             elevation: 0,
43             shadowColor: Colors.black.withValues(alpha: 0.04),
44             animationDuration: const Duration(milliseconds: 120),
45             padding: const EdgeInsets.symmetric(
46               horizontal: 20,
47               vertical: 14,
48             ),
49             shape: RoundedRectangleBorder(
50               borderRadius: BorderRadius.circular(radius),
51               side: borderSide ?? BorderSide.none,
52             ),
53           ),
54           child: PrimaryElevatedButtonLabel(
55             isLoading: isLoading,
56             text: text,
57             textStyle: textStyle,
58           ),
59         )
60         : ElevatedButton.icon(
61           onPressed: isLoading ? null : onPressed,
62           style: ElevatedButton.styleFrom(
63             backgroundColor: backgroundColor ?? ColorsManager.primaryColor,
64             foregroundColor: Colors.white,
65             elevation: 0,
66             shadowColor: Colors.black.withValues(alpha: 0.04),
67             animationDuration: const Duration(milliseconds: 120),
68             padding: const EdgeInsets.symmetric(
69               horizontal: 20,
70               vertical: 14,
71             ),
72             shape: RoundedRectangleBorder(
73               borderRadius: BorderRadius.circular(radius),
74               side: borderSide ?? BorderSide.none,
75             ),
76           ),
77           icon: icon!,
78           label: PrimaryElevatedButtonLabel(
79             isLoading: isLoading,
80             text: text,
81             textStyle: textStyle,
82           ),
83         ),
84     );
85   }
86 }
87

```

```

1 import 'package:flutter/services.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:flutter/material.dart';
5 import 'package:flutter_bloc/flutter_bloc.dart';
6 import 'package:moean/core/utils/cubit/theme/theme_cubit.dart';

```

```

7 import 'package:moean/core/utils/cubit/theme/theme_state.dart';
8
9 class PrimaryTextField extends StatefulWidget {
10   final TextEditingController controller;
11   final String? label;
12   final String hint;
13   final bool isPassword;
14   final String? Function(String?)? validator;
15   final Widget? prefixIcon;
16   final Widget? suffixIcon;
17   final TextInputType keyboardType;
18   final TextInputAction textInputAction;
19   final bool enabled;
20   final Color? fillColor;
21   final Color? textColor;
22   final Color? hintColor;
23   final bool readOnly;
24   final VoidCallback? onTap;
25   final ValueChanged<String>? onChanged;
26   final FocusNode? focusNode;
27   final ValueChanged<String>? onFieldSubmitted;
28   final int maxLines;
29   final List<TextInputFormatter>? inputFormatters;
30   final TextDirection? textDirection;
31   final TextAlign textAlign;
32   final BoxConstraints? prefixIconConstraints;
33   final BoxConstraints? suffixIconConstraints;
34   final int? maxLength;
35
36   const PrimaryTextField({
37     super.key,
38     required this.controller,
39     this.label,
40     required this.hint,
41     this.isPassword = false,
42     this.validator,
43     this.prefixIcon,
44     this.suffixIcon,
45     this.keyboardType = TextInputType.text,
46     this.textInputAction = TextInputAction.next,
47     this.enabled = true,
48     this.fillColor,
49     this.textColor,
50     this.hintColor,
51     this.readOnly = false,
52     this.onTap,
53     this.onChanged,
54     this.focusNode,
55     this.onFieldSubmitted,
56     this.maxLines = 1,
57     this.inputFormatters,
58     this.textDirection,
59     this.textAlign = TextAlign.start,
60     this.prefixIconConstraints,
61     this.suffixIconConstraints,
62     this.maxLength,
63   });
64
65   @override
66   State<PrimaryTextField> createState() => _PrimaryTextFieldState();
67 }
68
69 class _PrimaryTextFieldState extends State<PrimaryTextField> {
70   late FocusNode _focusNode;
71   late bool _ownsFocusNode;
72
73   @override
74   void initState() {
75     super.initState();
76     _focusNode = widget.focusNode ?? FocusNode();
77     _ownsFocusNode = widget.focusNode == null;
78     _focusNode.addListener(_onFocusChange);
79   }
80
81   @override

```

```

82 void didUpdateWidget(covariant PrimaryTextField oldWidget) {
83     super.didUpdateWidget(oldWidget);
84
85     if (oldWidget.focusNode != widget.focusNode) {
86         _focusNode.removeListener(_onFocusChange);
87         if (_ownsFocusNode) {
88             _focusNode.dispose();
89         }
90
91         _focusNode = widget.focusNode ?? FocusNode();
92         _ownsFocusNode = widget.focusNode == null;
93         _focusNode.addListener(_onFocusChange);
94     }
95 }
96
97 void _onFocusChange() {
98     if (mounted) {
99         setState(() {});
100     }
101 }
102
103 @override
104 void dispose() {
105     _focusNode.removeListener(_onFocusChange);
106     if (_ownsFocusNode) {
107         _focusNode.dispose();
108     }
109     super.dispose();
110 }
111
112 @override
113 Widget build(BuildContext context) {
114     return BlocBuilder<ThemeCubit, ThemeState>(
115         builder: (context, state) {
116             return TextFormField(
117                 controller: widget.controller,
118                 focusNode: _focusNode,
119                 obscureText: widget.isPassword,
120                 validator: widget.validator,
121                 keyboardType: widget.keyboardType,
122                 textInputAction: widget.textInputAction,
123                 inputFormatters: widget.inputFormatters,
124                 textDirection: widget.textDirection,
125                 textAlign: widget.textAlign,
126                 maxLength: widget.maxLength,
127                 maxLines: widget.maxLines,
128                 style: TextStyleManager.regular14.copyWith(
129                     color: widget.textColor ?? ColorsManager.textPrimary,
130                 ),
131                 enabled: widget.enabled,
132                 readOnly: widget.readOnly,
133                 onTap: widget.onTap,
134                 onChanged: widget.onChanged,
135                 onFieldSubmitted: widget.onFieldSubmitted,
136                 decoration: InputDecoration(
137                     // تفعيل الخلفية هنا بدلاً من الـ
138                     filled: true,
139                     fillColor: widget.fillColor ?? ColorsManager.surfacePrimary,
140                     counterText: widget.maxLength != null ? '': null,
141                     labelText: widget.label,
142                     labelStyle: TextStyleManager.regular14.copyWith(
143                         color: widget.textColor ?? ColorsManager.textSecondary,
144                     ),
145                     hintText: widget.hint,
146                     hintStyle: TextStyleManager.regular14.copyWith(
147                         color: widget.hintColor ?? ColorsManager.textSecondary.withValues(alpha: 0.85),
148                     ),
149                     contentPadding: const EdgeInsets.symmetric(
150                         horizontal: 20,
151                         vertical: 16,
152                     ),
153                     prefixIcon: widget.prefixIcon,
154                     prefixIconConstraints: widget.prefixIconConstraints,
155                     prefixIconColor: widget.textColor ?? ColorsManager.textSecondary,
156                     suffixIcon: widget.suffixIcon,

```

```

157     suffixIconConstraints: widget.suffixIconConstraints,
158     suffixIconColor: widget.textColor ?? ColorsManager.textSecondary,
159
160     // هنا نقوم بتعريف الحدود لكل الحالات وسيقوم فلتر بمعالجة حجم الخطأ تلقائياً
161     border: OutlineInputBorder(
162       borderRadius: BorderRadius.circular(24),
163       borderSide: BorderSide(color: ColorsManager.borderColor),
164     ),
165     enabledBorder: OutlineInputBorder(
166       borderRadius: BorderRadius.circular(24),
167       borderSide: BorderSide(color: ColorsManager.borderColor),
168     ),
169     focusedBorder: OutlineInputBorder(
170       borderRadius: BorderRadius.circular(24),
171       borderSide: BorderSide(color: ColorsManager.themeActiveAccent),
172     ),
173     errorBorder: OutlineInputBorder(
174       borderRadius: BorderRadius.circular(24),
175       borderSide: const BorderSide(color: Colors.red), // لون الحقل عند الخطأ
176     ),
177     focusedErrorBorder: OutlineInputBorder(
178       borderRadius: BorderRadius.circular(24),
179       borderSide: const BorderSide(color: Colors.red, width: 2),
180     ),
181   ),
182 );
183 },
184 );
185 }
186 }
187

```

Lines • Dart 127

Core / Utils

#36 – lib/core/utils/constants/primary/primary_two_week_calendar.dart

```

1 // import 'package:flutter/material.dart';
2 // import 'package:intl/intl.dart';
3 // import 'package:moean/core/theme/colors.dart';
4 // import 'package:moean/core/theme/text_styles.dart';
5 // import 'package:moean/core/utils/constants/spacing.dart';
6
7 // class PrimaryTwoWeekCalendar extends StatelessWidget {
8 //   final DateTime currentWeekStart;
9 //   final DateTime selectedDate;
10 //   final VoidCallback onPreviousWeek;
11 //   final VoidCallback onNextWeek;
12 //   final Function(DateTime) onDateSelected;
13
14 //   const PrimaryTwoWeekCalendar({
15 //     super.key,
16 //     required this.currentWeekStart,
17 //     required this.selectedDate,
18 //     required this.onPreviousWeek,
19 //     required this.onNextWeek,
20 //     required this.onDateSelected,
21 //   });
22
23 //   @override
24 //   Widget build(BuildContext context) {
25 //     return Container(
26 //       padding: const EdgeInsets.all(16),
27 //       decoration: BoxDecoration(
28 //         color: ColorsManager.surfacePrimary,
29 //         borderRadius: BorderRadius.circular(24),
30 //         border: Border.all(color: ColorsManager.borderColor),
31 //         boxShadow: [
32 //           BoxShadow(
33 //             color: Colors.black.withValues(alpha: 0.04),
34 //             blurRadius: 20,
35 //             offset: const Offset(0, 10),
36 //           ),
37 //         ],
38 //       ),
39 //       child: Column(

```

```

40 //      crossAxisAlignment: CrossAxisAlignment.start,
41 //      children: [
42 //          Row(
43 //              mainAxisAlignment: MainAxisAlignment.spaceBetween,
44 //              children: [
45 //                  Text(
46 //                      DateFormat('MMMM yyyy').format(currentWeekStart),
47 //                      style: TextStylesManager.bold16.copyWith(color: ColorsManager.textPrimary),
48 //                  ),
49 //                  Row(
50 //                      children: [
51 //                          IconButton(
52 //                              icon: const Icon(Icons.chevron_left, size: 24),
53 //                              onPressed: onPreviousWeek,
54 //                          ),
55 //                          IconButton(
56 //                              icon: const Icon(Icons.chevron_right, size: 24),
57 //                              onPressed: onNextWeek,
58 //                          ),
59 //                      ],
60 //                  ),
61 //              ],
62 //          ),
63 //          verticalSpace12,
64 //          Row(
65 //              mainAxisAlignment: MainAxisAlignment.spaceAround,
66 //              children: List.generate(7, (index) {
67 //                  final dayDate = currentWeekStart.add(Duration(days: index));
68 //                  final dayStr = DateFormat('EE').format(dayDate).toUpperCase(); // e.g., MO, TU
69 //                  return SizedBox(
70 //                      width: 32,
71 //                      child: Center(
72 //                          child: Text(
73 //                              dayStr.substring(0, 2),
74 //                              style: TextStylesManager.bold12.copyWith(color: Colors.grey),
75 //                          ),
76 //                      ),
77 //                  );
78 //              }),
79 //          ),
80 //          verticalSpace8,
81 //          GridView.builder(
82 //              shrinkWrap: true,
83 //              physics: const NeverScrollableScrollPhysics(),
84 //              gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
85 //                  crossAxisCount: 7,
86 //                  mainAxisSpacing: 8,
87 //                  crossAxisSpacing: 8,
88 //              ),
89 //              itemCount: 14,
90 //              itemBuilder: (context, index) {
91 //                  final date = currentWeekStart.add(Duration(days: index));
92 //                  final isSelected = date.year == selectedDate.year &&
93 //                      date.month == selectedDate.month &&
94 //                      date.day == selectedDate.day;
95 //
96 //                  final now = DateTime.now();
97 //                  final today = DateTime(now.year, now.month, now.day);
98 //                  final currentDate = DateTime(date.year, date.month, date.day);
99 //                  final isBeforeToday = currentDate.isBefore(today);
100 //
101 //                  return GestureDetector(
102 //                      onTap: isBeforeToday ? null : () => onDateSelected(date),
103 //                      child: Container(
104 //                          decoration: BoxDecoration(
105 //                              color: isSelected ? ColorsManager.primaryColor : Colors.transparent,
106 //                              shape: BoxShape.circle,
107 //                          ),
108 //                          child: Center(
109 //                              child: Text(
110 //                                  '${date.day}',
111 //                                  style: TextStylesManager.medium14.copyWith(
112 //                                      color: isSelected
113 //                                          ? Colors.white
114 //                                          : (isBeforeToday ? Colors.grey.withValues(alpha: 0.3) : ColorsManager.textPrim

```

```

ary),
115 //           ),
116 //           ),
117 //           ),
118 //           ),
119 //           );
120 //       },
121 //     ),
122 //   ],
123 // ),
124 // );
125 // }
126 // }
127

```

Lines • Dart 119

Core / Utils

#37 – lib/core/utils/constants/primary/upgrade_prompt_bottom_sheet.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
6 import 'package:moean/core/utils/constants/routes.dart';
7 import 'package:moean/core/utils/constants/spacing.dart';
8 import 'package:moean/core/utils/extensions/context_extension.dart';
9
10 class UpgradePromptBottomSheet extends StatelessWidget {
11   final String message;
12   final bool isQuotaExceeded;
13
14   const UpgradePromptBottomSheet({
15     super.key,
16     required this.message,
17     required this.isQuotaExceeded,
18   });
19
20   static Future<void> show(BuildContext context, {required String message, required bool isQuotaExceeded}) {
21     return showModalBottomSheet(
22       context: context,
23       isScrollControlled: true,
24       backgroundColor: ColorsManager.surfacePrimary,
25       shape: const RoundedRectangleBorder(
26         borderRadius: BorderRadius.vertical(top: Radius.circular(24)),
27       ),
28       builder: (context) => UpgradePromptBottomSheet(
29         message: message,
30         isQuotaExceeded: isQuotaExceeded,
31       ),
32     );
33   }
34
35   @override
36   Widget build(BuildContext context) {
37     return Padding(
38       padding: EdgeInsets.only(
39         left: 20,
40         right: 20,
41         top: 32,
42         bottom: MediaQuery.of(context).padding.bottom + 20,
43       ),
44       child: Column(
45         mainAxisAlignment: MainAxisAlignment.min,
46         crossAxisAlignment: CrossAxisAlignment.center,
47         children: [
48           Icon(
49             isQuotaExceeded ? Icons.hourglass_empty_rounded : Icons.workspace_premium_rounded,
50             size: 64,
51             color: ColorsManager.primaryColor,
52           ),
53           verticalSpace16,
54           Text(
55             isQuotaExceeded ? 'نفد الرصيد!' : 'ترقية الحساب المطلوبة',
56             style: TextStylesManager.bold20.copyWith(color: ColorsManager.mainText),

```



```

57         textAlign: TextAlign.center,
58     ),
59     verticalSpace8,
60     Text(
61         message,
62         style: TextStyleManager.regular14.copyWith(color: ColorsManager.secondaryText),
63         textAlign: TextAlign.center,
64     ),
65     verticalSpace24,
66     Container(
67         padding: const EdgeInsets.all(16),
68         decoration: BoxDecoration(
69             color: ColorsManager.primaryColor.withValues(alpha: 0.1),
70             borderRadius: BorderRadius.circular(16),
71         ),
72         child: Column(
73             children: [
74                 _buildBenefit('وصول غير محدود لكل المزايا'),
75                 verticalSpace8,
76                 _buildBenefit('أولوية الدعم الفني والمساعدة'),
77                 verticalSpace8,
78                 _buildBenefit('إعداد الدروس بنقرة واحدة'),
79             ],
80         ),
81     ),
82     verticalSpace32,
83     PrimaryElevatedButton(
84         text: 'عرض باقات الاشتراك',
85         onPressed: () {
86             Navigator.pop(context);
87             context.push(Routes.checkout);
88         },
89         width: double.infinity,
90     ),
91     verticalSpace16,
92     TextButton(
93         onPressed: () => Navigator.pop(context),
94         child: Text(
95             'إخفاء',
96             style: TextStyleManager.bold14.copyWith(color: ColorsManager.secondaryText),
97         ),
98     ),
99 ],
100 ),
101 );
102 }
103
104 Widget _buildBenefit(String text) {
105     return Row(
106         children: [
107             Icon(Icons.check_circle_rounded, color: ColorsManager.primaryColor, size: 20),
108             horizontalSpace12,
109             Expanded(
110                 child: Text(
111                     text,
112                     style: TextStyleManager.medium14.copyWith(color: ColorsManager.mainText),
113                 ),
114             ),
115         ],
116     );
117 }
118 }
119

```

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/di/injections.dart';
3 import 'package:moean/features/home/presentation/widgets/chose_app.dart';
4 import 'package:moean/features/home/presentation/widgets/download_extention.dart';
5 import 'package:moean/features/home/presentation/widgets/extension_usage_slider.dart';
6 import 'package:moean/features/layout/presentation/screen/main_layout_screen.dart';
7 import 'package:moean/features/login/presentation/screen/login_screen.dart';

```

```

8 import 'package:moean/features/login/presentation/screen/microsoft_login_screen.dart';
9 import 'package:moean/features/register/presentation/screen/register_screen.dart';
10 import 'package:moean/features/schedule/presentation/screen/schedule_screen.dart';
11 import 'package:moean/features/profile/presentation/screen/profile_screen.dart';
12 import 'package:moean/features/profile/presentation/screen/settings_screen.dart';
13 import 'package:moean/features/profile/presentation/screen/change_password_screen.dart';
14 import 'package:moean/features/verify_email/presentation/screen/verify_email_screen.dart';
15 import 'package:moean/features/forgot_password/presentation/screen/forgot_password_screen.dart';
16 import 'package:moean/features/forgot_password/presentation/screen/reset_password_screen.dart';
17 import 'package:moean/features/account_suspended/presentation/screen/account_suspended_screen.dart';
18
19 import 'package:moean/features/admin/teachers/presentation/screen/admin_teachers_screen.dart';
20 import 'package:moean/features/admin/contact/presentation/screen/admin_contact_screen.dart';
21 import 'package:moean/features/admin/contact/presentation/screen/admin_ticket_details_screen.dart';
22 import 'package:moean/features/admin/contact/presentation/cubit/admin_contact_cubit.dart';
23 import 'package:moean/features/admin/payments/presentation/screen/admin_payments_screen.dart';
24 import 'package:moean/features/contact/presentation/screen/contact_support_screen.dart';
25 import 'package:moean/features/contact/presentation/screen/create_ticket_screen.dart';
26 import 'package:moean/features/contact/presentation/screen/my_tickets_screen.dart';
27 import 'package:flutter_bloc/flutter_bloc.dart';
28 import 'package:moean/features/contact/presentation/cubit/contact_cubit.dart';
29 import 'package:moean/features/contact/presentation/screen/ticket_details_screen.dart';
30 import 'package:moean/features/reports/presentation/screen/report_screen.dart';
31 import 'package:moean/features/reports/presentation/cubit/report_cubit.dart';
32 import 'package:moean/features/payment/presentation/cubit/payment_cubit.dart';
33 import 'package:moean/features/payment/presentation/screen/checkout_screen.dart';
34 import 'package:moean/features/payment/presentation/screen/bank_transfer_screen.dart';
35 import 'package:moean/features/payment/presentation/screen/myfatoorah_payment_screen.dart';
36 import 'package:moean/features/payment/presentation/screen/payment_result_screen.dart';
37 import 'package:moean/features/payment/presentation/screen/payment_history_screen.dart';
38
39 // Exam Generation Imports
40 import 'package:moean/features/exam_generation/presentation/screens/exam_info_screen.dart';
41 import 'package:moean/features/exam_generation/presentation/screens/lesson_selection_screen.dart';
42 import 'package:moean/features/exam_generation/presentation/screens/question_counts_screen.dart';
43 import 'package:moean/features/exam_generation/presentation/screens/exam_preview_screen.dart';
44 import 'package:moean/features/exam_generation/presentation/cubit/exam_info_cubit.dart';
45 import 'package:moean/features/exam_generation/presentation/cubit/lesson_selection_cubit.dart';
46 import 'package:moean/features/exam_generation/presentation/cubit/question_count_cubit.dart';
47 import 'package:moean/features/exam_generation/presentation/cubit/exam_preview_cubit.dart';
48 import 'package:moean/features/exam_generation/presentation/cubit/generate_exam_cubit.dart';
49 import 'package:moean/features/exam_generation/presentation/cubit/my_exams_cubit.dart';
50 import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
51 import 'package:moean/features/exam_generation/presentation/screens/my_exams_screen.dart';
52 import 'package:moean/features/exam_generation/presentation/screens/custom_questions_screen.dart';
53 import 'package:moean/features/exam_generation/presentation/cubit/custom_questions_cubit.dart';
54 import 'package:moean/features/exam_generation/presentation/screens/manual_exam_screen.dart';
55 import 'package:moean/features/exam_generation/presentation/screens/bank_questions_screen.dart';
56 import 'package:moean/features/exam_generation/presentation/cubit/bank_questions_cubit.dart';
57 import 'package:moean/features/admin/exams/presentation/screen/admin_exams_screen.dart';
58 import 'package:moean/features/admin/exams/presentation/cubit/admin_exams_cubit.dart';
59 import 'package:moean/features/search/presentation/screen/search_screen.dart';
60 import 'package:moean/features/search/presentation/cubit/search_cubit.dart';
61 import 'package:moean/features/certificates/presentation/screens/certificate_screen.dart';
62 import 'package:moean/features/certificates/presentation/cubit/certificate_cubit.dart';
63 import 'package:moean/features/presentations/presentation/screens/presentations_screen.dart';
64 import 'package:moean/features/presentations/presentation/cubit/presentations_cubit.dart';
65 import 'package:moean/features/presentations/data/repositories/presentations_repository.dart';
66 import 'package:moean/features/referral/presentation/cubit/referral_cubit.dart';
67 import 'package:moean/features/referral/presentation/screen/referral_screen.dart';
68 import 'package:moean/features/admin/promo/presentation/cubit/admin_promo_cubit.dart';
69 import 'package:moean/features/admin/promo/presentation/screen/admin_promo_screen.dart';
70 import 'package:moean/features/privacy_policy/presentation/cubit/privacy_policy_cubit.dart';
71 import 'package:moean/features/privacy_policy/presentation/screen/privacy_policy_screen.dart';
72 import 'package:moean/features/curriculum/presentation/cubit/curriculum_distribution_cubit.dart';
73 import 'package:moean/features/curriculum/presentation/cubit/curriculum_books_cubit.dart';
74 import 'package:moean/features/curriculum/presentation/screens/curriculum_distribution_screen.dart';
75 import 'package:moean/features/curriculum/presentation/screens/curriculum_books_screen.dart';
76
77 class Routes {
78   static const String splash = '/splash';
79   static const String onboarding = '/onboarding';
80   static const String home = '/';
81   static const String login = '/login';
82   static const String register = '/register';

```

```

83 static const String forgotPassword = '/forgot-password';
84 static const String resetPassword = '/reset-password';
85 static const String accountSuspended = '/account-suspended';
86 static const String privacyPolicy = '/privacy-policy';
87 static const String verifyEmail = '/verify-email';
88 static const String loginMicrosoft = '/login/microsoft';
89 static const String schedule = '/schedule';
90 static const String profile = '/profile';
91 static const String settings = '/settings';
92 static const String changePassword = '/change-password';
93 static const String addextension = '/add-extension';
94 static const String choseapp = '/chose-app';
95 static const String extensionUsage = '/extension-usage';
96 static const String adminTeachers = '/admin/teachers';
97 static const String adminContact = '/admin/contact';
98 static const String adminTicketDetails = '/admin/contact/details';
99 static const String adminPayments = '/admin/payments';
100 static const String contact = '/contact';
101 static const String createTicket = '/contact/create-ticket';
102 static const String myTickets = '/contact/my-tickets';
103 static const String ticketDetails = '/contact/my-tickets/details';
104 static const String reports = '/reports';
105 static const String checkout = '/checkout';
106 static const String bankTransfer = '/bank-transfer';
107 static const String myfatoorahPayment = '/myfatoorah-payment';
108 static const String paymentResult = '/payment-result';
109 static const String paymentHistory = '/payment-history';
110
111 // Exam Generation
112 static const String examGenerationInfo = '/exam-generation/info';
113 static const String examGenerationLessons = '/exam-generation/lessons';
114 static const String examGenerationCounts = '/exam-generation/counts';
115 static const String examGenerationPreview = '/exam-generation/preview';
116 static const String myExams = '/my-exams';
117 static const String customQuestions = '/my-custom-questions';
118 static const String manualExam = '/manual-exam';
119 static const String examBankQuestions = '/exam-generation/bank-questions';
120 static const String adminExams = '/admin/exams';
121 static const String search = '/search';
122 static const String certificates = '/certificates';
123 static const String presentations = '/presentations';
124 static const String referral = '/referral';
125 static const String adminPromo = '/admin/promo';
126 static const String curriculumDistribution = '/curriculum/distribution';
127 static const String curriculumBooks = '/curriculum/books';
128
129
130
131 static Map<String, WidgetBuilder> get routes => {
132   home: (context) => const MainLayoutScreen(),
133   login: (context) => const LoginScreen(),
134   register: (context) => const RegisterScreen(),
135   forgotPassword: (context) => const ForgotPasswordScreen(),
136   resetPassword: (context) {
137     final args = ModalRoute.of(context)?.settings.arguments;
138     String token = '';
139     String email = '';
140     if (args is Map<String, dynamic>) {
141       token = args['token'] as String? ?? '';
142       email = args['email'] as String? ?? '';
143     }
144     return ResetPasswordScreen(token: token, email: email);
145   },
146   accountSuspended: (context) {
147     final args = ModalRoute.of(context)?.settings.arguments;
148     String? email;
149     String? password;
150     String? name;
151     String? phone;
152     if (args is Map<String, dynamic>) {
153       email = args['email'] as String?;
154       password = args['password'] as String?;
155       name = args['name'] as String?;
156       phone = args['phone'] as String?;
157     }

```

```

158     return AccountSuspendedScreen(
159         email: email,
160         password: password,
161         name: name,
162         phone: phone,
163     );
164 },
165 verifyEmail: (context) {
166     final email = ModalRoute.of(context)?.settings.arguments as String? ?? '';
167     return VerifyEmailScreen(email: email);
168 },
169 loginMicrosoft: (context) => const MicrosoftLoginScreen(),
170 schedule: (context) => const ScheduleScreen(),
171 profile: (context) => const ProfileScreen(),
172 settings: (context) => const SettingsScreen(),
173 changePassword: (context) => const ChangePasswordScreen(),
174 addextension: (context) => const DownloadExtension(),
175 choseapp: (context) => const ChoseApp(),
176 extensionUsage: (context) => const ExtensionUsageSlider(),
177 adminTeachers: (context) => const AdminTeachersScreen(),
178 adminContact: (context) => const AdminContactScreen(),
179 adminTicketDetails: (context) => BlocProvider(create: (context) => AdminContactCubit(), child: const Admin
nTicketDetailsScreen()),
180 adminPayments: (context) => const AdminPaymentsScreen(),
181 adminExams: (context) => BlocProvider(create: (_) => sl<AdminExamsCubit>()..fetchPendingQuestions(), chil
d: const AdminExamsScreen()),
182 contact: (context) => BlocProvider(create: (context) => ContactCubit(), child: const ContactSupportScreen
()),
183 createTicket: (context) => BlocProvider(create: (context) => ContactCubit(), child: const CreateTicketScr
een()),
184 myTickets: (context) => BlocProvider(create: (context) => ContactCubit(), child: const MyTicketsScreen
()),
185 ticketDetails: (context) => BlocProvider(create: (context) => ContactCubit(), child: const TicketDetailsS
creen()),
186 reports: (context) => BlocProvider(create: (_) => ReportCubit(), child: const ReportScreen()),
187 checkout: (context) => BlocProvider(create: (_) => PaymentCubit(), child: const CheckoutScreen()),
188 bankTransfer: (context) => BlocProvider(create: (_) => PaymentCubit(), child: const BankTransferScreen
()),
189 myfatoorahPayment: (context) => BlocProvider(create: (_) => PaymentCubit(), child: const MyfatoorahPaymen
tScreen()),
190 paymentResult: (context) => const PaymentResultScreen(),
191 paymentHistory: (context) => BlocProvider(create: (_) => PaymentCubit(), child: const PaymentHistoryScre
n()),
192 examGenerationInfo: (context) => BlocProvider.value(value: sl<ExamInfoCubit>(), child: const ExamInfoScre
en()),
193 examGenerationLessons: (context) => MultiBlocProvider(
194     providers: [
195         BlocProvider.value(value: sl<LessonSelectionCubit>()),
196         BlocProvider.value(value: sl<ExamInfoCubit>()),
197     ],
198     child: const LessonSelectionScreen(),
199 ),
200 examGenerationCounts: (context) => MultiBlocProvider(
201     providers: [
202         BlocProvider.value(value: sl<QuestionCountCubit>()),
203         BlocProvider(create: (_) => sl<GenerateExamCubit>()),
204         BlocProvider.value(value: sl<LessonSelectionCubit>()),
205         BlocProvider.value(value: sl<ExamInfoCubit>()),
206     ],
207     child: const QuestionCountsScreen(),
208 ),
209 myExams: (context) => BlocProvider(
210     create: (_) => sl<MyExamsCubit>(),
211     child: const MyExamsScreen(),
212 ),
213 customQuestions: (context) => BlocProvider(
214     create: (_) => sl<CustomQuestionsCubit>(),
215     child: const CustomQuestionsScreen(),
216 ),
217 examGenerationPreview: (context) {
218     final initialExam = ModalRoute.of(context)!.settings.arguments as ExamEntity;
219     return BlocProvider(
220         create: (_) => sl<ExamPreviewCubit>(),
221         child: ExamPreviewScreen(initialExam: initialExam),
222     );

```

```

223     },
224     manualExam: (context) => const ManualExamScreen(),
225     search: (context) => BlocProvider(
226       create: (_) => SearchCubit(),
227       child: const SearchScreen(),
228     ),
229     certificates: (context) => BlocProvider(
230       create: (_) => CertificateCubit(),
231       child: const CertificateScreen(),
232     ),
233     examBankQuestions: (context) => MultiBlocProvider(
234       providers: [
235         BlocProvider(create: (_) => sl<BankQuestionsCubit>()),
236         BlocProvider.value(value: sl<LessonSelectionCubit>()),
237         BlocProvider.value(value: sl<ExamInfoCubit>()),
238       ],
239       child: const BankQuestionsScreen(),
240     ),
241     presentations: (context) => BlocProvider(
242       create: (_) => PresentationsCubit(PresentationsRepository()),
243       child: const PresentationsScreen(),
244     ),
245     referral: (context) => BlocProvider(
246       create: (_) => ReferralCubit(),
247       child: const ReferralScreen(),
248     ),
249     adminPromo: (context) => BlocProvider(
250       create: (_) => AdminPromoCubit(),
251       child: const AdminPromoScreen(),
252     ),
253     privacyPolicy: (context) => BlocProvider(
254       create: (_) => PrivacyPolicyCubit(),
255       child: const PrivacyPolicyScreen(),
256     ),
257     curriculumDistribution: (context) => MultiBlocProvider(
258       providers: [
259         BlocProvider(create: (_) => CurriculumDistributionCubit()),
260         BlocProvider(create: (_) => CurriculumBooksCubit()),
261       ],
262       child: const CurriculumDistributionScreen(),
263     ),
264     curriculumBooks: (context) => MultiBlocProvider(
265       providers: [
266         BlocProvider(create: (_) => CurriculumDistributionCubit()),
267         BlocProvider(create: (_) => CurriculumBooksCubit()),
268       ],
269       child: const CurriculumBooksScreen(),
270     ),
271   };
272 }
273

```

Lines • Dart 8 Core / Utils

#39 – lib/core/utils/constants/secrets.dart

```

1 class Secrets {
2   static const String myfatoorahApiKey = '*****_REDACTED_API_KEY_FOR_SAIP_REGISTRATION_*****';
3   static const String myfatoorahBaseUrl = 'https://api-sa.myfatoorah.com/';
4   static const String myfatoorahPortalHost = 'https://sa.myfatoorah.com/';
5   static const String myfatoorahCallbackUrl = 'http://localhost:3000/payment/myfatoorah/callback';
6   static const String myfatoorahErrorUrl = 'http://localhost:3000/payment/myfatoorah/error';
7 }
8

```

Lines • Dart 126 Core / Utils

#40 – lib/core/utils/constants/spacing.dart

```

1 import 'package:flutter/material.dart';
2
3 SizedBox get horizontalSpace2 => const SizedBox(width: 2);
4
5 SizedBox get horizontalSpace3 => const SizedBox(width: 3);
6
7 SizedBox get horizontalSpace4 => const SizedBox(width: 4);

```

```
8
9   SizedBox get horizontalSpace6 => const SizedBox(width: 6);
10
11   SizedBox get horizontalSpace8 => const SizedBox(width: 8);
12
13   SizedBox get horizontalSpace14 => const SizedBox(width: 14);
14
15   SizedBox get horizontalSpace10 => const SizedBox(width: 10);
16
17   SizedBox get horizontalSpace12 => const SizedBox(width: 12);
18
19   SizedBox get horizontalSpace16 => const SizedBox(width: 16);
20
21   SizedBox get horizontalSpace18 => const SizedBox(width: 18);
22
23   SizedBox get horizontalSpace20 => const SizedBox(width: 20);
24
25   SizedBox get horizontalSpace22 => const SizedBox(width: 22);
26
27   SizedBox get horizontalSpace23 => const SizedBox(width: 23);
28
29   SizedBox get horizontalSpace24 => const SizedBox(width: 24);
30
31   SizedBox get horizontalSpace26 => const SizedBox(width: 26);
32
33   SizedBox get horizontalSpace30 => const SizedBox(width: 30);
34
35   SizedBox get horizontalSpace32 => const SizedBox(width: 32);
36
37   SizedBox get horizontalSpace48 => const SizedBox(width: 48);
38
39   SizedBox get horizontalSpace96 => const SizedBox(width: 96);
40
41   SizedBox get verticalSpace2 => const SizedBox(height: 2);
42
43   SizedBox get verticalSpace4 => const SizedBox(height: 4);
44
45   SizedBox get verticalSpace5 => const SizedBox(height: 5);
46
47   SizedBox get verticalSpace6 => const SizedBox(height: 6);
48
49   SizedBox get verticalSpace7 => const SizedBox(height: 7);
50
51   SizedBox get verticalSpace8 => const SizedBox(height: 8);
52
53   SizedBox get verticalSpace10 => const SizedBox(height: 10);
54
55   SizedBox get verticalSpace12 => const SizedBox(height: 12);
56
57   SizedBox get verticalSpace14 => const SizedBox(height: 12);
58
59   SizedBox get verticalSpace16 => const SizedBox(height: 16);
60
61   SizedBox get verticalSpace18 => const SizedBox(height: 16);
62
63   SizedBox get verticalSpace20 => const SizedBox(height: 20);
64
65   SizedBox get verticalSpace22 => const SizedBox(height: 22);
66
67   SizedBox get verticalSpace24 => const SizedBox(height: 24);
68
69   SizedBox get verticalSpace26 => const SizedBox(height: 26);
70
71   SizedBox get verticalSpace28 => const SizedBox(height: 28);
72
73   SizedBox get verticalSpace30 => const SizedBox(height: 30);
74
75   SizedBox get verticalSpace32 => const SizedBox(height: 32);
76
77   SizedBox get verticalSpace38 => const SizedBox(height: 38);
78
79   SizedBox get verticalSpace40 => const SizedBox(height: 40);
80
81   SizedBox get verticalSpace44 => const SizedBox(height: 44);
82
```

```

83   SizedBox get verticalSpace48 => const SizedBox(height: 48);
84
85   SizedBox get verticalSpace50 => const SizedBox(height: 50);
86
87   SizedBox get verticalSpace52 => const SizedBox(height: 52);
88
89   SizedBox get verticalSpace54 => const SizedBox(height: 54);
90
91   SizedBox get verticalSpace56 => const SizedBox(height: 56);
92
93   SizedBox get verticalSpace58 => const SizedBox(height: 58);
94
95   SizedBox get verticalSpace60 => const SizedBox(height: 60);
96
97   SizedBox get verticalSpace62 => const SizedBox(height: 62);
98
99   SizedBox get verticalSpace64 => const SizedBox(height: 64);
100
101   SizedBox get verticalSpace66 => const SizedBox(height: 66);
102
103   SizedBox get verticalSpace68 => const SizedBox(height: 68);
104
105   SizedBox get verticalSpace70 => const SizedBox(height: 70);
106
107   SizedBox get verticalSpace80 => const SizedBox(height: 80);
108
109   SizedBox get verticalSpace85 => const SizedBox(height: 85);
110
111   SizedBox get verticalSpace90 => const SizedBox(height: 90);
112
113   SizedBox get verticalSpace95 => const SizedBox(height: 95);
114
115   SizedBox get verticalSpace96 => const SizedBox(height: 96);
116
117   SizedBox get verticalSpace100 => const SizedBox(height: 100);
118
119   SizedBox get verticalSpace120 => const SizedBox(height: 120);
120
121   SizedBox get verticalSpace140 => const SizedBox(height: 140);
122
123   SizedBox get verticalSpace160 => const SizedBox(height: 160);
124
125   SizedBox get verticalSpace500 => const SizedBox(height: 500);
126

```

Lines • Dart 9 Core / Utils

#41 – lib/core/utils/constants/translations.dart

```

1  class TranslationModel {
2    final Map<String, String> data;
3
4    TranslationModel.fromJson(Map<String, dynamic> json)
5      : data = json.map((key, value) => MapEntry(key, value.toString()));
6
7    String get(String key) => data[key] ?? key;
8  }
9

```

Lines • Dart 799 Core / Utils

#42 – lib/core/utils/cubit/auth/auth_cubit.dart

```

1  // import 'dart:async';
2  // import 'dart:convert';
3
4  // import 'package:moean/core/network/local/cache_helper.dart';
5  // import 'package:moean/core/network/remote/api_endpoints.dart';
6  // import 'package:moean/core/network/remote/dio_helper.dart';
7  // import 'package:moean/core/utils/constants/constants.dart';
8  // import 'package:moean/core/utils/cubit/auth/auth_state.dart';
9  // import 'package:moean/core/models/user_model.dart';
10 // import 'package:moean/core/models/register_request.dart';
11 // import 'package:moean/core/network/remote/api_service.dart';
12 // import 'package:moean/core/helpers/fcm_helper.dart';
13 // import 'package:moean/main.dart';

```

```

14 // import 'package:flutter/material.dart';
15 // import 'package:flutter_bloc/flutter_bloc.dart';
16 // import 'package:moean/core/di/injections.dart';
17 // import 'package:moean/features/profile/presentation/cubit/profile_cubit.dart';
18 // import 'package:moean/features/notifications/logic/cubit/notification_cubit.dart';
19
20
21 // AuthCubit get authCubit => AuthCubit.get(navigatorKey.currentContext!);
22
23 // class AuthCubit extends Cubit<AuthState> {
24 //   AuthCubit() : super(AuthInitialState());
25 //   int registerCallCount = 0;
26 //   int otpCount = 0;
27
28 //   static AuthCubit get (BuildContext context) => BlocProvider.of(context);
29
30 //   // Login Controllers
31 //   final TextEditingController emailController = TextEditingController();
32 //   final TextEditingController passwordController = TextEditingController();
33
34 //   // Register Controllers
35 //   final TextEditingController fullNameController = TextEditingController();
36 //   final TextEditingController phoneController = TextEditingController();
37 //   final TextEditingController nationalIdController = TextEditingController();
38 //   final TextEditingController otpController = TextEditingController();
39 //   final TextEditingController confirmPasswordController =
40 //     TextEditingController();
41 //   final TextEditingController nationalIdRegisterController =
42 //     TextEditingController();
43 //   final TextEditingController dateController = TextEditingController();
44
45 //   // Forgot Password Controllers
46 //   final TextEditingController forgotLoginController = TextEditingController();
47 //   final TextEditingController resetOtpController = TextEditingController();
48 //   final TextEditingController newPasswordController = TextEditingController();
49 //   final TextEditingController confirmNewPasswordController =
50 //     TextEditingController();
51
52 //   bool isShowNewPassword = false;
53 //   bool isShowConfirmNewPassword = false;
54 //   bool isShowCurrentPassword = false;
55 //   bool isShowChangePassword = false;
56 //   bool isShowConfirmChangePassword = false;
57
58 //   final TextEditingController currentPasswordController = TextEditingController();
59 //   final TextEditingController changePasswordController = TextEditingController();
60 //   final TextEditingController confirmChangePasswordController = TextEditingController();
61
62 //   bool isShowPassword = false;
63 //   bool isShowConfirmPassword = false;
64 //   int selectedGender = 0; // 0 for male, 1 for female
65 //   bool isAgreedToTerms = false;
66
67 //   // Resend OTP Timer
68 //   Timer? _resendTimer;
69 //   int resendCountdown = 0;
70
71 //   void startResendTimer() {
72 //     resendCountdown = 180; // 3 minutes
73 //     _resendTimer?.cancel();
74 //     _resendTimer = Timer.periodic(const Duration(seconds: 1), (timer) {
75 //       if (resendCountdown > 0) {
76 //         resendCountdown--;
77 //         notifyUI();
78 //       } else {
79 //         _resendTimer?.cancel();
80 //         notifyUI();
81 //       }
82 //     });
83 //   }
84
85 //   @override
86 //   Future<void> close() {
87 //     _resendTimer?.cancel();
88 //     return super.close();

```



```

89 // }
90
91 // void changePasswordVisibility() {
92 //   isShowPassword = !isShowPassword;
93 //   emit(AuthShowPasswordState());
94 // }
95
96 // void changeConfirmPasswordVisibility() {
97 //   isShowConfirmPassword = !isShowConfirmPassword;
98 //   emit(AuthShowPasswordState());
99 // }
100
101 // void notifyUI() => emit(AuthUpdateUIState());
102
103 // void changeGender(int value) {
104 //   selectedGender = value;
105 //   emit(AuthUpdateUIState());
106 // }
107
108 // void toggleTermsAgreement(bool value) {
109 //   isAgreedToTerms = value;
110 //   emit(AuthUpdateUIState());
111 // }
112
113 // Future<void> selectDate(BuildContext context) async {
114 //   final DateTime? picked = await showDatePicker(
115 //     context: context,
116 //     initialDate: DateTime.now(),
117 //     firstDate: DateTime(1900),
118 //     lastDate: DateTime.now(),
119 //   );
120
121 //   if (picked != null) {
122 //     final formattedDate =
123 //       "${picked.year}-${picked.month.toString().padLeft(2, '0')}-${picked.day.toString().padLeft(2,
'0')}";
124 //     dateController.text = formattedDate;
125 //     emit(AuthUpdateUIState());
126 //   }
127 // }
128
129 // UserModel? userModel;
130 // int? pendingUserId;
131
132 // Future<void> login() async {
133 //   emit(AuthLoginLoadingState());
134
135 //   final String loginValue = emailController.text.trim();
136 //   final String passwordValue = passwordController.text.trim();
137
138 //   debugPrint('🔍 Debug Login - Login: $loginValue');
139 //   debugPrint('🔍 Debug Login - Password: $passwordValue');
140
141 //   final result = await DioHelper.postData(
142 //     url: loginApi,
143 //     data: {
144 //       'login': loginValue,
145 //       'password': passwordValue,
146 //     },
147 //   );
148 //   result.fold(
149 //     (failure) {
150 //       debugPrint('❌ Login Error: $failure');
151 //       emit(AuthLoginErrorState(message: failure.toString()));
152 //     },
153 //     (response) {
154 //       final responseData = response.data as Map<String, dynamic>;
155
156 //       if (responseData == null) {
157 //         emit(AuthLoginErrorState(message: 'Invalid login response'));
158 //         return;
159 //       }
160
161 //       final bool isSuccess =
162 //         (responseData['success'] == true ||

```

```

163 //      (responseData['data'] != null &&
164 //      (responseData['data']['token'] != null ||
165 //      responseData['data']['access_token'] != null)) ||
166 //      responseData['token'] != null ||
167 //      responseData['access_token'] != null ||
168 //      (responseData['message']?.toString().toLowerCase().contains(
169 //      'success',
170 //      ) ??
171 //      false));
172
173 //      if (isSuccess) {
174 //      final bool hasToken = _storeTokenFromResponse(responseData);
175 //      if (!hasToken) {
176 //      debugPrint('❌ Login response has no token: $responseData');
177 //      emit(
178 //      AuthLoginErrorState(
179 //      message: 'Login succeeded but token is missing. Please retry.',
180 //      ),
181 //      );
182 //      return;
183 //      }
184
185 //      _handleLoginSuccess(responseData);
186 //      debugPrint('✅ Login Success Header: ${responseData['message']}');
187 //      emit(AuthLoginSuccessState());
188 //      } else {
189 //      debugPrint('❌ Login Failed Header: ${responseData['message']}');
190 //      emit(
191 //      AuthLoginErrorState(
192 //      message: responseData['message'] ?? 'Login failed',
193 //      ),
194 //      );
195 //      }
196 //      },
197 //      );
198 //      }
199
200 //      Future<void> register() async {
201 //      registerCallCount++;
202
203 //      debugPrint(
204 //      '🔴 Register button pressed: $registerCallCount time(s)'
205 //      );
206
207 //      emit(AuthRegisterLoadingState());
208 //      final request = RegisterRequest(
209 //      name: fullNameController.text.trim(),
210 //      phone: phoneController.text.trim(),
211 //      email: emailController.text.trim(),
212 //      password: passwordController.text,
213 //      passwordConfirmation: confirmPasswordController.text,
214 //      nationalId: nationalIdController.text.trim(),
215 //      dateOfBirth: dateController.text.trim(),
216 //      gender: selectedGender == 0 ? 'male' : 'female',
217 //      );
218 //      debugPrint('🔴 Registration Request: ${request.toJson()}');
219 //      debugPrint('🔴 Calling Register API...');
220 //      try {
221 //      final response = await ApiService.registerUser(request);
222
223 //      final bool isSuccess =
224 //      response.success ||
225 //      (response.message?.toLowerCase().contains('success') ?? false);
226
227 //      if (isSuccess) {
228 //      debugPrint('✅ Register Success');
229 //      pendingUserId = response.userId;
230 //      startResendTimer();
231 //      emit(AuthRegisterSuccessState());
232 //      } else {
233 //      debugPrint('❌ Register Failed Header: ${response.message}');
234 //      emit(
235 //      AuthRegisterErrorState(
236 //      message: response.message ?? 'Registration failed',
237 //      ),

```

```

238 //     );
239 //     }
240 //   } catch (e) {
241 //     emit(AuthRegisterErrorState(message: e.toString()));
242 //   }
243 // }
244
245 // void _handleLoginSuccess(Map<String, dynamic> responseBody) {
246 //   final innerData = responseBody['data'] as Map<String, dynamic>?;
247
248 //   // Ensure token is extracted and stored from any supported response shape.
249 //   _storeTokenFromResponse(responseBody);
250
251 //   if (innerData != null) {
252 //     if (innerData['user'] != null) {
253 //       userModel = UserModel.fromJson(
254 //         innerData['user'] as Map<String, dynamic>,
255 //       );
256 //       CacheHelper.saveData(
257 //         key: 'cached_user',
258 //         value: jsonEncode(userModel!.toJson()),
259 //       );
260 //     }
261 //   }
262 //   _clearControllers();
263 //   fetchBootstrapData();
264 //   fetchFreshProfile();
265 // }
266
267 // bool _storeTokenFromResponse(Map<String, dynamic> responseData) {
268 //   final innerData = responseData['data'] as Map<String, dynamic>?;
269 //   final extractedToken =
270 //     (innerData != null
271 //       ? (innerData['token'] ?? innerData['access_token'])
272 //       : (responseData['token'] ?? responseData['access_token']))
273 //     as String?;
274
275 //   if (extractedToken == null || extractedToken.isEmpty) {
276 //     return false;
277 //   }
278
279 //   token = extractedToken;
280 //   CacheHelper.saveData(key: 'auth_token', value: token);
281 //   ApiService.setToken(extractedToken);
282 //   FcmHelper.getAndSendTokenToServer();
283 //   return true;
284 // }
285
286 // Future<bool> _autoLoginAfterOtpVerification() async {
287 //   final loginValue = phoneController.text.trim();
288 //   final passwordValue = passwordController.text.trim();
289
290 //   if (loginValue.isEmpty || passwordValue.isEmpty) {
291 //     return false;
292 //   }
293
294 //   final loginResult = await DioHelper.postData(
295 //     url: loginApi,
296 //     data: {
297 //       'login': loginValue,
298 //       'password': passwordValue,
299 //     },
300 //   );
301
302 //   bool loggedIn = false;
303 //   loginResult.fold(
304 //     (failure) {
305 //       debugPrint('✗ Auto-login after OTP failed: $failure');
306 //     },
307 //     (response) {
308 //       final loginResponseData = response.data as Map<String, dynamic>?;
309 //       if (loginResponseData != null) {
310 //         loggedIn = _storeTokenFromResponse(loginResponseData);
311 //         if (loggedIn && loginResponseData['data'] is Map<String, dynamic>) {
312 //           _handleLoginSuccess(loginResponseData);

```

```

313 //      }
314 //      }
315 //      },
316 //      );
317
318 //      return loggedIn;
319 //  }
320
321 //  void loadCachedUser() {
322 //      final cached = CacheHelper.getData(key: 'cached_user');
323
324 //      if (cached != null) {
325 //          if (cached is String) {
326 //              userModel = UserModel.fromJson(jsonDecode(cached));
327 //          } else if (cached is Map<String, dynamic>) {
328 //              userModel = UserModel.fromJson(cached);
329 //          }
330
331 //          emit(AuthUserLoadedFromCacheState());
332 //          fetchBootstrapData();
333 //          fetchFreshProfile();
334 //      }
335 //  }
336
337 //  Future<void> fetchFreshProfile() async {
338 //      final result = await DioHelper.getData(url: patientProfileApi);
339 //      result.fold(
340 //          (error) => debugPrint('Error fetching fresh profile in AuthCubit: $error'),
341 //          (response) {
342 //              try {
343 //                  final data = response.data['data'];
344 //                  if (data != null) {
345 //                      final avatar = data['profile']['avatar'] ?? data['patient']['avatar'];
346 //                      final name = data['patient']['name'] ?? data['user']['name'];
347
348 //                      if (userModel != null) {
349 //                          updateUser(userModel!.copyWith(
350 //                              avatar: (avatar != null && avatar.toString().trim().isEmpty) ? avatar : userModel!.avatar,
351 //                              name: (name != null && name.toString().trim().isEmpty) ? name : userModel!.name,
352 //                          ));
353 //                      }
354 //                  } catch (e) {
355 //                      debugPrint('Error parsing fresh profile: $e');
356 //                  }
357 //              },
358 //          );
359 //      }
360 //  }
361
362 //  Future<void> fetchBootstrapData() async {
363 //      final result = await DioHelper.getData(url: bootstrapApi);
364 //      result.fold(
365 //          (failure) {
366 //              debugPrint('❌ Bootstrap API Error: $failure');
367 //          },
368 //          (response) {
369 //              final responseData = response.data;
370 //              debugPrint('✅ Bootstrap API Success');
371 //              if (responseData != null) {
372 //                  CacheHelper.saveData(
373 //                      key: 'bootstrap_data',
374 //                      value: jsonEncode(responseData),
375 //                  );
376 //              }
377 //          },
378 //      );
379 //  }
380
381 //  void _clearControllers() {
382 //      emailController.clear();
383 //      passwordController.clear();
384 //      fullNameController.clear();
385 //      phoneController.clear();
386 //      nationalIdController.clear();

```

```

387 // confirmPasswordController.clear();
388 // dateController.clear();
389 // otpController.clear();
390 // forgotLoginController.clear();
391 // resetOtpController.clear();
392 // newPasswordController.clear();
393 // confirmNewPasswordController.clear();
394 // currentPasswordController.clear();
395 // changePasswordController.clear();
396 // confirmChangePasswordController.clear();
397 // isAgreedToTerms = false;
398 // }
399
400 // Future<void> sendOtp() async {
401 //   otpCount++;
402 //   debugPrint('🔴 Send OTP called: $otpCount');
403 //   emit(AuthSendOtpLoadingState());
404 //   final result = await DioHelper.postData(
405 //     url: sendOtpApi,
406 //     data: {
407 //       'login': phoneController.text.trim().isEmpty
408 //         ? emailController.text.trim()
409 //         : phoneController.text.trim(),
410 //     },
411 //   );
412 //   result.fold(
413 //     (failure) {
414 //       debugPrint('❌ Send OTP Error: $failure');
415 //       emit(AuthSendOtpErrorState(message: failure.toString()));
416 //     },
417 //     (response) {
418 //       final responseData = response.data as Map<String, dynamic>;
419 //       final bool isSuccess =
420 //         responseData != null &&
421 //         (responseData['success'] == true ||
422 //           (responseData['message']?.toString().toLowerCase().contains(
423 //             'success',
424 //           ) ??
425 //             false));
426 //
427 //       if (isSuccess) {
428 //         debugPrint('✅ Send OTP Success header: ${response.data}');
429 //         startResendTimer();
430 //         emit(AuthSendOtpSuccessState());
431 //       } else {
432 //         debugPrint('❌ Send OTP Error header: ${response.data}');
433 //         emit(
434 //           AuthSendOtpErrorState(
435 //             message: responseData?['message'] ?? 'Failed to send OTP',
436 //           ),
437 //         );
438 //       }
439 //     },
440 //   );
441 // }
442
443 // Future<void> resendOtp() async {
444 //   final userId = pendingUserId ?? userModel?.id;
445 //   if (userId == null) {
446 //     debugPrint('🚨 No user_id found, falling back to sendOtp');
447 //     return sendOtp();
448 //   }
449 //
450 //   emit(AuthSendOtpLoadingState());
451 //   final result = await DioHelper.postData(
452 //     url: resendOtpApi,
453 //     data: {
454 //       'user_id': userId,
455 //     },
456 //   );
457 //   result.fold(
458 //     (failure) {
459 //       debugPrint('❌ Resend OTP Error: $failure');
460 //       emit(AuthSendOtpErrorState(message: failure.toString()));
461 //     },

```



```

537 //     }
538 //     },
539 //     );
540 // }
541
542 // Future<void> logout() async {
543 //     emit(AuthLogoutLoadingState());
544 //     final result = await DioHelper.postData(
545 //         url: logoutApi,
546 //         data: {},
547 //     );
548
549 //     result.fold(
550 //         (failure) {
551 //             debugPrint('❌ Logout Error: $failure');
552 //             // Still clear local data even if API fails to ensure user can "logout"
553 //             _clearLocalAuthData();
554 //             emit(AuthLogoutSuccessState());
555 //         },
556 //         (response) {
557 //             debugPrint('✅ Logout Success');
558 //             _clearLocalAuthData();
559 //             emit(AuthLogoutSuccessState());
560 //         },
561 //     );
562 // }
563
564 // Future<void> logoutAll() async {
565 //     emit(AuthLogoutAllLoadingState());
566 //     final result = await DioHelper.postData(
567 //         url: logoutAllApi,
568 //         data: {},
569 //     );
570
571 //     result.fold(
572 //         (failure) {
573 //             debugPrint('❌ Logout All Error: $failure');
574 //             _clearLocalAuthData();
575 //             emit(AuthLogoutAllSuccessState());
576 //         },
577 //         (response) {
578 //             debugPrint('✅ Logout All Success');
579 //             _clearLocalAuthData();
580 //             emit(AuthLogoutAllSuccessState());
581 //         },
582 //     );
583 // }
584
585 // Future<void> _clearLocalAuthData() async {
586 //     token = null;
587 //     userModel = null;
588 //     await CacheHelper.removeData(key: 'auth_token');
589 //     await CacheHelper.removeData(key: 'cached_user');
590 //     ApiService.setToken('');
591 //     sl<ProfileCubit>().clearData();
592
593 //     final context = navigatorKey.currentContext;
594 //     if (context != null) {
595 //         NotificationCubit.get(context).clearData();
596 //     }
597 // }
598
599 // void updateUser(UserModel newUser) {
600 //     userModel = newUser;
601 //     CacheHelper.saveData(
602 //         key: 'cached_user',
603 //         value: jsonEncode(userModel!.toJson()),
604 //     );
605 //     emit(AuthUserUpdatedState());
606 // }
607
608 // void updateUserAvatar(String? avatar) {
609 //     if (userModel != null) {
610 //         userModel = userModel!.copyWith(avatar: avatar);
611 //         CacheHelper.saveData(

```

```

612 //         key: 'cached_user',
613 //         value: jsonEncode(userModel!.toJson()),
614 //     );
615 //     emit(AuthUserUpdatedState());
616 // }
617 // }
618
619 // void changeNewPasswordVisibility() {
620 //     isShowNewPassword = !isShowNewPassword;
621 //     emit(AuthUpdateUIState());
622 // }
623
624 // void changeConfirmNewPasswordVisibility() {
625 //     isShowConfirmNewPassword = !isShowConfirmNewPassword;
626 //     emit(AuthUpdateUIState());
627 // }
628
629 // Future<void> sendForgotPasswordOtp() async {
630 //     emit(AuthForgotPasswordLoadingState());
631 //     debugPrint('🚀 Forgot Password Request Data: {"login": "${forgotLoginController.text.trim()}"}');
632 //     final result = await DioHelper.postData(
633 //         url: forgotPasswordApi,
634 //         data: {'login': forgotLoginController.text.trim()},
635 //     );
636 //     result.fold(
637 //         (failure) {
638 //             debugPrint('❌ Forgot Password Error: $failure');
639 //             emit(AuthForgotPasswordErrorState(message: failure.toString()));
640 //         },
641 //         (response) {
642 //             final responseData = response.data as Map<String, dynamic>;
643 //             final bool isSuccess =
644 //                 responseData != null &&
645 //                 (responseData['success'] == true ||
646 //                 (responseData['message']
647 //                     ?.toString()
648 //                     .toLowerCase()
649 //                     .contains('sent') ??
650 //                     false) ||
651 //                 (responseData['message']
652 //                     ?.toString()
653 //                     .toLowerCase()
654 //                     .contains('success') ??
655 //                     false));
656 //
657 //             if (isSuccess) {
658 //                 debugPrint('✅ Forgot Password OTP Sent');
659 //                 emit(AuthForgotPasswordSuccessState());
660 //             } else {
661 //                 emit(
662 //                     AuthForgotPasswordErrorState(
663 //                         message:
664 //                             responseData?['message'] ?? 'Failed to send reset code',
665 //                     ),
666 //                 );
667 //             }
668 //         },
669 //     );
670 // }
671
672 // Future<void> resetPassword((bool logoutAllDevices = false)) async {
673 //     final loginValue = forgotLoginController.text.trim().isEmpty
674 //         ? emailController.text.trim()
675 //         : forgotLoginController.text.trim();
676 //     final otpValue = resetOtpController.text.trim();
677 //
678 //     if (loginValue.isEmpty || otpValue.isEmpty) {
679 //         emit(AuthResetPasswordErrorState(
680 //             message: 'Missing session data (login/OTP). Please restart the recovery flow.',
681 //         ));
682 //         return;
683 //     }
684 //
685 //     emit(AuthResetPasswordLoadingState());
686 //     final requestData = {

```



```

687 //      'login': loginValue,
688 //      'otp': otpValue,
689 //      'password': newPasswordController.text,
690 //      'password_confirmation': confirmNewPasswordController.text,
691 //      'logout_all_devices': logoutAllDevices ? 1 : 0,
692 //    };
693 //    debugPrint('🚀 Reset Password Request Data: $requestData');
694
695 //    final result = await DioHelper.postData(
696 //      url: resetPasswordApi,
697 //      data: requestData,
698 //    );
699 //    result.fold(
700 //      (failure) {
701 //        debugPrint('❌ Reset Password Error: $failure');
702 //        emit(AuthResetPasswordErrorState(message: failure.toString()));
703 //      },
704 //      (response) {
705 //        final responseData = response.data as Map<String, dynamic>;
706 //        final bool isSuccess =
707 //          responseData != null &&
708 //          (responseData['success'] == true ||
709 //            (responseData['message']
710 //              ?.toString()
711 //                .toLowerCase()
712 //                  .contains('success') ??
713 //                false));
714
715 //        if (isSuccess) {
716 //          debugPrint('✅ Reset Password Success');
717 //          resetOtpController.clear();
718 //          newPasswordController.clear();
719 //          confirmNewPasswordController.clear();
720 //          emit(AuthResetPasswordSuccessState());
721 //        } else {
722 //          emit(
723 //            AuthResetPasswordErrorState(
724 //              message: responseData?['message'] ?? 'Failed to reset password',
725 //            ),
726 //          );
727 //        }
728 //      },
729 //    );
730 //  }
731
732 //  void changeCurrentPasswordVisibility() {
733 //    isShowCurrentPassword = !isShowCurrentPassword;
734 //    emit(AuthUpdateUIState());
735 //  }
736
737 //  void changeChangePasswordVisibility() {
738 //    isShowChangePassword = !isShowChangePassword;
739 //    emit(AuthUpdateUIState());
740 //  }
741
742 //  void changeConfirmChangePasswordVisibility() {
743 //    isShowConfirmChangePassword = !isShowConfirmChangePassword;
744 //    emit(AuthUpdateUIState());
745 //  }
746
747 //  Future<void> changePassword({bool logoutAllDevices = false}) async {
748 //    emit(AuthChangePasswordLoadingState());
749 //    final requestData = {
750 //      'current_password': currentPasswordController.text,
751 //      'password': changePasswordController.text,
752 //      'password_confirmation': confirmChangePasswordController.text,
753 //    };
754 //    debugPrint('🚀 Change Password Request Data: $requestData');
755
756 //    final result = await DioHelper.postData(
757 //      url: changePasswordApi,
758 //      data: requestData,
759 //    );
760
761 //    result.fold(

```

```

762 //      (failure) {
763 //          debugPrint('❌ Change Password Error: $failure');
764 //          emit(AuthChangePasswordErrorState(message: failure.toString()));
765 //      },
766 //      (response) async {
767 //          final responseData = response.data as Map<String, dynamic>;
768 //          final bool isSuccess =
769 //              responseData != null &&
770 //              (responseData['success'] == true ||
771 //              (responseData['message']
772 //                  ?.toString()
773 //                  .toLowerCase()
774 //                  .contains('success') ??
775 //              false));
776
777 //          if (isSuccess) {
778 //              debugPrint('✅ Change Password Success');
779 //              currentPasswordController.clear();
780 //              changePasswordController.clear();
781 //              confirmChangePasswordController.clear();
782
783 //              if (logoutAllDevices) {
784 //                  await logoutAll();
785 //              } else {
786 //                  emit(AuthChangePasswordSuccessState());
787 //              }
788 //          } else {
789 //              emit(
790 //                  AuthChangePasswordErrorState(
791 //                      message: responseData?['message'] ?? 'Failed to change password',
792 //                      ),
793 //              );
794 //          }
795 //      },
796 //      );
797 //  }
798 // }
799

```

Lines • Dart 98 Core / Utils

#43 – lib/core/utils/cubit/auth/auth_state.dart

```

1  abstract class AuthState {}
2
3  class AuthInitialState extends AuthState {}
4
5  class AuthSendOtpLoadingState extends AuthState {}
6
7  class AuthSendOtpSuccessState extends AuthState {}
8
9  class AuthSendOtpErrorState extends AuthState {
10     final String message;
11     AuthSendOtpErrorState({required this.message});
12 }
13
14 class AuthVerifyOtpLoadingState extends AuthState {}
15
16 class AuthVerifyOtpSuccessState extends AuthState {}
17
18 class AuthVerifyOtpErrorState extends AuthState {
19     final String message;
20     AuthVerifyOtpErrorState({required this.message});
21 }
22
23 class AuthShowPasswordState extends AuthState {}
24
25 class AuthLoginLoadingState extends AuthState {}
26
27 class AuthLoginSuccessState extends AuthState {}
28
29 class AuthLoginErrorState extends AuthState {
30     final String message;
31
32     AuthLoginErrorState({required this.message});

```

```

33 }
34
35 class AuthUserLoadedFromCacheState extends AuthState {}
36
37 class AuthRegisterLoadingState extends AuthState {}
38
39 class AuthRegisterSuccessState extends AuthState {}
40
41 class AuthRegisterErrorState extends AuthState {
42   final String message;
43
44   AuthRegisterErrorState({required this.message});
45 }
46
47 class AuthUpdateUIState extends AuthState {}
48
49 class AuthUserUpdatedState extends AuthState {}
50
51 // — Forgot Password Flow —————
52
53 class AuthForgotPasswordLoadingState extends AuthState {}
54
55 class AuthForgotPasswordSuccessState extends AuthState {}
56
57 class AuthForgotPasswordErrorState extends AuthState {
58   final String message;
59   AuthForgotPasswordErrorState({required this.message});
60 }
61
62 class AuthResetPasswordLoadingState extends AuthState {}
63
64 class AuthResetPasswordSuccessState extends AuthState {}
65
66 class AuthResetPasswordErrorState extends AuthState {
67   final String message;
68   AuthResetPasswordErrorState({required this.message});
69 }
70
71 // — Change Password & Logout —————
72 class AuthChangePasswordLoadingState extends AuthState {}
73
74 class AuthChangePasswordSuccessState extends AuthState {}
75
76 class AuthChangePasswordErrorState extends AuthState {
77   final String message;
78   AuthChangePasswordErrorState({required this.message});
79 }
80
81 class AuthLogoutLoadingState extends AuthState {}
82
83 class AuthLogoutSuccessState extends AuthState {}
84
85 class AuthLogoutErrorState extends AuthState {
86   final String message;
87   AuthLogoutErrorState({required this.message});
88 }
89
90 class AuthLogoutAllLoadingState extends AuthState {}
91
92 class AuthLogoutAllSuccessState extends AuthState {}
93
94 class AuthLogoutAllErrorState extends AuthState {
95   final String message;
96   AuthLogoutAllErrorState({required this.message});
97 }
98

```

```

1 import 'dart:convert';
2 import 'package:moean/core/di/injections.dart';
3 import 'package:flutter/material.dart';
4 import 'package:flutter/services.dart';

```

```

5 import 'package:flutter_bloc/flutter_bloc.dart';
6 import 'package:moean/core/network/local/cache_helper.dart';
7 import 'package:moean/core/utils/constants/translations.dart';
8 import 'package:moean/core/utils/cubit/theme/theme_state.dart';
9
10 ThemeCubit get themeCubit => sl<ThemeCubit>();
11
12 class ThemeCubit extends Cubit<ThemeState> {
13   ThemeCubit() : super(ThemeInitialState());
14   static ThemeCubit get(BuildContext context) => BlocProvider.of(context);
15
16   void init({bool isDark = false}) {
17     _isDarkMode = isDark;
18   }
19
20   bool _isDarkMode = false;
21
22   bool get isDarkMode => _isDarkMode;
23
24   void changeTheme({bool? fromShared}) {
25     _isDarkMode = fromShared ?? !_isDarkMode;
26     CacheHelper.saveData(key: 'isDark', value: _isDarkMode);
27     emit(ThemeChangeThemeState());
28   }
29
30   bool _isArabicLang = false;
31   TranslationModel? _translationModel;
32
33   bool get isArabicLang => _isArabicLang;
34
35   TranslationModel? get translationModel => _translationModel;
36
37   Future<void> changeLanguage() async {
38     try {
39       emit(ThemeLanguageLoadingState());
40
41       final newIsArabic = !_isArabicLang;
42       final String translations = await rootBundle.loadString(
43         'assets/translations/${newIsArabic ? 'ar' : 'en'}.json',
44       );
45
46       final model = TranslationModel.fromJson(json.decode(translations));
47       _isArabicLang = newIsArabic;
48       _translationModel = model;
49
50       await CacheHelper.saveData(key: 'isArabicLang', value: _isArabicLang);
51       emit(ThemeLanguageUpdatedState());
52     } catch (e) {
53       emit(ThemeLanguageErrorState(e.toString()));
54     }
55   }
56
57   Future<void> initializeLanguage({
58     required bool isArabic,
59     required String translations,
60   }) async {
61     try {
62       _isArabicLang = isArabic;
63       _translationModel = TranslationModel.fromJson(json.decode(translations));
64       CacheHelper.saveData(key: 'isArabicLang', value: _isArabicLang);
65       emit(ThemeLanguageSuccessState());
66     } catch (e) {
67       emit(ThemeLanguageErrorState(e.toString()));
68     }
69   }
70 }
71

```

```

1 abstract class ThemeState {}
2
3 class ThemeInitialState extends ThemeState {}

```

```

4
5 class ThemeChangeThemeState extends ThemeState {}
6
7 class ThemeLanguageUpdatedState extends ThemeState {}
8
9 class ThemeLanguageLoadingState extends ThemeState {}
10
11 class ThemeLanguageSuccessState extends ThemeState {}
12
13 class ThemeLanguageErrorState extends ThemeState {
14   final String error;
15
16   ThemeLanguageErrorState(this.error);
17 }
18

```

Lines • Dart 43

Core / Utils

#46 – lib/core/utils/extensions/context_extension.dart

```

1 import 'package:flutter/material.dart';
2
3 extension NavigationExtension on BuildContext {
4   void push<ARG>({
5     String routeName, {
6     ARG? arguments,
7   }) => Navigator.pushNamed(
8     this,
9     routeName,
10    arguments: arguments,
11  );
12
13   void pushReplacement<ARG>({
14     String routeName, {
15     ARG? arguments,
16   }) => Navigator.pushReplacementNamed(
17     this,
18     routeName,
19     arguments: arguments,
20   );
21
22   void pushNamedAndRemoveUntil<ARG>({
23     String routeName,
24     RoutePredicate predicate, {
25     ARG? arguments,
26   }) => Navigator.pushNamedAndRemoveUntil(
27     this,
28     routeName,
29     predicate,
30     arguments: arguments,
31   );
32
33   void pop([Object? result]) => Navigator.pop(this, result);
34
35   ARG? getArg<ARG>() {
36     final args = ModalRoute.of(this)?.settings.arguments;
37     if (args is ARG) {
38       return args;
39     }
40     return null;
41   }
42 }
43

```

Lines • Dart 19

Core / Utils

#47 – lib/core/utils/extensions/date_formatter.dart

```

1 // import 'package:intl/intl.dart';
2
3 // extension DateFormatter on String {
4 //   String toFormattedDate() {
5 //     try {
6 //       final DateTime dt = DateTime.parse(this);
7 //       return DateFormat('dd/MM/yyyy').format(dt);
8 //     } catch (e) {

```

```

9 //      return this;
10 //    }
11 //  }
12 // }
13
14 // extension DateTimeFormatter on DateTime {
15 //   String toFormattedDate() {
16 //     return DateFormat('dd/MM/yyyy').format(this);
17 //   }
18 // }
19

```

Lines • Dart 70

Core / Utils

#48 – lib/core/utils/formatters/saudi_phone_input_formatter.dart

```

1 import 'package:flutter/services.dart';
2
3 /// Formatter to normalize Saudi phone numbers in real-time.
4 /// - Converts Arabic-Indic and Eastern-Arabic numerals to English ASCII digits.
5 /// - Strips whitespace, hyphens, and any non-digit characters.
6 /// - Normalizes international prefixes (+966, 00966, 966) if pasted.
7 /// - Enforces maximum length of 10 digits (e.g. 05XXXXXXX).
8 class SaudiPhoneInputFormatter extends TextInputFormatter {
9   @override
10   TextEditingValue formatEditUpdate(
11     TextEditingValue oldValue,
12     TextEditingValue newValue,
13   ) {
14     if (newValue.text.isEmpty) {
15       return newValue;
16     }
17
18     final normalized = normalizePhoneNumber(newValue.text);
19     final truncated =
20       normalized.length > 10 ? normalized.substring(0, 10) : normalized;
21
22     // Calculate cursor position after formatting
23     int newOffset = truncated.length;
24     if (newValue.selection.end <= newValue.text.length) {
25       final textBeforeCursor =
26         newValue.text.substring(0, newValue.selection.end);
27       final normalizedBefore = normalizePhoneNumber(textBeforeCursor);
28       newOffset = normalizedBefore.length.clamp(0, truncated.length);
29     }
30
31     return TextEditingValue(
32       text: truncated,
33       selection: TextSelection.collapsed(offset: newOffset),
34     );
35   }
36
37   /// Utility method to convert Arabic numerals and strip invalid chars
38   static String normalizePhoneNumber(String input) {
39     if (input.isEmpty) return input;
40
41     // Arabic-Indic & Eastern-Arabic digits mapping
42     const arabicDigits = ['٠', '١', '٢', '٣', '٤', '٥', '٦', '٧', '٨', '٩'];
43     const easternArabicDigits = ['٠', '١', '٢', '٣', '٤', '٥', '٦', '٧', '٨', '٩'];
44
45     String normalized = input;
46     for (int i = 0; i < 10; i++) {
47       normalized = normalized.replaceAll(arabicDigits[i], i.toString());
48       normalized = normalized.replaceAll(easternArabicDigits[i], i.toString());
49     }
50
51     // Strip all non-digit characters (spaces, dashes, parentheses, +, etc.)
52     normalized = normalized.replaceAll(RegExp(r'\D'), '');
53
54     // Handle country codes (+966 / 00966 / 966)
55     if (normalized.startsWith('00966')) {
56       normalized = normalized.substring(5);
57       if (!normalized.startsWith('0')) {
58         normalized = '0$normalized';
59       }
60     }
61   }
62 }

```

```

60     } else if (normalized.startsWith('966')) {
61         normalized = normalized.substring(3);
62         if (!normalized.startsWith('0')) {
63             normalized = '0$normalized';
64         }
65     }
66
67     return normalized;
68 }
69 }
70

```

Lines • Dart 77

Core / Utils

#49 – lib/core/utils/pdf_export_helper.dart

```

1  import 'dart:typed_data';
2  import 'package:flutter/material.dart';
3  import 'package:screenshot/screenshot.dart';
4  import 'package:pdf/pdf.dart';
5  import 'package:pdf/widgets.dart' as pw;
6  import 'package:printing/printing.dart';
7  import 'package:moean/core/theme/text_styles.dart';
8  import 'package:moean/core/theme/colors.dart';
9
10 class PdfExportHelper {
11     static Future<void> exportWidgetsToPdfPages({
12         required BuildContext context,
13         required List<Widget> pages,
14         required String title,
15     }) async {
16         showDialog(
17             context: context,
18             barrierDismissible: false,
19             builder: (_) => const Center(child: CircularProgressIndicator()),
20         );
21
22         try {
23             final screenshotController = ScreenshotController();
24             final pdf = pw.Document();
25
26             for (var pageWidget in pages) {
27                 final fullWidget = Directionality(
28                     textDirection: TextDirection.rtl,
29                     child: Theme(
30                         data: Theme.of(context),
31                         child: Material(
32                             color: ColorsManager.background,
33                             child: Padding(
34                                 padding: const EdgeInsets.all(24.0),
35                                 child: pageWidget,
36                             ),
37                         ),
38                     ),
39                 );
40
41                 final Uint8List imageBytes = await screenshotController.captureFromWidget(
42                     fullWidget,
43                     context: context,
44                     delay: const Duration(milliseconds: 100),
45                     pixelRatio: 2.0,
46                 );
47
48                 final image = pw.MemoryImage(imageBytes);
49                 pdf.addPage(
50                     pw.Page(
51                         pageFormat: PdfPageFormat.a4,
52                         build: (pw.Context context) {
53                             return pw.Center(
54                                 child: pw.Image(image, fit: pw.BoxFit.contain),
55                             );
56                         },
57                     ),
58                 );
59             }

```

```

60
61     if (context.mounted) Navigator.pop(context);
62
63     await Printing.layoutPdf(
64       onLayout: (PdfPageFormat format) async => pdf.save(),
65       name: '$title.pdf',
66     );
67   } catch (e) {
68     if (context.mounted) {
69       Navigator.pop(context);
70       ScaffoldMessenger.of(context).showSnackBar(
71         SnackBar(content: Text('حدث خطأ أثناء التصدير: $e'), backgroundColor: Colors.red),
72       );
73     }
74   }
75 }
76 }
77

```

Lines • Dart 6

Core / Utils

#50 – lib/core/utils/platform_url/platform_url_stub.dart

```

1  /// Stub implementation for non-web platforms.
2  /// Opening a URL in a new tab is a no-op on mobile/desktop.
3  void openUrlInNewTab(String url) {
4    // No-op on non-web platforms
5  }
6

```

Lines • Dart 7

Core / Utils

#51 – lib/core/utils/platform_url/platform_url_web.dart

```

1
2  import 'dart:html' as html;
3
4  void openUrlInNewTab(String url) {
5    html.window.open(url, '_blank');
6  }
7

```

Lines • Dart 3

Core / Utils

#52 – lib/core/utils/platform_url/platform_url.dart

```

1  export 'platform_url_stub.dart'
2    if (dart.library.html) 'platform_url_web.dart';
3

```

Lines • Dart 52

Core / Widgets

#53 – lib/core/widgets/background_wrapper.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/utils/cubit/theme/theme_cubit.dart';
5  import 'package:moean/core/utils/cubit/theme/theme_state.dart';
6
7  class BackgroundWrapper extends StatelessWidget {
8    final Widget child;
9
10    const BackgroundWrapper({super.key, required this.child});
11
12    @override
13    Widget build(BuildContext context) {
14      return BlocBuilder<ThemeCubit, ThemeState>(
15        builder: (context, state) {
16          return Container(
17            color: ColorsManager.background,
18            child: Stack(
19              children: [
20                Positioned(
21                  top: -120,
22                  right: -80,
23                  child: Container(

```



```

24         width: 200,
25         height: 200,
26         decoration: BoxDecoration(
27             shape: BoxShape.circle,
28             color: ColorsManager.primaryColor.withValues(alpha: 0.15),
29         ),
30     ),
31 ),
32 Positioned(
33     top: -40,
34     right: -40,
35     child: Container(
36         width: 100,
37         height: 100,
38         decoration: BoxDecoration(
39             shape: BoxShape.circle,
40             color: ColorsManager.primaryColor.withValues(alpha: 0.8),
41         ),
42     ),
43 ),
44     child,
45 ],
46 ),
47 );
48 },
49 );
50 }
51 }
52

```

Lines • Dart 139

Core / Widgets

#54 – lib/core/widgets/custom_app_dialog.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/spacing.dart';
5
6  class CustomAppDialog extends StatelessWidget {
7      final String title;
8      final String message;
9      final String? confirmText;
10     final String? cancelText;
11     final VoidCallback? onConfirm;
12     final VoidCallback? onCancel;
13     final IconData? icon;
14     final Color? iconColor;
15     final Widget? child;
16
17     const CustomAppDialog({
18         super.key,
19         required this.title,
20         required this.message,
21         this.confirmText,
22         this.cancelText,
23         this.onConfirm,
24         this.onCancel,
25         this.icon,
26         this.iconColor,
27         this.child,
28     });
29
30     @override
31     Widget build(BuildContext context) {
32         return Dialog(
33             shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(24)),
34             elevation: 0,
35             backgroundColor: Colors.transparent,
36             child: Container(
37                 padding: const EdgeInsets.all(24),
38                 decoration: BoxDecoration(
39                     shape: BoxShape.rectangle,
40                     color: ColorsManager.surfacePrimary,
41                     borderRadius: BorderRadius.circular(24),

```

```

42     boxShadow: [
43       BoxShadow(
44         color: Colors.black.withValues(alpha: 0.1),
45         offset: const Offset(0, 10),
46         blurRadius: 20,
47       ),
48     ],
49   ),
50   child: Column(
51     mainAxisAlignment: MainAxisAlignment.min,
52     children: <Widget>[
53       if (icon != null) ...[
54         Container(
55           padding: const EdgeInsets.all(16),
56           decoration: BoxDecoration(
57             color: (iconColor ?? ColorsManager.primaryAction)
58               .withValues(alpha: 0.1),
59             shape: BoxShape.circle,
60           ),
61           child: Icon(
62             icon,
63             color: iconColor ?? ColorsManager.primaryAction,
64             size: 32,
65           ),
66         ),
67         verticalSpace20,
68       ],
69       Text(
70         title,
71         style: TextStylesManager.bold20.copyWith(
72           color: ColorsManager.textPrimary,
73         ),
74         textAlign: TextAlign.center,
75       ),
76       verticalSpace12,
77       Text(
78         message,
79         style: TextStylesManager.medium14.copyWith(
80           color: ColorsManager.textPrimary,
81         ),
82         textAlign: TextAlign.center,
83       ),
84       if (child != null) ...[
85         verticalSpace16,
86         child!,
87       ],
88       verticalSpace24,
89       Row(
90         children: [
91           if (cancelText != null)
92             Expanded(
93               child: OutlinedButton(
94                 onPressed: onCancel ?? () => Navigator.pop(context),
95                 style: OutlinedButton.styleFrom(
96                   padding: const EdgeInsets.symmetric(vertical: 14),
97                   side: BorderSide(color: ColorsManager.borderColor),
98                   shape: RoundedRectangleBorder(
99                     borderRadius: BorderRadius.circular(12),
100                 ),
101               ),
102               child: Text(
103                 cancelText!,
104                 style: TextStylesManager.bold14.copyWith(
105                   color: ColorsManager.textPrimary,
106                 ),
107               ),
108             ),
109         ],
110         if (cancelText != null && confirmText != null) horizontalSpace12,
111         if (confirmText != null)
112           Expanded(
113             child: ElevatedButton(
114               onPressed: onConfirm,
115               style: ElevatedButton.styleFrom(
116                 backgroundColor: iconColor ?? ColorsManager.primaryAction,

```

```

117         padding: const EdgeInsets.symmetric(vertical: 14),
118         elevation: 0,
119         shape: RoundedRectangleBorder(
120             borderRadius: BorderRadius.circular(12),
121         ),
122     ),
123     child: Text(
124         confirmText!,
125         style: TextStylesManager.bold14.copyWith(
126             color: Colors.white,
127         ),
128     ),
129 ),
130 ],
131 ],
132 ),
133 ],
134 ),
135 ),
136 );
137 }
138 }
139

```

Lines • Dart 36

Core / Widgets

#55 – lib/core/widgets/dot_gridPainter.dart

```

1  import 'package:flutter/material.dart';
2
3  /// A [CustomPainter] that draws a uniform dot-grid pattern.
4  /// Used as a decorative background element across multiple screens.
5  class DotGridPainter extends CustomPainter {
6      final Color color;
7      final double spacing;
8      final double dotRadius;
9
10     const DotGridPainter({
11         required this.color,
12         this.spacing = 16.0,
13         this.dotRadius = 2.0,
14     });
15
16     @override
17     void paint(Canvas canvas, Size size) {
18         final paint = Paint()
19             ..color = color
20             ..style = PaintingStyle.fill;
21
22         for (double x = 0; x < size.width; x += spacing) {
23             for (double y = 0; y < size.height; y += spacing) {
24                 canvas.drawCircle(Offset(x, y), dotRadius, paint);
25             }
26         }
27     }
28
29     @override
30     bool shouldRepaint(covariant DotGridPainter oldDelegate) {
31         return oldDelegate.color != color ||
32             oldDelegate.spacing != spacing ||
33             oldDelegate.dotRadius != dotRadius;
34     }
35 }
36

```

Lines • Dart 63

Core / Widgets

#56 – lib/core/widgets/logout_all_option.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6
7  class LogoutAllOption extends StatelessWidget {

```

```

8   final bool value;
9   final ValueChanged<bool> onChanged;
10
11   const LogoutAllOption({
12     super.key,
13     required this.value,
14     required this.onChanged,
15   });
16
17   @override
18   Widget build(BuildContext context) {
19     return Container(
20       padding: const EdgeInsets.all(16),
21       decoration: BoxDecoration(
22         color: ColorsManager.surfacePrimary,
23         borderRadius: BorderRadius.circular(16),
24         border: Border.all(color: ColorsManager.borderColor),
25       ),
26       child: Column(
27         crossAxisAlignment: CrossAxisAlignment.start,
28         children: [
29           Row(
30             children: [
31               Expanded(
32                 child: Column(
33                   crossAxisAlignment: CrossAxisAlignment.start,
34                   children: [
35                     Text(
36                       appTranslation().get('logout_all_devices'),
37                       style: TextStylesManager.bold14.copyWith(
38                         color: ColorsManager.textPrimary,
39                       ),
40                     ),
41                     verticalSpace4,
42                     Text(
43                       appTranslation().get('logout_all_devices_desc'),
44                       style: TextStylesManager.regular12.copyWith(
45                         color: ColorsManager.textSecondary,
46                       ),
47                     ),
48                   ],
49                 ),
50               ),
51               Switch.adaptive(
52                 value: value,
53                 onChanged: onChanged,
54                 activeTrackColor: ColorsManager.primaryColor,
55               ),
56             ],
57           ),
58         ],
59       ),
60     );
61   }
62 }
63

```

Lines • Dart 144

Core / Widgets

#57 – lib/core/widgets/session_expired_dialog.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/spacing.dart';
5  import 'package:moean/core/utils/extensions/context_extension.dart';
6  import 'package:moean/core/utils/constants/routes.dart';
7
8  /// Dialog shown when the Madrasati school session has expired AND the silent
9  /// background refresh has already failed (e.g., the Microsoft refresh token
10 /// is expired or revoked).
11 ///
12 /// At this point the user must perform a manual WebView login. This dialog
13 /// provides a clear CTA to navigate to the Microsoft login screen.
14 ///

```

```

15  /// Note: The automatic silent refresh is now handled transparently by
16  /// [MadrasatiSessionInterceptor] at the Dio layer – this dialog only appears
17  /// when that interceptor cannot recover the session automatically.
18  class SessionExpiredDialog extends StatelessWidget {
19    const SessionExpiredDialog({super.key});
20
21    static Future<void> show(BuildContext context) {
22      return showDialog<void>(
23        context: context,
24        barrierDismissible: false,
25        builder: (_) => const SessionExpiredDialog(),
26      );
27    }
28
29    @override
30    Widget build(BuildContext context) {
31      return Dialog(
32        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(24)),
33        elevation: 0,
34        backgroundColor: Colors.transparent,
35        child: Container(
36          padding: const EdgeInsets.all(24),
37          decoration: BoxDecoration(
38            shape: BoxShape.rectangle,
39            color: ColorsManager.surfacePrimary,
40            borderRadius: BorderRadius.circular(24),
41            boxShadow: [
42              BoxShadow(
43                color: Colors.black.withValues(alpha: 0.12),
44                offset: const Offset(0, 10),
45                blurRadius: 24,
46              ),
47            ],
48          ),
49          child: Column(
50            mainAxisAlignment: MainAxisAlignment.min,
51            children: [
52              // — Icon —————
53              Container(
54                padding: const EdgeInsets.all(16),
55                decoration: BoxDecoration(
56                  color: Colors.orange.withValues(alpha: 0.12),
57                  shape: BoxShape.circle,
58                ),
59                child: const Icon(
60                  Icons.lock_clock_rounded,
61                  color: Colors.orange,
62                  size: 36,
63                ),
64              ),
65              verticalSpace20,
66
67              // — Title —————
68              Text(
69                'انتهت جلسة مدرستي',
70                style: TextStyleManager.bold20.copyWith(
71                  color: ColorsManager.textPrimary,
72                ),
73                textAlign: TextAlign.center,
74              ),
75              verticalSpace12,
76
77              // — Message —————
78              Text(
79                'يرجى تسجيل الدخول في مدرستي مجدداً للاستمرار.\nتعذر تجديد الجلسة تلقائياً',
80                style: TextStyleManager.medium14.copyWith(
81                  color: ColorsManager.textPrimary,
82                ),
83                textAlign: TextAlign.center,
84              ),
85              verticalSpace24,
86
87              // — Buttons —————
88              Row(
89                children: [

```

```

90         // Dismiss
91         Expanded(
92           child: OutlinedButton(
93             onPressed: () => Navigator.of(context).pop(),
94             style: OutlinedButton.styleFrom(
95               padding: const EdgeInsets.symmetric(vertical: 14),
96               side: BorderSide(color: ColorsManager.borderColor),
97               shape: RoundedRectangleBorder(
98                 borderRadius: BorderRadius.circular(12),
99               ),
100            ),
101            child: Text(
102              'لاخفا',
103              style: TextStylesManager.bold14.copyWith(
104                color: ColorsManager.textPrimary,
105              ),
106            ),
107          ),
108        ),
109        horizontalSpace12,
110
111        // Login
112        Expanded(
113          child: ElevatedButton(
114            onPressed: () {
115              Navigator.of(context).pop();
116              // context.push(Routes.loginMicrosoft);
117              context.push(Routes.home);
118            },
119            style: ElevatedButton.styleFrom(
120              backgroundColor: ColorsManager.primaryAction,
121              padding: const EdgeInsets.symmetric(vertical: 14),
122              elevation: 0,
123              shape: RoundedRectangleBorder(
124                borderRadius: BorderRadius.circular(12),
125              ),
126            ),
127            child: Text(
128              'تسجيل الدخول',
129              style: TextStylesManager.bold14.copyWith(
130                color: Colors.white,
131              ),
132            ),
133          ),
134        ),
135      ),
136    ],
137  ),
138 ],
139 ),
140 ),
141 );
142 }
143 }
144

```

Lines • Dart 70 Core / Widgets

#58 – lib/core/widgets/session_monitor_wrapper.dart

```

1  import 'dart:async';
2  import 'package:flutter/material.dart';
3  import 'package:moean/core/di/injections.dart';
4  import 'package:moean/core/services/madrasati_session_service.dart';
5  import 'package:moean/core/widgets/session_expired_dialog.dart';
6  import 'package:moean/main.dart';
7
8  /// A transparent widget that sits at the top of the widget tree (inside
9  /// [MaterialApp.builder]) and listens globally for Madrasati session expiry.
10 ///
11 /// When [MadrasatiSessionService.onSessionExpired] fires it shows
12 /// [SessionExpiredDialog] via the [navigatorKey], ensuring the dialog
13 /// appears regardless of which screen the user is currently on.
14 class SessionMonitorWrapper extends StatefulWidget {
15   final Widget child;

```

```

16
17   const SessionMonitorWrapper({super.key, required this.child});
18
19   @override
20   State<SessionMonitorWrapper> createState() => _SessionMonitorWrapperState();
21 }
22
23 class _SessionMonitorWrapperState extends State<SessionMonitorWrapper> {
24   StreamSubscription<void>? _subscription;
25   bool _dialogShowing = false;
26
27   @override
28   void initState() {
29     super.initState();
30     _subscribe();
31   }
32
33   void _subscribe() {
34     try {
35       _subscription =
36         sl<MadrasatiSessionService>().onSessionExpired.listen((_) {
37           _showExpiredDialog();
38         });
39     } catch (e) {
40       debugPrint('SessionMonitorWrapper: could not subscribe: $e');
41     }
42   }
43
44   Future<void> _showExpiredDialog() async {
45     if (_dialogShowing) return; // Prevent stacking multiple dialogs
46     _dialogShowing = true;
47
48     final context = navigatorKey.currentContext;
49     if (context == null) {
50       _dialogShowing = false;
51       return;
52     }
53
54     debugPrint('🔔 SessionMonitorWrapper: showing session expired dialog');
55
56     await SessionExpiredDialog.show(context);
57
58     _dialogShowing = false;
59   }
60
61   @override
62   void dispose() {
63     _subscription?.cancel();
64     super.dispose();
65   }
66
67   @override
68   Widget build(BuildContext context) => widget.child;
69 }
70

```

Lines • 213
Dart

Feature / Account #59 -

Suspended lib/features/account_suspended/presentation/cubit/account_suspended_cubit.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/models/login_request.dart';
4  import 'package:moean/core/network/remote/api_service.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/features/account_suspended/presentation/cubit/account_suspended_state.dart';
7  import 'package:url_launcher/url_launcher.dart';
8
9  class AccountSuspendedCubit extends Cubit<AccountSuspendedState> {
10    static AccountSuspendedCubit get(BuildContext context) =>
11      BlocProvider.of<AccountSuspendedCubit>(context);
12
13    final GlobalKey<FormState> formKey = GlobalKey<FormState>();
14    final TextEditingController subjectController = TextEditingController(

```

```

15     text: 'طلب مراجعة وفك تعليق الحساب',
16 );
17 final TextEditingController messageController = TextEditingController();
18
19 AccountSuspendedCubit() : super(AccountSuspendedInitialState()) {
20     try {
21         subjectController.text = appTranslation().get('account_suspended_subject_default');
22     } catch (_) {}
23 }
24
25 String selectedType = 'technical_support';
26 List<dynamic> contactTypes = [
27     {'value': 'technical_support', 'label': 'دعم فني'},
28     {'value': 'billing', 'label': 'الفوترة والاشتراكات'},
29     {'value': 'feature_request', 'label': 'اقتراح ميزة'},
30     {'value': 'general', 'label': 'استفسار عام'},
31     {'value': 'other', 'label': 'أخرى'},
32 ];
33
34 Future<void> loadContactTypes() async {
35     final result = await ApiService.getContactTypes();
36     if (isClosed) return;
37     result.fold(
38         (_) {},
39         (data) {
40             if (data['types'] is List && (data['types'] as List).isNotEmpty) {
41                 contactTypes = data['types'] as List;
42                 if (!isClosed) {
43                     emit(AccountSuspendedTypesLoadedState(types: contactTypes));
44                 }
45             }
46         },
47     );
48 }
49
50 void selectType(String type) {
51     selectedType = type;
52     emit(AccountSuspendedTypeSelectedState(selectedType: type));
53 }
54
55 Future<void> checkStatus({String? email, String? password}) async {
56     if (!isClosed) emit(AccountSuspendedCheckingState());
57
58     if (email != null && email.isNotEmpty && password != null && password.isNotEmpty) {
59         final result = await ApiService.loginUser(
60             LoginRequest(email: email, password: password),
61         );
62         if (isClosed) return;
63
64         result.fold(
65             (error) {
66                 if (!isClosed) {
67                     final isSuspended = error.contains('account_suspended') ||
68                         error.contains('تعليق') ||
69                         error.contains('موقوف');
70                     if (isSuspended) {
71                         emit(
72                             AccountSuspendedStillSuspendedState(
73                                 message: appTranslation().get('account_suspended_still_suspended'),
74                             ),
75                         );
76                     } else {
77                         emit(AccountSuspendedStillSuspendedState(message: error));
78                     }
79                 }
80             },
81             (response) {
82                 if (!isClosed) {
83                     if (response.user.isActive) {
84                         emit(AccountSuspendedActiveState());
85                     } else {
86                         emit(
87                             AccountSuspendedStillSuspendedState(
88                                 message: appTranslation().get('account_suspended_still_suspended'),
89                             ),

```



```

90         );
91     }
92 }
93 },
94 );
95 return;
96 }
97
98 final meResult = await ApiService.getMe();
99 if (isClosed) return;
100
101 meResult.fold(
102     (error) {
103         if (!isClosed) {
104             emit(
105                 AccountSuspendedStillSuspendedState(
106                     message: appTranslation().get('account_suspended_still_suspended'),
107                 ),
108             );
109         }
110     },
111     (user) {
112         if (!isClosed) {
113             if (user.isActive) {
114                 emit(AccountSuspendedActiveState());
115             } else {
116                 emit(
117                     AccountSuspendedStillSuspendedState(
118                         message: appTranslation().get('account_suspended_still_suspended'),
119                     ),
120                 );
121             }
122         }
123     },
124 );
125 }
126
127 Future<void> submitReviewTicket({
128     String? email,
129     String? name,
130     String? phone,
131 }) async {
132     if (!(formKey.currentState?.validate() ?? false)) return;
133
134     if (!isClosed) emit(AccountSuspendedTicketSubmittingState());
135
136     final subject = subjectController.text.trim();
137     final messageDetails = messageController.text.trim();
138     final combinedMessage = subject.isNotEmpty
139         ? '$subject\n$messageDetails'
140         : messageDetails;
141
142     final result = await ApiService.submitContactRequest(
143         name: (name != null && name.isNotEmpty) ? name : 'مستخدم معلق',
144         email: (email != null && email.isNotEmpty) ? email : 'suspended@user.com',
145         phone: phone,
146         type: selectedType,
147         message: combinedMessage,
148     );
149
150     if (isClosed) return;
151
152     result.fold(
153         (error) {
154             if (!isClosed) emit(AccountSuspendedTicketErrorState(message: error));
155         },
156         (_) {
157             messageController.clear();
158             if (!isClosed) {
159                 emit(
160                     AccountSuspendedTicketSuccessState(
161                         message: appTranslation().get('account_suspended_ticket_sent_success'),
162                     ),
163                 );
164             }
165         }
166     );

```

```

165     },
166   );
167 }
168
169 Future<void> launchWhatsApp(String phone) async {
170   final cleanPhone = phone.replaceAll(RegExp(r'^0-9']), '');
171   final formattedPhone = cleanPhone.startsWith('0')
172     ? '966${cleanPhone.substring(1)}'
173     : cleanPhone;
174   final uri = Uri.parse('https://wa.me/$formattedPhone');
175   if (await canLaunchUrl(uri)) {
176     await launchUrl(uri, mode: LaunchMode.externalApplication);
177   }
178 }
179
180 Future<void> launchEmail(String email, {String? subject, String? body}) async {
181   final defaultSubject = 'طلب مراجعة وفك تعليق الحساب';
182   final uri = Uri(
183     scheme: 'mailto',
184     path: email,
185     query: 'subject=${Uri.encodeComponent(subject ?? defaultSubject)}',
186   );
187   try {
188     final launched = await launchUrl(uri, mode: LaunchMode.externalApplication);
189     if (!launched) {
190       await launchUrl(uri);
191     }
192   } catch (_) {
193     try {
194       await launchUrl(uri);
195     } catch (_) {}
196   }
197 }
198
199 Future<void> launchPhone(String phone) async {
200   final uri = Uri(scheme: 'tel', path: phone);
201   if (await canLaunchUrl(uri)) {
202     await launchUrl(uri);
203   }
204 }
205
206 @override
207 Future<void> close() {
208   subjectController.dispose();
209   messageController.dispose();
210   return super.close();
211 }
212 }
213

```

Lines • 35
Dart

Feature / Account #60 -

Suspended lib/features/account_suspended/presentation/cubit/account_suspended_state.dart

```

1  abstract class AccountSuspendedState {}
2
3  class AccountSuspendedInitialState extends AccountSuspendedState {}
4
5  class AccountSuspendedCheckingState extends AccountSuspendedState {}
6
7  class AccountSuspendedActiveState extends AccountSuspendedState {}
8
9  class AccountSuspendedStillSuspendedState extends AccountSuspendedState {
10    final String message;
11    AccountSuspendedStillSuspendedState({required this.message});
12  }
13
14  class AccountSuspendedTypesLoadedState extends AccountSuspendedState {
15    final List<dynamic> types;
16    AccountSuspendedTypesLoadedState({required this.types});
17  }
18
19  class AccountSuspendedTypeSelectedState extends AccountSuspendedState {
20    final String selectedType;

```

```

21   AccountSuspendedTypeSelectedState({required this.selectedType});
22 }
23
24 class AccountSuspendedTicketSubmittingState extends AccountSuspendedState {}
25
26 class AccountSuspendedTicketSuccessState extends AccountSuspendedState {
27   final String message;
28   AccountSuspendedTicketSuccessState({required this.message});
29 }
30
31 class AccountSuspendedTicketErrorState extends AccountSuspendedState {
32   final String message;
33   AccountSuspendedTicketErrorState({required this.message});
34 }
35

```

Lines • 108
Dart

Feature / Account #61 –

Suspended [lib/features/account_suspended/presentation/screen/account_suspended_screen.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/utils/constants/routes.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6  import 'package:moean/core/utils/extensions/context_extension.dart';
7  import 'package:moean/features/account_suspended/presentation/cubit/account_suspended_cubit.dart';
8  import 'package:moean/features/account_suspended/presentation/cubit/account_suspended_state.dart';
9  import 'package:moean/features/account_suspended/presentation/widgets/suspended_email_card_widget.dart';
10 import 'package:moean/features/account_suspended/presentation/widgets/suspended_notice_card_widget.dart';
11 import 'package:moean/features/account_suspended/presentation/widgets/suspended_ticket_form_card_widget.dar
t';
12 import 'package:moean/features/account_suspended/presentation/widgets/suspended_whatsapp_card_widget.dart';
13 import 'package:moean/features/account_suspended/presentation/widgets/suspended_working_hours_card_widget.dar
t';
14
15 class AccountSuspendedScreen extends StatelessWidget {
16   final String? email;
17   final String? password;
18   final String? name;
19   final String? phone;
20
21   const AccountSuspendedScreen({
22     super.key,
23     this.email,
24     this.password,
25     this.name,
26     this.phone,
27   });
28
29   @override
30   Widget build(BuildContext context) {
31     return BlocProvider(
32       create: (_) => AccountSuspendedCubit().loadContactTypes(),
33       child: BlocListener<AccountSuspendedCubit, AccountSuspendedState>(
34         listener: _onStateChanged,
35         child: Scaffold(
36           backgroundColor: ColorsManager.background,
37           appBar: AppBar(
38             backgroundColor: ColorsManager.background,
39             elevation: 0,
40             leading: IconButton(
41               icon: Icon(
42                 Icons.arrow_back_ios_new_rounded,
43                 color: ColorsManager.textPrimary,
44                 size: 20,
45               ),
46               onPressed: () => Navigator.of(context).pop(),
47             ),
48           ),
49           body: SafeArea(
50             child: SingleChildScrollView(
51               padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 12),
52               child: Column(

```

```

53         crossAxisAlignment: CrossAxisAlignment.stretch,
54         children: [
55             SuspendedNoticeCardWidget(
56                 email: email,
57                 password: password,
58             ),
59             verticalSpace16,
60             const SuspendedWhatsappCardWidget(),
61             verticalSpace16,
62             const SuspendedEmailCardWidget(),
63             verticalSpace16,
64             const SuspendedWorkingHoursCardWidget(),
65             verticalSpace16,
66             SuspendedTicketFormCardWidget(
67                 email: email,
68                 name: name,
69                 phone: phone,
70             ),
71             verticalSpace32,
72         ],
73     ),
74 ),
75 ),
76 ),
77 ),
78 );
79 }
80
81 void _onStateChanged(BuildContext context, AccountSuspendedState state) {
82     if (state is AccountSuspendedActiveState) {
83         context.pushNamedAndRemoveUntil(Routes.home, (route) => false);
84     } else if (state is AccountSuspendedStillSuspendedState) {
85         ScaffoldMessenger.of(context).showSnackBar(
86             SnackBar(
87                 content: Text(state.message),
88                 backgroundColor: ColorsManager.errorColor,
89             ),
90         );
91     } else if (state is AccountSuspendedTicketSuccessState) {
92         ScaffoldMessenger.of(context).showSnackBar(
93             SnackBar(
94                 content: Text(state.message),
95                 backgroundColor: ColorsManager.successColor,
96             ),
97         );
98     } else if (state is AccountSuspendedTicketErrorState) {
99         ScaffoldMessenger.of(context).showSnackBar(
100             SnackBar(
101                 content: Text(state.message),
102                 backgroundColor: ColorsManager.errorColor,
103             ),
104         );
105     }
106 }
107 }
108

```

Lines • 42
Dart

Feature / Account #62 -
Suspended

[lib/features/account_suspended/presentation/widgets/suspended_badge_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6
7  class SuspendedBadgeWidget extends StatelessWidget {
8      const SuspendedBadgeWidget({super.key});
9
10     @override
11     Widget build(BuildContext context) {
12         return Container(
13             padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 6),

```

```

14     decoration: BoxDecoration(
15       color: ColorsManager.suspendedBadgeBg,
16       borderRadius: BorderRadius.circular(20),
17       border: Border.all(
18         color: ColorsManager.suspendedBadgeBorder,
19         width: 1,
20       ),
21     ),
22     child: Row(
23       mainAxisAlignment: MainAxisAlignment.min,
24       children: [
25         Icon(
26           Icons.error_outline_rounded,
27           size: 16,
28           color: ColorsManager.suspendedBadgeText,
29         ),
30         horizontalSpace6,
31         Text(
32           appTranslation().get('account_suspended_badge'),
33           style: TextStylesManager.bold13.copyWith(
34             color: ColorsManager.suspendedBadgeText,
35           ),
36         ),
37       ],
38     ),
39   );
40 }
41 }
42

```

Lines 95

• Dart

Feature / Account #63 -

Suspended

lib/features/account_suspended/presentation/widgets/suspended_email_card_widget.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/spacing.dart';
6 import 'package:moean/features/account_suspended/presentation/cubit/account_suspended_cubit.dart';
7
8 class SuspendedEmailCardWidget extends StatelessWidget {
9   const SuspendedEmailCardWidget({super.key});
10
11   @override
12   Widget build(BuildContext context) {
13     final cubit = AccountSuspendedCubit.get(context);
14
15     return Container(
16       width: double.infinity,
17       padding: const EdgeInsets.all(20),
18       decoration: BoxDecoration(
19         color: ColorsManager.surfacePrimary,
20         borderRadius: BorderRadius.circular(20),
21         border: Border.all(
22           color: ColorsManager.borderColor,
23           width: 1,
24         ),
25       ),
26       child: Column(
27         crossAxisAlignment: CrossAxisAlignment.stretch,
28         children: [
29           Align(
30             alignment: AlignmentDirectional.topEnd,
31             child: InkWell(
32               onTap: () => cubit.launchEmail('qraura0@gmail.com'),
33               borderRadius: BorderRadius.circular(14),
34               child: Container(
35                 width: 48,
36                 height: 48,
37                 decoration: BoxDecoration(
38                   color: ColorsManager.emailIconBg,
39                   borderRadius: BorderRadius.circular(14),
40                 ),

```

```

41         child: const Center(
42             child: Icon(
43                 Icons.mail_outline_rounded,
44                 color: ColorsManager.emailIconColor,
45                 size: 24,
46             ),
47         ),
48     ),
49 ),
50 ),
51 verticalSpace12,
52 Text(
53     appTranslation().get('account_suspended_email_title'),
54     style: TextStylesManager.bold18.copyWith(
55         color: ColorsManager.textSecondaryDark,
56     ),
57 ),
58 verticalSpace6,
59 Text(
60     appTranslation().get('account_suspended_email_desc'),
61     style: TextStylesManager.regular13.copyWith(
62         color: ColorsManager.placeholder,
63     ),
64 ),
65 verticalSpace16,
66 InkWell(
67     onTap: () => cubit.launchEmail('qraura0@gmail.com'),
68     borderRadius: BorderRadius.circular(8),
69     child: Padding(
70         padding: const EdgeInsets.symmetric(vertical: 4),
71         child: Row(
72             mainAxisAlignment: MainAxisAlignment.min,
73             children: [
74                 const Icon(
75                     Icons.open_in_new_rounded,
76                     size: 16,
77                     color: ColorsManager.emailIconColor,
78                 ),
79                 horizontalSpace6,
80                 Text(
81                     'qraura0@gmail.com',
82                     style: TextStylesManager.bold14.copyWith(
83                         color: ColorsManager.emailIconColor,
84                     ),
85                 ),
86             ],
87         ),
88     ),
89 ),
90 ],
91 ),
92 );
93 }
94 }
95

```

26
Lines
•
Dart

Feature /
Account
Suspended

#64 -

[lib/features/account_suspended/presentation/widgets/suspended_header_shield_icon_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3
4  class SuspendedHeaderShieldIconWidget extends StatelessWidget {
5      const SuspendedHeaderShieldIconWidget({super.key});
6
7      @override
8      Widget build(BuildContext context) {
9          return Container(
10             width: 68,
11             height: 68,
12             decoration: BoxDecoration(

```

```

13     color: ColorsManager.suspendedIconBg,
14     shape: BoxShape.circle,
15   ),
16   child: const Center(
17     child: Icon(
18       Icons.gpp_maybe_outlined,
19       color: ColorsManager.errorColor,
20       size: 36,
21     ),
22   ),
23 );
24 }
25 }
26

```

Lines 78
• Dart

Feature /
Account
Suspended

#65 -

[lib/features/account_suspended/presentation/widgets/suspended_notice_card_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6  import 'package:moean/features/account_suspended/presentation/widgets/suspended_badge_widget.dart';
7  import 'package:moean/features/account_suspended/presentation/widgets/suspended_header_shield_icon_widget.dar
t';
8  import 'package:moean/features/account_suspended/presentation/widgets/suspended_status_check_widget.dart';
9
10 class SuspendedNoticeCardWidget extends StatelessWidget {
11   final String? email;
12   final String? password;
13
14   const SuspendedNoticeCardWidget({
15     super.key,
16     this.email,
17     this.password,
18   });
19
20   @override
21   Widget build(BuildContext context) {
22     return Container(
23       width: double.infinity,
24       padding: const EdgeInsets.all(20),
25       decoration: BoxDecoration(
26         color: ColorsManager.surfacePrimary,
27         borderRadius: BorderRadius.circular(24),
28         border: Border.all(
29           color: ColorsManager.suspendedCardBorder,
30           width: 1.5,
31         ),
32         boxShadow: [
33           BoxShadow(
34             color: ColorsManager.errorColor.withValues(alpha: 0.04),
35             blurRadius: 16,
36             offset: const Offset(0, 4),
37           ),
38         ],
39     ),
40     child: Column(
41       crossAxisAlignment: CrossAxisAlignment.center,
42       children: [
43         const SuspendedHeaderShieldIconWidget(),
44         verticalSpace14,
45         const SuspendedBadgeWidget(),
46         verticalSpace16,
47         Text(
48           appTranslation().get('account_suspended_title'),
49           style: TextStylesManager.bold20.copyWith(
50             color: ColorsManager.textSecondaryDark,
51           ),
52           textAlign: TextAlign.center,
53         ),

```

```

54         verticalSpace12,
55         Text(
56             appTranslation().get('account_suspended_desc'),
57             style: TextStylesManager.regular13.copyWith(
58                 color: ColorsManager.placeholder,
59                 height: 1.6,
60             ),
61             textAlign: TextAlign.center,
62         ),
63         verticalSpace20,
64         Divider(
65             color: ColorsManager.borderLightGray,
66             thickness: 1,
67         ),
68         verticalSpace16,
69         SuspendedStatusCheckWidget(
70             email: email,
71             password: password,
72         ),
73     ],
74 ),
75 );
76 }
77 }
78

```

Lines 81
• Dart

Feature /
Account
Suspended

#66 -

lib/features/account_suspended/presentation/widgets/suspended_status_check_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
7  import 'package:moean/core/utils/constants/spacing.dart';
8  import 'package:moean/features/account_suspended/presentation/cubit/account_suspended_cubit.dart';
9  import 'package:moean/features/account_suspended/presentation/cubit/account_suspended_state.dart';
10
11  class SuspendedStatusCheckWidget extends StatelessWidget {
12      final String? email;
13      final String? password;
14
15      const SuspendedStatusCheckWidget({
16          super.key,
17          this.email,
18          this.password,
19      });
20
21      @override
22      Widget build(BuildContext context) {
23          return BlocBuilder<AccountSuspendedCubit, AccountSuspendedState>(
24              buildWhen: (previous, current) =>
25                  current is AccountSuspendedCheckingState ||
26                  current is AccountSuspendedActiveState ||
27                  current is AccountSuspendedStillSuspendedState,
28              builder: (context, state) {
29                  final cubit = AccountSuspendedCubit.get(context);
30                  final isChecking = state is AccountSuspendedCheckingState;
31
32                  return Column(
33                      crossAxisAlignment: CrossAxisAlignment.stretch,
34                      children: [
35                          Row(
36                              children: [
37                                  const Icon(
38                                      Icons.access_time_rounded,
39                                      size: 18,
40                                      color: ColorsManager.brandGold,
41                                  ),
42                                  horizontalSpace6,
43                                  Expanded(

```



```

44         child: Text(
45             appTranslation().get('account_suspended_check_prompt'),
46             style: TextStylesManager.medium13.copyWith(
47                 color: ColorsManager.textPrimary,
48             ),
49         ),
50     ),
51 ],
52 ),
53 verticalSpace12,
54 Align(
55     alignment: Alignment.centerLeft,
56     child: PrimaryElevatedButton(
57         width: 200,
58         height: 44,
59         radius: 12,
60         text: appTranslation().get('account_suspended_check_status_btn'),
61         isLoading: isChecking,
62         icon: const Icon(
63             Icons.sync_rounded,
64             size: 20,
65             color: ColorsManager.white,
66         ),
67         onPressed: () {
68             cubit.checkStatus(
69                 email: email,
70                 password: password,
71             );
72         },
73     ),
74 ),
75 ],
76 );
77 },
78 );
79 }
80 }
81

```

131
Lines •
Dart

Feature /
Account
Suspended

#67 –

lib/features/account_suspended/presentation/widgets/suspended_ticket_form_card_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
7  import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
8  import 'package:moean/core/utils/constants/spacing.dart';
9  import 'package:moean/features/account_suspended/presentation/cubit/account_suspended_cubit.dart';
10 import 'package:moean/features/account_suspended/presentation/cubit/account_suspended_state.dart';
11 import 'package:moean/features/account_suspended/presentation/widgets/suspended_ticket_type_dropdown_widget.d
art';
12
13 class SuspendedTicketFormCardWidget extends StatelessWidget {
14     final String? email;
15     final String? name;
16     final String? phone;
17
18     const SuspendedTicketFormCardWidget({
19         super.key,
20         this.email,
21         this.name,
22         this.phone,
23     });
24
25     @override
26     Widget build(BuildContext context) {
27         return BlocBuilder<AccountSuspendedCubit, AccountSuspendedState>(
28             buildWhen: (previous, current) =>
29                 current is AccountSuspendedTicketSubmittingState ||

```

```

30     current is AccountSuspendedTicketSuccessState ||
31     current is AccountSuspendedTicketErrorState,
32 builder: (context, state) {
33     final cubit = AccountSuspendedCubit.get(context);
34     final isSubmitting = state is AccountSuspendedTicketSubmittingState;
35
36     return Container(
37       width: double.infinity,
38       padding: const EdgeInsets.all(20),
39       decoration: BoxDecoration(
40         color: ColorsManager.surfacePrimary,
41         borderRadius: BorderRadius.circular(24),
42         border: Border.all(
43           color: ColorsManager.borderColor,
44           width: 1,
45         ),
46       ),
47       child: Form(
48         key: cubit.formKey,
49         child: Column(
50           crossAxisAlignment: CrossAxisAlignment.stretch,
51           children: [
52             Text(
53               appTranslation().get('account_suspended_form_title'),
54               style: TextStylesManager.bold18.copyWith(
55                 color: ColorsManager.textSecondaryDark,
56               ),
57             ),
58             verticalSpace6,
59             Text(
60               appTranslation().get('account_suspended_form_desc'),
61               style: TextStylesManager.regular13.copyWith(
62                 color: ColorsManager.placeholder,
63                 height: 1.5,
64               ),
65             ),
66             verticalSpace20,
67             Text(
68               appTranslation().get('account_suspended_ticket_type_label'),
69               style: TextStylesManager.bold14.copyWith(
70                 color: ColorsManager.textSecondaryDark,
71               ),
72             ),
73             verticalSpace8,
74             const SuspendedTicketTypeDropdownWidget(),
75             verticalSpace16,
76             Text(
77               appTranslation().get('account_suspended_subject_label'),
78               style: TextStylesManager.bold14.copyWith(
79                 color: ColorsManager.textSecondaryDark,
80               ),
81             ),
82             verticalSpace8,
83             PrimaryTextField(
84               controller: cubit.subjectController,
85               hint: appTranslation().get('account_suspended_subject_default'),
86             ),
87             verticalSpace16,
88             Text(
89               appTranslation().get('account_suspended_message_label'),
90               style: TextStylesManager.bold14.copyWith(
91                 color: ColorsManager.textSecondaryDark,
92               ),
93             ),
94             verticalSpace8,
95             PrimaryTextField(
96               controller: cubit.messageController,
97               hint: appTranslation().get('account_suspended_message_hint'),
98               maxLines: 4,
99               validator: (value) {
100                 if (value == null || value.trim().isEmpty) {
101                   return appTranslation().get('account_suspended_message_required');
102                 }
103                 return null;
104               },

```

```

105         ),
106         verticalSpace24,
107         PrimaryElevatedButton(
108             text: appTranslation().get('account_suspended_submit_btn'),
109             isLoading: isSubmitting,
110             icon: const Icon(
111                 Icons.send_rounded,
112                 size: 18,
113                 color: ColorsManager.white,
114             ),
115             onPressed: () {
116                 cubit.submitReviewTicket(
117                     email: email,
118                     name: name,
119                     phone: phone,
120                 );
121             },
122         ),
123     ],
124 ),
125 ),
126 );
127 },
128 );
129 }
130 }
131

```

70
Lines
•
Dart

Feature /
Account
Suspended

#68 -

lib/features/account_suspended/presentation/widgets/suspended_ticket_type_dropdown_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/features/account_suspended/presentation/cubit/account_suspended_cubit.dart';
6  import 'package:moean/features/account_suspended/presentation/cubit/account_suspended_state.dart';
7
8  class SuspendedTicketTypeDropdownWidget extends StatelessWidget {
9      const SuspendedTicketTypeDropdownWidget({super.key});
10
11      @override
12      Widget build(BuildContext context) {
13          return BlocBuilder<AccountSuspendedCubit, AccountSuspendedState>(
14              buildWhen: (previous, current) =>
15                  current is AccountSuspendedTypesLoadedState ||
16                  current is AccountSuspendedTypeSelectedState,
17              builder: (context, state) {
18                  final cubit = AccountSuspendedCubit.get(context);
19
20                  final items = cubit.contactTypes.map<DropdownMenuItem<String>>((type) {
21                      final val = (type is Map ? type['value'] : type.toString()).toString();
22                      final lbl = (type is Map ? type['label'] : type.toString()).toString();
23                      return DropdownMenuItem<String>(
24                          value: val,
25                          child: Text(
26                              lbl,
27                              style: TextStylesManager.regular14.copyWith(
28                                  color: ColorsManager.textPrimary,
29                              ),
30                      ),
31                  );
32              }).toList();
33
34              final isValidSelection =
35                  cubit.contactTypes.any((t) => (t is Map ? t['value'] : t.toString()).toString() == cubit.selected
Type);
36
37              final value = isValidSelection ? cubit.selectedType : (items.isNotEmpty ? items.first.value : null);
38
39              return Container(
40                  decoration: BoxDecoration(

```

```

40         color: ColorsManager.surfacePrimary,
41         borderRadius: BorderRadius.circular(24),
42         border: Border.all(
43             color: ColorsManager.borderColor,
44         ),
45     ),
46     padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),
47     child: DropdownButtonHideUnderline(
48         child: DropdownButton<String>(
49             isExpanded: true,
50             value: value,
51             icon: const Icon(
52                 Icons.keyboard_arrow_down_rounded,
53                 color: ColorsManager.textSecondaryDark,
54             ),
55             dropdownColor: ColorsManager.surfacePrimary,
56             borderRadius: BorderRadius.circular(16),
57             items: items,
58             onChanged: (newVal) {
59                 if (newVal != null) {
60                     cubit.selectType(newVal);
61                 }
62             },
63         ),
64     ),
65 );
66 },
67 );
68 }
69 }
70

```

91
Lines •
Dart

Feature /
Account
Suspended

#69 –

[lib/features/account_suspended/presentation/widgets/suspended_whatsapp_card_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6  import 'package:moean/features/account_suspended/presentation/cubit/account_suspended_cubit.dart';
7
8  class SuspendedWhatsappCardWidget extends StatelessWidget {
9      const SuspendedWhatsappCardWidget({super.key});
10
11      @override
12      Widget build(BuildContext context) {
13          final cubit = AccountSuspendedCubit.get(context);
14
15          return Container(
16              width: double.infinity,
17              padding: const EdgeInsets.all(20),
18              decoration: BoxDecoration(
19                  color: ColorsManager.surfacePrimary,
20                  borderRadius: BorderRadius.circular(20),
21                  border: Border.all(
22                      color: ColorsManager.borderColor,
23                      width: 1,
24                  ),
25              ),
26              child: Column(
27                  crossAxisAlignment: CrossAxisAlignment.stretch,
28                  children: [
29                      Align(
30                          alignment: AlignmentDirectional.topEnd,
31                          child: Container(
32                              width: 48,
33                              height: 48,
34                              decoration: BoxDecoration(
35                                  color: ColorsManager.whatsappIconBg,
36                                  borderRadius: BorderRadius.circular(14),
37                              ),

```

```

38         child: const Center(
39           child: Icon(
40             Icons.chat_bubble_outline_rounded,
41             color: ColorsManager.whatsappIconColor,
42             size: 24,
43           ),
44         ),
45       ),
46     ),
47     verticalSpace12,
48     Text(
49       appTranslation().get('account_suspended_whatsapp_title'),
50       style: TextStylesManager.bold18.copyWith(
51         color: ColorsManager.textSecondaryDark,
52       ),
53     ),
54     verticalSpace6,
55     Text(
56       appTranslation().get('account_suspended_whatsapp_desc'),
57       style: TextStylesManager.regular13.copyWith(
58         color: ColorsManager.placeholder,
59       ),
60     ),
61     verticalSpace16,
62     InkWell(
63       onTap: () => cubit.launchWhatsApp('01229681818'),
64       borderRadius: BorderRadius.circular(8),
65       child: Padding(
66         padding: const EdgeInsets.symmetric(vertical: 4),
67         child: Row(
68           mainAxisAlignment: MainAxisAlignment.min,
69           children: [
70             Icon(
71               Icons.open_in_new_rounded,
72               size: 16,
73               color: ColorsManager.primaryColor,
74             ),
75             horizontalSpace6,
76             Text(
77               appTranslation().get('account_suspended_whatsapp_btn'),
78               style: TextStylesManager.bold14.copyWith(
79                 color: ColorsManager.primaryColor,
80               ),
81             ),
82           ],
83         ),
84       ),
85     ),
86   ],
87 ),
88 );
89 }
90 }
91

```

92
Lines
•
Dart

Feature /
Account
Suspended

#70 -

lib/features/account_suspended/presentation/widgets/suspended_working_hours_card_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6  import 'package:moean/features/account_suspended/presentation/cubit/account_suspended_cubit.dart';
7
8  class SuspendedWorkingHoursCardWidget extends StatelessWidget {
9    const SuspendedWorkingHoursCardWidget({super.key});
10
11    @override
12    Widget build(BuildContext context) {
13      final cubit = AccountSuspendedCubit.get(context);

```

```

14
15 return Container(
16   width: double.infinity,
17   padding: const EdgeInsets.all(20),
18   decoration: BoxDecoration(
19     color: ColorsManager.surfacePrimary,
20     borderRadius: BorderRadius.circular(20),
21     border: Border.all(
22       color: ColorsManager.borderColor,
23       width: 1,
24     ),
25   ),
26   child: Column(
27     crossAxisAlignment: CrossAxisAlignment.stretch,
28     children: [
29       Align(
30         alignment: AlignmentDirectional.topEnd,
31         child: Container(
32           width: 48,
33           height: 48,
34           decoration: BoxDecoration(
35             color: ColorsManager.workingHoursIconBg,
36             borderRadius: BorderRadius.circular(14),
37           ),
38           child: const Center(
39             child: Icon(
40               Icons.headset_mic_outlined,
41               color: ColorsManager.workingHoursIconColor,
42               size: 24,
43             ),
44           ),
45         ),
46       ),
47       verticalSpace12,
48       Text(
49         appTranslation().get('account_suspended_working_hours_title'),
50         style: TextStyleManager.bold18.copyWith(
51           color: ColorsManager.textSecondaryDark,
52         ),
53       ),
54       verticalSpace6,
55       Text(
56         appTranslation().get('account_suspended_working_hours_desc'),
57         style: TextStyleManager.regular13.copyWith(
58           color: ColorsManager.placeholder,
59         ),
60       ),
61       verticalSpace16,
62       InkWell(
63         onTap: () => cubit.launchPhone('01229681818'),
64         borderRadius: BorderRadius.circular(8),
65         child: Padding(
66           padding: const EdgeInsets.symmetric(vertical: 4),
67           child: Text.rich(
68             TextSpan(
69               children: [
70                 TextSpan(
71                   text: appTranslation().get('account_suspended_phone_prefix'),
72                   style: TextStyleManager.regular14.copyWith(
73                     color: ColorsManager.placeholder,
74                   ),
75                 ),
76                 TextSpan(
77                   text: appTranslation().get('account_suspended_phone_number'),
78                   style: TextStyleManager.bold14.copyWith(
79                     color: ColorsManager.textSecondaryDark,
80                   ),
81               ],
82             ),
83           ),
84         ),
85       ),
86     ],
87   ),
88 )

```

```
89     );
90   }
91 }
92
```

Lines • Dart 126

Feature / Admin

#71 – lib/features/admin/contact/presentation/cubit/admin_contact_cubit.dart

```
1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/network/remote/api_service.dart';
4  import 'package:moean/features/admin/contact/presentation/cubit/admin_contact_state.dart';
5
6  class AdminContactCubit extends Cubit<AdminContactState> {
7    AdminContactCubit() : super(AdminContactInitial());
8
9    static AdminContactCubit get(BuildContext context) => BlocProvider.of(context);
10
11    Map<String, dynamic>? stats;
12    List<dynamic> tickets = [];
13
14    Future<void> getStats() async {
15      if (!isClosed) emit(AdminContactStatsLoading());
16
17      final result = await ApiService.getAdminContactStats();
18      if (isClosed) return;
19      result.fold(
20        (error) {
21          if (!isClosed) emit(AdminContactStatsError(error));
22        },
23        (data) {
24          stats = data;
25          if (!isClosed) emit(AdminContactStatsLoaded(data));
26        },
27      );
28    }
29
30    Future<void> getTickets({String? status, String? type, String? search}) async {
31      if (!isClosed) emit(AdminContactTicketsLoading());
32
33      final result = await ApiService.getAdminContactTickets(
34        status: status,
35        type: type,
36        search: search,
37      );
38      if (isClosed) return;
39
40      result.fold(
41        (error) {
42          if (!isClosed) emit(AdminContactTicketsError(error));
43        },
44        (data) {
45          tickets = data['data'] ?? [];
46          if (!isClosed) emit(AdminContactTicketsLoaded(tickets));
47        },
48      );
49    }
50
51    Map<String, dynamic>? currentTicket;
52
53    Future<void> getTicketDetails(int id) async {
54      if (!isClosed) emit(AdminContactTicketDetailsLoading());
55
56      final result = await ApiService.getAdminContactTicketDetails(id: id);
57      if (isClosed) return;
58      result.fold(
59        (error) {
60          if (!isClosed) emit(AdminContactTicketDetailsError(error));
61        },
62        (data) {
63          currentTicket = data;
64          if (!isClosed) emit(AdminContactTicketDetailsLoaded(data));
65        },
66      );
67    }
68  }
```

```

67     }
68
69     Future<void> updateTicket({required int id, String? status, String? adminNotes}) async {
70       if (!isClosed) emit(AdminContactTicketUpdateLoading());
71
72       final result = await ApiService.updateAdminContactTicket(
73         id: id,
74         status: status,
75         adminNotes: adminNotes,
76       );
77       if (isClosed) return;
78
79       result.fold(
80         (error) {
81           if (!isClosed) emit(AdminContactTicketUpdateError(error));
82         },
83         (data) {
84           if (!isClosed) emit(AdminContactTicketUpdateSuccess(data['message'] ?? 'تم التحديث بنجاح'));
85           getTicketDetails(id);
86         },
87       );
88     }
89
90     Future<void> replyToTicket({required int id, required String body}) async {
91       if (!isClosed) emit(AdminContactTicketReplyLoading());
92
93       final result = await ApiService.replyAdminContactTicket(
94         id: id,
95         body: body,
96       );
97       if (isClosed) return;
98
99       result.fold(
100         (error) {
101           if (!isClosed) emit(AdminContactTicketReplyError(error));
102         },
103         (data) {
104           if (!isClosed) emit(AdminContactTicketReplySuccess(data['message'] ?? 'تم إرسال الرد بنجاح'));
105           getTicketDetails(id);
106         },
107       );
108     }
109
110     Future<void> deleteTicket(int id) async {
111       if (!isClosed) emit(AdminContactTicketDeleteLoading());
112
113       final result = await ApiService.deleteAdminContactTicket(id: id);
114       if (isClosed) return;
115
116       result.fold(
117         (error) {
118           if (!isClosed) emit(AdminContactTicketDeleteError(error));
119         },
120         (_) {
121           if (!isClosed) emit(AdminContactTicketDeleteSuccess());
122         },
123       );
124     }
125   }
126

```

Lines • Dart 82

Feature / Admin

#72 – lib/features/admin/contact/presentation/cubit/admin_contact_state.dart

```

1  import 'package:flutter/material.dart';
2
3  @immutable
4  abstract class AdminContactState {}
5
6  class AdminContactInitial extends AdminContactState {}
7
8  // Stats States
9  class AdminContactStatsLoading extends AdminContactState {}
10

```



```
11 class AdminContactStatsLoaded extends AdminContactState {
12     final Map<String, dynamic> stats;
13     AdminContactStatsLoaded(this.stats);
14 }
15
16 class AdminContactStatsError extends AdminContactState {
17     final String error;
18     AdminContactStatsError(this.error);
19 }
20
21 // Tickets States
22 class AdminContactTicketsLoading extends AdminContactState {}
23
24 class AdminContactTicketsLoaded extends AdminContactState {
25     final List<dynamic> tickets;
26     AdminContactTicketsLoaded(this.tickets);
27 }
28
29 class AdminContactTicketsError extends AdminContactState {
30     final String error;
31     AdminContactTicketsError(this.error);
32 }
33
34 // Ticket Details States
35 class AdminContactTicketDetailsLoading extends AdminContactState {}
36
37 class AdminContactTicketDetailsLoaded extends AdminContactState {
38     final Map<String, dynamic> ticket;
39     AdminContactTicketDetailsLoaded(this.ticket);
40 }
41
42 class AdminContactTicketDetailsError extends AdminContactState {
43     final String error;
44     AdminContactTicketDetailsError(this.error);
45 }
46
47 // Update Ticket States
48 class AdminContactTicketUpdateLoading extends AdminContactState {}
49
50 class AdminContactTicketUpdateSuccess extends AdminContactState {
51     final String message;
52     AdminContactTicketUpdateSuccess(this.message);
53 }
54
55 class AdminContactTicketUpdateError extends AdminContactState {
56     final String error;
57     AdminContactTicketUpdateError(this.error);
58 }
59
60 // Reply Ticket States
61 class AdminContactTicketReplyLoading extends AdminContactState {}
62
63 class AdminContactTicketReplySuccess extends AdminContactState {
64     final String message;
65     AdminContactTicketReplySuccess(this.message);
66 }
67
68 class AdminContactTicketReplyError extends AdminContactState {
69     final String error;
70     AdminContactTicketReplyError(this.error);
71 }
72
73 // Delete Ticket States
74 class AdminContactTicketDeleteLoading extends AdminContactState {}
75
76 class AdminContactTicketDeleteSuccess extends AdminContactState {}
77
78 class AdminContactTicketDeleteError extends AdminContactState {
79     final String error;
80     AdminContactTicketDeleteError(this.error);
81 }
82
```

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/spacing.dart';
7 import 'package:moean/features/admin/contact/presentation/cubit/admin_contact_cubit.dart';
8 import 'package:moean/features/admin/contact/presentation/widgets/admin_contact_stats_widget.dart';
9 import 'package:moean/features/admin/contact/presentation/widgets/admin_contact_list_widget.dart';
10
11 class AdminContactScreen extends StatelessWidget {
12   const AdminContactScreen({super.key});
13
14   @override
15   Widget build(BuildContext context) {
16     return BlocProvider(
17       create: (context) => AdminContactCubit()..getStats()..getTickets(),
18       child: Directionality(
19         textDirection: TextDirection.rtl,
20         child: Scaffold(
21           backgroundColor: ColorsManager.background,
22           appBar: AppBar(
23             backgroundColor: ColorsManager.surfacePrimary,
24             elevation: 0,
25             title: Text(
26               appTranslation().get('admin_contact_title'),
27               style: TextStylesManager.bold18.copyWith(color: ColorsManager.mainText),
28             ),
29             centerTitle: true,
30             iconTheme: IconThemeData(color: ColorsManager.mainText),
31           ),
32           body: Padding(
33             padding: EdgeInsets.all(16.0),
34             child: Column(
35               children: [
36                 AdminContactStatsWidget(),
37                 verticalSpace24,
38                 Expanded(child: AdminContactListWidget()),
39               ],
40             ),
41           ),
42         ),
43       ),
44     );
45   }
46 }
47

```

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/spacing.dart';
7 import 'package:moean/core/utils/extensions/context_extension.dart';
8 import 'package:moean/features/admin/contact/presentation/cubit/admin_contact_cubit.dart';
9 import 'package:moean/features/admin/contact/presentation/cubit/admin_contact_state.dart';
10 import 'package:moean/features/admin/contact/presentation/widgets/admin_ticket_info_card.dart';
11 import 'package:moean/features/admin/contact/presentation/widgets/admin_ticket_chat_history.dart';
12
13 class AdminTicketDetailsScreen extends StatefulWidget {
14   const AdminTicketDetailsScreen({super.key});
15
16   @override
17   State<AdminTicketDetailsScreen> createState() => _AdminTicketDetailsScreenState();
18 }
19
20 class _AdminTicketDetailsScreenState extends State<AdminTicketDetailsScreen> {

```

```

1 late int ticketId;
2 bool _isInit = false;
3
4
5 @override
6 void didChangeDependencies() {
7   super.didChangeDependencies();
8   if (!_isInit) {
9     final args = context.getArg<Map<String, dynamic>>();
10    ticketId = args?['id'] ?? 0;
11    if (ticketId != 0) {
12      AdminContactCubit.get(context).getTicketDetails(ticketId);
13    }
14    _isInit = true;
15  }
16 }
17
18 @override
19 Widget build(BuildContext context) {
20   return Directionality(
21     textDirection: TextDirection.rtl,
22     child: Scaffold(
23       backgroundColor: ColorsManager.background,
24       appBar: AppBar(
25         backgroundColor: ColorsManager.background,
26         elevation: 0,
27         leading: IconButton(
28           icon: Icon(Icons.arrow_back, color: ColorsManager.mainText),
29           onPressed: () => context.pop(true), // pass true to refresh list if needed
30         ),
31         title: Text(
32           appTranslation().get('admin_ticket_details'),
33           style: TextStylesManager.bold18.copyWith(color: ColorsManager.mainText),
34         ),
35         centerTitle: true,
36       ),
37       body: BlocConsumer<AdminContactCubit, AdminContactState>(
38         listener: (context, state) {
39           if (state is AdminContactTicketUpdateSuccess) {
40             ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(state.message, style: TextStylesManager.bold14.copyWith(color: ColorsManager.white)), backgroundColor: Colors.green));
41           } else if (state is AdminContactTicketUpdateError) {
42             ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(state.error, style: TextStylesManager.bold14.copyWith(color: ColorsManager.white)), backgroundColor: Colors.red));
43           } else if (state is AdminContactTicketReplySuccess) {
44             ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(state.message, style: TextStylesManager.bold14.copyWith(color: ColorsManager.white)), backgroundColor: Colors.green));
45           } else if (state is AdminContactTicketReplyError) {
46             ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(state.error, style: TextStylesManager.bold14.copyWith(color: ColorsManager.white)), backgroundColor: Colors.red));
47           } else if (state is AdminContactTicketDeleteSuccess) {
48             ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(appTranslation().get('ticket_deleted'), style: TextStylesManager.bold14.copyWith(color: ColorsManager.white)), backgroundColor: Colors.green));
49             context.pop(true);
50           } else if (state is AdminContactTicketDeleteError) {
51             ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(state.error, style: TextStylesManager.bold14.copyWith(color: ColorsManager.white)), backgroundColor: Colors.red));
52           }
53         },
54         buildWhen: (previous, current) => current is AdminContactTicketDetailsLoading || current is AdminContactTicketDetailsLoaded || current is AdminContactTicketDetailsError,
55         builder: (context, state) {
56           if (state is AdminContactTicketDetailsLoading || AdminContactCubit.get(context).currentTicket == null) {
57             return const Center(child: CircularProgressIndicator());
58           } else if (state is AdminContactTicketDetailsError) {
59             return Center(child: Text(state.error, style: TextStylesManager.bold16.copyWith(color: ColorsManager.mainText)));
60           }
61
62           final ticket = AdminContactCubit.get(context).currentTicket!;
63           return Padding(
64             padding: const EdgeInsets.all(24.0),
65             child: LayoutBuilder(
66               builder: (context, constraints) {
67                 if (constraints.maxWidth > 800) {

```

```

87         return Row(
88             crossAxisAlignment: CrossAxisAlignment.start,
89             children: [
90                 Expanded(
91                     flex: 2,
92                     child: AdminTicketChatHistory(ticket: ticket),
93                 ),
94                 horizontalSpace24,
95                 Expanded(
96                     flex: 1,
97                     child: AdminTicketInfoCard(ticket: ticket),
98                 ),
99             ],
100         );
101     } else {
102         return SingleChildScrollView(
103             child: Column(
104                 crossAxisAlignment: CrossAxisAlignment.stretch,
105                 children: [
106                     AdminTicketInfoCard(ticket: ticket),
107                     verticalSpace24,
108                     AdminTicketChatHistory(ticket: ticket),
109                 ],
110             ),
111         );
112     }
113 },
114 ),
115 );
116 },
117 ),
118 ),
119 );
120 }
121 }
122

```

Lines • 160
Dart

Feature / #75 -

Admin [lib/features/admin/contact/presentation/widgets/admin_contact_list_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7  import 'package:moean/core/utils/constants/routes.dart';
8  import 'package:moean/core/utils/extensions/context_extension.dart';
9  import 'package:moean/features/admin/contact/presentation/cubit/admin_contact_cubit.dart';
10 import 'package:moean/features/admin/contact/presentation/cubit/admin_contact_state.dart';
11
12 class AdminContactListWidget extends StatelessWidget {
13     const AdminContactListWidget({super.key});
14
15     @override
16     Widget build(BuildContext context) {
17         return BlocBuilder<AdminContactCubit, AdminContactState>(
18             buildWhen: (previous, current) => current is AdminContactTicketsLoading || current is AdminContactTicketsLoaded || current is AdminContactTicketsError,
19             builder: (context, state) {
20                 if (state is AdminContactTicketsLoading) {
21                     return const Center(child: CircularProgressIndicator());
22                 } else if (state is AdminContactTicketsLoaded) {
23                     final tickets = state.tickets;
24                     if (tickets.isEmpty) {
25                         return Center(
26                             child: Text(
27                                 appTranslation().get('no_tickets') ?? 'لا توجد تذاكر',
28                                 style: TextStylesManager.bold16.copyWith(color: ColorsManager.secondaryText),
29                             ),
30                         );
31                     }
32                     return ListView.separated(

```

```

33         itemCount: tickets.length,
34         separatorBuilder: (context, index) => verticalSpace12,
35         itemBuilder: (context, index) {
36             final ticket = tickets[index];
37             return TicketListItemWidget(ticket: ticket);
38         },
39     );
40 } else if (state is AdminContactTicketsError) {
41     return Center(child: Text(state.error, style: TextStyleManager.bold14.copyWith(color: Colors.re
d)));
42 }
43 return const SizedBox.shrink();
44 },
45 );
46 }
47 }
48
49 class TicketListItemWidget extends StatelessWidget {
50     final dynamic ticket;
51
52     const TicketListItemWidget({super.key, required this.ticket});
53
54     @override
55     Widget build(BuildContext context) {
56         final status = ticket['status'] ?? 'pending';
57         final statusLabel = ticket['status_label'] ?? 'قيد الانتظار';
58         final name = ticket['name'] ?? 'بدون اسم';
59         final email = ticket['email'] ?? '';
60         final typeLabel = ticket['type_label'] ?? '';
61         final createdAt = ticket['created_at'] != null ? DateTime.tryParse(ticket['created_at']) : null;
62
63         Color statusColor;
64         switch (status) {
65             case 'resolved':
66                 statusColor = Colors.green;
67                 break;
68             case 'in_progress':
69                 statusColor = Colors.blue;
70                 break;
71             case 'pending':
72                 statusColor = Colors.orange;
73                 break;
74             default:
75                 statusColor = Colors.grey;
76         }
77
78         return InkWell(
79             onTap: () {
80                 Navigator.pushNamed(context, Routes.adminTicketDetails, arguments: {'id': ticket['id']}).then((result) {
81                     if (result == true) {
82                         if (context.mounted) {
83                             AdminContactCubit.get(context).getStats();
84                             AdminContactCubit.get(context).getTickets();
85                         }
86                     }
87                 });
88             },
89             borderRadius: BorderRadius.circular(12),
90             child: Container(
91                 padding: const EdgeInsets.all(16),
92                 decoration: BoxDecoration(
93                     color: ColorsManager.surfacePrimary,
94                     borderRadius: BorderRadius.circular(12),
95                     border: Border.all(color: ColorsManager.borderColor),
96                 ),
97             child: Column(
98                 crossAxisAlignment: CrossAxisAlignment.start,
99                 children: [
100                     Row(
101                         mainAxisAlignment: MainAxisAlignment.spaceBetween,
102                         children: [
103                             Text(
104                                 name,
105                                 style: TextStyleManager.bold16.copyWith(color: ColorsManager.mainText),

```

```

106     ),
107     Container(
108       padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
109       decoration: BoxDecoration(
110         color: statusColor.withValues(alpha: 0.1),
111         borderRadius: BorderRadius.circular(20),
112         border: Border.all(color: statusColor.withValues(alpha: 0.5)),
113       ),
114       child: Row(
115         children: [
116           Icon(Icons.info_outline, size: 14, color: statusColor),
117           horizontalSpace6,
118           Text(
119             statusLabel,
120             style: TextStylesManager.bold12.copyWith(color: statusColor),
121           ),
122         ],
123       ),
124     ),
125   ],
126 ),
127 verticalSpace4,
128 Text(
129   email,
130   style: TextStylesManager.regular12.copyWith(color: ColorsManager.secondaryText),
131 ),
132 verticalSpace12,
133 Row(
134   mainAxisAlignment: MainAxisAlignment.spaceBetween,
135   children: [
136     Container(
137       padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
138       decoration: BoxDecoration(
139         color: ColorsManager.isDark ? Colors.white.withValues(alpha: 0.05) : Colors.grey.shade100,
140         borderRadius: BorderRadius.circular(20),
141       ),
142       child: Text(
143         typeLabel,
144         style: TextStylesManager.bold12.copyWith(color: ColorsManager.mainText),
145       ),
146     ),
147     if (createdAt != null)
148       Text(
149         '${createdAt.year}-${createdAt.month.toString().padLeft(2, '0')}-${createdAt.day.toString
150         ().padLeft(2, '0')}}',
151         style: TextStylesManager.regular12.copyWith(color: ColorsManager.secondaryText),
152       ),
153   ],
154 ),
155 ],
156 ),
157 );
158 }
159 }
160

```

Lines • 132
Dart

Feature / #76 –
Admin

[lib/features/admin/contact/presentation/widgets/admin_contact_stats_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7  import 'package:moean/features/admin/contact/presentation/cubit/admin_contact_cubit.dart';
8  import 'package:moean/features/admin/contact/presentation/cubit/admin_contact_state.dart';
9
10 class AdminContactStatsWidget extends StatelessWidget {
11   const AdminContactStatsWidget({super.key});
12
13   @override

```

```

14   Widget build(BuildContext context) {
15     return BlocBuilder<AdminContactCubit, AdminContactState>(
16       buildWhen: (previous, current) => current is AdminContactStatsLoading || current is AdminContactStatsLo
added | current is AdminContactStatsError,
17     builder: (context, state) {
18       if (state is AdminContactStatsLoading) {
19         return const Center(child: CircularProgressIndicator());
20       } else if (state is AdminContactStatsLoaded) {
21         final stats = state.stats;
22         final total = stats['total'] ?? 0;
23         final byStatus = stats['by_status'] ?? {};
24         final resolved = byStatus['resolved'] ?? 0;
25         final inProgress = byStatus['in_progress'] ?? 0;
26         final pending = byStatus['pending'] ?? 0;
27         final closed = byStatus['closed'] ?? 0;
28
29         return SingleChildScrollView(
30           scrollDirection: Axis.horizontal,
31           child: Row(
32             children: [
33               StatCardWidget(
34                 title: appTranslation().get('total_tickets') ,
35                 value: total.toString(),
36                 icon: Icons.chat_bubble_outline,
37                 color: ColorsManager.primaryColor,
38                 bgColor: ColorsManager.primaryColor.withValues(alpha: 0.1),
39               ),
40               horizontalSpacel2,
41               StatCardWidget(
42                 title: appTranslation().get('resolved'),
43                 value: resolved.toString(),
44                 icon: Icons.check_circle_outline,
45                 color: Colors.green,
46                 bgColor: Colors.green.withValues(alpha: 0.1),
47               ),
48               horizontalSpacel2,
49               StatCardWidget(
50                 title: appTranslation().get('in_progress'),
51                 value: inProgress.toString(),
52                 icon: Icons.access_time,
53                 color: Colors.blue,
54                 bgColor: Colors.blue.withValues(alpha: 0.1),
55               ),
56               horizontalSpacel2,
57               StatCardWidget(
58                 title: appTranslation().get('pending'),
59                 value: pending.toString(),
60                 icon: Icons.hourglass_empty,
61                 color: Colors.orange,
62                 bgColor: Colors.orange.withValues(alpha: 0.1),
63               ),
64               horizontalSpacel2,
65               StatCardWidget(
66                 title: appTranslation().get('closed'),
67                 value: closed.toString(),
68                 icon: Icons.cancel_outlined,
69                 color: Colors.grey,
70                 bgColor: Colors.grey.withValues(alpha: 0.1),
71               ),
72             ],
73           ),
74         );
75       } else if (state is AdminContactStatsError) {
76         return Center(child: Text(state.error, style: TextStyleManager.bold14.copyWith(color: Colors.re
d)));
77       }
78       return const SizedBox.shrink();
79     },
80   );
81 }
82
83
84 class StatCardWidget extends StatelessWidget {
85   final String title;
86   final String value;

```

```

87   final IconData icon;
88   final Color color;
89   final Color bgColor;
90
91   const StatCardWidget({
92     super.key,
93     required this.title,
94     required this.value,
95     required this.icon,
96     required this.color,
97     required this.bgColor,
98   });
99
100  @override
101  Widget build(BuildContext context) {
102    return Container(
103      width: 140,
104      padding: const EdgeInsets.all(16),
105      decoration: BoxDecoration(
106        color: bgColor,
107        borderRadius: BorderRadius.circular(16),
108      ),
109      child: Column(
110        crossAxisAlignment: CrossAxisAlignment.start,
111        children: [
112          Row(
113            mainAxisAlignment: MainAxisAlignment.spaceBetween,
114            children: [
115              Icon(icon, color: color),
116              Text(
117                value,
118                style: TextStyleManager.bold24.copyWith(color: ColorsManager.mainText),
119              ),
120            ],
121          ),
122          verticalSpace16,
123          Text(
124            title,
125            style: TextStyleManager.bold14.copyWith(color: ColorsManager.mainText),
126          ),
127        ],
128      ),
129    );
130  }
131 }
132

```

Lines • 368
Dart

Feature / #77 -

Admin lib/features/admin/contact/presentation/widgets/admin_ticket_chat_history.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
6  import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
7  import 'package:moean/core/utils/constants/spacing.dart';
8  import 'package:moean/features/admin/contact/presentation/cubit/admin_contact_cubit.dart';
9
10 class AdminTicketChatHistory extends StatefulWidget {
11   final Map<String, dynamic> ticket;
12
13   const AdminTicketChatHistory({super.key, required this.ticket});
14
15   @override
16   State<AdminTicketChatHistory> createState() => _AdminTicketChatHistoryState();
17 }
18
19 class _AdminTicketChatHistoryState extends State<AdminTicketChatHistory> {
20   final TextEditingController _replyController = TextEditingController();
21
22   @override
23   void dispose() {

```



```

24     _replyController.dispose();
25     super.dispose();
26 }
27
28 @override
29 Widget build(BuildContext context) {
30     return Column(
31         crossAxisAlignment: CrossAxisAlignment.stretch,
32         children: [
33             _buildTopCard(context),
34             verticalSpace24,
35             _buildChatCard(context),
36         ],
37     );
38 }
39
40 Widget _buildTopCard(BuildContext context) {
41     final status = widget.ticket['status'] ?? 'pending';
42     final statusLabel = widget.ticket['status_label'] ?? appTranslation().get('pending');
43     final user = widget.ticket['user'];
44     final name = widget.ticket['name'] ?? user?['name'] ?? '';
45     final email = widget.ticket['email'] ?? user?['email'] ?? '';
46     final phone = widget.ticket['phone'] ?? user?['phone'] ?? '';
47     final typeLabel = widget.ticket['type_label'] ?? '';
48     final message = widget.ticket['message'] ?? '';
49     final createdAt = _formatDate(widget.ticket['created_at']);
50
51     Color statusColor;
52     switch (status) {
53         case 'resolved':
54             statusColor = Colors.green;
55             break;
56         case 'in_progress':
57             statusColor = Colors.blue;
58             break;
59         case 'pending':
60             statusColor = Colors.orange;
61             break;
62         default:
63             statusColor = Colors.grey;
64     }
65
66     return Container(
67         padding: const EdgeInsets.all(24),
68         decoration: BoxDecoration(
69             color: ColorsManager.surfacePrimary,
70             borderRadius: BorderRadius.circular(16),
71             boxShadow: [
72                 BoxShadow(
73                     color: Colors.black.withValues(alpha: 0.05),
74                     blurRadius: 10,
75                     offset: const Offset(0, 4),
76                 ),
77             ],
78     ),
79     child: Column(
80         crossAxisAlignment: CrossAxisAlignment.start,
81         children: [
82             Row(
83                 children: [
84                     Expanded(
85                         child: Wrap(
86                             crossAxisAlignment: WrapCrossAlignment.center,
87                             spacing: 12,
88                             runSpacing: 8,
89                             children: [
90                                 Text(
91                                     name,
92                                     style: TextStylesManager.bold18.copyWith(color: ColorsManager.primaryColor),
93                                 ),
94                                 Container(
95                                     padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
96                                     decoration: BoxDecoration(
97                                         color: statusColor.withValues(alpha: 0.1),
98                                         borderRadius: BorderRadius.circular(20),

```

```

99         border: Border.all(color: statusColor.withValues(alpha: 0.5)),
100     ),
101     child: Row(
102         mainAxisAlignment: MainAxisAlignment.min,
103         children: [
104             Icon(Icons.access_time, size: 14, color: statusColor),
105             horizontalSpace6,
106             Text(
107                 statusLabel,
108                 style: TextStylesManager.bold12.copyWith(color: statusColor),
109             ),
110         ],
111     ),
112 ),
113 ],
114 ),
115 ),
116 IconButton(
117     onPressed: () {
118         _showDeleteDialog(context, widget.ticket['id']);
119     },
120     icon: const Icon(Icons.delete_outline, color: Colors.red),
121     style: IconButton.styleFrom(
122         backgroundColor: Colors.red.withValues(alpha: 0.1),
123     ),
124 ),
125 ],
126 ),
127 verticalSpace16,
128 Wrap(
129     spacing: 16,
130     runSpacing: 12,
131     crossAxisAlignment: WrapCrossAlignment.center,
132     children: [
133         if (email.isNotEmpty)
134             Row(
135                 mainAxisAlignment: MainAxisAlignment.min,
136                 children: [
137                     Icon(Icons.email_outlined, size: 16, color: ColorsManager.secondaryText),
138                     horizontalSpace8,
139                     Text(email, style: TextStylesManager.regular14.copyWith(color: ColorsManager.secondaryText)),
140                 ],
141             ),
142         if (phone.isNotEmpty)
143             Row(
144                 mainAxisAlignment: MainAxisAlignment.min,
145                 children: [
146                     Icon(Icons.phone_outlined, size: 16, color: ColorsManager.secondaryText),
147                     horizontalSpace8,
148                     Text(phone, style: TextStylesManager.regular14.copyWith(color: ColorsManager.secondaryText)),
149                 ],
150             ),
151         Container(
152             padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
153             decoration: BoxDecoration(
154                 color: ColorsManager.statusSuccess.withValues(alpha: 0.1),
155                 borderRadius: BorderRadius.circular(20),
156             ),
157             child: Text(typeLabel, style: TextStylesManager.bold12.copyWith(color: ColorsManager.statusSuccess)),
158         ),
159         Text(createdAt, style: TextStylesManager.regular12.copyWith(color: ColorsManager.secondaryText)),
160     ],
161 ),
162 verticalSpace24,
163 Container(
164     width: double.infinity,
165     padding: const EdgeInsets.all(16),
166     decoration: BoxDecoration(
167         color: ColorsManager.background,
168         borderRadius: BorderRadius.circular(12),
169         border: Border.all(color: ColorsManager.borderColor),

```

```

170     ),
171     child: Column(
172       crossAxisAlignment: CrossAxisAlignment.start,
173       children: [
174         Text(
175           appTranslation().get('original_message'),
176           style: TextStyleManager.bold12.copyWith(color: ColorsManager.secondaryText),
177         ),
178         verticalSpace12,
179         Text(
180           message,
181           style: TextStyleManager.regular14.copyWith(color: ColorsManager.mainText),
182         ),
183       ],
184     ),
185   ),
186 ],
187 ),
188 );
189 }
190
191 Widget _buildChatCard(BuildContext context) {
192   final replies = (widget.ticket['replies'] as List<dynamic>?) ?? [];
193
194   return Container(
195     decoration: BoxDecoration(
196       color: ColorsManager.surfacePrimary,
197       borderRadius: BorderRadius.circular(16),
198       boxShadow: [
199         BoxShadow(
200           color: Colors.black.withValues(alpha: 0.05),
201           blurRadius: 10,
202           offset: const Offset(0, 4),
203         ),
204       ],
205     ),
206     child: Column(
207       crossAxisAlignment: CrossAxisAlignment.stretch,
208       children: [
209         Padding(
210           padding: const EdgeInsets.all(24.0),
211           child: Row(
212             children: [
213               Icon(Icons.chat_bubble_outline, color: ColorsManager.primaryColor),
214               horizontalSpace12,
215               Text(
216                 '${appTranslation().get('chat_history')} (${replies.length}}',
217                 style: TextStyleManager.bold16.copyWith(color: ColorsManager.primaryColor),
218               ),
219             ],
220           ),
221         ),
222         const Divider(height: 1),
223         if (replies.isNotEmpty)
224           Padding(
225             padding: const EdgeInsets.all(24.0),
226             child: ListView.separated(
227               shrinkWrap: true,
228               physics: const NeverScrollableScrollPhysics(),
229               itemCount: replies.length,
230               separatorBuilder: (context, index) => verticalSpace24,
231               itemBuilder: (context, index) {
232                 final reply = replies[index];
233                 final isClient = reply['sender_type'] == 'client';
234                 final senderName = reply['sender_name'] ?? (isClient ? appTranslation().get('client') : app
Translation().get('admin'));
235                 final date = _formatDateWithTime(reply['created_at']);
236                 final body = reply['body'] ?? '';
237
238                 return Column(
239                   crossAxisAlignment: isClient ? CrossAxisAlignment.start : CrossAxisAlignment.end,
240                   children: [
241                     Row(
242                       mainAxisAlignment: isClient ? MainAxisAlignment.start : MainAxisAlignment.end,
243                       children: [

```

```

244         Flexible(
245             child: Text(
246                 senderName,
247                 style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainText),
248                 overflow: TextOverflow.ellipsis,
249             ),
250         ),
251         horizontalSpace8,
252         Text(
253             date,
254             style: TextStylesManager.regular12.copyWith(color: ColorsManager.secondaryText),
255         ),
256     ],
257 ),
258 verticalSpace8,
259 Container(
260     padding: const EdgeInsets.all(16),
261     decoration: BoxDecoration(
262         color: isClient ? ColorsManager.primaryColor.withValues(alpha: 0.1) : ColorsManager
r.background,
263         borderRadius: BorderRadius.circular(12).copyWith(
264             bottomRight: isClient ? Radius.zero : const Radius.circular(12),
265             bottomLeft: isClient ? const Radius.circular(12) : Radius.zero,
266         ),
267     ),
268     child: Text(
269         body,
270         style: TextStylesManager.regular14.copyWith(color: ColorsManager.mainText),
271     ),
272 ),
273 ],
274 );
275 },
276 ),
277 ),
278 Padding(
279     padding: const EdgeInsets.all(8.0),
280     child: Row(
281         children: [
282             Expanded(
283                 child: PrimaryTextField(
284                     controller: _replyController,
285                     hint: appTranslation().get('write_your_reply_here'),
286                 ),
287             ),
288             horizontalSpace16,
289             PrimaryElevatedButton(
290                 width: 80,
291                 text: '',
292                 icon: const Icon(Icons.send_sharp, color: Colors.white),
293                 onPressed: () {
294                     if (_replyController.text.trim().isNotEmpty) {
295                         AdminContactCubit.get(context).replyToTicket(
296                             id: widget.ticket['id'],
297                             body: _replyController.text.trim(),
298                         );
299                         _replyController.clear();
300                     }
301                 },
302             ),
303         ],
304     ),
305 ),
306 ],
307 ),
308 );
309 );
310 }
311
312 void _showDeleteDialog(BuildContext context, int id) {
313     showDialog(
314         context: context,
315         builder: (dialogContext) {
316             return Directionality(
317                 textDirection: TextDirection.rtl,

```

```

318         child: AlertDialog(
319             backgroundColor: ColorsManager.surfacePrimary,
320             title: Text(appTranslation().get('delete_ticket'), style: TextStylesManager.bold16.copyWith(color: ColorsManager.mainText)),
321             content: Text(appTranslation().get('delete_ticket_confirm'), style: TextStylesManager.regular14.copyWith(color: ColorsManager.mainText)),
322             actions: [
323                 TextButton(
324                     onPressed: () => Navigator.pop(dialogContext),
325                     child: Text(appTranslation().get('cancel'), style: TextStylesManager.bold14.copyWith(color: ColorsManager.primaryColor)),
326                 ),
327                 ElevatedButton(
328                     onPressed: () {
329                         Navigator.pop(dialogContext);
330                         AdminContactCubit.get(context).deleteTicket(id);
331                     },
332                     style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
333                     child: Text(appTranslation().get('delete'), style: TextStylesManager.bold14.copyWith(color: ColorsManager.white)),
334                 ),
335             ],
336         ),
337     );
338 },
339 );
340 }
341
342 String _formatDate(String? dateStr) {
343     if (dateStr == null) return '';
344     final dt = DateTime.tryParse(dateStr);
345     if (dt == null) return '';
346     return '${dt.day} ${_getMonthName(dt.month)} ${dt.year}';
347 }
348
349 String _formatDateWithTime(String? dateStr) {
350     if (dateStr == null) return '';
351     final dt = DateTime.tryParse(dateStr);
352     if (dt == null) return '';
353     final amPm = dt.hour >= 12 ? 'ص' : 'م';
354     final hour = dt.hour > 12 ? dt.hour - 12 : (dt.hour == 0 ? 12 : dt.hour);
355     final minute = dt.minute.toString().padLeft(2, '0');
356     return '${dt.day} ${_getMonthName(dt.month)} ${dt.year} $hour:$minute $amPm';
357 }
358
359 String _getMonthName(int month) {
360     const months = [
361         '', 'يناير', 'فبراير', 'مارس', 'أبريل', 'مايو', 'يونيو',
362         'يوليو', 'أغسطس', 'سبتمبر', 'أكتوبر', 'نوفمبر', 'ديسمبر'
363     ];
364     if (month >= 1 && month <= 12) return months[month];
365     return month.toString();
366 }
367 }
368

```

Lines • 199
Dart

Feature / #78 -

Admin lib/features/admin/contact/presentation/widgets/admin_ticket_info_card.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
6  import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
7  import 'package:moean/core/utils/constants/spacing.dart';
8  import 'package:moean/features/admin/contact/presentation/cubit/admin_contact_cubit.dart';
9
10 class AdminTicketInfoCard extends StatefulWidget {
11     final Map<String, dynamic> ticket;
12
13     const AdminTicketInfoCard({super.key, required this.ticket});
14

```

```

15  @override
16  State<AdminTicketInfoCard> createState() => _AdminTicketInfoCardState();
17  }
18
19  class _AdminTicketInfoCardState extends State<AdminTicketInfoCard> {
20    late String selectedStatus;
21    late TextEditingController notesController;
22
23    @override
24    void initState() {
25      super.initState();
26      selectedStatus = widget.ticket['status'] ?? 'pending';
27      notesController = TextEditingController(text: widget.ticket['admin_notes'] ?? '');
28    }
29
30    @override
31    void dispose() {
32      notesController.dispose();
33      super.dispose();
34    }
35
36    @override
37    Widget build(BuildContext context) {
38      return Column(
39        crossAxisAlignment: CrossAxisAlignment.stretch,
40        children: [
41          _buildManagementCard(context),
42          verticalSpace24,
43          _buildDetailsCard(context),
44        ],
45      );
46    }
47
48    Widget _buildManagementCard(BuildContext context) {
49      return Container(
50        padding: const EdgeInsets.all(24),
51        decoration: BoxDecoration(
52          color: ColorsManager.surfacePrimary,
53          borderRadius: BorderRadius.circular(16),
54          boxShadow: [
55            BoxShadow(
56              color: Colors.black.withValues(alpha: 0.05),
57              blurRadius: 10,
58              offset: const Offset(0, 4),
59            ),
60          ],
61        ),
62        child: Column(
63          crossAxisAlignment: CrossAxisAlignment.start,
64          children: [
65            Row(
66              mainAxisAlignment: MainAxisAlignment.spaceBetween,
67              children: [
68                Text(
69                  appTranslation().get('ticket_management'),
70                  style: TextStyleManager.bold16.copyWith(color: ColorsManager.primaryColor),
71                ),
72                Icon(Icons.filter_alt_outlined, color: ColorsManager.primaryColor),
73              ],
74            ),
75            verticalSpace16,
76            Text(
77              appTranslation().get('status'),
78              style: TextStyleManager.regular12.copyWith(color: ColorsManager.mainText),
79            ),
80            verticalSpace8,
81            Container(
82              padding: const EdgeInsets.symmetric(horizontal: 16),
83              decoration: BoxDecoration(
84                border: Border.all(color: ColorsManager.borderColor),
85                borderRadius: BorderRadius.circular(8),
86              ),
87              child: DropdownButtonHideUnderline(
88                child: DropdownButton<String>(
89                  value: selectedStatus,

```

```

90         isExpanded: true,
91         items: [
92             DropdownMenuItem(value: 'pending', child: Text(appTranslation().get('pending'))),
93             DropdownMenuItem(value: 'in_progress', child: Text(appTranslation().get('in_progress'))),
94             DropdownMenuItem(value: 'resolved', child: Text(appTranslation().get('resolved'))),
95             DropdownMenuItem(value: 'closed', child: Text(appTranslation().get('closed'))),
96         ],
97         onChanged: (value) {
98             if (value != null) {
99                 setState(() {
100                     selectedStatus = value;
101                 });
102             }
103         },
104     ),
105 ),
106 ),
107 verticalSpace16,
108 Row(
109     children: [
110         Icon(Icons.note_alt_outlined, size: 16, color: ColorsManager.secondaryText),
111         horizontalSpace8,
112         Text(
113             appTranslation().get('internal_notes'),
114             style: TextStyleManager.regular12.copyWith(color: ColorsManager.mainText),
115         ),
116     ],
117 ),
118 verticalSpace8,
119 PrimaryTextField(
120     controller: notesController,
121     hint: appTranslation().get('add_internal_notes'),
122     maxLines: 4,
123 ),
124 verticalSpace24,
125 PrimaryElevatedButton(
126     text: appTranslation().get('save_changes'),
127     icon: const Icon(Icons.check, color: Colors.white),
128     onPressed: () {
129         AdminContactCubit.get(context).updateTicket(
130             id: widget.ticket['id'],
131             status: selectedStatus,
132             adminNotes: notesController.text,
133         );
134     },
135 ),
136 ],
137 ),
138 );
139 }
140
141 Widget _buildDetailsCard(BuildContext context) {
142     return Container(
143         padding: const EdgeInsets.all(24),
144         decoration: BoxDecoration(
145             color: ColorsManager.surfacePrimary,
146             borderRadius: BorderRadius.circular(16),
147             boxShadow: [
148                 BoxShadow(
149                     color: Colors.black.withValues(alpha: 0.05),
150                     blurRadius: 10,
151                     offset: const Offset(0, 4),
152                 ),
153             ],
154         ),
155         child: Column(
156             crossAxisAlignment: CrossAxisAlignment.start,
157             children: [
158                 Text(
159                     appTranslation().get('ticket_details'),
160                     style: TextStyleManager.bold16.copyWith(color: ColorsManager.primaryColor),
161                 ),
162                 verticalSpace24,
163                 _buildDetailRow(appTranslation().get('id'), '#${widget.ticket['id']}'),
164                 const Divider(),

```

```

165         _buildDetailRow(appTranslation().get('type'), widget.ticket['type_label'] ?? ''),
166         const Divider(),
167         _buildDetailRow(appTranslation().get('replies'), '${widget.ticket['replies_count'] ?? 0} ${appTrans
lation().get('reply')})'),
168         const Divider(),
169         _buildDetailRow(appTranslation().get('unread_admin'), '${widget.ticket['unread_by_admin_count'] ??
0}'),
170         const Divider(),
171         _buildDetailRow(appTranslation().get('unread_client'), '${widget.ticket['unread_by_client_count']
?? 0}'),
172         const Divider(),
173         _buildDetailRow(appTranslation().get('created_at'), _formatDate(widget.ticket['created_at'])),
174     ],
175 ),
176 );
177 }
178
179 Widget _buildDetailRow(String title, String value) {
180     return Padding(
181         padding: const EdgeInsets.symmetric(vertical: 8.0),
182         child: Row(
183             mainAxisAlignment: MainAxisAlignment.spaceBetween,
184             children: [
185                 Text(title, style: TextStyleManager.regular14.copyWith(color: ColorsManager.secondaryText)),
186                 Text(value, style: TextStyleManager.bold14.copyWith(color: ColorsManager.mainText)),
187             ],
188         ),
189     );
190 }
191
192 String _formatDate(String? dateStr) {
193     if (dateStr == null) return '';
194     final dt = DateTime.tryParse(dateStr);
195     if (dt == null) return '';
196     return '${dt.year}-${dt.month.toString().padLeft(2, '0')}-${dt.day.toString().padLeft(2, '0')}';
197 }
198 }
199

```

Lines • 68
Dart

Feature / #79 –
Admin

[lib/features/admin/exams/data/datasources/admin_exam_remote_data_source.dart](#)

```

1 import 'package:moean/core/network/remote/api_endpoints.dart';
2 import 'package:moean/core/network/remote/dio_helper.dart';
3 import 'package:moean/features/admin/exams/data/models/admin_exam_models.dart';
4
5 abstract class AdminExamRemoteDataSource {
6     Future<AdminQuestionPaginationModel> getPendingQuestions({
7         int page = 1,
8         String? type,
9         String? difficulty,
10        int? lessonId,
11    });
12
13    Future<AdminQuestionModel> reviewQuestion({
14        required int questionId,
15        required String decision,
16        String? rejectionReason,
17    });
18 }
19
20 class AdminExamRemoteDataSourceImpl implements AdminExamRemoteDataSource {
21     AdminExamRemoteDataSourceImpl();
22
23     @override
24     Future<AdminQuestionPaginationModel> getPendingQuestions({
25         int page = 1,
26         String? type,
27         String? difficulty,
28         int? lessonId,
29     }) async {
30         final queryParameters = <String, dynamic>{
31             'page': page,

```



```

32     };
33     if (type != null) queryParameters['type'] = type;
34     if (difficulty != null) queryParameters['difficulty'] = difficulty;
35     if (lessonId != null) queryParameters['lesson_id'] = lessonId;
36
37     final response = await DioHelper.getData(
38       url: adminPendingQuestionsApi,
39       query: queryParameters,
40     );
41
42     return response.fold(
43       (error) => throw Exception(error),
44       (data) => AdminQuestionPaginationModel.fromJson(data.data),
45     );
46   }
47
48   @override
49   Future<AdminQuestionModel> reviewQuestion({
50     required int questionId,
51     required String decision,
52     String? rejectionReason,
53   }) async {
54     final response = await DioHelper.patchData(
55       url: adminReviewQuestionApi(questionId),
56       data: {
57         'decision': decision,
58         if (rejectionReason != null) 'rejection_reason': rejectionReason,
59       },
60     );
61
62     return response.fold(
63       (error) => throw Exception(error),
64       (data) => AdminQuestionModel.fromJson(data.data['question']),
65     );
66   }
67 }
68

```

Lines • Dart 90

Feature / Admin

#80 – lib/features/admin/exams/data/models/admin_exam_models.dart

```

1  import 'package:moean/features/admin/exams/domain/entities/admin_exam_entities.dart';
2  import 'package:moean/features/exam_generation/data/models/exam_models.dart';
3
4  class CreatorModel extends CreatorEntity {
5    CreatorModel({
6      required super.teacherId,
7      required super.name,
8      required super.email,
9    });
10
11    factory CreatorModel.fromJson(Map<String, dynamic> json) {
12      return CreatorModel(
13        teacherId: json['teacher_id'] as int? ?? 0,
14        name: json['name'] as String? ?? '',
15        email: json['email'] as String? ?? '',
16      );
17    }
18  }
19
20  class AdminQuestionModel extends AdminQuestionEntity {
21    AdminQuestionModel({
22      required super.id,
23      required super.lessonId,
24      required super.type,
25      required super.questionText,
26      required super.options,
27      required super.correctAnswer,
28      required super.source,
29      required super.usageCount,
30      required super.questionOrder,
31      required super.points,
32      required super.reviewStatus,
33      required super.difficulty,

```

```

34     super.creator,
35   });
36
37   factory AdminQuestionModel.fromJson(Map<String, dynamic> json) {
38     final type = json['type'] as String? ?? '';
39     dynamic parsedOptions;
40
41     if (type == 'matching') {
42       parsedOptions = MatchingOptionsModel.fromJson(json['options'] as Map<String, dynamic>? ?? {});
43     } else if (type == 'mcq') {
44       parsedOptions = (json['options'] as List<dynamic>?)?.map((e) => e.toString()).toList() ?? [];
45     } else {
46       parsedOptions = json['options'];
47     }
48
49     return AdminQuestionModel(
50       id: json['id'] as int? ?? 0,
51       lessonId: json['lesson_id'] as int? ?? 0,
52       type: type,
53       questionText: json['question_text'] as String? ?? '',
54       options: parsedOptions,
55       correctAnswer: json['correct_answer'] as String? ?? '',
56       source: json['source'] as String? ?? '',
57       usageCount: json['usage_count'] as int? ?? 0,
58       questionOrder: json['question_order'] as int? ?? 0,
59       points: (json['points'] as num?)?.toInt() ?? 0,
60       reviewStatus: json['review_status'] as String? ?? '',
61       difficulty: json['difficulty'] as String? ?? '',
62       creator: json['creator'] != null ? CreatorModel.fromJson(json['creator'] as Map<String, dynamic>) : nul
1,
63     );
64   }
65 }
66
67 class AdminQuestionPaginationModel extends AdminQuestionPaginationEntity {
68   AdminQuestionPaginationModel({
69     required super.currentPage,
70     required super.lastPage,
71     required super.perPage,
72     required super.total,
73     required super.data,
74   });
75
76   factory AdminQuestionPaginationModel.fromJson(Map<String, dynamic> json) {
77     final meta = json['meta'] as Map<String, dynamic>? ?? {};
78     return AdminQuestionPaginationModel(
79       currentPage: meta['current_page'] as int? ?? 1,
80       lastPage: meta['last_page'] as int? ?? 1,
81       perPage: meta['per_page'] as int? ?? 20,
82       total: meta['total'] as int? ?? 0,
83       data: (json['data'] as List<dynamic>?)
84         ?.map((e) => AdminQuestionModel.fromJson(e as Map<String, dynamic>))
85         .toList() ??
86       [],
87     );
88   }
89 }
90

```

Lines • Dart 57

Feature / Admin

#81 – lib/features/admin/exams/data/repositories/admin_exam_repository_impl.dart

```

1  import 'package:dartz/dartz.dart';
2  import 'package:dio/dio.dart';
3  import 'package:moean/core/errors/failures.dart';
4  import 'package:moean/features/admin/exams/data/datasources/admin_exam_remote_data_source.dart';
5  import 'package:moean/features/admin/exams/domain/entities/admin_exam_entities.dart';
6  import 'package:moean/features/admin/exams/domain/repositories/admin_exam_repository.dart';
7
8  class AdminExamRepositoryImpl implements AdminExamRepository {
9     final AdminExamRemoteDataSource remoteDataSource;
10
11     AdminExamRepositoryImpl(this.remoteDataSource);
12

```

```

13 @override
14 Future<Either<Failure, AdminQuestionPaginationEntity>> getPendingQuestions({
15   int page = 1,
16   String? type,
17   String? difficulty,
18   int? lessonId,
19 }) async {
20   try {
21     final result = await remoteDataSource.getPendingQuestions(
22       page: page,
23       type: type,
24       difficulty: difficulty,
25       lessonId: lessonId,
26     );
27     return Right(result);
28   } catch (e) {
29     if (e is DioException) {
30       return Left(ServerFailure(e.message ?? e.toString()));
31     }
32     return Left(ServerFailure(e.toString()));
33   }
34 }
35
36 @override
37 Future<Either<Failure, AdminQuestionEntity>> reviewQuestion({
38   required int questionId,
39   required String decision,
40   String? rejectionReason,
41 }) async {
42   try {
43     final result = await remoteDataSource.reviewQuestion(
44       questionId: questionId,
45       decision: decision,
46       rejectionReason: rejectionReason,
47     );
48     return Right(result);
49   } catch (e) {
50     if (e is DioException) {
51       return Left(ServerFailure(e.message ?? e.toString()));
52     }
53     return Left(ServerFailure(e.toString()));
54   }
55 }
56 }
57

```

Lines • Dart 52

Feature / Admin

#82 – lib/features/admin/exams/domain/entities/admin_exam_entities.dart

```

1 import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
2
3 class CreatorEntity {
4   final int teacherId;
5   final String name;
6   final String email;
7
8   CreatorEntity({
9     required this.teacherId,
10    required this.name,
11    required this.email,
12  });
13 }
14
15 class AdminQuestionEntity extends QuestionEntity {
16   final CreatorEntity? creator;
17   final String reviewStatus;
18   final String difficulty;
19
20   AdminQuestionEntity({
21     required super.id,
22     required super.lessonId,
23     required super.type,
24     required super.questionText,
25     required super.options,

```

```

26     required super.correctAnswer,
27     required super.source,
28     required super.usageCount,
29     required super.questionOrder,
30     required super.points,
31     required this.reviewStatus,
32     required this.difficulty,
33     this.creator,
34   });
35 }
36
37 class AdminQuestionPaginationEntity {
38   final int currentPage;
39   final int lastPage;
40   final int perPage;
41   final int total;
42   final List<AdminQuestionEntity> data;
43
44   AdminQuestionPaginationEntity({
45     required this.currentPage,
46     required this.lastPage,
47     required this.perPage,
48     required this.total,
49     required this.data,
50   });
51 }
52

```

Lines • Dart 19

Feature / Admin

#83 – lib/features/admin/exams/domain/repositories/admin_exam_repository.dart

```

1  import 'package:dartz/dartz.dart';
2  import 'package:moean/core/errors/failures.dart';
3  import 'package:moean/features/admin/exams/domain/entities/admin_exam_entities.dart';
4
5  abstract class AdminExamRepository {
6    Future<Either<Failure, AdminQuestionPaginationEntity>> getPendingQuestions({
7      int page = 1,
8      String? type,
9      String? difficulty,
10     int? lessonId,
11   });
12
13   Future<Either<Failure, AdminQuestionEntity>> reviewQuestion({
14     required int questionId,
15     required String decision,
16     String? rejectionReason,
17   });
18 }
19

```

Lines • Dart 25

Feature / Admin

#84 – lib/features/admin/exams/domain/usecases/get_pending_questions_usecase.dart

```

1  import 'package:dartz/dartz.dart';
2  import 'package:moean/core/errors/failures.dart';
3  import 'package:moean/features/admin/exams/domain/entities/admin_exam_entities.dart';
4  import 'package:moean/features/admin/exams/domain/repositories/admin_exam_repository.dart';
5
6  class GetPendingQuestionsUseCase {
7    final AdminExamRepository repository;
8
9    GetPendingQuestionsUseCase(this.repository);
10
11   Future<Either<Failure, AdminQuestionPaginationEntity>> call({
12     int page = 1,
13     String? type,
14     String? difficulty,
15     int? lessonId,
16   }) {
17     return repository.getPendingQuestions(
18       page: page,
19       type: type,
20       difficulty: difficulty,

```

```

21     lessonId: lessonId,
22   );
23 }
24 }
25

```

Lines • Dart 23 Feature / Admin

#85 – lib/features/admin/exams/domain/usecases/review_question_usecase.dart

```

1  import 'package:dartz/dartz.dart';
2  import 'package:moean/core/errors/failures.dart';
3  import 'package:moean/features/admin/exams/domain/entities/admin_exam_entities.dart';
4  import 'package:moean/features/admin/exams/domain/repositories/admin_exam_repository.dart';
5
6  class ReviewQuestionUseCase {
7    final AdminExamRepository repository;
8
9    ReviewQuestionUseCase(this.repository);
10
11    Future<Either<Failure, AdminQuestionEntity>> call({
12      required int questionId,
13      required String decision,
14      String? rejectionReason,
15    }) {
16      return repository.reviewQuestion(
17        questionId: questionId,
18        decision: decision,
19        rejectionReason: rejectionReason,
20      );
21    }
22  }
23

```

Lines • Dart 122 Feature / Admin

#86 – lib/features/admin/exams/presentation/cubit/admin_exams_cubit.dart

```

1  import 'package:flutter_bloc/flutter_bloc.dart';
2  import 'package:moean/features/admin/exams/domain/entities/admin_exam_entities.dart';
3  import 'package:moean/features/admin/exams/domain/usecases/get_pending_questions_usecase.dart';
4  import 'package:moean/features/admin/exams/domain/usecases/review_question_usecase.dart';
5  import 'package:moean/features/admin/exams/presentation/cubit/admin_exams_state.dart';
6
7  class AdminExamsCubit extends Cubit<AdminExamsState> {
8    final GetPendingQuestionsUseCase getPendingQuestionsUseCase;
9    final ReviewQuestionUseCase reviewQuestionUseCase;
10
11    AdminExamsCubit({
12      required this.getPendingQuestionsUseCase,
13      required this.reviewQuestionUseCase,
14    }) : super(AdminExamsInitial());
15
16    AdminQuestionPaginationEntity? currentPagination;
17    final List<AdminQuestionEntity> _questions = [];
18
19    Future<void> fetchPendingQuestions({
20      int page = 1,
21      String? type,
22      String? difficulty,
23      int? lessonId,
24    }) async {
25      if (page == 1) {
26        if (!isClosed) emit(AdminExamsLoading());
27        _questions.clear();
28        currentPagination = null;
29      }
30
31      final result = await getPendingQuestionsUseCase(
32        page: page,
33        type: type,
34        difficulty: difficulty,
35        lessonId: lessonId,
36      );
37      if (isClosed) return;
38

```

```

39     result.fold(
40         (failure) {
41             if (page == 1) {
42                 if (!isClosed) emit(AdminExamsError(failure.message));
43             }
44         },
45         (pagination) {
46             currentPagination = pagination;
47             if (page == 1) {
48                 _questions.addAll(pagination.data);
49             } else {
50                 _questions.addAll(pagination.data);
51             }
52
53             // We replace the pagination data with our accumulated list so the UI gets all items
54             final updatedPagination = AdminQuestionPaginationEntity(
55                 currentPage: pagination.currentPage,
56                 lastPage: pagination.lastPage,
57                 perPage: pagination.perPage,
58                 total: pagination.total,
59                 data: List.from(_questions),
60             );
61             currentPagination = updatedPagination;
62             if (!isClosed) emit(AdminExamsLoaded(updatedPagination));
63         },
64     );
65 }
66
67 Future<void> reviewQuestion(int questionId, String decision, [String? rejectionReason]) async {
68     final currentState = state;
69
70     if (!isClosed) emit(AdminExamReviewLoading(questionId));
71
72     final result = await reviewQuestionUseCase(
73         questionId: questionId,
74         decision: decision,
75         rejectionReason: rejectionReason,
76     );
77     if (isClosed) return;
78
79     result.fold(
80         (failure) {
81             if (!isClosed) {
82                 emit(AdminExamReviewError(failure.message));
83                 if (currentState is AdminExamsLoaded) {
84                     emit(currentState);
85                 } else if (currentPagination != null) {
86                     final updatedPagination = AdminQuestionPaginationEntity(
87                         currentPage: currentPagination!.currentPage,
88                         lastPage: currentPagination!.lastPage,
89                         perPage: currentPagination!.perPage,
90                         total: currentPagination!.total,
91                         data: List.from(_questions),
92                     );
93                     currentPagination = updatedPagination;
94                     emit(AdminExamsLoaded(updatedPagination));
95                 }
96             }
97         },
98         (updatedQuestion) {
99             final index = _questions.indexWhere((q) => q.id == questionId);
100             if (index != -1) {
101                 _questions[index] = updatedQuestion;
102             }
103             if (!isClosed) {
104                 emit(AdminExamReviewSuccess(updatedQuestion));
105
106                 if (currentPagination != null) {
107                     final updatedPagination = AdminQuestionPaginationEntity(
108                         currentPage: currentPagination!.currentPage,
109                         lastPage: currentPagination!.lastPage,
110                         perPage: currentPagination!.perPage,
111                         total: currentPagination!.total,
112                         data: List.from(_questions),
113                     );

```

```

114         currentPagination = updatedPagination;
115         emit (AdminExamsLoaded (updatedPagination));
116     }
117 }
118 },
119 );
120 }
121 }
122

```

Lines • Dart 38

Feature / Admin

#87 – lib/features/admin/exams/presentation/cubit/admin_exams_state.dart

```

1  import 'package:moean/features/admin/exams/domain/entities/admin_exam_entities.dart';
2
3  abstract class AdminExamsState {}
4
5  class AdminExamsInitial extends AdminExamsState {}
6
7  class AdminExamsLoading extends AdminExamsState {}
8
9  class AdminExamsLoaded extends AdminExamsState {
10     final AdminQuestionPaginationEntity pagination;
11
12     AdminExamsLoaded(this.pagination);
13 }
14
15 class AdminExamsError extends AdminExamsState {
16     final String message;
17
18     AdminExamsError(this.message);
19 }
20
21 class AdminExamReviewLoading extends AdminExamsState {
22     final int questionId;
23
24     AdminExamReviewLoading(this.questionId);
25 }
26
27 class AdminExamReviewSuccess extends AdminExamsState {
28     final AdminQuestionEntity updatedQuestion;
29
30     AdminExamReviewSuccess(this.updatedQuestion);
31 }
32
33 class AdminExamReviewError extends AdminExamsState {
34     final String message;
35
36     AdminExamReviewError(this.message);
37 }
38

```

Lines • Dart 773

Feature / Admin

#88 – lib/features/admin/exams/presentation/screen/admin_exams_screen.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/routes.dart';
7  import 'package:moean/core/utils/constants/spacing.dart';
8  import 'package:moean/core/utils/extensions/context_extension.dart';
9  import 'package:moean/features/admin/exams/domain/entities/admin_exam_entities.dart';
10 import 'package:moean/features/admin/exams/presentation/cubit/admin_exams_cubit.dart';
11 import 'package:moean/features/admin/exams/presentation/cubit/admin_exams_state.dart';
12 import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
13
14 class AdminExamsScreen extends StatelessWidget {
15     const AdminExamsScreen({super.key});
16
17     @override
18     Widget build(BuildContext context) {
19         return BlocConsumer<AdminExamsCubit, AdminExamsState>(

```

```

20 listener: (context, state) {
21   if (state is AdminExamReviewSuccess) {
22     ScaffoldMessenger.of(context).showSnackBar(
23       SnackBar(
24         content: Text(appTranslation().get('admin_exam_review_success')),
25         backgroundColor: Colors.green,
26       ),
27     );
28   } else if (state is AdminExamReviewError) {
29     ScaffoldMessenger.of(context).showSnackBar(
30       SnackBar(content: Text(state.message), backgroundColor: Colors.red),
31     );
32   } else if (state is AdminExamsError) {
33     ScaffoldMessenger.of(context).showSnackBar(
34       SnackBar(content: Text(state.message), backgroundColor: Colors.red),
35     );
36   }
37 },
38 builder: (context, state) {
39   final cubit = context.read<AdminExamsCubit>();
40   final scrollController = ScrollController();
41
42   scrollController.addListener(() {
43     if (scrollController.position.pixels >=
44       scrollController.position.maxScrollExtent - 200) {
45       if (state is! AdminExamsLoading && cubit.currentPagination != null) {
46         if (cubit.currentPagination!.currentPage < cubit.currentPagination!.lastPage) {
47           cubit.fetchPendingQuestions(page: cubit.currentPagination!.currentPage + 1);
48         }
49       }
50     }
51   });
52
53   List<AdminQuestionEntity> questions = [];
54   if (cubit.currentPagination != null) {
55     questions = cubit.currentPagination!.data;
56   }
57
58   return Scaffold(
59     backgroundColor: ColorsManager.background,
60     appBar: AppBar(
61       backgroundColor: ColorsManager.primaryColor,
62       automaticallyImplyLeading: false,
63       title: Row(
64         children: [
65           const Icon(Icons.verified_outlined, color: Colors.white),
66           horizontalSpace8,
67           Text(
68             appTranslation().get('admin_pending_questions_title'),
69             style: TextStylesManager.bold18.copyWith(color: Colors.white),
70           ),
71         ],
72       ),
73     actions: [
74       IconButton(
75         icon: const Icon(Icons.refresh, color: Colors.white),
76         onPressed: () => cubit.fetchPendingQuestions(),
77       ),
78       Padding(
79         padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
80         child: ElevatedButton.icon(
81           style: ElevatedButton.styleFrom(
82             backgroundColor: Colors.white,
83             foregroundColor: ColorsManager.primaryColor,
84           ),
85           onPressed: () {
86             context.push(Routes.home);
87           },
88           icon: const Icon(Icons.home),
89           label: Text(
90             appTranslation().get('home'),
91             style: TextStylesManager.medium16,
92           ),
93         ),
94       ),

```



```

95     ],
96   ),
97   body: SafeArea(
98     child: Padding(
99       padding: const EdgeInsets.all(16.0),
100     child: Column(
101       children: [
102         Expanded(
103           child: (state is AdminExamsLoading && questions.isEmpty)
104             ? const Center(child: CircularProgressIndicator())
105             : questions.isEmpty
106               ? Center(
107                 child: Text(
108                   appTranslation().get('no_pending_questions'),
109                   style: TextStylesManager.medium16,
110                 ),
111               )
112             : ListView.separated(
113                 controller: scrollController,
114                 itemCount: questions.length + (state is AdminExamsLoading ? 1 : 0),
115                 separatorBuilder: (context, index) => verticalSpace16,
116                 itemBuilder: (context, index) {
117                   if (index >= questions.length) {
118                     return const Center(child: CircularProgressIndicator());
119                   }
120                   final question = questions[index];
121                   return _buildQuestionCard(context, question, state);
122                 },
123               ),
124       ],
125     ),
126   ),
127 ),
128 ),
129 );
130 },
131 );
132 }
133
134 Widget _buildQuestionCard(BuildContext context, AdminQuestionEntity question, AdminExamsState state) {
135   final cubit = context.read<AdminExamsCubit>();
136   final isLoading = state is AdminExamReviewLoading && state.questionId == question.id;
137
138   return Container(
139     padding: const EdgeInsets.all(16),
140     decoration: BoxDecoration(
141       color: Theme.of(context).cardColor,
142       borderRadius: BorderRadius.circular(12),
143       border: Border.all(color: Colors.grey.withValues(alpha: 0.2)),
144       boxShadow: [
145         BoxShadow(
146           color: Colors.black.withValues(alpha: 0.05),
147           blurRadius: 4,
148           offset: const Offset(0, 2),
149         ),
150       ],
151     ),
152     child: Column(
153       crossAxisAlignment: CrossAxisAlignment.start,
154       children: [
155         Row(
156           mainAxisAlignment: MainAxisAlignment.spaceBetween,
157           children: [
158             Chip(
159               label: Text(
160                 _getQuestionTypeName(question.type),
161                 style: TextStylesManager.medium12.copyWith(color: Colors.white),
162               ),
163               backgroundColor: ColorsManager.primaryColor,
164             ),
165             if (question.reviewStatus == 'pending')
166               Container(
167                 padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
168                 decoration: BoxDecoration(
169                   color: Colors.orange.withValues(alpha: 0.1),

```

```

170         borderRadius: BorderRadius.circular(8),
171         border: Border.all(color: Colors.orange),
172     ),
173     child: Text(
174         appTranslation().get('pending'),
175         style: TextStyleManager.medium12.copyWith(color: Colors.orange),
176     ),
177 )
178 else if (question.reviewStatus == 'approved')
179     Container(
180         padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
181         decoration: BoxDecoration(
182             color: Colors.green.withValues(alpha: 0.1),
183             borderRadius: BorderRadius.circular(8),
184             border: Border.all(color: Colors.green),
185         ),
186         child: Text(
187             appTranslation().get('approved'),
188             style: TextStyleManager.medium12.copyWith(color: Colors.green),
189         ),
190     )
191 else if (question.reviewStatus == 'rejected')
192     Container(
193         padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
194         decoration: BoxDecoration(
195             color: Colors.red.withValues(alpha: 0.1),
196             borderRadius: BorderRadius.circular(8),
197             border: Border.all(color: Colors.red),
198         ),
199         child: Text(
200             appTranslation().get('rejected'),
201             style: TextStyleManager.medium12.copyWith(color: Colors.red),
202         ),
203     ),
204 ],
205 ),
206 verticalSpace12,
207 Text(
208     question.questionText,
209     style: TextStyleManager.bold16,
210 ),
211 verticalSpace12,
212 _buildOptionsView(question),
213 verticalSpace12,
214 Container(
215     padding: const EdgeInsets.all(12),
216     decoration: BoxDecoration(
217         color: Colors.green.withValues(alpha: 0.05),
218         borderRadius: BorderRadius.circular(8),
219         border: Border.all(color: Colors.green.withValues(alpha: 0.3)),
220     ),
221     child: Row(
222         children: [
223             const Icon(Icons.check_circle_outline, color: Colors.green, size: 20),
224             horizontalSpace8,
225             Expanded(
226                 child: Text(
227                     "الإجابة الصحيحة: ${question.correctAnswer}",
228                     style: TextStyleManager.medium14.copyWith(color: Colors.green),
229                 ),
230             ),
231         ],
232     ),
233 ),
234 verticalSpace12,
235 if (question.creator != null)
236     Row(
237         children: [
238             const Icon(Icons.person_outline, size: 16, color: Colors.grey),
239             horizontalSpace4,
240             Text(
241                 question.creator!.name,
242                 style: TextStyleManager.regular14.copyWith(color: Colors.grey),
243             ),
244             const Spacer(),

```

```

245         Text(
246             "المصوبة: ${_getDifficultyName(question.difficulty)}",
247             style: TextStylesManager.regular14.copyWith(color: Colors.grey),
248         ),
249     ],
250 ),
251 if (question.reviewStatus == 'pending') ...[
252     verticalSpace16,
253     const Divider(),
254     verticalSpace8,
255     Row(
256         mainAxisAlignment: MainAxisAlignment.end,
257         children: [
258             if (isLoading)
259                 const SizedBox(
260                     width: 24,
261                     height: 24,
262                     child: CircularProgressIndicator(strokeWidth: 2),
263                 )
264             else ...[
265                 OutlinedButton.icon(
266                     onPressed: () => _showRejectDialog(context, question, cubit),
267                     icon: const Icon(Icons.close, color: Colors.red),
268                     label: Text(
269                         appTranslation().get('reject'),
270                         style: TextStylesManager.medium14.copyWith(color: Colors.red),
271                     ),
272                     style: OutlinedButton.styleFrom(
273                         side: const BorderSide(color: Colors.red),
274                     ),
275                 ),
276                 horizontalSpace12,
277                 ElevatedButton.icon(
278                     onPressed: () => _showApproveDialog(context, question, cubit),
279                     icon: const Icon(Icons.check, color: Colors.white),
280                     label: Text(
281                         appTranslation().get('approve'),
282                         style: TextStylesManager.medium14.copyWith(color: Colors.white),
283                     ),
284                     style: ElevatedButton.styleFrom(
285                         backgroundColor: Colors.green,
286                     ),
287                 ),
288             ],
289         ],
290     ),
291 ],
292 ],
293 ),
294 );
295 }
296
297 Widget _buildOptionsView(AdminQuestionEntity question) {
298     if (question.type == 'mcq' && question.options is List) {
299         final options = question.options as List;
300         return Column(
301             crossAxisAlignment: CrossAxisAlignment.start,
302             children: options.map((option) {
303                 return Padding(
304                     padding: const EdgeInsets.only(bottom: 8.0),
305                     child: Row(
306                         children: [
307                             const Icon(Icons.radio_button_unchecked, size: 16, color: Colors.grey),
308                             horizontalSpace8,
309                             Expanded(
310                                 child: Text(
311                                     option.toString(),
312                                     style: TextStylesManager.regular14,
313                                 ),
314                             ),
315                         ],
316                     ),
317                 );
318             }).toList(),
319         );

```

```

320     } else if (question.type == 'matching' && question.options != null) {
321         try {
322             final columnA = (question.options.columnA as List);
323             final columnB = (question.options.columnB as List);
324             return Row(
325                 crossAxisAlignment: CrossAxisAlignment.start,
326                 children: [
327                     Expanded(
328                         child: Column(
329                             crossAxisAlignment: CrossAxisAlignment.start,
330                             children: columnA.map((e) => Text("• $e")).toList(),
331                         ),
332                     ),
333                     horizontalSpace16,
334                     Expanded(
335                         child: Column(
336                             crossAxisAlignment: CrossAxisAlignment.start,
337                             children: columnB.map((e) => Text("• $e")).toList(),
338                         ),
339                     ),
340                 ],
341             );
342         } catch (e) {
343             return const SizedBox();
344         }
345     }
346     return const SizedBox();
347 }
348
349 String _getQuestionTypeName(String type) {
350     switch (type.toLowerCase()) {
351         case 'mcq': return 'اختيار من متعدد';
352         case 'matching': return 'مزاوجة';
353         case 'true_false': return 'صح وخطأ';
354         case 'fill_blank':
355         case 'fill_in_the_blank': return 'أكمل الفراغ';
356         default: return type;
357     }
358 }
359
360 String _getDifficultyName(String difficulty) {
361     switch (difficulty.toLowerCase()) {
362         case 'easy': return 'سهل';
363         case 'medium': return 'متوسط';
364         case 'hard': return 'صعب';
365         default: return difficulty;
366     }
367 }
368
369 void _showApproveDialog(BuildContext context, AdminQuestionEntity question, AdminExamsCubit cubit) {
370     showDialog(
371         context: context,
372         builder: (dialogContext) {
373             final theme = Theme.of(dialogContext);
374             final isDark = theme.brightness == Brightness.dark;
375             final dialogBg = theme.dialogTheme.backgroundColor ?? theme.colorScheme.surface;
376             final textColor = theme.colorScheme.onSurface;
377             final chipBg = isDark ? const Color(0xFF2A3A4A) : const Color(0xFFEDF2F7);
378             final chipTextColor = isDark ? const Color(0xFF90CAF9) : const Color(0xFFD3557);
379             final typeChipBg = isDark ? const Color(0xFF1A3A30) : const Color(0xFFE6F4F1);
380             final answerBg = isDark ? const Color(0xFF3A2E1A) : const Color(0xFFFFF7E6);
381             final answerTextColor = isDark ? const Color(0xFFFFFCC80) : const Color(0xFFB57D2C);
382             final infoBg = isDark ? const Color(0xFF1A3A30) : const Color(0xFFE8F5F2);
383
384             return Dialog(
385                 backgroundColor: dialogBg,
386                 shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
387                 child: Column(
388                     mainAxisAlignment: MainAxisAlignment.min,
389                     children: [
390                         Container(
391                             padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
392                             decoration: const BoxDecoration(
393                                 color: Color(0xFF1E9B75),
394                                 borderRadius: BorderRadius.vertical(top: Radius.circular(16)),

```

```

395     ),
396     child: Row(
397       mainAxisAlignment: MainAxisAlignment.spaceBetween,
398       children: [
399         IconButton(
400           icon: const Icon(Icons.close, color: Colors.white),
401           onPressed: () => Navigator.pop(dialogContext),
402         ),
403         Column(
404           crossAxisAlignment: CrossAxisAlignment.end,
405           children: [
406             Text(
407               'تأكيد الإجراء',
408               style: TextStylesManager.regular12.copyWith(color: Colors.white),
409             ),
410             Text(
411               'اعتماد السؤال',
412               style: TextStylesManager.bold18.copyWith(color: Colors.white),
413             ),
414           ],
415         ),
416       ],
417     ),
418   ),
419   Padding(
420     padding: const EdgeInsets.all(16.0),
421     child: Column(
422       children: [
423         Container(
424           padding: const EdgeInsets.all(12),
425           decoration: BoxDecoration(
426             border: Border.all(color: Colors.grey.withValues(alpha: 0.2)),
427             borderRadius: BorderRadius.circular(12),
428           ),
429           child: Column(
430             crossAxisAlignment: CrossAxisAlignment.end,
431             children: [
432               Row(
433                 mainAxisAlignment: MainAxisAlignment.end,
434                 children: [
435                   Chip(
436                     label: Text(
437                       _getDifficultyName(question.difficulty),
438                       style: TextStylesManager.medium12.copyWith(color: chipTextColor),
439                     ),
440                     backgroundColor: chipBg,
441                     side: BorderSide.none,
442                   ),
443                   horizontalSpace8,
444                   Chip(
445                     label: Text(
446                       _getQuestionTypeName(question.type),
447                       style: TextStylesManager.medium12.copyWith(color: const Color(0xFF1E9B75)),
448                     ),
449                     backgroundColor: typeChipBg,
450                     side: BorderSide.none,
451                   ),
452                 ],
453               ),
454               verticalSpace12,
455               Text(
456                 question.questionText,
457                 style: TextStylesManager.bold16.copyWith(color: textColor),
458                 textAlign: TextAlign.right,
459               ),
460               verticalSpace12,
461               Container(
462                 width: double.infinity,
463                 padding: const EdgeInsets.all(12),
464                 decoration: BoxDecoration(
465                   color: answerBg,
466                   borderRadius: BorderRadius.circular(8),
467                 ),
468                 child: Text(
469                   'الإجابة الصحيحة: ${question.correctAnswer}',

```

```

470         style: TextStyleManager.bold14.copyWith(color: answerTextColor),
471         textAlign: TextAlign.center,
472     ),
473 ),
474 ],
475 ),
476 ),
477 verticalSpace16,
478 Container(
479     width: double.infinity,
480     padding: const EdgeInsets.all(16),
481     decoration: BoxDecoration(
482         color: infoBg,
483         borderRadius: BorderRadius.circular(12),
484     ),
485     child: Text(
486         'بعد الاعتماد، سيظهر هذا السؤال ضمن الأسئلة الجاهزة التي يمكن للمعلمين اختيارها',
487         style: TextStyleManager.bold14.copyWith(color: const Color(0xFF1E9B75)),
488         textAlign: TextAlign.center,
489     ),
490 ),
491 verticalSpace24,
492 Row(
493     mainAxisAlignment: MainAxisAlignment.spaceBetween,
494     children: [
495         Expanded(
496             child: OutlinedButton(
497                 onPressed: () => Navigator.pop(dialogContext),
498                 style: OutlinedButton.styleFrom(
499                     side: BorderSide(color: Colors.grey.withValues(alpha: 0.3)),
500                     shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
501                     padding: const EdgeInsets.symmetric(vertical: 14),
502                 ),
503                 child: Text(
504                     'إلغاء',
505                     style: TextStyleManager.bold14.copyWith(color: Colors.grey),
506                 ),
507             ),
508         ),
509         horizontalSpace12,
510         Expanded(
511             flex: 2,
512             child: ElevatedButton.icon(
513                 onPressed: () {
514                     Navigator.pop(dialogContext);
515                     cubit.reviewQuestion(question.id, 'approved');
516                 },
517                 icon: const Icon(Icons.check_circle_outline, color: Colors.white),
518                 label: Text(
519                     'نعم، اعتمد السؤال',
520                     style: TextStyleManager.bold14.copyWith(color: Colors.white),
521                 ),
522                 style: ElevatedButton.styleFrom(
523                     backgroundColor: const Color(0xFF1E9B75),
524                     shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
525                     padding: const EdgeInsets.symmetric(vertical: 14),
526                 ),
527             ),
528         ),
529     ],
530 ),
531 ],
532 ),
533 ),
534 ],
535 ),
536 );
537 },
538 );
539 }
540
541 void _showRejectDialog(BuildContext context, AdminQuestionEntity question, AdminExamsCubit cubit) {
542     showDialog(
543         context: context,
544         builder: (dialogContext) {

```

```

545     return _RejectQuestionDialog(
546         question: question,
547         onReject: (reason) {
548             cubit.reviewQuestion(
549                 question.id,
550                 'rejected',
551                 reason.isNotEmpty ? reason : null,
552             );
553         },
554         getQuestionTypeName: _getQuestionTypeName,
555         getDifficultyName: _getDifficultyName,
556     );
557 },
558 );
559 }
560 }
561
562 class _RejectQuestionDialog extends StatefulWidget {
563     final AdminQuestionEntity question;
564     final Function(String) onReject;
565     final String Function(String) getQuestionTypeName;
566     final String Function(String) getDifficultyName;
567
568     const _RejectQuestionDialog({
569         required this.question,
570         required this.onReject,
571         required this.getQuestionTypeName,
572         required this.getDifficultyName,
573     });
574
575     @override
576     State<_RejectQuestionDialog> createState() => _RejectQuestionDialogState();
577 }
578
579 class _RejectQuestionDialogState extends State<_RejectQuestionDialog> {
580     late final TextEditingController _reasonController;
581
582     @override
583     void initState() {
584         super.initState();
585         _reasonController = TextEditingController();
586     }
587
588     @override
589     void dispose() {
590         _reasonController.dispose();
591         super.dispose();
592     }
593
594     @override
595     Widget build(BuildContext context) {
596         final theme = Theme.of(context);
597         final isDark = theme.brightness == Brightness.dark;
598         final dialogBg = theme.dialogTheme.backgroundColor ?? theme.colorScheme.surface;
599         final textColor = theme.colorScheme.onSurface;
600         final chipBg = isDark ? const Color(0xFF2A3A4A) : const Color(0xFFEDF2F7);
601         final chipTextColor = isDark ? const Color(0xFF90CAF9) : const Color(0xFF1D3557);
602         final typeChipBg = isDark ? const Color(0xFF1A3A30) : const Color(0xFFE6F4F1);
603         final answerBg = isDark ? const Color(0xFF3A2E1A) : const Color(0xFFFFF7E6);
604         final answerTextColor = isDark ? const Color(0xFFFFCC80) : const Color(0xFFB57D2C);
605         final rejectInfoBg = isDark ? const Color(0xFF3A1A1A) : const Color(0xFFFFEBEE);
606
607         return Dialog(
608             backgroundColor: dialogBg,
609             shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
610             child: SingleChildScrollView(
611                 child: Column(
612                     mainAxisAlignment: MainAxisAlignment.min,
613                     children: [
614                         Container(
615                             padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
616                             decoration: const BoxDecoration(
617                                 color: Color(0xFFD32F2F),
618                                 borderRadius: BorderRadius.vertical(top: Radius.circular(16)),
619                             ),

```

```

620 child: Row(
621   mainAxisAlignment: MainAxisAlignment.spaceBetween,
622   children: [
623     IconButton(
624       icon: const Icon(Icons.close, color: Colors.white),
625       onPressed: () => Navigator.pop(context),
626     ),
627     Column(
628       crossAxisAlignment: CrossAxisAlignment.end,
629       children: [
630         Text(
631           'تأكيد الإجراء',
632           style: TextStyleManager.regular12.copyWith(color: Colors.white),
633         ),
634         Text(
635           'رفض السؤال',
636           style: TextStyleManager.bold18.copyWith(color: Colors.white),
637         ),
638       ],
639     ),
640   ],
641 ),
642 ),
643 Padding(
644   padding: const EdgeInsets.all(16.0),
645   child: Column(
646     children: [
647       Container(
648         padding: const EdgeInsets.all(12),
649         decoration: BoxDecoration(
650           border: Border.all(color: Colors.grey.withValues(alpha: 0.2)),
651           borderRadius: BorderRadius.circular(12),
652         ),
653       child: Column(
654         crossAxisAlignment: CrossAxisAlignment.end,
655         children: [
656           Row(
657             mainAxisAlignment: MainAxisAlignment.end,
658             children: [
659               Chip(
660                 label: Text(
661                   widget.getDifficultyName(widget.question.difficulty),
662                   style: TextStyleManager.medium12.copyWith(color: chipTextColor),
663                 ),
664                 backgroundColor: chipBg,
665                 side: BorderSide.none,
666               ),
667               horizontalSpace8,
668               Chip(
669                 label: Text(
670                   widget.getQuestionTypeName(widget.question.type),
671                   style: TextStyleManager.medium12.copyWith(color: const Color(0xFF1E9B75)),
672                 ),
673                 backgroundColor: typeChipBg,
674                 side: BorderSide.none,
675               ),
676             ],
677           ),
678           verticalSpace12,
679           Text(
680             widget.question.questionText,
681             style: TextStyleManager.bold16.copyWith(color: textColor),
682             textAlign: TextAlign.right,
683           ),
684           verticalSpace12,
685           Container(
686             width: double.infinity,
687             padding: const EdgeInsets.all(12),
688             decoration: BoxDecoration(
689               color: answerBg,
690               borderRadius: BorderRadius.circular(8),
691             ),
692             child: Text(
693               'الإجابة الصحيحة: ${widget.question.correctAnswer}',
694               style: TextStyleManager.bold14.copyWith(color: answerTextColor),

```



```

695         textAlign: TextAlign.center,
696     ),
697 ),
698 ],
699 ),
700 ),
701 verticalSpace16,
702 Container(
703     width: double.infinity,
704     padding: const EdgeInsets.all(16),
705     decoration: BoxDecoration(
706         color: rejectInfoBg,
707         borderRadius: BorderRadius.circular(12),
708     ),
709     child: Text(
710         'بعد الرفض، لن يظهر هذا السؤال ضمن الأسئلة الجاهزة، وسيظل محفوظاً في اختبار المعلم فقط',
711         style: TextStyleManager.bold14.copyWith(color: const Color(0xFFD32F2F)),
712         textAlign: TextAlign.center,
713     ),
714 ),
715 verticalSpace16,
716 PrimaryTextField(
717     controller: _reasonController,
718     hint: 'سبب الرفض (اختياري)',
719     maxLines: 3,
720     keyboardType: TextInputType.multiline,
721     textInputAction: TextInputAction.newline,
722 ),
723 verticalSpace24,
724 Row(
725     mainAxisAlignment: MainAxisAlignment.spaceBetween,
726     children: [
727         Expanded(
728             child: OutlinedButton(
729                 onPressed: () => Navigator.pop(context),
730                 style: OutlinedButton.styleFrom(
731                     side: BorderSide(color: Colors.grey.withValues(alpha: 0.3)),
732                     shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
733                     padding: const EdgeInsets.symmetric(vertical: 14),
734                 ),
735                 child: Text(
736                     'إلغاء',
737                     style: TextStyleManager.bold14.copyWith(color: Colors.grey),
738                 ),
739             ),
740         ),
741         horizontalSpace12,
742         Expanded(
743             flex: 2,
744             child: ElevatedButton.icon(
745                 onPressed: () {
746                     final reason = _reasonController.text.trim();
747                     Navigator.pop(context);
748                     widget.onReject(reason);
749                 },
750                 icon: const Icon(Icons.cancel_outlined, color: Colors.white),
751                 label: Text(
752                     'نعم، ارفض السؤال',
753                     style: TextStyleManager.bold14.copyWith(color: Colors.white),
754                 ),
755                 style: ElevatedButton.styleFrom(
756                     backgroundColor: const Color(0xFFD32F2F),
757                     shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
758                     padding: const EdgeInsets.symmetric(vertical: 14),
759                 ),
760             ),
761         ),
762     ],
763 ),
764 ],
765 ),
766 ),
767 ],
768 ),
769 ),

```

```
770 );
771 }
772 }
773
```

Lines • 82
Dart

Feature / #89 -

Admin lib/features/admin/payments/data/repositories/admin_payments_repository.dart

```
1 import 'package:dartz/dartz.dart';
2 import 'package:moean/core/network/remote/api_endpoints.dart';
3 import 'package:moean/core/network/remote/dio_helper.dart';
4 import 'package:moean/features/payment/data/models/payment_model.dart';
5
6 class AdminPaymentsRepository {
7   Future<Either<String, Map<String, dynamic>>> getPayments({
8     String? filter,
9     String? search,
10    int? page,
11  }) async {
12    try {
13      final Map<String, dynamic> queryParameters = {};
14      if (filter != null && filter.isNotEmpty && filter != 'all') {
15        queryParameters['filter'] = filter;
16      }
17      if (search != null && search.isNotEmpty) {
18        queryParameters['search'] = search;
19      }
20      if (page != null) {
21        queryParameters['page'] = page;
22      }
23
24      final responseEither = await DioHelper.getData(
25        url: adminPaymentsApi,
26        query: queryParameters,
27      );
28
29      return responseEither.fold(
30        (error) => Left(error),
31        (response) {
32          final List<dynamic> dataList = response.data['data'];
33          final List<PaymentModel> payments = dataList
34            .map((json) => PaymentModel.fromJson(json as Map<String, dynamic>))
35            .toList();
36
37          final meta = response.data['meta'];
38          final int lastPage = meta != null ? meta['last_page'] as int : 1;
39
40          return Right({
41            'payments': payments,
42            'lastPage': lastPage,
43          });
44        },
45      );
46    } catch (e) {
47      return Left(e.toString());
48    }
49  }
50
51  Future<Either<String, String>> approvePayment(int paymentId) async {
52    try {
53      final responseEither = await DioHelper.postData(
54        url: '$adminPaymentsApi/$paymentId/approve',
55        data: {},
56      );
57      return responseEither.fold(
58        (error) => Left(error),
59        (response) => Right(response.data['message'] ?? 'Payment approved successfully'),
60      );
61    } catch (e) {
62      return Left(e.toString());
63    }
64  }
65}
```

```

66 Future<Either<String, String>> rejectPayment(int paymentId) async {
67   try {
68     final responseEither = await DioHelper.postData(
69       url: '$adminPaymentsApi/$paymentId/reject',
70       data: {},
71     );
72     return responseEither.fold(
73       (error) => Left(error),
74       (response) => Right(response.data['message'] ?? 'Payment rejected successfully'),
75     );
76   } catch (e) {
77     return Left(e.toString());
78   }
79 }
80 }
81
82

```

Lines • Dart 145

Feature / Admin

#90 — lib/features/admin/payments/presentation/cubit/admin_payments_cubit.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/features/admin/payments/data/repositories/admin_payments_repository.dart';
4  import 'package:moean/features/admin/payments/presentation/cubit/admin_payments_state.dart';
5  import 'package:moean/features/payment/data/models/payment_model.dart';
6
7  class AdminPaymentsCubit extends Cubit<AdminPaymentsState> {
8    final AdminPaymentsRepository repository;
9
10   AdminPaymentsCubit(this.repository) : super(AdminPaymentsInitial());
11
12   static AdminPaymentsCubit get(BuildContext context) => BlocProvider.of<AdminPaymentsCubit>(context);
13
14   List<PaymentModel> paymentsList = [];
15   int currentPage = 1;
16   int lastPage = 1;
17   String currentFilter = 'all';
18   String currentSearch = '';
19   bool hasMoreData = true;
20
21   Future<void> getPayments({
22     bool loadMore = false,
23     String? filter,
24     String? search,
25   }) async {
26     if (loadMore) {
27       if (!hasMoreData) return;
28       currentPage++;
29     } else {
30       currentPage = 1;
31       paymentsList.clear();
32       hasMoreData = true;
33       if (filter != null) currentFilter = filter;
34       if (search != null) currentSearch = search;
35     }
36
37     if (!isClosed) emit(GetAdminPaymentsLoadingState());
38
39     final result = await repository.getPayments(
40       page: currentPage,
41       filter: currentFilter,
42       search: currentSearch,
43     );
44     if (isClosed) return;
45
46     result.fold(
47       (failureMessage) {
48         if (!isClosed) emit(GetAdminPaymentsErrorState(failureMessage));
49       },
50       (data) {
51         final List<PaymentModel> newPayments = data['payments'];
52         lastPage = data['lastPage'];
53

```

```

54         if (newPayments.isEmpty || currentPage >= lastPage) {
55             hasMoreData = false;
56         }
57
58         if (loadMore) {
59             paymentsList.addAll(newPayments);
60         } else {
61             paymentsList = newPayments;
62         }
63
64         if (!isClosed) emit(GetAdminPaymentsSuccessState());
65     },
66 );
67 }
68
69 void approvePayment(int paymentId) async {
70     if (!isClosed) emit(AdminPaymentActionLoadingState());
71
72     final result = await repository.approvePayment(paymentId);
73     if (isClosed) return;
74
75     result.fold(
76         (failureMessage) {
77             if (!isClosed) emit(AdminPaymentActionErrorState(failureMessage));
78         },
79         (message) {
80             // Find and update the payment status locally
81             final index = paymentsList.indexWhere((p) => p.id == paymentId);
82             if (index != -1) {
83                 final p = paymentsList[index];
84                 paymentsList[index] = PaymentModel(
85                     id: p.id,
86                     orderId: p.orderId,
87                     paymentMethod: p.paymentMethod,
88                     amount: p.amount,
89                     currency: p.currency,
90                     status: 'paid',
91                     gateway: p.gateway,
92                     moyasarPaymentId: p.moyasarPaymentId,
93                     paidAt: p.paidAt,
94                     verifiedAt: p.verifiedAt,
95                     receiptPath: p.receiptPath,
96                     receiptUrl: p.receiptUrl,
97                     createdAt: p.createdAt,
98                     order: p.order,
99                     user: p.user,
100                 );
101             }
102             if (!isClosed) emit(AdminPaymentActionSuccessState(message));
103         },
104     );
105 }
106
107 void rejectPayment(int paymentId) async {
108     if (!isClosed) emit(AdminPaymentActionLoadingState());
109
110     final result = await repository.rejectPayment(paymentId);
111     if (isClosed) return;
112
113     result.fold(
114         (failureMessage) {
115             if (!isClosed) emit(AdminPaymentActionErrorState(failureMessage));
116         },
117         (message) {
118             // Find and update the payment status locally
119             final index = paymentsList.indexWhere((p) => p.id == paymentId);
120             if (index != -1) {
121                 final p = paymentsList[index];
122                 paymentsList[index] = PaymentModel(
123                     id: p.id,
124                     orderId: p.orderId,
125                     paymentMethod: p.paymentMethod,
126                     amount: p.amount,
127                     currency: p.currency,
128                     status: 'rejected',

```

```

129         gateway: p.gateway,
130         moyasarPaymentId: p.moyasarPaymentId,
131         paidAt: p.paidAt,
132         verifiedAt: p.verifiedAt,
133         receiptPath: p.receiptPath,
134         receiptUrl: p.receiptUrl,
135         createdAt: p.createdAt,
136         order: p.order,
137         user: p.user,
138     );
139 }
140 if (!isClosed) emit(AdminPaymentActionSuccessState(message));
141 },
142 );
143 }
144 }
145

```

Lines • Dart 27

Feature / Admin

#91 – lib/features/admin/payments/presentation/cubit/admin_payments_state.dart

```

1 import 'package:moean/features/payment/data/models/payment_model.dart';
2
3 abstract class AdminPaymentsState {}
4
5 class AdminPaymentsInitial extends AdminPaymentsState {}
6
7 class GetAdminPaymentsLoadingState extends AdminPaymentsState {}
8
9 class GetAdminPaymentsSuccessState extends AdminPaymentsState {}
10
11 class GetAdminPaymentsErrorState extends AdminPaymentsState {
12     final String message;
13     GetAdminPaymentsErrorState(this.message);
14 }
15
16 class AdminPaymentActionLoadingState extends AdminPaymentsState {}
17
18 class AdminPaymentActionSuccessState extends AdminPaymentsState {
19     final String message;
20     AdminPaymentActionSuccessState(this.message);
21 }
22
23 class AdminPaymentActionErrorState extends AdminPaymentsState {
24     final String message;
25     AdminPaymentActionErrorState(this.message);
26 }
27

```

Lines • Dart 136

Feature / Admin

#92 – lib/features/admin/payments/presentation/screen/admin_payments_screen.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/routes.dart';
7 import 'package:moean/core/utils/constants/spacing.dart';
8 import 'package:moean/core/utils/extensions/context_extension.dart';
9 import 'package:moean/features/admin/payments/data/repositories/admin_payments_repository.dart';
10 import 'package:moean/features/admin/payments/presentation/cubit/admin_payments_cubit.dart';
11 import 'package:moean/features/admin/payments/presentation/cubit/admin_payments_state.dart';
12 import 'package:moean/features/admin/payments/presentation/widgets/admin_payments_search_filter_widget.dart';
13 import 'package:moean/features/admin/payments/presentation/widgets/admin_payments_table_widget.dart';
14
15 class AdminPaymentsScreen extends StatelessWidget {
16     const AdminPaymentsScreen({super.key});
17
18     @override
19     Widget build(BuildContext context) {
20         return BlocProvider(
21             create: (context) => AdminPaymentsCubit(AdminPaymentsRepository())..getPayments(),
22             child: BlocConsumer<AdminPaymentsCubit, AdminPaymentsState>{

```

```

23 listener: (context, state) {
24   if (state is AdminPaymentActionSuccessState) {
25     ScaffoldMessenger.of(context).showSnackBar(
26       SnackBar(content: Text(state.message), backgroundColor: Colors.green),
27     );
28   } else if (state is AdminPaymentActionErrorState || state is GetAdminPaymentsErrorState) {
29     final msg = state is AdminPaymentActionErrorState
30       ? state.message
31       : (state as GetAdminPaymentsErrorState).message;
32     ScaffoldMessenger.of(context).showSnackBar(
33       SnackBar(content: Text(msg), backgroundColor: Colors.red),
34     );
35   }
36 },
37 builder: (context, state) {
38   final cubit = AdminPaymentsCubit.get(context);
39   final scrollController = ScrollController();
40
41   // Infinite scrolling
42   scrollController.addListener(() {
43     if (scrollController.position.pixels >= scrollController.position.maxScrollExtent - 200) {
44       if (state is! GetAdminPaymentsLoadingState) {
45         cubit.getPayments(loadMore: true);
46       }
47     }
48   });
49
50   return Scaffold(
51     backgroundColor: ColorsManager.background,
52     appBar: AppBar(
53       backgroundColor: ColorsManager.primaryColor,
54       automaticallyImplyLeading: false,
55       title: Row(
56         mainAxisAlignment: MainAxisAlignment.min,
57         children: [
58           const Icon(Icons.payment, color: Colors.white),
59           horizontalSpace8,
60           Flexible(
61             child: Text(
62               appTranslation().get('admin_payments_title'),
63               style: TextStyleManager.bold18.copyWith(color: Colors.white),
64               overflow: TextOverflow.ellipsis,
65             ),
66           ),
67         ],
68       ),
69     actions: [
70       IconButton(
71         icon: const Icon(Icons.refresh, color: Colors.white),
72         onPressed: () => cubit.getPayments(),
73       ),
74       Padding(
75         padding: const EdgeInsetsDirectional.only(end: 12.0),
76         child: ElevatedButton.icon(
77           style: ElevatedButton.styleFrom(
78             backgroundColor: Colors.white,
79             foregroundColor: ColorsManager.primaryColor,
80             elevation: 0,
81             padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
82             minimumSize: Size.zero,
83             tapTargetSize: MaterialTapTargetSize.shrinkWrap,
84             shape: RoundedRectangleBorder(
85               borderRadius: BorderRadius.circular(8),
86             ),
87           ),
88           onPressed: () {
89             context.push(Routes.home);
90           },
91           icon: const Icon(Icons.home, size: 18),
92           label: Text(
93             appTranslation().get('home'),
94             style: TextStyleManager.medium14.copyWith(
95               color: ColorsManager.primaryColor,
96               fontWeight: FontWeight.bold,
97             ),

```

```

98         ),
99     ),
100 ),
101 ],
102 ),
103 body: SafeArea(
104   child: Padding(
105     padding: const EdgeInsets.all(16.0),
106     child: Column(
107       children: [
108         const AdminPaymentsSearchFilterWidget(),
109         verticalSpace24,
110         Expanded(
111           child: Container(
112             clipBehavior: Clip.hardEdge,
113             decoration: BoxDecoration(
114               color: Theme.of(context).cardColor,
115               borderRadius: BorderRadius.circular(12),
116               border: Border.all(color: Colors.grey.withValues(alpha: 0.2)),
117             ),
118             child: state is GetAdminPaymentsLoadingState && cubit.paymentsList.isEmpty
119               ? const Center(child: CircularProgressIndicator())
120               : AdminPaymentsTableWidget(
121                 payments: cubit.paymentsList,
122                 scrollController: scrollController,
123               ),
124           ),
125         ),
126       ],
127     ),
128   ),
129 ),
130 );
131 },
132 ),
133 );
134 }
135 }
136

```

Lines 118

• Dart

Feature / #93 -

Admin

[lib/features/admin/payments/presentation/widgets/admin_payments_search_filter_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/utils/constants/constants.dart';
4  import 'package:moean/core/utils/constants/spacing.dart';
5  import 'package:moean/features/admin/payments/presentation/cubit/admin_payments_cubit.dart';
6
7  class AdminPaymentsSearchFilterWidget extends StatefulWidget {
8    const AdminPaymentsSearchFilterWidget({super.key});
9
10   @override
11   State<AdminPaymentsSearchFilterWidget> createState() => _AdminPaymentsSearchFilterWidgetState();
12 }
13
14 class _AdminPaymentsSearchFilterWidgetState extends State<AdminPaymentsSearchFilterWidget> {
15   final TextEditingController _searchController = TextEditingController();
16
17   // Filter options
18   final List<String> _filters = [
19     'all',
20     'pending',
21     'paid',
22     'failed',
23     'manual',
24     'moyasar'
25   ];
26
27   @override
28   void dispose() {
29     _searchController.dispose();
30     super.dispose();

```

```

31 }
32
33 void _onSearch() {
34   AdminPaymentsCubit.get(context).getPayments(
35     search: _searchController.text.trim(),
36   );
37 }
38
39 void _onFilterChanged(String filter) {
40   AdminPaymentsCubit.get(context).getPayments(
41     filter: filter,
42   );
43 }
44
45 @override
46 Widget build(BuildContext context) {
47   final cubit = AdminPaymentsCubit.get(context);
48
49   return Container(
50     padding: const EdgeInsets.all(16),
51     decoration: BoxDecoration(
52       color: Theme.of(context).cardColor,
53       borderRadius: BorderRadius.circular(12),
54       border: Border.all(color: Colors.grey.withValues(alpha: 0.2)),
55     ),
56     child: Column(
57       crossAxisAlignment: CrossAxisAlignment.start,
58       children: [
59         Row(
60           children: [
61             Expanded(
62               child: TextField(
63                 controller: _searchController,
64                 decoration: InputDecoration(
65                   hintText: appTranslation().get('admin_payments_search_hint'),
66                   prefixIcon: const Icon(Icons.search),
67                   border: OutlineInputBorder(
68                     borderRadius: BorderRadius.circular(8),
69                   ),
70                   contentPadding: const EdgeInsets.symmetric(horizontal: 16),
71                 ),
72                 onSubmitted: (_) => _onSearch(),
73             ),
74             horizontalSpace16,
75             ElevatedButton(
76               onPressed: _onSearch,
77               style: ElevatedButton.styleFrom(
78                 backgroundColor: ColorsManager.primaryColor,
79                 foregroundColor: Colors.white,
80                 padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
81                 shape: RoundedRectangleBorder(
82                   borderRadius: BorderRadius.circular(8),
83                 ),
84             ),
85             child: Text('بحث'),
86           ],
87         ),
88         verticalSpace16,
89         SingleChildScrollView(
90           scrollDirection: Axis.horizontal,
91           child: Row(
92             children: _filters.map((filter) {
93               final isSelected = cubit.currentFilter == filter;
94               return Padding(
95                 padding: const EdgeInsets.only(left: 8.0),
96                 child: FilterChip(
97                   label: Text(appTranslation().get('filter_$filter')),
98                   selected: isSelected,
99                   onSelected: (_) => _onFilterChanged(filter),
100                   selectedColor: ColorsManager.primaryColor.withValues(alpha: 0.2),
101                   checkmarkColor: ColorsManager.primaryColor,
102                   labelStyle: TextStyle(
103                     color: isSelected ? ColorsManager.primaryColor : Theme.of(context).textTheme.bodyMediu

```



```

m?.color,
106         fontWeight: isSelected ? FontWeight.bold : FontWeight.normal,
107     ),
108 ),
109 );
110     }).toList(),
111 ),
112 ),
113 ],
114 ),
115 );
116 }
117 }
118

```

Lines • 192
Dart

Feature / #94 -

Admin [lib/features/admin/payments/presentation/widgets/admin_payments_table_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/features/admin/payments/presentation/cubit/admin_payments_cubit.dart';
6  import 'package:moean/features/payment/data/models/payment_model.dart';
7  import 'package:url_launcher/url_launcher.dart';
8
9  class AdminPaymentsTableWidget extends StatelessWidget {
10     final List<PaymentModel> payments;
11     final ScrollController scrollController;
12
13     const AdminPaymentsTableWidget({
14         super.key,
15         required this.payments,
16         required this.scrollController,
17     });
18
19     Color _getStatusColor(String status) {
20         switch (status.toLowerCase()) {
21             case 'paid':
22                 return Colors.green;
23             case 'failed':
24             case 'rejected':
25             case 'cancelled':
26                 return Colors.red;
27             case 'pending':
28             case 'processing':
29             case 'waiting_verification':
30                 return Colors.orange;
31             default:
32                 return Colors.grey;
33         }
34     }
35
36     String _getMethodText(String method) {
37         if (method == 'manual_bank_transfer' || method == 'manual') {
38             return appTranslation().get('payment_method_manual');
39         } else if (method == 'moyasar') {
40             return 'Moyasar';
41         } else if (method == 'myfatoorah') {
42             return 'MyFatoorah';
43         }
44         return method.isEmpty ? '-' : method;
45     }
46
47     @override
48     Widget build(BuildContext context) {
49         if (payments.isEmpty) {
50             return Center(
51                 child: Column(
52                     mainAxisAlignment: MainAxisAlignment.center,
53                     children: [
54                         Icon(Icons.payment_outlined, size: 64, color: ColorsManager.primaryColor.withValues(alpha: 0.2)),
55                         const SizedBox(height: 16),

```

```

56         Text(
57             appTranslation().get('no_payments_yet'),
58             style: TextStylesManager.bold14.copyWith(color: ColorsManager.primaryColor),
59         ),
60     ],
61 ),
62 );
63 }
64
65 return SingleChildScrollView(
66     scrollDirection: Axis.horizontal,
67     child: SingleChildScrollView(
68         controller: scrollController,
69         child: DataTable(
70             headingRowColor: WidgetStateProperty.all(ColorsManager.primaryColor.withValues(alpha: 0.1)),
71             columns: [
72                 DataColumn(label: Text(appTranslation().get('admin_col_payment_id'), style: TextStylesManager.bold14)),
73                 DataColumn(label: Text(appTranslation().get('admin_col_user'), style: TextStylesManager.bold14)),
74                 DataColumn(label: Text(appTranslation().get('admin_col_service'), style: TextStylesManager.bold14)),
75                 DataColumn(label: Text(appTranslation().get('admin_col_method'), style: TextStylesManager.bold14)),
76                 DataColumn(label: Text(appTranslation().get('admin_col_amount'), style: TextStylesManager.bold14)),
77                 DataColumn(label: Text(appTranslation().get('admin_col_status'), style: TextStylesManager.bold14)),
78                 DataColumn(label: Text(appTranslation().get('admin_col_receipt'), style: TextStylesManager.bold14)),
79                 DataColumn(label: Text(appTranslation().get('admin_col_actions'), style: TextStylesManager.bold14)),
80             ],
81             rows: payments.map((payment) {
82                 final hasReceipt = payment.receiptUrl != null;
83                 final isManualProcessing = (payment.paymentMethod == 'manual_bank_transfer' || payment.paymentMethod == 'manual') &&
84                     payment.status == 'waiting_verification';
85
86                 return DataRow(
87                     cells: [
88                         DataCell(Text('#${payment.id}')),
89                         DataCell(Column(
90                             crossAxisAlignment: CrossAxisAlignment.start,
91                             mainAxisAlignment: MainAxisAlignment.center,
92                             children: [
93                                 Text(payment.user?.name ?? '-', style: const TextStyle(fontWeight: FontWeight.bold)),
94                                 Text(payment.user?.email ?? '-', style: const TextStyle(fontSize: 12, color: Colors.green)),
95                             ],
96                         )),
97                         DataCell(Text(payment.order?.service?.name ?? '-')),
98                         DataCell(
99                             Container(
100                                 padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
101                                 decoration: BoxDecoration(
102                                     color: Colors.green.withValues(alpha: 0.1),
103                                     borderRadius: BorderRadius.circular(16),
104                                 ),
105                                 child: Row(
106                                     mainAxisAlignment: MainAxisAlignment.min,
107                                     children: [
108                                         if (payment.paymentMethod == 'manual_bank_transfer' || payment.paymentMethod == 'manual')
109                                             const Icon(Icons.account_balance, size: 14, color: Colors.green),
110                                         if (payment.paymentMethod == 'myfattoorah' || payment.paymentMethod == 'moyasar')
111                                             const Icon(Icons.credit_card, size: 14, color: Colors.green),
112                                         if (payment.paymentMethod.isNotEmpty)
113                                             const SizedBox(width: 4),
114                                         Text(
115                                             _getMethodText(payment.paymentMethod),
116                                             style: const TextStyle(color: Colors.green, fontSize: 12),
117                                         ),
118                                     ],
119                                 ),
120                     ],
121                 ),
122             ),
123         ),
124     ),
125 );

```

```

121         ),
122         DataCell(Text('${payment.currency} ${payment.amount}', style: const TextStyle(fontWeight: Font
tWeight.bold))),
123         DataCell(
124             Container(
125                 padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
126                 decoration: BoxDecoration(
127                     color: _getStatusColor(payment.status).withValues(alpha: 0.1),
128                     borderRadius: BorderRadius.circular(16),
129                 ),
130                 child: Row(
131                     mainAxisAlignment: MainAxisAlignment.min,
132                     children: [
133                         Icon(
134                             payment.status == 'paid' ? Icons.check_circle_outline :
135                             (payment.status == 'failed' || payment.status == 'rejected' ? Icons.error_outline :
Icons.pending_outlined),
136                             size: 14,
137                             color: _getStatusColor(payment.status)
138                         ),
139                         const SizedBox(width: 4),
140                         Text(
141                             appTranslation().get('status_${payment.status}'),
142                             style: TextStyle(color: _getStatusColor(payment.status), fontSize: 12),
143                         ),
144                     ],
145                 ),
146             ),
147         ),
148         DataCell(
149             hasReceipt
150                 ? TextButton(
151                     onPressed: () async {
152                         final uri = Uri.parse(payment.receiptUrl!);
153                         if (await canLaunchUrl(uri)) {
154                             await launchUrl(uri);
155                         }
156                     },
157                     child: Text(appTranslation().get('view'), style: TextStyle(color: ColorsManager.prim
aryColor)),
158                 )
159                 : const Text('-'),
160         ),
161         DataCell(
162             isManualProcessing
163                 ? Row(
164                     mainAxisAlignment: MainAxisAlignment.min,
165                     children: [
166                         IconButton(
167                             icon: const Icon(Icons.check_circle, color: Colors.green),
168                             tooltip: appTranslation().get('approve'),
169                             onPressed: () {
170                                 AdminPaymentsCubit.get(context).approvePayment(payment.id);
171                             },
172                         ),
173                         IconButton(
174                             icon: const Icon(Icons.cancel, color: Colors.red),
175                             tooltip: appTranslation().get('reject'),
176                             onPressed: () {
177                                 AdminPaymentsCubit.get(context).rejectPayment(payment.id);
178                             },
179                         ),
180                     ],
181                 )
182                 : Text(appTranslation().get('no_action'), style: const TextStyle(color: Colors.grey)),
183         ),
184     ],
185 );
186 }).toList(),
187 ),
188 ),
189 );
190 }

```

```
191 }
192
```

Lines • Dart 79

Feature / Admin

#95 – lib/features/admin/promo/data/models/admin_promo_models.dart

```
1 class AdminPromoCodeModel {
2   final int id;
3   final String code;
4   final String name;
5   final String description;
6   final String discountType;
7   final double discountValue;
8   final bool isActive;
9   final String userTarget;
10  final int? maxRedemptions;
11  final int? maxRedemptionsPerUser;
12  final int redemptionsCount;
13  final String? startsAt;
14  final String? expiresAt;
15
16  const AdminPromoCodeModel({
17    required this.id,
18    required this.code,
19    required this.name,
20    required this.description,
21    required this.discountType,
22    required this.discountValue,
23    required this.isActive,
24    required this.userTarget,
25    this.maxRedemptions,
26    this.maxRedemptionsPerUser,
27    required this.redemptionsCount,
28    this.startsAt,
29    this.expiresAt,
30  });
31
32  factory AdminPromoCodeModel.fromJson(Map<String, dynamic> json) {
33    return AdminPromoCodeModel(
34      id: int.tryParse(json['id']?.toString() ?? '0') ?? 0,
35      code: json['code']?.toString() ?? '',
36      name: json['name']?.toString() ?? '',
37      description: json['description']?.toString() ?? '',
38      discountType: json['discount_type']?.toString() ?? 'percentage',
39      discountValue: double.tryParse(json['discount_value']?.toString() ?? '0') ?? 0.0,
40      isActive: json['is_active'] == true || json['is_active'] == 1 || json['is_active'] == '1',
41      userTarget: json['user_target']?.toString() ?? 'all',
42      maxRedemptions: json['max_redemptions'] != null ? int.tryParse(json['max_redemptions'].toString()) : nu
11,
43      maxRedemptionsPerUser: json['max_redemptions_per_user'] != null ? int.tryParse(json['max_redemptions_pe
r_user'].toString()) : null,
44      redemptionsCount: int.tryParse(json['redemptions_count']?.toString() ?? '0') ?? 0,
45      startsAt: json['starts_at']?.toString(),
46      expiresAt: json['expires_at']?.toString(),
47    );
48  }
49 }
50
51 class AdminReferralStatsModel {
52   final int totalReferralLinks;
53   final int totalRegistrations;
54   final int qualifiedReferrals;
55   final int rewardsGenerated;
56   final int rewardsUsed;
57   final double conversionRatePct;
58
59   const AdminReferralStatsModel({
60     required this.totalReferralLinks,
61     required this.totalRegistrations,
62     required this.qualifiedReferrals,
63     required this.rewardsGenerated,
64     required this.rewardsUsed,
65     required this.conversionRatePct,
66   });
```

```

67
68 factory AdminReferralStatsModel.fromJson(Map<String, dynamic> json) {
69   return AdminReferralStatsModel(
70     totalReferralLinks: int.tryParse(json['total_referral_links']?.toString() ?? '0') ?? 0,
71     totalRegistrations: int.tryParse(json['total_registrations']?.toString() ?? '0') ?? 0,
72     qualifiedReferrals: int.tryParse(json['qualified_referrals']?.toString() ?? '0') ?? 0,
73     rewardsGenerated: int.tryParse(json['rewards_generated']?.toString() ?? '0') ?? 0,
74     rewardsUsed: int.tryParse(json['rewards_used']?.toString() ?? '0') ?? 0,
75     conversionRatePct: double.tryParse(json['conversion_rate_pct']?.toString() ?? '0') ?? 0.0,
76   );
77 }
78 }
79

```

Lines • Dart 95

Feature / Admin

#96 – lib/features/admin/promo/presentation/cubit/admin_promo_cubit.dart

```

1  import 'package:flutter/widgets.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/network/remote/api_service.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/features/admin/promo/data/models/admin_promo_models.dart';
6  import 'package:moean/features/admin/promo/presentation/cubit/admin_promo_state.dart';
7
8  class AdminPromoCubit extends Cubit<AdminPromoState> {
9    AdminPromoCubit() : super(AdminPromoInitial());
10
11    static AdminPromoCubit get(BuildContext context) => BlocProvider.of(context);
12
13    AdminReferralStatsModel? stats;
14    List<AdminPromoCodeModel> promoCodes = [];
15
16    Future<void> loadAll() async {
17      if (!isClosed) emit(AdminPromoLoading());
18      final statsResult = await ApiService.getAdminReferralStats();
19      final promoResult = await ApiService.getAdminPromoCodes();
20      if (isClosed) return;
21
22      statsResult.fold(
23        (error) {
24          if (!isClosed) emit(AdminPromoError(error));
25        },
26        (statsData) {
27          promoResult.fold(
28            (error) {
29              if (!isClosed) emit(AdminPromoError(error));
30            },
31            (promoData) {
32              stats = AdminReferralStatsModel.fromJson(statsData);
33              final list = promoData['data'] as List<dynamic>? ?? [];
34              promoCodes = list
35                .map((e) => AdminPromoCodeModel.fromJson(e as Map<String, dynamic>))
36                .toList();
37              if (!isClosed) emit(AdminPromoLoaded(stats: stats!, promoCodes: promoCodes));
38            },
39          );
40        },
41      );
42    }
43
44    Future<void> createPromoCode(Map<String, dynamic> data) async {
45      if (!isClosed) emit(AdminPromoCreating());
46      final result = await ApiService.createAdminPromoCode(data);
47      if (isClosed) return;
48      result.fold(
49        (error) {
50          if (!isClosed) emit(AdminPromoActionError(error));
51        },
52        (_) {
53          if (!isClosed) emit(AdminPromoActionSuccess(appTranslation().get('admin_promo_success_create')));
54          loadAll();
55        },
56      );
57    }
58

```

```

58
59 Future<void> togglePromoCode(AdminPromoCodeModel promo) async {
60   if (!isClosed) emit(AdminPromoActionLoading());
61   final result = promo.isActive
62     ? await ApiService.deactivateAdminPromoCode(promo.id)
63     : await ApiService.activateAdminPromoCode(promo.id);
64   if (isClosed) return;
65
66   result.fold(
67     (error) {
68       if (!isClosed) emit(AdminPromoActionError(error));
69     },
70     (_) {
71       final msg = promo.isActive
72         ? appTranslation().get('admin_promo_success_deactivate')
73         : appTranslation().get('admin_promo_success_activate');
74       if (!isClosed) emit(AdminPromoActionSuccess(msg));
75       loadAll();
76     },
77   );
78 }
79
80 Future<void> deletePromoCode(int id) async {
81   if (!isClosed) emit(AdminPromoActionLoading());
82   final result = await ApiService.deleteAdminPromoCode(id);
83   if (isClosed) return;
84   result.fold(
85     (error) {
86       if (!isClosed) emit(AdminPromoActionError(error));
87     },
88     (_) {
89       if (!isClosed) emit(AdminPromoActionSuccess(appTranslation().get('admin_promo_success_delete')));
90       loadAll();
91     },
92   );
93 }
94 }
95

```

Lines • Dart 34

Feature / Admin

#97 – lib/features/admin/promo/presentation/cubit/admin_promo_state.dart

```

1  import 'package:moean/features/admin/promo/data/models/admin_promo_models.dart';
2
3  abstract class AdminPromoState {}
4
5  class AdminPromoInitial extends AdminPromoState {}
6
7  class AdminPromoLoading extends AdminPromoState {}
8
9  class AdminPromoLoaded extends AdminPromoState {
10    final AdminReferralStatsModel stats;
11    final List<AdminPromoCodeModel> promoCodes;
12    AdminPromoLoaded({required this.stats, required this.promoCodes});
13  }
14
15  class AdminPromoError extends AdminPromoState {
16    final String error;
17    AdminPromoError(this.error);
18  }
19
20  // Actions
21  class AdminPromoActionLoading extends AdminPromoState {}
22
23  class AdminPromoActionSuccess extends AdminPromoState {
24    final String message;
25    AdminPromoActionSuccess(this.message);
26  }
27
28  class AdminPromoActionError extends AdminPromoState {
29    final String error;
30    AdminPromoActionError(this.error);
31  }
32

```

```
33 class AdminPromoCreating extends AdminPromoState {}
34
```

Lines • Dart 176

Feature / Admin

#98 – lib/features/admin/promo/presentation/screen/admin_promo_screen.dart

```
1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/primary/conditional_builder.dart';
7 import 'package:moean/core/utils/constants/spacing.dart';
8 import 'package:moean/core/utils/extensions/context_extension.dart';
9 import 'package:moean/features/admin/promo/presentation/cubit/admin_promo_cubit.dart';
10 import 'package:moean/features/admin/promo/presentation/cubit/admin_promo_state.dart';
11 import 'package:moean/features/admin/promo/presentation/widgets/admin_promo_body_widget.dart';
12 import 'package:moean/features/admin/promo/presentation/widgets/admin_promo_create_dialog.dart';
13
14 class AdminPromoScreen extends StatefulWidget {
15   const AdminPromoScreen({super.key});
16
17   @override
18   State<AdminPromoScreen> createState() => _AdminPromoScreenState();
19 }
20
21 class _AdminPromoScreenState extends State<AdminPromoScreen> {
22   @override
23   void initState() {
24     super.initState();
25     AdminPromoCubit.get(context).loadAll();
26   }
27
28   void _showCreateDialog(BuildContext context) {
29     showDialog<void>(
30       context: context,
31       builder: (_) => BlocProvider.value(
32         value: AdminPromoCubit.get(context),
33         child: const AdminPromoCreateDialog(),
34       ),
35     );
36   }
37
38   @override
39   Widget build(BuildContext context) {
40     return Directionality(
41       textDirection: TextDirection.rtl,
42       child: Scaffold(
43         backgroundColor: ColorsManager.background,
44         appBar: AppBar(
45           backgroundColor: ColorsManager.primaryColor,
46           elevation: 0,
47           automaticallyImplyLeading: false,
48           title: Row(
49             mainAxisAlignment: MainAxisAlignment.min,
50             children: [
51               const Icon(Icons.discount_outlined, color: Colors.white, size: 22),
52               horizontalSpace8,
53               Flexible(
54                 child: Text(
55                   appTranslation().get('admin_promo_title'),
56                   style: TextStylesManager.bold18.copyWith(color: Colors.white),
57                   overflow: TextOverflow.ellipsis,
58                 ),
59               ),
60             ],
61           ),
62           actions: [
63             // Refresh
64             IconButton(
65               icon: const Icon(Icons.refresh_rounded, color: Colors.white),
66               onPressed: () => AdminPromoCubit.get(context).loadAll(),
67             ),
68             // Back home
```

```

69     Padding(
70       padding: const EdgeInsetsDirectional.only(end: 12.0),
71       child: ElevatedButton.icon(
72         style: ElevatedButton.styleFrom(
73           backgroundColor: Colors.white,
74           foregroundColor: ColorsManager.primaryColor,
75           elevation: 0,
76           padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
77           minimumSize: Size.zero,
78           tapTargetSize: MaterialTapTargetSize.shrinkWrap,
79           shape: RoundedRectangleBorder(
80             borderRadius: BorderRadius.circular(8),
81           ),
82         ),
83         onPressed: () => context.pop(),
84         icon: const Icon(Icons.home_rounded, size: 18),
85         label: Text(
86           appTranslation().get('home'),
87           style: TextStyleManager.medium14.copyWith(
88             color: ColorsManager.primaryColor,
89             fontWeight: FontWeight.bold,
90           ),
91         ),
92       ),
93     ),
94   ],
95 ),
96 floatingActionButton: FloatingActionButton.extended(
97   backgroundColor: ColorsManager.primaryColor,
98   onPressed: () => _showCreateDialog(context),
99   icon: const Icon(Icons.add_rounded, color: Colors.white),
100   label: Text(
101     appTranslation().get('admin_promo_create_btn'),
102     style:
103       TextStyleManager.medium14.copyWith(color: Colors.white),
104   ),
105 ),
106 body: SafeArea(
107   child: BlocConsumer<AdminPromoCubit, AdminPromoState>(
108     listenWhen: (_, s) =>
109       s is AdminPromoActionSuccess || s is AdminPromoActionError,
110     listener: (context, state) {
111       if (state is AdminPromoActionSuccess) {
112         ScaffoldMessenger.of(context).showSnackBar(SnackBar(
113           content: Text(state.message, textAlign: TextAlign.center),
114           backgroundColor: ColorsManager.primaryColor,
115           behavior: SnackBarBehavior.floating,
116           shape: RoundedRectangleBorder(
117             borderRadius: BorderRadius.circular(12)),
118         ));
119         Navigator.of(context, rootNavigator: true)
120           .popUntil((_) => true);
121       } else if (state is AdminPromoActionError) {
122         ScaffoldMessenger.of(context).showSnackBar(SnackBar(
123           content: Text(state.error, textAlign: TextAlign.center),
124           backgroundColor: ColorsManager.errorColor,
125           behavior: SnackBarBehavior.floating,
126           shape: RoundedRectangleBorder(
127             borderRadius: BorderRadius.circular(12)),
128         ));
129       }
130     },
131     buildWhen: (_, s) =>
132       s is AdminPromoLoading ||
133       s is AdminPromoLoaded ||
134       s is AdminPromoError ||
135       s is AdminPromoActionLoading,
136     builder: (context, state) {
137       final cubit = AdminPromoCubit.get(context);
138       return ConditionalBuilder(
139         loadingState: state is AdminPromoLoading || state is AdminPromoActionLoading,
140         errorState: state is AdminPromoError,
141         emptyState: false,
142         errorBuilder: (_) => Center(
143           child: Column(

```



```

144         mainAxisSize: MainAxisSize.min,
145         children: [
146           Icon(Icons.cloud_off_outlined,
147             size: 48,
148             color: ColorsManager.textSecondary
149               .withValues(alpha: 0.4)),
150           verticalSpace12,
151           Text((state as AdminPromoError).error,
152             style: TextStylesManager.regular14.copyWith(
153               color: ColorsManager.textSecondary),
154             textAlign: TextAlign.center),
155           verticalSpace16,
156           TextButton.icon(
157             onPressed: () => cubit.loadAll(),
158             icon: const Icon(Icons.refresh_rounded),
159             label: Text(appTranslation().get('admin_promo_refresh')),
160           ),
161         ],
162       ),
163     ),
164     successBuilder: (_) => AdminPromoBodyWidget(
165       stats: cubit.stats!,
166       promoCodes: cubit.promoCodes,
167     ),
168   );
169 },
170 ),
171 ),
172 ),
173 );
174 }
175 }
176

```

Lines • Dart 151

Feature / Admin

#99 – lib/features/admin/promo/presentation/widgets/admin_promo_body_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6  import 'package:moean/features/admin/promo/data/models/admin_promo_models.dart';
7  import 'package:moean/features/admin/promo/presentation/cubit/admin_promo_cubit.dart';
8  import 'package:moean/features/admin/promo/presentation/widgets/admin_promo_stat_card_widget.dart';
9  import 'package:moean/features/admin/promo/presentation/widgets/admin_promo_table_row_widget.dart';
10
11  class AdminPromoBodyWidget extends StatelessWidget {
12    final AdminReferralStatsModel stats;
13    final List<AdminPromoCodeModel> promoCodes;
14
15    const AdminPromoBodyWidget({
16      super.key,
17      required this.stats,
18      required this.promoCodes,
19    });
20
21    void _showDeleteConfirm(BuildContext context, int id) {
22      showDialog<void>(
23        context: context,
24        builder: (_) => Directionality(
25          textDirection: TextDirection.rtl,
26          child: AlertDialog(
27            backgroundColor: ColorsManager.surfacePrimary,
28            shape:
29              RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
30            title: Text(
31              appTranslation().get('admin_promo_delete_confirm'),
32              style:
33                TextStylesManager.bold16.copyWith(color: ColorsManager.textPrimary),
34            ),
35            actions: [
36              TextButton(
37                onPressed: () => Navigator.of(context).pop(),

```

```

38         child: Text(
39             appTranslation().get('admin_promo_delete_no'),
40             style: TextStylesManager.medium14
41             .copyWith(color: ColorsManager.textSecondary),
42         ),
43     ),
44     TextButton(
45         onPressed: () {
46             Navigator.of(context).pop();
47             AdminPromoCubit.get(context).deletePromoCode(id);
48         },
49         child: Text(
50             appTranslation().get('admin_promo_delete_yes'),
51             style: TextStylesManager.bold14
52             .copyWith(color: ColorsManager.errorColor),
53         ),
54     ),
55 ],
56 ),
57 ),
58 );
59 }
60
61 @override
62 Widget build(BuildContext context) {
63     return SingleChildScrollView(
64         padding: const EdgeInsets.all(20),
65         child: Column(
66             crossAxisAlignment: CrossAxisAlignment.end,
67             children: [
68                 // Stats cards
69                 AdminPromoStatCardWidget(
70                     label: appTranslation().get('admin_promo_stat_registrations'),
71                     value: '${stats.totalRegistrations}',
72                     icon: Icons.people_alt_outlined,
73                     iconBgColor: ColorsManager.primaryColor.withValues(alpha: 0.1),
74                     iconColor: ColorsManager.primaryColor,
75                 ),
76                 verticalSpace12,
77                 AdminPromoStatCardWidget(
78                     label: appTranslation().get('admin_promo_stat_qualified'),
79                     value: '${stats.qualifiedReferrals}',
80                     icon: Icons.workspace_premium_outlined,
81                     iconBgColor: const Color(0xFFE7F6),
82                     iconColor: const Color(0xFF7B1FA2),
83                 ),
84                 verticalSpace12,
85                 AdminPromoStatCardWidget(
86                     label: appTranslation().get('admin_promo_stat_rewards'),
87                     value: '${stats.rewardsUsed} / ${stats.rewardsGenerated}',
88                     icon: Icons.redeem_outlined,
89                     iconBgColor: const Color(0xFFFFF3E0),
90                     iconColor: const Color(0xFFFF8C00),
91                 ),
92                 verticalSpace12,
93                 AdminPromoStatCardWidget(
94                     label: appTranslation().get('admin_promo_stat_conversion'),
95                     value: '${stats.conversionRatePct.toStringAsFixed(1)}%',
96                     icon: Icons.trending_up_rounded,
97                     iconBgColor: const Color(0xFFE8F5E9),
98                     iconColor: const Color(0xFF2E7D32),
99                 ),
100                verticalSpace24,
101                // Promo codes table
102                Container(
103                    width: double.infinity,
104                    padding: const EdgeInsets.all(16),
105                    decoration: BoxDecoration(
106                        color: ColorsManager.surfacePrimary,
107                        borderRadius: BorderRadius.circular(16),
108                        border: Border.all(color: ColorsManager.borderColor),
109                    ),
110                    child: Column(
111                        crossAxisAlignment: CrossAxisAlignment.end,
112                        children: [

```

```

113         Text(
114             appTranslation().get('admin_promo_table_title'),
115             style: TextStylesManager.bold16
116             .copyWith(color: ColorsManager.textPrimary),
117         ),
118         verticalSpace16,
119         if (promoCodes.isEmpty)
120             Padding(
121                 padding: const EdgeInsets.symmetric(vertical: 24),
122                 child: Center(
123                     child: Text(
124                         appTranslation().get('admin_promo_empty'),
125                         style: TextStylesManager.regular14
126                         .copyWith(color: ColorsManager.textSecondary),
127                     ),
128                 ),
129             )
130         else
131             ListView.separated(
132                 shrinkWrap: true,
133                 physics: const NeverScrollableScrollPhysics(),
134                 itemCount: promoCodes.length,
135                 separatorBuilder: (_, _) => verticalSpace10,
136                 itemBuilder: (_, i) => AdminPromoTableRowWidget(
137                     promo: promoCodes[i],
138                     onDelete: () =>
139                         _showDeleteConfirm(context, promoCodes[i].id),
140                 ),
141             ),
142     ],
143 ),
144 ),
145     verticalSpace24,
146 ],
147 ),
148 );
149 }
150 }
151

```

Lines • 252
Dart

Feature / #100 -

Admin lib/features/admin/promo/presentation/widgets/admin_promo_create_dialog.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7  import 'package:moean/features/admin/promo/presentation/cubit/admin_promo_cubit.dart';
8  import 'package:moean/features/admin/promo/presentation/cubit/admin_promo_state.dart';
9  import 'package:flutter_bloc/flutter_bloc.dart';
10
11  class AdminPromoCreateDialog extends StatefulWidget {
12      const AdminPromoCreateDialog({super.key});
13
14      @override
15      State<AdminPromoCreateDialog> createState() => _AdminPromoCreateDialogState();
16  }
17
18  class _AdminPromoCreateDialogState extends State<AdminPromoCreateDialog> {
19      final _formKey = GlobalKey<FormState>();
20      late final TextEditingController _codeCtrl;
21      late final TextEditingController _nameCtrl;
22      late final TextEditingController _valueCtrl;
23      late final TextEditingController _maxUseCtrl;
24      String _discountType = 'percentage';
25      String _userTarget = 'all';
26
27      @override
28      void initState() {
29          super.initState();
30          _codeCtrl = TextEditingController();

```

```

31     _nameCtrl = TextEditingController();
32     _valueCtrl = TextEditingController();
33     _maxUseCtrl = TextEditingController();
34 }
35
36 @override
37 void dispose() {
38     _codeCtrl.dispose();
39     _nameCtrl.dispose();
40     _valueCtrl.dispose();
41     _maxUseCtrl.dispose();
42     super.dispose();
43 }
44
45 void _submit(BuildContext context) {
46     if (!_formKey.currentState!.validate()) return;
47     AdminPromoCubit.get(context).createPromoCode({
48         'code': _codeCtrl.text.trim().toUpperCase(),
49         'name': _nameCtrl.text.trim(),
50         'discount_type': _discountType,
51         'discount_value': double.tryParse(_valueCtrl.text) ?? 0,
52         'max_redemptions': int.tryParse(_maxUseCtrl.text),
53         'max_redemptions_per_user': 1,
54         'is_active': true,
55         'user_target': _userTarget,
56     });
57 }
58
59 Widget _buildDropdown({
60     required String label,
61     required String value,
62     required List<DropdownMenuItem<String>> items,
63     required void Function(String?) onChanged,
64 }) {
65     return Column(
66         crossAxisAlignment: CrossAxisAlignment.start,
67         children: [
68             if (label.isNotEmpty) ...[
69                 Text(
70                     label,
71                     style: TextStylesManager.regular14.copyWith(
72                         color: ColorsManager.textSecondary,
73                     ),
74                 ),
75                 verticalSpace4,
76             ],
77             DropdownButtonFormField<String>({
78                 isExpanded: true,
79                 initialValue: value,
80                 icon: const Icon(Icons.keyboard_arrow_down_rounded),
81                 decoration: InputDecoration(
82                     filled: true,
83                     fillColor: ColorsManager.surfacePrimary,
84                     contentPadding: const EdgeInsets.symmetric(horizontal: 20, vertical: 16),
85                     border: OutlineInputBorder(
86                         borderRadius: BorderRadius.circular(24),
87                         borderSide: BorderSide(color: ColorsManager.borderColor),
88                     ),
89                     enabledBorder: OutlineInputBorder(
90                         borderRadius: BorderRadius.circular(24),
91                         borderSide: BorderSide(color: ColorsManager.borderColor),
92                     ),
93                     focusedBorder: OutlineInputBorder(
94                         borderRadius: BorderRadius.circular(24),
95                         borderSide: BorderSide(color: ColorsManager.themeActiveAccent),
96                     ),
97                 ),
98                 items: items,
99                 onChanged: onChanged,
100             ),
101         ],
102     );
103 }
104
105 @override

```

```

106 Widget build(BuildContext context) {
107   return Directionality(
108     textDirection: TextDirection.rtl,
109     child: Dialog(
110       backgroundColor: ColorsManager.surfacePrimary,
111       shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(20)),
112       child: SingleChildScrollView(
113         padding: const EdgeInsets.all(24),
114         child: Form(
115           key: _formKey,
116           child: Column(
117             mainAxisAlignment: MainAxisAlignment.min,
118             crossAxisAlignment: CrossAxisAlignment.stretch,
119             children: [
120               // Header
121               Row(
122                 mainAxisAlignment: MainAxisAlignment.spaceBetween,
123                 children: [
124                   Text(
125                     appTranslation().get('admin_promo_create_title'),
126                     style: TextStyleManager.bold18.copyWith(color: ColorsManager.primaryColor),
127                   ),
128                   IconButton(
129                     icon: const Icon(Icons.close_rounded, color: Colors.grey),
130                     onPressed: () => Navigator.of(context).pop(),
131                   ),
132                 ],
133               ),
134               verticalSpace20,
135               // Code
136               PrimaryTextField(
137                 controller: _codeCtrl,
138                 hint: appTranslation().get('admin_promo_form_code_hint'),
139                 label: 'رمز الكود (Code)',
140                 validator: (v) => (v == null || v.isEmpty) ? 'مطلوب' : null,
141               ),
142               verticalSpace12,
143               // Name
144               PrimaryTextField(
145                 controller: _nameCtrl,
146                 hint: 'مثال: خصم العودة للمدارس',
147                 label: 'اسم الحملة',
148                 validator: (v) => (v == null || v.isEmpty) ? 'مطلوب' : null,
149               ),
150               verticalSpace12,
151               // Type
152               _buildDropdown(
153                 label: 'نوع الخصم',
154                 value: _discountType,
155                 items: const [
156                   DropdownMenuItem(value: 'percentage', child: Text('(%) نسبة مئوية')),
157                   DropdownMenuItem(value: 'fixed_amount', child: Text('مبلغ ثابت')),
158                 ],
159                 onChanged: (val) {
160                   if (val != null) setState(() => _discountType = val);
161                 },
162               ),
163               verticalSpace12,
164               // Value
165               PrimaryTextField(
166                 controller: _valueCtrl,
167                 hint: '15',
168                 label: 'قيمة الخصم',
169                 keyboardType: TextInputType.number,
170                 validator: (v) => (v == null || v.isEmpty) ? 'مطلوب' : null,
171               ),
172               verticalSpace12,
173               // Target
174               _buildDropdown(
175                 label: 'المستهدفين',

```

```

181         value: _userTarget,
182         items: const [
183             DropdownMenuItem(value: 'all', child: Text('الجميع', overflow: TextOverflow.ellipsis)),
184             DropdownMenuItem(value: 'existing_users', child: Text('المستخدمين المشتركين فقط', overflo
w: TextOverflow.ellipsis)),
185             DropdownMenuItem(value: 'new_users', child: Text('المستخدمين الجدد فقط', overflow: TextOv
erflow.ellipsis)),
186         ],
187         onChanged: (val) {
188             if (val != null) setState(() => _userTarget = val);
189         },
190     ),
191     verticalSpace12,
192
193     // Max Use
194     PrimaryTextField(
195         controller: _maxUseCtrl,
196         hint: 'مثال: 1000',
197         label: 'حدد الاستخدام الأقصى (اختياري)',
198         keyboardType: TextInputType.number,
199     ),
200     verticalSpace24,
201
202     // Action Buttons
203     BlocBuilder<AdminPromoCubit, AdminPromoState>(
204         buildWhen: (_, s) =>
205             s is AdminPromoCreating || s is AdminPromoActionSuccess || s is AdminPromoActionError,
206         builder: (context, state) {
207             final isLoading = state is AdminPromoCreating;
208             return Row(
209                 children: [
210                     Expanded(
211                         child: ElevatedButton(
212                             style: ElevatedButton.styleFrom(
213                                 backgroundColor: ColorsManager.primaryColor,
214                                 foregroundColor: Colors.white,
215                                 padding: const EdgeInsets.symmetric(vertical: 16),
216                                 shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(24)),
217                             ),
218                             onPressed: isLoading ? null : () => _submit(context),
219                             child: isLoading
220                                 ? const SizedBox(
221                                     width: 20, height: 20,
222                                     child: CircularProgressIndicator(color: Colors.white, strokeWidth: 2),
223                                 )
224                                 : Text('حفظ', style: TextStyleManager.bold14),
225                         ),
226                     ),
227                     horizontalSpace12,
228                     Expanded(
229                         child: OutlinedButton(
230                             style: OutlinedButton.styleFrom(
231                                 foregroundColor: ColorsManager.textPrimary,
232                                 padding: const EdgeInsets.symmetric(vertical: 16),
233                                 shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(24)),
234                                 side: BorderSide(color: ColorsManager.borderColor),
235                             ),
236                             onPressed: isLoading ? null : () => Navigator.of(context).pop(),
237                             child: Text('إلغاء', style: TextStyleManager.bold14),
238                         ),
239                     ),
240                 ],
241             );
242         },
243     ),
244 ],
245 ),
246 ),
247 ),
248 ),
249 );
250 }
251 }
252

```

```
1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/spacing.dart';
5
6 class AdminPromoStatCardWidget extends StatelessWidget {
7   final String label;
8   final String value;
9   final IconData icon;
10  final Color iconBgColor;
11  final Color iconColor;
12
13  const AdminPromoStatCardWidget({
14    super.key,
15    required this.label,
16    required this.value,
17    required this.icon,
18    required this.iconBgColor,
19    required this.iconColor,
20  });
21
22  @override
23  Widget build(BuildContext context) {
24    return Container(
25      width: double.infinity,
26      padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 18),
27      decoration: BoxDecoration(
28        color: ColorsManager.surfacePrimary,
29        borderRadius: BorderRadius.circular(16),
30        border: Border.all(color: ColorsManager.borderColor),
31      ),
32      child: Row(
33        children: [
34          Container(
35            width: 48,
36            height: 48,
37            decoration: BoxDecoration(
38              color: iconBgColor,
39              borderRadius: BorderRadius.circular(14),
40            ),
41            child: Icon(icon, color: iconColor, size: 24),
42          ),
43          horizontalSpace16,
44          Expanded(
45            child: Column(
46              crossAxisAlignment: CrossAxisAlignment.end,
47              children: [
48                Text(
49                  label,
50                  style: TextStylesManager.regular12.copyWith(
51                    color: ColorsManager.textSecondary,
52                  ),
53                ),
54                verticalSpace4,
55                Text(
56                  value,
57                  style: TextStylesManager.bold20.copyWith(
58                    color: ColorsManager.textPrimary,
59                  ),
60                ),
61              ],
62            ),
63          ),
64        ],
65      ),
66    );
67  }
68 }
69
```

```
1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/spacing.dart';
6 import 'package:moean/features/admin/promo/data/models/admin_promo_models.dart';
7 import 'package:moean/features/admin/promo/presentation/cubit/admin_promo_cubit.dart';
8
9 class AdminPromoTableRowWidget extends StatelessWidget {
10   final AdminPromoCodeModel promo;
11   final VoidCallback onDelete;
12
13   const AdminPromoTableRowWidget({
14     super.key,
15     required this.promo,
16     required this.onDelete,
17   });
18
19   String _targetLabel(String target) {
20     switch (target) {
21       case 'new_users':
22         return appTranslation().get('admin_promo_target_new');
23       case 'existing_users':
24         return appTranslation().get('admin_promo_target_existing');
25       default:
26         return appTranslation().get('admin_promo_target_all');
27     }
28   }
29
30   String _valueDisplay(AdminPromoCodeModel p) {
31     final valueStr = p.discountValue.truncateToDouble() == p.discountValue
32       ? p.discountValue.toInt().toString()
33       : p.discountValue.toStringAsFixed(2);
34     return p.discountType == 'percentage'
35       ? '$valueStr%'
36       : '$valueStr %';
37   }
38
39   @override
40   Widget build(BuildContext context) {
41     return Container(
42       padding: const EdgeInsets.all(12),
43       decoration: BoxDecoration(
44         color: ColorsManager.surfacePrimary,
45         borderRadius: BorderRadius.circular(12),
46         border: Border.all(color: ColorsManager.borderColor),
47       ),
48       child: Column(
49         crossAxisAlignment: CrossAxisAlignment.start,
50         children: [
51           // Row 1: Code + Name on start, Status badge + Actions on end
52           Row(
53             mainAxisAlignment: MainAxisAlignment.spaceBetween,
54             children: [
55               // Code + Name
56               Expanded(
57                 child: Column(
58                   crossAxisAlignment: CrossAxisAlignment.start,
59                   children: [
60                     Text(
61                       promo.code,
62                       style: TextStylesManager.bold14.copyWith(
63                         color: ColorsManager.primaryColor,
64                       ),
65                     ),
66                     Text(
67                       promo.name,
68                       style: TextStylesManager.regular12.copyWith(
69
```



```

72         color: ColorsManager.textSecondary,
73     ),
74     maxLines: 1,
75     overflow: TextOverflow.ellipsis,
76 ),
77 ],
78 ),
79 ),
80 horizontalSpace8,
81 // Status Badge + Actions
82 Row(
83     mainAxisSize: MainAxisSize.min,
84     children: [
85         // Status badge
86         Container(
87             padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
88             decoration: BoxDecoration(
89                 color: promo.isActive
90                     ? const Color(0xFF22C55E).withValues(alpha: 0.12)
91                     : ColorsManager.errorColor.withValues(alpha: 0.1),
92                 borderRadius: BorderRadius.circular(20),
93             ),
94             child: Text(
95                 promo.isActive
96                     ? appTranslation().get('admin_promo_status_active')
97                     : appTranslation().get('admin_promo_status_inactive'),
98                 style: TextStylesManager.bold12.copyWith(
99                     color: promo.isActive
100                        ? const Color(0xFF16A34A)
101                        : ColorsManager.errorColor,
102                 ),
103             ),
104         ),
105         horizontalSpace8,
106         // Toggle activate/deactivate
107         IconButton(
108             onPressed: () =>
109                 AdminPromoCubit.get(context).togglePromoCode(promo),
110             icon: Icon(
111                 promo.isActive
112                     ? Icons.toggle_on_rounded
113                     : Icons.toggle_off_rounded,
114                 color: promo.isActive
115                     ? ColorsManager.primaryColor
116                     : ColorsManager.textSecondary,
117                 size: 28,
118             ),
119             padding: EdgeInsets.zero,
120             constraints: const BoxConstraints(),
121         ),
122         horizontalSpace8,
123         // Delete
124         IconButton(
125             onPressed: onDelete,
126             icon: const Icon(
127                 Icons.delete_outline_rounded,
128                 color: ColorsManager.errorColor,
129                 size: 20,
130             ),
131             padding: EdgeInsets.zero,
132             constraints: const BoxConstraints(),
133         ),
134     ],
135 ),
136 ],
137 ),
138 const Divider(height: 16, thickness: 0.6),
139 // Row 2: Info chips (Wrap avoids overflow)
140 Wrap(
141     spacing: 8,
142     runSpacing: 6,
143     children: [
144         _InfoChip(
145             label: _valueDisplay(promo),
146             icon: Icons.discount_outlined,

```

```

147         ),
148         _InfoChip(
149           label: _targetLabel(promo.userTarget),
150           icon: Icons.flag_outlined,
151         ),
152         _InfoChip(
153           label: '${promo.redemptionsCount}/${promo.maxRedemptions ?? '∞'}',
154           icon: Icons.people_outline,
155         ),
156       ],
157     ),
158   ],
159 ),
160 );
161 }
162 }
163
164 class _InfoChip extends StatelessWidget {
165   final String label;
166   final IconData icon;
167
168   const _InfoChip({required this.label, required this.icon});
169
170   @override
171   Widget build(BuildContext context) {
172     return Container(
173       padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
174       decoration: BoxDecoration(
175         color: ColorsManager.background,
176         borderRadius: BorderRadius.circular(8),
177         border: Border.all(color: ColorsManager.borderColor.withValues(alpha: 0.6)),
178       ),
179       child: Row(
180         mainAxisAlignment: MainAxisAlignment.min,
181         children: [
182           Icon(icon, size: 14, color: ColorsManager.textSecondary),
183           const SizedBox(width: 4),
184           Text(
185             label,
186             style: TextStylesManager.medium12
187               .copyWith(color: ColorsManager.textPrimary),
188           ),
189         ],
190       ),
191     );
192   }
193 }
194

```

Lines • 173
Dart

Feature / #103 -

Admin lib/features/admin/teachers/data/repositories/admin_teachers_repository.dart

```

1  import 'package:dartz/dartz.dart';
2  import 'package:moean/core/network/remote/api_endpoints.dart';
3  import 'package:moean/core/network/remote/dio_helper.dart';
4  import 'package:moean/core/models/admin_teacher_model.dart';
5  import 'dart:convert';
6
7  class AdminTeachersRepository {
8    static dynamic _decodeData(dynamic data) {
9      if (data is String) {
10        try {
11          return jsonDecode(data);
12        } catch (_) {
13          return data;
14        }
15      }
16      return data;
17    }
18
19    static Future<Either<String, AdminTeacherPaginationModel>> getTeachers({
20      String? search,
21      String? status,

```

```

22     int page = 1,
23     int perPage = 20,
24 }) async {
25     final queryParams = {
26       'page': page,
27       'per_page': perPage,
28       if (search != null && search.isNotEmpty) 'search': search,
29       if (status != null && status.isNotEmpty) 'status': status,
30     };
31
32     final response = await DioHelper.getData(
33       url: adminTeachersApi,
34       query: queryParams,
35     );
36
37     return response.fold(
38       (error) => Left(error),
39       (res) => Right(AdminTeacherPaginationModel.fromJson(_decodeData(res.data))),
40     );
41   }
42
43   static Future<Either<String, List<AdminTeacherSubscriptionModel>>> getSubscriptions() async {
44     final response = await DioHelper.getData(
45       url: subscriptionsApi,
46     );
47
48     return response.fold(
49       (error) => Left(error),
50       (res) {
51         final data = _decodeData(res.data) as List;
52         return Right(data.map((e) => AdminTeacherSubscriptionModel.fromJson(e)).toList());
53       },
54     );
55   }
56
57   static Future<Either<String, AdminTeacherModel>> createTeacher({
58     required String name,
59     required String email,
60     required String phone,
61     required String password,
62     required int subscriptionId,
63 }) async {
64     final response = await DioHelper.postData(
65       url: adminTeachersApi,
66       data: {
67         'name': name,
68         'email': email,
69         'phone': phone,
70         'password': password,
71         'subscription_id': subscriptionId,
72       },
73     );
74
75     return response.fold(
76       (error) => Left(error),
77       (res) => Right(AdminTeacherModel.fromJson(_decodeData(res.data))),
78     );
79   }
80
81   static Future<Either<String, AdminTeacherModel>> updateTeacher({
82     required int id,
83     String? name,
84     String? email,
85     String? phone,
86     bool? active,
87     int? subscriptionId,
88     String? subscriptionEndsAt,
89     String? password,
90 }) async {
91     final data = <String, dynamic>{};
92     if (name != null) data['name'] = name;
93     if (email != null) data['email'] = email;
94     if (phone != null) data['phone'] = phone;
95     if (active != null) data['active'] = active;
96     if (subscriptionId != null) data['subscription_id'] = subscriptionId;

```

```

97     if (subscriptionEndsAt != null) data['subscription_ends_at'] = subscriptionEndsAt;
98     if (password != null) data['password'] = password;
99
100    final response = await DioHelper.patchData(
101      url: '$adminTeachersApi/$id',
102      data: data,
103    );
104
105    return response.fold(
106      (error) => Left(error),
107      (res) => Right(AdminTeacherModel.fromJson(_decodeData(res.data))),
108    );
109  }
110
111  static Future<Either<String, AdminTeacherModel>> renewSubscription({
112    required int id,
113    int? subscriptionId,
114    int? months,
115  }) async {
116    final data = <String, dynamic>{};
117    if (subscriptionId != null) data['subscription_id'] = subscriptionId;
118    if (months != null) data['months'] = months;
119
120    final response = await DioHelper.postData(
121      url: '$adminTeachersApi/$id/renew-subscription',
122      data: data,
123    );
124
125    return response.fold(
126      (error) => Left(error),
127      (res) => Right(AdminTeacherModel.fromJson(_decodeData(res.data))),
128    );
129  }
130
131  static Future<Either<String, AdminTeacherModel>> removeSubscription({
132    required int id,
133  }) async {
134    final response = await DioHelper.postData(
135      url: '$adminTeachersApi/$id/remove-subscription',
136    );
137
138    return response.fold(
139      (error) => Left(error),
140      (res) => Right(AdminTeacherModel.fromJson(_decodeData(res.data))),
141    );
142  }
143
144  static Future<Either<String, String>> resetPassword({
145    required int id,
146  }) async {
147    final response = await DioHelper.postData(
148      url: '$adminTeachersApi/$id/reset-password',
149    );
150
151    return response.fold(
152      (error) => Left(error),
153      (res) {
154        final data = _decodeData(res.data);
155        return Right(data['plain_password'] as String? ?? 'تمت إعادة التعيين بنجاح');
156      },
157    );
158  }
159
160  static Future<Either<String, bool>> deleteTeacher({
161    required int id,
162  }) async {
163    final response = await DioHelper.deleteData(
164      url: '$adminTeachersApi/$id',
165    );
166
167    return response.fold(
168      (error) => Left(error),
169      (res) => const Right(true),
170    );
171  }

```

```
172 }
173
```

Lines • Dart 224

Feature / Admin

#104 – lib/features/admin/teachers/presentation/cubit/admin_teachers_cubit.dart

```
1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/models/admin_teacher_model.dart';
4 import 'package:moean/features/admin/teachers/data/repositories/admin_teachers_repository.dart';
5 import 'package:moean/features/admin/teachers/presentation/cubit/admin_teachers_state.dart';
6
7 class AdminTeachersCubit extends Cubit<AdminTeachersState> {
8   AdminTeachersCubit() : super(AdminTeachersInitial());
9
10  static AdminTeachersCubit get(BuildContext context) => BlocProvider.of(context);
11
12  AdminTeacherPaginationModel? paginationModel;
13  List<AdminTeacherModel> teachersList = [];
14  List<AdminTeacherSubscriptionModel> subscriptionsList = [];
15
16  String? searchQuery;
17  String? statusFilter;
18  int currentPage = 1;
19  bool isLastPage = false;
20  int perPage = 20;
21
22  TextEditingController searchController = TextEditingController();
23
24  void getSubscriptions() async {
25    final result = await AdminTeachersRepository.getSubscriptions();
26    if (isClosed) return;
27    result.fold(
28      (error) {}, // Handle error if needed
29      (data) {
30        subscriptionsList = data;
31        if (!isClosed) emit(AdminTeacherFilterChangedState()); // Trigger a rebuild to update dialogs
32      },
33    );
34  }
35
36  void getTeachers({bool loadMore = false}) async {
37    if (subscriptionsList.isEmpty) getSubscriptions();
38    if (loadMore) {
39      if (isLastPage) return;
40      currentPage++;
41    } else {
42      currentPage = 1;
43      teachersList.clear();
44      if (!isClosed) emit(GetTeachersLoadingState());
45    }
46
47    final result = await AdminTeachersRepository.getTeachers(
48      page: currentPage,
49      perPage: perPage,
50      search: searchQuery,
51      status: statusFilter,
52    );
53    if (isClosed) return;
54
55    result.fold(
56      (error) {
57        if (!isClosed) emit(GetTeachersErrorState(error));
58      },
59      (data) {
60        paginationModel = data;
61        if (loadMore) {
62          teachersList.addAll(data.data);
63        } else {
64          teachersList = List.from(data.data);
65        }
66        isLastPage = data.currentPage >= data.lastPage;
67        if (!isClosed) emit(GetTeachersSuccessState(data));
68      },
69    );
70  }
71}
```

```

69     );
70 }
71
72 void onSearchChanged(String value) {
73     searchQuery = value.trim();
74     if (searchQuery!.isEmpty) {
75         searchQuery = null;
76     }
77     if (!isClosed) emit(AdminTeacherFilterChangedState());
78     getTeachers();
79 }
80
81 void onStatusFilterChanged(String? value) {
82     statusFilter = value;
83     if (!isClosed) emit(AdminTeacherFilterChangedState());
84     getTeachers();
85 }
86
87 Future<void> addTeacher({
88     required String name,
89     required String email,
90     required String phone,
91     required String password,
92     required int subscriptionId,
93 }) async {
94     if (!isClosed) emit(AdminTeacherActionLoadingState());
95     final result = await AdminTeachersRepository.createTeacher(
96         name: name,
97         email: email,
98         phone: phone,
99         password: password,
100         subscriptionId: subscriptionId,
101     );
102     if (isClosed) return;
103
104     result.fold(
105         (error) {
106             if (!isClosed) emit(AdminTeacherActionErrorState(error));
107         },
108         (teacher) {
109             if (!isClosed) emit(AdminTeacherActionSuccessState('تم إضافة المعلم بنجاح'));
110             getTeachers();
111         },
112     );
113 }
114
115 Future<void> updateTeacher({
116     required int id,
117     String? name,
118     String? email,
119     String? phone,
120     bool? active,
121     int? subscriptionId,
122     String? subscriptionEndsAt,
123     String? password,
124 }) async {
125     if (!isClosed) emit(AdminTeacherActionLoadingState());
126     final result = await AdminTeachersRepository.updateTeacher(
127         id: id,
128         name: name,
129         email: email,
130         phone: phone,
131         active: active,
132         subscriptionId: subscriptionId,
133         subscriptionEndsAt: subscriptionEndsAt,
134         password: password,
135     );
136     if (isClosed) return;
137
138     result.fold(
139         (error) {
140             if (!isClosed) emit(AdminTeacherActionErrorState(error));
141         },
142         (teacher) {
143             if (!isClosed) emit(AdminTeacherActionSuccessState('تم تحديث بيانات المعلم بنجاح'));

```

```

144         getTeachers();
145     },
146 );
147 }
148
149 Future<void> renewSubscription({
150     required int id,
151     int? subscriptionId,
152     int? months,
153 }) async {
154     if (!isClosed) emit(AdminTeacherActionLoadingState());
155     final result = await AdminTeachersRepository.renewSubscription(
156         id: id,
157         subscriptionId: subscriptionId,
158         months: months,
159     );
160     if (isClosed) return;
161
162     result.fold(
163         (error) {
164             if (!isClosed) emit(AdminTeacherActionErrorState(error));
165         },
166         (teacher) {
167             if (!isClosed) emit(AdminTeacherActionSuccessState('تم تجديد الاشتراك بنجاح'));
168             getTeachers();
169         },
170     );
171 }
172
173 Future<void> removeSubscription({required int id}) async {
174     if (!isClosed) emit(AdminTeacherActionLoadingState());
175     final result = await AdminTeachersRepository.removeSubscription(id: id);
176     if (isClosed) return;
177
178     result.fold(
179         (error) {
180             if (!isClosed) emit(AdminTeacherActionErrorState(error));
181         },
182         (teacher) {
183             if (!isClosed) emit(AdminTeacherActionSuccessState('تم إلغاء الاشتراك بنجاح'));
184             getTeachers();
185         },
186     );
187 }
188
189 Future<void> resetPassword({required int id}) async {
190     if (!isClosed) emit(AdminTeacherActionLoadingState());
191     final result = await AdminTeachersRepository.resetPassword(id: id);
192     if (isClosed) return;
193
194     result.fold(
195         (error) {
196             if (!isClosed) emit(AdminTeacherActionErrorState(error));
197         },
198         (password) {
199             if (!isClosed) emit(AdminTeacherPasswordResetSuccessState('تم إعادة تعيين كلمة المرور بنجاح', password, id));
200         },
201     );
202 }
203
204 Future<void> deleteTeacher({required int id}) async {
205     if (!isClosed) emit(AdminTeacherActionLoadingState());
206     final result = await AdminTeachersRepository.deleteTeacher(id: id);
207     if (isClosed) return;
208
209     result.fold(
210         (error) {
211             if (!isClosed) emit(AdminTeacherActionErrorState(error));
212         },
213         (success) {
214             if (!isClosed) emit(AdminTeacherActionSuccessState('تم حذف المعلم بنجاح'));
215             getTeachers();
216         },
217     );

```

```

218     }
219
220     Future<void> toggleTeacherStatus({required AdminTeacherModel teacher}) async {
221       await updateTeacher(id: teacher.id, active: !teacher.active);
222     }
223   }
224

```

Lines • Dart 44

Feature / Admin

#105 – lib/features/admin/teachers/presentation/cubit/admin_teachers_state.dart

```

1  import 'package:moean/core/models/admin_teacher_model.dart';
2
3  abstract class AdminTeachersState {}
4
5  class AdminTeachersInitial extends AdminTeachersState {}
6
7  class GetTeachersLoadingState extends AdminTeachersState {}
8
9  class GetTeachersSuccessState extends AdminTeachersState {
10     final AdminTeacherPaginationModel paginationModel;
11
12     GetTeachersSuccessState(this.paginationModel);
13 }
14
15 class GetTeachersErrorState extends AdminTeachersState {
16     final String message;
17
18     GetTeachersErrorState(this.message);
19 }
20
21 class AdminTeacherActionLoadingState extends AdminTeachersState {}
22
23 class AdminTeacherActionSuccessState extends AdminTeachersState {
24     final String message;
25
26     AdminTeacherActionSuccessState(this.message);
27 }
28
29 class AdminTeacherPasswordResetSuccessState extends AdminTeachersState {
30     final String message;
31     final String plainPassword;
32     final int teacherId;
33
34     AdminTeacherPasswordResetSuccessState(this.message, this.plainPassword, this.teacherId);
35 }
36
37 class AdminTeacherActionErrorState extends AdminTeachersState {
38     final String message;
39
40     AdminTeacherActionErrorState(this.message);
41 }
42
43 class AdminTeacherFilterChangedState extends AdminTeachersState {}
44

```

Lines • 239
Dart

Feature /
Admin

#106 –

lib/features/admin/teachers/presentation/screen/admin_teachers_screen.dart

```

1  import 'dart:math';
2  import 'package:flutter/material.dart';
3  import 'package:flutter/services.dart';
4  import 'package:flutter_bloc/flutter_bloc.dart';
5  import 'package:moean/core/theme/colors.dart';
6  import 'package:moean/core/theme/text_styles.dart';
7  import 'package:moean/core/utils/constants/constants.dart';
8  import 'package:moean/core/utils/constants/routes.dart';
9  import 'package:moean/core/utils/constants/spacing.dart';
10 import 'package:moean/core/utils/extensions/context_extension.dart';
11 import 'package:moean/features/admin/teachers/presentation/cubit/admin_teachers_cubit.dart';
12 import 'package:moean/features/admin/teachers/presentation/cubit/admin_teachers_state.dart';
13 import 'package:moean/features/admin/teachers/presentation/widgets/admin_stats_cards_widget.dart';
14 import 'package:moean/features/admin/teachers/presentation/widgets/admin_teacher_action_dialogs.dart';

```



```

15 import 'package:moean/features/admin/teachers/presentation/widgets/admin_teachers_search_filter_widget.dart';
16 import 'package:moean/features/admin/teachers/presentation/widgets/admin_teachers_table_widget.dart';
17
18 class AdminTeachersScreen extends StatelessWidget {
19   const AdminTeachersScreen({super.key});
20
21   @override
22   Widget build(BuildContext context) {
23     return BlocProvider(
24       create: (context) => AdminTeachersCubit()..getTeachers(),
25       child: BlocConsumer<AdminTeachersCubit, AdminTeachersState>(
26         listener: (context, state) {
27           if (state is AdminTeacherActionSuccessState) {
28             ScaffoldMessenger.of(context).showSnackBar(
29               SnackBar(content: Text(state.message), backgroundColor: Colors.green),
30             );
31           } else if (state is AdminTeacherPasswordResetSuccessState) {
32             showDialog(
33               context: context,
34               builder: (ctx) {
35                 final passwordController = TextEditingController(text: state.plainPassword);
36                 return AlertDialog(
37                   title: Text(appTranslation().get('new_password')),
38                   content: Column(
39                     mainAxisAlignment: MainAxisAlignment.min,
40                     crossAxisAlignment: CrossAxisAlignment.start,
41                     children: [
42                       Text(appTranslation().get('about_password')),
43                       verticalSpace12,
44                       Row(
45                         children: [
46                           Expanded(
47                             child: TextField(
48                               controller: passwordController,
49                               decoration: InputDecoration(
50                                 labelText: appTranslation().get('password'),
51                                 border: OutlineInputBorder(),
52                               ),
53                           ),
54                         ],
55                       horizontalSpace8,
56                       IconButton(
57                         onPressed: () {
58                           const chars = 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#\$%&*';
59
60                           final random = Random();
61                           final newPassword = String.fromCharCode(Iterable.generate(
62                             8, (_) => chars.codeUnitAt(random.nextInt(chars.length))));
63                           passwordController.text = newPassword;
64
65                           icon: const Icon(Icons.autorenew),
66                           tooltip: appTranslation().get('regenerate_password'),
67                         ),
68                       IconButton(
69                         onPressed: () {
70                           Clipboard.setData(ClipboardData(text: passwordController.text));
71                           ScaffoldMessenger.of(context).showSnackBar(
72                             SnackBar(content: Text(appTranslation().get('copy_success')), backgroundColo
73                             r: Colors.green),
74                           );
75                         },
76                       icon: const Icon(Icons.copy),
77                       tooltip: appTranslation().get('copy_password'),
78                     ],
79                   ),
80                 ],
81               ),
82               actions: [
83                 TextButton(
84                   onPressed: () => Navigator.pop(ctx),
85                   child: Text(appTranslation().get('close')),
86                 ),
87                 ElevatedButton(
88                   onPressed: () {

```

```

88         if (passwordController.text != state.plainPassword) {
89             AdminTeachersCubit.get(context).updateTeacher(
90                 id: state.teacherId,
91                 password: passwordController.text,
92             );
93         }
94         Navigator.pop(ctx);
95     },
96     child: Text(appTranslation().get('save_changes')),
97 ),
98 ],
99 );
100 },
101 );
102 } else if (state is AdminTeacherActionErrorState || state is GetTeachersErrorState) {
103     final msg = state is AdminTeacherActionErrorState
104         ? state.message
105         : (state as GetTeachersErrorState).message;
106     ScaffoldMessenger.of(context).showSnackBar(
107         SnackBar(content: Text(msg), backgroundColor: Colors.red),
108     );
109 }
110 },
111 builder: (context, state) {
112     final cubit = AdminTeachersCubit.get(context);
113     final scrollController = ScrollController();
114
115     // Infinite scrolling
116     scrollController.addListener(() {
117         if (scrollController.position.pixels >= scrollController.position.maxScrollExtent - 200) {
118             if (state is! GetTeachersLoadingState) {
119                 cubit.getTeachers(loadMore: true);
120             }
121         }
122     });
123
124     return Scaffold(
125         backgroundColor: ColorsManager.background,
126         appBar: AppBar(
127             backgroundColor: ColorsManager.primaryColor,
128             automaticallyImplyLeading: false,
129             title: Row(
130                 mainAxisAlignment: MainAxisAlignment.min,
131                 children: [
132                     const Icon(Icons.shield_outlined, color: Colors.white),
133                     horizontalSpace8,
134                     Flexible(
135                         child: Text(
136                             appTranslation().get('admin_teachers_title'),
137                             style: TextStyleManager.bold18.copyWith(color: Colors.white),
138                             overflow: TextOverflow.ellipsis,
139                         ),
140                     ),
141                 ],
142             ),
143         actions: [
144             IconButton(
145                 icon: const Icon(Icons.refresh, color: Colors.white),
146                 onPressed: () => cubit.getTeachers(),
147             ),
148             Padding(
149                 padding: const EdgeInsetsDirectional.only(end: 12.0),
150                 child: ElevatedButton.icon(
151                     style: ElevatedButton.styleFrom(
152                         backgroundColor: Colors.white,
153                         foregroundColor: ColorsManager.primaryColor,
154                         elevation: 0,
155                         padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
156                         minimumSize: Size.zero,
157                         tapTargetSize: MaterialTapTargetSize.shrinkWrap,
158                         shape: RoundedRectangleBorder(
159                             borderRadius: BorderRadius.circular(8),
160                         ),
161                     ),
162                     onPressed: () {

```

```

163         context.push(Routes.home);
164     },
165     icon: const Icon(Icons.home, size: 18),
166     label: Text(
167       appTranslation().get('home'),
168       style: TextStyleManager.medium14.copyWith(
169         color: ColorsManager.primaryColor,
170         fontWeight: FontWeight.bold,
171     ),
172   ),
173 ),
174 ),
175 ],
176 ),
177 body: SafeArea(
178   child: Padding(
179     padding: const EdgeInsets.all(16.0),
180     child: Column(
181       children: [
182         const AdminTeachersSearchFilterWidget(),
183
184         verticalSpace24,
185         const AdminStatsCardsWidget(),
186
187         verticalSpace24,
188         Row(
189           mainAxisAlignment: MainAxisAlignment.end,
190           children: [
191             // ElevatedButton.icon(
192             //   style: ElevatedButton.styleFrom(
193             //     backgroundColor: ColorsManager.secondaryColor,
194             //     foregroundColor: Colors.white,
195             //   ),
196             //   onPressed: () => context.push(Routes.adminPayments),
197             //   icon: const Icon(Icons.payments_outlined),
198             //   label: Text(appTranslation().get('admin_payments_title') ?? 'Payments'),
199             // ),
200             // horizontalSpace8,
201             ElevatedButton.icon(
202               style: ElevatedButton.styleFrom(
203                 backgroundColor: ColorsManager.primaryColor,
204                 foregroundColor: Colors.white,
205               ),
206               onPressed: () => showAddTeacherDialog(context),
207               icon: const Icon(Icons.add),
208               label: Text(appTranslation().get('admin_add_teacher')),
209             ),
210           ],
211         ),
212         verticalSpace16,
213         Expanded(
214           child: Container(
215             clipBehavior: Clip.hardEdge,
216             decoration: BoxDecoration(
217               color: Theme.of(context).cardColor,
218               borderRadius: BorderRadius.circular(12),
219               border: Border.all(color: Colors.grey.withValues(alpha: 0.2)),
220             ),
221             child: state is GetTeachersLoadingState && cubit.teachersList.isEmpty
222               ? const Center(child: CircularProgressIndicator())
223               : AdminTeachersTableWidget(
224                 teachers: cubit.teachersList,
225                 scrollController: scrollController,
226               ),
227           ),
228         ),
229       ],
230     ),
231   ),
232 ),
233 );
234 },
235 ),
236 );
237 }

```

238 }
239

Lines • 116
Dart

Feature / #107 -

Admin lib/features/admin/teachers/presentation/widgets/admin_stats_cards_widget.dart

```
1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/spacing.dart';
7 import 'package:moean/features/admin/teachers/presentation/cubit/admin_teachers_cubit.dart';
8 import 'package:moean/features/admin/teachers/presentation/cubit/admin_teachers_state.dart';
9
10 class AdminStatsCardsWidget extends StatelessWidget {
11   const AdminStatsCardsWidget({super.key});
12
13   @override
14   Widget build(BuildContext context) {
15     return BlocBuilder<AdminTeachersCubit, AdminTeachersState>(
16       builder: (context, state) {
17         final cubit = AdminTeachersCubit.get(context);
18         final total = cubit.paginationModel?.total ?? 0;
19
20         // Since we don't have a dedicated stats API, we'll calculate from loaded data
21         // or display the total for the first card as an example.
22         final activeCount = cubit.teachersList.where((t) => t.active).length;
23         final suspendedCount = cubit.teachersList.where((t) => !t.active).length;
24         final subscribedCount = cubit.teachersList.where((t) => t.subscription?.slug != 'free').length;
25
26         return LayoutBuilder(
27           builder: (context, constraints) {
28             final isMobile = constraints.maxWidth < 600;
29             return GridView.count(
30               crossAxisCount: isMobile ? 2 : 4,
31               shrinkWrap: true,
32               physics: const NeverScrollableScrollPhysics(),
33               crossAxisSpacing: 12,
34               mainAxisSpacing: 12,
35               childAspectRatio: isMobile ? 1.8 : 2.5,
36               children: [
37                 _buildStatCard(
38                   title: appTranslation().get('admin_total_teachers'),
39                   count: total,
40                   icon: Icons.people_outline,
41                   color: Colors.red,
42                   bgColor: Colors.red.withValues(alpha: 0.1),
43                 ),
44                 _buildStatCard(
45                   title: appTranslation().get('admin_active_teachers'),
46                   count: activeCount,
47                   icon: Icons.person_outline,
48                   color: Colors.purple,
49                   bgColor: Colors.purple.withValues(alpha: 0.1),
50                 ),
51                 _buildStatCard(
52                   title: appTranslation().get('admin_suspended_accounts'),
53                   count: suspendedCount,
54                   icon: Icons.person_off_outlined,
55                   color: ColorsManager.primaryColor,
56                   bgColor: ColorsManager.primaryColor.withValues(alpha: 0.1),
57                 ),
58                 _buildStatCard(
59                   title: appTranslation().get('admin_subscribed_plans'),
60                   count: subscribedCount,
61                   icon: Icons.bolt_outlined,
62                   color: Colors.green,
63                   bgColor: Colors.green.withValues(alpha: 0.1),
64                 ),
65               ],
66             );
67           },
68         );
69     }
70   }
71 }
```

```

68     );
69   },
70 );
71 }
72
73 Widget _buildStatCard({
74   required String title,
75   required int count,
76   required IconData icon,
77   required Color color,
78   required Color bgColor,
79 }) {
80   return Container(
81     padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 10),
82     decoration: BoxDecoration(
83       color: bgColor,
84       borderRadius: BorderRadius.circular(12),
85       border: Border.all(color: color.withValues(alpha: 0.3)),
86     ),
87     child: Row(
88       mainAxisAlignment: MainAxisAlignment.spaceBetween,
89       children: [
90         Expanded(
91           child: Column(
92             crossAxisAlignment: CrossAxisAlignment.start,
93             mainAxisAlignment: MainAxisAlignment.center,
94             children: [
95               Text(
96                 count.toString(),
97                 style: TextStylesManager.bold20.copyWith(color: color),
98               ),
99               verticalSpace4,
100              Text(
101                title,
102                style: TextStylesManager.medium12.copyWith(color: color, height: 1.2),
103                maxLines: 2,
104                overflow: TextOverflow.ellipsis,
105              ),
106            ],
107          ),
108        ),
109        horizontalSpace4,
110        Icon(icon, color: color, size: 26),
111      ],
112    ),
113 );
114 }
115 }
116

```

Lines • 465

Dart

Feature / #108 -

Admin [lib/features/admin/teachers/presentation/widgets/admin_teacher_action_dialogs.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter/services.dart';
3  import 'package:moean/core/models/admin_teacher_model.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7  import 'package:moean/features/admin/teachers/presentation/cubit/admin_teachers_cubit.dart';
8  import 'package:moean/core/theme/colors.dart';
9
10 void showAddTeacherDialog(BuildContext context) {
11   final cubit = AdminTeachersCubit.get(context);
12   final nameController = TextEditingController();
13   final emailController = TextEditingController();
14   final phoneController = TextEditingController();
15   final passwordController = TextEditingController();
16   int subscriptionId = 1; // Default free
17
18   showDialog(
19     context: context,
20     builder: (dialogContext) {

```

```

21 return AlertDialog(
22   title: Text(appTranslation().get('admin_add_teacher')),
23   content: SingleChildScrollView(
24     child: Column(
25       mainAxisAlignment: MainAxisAlignment.min,
26       children: [
27         TextField(
28           controller: nameController,
29           decoration: InputDecoration(labelText: appTranslation().get('full_name')),
30         ),
31         verticalSpace12,
32         TextField(
33           controller: emailController,
34           decoration: InputDecoration(labelText: appTranslation().get('admin_email')),
35         ),
36         verticalSpace12,
37         TextField(
38           controller: phoneController,
39           keyboardType: TextInputType.phone,
40           inputFormatters: [FilteringTextInputFormatter.digitsOnly],
41           decoration: InputDecoration(labelText: appTranslation().get('admin_phone')),
42         ),
43         verticalSpace12,
44         TextField(
45           controller: passwordController,
46           decoration: InputDecoration(labelText: appTranslation().get('password')),
47           obscureText: true,
48         ),
49       ],
50     ),
51   ),
52   actions: [
53     TextButton(
54       onPressed: () => Navigator.pop(dialogContext),
55       child: Text(appTranslation().get('cancel')),
56     ),
57     ElevatedButton(
58       onPressed: () {
59         cubit.addTeacher(
60           name: nameController.text,
61           email: emailController.text,
62           phone: phoneController.text,
63           password: passwordController.text,
64           subscriptionId: subscriptionId,
65         );
66         Navigator.pop(dialogContext);
67       },
68       child: Text(appTranslation().get('admin_add_teacher')),
69     ),
70   ],
71 );
72 },
73 );
74 }
75
76 void showEditTeacherDialog(BuildContext context, AdminTeacherModel teacher) {
77   final cubit = AdminTeachersCubit.get(context);
78   final nameController = TextEditingController(text: teacher.user.name);
79   final phoneController = TextEditingController(text: teacher.user.phone ?? '');
80   final endDateController = TextEditingController(text: teacher.subscriptionEndsAt ?? '');
81
82   int? selectedPlanId = teacher.subscription?.id;
83   bool isActive = teacher.active;
84
85   showDialog(
86     context: context,
87     builder: (dialogContext) {
88       return StatefulBuilder(
89         builder: (context, setState) {
90           return Dialog(
91             shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
92             child: Container(
93               width: 600,
94               padding: const EdgeInsets.all(24),
95               child: SingleChildScrollView(

```

```

96         child: Column(
97             mainAxisAlignment: MainAxisAlignment.min,
98             crossAxisAlignment: CrossAxisAlignment.start,
99             children: [
100                 // Header
101                 Row(
102                     mainAxisAlignment: MainAxisAlignment.spaceBetween,
103                     children: [
104                         Row(
105                             children: [
106                                 Container(
107                                     padding: const EdgeInsets.all(12),
108                                     decoration: const BoxDecoration(
109                                         color: Color(0xFFECA332), // orange from screenshot
110                                         shape: BoxShape.circle,
111                                     ),
112                                     child: const Icon(Icons.edit, color: Colors.white),
113                                 ),
114                                 horizontalSpace16,
115                                 Column(
116                                     crossAxisAlignment: CrossAxisAlignment.start,
117                                     children: [
118                                         Text('تعديل بيانات المعلم', style: TextStyleManager.bold18.copyWith(color: ColorsManager.primaryColor)),
119                                         verticalSpace4,
120                                         Text(teacher.user.email, style: TextStyleManager.medium14.copyWith(color: ColorsManager.secondaryText)),
121                                     ],
122                                 ),
123                             ],
124                         ),
125                         IconButton(
126                             icon: const Icon(Icons.close),
127                             onPressed: () => Navigator.pop(dialogContext),
128                         ),
129                     ],
130                 ),
131                 const Divider(height: 32),
132
133                 // Name
134                 Text('الاسم الكامل *', style: TextStyleManager.bold14.copyWith(color: ColorsManager.primaryColor)),
135                 verticalSpace8,
136                 TextField(
137                     controller: nameController,
138                     decoration: InputDecoration(
139                         border: OutlineInputBorder(borderRadius: BorderRadius.circular(12), borderSide: BorderSide(color: ColorsManager.borderColor)),
140                         enabledBorder: OutlineInputBorder(borderRadius: BorderRadius.circular(12), borderSide: BorderSide(color: ColorsManager.borderColor)),
141                         contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
142                     ),
143                 ),
144                 verticalSpace16,
145
146                 // Row for Plan and Phone
147                 Row(
148                     crossAxisAlignment: CrossAxisAlignment.start,
149                     children: [
150                         Expanded(
151                             child: Column(
152                                 crossAxisAlignment: CrossAxisAlignment.start,
153                                 children: [
154                                     Text('خطه الاشتراك', style: TextStyleManager.bold14.copyWith(color: ColorsManager.primaryColor)),
155                                     verticalSpace8,
156                                     DropdownButtonFormField<int?>(
157                                         initialValue: selectedPlanId,
158                                         isExpanded: true,
159                                         style: TextStyleManager.medium14.copyWith(color: ColorsManager.mainText),
160                                         decoration: InputDecoration(
161                                             border: OutlineInputBorder(borderRadius: BorderRadius.circular(12), borderSide: BorderSide(color: ColorsManager.borderColor)),
162                                             enabledBorder: OutlineInputBorder(borderRadius: BorderRadius.circular(12), borderSide: BorderSide(color: ColorsManager.borderColor)),

```

```

163         contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
164     ),
165     items: [
166         const DropdownMenuItem(value: null, child: Text('بدون خطة', overflow: TextO
verflow.ellipsis)),
167         ...cubit.subscriptionsList.map((sub) => DropdownMenuItem(
168             value: sub.id,
169             child: Text(sub.name, overflow: TextOverflow.ellipsis),
170         )),
171         if (selectedPlanId != null && !cubit.subscriptionsList.any((sub) => sub.id
== selectedPlanId))
172             DropdownMenuItem(
173                 value: selectedPlanId,
174                 child: Text(teacher.subscription?.name ?? 'خطة ($selectedPlanId)', over
flow: TextOverflow.ellipsis),
175             ),
176     ],
177     onChanged: (val) {
178         setState(() {
179             selectedPlanId = val;
180         });
181     },
182 ),
183 ],
184 ),
185 ),
186 horizontalSpace16,
187 Expanded(
188     child: Column(
189         crossAxisAlignment: CrossAxisAlignment.start,
190         children: [
191             Text('رقم الجوال', style: TextStyleManager.bold14.copyWith(color: ColorsManage
r.primaryColor)),
192             verticalSpace8,
193             TextField(
194                 controller: phoneController,
195                 keyboardType: TextInputType.phone,
196                 inputFormatters: [FilteringTextInputFormatter.digitsOnly],
197                 decoration: InputDecoration(
198                     border: OutlineInputBorder(borderRadius: BorderRadius.circular(12), borderS
ide: BorderSide(color: ColorsManager.borderColor)),
199                     enabledBorder: OutlineInputBorder(borderRadius: BorderRadius.circular(12),
borderSide: BorderSide(color: ColorsManager.borderColor)),
200                     contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
201                 ),
202             ),
203         ],
204     ),
205 ),
206 ],
207 ),
208 verticalSpace16,
209
210 // Row for Quick Actions and End Date
211 Row(
212     crossAxisAlignment: CrossAxisAlignment.start,
213     children: [
214         Expanded(
215             child: Column(
216                 crossAxisAlignment: CrossAxisAlignment.start,
217                 children: [
218                     Text('إجراءات سريعة للاشتراك', style: TextStyleManager.bold14.copyWith(color: C
olorsManager.primaryColor)),
219                     verticalSpace8,
220                     Row(
221                         children: [
222                             Expanded(
223                                 child: OutlinedButton(
224                                     onPressed: () {
225                                         cubit.renewSubscription(id: teacher.id, months: 6);
226                                         Navigator.pop(dialogContext);
227                                     },
228                                     style: OutlinedButton.styleFrom(
229                                         padding: const EdgeInsets.symmetric(vertical: 12),
230                                         shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(1

```



```

2)),
231 |                 side: BorderSide(color: ColorsManager.primaryColor.withValues(alpha:
0.3)),
232 |                 backgroundColor: ColorsManager.isDark ? ColorsManager.primaryColor.wi
thValues(alpha: 0.1) : const Color(0xFFE8F5E9),
233 |             ),
234 |             child: FittedBox(child: Text('أشهرنا (6+)' , textAlign: TextAlign.c
enter, style: TextStyle(color: ColorsManager.primaryColor, fontSize: 12, fontWeight: FontWeight.bold))),
235 |         ),
236 |     ),
237 |     horizontalSpace8,
238 |     Expanded(
239 |       child: OutlinedButton(
240 |         onPressed: () {
241 |           cubit.removeSubscription(id: teacher.id);
242 |           Navigator.pop(dialogContext);
243 |         },
244 |         style: OutlinedButton.styleFrom(
245 |           padding: const EdgeInsets.symmetric(vertical: 12),
246 |           shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(1
2)),
247 |           side: BorderSide(color: Colors.red.withValues(alpha: 0.3)),
248 |           backgroundColor: ColorsManager.isDark ? Colors.red.withValues(alpha:
0.1) : const Color(0xFFFFFEBEE),
249 |         ),
250 |         child: const FittedBox(child: Text('إلغاء الاشتراك' , textAlign: TextAlig
n.center, style: TextStyle(color: Colors.red, fontSize: 12, fontWeight: FontWeight.bold))),
251 |       ),
252 |     ),
253 |   ],
254 | ),
255 | ],
256 | ),
257 | ),
258 | horizontalSpace16,
259 | Expanded(
260 |   child: Column(
261 |     crossAxisAlignment: CrossAxisAlignment.start,
262 |     children: [
263 |       Text(appTranslation().get("expire_date"), style: TextStylesManager.bold14.copyWith
with(color: ColorsManager.primaryColor)),
264 |       verticalSpace8,
265 |       TextField(
266 |         controller: endDateController,
267 |         readOnly: true,
268 |         onTap: () async {
269 |           final date = await showDatePicker(
270 |             context: context,
271 |             initialDate: DateTime.now(),
272 |             firstDate: DateTime(2020),
273 |             lastDate: DateTime(2030),
274 |           );
275 |           if (date != null) {
276 |             setState(() {
277 |               endDateController.text = "${date.year}-${date.month.toString().padLeft
(2, '0')}-${date.day.toString().padLeft(2, '0')}";
278 |             });
279 |           }
280 |         },
281 |         decoration: InputDecoration(
282 |           hintText: 'mm/dd/yyyy',
283 |           suffixIcon: const Icon(Icons.calendar_today, size: 20),
284 |           border: OutlineInputBorder(borderRadius: BorderRadius.circular(12), borderS
ide: BorderSide(color: ColorsManager.borderColor)),
285 |           enabledBorder: OutlineInputBorder(borderRadius: BorderRadius.circular(12),
borderSide: BorderSide(color: ColorsManager.borderColor)),
286 |           contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
287 |         ),
288 |       ),
289 |     ],
290 |   ),
291 | ),
292 | ],
293 | ),
294 | verticalSpace16,

```

```

295
296 // Account Status
297 Container(
298   padding: const EdgeInsets.all(16),
299   decoration: BoxDecoration(
300     color: ColorsManager.isDark ? ColorsManager.surfaceDark : const Color(0xFFFF8FBF8),
301     borderRadius: BorderRadius.circular(12),
302     border: Border.all(color: ColorsManager.borderColor),
303   ),
304   child: Row(
305     mainAxisAlignment: MainAxisAlignment.spaceBetween,
306     children: [
307       Column(
308         crossAxisAlignment: CrossAxisAlignment.start,
309         children: [
310           Text(appTranslation().get('account_status'), style: TextStyleManager.bold16.co
pyWith(color: ColorsManager.primaryColor)),
311           verticalSpace4,
312           Text('الحساب نشط ويمكن تسجيل الدخول', style: TextStyleManager.medium12.copyWith(color: ColorsManager.secondaryText)),
313         ],
314       ),
315       Switch(
316         value: isActive,
317         onChanged: (val) {
318           setState(() {
319             isActive = val;
320           });
321         },
322         activeThumbColor: const Color(0xFF00B778),
323       ),
324     ],
325   ),
326 ),
327 verticalSpace24,
328
329 // Actions
330 Row(
331   children: [
332     Expanded(
333       child: ElevatedButton(
334         onPressed: () {
335           cubit.updateTeacher(
336             id: teacher.id,
337             name: nameController.text.isNotEmpty ? nameController.text : null,
338             phone: phoneController.text.isNotEmpty ? phoneController.text : null,
339             subscriptionId: selectedPlanId,
340             active: isActive,
341             subscriptionEndsAt: endDateController.text.isNotEmpty ? endDateController.tex
t : null,
342           );
343           Navigator.pop(dialogContext);
344         },
345         style: ElevatedButton.styleFrom(
346           backgroundColor: const Color(0xFFECA332),
347           padding: const EdgeInsets.symmetric(vertical: 16),
348           shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
349         ),
350         child: Row(
351           mainAxisAlignment: MainAxisAlignment.center,
352           children: [
353             Icon(Icons.check, color: Colors.white, size: 20),
354             horizontalSpace8,
355             Text(appTranslation().get('save_changes'), style: TextStyle(color: Colors.whi
te, fontSize: 16, fontWeight: FontWeight.bold)),
356           ],
357         ),
358       ),
359     ),
360     horizontalSpace16,
361     Expanded(
362       child: OutlinedButton(
363         onPressed: () => Navigator.pop(dialogContext),
364         style: OutlinedButton.styleFrom(
365           padding: const EdgeInsets.symmetric(vertical: 16),

```

```

366         shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
367         side: BorderSide(color: ColorsManager.borderColor),
368     ),
369     child: Text(appTranslation().get('close'), style: TextStyle(color: ColorsManager.
mainText, fontSize: 16, fontWeight: FontWeight.bold)),
370     ),
371   ),
372 ],
373 ),
374 ],
375 ),
376 ),
377 ),
378 );
379 }
380 );
381 },
382 );
383 }
384
385 void showResetPasswordDialog(BuildContext context, AdminTeacherModel teacher) {
386   final cubit = AdminTeachersCubit.get(context);
387   showDialog(
388     context: context,
389     builder: (dialogContext) {
390       return AlertDialog(
391         title: Text(appTranslation().get('admin_reset_password')),
392         content: Text('هل أنت متأكد من إعادة تعيين كلمة المرور لـ ${teacher.user.name}?'),
393         actions: [
394           TextButton(
395             onPressed: () => Navigator.pop(dialogContext),
396             child: Text(appTranslation().get('cancel')),
397           ),
398           ElevatedButton(
399             onPressed: () {
400               cubit.resetPassword(id: teacher.id);
401               Navigator.pop(dialogContext);
402             },
403             child: Text(appTranslation().get('admin_reset_password')),
404           ),
405         ],
406       );
407     },
408   );
409 }
410
411 void showToggleStatusDialog(BuildContext context, AdminTeacherModel teacher) {
412   final cubit = AdminTeachersCubit.get(context);
413   final actionStr = teacher.active ? appTranslation().get('admin_suspend') : appTranslation().get('admin_acti
vate');
414   showDialog(
415     context: context,
416     builder: (dialogContext) {
417       return AlertDialog(
418         title: Text(actionStr),
419         content: Text('هل أنت متأكد أنك تريد $actionStr ${teacher.user.name}?'),
420         actions: [
421           TextButton(
422             onPressed: () => Navigator.pop(dialogContext),
423             child: Text(appTranslation().get('cancel')),
424           ),
425           ElevatedButton(
426             onPressed: () {
427               cubit.toggleTeacherStatus(teacher: teacher);
428               Navigator.pop(dialogContext);
429             },
430             style: ElevatedButton.styleFrom(backgroundColor: teacher.active ? Colors.red : Colors.green),
431             child: Text(actionStr),
432           ),
433         ],
434       );
435     },
436   );
437 }
438

```

```

439 void showDeleteTeacherDialog(BuildContext context, AdminTeacherModel teacher) {
440   final cubit = AdminTeachersCubit.get(context);
441   showDialog(
442     context: context,
443     builder: (dialogContext) {
444       return AlertDialog(
445         title: Text(appTranslation().get('admin_delete')),
446         content: Text(appTranslation().get('admin_confirm_delete')),
447         actions: [
448           TextButton(
449             onPressed: () => Navigator.pop(dialogContext),
450             child: Text(appTranslation().get('cancel')),
451           ),
452           ElevatedButton(
453             onPressed: () {
454               cubit.deleteTeacher(id: teacher.id);
455               Navigator.pop(dialogContext);
456             },
457             style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
458             child: Text(appTranslation().get('admin_delete')),
459           ),
460         ],
461       );
462     },
463   );
464 }
465

```

Lines 106

• Dart

Feature / #109 -

Admin [lib/features/admin/teachers/presentation/widgets/admin_teachers_search_filter_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
7  import 'package:moean/core/utils/constants/spacing.dart';
8  import 'package:moean/core/utils/cubit/theme/theme_cubit.dart';
9  import 'package:moean/core/utils/cubit/theme/theme_state.dart';
10 import 'package:moean/features/admin/teachers/presentation/cubit/admin_teachers_cubit.dart';
11
12 class AdminTeachersSearchFilterWidget extends StatelessWidget {
13   const AdminTeachersSearchFilterWidget({super.key});
14
15   @override
16   Widget build(BuildContext context) {
17     final cubit = AdminTeachersCubit.get(context);
18
19     return BlocBuilder<ThemeCubit, ThemeState>(
20       builder: (context, state) {
21         return Row(
22           children: [
23             Expanded(
24               flex: 5,
25               child: PrimaryTextField(
26                 controller: cubit.searchController,
27                 onChanged: cubit.onSearchChanged,
28                 prefixIcon: const Icon(Icons.search, color: Colors.grey),
29                 hint: appTranslation().get('admin_search_hint'),
30               ),
31             ),
32             horizontalSpace8,
33             Expanded(
34               flex: 2,
35               child: Container(
36                 height: 48,
37                 padding: const EdgeInsets.symmetric(horizontal: 10),
38                 decoration: BoxDecoration(
39                   color: ColorsManager.surfacePrimary,
40                   borderRadius: BorderRadius.circular(24),
41                   border: Border.all(color: ColorsManager.borderColor),
42                 ),

```

```

43         child: DropdownButtonHideUnderline(
44             child: DropdownButton<String>(
45                 dropdownColor: ColorsManager.surfacePrimary,
46                 value: cubit.statusFilter,
47                 hint: Text(
48                     appTranslation().get('admin_all_status'),
49                     style: TextStylesManager.regular14.copyWith(
50                         color: ColorsManager.textSecondary.withValues(alpha: 0.85),
51                     ),
52                 overflow: TextOverflow.ellipsis,
53                 maxLines: 1,
54             ),
55             isExpanded: true,
56             icon: Icon(Icons.arrow_drop_down, color: ColorsManager.textPrimary, size: 20),
57             style: TextStylesManager.regular14.copyWith(
58                 color: ColorsManager.textPrimary,
59             ),
60             items: [
61                 DropdownMenuItem(
62                     value: null,
63                     child: Text(
64                         appTranslation().get('admin_all_status'),
65                         style: TextStylesManager.regular14.copyWith(
66                             color: ColorsManager.textPrimary,
67                         ),
68                     overflow: TextOverflow.ellipsis,
69                     maxLines: 1,
70                 ),
71             ],
72                 DropdownMenuItem(
73                     value: 'active',
74                     child: Text(
75                         appTranslation().get('admin_active'),
76                         style: TextStylesManager.regular14.copyWith(
77                             color: ColorsManager.textPrimary,
78                         ),
79                     overflow: TextOverflow.ellipsis,
80                     maxLines: 1,
81                 ),
82             ),
83                 DropdownMenuItem(
84                     value: 'inactive',
85                     child: Text(
86                         appTranslation().get('admin_suspended'),
87                         style: TextStylesManager.regular14.copyWith(
88                             color: ColorsManager.textPrimary,
89                         ),
90                     overflow: TextOverflow.ellipsis,
91                     maxLines: 1,
92                 ),
93             ],
94         ],
95         onChanged: cubit.onStatusFilterChanged,
96     ),
97 ),
98 ),
99 ),
100 ],
101 );
102 },
103 );
104 }
105 }
106

```

Lines • 800
Dart

Feature / #110 –
Admin

[lib/features/admin/teachers/presentation/widgets/admin_teachers_table_widget.dart](#)

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/models/admin_teacher_model.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/constants.dart';

```

```

6 import 'package:moean/core/utils/constants/spacing.dart';
7 import 'package:moean/features/admin/teachers/presentation/widgets/admin_teacher_action_dialogs.dart';
8
9 class AdminTeachersTableWidget extends StatefulWidget {
10   final List<AdminTeacherModel> teachers;
11   final ScrollController scrollController;
12
13   const AdminTeachersTableWidget({
14     super.key,
15     required this.teachers,
16     required this.scrollController,
17   });
18
19   @override
20   State<AdminTeachersTableWidget> createState() => _AdminTeachersTableWidgetState();
21 }
22
23 class _AdminTeachersTableWidgetState extends State<AdminTeachersTableWidget> {
24   final ScrollController _horizontalController = ScrollController();
25
26   static const _arabicMonths = [
27     'يناير',
28     'فبراير',
29     'مارس',
30     'أبريل',
31     'مايو',
32     'يونيو',
33     'يوليو',
34     'أغسطس',
35     'سبتمبر',
36     'أكتوبر',
37     'نوفمبر',
38     'ديسمبر'
39   ];
40
41   static String _getMonthName(int month) {
42     if (month >= 1 && month <= 12) {
43       return _arabicMonths[month - 1];
44     }
45     return '$month';
46   }
47
48   @override
49   void dispose() {
50     _horizontalController.dispose();
51     super.dispose();
52   }
53
54   @override
55   Widget build(BuildContext context) {
56     if (widget.teachers.isEmpty) {
57       return const Center(
58         child: Padding(
59           padding: EdgeInsets.all(32.0),
60           child: Text('لا توجد بيانات'),
61         ),
62       );
63     }
64
65     final isDark = Theme.of(context).brightness == Brightness.dark;
66     final headerColor = isDark ? const Color(0xFF2C2C2E) : Colors.grey.withValues(alpha: 0.1);
67     final borderColor = isDark ? Colors.grey[800]! : Colors.grey.withValues(alpha: 0.2);
68
69     const double colTeacher = 190;
70     const double colPhone = 120;
71     const double colPlan = 175;
72     const double colStatus = 95;
73     const double colDate = 100;
74     const double colActions = 140;
75     const double totalWidth = colTeacher + colPhone + colPlan + colStatus + colDate + colActions;
76
77     return Scrollbar(
78       controller: _horizontalController,
79       thumbVisibility: true,
80       child: SingleChildScrollView(

```

```

81         controller: _horizontalController,
82         scrollDirection: Axis.horizontal,
83         child: SizedBox(
84             width: totalWidth,
85             child: Column(
86                 children: [
87                     // Header Row
88                     Container(
89                         height: 50,
90                         decoration: BoxDecoration(
91                             color: headerColor,
92                             border: Border(bottom: BorderSide(color: borderColor)),
93                     ),
94                     child: Row(
95                         children: [
96                             _buildHeaderCell(appTranslation().get('admin_teacher'), colTeacher),
97                             _buildHeaderCell(appTranslation().get('admin_phone'), colPhone),
98                             _buildHeaderCell(appTranslation().get('admin_plan'), colPlan, alignment: AlignmentDirectional.center),
99                             _buildHeaderCell(appTranslation().get('admin_status'), colStatus, alignment: AlignmentDirectional.center),
100                             _buildHeaderCell(appTranslation().get('admin_join_date'), colDate, alignment: AlignmentDirectional.center),
101                             _buildHeaderCell(appTranslation().get('admin_actions'), colActions, alignment: AlignmentDirectional.center),
102                         ],
103                     ),
104                 ],
105                 // Body
106                 Expanded(
107                     child: ListView.separated(
108                         controller: widget.scrollController,
109                         physics: const AlwaysScrollableScrollPhysics(),
110                         itemCount: widget.teachers.length,
111                         separatorBuilder: (context, index) => Divider(color: borderColor, height: 1),
112                         itemBuilder: (context, index) {
113                             return _buildDataRow(context, widget.teachers[index], colTeacher, colPhone, colPlan, colStatus, colDate, colActions);
114                         },
115                     ),
116                 ),
117             ],
118         ),
119     ),
120 ],
121 );
122 }
123
124 Widget _buildHeaderCell(String text, double width, {AlignmentDirectional alignment = AlignmentDirectional.centerStart}) {
125     return SizedBox(
126         width: width,
127         child: Padding(
128             padding: const EdgeInsets.symmetric(horizontal: 10.0),
129             child: Align(
130                 alignment: alignment,
131                 child: Text(text, style: TextStyleManager.bold14),
132             ),
133         ),
134     );
135 }
136
137 Widget _buildDataRow(
138     BuildContext context,
139     AdminTeacherModel teacher,
140     double colTeacher,
141     double colPhone,
142     double colPlan,
143     double colStatus,
144     double colDate,
145     double colActions,
146 ) {
147     return SizedBox(
148         height: 76,
149         child: Row(

```

```

150     children: [
151       // Teacher Name & Email
152       _buildTeacherCell(context, teacher, colTeacher),
153       // Phone
154       SizedBox(
155         width: colPhone,
156         child: Padding(
157           padding: const EdgeInsets.symmetric(horizontal: 8.0),
158           child: Align(
159             alignment: AlignmentDirectional.centerStart,
160             child: Text(teacher.user.phone ?? '-', style: TextStyleManager.medium14),
161           ),
162         ),
163       ),
164       // Plan & Expiry
165       _buildPlanAndExpiryCell(context, teacher, colPlan),
166       // Status
167       SizedBox(
168         width: colStatus,
169         child: Padding(
170           padding: const EdgeInsets.symmetric(horizontal: 4.0),
171           child: Align(
172             alignment: Alignment.center,
173             child: Container(
174               padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
175               decoration: BoxDecoration(
176                 color: teacher.active ? Colors.green.withValues(alpha: 0.1) : Colors.red.withValues(alpha: 0.1),
177                 borderRadius: BorderRadius.circular(16),
178               ),
179               child: Row(
180                 mainAxisAlignment: MainAxisAlignment.min,
181                 children: [
182                   Icon(Icons.circle, size: 7, color: teacher.active ? Colors.green : Colors.red),
183                   horizontalSpace4,
184                   Text(
185                     teacher.active ? appTranslation().get('admin_active') : appTranslation().get('admin_suspended'),
186                     style: TextStyleManager.medium12.copyWith(
187                       color: teacher.active ? Colors.green : Colors.red,
188                     ),
189                 ),
190               ],
191             ),
192           ),
193         ),
194       ),
195     ],
196     // Join Date
197     _buildJoinDateCell(context, teacher.user.createdAt, colDate),
198     // Actions
199     _buildActionsCell(context, teacher, colActions),
200   ],
201 ),
202 );
203 }
204
205 Widget _buildTeacherCell(BuildContext context, AdminTeacherModel teacher, double width) {
206   final avatarLetter = teacher.user.name.isNotEmpty ? teacher.user.name.substring(0, 1) : '?';
207
208   return SizedBox(
209     width: width,
210     child: Padding(
211       padding: const EdgeInsets.symmetric(horizontal: 10.0),
212       child: Row(
213         children: [
214           CircleAvatar(
215             radius: 16,
216             backgroundColor: ColorsManager.primaryColor.withValues(alpha: 0.2),
217             child: Text(avatarLetter, style: TextStyle(color: ColorsManager.primaryColor, fontWeight: FontWeight.bold, fontSize: 13)),
218           ),
219           horizontalSpace8,
220           Expanded(
221             child: Column(

```



```

222         mainAxisAlignment: MainAxisAlignment.center,
223         crossAxisAlignment: CrossAxisAlignment.start,
224         children: [
225             Text(
226                 teacher.user.name,
227                 style: TextStylesManager.medium14,
228                 overflow: TextOverflow.ellipsis,
229                 maxLines: 1,
230             ),
231             Text(
232                 teacher.user.email,
233                 style: TextStylesManager.regular12.copyWith(color: Colors.grey, fontSize: 11),
234                 overflow: TextOverflow.ellipsis,
235                 maxLines: 1,
236             ),
237         ],
238     ),
239 ),
240 ],
241 ),
242 ),
243 );
244 }
245
246 Widget _buildJoinDateCell(BuildContext context, String? createdAt, double width) {
247     if (createdAt == null || createdAt.isEmpty) {
248         return SizedBox(
249             width: width,
250             child: const Center(child: Text('-', style: TextStyle(color: Colors.grey))),
251         );
252     }
253     final dt = DateTime.tryParse(createdAt);
254     if (dt == null) {
255         return SizedBox(
256             width: width,
257             child: Center(child: Text(createdAt.split('T').first, style: TextStylesManager.medium12)),
258         );
259     }
260     final monthName = _getMonthName(dt.month);
261
262     return SizedBox(
263         width: width,
264         child: Column(
265             mainAxisAlignment: MainAxisAlignment.center,
266             crossAxisAlignment: CrossAxisAlignment.center,
267             children: [
268                 Text(
269                     '${dt.day}',
270                     style: const TextStyle(fontSize: 11, fontWeight: FontWeight.w600, height: 1.1),
271                 ),
272                 Text(
273                     monthName,
274                     style: const TextStyle(fontSize: 11, fontWeight: FontWeight.w600, height: 1.1),
275                 ),
276                 Text(
277                     '${dt.year}',
278                     style: const TextStyle(fontSize: 10.5, color: Colors.grey, height: 1.1),
279                 ),
280             ],
281         ),
282     );
283 }
284
285 Widget _buildActionsCell(BuildContext context, AdminTeacherModel teacher, double width) {
286     return SizedBox(
287         width: width,
288         child: Align(
289             alignment: Alignment.center,
290             child: Row(
291                 mainAxisAlignment: MainAxisAlignment.min,
292                 mainAxisAlignment: MainAxisAlignment.center,
293                 children: [
294                     _buildActionBtn(
295                         icon: Icons.edit_outlined,
296                         color: Colors.purple,

```

```

297         onTap: () => showEditTeacherDialog(context, teacher),
298         tooltip: appTranslation().get('admin_edit'),
299     ),
300     const SizedBox(width: 2),
301     _buildActionBtn(
302         icon: Icons.vpn_key_outlined,
303         color: Colors.orange,
304         onTap: () => showResetPasswordDialog(context, teacher),
305         tooltip: appTranslation().get('admin_reset_password'),
306     ),
307     const SizedBox(width: 2),
308     _buildActionBtn(
309         icon: Icons.power_settings_new,
310         color: Colors.red,
311         onTap: () => showToggleStatusDialog(context, teacher),
312         tooltip: teacher.active ? appTranslation().get('admin_suspend') : appTranslation().get('admin_a
ctivate'),
313     ),
314     const SizedBox(width: 2),
315     _buildActionBtn(
316         icon: Icons.delete_outline,
317         color: Colors.redAccent,
318         onTap: () => showDeleteTeacherDialog(context, teacher),
319         tooltip: appTranslation().get('admin_delete'),
320     ),
321 ],
322 ),
323 ),
324 );
325 }
326
327 Widget _buildActionBtn({
328     required IconData icon,
329     required Color color,
330     required VoidCallback onTap,
331     required String tooltip,
332 }) {
333     return Tooltip(
334         message: tooltip,
335         child: Material(
336             color: Colors.transparent,
337             child: InkWell(
338                 onTap: onTap,
339                 borderRadius: BorderRadius.circular(6),
340                 child: Padding(
341                     padding: const EdgeInsets.all(5.0),
342                     child: Icon(icon, size: 18, color: color),
343                 ),
344             ),
345         ),
346     );
347 }
348
349 Widget _buildPlanAndExpiryCell(BuildContext context, AdminTeacherModel teacher, double width) {
350     final isDark = Theme.of(context).brightness == Brightness.dark;
351
352     final isSubscribedActive = teacher.isSubscribed && !teacher.isExpired && !teacher.isFreePlan;
353     final isTrialActive = teacher.isInTrial && !teacher.isExpired;
354     final isTrialEnded = teacher.isTrialExpired || (teacher.isExpired && !teacher.isSubscribed);
355     final isSubscribedExpired = teacher.isSubscribed && teacher.isExpired;
356     final isFree = teacher.isFreePlan && !isTrialActive && !isTrialEnded && !isSubscribedActive && !isSubscri
bedExpired && teacher.effectiveEndsAt == null;
357
358     return SizedBox(
359         width: width,
360         child: Padding(
361             padding: const EdgeInsets.symmetric(horizontal: 4.0, vertical: 2.0),
362             child: Column(
363                 mainAxisAlignment: MainAxisAlignment.center,
364                 crossAxisAlignment: CrossAxisAlignment.center,
365                 children: [
366                     // Top Pill Badge
367                     _buildTopPillBadge(
368                         context,
369                         teacher,

```

```

370         isSubscribedActive: isSubscribedActive,
371         isTrialActive: isTrialActive,
372         isTrialEnded: isTrialEnded,
373         isSubscribedExpired: isSubscribedExpired,
374         isFree: isFree,
375         isDark: isDark,
376     ),
377     const SizedBox(height: 4),
378     // Bottom Details
379     _buildExpiryDetails(
380         context,
381         teacher,
382         isSubscribedActive: isSubscribedActive,
383         isTrialActive: isTrialActive,
384         isTrialEnded: isTrialEnded,
385         isSubscribedExpired: isSubscribedExpired,
386         isFree: isFree,
387         isDark: isDark,
388     ),
389 ],
390 ),
391 ),
392 );
393 }
394
395 Widget _buildTopPillBadge(
396     BuildContext context,
397     AdminTeacherModel teacher, {
398     required bool isSubscribedActive,
399     required bool isTrialActive,
400     required bool isTrialEnded,
401     required bool isSubscribedExpired,
402     required bool isFree,
403     required bool isDark,
404 }) {
405     if (isTrialActive) {
406         final bg = isDark ? const Color(0xFF451A03) : const Color(0xFFFFFBEB);
407         final border = isDark ? const Color(0xFFD97706) : const Color(0xFFDE68A);
408         final text = isDark ? const Color(0xFFBBF24) : const Color(0xFFB45309);
409
410         return Container(
411             padding: const EdgeInsets.symmetric(horizontal: 9, vertical: 2.5),
412             decoration: BoxDecoration(
413                 color: bg,
414                 borderRadius: BorderRadius.circular(16),
415                 border: Border.all(color: border, width: 1),
416             ),
417             child: Row(
418                 mainAxisAlignment: MainAxisAlignment.min,
419                 children: [
420                     Text(
421                         'تجربة مجانية',
422                         style: TextStyle(
423                             fontSize: 10.5,
424                             fontWeight: FontWeight.bold,
425                             color: text,
426                             height: 1.1,
427                         ),
428                     ),
429                     const SizedBox(width: 4),
430                     Icon(Icons.access_time_outlined, size: 12.5, color: text),
431                 ],
432             ),
433         );
434     }
435
436     if (isTrialEnded || isSubscribedExpired) {
437         final bg = isDark ? const Color(0xFF450A0A) : const Color(0xFFFFF2F2);
438         final border = isDark ? const Color(0xFFDC2626) : const Color(0xFFFFECACA);
439         final text = isDark ? const Color(0xFFFF8717) : const Color(0xFFDC2626);
440
441         return Container(
442             padding: const EdgeInsets.symmetric(horizontal: 9, vertical: 2),
443             decoration: BoxDecoration(
444                 color: bg,

```

```

445         borderRadius: BorderRadius.circular(16),
446         border: Border.all(color: border, width: 1),
447     ),
448     child: Row(
449       mainAxisAlignment: MainAxisAlignment.min,
450       children: [
451         Column(
452           mainAxisAlignment: MainAxisAlignment.min,
453           children: [
454             Text(
455               isTrialEnded ? 'انتهى' : 'انتهت',
456               style: TextStyle(
457                 fontSize: 9.5,
458                 fontWeight: FontWeight.bold,
459                 color: text,
460                 height: 1.1,
461               ),
462             ),
463             Text(
464               isTrialEnded ? 'الاشتراك' : 'التجربة',
465               style: TextStyle(
466                 fontSize: 9.5,
467                 fontWeight: FontWeight.bold,
468                 color: text,
469                 height: 1.1,
470               ),
471             ),
472           ],
473         ),
474         const SizedBox(width: 4),
475         Icon(Icons.error_outline_rounded, size: 13, color: text),
476       ],
477     ),
478   );
479 }
480
481 if (isSubscribedActive) {
482   final bg = isDark ? const Color(0xFF042F2E) : const Color(0xFFE0F2F1);
483   final border = isDark ? const Color(0xFF0D9488) : const Color(0xFF99F6E4);
484   final text = isDark ? const Color(0xFF2DD4BF) : const Color(0xFF0F766E);
485   final planName = teacher.subscription?.name ?? 'فصل دراسي واحد';
486
487   final words = planName.trim().split(' ');
488   Widget titleWidget;
489   if (words.length >= 3) {
490     titleWidget = Column(
491       mainAxisAlignment: MainAxisAlignment.min,
492       children: [
493         Text(
494           '${words[0]} ${words[1]}',
495           style: TextStyle(fontSize: 9.5, fontWeight: FontWeight.bold, color: text, height: 1.1),
496         ),
497         Text(
498           words.sublist(2).join(' '),
499           style: TextStyle(fontSize: 9.5, fontWeight: FontWeight.bold, color: text, height: 1.1),
500         ),
501       ],
502     );
503   } else if (words.length == 2) {
504     titleWidget = Column(
505       mainAxisAlignment: MainAxisAlignment.min,
506       children: [
507         Text(
508           words[0],
509           style: TextStyle(fontSize: 9.5, fontWeight: FontWeight.bold, color: text, height: 1.1),
510         ),
511         Text(
512           words[1],
513           style: TextStyle(fontSize: 9.5, fontWeight: FontWeight.bold, color: text, height: 1.1),
514         ),
515       ],
516     );
517   } else {
518     titleWidget = Text(
519       planName,

```

```

520         style: TextStyle(fontSize: 10, fontWeight: FontWeight.bold, color: text, height: 1.1),
521     );
522 }
523
524 return Container(
525     padding: const EdgeInsets.symmetric(horizontal: 9, vertical: 2),
526     decoration: BoxDecoration(
527         color: bg,
528         borderRadius: BorderRadius.circular(16),
529         border: Border.all(color: border, width: 1),
530     ),
531     child: Row(
532         mainAxisAlignment: MainAxisAlignment.min,
533         children: [
534             titleWidget,
535             const SizedBox(width: 4),
536             Icon(Icons.bolt_rounded, size: 14, color: text),
537         ],
538     ),
539 );
540 }
541
542 final bg = isDark ? Colors.grey[800]!.withValues(alpha: 0.5) : const Color(0xFFFF3F4F6);
543 final border = isDark ? Colors.grey[700]! : const Color(0xFFE5E7EB);
544 final text = isDark ? Colors.grey[400]! : const Color(0xFF6B7280);
545
546 return Container(
547     padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 3),
548     decoration: BoxDecoration(
549         color: bg,
550         borderRadius: BorderRadius.circular(16),
551         border: Border.all(color: border, width: 1),
552     ),
553     child: Text(
554         'مجاني',
555         style: TextStyle(
556             fontSize: 11,
557             fontWeight: FontWeight.bold,
558             color: text,
559             height: 1.1,
560         ),
561     ),
562 );
563 }
564
565 Widget _buildExpiryDetails(
566     BuildContext context,
567     AdminTeacherModel teacher, {
568     required bool isSubscribedActive,
569     required bool isTrialActive,
570     required bool isTrialEnded,
571     required bool isSubscribedExpired,
572     required bool isFree,
573     required bool isDark,
574 }) {
575     if (isFree || teacher.effectiveEndsAt == null) {
576         return Text(
577             '-',
578             style: TextStyle(
579                 fontSize: 14,
580                 fontWeight: FontWeight.bold,
581                 color: isDark ? Colors.grey[500] : Colors.grey[400],
582             ),
583         );
584     }
585
586     final dt = teacher.effectiveEndsAt!;
587     final monthName = _getMonthName(dt.month);
588     final days = teacher.dynamicDaysRemaining;
589
590     if (isTrialActive) {
591         final boxBg = isDark ? const Color(0xFF451A03) : const Color(0xFFFFEF3C7);
592         final boxBorder = isDark ? const Color(0xFFD97706) : const Color(0xFFFFDE68A);
593         final textColor = isDark ? const Color(0xFFFFBBF24) : const Color(0xFF92400E);
594         final calendarColor = isDark ? const Color(0xFFFF59E0B) : const Color(0xFFD97706);

```

```

595
596 return Row(
597   mainAxisSize: MainAxisSize.min,
598   mainAxisAlignment: MainAxisAlignment.center,
599   crossAxisAlignment: CrossAxisAlignment.center,
600   children: [
601     // Countdown badge
602     Container(
603       padding: const EdgeInsets.symmetric(horizontal: 5, vertical: 2),
604       decoration: BoxDecoration(
605         color: boxBg,
606         borderRadius: BorderRadius.circular(6),
607         border: Border.all(color: boxBorder, width: 0.8),
608       ),
609       child: Column(
610         mainAxisSize: MainAxisSize.min,
611         children: [
612           Text(
613             'متبقي',
614             style: TextStyle(
615               fontSize: 8.5,
616               fontWeight: FontWeight.bold,
617               color: textColor,
618               height: 1.1,
619             ),
620           ),
621           Text(
622             days > 2 && days <= 10 ? '$days أيام' : '$days يوم',
623             style: TextStyle(
624               fontSize: 8.5,
625               fontWeight: FontWeight.bold,
626               color: textColor,
627               height: 1.1,
628             ),
629           ),
630         ],
631       ),
632     ),
633     const SizedBox(width: 5),
634     // Date text + Calendar Icon
635     Row(
636       mainAxisSize: MainAxisSize.min,
637       children: [
638         Column(
639           mainAxisSize: MainAxisSize.min,
640           crossAxisAlignment: CrossAxisAlignment.start,
641           children: [
642             Text(
643               'تنتهي: ${dt.day}',
644               style: TextStyle(
645                 fontSize: 9.5,
646                 fontWeight: FontWeight.w600,
647                 color: textColor,
648                 height: 1.1,
649               ),
650             ),
651             Text(
652               '$monthName ${dt.year}',
653               style: TextStyle(
654                 fontSize: 9.5,
655                 fontWeight: FontWeight.w600,
656                 color: textColor,
657                 height: 1.1,
658               ),
659             ),
660           ],
661         ),
662         const SizedBox(width: 3),
663         Icon(Icons.calendar_today_outlined, size: 13, color: calendarColor),
664       ],
665     ),
666   ],
667 );
668 }
669

```

```

670 if (isTrialEnded || isSubscribedExpired) {
671   final textColor = isDark ? const Color(0xFF87171) : const Color(0xFFDC2626);
672
673   return Row(
674     mainAxisSize: MainAxisSize.min,
675     mainAxisAlignment: MainAxisAlignment.center,
676     crossAxisAlignment: CrossAxisAlignment.center,
677     children: [
678       Column(
679         mainAxisSize: MainAxisSize.min,
680         crossAxisAlignment: CrossAxisAlignment.center,
681         children: [
682           Text(
683             'انتهى: ${dt.day} $monthName',
684             style: TextStyle(
685               fontSize: 9.5,
686               fontWeight: FontWeight.w600,
687               color: textColor,
688               height: 1.1,
689             ),
690           ),
691           Text(
692             '${dt.year}',
693             style: TextStyle(
694               fontSize: 9.5,
695               fontWeight: FontWeight.w600,
696               color: textColor,
697               height: 1.1,
698             ),
699           ),
700         ],
701       ),
702       const SizedBox(width: 3),
703       Icon(Icons.calendar_today_outlined, size: 13, color: textColor),
704     ],
705   );
706 }
707
708 if (isSubscribedActive) {
709   final boxBg = isDark ? const Color(0xFF042F2E) : const Color(0xFFCCFBF1);
710   final boxBorder = isDark ? const Color(0xFF0D9488) : const Color(0xFF99F6E4);
711   final textColor = isDark ? const Color(0xFF2DD4BF) : const Color(0xFF0F766E);
712   final calendarColor = isDark ? const Color(0xFF14B8A6) : const Color(0xFF0D9488);
713
714   return Row(
715     mainAxisSize: MainAxisSize.min,
716     mainAxisAlignment: MainAxisAlignment.center,
717     crossAxisAlignment: CrossAxisAlignment.center,
718     children: [
719       // Countdown badge
720       Container(
721         padding: const EdgeInsets.symmetric(horizontal: 5, vertical: 2),
722         decoration: BoxDecoration(
723           color: boxBg,
724           borderRadius: BorderRadius.circular(6),
725           border: Border.all(color: boxBorder, width: 0.8),
726         ),
727       child: Column(
728         mainAxisSize: MainAxisSize.min,
729         children: [
730           Text(
731             '$days',
732             style: TextStyle(
733               fontSize: 9.5,
734               fontWeight: FontWeight.bold,
735               color: textColor,
736               height: 1.1,
737             ),
738           ),
739           Text(
740             'يوم',
741             style: TextStyle(
742               fontSize: 8.5,
743               fontWeight: FontWeight.bold,
744               color: textColor,

```

```

745         height: 1.1,
746       ),
747     ),
748   ],
749 ),
750 ),
751 const SizedBox(width: 5),
752 // Date text + Calendar Icon
753 Row(
754   mainAxisAlignment: MainAxisAlignment.min,
755   children: [
756     Column(
757       mainAxisAlignment: MainAxisAlignment.min,
758       crossAxisAlignment: CrossAxisAlignment.start,
759       children: [
760         Text(
761           'ينتهي: ${dt.day}',
762           style: TextStyle(
763             fontSize: 9.5,
764             fontWeight: FontWeight.w600,
765             color: textColor,
766             height: 1.1,
767           ),
768         ),
769         Text(
770           '$monthName ${dt.year}',
771           style: TextStyle(
772             fontSize: 9.5,
773             fontWeight: FontWeight.w600,
774             color: textColor,
775             height: 1.1,
776           ),
777         ),
778       ],
779     ),
780     const SizedBox(width: 3),
781     Icon(Icons.calendar_today_outlined, size: 13, color: calendarColor),
782   ],
783 ),
784 ],
785 );
786 }
787
788 return Text(
789   '-',
790   style: TextStyle(
791     fontSize: 14,
792     fontWeight: FontWeight.bold,
793     color: isDark ? Colors.grey[500] : Colors.grey[400],
794   ),
795 );
796 }
797 }
798
799
800

```

Lines • Dart 138

Feature / Certificates

#111 – lib/features/certificates/data/models/certificate_template_model.dart

```

1  import 'package:flutter/material.dart';
2
3  /// Visual style of each certificate template background.
4  enum CertificateTemplateStyle {
5    nationalRibbon,
6    luxuryFrame,
7    blueAcademic,
8    royalGold,
9    modernCorner,
10   minimalStamp,
11  }
12
13  class CertificateTemplateModel {
14    final int id;

```



```

15
16 /// Primary accent colour (top gradient, pills, borders).
17 final Color primaryColor;
18
19 /// Darker shade of primary (gradient end, corner accents).
20 final Color darkColor;
21
22 /// Soft / light background colour.
23 final Color bgColor;
24
25 /// Gold accent colour (arc borders, frame borders).
26 final Color goldColor;
27
28 /// Main text colour.
29 final Color textColor;
30
31 /// Kept for backward-compatibility with the PDF generation layer.
32 final Color accentColor;
33
34 /// Background visual style applied by [CertificateBgWidget].
35 final CertificateTemplateStyle style;
36
37 /// Short Arabic name shown in the selector (e.g. "قالب 1").
38 final String name;
39
40 /// Descriptive Arabic label shown below the name (e.g. "رسمي سعودي").
41 final String label;
42
43 const CertificateTemplateModel({
44   required this.id,
45   required this.primaryColor,
46   required this.darkColor,
47   required this.bgColor,
48   required this.goldColor,
49   required this.textColor,
50   required this.accentColor,
51   required this.style,
52   required this.name,
53   required this.label,
54 });
55
56 /// All 6 certificate templates matching the React design reference.
57 static const List<CertificateTemplateModel> all = [
58   // — Template 1 — nationalRibbon —————
59   CertificateTemplateModel(
60     id: 0,
61     primaryColor: Color(0xFF0E9F86),
62     darkColor: Color(0xFF065F55),
63     bgColor: Color(0xFFFF4FFFB),
64     goldColor: Color(0xFFE2AD3B),
65     textColor: Color(0xFF1E293B),
66     accentColor: Color(0xFF24B998),
67     style: CertificateTemplateStyle.nationalRibbon,
68     name: 'قالب 1',
69     label: 'رسمي سعودي',
70   ),
71   // — Template 2 — luxuryFrame —————
72   CertificateTemplateModel(
73     id: 1,
74     primaryColor: Color(0xFF238B83),
75     darkColor: Color(0xFF163E47),
76     bgColor: Color(0xFFFF8FCFB),
77     goldColor: Color(0xFFCFA44A),
78     textColor: Color(0xFF1E293B),
79     accentColor: Color(0xFF2FB8AD),
80     style: CertificateTemplateStyle.luxuryFrame,
81     name: 'قالب 2',
82     label: 'إطار فاخر',
83   ),
84   // — Template 3 — blueAcademic —————
85   CertificateTemplateModel(
86     id: 2,
87     primaryColor: Color(0xFF2377B8),
88     darkColor: Color(0xFF153B5B),
89     bgColor: Color(0xFFFF4FAFF),

```

```

90     goldColor: Color(0xFFE2AD3B),
91     textColor: Color(0xFF1E293B),
92     accentColor: Color(0xFF3B8FD8),
93     style: CertificateTemplateStyle.blueAcademic,
94     name: '3 قالب',
95     label: 'تعليمي أزرق',
96   ),
97   // — Template 4 — royalGold —————
98   CertificateTemplateModel(
99     id: 3,
100    primaryColor: Color(0xFFD7A13B),
101    darkColor: Color(0xFF20242B),
102    bgColor: Color(0xFFFFF9EC),
103    goldColor: Color(0xFFFF0C96A),
104    textColor: Color(0xFF1E293B),
105    accentColor: Color(0xFFEDBC5A),
106    style: CertificateTemplateStyle.royalGold,
107    name: '4 قالب',
108    label: 'ذهبي ملكي',
109  ),
110  // — Template 5 — modernCorner —————
111  CertificateTemplateModel(
112    id: 4,
113    primaryColor: Color(0xFF16A085),
114    darkColor: Color(0xFF123C4A),
115    bgColor: Color(0xFFFFFDF8),
116    goldColor: Color(0xFFF4BE3B),
117    textColor: Color(0xFF1E293B),
118    accentColor: Color(0xFF24B998),
119    style: CertificateTemplateStyle.modernCorner,
120    name: '5 قالب',
121    label: 'زوايا حديثة',
122  ),
123  // — Template 6 — minimalStamp —————
124  CertificateTemplateModel(
125    id: 5,
126    primaryColor: Color(0xFF0E7A5E),
127    darkColor: Color(0xFF075244),
128    bgColor: Color(0xFFFF7FCFA),
129    goldColor: Color(0xFFE2AD3B),
130    textColor: Color(0xFF1E293B),
131    accentColor: Color(0xFF1A9E7A),
132    style: CertificateTemplateStyle.minimalStamp,
133    name: '6 قالب',
134    label: 'ختم بسيط',
135  ),
136 ];
137 }
138

```

Lines • Dart 27

Feature / Certificates

#112 – lib/features/certificates/domain/entities/certificate_data.dart

```

1  class CertificateData {
2    final String studentName;
3    final String schoolName;
4    final String className;
5    final String teacherName;
6    final String principalName;
7    final String certDate;
8    final String certText;
9
10   /// 'male' or 'female'
11   final String gender;
12
13   const CertificateData({
14     required this.studentName,
15     required this.schoolName,
16     required this.className,
17     required this.teacherName,
18     required this.principalName,
19     required this.certDate,
20     required this.certText,
21     required this.gender,

```

```

22    });
23
24    /// "للطالب" or "للطالبة" based on gender
25    String get studentLabel => gender == 'female' ? 'للطالب' : 'للطالبة';
26  }
27

```

Lines • Dart 24

Feature / Certificates

#113 – lib/features/certificates/domain/entities/certificate_entity.dart

```

1  class CertificateEntity {
2    final String studentName;
3    final String gender;
4    final String schoolName;
5    final String className;
6    final String directorName;
7    final String teacherName;
8    final String certDate;
9    final String certText;
10   final int templateIndex;
11
12   const CertificateEntity({
13     required this.studentName,
14     required this.gender,
15     required this.schoolName,
16     required this.className,
17     required this.directorName,
18     required this.teacherName,
19     required this.certDate,
20     required this.certText,
21     required this.templateIndex,
22   });
23 }
24

```

Lines • Dart 225

Feature / Certificates

#114 – lib/features/certificates/presentation/cubit/certificate_cubit.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter/services.dart';
3  import 'package:flutter_bloc/flutter_bloc.dart';
4  import 'package:moean/core/network/remote/api_service.dart';
5  import 'package:moean/features/certificates/data/models/certificate_template_model.dart';
6  import 'package:moean/features/certificates/domain/entities/certificate_data.dart';
7  import 'package:moean/features/certificates/presentation/widgets/certificate_preview_widget.dart';
8  import 'package:pdf/pdf.dart';
9  import 'package:pdf/widgets.dart' as pw;
10 import 'package:printing/printing.dart';
11 import 'package:screenshot/screenshot.dart';
12
13 part 'certificate_state.dart';
14
15 class CertificateCubit extends Cubit<CertificateState> {
16   CertificateCubit() : super(CertificateInitial()) {
17     _addControllerListeners();
18   }
19
20   static CertificateCubit get(BuildContext context) => BlocProvider.of(context);
21
22   final TextEditingController studentNamesController = TextEditingController();
23   final TextEditingController schoolNameController = TextEditingController();
24   final TextEditingController classNameController = TextEditingController();
25   final TextEditingController directorNameController = TextEditingController();
26   final TextEditingController teacherNameController = TextEditingController();
27   final TextEditingController certTextController = TextEditingController();
28
29   int selectedTemplate = 0;
30   String selectedGender = 'male';
31   DateTime selectedDate = DateTime.now();
32
33   Future<void> loadTeacherProfile() async {
34     if (!isClosed) emit(CertificateLoading());
35     final result = await ApiService.getProfile();
36     if (isClosed) return;

```

```

37     result.fold(
38         (_) {
39             if (!isClosed) emit(const CertificateLoaded());
40         },
41         (profile) {
42             teacherNameController.text = profile.user.name;
43             if (!isClosed) emit(const CertificateLoaded());
44         },
45     );
46 }
47
48 void selectTemplate(int index) {
49     selectedTemplate = index;
50     if (!isClosed) emit(CertificateTemplateSelected(index));
51 }
52
53 void selectGender(String gender) {
54     selectedGender = gender;
55     if (!isClosed) emit(CertificateGenderChanged(gender));
56 }
57
58 void selectDate(DateTime date) {
59     selectedDate = date;
60     if (!isClosed) emit(CertificateDateChanged(date));
61 }
62
63 void selectReadyText(String text) {
64     certTextController.text = text;
65     if (!isClosed) emit(CertificateReadyTextSelected(text));
66 }
67
68 List<String> get studentNames => studentNamesController.text
69     .split('\n')
70     .map((e) => e.trim())
71     .where((e) => e.isNotEmpty)
72     .toList();
73
74 Future<void> generatePdf(BuildContext context) async {
75     if (studentNames.isEmpty) {
76         if (!isClosed) emit(const CertificateError('no_students'));
77         return;
78     }
79     if (!isClosed) emit(CertificateGenerating());
80     try {
81         await Printing.layoutPdf(
82             onLayout: (_) => _buildPdfBytes(context),
83             name: 'certificates',
84             format: PdfPageFormat.a4,
85         );
86         if (!isClosed) emit(const CertificateLoaded());
87     } catch (e) {
88         if (!isClosed) emit(CertificateError(e.toString()));
89     }
90 }
91
92 Future<Uint8List> _buildPdfBytes(BuildContext context) async {
93     final doc = pw.Document();
94     final template = CertificateTemplateModel.all[selectedTemplate];
95     final formattedDate =
96         '${selectedDate.year}-${selectedDate.month.toString().padLeft(2, '0')}-${selectedDate.day.toString().
97         padLeft(2, '0')}';
98     final screenshotController = ScreenshotController();
99
100     for (final name in studentNames) {
101         final data = CertificateData(
102             studentName: name,
103             schoolName: schoolNameController.text,
104             className: classNameController.text,
105             teacherName: teacherNameController.text,
106             principalName: directorNameController.text,
107             certDate: formattedDate,
108             certText: certTextController.text,
109             gender: selectedGender,
110         );

```

```

111
112     final widget = Directionality(
113         textDirection: TextDirection.rtl,
114         child: Material(
115             color: Colors.white,
116             child: RotatedBox(
117                 quarterTurns: 1,
118                 child: CertificatePreviewWidget(
119                     template: template,
120                     data: data,
121                     availableWidth: 1080.0,
122                 ),
123             ),
124         ),
125     );
126
127     // Render image at 1.5 pixel ratio for high print quality
128     final imageBytes = await screenshotController.captureFromWidget(
129         widget,
130         delay: const Duration(milliseconds: 150),
131         pixelRatio: 1.5,
132         context: context,
133         targetSize: const Size(1080.0 / 1.414, 1080.0),
134     );
135
136     final pdfImage = pw.MemoryImage(imageBytes);
137
138     doc.addPage(
139         pw.Page(
140             pageFormat: PdfPageFormat.a4,
141             margin: pw.EdgeInsets.zero,
142             build: (context) {
143                 return pw.FullPage(
144                     ignoreMargins: true,
145                     child: pw.Image(pdfImage, fit: pw.BoxFit.fill),
146                 );
147             },
148         ),
149     );
150 }
151
152 return doc.save();
153 }
154
155 // — Listeners —————
156
157 void _addControllerListeners() {
158     studentNamesController.addListener(_onPreviewChange);
159     schoolNameController.addListener(_onPreviewChange);
160     classNameController.addListener(_onPreviewChange);
161     directorNameController.addListener(_onPreviewChange);
162     teacherNameController.addListener(_onPreviewChange);
163     certTextController.addListener(_onPreviewChange);
164 }
165
166 void _removeControllerListeners() {
167     studentNamesController.removeListener(_onPreviewChange);
168     schoolNameController.removeListener(_onPreviewChange);
169     classNameController.removeListener(_onPreviewChange);
170     directorNameController.removeListener(_onPreviewChange);
171     teacherNameController.removeListener(_onPreviewChange);
172     certTextController.removeListener(_onPreviewChange);
173 }
174
175 void _onPreviewChange() {
176     if (!isClosed) emit(const CertificatePreviewUpdated());
177 }
178
179 // — Preview data —————
180
181 /// Builds a [CertificateData] snapshot from the current form state for the
182 /// live certificate preview. Shows placeholder text when fields are empty.
183 CertificateData buildPreviewData() {
184     final names = studentNames;
185     final date = selectedDate;

```

```

186     final formattedDate =
187         '${date.year}-${date.month.toString().padLeft(2, '0')}-${date.day.toString().padLeft(2, '0')}';
188
189     return CertificateData(
190         studentName: names.isNotEmpty ? names.first : 'اسم الطالب',
191         schoolName: schoolNameController.text.isNotEmpty
192             ? schoolNameController.text
193             : 'مدرسة حضر النموذجية',
194         className: classNameController.text.isNotEmpty
195             ? classNameController.text
196             : 'الصف الأول المتوسط',
197         teacherName: teacherNameController.text.isNotEmpty
198             ? teacherNameController.text
199             : 'اسم المعلم',
200         principalName: directorNameController.text.isNotEmpty
201             ? directorNameController.text
202             : 'اسم المدير',
203         certDate: formattedDate,
204         certText: certTextController.text.isNotEmpty
205             ? certTextController.text
206             : 'لتفوقه الدراسي وسمو أخلاقه، ونتمنى له دوام التفوق والنجاح بإذن الله.',
207         gender: selectedGender,
208     );
209 }
210
211 // — Lifecycle —————
212
213 @override
214 Future<void> close() {
215     _removeControllerListeners();
216     studentNamesController.dispose();
217     schoolNameController.dispose();
218     classNameController.dispose();
219     directorNameController.dispose();
220     teacherNameController.dispose();
221     certTextController.dispose();
222     return super.close();
223 }
224 }
225

```

Lines • Dart 45

Feature / Certificates

#115 — lib/features/certificates/presentation/cubit/certificate_state.dart

```

1  part of 'certificate_cubit.dart';
2
3  abstract class CertificateState {
4      const CertificateState();
5  }
6
7  class CertificateInitial extends CertificateState {}
8
9  class CertificateLoading extends CertificateState {}
10
11 class CertificateLoaded extends CertificateState {
12     const CertificateLoaded();
13 }
14
15 class CertificateTemplateSelected extends CertificateState {
16     final int index;
17     const CertificateTemplateSelected(this.index);
18 }
19
20 class CertificateGenderChanged extends CertificateState {
21     final String gender;
22     const CertificateGenderChanged(this.gender);
23 }
24
25 class CertificateDateChanged extends CertificateState {
26     final DateTime date;
27     const CertificateDateChanged(this.date);
28 }
29
30 class CertificateReadyTextSelected extends CertificateState {

```

```

31   final String text;
32   const CertificateReadyTextSelected(this.text);
33 }
34
35 class CertificatePreviewUpdated extends CertificateState {
36   const CertificatePreviewUpdated();
37 }
38
39 class CertificateGenerating extends CertificateState {}
40
41 class CertificateError extends CertificateState {
42   final String message;
43   const CertificateError(this.message);
44 }
45

```

Lines • Dart 161

Feature / Certificates #116 – lib/features/certificates/presentation/screens/certificate_screen.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7  import 'package:moean/features/certificates/presentation/cubit/certificate_cubit.dart';
8  import 'package:moean/features/certificates/presentation/widgets/certificate_form_widget.dart';
9  import 'package:moean/features/certificates/presentation/widgets/certificate_gender_tab_widget.dart';
10 import 'package:moean/features/certificates/presentation/widgets/certificate_generate_button_widget.dart';
11 import 'package:moean/features/certificates/presentation/widgets/certificate_template_grid_widget.dart';
12
13 class CertificateScreen extends StatefulWidget {
14   const CertificateScreen({super.key});
15
16   @override
17   State<CertificateScreen> createState() => _CertificateScreenState();
18 }
19
20 class _CertificateScreenState extends State<CertificateScreen> {
21   @override
22   void initState() {
23     super.initState();
24     CertificateCubit.get(context).loadTeacherProfile();
25   }
26
27   @override
28   Widget build(BuildContext context) {
29     return Directionality(
30       textDirection: TextDirection.rtl,
31       child: Scaffold(
32         backgroundColor: ColorsManager.background,
33         appBar: AppBar(
34           backgroundColor: ColorsManager.background,
35           elevation: 0,
36           centerTitle: true,
37           title: Text(
38             appTranslation().get('certificates'),
39             style: TextStylesManager.bold18
40               .copyWith(color: ColorsManager.mainText),
41           ),
42         leading: IconButton(
43           onPressed: () => Navigator.pop(context),
44           icon: Icon(
45             Icons.arrow_back_ios_rounded,
46             color: ColorsManager.mainText,
47             size: 20,
48           ),
49         ),
50       ),
51       body: BlocBuilder<CertificateCubit, CertificateState>(
52         buildWhen: (prev, curr) =>
53           curr is CertificateLoading || curr is CertificateLoaded,
54         builder: (context, state) {
55           if (state is CertificateLoading) {

```

```

56         return Center(
57             child: CircularProgressIndicator(
58                 color: ColorsManager.primaryColor,
59             ),
60         );
61     }
62     return const _CertificateBodyWidget();
63 },
64 ),
65 ),
66 );
67 }
68 }
69
70 class _CertificateBodyWidget extends StatelessWidget {
71     const _CertificateBodyWidget();
72
73     @override
74     Widget build(BuildContext context) {
75         return CustomScrollView(
76             slivers: [
77                 SliverPadding(
78                     padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 16),
79                     sliver: SliverList(
80                         delegate: SliverChildListDelegate([
81                             Row(
82                                 mainAxisAlignment: MainAxisAlignment.spaceBetween,
83                                 children: [
84                                     Image.asset('assets/images/roaa.png', height: 38),
85                                     Image.asset('assets/images/ministry.jpg', height: 38),
86                                 ],
87                             ),
88                             verticalSpace20,
89                             const _CertificateSectionHeaderWidget(),
90                             verticalSpace20,
91                             const CertificateGenderTabWidget(),
92                             verticalSpace20,
93                             const CertificateTemplateGridWidget(),
94                             verticalSpace24,
95                             const CertificateFormWidget(),
96                         ]),
97                     ),
98                 ),
99                 SliverFillRemaining(
100                     hasScrollBody: false,
101                     child: Padding(
102                         padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 16),
103                         child: Column(
104                             mainAxisAlignment: MainAxisAlignment.end,
105                             children: [
106                                 const CertificateGenerateButtonWidget(),
107                                 verticalSpace20,
108                             ],
109                         ),
110                     ),
111                 ),
112             ],
113         );
114     }
115 }
116
117 class _CertificateSectionHeaderWidget extends StatelessWidget {
118     const _CertificateSectionHeaderWidget();
119
120     @override
121     Widget build(BuildContext context) {
122         return Column(
123             crossAxisAlignment: CrossAxisAlignment.start,
124             children: [
125                 Row(
126                     children: [
127                         Container(
128                             padding: const EdgeInsets.all(10),
129                             decoration: BoxDecoration(
130                                 color: ColorsManager.primaryColor.withValues(alpha: 0.12),

```



```

131         borderRadius: BorderRadius.circular(14),
132     ),
133     child: Icon(
134         Icons.workspace_premium_rounded,
135         color: ColorsManager.primaryColor,
136         size: 24,
137     ),
138 ),
139 horizontalSpace12,
140 Column(
141     crossAxisAlignment: CrossAxisAlignment.start,
142     children: [
143         Text(
144             appTranslation().get('cert_section_title'),
145             style: TextStylesManager.bold20
146             .copyWith(color: ColorsManager.mainText),
147         ),
148         Text(
149             appTranslation().get('cert_choose_template'),
150             style: TextStylesManager.regular13
151             .copyWith(color: ColorsManager.secondaryText),
152         ),
153     ],
154 ),
155 ],
156 ),
157 ],
158 );
159 }
160 }
161

```

Lines • 35
Dart

Feature / #117 –
Certificates

[lib/features/certificates/presentation/widgets/certificate_bg_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/features/certificates/data/models/certificate_template_model.dart';
3  import 'package:moean/features/certificates/presentation/widgets/templates/cert_bg_blue_academic.dart';
4  import 'package:moean/features/certificates/presentation/widgets/templates/cert_bg_luxury_frame.dart';
5  import 'package:moean/features/certificates/presentation/widgets/templates/cert_bg_minimal_stamp.dart';
6  import 'package:moean/features/certificates/presentation/widgets/templates/cert_bg_modern_corner.dart';
7  import 'package:moean/features/certificates/presentation/widgets/templates/cert_bg_national_ribbon.dart';
8  import 'package:moean/features/certificates/presentation/widgets/templates/cert_bg_royal_gold.dart';
9
10  /// Routes to the correct background widget based on [template.style].
11  /// Used as the bottom layer of [CertificatePreviewWidget].
12  class CertificateBgWidget extends StatelessWidget {
13      final CertificateTemplateModel template;
14
15      const CertificateBgWidget({super.key, required this.template});
16
17      @override
18      Widget build(BuildContext context) {
19          return switch (template.style) {
20              CertificateTemplateStyle.nationalRibbon =>
21                  CertBgNationalRibbon(template: template),
22              CertificateTemplateStyle.luxuryFrame =>
23                  CertBgLuxuryFrame(template: template),
24              CertificateTemplateStyle.blueAcademic =>
25                  CertBgBlueAcademic(template: template),
26              CertificateTemplateStyle.royalGold =>
27                  CertBgRoyalGold(template: template),
28              CertificateTemplateStyle.modernCorner =>
29                  const CertBgModernCorner(),
30              CertificateTemplateStyle.minimalStamp =>
31                  CertBgMinimalStamp(template: template),
32          };
33      }
34  }
35

```

Lines • 41
Dart

Feature / #118 -
Certificates

[lib/features/certificates/presentation/widgets/certificate_dot_pattern_widget.dart](#)

```
1 import 'package:flutter/material.dart';
2
3 /// Repeating radial dot pattern matching the React `Pattern` component:
4 /// `radial-gradient(circle_at_20px_20px, #000 2px, transparent 0)`
5 /// repeated at 42px x 42px intervals, opacity 0.08.
6 class CertificateDotPatternWidget extends StatelessWidget {
7   const CertificateDotPatternWidget({super.key});
8
9   @override
10  Widget build(BuildContext context) {
11    return Positioned.fill(
12      child: CustomPaint(
13        painter: _DotPatternPainter(),
14      ),
15    );
16  }
17 }
18
19 class _DotPatternPainter extends CustomPainter {
20   const _DotPatternPainter();
21
22   @override
23   void paint(Canvas canvas, Size size) {
24     final paint = Paint()
25       ..color = Colors.black.withValues(alpha: 0.08)
26       ..style = PaintingStyle.fill;
27
28     const spacing = 42.0;
29     const dotRadius = 2.0;
30
31     for (double x = spacing * 0.5; x < size.width + spacing; x += spacing) {
32       for (double y = spacing * 0.5; y < size.height + spacing; y += spacing) {
33         canvas.drawCircle(Offset(x, y), dotRadius, paint);
34       }
35     }
36   }
37
38   @override
39   bool shouldRepaint(covariant CustomPainter oldDelegate) => false;
40 }
41
```

Lines • 178
Dart

Feature / #119 -
Certificates

[lib/features/certificates/presentation/widgets/certificate_form_widget.dart](#)

```
1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/spacing.dart';
7 import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
8 import 'package:moean/features/certificates/presentation/cubit/certificate_cubit.dart';
9 import 'package:moean/features/certificates/presentation/widgets/certificate_ready_texts_widget.dart';
10
11 class CertificateFormWidget extends StatelessWidget {
12   const CertificateFormWidget({super.key});
13
14   @override
15   Widget build(BuildContext context) {
16     final cubit = CertificateCubit.get(context);
17     return Column(
18       crossAxisAlignment: CrossAxisAlignment.start,
19       children: [
20         _SectionLabel(label: appTranslation().get('cert_student_names')),
21         verticalSpace8,
22         PrimaryTextField(
23           controller: cubit.studentNamesController,
24           hint: appTranslation().get('cert_student_names_hint'),
25         ),
26       ],
27     );
28   }
29 }

```

```

25     maxLines: 5,
26     keyboardType: TextInputType.multiline,
27     textInputAction: TextInputAction.newline,
28   ),
29   verticalSpace16,
30   _SectionLabel(label: appTranslation().get('cert_school_name')),
31   verticalSpace8,
32   PrimaryTextField(
33     controller: cubit.schoolNameController,
34     hint: appTranslation().get('cert_school_name'),
35   ),
36   verticalSpace16,
37   Row(
38     children: [
39       Expanded(
40         child: Column(
41           crossAxisAlignment: CrossAxisAlignment.start,
42           children: [
43             _SectionLabel(label: appTranslation().get('cert_director_name')),
44             verticalSpace8,
45             PrimaryTextField(
46               controller: cubit.directorNameController,
47               hint: appTranslation().get('cert_director_name'),
48             ),
49           ],
50         ),
51       ),
52       horizontalSpace12,
53       Expanded(
54         child: Column(
55           crossAxisAlignment: CrossAxisAlignment.start,
56           children: [
57             _SectionLabel(label: appTranslation().get('cert_class_name')),
58             verticalSpace8,
59             PrimaryTextField(
60               controller: cubit.classNameController,
61               hint: appTranslation().get('cert_class_name'),
62             ),
63           ],
64         ),
65       ),
66     ],
67   ),
68   verticalSpace16,
69   Row(
70     children: [
71       Expanded(
72         child: Column(
73           crossAxisAlignment: CrossAxisAlignment.start,
74           children: [
75             _SectionLabel(label: appTranslation().get('cert_date')),
76             verticalSpace8,
77             _CertDatePickerWidget(),
78           ],
79         ),
80       ),
81       horizontalSpace12,
82       Expanded(
83         child: Column(
84           crossAxisAlignment: CrossAxisAlignment.start,
85           children: [
86             _SectionLabel(label: appTranslation().get('cert_teacher_name')),
87             verticalSpace8,
88             PrimaryTextField(
89               controller: cubit.teacherNameController,
90               hint: appTranslation().get('cert_teacher_name'),
91             ),
92           ],
93         ),
94       ),
95     ],
96   ),
97   verticalSpace16,
98   _SectionLabel(label: appTranslation().get('cert_text')),
99   verticalSpace8,

```

```

100     PrimaryTextField(
101       controller: cubit.certTextController,
102       hint: appTranslation().get('cert_text'),
103       maxLines: 3,
104       keyboardType: TextInputType.multiline,
105       textInputAction: TextInputAction.newline,
106     ),
107     verticalSpace16,
108     const CertificateReadyTextsWidget(),
109   ],
110 );
111 }
112 }
113
114 class _SectionLabel extends StatelessWidget {
115   final String label;
116   const _SectionLabel({required this.label});
117
118   @override
119   Widget build(BuildContext context) {
120     return Text(
121       label,
122       style: TextStylesManager.bold13.copyWith(color: ColorsManager.mainText),
123     );
124   }
125 }
126
127 class _CertDatePickerWidget extends StatelessWidget {
128   @override
129   Widget build(BuildContext context) {
130     return BlocBuilder<CertificateCubit, CertificateState>(
131       buildWhen: (prev, curr) => curr is CertificateDateChanged,
132       builder: (context, state) {
133         final cubit = CertificateCubit.get(context);
134         final date = cubit.selectedDate;
135         final formatted =
136           '${date.year}-${date.month.toString().padLeft(2, '0')}-${date.day.toString().padLeft(2, '0')}';
137         return GestureDetector(
138           onTap: () async {
139             final picked = await showDatePicker(
140               context: context,
141               initialDate: cubit.selectedDate,
142               firstDate: DateTime(2020),
143               lastDate: DateTime(2030),
144             );
145             if (picked != null) cubit.selectDate(picked);
146           },
147           child: Container(
148             padding:
149               const EdgeInsets.symmetric(horizontal: 16, vertical: 16),
150             decoration: BoxDecoration(
151               color: ColorsManager.surfacePrimary,
152               borderRadius: BorderRadius.circular(24),
153               border: Border.all(color: ColorsManager.borderColor),
154             ),
155             child: Row(
156               children: [
157                 Icon(
158                   Icons.calendar_today_outlined,
159                   size: 16,
160                   color: ColorsManager.primaryColor,
161                 ),
162                 horizontalSpace8,
163                 Expanded(
164                   child: Text(
165                     formatted,
166                     style: TextStylesManager.regular14
167                       .copyWith(color: ColorsManager.mainText),
168                   ),
169                 ),
170               ],
171             ),
172           ),
173         );
174       },
175     );

```

```
175     );
176   }
177 }
178
```

Lines • 86
Dart

Feature / #120 -
Certificates [lib/features/certificates/presentation/widgets/certificate_gender_tab_widget.dart](#)

```
1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7  import 'package:moean/features/certificates/presentation/cubit/certificate_cubit.dart';
8
9  class CertificateGenderTabWidget extends StatelessWidget {
10    const CertificateGenderTabWidget({super.key});
11
12    @override
13    Widget build(BuildContext context) {
14      return BlocBuilder<CertificateCubit, CertificateState>(
15        buildWhen: (prev, curr) => curr is CertificateGenderChanged,
16        builder: (context, state) {
17          final cubit = CertificateCubit.get(context);
18          final isMale = cubit.selectedGender == 'male';
19          return Container(
20            decoration: BoxDecoration(
21              color: ColorsManager.surfacePrimary,
22              borderRadius: BorderRadius.circular(16),
23              border: Border.all(color: ColorsManager.borderColor),
24            ),
25            padding: const EdgeInsets.all(4),
26            child: Row(
27              children: [
28                _TabItem(
29                  label: appTranslation().get('cert_student_type_female'),
30                  isSelected: !isMale,
31                  onTap: () => cubit.selectGender('female'),
32                ),
33                horizontalSpace8,
34                _TabItem(
35                  label: appTranslation().get('cert_student_type_male'),
36                  isSelected: isMale,
37                  onTap: () => cubit.selectGender('male'),
38                ),
39              ],
40            ),
41          );
42        },
43      );
44    }
45  }
46
47  class _TabItem extends StatelessWidget {
48    final String label;
49    final bool isSelected;
50    final VoidCallback onTap;
51
52    const _TabItem({
53      required this.label,
54      required this.isSelected,
55      required this.onTap,
56    });
57
58    @override
59    Widget build(BuildContext context) {
60      return Expanded(
61        child: GestureDetector(
62          onTap: onTap,
63          child: AnimatedContainer(
64            duration: const Duration(milliseconds: 250),
65            padding: const EdgeInsets.symmetric(vertical: 12),
```

```

66         decoration: BoxDecoration(
67             color: isSelected
68                 ? ColorsManager.primaryColor
69                 : Colors.transparent,
70             borderRadius: BorderRadius.circular(12),
71         ),
72         alignment: Alignment.center,
73         child: Text(
74             label,
75             style: TextStylesManager.bold14.copyWith(
76                 color: isSelected
77                     ? ColorsManager.white
78                     : ColorsManager.secondaryText,
79             ),
80         ),
81     ),
82 ),
83 );
84 }
85 }
86

```

Lines 114
• Dart

Feature / #121 –
Certificates lib/features/certificates/presentation/widgets/certificate_generate_button_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7  import 'package:moean/features/certificates/presentation/cubit/certificate_cubit.dart';
8
9  class CertificateGenerateButtonWidget extends StatelessWidget {
10     const CertificateGenerateButtonWidget({super.key});
11
12     @override
13     Widget build(BuildContext context) {
14         return BlocBuilder<CertificateCubit, CertificateState>(
15             buildWhen: (prev, curr) =>
16                 curr is CertificateGenerating ||
17                 curr is CertificateLoaded ||
18                 curr is CertificateError,
19             builder: (context, state) {
20                 final cubit = CertificateCubit.get(context);
21                 final isGenerating = state is CertificateGenerating;
22                 final count = cubit.studentNames.length;
23
24                 return Column(
25                     children: [
26                         if (state is CertificateError)
27                             Container(
28                                 margin: const EdgeInsets.only(bottom: 12),
29                                 padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 10),
30                                 decoration: BoxDecoration(
31                                     color: ColorsManager.errorColor.withValues(alpha: 0.1),
32                                     borderRadius: BorderRadius.circular(12),
33                                     border: Border.all(
34                                         color: ColorsManager.errorColor.withValues(alpha: 0.4),
35                                     ),
36                             ),
37                         child: Row(
38                             children: [
39                                 Icon(Icons.error_outline,
40                                     color: ColorsManager.errorColor, size: 18),
41                                 horizontalSpace8,
42                                 Expanded(
43                                     child: Text(
44                                         state.message == 'no_students'
45                                             ? appTranslation().get('cert_no_students')
46                                             : appTranslation().get('cert_error'),
47                                         style: TextStylesManager.regular12
48                                             .copyWith(color: ColorsManager.errorColor),

```

```

49         ),
50     ),
51 ],
52 ),
53 ),
54 SizedBox(
55   width: double.infinity,
56   child: ElevatedButton(
57     onPressed: isGenerating ? null : () => cubit.generatePdf(context),
58     style: ElevatedButton.styleFrom(
59       backgroundColor: ColorsManager.primaryColor,
60       disabledBackgroundColor:
61         ColorsManager.primaryColor.withValues(alpha: 0.5),
62       padding: const EdgeInsets.symmetric(vertical: 18),
63       shape: RoundedRectangleBorder(
64         borderRadius: BorderRadius.circular(24),
65       ),
66       elevation: 0,
67     ),
68     child: isGenerating
69       ? Row(
70         mainAxisAlignment: MainAxisAlignment.center,
71         children: [
72           const SizedBox(
73             width: 18,
74             height: 18,
75             child: CircularProgressIndicator(
76               strokeWidth: 2,
77               color: Colors.white,
78             ),
79           ),
80           horizontalSpace10,
81           Text(
82             appTranslation().get('cert_generating'),
83             style: TextStylesManager.bold16
84               .copyWith(color: ColorsManager.white),
85           ),
86         ],
87       )
88       : Row(
89         mainAxisAlignment: MainAxisAlignment.center,
90         children: [
91           const Icon(
92             Icons.picture_as_pdf_outlined,
93             color: Colors.white,
94             size: 20,
95           ),
96           horizontalSpace10,
97           Text(
98             count > 0
99               ? '${appTranslation().get('cert_generate')} ($count)'
100               : appTranslation().get('cert_generate'),
101             style: TextStylesManager.bold16
102               .copyWith(color: ColorsManager.white),
103           ),
104         ],
105       ),
106     ),
107   ),
108 ],
109 );
110 },
111 );
112 }
113 }
114

```

Lines • 68
Dart

Feature /
Certificates

#122 -

lib/features/certificates/presentation/widgets/certificate_logo_slot_widget.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:google_fonts/google_fonts.dart';
3 import 'package:moean/features/certificates/data/models/certificate_template_model.dart';

```

```

4
5 /// A 128×96 white rounded box that shows either an image asset or a text
6 /// fallback – matching the React `LogoSlot` component exactly.
7 ///
8 /// Container spec: `h-24 w-32 rounded-3xl bg-white/95 p-2 shadow-xl ring-1 ring-black/5`
9 /// All dimensions are at the 1080-base scale.
10 class CertificateLogoSlotWidget extends StatelessWidget {
11   final String assetPath;
12   final double imageWidth;
13   final double imageHeight;
14   final String fallbackLine1;
15   final String fallbackLine2;
16   final CertificateTemplateModel template;
17
18   const CertificateLogoSlotWidget({
19     super.key,
20     required this.assetPath,
21     required this.imageWidth,
22     required this.imageHeight,
23     required this.fallbackLine1,
24     required this.fallbackLine2,
25     required this.template,
26   });
27
28   @override
29   Widget build(BuildContext context) {
30     return Container(
31       width: 128,
32       height: 96,
33       padding: const EdgeInsets.all(8),
34       decoration: BoxDecoration(
35         color: Colors.white.withValues(alpha: 0.95),
36         borderRadius: BorderRadius.circular(24),
37         boxShadow: [
38           BoxShadow(
39             color: Colors.black.withValues(alpha: 0.14),
40             blurRadius: 28,
41             offset: const Offset(0, 12),
42           ),
43         ],
44         border: Border.all(
45           color: Colors.black.withValues(alpha: 0.05),
46         ),
47       ),
48       alignment: Alignment.center,
49       child: Image.asset(
50         assetPath,
51         width: imageWidth,
52         height: imageHeight,
53         fit: BoxFit.contain,
54         errorBuilder: (context, error, stackTrace) => Text(
55           '$fallbackLine1\n$fallbackLine2',
56           textAlign: TextAlign.center,
57           style: GoogleFonts.tajawal(
58             fontSize: 12,
59             fontWeight: FontWeight.w900,
60             color: template.darkColor,
61             height: 1.4,
62           ),
63         ),
64       ),
65     );
66   }
67 }
68

```

Lines 130

• Dart

Feature / #123 –

Certificates

[lib/features/certificates/presentation/widgets/certificate_preview_section_widget.dart](#)

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';

```



```

5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/spacing.dart';
7 import 'package:moean/features/certificates/data/models/certificate_template_model.dart';
8 import 'package:moean/features/certificates/presentation/cubit/certificate_cubit.dart';
9 import 'package:moean/features/certificates/presentation/widgets/certificate_preview_widget.dart';
10
11 /// The preview section shown between the template grid and the generate button.
12 ///
13 /// It wraps [CertificatePreviewWidget] in a [BlocBuilder] so the preview
14 /// instantly rebuilds whenever the user changes:
15 ///   - template, gender, date, ready-text (cubit states)
16 ///   - any text field (CertificatePreviewUpdated emitted by controller listeners)
17 class CertificatePreviewSectionWidget extends StatelessWidget {
18   const CertificatePreviewSectionWidget({super.key});
19
20   @override
21   Widget build(BuildContext context) {
22     return BlocBuilder<CertificateCubit, CertificateState>(
23       buildWhen: (prev, curr) =>
24         curr is CertificateTemplateSelected ||
25         curr is CertificateGenderChanged ||
26         curr is CertificateDateChanged ||
27         curr is CertificateReadyTextSelected ||
28         curr is CertificatePreviewUpdated ||
29         curr is CertificateLoaded,
30       builder: (context, state) {
31         final cubit = CertificateCubit.get(context);
32         final template = CertificateTemplateModel.all[cubit.selectedTemplate];
33         final data = cubit.buildPreviewData();
34
35         return Column(
36           crossAxisAlignment: CrossAxisAlignment.start,
37           children: [
38             // — Section header —————
39             _PreviewHeaderWidget(template: template, cubit: cubit),
40             verticalSpace12,
41
42             // — Horizontally scrollable certificate canvas —————
43             // The inner [CertificatePreviewWidget] enforces a minimum
44             // display width of 680 px, so narrow phones can scroll.
45             LayoutBuilder(
46               builder: (context, constraints) {
47                 return Container(
48                   decoration: BoxDecoration(
49                     color: ColorsManager.surfacePrimary,
50                     borderRadius: BorderRadius.circular(20),
51                     border: Border.all(color: ColorsManager.borderColor),
52                     boxShadow: [
53                       BoxShadow(
54                         color: Colors.black.withValues(alpha: 0.06),
55                         blurRadius: 20,
56                         offset: const Offset(0, 6),
57                       ),
58                     ],
59                 ),
60                 padding: const EdgeInsets.all(10),
61                 child: ClipRRect(
62                   borderRadius: BorderRadius.circular(12),
63                   child: SingleChildScrollView(
64                     scrollDirection: Axis.horizontal,
65                     child: CertificatePreviewWidget(
66                       availableWidth: constraints.maxWidth - 20, // adjust for padding
67                       template: template,
68                       data: data,
69                     ),
70                   ),
71                 ),
72               );
73             ],
74           ),
75         ],
76       );
77     },
78   );
79 }

```

```

80 }
81
82 /// Header bar: "معينة الشهادة" label + template name + student count.
83 class _PreviewHeaderWidget extends StatelessWidget {
84   final CertificateTemplateModel template;
85   final CertificateCubit cubit;
86
87   const _PreviewHeaderWidget({
88     required this.template,
89     required this.cubit,
90   });
91
92   @override
93   Widget build(BuildContext context) {
94     final count = cubit.studentNames.length;
95
96     return Row(
97       mainAxisAlignment: MainAxisAlignment.spaceBetween,
98       children: [
99         Column(
100           crossAxisAlignment: CrossAxisAlignment.start,
101           children: [
102             Text(
103               appTranslation().get('cert_preview_label'),
104               style: TextStyleManager.bold12
105                 .copyWith(color: ColorsManager.primaryColor),
106             ),
107             Text(
108               '${template.name} - ${template.label}',
109               style: TextStyleManager.bold16
110                 .copyWith(color: ColorsManager.mainText),
111             ),
112           ],
113         ),
114         Container(
115           padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
116           decoration: BoxDecoration(
117             color: ColorsManager.primaryColor.withValues(alpha: 0.1),
118             borderRadius: BorderRadius.circular(20),
119           ),
120           child: Text(
121             '${count > 0 ? count : 1} ${appTranslation().get('cert_preview_count_suffix')}',
122             style: TextStyleManager.bold12
123               .copyWith(color: ColorsManager.primaryColor),
124           ),
125         ),
126       ],
127     );
128   }
129 }
130

```

Lines • 520
Dart

Feature / #124 -
Certificates

[lib/features/certificates/presentation/widgets/certificate_preview_widget.dart](#)

```

1 import 'dart:math' as math;
2
3 import 'package:flutter/material.dart';
4 import 'package:google_fonts/google_fonts.dart';
5 import 'package:moean/features/certificates/data/models/certificate_template_model.dart';
6 import 'package:moean/features/certificates/domain/entities/certificate_data.dart';
7 import 'package:moean/features/certificates/presentation/widgets/certificate_bg_widget.dart';
8 import 'package:moean/features/certificates/presentation/widgets/certificate_logo_slot_widget.dart';
9 import 'package:moean/features/certificates/presentation/widgets/certificate_signature_widget.dart';
10
11 /// Full-fidelity live certificate preview.
12 ///
13 /// Matches the React `CertificatePreview` component pixel-for-pixel:
14 /// - aspect ratio 1.414 : 1 (A4 landscape)
15 /// - built internally at 1080 × 763.5 px (React base size)
16 /// - scaled via [Transform.scale] to fit the available width
17 /// - minimum display width 680 px with horizontal scroll support
18 ///

```

```

19 /// The parent must wrap this widget in a [SingleChildScrollView] with
20 /// `scrollDirection: Axis.horizontal` to allow the minimum width on narrow
21 /// screens.
22 class CertificatePreviewWidget extends StatelessWidget {
23   final CertificateTemplateModel template;
24   final CertificateData data;
25   final double availableWidth;
26
27   const CertificatePreviewWidget({
28     super.key,
29     required this.template,
30     required this.data,
31     required this.availableWidth,
32   });
33
34   // — Geometry constants (all in React / 1080-base pixels) —————
35   static const double _baseWidth = 1080.0;
36   static const double _baseHeight = _baseWidth / 1.414; // ≈ 763.5 px
37
38   /// Minimum display width (px) before the preview becomes horizontally
39   /// scrollable. Chosen so text remains legible at the resulting scale.
40   static const double _minDisplayWidth = 680.0;
41
42   @override
43   Widget build(BuildContext context) {
44     final displayWidth = math.max(availableWidth, _minDisplayWidth);
45     final scale = displayWidth / _baseWidth;
46     final displayHeight = _baseHeight * scale;
47
48     return SizedBox(
49       width: displayWidth,
50       height: displayHeight,
51       child: FittedBox(
52         fit: BoxFit.contain,
53         alignment: Alignment.topLeft,
54         child: SizedBox(
55           width: _baseWidth,
56           height: _baseHeight,
57           child: ClipRect(
58             child: _CertificateCanvas(
59               template: template,
60               data: data,
61             ),
62           ),
63         ),
64       ),
65     );
66   }
67 }
68
69 // — Canvas (built at 1080×763.5 base) —————
70
71 class _CertificateCanvas extends StatelessWidget {
72   final CertificateTemplateModel template;
73   final CertificateData data;
74
75   const _CertificateCanvas({
76     required this.template,
77     required this.data,
78   });
79
80   @override
81   Widget build(BuildContext context) {
82     return Stack(
83       clipBehavior: Clip.hardEdge,
84       children: [
85         // — Layer 0: Template background —————
86         Positioned.fill(
87           child: CertificateBgWidget(template: template),
88         ),
89
90         // — Layer 1: Content (z-10 in React) —————
91         Positioned.fill(
92           child: Directionality(
93             textDirection: TextDirection.rtl,

```

```

94     child: Padding(
95       // px-20 py-12 → horizontal 80px, vertical 48px
96       padding: const EdgeInsets.symmetric(
97         horizontal: 80,
98         vertical: 48,
99       ),
100     child: FittedBox(
101       fit: BoxFit.scaleDown,
102       child: SizedBox(
103         width: 920, // 1080 (baseWidth) - 160 (horizontal padding)
104         child: Column(
105           crossAxisAlignment: CrossAxisAlignment.center,
106           children: [
107             _LogoRow(template: template),
108             const SizedBox(height: 32),
109             _TitleBlock(template: template, data: data),
110             const SizedBox(height: 32),
111             _StudentRow(template: template, data: data),
112             const SizedBox(height: 40),
113             _ReasonText(data: data),
114             const SizedBox(height: 24),
115             _FooterRow(template: template, data: data),
116           ],
117         ),
118       ),
119     ),
120   ),
121 ),
122 ),
123 ],
124 );
125 }
126 }
127
128 // — Logo row —————
129
130 /// React: `dir="ltr"` row with Vision-2030 on the left, Ministry on the right.
131 /// `gap-5` = 20px between logos.
132 class _LogoRow extends StatelessWidget {
133   final CertificateTemplateModel template;
134
135   const _LogoRow({required this.template});
136
137   @override
138   Widget build(BuildContext context) {
139     return Directionality(
140       textDirection: TextDirection.ltr,
141       child: Row(
142         mainAxisAlignment: MainAxisAlignment.center,
143         children: [
144           // 2030 رؤية - left slot
145           CertificateLogoSlotWidget(
146             assetPath: 'assets/images/roaa.png',
147             imageWidth: 104,
148             imageHeight: 72,
149             fallbackLine1: 'رؤية',
150             fallbackLine2: '2030',
151             template: template,
152           ),
153           const SizedBox(width: 20),
154           // وزارة التعليم - right slot
155           CertificateLogoSlotWidget(
156             assetPath: 'assets/images/ministry.jpg',
157             imageWidth: 112,
158             imageHeight: 78,
159             fallbackLine1: 'وزارة',
160             fallbackLine2: 'التعليم',
161             template: template,
162           ),
163         ],
164       ),
165     );
166   }
167 }
168

```

```

169 // — Title + school line —————
170
171 class _TitleBlock extends StatelessWidget {
172   final CertificateTemplateModel template;
173   final CertificateData data;
174
175   const _TitleBlock({required this.template, required this.data});
176
177   @override
178   Widget build(BuildContext context) {
179     return Column(
180       children: [
181         // "شهادة شكر وتقدير" — text-5xl (48px) font-black, color=dark, drop-shadow
182         Text(
183           'شهادة شكر وتقدير',
184           textAlign: TextAlign.center,
185           style: GoogleFonts.tajawal(
186             fontSize: 48,
187             fontWeight: FontWeight.w900,
188             color: template.darkColor,
189             shadows: [
190               Shadow(
191                 color: Colors.black.withValues(alpha: 0.12),
192                 offset: const Offset(0, 2),
193                 blurRadius: 4,
194               ),
195             ],
196           ),
197         ),
198         const SizedBox(height: 16),
199         // School line — text-2xl (24px) font-bold
200         _SchoolLine(template: template, data: data),
201       ],
202     );
203   }
204 }
205
206 /// React: `يسر إدارة مدرسة` + pill(schoolName) + "الشكر والتقدير"
207 /// pill: min-w-[285px] justify-center rounded-full bg-accent px-8 py-2 text-white shadow-md mx-3
208 class _SchoolLine extends StatelessWidget {
209   final CertificateTemplateModel template;
210   final CertificateData data;
211
212   const _SchoolLine({required this.template, required this.data});
213
214   @override
215   Widget build(BuildContext context) {
216     const slateStyle = TextStyle(
217       fontSize: 24,
218       fontWeight: FontWeight.w700,
219       color: Color(0xFF475569),
220       fontFamily: 'Tajawal',
221     );
222
223     return Wrap(
224       alignment: WrapAlignment.center,
225       crossAxisAlignment: WrapCrossAlignment.center,
226       spacing: 12,
227       runSpacing: 8,
228       children: [
229         Text('يسر إدارة مدرسة', style: slateStyle),
230
231         // school name pill
232         Container(
233           constraints: const BoxConstraints(minWidth: 285),
234           padding: const EdgeInsets.symmetric(horizontal: 32, vertical: 8),
235           decoration: BoxDecoration(
236             color: template.primaryColor,
237             borderRadius: BorderRadius.circular(999),
238             boxShadow: [
239               BoxShadow(
240                 color: template.primaryColor.withValues(alpha: 0.35),
241                 blurRadius: 12,
242                 offset: const Offset(0, 4),
243               ),

```

```

244     ],
245   ),
246   alignment: Alignment.center,
247   child: Text(
248     data.schoolName,
249     textAlign: TextAlign.center,
250     style: GoogleFonts.tajawal(
251       fontSize: 24,
252       fontWeight: FontWeight.w700,
253       color: Colors.white,
254     ),
255   ),
256 ),
257
258 Text('أن تتقدم بوافر الشكر والتقدير', style: slateStyle),
259 ],
260 );
261 }
262 }
263
264 // — Student row —————
265
266 /// React: 4-column grid `[1fr auto 1.25fr auto]`, text-2xl, gap-5.
267 /// Columns (RTL order, right→left):
268 ///   "الطالب:" | name pill (min-w 420) | class pill (1.25fr) | "الصف"
269 class _StudentRow extends StatelessWidget {
270   final CertificateTemplateModel template;
271   final CertificateData data;
272
273   const _StudentRow({required this.template, required this.data});
274
275   @override
276   Widget build(BuildContext context) {
277     return Row(
278       crossAxisAlignment: CrossAxisAlignment.center,
279       children: [
280         // Col 1 — "الطالب:" (1fr, text-left in RTL = align start)
281         Expanded(
282           flex: 4,
283           child: Text(
284             data.studentLabel,
285             textAlign: TextAlign.start,
286             style: GoogleFonts.tajawal(
287               fontSize: 24,
288               fontWeight: FontWeight.w700,
289               color: const Color(0xFF475569),
290             ),
291           ),
292         ),
293         const SizedBox(width: 20),
294
295         // Col 2 — student name pill min-w-[420px] gradient rounded-full
296         ConstrainedBox(
297           constraints: const BoxConstraints(minWidth: 420),
298           child: Container(
299             padding: const EdgeInsets.symmetric(horizontal: 32, vertical: 12),
300             decoration: BoxDecoration(
301               borderRadius: BorderRadius.circular(999),
302               gradient: LinearGradient(
303                 begin: Alignment.centerRight,
304                 end: Alignment.centerLeft,
305                 colors: [template.primaryColor, template.darkColor],
306               ),
307               boxShadow: [
308                 BoxShadow(
309                   color: template.darkColor.withValues(alpha: 0.40),
310                   blurRadius: 20,
311                   offset: const Offset(0, 8),
312                 ),
313               ],
314             ),
315           alignment: Alignment.center,
316           child: Text(
317             data.studentName,
318             textAlign: TextAlign.center,

```

```

319         style: GoogleFonts.tajawal(
320             fontSize: 30,
321             fontWeight: FontWeight.w900,
322             color: Colors.white,
323         ),
324     ),
325 ),
326 ),
327 const SizedBox(width: 20),
328
329 // Col 3 - class pill (1.25fr, rounded-full, bg-accent)
330 Expanded(
331     flex: 5,
332     child: Container(
333         padding: const EdgeInsets.symmetric(horizontal: 32, vertical: 12),
334         decoration: BoxDecoration(
335             color: template.primaryColor,
336             borderRadius: BorderRadius.circular(999),
337             boxShadow: [
338                 BoxShadow(
339                     color: template.primaryColor.withValues(alpha: 0.30),
340                     blurRadius: 12,
341                     offset: const Offset(0, 4),
342                 ),
343             ],
344         ),
345         alignment: Alignment.center,
346         child: Text(
347             data.className,
348             textAlign: TextAlign.center,
349             style: GoogleFonts.tajawal(
350                 fontSize: 24,
351                 fontWeight: FontWeight.w700,
352                 color: Colors.white,
353             ),
354         ),
355     ),
356 ),
357 const SizedBox(width: 20),
358
359 // Col 4 - "الصف" label (auto)
360 Text(
361     'الصف',
362     style: GoogleFonts.tajawal(
363         fontSize: 24,
364         fontWeight: FontWeight.w700,
365         color: const Color(0xFF475569),
366     ),
367 ),
368 ],
369 );
370 }
371 }
372
373 // — Reason / certificate body text —————
374
375 /// React: `mx-auto mt-10 max-w-[850px] text-center text-3xl font-bold
376 ///         leading-[1.85] text-slate-800`
377 class _ReasonText extends StatelessWidget {
378     final CertificateData data;
379
380     const _ReasonText({required this.data});
381
382     @override
383     Widget build(BuildContext context) {
384         return Center(
385             child: ConstrainedBox(
386                 constraints: const BoxConstraints(maxWidth: 850),
387                 child: Text(
388                     data.certText,
389                     textAlign: TextAlign.center,
390                     style: GoogleFonts.tajawal(
391                         fontSize: 30,
392                         fontWeight: FontWeight.w700,
393                         height: 1.85,

```

```

394         color: const Color(0xFF1E293B),
395     ),
396 ),
397 ),
398 );
399 }
400 }
401
402 // — Footer row —————
403
404 /// React: `mt-auto grid grid-cols-3 items-end gap-8`
405 /// Col1: Signature(principal)
406 /// Col2: circular seal + date pill (centered)
407 /// Col3: Signature(teacher)
408 class _FooterRow extends StatelessWidget {
409     final CertificateTemplateModel template;
410     final CertificateData data;
411
412     const _FooterRow({required this.template, required this.data});
413
414     @override
415     Widget build(BuildContext context) {
416         return Row(
417             crossAxisAlignment: CrossAxisAlignment.end,
418             children: [
419                 // Principal signature
420                 Expanded(
421                     child: Center(
422                         child: CertificateSignatureWidget(
423                             title: 'المدير',
424                             name: data.principalName,
425                             template: template,
426                         ),
427                     ),
428                 ),
429                 const SizedBox(width: 32),
430
431                 // Seal + date
432                 _SealAndDate(template: template, data: data),
433                 const SizedBox(width: 32),
434
435                 // Teacher signature
436                 Expanded(
437                     child: Center(
438                         child: CertificateSignatureWidget(
439                             title: 'المعلم',
440                             name: data.teacherName,
441                             template: template,
442                         ),
443                     ),
444                 ),
445             ],
446         );
447     }
448 }
449
450 /// React:
451 /// ```
452 /// <div h-24 w-24 rounded-full border-[5px] border-accent bg-white/70 shadow-xl
453 ///   text-11px font-black leading-5 text-dark>شهادة<br/>حضر</div>
454 /// <p rounded-full bg-soft px-5 py-2 text-sm font-black text-dark>{date}</p>
455 /// ```
456 class _SealAndDate extends StatelessWidget {
457     final CertificateTemplateModel template;
458     final CertificateData data;
459
460     const _SealAndDate({required this.template, required this.data});
461
462     @override
463     Widget build(BuildContext context) {
464         return Column(
465             mainAxisAlignment: MainAxisAlignment.min,
466             children: [
467                 // Circular seal
468                 Container(

```



```

469 width: 96,
470 height: 96,
471 decoration: BoxDecoration(
472   shape: BoxShape.circle,
473   color: Colors.white.withValues(alpha: 0.70),
474   border: Border.all(
475     color: template.primaryColor,
476     width: 5,
477   ),
478   boxShadow: [
479     BoxShadow(
480       color: Colors.black.withValues(alpha: 0.18),
481       blurRadius: 24,
482       offset: const Offset(0, 8),
483     ),
484   ],
485 ),
486 alignment: Alignment.center,
487 child: Text(
488   'حضر شهادة',
489   textAlign: TextAlign.center,
490   style: GoogleFonts.tajawal(
491     fontSize: 11,
492     fontWeight: FontWeight.w900,
493     height: 1.5,
494     color: template.darkColor,
495   ),
496 ),
497 ),
498 const SizedBox(height: 12),
499
500 // Date pill
501 Container(
502   padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 8),
503   decoration: BoxDecoration(
504     color: template.bgColor,
505     borderRadius: BorderRadius.circular(999),
506   ),
507   child: Text(
508     data.certDate,
509     style: GoogleFonts.tajawal(
510       fontSize: 14,
511       fontWeight: FontWeight.w900,
512       color: template.darkColor,
513     ),
514   ),
515 ),
516 ],
517 );
518 }
519 }
520

```

Lines • 80
Dart

Feature / #125 -

Certificates lib/features/certificates/presentation/widgets/certificate_ready_texts_widget.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/spacing.dart';
6 import 'package:moean/features/certificates/presentation/cubit/certificate_cubit.dart';
7
8 class CertificateReadyTextsWidget extends StatelessWidget {
9   const CertificateReadyTextsWidget({super.key});
10
11   static const List<String> _readyTexts = [
12     'لتفوقها الدراسي وسمو أخلاقها، ونتمنى لها دوام التفوق والنجاح بإذن الله',
13     'لتميزه في المشاركة الصفية والانضباط مع تمنياتنا لها بمزيد من التقدم',
14     'لحسن سلوكه وتعاونه مع زملائه ومعلميه، ونتمنى له مزيداً من العطاء',
15     'لالتزامه بالحضور المنتظم وحرصه على التعلم، ونتمنى له التوفيق والسداد',
16     'تقديرًا لجهوده المبذولة وإبداعه المتميز في العملية التعليمية',
17   ];

```

```

18
19 @override
20 Widget build(BuildContext context) {
21   return Column(
22     crossAxisAlignment: CrossAxisAlignment.start,
23     children: [
24       Text(
25         appTranslation().get('cert_ready_texts'),
26         style: TextStylesManager.bold14
27           .copyWith(color: ColorsManager.mainText),
28       ),
29       verticalSpace8,
30       ..._readyTexts.map(
31         (text) => _ReadyTextItem(text: text),
32       ),
33     ],
34   );
35 }
36 }
37
38 class _ReadyTextItem extends StatelessWidget {
39   final String text;
40
41   const _ReadyTextItem({required this.text});
42
43   @override
44   Widget build(BuildContext context) {
45     return GestureDetector(
46       onTap: () => CertificateCubit.get(context).selectReadyText(text),
47       child: Container(
48         margin: const EdgeInsets.only(bottom: 8),
49         padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 10),
50         decoration: BoxDecoration(
51           color: ColorsManager.primaryColor.withValues(alpha: 0.06),
52           borderRadius: BorderRadius.circular(12),
53           border: Border.all(
54             color: ColorsManager.primaryColor.withValues(alpha: 0.2),
55           ),
56         ),
57         child: Row(
58           children: [
59             Icon(
60               Icons.arrow_back_ios_new_rounded,
61               size: 12,
62               color: ColorsManager.primaryColor,
63             ),
64             horizontalSpace8,
65             Expanded(
66               child: Text(
67                 text,
68                 style: TextStylesManager.regular12
69                   .copyWith(color: ColorsManager.mainText),
70                 maxLines: 2,
71                 overflow: TextOverflow.ellipsis,
72               ),
73             ),
74           ],
75         ),
76       ),
77     );
78 }
79 }
80

```

Lines • 62
Dart

Feature / #126 -
Certificates

[lib/features/certificates/presentation/widgets/certificate_signature_widget.dart](#)

```

1 import 'package:flutter/material.dart';
2 import 'package:google_fonts/google_fonts.dart';
3 import 'package:moean/features/certificates/data/models/certificate_template_model.dart';
4
5 /// Signature block used in the certificate footer.
6 ///

```

```

7  /// Matches the React `Signature` component:
8  /// ```
9  /// <p text-2xl font-bold>{title}</p>
10 /// <div mx-auto mt-3 h-1 w-56 rounded-full bg-accent />
11 /// <p mt-4 text-2xl font-bold>{name}</p>
12 /// ```
13 /// All dimensions are at the 1080-base scale (no scaling needed – the parent
14 /// [CertificatePreviewWidget] applies Transform.scale).
15 class CertificateSignatureWidget extends StatelessWidget {
16   final String title;
17   final String name;
18   final CertificateTemplateModel template;
19
20   const CertificateSignatureWidget({
21     super.key,
22     required this.title,
23     required this.name,
24     required this.template,
25   });
26
27   @override
28   Widget build(BuildContext context) {
29     return Column(
30       mainAxisAlignment: MainAxisAlignment.min,
31       children: [
32         Text(
33           title,
34           style: GoogleFonts.tajawal(
35             fontSize: 24,
36             fontWeight: FontWeight.w700,
37             color: const Color(0xFF0F172A),
38           ),
39         ),
40         const SizedBox(height: 12),
41         Container(
42           width: 224,
43           height: 4,
44           decoration: BoxDecoration(
45             color: template.primaryColor,
46             borderRadius: BorderRadius.circular(999),
47           ),
48         ),
49         const SizedBox(height: 16),
50         Text(
51           name,
52           style: GoogleFonts.tajawal(
53             fontSize: 24,
54             fontWeight: FontWeight.w700,
55             color: const Color(0xFF0F172A),
56           ),
57         ),
58       ],
59     );
60   }
61 }
62

```

Lines • 309
Dart

Feature / #127 –
Certificates

[lib/features/certificates/presentation/widgets/certificate_template_card_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/features/certificates/data/models/certificate_template_model.dart';
4
5  /// Template selector card with a pixel-matched mini thumbnail.
6  ///
7  /// The thumbnail replicates the React `TemplateThumbnail` component:
8  /// ```
9  /// Soft-bg base
10 /// Top gradient header (h-12 of h-28 container ≈ 43%)
11 /// Two white/85% logo-box rects in the top corners
12 /// Two content-line placeholders in the center
13 /// Accent-colored strip bottom-left, dark-colored strip bottom-right

```



```

100         template.label,
101         style: TextStyle(
102           fontSize: 10,
103           fontWeight: FontWeight.w600,
104           color: ColorsManager.secondaryText,
105         ),
106       ),
107     ],
108   ),
109   if (isSelected)
110     Container(
111       width: 20,
112       height: 20,
113       decoration: BoxDecoration(
114         color: ColorsManager.primaryColor,
115         shape: BoxShape.circle,
116       ),
117       child: const Icon(
118         Icons.check_rounded,
119         color: Colors.white,
120         size: 13,
121       ),
122     ),
123   ],
124 ),
125 );
126 }
127
128 // Mini thumbnail matching the React TemplateThumbnail:
129 // ```
130 // <div relative h-28 overflow-hidden style={{background: soft}}>
131 //   <div absolute inset-x-0 top-0 h-12 gradient(accent-dark) />
132 //   <div absolute left-4 top-4 h-6 w-10 rounded-lg bg-white/85 /> ← logo box L
133 //   <div absolute right-4 top-4 h-6 w-10 rounded-lg bg-white/85 /> ← logo box R
134 //   <div absolute left-1/2 top-14 h-2.5 w-36 -translate-x-1/2 rounded-full bg-slate-300 />
135 //   <div absolute left-1/2 top-19 h-2 w-24 -translate-x-1/2 rounded-full bg-slate-200 />
136 //   <div absolute bottom-0 left-0 h-8 w-32 rounded-tr-[42px] accent/72 />
137 //   <div absolute bottom-0 right-0 h-8 w-32 rounded-tl-[42px] dark/72 />
138 //   {modernCorner: 2 extra colored squares at top-left and bottom-right}
139 // </div>
140 // ```
141 // All px values are Tailwind 1-unit=4px: h-12=48, h-8=32, w-10=40, w-32=128,
142 // h-6=24, h-2.5=10, h-2=8, w-36=144, w-24=96, top-14=56, top-19=76, left-4=16.
143 class _TemplateThumbnail extends StatelessWidget {
144   final CertificateTemplateModel template;
145
146   const _TemplateThumbnail({required this.template});
147
148   bool get _isRoyalGold =>
149     template.style == CertificateTemplateStyle.royalGold;
150
151   bool get _isModernCorner =>
152     template.style == CertificateTemplateStyle.modernCorner;
153
154   @override
155   Widget build(BuildContext context) {
156     return LayoutBuilder(
157       builder: (context, constraints) {
158         final w = constraints.maxWidth;
159
160         // Scale factor from the React 200px-wide thumbnail reference
161         final s = w / 200.0;
162
163         return ClipRect(
164           child: Stack(
165             clipBehavior: Clip.hardEdge,
166             children: [
167               // — Base background —————

```

```

164     Positioned.fill(
165         child: Container(color: template.bgColor),
166     ),
167
168     // — Top gradient header  h-12 (48px) —————
169     Positioned(
170         left: 0,
171         right: 0,
172         top: 0,
173         child: Container(
174             height: 48 * s,
175             decoration: BoxDecoration(
176                 gradient: LinearGradient(
177                     begin: Alignment.centerLeft,
178                     end: Alignment.centerRight,
179                     colors: _isRoyalGold
180                         ? [
181                             template.darkColor,
182                             const Color(0xFF4A3416),
183                             template.primaryColor,
184                         ]
185                         : [template.primaryColor, template.darkColor],
186                 ),
187             ),
188         ),
189     ),
190
191     // — Logo box left  left-4(16) top-4(16) h-6(24) w-10(40) —
192     Positioned(
193         left: 16 * s,
194         top: 16 * s,
195         child: Container(
196             width: 40 * s,
197             height: 24 * s,
198             decoration: BoxDecoration(
199                 color: Colors.white.withValues(alpha: 0.85),
200                 borderRadius: BorderRadius.circular(6 * s),
201             ),
202         ),
203     ),
204
205     // — Logo box right  right-4(16) top-4(16) h-6(24) w-10(40) —
206     Positioned(
207         right: 16 * s,
208         top: 16 * s,
209         child: Container(
210             width: 40 * s,
211             height: 24 * s,
212             decoration: BoxDecoration(
213                 color: Colors.white.withValues(alpha: 0.85),
214                 borderRadius: BorderRadius.circular(6 * s),
215             ),
216         ),
217     ),
218
219     // — Title line placeholder  top-14(56) h-2.5(10) w-36(144) —
220     Positioned(
221         left: w / 2 - 72 * s,
222         top: 56 * s,
223         child: Container(
224             width: 144 * s,
225             height: 10 * s,
226             decoration: BoxDecoration(
227                 color: const Color(0xFFCBD5E1),
228                 borderRadius: BorderRadius.circular(5 * s),
229             ),
230         ),
231     ),
232
233     // — Sub-line placeholder  top-19(76) h-2(8) w-24(96) —
234     Positioned(
235         left: w / 2 - 48 * s,
236         top: 76 * s,
237         child: Container(
238             width: 96 * s,

```

```

239         height: 8 * s,
240         decoration: BoxDecoration(
241             color: const Color(0xFFE2E8F0),
242             borderRadius: BorderRadius.circular(4 * s),
243         ),
244     ),
245 ),
246
247 // — Bottom-left accent strip h-8(32) w-32(128) rounded-tr-[42px] —
248 Positioned(
249     left: 0,
250     bottom: 0,
251     child: Container(
252         width: 128 * s,
253         height: 32 * s,
254         decoration: BoxDecoration(
255             color: template.primaryColor.withValues(alpha: 0.72),
256             borderRadius: BorderRadius.only(
257                 topRight: Radius.circular(42 * s),
258             ),
259         ),
260     ),
261 ),
262
263 // — Bottom-right dark strip h-8(32) w-32(128) rounded-tl-[42px] —
264 Positioned(
265     right: 0,
266     bottom: 0,
267     child: Container(
268         width: 128 * s,
269         height: 32 * s,
270         decoration: BoxDecoration(
271             color: template.darkColor.withValues(alpha: 0.72),
272             borderRadius: BorderRadius.only(
273                 topLeft: Radius.circular(42 * s),
274             ),
275         ),
276     ),
277 ),
278
279 // — modernCorner: extra colored squares —————
280 if (_isModernCorner) ...[
281     // top-left pink h-14(56) w-14(56)
282     Positioned(
283         left: 0,
284         top: 0,
285         child: Container(
286             width: 56 * s,
287             height: 56 * s,
288             color: const Color(0xFFFF4436C),
289         ),
290     ),
291     // bottom-right blue h-14(56) w-14(56)
292     Positioned(
293         right: 0,
294         bottom: 0,
295         child: Container(
296             width: 56 * s,
297             height: 56 * s,
298             color: const Color(0xFF2377B8),
299         ),
300     ),
301 ],
302 ],
303 ),
304 );
305 },
306 );
307 }
308 }
309

```

[illegible]


```

71         horizontal: 8, vertical: 3),
72         decoration: BoxDecoration(
73           color: Colors.black.withValues(alpha: 0.45),
74           borderRadius: BorderRadius.circular(8),
75         ),
76         child: Text(
77           '${appTranslation().get('cert_template_label')} ${index + 1}',
78           style: TextStylesManager.bold10
79             .copyWith(color: Colors.white),
80         ),
81       ),
82     ),
83   ],
84 );
85 },
86 ),
87 ],
88 );
89 },
90 );
91 }
92 }
93

```

Lines • 70
Dart

Feature / #129 -

Certificates [lib/features/certificates/presentation/widgets/templates/cert_bg_blue_academic.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/features/certificates/data/models/certificate_template_model.dart';
3  import 'package:moean/features/certificates/presentation/widgets/certificate_dot_pattern_widget.dart';
4
5  /// Template 3 - "تعليمي أزرق" (Blue Academic)
6  ///
7  /// React reference (default / fallback case):
8  /// ```
9  /// <div absolute inset-0 bg-soft />
10 /// <div absolute inset-8 rounded-[40px] border-[3px] border-accent/30 />
11 /// <div absolute inset-x-0 top-0 h-28 bg-gradient-to-l from-accent to-dark />
12 /// <Pattern />
13 /// ```
14 /// Tailwind: h-28 = 112px, inset-8 = 32px.
15 /// `bg-gradient-to-l from-A to-B` → right=A (accent), left=B (dark).
16 class CertBgBlueAcademic extends StatelessWidget {
17   final CertificateTemplateModel template;
18
19   const CertBgBlueAcademic({super.key, required this.template});
20
21   @override
22   Widget build(BuildContext context) {
23     return Stack(
24       children: [
25         // — Base soft background —————
26         Positioned.fill(
27           child: Container(color: template.bgColor),
28         ),
29
30         // — Border frame inset-8 rounded-[40px] border-[3px] accent/30 —
31         Positioned(
32           left: 32,
33           right: 32,
34           top: 32,
35           bottom: 32,
36           child: Container(
37             decoration: BoxDecoration(
38               borderRadius: BorderRadius.circular(40),
39               border: Border.all(
40                 color: template.primaryColor.withValues(alpha: 0.30),
41                 width: 3,
42               ),
43             ),
44           ),
45         ),
46

```

```

47 // — Top gradient bar h-28 from-accent(right) to-dark(left) —
48 Positioned(
49   left: 0,
50   right: 0,
51   top: 0,
52   child: Container(
53     height: 112, // h-28 = 112px
54     decoration: BoxDecoration(
55       gradient: LinearGradient(
56         begin: Alignment.centerRight,
57         end: Alignment.centerLeft,
58         colors: [template.primaryColor, template.darkColor],
59       ),
60     ),
61   ),
62 ),
63
64 // — Repeating dot pattern —
65 const CertificateDotPatternWidget(),
66 ],
67 );
68 }
69 }
70

```

Lines • 110
Dart

Feature / #130 —
Certificates [lib/features/certificates/presentation/widgets/templates/cert_bg_luxury_frame.dart](#)

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/features/certificates/data/models/certificate_template_model.dart';
3
4 /// Template 2 - "إطار فاخر" (Luxury Frame)
5 ///
6 /// React reference:
7 /// ```
8 /// <div absolute inset-0 bg-soft />
9 /// <div absolute inset-8 rounded-[44px] border-[4px] border-accent/35 />
10 /// <div absolute inset-14 rounded-[34px] border border-accent/35 />
11 /// <div absolute left-10 top-10 h-24 w-24 rounded-br-[80px] border-l-[8px] border-t-[8px] border-accent />
12 /// <div absolute right-10 bottom-10 h-24 w-24 rounded-tl-[80px] border-b-[8px] border-r-[8px] border-accent
/>
13 /// ```
14 /// Tailwind scale: 1 unit = 4px.
15 /// inset-8 = 32px, inset-14 = 56px
16 /// h-24 w-24 = 96×96px, left-10 top-10 = 40px, rounded-*-[80px] = 80px radius
17 class CertBgLuxuryFrame extends StatelessWidget {
18   final CertificateTemplateModel template;
19
20   const CertBgLuxuryFrame({super.key, required this.template});
21
22   @override
23   Widget build(BuildContext context) {
24     final accent = template.primaryColor;
25
26     return Stack(
27       children: [
28         // — Base soft background —
29         Positioned.fill(
30           child: Container(color: template.bgColor),
31         ),
32
33         // — Outer frame inset-8 rounded-[44px] border-[4px] accent/35 —
34         Positioned(
35           left: 32,
36           right: 32,
37           top: 32,
38           bottom: 32,
39           child: Container(
40             decoration: BoxDecoration(
41               borderRadius: BorderRadius.circular(44),
42               border: Border.all(
43                 color: accent.withValues(alpha: 0.35),
44                 width: 4,

```

```

45     ),
46     ),
47   ),
48 ),
49
50 // — Inner frame inset-14 rounded-[34px] border-[1px] accent/35 —
51 Positioned(
52   left: 56,
53   right: 56,
54   top: 56,
55   bottom: 56,
56   child: Container(
57     decoration: BoxDecoration(
58       borderRadius: BorderRadius.circular(34),
59       border: Border.all(
60         color: accent.withValues(alpha: 0.35),
61         width: 1,
62       ),
63     ),
64   ),
65 ),
66
67 // — Top-left L-shaped corner accent —————
68 // `h-24 w-24 rounded-br-[80px] border-l-[8px] border-t-[8px]`
69 Positioned(
70   left: 40,
71   top: 40,
72   child: Container(
73     width: 96,
74     height: 96,
75     decoration: BoxDecoration(
76       borderRadius: const BorderRadius.only(
77         bottomRight: Radius.circular(80),
78       ),
79       border: Border(
80         left: BorderSide(color: accent, width: 8),
81         top: BorderSide(color: accent, width: 8),
82       ),
83     ),
84   ),
85 ),
86
87 // — Bottom-right L-shaped corner accent —————
88 // `h-24 w-24 rounded-tl-[80px] border-b-[8px] border-r-[8px]`
89 Positioned(
90   right: 40,
91   bottom: 40,
92   child: Container(
93     width: 96,
94     height: 96,
95     decoration: BoxDecoration(
96       borderRadius: const BorderRadius.only(
97         topLeft: Radius.circular(80),
98       ),
99       border: Border(
100         right: BorderSide(color: accent, width: 8),
101         bottom: BorderSide(color: accent, width: 8),
102       ),
103     ),
104   ),
105 ),
106 ],
107 );
108 }
109 }
110

```

Lines • 69
Dart

Feature / #131 —
Certificates

lib/features/certificates/presentation/widgets/templates/cert_bg_minimal_stamp.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/features/certificates/data/models/certificate_template_model.dart';
3 import 'package:moean/features/certificates/presentation/widgets/certificate_dot_pattern_widget.dart';

```

```

4
5 /// Template 6 - "ختم بسيط" (Minimal Stamp)
6 ///
7 /// React reference (same default structure as blueAcademic, different colors):
8 /// ```
9 /// <div absolute inset-0 bg-soft />
10 /// <div absolute inset-8 rounded-[40px] border-[3px] border-accent/30 />
11 /// <div absolute inset-x-0 top-0 h-28 bg-gradient-to-l from-accent to-dark />
12 /// <Pattern />
13 /// ```
14 /// Tailwind: h-28 = 112px, inset-8 = 32px.
15 class CertBgMinimalStamp extends StatelessWidget {
16   final CertificateTemplateModel template;
17
18   const CertBgMinimalStamp({super.key, required this.template});
19
20   @override
21   Widget build(BuildContext context) {
22     return Stack(
23       children: [
24         // — Base soft background —————
25         Positioned.fill(
26           child: Container(color: template.bgColor),
27         ),
28
29         // — Border frame inset-8 rounded-[40px] border-[3px] accent/30 —
30         Positioned(
31           left: 32,
32           right: 32,
33           top: 32,
34           bottom: 32,
35           child: Container(
36             decoration: BoxDecoration(
37               borderRadius: BorderRadius.circular(40),
38               border: Border.all(
39                 color: template.primaryColor.withValues(alpha: 0.30),
40                 width: 3,
41               ),
42             ),
43           ),
44         ),
45
46         // — Top gradient bar h-28 from-accent(right) to-dark(left) ———
47         Positioned(
48           left: 0,
49           right: 0,
50           top: 0,
51           child: Container(
52             height: 112, // h-28
53             decoration: BoxDecoration(
54               gradient: LinearGradient(
55                 begin: Alignment.centerRight,
56                 end: Alignment.centerLeft,
57                 colors: [template.primaryColor, template.darkColor],
58               ),
59             ),
60           ),
61         ),
62
63         // — Repeating dot pattern —————
64         const CertificateDotPatternWidget(),
65       ],
66     );
67   }
68 }
69

```

Lines • 149
Dart

Feature / #132 -
Certificates

lib/features/certificates/presentation/widgets/templates/cert_bg_modern_corner.dart

```

1 import 'package:flutter/material.dart';
2
3 /// Template 5 - "زوايا حديثة" (Modern Corner)

```

```

4  ///
5  /// React reference:
6  /// ```
7  /// <div absolute inset-0 bg-white />
8  /// <div absolute left-0 top-0 h-64 w-52 bg-[#F4436C] style={{clipPath:"polygon(0 0,100% 0,0 100%)"}} />
9  /// <div absolute right-0 top-0 h-64 w-52 bg-[#FFC857] style={{clipPath:"polygon(0 0,100% 0,100% 100%)"}}
/>
10 /// <div absolute left-0 bottom-0 h-56 w-52 bg-[#0E9F86] style={{clipPath:"polygon(0 0,0 100%,100% 100%)"}}
/>
11 /// <div absolute right-0 bottom-0 h-56 w-52 bg-[#2377B8] style={{clipPath:"polygon(100% 0,0 100%,100% 10
0%)"}} />
12 /// ```
13 /// Tailwind: h-64 = 256px, h-56 = 224px, w-52 = 208px.
14 ///
15 /// ClipPath polygons map as follows to Flutter CustomClipper(Path):
16 /// `polygon(0 0, 100% 0, 0 100%)` → top-left right triangle
17 /// `polygon(0 0, 100% 0, 100% 100%)` → top-right right triangle
18 /// `polygon(0 0, 0 100%, 100% 100%)` → bottom-left right triangle
19 /// `polygon(100% 0, 0 100%, 100% 100%)` → bottom-right right triangle
20 class CertBgModernCorner extends StatelessWidget {
21   const CertBgModernCorner({super.key});
22
23   @override
24   Widget build(BuildContext context) {
25     return Stack(
26       children: [
27         // — White base —————
28         Positioned.fill(child: Container(color: Colors.white)),
29
30         // — Top-left pink triangle h-64(256) w-52(208) —————
31         Positioned(
32           left: 0,
33           top: 0,
34           child: ClipPath(
35             clipper: const _TopLeftTriangleClipper(),
36             child: Container(
37               width: 208,
38               height: 256,
39               color: const Color(0xFFFF4436C),
40             ),
41           ),
42         ),
43
44         // — Top-right yellow/gold triangle h-64(256) w-52(208) —————
45         Positioned(
46           right: 0,
47           top: 0,
48           child: ClipPath(
49             clipper: const _TopRightTriangleClipper(),
50             child: Container(
51               width: 208,
52               height: 256,
53               color: const Color(0xFFFFC857),
54             ),
55           ),
56         ),
57
58         // — Bottom-left green triangle h-56(224) w-52(208) —————
59         Positioned(
60           left: 0,
61           bottom: 0,
62           child: ClipPath(
63             clipper: const _BottomLeftTriangleClipper(),
64             child: Container(
65               width: 208,
66               height: 224,
67               color: const Color(0xFF0E9F86),
68             ),
69           ),
70         ),
71
72         // — Bottom-right blue triangle h-56(224) w-52(208) —————
73         Positioned(
74           right: 0,
75           bottom: 0,

```

```

76         child: ClipPath(
77             clipper: const _BottomRightTriangleClipper(),
78             child: Container(
79                 width: 208,
80                 height: 224,
81                 color: const Color(0xFF2377B8),
82             ),
83         ),
84     ),
85 ],
86 );
87 }
88 }
89
90 // — Clip path implementations —————
91
92 /// polygon(0 0, 100% 0, 0 100%) – top-left right triangle
93 class _TopLeftTriangleClipper extends CustomClipper<Path> {
94     const _TopLeftTriangleClipper();
95
96     @override
97     Path getClip(Size size) => Path()
98         ..lineTo(size.width, 0)
99         ..lineTo(0, size.height)
100         ..close();
101
102     @override
103     bool shouldReclip(covariant CustomClipper<Path> _) => false;
104 }
105
106 /// polygon(0 0, 100% 0, 100% 100%) – top-right right triangle
107 class _TopRightTriangleClipper extends CustomClipper<Path> {
108     const _TopRightTriangleClipper();
109
110     @override
111     Path getClip(Size size) => Path()
112         ..moveTo(0, 0)
113         ..lineTo(size.width, 0)
114         ..lineTo(size.width, size.height)
115         ..close();
116
117     @override
118     bool shouldReclip(covariant CustomClipper<Path> _) => false;
119 }
120
121 /// polygon(0 0, 0 100%, 100% 100%) – bottom-left right triangle
122 class _BottomLeftTriangleClipper extends CustomClipper<Path> {
123     const _BottomLeftTriangleClipper();
124
125     @override
126     Path getClip(Size size) => Path()
127         ..lineTo(0, size.height)
128         ..lineTo(size.width, size.height)
129         ..close();
130
131     @override
132     bool shouldReclip(covariant CustomClipper<Path> _) => false;
133 }
134
135 /// polygon(100% 0, 0 100%, 100% 100%) – bottom-right right triangle
136 class _BottomRightTriangleClipper extends CustomClipper<Path> {
137     const _BottomRightTriangleClipper();
138
139     @override
140     Path getClip(Size size) => Path()
141         ..moveTo(size.width, 0)
142         ..lineTo(0, size.height)
143         ..lineTo(size.width, size.height)
144         ..close();
145
146     @override
147     bool shouldReclip(covariant CustomClipper<Path> _) => false;
148 }
149

```

```
1 import 'package:flutter/material.dart';
2 import 'package:moean/features/certificates/data/models/certificate_template_model.dart';
3 import 'package:moean/features/certificates/presentation/widgets/certificate_dot_pattern_widget.dart';
4
5 /// Template 1 - "رسمي سعودي" (National Ribbon)
6 ///
7 /// React reference:
8 /// ```
9 /// <div absolute inset-x-0 top-0 h-[215px] bg-gradient-to-br from-accent to-dark />
10 /// <div absolute inset-x-[-6%] top-[132px] h-[120px] rounded-[0_0_50%_50%] bg-white />
11 /// <div absolute inset-x-14 top-[236px] h-px bg-accent/25 />
12 /// <Pattern />
13 /// ```
14 /// All coordinates assume the 1080-base width; the parent scales via Transform.scale.
15 class CertBgNationalRibbon extends StatelessWidget {
16   final CertificateTemplateModel template;
17
18   const CertBgNationalRibbon({super.key, required this.template});
19
20   @override
21   Widget build(BuildContext context) {
22     // The React cert is 1080×763.5. The arch is absolutely wider (-6% = -64.8px each side).
23     // We hard-code the 1080-base arithmetic and let the parent Transform.scale handle display.
24     const certWidth = 1080.0;
25     const overdraw = certWidth * 0.06; // 64.8px
26
27     return Stack(
28       clipBehavior: Clip.none,
29       children: [
30         // — Base background —————
31         Positioned.fill(
32           child: Container(color: Colors.white),
33         ),
34
35         // — Gradient header band h-[215px] bg-gradient-to-br —————
36         Positioned(
37           left: 0,
38           right: 0,
39           top: 0,
40           child: Container(
41             height: 215,
42             decoration: BoxDecoration(
43               gradient: LinearGradient(
44                 begin: Alignment.topLeft,
45                 end: Alignment.bottomRight,
46                 colors: [template.primaryColor, template.darkColor],
47               ),
48             ),
49           ),
50         ),
51
52         // — White curved arch top-[132px] h-[120px] rounded-[0_0_50%_50%] —
53         // Extends -6% beyond each side to create the elliptical curve at the bottom.
54         Positioned(
55           top: 132,
56           left: -overdraw,
57           right: -overdraw,
58           child: LayoutBuilder(
59             builder: (_, constraints) {
60               final w = certWidth + overdraw * 2; // 1209.6
61               return Container(
62                 height: 120,
63                 decoration: BoxDecoration(
64                   color: Colors.white,
65                   borderRadius: BorderRadius.only(
66                     bottomLeft: Radius.elliptical(w / 2, 120),
67                     bottomRight: Radius.elliptical(w / 2, 120),
68                   ),
69                 ),
70             );
71           ),
72         ),
73       ],
74     );
75   }
76 }
```

```

72     ),
73     ),
74
75     // — Thin horizontal divider inset-x-14 top-[236px] h-px accent/25 —
76     Positioned(
77       left: 56, // inset-x-14 = 14*4 = 56px
78       right: 56,
79       top: 236,
80       child: Container(
81         height: 1,
82         color: template.primaryColor.withValues(alpha: 0.25),
83       ),
84     ),
85
86     // — Repeating dot pattern —————
87     const CertificateDotPatternWidget(),
88   ],
89 );
90 }
91 }
92

```

Lines • 101
Dart

Feature / #134 —
Certificates

lib/features/certificates/presentation/widgets/templates/cert_bg_royal_gold.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/features/certificates/data/models/certificate_template_model.dart';
3  import 'package:moean/features/certificates/presentation/widgets/certificate_dot_pattern_widget.dart';
4
5  /// Template 4 - "ذهبي ملكي" (Royal Gold)
6  ///
7  /// React reference:
8  /// ```
9  /// <div absolute inset-0 bg-soft />
10 /// <div absolute inset-x-0 top-0 h-36 bg-gradient-to-l from-dark via-#3B2F1E to-dark />
11 /// <div absolute inset-x-20 top-12 h-28 rounded-b-[100px] border-b-[12px] border-gold />
12 /// <div absolute inset-10 rounded-[42px] border-[3px] border-gold/75 />
13 /// <Pattern />
14 /// ```
15 /// Tailwind:
16 ///   h-36 = 144px, inset-x-20 = 80px, top-12 = 48px, h-28 = 112px,
17 ///   rounded-b-[100px] = 100px bottom radius only, border-b-[12px] = 12px bottom border only,
18 ///   inset-10 = 40px.
19 class CertBgRoyalGold extends StatelessWidget {
20   final CertificateTemplateModel template;
21
22   const CertBgRoyalGold({super.key, required this.template});
23
24   static const Color _midBrown = Color(0xFF3B2F1E);
25
26   @override
27   Widget build(BuildContext context) {
28     final dark = template.darkColor;
29     final gold = template.goldColor;
30
31     return Stack(
32       children: [
33         // — Base soft background —————
34         Positioned.fill(
35           child: Container(color: template.bgColor),
36         ),
37
38         // — Dark header h-36 gradient from-dark via-midBrown to-dark —
39         // `bg-gradient-to-l from-dark via-#3B2F1E to-dark` (symmetric)
40         Positioned(
41           left: 0,
42           right: 0,
43           top: 0,
44           child: Container(
45             height: 144, // h-36
46             decoration: BoxDecoration(
47               gradient: LinearGradient(
48                 begin: Alignment.centerRight,

```



```

49         end: Alignment.centerLeft,
50         colors: [dark, _midBrown, dark],
51         stops: const [0.0, 0.5, 1.0],
52     ),
53 ),
54 ),
55 ),
56
57 // — Gold arc inset-x-20 top-12 h-28 rounded-b-[100px] border-b-[12px] gold —
58 // Only the bottom border and bottom border-radius is drawn – creating a
59 // "U-shaped" gold arc that peeks below the dark header.
60 Positioned(
61   left: 80, // inset-x-20 = 80px
62   right: 80,
63   top: 48, // top-12 = 48px
64   child: Container(
65     height: 112, // h-28
66     decoration: BoxDecoration(
67       borderRadius: const BorderRadius.only(
68         bottomLeft: Radius.circular(100),
69         bottomRight: Radius.circular(100),
70       ),
71       border: Border(
72         bottom: BorderSide(color: gold, width: 12),
73       ),
74     ),
75   ),
76 ),
77
78 // — Gold full frame inset-10 rounded-[42px] border-[3px] gold/75 —
79 Positioned(
80   left: 40, // inset-10 = 40px
81   right: 40,
82   top: 40,
83   bottom: 40,
84   child: Container(
85     decoration: BoxDecoration(
86       borderRadius: BorderRadius.circular(42),
87       border: Border.all(
88         color: gold.withValues(alpha: 0.75),
89         width: 3,
90       ),
91     ),
92   ),
93 ),
94
95 // — Repeating dot pattern —————
96 const CertificateDotPatternWidget(),
97 ],
98 );
99 }
100 }
101

```

Lines • Dart 108

Feature / Contact

#135 – lib/features/contact/presentation/cubit/contact_cubit.dart

```

1  import 'package:flutter_bloc/flutter_bloc.dart';
2  import 'package:moean/core/network/remote/api_service.dart';
3  import 'package:moean/features/contact/presentation/cubit/contact_state.dart';
4
5  class ContactCubit extends Cubit<ContactState> {
6    ContactCubit() : super(ContactInitial());
7
8    static ContactCubit get(context) => BlocProvider.of(context);
9
10   List<dynamic> types = [];
11   List<dynamic> myTickets = [];
12
13   Future<void> getContactTypes() async {
14     if (!isClosed) emit(ContactTypesLoading());
15
16     final result = await ApiService.getContactTypes();
17     if (isClosed) return;

```

```

18     result.fold(
19         (error) {
20             if (!isClosed) emit(ContactTypesError(error));
21         },
22         (data) {
23             types = data['types'] ?? [];
24             if (!isClosed) emit(ContactTypesLoaded(types));
25         },
26     );
27 }
28
29 Future<void> submitContactRequest({
30     required String name,
31     required String email,
32     String? phone,
33     required String type,
34     required String message,
35 }) async {
36     if (!isClosed) emit(ContactSubmitLoading());
37
38     final result = await ApiService.submitContactRequest(
39         name: name,
40         email: email,
41         phone: phone,
42         type: type,
43         message: message,
44     );
45     if (isClosed) return;
46
47     result.fold(
48         (error) {
49             if (!isClosed) emit(ContactSubmitError(error));
50         },
51         (data) {
52             if (!isClosed) emit(ContactSubmitSuccess(data['message'] ?? 'تم إرسال طلبك بنجاح.));
53         },
54     );
55 }
56
57 Future<void> getMyTickets() async {
58     if (!isClosed) emit(ContactMyTicketsLoading());
59
60     final result = await ApiService.getMyContactTickets();
61     if (isClosed) return;
62     result.fold(
63         (error) {
64             if (!isClosed) emit(ContactMyTicketsError(error));
65         },
66         (data) {
67             myTickets = data['data'] ?? [];
68             if (!isClosed) emit(ContactMyTicketsLoaded(myTickets));
69         },
70     );
71 }
72
73 Map<String, dynamic>? currentTicket;
74
75 Future<void> getTicketDetails(int id) async {
76     if (!isClosed) emit(ContactTicketDetailsLoading());
77
78     final result = await ApiService.getMyContactTicketDetails(id: id);
79     if (isClosed) return;
80     result.fold(
81         (error) {
82             if (!isClosed) emit(ContactTicketDetailsError(error));
83         },
84         (data) {
85             currentTicket = data['ticket'] ?? data;
86             if (!isClosed) emit(ContactTicketDetailsLoaded());
87         },
88     );
89 }
90
91 Future<void> replyToTicket({required int id, required String body}) async {
92     if (!isClosed) emit(ContactTicketReplyLoading());

```

```

93
94     final result = await ApiService.replyContactTicket(id: id, body: body);
95     if (isClosed) return;
96     result.fold(
97       (error) {
98         if (!isClosed) emit(ContactTicketReplyError(error));
99       },
100      (data) {
101        if (!isClosed) emit(ContactTicketReplySuccess(data['message'] ?? 'تم إرسال الرد بنجاح'));
102        // Refresh ticket details to get the new reply
103        getTicketDetails(id);
104      },
105    );
106  }
107 }
108

```

Lines • Dart 69

Feature / Contact

#136 – lib/features/contact/presentation/cubit/contact_state.dart

```

1  import 'package:flutter/material.dart';
2
3  @immutable
4  abstract class ContactState {}
5
6  class ContactInitial extends ContactState {}
7
8  // Types
9  class ContactTypesLoading extends ContactState {}
10
11 class ContactTypesLoaded extends ContactState {
12   final List<dynamic> types;
13   ContactTypesLoaded(this.types);
14 }
15
16 class ContactTypesError extends ContactState {
17   final String error;
18   ContactTypesError(this.error);
19 }
20
21 // Submit Request
22 class ContactSubmitLoading extends ContactState {}
23
24 class ContactSubmitSuccess extends ContactState {
25   final String message;
26   ContactSubmitSuccess(this.message);
27 }
28
29 class ContactSubmitError extends ContactState {
30   final String error;
31   ContactSubmitError(this.error);
32 }
33
34 // My Tickets
35 class ContactMyTicketsLoading extends ContactState {}
36
37 class ContactMyTicketsLoaded extends ContactState {
38   final List<dynamic> tickets;
39   ContactMyTicketsLoaded(this.tickets);
40 }
41
42 class ContactMyTicketsError extends ContactState {
43   final String error;
44   ContactMyTicketsError(this.error);
45 }
46
47 // Ticket Details
48 class ContactTicketDetailsLoading extends ContactState {}
49
50 class ContactTicketDetailsLoaded extends ContactState {}
51
52 class ContactTicketDetailsError extends ContactState {
53   final String error;
54   ContactTicketDetailsError(this.error);

```

```

55 }
56
57 // Reply
58 class ContactTicketReplyLoading extends ContactState {}
59
60 class ContactTicketReplySuccess extends ContactState {
61   final String message;
62   ContactTicketReplySuccess(this.message);
63 }
64
65 class ContactTicketReplyError extends ContactState {
66   final String error;
67   ContactTicketReplyError(this.error);
68 }
69

```

Lines • Dart 158

Feature / Contact

#137 – lib/features/contact/presentation/screen/contact_support_screen.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
6  import 'package:moean/core/utils/constants/routes.dart';
7  import 'package:moean/core/utils/constants/spacing.dart';
8  import 'package:moean/core/utils/extensions/context_extension.dart';
9
10 class ContactSupportScreen extends StatelessWidget {
11   const ContactSupportScreen({super.key});
12
13   @override
14   Widget build(BuildContext context) {
15     return Directionality(
16       textDirection: TextDirection.rtl,
17       child: Scaffold(
18         backgroundColor: ColorsManager.background,
19         appBar: AppBar(
20           backgroundColor: ColorsManager.background,
21           elevation: 0,
22           leading: IconButton(
23             icon: Icon(Icons.arrow_back, color: ColorsManager.mainText),
24             onPressed: () => context.pop(),
25           ),
26         ),
27         body: SafeArea(
28           child: Padding(
29             padding: const EdgeInsets.symmetric(horizontal: 24.0),
30             child: Column(
31               crossAxisAlignment: CrossAxisAlignment.stretch,
32               children: [
33                 Text(
34                   appTranslation().get('contact_support_title'),
35                   style: TextStyleManager.bold24.copyWith(color: ColorsManager.primaryColor, height: 1.4),
36                   textAlign: TextAlign.right,
37                 ),
38                 verticalSpace12,
39                 Text(
40                   appTranslation().get('contact_support_subtitle'),
41                   style: TextStyleManager.regular14.copyWith(color: ColorsManager.secondaryText),
42                   textAlign: TextAlign.right,
43                 ),
44                 verticalSpace32,
45                 Expanded(
46                   child: GridView.count(
47                     crossAxisCount: 2,
48                     crossAxisSpacing: 16,
49                     mainAxisSpacing: 16,
50                     childAspectRatio: 1.1,
51                     children: [
52                       ContactOptionWidget(
53                         icon: Icons.chat_bubble_outline,
54                         title: appTranslation().get('whatsapp_support'),
55                         subtitle: appTranslation().get('fast_communication_support'),

```

```

56         ),
57         ContactOptionWidget(
58             icon: Icons.email_outlined,
59             title: appTranslation().get('email_support'),
60             subtitle: appTranslation().get('support_email'),
61         ),
62         ContactOptionWidget(
63             icon: Icons.location_on_outlined,
64             title: appTranslation().get('location'),
65             subtitle: appTranslation().get('saudi_arabia'),
66         ),
67         ContactOptionWidget(
68             icon: Icons.access_time,
69             title: appTranslation().get('working_hours'),
70             subtitle: appTranslation().get('working_hours_details'),
71         ),
72     ],
73 ),
74 ),
75 Container(
76     padding: const EdgeInsets.all(16),
77     decoration: BoxDecoration(
78         color: ColorsManager.primaryColor.withValues(alpha: 0.1),
79         borderRadius: BorderRadius.circular(16),
80     ),
81     child: Row(
82         children: [
83             Icon(Icons.headset_mic_outlined, color: ColorsManager.primaryColor),
84             horizontalSpace12,
85             Expanded(
86                 child: Text(
87                     appTranslation().get('team_close_to_you'),
88                     style: TextStylesManager.bold12.copyWith(color: ColorsManager.primaryColor),
89                 ),
90             ),
91         ],
92     ),
93 ),
94 verticalSpace24,
95 PrimaryElevatedButton(
96     text: appTranslation().get('my_past_requests'),
97     onPressed: () {
98         context.push(Routes.myTickets);
99     },
100 ),
101 verticalSpace24,
102 ],
103 ),
104 ),
105 ),
106 ),
107 );
108 }
109 }
110
111 class ContactOptionWidget extends StatelessWidget {
112     final IconData icon;
113     final String title;
114     final String subtitle;
115
116     const ContactOptionWidget({
117         super.key,
118         required this.icon,
119         required this.title,
120         required this.subtitle,
121     });
122
123     @override
124     Widget build(BuildContext context) {
125         return Container(
126             padding: const EdgeInsets.all(16),
127             decoration: BoxDecoration(
128                 color: ColorsManager.surfacePrimary,
129                 borderRadius: BorderRadius.circular(16),
130                 boxShadow: [

```

```

131     BoxShadow(
132       color: Colors.black.withValues(alpha: 0.05),
133       blurRadius: 10,
134       offset: const Offset(0, 4),
135     ),
136   ],
137 ),
138 child: Column(
139   crossAxisAlignment: CrossAxisAlignment.start,
140   mainAxisAlignment: MainAxisAlignment.center,
141   children: [
142     Icon(icon, color: ColorsManager.primaryColor, size: 28),
143     verticalSpace12,
144     Text(
145       title,
146       style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainText),
147     ),
148     verticalSpace4,
149     Text(
150       subtitle,
151       style: TextStylesManager.regular10.copyWith(color: ColorsManager.secondaryText),
152     ),
153   ],
154 ),
155 );
156 }
157 }
158

```

Lines • Dart 328

Feature / Contact

#138 – lib/features/contact/presentation/screen/create_ticket_screen.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
7  import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
8  import 'package:moean/core/utils/constants/spacing.dart';
9  import 'package:moean/core/utils/extensions/context_extension.dart';
10 import 'package:moean/features/contact/presentation/cubit/contact_cubit.dart';
11 import 'package:moean/features/contact/presentation/cubit/contact_state.dart';
12 import 'package:moean/core/utils/constants/primary/conditional_builder.dart';
13
14 class CreateTicketScreen extends StatefulWidget {
15   const CreateTicketScreen({super.key});
16
17   @override
18   State<CreateTicketScreen> createState() => _CreateTicketScreenState();
19 }
20
21 class _CreateTicketScreenState extends State<CreateTicketScreen> {
22   final _formKey = GlobalKey<FormState>();
23   final _nameController = TextEditingController();
24   final _emailController = TextEditingController();
25   final _phoneController = TextEditingController();
26   final _messageController = TextEditingController();
27   String? _selectedType;
28
29   @override
30   void initState() {
31     super.initState();
32     ContactCubit.get(context).getContactTypes();
33   }
34
35   @override
36   void dispose() {
37     _nameController.dispose();
38     _emailController.dispose();
39     _phoneController.dispose();
40     _messageController.dispose();
41     super.dispose();
42   }

```

```

43
44 @override
45 Widget build(BuildContext context) {
46   return BlocConsumer<ContactCubit, ContactState>(
47     listener: (context, state) {
48       if (state is ContactSubmitSuccess) {
49         ScaffoldMessenger.of(context).showSnackBar(
50           SnackBar(content: Text(state.message, style: TextStyleManager.bold14)),
51         );
52         context.pop();
53       } else if (state is ContactSubmitError) {
54         ScaffoldMessenger.of(context).showSnackBar(
55           SnackBar(content: Text(state.error, style: TextStyleManager.bold14)),
56         );
57       }
58     },
59     builder: (context, state) {
60       final cubit = ContactCubit.get(context);
61       return Directionality(
62         textDirection: TextDirection.rtl,
63         child: Scaffold(
64           backgroundColor: ColorsManager.background,
65           appBar:
66             // AppBar(
67             //   backgroundColor: ColorsManager.background,
68             //   elevation: 0,
69             //   centerTitle: true,
70             //   title: Text(
71             //     appTranslation().get('new_request'),
72             //     style: TextStyleManager.bold18.copyWith(color: ColorsManager.mainText),
73             //   ),
74             //   leading: IconButton(
75             //     icon: Icon(Icons.arrow_forward, color: ColorsManager.mainText),
76             //     onPressed: () => context.pop(),
77             //   ),
78             // ),
79             AppBar(
80               backgroundColor: ColorsManager.background,
81               elevation: 0,
82               centerTitle: true,
83               title: Text(
84                 appTranslation().get('new_request'),
85                 style: TextStyleManager.bold18.copyWith(color: ColorsManager.mainText),
86               ),
87               leading: IconButton(
88                 onPressed: () => Navigator.pop(context),
89                 icon: Icon(
90                   Icons.arrow_back_ios_rounded,
91                   color: ColorsManager.mainText,
92                   size: 20,
93                 ),
94             ),
95           ),
96         body: SingleChildScrollView(
97           padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
98           child: Form(
99             key: _formKey,
100             child: Column(
101               crossAxisAlignment: CrossAxisAlignment.stretch,
102               children: [
103                 const CreateTicketInfoWidget(),
104                 verticalSpace24,
105                 Text(
106                   appTranslation().get('name'),
107                   style: TextStyleManager.bold14,
108                 ),
109                 verticalSpace8,
110                 PrimaryTextField(
111                   controller: _nameController,
112                   hint: appTranslation().get('name'),
113                   prefixIcon: const Icon(Icons.person_outline),
114                   validator: (value) {
115                     if (value == null || value.isEmpty) {
116                       return appTranslation().get('field_required');
117                     }

```

```

118         return null;
119     },
120 ),
121 verticalSpace16,
122 Text(
123     appTranslation().get('email'),
124     style: TextStylesManager.bold14,
125 ),
126 verticalSpace8,
127 PrimaryTextField(
128     controller: _emailController,
129     hint: appTranslation().get('email'),
130     prefixIcon: const Icon(Icons.email_outlined),
131     keyboardType: TextInputType.emailAddress,
132     validator: (value) {
133         if (value == null || value.isEmpty) {
134             return appTranslation().get('field_required');
135         }
136         return null;
137     },
138 ),
139 verticalSpace16,
140 Text(
141     appTranslation().get('phone'),
142     style: TextStylesManager.bold14,
143 ),
144 verticalSpace8,
145 PrimaryTextField(
146     controller: _phoneController,
147     hint: '966 5X XXX XXXX+',
148     prefixIcon: const Icon(Icons.phone_outlined),
149     keyboardType: TextInputType.phone,
150 ),
151 verticalSpace16,
152 Text(
153     appTranslation().get('request_type'),
154     style: TextStylesManager.bold14,
155 ),
156 verticalSpace8,
157 CreateTicketTypesDropdownWidget(
158     cubit: cubit,
159     state: state,
160     selectedType: _selectedType,
161     onChanged: (value) {
162         setState(() {
163             _selectedType = value;
164         });
165     },
166 ),
167 verticalSpace16,
168 Text(
169     appTranslation().get('message'),
170     style: TextStylesManager.bold14,
171 ),
172 verticalSpace8,
173 PrimaryTextField(
174     controller: _messageController,
175     hint: appTranslation().get('write_details_here'),
176     maxLines: 4,
177     validator: (value) {
178         if (value == null || value.isEmpty) {
179             return appTranslation().get('field_required');
180         }
181         return null;
182     },
183 ),
184 verticalSpace32,
185 ConditionalBuilder(
186     loadingState: state is ContactSubmitLoading,
187     successBuilder: (context) => PrimaryElevatedButton(
188         text: appTranslation().get('send_request'),
189         icon: const Icon(Icons.send_outlined, color: ColorsManager.white, size: 20),
190         onPressed: () {
191             if (_formKey.currentState!.validate()) {
192                 if (_selectedType == null) {

```



```

193 ScaffoldMessenger.of(context).showSnackBar(
194     SnackBar(content: Text(appTranslation().get('please_select_request_type'))),
195 );
196 return;
197 }
198 cubit.submitContactRequest(
199     name: _nameController.text,
200     email: _emailController.text,
201     phone: _phoneController.text,
202     type: _selectedType!,
203     message: _messageController.text,
204 );
205 }
206 },
207 ),
208 ),
209     verticalSpace32,
210 ],
211 ),
212 ),
213 ),
214 ),
215 );
216 },
217 );
218 }
219
220 }
221
222 class CreateTicketInfoWidget extends StatelessWidget {
223     const CreateTicketInfoWidget({super.key});
224
225     @override
226     Widget build(BuildContext context) {
227         return Container(
228             padding: const EdgeInsets.all(16),
229             decoration: BoxDecoration(
230                 color: ColorsManager.surfacePrimary,
231                 borderRadius: BorderRadius.circular(16),
232                 border: Border.all(color: ColorsManager.statusSuccess.withValues(alpha: 0.3)),
233             ),
234             child: Column(
235                 crossAxisAlignment: CrossAxisAlignment.start,
236                 children: [
237                     Row(
238                         children: [
239                             Icon(Icons.shield_outlined, color: ColorsManager.statusWarning),
240                             horizontalSpace8,
241                             Text(
242                                 appTranslation().get('before_sending_request'),
243                                 style: TextStyleManager.bold14.copyWith(color: ColorsManager.mainText),
244                             ),
245                         ],
246                     ),
247                     verticalSpace12,
248                     Row(
249                         crossAxisAlignment: CrossAxisAlignment.start,
250                         children: [
251                             const Icon(Icons.check_circle_outline, color: ColorsManager.statusSuccess, size: 18),
252                             horizontalSpace8,
253                             Expanded(
254                                 child: Text(
255                                     appTranslation().get('help_us_help_you'),
256                                     style: TextStyleManager.regular12.copyWith(color: ColorsManager.secondaryText),
257                                 ),
258                             ),
259                         ],
260                     ),
261                     verticalSpace8,
262                     Row(
263                         crossAxisAlignment: CrossAxisAlignment.start,
264                         children: [
265                             const Icon(Icons.check_circle_outline, color: ColorsManager.statusSuccess, size: 18),
266                             horizontalSpace8,
267                             Expanded(

```

```

268         child: Text(
269             appTranslation().get('where_error_happened'),
270             style: TextStyleManager.regular12.copyWith(color: ColorsManager.secondaryText),
271         ),
272     ),
273 ],
274 ),
275 ],
276 ),
277 );
278 }
279 }
280
281 class CreateTicketTypesDropdownWidget extends StatelessWidget {
282     final ContactCubit cubit;
283     final ContactState state;
284     final String? selectedType;
285     final ValueChanged<String?> onChanged;
286
287     const CreateTicketTypesDropdownWidget({
288         super.key,
289         required this.cubit,
290         required this.state,
291         required this.selectedType,
292         required this.onChanged,
293     });
294
295     @override
296     Widget build(BuildContext context) {
297         if (state is ContactTypesLoading) {
298             return const Center(child: CircularProgressIndicator());
299         }
300         return Container(
301             padding: const EdgeInsets.symmetric(horizontal: 16),
302             decoration: BoxDecoration(
303                 color: ColorsManager.surfacePrimary,
304                 borderRadius: BorderRadius.circular(12),
305                 border: Border.all(color: ColorsManager.borderColor),
306             ),
307             child: DropdownButtonHideUnderline(
308                 child: DropdownButton<String>(
309                     value: selectedType,
310                     isExpanded: true,
311                     hint: Text(
312                         appTranslation().get('select_request_type'),
313                         style: TextStyleManager.regular14.copyWith(color: ColorsManager.secondaryText),
314                     ),
315                     items: cubit.types.map<DropdownMenuItem<String>>((dynamic type) {
316                         return DropdownMenuItem<String>(
317                             value: type['value'],
318                             child: Text(type['label'], style: TextStyleManager.regular14.copyWith(color: ColorsManager.ma
319 nText))),
319                             );
320                     }).toList(),
321                     onChanged: onChanged,
322                 ),
323             ),
324         );
325     }
326 }
327
328

```

Lines • Dart 227

Feature / Contact

#139 – lib/features/contact/presentation/screen/my_tickets_screen.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
7 import 'package:moean/core/utils/constants/routes.dart';
8 import 'package:moean/core/utils/constants/spacing.dart';

```

```

9 import 'package:moean/core/utils/extensions/context_extension.dart';
10 import 'package:moean/features/contact/presentation/cubit/contact_cubit.dart';
11 import 'package:moean/features/contact/presentation/cubit/contact_state.dart';
12
13 class MyTicketsScreen extends StatefulWidget {
14   const MyTicketsScreen({super.key});
15
16   @override
17   State<MyTicketsScreen> createState() => _MyTicketsScreenState();
18 }
19
20 class _MyTicketsScreenState extends State<MyTicketsScreen> {
21   @override
22   void initState() {
23     super.initState();
24     ContactCubit.get(context).getMyTickets();
25   }
26
27   @override
28   Widget build(BuildContext context) {
29     return BlocBuilder<ContactCubit, ContactState>(
30       buildWhen: (previous, current) => current is ContactMyTicketsLoading || current is ContactMyTicketsLoad
ed || current is ContactMyTicketsError,
31       builder: (context, state) {
32         return Directionality(
33           textDirection: TextDirection.rtl,
34           child: Scaffold(
35             backgroundColor: ColorsManager.background,
36             appBar: AppBar(
37               backgroundColor: ColorsManager.background,
38               elevation: 0,
39               centerTitle: true,
40               title: Text(
41                 appTranslation().get('my_requests_conversations'),
42                 style: TextStyleManager.bold18.copyWith(color: ColorsManager.mainText),
43               ),
44               leading: IconButton(
45                 icon: Icon(Icons.arrow_back_ios, color: ColorsManager.mainText),
46                 onPressed: () => context.pop(),
47               ),
48               actions: [
49                 IconButton(
50                   icon: Icon(Icons.refresh, color: ColorsManager.primaryColor),
51                   onPressed: () => ContactCubit.get(context).getMyTickets(),
52                 ),
53               ],
54             ),
55             body: SafeArea(
56               child: Padding(
57                 padding: const EdgeInsets.symmetric(horizontal: 24.0, vertical: 16),
58                 child: Column(
59                   crossAxisAlignment: CrossAxisAlignment.stretch,
60                   children: [
61                     Expanded(
62                       child: _buildTicketsList(state),
63                     ),
64                     verticalSpace16,
65                     PrimaryElevatedButton(
66                       text: appTranslation().get('new_request'),
67                       onPressed: () {
68                         Navigator.pushNamed(context, Routes.createTicket).then((_) {
69                           if (context.mounted) {
70                             ContactCubit.get(context).getMyTickets();
71                           }
72                         });
73                       },
74                     ),
75                   ],
76                 ),
77               ),
78             ),
79           );
80         );
81       },
82     );

```

```

83     }
84
85     Widget _buildTicketsList(ContactState state) {
86       if (state is ContactMyTicketsLoading) {
87         return const Center(child: CircularProgressIndicator());
88       } else if (state is ContactMyTicketsError) {
89         return Center(child: Text(state.error, style: TextStyleManager.bold14.copyWith(color: Colors.red)));
90       } else if (state is ContactMyTicketsLoaded) {
91         final tickets = state.tickets;
92         if (tickets.isEmpty) {
93           return Center(
94             child: Text(
95               appTranslation().get('no_tickets'),
96               style: TextStyleManager.bold16.copyWith(color: Colors.grey),
97             ),
98           );
99         }
100         return ListView.separated(
101           itemCount: tickets.length,
102           separatorBuilder: (context, index) => verticalSpace16,
103           itemBuilder: (context, index) {
104             final ticket = tickets[index];
105             return TeacherTicketItemWidget(ticket: ticket);
106           },
107         );
108       }
109       return const SizedBox.shrink();
110     }
111   }
112
113   class TeacherTicketItemWidget extends StatelessWidget {
114     final dynamic ticket;
115
116     const TeacherTicketItemWidget({super.key, required this.ticket});
117
118     @override
119     Widget build(BuildContext context) {
120       final status = ticket['status'] ?? 'pending';
121       final statusLabel = ticket['status_label'] ?? appTranslation().get('pending');
122       final typeLabel = ticket['type_label'] ?? '';
123       final message = ticket['message'] ?? '';
124       final repliesCount = ticket['replies_count'] ?? 0;
125       final createdAt = ticket['created_at'] != null ? DateTime.tryParse(ticket['created_at']) : null;
126       final ticketId = ticket['id'] != null ? '#${ticket['id']}' : '';
127
128       Color statusColor;
129       switch (status) {
130         case 'resolved':
131           statusColor = Colors.green;
132           break;
133         case 'in_progress':
134           statusColor = Colors.blue;
135           break;
136         case 'pending':
137           statusColor = Colors.orange;
138           break;
139         default:
140           statusColor = Colors.grey;
141       }
142
143       return InkWell(
144         onTap: () {
145           Navigator.pushNamed(context, Routes.ticketDetails, arguments: {'id': ticket['id']}).then((_) {
146             if (context.mounted) {
147               ContactCubit.get(context).getMyTickets();
148             }
149           });
150         },
151         borderRadius: BorderRadius.circular(12),
152         child: Container(
153           padding: const EdgeInsets.all(16),
154           decoration: BoxDecoration(
155             color: ColorsManager.surfacePrimary,
156             borderRadius: BorderRadius.circular(12),
157             border: Border.all(color: ColorsManager.borderColor),

```

```

158     ),
159     child: Column(
160       crossAxisAlignment: CrossAxisAlignment.start,
161       children: [
162         Row(
163           mainAxisAlignment: MainAxisAlignment.spaceBetween,
164           children: [
165             Expanded(
166               child: Text(
167                 message,
168                 maxLines: 1,
169                 overflow: TextOverflow.ellipsis,
170                 style: TextStyleManager.bold16.copyWith(color: ColorsManager.mainText),
171               ),
172             ),
173             horizontalSpace8,
174             Container(
175               padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
176               decoration: BoxDecoration(
177                 color: statusColor.withValues(alpha: 0.1),
178                 borderRadius: BorderRadius.circular(20),
179                 border: Border.all(color: statusColor.withValues(alpha: 0.5)),
180               ),
181               child: Row(
182                 children: [
183                   Icon(Icons.info_outline, size: 14, color: statusColor),
184                   horizontalSpace6,
185                   Text(
186                     statusLabel,
187                     style: TextStyleManager.bold12.copyWith(color: statusColor),
188                   ),
189                 ],
190               ),
191             ),
192           ],
193         ),
194         verticalSpace4,
195         Text(
196           '$ticketId • $repliesCount ${appTranslation().get('replies') ?? 'رد'}',
197           style: TextStyleManager.regular12.copyWith(color: ColorsManager.secondaryText),
198         ),
199         verticalSpace12,
200         Row(
201           mainAxisAlignment: MainAxisAlignment.spaceBetween,
202           children: [
203             Container(
204               padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
205               decoration: BoxDecoration(
206                 color: ColorsManager.isDark ? Colors.white.withValues(alpha: 0.05) : Colors.grey.shade10
0,
207                 borderRadius: BorderRadius.circular(20),
208               ),
209               child: Text(
210                 typeLabel,
211                 style: TextStyleManager.bold12.copyWith(color: ColorsManager.mainText),
212               ),
213             ),
214             if (createdAt != null)
215               Text(
216                 '${createdAt.year}-${createdAt.month.toString().padLeft(2, '0')}-${createdAt.day.toString
().padLeft(2, '0')}',
217                 style: TextStyleManager.regular12.copyWith(color: ColorsManager.secondaryText),
218               ),
219             ],
220           ),
221         ],
222       ),
223     ),
224   );
225 }
226 }
227

```

```
1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/spacing.dart';
7 import 'package:moean/core/utils/extensions/context_extension.dart';
8 import 'package:moean/features/contact/presentation/cubit/contact_cubit.dart';
9 import 'package:moean/features/contact/presentation/cubit/contact_state.dart';
10 import 'package:moean/features/contact/presentation/widgets/ticket_chat_history.dart';
11
12 class TicketDetailsScreen extends StatefulWidget {
13   const TicketDetailsScreen({super.key});
14
15   @override
16   State<TicketDetailsScreen> createState() => _TicketDetailsScreenState();
17 }
18
19 class _TicketDetailsScreenState extends State<TicketDetailsScreen> {
20   late int ticketId;
21   bool _isInit = false;
22
23   @override
24   void didChangeDependencies() {
25     super.didChangeDependencies();
26     if (!_isInit) {
27       final args = context.getArg<Map<String, dynamic>>();
28       ticketId = args?['id'] ?? 0;
29       if (ticketId != 0) {
30         ContactCubit.get(context).getTicketDetails(ticketId);
31       }
32       _isInit = true;
33     }
34   }
35
36   @override
37   Widget build(BuildContext context) {
38     return Directionality(
39       textDirection: TextDirection.rtl,
40       child: Scaffold(
41         backgroundColor: ColorsManager.background,
42         appBar: AppBar(
43           backgroundColor: ColorsManager.background,
44           elevation: 0,
45           leading: IconButton(
46             icon: const Icon(Icons.arrow_back, color: ColorsManager.black),
47             onPressed: () => context.pop(true),
48           ),
49           title: Column(
50             children: [
51               Text(
52                 appTranslation().get('ticket_details') ?? 'تفاصيل التذكرة',
53                 style: TextStylesManager.bold16.copyWith(color: ColorsManager.black),
54               ),
55               verticalSpace4,
56               Text(
57                 '${appTranslation().get('ticket_number') ?? 'التذكرة رقم'} #$ticketId',
58                 style: TextStylesManager.regular14.copyWith(color: Colors.grey.shade600),
59               ),
60             ],
61           ),
62           centerTitle: true,
63         ),
64         body: BlocConsumer<ContactCubit, ContactState>(
65           listener: (context, state) {
66             if (state is ContactTicketReplySuccess) {
67               ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(state.message, style: TextStylesManager.bold14.copyWith(color: ColorsManager.white)), backgroundColor: Colors.green));
68             } else if (state is ContactTicketReplyError) {
69               ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(state.error, style: TextStylesManager.bold14.copyWith(color: ColorsManager.white)), backgroundColor: Colors.red));
70             }
71           },
72         ),
73       ),
74     );
75   }
76 }
```

```

71         },
72         buildWhen: (previous, current) => current is ContactTicketDetailsLoading || current is ContactTicke
tDetailsLoaded || current is ContactTicketDetailsError,
73         builder: (context, state) {
74             if (state is ContactTicketDetailsLoading || ContactCubit.get(context).currentTicket == null) {
75                 return const Center(child: CircularProgressIndicator());
76             } else if (state is ContactTicketDetailsError) {
77                 return Center(child: Text(state.error, style: TextStyleManager.bold16.copyWith(color: Colors.r
ed)));
78             }
79
80             final ticket = ContactCubit.get(context).currentTicket!;
81             return Padding(
82                 padding: const EdgeInsets.all(24.0),
83                 child: SingleChildScrollView(
84                     child: Center(
85                         child: ConstrainedBox(
86                             constraints: const BoxConstraints(maxWidth: 800),
87                             child: TicketChatHistory(ticket: ticket),
88                         ),
89                     ),
90                 ),
91             );
92         },
93     ),
94 ),
95 );
96 }
97 }
98

```

Lines • Dart 368

Feature / Contact

#141 – lib/features/contact/presentation/widgets/ticket_chat_history.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6  import 'package:moean/features/contact/presentation/cubit/contact_cubit.dart';
7
8  class TicketChatHistory extends StatefulWidget {
9      final Map<String, dynamic> ticket;
10
11      const TicketChatHistory({super.key, required this.ticket});
12
13      @override
14      State<TicketChatHistory> createState() => _TicketChatHistoryState();
15  }
16
17  class _TicketChatHistoryState extends State<TicketChatHistory> {
18      final TextEditingController _replyController = TextEditingController();
19
20      @override
21      void dispose() {
22          _replyController.dispose();
23          super.dispose();
24      }
25
26      @override
27      Widget build(BuildContext context) {
28          return Column(
29              crossAxisAlignment: CrossAxisAlignment.stretch,
30              children: [
31                  _buildTopCard(context),
32                  verticalSpace24,
33                  _buildChatCard(context),
34              ],
35          );
36      }
37
38      Widget _buildTopCard(BuildContext context) {
39          final statusLabel = widget.ticket['status_label'] ?? 'تم الحل';
40          final typeLabel = widget.ticket['type_label'] ?? 'اقتراح ميزة';

```

```

41 final message = widget.ticket['message'] ?? '';
42 final createdAt = _formatDateDetails(widget.ticket['created_at']);
43
44 return Container(
45   padding: const EdgeInsets.all(24),
46   decoration: BoxDecoration(
47     color: ColorsManager.isDark ? ColorsManager.surfacePrimary : Colors.white,
48     borderRadius: BorderRadius.circular(16),
49     border: Border.all(color: ColorsManager.isDark ? Colors.transparent : Colors.grey.shade200),
50     boxShadow: [
51       if (!ColorsManager.isDark)
52         BoxShadow(
53           color: Colors.black.withValues(alpha: 0.03),
54           blurRadius: 10,
55           offset: const Offset(0, 4),
56         ),
57     ],
58 ),
59 child: Column(
60   crossAxisAlignment: CrossAxisAlignment.start,
61   children: [
62     Row(
63       mainAxisAlignment: MainAxisAlignment.spaceBetween,
64       children: [
65         Text(
66           appTranslation().get('ticket_summary') ?? 'ملخص التذكرة',
67           style: TextStyleManager.bold16.copyWith(color: ColorsManager.mainText),
68         ),
69         Container(
70           padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 6),
71           decoration: BoxDecoration(
72             color: Colors.green.withValues(alpha: 0.1),
73             borderRadius: BorderRadius.circular(20),
74             border: Border.all(color: Colors.green.withValues(alpha: 0.5)),
75           ),
76           child: Text(
77             statusLabel,
78             style: TextStyleManager.bold12.copyWith(color: Colors.green.shade700),
79           ),
80         ),
81       ],
82     ),
83     verticalSpace16,
84     Row(
85       mainAxisAlignment: MainAxisAlignment.spaceBetween,
86       crossAxisAlignment: CrossAxisAlignment.end,
87       children: [
88         Column(
89           crossAxisAlignment: CrossAxisAlignment.start,
90           children: [
91             Text(appTranslation().get('date') ?? 'التاريخ', style: TextStyleManager.regular12.copyWith
92 (color: ColorsManager.secondaryText)),
93             verticalSpace4,
94             Text(createdAt, style: TextStyleManager.bold14.copyWith(color: ColorsManager.mainText)),
95           ],
96         ),
97         Container(
98           padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 6),
99           decoration: BoxDecoration(
100             color: Colors.green.withValues(alpha: 0.1),
101             borderRadius: BorderRadius.circular(20),
102           ),
103           child: Text(typeLabel, style: TextStyleManager.bold12.copyWith(color: Colors.green.shade70
104 0)),
105         ),
106       ],
107     ),
108     verticalSpace24,
109     Container(
110       width: double.infinity,
111       padding: const EdgeInsets.all(16),
112       decoration: BoxDecoration(
113         color: ColorsManager.isDark ? Colors.white.withValues(alpha: 0.05) : const Color(0xFFFFF8F9FA),
114         borderRadius: BorderRadius.circular(12),
115         border: Border.all(color: ColorsManager.isDark ? Colors.transparent : Colors.grey.shade200),

```



```

114     ),
115     child: Column(
116       crossAxisAlignment: CrossAxisAlignment.start,
117       children: [
118         Row(
119           children: [
120             Expanded(
121               child: Text(
122                 appTranslation().get('original_message') ?? 'نص الرسالة الأصلي',
123                 style: TextStyleManager.bold14.copyWith(color: ColorsManager.mainText),
124               ),
125             ),
126           ],
127         ),
128         verticalSpace12,
129         Text(
130           message,
131           style: TextStyleManager.regular14.copyWith(color: ColorsManager.mainText),
132         ),
133       ],
134     ),
135   ),
136 ],
137 ),
138 );
139 }
140
141 Widget _buildChatCard(BuildContext context) {
142   final replies = (widget.ticket['replies'] as List<dynamic>?) ?? [];
143
144   return Container(
145     decoration: BoxDecoration(
146       color: ColorsManager.isDark ? ColorsManager.surfacePrimary : Colors.white,
147       borderRadius: BorderRadius.circular(16),
148       border: Border.all(color: ColorsManager.isDark ? Colors.transparent : Colors.grey.shade200),
149       boxShadow: [
150         if (!ColorsManager.isDark)
151           BoxShadow(
152             color: Colors.black.withValues(alpha: 0.03),
153             blurRadius: 10,
154             offset: const Offset(0, 4),
155           ),
156       ],
157   ),
158   child: Column(
159     crossAxisAlignment: CrossAxisAlignment.stretch,
160     children: [
161       Padding(
162         padding: const EdgeInsets.all(24.0),
163         child: Row(
164           mainAxisAlignment: MainAxisAlignment.spaceBetween,
165           children: [
166             Expanded(
167               child: Text(
168                 appTranslation().get('original_message') ?? 'نص الرسالة الأصلي',
169                 style: TextStyleManager.bold16.copyWith(color: ColorsManager.mainText),
170               ),
171             ),
172             Row(
173               mainAxisAlignment: MainAxisAlignment.min,
174               children: [
175                 Text(
176                   '${appTranslation().get('chat_history') ?? 'المحادثة'} (${replies.length})',
177                   style: TextStyleManager.bold16.copyWith(color: Colors.teal),
178                 ),
179                 horizontalSpace8,
180                 const Icon(Icons.chat_bubble_outline, color: Colors.teal),
181               ],
182             ),
183           ],
184         ),
185       ),
186       Divider(height: 1, color: ColorsManager.isDark ? Colors.grey.shade800 : Colors.grey.shade200),
187       if (replies.isNotEmpty)
188         Padding(

```

```

189         padding: const EdgeInsets.all(24.0),
190         child: ListView.separated(
191             shrinkWrap: true,
192             physics: const NeverScrollableScrollPhysics(),
193             itemCount: replies.length,
194             separatorBuilder: (context, index) => verticalSpace24,
195             itemBuilder: (context, index) {
196                 final reply = replies[index];
197                 final isClient = reply['sender_type'] == 'client';
198                 final body = reply['body'] ?? '';
199                 final rawDate = reply['created_at'];
200
201                 if (isClient) {
202                     return _buildClientMessage(body, rawDate);
203                 } else {
204                     final senderNameAr = reply['sender_name'] ?? 'مدير النظام';
205                     final senderNameEn = ''; // Removed placeholder
206                     return _buildAdminMessage(senderNameAr, senderNameEn, body, rawDate);
207                 }
208             },
209         ),
210     ),
211     Padding(
212         padding: const EdgeInsets.all(24.0),
213         child: Row(
214             crossAxisAlignment: CrossAxisAlignment.end,
215             children: [
216                 InkWell(
217                     onTap: () {
218                         if (_replyController.text.trim().isEmpty) {
219                             ContactCubit.get(context).replyToTicket(
220                                 id: widget.ticket['id'],
221                                 body: _replyController.text.trim(),
222                             );
223                             _replyController.clear();
224                         }
225                     },
226                     borderRadius: BorderRadius.circular(12),
227                     child: Container(
228                         padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 14),
229                         decoration: BoxDecoration(
230                             color: Colors.teal,
231                             borderRadius: BorderRadius.circular(12),
232                         ),
233                         alignment: Alignment.center,
234                         child: Row(
235                             mainAxisAlignment: MainAxisAlignment.min,
236                             children: [
237                                 const Icon(Icons.send_outlined, color: Colors.white, size: 20),
238
239                             ],
240                         ),
241                     ),
242                 ),
243                 horizontalSpace16,
244                 Expanded(
245                     child: TextField(
246                         controller: _replyController,
247                         maxLines: 5,
248                         minLines: 1,
249                         decoration: InputDecoration(
250                             hintText: appTranslation().get('write_your_reply_here'),
251                             hintStyle: TextStyleManager.regular14.copyWith(color: ColorsManager.secondaryText),
252                             border: InputBorder.none,
253                             contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
254                         ),
255                     ),
256                 ),
257             ],
258         ),
259     ),
260 ],
261 ),
262 );
263 }

```

```

264
265 Widget _buildClientMessage(String body, String? rawDate) {
266   return Column(
267     crossAxisAlignment: CrossAxisAlignment.start, // Right aligned in RTL
268     children: [
269       Text(
270         '${appTranslation().get('chat_date')} / ${_formatDateShort(rawDate)}',
271         style: TextStyleManager.regular12.copyWith(color: ColorsManager.secondaryText),
272       ),
273       verticalSpace8,
274       Container(
275         padding: const EdgeInsets.all(16),
276         decoration: BoxDecoration(
277           color: ColorsManager.isDark ? Colors.white.withValues(alpha: 0.05) : const Color(0xFFFF8F9FA),
278           borderRadius: BorderRadius.circular(16).copyWith(
279             topRight: const Radius.circular(4), // Make the bubble point slightly
280           ),
281         ),
282         child: Text(
283           body,
284           style: TextStyleManager.regular14.copyWith(color: ColorsManager.mainText),
285         ),
286       ),
287     ],
288   );
289 }
290
291 Widget _buildAdminMessage(String senderNameAr, String senderNameEn, String body, String? rawDate) {
292   return Column(
293     crossAxisAlignment: CrossAxisAlignment.end, // Left aligned in RTL
294     children: [
295       Row(
296         mainAxisAlignment: MainAxisAlignment.end,
297         crossAxisAlignment: CrossAxisAlignment.center,
298         children: [
299           Text(
300             _formatDateShort(rawDate),
301             style: TextStyleManager.regular12.copyWith(color: ColorsManager.secondaryText),
302           ),
303           horizontalSpace12,
304           Column(
305             crossAxisAlignment: CrossAxisAlignment.end,
306             children: [
307               Text(senderNameAr, style: TextStyleManager.bold14.copyWith(color: ColorsManager.mainText)),
308               if (senderNameEn.isNotEmpty) Text(senderNameEn, style: TextStyleManager.regular12.copyWith(c
309             ],
310           ),
311           horizontalSpace12,
312           CircleAvatar(
313             radius: 20,
314             backgroundColor: Colors.blueGrey,
315             child: Text(
316               senderNameAr.isNotEmpty ? senderNameAr[0].toUpperCase() : 'A',
317               style: TextStyleManager.bold16.copyWith(color: Colors.white),
318             ),
319           ),
320         ],
321       ),
322       verticalSpace8,
323       Container(
324         padding: const EdgeInsets.all(16),
325         decoration: BoxDecoration(
326           color: ColorsManager.isDark ? Colors.white.withValues(alpha: 0.05) : const Color(0xFFFF8F9FA),
327           borderRadius: BorderRadius.circular(16).copyWith(
328             topLeft: const Radius.circular(4),
329           ),
330         ),
331         child: Text(
332           body,
333           style: TextStyleManager.regular14.copyWith(color: ColorsManager.mainText),
334         ),
335       ),
336     ],
337   );

```

```

338 }
339
340 String _formatDateDetails(String? dateStr) {
341   if (dateStr == null) return '9 2026 يوليو';
342   final dt = DateTime.tryParse(dateStr);
343   if (dt == null) return '9 2026 يوليو';
344   const months = [
345     'يناير', 'فبراير', 'مارس', 'أبريل', 'مايو', 'يونيو',
346     'يوليو', 'أغسطس', 'سبتمبر', 'أكتوبر', 'نوفمبر', 'ديسمبر'
347   ];
348   return '${dt.day} ${months[dt.month]} ${dt.year}';
349 }
350
351 String _formatDateShort(String? dateStr) {
352   if (dateStr == null) return '10:02 / ص AM يوليو';
353   final dt = DateTime.tryParse(dateStr);
354   if (dt == null) return '10:02 / ص AM يوليو';
355
356   final amPm = dt.hour >= 12 ? 'م PM' : 'ص AM';
357   final hour = dt.hour > 12 ? dt.hour - 12 : (dt.hour == 0 ? 12 : dt.hour);
358   final minute = dt.minute.toString().padLeft(2, '0');
359
360   const months = [
361     'يناير', 'فبراير', 'مارس', 'أبريل', 'مايو', 'يونيو',
362     'يوليو', 'أغسطس', 'سبتمبر', 'أكتوبر', 'نوفمبر', 'ديسمبر'
363   ];
364
365   return '${dt.day} ${months[dt.month]} / $hour:$minute $amPm';
366 }
367 }
368

```

Lines • 231
Dart

Feature / #142 –
Curriculum lib/features/curriculum/data/models/curriculum_distribution_models.dart

```

1 // Models for Curriculum Distribution & Books feature
2 // Based on the API shape described in CURRICULUM_DISTRIBUTION_AND_BOOKS.md
3
4 // _____
5 // Plans
6 // _____
7
8 class CurriculumPlanModel {
9   final int id;
10  final String subjectName;
11  final String gradeName;
12  final int semester;
13  final String academicYear;
14
15  const CurriculumPlanModel({
16    required this.id,
17    required this.subjectName,
18    required this.gradeName,
19    required this.semester,
20    required this.academicYear,
21  });
22
23  factory CurriculumPlanModel.fromJson(Map<String, dynamic> json) {
24    return CurriculumPlanModel(
25      id: int.tryParse(json['id']?.toString() ?? '') ?? 0,
26      subjectName: json['subject_name'] as String? ?? '',
27      gradeName: json['grade_name'] as String? ?? '',
28      semester: int.tryParse(json['semester']?.toString() ?? '') ?? 1,
29      academicYear: json['academic_year'] as String? ?? '',
30    );
31  }
32 }
33
34 // _____
35 // Week items (lessons inside a week)
36 // _____
37
38 class CurriculumWeekItem {

```

```

39 final int id;
40 final String title;
41 final String unit;
42 final String kind; // lesson | review | exam | intro | activity
43 final int? lessonId;
44 final bool canPrepare;
45
46 const CurriculumWeekItem({
47   required this.id,
48   required this.title,
49   required this.unit,
50   required this.kind,
51   this.lessonId,
52   required this.canPrepare,
53 });
54
55 factory CurriculumWeekItem.fromJson(Map<String, dynamic> json) {
56   return CurriculumWeekItem(
57     id: int.tryParse(json['id']?.toString() ?? '') ?? 0,
58     title: json['title'] as String? ?? '',
59     unit: json['unit'] as String? ?? '',
60     kind: json['kind'] as String? ?? 'lesson',
61     lessonId: int.tryParse(json['lesson_id']?.toString() ?? ''),
62     canPrepare: json['can_prepare'] as bool? ?? false,
63   );
64 }
65 }
66
67 // _____
68 // Week card (teaching | holiday | exam)
69 // _____
70
71 class CurriculumWeekModel {
72   final String type; // teaching | holiday | exam
73   final int? weekNumber;
74   final String? title;
75   final String startsOn;
76   final String endsOn;
77   final String? startsOnHijri;
78   final String? endsOnHijri;
79   final bool isCurrent;
80   final List<String> notes;
81   final List<CurriculumWeekItem> items;
82
83   const CurriculumWeekModel({
84     required this.type,
85     this.weekNumber,
86     this.title,
87     required this.startsOn,
88     required this.endsOn,
89     this.startsOnHijri,
90     this.endsOnHijri,
91     required this.isCurrent,
92     required this.notes,
93     required this.items,
94   });
95
96   bool get isTeaching => type == 'teaching';
97   bool get isHoliday => type == 'holiday';
98   bool get isExam => type == 'exam';
99
100   factory CurriculumWeekModel.fromJson(Map<String, dynamic> json) {
101     final rawItems = json['items'] as List<dynamic>? ?? [];
102     final rawNotes = json['notes'] as List<dynamic>? ?? [];
103     return CurriculumWeekModel(
104       type: json['type'] as String? ?? 'teaching',
105       weekNumber: int.tryParse(json['week_number']?.toString() ?? ''),
106       title: json['title'] as String?,
107       startsOn: json['starts_on'] as String? ?? '',
108       endsOn: json['ends_on'] as String? ?? '',
109       startsOnHijri: json['starts_on_hijri'] as String?,
110       endsOnHijri: json['ends_on_hijri'] as String?,
111       isCurrent: json['is_current'] as bool? ?? false,
112       notes: rawNotes.map((e) => e.toString()).toList(),
113       items: rawItems

```

```

114         .map((e) => CurriculumWeekItem.fromJson(e as Map<String, dynamic>))
115         .toList(),
116     );
117 }
118 }
119
120 // =====
121 // Full plan detail (plan meta + all weeks)
122 // =====
123
124 class CurriculumPlanDetailModel {
125     final CurriculumPlanModel plan;
126     final List<CurriculumWeekModel> weeks;
127
128     const CurriculumPlanDetailModel({
129         required this.plan,
130         required this.weeks,
131     });
132
133     factory CurriculumPlanDetailModel.fromJson(Map<String, dynamic> json) {
134         // The backend may wrap plan metadata under a 'plan' key or inline it
135         final planData = (json['plan'] as Map<String, dynamic>?) ?? json;
136         final rawWeeks = json['weeks'] as List<dynamic>? ?? [];
137         return CurriculumPlanDetailModel(
138             plan: CurriculumPlanModel.fromJson(planData),
139             weeks: rawWeeks
140                 .map((e) => CurriculumWeekModel.fromJson(e as Map<String, dynamic>))
141                 .toList(),
142         );
143     }
144 }
145
146 // =====
147 // Books
148 // =====
149
150 class CurriculumBookModel {
151     final int id;
152     final String title;
153     final String subjectName;
154     final String? sizeMb;
155     final String? coverUrl;
156
157     const CurriculumBookModel({
158         required this.id,
159         required this.title,
160         required this.subjectName,
161         this.sizeMb,
162         this.coverUrl,
163     });
164
165     factory CurriculumBookModel.fromJson(Map<String, dynamic> json) {
166         // Some APIs might return the ID under a different key, e.g. book_id or uuid.
167         final rawId = json['id'] ?? json['book_id'] ?? json['uuid'] ?? 0;
168
169         return CurriculumBookModel(
170             id: int.tryParse(rawId.toString()) ?? 0,
171             title: json['title'] as String? ?? json['name'] as String? ?? '',
172             subjectName: json['subject_name'] as String? ?? '',
173             sizeMb: json['size_mb']?.toString(),
174             coverUrl: json['cover_url'] as String?,
175         );
176     }
177 }
178
179 class CurriculumBookDownloadModel {
180     final String url;
181     final String expiresAt;
182     final String fileName;
183     final double sizeMb;
184
185     const CurriculumBookDownloadModel({
186         required this.url,
187         required this.expiresAt,
188         required this.fileName,

```

```

189     required this.sizeMb,
190   });
191
192   factory CurriculumBookDownloadModel.fromJson(Map<String, dynamic> json) {
193     return CurriculumBookDownloadModel(
194       url: json['url'] as String? ?? '',
195       expiresAt: json['expires_at'] as String? ?? '',
196       fileName: json['file_name'] as String? ?? '',
197       sizeMb: double.tryParse(json['size_mb']?.toString() ?? '') ?? 0.0,
198     );
199   }
200 }
201
202 // _____
203 // Progress (teacher-specific)
204 // _____
205
206 class CurriculumProgressModel {
207   final int totalWeeks;
208   final int completedWeeks;
209   final int currentWeekNumber;
210   final String status; // ahead | on_track | behind
211   final int weeksAheadOrBehind;
212
213   const CurriculumProgressModel({
214     required this.totalWeeks,
215     required this.completedWeeks,
216     required this.currentWeekNumber,
217     required this.status,
218     required this.weeksAheadOrBehind,
219   });
220
221   factory CurriculumProgressModel.fromJson(Map<String, dynamic> json) {
222     return CurriculumProgressModel(
223       totalWeeks: json['total_weeks'] as int? ?? 0,
224       completedWeeks: json['completed_weeks'] as int? ?? 0,
225       currentWeekNumber: json['current_week_number'] as int? ?? 0,
226       status: json['status'] as String? ?? 'on_track',
227       weeksAheadOrBehind: json['weeks_ahead_or_behind'] as int? ?? 0,
228     );
229   }
230 }
231

```

Lines • Dart 201

Feature / Curriculum

#143 – lib/features/curriculum/data/repositories/curriculum_repository.dart

```

1  import 'package:dartz/dartz.dart';
2  import 'package:moean/core/network/remote/api_endpoints.dart';
3  import 'package:moean/core/network/remote/dio_helper.dart';
4  import 'package:moean/features/curriculum/data/models/curriculum_distribution_models.dart';
5
6  /// Repository covering all 7 curriculum endpoints described in
7  /// CURRICULUM_DISTRIBUTION_AND_BOOKS.md § 6. API
8  class CurriculumRepository {
9    // — Public: Plans —————
10
11    /// GET /api/curriculum/plans?subject_id=&semester=
12    Future<Either<String, List<CurriculumPlanModel>>> getPlans({
13      int? subjectId,
14      int? semester,
15    }) async {
16      final query = <String, dynamic>{};
17      if (subjectId != null) query['subject_id'] = subjectId;
18      if (semester != null) query['semester'] = semester;
19
20      final response = await DioHelper.getData(url: curriculumPlansApi, query: query);
21      return response.fold(
22        (error) => Left(error),
23        (res) {
24          try {
25            final data = res.data;
26            final list = (data['data'] ?? data) as List<dynamic>;
27            return Right(

```

```

28         list
29         .map((e) => CurriculumPlanModel.fromJson(e as Map<String, dynamic>))
30         .toList(),
31     );
32     } catch (e) {
33         return Left(e.toString());
34     }
35 },
36 );
37 }
38
39 /// GET /api/curriculum/plans/{plan}?region=west
40 Future<Either<String, CurriculumPlanDetailModel>> getPlanDetail(
41     int planId, {
42     String? region,
43 }) async {
44     final query = <String, dynamic>{};
45     if (region != null) query['region'] = region;
46
47     final response = await DioHelper.getData(
48         url: curriculumPlanDetailApi(planId),
49         query: query,
50     );
51     return response.fold(
52         (error) => Left(error),
53         (res) {
54             try {
55                 final data = (res.data['data'] ?? res.data) as Map<String, dynamic>;
56                 return Right(CurriculumPlanDetailModel.fromJson(data));
57             } catch (e) {
58                 return Left(e.toString());
59             }
60         },
61     );
62 }
63
64 /// GET /api/curriculum/plans/{plan}/weeks/current?date=
65 Future<Either<String, CurriculumWeekModel>> getCurrentWeek(
66     int planId, {
67     String? date,
68 }) async {
69     final query = <String, dynamic>{};
70     if (date != null) query['date'] = date;
71
72     final response = await DioHelper.getData(
73         url: curriculumPlanCurrentWeekApi(planId),
74         query: query,
75     );
76     return response.fold(
77         (error) => Left(error),
78         (res) {
79             try {
80                 final data = (res.data['data'] ?? res.data) as Map<String, dynamic>;
81                 return Right(CurriculumWeekModel.fromJson(data));
82             } catch (e) {
83                 return Left(e.toString());
84             }
85         },
86     );
87 }
88
89 // — Public: Books —————
90
91 /// GET /api/curriculum/books?subject_id=|grade_id=
92 Future<Either<String, List<CurriculumBookModel>>> getBooks({
93     int? subjectId,
94     int? gradeId,
95 }) async {
96     final query = <String, dynamic>{};
97     if (subjectId != null) query['subject_id'] = subjectId;
98     if (gradeId != null) query['grade_id'] = gradeId;
99
100     final response = await DioHelper.getData(url: curriculumBooksApi, query: query);
101     return response.fold(
102         (error) => Left(error),

```



```

103     (res) {
104         try {
105             final data = res.data;
106             final list = (data['data'] ?? data) as List<dynamic>;
107
108             List<CurriculumBookModel> books = [];
109             for (var item in list) {
110                 if (item is Map<String, dynamic>) {
111                     // If the item has a 'books' array, it means the response is grouped by subject
112                     if (item.containsKey('books') && item['books'] is List) {
113                         final subjectName = item['subject_name'] ?? item['name'] ?? '';
114                         for (var bookJson in item['books']) {
115                             final b = bookJson as Map<String, dynamic>;
116                             // Inject the subject name into the book json so the model parses it
117                             b['subject_name'] = subjectName;
118                             books.add(CurriculumBookModel.fromJson(b));
119                         }
120                     } else {
121                         // Otherwise, it's a flat list of books
122                         books.add(CurriculumBookModel.fromJson(item));
123                     }
124                 }
125             }
126
127             return Right(books);
128         } catch (e) {
129             return Left(e.toString());
130         }
131     },
132 );
133 }
134
135 /// GET /api/curriculum/books/{book}/download → { url, expires_at, file_name, size_mb }
136 Future<Either<String, CurriculumBookDownloadModel>> getBookDownloadUrl(
137     int bookId,
138 ) async {
139     final response = await DioHelper.getData(
140         url: curriculumBookDownloadApi(bookId),
141     );
142     return response.fold(
143         (error) => Left(error),
144         (res) {
145             try {
146                 final data = (res.data['data'] ?? res.data) as Map<String, dynamic>;
147                 return Right(CurriculumBookDownloadModel.fromJson(data));
148             } catch (e) {
149                 return Left(e.toString());
150             }
151         },
152     );
153 }
154
155 // — Authenticated: Teacher-specific —————
156
157 /// GET /api/curriculum/progress?subject_id=&semester=
158 Future<Either<String, CurriculumProgressModel>> getProgress({
159     required int subjectId,
160     required int semester,
161 }) async {
162     final response = await DioHelper.getData(
163         url: curriculumProgressApi,
164         query: {'subject_id': subjectId, 'semester': semester},
165     );
166     return response.fold(
167         (error) => Left(error),
168         (res) {
169             try {
170                 final data = (res.data['data'] ?? res.data) as Map<String, dynamic>;
171                 return Right(CurriculumProgressModel.fromJson(data));
172             } catch (e) {
173                 return Left(e.toString());
174             }
175         },
176     );
177 }

```

```

178
179 /// POST /api/curriculum/plans/{plan}/weeks/{week}/prepare
180 Future<Either<String, Map<String, dynamic>>> prepareWeek({
181   required int planId,
182   required int weekId,
183 }) async {
184   final response = await DioHelper.postData(
185     url: curriculumPlanWeekPrepareApi(planId, weekId),
186     data: {},
187   );
188   return response.fold(
189     (error) => Left(error),
190     (res) {
191       try {
192         final data = (res.data['data'] ?? res.data) as Map<String, dynamic>;
193         return Right(data);
194       } catch (e) {
195         return Left(e.toString());
196       }
197     },
198   );
199 }
200 }
201

```

Lines • Dart 40 Feature / Curriculum #144 – lib/features/curriculum/presentation/cubit/curriculum_books_cubit.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/features/curriculum/data/models/curriculum_distribution_models.dart';
4  import 'package:moean/features/curriculum/data/repositories/curriculum_repository.dart';
5
6  part 'curriculum_books_state.dart';
7
8  class CurriculumBooksCubit extends Cubit<CurriculumBooksState> {
9    static CurriculumBooksCubit get(BuildContext context) => BlocProvider.of(context);
10
11    CurriculumBooksCubit() : super(CurriculumBooksInitial());
12
13    final _repo = CurriculumRepository();
14    List<CurriculumBookModel> _books = [];
15
16    // — Load books list (optionally filtered) —————
17    Future<void> loadBooks({int? subjectId, int? gradeId}) async {
18      emit(CurriculumBooksLoading());
19      final result = await _repo.getBooks(subjectId: subjectId, gradeId: gradeId);
20      result.fold(
21        (error) => emit(CurriculumBooksError(message: error)),
22        (books) {
23          _books = books;
24          emit(CurriculumBooksLoaded(books: books));
25        },
26      );
27    }
28
29    // — Fetch a signed R2 download URL for a book —————
30    Future<void> getDownloadUrl(int bookId) async {
31      emit(CurriculumBooksDownloadLoading(bookId: bookId, books: _books));
32      final result = await _repo.getBookDownloadUrl(bookId);
33      result.fold(
34        (error) => emit(CurriculumBooksError(message: error, books: _books)),
35        (download) =>
36          emit(CurriculumBooksDownloadReady(download: download, books: _books)),
37      );
38    }
39  }
40

```

Lines • Dart 31 Feature / Curriculum #145 – lib/features/curriculum/presentation/cubit/curriculum_books_state.dart

```

1  part of 'curriculum_books_cubit.dart';
2

```

```

3  abstract class CurriculumBooksState {}
4
5  class CurriculumBooksInitial extends CurriculumBooksState {}
6
7  class CurriculumBooksLoading extends CurriculumBooksState {}
8
9  class CurriculumBooksLoaded extends CurriculumBooksState {
10     final List<CurriculumBookModel> books;
11     CurriculumBooksLoaded({required this.books});
12 }
13
14 class CurriculumBooksError extends CurriculumBooksState {
15     final List<CurriculumBookModel> books; // keep existing list visible
16     final String message;
17     CurriculumBooksError({required this.message, this.books = const []});
18 }
19
20 class CurriculumBooksDownloadLoading extends CurriculumBooksState {
21     final int bookId;
22     final List<CurriculumBookModel> books;
23     CurriculumBooksDownloadLoading({required this.bookId, required this.books});
24 }
25
26 class CurriculumBooksDownloadReady extends CurriculumBooksState {
27     final CurriculumBookDownloadModel download;
28     final List<CurriculumBookModel> books;
29     CurriculumBooksDownloadReady({required this.download, required this.books});
30 }
31

```

Lines • 98
Dart

Feature / #146 –
Curriculum [lib/features/curriculum/presentation/cubit/curriculum_distribution_cubit.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/features/curriculum/data/models/curriculum_distribution_models.dart';
4  import 'package:moean/features/curriculum/data/repositories/curriculum_repository.dart';
5
6  part 'curriculum_distribution_state.dart';
7
8  class CurriculumDistributionCubit
9      extends Cubit<CurriculumDistributionState> {
10     static CurriculumDistributionCubit get(BuildContext context) => BlocProvider.of(context);
11
12     CurriculumDistributionCubit() : super(CurriculumDistributionInitial());
13
14     final _repo = CurriculumRepository();
15
16     // Retained for cross-call context
17     int? selectedSubjectId;
18     int? selectedSemester;
19     int? selectedPlanId;
20
21     // — Load plans for a subject/semester —————
22     Future<void> loadPlans({int? subjectId, int? semester}) async {
23         selectedSubjectId = subjectId;
24         selectedSemester = semester;
25         emit(CurriculumDistributionLoading());
26
27         final result = await _repo.getPlans(subjectId: subjectId, semester: semester);
28         result.fold(
29             (error) => emit(CurriculumDistributionError(message: error)),
30             (plans) {
31                 emit(CurriculumDistributionPlansLoaded(plans: plans));
32                 // Auto-load the first plan's detail right away
33                 if (plans.isNotEmpty) loadPlanDetail(plans.first.id);
34             },
35         );
36     }
37
38     // — Load full timeline for a plan —————
39     Future<void> loadPlanDetail(int planId, {String? region}) async {
40         selectedPlanId = planId;

```

```

41   emit(CurriculumDistributionLoading());
42
43   final detailResult = await _repo.getPlanDetail(planId, region: region);
44   detailResult.fold(
45     (error) => emit(CurriculumDistributionError(message: error)),
46     (detail) async {
47       CurriculumProgressModel? progress;
48       if (selectedSubjectId != null && selectedSemester != null) {
49         final progResult = await _repo.getProgress(
50           subjectId: selectedSubjectId!,
51           semester: selectedSemester!,
52         );
53         progResult.fold((_) {}, (p) => progress = p);
54       }
55       if (!isClosed) {
56         emit(CurriculumDistributionDetailLoaded(
57           detail: detail,
58           progress: progress,
59         ));
60       }
61     },
62   );
63 }
64
65 // — Bulk-prepare a whole week —————
66 Future<void> prepareWeek(int weekId) async {
67   if (selectedPlanId == null) return;
68
69   // Keep current detail/progress visible while preparing
70   final current = state;
71   CurriculumPlanDetailModel? currentDetail;
72   CurriculumProgressModel? currentProgress;
73   if (current is CurriculumDistributionDetailLoaded) {
74     currentDetail = current.detail;
75     currentProgress = current.progress;
76   }
77   if (currentDetail != null) {
78     emit(CurriculumDistributionPreparing(
79       detail: currentDetail,
80       progress: currentProgress,
81     ));
82   }
83
84   final result = await _repo.prepareWeek(
85     planId: selectedPlanId!,
86     weekId: weekId,
87   );
88   result.fold(
89     (error) => emit(CurriculumDistributionError(message: error)),
90     (data) => emit(CurriculumDistributionPrepareSuccess(
91       detail: currentDetail!,
92       progress: currentProgress,
93       result: data,
94     )),
95   );
96 }
97 }
98

```

Lines • 41
Dart

Feature / #147 —
Curriculum lib/features/curriculum/presentation/cubit/curriculum_distribution_state.dart

```

1  part of 'curriculum_distribution_cubit.dart';
2
3  abstract class CurriculumDistributionState {}
4
5  class CurriculumDistributionInitial extends CurriculumDistributionState {}
6
7  class CurriculumDistributionLoading extends CurriculumDistributionState {}
8
9  class CurriculumDistributionPlansLoaded extends CurriculumDistributionState {
10   final List<CurriculumPlanModel> plans;
11   CurriculumDistributionPlansLoaded({required this.plans});

```

```

12 }
13
14 class CurriculumDistributionDetailLoaded extends CurriculumDistributionState {
15   final CurriculumPlanDetailModel detail;
16   final CurriculumProgressModel? progress;
17   CurriculumDistributionDetailLoaded({required this.detail, this.progress});
18 }
19
20 class CurriculumDistributionError extends CurriculumDistributionState {
21   final String message;
22   CurriculumDistributionError({required this.message});
23 }
24
25 class CurriculumDistributionPreparing extends CurriculumDistributionState {
26   final CurriculumPlanDetailModel detail;
27   final CurriculumProgressModel? progress;
28   CurriculumDistributionPreparing({required this.detail, this.progress});
29 }
30
31 class CurriculumDistributionPrepareSuccess extends CurriculumDistributionState {
32   final CurriculumPlanDetailModel detail;
33   final CurriculumProgressModel? progress;
34   final Map<String, dynamic> result;
35   CurriculumDistributionPrepareSuccess({
36     required this.detail,
37     this.progress,
38     required this.result,
39   });
40 }
41

```

Lines • 389
Dart

Feature / #148 -
Curriculum lib/features/curriculum/presentation/screens/curriculum_books_screen.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:url_launcher/url_launcher.dart';
4 import 'package:moean/core/theme/colors.dart';
5 import 'package:moean/core/theme/text_styles.dart';
6 import 'package:moean/core/utils/constants/spacing.dart';
7 import 'package:moean/features/curriculum/data/models/curriculum_distribution_models.dart';
8 import 'package:moean/features/curriculum/presentation/cubit/curriculum_books_cubit.dart';
9 import 'package:moean/core/network/remote/api_service.dart';
10 import 'package:moean/features/exam_generation/data/models/curriculum_models.dart';
11 import 'package:moean/features/curriculum/presentation/widgets/subject_picker_section.dart';
12 import 'package:moean/features/curriculum/presentation/cubit/curriculum_distribution_cubit.dart';
13 import 'package:moean/features/curriculum/presentation/widgets/region_legend_row.dart';
14 import 'package:moean/features/curriculum/presentation/widgets/weeks_grid.dart';
15 import 'package:moean/features/curriculum/presentation/widgets/inline_books_section.dart';
16 import 'package:moean/features/curriculum/presentation/widgets/curriculum_error_view.dart';
17
18 class CurriculumBooksScreen extends StatefulWidget {
19   const CurriculumBooksScreen({super.key});
20
21   @override
22   State<CurriculumBooksScreen> createState() => _CurriculumBooksScreenState();
23 }
24
25 class _CurriculumBooksScreenState extends State<CurriculumBooksScreen> {
26   List<CurriculumStageModel> _stages = [];
27   List<CurriculumGradeModel> _grades = [];
28   List<CurriculumSubjectModel> _subjects = [];
29
30   CurriculumStageModel? _selectedStage;
31   CurriculumGradeModel? _selectedGrade;
32   CurriculumSubjectModel? _selectedSubject;
33   int _selectedSemester = 1;
34
35   bool _loadingStages = true;
36   String? _pickerError;
37
38   @override
39   void initState() {

```

```

40     super.initState();
41     _loadStages();
42 }
43
44 Future<void> _loadStages() async {
45     setState(() {
46         _loadingStages = true;
47         _pickerError = null;
48     });
49     final result = await ApiService.getSubjects();
50     if (!mounted) return;
51     result.fold(
52         (failure) => setState(() {
53             _loadingStages = false;
54             _pickerError = failure.toString();
55         }),
56         (stages) => setState(() {
57             _stages = stages;
58             _loadingStages = false;
59         }),
60     );
61 }
62
63 void _onStageSelected(CurriculumStageModel stage) {
64     setState(() {
65         _selectedStage = stage;
66         _selectedGrade = null;
67         _selectedSubject = null;
68         _grades = stage.grades;
69         _subjects = [];
70     });
71 }
72
73 void _onGradeSelected(CurriculumGradeModel grade) {
74     setState(() {
75         _selectedGrade = grade;
76         _selectedSubject = null;
77         _subjects = grade.subjects;
78     });
79 }
80
81 void _onSubjectSelected(CurriculumSubjectModel subject) {
82     setState(() => _selectedSubject = subject);
83     CurriculumBooksCubit.get(context).loadBooks(subjectId: subject.id);
84     CurriculumDistributionCubit.get(context).loadPlans(
85         subjectId: subject.id,
86         semester: _selectedSemester,
87     );
88 }
89
90 void _onSemesterChanged(int s) {
91     setState(() => _selectedSemester = s);
92     if (_selectedSubject != null) {
93         CurriculumDistributionCubit.get(context).loadPlans(
94             subjectId: _selectedSubject!.id,
95             semester: s,
96         );
97     }
98 }
99
100 String _selectedRegion = 'general';
101
102 void _onRegionChanged(String region) {
103     setState(() => _selectedRegion = region);
104     final cubit = CurriculumDistributionCubit.get(context);
105     if (cubit.selectedPlanId != null) {
106         cubit.loadPlanDetail(
107             cubit.selectedPlanId!,
108             region: region == 'west' ? 'west' : null,
109         );
110     }
111 }
112
113 @override
114 Widget build(BuildContext context) {

```

```

115     return Directionality(
116       textDirection: TextDirection.rtl,
117       child: Scaffold(
118         backgroundColor: ColorsManager.background,
119         appBar: AppBar(
120           backgroundColor: ColorsManager.background,
121           elevation: 0,
122           title: Text(
123             'الكتب الدراسية',
124             style: TextStyleManager.bold16.copyWith(color: ColorsManager.mainText),
125           ),
126           centerTitle: true,
127           leading: IconButton(
128             onPressed: () => Navigator.pop(context),
129             icon: const Icon(Icons.arrow_back_ios_new_rounded, size: 20),
130             color: ColorsManager.mainText,
131           ),
132         ),
133         body: Column(
134           children: [
135             SubjectPickerSection(
136               stages: _stages,
137               grades: _grades,
138               subjects: _subjects,
139               selectedStage: _selectedStage,
140               selectedGrade: _selectedGrade,
141               selectedSubject: _selectedSubject,
142               selectedSemester: _selectedSemester,
143               loading: _loadingStages,
144               error: _pickerError,
145               onStageSelected: _onStageSelected,
146               onGradeSelected: _onGradeSelected,
147               onSubjectSelected: _onSubjectSelected,
148               onSemesterChanged: _onSemesterChanged,
149             ),
150             if (_selectedSubject != null) const InlineBooksSection(),
151             RegionLegendRow(
152               selectedRegion: _selectedRegion,
153               onRegionChanged: _onRegionChanged,
154             ),
155             Expanded(
156               child: BlocConsumer<CurriculumDistributionCubit, CurriculumDistributionState>(
157                 listener: (context, state) {
158                   if (state is CurriculumDistributionPrepareSuccess) {
159                     ScaffoldMessenger.of(context).showSnackBar(
160                       const SnackBar(
161                         content: Text('تم تحضير الأسبوع بنجاح'),
162                         backgroundColor: Colors.green,
163                       ),
164                     );
165                   }
166                   if (state is CurriculumDistributionError) {
167                     ScaffoldMessenger.of(context).showSnackBar(
168                       SnackBar(
169                         content: Text(state.message),
170                         backgroundColor: Colors.red,
171                       ),
172                     );
173                   }
174                 },
175                 builder: (context, state) {
176                   if (_selectedSubject == null) {
177                     return Center(
178                       child: Text(
179                         'الرجاء اختيار المادة لعرض الكتب والتوزيع',
180                         style: TextStyleManager.medium14.copyWith(color: ColorsManager.secondaryText),
181                       ),
182                     );
183                   }
184
185                   if (state is CurriculumDistributionLoading ||
186                       state is CurriculumDistributionInitial ||
187                       state is CurriculumDistributionPlansLoaded) {
188                     return const Center(child: CircularProgressIndicator());
189                   }

```

```

190
191     CurriculumPlanDetailModel? detail;
192     CurriculumProgressModel? progress;
193     bool isPreparing = false;
194
195     if (state is CurriculumDistributionDetailLoaded) {
196         detail = state.detail;
197         progress = state.progress;
198     } else if (state is CurriculumDistributionPreparing) {
199         detail = state.detail;
200         progress = state.progress;
201         isPreparing = true;
202     } else if (state is CurriculumDistributionPrepareSuccess) {
203         detail = state.detail;
204         progress = state.progress;
205     } else if (state is CurriculumDistributionError) {
206         return CurriculumErrorView(
207             message: state.message,
208             onRetry: () => CurriculumDistributionCubit.get(context).loadPlans(
209                 subjectId: _selectedSubject!.id,
210                 semester: _selectedSemester,
211             ),
212         );
213     }
214
215     if (detail == null) return const SizedBox.shrink();
216
217     return Stack(
218         children: [
219             WeeksGrid(
220                 detail: detail,
221                 progress: progress,
222                 onPrepareTap: (weekId) {}, // Disabled as per user request
223             ),
224             if (isPreparing)
225                 const Positioned.fill(
226                     child: ColoredBox(
227                         color: Color(0x44000000),
228                         child: Center(child: CircularProgressIndicator()),
229                     ),
230                 ),
231         ],
232     );
233
234     },
235
236     ],
237
238     ),
239
240     );
241 }
242
243 // _____
244 // Book card
245 // _____
246
247 class _BookCard extends StatelessWidget {
248     final CurriculumBookModel book;
249     final bool isDownloading;
250     final VoidCallback onDownload;
251
252     const _BookCard({
253         required this.book,
254         required this.isDownloading,
255         required this.onDownload,
256     });
257
258     @override
259     Widget build(BuildContext context) {
260         return Container(
261             padding: const EdgeInsets.all(14),
262             decoration: BoxDecoration(
263                 color: ColorsManager.surfacePrimary,
264                 borderRadius: BorderRadius.circular(14),

```



```

265     border: Border.all(
266       color: ColorsManager.primaryColor.withOpacity(0.15)),
267     boxShadow: [
268       BoxShadow(
269         color: Colors.black.withOpacity(0.04),
270         blurRadius: 8,
271         offset: const Offset(0, 2),
272       ),
273     ],
274   ),
275   child: Row(
276     children: [
277       // PDF icon
278       Container(
279         width: 52,
280         height: 52,
281         decoration: BoxDecoration(
282           color: Colors.red.withOpacity(0.08),
283           borderRadius: BorderRadius.circular(12),
284         ),
285         child: const Icon(Icons.picture_as_pdf_outlined,
286           color: Colors.red, size: 28),
287       ),
288       horizontalSpace12,
289       // Info
290       Expanded(
291         child: Column(
292           crossAxisAlignment: CrossAxisAlignment.start,
293           children: [
294             Text(
295               book.title,
296               style: TextStylesManager.bold14
297                 .copyWith(color: ColorsManager.mainText),
298               maxLines: 2,
299               overflow: TextOverflow.ellipsis,
300             ),
301             verticalSpace4,
302             Text(
303               book.subjectName,
304               style: TextStylesManager.regular12
305                 .copyWith(color: ColorsManager.secondaryText),
306             ),
307             if (book.sizeMb != null) ...[
308               verticalSpace2,
309               Text(
310                 '${book.sizeMb} ميجا',
311                 style: TextStylesManager.regular12.copyWith(
312                   color: ColorsManager.secondaryText, fontSize: 10),
313               ),
314             ],
315           ],
316         ),
317       ),
318       horizontalSpace12,
319       // Download button - fetches signed URL then launches it
320       GestureDetector(
321         onTap: isDownloading ? null : onDownload,
322         child: AnimatedContainer(
323           duration: const Duration(milliseconds: 200),
324           padding:
325             const EdgeInsets.symmetric(horizontal: 14, vertical: 10),
326           decoration: BoxDecoration(
327             color: ColorsManager.primaryColor,
328             borderRadius: BorderRadius.circular(10),
329           ),
330           child: isDownloading
331             ? const SizedBox(
332               width: 16,
333               height: 16,
334               child: CircularProgressIndicator(
335                 strokeWidth: 2,
336                 color: Colors.white,
337               ),
338             )
339             : Row(

```

```

340         mainAxisSize: MainAxisSize.min,
341         children: [
342             const Icon(Icons.download_rounded,
343                 color: Colors.white, size: 16),
344             const SizedBox(width: 4),
345             Text('تحميل',
346                 style: TextStylesManager.bold12
347                     .copyWith(color: Colors.white)),
348         ],
349     ),
350 ),
351 ),
352 ],
353 ),
354 );
355 }
356 }
357
358 class _FullError extends StatelessWidget {
359     final String message;
360     final VoidCallback onRetry;
361     const _FullError({required this.message, required this.onRetry});
362
363     @override
364     Widget build(BuildContext context) {
365         return Center(
366             child: Padding(
367                 padding: const EdgeInsets.symmetric(horizontal: 24),
368                 child: Column(
369                     mainAxisAlignment: MainAxisAlignment.center,
370                     children: [
371                         const Icon(Icons.error_outline, size: 52, color: Colors.red),
372                         verticalSpace12,
373                         Text(message,
374                             textAlign: TextAlign.center,
375                             style: TextStylesManager.regular14
376                                 .copyWith(color: ColorsManager.secondaryText)),
377                         verticalSpace16,
378                         ElevatedButton.icon(
379                             onPressed: onRetry,
380                             icon: const Icon(Icons.refresh_rounded),
381                             label: const Text('إعادة المحاولة'),
382                         ),
383                     ],
384                 ),
385             ),
386         );
387     }
388 }
389

```

Lines • 248
Dart

Feature / #149 –
Curriculum

[lib/features/curriculum/presentation/screens/curriculum_distribution_screen.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/network/remote/api_service.dart';
4  import 'package:moean/core/theme/colors.dart';
5  import 'package:moean/core/theme/text_styles.dart';
6  import 'package:moean/features/curriculum/data/models/curriculum_distribution_models.dart';
7  import 'package:moean/features/curriculum/presentation/cubit/curriculum_distribution_cubit.dart';
8  import 'package:moean/features/exam_generation/data/models/curriculum_models.dart';
9  import 'package:moean/features/curriculum/presentation/cubit/curriculum_books_cubit.dart';
10 import 'package:moean/features/curriculum/presentation/widgets/subject_picker_section.dart';
11 import 'package:moean/features/curriculum/presentation/widgets/region_legend_row.dart';
12 import 'package:moean/features/curriculum/presentation/widgets/weeks_grid.dart';
13 import 'package:moean/features/curriculum/presentation/widgets/inline_books_section.dart';
14 import 'package:moean/features/curriculum/presentation/widgets/curriculum_error_view.dart';
15 import 'package:moean/core/utils/pdf_export_helper.dart';
16
17 class CurriculumDistributionScreen extends StatefulWidget {
18     const CurriculumDistributionScreen({super.key});
19

```

```

20  @override
21  State<CurriculumDistributionScreen> createState() =>
22    _CurriculumDistributionScreenState();
23  }
24
25  class _CurriculumDistributionScreenState
26    extends State<CurriculumDistributionScreen> {
27    // Stage / Grade / Subject tree
28    List<CurriculumStageModel> _stages = [];
29    List<CurriculumGradeModel> _grades = [];
30    List<CurriculumSubjectModel> _subjects = [];
31
32    CurriculumStageModel? _selectedStage;
33    CurriculumGradeModel? _selectedGrade;
34    CurriculumSubjectModel? _selectedSubject;
35    int _selectedSemester = 1;
36
37    bool _loadingStages = true;
38    String? _pickerError;
39
40    // Region switcher
41    String _selectedRegion = 'general'; // 'general' | 'west'
42
43    @override
44    void initState() {
45      super.initState();
46      _loadStages();
47    }
48
49    Future<void> _loadStages() async {
50      setState(() {
51        _loadingStages = true;
52        _pickerError = null;
53      });
54      final result = await ApiService.getSubjects();
55      if (!mounted) return;
56      result.fold(
57        (failure) => setState(() {
58          _loadingStages = false;
59          _pickerError = failure.toString();
60        }),
61        (stages) => setState(() {
62          _stages = stages;
63          _loadingStages = false;
64        }),
65      );
66    }
67
68    void _onStageSelected(CurriculumStageModel stage) {
69      setState(() {
70        _selectedStage = stage;
71        _selectedGrade = null;
72        _selectedSubject = null;
73        _grades = stage.grades;
74        _subjects = [];
75      });
76    }
77
78    void _onGradeSelected(CurriculumGradeModel grade) {
79      setState(() {
80        _selectedGrade = grade;
81        _selectedSubject = null;
82        _subjects = grade.subjects;
83      });
84    }
85
86    void _onSubjectSelected(CurriculumSubjectModel subject) {
87      setState(() => _selectedSubject = subject);
88      CurriculumDistributionCubit.get(context).loadPlans(
89        subjectId: subject.id,
90        semester: _selectedSemester,
91      );
92      CurriculumBooksCubit.get(context).loadBooks(subjectId: subject.id);
93    }
94

```



```

170         content: Text('تم تحضير الأسبوع بنجاح ✔'),
171         backgroundColor: Colors.green,
172     ),
173 );
174 }
175 if (state is CurriculumDistributionError) {
176     ScaffoldMessenger.of(context).showSnackBar(
177         SnackBar(
178             content: Text(state.message),
179             backgroundColor: Colors.red,
180         ),
181     );
182 }
183 },
184 builder: (context, state) {
185     if (_selectedSubject == null) {
186         return EmptyPickerView();
187     }
188
189     if (state is CurriculumDistributionLoading ||
190         state is CurriculumDistributionInitial ||
191         state is CurriculumDistributionPlansLoaded) {
192         return const Center(child: CircularProgressIndicator());
193     }
194
195     CurriculumPlanDetailModel? detail;
196     CurriculumProgressModel? progress;
197     bool isPreparing = false;
198
199     if (state is CurriculumDistributionDetailLoaded) {
200         detail = state.detail;
201         progress = state.progress;
202     } else if (state is CurriculumDistributionPreparing) {
203         detail = state.detail;
204         progress = state.progress;
205         isPreparing = true;
206     } else if (state is CurriculumDistributionPrepareSuccess) {
207         detail = state.detail;
208         progress = state.progress;
209     } else if (state is CurriculumDistributionError) {
210         return CurriculumErrorView(
211             message: state.message,
212             onRetry: () =>
213                 CurriculumDistributionCubit.get(context).loadPlans(
214                     subjectId: _selectedSubject!.id,
215                     semester: _selectedSemester,
216                 ),
217         );
218     }
219
220     if (detail == null) return const SizedBox.shrink();
221
222     return Stack(
223         children: [
224             WeeksGrid(
225                 detail: detail,
226                 progress: progress,
227                 onPrepareTap: (weekId) => CurriculumDistributionCubit.get(context).prepareWeek(weekId),
228             ),
229             if (isPreparing)
230                 const Positioned.fill(
231                     child: ColoredBox(
232                         color: Color(0x44000000),
233                         child: Center(child: CircularProgressIndicator()),
234                     ),
235                 ),
236         ],
237     );
238 },
239 ),
240 ],
241 ),
242 ),
243 ),

```

```
244     );
245   }
246 }
247
248
```

Lines • Dart 62

Feature / Curriculum

#150 – lib/features/curriculum/presentation/widgets/curriculum_error_view.dart

```
1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/spacing.dart';
5
6  class CurriculumErrorView extends StatelessWidget {
7    final String message;
8    final VoidCallback onRetry;
9
10   const CurriculumErrorView({super.key, required this.message, required this.onRetry});
11
12   @override
13   Widget build(BuildContext context) {
14     return Center(
15       child: Padding(
16         padding: const EdgeInsets.symmetric(horizontal: 24),
17         child: Column(
18           mainAxisAlignment: MainAxisAlignment.center,
19           children: [
20             const Icon(Icons.error_outline, size: 52, color: Colors.red),
21             verticalSpace12,
22             Text(message,
23               textAlign: TextAlign.center,
24               style: TextStylesManager.regular14
25                 .copyWith(color: ColorsManager.secondaryText)),
26             verticalSpace16,
27             ElevatedButton.icon(
28               onPressed: onRetry,
29               icon: const Icon(Icons.refresh_rounded),
30               label: const Text('إعادة المحاولة'),
31             ),
32           ],
33         ),
34       ),
35     );
36   }
37 }
38
39 class EmptyPickerView extends StatelessWidget {
40   const EmptyPickerView({super.key});
41
42   @override
43   Widget build(BuildContext context) {
44     return Center(
45       child: Column(
46         mainAxisAlignment: MainAxisAlignment.center,
47         children: [
48           Icon(Icons.calendar_month_outlined,
49             size: 64,
50             color: ColorsManager.primaryColor.withOpacity(0.4)),
51           verticalSpace16,
52           Text(
53             'اختر المرحلة والصف والمادة لعرض التوزيع',
54             style: TextStylesManager.regular14
55               .copyWith(color: ColorsManager.secondaryText),
56           ),
57         ],
58       ),
59     );
60   }
61 }
62
```

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
5 import 'package:moean/core/utils/constants/spacing.dart';
6
7 class ExportPdfCard extends StatefulWidget {
8   final Future<void> Function(String school, String teacher, String manager)? onExport;
9
10  const ExportPdfCard({super.key, this.onExport});
11
12  @override
13  State<ExportPdfCard> createState() => _ExportPdfCardState();
14 }
15
16 class _ExportPdfCardState extends State<ExportPdfCard> {
17   final _schoolController = TextEditingController();
18   final _teacherController = TextEditingController();
19   final _managerController = TextEditingController();
20   bool _isExporting = false;
21
22   @override
23   void dispose() {
24     _schoolController.dispose();
25     _teacherController.dispose();
26     _managerController.dispose();
27     super.dispose();
28   }
29
30   @override
31   Widget build(BuildContext context) {
32     return Container(
33       margin: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
34       padding: const EdgeInsets.all(16),
35       decoration: BoxDecoration(
36         color: ColorsManager.surfacePrimary,
37         borderRadius: BorderRadius.circular(16),
38         border: Border.all(color: ColorsManager.borderLightGray),
39         boxShadow: [
40           BoxShadow(
41             color: Colors.black.withOpacity(0.04),
42             blurRadius: 10,
43             offset: const Offset(0, 4),
44           ),
45         ],
46       ),
47       child: Column(
48         crossAxisAlignment: CrossAxisAlignment.start,
49         children: [
50           Row(
51             children: [
52               Container(
53                 padding: const EdgeInsets.all(8),
54                 decoration: BoxDecoration(
55                   color: ColorsManager.primaryColor.withOpacity(0.1),
56                   borderRadius: BorderRadius.circular(10),
57                 ),
58               child: Icon(Icons.print_outlined, color: ColorsManager.primaryColor, size: 22),
59             ),
60             horizontalSpace12,
61             Expanded(
62               child: Column(
63                 crossAxisAlignment: CrossAxisAlignment.start,
64                 children: [
65                   Text('تنزيل التوزيع', style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainText)),
66                   verticalSpace2,
67                   Text('PDF أضف بيانات مدرستك ثم اطبع الورقة أو احفظها',
68                     style: TextStylesManager.regular12.copyWith(color: ColorsManager.secondaryText, fontS
69 size: 10)),
70                 ],
71             ),

```

```

71     ),
72     ],
73   ),
74   verticalSpace16,
75   Text('اسم المدرسة', style: TextStylesManager.bold12.copyWith(color: ColorsManager.mainText)),
76   verticalSpace8,
77   PrimaryTextField(controller: _schoolController, hint: 'مثال: ابتدائية الأندلس'),
78   verticalSpace12,
79   Text('اسم المعلم / ة', style: TextStylesManager.bold12.copyWith(color: ColorsManager.mainText)),
80   verticalSpace8,
81   PrimaryTextField(controller: _teacherController, hint: 'اسم المعلم'),
82   verticalSpace12,
83   Text('اسم المدير / ة', style: TextStylesManager.bold12.copyWith(color: ColorsManager.mainText)),
84   verticalSpace8,
85   PrimaryTextField(controller: _managerController, hint: 'اسم المدير'),
86   verticalSpace16,
87   SizedBox(
88     width: double.infinity,
89     child: ElevatedButton.icon(
90       onPressed: _isExporting ? null : () async {
91         if (widget.onExport != null) {
92           setState(() => _isExporting = true);
93           await widget.onExport!(
94             _schoolController.text,
95             _teacherController.text,
96             _managerController.text,
97           );
98           setState(() => _isExporting = false);
99         } else {
100           ScaffoldMessenger.of(context).showSnackBar(
101             const SnackBar(content: Text('سيتم توفير هذه الميزة قريباً')),
102           );
103         }
104       },
105       icon: _isExporting
106         ? const SizedBox(width: 18, height: 18, child: CircularProgressIndicator(color: Colors.white
e, strokeWidth: 2))
107         : const Icon(Icons.print, size: 18),
108       label: Text(_isExporting ? 'تنزيل' : 'طباعة / PDF جاري التصدير...'),
109       style: ElevatedButton.styleFrom(
110         backgroundColor: const Color(0xFF13192B), // very dark color from screenshot
111         foregroundColor: Colors.white,
112         padding: const EdgeInsets.symmetric(vertical: 12),
113         shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(10)),
114       ),
115     ),
116   ),
117 ],
118 ),
119 );
120 }
121
122 // Allowed since it returns Widget but is a simple layout helper that calls a global Primary component
123 // Wait, the rule says: "Private methods that return Widget are strictly forbidden inside page or widget fi
les"
124 // Let me inline it instead to be safe!
125 }
126

```

Lines • Dart 147

Feature / Curriculum

#152 – lib/features/curriculum/presentation/widgets/inline_books_section.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:url_launcher/url_launcher.dart';
4 import 'package:moean/core/theme/colors.dart';
5 import 'package:moean/core/theme/text_styles.dart';
6 import 'package:moean/core/utils/constants/spacing.dart';
7 import 'package:moean/features/curriculum/data/models/curriculum_distribution_models.dart';
8 import 'package:moean/features/curriculum/presentation/cubit/curriculum_books_cubit.dart';
9
10 class InlineBooksSection extends StatelessWidget {
11   const InlineBooksSection({super.key});
12

```



```

13 @override
14 Widget build(BuildContext context) {
15   return BlocConsumer<CurriculumBooksCubit, CurriculumBooksState>(
16     listener: (context, state) {
17       if (state is CurriculumBooksDownloadReady) {
18         _openDownloadUrl(context, state.download.url);
19       }
20     },
21     builder: (context, state) {
22       List<CurriculumBookModel> books = [];
23       int? loadingBookId;
24
25       if (state is CurriculumBooksLoading) {
26         return const Padding(
27           padding: EdgeInsets.all(16.0),
28           child: Center(child: CircularProgressIndicator()),
29         );
30       } else if (state is CurriculumBooksLoaded) {
31         books = state.books;
32       } else if (state is CurriculumBooksDownloadLoading) {
33         books = state.books;
34         loadingBookId = state.bookId;
35       } else if (state is CurriculumBooksDownloadReady) {
36         books = state.books;
37       } else if (state is CurriculumBooksError) {
38         return const SizedBox.shrink(); // hide section if error loading books inline
39       }
40
41       if (books.isEmpty) return const SizedBox.shrink();
42
43       return Padding(
44         padding: const EdgeInsets.fromLTRB(16, 12, 16, 0),
45         child: Column(
46           children: books.map((book) {
47             final isDownloading = loadingBookId == book.id;
48             return Container(
49               margin: const EdgeInsets.only(bottom: 8),
50               padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
51               decoration: BoxDecoration(
52                 color: ColorsManager.surfacePrimary,
53                 borderRadius: BorderRadius.circular(12),
54                 border: Border.all(color: ColorsManager.borderLightGray),
55                 boxShadow: [
56                   BoxShadow(
57                     color: Colors.black.withOpacity(0.04),
58                     blurRadius: 4,
59                     offset: const Offset(0, 2),
60                   ),
61                 ],
62             ),
63             child: Row(
64               children: [
65                 Container(
66                   width: 44,
67                   height: 44,
68                   decoration: BoxDecoration(
69                     color: Colors.red.withOpacity(0.08),
70                     borderRadius: BorderRadius.circular(10),
71                   ),
72                   child: const Icon(Icons.picture_as_pdf_outlined, color: Colors.red, size: 24),
73                 ),
74                 horizontalSpace12,
75                 Expanded(
76                   child: Column(
77                     crossAxisAlignment: CrossAxisAlignment.start,
78                     children: [
79                       Text(
80                         book.title,
81                         style: TextStyleManager.bold12.copyWith(color: ColorsManager.mainText),
82                         maxLines: 2,
83                         overflow: TextOverflow.ellipsis,
84                       ),
85                       if (book.sizeMb != null) ...[
86                         verticalSpace2,
87                         Text(

```

```

88         '${book.sizeMb} ميجا',
89         style: TextStylesManager.regular12.copyWith(
90             color: ColorsManager.secondaryText, fontSize: 10),
91     ),
92 ],
93 ],
94 ),
95 ),
96 horizontalSpace12,
97 GestureDetector(
98     onTap: isDownloading ? null : () => CurriculumBooksCubit.get(context).getDownloadUrl(bo
ok.id),
99     child: Container(
100         padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 8),
101         decoration: BoxDecoration(
102             color: const Color(0xFF13192B), // very dark color from screenshot
103             borderRadius: BorderRadius.circular(8),
104         ),
105         child: isDownloading
106             ? const SizedBox(
107                 width: 14,
108                 height: 14,
109                 child: CircularProgressIndicator(strokeWidth: 2, color: Colors.white),
110             )
111             : Row(
112                 mainAxisAlignment: MainAxisAlignment.min,
113                 children: [
114                     const Icon(Icons.download_rounded, color: Colors.white, size: 14),
115                     const SizedBox(width: 4),
116                     Text('تحميل', style: TextStylesManager.bold12.copyWith(color: Colors.white,
fontSize: 10)),
117                 ],
118             ),
119     ),
120 ],
121 ],
122 ),
123 );
124 }).toList(),
125 ),
126 );
127 },
128 );
129 }
130 }
131
132 Future<void> _openDownloadUrl(BuildContext context, String url) async {
133     final uri = Uri.tryParse(url);
134     if (uri != null && await canLaunchUrl(uri)) {
135         await launchUrl(uri, mode: LaunchMode.externalApplication);
136     } else {
137         if (context.mounted) {
138             ScaffoldMessenger.of(context).showSnackBar(
139                 const SnackBar(
140                     content: Text('تعذر فتح رابط التحميل'),
141                     backgroundColor: Colors.red,
142                 ),
143             );
144         }
145     }
146 }
147

```

Lines • Dart 118

Feature / Curriculum

#153 – lib/features/curriculum/presentation/widgets/region_legend_row.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4
5 class RegionLegendRow extends StatelessWidget {
6     final String selectedRegion;
7     final void Function(String) onRegionChanged;
8

```

```

9   const RegionLegendRow({
10     super.key,
11     required this.selectedRegion,
12     required this.onRegionChanged,
13   });
14
15   @override
16   Widget build(BuildContext context) {
17     return Padding(
18       padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 8),
19       child: Row(
20         children: [
21           Text('نوع التوزيع',
22             style: TextStyleManager.regular12
23               .copyWith(color: ColorsManager.secondaryText)),
24           const SizedBox(width: 6),
25           RegionChip(
26             label: 'عام',
27             selected: selectedRegion == 'general',
28             onTap: () => onRegionChanged('general'),
29           ),
30           const SizedBox(width: 6),
31           RegionChip(
32             label: 'المنطقة الغربية',
33             selected: selectedRegion == 'west',
34             onTap: () => onRegionChanged('west'),
35           ),
36           const Spacer(),
37           LegendDot(color: ColorsManager.primaryColor, label: 'أسبوع دراسي'),
38           const SizedBox(width: 4),
39           const LegendDot(color: Colors.orange, label: 'إجازة'),
40           const SizedBox(width: 4),
41           LegendDot(color: ColorsManager.errorColor, label: 'اختبار'),
42         ],
43       ),
44     );
45   }
46 }
47
48 class RegionChip extends StatelessWidget {
49   final String label;
50   final bool selected;
51   final VoidCallback onTap;
52
53   const RegionChip({
54     super.key,
55     required this.label,
56     required this.selected,
57     required this.onTap,
58   });
59
60   @override
61   Widget build(BuildContext context) {
62     return GestureDetector(
63       onTap: onTap,
64       child: Container(
65         padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
66         decoration: BoxDecoration(
67           color: selected
68             ? ColorsManager.primaryColor.withOpacity(0.12)
69             : Colors.transparent,
70           borderRadius: BorderRadius.circular(16),
71           border: Border.all(
72             color:
73               selected ? ColorsManager.primaryColor : ColorsManager.borderLightGray,
74           ),
75         ),
76         child: Text(
77           label,
78           style: TextStyleManager.regular12.copyWith(
79             color: selected
80               ? ColorsManager.primaryColor
81               : ColorsManager.secondaryText,
82             fontSize: 11,
83           ),

```

```

84     ),
85     ),
86   );
87 }
88 }
89
90 class LegendDot extends StatelessWidget {
91   final Color color;
92   final String label;
93
94   const LegendDot({
95     super.key,
96     required this.color,
97     required this.label,
98   });
99
100   @override
101   Widget build(BuildContext context) {
102     return Row(
103       mainAxisAlignment: MainAxisAlignment.min,
104       children: [
105         Container(
106           width: 8,
107           height: 8,
108           decoration: BoxDecoration(color: color, shape: BoxShape.circle),
109         ),
110         const SizedBox(width: 2),
111         Text(label,
112           style: TextStylesManager.regular12
113             .copyWith(color: ColorsManager.secondaryText, fontSize: 9)),
114       ],
115     );
116   }
117 }
118

```

Lines • 274
Dart

Feature / #154 –
Curriculum [lib/features/curriculum/presentation/widgets/subject_picker_section.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/spacing.dart';
5  import 'package:moean/features/exam_generation/data/models/curriculum_models.dart';
6
7  class SubjectPickerSection extends StatelessWidget {
8    final List<CurriculumStageModel> stages;
9    final List<CurriculumGradeModel> grades;
10   final List<CurriculumSubjectModel> subjects;
11   final CurriculumStageModel? selectedStage;
12   final CurriculumGradeModel? selectedGrade;
13   final CurriculumSubjectModel? selectedSubject;
14   final int selectedSemester;
15   final bool loading;
16   final String? error;
17   final void Function(CurriculumStageModel) onStageSelected;
18   final void Function(CurriculumGradeModel) onGradeSelected;
19   final void Function(CurriculumSubjectModel) onSubjectSelected;
20   final void Function(int) onSemesterChanged;
21
22   const SubjectPickerSection({
23     super.key,
24     required this.stages,
25     required this.grades,
26     required this.subjects,
27     required this.selectedStage,
28     required this.selectedGrade,
29     required this.selectedSubject,
30     required this.selectedSemester,
31     required this.loading,
32     required this.error,
33     required this.onStageSelected,
34     required this.onGradeSelected,

```

```

35     required this.onSubjectSelected,
36     required this.onSemesterChanged,
37   });
38
39   @override
40   Widget build(BuildContext context) {
41     return Container(
42       padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
43       decoration: BoxDecoration(
44         color: ColorsManager.surfacePrimary,
45         boxShadow: [
46           BoxShadow(
47             color: Colors.black.withOpacity(0.04),
48             blurRadius: 8,
49             offset: const Offset(0, 2),
50           ),
51         ],
52     ),
53     child: Column(
54       crossAxisAlignment: CrossAxisAlignment.start,
55       children: [
56         if (loading)
57           const Center(
58             child: Padding(
59               padding: EdgeInsets.all(8),
60               child: CircularProgressIndicator(strokeWidth: 2),
61             ),
62           ),
63         else if (error != null)
64           Text(error!,
65             style:
66               TextStyleManager.regular12.copyWith(color: Colors.red))
67         else ...[
68           PickerDropdown(
69             label: 'المرحلة الدراسية',
70             icon: Icons.school_outlined,
71             value: selectedStage?.name,
72             onTap: () => _showPicker<CurriculumStageModel>(
73               context,
74               title: 'اختر المرحلة',
75               items: stages,
76               getName: (e) => e.name,
77               onSelect: onStageSelected,
78             ),
79         ],
80         if (selectedStage != null) ...[
81           verticalSpace8,
82           PickerDropdown(
83             label: 'الصف الدراسي',
84             icon: Icons.class_outlined,
85             value: selectedGrade?.name,
86             onTap: () => _showPicker<CurriculumGradeModel>(
87               context,
88               title: 'اختر الصف',
89               items: grades,
90               getName: (e) => e.name,
91               onSelect: onGradeSelected,
92             ),
93         ],
94         ],
95         if (selectedGrade != null) ...[
96           verticalSpace8,
97           PickerDropdown(
98             label: 'المادة',
99             icon: Icons.book_outlined,
100            value: selectedSubject?.name,
101            onTap: () => _showPicker<CurriculumSubjectModel>(
102              context,
103              title: 'اختر المادة',
104              items: subjects,
105              getName: (e) => e.name,
106              onSelect: onSubjectSelected,
107            ),
108          ],
109        ],

```

```

110     ],
111     if (selectedSubject != null) ...[
112         verticalSpace10,
113         Row(
114             children: [
115                 Text('توزيع المنهج',
116                     style: TextStylesManager.bold14
117                         .copyWith(color: ColorsManager.mainText)),
118                 const Spacer(),
119                 SemesterChip(
120                     label: 'الفصل الأول',
121                     selected: selectedSemester == 1,
122                     onTap: () => onSemesterChanged(1),
123                 ),
124                 horizontalSpace8,
125                 SemesterChip(
126                     label: 'الفصل الثاني',
127                     selected: selectedSemester == 2,
128                     onTap: () => onSemesterChanged(2),
129                 ),
130             ],
131         ),
132     ],
133 ],
134 ),
135 );
136 }
137
138 void _showPicker<T>(  

139     BuildContext context, {  

140     required String title,  

141     required List<T> items,  

142     required String Function(T) getName,  

143     required void Function(T) onSelect,  

144 }) {  

145     showModalBottomSheet(  

146         context: context,  

147         shape: const RoundedRectangleBorder(  

148             borderRadius: BorderRadius.vertical(top: Radius.circular(20)),  

149         ),  

150         builder: (_) => Directionality(  

151             textDirection: TextDirection.rtl,  

152             child: Column(  

153                 mainAxisAlignment: MainAxisAlignment.min,  

154                 children: [  

155                     const SizedBox(height: 12),  

156                     Container(  

157                         width: 40,  

158                         height: 4,  

159                         decoration: BoxDecoration(  

160                             color: Colors.grey.shade300,  

161                             borderRadius: BorderRadius.circular(2),  

162                         ),  

163                     ),  

164                     const SizedBox(height: 12),  

165                     Text(title, style: TextStylesManager.bold16),  

166                     const SizedBox(height: 8),  

167                     const Divider(),  

168                     Flexible(  

169                         child: ListView.separated(  

170                             shrinkWrap: true,  

171                             itemCount: items.length,  

172                             separatorBuilder: (_, __) => const Divider(height: 1),  

173                             itemBuilder: (_, i) => ListTile(  

174                                 title: Text(getName(items[i]),  

175                                     style: TextStylesManager.regular14),  

176                                 onTap: () {  

177                                     Navigator.pop(context);  

178                                     onSelect(items[i]);  

179                                 },  

180                             ),  

181                         ),  

182                     ),  

183                     const SizedBox(height: 16),  

184                 ],

```

```

185     ),
186     ),
187     );
188 }
189 }
190
191 class PickerDropdown extends StatelessWidget {
192   final String label;
193   final IconData icon;
194   final String? value;
195   final VoidCallback onTap;
196
197   const PickerDropdown({
198     super.key,
199     required this.label,
200     required this.icon,
201     required this.value,
202     required this.onTap,
203   });
204
205   @override
206   Widget build(BuildContext context) {
207     return GestureDetector(
208       onTap: onTap,
209       child: Container(
210         padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
211         decoration: BoxDecoration(
212           color: ColorsManager.background,
213           borderRadius: BorderRadius.circular(12),
214           border: Border.all(
215             color: ColorsManager.primaryColor.withOpacity(0.2),
216           ),
217         ),
218         child: Row(
219           children: [
220             Icon(icon, color: ColorsManager.primaryColor, size: 18),
221             const SizedBox(width: 10),
222             Expanded(
223               child: Text(
224                 value ?? label,
225                 style: TextStylesManager.regular14.copyWith(
226                   color: value != null
227                     ? ColorsManager.mainText
228                     : ColorsManager.secondaryText,
229               ),
230             ),
231           ],
232           Icon(Icons.keyboard_arrow_down_rounded,
233             size: 20, color: ColorsManager.secondaryText),
234         ],
235       ),
236     ),
237   );
238 }
239 }
240
241 class SemesterChip extends StatelessWidget {
242   final String label;
243   final bool selected;
244   final VoidCallback onTap;
245
246   const SemesterChip(
247     {super.key, required this.label, required this.selected, required this.onTap});
248
249   @override
250   Widget build(BuildContext context) {
251     return GestureDetector(
252       onTap: onTap,
253       child: AnimatedContainer(
254         duration: const Duration(milliseconds: 200),
255         padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
256         decoration: BoxDecoration(
257           color: selected ? ColorsManager.primaryColor : ColorsManager.background,
258           borderRadius: BorderRadius.circular(20),
259           border: Border.all(

```

```

260         color: ColorsManager.primaryColor,
261         width: selected ? 0 : 1,
262     ),
263 ),
264 child: Text(
265     label,
266     style: TextStylesManager.bold12.copyWith(
267         color: selected ? Colors.white : ColorsManager.primaryColor,
268     ),
269 ),
270 ),
271 );
272 }
273 }
274

```

Lines • Dart 339

Feature / Curriculum

#155 – lib/features/curriculum/presentation/widgets/weeks_grid.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/spacing.dart';
5  import 'package:moean/core/utils/pdf_export_helper.dart';
6  import 'package:moean/features/curriculum/data/models/curriculum_distribution_models.dart';
7  import 'export_pdf_card.dart';
8
9  class WeeksGrid extends StatelessWidget {
10     final CurriculumPlanDetailModel detail;
11     final CurriculumProgressModel? progress;
12     final void Function(int weekId) onPrepareTap;
13
14     const WeeksGrid({
15         super.key,
16         required this.detail,
17         this.progress,
18         required this.onPrepareTap,
19     });
20
21     @override
22     Widget build(BuildContext context) {
23         return CustomScrollView(
24             slivers: [
25                 if (progress != null)
26                     SliverToBoxAdapter(child: ProgressBar(progress: progress!)),
27                 SliverPadding(
28                     padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 8),
29                     sliver: SliverGrid(
30                         gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
31                             crossAxisCount: 2,
32                             mainAxisSpacing: 10,
33                             crossAxisSpacing: 10,
34                             childAspectRatio: 0.82,
35                         ),
36                         delegate: SliverChildBuilderDelegate(
37                             (_, i) => WeekCard(
38                                 week: detail.weeks[i],
39                                 onPrepareTap: onPrepareTap,
40                             ),
41                             childCount: detail.weeks.length,
42                         ),
43                     ),
44                 SliverToBoxAdapter(
45                     child: ExportPdfCard(
46                         onExport: (school, teacher, manager) async {
47                             await PdfExportHelper.exportWidgetsToPdfPages(
48                                 context: context,
49                                 title: 'توزيع المناهج_${detail.plan.subjectName}',
50                                 pages: _buildPdfPages(detail, school, teacher, manager),
51                             );
52                         },
53                     ),
54                 ),
55             ],
56         );
57     }
58 }
59

```



```

56         const SliverToBoxAdapter(child: SizedBox(height: 40)),
57     ],
58 );
59 }
60
61 List<Widget> _buildPdfPages(
62     CurriculumPlanDetailModel detail,
63     String school,
64     String teacher,
65     String manager,
66 ) {
67     final List<Widget> pages = [];
68     const int itemsPerPage = 6;
69
70     for (int i = 0; i < detail.weeks.length; i += itemsPerPage) {
71         final int end = (i + itemsPerPage < detail.weeks.length)
72             ? i + itemsPerPage
73             : detail.weeks.length;
74         final weekSubset = detail.weeks.sublist(i, end);
75
76         pages.add(
77             Container(
78                 color: ColorsManager.background,
79                 padding: const EdgeInsets.all(16),
80                 child: Column(
81                     crossAxisAlignment: CrossAxisAlignment.stretch,
82                     mainAxisAlignment: MainAxisAlignment.min,
83                     children: [
84                         Row(
85                             mainAxisAlignment: MainAxisAlignment.spaceBetween,
86                             children: [
87                                 if (school.isNotEmpty)
88                                     Text('المدرسة: $school',
89                                         style: TextStyleManager.bold14
90                                         .copyWith(color: ColorsManager.mainText)),
91                                 if (teacher.isNotEmpty)
92                                     Text('المعلم: $teacher',
93                                         style: TextStyleManager.bold14
94                                         .copyWith(color: ColorsManager.mainText)),
95                                 if (manager.isNotEmpty)
96                                     Text('المدير: $manager',
97                                         style: TextStyleManager.bold14
98                                         .copyWith(color: ColorsManager.mainText)),
99                             ],
100                         ),
101                         const SizedBox(height: 20),
102                         Text(
103                             '$detail.plan.subjectName - $detail.plan.gradeName',
104                             style: TextStyleManager.bold16
105                             .copyWith(color: ColorsManager.mainText),
106                             textAlign: TextAlign.center,
107                         ),
108                         verticalSpace16,
109                         Wrap(
110                             spacing: 10,
111                             runSpacing: 10,
112                             alignment: WrapAlignment.center,
113                             children: weekSubset.map((week) {
114                                 return SizedBox(
115                                     width: 160,
116                                     height: 220,
117                                     child: WeekCard(week: week, onPrepareTap: (_) {}),
118                                 );
119                             }).toList(),
120                         ),
121                     ],
122                 ),
123             ),
124         );
125     }
126     return pages;
127 }
128 }
129
130 class ProgressBar extends StatelessWidget {

```

```

131 final CurriculumProgressModel progress;
132 const ProgressBar({super.key, required this.progress});
133
134 @override
135 Widget build(BuildContext context) {
136   final Color statusColor = progress.status == 'ahead'
137     ? Colors.green
138     : progress.status == 'behind'
139     ? Colors.red
140     : ColorsManager.primaryColor;
141   final String statusLabel = progress.status == 'ahead'
142     ? 'أسبوع متقدم ${progress.weeksAheadOrBehind}'
143     : progress.status == 'behind'
144     ? 'أسبوع متأخر ${progress.weeksAheadOrBehind}'
145     : 'على المسار الصحيح';
146
147   return Container(
148     margin: const EdgeInsets.fromLTRB(12, 8, 12, 0),
149     padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 10),
150     decoration: BoxDecoration(
151       color: statusColor.withOpacity(0.08),
152       borderRadius: BorderRadius.circular(12),
153       border: Border.all(color: statusColor.withOpacity(0.3)),
154     ),
155     child: Row(
156       children: [
157         Icon(
158           progress.status == 'ahead'
159             ? Icons.trending_up
160             : progress.status == 'behind'
161             ? Icons.trending_down
162             : Icons.check_circle_outline,
163           color: statusColor,
164           size: 20,
165         ),
166         const SizedBox(width: 8),
167         Text(statusLabel,
168           style:
169             TextStyleManager.bold12.copyWith(color: statusColor)),
170         const Spacer(),
171         Text('${progress.completedWeeks}/${progress.totalWeeks} أسبوع',
172           style: TextStyleManager.regular12
173             .copyWith(color: ColorsManager.secondaryText)),
174       ],
175     ),
176   );
177 }
178 }
179
180 class WeekCard extends StatelessWidget {
181   final CurriculumWeekModel week;
182   final void Function(int) onPrepareTap;
183
184   const WeekCard({super.key, required this.week, required this.onPrepareTap});
185
186   @override
187   Widget build(BuildContext context) {
188     final Color headerColor = week.isHoliday
189       ? Colors.orange
190       : week.isExam
191       ? ColorsManager.errorColor
192       : ColorsManager.primaryColor;
193
194     return Container(
195       decoration: BoxDecoration(
196         color: ColorsManager.surfacePrimary,
197         borderRadius: BorderRadius.circular(12),
198         border: Border.all(
199           color: week.isCurrent
200             ? ColorsManager.primaryColor
201             : ColorsManager.borderLightGray,
202           width: week.isCurrent ? 2 : 1,
203         ),
204       ),
205       boxShadow: [
206         BoxShadow(

```

```

206         color: Colors.black.withOpacity(0.04),
207         blurRadius: 6,
208         offset: const Offset(0, 2),
209     ),
210 ],
211 ),
212 child: Column(
213     crossAxisAlignment: CrossAxisAlignment.stretch,
214     children: [
215         // Header bar
216         Container(
217             padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 7),
218             decoration: BoxDecoration(
219                 color: headerColor,
220                 borderRadius:
221                     const BorderRadius.vertical(top: Radius.circular(11)),
222             ),
223             child: Row(
224                 mainAxisAlignment: MainAxisAlignment.spaceBetween,
225                 children: [
226                     Flexible(
227                         child: Text(
228                             week.isHoliday
229                                 ? (week.title ?? 'إجازة')
230                                 : week.isExam
231                                     ? (week.title ?? 'اختبارات')
232                                     : 'الأسبوع ${week.weekNumber}',
233                         style: TextStyleManager.bold12
234                             .copyWith(color: Colors.white, fontSize: 11),
235                         overflow: TextOverflow.ellipsis,
236                     ),
237                 ],
238                 if (week.isCurrent)
239                     Container(
240                         padding: const EdgeInsets.symmetric(
241                             horizontal: 5, vertical: 2),
242                         decoration: BoxDecoration(
243                             color: Colors.white.withOpacity(0.25),
244                             borderRadius: BorderRadius.circular(6),
245                         ),
246                         child: Text('الحالي',
247                             style: TextStyleManager.regular12
248                                 .copyWith(color: Colors.white, fontSize: 9)),
249                     ),
250             ],
251         ),
252     ],
253     // Gregorian date
254     Padding(
255         padding: const EdgeInsets.fromLTRB(8, 6, 8, 2),
256         child: Text(
257             '${week.startsOn} - ${week.endsOn}',
258             style: TextStyleManager.regular12
259                 .copyWith(color: ColorsManager.secondaryText, fontSize: 9),
260             maxLines: 1,
261             overflow: TextOverflow.ellipsis,
262         ),
263     ),
264     // Hijri date
265     if (week.startsOnHijri != null)
266         Padding(
267             padding: const EdgeInsets.fromLTRB(8, 0, 8, 4),
268             child: Text(
269                 '${week.startsOnHijri} - ${week.endsOnHijri}',
270                 style: TextStyleManager.regular12
271                     .copyWith(color: ColorsManager.secondaryText, fontSize: 9),
272                 maxLines: 1,
273                 overflow: TextOverflow.ellipsis,
274             ),
275         ),
276     const Divider(height: 1),
277     // Content
278     Expanded(
279         child: week.isHoliday || week.isExam
280             ? Center(

```

```

281         child: Column(
282             mainAxisAlignment: MainAxisAlignment.center,
283             children: [
284                 Icon(
285                     week.isHoliday
286                     ? Icons.beach_access_rounded
287                     : Icons.quiz_outlined,
288                     color: headerColor.withOpacity(0.7),
289                     size: 28,
290                 ),
291                 const SizedBox(height: 4),
292                 Text(
293                     week.title ?? '',
294                     textAlign: TextAlign.center,
295                     style: TextStylesManager.regular12.copyWith(
296                         color: headerColor,
297                         fontWeight: FontWeight.w600,
298                         fontSize: 11,
299                     ),
300                 ),
301             ],
302         ),
303     ),
304     : ListView.builder(
305         padding:
306             const EdgeInsets.symmetric(horizontal: 8, vertical: 6),
307         itemCount: week.items.length,
308         itemBuilder: (_, i) {
309             final item = week.items[i];
310             return Padding(
311                 padding: const EdgeInsets.only(bottom: 2),
312                 child: Row(
313                     crossAxisAlignment: CrossAxisAlignment.start,
314                     children: [
315                         Text('• ',
316                             style: TextStyle(
317                                 fontSize: 11,
318                                 color: ColorsManager.primaryColor)),
319                         Expanded(
320                             child: Text(
321                                 item.title,
322                                 style: TextStylesManager.regular12.copyWith(
323                                     fontSize: 11, height: 1.3),
324                                 maxLines: 2,
325                                 overflow: TextOverflow.ellipsis,
326                             ),
327                         ),
328                     ],
329                 ),
330             );
331         },
332     ),
333 ),
334 ],
335 ),
336 );
337 }
338 }
339

```

Lines • 82
Dart

Feature / Exam #156 –
Generation

[lib/features/exam_generation/data/datasources/local_curriculum_provider.dart](#)

```

1 class LocalCurriculumProvider {
2     // Simulating Grade Stage, Grade, Subject, Unit structure
3     // This is a temporary local provider until backend endpoints are available.
4
5     static const List<String> gradeStages = [
6         'المرحلة الابتدائية',
7         'المرحلة المتوسطة',
8         'المرحلة الثانوية',
9     ];
10

```

```

11 static const Map<String, List<String>> gradesByStage = {
12     'المرحلة الابتدائية': ['الصف الأول', 'الصف الثاني', 'الصف الثالث', 'الصف الرابع', 'الصف الخامس', 'الصف السادس',
13     'المرحلة المتوسطة': ['الصف الأول المتوسط', 'الصف الثاني المتوسط', 'الصف الثالث المتوسط'],
14     'المرحلة الثانوية': ['الصف الأول الثانوي', 'الصف الثاني الثانوي', 'الصف الثالث الثانوي'],
15 };
16
17 static const List<String> subjects = [
18     'القرآن الكريم',
19     'الدراسات الإسلامية',
20     'اللغة العربية',
21     'الرياضيات',
22     'العلوم',
23     'اللغة الإنجليزية',
24     'الدراسات الاجتماعية',
25     'الحاسب وتقنية المعلومات',
26 ];
27
28 static const List<String> semesters = [
29     'الفصل الدراسي الأول',
30     'الفصل الدراسي الثاني',
31 ];
32
33 // Dummy Units based on Subject
34 static const Map<String, List<String>> unitsBySubject = {
35     'العلوم': [
36         'طبيعة العلم',
37         'الخلايا وتصنيف المخلوقات الحية',
38         'الحيوانات اللافقارية',
39         'الحيوانات الفقارية',
40     ],
41     'الرياضيات': [
42         'الجبر',
43         'الهندسة',
44         'الإحصاء والاحتمالات',
45         'المعادلات',
46     ],
47     // Default fallback
48     'default': [
49         'الوحدة الأولى',
50         'الوحدة الثانية',
51         'الوحدة الثالثة',
52         'الوحدة الرابعة',
53     ],
54 };
55
56 // Dummy Lessons based on Unit
57 static List<Map<String, dynamic>> getLessonsForUnit(String unit) {
58     if (unit == 'طبيعة العلم') {
59         return [
60             {'id': 101, 'name': 'العلم وعملياته'},
61             {'id': 102, 'name': 'النماذج العلمية'},
62             {'id': 103, 'name': 'تقويم التفسيرات العلمية'},
63         ];
64     }
65     if (unit == 'الخلايا وتصنيف المخلوقات الحية') {
66         return [
67             {'id': 104, 'name': 'عالم الخلايا'},
68             {'id': 105, 'name': 'وظائف الخلايا'},
69             {'id': 106, 'name': 'تصنيف المخلوقات الحية'},
70         ];
71     }
72
73     // Fallback
74     return [
75         {'id': DateTime.now().millisecondsSinceEpoch % 1000 + 1, 'name': '$unit - الدرس الأول'},
76         {'id': DateTime.now().millisecondsSinceEpoch % 1000 + 2, 'name': '$unit - الدرس الثاني'},
77         {'id': DateTime.now().millisecondsSinceEpoch % 1000 + 3, 'name': '$unit - الدرس الثالث'},
78         {'id': DateTime.now().millisecondsSinceEpoch % 1000 + 4, 'name': '$unit - الدرس الرابع'},
79     ];
80 }
81 }
82

```

```
1 class CurriculumStageModel {
2   final int id;
3   final String name;
4   final List<CurriculumGradeModel> grades;
5
6   CurriculumStageModel({required this.id, required this.name, required this.grades});
7
8   factory CurriculumStageModel.fromJson(Map<String, dynamic> json) {
9     return CurriculumStageModel(
10       id: json['id'] ?? 0,
11       name: json['name'] ?? '',
12       grades: (json['grades'] as List?)?.map((e) => CurriculumGradeModel.fromJson(e)).toList() ?? [],
13     );
14   }
15 }
16
17 class CurriculumGradeModel {
18   final int id;
19   final String name;
20   final String? track;
21   final List<CurriculumSubjectModel> subjects;
22
23   CurriculumGradeModel({required this.id, required this.name, this.track, required this.subjects});
24
25   factory CurriculumGradeModel.fromJson(Map<String, dynamic> json) {
26     return CurriculumGradeModel(
27       id: json['id'] ?? 0,
28       name: json['name'] ?? '',
29       track: json['track']?.toString(),
30       subjects: (json['subjects'] as List?)?.map((e) => CurriculumSubjectModel.fromJson(e)).toList() ?? [],
31     );
32   }
33 }
34
35 class CurriculumSubjectModel {
36   final int id;
37   final String name;
38
39   CurriculumSubjectModel({required this.id, required this.name});
40
41   factory CurriculumSubjectModel.fromJson(Map<String, dynamic> json) {
42     return CurriculumSubjectModel(
43       id: json['id'] ?? 0,
44       name: json['name'] ?? '',
45     );
46   }
47 }
48
49 class SubjectDetailsModel {
50   final int subjectId;
51   final String subjectName;
52   final List<CurriculumChapterModel> chapters;
53
54   SubjectDetailsModel({required this.subjectId, required this.subjectName, required this.chapters});
55
56   factory SubjectDetailsModel.fromJson(Map<String, dynamic> json) {
57     return SubjectDetailsModel(
58       subjectId: json['subject_id'] ?? 0,
59       subjectName: json['subject_name'] ?? '',
60       chapters: (json['chapters'] as List?)?.map((e) => CurriculumChapterModel.fromJson(e)).toList() ?? [],
61     );
62   }
63 }
64
65 class CurriculumChapterModel {
66   final int chapterId;
67   final String title;
68   final String? unitId;
69   final String? unitName;
70   final String semester;
71   final List<CurriculumLessonModel> lessons;
72 }
```

```

73 CurriculumChapterModel({
74   required this.chapterId,
75   required this.title,
76   this.unitId,
77   this.unitName,
78   required this.semester,
79   required this.lessons,
80 });
81
82 factory CurriculumChapterModel.fromJson(Map<String, dynamic> json) {
83   return CurriculumChapterModel(
84     chapterId: json['chapter_id'] ?? 0,
85     title: json['title'] ?? '',
86     unitId: json['unit_id']?.toString(),
87     unitName: json['unit_name']?.toString(),
88     semester: json['semester']?.toString() ?? '1',
89     lessons: (json['lessons'] as List?)?.map((e) => CurriculumLessonModel.fromJson(e)).toList() ?? [],
90   );
91 }
92 }
93
94 class CurriculumLessonModel {
95   final int id;
96   final String title;
97   final int orderIndex;
98   final String semester;
99   final int? sourceLessonId;
100  final int? lessonApiId;
101
102  CurriculumLessonModel({
103    required this.id,
104    required this.title,
105    required this.orderIndex,
106    required this.semester,
107    this.sourceLessonId,
108    this.lessonApiId,
109  });
110
111  factory CurriculumLessonModel.fromJson(Map<String, dynamic> json) {
112    return CurriculumLessonModel(
113      id: json['id'] ?? 0,
114      title: json['title'] ?? json['name'] ?? '',
115      orderIndex: json['order_index'] ?? 0,
116      semester: json['semester']?.toString() ?? '1',
117      sourceLessonId: json['source_lesson_id'],
118      lessonApiId: json['lesson_api_id'],
119    );
120  }
121 }
122

```

Lines • Dart 165

Feature / Exam Generation

#158 – lib/features/exam_generation/data/models/exam_models.dart

```

1 import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
2
3 class ExamModel extends ExamEntity {
4   ExamModel({
5     required super.id,
6     required super.teacherId,
7     required super.title,
8     required super.status,
9     required super.totalPoints,
10    required super.createdAt,
11    required super.updatedAt,
12    required super.questions,
13  });
14
15  factory ExamModel.fromJson(Map<String, dynamic> json) {
16    return ExamModel(
17      id: json['id'] as int? ?? 0,
18      teacherId: json['teacher_id'] as int? ?? 0,
19      title: json['title'] as String? ?? '',
20      status: json['status'] as String? ?? '',

```

```

21     totalPoints: (json['total_points'] as num?)?.toInt() ?? 0,
22     createdAt: json['created_at'] as String? ?? '',
23     updatedAt: json['updated_at'] as String? ?? '',
24     questions: (json['questions'] as List<dynamic>?)
25         ?.map((e) => QuestionModel.fromJson(e as Map<String, dynamic>))
26         .toList() ??
27         [],
28     );
29 }
30 }
31
32 class QuestionModel extends QuestionEntity {
33     QuestionModel({
34         required super.id,
35         required super.lessonId,
36         required super.type,
37         required super.questionText,
38         required super.options,
39         required super.correctAnswer,
40         required super.source,
41         required super.usageCount,
42         required super.questionOrder,
43         required super.points,
44         super.reviewStatus,
45         super.rejectionReason,
46     });
47
48     factory QuestionModel.fromJson(Map<String, dynamic> json) {
49         final type = json['type'] as String? ?? '';
50         dynamic parsedOptions;
51
52         if (type == 'matching') {
53             parsedOptions = MatchingOptionsModel.fromJson(json['options'] as Map<String, dynamic>? ?? {});
54         } else if (type == 'mcq') {
55             parsedOptions = (json['options'] as List<dynamic>?)?.map((e) => e.toString()).toList() ?? [];
56         } else {
57             parsedOptions = json['options'];
58         }
59
60         return QuestionModel(
61             id: json['id'] as int? ?? 0,
62             lessonId: json['lesson_id'] as int? ?? 0,
63             type: type,
64             questionText: json['question_text'] as String? ?? '',
65             options: parsedOptions,
66             correctAnswer: json['correct_answer'] as String? ?? '',
67             source: json['source'] as String? ?? '',
68             usageCount: json['usage_count'] as int? ?? 0,
69             questionOrder: json['question_order'] as int? ?? 0,
70             points: (json['points'] as num?)?.toInt() ?? 0,
71             reviewStatus: json['review_status'] as String? ?? '',
72             rejectionReason: json['rejection_reason'] as String?,
73         );
74     }
75 }
76
77 class MatchingOptionsModel {
78     final List<String> columnA;
79     final List<String> columnB;
80
81     MatchingOptionsModel({
82         required this.columnA,
83         required this.columnB,
84     });
85
86     factory MatchingOptionsModel.fromJson(Map<String, dynamic> json) {
87         return MatchingOptionsModel(
88             columnA: (json['column_a'] as List<dynamic>?)?.map((e) => e.toString()).toList() ?? [],
89             columnB: (json['column_b'] as List<dynamic>?)?.map((e) => e.toString()).toList() ?? [],
90         );
91     }
92 }
93
94 class ExamListModel extends ExamListEntity {
95     ExamListModel({

```



```

96     required super.id,
97     required super.title,
98     required super.status,
99     required super.questionsCount,
100    required super.totalPoints,
101    required super.createdAt,
102    required super.updatedAt,
103  });
104
105  factory ExamListModel.fromJson(Map<String, dynamic> json) {
106    return ExamListModel(
107      id: json['id'] as int? ?? 0,
108      title: json['title'] as String? ?? '',
109      status: json['status'] as String? ?? '',
110      questionsCount: json['questions_count'] as int? ?? 0,
111      totalPoints: (json['total_points'] as num?)?.toInt() ?? 0,
112      createdAt: json['created_at'] as String? ?? '',
113      updatedAt: json['updated_at'] as String? ?? '',
114    );
115  }
116 }
117
118 class ExamPaginationModel extends ExamPaginationEntity {
119   ExamPaginationModel({
120     required super.currentPage,
121     required super.lastPage,
122     required super.perPage,
123     required super.total,
124     required super.data,
125   });
126
127   factory ExamPaginationModel.fromJson(Map<String, dynamic> json) {
128     final meta = json['meta'] as Map<String, dynamic>? ?? {};
129     return ExamPaginationModel(
130       currentPage: meta['current_page'] as int? ?? 1,
131       lastPage: meta['last_page'] as int? ?? 1,
132       perPage: meta['per_page'] as int? ?? 20,
133       total: meta['total'] as int? ?? 0,
134       data: (json['data'] as List<dynamic>?)
135         ?.map((e) => ExamListModel.fromJson(e as Map<String, dynamic>))
136         .toList() ??
137       [],
138     );
139   }
140 }
141
142 class QuestionPaginationModel extends QuestionPaginationEntity {
143   QuestionPaginationModel({
144     required super.currentPage,
145     required super.lastPage,
146     required super.perPage,
147     required super.total,
148     required super.data,
149   });
150
151   factory QuestionPaginationModel.fromJson(Map<String, dynamic> json) {
152     final meta = json['meta'] as Map<String, dynamic>? ?? {};
153     return QuestionPaginationModel(
154       currentPage: meta['current_page'] as int? ?? 1,
155       lastPage: meta['last_page'] as int? ?? 1,
156       perPage: meta['per_page'] as int? ?? 20,
157       total: meta['total'] as int? ?? 0,
158       data: (json['data'] as List<dynamic>?)
159         ?.map((e) => QuestionModel.fromJson(e as Map<String, dynamic>))
160         .toList() ??
161       [],
162     );
163   }
164 }
165

```

```
1 class GenerateExamRequest {
2   final String? title;
3   final String grade;
4   final String subject;
5   final List<LessonRequest> lessons;
6
7   GenerateExamRequest({
8     this.title,
9     required this.grade,
10    required this.subject,
11    required this.lessons,
12  });
13
14  Map<String, dynamic> toJson() {
15    return {
16      if (title != null && title!.isNotEmpty) 'title': title,
17      'grade': grade,
18      'subject': subject,
19      'lessons': lessons.map((e) => e.toJson()).toList(),
20    };
21  }
22 }
23
24 class LessonRequest {
25   final int lessonId;
26   final String lessonName;
27   final RequestedCounts requestedCounts;
28   final List<int>? selectedQuestionIds;
29
30   LessonRequest({
31     required this.lessonId,
32     required this.lessonName,
33     required this.requestedCounts,
34     this.selectedQuestionIds,
35   });
36
37   Map<String, dynamic> toJson() {
38     return {
39       'lesson_id': lessonId,
40       'lesson_name': lessonName,
41       'requested_counts': requestedCounts.toJson(),
42       if (selectedQuestionIds != null && selectedQuestionIds!.isNotEmpty) 'selected_question_ids': selectedQu
43     };
44   }
45 }
46
47 class RequestedCounts {
48   final int mcq;
49   final int trueFalse;
50   final int fillBlank;
51   final int essay;
52   final int matching;
53
54   const RequestedCounts({
55     required this.mcq,
56     required this.trueFalse,
57     required this.fillBlank,
58     required this.essay,
59     required this.matching,
60   });
61
62   Map<String, dynamic> toJson() {
63     // Backend requires all keys even if 0
64     return {
65       'mcq': mcq,
66       'true_false': trueFalse,
67       'fill_blank': fillBlank,
68       'essay': essay,
69       'matching': matching,
70     };
71   }
72 }
```

```

72 }
73
74 class UpdateQuestionPointsRequest {
75   final List<QuestionPoints> questions;
76
77   UpdateQuestionPointsRequest({required this.questions});
78
79   Map<String, dynamic> toJson() {
80     return {
81       'questions': questions.map((e) => e.toJson()).toList(),
82     };
83   }
84 }
85
86 class QuestionPoints {
87   final int questionId;
88   final double points;
89
90   QuestionPoints({
91     required this.questionId,
92     required this.points,
93   });
94
95   Map<String, dynamic> toJson() {
96     return {
97       'question_id': questionId,
98       'points': points,
99     };
100   }
101 }
102

```

Lines • 293
Dart

Feature / Exam #160 –

Generation [lib/features/exam_generation/data/repositories/exam_repository_impl.dart](#)

```

1 import 'package:dartz/dartz.dart';
2 import 'package:moean/core/errors/failures.dart';
3 import 'package:moean/core/network/remote/api_service.dart';
4 import 'package:moean/features/exam_generation/data/models/exam_models.dart';
5 import 'package:moean/features/exam_generation/data/models/exam_requests.dart';
6 import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
7 import 'package:moean/features/exam_generation/domain/repositories/exam_repository.dart';
8
9 class ExamRepositoryImpl implements ExamRepository {
10   @override
11   Future<Either<Failure, ExamEntity>> generateExam({
12     required String title,
13     required String grade,
14     required String subject,
15     required List<Map<String, dynamic>> lessons,
16   }) async {
17     final request = GenerateExamRequest(
18       title: title,
19       grade: grade,
20       subject: subject,
21       lessons: lessons.map((e) => LessonRequest(
22         lessonId: e['lesson_id'] as int? ?? 0,
23         lessonName: e['lesson_name'] as String? ?? '',
24         selectedQuestionIds: e['selected_question_ids'] != null ? List<int>.from(e['selected_question_ids'])
: null,
25         requestedCounts: RequestedCounts(
26           mcq: e['requested_counts']['mcq'] ?? 0,
27           trueFalse: e['requested_counts']['true_false'] ?? 0,
28           fillBlank: e['requested_counts']['fill_blank'] ?? 0,
29           essay: e['requested_counts']['essay'] ?? 0,
30           matching: e['requested_counts']['matching'] ?? 0,
31         ),
32       )),
33     );
34
35     final result = await ApiService.generateExam(request.toJson());
36
37     return result.fold(

```

```

38     (errorString) => Left(_parseApiError(errorString)),
39     (data) {
40         if (data['success'] == true && data['exam'] != null) {
41             try {
42                 final examModel = ExamModel.fromJson(data['exam']);
43                 return Right(examModel);
44             } catch (e) {
45                 return Left(ServerFailure('Failed to parse generated exam: $e'));
46             }
47         } else {
48             return Left(AiGenerationFailure(data['message']?.toString() ?? 'AI Generation Failed'));
49         }
50     },
51 );
52 }
53
54 @override
55 Future<Either<Failure, ExamEntity>> getExam(int id) async {
56     final result = await ApiService.getExam(id);
57
58     return result.fold(
59         (errorString) => Left(_parseApiError(errorString)),
60         (data) {
61             if (data['success'] == true && data['exam'] != null) {
62                 try {
63                     final examModel = ExamModel.fromJson(data['exam']);
64                     return Right(examModel);
65                 } catch (e) {
66                     return Left(ServerFailure('Failed to parse exam: $e'));
67                 }
68             } else {
69                 return const Left(NotFoundFailure('Exam not found'));
70             }
71         },
72     );
73 }
74
75 @override
76 Future<Either<Failure, ExamEntity>> updateExamPoints(int id, List<Map<String, dynamic>> questions) async {
77     final request = UpdateQuestionPointsRequest(
78         questions: questions.map((e) => QuestionPoints(
79             questionId: e['question_id'] as int? ?? 0,
80             points: (e['points'] as num?)?.toDouble() ?? 0.0,
81         )).toList(),
82     );
83
84     final result = await ApiService.updateExamPoints(id, request.toJson());
85
86     return result.fold(
87         (errorString) => Left(_parseApiError(errorString)),
88         (data) {
89             if (data['success'] == true && data['exam'] != null) {
90                 try {
91                     final examModel = ExamModel.fromJson(data['exam']);
92                     return Right(examModel);
93                 } catch (e) {
94                     return Left(ServerFailure('Failed to parse exam: $e'));
95                 }
96             } else {
97                 return const Left(ValidationFailure('Failed to update points', {}));
98             }
99         },
100     );
101 }
102
103 @override
104 Future<Either<Failure, ExamPaginationEntity>> getMyExams({
105     int page = 1,
106     int perPage = 20,
107     String? status,
108     String? search,
109 }) async {
110     final result = await ApiService.getMyExams(
111         page: page,
112         perPage: perPage,

```

```

113         status: status,
114         search: search,
115     );
116
117     return result.fold(
118         (errorString) => Left(_parseApiError(errorString)),
119         (data) {
120             if (data['success'] == true) {
121                 try {
122                     final paginationModel = ExamPaginationModel.fromJson(data);
123                     return Right(paginationModel);
124                 } catch (e) {
125                     return Left(ServerFailure('Failed to parse exams list: $e'));
126                 }
127             } else {
128                 return Left(ServerFailure(data['message']?.toString() ?? 'Failed to fetch exams'));
129             }
130         },
131     );
132 }
133
134 @override
135 Future<Either<Failure, ExamEntity>> publishExam(int id) async {
136     final result = await ApiService.publishExam(id);
137
138     return result.fold(
139         (errorString) => Left(_parseApiError(errorString)),
140         (data) {
141             if (data['success'] == true && data['exam'] != null) {
142                 try {
143                     final examModel = ExamModel.fromJson(data['exam']);
144                     return Right(examModel);
145                 } catch (e) {
146                     return Left(ServerFailure('Failed to parse published exam: $e'));
147                 }
148             } else {
149                 return Left(ServerFailure(data['message']?.toString() ?? 'Failed to publish exam'));
150             }
151         },
152     );
153 }
154
155 @override
156 Future<Either<Failure, bool>> deleteExam(int id) async {
157     final result = await ApiService.deleteExam(id);
158
159     return result.fold(
160         (errorString) => Left(_parseApiError(errorString)),
161         (data) {
162             return const Right(true);
163         },
164     );
165 }
166
167 @override
168 Future<Either<Failure, Map<String, dynamic>>> getExams({int page = 1, int perPage = 20, String? status}) as
ync {
169     final result = await ApiService.getExams(page: page, perPage: perPage, status: status);
170
171     return result.fold(
172         (errorString) => Left(_parseApiError(errorString)),
173         (data) {
174             if (data['success'] == true) {
175                 return Right(data);
176             } else {
177                 return Left(ServerFailure(data['message']?.toString() ?? 'Failed to fetch exams'));
178             }
179         },
180     );
181 }
182
183 @override
184 Future<Either<Failure, ExamEntity>> addManualQuestion(Map<String, dynamic> request) async {
185     final result = await ApiService.addManualQuestion(request);
186

```

```

187     return result.fold(
188         (errorString) => Left(_parseApiError(errorString)),
189         (data) {
190             if (data['success'] == true && data['exam'] != null) {
191                 try {
192                     final examModel = ExamModel.fromJson(data['exam']);
193                     return Right(examModel);
194                 } catch (e) {
195                     return Left(ServerFailure('Failed to parse exam: $e'));
196                 }
197             } else {
198                 return const Left(ValidationFailure('Failed to add question', {}));
199             }
200         },
201     );
202 }
203
204 Failure _parseApiError(String errorString) {
205     if (errorString.startsWith('__402__')) {
206         final parts = errorString.split(':');
207         final code = parts.length > 1 ? parts[1] : 'quota_exceeded';
208         final msg = parts.length > 2 ? parts.sublist(2).join(':') : 'ترقية الحساب المطلوبة';
209         return PaymentRequiredFailure(msg, code: code);
210     }
211     if (errorString.toLowerCase().contains('unauthenticated') || errorString.contains('401')) {
212         return const UnauthorizedFailure('Unauthenticated');
213     }
214     if (errorString.contains('403')) {
215         return const ForbiddenFailure('Access Denied');
216     }
217     if (errorString.contains('404')) {
218         return const NotFoundFailure('Not Found');
219     }
220     if (errorString.contains('409')) {
221         return const ExamLockedFailure('Exam is no longer a draft');
222     }
223     if (errorString.contains('422')) {
224         return ValidationFailure(errorString, {});
225     }
226     if (errorString.contains('502')) {
227         return const AiGenerationFailure('حدث خطأ أثناء انشاء الأسئلة، الرجاء المحاولة مرة أخرى لاحقاً');
228     }
229     if (errorString.contains('503') || errorString.contains('504') || errorString.contains('timeout')) {
230         return const TimeoutFailure('Service unavailable or timed out');
231     }
232     return ServerFailure(errorString);
233 }
234
235 @override
236 Future<Either<Failure, QuestionPaginationEntity>> getMyQuestions({
237     int page = 1,
238     int perPage = 20,
239     String? type,
240     String? difficulty,
241     String? reviewStatus,
242     int? lessonId,
243     String? search,
244 }) async {
245     final result = await ApiService.getMyQuestions(
246         page: page,
247         perPage: perPage,
248         type: type,
249         difficulty: difficulty,
250         reviewStatus: reviewStatus,
251         lessonId: lessonId,
252         search: search,
253     );
254
255     return result.fold(
256         (errorString) => Left(_parseApiError(errorString)),
257         (data) {
258             if (data['success'] == true) {
259                 try {
260                     final paginationModel = QuestionPaginationModel.fromJson(data);
261                     return Right(paginationModel);

```

```

262         } catch (e) {
263             return Left(ServerFailure('Failed to parse questions list: $e'));
264         }
265     } else {
266         return Left(ServerFailure(data['message']?.toString() ?? 'Failed to fetch questions'));
267     }
268 },
269 );
270 }
271
272 @override
273 Future<Either<Failure, QuestionEntity>> updateCustomQuestion(int id, Map<String, dynamic> data) async {
274     final result = await ApiService.updateCustomQuestion(id, data);
275
276     return result.fold(
277         (errorString) => Left(_parseApiError(errorString)),
278         (dataResponse) {
279             if (dataResponse['success'] == true && dataResponse['question'] != null) {
280                 try {
281                     final questionModel = QuestionModel.fromJson(dataResponse['question']);
282                     return Right(questionModel);
283                 } catch (e) {
284                     return Left(ServerFailure('Failed to parse updated question: $e'));
285                 }
286             } else {
287                 return Left(ValidationFailure(dataResponse['message']?.toString() ?? 'Failed to update question',
288 {}));
289             }
290         },
291     );
292 }
293

```

Lines • Dart 370

Feature / Exam Generation

#161 – lib/features/exam_generation/data/services/exam_pdf_service.dart

```

1  import 'dart:typed_data';
2  import 'package:flutter/services.dart' show rootBundle;
3  import 'package:pdf/pdf.dart';
4  import 'package:pdf/widgets.dart' as pw;
5  import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
6  import 'package:moean/features/exam_generation/data/models/exam_models.dart';
7
8  class ExamPdfService {
9
10     static final PdfColor _primaryColor = PdfColor.fromHex('073F49');
11     static final PdfColor _headerBg = PdfColor.fromHex('0B4A45');
12     static final PdfColor _textDark = PdfColor.fromHex('1E293B');
13
14     static bool isEnglishText(String text) {
15         if (text.trim().isEmpty) return false;
16
17         int englishCount = 0;
18         int arabicCount = 0;
19
20         for (final rune in text.runes) {
21             if ((rune >= 0x0041 && rune <= 0x005A) || (rune >= 0x0061 && rune <= 0x007A)) {
22                 englishCount++;
23             } else if (rune >= 0x0600 && rune <= 0x06FF) {
24                 arabicCount++;
25             }
26         }
27
28         return englishCount > arabicCount;
29     }
30
31     static Future<Uint8List> generatePdf({
32         required ExamEntity exam,
33         required bool showAnswers,
34         required String teacherName,
35         required String schoolName,
36         required String educationOffice,
37     }) async {

```

```

38 final ByteData fontData = await rootBundle.load('assets/fonts/Tajawal-Regular.ttf');
39 final pw.Font ttf = pw.Font.ttf(fontData);
40
41 final ByteData boldFontData = await rootBundle.load('assets/fonts/Tajawal-Bold.ttf');
42 final pw.Font ttfBold = pw.Font.ttf(boldFontData);
43
44 final ByteData ministryImageData = await rootBundle.load('assets/images/ministry.jpg');
45 final pw.MemoryImage ministryImage = pw.MemoryImage(ministryImageData.buffer.asUint8List());
46
47 final ByteData logoImageData = await rootBundle.load('assets/images/logo_icon-removebg-preview.png');
48 final pw.MemoryImage logoImage = pw.MemoryImage(logoImageData.buffer.asUint8List());
49
50 final pdf = pw.Document(
51   theme: pw.ThemeData.withFont(
52     base: ttf,
53     bold: ttfBold,
54   ),
55 );
56
57 final pw.TextStyle arabicRegular = pw.TextStyle(
58   fontSize: 12,
59   color: _textDark,
60 );
61 final pw.TextStyle arabicBold = pw.TextStyle(
62   fontSize: 12,
63   fontWeight: pw.FontWeight.bold,
64   color: _textDark,
65 );
66 final pw.TextStyle titleStyle = pw.TextStyle(
67   fontSize: 16,
68   fontWeight: pw.FontWeight.bold,
69   color: _primaryColor,
70 );
71 final pw.TextStyle headerWhite = pw.TextStyle(
72   fontSize: 10,
73   fontWeight: pw.FontWeight.bold,
74   color: PdfColors.white,
75 );
76
77 final questions = List<QuestionEntity>.from(exam.questions)
78   ..sort((a, b) => a.questionOrder.compareTo(b.questionOrder));
79
80 final isExamTitleEnglish = isEnglishText(exam.title);
81
82 pdf.addPage(
83   pw.MultiPage(
84     pageFormat: PdfPageFormat.a4,
85     textDirection: pw.TextDirection.rtl,
86     margin: const pw.EdgeInsets.all(24),
87     build: (pw.Context context) {
88       return [
89         // Header
90         _buildPageHeader(
91           title: exam.title,
92           teacherName: teacherName,
93           schoolName: schoolName,
94           educationOffice: educationOffice,
95           date: exam.createdAt.split('T')[0],
96           titleStyle: headerWhite.copyWith(fontSize: 16),
97           headerStyle: headerWhite.copyWith(fontSize: 10, color: PdfColors.cyan200),
98           arabicRegular: arabicRegular,
99           ministryImage: ministryImage,
100          logoImage: logoImage,
101        ),
102        pw.SizedBox(height: 20),
103
104        pw.Center(
105          child: pw.Text(
106            exam.title,
107            style: titleStyle,
108            textDirection: isExamTitleEnglish ? pw.TextDirection.ltr : pw.TextDirection.rtl,
109          ),
110        ),
111        pw.SizedBox(height: 20),
112

```



```

184         child: pw.Image(
185             ministryImage,
186             fit: pw.BoxFit.cover,
187         ),
188     ),
189 ),
190 ),
191 ],
192 ),
193 ],
194 ),
195 );
196 }
197
198 static pw.Widget _buildQuestion(
199     QuestionEntity q, pw.TextStyle regular, pw.TextStyle bold, bool showAnswers) {
200     final isEnglish = isEnglishText(q.questionText);
201     final textDir = isEnglish ? pw.TextDirection.ltr : pw.TextDirection.rtl;
202
203     return pw.Directionality(
204         textDirection: textDir,
205         child: pw.Padding(
206             padding: const pw.EdgeInsets.only(bottom: 20),
207             child: pw.Column(
208                 crossAxisAlignment: pw.CrossAxisAlignment.start,
209                 children: [
210                     pw.Row(
211                         crossAxisAlignment: pw.CrossAxisAlignment.start,
212                         children: [
213                             pw.Text('${q.questionOrder}.', style: bold, textDirection: textDir),
214                             pw.SizedBox(width: 4),
215                             pw.Expanded(
216                                 child: pw.Text(q.questionText, style: bold, textDirection: textDir),
217                             ),
218                             pw.SizedBox(width: 8),
219                             pw.Text(
220                                 isEnglish ? '[$ {q.points} point${q.points > 1 ? "s" : ""}]' : '[$ {q.points} درجة]',
221                                 style: regular.copyWith(color: PdfColors.grey700, fontSize: 10),
222                                 textDirection: textDir,
223                             ),
224                         ],
225                     ),
226                     pw.SizedBox(height: 10),
227                     _buildQuestionOptions(q, regular, bold, showAnswers, isEnglish),
228                     if (showAnswers && q.type != 'matching' && q.type != 'mcq') ...[
229                         pw.SizedBox(height: 10),
230                         pw.Text(
231                             isEnglish ? 'Answer: ${q.correctAnswer}' : 'الإجابة: ${q.correctAnswer}',
232                             style: bold.copyWith(color: PdfColors.green700),
233                             textDirection: textDir,
234                         ),
235                     ],
236                 ],
237             ),
238         ),
239     );
240 }
241
242 static pw.Widget _buildQuestionOptions(
243     QuestionEntity q, pw.TextStyle regular, pw.TextStyle bold, bool showAnswers, bool isEnglish) {
244     final textDir = isEnglish ? pw.TextDirection.ltr : pw.TextDirection.rtl;
245
246     if (q.type == 'mcq') {
247         final options = List<String>.from(q.options ?? []);
248         return pw.Column(
249             crossAxisAlignment: pw.CrossAxisAlignment.start,
250             children: options.map((opt) {
251                 final isCorrect = opt == q.correctAnswer;
252                 return pw.Padding(
253                     padding: const pw.EdgeInsets.only(bottom: 4),
254                     child: pw.Row(
255                         crossAxisAlignment: pw.CrossAxisAlignment.center,
256                         children: [
257                             pw.Container(
258                                 width: 12,

```

```

259         height: 12,
260         decoration: pw.BoxDecoration(
261             shape: pw.BoxShape.circle,
262             border: pw.Border.all(color: PdfColors.grey),
263             color: (showAnswers && isCorrect) ? PdfColors.green700 : null,
264         ),
265     ),
266     pw.SizedBox(width: 8),
267     pw.Expanded(
268         child: pw.Text(
269             opt,
270             style: (showAnswers && isCorrect)
271                 ? bold.copyWith(color: PdfColors.green700)
272                 : regular,
273             textDirection: textDir,
274         ),
275     ),
276 ],
277 ),
278 );
279 }).toList(),
280 );
281 } else if (q.type == 'true_false') {
282     final isTrueCorrect = showAnswers && (q.correctAnswer == 'صح' || q.correctAnswer.toLowerCase() == 'true');
283     final isFalseCorrect = showAnswers && (q.correctAnswer == 'خطأ' || q.correctAnswer.toLowerCase() == 'false');
284
285     return pw.Row(
286         mainAxisAlignment: pw.MainAxisAlignment.spaceEvenly,
287         children: [
288             pw.Row(
289                 children: [
290                     pw.Container(
291                         width: 12,
292                         height: 12,
293                         decoration: pw.BoxDecoration(
294                             shape: pw.BoxShape.circle,
295                             border: pw.Border.all(color: PdfColors.grey),
296                             color: isTrueCorrect ? PdfColors.green700 : null,
297                         ),
298                     ),
299                     pw.SizedBox(width: 8),
300                     pw.Text(isEnglish ? 'True' : 'صح', style: regular, textDirection: textDir),
301                 ],
302             ),
303             pw.Row(
304                 children: [
305                     pw.Container(
306                         width: 12,
307                         height: 12,
308                         decoration: pw.BoxDecoration(
309                             shape: pw.BoxShape.circle,
310                             border: pw.Border.all(color: PdfColors.grey),
311                             color: isFalseCorrect ? PdfColors.green700 : null,
312                         ),
313                     ),
314                     pw.SizedBox(width: 8),
315                     pw.Text(isEnglish ? 'False' : 'خطأ', style: regular, textDirection: textDir),
316                 ],
317             ),
318         ],
319     );
320 } else if (q.type == 'matching') {
321     List<String> columnA = [];
322     List<String> columnB = [];
323
324     if (q.options is MatchingOptionsModel) {
325         columnA = (q.options as MatchingOptionsModel).columnA;
326         columnB = (q.options as MatchingOptionsModel).columnB;
327     } else if (q.options is Map) {
328         columnA = List<String>.from(q.options['column_a'] ?? []);
329         columnB = List<String>.from(q.options['column_b'] ?? []);
330     }
331 }

```

```

332     return pw.Row(
333       crossAxisAlignment: pw.CrossAxisAlignment.start,
334       children: [
335         pw.Expanded(
336           child: pw.Column(
337             crossAxisAlignment: pw.CrossAxisAlignment.start,
338             children: columnA.map((a) => pw.Padding(
339               padding: const pw.EdgeInsets.only(bottom: 8),
340               child: pw.Text('• $a', style: regular, textDirection: textDir),
341             )),
342           ),
343         ),
344         pw.SizedBox(width: 20),
345         pw.Expanded(
346           child: pw.Column(
347             crossAxisAlignment: pw.CrossAxisAlignment.start,
348             children: columnB.map((b) => pw.Padding(
349               padding: const pw.EdgeInsets.only(bottom: 8),
350               child: pw.Text('• $b', style: regular, textDirection: textDir),
351             )),
352           ),
353         ),
354       ],
355     );
356   } else if (q.type == 'essay') {
357     return pw.Container(
358       height: 60,
359       width: double.infinity,
360       decoration: pw.BoxDecoration(
361         border: pw.Border.all(color: PdfColors.grey300),
362         borderRadius: const pw.BorderRadius.all(pw.Radius.circular(8)),
363       ),
364     );
365   }
366   return pw.SizedBox();
367 }
368 }
369
370

```

Lines • Dart 124

Feature / Exam Generation

#162 – lib/features/exam_generation/domain/entities/exam_entities.dart

```

1  class ExamEntity {
2    final int id;
3    final int teacherId;
4    final String title;
5    final String status;
6    final int totalPoints;
7    final String createdAt;
8    final String updatedAt;
9    final List<QuestionEntity> questions;
10
11    ExamEntity({
12      required this.id,
13      required this.teacherId,
14      required this.title,
15      required this.status,
16      required this.totalPoints,
17      required this.createdAt,
18      required this.updatedAt,
19      required this.questions,
20    });
21  }
22
23  class QuestionEntity {
24    final int id;
25    final int lessonId;
26    final String type;
27    final String questionText;
28    final dynamic options; // Will be properly typed in models depending on question type
29    final String correctAnswer;
30    final String source;
31    final int usageCount;

```

```

32     final int questionOrder;
33     final int points;
34     final String reviewStatus;
35     final String? rejectionReason;
36
37     QuestionEntity({
38         required this.id,
39         required this.lessonId,
40         required this.type,
41         required this.questionText,
42         required this.options,
43         required this.correctAnswer,
44         required this.source,
45         required this.usageCount,
46         required this.questionOrder,
47         required this.points,
48         this.reviewStatus = '',
49         this.rejectionReason,
50     });
51 }
52
53 class RequestedCountsEntity {
54     final int mcq;
55     final int trueFalse;
56     final int fillBlank;
57     final int essay;
58     final int matching;
59
60     const RequestedCountsEntity({
61         required this.mcq,
62         required this.trueFalse,
63         required this.fillBlank,
64         required this.essay,
65         required this.matching,
66     });
67
68     bool get hasQuestions => mcq > 0 || trueFalse > 0 || fillBlank > 0 || essay > 0 || matching > 0;
69
70     int get total => mcq + trueFalse + fillBlank + essay + matching;
71 }
72
73 class ExamListEntity {
74     final int id;
75     final String title;
76     final String status;
77     final int questionsCount;
78     final int totalPoints;
79     final String createdAt;
80     final String updatedAt;
81
82     ExamListEntity({
83         required this.id,
84         required this.title,
85         required this.status,
86         required this.questionsCount,
87         required this.totalPoints,
88         required this.createdAt,
89         required this.updatedAt,
90     });
91 }
92
93 class ExamPaginationEntity {
94     final int currentPage;
95     final int lastPage;
96     final int perPage;
97     final int total;
98     final List<ExamListEntity> data;
99
100     ExamPaginationEntity({
101         required this.currentPage,
102         required this.lastPage,
103         required this.perPage,
104         required this.total,
105         required this.data,
106     });

```

```

107 }
108
109 class QuestionPaginationEntity {
110   final int currentPage;
111   final int lastPage;
112   final int perPage;
113   final int total;
114   final List<QuestionEntity> data;
115
116   QuestionPaginationEntity({
117     required this.currentPage,
118     required this.lastPage,
119     required this.perPage,
120     required this.total,
121     required this.data,
122   });
123 }
124

```

Lines • 44
Dart

Feature / Exam #163 -
Generation

lib/features/exam_generation/domain/repositories/exam_repository.dart

```

1 import 'package:dartz/dartz.dart';
2 import 'package:moean/core/errors/failures.dart';
3 import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
4
5 abstract class ExamRepository {
6   Future<Either<Failure, ExamEntity>> generateExam({
7     required String title,
8     required String grade,
9     required String subject,
10    required List<Map<String, dynamic>> lessons,
11  });
12
13   Future<Either<Failure, ExamEntity>> getExam(int id);
14
15   Future<Either<Failure, ExamEntity>> updateExamPoints(int id, List<Map<String, dynamic>> questions);
16
17   Future<Either<Failure, Map<String, dynamic>>> getExams({int page = 1, int perPage = 20, String? status});
18
19   Future<Either<Failure, ExamPaginationEntity>> getMyExams({
20     int page = 1,
21     int perPage = 20,
22     String? status,
23     String? search,
24   });
25
26   Future<Either<Failure, ExamEntity>> publishExam(int id);
27
28   Future<Either<Failure, bool>> deleteExam(int id);
29
30   Future<Either<Failure, ExamEntity>> addManualQuestion(Map<String, dynamic> request);
31
32   Future<Either<Failure, QuestionPaginationEntity>> getMyQuestions({
33     int page = 1,
34     int perPage = 20,
35     String? type,
36     String? difficulty,
37     String? reviewStatus,
38     int? lessonId,
39     String? search,
40   });
41
42   Future<Either<Failure, QuestionEntity>> updateCustomQuestion(int id, Map<String, dynamic> data);
43 }
44

```

Lines • 15
Dart

Feature / Exam #164 -
Generation

lib/features/exam_generation/domain/usecases/add_manual_question_usecase.dart

```

1 import 'package:dartz/dartz.dart';
2 import 'package:moean/core/errors/failures.dart';

```

```

3 import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
4 import 'package:moean/features/exam_generation/domain/repositories/exam_repository.dart';
5
6 class AddManualQuestionUseCase {
7   final ExamRepository repository;
8
9   AddManualQuestionUseCase(this.repository);
10
11   Future<Either<Failure, ExamEntity>> call(Map<String, dynamic> request) async {
12     return await repository.addManualQuestion(request);
13   }
14 }
15

```

Lines • 25
Dart

Feature / Exam #165 –
Generation

lib/features/exam_generation/domain/usecases/generate_exam_usecase.dart

```

1 import 'package:dartz/dartz.dart';
2 import 'package:moean/core/errors/failures.dart';
3 import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
4 import 'package:moean/features/exam_generation/domain/repositories/exam_repository.dart';
5
6 class GenerateExamUseCase {
7   final ExamRepository repository;
8
9   GenerateExamUseCase(this.repository);
10
11   Future<Either<Failure, ExamEntity>> execute({
12     required String title,
13     required String grade,
14     required String subject,
15     required List<Map<String, dynamic>> lessons,
16   }) {
17     return repository.generateExam(
18       title: title,
19       grade: grade,
20       subject: subject,
21       lessons: lessons,
22     );
23   }
24 }
25

```

Lines • Dart 15

Feature / Exam Generation

#166 – lib/features/exam_generation/domain/usecases/get_exam_usecase.dart

```

1 import 'package:dartz/dartz.dart';
2 import 'package:moean/core/errors/failures.dart';
3 import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
4 import 'package:moean/features/exam_generation/domain/repositories/exam_repository.dart';
5
6 class GetExamUseCase {
7   final ExamRepository repository;
8
9   GetExamUseCase(this.repository);
10
11   Future<Either<Failure, ExamEntity>> execute(int id) {
12     return repository.getExam(id);
13   }
14 }
15

```

Lines • 31
Dart

Feature / Exam #167 –
Generation

lib/features/exam_generation/domain/usecases/get_my_questions_usecase.dart

```

1 import 'package:dartz/dartz.dart';
2 import 'package:moean/core/errors/failures.dart';
3 import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
4 import 'package:moean/features/exam_generation/domain/repositories/exam_repository.dart';
5
6 class GetMyQuestionsUseCase {

```

```

7   final ExamRepository repository;
8
9   GetMyQuestionsUseCase(this.repository);
10
11  Future<Either<Failure, QuestionPaginationEntity>> execute({
12    int page = 1,
13    int perPage = 20,
14    String? type,
15    String? difficulty,
16    String? reviewStatus,
17    int? lessonId,
18    String? search,
19  }) {
20    return repository.getMyQuestions(
21      page: page,
22      perPage: perPage,
23      type: type,
24      difficulty: difficulty,
25      reviewStatus: reviewStatus,
26      lessonId: lessonId,
27      search: search,
28    );
29  }
30 }
31

```

Lines • 15
Dart

Feature / Exam #168 –

Generation [lib/features/exam_generation/domain/usecases/update_custom_question_usecase.dart](#)

```

1  import 'package:dartz/dartz.dart';
2  import 'package:moean/core/errors/failures.dart';
3  import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
4  import 'package:moean/features/exam_generation/domain/repositories/exam_repository.dart';
5
6  class UpdateCustomQuestionUseCase {
7    final ExamRepository repository;
8
9    UpdateCustomQuestionUseCase(this.repository);
10
11    Future<Either<Failure, QuestionEntity>> execute(int id, Map<String, dynamic> data) {
12      return repository.updateCustomQuestion(id, data);
13    }
14  }
15

```

Lines • 15
Dart

Feature / Exam #169 –

Generation [lib/features/exam_generation/domain/usecases/update_question_points_usecase.dart](#)

```

1  import 'package:dartz/dartz.dart';
2  import 'package:moean/core/errors/failures.dart';
3  import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
4  import 'package:moean/features/exam_generation/domain/repositories/exam_repository.dart';
5
6  class UpdateQuestionPointsUseCase {
7    final ExamRepository repository;
8
9    UpdateQuestionPointsUseCase(this.repository);
10
11    Future<Either<Failure, ExamEntity>> execute(int examId, List<Map<String, dynamic>> questions) {
12      return repository.updateExamPoints(examId, questions);
13    }
14  }
15

```

Lines • 158
Dart

Feature / Exam #170 –

Generation [lib/features/exam_generation/presentation/cubit/bank_questions_cubit.dart](#)

```

1  import 'package:flutter_bloc/flutter_bloc.dart';
2  import 'package:moean/core/network/remote/api_service.dart';
3

```



```

4 abstract class BankQuestionsState {}
5
6 class BankQuestionsInitial extends BankQuestionsState {}
7
8 class BankQuestionsLoading extends BankQuestionsState {}
9
10 class BankQuestionsError extends BankQuestionsState {
11     final String message;
12     BankQuestionsError(this.message);
13 }
14
15 class BankQuestionsUpdated extends BankQuestionsState {
16     final List<Map<String, dynamic>> questions;
17     final List<int> selectedQuestionIds;
18     final Map<int, List<int>> allSelectedQuestionIds;
19     final int currentPage;
20     final int lastPage;
21     final bool hasMore;
22     final String? selectedType;
23
24     BankQuestionsUpdated({
25         required this.questions,
26         required this.selectedQuestionIds,
27         required this.allSelectedQuestionIds,
28         required this.currentPage,
29         required this.lastPage,
30         required this.hasMore,
31         this.selectedType,
32     });
33 }
34
35 class BankQuestionsCubit extends Cubit<BankQuestionsState> {
36     BankQuestionsCubit() : super(BankQuestionsInitial());
37
38     List<Map<String, dynamic>> _allQuestions = [];
39     final Map<int, List<int>> _allSelectedQuestionIds = {};
40     int _currentLessonId = -1;
41     int _currentPage = 1;
42     int _lastPage = 1;
43     bool _isLoadingMore = false;
44     String? _selectedType;
45
46     void setFilter(String? type) {
47         if (_selectedType == type) return;
48         _selectedType = type;
49         if (_currentLessonId != -1) {
50             _currentPage = 1;
51             _allQuestions = [];
52             if (!isClosed) emit(BankQuestionsLoading());
53             loadQuestionsForLesson(_currentLessonId, refresh: true);
54         }
55     }
56
57     Future<void> loadQuestionsForLesson(int lessonId, {bool refresh = false}) async {
58         if (refresh || _currentLessonId != lessonId) {
59             _currentLessonId = lessonId;
60             _currentPage = 1;
61             _allQuestions = [];
62             // Do not clear _allSelectedQuestionIds to keep selection across lessons
63             if (!isClosed) emit(BankQuestionsLoading());
64         } else if (_currentPage >= _lastPage || _isLoadingMore) {
65             return;
66         } else {
67             _currentPage++;
68             _isLoadingMore = true;
69             // Emit state to show loading more if needed, but here we just update state silently
70         }
71
72         final result = await ApiService.getLessonQuestions(
73             lessonId: _currentLessonId,
74             type: _selectedType,
75             page: _currentPage,
76         );
77
78         if (isClosed) return;

```

```

79
80 result.fold(
81     (error) {
82         if (_currentPage == 1) {
83             if (!isClosed) emit(BankQuestionsError(error.toString()));
84         } else {
85             _currentPage--;
86             _isLoadingMore = false;
87         }
88     },
89     (data) {
90         final List<dynamic> questionsData = data['data'] ?? [];
91         final meta = data['meta'] ?? {};
92
93         _lastPage = meta['last_page'] ?? 1;
94
95         for (var q in questionsData) {
96             _allQuestions.add({
97                 'id': q['id'],
98                 'type': q['type'],
99                 'difficulty': q['difficulty'],
100                 'question_text': q['question_text'],
101                 'options': q['options'],
102                 'correct_answer': q['correct_answer'],
103             });
104         }
105
106         _isLoadingMore = false;
107         _emitState();
108     },
109 );
110 }
111
112 void toggleQuestion(int questionId) {
113     if (_currentLessonId == -1) return;
114     _allSelectedQuestionIds[_currentLessonId] ??= [];
115
116     if (_allSelectedQuestionIds[_currentLessonId]!.contains(questionId)) {
117         _allSelectedQuestionIds[_currentLessonId]!.remove(questionId);
118     } else {
119         _allSelectedQuestionIds[_currentLessonId]!.add(questionId);
120     }
121     _emitState();
122 }
123
124 void selectAll() {
125     if (_currentLessonId == -1) return;
126     _allSelectedQuestionIds[_currentLessonId] ??= [];
127
128     for (var q in _allQuestions) {
129         if (!_allSelectedQuestionIds[_currentLessonId]!.contains(q['id'])) {
130             _allSelectedQuestionIds[_currentLessonId]!.add(q['id']);
131         }
132     }
133     _emitState();
134 }
135
136 void clearSelection() {
137     if (_currentLessonId == -1) return;
138     _allSelectedQuestionIds[_currentLessonId]?.clear();
139     _emitState();
140 }
141
142 void _emitState() {
143     if (isClosed) return;
144     final selectedForCurrent = _allSelectedQuestionIds[_currentLessonId] ?? [];
145     emit(BankQuestionsUpdated(
146         questions: List.from(_allQuestions),
147         selectedQuestionIds: List.from(selectedForCurrent),
148         allSelectedQuestionIds: Map.from(_allSelectedQuestionIds),
149         currentPage: _currentPage,
150         lastPage: _lastPage,
151         hasMore: _currentPage < _lastPage,
152         selectedType: _selectedType,
153     ));

```

```

154     }
155
156     Map<int, List<int>> get allSelectedQuestionIds => Map.from(_allSelectedQuestionIds);
157 }
158

```

Lines • 140
Dart

Feature / Exam #171 -

Generation [lib/features/exam_generation/presentation/cubit/custom_questions_cubit.dart](#)

```

1  import 'package:flutter_bloc/flutter_bloc.dart';
2  import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
3  import 'package:moean/features/exam_generation/domain/usecases/get_my_questions_usecase.dart';
4  import 'package:moean/features/exam_generation/domain/usecases/update_custom_question_usecase.dart';
5  import 'package:moean/features/exam_generation/presentation/cubit/custom_questions_state.dart';
6
7  class CustomQuestionsCubit extends Cubit<CustomQuestionsState> {
8    final GetMyQuestionsUseCase _getMyQuestionsUseCase;
9    final UpdateCustomQuestionUseCase _updateCustomQuestionUseCase;
10
11    int _currentPage = 1;
12    bool _isLastPage = false;
13    String _selectedStatus = 'all';
14    String _searchQuery = '';
15    QuestionPaginationEntity? _currentPagination;
16
17    CustomQuestionsCubit(
18      this._getMyQuestionsUseCase,
19      this._updateCustomQuestionUseCase,
20    ) : super(CustomQuestionsInitial());
21
22    static CustomQuestionsCubit get(context) => BlocProvider.of(context);
23
24    Future<void> fetchQuestions({bool isRefresh = false}) async {
25      if (isRefresh) {
26        _currentPage = 1;
27        _isLastPage = false;
28        _currentPagination = null;
29        emit(CustomQuestionsLoading());
30      } else {
31        if (_isLastPage || state is CustomQuestionsLoading) return;
32        emit(CustomQuestionsLoading(isPagination: true));
33      }
34
35      String? statusFilter = _selectedStatus == 'all' ? null : _selectedStatus;
36      String? searchFilter = _searchQuery.isEmpty ? null : _searchQuery;
37
38      final result = await _getMyQuestionsUseCase.execute(
39        page: _currentPage,
40        reviewStatus: statusFilter,
41        search: searchFilter,
42      );
43
44      result.fold(
45        (failure) {
46          if (_currentPagination != null) {
47            emit(CustomQuestionsLoaded(
48              questions: _currentPagination!,
49              selectedStatus: _selectedStatus,
50              searchQuery: _searchQuery,
51            ));
52          } else {
53            emit(CustomQuestionsError(failure.message));
54          }
55        },
56        (pagination) {
57          _isLastPage = pagination.currentPage >= pagination.lastPage;
58
59          if (isRefresh || _currentPagination == null) {
60            _currentPagination = pagination;
61          } else {
62            final currentQuestions = List<QuestionEntity>.from(_currentPagination!.data);
63            currentQuestions.addAll(pagination.data);
64            _currentPagination = QuestionPaginationEntity(

```

```

65         currentPage: pagination.currentPage,
66         lastPage: pagination.lastPage,
67         perPage: pagination.perPage,
68         total: pagination.total,
69         data: currentQuestions,
70     );
71 }
72
73 if (!_isLastPage) {
74     _currentPage++;
75 }
76
77 emit(CustomQuestionsLoaded(
78     questions: _currentPagination!,
79     selectedStatus: _selectedStatus,
80     searchQuery: _searchQuery,
81 ));
82 },
83 );
84 }
85
86 void changeStatus(String status) {
87     _selectedStatus = status;
88     fetchQuestions(isRefresh: true);
89 }
90
91 void searchQuestions(String query) {
92     _searchQuery = query;
93     fetchQuestions(isRefresh: true);
94 }
95
96 Future<void> updateQuestion(int questionId, Map<String, dynamic> data) async {
97     emit(CustomQuestionActionLoading());
98
99     final result = await _updateCustomQuestionUseCase.execute(questionId, data);
100
101     result.fold(
102         (failure) {
103             emit(CustomQuestionActionError(failure.message));
104             if (_currentPagination != null) {
105                 emit(CustomQuestionsLoaded(
106                     questions: _currentPagination!,
107                     selectedStatus: _selectedStatus,
108                     searchQuery: _searchQuery,
109                 ));
110             }
111         },
112         (updatedQuestion) {
113             emit(CustomQuestionActionSuccess('تم تحديث السؤال بنجاح'));
114
115             if (_currentPagination != null) {
116                 final updatedList = _currentPagination!.data.map((q) {
117                     return q.id == questionId ? updatedQuestion : q;
118                 }).toList();
119
120                 _currentPagination = QuestionPaginationEntity(
121                     currentPage: _currentPagination!.currentPage,
122                     lastPage: _currentPagination!.lastPage,
123                     perPage: _currentPagination!.perPage,
124                     total: _currentPagination!.total,
125                     data: updatedList,
126                 );
127
128                 emit(CustomQuestionsLoaded(
129                     questions: _currentPagination!,
130                     selectedStatus: _selectedStatus,
131                     searchQuery: _searchQuery,
132                 ));
133             } else {
134                 fetchQuestions(isRefresh: true);
135             }
136         },
137     );
138 }

```

139 }
140

Lines • 55
Dart

Feature / Exam #172 -

Generation [lib/features/exam_generation/presentation/cubit/custom_questions_state.dart](#)

```
1 import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
2
3 abstract class CustomQuestionsState {}
4
5 class CustomQuestionsInitial extends CustomQuestionsState {}
6
7 class CustomQuestionsLoading extends CustomQuestionsState {
8   final bool isPagination;
9   CustomQuestionsLoading({this.isPagination = false});
10 }
11
12 class CustomQuestionsLoaded extends CustomQuestionsState {
13   final QuestionPaginationEntity questions;
14   final String selectedStatus;
15   final String searchQuery;
16
17   CustomQuestionsLoaded({
18     required this.questions,
19     this.selectedStatus = 'all',
20     this.searchQuery = '',
21   });
22
23   CustomQuestionsLoaded copyWith({
24     QuestionPaginationEntity? questions,
25     String? selectedStatus,
26     String? searchQuery,
27   }) {
28     return CustomQuestionsLoaded(
29       questions: questions ?? this.questions,
30       selectedStatus: selectedStatus ?? this.selectedStatus,
31       searchQuery: searchQuery ?? this.searchQuery,
32     );
33   }
34 }
35
36 class CustomQuestionsError extends CustomQuestionsState {
37   final String message;
38
39   CustomQuestionsError(this.message);
40 }
41
42 class CustomQuestionActionLoading extends CustomQuestionsState {}
43
44 class CustomQuestionActionSuccess extends CustomQuestionsState {
45   final String message;
46
47   CustomQuestionActionSuccess(this.message);
48 }
49
50 class CustomQuestionActionError extends CustomQuestionsState {
51   final String message;
52
53   CustomQuestionActionError(this.message);
54 }
55
```

Lines • 250
Dart

Feature / Exam #173 -

Generation [lib/features/exam_generation/presentation/cubit/exam_info_cubit.dart](#)

```
1 import 'package:flutter_bloc/flutter_bloc.dart';
2 import 'package:moean/core/network/remote/api_service.dart';
3 import 'package:moean/features/exam_generation/data/models/curriculum_models.dart';
4
5 enum GenerationSource {
6   aiOnly,
7   mixed,
```

```

8 }
9 abstract class ExamInfoState {}
10
11 class ExamInfoInitial extends ExamInfoState {}
12
13 class ExamInfoLoading extends ExamInfoState {}
14
15 class ExamInfoError extends ExamInfoState {
16     final String message;
17     ExamInfoError(this.message);
18 }
19
20 class ExamInfoUpdated extends ExamInfoState {
21     final CurriculumStageModel? selectedStage;
22     final String? selectedSemester;
23     final String? selectedTrack;
24     final CurriculumGradeModel? selectedGrade;
25     final CurriculumSubjectModel? selectedSubject;
26     final String? unitName;
27     final String? examTitle;
28     final List<CurriculumStageModel> stages;
29     final String difficulty;
30     final GenerationSource generationSource;
31     final Map<int, List<int>> selectedBankQuestionIds;
32     final List<String> availableUnits;
33     final bool isUnitsLoading;
34
35     ExamInfoUpdated({
36         required this.selectedStage,
37         required this.selectedSemester,
38         required this.selectedTrack,
39         required this.selectedGrade,
40         required this.selectedSubject,
41         this.unitName,
42         this.examTitle,
43         required this.stages,
44         required this.difficulty,
45         required this.generationSource,
46         required this.selectedBankQuestionIds,
47         required this.availableUnits,
48         required this.isUnitsLoading,
49     });
50 }
51
52 class ExamInfoCubit extends Cubit<ExamInfoState> {
53     ExamInfoCubit() : super(ExamInfoInitial()) {
54         _loadSubjects();
55     }
56
57     CurriculumStageModel? selectedStage;
58     String? selectedSemester;
59     String? selectedTrack;
60     CurriculumGradeModel? selectedGrade;
61     CurriculumSubjectModel? selectedSubject;
62     String? unitName;
63     String? examTitle;
64
65     String difficulty = 'medium';
66     GenerationSource generationSource = GenerationSource.aiOnly;
67     final Map<int, List<int>> selectedBankQuestionIds = {};
68
69     List<CurriculumStageModel> stages = [];
70     List<String> availableUnits = [];
71     bool isUnitsLoading = false;
72
73     List<String> get availableTracks {
74         if (selectedStage == null) return [];
75         return selectedStage!.grades
76             .map((g) => g.track)
77             .whereType<String>()
78             .where((t) => t.isNotEmpty)
79             .toSet()
80             .toList();
81     }
82 }

```

```

83 List<CurriculumGradeModel> get availableGrades {
84     if (selectedStage == null) return [];
85     if (availableTracks.isEmpty) {
86         if (selectedTrack == null) return [];
87         return selectedStage!.grades.where((g) => g.track == selectedTrack).toList();
88     }
89     return selectedStage!.grades;
90 }
91
92 Future<void> _loadSubjects() async {
93     if (!isClosed) emit(ExamInfoLoading());
94     final endpoint = '/subjects';
95
96     // --- Console Print ---
97     print('===== API CALL =====');
98     print('ENDPOINT: $endpoint');
99     print('METHOD: GET');
100    print('BODY: {}');
101    print('=====');
102    // -----
103
104    final result = await ApiService.getSubjects();
105    if (isClosed) return;
106    result.fold(
107        (dynamic failure) {
108            if (!isClosed) emit(ExamInfoError(failure?.message ?? 'Unknown error'));
109        },
110        (data) {
111            stages = data;
112            _emitUpdated();
113        },
114    );
115 }
116
117 void retry() {
118     _loadSubjects();
119 }
120
121 void updateInfo({
122     CurriculumStageModel? stage,
123     String? semester,
124     String? track,
125     CurriculumGradeModel? grade,
126     CurriculumSubjectModel? subject,
127     String? unitName,
128     String? examTitle,
129     String? difficulty,
130 }) {
131     if (stage != null) {
132         selectedStage = stage;
133         selectedSemester = null;
134         selectedTrack = null;
135         selectedGrade = null;
136         selectedSubject = null;
137         this.unitName = null;
138         this.examTitle = null;
139     }
140     if (semester != null) {
141         selectedSemester = semester;
142     }
143     if (track != null) {
144         selectedTrack = track;
145         selectedGrade = null;
146         selectedSubject = null;
147         this.unitName = null;
148         this.examTitle = null;
149     }
150     if (grade != null) {
151         selectedGrade = grade;
152         selectedSubject = null;
153         this.unitName = null;
154         this.examTitle = null;
155     }
156     if (subject != null) {
157         selectedSubject = subject;

```

```

158     this.unitName = null;
159     this.examTitle = null;
160     _loadUnitsForSubject(subject.id);
161 }
162 if (unitName != null) {
163     this.unitName = unitName;
164 }
165 if (examTitle != null) {
166     this.examTitle = examTitle;
167 }
168 if (difficulty != null) {
169     this.difficulty = difficulty;
170 }
171
172 _emitUpdated();
173 }
174
175 Future<void> _loadUnitsForSubject(int subjectId) async {
176     isUnitsLoading = true;
177     availableUnits = [];
178     final endpoint = '/subjects/$subjectId/lessons';
179
180     // --- Console Print ---
181     print('===== API CALL =====');
182     print('ENDPOINT: $endpoint');
183     print('METHOD: GET');
184     print('BODY: {}');
185     print('=====');
186     // -----
187
188     _emitUpdated();
189
190     final result = await ApiService.getSubjectLessons(subjectId);
191     if (isClosed) return;
192
193     isUnitsLoading = false;
194     result.fold(
195         (failure) {
196             _emitUpdated();
197         },
198         (data) {
199             final units = data.chapters
200                 .where((c) => selectedSemester == null || c.semester == selectedSemester)
201                 .map((c) => (c.unitName != null && c.unitName!.isNotEmpty) ? c.unitName! : c.title)
202                 .where((u) => u.isNotEmpty)
203                 .toSet()
204                 .toList();
205             availableUnits = units;
206             _emitUpdated();
207         }
208     );
209 }
210
211 void updateSelectedBankQuestions(Map<int, List<int>> questionIdsMap) {
212     selectedBankQuestionIds.clear();
213     selectedBankQuestionIds.addAll(questionIdsMap);
214     _emitUpdated();
215 }
216
217 void updateGenerationSource(GenerationSource source) {
218     generationSource = source;
219     _emitUpdated();
220 }
221
222 void _emitUpdated() {
223     if (!isClosed) {
224         emit(ExamInfoUpdated(
225             selectedStage: selectedStage,
226             selectedSemester: selectedSemester,
227             selectedTrack: selectedTrack,
228             selectedGrade: selectedGrade,
229             selectedSubject: selectedSubject,
230             unitName: unitName,
231             examTitle: examTitle,
232             stages: stages,

```



```

233     difficulty: difficulty,
234     generationSource: generationSource,
235     selectedBankQuestionIds: Map.from(selectedBankQuestionIds),
236     availableUnits: availableUnits,
237     isUnitsLoading: isUnitsLoading,
238   ));
239 }
240 }
241
242 bool get isValid =>
243   selectedStage != null &&
244   selectedSemester != null &&
245   (availableTracks.isEmpty || selectedTrack != null) &&
246   selectedGrade != null &&
247   selectedSubject != null &&
248   difficulty.isNotEmpty;
249 }
250

```

Lines • 215
Dart

Feature / Exam #174 -

Generation [lib/features/exam_generation/presentation/cubit/exam_preview_cubit.dart](#)

```

1  import 'package:flutter_bloc/flutter_bloc.dart';
2  import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
3  import 'package:moean/features/exam_generation/domain/usecases/get_exam_usecase.dart';
4  import 'package:moean/features/exam_generation/domain/usecases/update_question_points_usecase.dart';
5  import 'package:moean/features/exam_generation/domain/repositories/exam_repository.dart';
6  import 'package:moean/core/errors/failures.dart';
7
8  abstract class ExamPreviewState {}
9
10 class ExamPreviewInitial extends ExamPreviewState {}
11
12 class ExamPreviewLoading extends ExamPreviewState {}
13
14 class ExamPreviewLoaded extends ExamPreviewState {
15   final ExamEntity exam;
16   final bool isTeacherMode;
17   final bool showAnswers;
18
19   // Local changes for provisional calculation before saving
20   final Map<int, double> pendingPointsChanges;
21
22   ExamPreviewLoaded({
23     required this.exam,
24     this.isTeacherMode = true,
25     this.showAnswers = true,
26     this.pendingPointsChanges = const {},
27   });
28
29   double get provisionalTotalPoints {
30     double total = 0.0;
31     for (var q in exam.questions) {
32       if (pendingPointsChanges.containsKey(q.id)) {
33         total += pendingPointsChanges[q.id]!;
34       } else {
35         total += q.points;
36       }
37     }
38     return total;
39   }
40
41   ExamPreviewLoaded copyWith({
42     ExamEntity? exam,
43     bool? isTeacherMode,
44     bool? showAnswers,
45     Map<int, double>? pendingPointsChanges,
46   }) {
47     return ExamPreviewLoaded(
48       exam: exam ?? this.exam,
49       isTeacherMode: isTeacherMode ?? this.isTeacherMode,
50       showAnswers: showAnswers ?? this.showAnswers,
51       pendingPointsChanges: pendingPointsChanges ?? this.pendingPointsChanges,

```

```

52     );
53 }
54 }
55
56 class ExamPreviewError extends ExamPreviewState {
57     final String message;
58     ExamPreviewError(this.message);
59 }
60
61 class ExamPreviewPaymentRequired extends ExamPreviewState {
62     final String message;
63     final String code;
64     final ExamPreviewLoaded loadedState; // So we can revert safely
65     ExamPreviewPaymentRequired(this.message, this.code, this.loadedState);
66 }
67
68 class ExamPreviewSaving extends ExamPreviewLoaded {
69     ExamPreviewSaving(ExamPreviewLoaded state) : super(
70         exam: state.exam,
71         isTeacherMode: state.isTeacherMode,
72         showAnswers: state.showAnswers,
73         pendingPointsChanges: state.pendingPointsChanges,
74     );
75 }
76
77 class ExamPreviewCubit extends Cubit<ExamPreviewState> {
78     final GetExamUseCase getExamUseCase;
79     final UpdateQuestionPointsUseCase updateQuestionPointsUseCase;
80     final ExamRepository examRepository;
81
82     ExamPreviewCubit({
83         required this.getExamUseCase,
84         required this.updateQuestionPointsUseCase,
85         required this.examRepository,
86     }) : super(ExamPreviewInitial());
87
88     Future<void> loadExam(int id) async {
89         if (!isClosed) emit(ExamPreviewLoading());
90
91         final result = await getExamUseCase.execute(id);
92         if (isClosed) return;
93
94         result.fold(
95             (failure) {
96                 if (!isClosed) emit(ExamPreviewError(failure.message));
97             },
98             (exam) {
99                 if (!isClosed) emit(ExamPreviewLoaded(exam: exam));
100             },
101         );
102     }
103
104     void toggleMode(bool isTeacher) {
105         if (state is ExamPreviewLoaded) {
106             final current = state as ExamPreviewLoaded;
107             if (!isClosed) {
108                 emit(ExamPreviewLoaded(
109                     exam: current.exam,
110                     isTeacherMode: isTeacher,
111                     showAnswers: current.showAnswers,
112                     pendingPointsChanges: current.pendingPointsChanges,
113                 ));
114             }
115         }
116     }
117
118     void toggleAnswers(bool show) {
119         if (state is ExamPreviewLoaded) {
120             final current = state as ExamPreviewLoaded;
121             if (!isClosed) {
122                 emit(ExamPreviewLoaded(
123                     exam: current.exam,
124                     isTeacherMode: current.isTeacherMode,
125                     showAnswers: show,
126                     pendingPointsChanges: current.pendingPointsChanges,

```

```

127     ));
128     }
129 }
130 }
131
132 void updatePendingPoints(int questionId, double points) {
133     if (state is ExamPreviewLoaded) {
134         final currentState = state as ExamPreviewLoaded;
135         final newPending = Map<int, double>.from(currentState.pendingPointsChanges);
136         newPending[questionId] = points;
137
138         if (!isClosed) {
139             emit(currentState.copyWith(
140                 pendingPointsChanges: newPending,
141             ));
142         }
143     }
144 }
145
146 Future<void> addManualQuestion(Map<String, dynamic> request) async {
147     if (state is ExamPreviewLoaded) {
148         final loadedState = state as ExamPreviewLoaded;
149         if (!isClosed) emit(ExamPreviewSaving(loadedState));
150
151         final result = await examRepository.addManualQuestion(request);
152         if (isClosed) return;
153
154         result.fold(
155             (failure) {
156                 if (!isClosed) {
157                     if (failure is PaymentRequiredFailure) {
158                         emit(ExamPreviewPaymentRequired(failure.message, failure.code, loadedState));
159                     } else {
160                         emit(ExamPreviewError(failure.message));
161                     }
162                     // Restore previous state
163                     emit(loadedState);
164                 }
165             },
166             (updatedExam) {
167                 if (!isClosed) {
168                     emit(ExamPreviewLoaded(
169                         exam: updatedExam,
170                         isTeacherMode: loadedState.isTeacherMode,
171                         showAnswers: loadedState.showAnswers,
172                         pendingPointsChanges: const {},
173                     ));
174                 }
175             },
176         );
177     }
178 }
179
180 Future<void> savePoints() async {
181     if (state is! ExamPreviewLoaded) return;
182
183     final current = state as ExamPreviewLoaded;
184     if (current.pendingPointsChanges.isEmpty) return; // nothing to save
185
186     final examId = current.exam.id;
187
188     // Prepare the list of questions to update
189     final updates = current.pendingPointsChanges.entries.map((e) {
190         return {
191             'question_id': e.key,
192             'points': e.value,
193         };
194     }).toList();
195
196     if (!isClosed) emit(ExamPreviewSaving(current));
197
198     final result = await updateQuestionPointsUseCase.execute(examId, updates);
199     if (isClosed) return;
200
201     result.fold(

```

```

202     (failure) {
203         if (!isClosed) {
204             emit(ExamPreviewError(failure.message));
205             // Reload exam after failure to reset UI to actual backend state
206             loadExam(examId);
207         }
208     },
209     (exam) {
210         if (!isClosed) emit(ExamPreviewLoaded(exam: exam, pendingPointsChanges: const {})); // Clear pending
on success
211     },
212 );
213 }
214 }
215

```

Lines • 91
Dart

Feature / Exam #175 -

Generation lib/features/exam_generation/presentation/cubit/generate_exam_cubit.dart

```

1  import 'package:flutter_bloc/flutter_bloc.dart';
2  import 'package:moean/core/errors/failures.dart';
3  import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
4  import 'package:moean/features/exam_generation/domain/usecases/generate_exam_usecase.dart';
5
6  abstract class GenerateExamState {}
7
8  class GenerateExamInitial extends GenerateExamState {}
9
10 class GenerateExamLoading extends GenerateExamState {}
11
12 class GenerateExamSuccess extends GenerateExamState {
13     final ExamEntity exam;
14     GenerateExamSuccess(this.exam);
15 }
16
17 class GenerateExamError extends GenerateExamState {
18     final String message;
19     GenerateExamError(this.message);
20 }
21
22 class GenerateExamPaymentRequired extends GenerateExamState {
23     final String message;
24     final String code;
25     GenerateExamPaymentRequired(this.message, this.code);
26 }
27
28 class GenerateExamCubit extends Cubit<GenerateExamState> {
29     final GenerateExamUseCase generateExamUseCase;
30
31     GenerateExamCubit(this.generateExamUseCase) : super(GenerateExamInitial());
32
33     Future<void> generate({
34         String? title,
35         required String grade,
36         required String subject,
37         required List<Map<String, dynamic>> lessons,
38         required Map<int, RequestedCountsEntity> counts,
39         Map<int, List<int>>? selectedQuestionIds,
40     }) async {
41         if (!isClosed) emit(GenerateExamLoading());
42
43         // Map the selected lessons with their respective requested counts to match the API contract
44         final List<Map<String, dynamic>> requestedLessons = [];
45         for (final lesson in lessons) {
46             final id = lesson['id'] as int;
47             final requestedCounts = counts[id];
48             final selectedIds = selectedQuestionIds?[id] ?? [];
49
50             if (requestedCounts != null || selectedIds.isNotEmpty) {
51                 requestedLessons.add({
52                     'lesson_id': id,
53                     'lesson_name': lesson['name'],
54                     if (selectedIds.isNotEmpty) 'selected_question_ids': selectedIds,

```

```

55     'requested_counts': {
56       'mcq': requestedCounts?.mcq ?? 0,
57       'true_false': requestedCounts?.trueFalse ?? 0,
58       'fill_blank': requestedCounts?.fillBlank ?? 0,
59       'essay': requestedCounts?.essay ?? 0,
60       'matching': requestedCounts?.matching ?? 0,
61     }
62   });
63 }
64 }
65
66 final result = await generateExamUseCase.execute(
67   title: title ?? '', // Title will be auto-generated by the backend if empty
68   grade: grade,
69   subject: subject,
70   lessons: requestedLessons,
71 );
72
73 if (isClosed) return;
74
75 result.fold(
76   (failure) {
77     if (!isClosed) {
78       if (failure is PaymentRequiredFailure) {
79         emit(GenerateExamPaymentRequired(failure.message, failure.code));
80       } else {
81         emit(GenerateExamError(failure.message));
82       }
83     }
84   },
85   (exam) {
86     if (!isClosed) emit(GenerateExamSuccess(exam));
87   },
88 );
89 }
90 }
91

```

Lines • 160
Dart

Feature / Exam #176 -

Generation lib/features/exam_generation/presentation/cubit/lesson_selection_cubit.dart

```

1  import 'package:flutter_bloc/flutter_bloc.dart';
2  import 'package:moean/core/network/remote/api_service.dart';
3  import 'package:moean/features/exam_generation/data/models/curriculum_models.dart';
4
5  abstract class LessonSelectionState {}
6
7  class LessonSelectionInitial extends LessonSelectionState {}
8
9  class LessonSelectionLoading extends LessonSelectionState {}
10
11 class LessonSelectionError extends LessonSelectionState {
12   final String message;
13   LessonSelectionError(this.message);
14 }
15
16 class LessonSelectionUpdated extends LessonSelectionState {
17   final SubjectDetailsModel? subjectDetails;
18   final List<CurriculumChapterModel> filteredChapters;
19   final List<CurriculumLessonModel> selectedLessons;
20   final String searchQuery;
21   final List<String> availableSemesters;
22   final String? selectedSemester;
23
24   LessonSelectionUpdated({
25     required this.subjectDetails,
26     required this.filteredChapters,
27     required this.selectedLessons,
28     required this.searchQuery,
29     required this.availableSemesters,
30     required this.selectedSemester,
31   });
32 }

```

```

33
34 class LessonSelectionCubit extends Cubit<LessonSelectionState> {
35     LessonSelectionCubit() : super(LessonSelectionInitial());
36
37     SubjectDetailsModel? _subjectDetails;
38     final List<CurriculumLessonModel> _selectedLessons = [];
39     String _searchQuery = '';
40     int? _currentSubjectId;
41     String? _selectedSemester;
42     String? _selectedUnitName;
43
44     Future<void> loadLessonsForSubjectId(int subjectId, {String? unitName}) async {
45         _selectedUnitName = unitName;
46         if (_currentSubjectId == subjectId && _subjectDetails != null) {
47             _emitState();
48             return; // Already loaded
49         }
50
51         _currentSubjectId = subjectId;
52         if (!isClosed) emit(LessonSelectionLoading());
53
54         final result = await ApiService.getSubjectLessons(subjectId);
55         if (isClosed) return;
56
57         result.fold(
58             (dynamic failure) {
59                 if (!isClosed) emit(LessonSelectionError(failure?.message ?? 'Unknown error'));
60             },
61             (details) {
62                 _subjectDetails = details;
63
64                 // Remove selected lessons that are no longer in this subject
65                 final allLessonsIds = details.chapters.expand((c) => c.lessons).map((l) => l.id).toSet();
66                 _selectedLessons.removeWhere((selected) => !allLessonsIds.contains(selected.id));
67
68                 _emitState();
69             },
70         );
71     }
72
73     void selectSemester(String? semester) {
74         _selectedSemester = semester;
75         _emitState();
76     }
77
78     void retry() {
79         if (_currentSubjectId != null) {
80             loadLessonsForSubjectId(_currentSubjectId!, unitName: _selectedUnitName);
81         }
82     }
83
84     void search(String query) {
85         _searchQuery = query.toLowerCase();
86         _emitState();
87     }
88
89     String _normalizeArabic(String text) {
90         return text
91             .replaceAll(' ' , ' ')
92             .replaceAll(' ' , ' ')
93             .replaceAll(' ' , ' ')
94             .replaceAll(' ' , ' ');
95     }
96
97     void toggleLesson(CurriculumLessonModel lesson) {
98         if (_selectedLessons.any((l) => l.id == lesson.id)) {
99             _selectedLessons.removeWhere((l) => l.id == lesson.id);
100         } else {
101             _selectedLessons.add(lesson);
102         }
103         _emitState();
104     }
105
106     void removeLesson(int lessonId) {
107         _selectedLessons.removeWhere((l) => l.id == lessonId);

```

```

108     _emitState();
109 }
110
111 void _emitState() {
112     final normalizedQuery = _normalizeArabic(_searchQuery);
113
114     List<CurriculumChapterModel> filtered = [];
115     if (_subjectDetails != null) {
116         for (var chapter in _subjectDetails!.chapters) {
117             if (_selectedSemester != null && chapter.semester != _selectedSemester) {
118                 continue;
119             }
120
121             if (_selectedUnitName != null && _selectedUnitName!.isNotEmpty) {
122                 final chapterUnitName = (chapter.unitName != null && chapter.unitName!.isNotEmpty) ? chapter.unitName! : chapter.title;
123                 if (chapterUnitName != _selectedUnitName) {
124                     continue;
125                 }
126             }
127
128             final matchingLessons = chapter.lessons.where((l) {
129                 final name = l.title.toLowerCase();
130                 final normalizedName = _normalizeArabic(name);
131                 return normalizedName.contains(normalizedQuery);
132             }).toList();
133
134             if (matchingLessons.isNotEmpty) {
135                 filtered.add(CurriculumChapterModel(
136                     chapterId: chapter.chapterId,
137                     title: chapter.title,
138                     semester: chapter.semester,
139                     unitId: chapter.unitId,
140                     unitName: chapter.unitName,
141                     lessons: matchingLessons,
142                 ));
143             }
144         }
145     }
146
147     emit(LessonSelectionUpdated(
148         subjectDetails: _subjectDetails,
149         filteredChapters: filtered,
150         selectedLessons: List.from(_selectedLessons),
151         searchQuery: _searchQuery,
152         availableSemesters: _subjectDetails?.chapters.map((c) => c.semester).toSet().toList() ?? [],
153         selectedSemester: _selectedSemester,
154     ));
155 }
156
157 List<CurriculumLessonModel> get selectedLessons => _selectedLessons;
158 bool get hasSelection => _selectedLessons.isNotEmpty;
159 }
160

```

Lines • 147
Dart

Feature / Exam #177 -

Generation lib/features/exam_generation/presentation/cubit/manual_exam_cubit.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/features/exam_generation/domain/usecases/add_manual_question_usecase.dart';
4 import 'manual_exam_state.dart';
5 import 'package:moean/core/errors/failures.dart';
6
7 class ManualExamCubit extends Cubit<ManualExamState> {
8     final AddManualQuestionUseCase addManualQuestionUseCase;
9
10     String selectedQuestionType = 'mcq';
11     int? selectedLessonId;
12
13     final TextEditingController gradeController = TextEditingController(text: '1');
14     final TextEditingController questionTextController = TextEditingController();
15     final TextEditingController correctAnswerController = TextEditingController();

```

```

16
17 List<TextEditingController> optionsControllers = [
18   TextEditingController(), TextEditingController(), TextEditingController(), TextEditingController()
19 ];
20
21 List<TextEditingController> columnAControllers = [TextEditingController(), TextEditingController()];
22 List<TextEditingController> columnBControllers = [TextEditingController(), TextEditingController()];
23
24 ManualExamCubit(this.addManualQuestionUseCase) : super(ManualExamInitial());
25
26 static ManualExamCubit get(context) => BlocProvider.of(context);
27
28 void changeQuestionType(String type) {
29   selectedQuestionType = type;
30   emit(ManualExamFormUpdated());
31 }
32
33 void updateGrade(String newGrade) {}
34
35 void updateLesson(int lessonId) {
36   selectedLessonId = lessonId;
37   emit(ManualExamFormUpdated());
38 }
39
40 void addOption() {
41   optionsControllers.add(TextEditingController());
42   emit(ManualExamFormUpdated());
43 }
44
45 void removeOption(int index) {
46   if (optionsControllers.length > 2) {
47     optionsControllers[index].dispose();
48     optionsControllers.removeAt(index);
49     emit(ManualExamFormUpdated());
50   }
51 }
52
53 void addMatchingItem() {
54   columnAControllers.add(TextEditingController());
55   columnBControllers.add(TextEditingController());
56   emit(ManualExamFormUpdated());
57 }
58
59 void removeMatchingItem(int index) {
60   if (columnAControllers.length > 2) {
61     columnAControllers[index].dispose();
62     columnBControllers[index].dispose();
63     columnAControllers.removeAt(index);
64     columnBControllers.removeAt(index);
65     emit(ManualExamFormUpdated());
66   }
67 }
68
69 Future<void> submitQuestion(int examId) async {
70   final questionText = questionTextController.text;
71   if (selectedLessonId == null || questionText.trim().isEmpty) {
72     emit(ManualExamError('Please fill all required fields'));
73     return;
74   }
75
76   if (!isClosed) emit(ManualExamLoading());
77
78   final request = <String, dynamic>{
79     'exam_id': examId,
80     'lesson_id': selectedLessonId,
81     'type': selectedQuestionType,
82     'question_text': questionText.trim(),
83     'points': double.tryParse(gradeController.text) ?? 1.0,
84   };
85
86   final correctAnswer = correctAnswerController.text.trim();
87
88   if (selectedQuestionType == 'mcq') {
89     final validOptions = optionsControllers.map((c) => c.text.trim()).where((o) => o.isNotEmpty).toList();
90     if (validOptions.length < 2) {

```



```

91     if (!isClosed) emit(ManualExamError('MCQ requires at least 2 options'));
92     return;
93   }
94   if (!validOptions.contains(correctAnswer)) {
95     if (!isClosed) emit(ManualExamError('Correct answer must match one of the options'));
96     return;
97   }
98   request['options'] = validOptions;
99   request['correct_answer'] = correctAnswer;
100 } else if (selectedQuestionType == 'matching') {
101   final validColA = columnAControllers.map((c) => c.text.trim()).where((o) => o.isNotEmpty).toList();
102   final validColB = columnBControllers.map((c) => c.text.trim()).where((o) => o.isNotEmpty).toList();
103   if (validColA.isEmpty || validColB.isEmpty) {
104     if (!isClosed) emit(ManualExamError('Matching columns cannot be empty'));
105     return;
106   }
107   request['options'] = {
108     'column_a': validColA,
109     'column_b': validColB,
110   };
111   request['correct_answer'] = correctAnswer;
112 } else {
113   // True/False, Essay, Fill Blank
114   request['correct_answer'] = correctAnswer;
115 }
116
117 final result = await addManualQuestionUseCase(request);
118 if (isClosed) return;
119
120 result.fold(
121   (failure) {
122     if (!isClosed) {
123       if (failure is PaymentRequiredFailure) {
124         emit(ManualExamPaymentRequired(failure.message, failure.code));
125       } else {
126         emit(ManualExamError(failure.message));
127       }
128     }
129   },
130   (exam) {
131     if (!isClosed) emit(ManualExamSuccess('Question added successfully'));
132   },
133 );
134 }
135
136 @override
137 Future<void> close() {
138   gradeController.dispose();
139   questionTextController.dispose();
140   correctAnswerController.dispose();
141   for (var c in optionsControllers) { c.dispose(); }
142   for (var c in columnAControllers) { c.dispose(); }
143   for (var c in columnBControllers) { c.dispose(); }
144   return super.close();
145 }
146 }
147

```

Lines • 25
Dart

Feature / Exam #178 -

Generation lib/features/exam_generation/presentation/cubit/manual_exam_state.dart

```

1  abstract class ManualExamState {}
2
3  class ManualExamInitial extends ManualExamState {}
4
5  class ManualExamLoading extends ManualExamState {}
6
7  class ManualExamSuccess extends ManualExamState {
8    final String message;
9    ManualExamSuccess(this.message);
10 }
11
12 class ManualExamError extends ManualExamState {

```

```

13   final String error;
14   ManualExamError(this.error);
15 }
16
17 class ManualExamPaymentRequired extends ManualExamState {
18   final String message;
19   final String code;
20   ManualExamPaymentRequired(this.message, this.code);
21 }
22
23 // Emitted when form validates successfully and updates
24 class ManualExamFormUpdated extends ManualExamState {}
25

```

Lines • 140
Dart

Feature / Exam #179 -

Generation lib/features/exam_generation/presentation/cubit/my_exams_cubit.dart

```

1  import 'dart:async';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/features/exam_generation/domain/repositories/exam_repository.dart';
4  import 'package:moean/features/exam_generation/presentation/cubit/my_exams_state.dart';
5
6  class MyExamsCubit extends Cubit<MyExamsState> {
7    final ExamRepository _examRepository;
8    Timer? _debounce;
9
10   // Internal state tracking
11   String _currentTab = 'all'; // 'all', 'draft', 'published'
12   String _searchQuery = '';
13   int _currentPage = 1;
14
15   MyExamsCubit(this._examRepository) : super(MyExamsInitial());
16
17   static MyExamsCubit get(context) => BlocProvider.of(context);
18
19   void fetchExams({bool isRefresh = false}) async {
20     if (isRefresh) {
21       _currentPage = 1;
22     }
23
24     if (_currentPage == 1) {
25       if (!isClosed) emit(MyExamsLoading());
26     }
27
28     String? statusQuery;
29     if (_currentTab == 'draft') {
30       statusQuery = 'draft';
31     } else if (_currentTab == 'published') {
32       statusQuery = 'published';
33     }
34
35     final result = await _examRepository.getMyExams(
36       page: _currentPage,
37       perPage: 20,
38       status: statusQuery,
39       search: _searchQuery.isNotEmpty ? _searchQuery : null,
40     );
41
42     if (isClosed) return;
43
44     result.fold(
45       (failure) {
46         if (!isClosed) emit(MyExamsError(failure.message));
47       },
48       (examsPagination) {
49         if (!isClosed) {
50           emit(MyExamsLoaded(
51             exams: examsPagination,
52             selectedTab: _currentTab,
53             searchQuery: _searchQuery,
54           ));
55         }
56       },
57     );
58

```

```

57     );
58 }
59
60 void changeTab(String tab) {
61     if (_currentTab == tab) return;
62     _currentTab = tab;
63     _currentPage = 1;
64     fetchExams();
65 }
66
67 void searchExams(String query) {
68     if (_searchQuery == query) return;
69
70     if (_debounce?.isActive ?? false) _debounce?.cancel();
71
72     _debounce = Timer(const Duration(milliseconds: 500), () {
73         if (isClosed) return;
74         _searchQuery = query;
75         _currentPage = 1;
76         fetchExams();
77     });
78 }
79
80 Future<void> publishExam(int examId) async {
81     final currentState = state;
82     if (currentState is! MyExamsLoaded) return;
83
84     if (!isClosed) emit(MyExamsActionLoading(action: 'publish', examId: examId));
85
86     final result = await _examRepository.publishExam(examId);
87
88     if (isClosed) return;
89
90     result.fold(
91         (failure) {
92             if (!isClosed) {
93                 emit(MyExamsActionError(failure.message));
94                 // Re-emit loaded state so UI recovers
95                 emit(currentState);
96             }
97         },
98         (exam) {
99             if (!isClosed) {
100                 emit(MyExamsActionSuccess('تم نشر الاختبار بنجاح'));
101                 fetchExams(isRefresh: true);
102             }
103         },
104     );
105 }
106
107 Future<void> deleteExam(int examId) async {
108     final currentState = state;
109     if (currentState is! MyExamsLoaded) return;
110
111     if (!isClosed) emit(MyExamsActionLoading(action: 'delete', examId: examId));
112
113     final result = await _examRepository.deleteExam(examId);
114
115     if (isClosed) return;
116
117     result.fold(
118         (failure) {
119             if (!isClosed) {
120                 emit(MyExamsActionError(failure.message));
121                 // Re-emit loaded state so UI recovers
122                 emit(currentState);
123             }
124         },
125         (_) {
126             if (!isClosed) {
127                 emit(MyExamsActionSuccess('تم حذف الاختبار بنجاح'));
128                 fetchExams(isRefresh: true);
129             }
130         },
131     );

```

```

132     }
133
134     @override
135     Future<void> close() {
136       _debounce?.cancel();
137       return super.close();
138     }
139   }
140

```

Lines • Dart 57 Feature / Exam Generation #180 – lib/features/exam_generation/presentation/cubit/my_exams_state.dart

```

1  import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
2
3  abstract class MyExamsState {}
4
5  class MyExamsInitial extends MyExamsState {}
6
7  class MyExamsLoading extends MyExamsState {}
8
9  class MyExamsLoaded extends MyExamsState {
10     final ExamPagingEntity exams;
11     final String selectedTab; // 'all', 'draft', 'published'
12     final String searchQuery;
13
14     MyExamsLoaded({
15       required this.exams,
16       this.selectedTab = 'all',
17       this.searchQuery = '',
18     });
19
20     MyExamsLoaded copyWith({
21       ExamPagingEntity? exams,
22       String? selectedTab,
23       String? searchQuery,
24     }) {
25       return MyExamsLoaded(
26         exams: exams ?? this.exams,
27         selectedTab: selectedTab ?? this.selectedTab,
28         searchQuery: searchQuery ?? this.searchQuery,
29       );
30     }
31   }
32
33   class MyExamsError extends MyExamsState {
34     final String message;
35
36     MyExamsError(this.message);
37   }
38
39   class MyExamsActionLoading extends MyExamsState {
40     final String action; // 'publish' or 'delete'
41     final int examId;
42
43     MyExamsActionLoading({required this.action, required this.examId});
44   }
45
46   class MyExamsActionSuccess extends MyExamsState {
47     final String message;
48
49     MyExamsActionSuccess(this.message);
50   }
51
52   class MyExamsActionError extends MyExamsState {
53     final String message;
54
55     MyExamsActionError(this.message);
56   }
57

```

```
1 import 'package:flutter_bloc/flutter_bloc.dart';
2 import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
3 import 'package:moean/features/exam_generation/data/models/curriculum_models.dart';
4
5 abstract class QuestionCountState {}
6
7 class QuestionCountInitial extends QuestionCountState {}
8
9 class QuestionCountUpdated extends QuestionCountState {
10   final Map<int, RequestedCountsEntity> counts;
11   final int totalQuestions;
12   final int totalLessons;
13
14   QuestionCountUpdated({
15     required this.counts,
16     required this.totalQuestions,
17     required this.totalLessons,
18   });
19 }
20
21 class QuestionCountCubit extends Cubit<QuestionCountState> {
22   QuestionCountCubit() : super(QuestionCountInitial());
23
24   // Store counts efficiently
25   final Map<int, RequestedCountsEntity> _counts = {};
26
27   void initForLessons(List<CurriculumLessonModel> selectedLessons) {
28     // Retain old counts for lessons still selected, initialize new ones to 0
29     final newCounts = <int, RequestedCountsEntity>{};
30     for (final lesson in selectedLessons) {
31       final int id = lesson.id;
32       if (_counts.containsKey(id)) {
33         newCounts[id] = _counts[id]!;
34       } else {
35         newCounts[id] = const RequestedCountsEntity(mcq: 0, trueFalse: 0, fillBlank: 0, essay: 0, matching:
0);
36       }
37     }
38     _counts.clear();
39     _counts.addAll(newCounts);
40     _emitState();
41   }
42
43   void updateCount(int lessonId, String type, int delta) {
44     if (!_counts.containsKey(lessonId)) return;
45     final current = _counts[lessonId]!;
46
47     int newMcq = current.mcq;
48     int newTrueFalse = current.trueFalse;
49     int newFillBlank = current.fillBlank;
50     int newEssay = current.essay;
51     int newMatching = current.matching;
52
53     switch (type) {
54       case 'mcq':
55         newMcq = (newMcq + delta).clamp(0, 100);
56         break;
57       case 'true_false':
58         newTrueFalse = (newTrueFalse + delta).clamp(0, 100);
59         break;
60       case 'fill_blank':
61         newFillBlank = (newFillBlank + delta).clamp(0, 100);
62         break;
63       case 'essay':
64         newEssay = (newEssay + delta).clamp(0, 100);
65         break;
66       case 'matching':
67         newMatching = (newMatching + delta).clamp(0, 100);
68         break;
69     }
70   }
```

```

71     _counts[lessonId] = RequestedCountsEntity(
72       mcq: newMcq,
73       trueFalse: newTrueFalse,
74       fillBlank: newFillBlank,
75       essay: newEssay,
76       matching: newMatching,
77     );
78
79     _emitState();
80   }
81
82   void _emitState() {
83     int total = 0;
84     for (final count in _counts.values) {
85       total += count.total;
86     }
87     emit(QuestionCountUpdated(
88       counts: Map.from(_counts),
89       totalQuestions: total,
90       totalLessons: _counts.length,
91     ));
92   }
93
94   Map<int, RequestedCountsEntity> get counts => _counts;
95
96   int get totalRequestedQuestions {
97     int total = 0;
98     for (final count in _counts.values) {
99       total += count.total;
100     }
101     return total;
102   }
103 }
104

```

Lines • 399
Dart

Feature / Exam #182 -

Generation lib/features/exam_generation/presentation/screens/bank_questions_screen.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7  import 'package:moean/core/utils/extensions/context_extension.dart';
8  import 'package:moean/features/exam_generation/presentation/cubit/bank_questions_cubit.dart';
9  import 'package:moean/features/exam_generation/presentation/cubit/exam_info_cubit.dart';
10 import 'package:moean/features/exam_generation/presentation/cubit/lesson_selection_cubit.dart';
11
12 class BankQuestionsScreen extends StatefulWidget {
13   const BankQuestionsScreen({super.key});
14
15   @override
16   State<BankQuestionsScreen> createState() => _BankQuestionsScreenState();
17 }
18
19 class _BankQuestionsScreenState extends State<BankQuestionsScreen> {
20   int? _selectedLessonId;
21
22   String _getQuestionTypeArabic(String type) {
23     switch (type.toLowerCase()) {
24       case 'mcq': return 'اختيار من متعدد';
25       case 'true_false': return 'صح أو خطأ';
26       case 'fill_blank': return 'أكمل الفراغ';
27       case 'essay': return 'سؤال مقالي';
28       case 'matching': return 'مطابقة';
29       default: return type;
30     }
31   }
32
33   String _getDifficultyArabic(String difficulty) {
34     switch (difficulty.toLowerCase()) {
35       case 'easy': return 'سهل';
36

```

```

36     case 'medium': return 'متوسط';
37     case 'hard': return 'صعب';
38     default: return difficulty.isEmpty ? 'متوسط' : difficulty;
39   }
40 }
41
42 @override
43 void initState() {
44   super.initState();
45   final selectedLessons = context.read<LessonSelectionCubit>().selectedLessons;
46   if (selectedLessons.isNotEmpty) {
47     _selectedLessonId = selectedLessons.first.id;
48     context.read<BankQuestionsCubit>().loadQuestionsForLesson(_selectedLessonId!);
49   }
50 }
51
52 void _onLessonSelected(int lessonId) {
53   if (_selectedLessonId != lessonId) {
54     setState(() {
55       _selectedLessonId = lessonId;
56     });
57     context.read<BankQuestionsCubit>().loadQuestionsForLesson(lessonId);
58   }
59 }
60
61 void _onContinue() {
62   final selectedIdsMap = context.read<BankQuestionsCubit>().allSelectedQuestionIds;
63   context.read<ExamInfoCubit>().updateSelectedBankQuestions(selectedIdsMap);
64   context.pop(); // Return to QuestionCountsScreen
65 }
66
67 @override
68 Widget build(BuildContext context) {
69   final selectedLessons = context.read<LessonSelectionCubit>().selectedLessons;
70
71   return Scaffold(
72     backgroundColor: ColorsManager.background,
73     appBar: AppBar(
74       backgroundColor: ColorsManager.background,
75       elevation: 0,
76       centerTitle: true,
77       title: Text(
78         'بنك الأسئلة',
79         style: TextStyleManager.bold18.copyWith(color: ColorsManager.mainText),
80       ),
81       leading: IconButton(
82         icon: Icon(Icons.arrow_back_ios, color: ColorsManager.mainText),
83         onPressed: () => context.pop(),
84       ),
85     ),
86     body: Column(
87       children: [
88         // Toggle Switch
89         Padding(
90           padding: const EdgeInsets.only(top: 16, left: 20, right: 20, bottom: 8),
91           child: Container(
92             padding: const EdgeInsets.all(4),
93             decoration: BoxDecoration(
94               color: ColorsManager.surfacePrimary,
95               borderRadius: BorderRadius.circular(12),
96               border: Border.all(color: ColorsManager.borderLightGray),
97             ),
98             child: Row(
99               children: [
100                 Expanded(
101                   child: Container(
102                     padding: const EdgeInsets.symmetric(vertical: 12),
103                     decoration: BoxDecoration(
104                       color: ColorsManager.primaryColor,
105                       borderRadius: BorderRadius.circular(8),
106                     ),
107                     alignment: Alignment.center,
108                     child: Text(
109                       'من بنك الأسئلة',
110                       style: TextStyleManager.bold14.copyWith(color: Colors.white),

```

```

111         ),
112     ),
113 ),
114 Expanded(
115     child: GestureDetector(
116         onTap: () => context.pop(),
117         child: Container(
118             padding: const EdgeInsets.symmetric(vertical: 12),
119             decoration: BoxDecoration(
120                 color: ColorsManager.surfacePrimary,
121                 borderRadius: BorderRadius.circular(8),
122             ),
123             alignment: Alignment.center,
124             child: Text(
125                 'إنشاء تلقائي',
126                 style: TextStylesManager.bold14.copyWith(color: ColorsManager.secondaryText),
127             ),
128         ),
129     ),
130 ),
131 ],
132 ),
133 ),
134 ),
135
136 // Lesson Selector
137 if (selectedLessons.isNotEmpty)
138     Container(
139         height: 60,
140         padding: const EdgeInsets.symmetric(vertical: 8),
141         child: ListView.separated(
142             padding: const EdgeInsets.symmetric(horizontal: 20),
143             scrollDirection: Axis.horizontal,
144             itemCount: selectedLessons.length,
145             separatorBuilder: (context, index) => horizontalSpace8,
146             itemBuilder: (context, index) {
147                 final lesson = selectedLessons[index];
148                 final lessonId = lesson.id;
149                 final isSelected = _selectedLessonId == lessonId;
150
151                 return ChoiceChip(
152                     label: Text(
153                         lesson.title,
154                         style: isSelected
155                             ? TextStylesManager.bold12.copyWith(color: Colors.white)
156                             : TextStylesManager.regular12.copyWith(color: ColorsManager.mainText),
157                     ),
158                     selected: isSelected,
159                     selectedColor: ColorsManager.primaryColor,
160                     backgroundColor: ColorsManager.surfacePrimary,
161                     shape: RoundedRectangleBorder(
162                         borderRadius: BorderRadius.circular(20),
163                         side: BorderSide(
164                             color: isSelected ? ColorsManager.primaryColor : ColorsManager.borderLightGray,
165                         ),
166                     ),
167                     onSelected: (_) => _onLessonSelected(lessonId),
168                 );
169             },
170         ),
171     ),
172
173 // Type Filter
174 BlocBuilder<BankQuestionsCubit, BankQuestionsState>(
175     buildWhen: (previous, current) {
176         if (previous is BankQuestionsUpdated && current is BankQuestionsUpdated) {
177             return previous.selectedType != current.selectedType;
178         }
179         return current is BankQuestionsUpdated || previous is BankQuestionsInitial;
180     },
181     builder: (context, state) {
182         String? currentType;
183         if (state is BankQuestionsUpdated) {
184             currentType = state.selectedType;
185         }
186     },
187 )

```



```

186
187     final filters = [
188         {'label': 'الكل', 'value': null},
189         {'label': 'اختياري', 'value': 'mcq'},
190         {'label': 'صح وخطأ', 'value': 'true_false'},
191         {'label': 'أكمل', 'value': 'fill_blank'},
192         {'label': 'مقالتي', 'value': 'essay'},
193         {'label': 'مطابقة', 'value': 'matching'},
194     ];
195
196     return Container(
197         height: 40,
198         margin: const EdgeInsets.only(bottom: 12),
199         child: ListView.separated(
200             padding: const EdgeInsets.symmetric(horizontal: 20),
201             scrollDirection: Axis.horizontal,
202             itemCount: filters.length,
203             separatorBuilder: (context, index) => horizontalSpace8,
204             itemBuilder: (context, index) {
205                 final filter = filters[index];
206                 final isSelected = currentType == filter['value'];
207
208                 return ChoiceChip(
209                     label: Text(
210                         filter['label'] as String,
211                         style: isSelected
212                             ? TextStylesManager.bold12.copyWith(color: Colors.white)
213                             : TextStylesManager.regular12.copyWith(color: ColorsManager.mainText),
214                     ),
215                     selected: isSelected,
216                     showCheckmark: false,
217                     selectedColor: ColorsManager.primaryColor,
218                     backgroundColor: ColorsManager.surfacePrimary,
219                     shape: RoundedRectangleBorder(
220                         borderRadius: BorderRadius.circular(8),
221                         side: BorderSide(
222                             color: isSelected ? ColorsManager.primaryColor : ColorsManager.borderLightGray,
223                         ),
224                     ),
225                     onSelect: (context, filter) => context.read<BankQuestionsCubit>().setFilter(filter['value'] as String?),
226                 );
227             },
228         ),
229     );
230 },
231 ),
232
233 Expanded(
234     child: BlocBuilder<BankQuestionsCubit, BankQuestionsState>(
235         builder: (context, state) {
236             if (state is BankQuestionsLoading) {
237                 return Center(child: CircularProgressIndicator(color: ColorsManager.primaryColor));
238             }
239
240             if (state is BankQuestionsError) {
241                 return Center(
242                     child: Column(
243                         mainAxisAlignment: MainAxisAlignment.center,
244                         children: [
245                             Text(state.message, style: TextStylesManager.bold14.copyWith(color: ColorsManager.errorColor)),
246                             verticalSpace16,
247                             PrimaryElevatedButton(
248                                 text: 'إعادة المحاولة',
249                                 onPressed: () {
250                                     if (_selectedLessonId != null) {
251                                         context.read<BankQuestionsCubit>().loadQuestionsForLesson(_selectedLessonId!, refresh: true);
252                                     }
253                                 },
254                             ),
255                         ],
256                     ),
257                 );

```



```

328         color: ColorsManager.primaryColor.withValues(alpha: 0.1),
329         borderRadius: BorderRadius.circular(8),
330     ),
331     child: Text(
332         _getDifficultyArabic(question['difficulty'] ?? ''),
333         style: TextStylesManager.regular12.copyWith(color: ColorsManage
r.primaryColor),
334     ),
335 ),
336 ],
337 ),
338 verticalSpace8,
339 Text(
340     question['question_text'] ?? '',
341     style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainTex
t),
342 ),
343 ],
344 ),
345 ),
346 ],
347 ),
348 ),
349 ),
350 );
351 },
352 );
353 }
354
355     return const SizedBox.shrink();
356 },
357 ),
358 ),
359
360 Container(
361     padding: const EdgeInsets.all(20),
362     decoration: BoxDecoration(
363         color: ColorsManager.surfacePrimary,
364         boxShadow: [
365             BoxShadow(color: Colors.black.withValues(alpha: 0.05), blurRadius: 10, offset: const Offset
(0, -5)),
366         ],
367     ),
368     child: SafeArea(
369         child: BlocBuilder<BankQuestionsCubit, BankQuestionsState>(
370             builder: (context, state) {
371                 int selectedCount = 0;
372                 if (state is BankQuestionsUpdated) {
373                     selectedCount = state.allSelectedQuestionIds.values.fold(0, (sum, list) => sum + list.len
gth);
374                 }
375                 return Column(
376                     children: [
377                         if (selectedCount > 0) ...[
378                             Text(
379                                 'سؤال $selectedCount تم تحديد',
380                                 style: TextStylesManager.bold14.copyWith(color: ColorsManager.primaryColor),
381                             ),
382                             verticalSpace12,
383                         ],
384                         PrimaryElevatedButton(
385                             text: 'تأكيد والعودة',
386                             onPressed: _onContinue,
387                         ),
388                     ],
389                 );
390             },
391         ),
392     ),
393 ),
394 ],
395 ),
396 );
397 }

```

```
398 }
399
```

Lines • 491
Dart

Feature / Exam #183 -

Generation lib/features/exam_generation/presentation/screens/custom_questions_screen.dart

```
1 import 'dart:convert';
2 import 'package:flutter/material.dart';
3 import 'package:flutter_bloc/flutter_bloc.dart';
4 import 'package:moean/core/theme/colors.dart';
5 import 'package:moean/core/theme/text_styles.dart';
6 import 'package:moean/core/utils/constants/constants.dart';
7 import 'package:moean/core/utils/constants/primary/conditional_builder.dart';
8 import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
9 import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
10 import 'package:moean/core/utils/constants/spacing.dart';
11 import 'package:moean/core/utils/extensions/context_extension.dart';
12 import 'package:moean/features/exam_generation/presentation/cubit/custom_questions_cubit.dart';
13 import 'package:moean/features/exam_generation/presentation/cubit/custom_questions_state.dart';
14 import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
15
16 class CustomQuestionsScreen extends StatefulWidget {
17   const CustomQuestionsScreen({super.key});
18
19   @override
20   State<CustomQuestionsScreen> createState() => _CustomQuestionsScreenState();
21 }
22
23 class _CustomQuestionsScreenState extends State<CustomQuestionsScreen> {
24   final ScrollController _scrollController = ScrollController();
25   final TextEditingController _searchController = TextEditingController();
26
27   @override
28   void initState() {
29     super.initState();
30     context.read<CustomQuestionsCubit>().fetchQuestions(isRefresh: true);
31     _scrollController.addListener(_onScroll);
32   }
33
34   void _onScroll() {
35     if (_scrollController.position.pixels >= _scrollController.position.maxScrollExtent - 200) {
36       context.read<CustomQuestionsCubit>().fetchQuestions();
37     }
38   }
39
40   @override
41   void dispose() {
42     _scrollController.dispose();
43     _searchController.dispose();
44     super.dispose();
45   }
46
47   @override
48   Widget build(BuildContext context) {
49     return Scaffold(
50       backgroundColor: ColorsManager.background,
51       appBar: AppBar(
52         backgroundColor: ColorsManager.background,
53         elevation: 0,
54         centerTitle: true,
55         title: Text(
56           appTranslation().get('my_custom_questions'),
57           style: TextStylesManager.bold18,
58         ),
59         leading: IconButton(
60           icon: Icon(Icons.arrow_back_ios, color: ColorsManager.mainText),
61           onPressed: () => context.pop(),
62         ),
63       ),
64       body: Directionality(
65         textDirection: TextDirection.rtl,
66         child: BlocConsumer<CustomQuestionsCubit, CustomQuestionsState>(
67           listener: (context, state) {
```

```

68         if (state is CustomQuestionActionSuccess) {
69             ScaffoldMessenger.of(context).showSnackBar(
70                 SnackBar(content: Text(state.message), backgroundColor: Colors.green),
71             );
72         } else if (state is CustomQuestionActionError) {
73             ScaffoldMessenger.of(context).showSnackBar(
74                 SnackBar(content: Text(state.message), backgroundColor: Colors.red),
75             );
76         } else if (state is CustomQuestionsError) {
77             ScaffoldMessenger.of(context).showSnackBar(
78                 SnackBar(content: Text(state.message), backgroundColor: Colors.red),
79             );
80         }
81     },
82     builder: (context, state) {
83         bool isLoading = state is CustomQuestionsLoading && !state.isPagination;
84         bool isPaginationLoading = state is CustomQuestionsLoading && state.isPagination;
85         QuestionPaginationEntity? paginationEntity;
86         String selectedStatus = 'all';
87
88         if (state is CustomQuestionsLoaded) {
89             paginationEntity = state.questions;
90             selectedStatus = state.selectedStatus;
91         }
92
93         return Padding(
94             padding: const EdgeInsets.all(16.0),
95             child: Column(
96                 children: [
97                     _buildHeaderFilters(context, selectedStatus),
98                     verticalSpace16,
99                     Expanded(
100                         child: ConditionalBuilder(
101                             loadingState: isLoading,
102                             emptyState: paginationEntity?.data.isEmpty ?? true,
103                             emptyBuilder: (context) => Center(
104                                 child: Text(
105                                     appTranslation().get('no_questions_found'),
106                                     style: TextStyleManager.bold16.copyWith(color: ColorsManager.placeholder),
107                                 ),
108                             ),
109                             successBuilder: (context) => ListView.separated(
110                                 controller: _scrollController,
111                                 itemCount: (paginationEntity?.data.length ?? 0) + (isPaginationLoading ? 1 : 0),
112                                 separatorBuilder: (_, __) => verticalSpace16,
113                                 itemBuilder: (context, index) {
114                                     if (index == paginationEntity?.data.length) {
115                                         return const Center(child: CircularProgressIndicator());
116                                     }
117                                     final question = paginationEntity!.data[index];
118                                     return _buildQuestionItem(context, question);
119                                 },
120                             ),
121                         ),
122                     ],
123                 ),
124             );
125     },
126 ),
127 ),
128 ),
129 );
130 }
131
132 Widget _buildHeaderFilters(BuildContext context, String selectedStatus) {
133     return Column(
134         children: [
135             PrimaryTextField(
136                 controller: _searchController,
137                 hint: appTranslation().get('search_questions'),
138                 prefixIcon: const Icon(Icons.search, color: ColorsManager.mutedDark),
139                 onChanged: (value) {
140                     context.read<CustomQuestionsCubit>().searchQuestions(value);
141                 },
142             ),

```

```

143     verticalSpace12,
144     SingleChildScrollView(
145       scrollDirection: Axis.horizontal,
146       child: Row(
147         children: [
148           _buildFilterChip(context, 'all', appTranslation().get('all'), selectedStatus),
149           horizontalSpace8,
150           _buildFilterChip(context, 'pending', appTranslation().get('pending'), selectedStatus),
151           horizontalSpace8,
152           _buildFilterChip(context, 'approved', appTranslation().get('approved'), selectedStatus),
153           horizontalSpace8,
154           _buildFilterChip(context, 'rejected', appTranslation().get('rejected'), selectedStatus),
155         ],
156       ),
157     ),
158   ],
159 );
160 }
161
162 Widget _buildFilterChip(BuildContext context, String statusKey, String label, String selectedStatus) {
163   final isSelected = selectedStatus == statusKey;
164
165   Color statusColor;
166   switch (statusKey) {
167     case 'approved':
168       statusColor = Colors.green;
169       break;
170     case 'rejected':
171       statusColor = Colors.red;
172       break;
173     case 'pending':
174       statusColor = Colors.orange;
175       break;
176     case 'all':
177     default:
178       statusColor = ColorsManager.primaryColor;
179       break;
180   }
181
182   return GestureDetector(
183     onTap: () {
184       context.read<CustomQuestionsCubit>().changeStatus(statusKey);
185     },
186     child: Container(
187       padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
188       decoration: BoxDecoration(
189         color: isSelected ? ColorsManager.background : statusColor.withValues(alpha: 0.1),
190         borderRadius: BorderRadius.circular(20),
191         border: Border.all(
192           color: isSelected ? statusColor : Colors.transparent,
193         ),
194       ),
195       child: Text(
196         label,
197         style: TextStylesManager.bold14.copyWith(
198           color: statusColor,
199         ),
200       ),
201     ),
202   );
203 }
204
205 Widget _buildQuestionItem(BuildContext context, QuestionEntity question) {
206   final isDark = Theme.of(context).brightness == Brightness.dark;
207
208   Color statusColor;
209   String statusText;
210
211   switch (question.reviewStatus) {
212     case 'approved':
213       statusColor = Colors.green;
214       statusText = appTranslation().get('approved');
215       break;
216     case 'rejected':
217       statusColor = Colors.red;

```

```

218         statusText = appTranslation().get('rejected');
219         break;
220     case 'pending':
221     default:
222         statusColor = Colors.orange;
223         statusText = appTranslation().get('pending');
224         break;
225     }
226
227     List<String> parsedOptions = [];
228     if (question.options is List) {
229         parsedOptions = (question.options as List).map((e) => e.toString()).toList();
230     } else if (question.options is String) {
231         try {
232             final decoded = jsonDecode(question.options as String);
233             if (decoded is List) {
234                 parsedOptions = decoded.map((e) => e.toString()).toList();
235             }
236         } catch (e) {
237             // Ignored
238         }
239     }
240
241     String typeLabel = _getQuestionTypeLabel(question.type);
242
243     return Container(
244         padding: const EdgeInsets.all(16),
245         decoration: BoxDecoration(
246             color: isDark ? const Color(0xFF2C2C2E) : ColorsManager.white,
247             borderRadius: BorderRadius.circular(16),
248             border: Border.all(color: ColorsManager.borderLight),
249         ),
250         child: Column(
251             crossAxisAlignment: CrossAxisAlignment.start,
252             children: [
253                 Row(
254                     mainAxisAlignment: MainAxisAlignment.spaceBetween,
255                     children: [
256                         Row(
257                             children: [
258                                 Container(
259                                     padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
260                                     decoration: BoxDecoration(
261                                         color: const Color(0xFFE8F5E9),
262                                         borderRadius: BorderRadius.circular(20),
263                                     ),
264                                     child: Text(
265                                         typeLabel,
266                                         style: TextStylesManager.bold12.copyWith(color: const Color(0xFF2E7D32)),
267                                     ),
268                                 ),
269                             ],
270                         ),
271                         if (question.reviewStatus != 'approved')
272                             GestureDetector(
273                                 onTap: () => _showEditQuestionSheet(context, question),
274                                 child: Container(
275                                     padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
276                                     decoration: BoxDecoration(
277                                         color: ColorsManager.primaryColor,
278                                         borderRadius: BorderRadius.circular(20),
279                                     ),
280                                     child: Row(
281                                         children: [
282                                             Text(
283                                                 appTranslation().get('edit_question') ?? 'تعديل السؤال',
284                                                 style: TextStylesManager.bold12.copyWith(color: ColorsManager.white),
285                                             ),
286                                             horizontalSpace4,
287                                             const Icon(Icons.edit_outlined, color: ColorsManager.white, size: 16),
288                                         ],
289                                     ),
290                                 ),
291                             ),
292                     ],
293                 ),
294             ],
295         ),
296     );

```

```

293         const SizedBox(),
294     ],
295 ),
296 verticalSpace16,
297 Row(
298   mainAxisAlignment: MainAxisAlignment.spaceBetween,
299   crossAxisAlignment: CrossAxisAlignment.start,
300   children: [
301     Expanded(
302       child: Text(
303         question.questionText,
304         style: TextStylesManager.bold16,
305         textAlign: TextAlign.right,
306       ),
307     ),
308     horizontalSpace12,
309     Row(
310       mainAxisAlignment: MainAxisAlignment.min,
311       children: [
312         Container(
313           padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
314           decoration: BoxDecoration(
315             color: statusColor.withValues(alpha: 0.1),
316             borderRadius: BorderRadius.circular(20),
317             border: Border.all(color: statusColor.withValues(alpha: 0.3)),
318           ),
319           child: Row(
320             children: [
321               if (question.reviewStatus == 'approved') ...[
322                 Icon(Icons.check_circle_outline, color: statusColor, size: 16),
323                 horizontalSpace4,
324               ],
325               Text(
326                 statusText,
327                 style: TextStylesManager.bold12.copyWith(color: statusColor),
328               ),
329             ],
330           ),
331         ),
332         horizontalSpace12,
333         Text(
334           'استخدم' '${question.usageCount} مرة',
335           style: TextStylesManager.bold12.copyWith(color: ColorsManager.mutedDark),
336         ),
337       ],
338     ),
339   ],
340 ),
341 if (question.reviewStatus == 'rejected' && question.rejectionReason != null && question.rejectionReason!.isNotEmpty) ...[
342   verticalSpace12,
343   Container(
344     padding: const EdgeInsets.all(12),
345     decoration: BoxDecoration(
346       color: Colors.red.withValues(alpha: 0.1),
347       borderRadius: BorderRadius.circular(8),
348       border: Border.all(color: Colors.red.withValues(alpha: 0.3)),
349     ),
350     child: Row(
351       children: [
352         const Icon(Icons.error_outline, color: Colors.red, size: 20),
353         horizontalSpace8,
354         Expanded(
355           child: Text(
356             '${appTranslation().get('rejection_reason')}: ${question.rejectionReason}',
357             style: TextStylesManager.medium14.copyWith(color: Colors.red),
358           ),
359         ),
360       ],
361     ),
362   ),
363 ],
364 if (parsedOptions.isNotEmpty) ...[
365   verticalSpace16,
366   ...parsedOptions.asMap().entries.map((entry) {

```



```

367         int index = entry.key;
368         String optionText = entry.value;
369         String prefix = _getOptionPrefix(index);
370
371         return Container(
372             margin: const EdgeInsets.only(bottom: 8),
373             padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
374             width: double.infinity,
375             decoration: BoxDecoration(
376                 color: isDark ? const Color(0xFF3A3A3C) : ColorsManager.white,
377                 borderRadius: BorderRadius.circular(12),
378                 border: Border.all(color: ColorsManager.borderLight),
379             ),
380             child: Text(
381                 '$prefix) $optionText',
382                 style: TextStylesManager.bold14,
383                 textAlign: TextAlign.right,
384             ),
385         );
386     })),
387 ],
388 if (question.correctAnswer.isNotEmpty) ...[
389     verticalSpace16,
390     Container(
391         width: double.infinity,
392         padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
393         decoration: BoxDecoration(
394             color: const Color(0xFFFFF8E1),
395             borderRadius: BorderRadius.circular(12),
396         ),
397         child: Text(
398             '${appTranslation().get('correct_answer')} ?? "الإجابة الصحيحة": ${question.correctAnswer}',
399             style: TextStylesManager.bold14.copyWith(color: const Color(0xFFFF57F17)),
400             textAlign: TextAlign.right,
401         ),
402     ),
403 ],
404 verticalSpace16,
405 Text(
406     'يظهر الآن ضمن الأسئلة الجاهزة للمعلمين.',
407     style: TextStylesManager.medium12.copyWith(color: ColorsManager.mutedDark),
408     textAlign: TextAlign.right,
409 ),
410 ],
411 ),
412 );
413 }
414
415 String _getQuestionTypeLabel(String type) {
416     switch (type) {
417         case 'mcq': return 'اختيار من متعدد';
418         case 'true_false': return 'صح وخطأ';
419         case 'essay': return 'مقالتي';
420         case 'fill_blank': return 'أكمل الفراغ';
421         case 'matching': return 'مزاوجة';
422         default: return type;
423     }
424 }
425
426 String _getOptionPrefix(int index) {
427     const prefixes = ['أ', 'ب', 'ج', 'د', 'هـ', 'و'];
428     if (index < prefixes.length) {
429         return prefixes[index];
430     }
431     return '${index + 1}';
432 }
433
434 void _showEditQuestionSheet(BuildContext context, QuestionEntity question) {
435     final TextEditingController textController = TextEditingController(text: question.questionText);
436     final TextEditingController correctAnswerController = TextEditingController(text: question.correctAnswer);
437
438     showModalBottomSheet(
439         context: context,
440         isScrollControlled: true,

```

```

441     shape: const RoundedRectangleBorder(
442       borderRadius: BorderRadius.vertical(top: Radius.circular(20)),
443     ),
444     builder: (ctx) {
445       return Padding(
446         padding: EdgeInsets.only(
447           bottom: MediaQuery.of(ctx).viewInsets.bottom,
448           left: 24,
449           right: 24,
450           top: 24,
451         ),
452         child: Column(
453           mainAxisAlignment: MainAxisAlignment.min,
454           crossAxisAlignment: CrossAxisAlignment.start,
455           children: [
456             Text(
457               appTranslation().get('edit_question'),
458               style: TextStylesManager.bold18,
459             ),
460             verticalSpace16,
461             PrimaryTextField(
462               controller: textController,
463               hint: appTranslation().get('question_text'),
464               maxLines: 3,
465             ),
466             verticalSpace16,
467             PrimaryTextField(
468               controller: correctAnswerController,
469               hint: appTranslation().get('correct_answer'),
470             ),
471             verticalSpace24,
472             PrimaryElevatedButton(
473               text: appTranslation().get('save'),
474               onPressed: () {
475                 final data = {
476                   'question_text': textController.text.trim(),
477                   'correct_answer': correctAnswerController.text.trim(),
478                 };
479                 context.read<CustomQuestionsCubit>().updateQuestion(question.id, data);
480                 Navigator.pop(ctx);
481               },
482             ),
483             verticalSpace24,
484           ],
485         ),
486       );
487     },
488   );
489 }
490 }
491

```

Lines • 462
Dart

Feature / Exam #184 -
Generation

lib/features/exam_generation/presentation/screens/exam_info_screen.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
6  import 'package:moean/core/utils/constants/routes.dart';
7  import 'package:moean/core/utils/constants/spacing.dart';
8  import 'package:moean/core/utils/extensions/context_extension.dart';
9  import 'package:moean/features/exam_generation/presentation/cubit/exam_info_cubit.dart';
10 import 'package:moean/features/reports/presentation/widgets/selection_bottom_sheet.dart';
11 import 'package:moean/features/exam_generation/data/models/curriculum_models.dart';
12
13 class ExamInfoScreen extends StatefulWidget {
14   const ExamInfoScreen({super.key});
15
16   @override
17   State<ExamInfoScreen> createState() => _ExamInfoScreenState();
18 }

```

```

19
20 class _ExamInfoScreenState extends State<ExamInfoScreen> {
21   late final TextEditingController _examTitleController;
22
23   @override
24   void initState() {
25     super.initState();
26     final cubit = context.read<ExamInfoCubit>();
27     _examTitleController = TextEditingController(text: cubit.examTitle);
28   }
29
30   @override
31   void dispose() {
32     _examTitleController.dispose();
33     super.dispose();
34   }
35   void _showSelector(String title, List<String> items, Function(String) onSelected) {
36     SelectionBottomSheet.show(
37       context: context,
38       title: title,
39       items: items,
40       initialSelectedItems: [],
41       isMultiSelect: false,
42       onSelectionConfirmed: (selected) {
43         if (selected.isNotEmpty) {
44           onSelected(selected.first);
45         }
46       },
47     );
48   }
49
50   Widget _buildBody(BuildContext context, ExamInfoState state, ExamInfoCubit cubit) {
51     if (state is ExamInfoLoading) {
52       return SafeArea(
53         child: Stack(
54           children: [
55             Align(
56               alignment: AlignmentDirectional.topStart,
57               child: Padding(
58                 padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 10),
59                 child: IconButton(
60                   padding: EdgeInsets.zero,
61                   constraints: const BoxConstraints(minWidth: 40, minHeight: 40),
62                   icon: Icon(Icons.arrow_back_ios, color: ColorsManager.mainText),
63                   onPressed: () => context.pop(),
64                 ),
65             ),
66           ],
67           const Center(child: CircularProgressIndicator()),
68         ],
69       ),
70     );
71   }
72
73   if (state is ExamInfoError) {
74     return SafeArea(
75       child: Stack(
76         children: [
77           Align(
78             alignment: AlignmentDirectional.topStart,
79             child: Padding(
80               padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 10),
81               child: IconButton(
82                 padding: EdgeInsets.zero,
83                 constraints: const BoxConstraints(minWidth: 40, minHeight: 40),
84                 icon: Icon(Icons.arrow_back_ios, color: ColorsManager.mainText),
85                 onPressed: () => context.pop(),
86               ),
87             ),
88           ),
89           Center(
90             child: Column(
91               mainAxisAlignment: MainAxisAlignment.center,
92               children: [
93                 Text(state.message, style: TextStylesManager.bold14.copyWith(color: ColorsManager.errorColor)

```

```

r)),
94         verticalSpace16,
95         PrimaryElevatedButton(
96             text: 'إعادة المحاولة',
97             onPressed: () => cubit.retry(),
98         )
99     ],
100 ),
101 ),
102 ],
103 ),
104 );
105 }
106
107 return Column(
108     children: [
109         Expanded(
110             child: SafeArea(
111                 bottom: false,
112                 child: SingleChildScrollView(
113                     padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 10),
114                     child: Column(
115                         crossAxisAlignment: CrossAxisAlignment.stretch,
116                         children: [
117                             // Header UI
118                             Stack(
119                                 alignment: Alignment.center,
120                                 children: [
121                                     Align(
122                                         alignment: AlignmentDirectional.centerStart,
123                                         child: IconButton(
124                                             padding: EdgeInsets.zero,
125                                             constraints: const BoxConstraints(minWidth: 40, minHeight: 40),
126                                             icon: Icon(Icons.arrow_back_ios, color: ColorsManager.mainText),
127                                             onPressed: () => context.pop(),
128                                         ),
129                                     Text(
130                                         'الخطوة الأولى',
131                                         style: TextStyleManager.bold14.copyWith(color: ColorsManager.primaryColor),
132                                     ),
133                                 ],
134                             ),
135                             verticalSpace8,
136                             Text(
137                                 'اختبار بيانات الاختبار',
138                                 textAlign: TextAlign.center,
139                                 style: TextStyleManager.bold24.copyWith(color: ColorsManager.mainText),
140                             ),
141                             verticalSpace8,
142                             Center(
143                                 child: Container(
144                                     padding: const EdgeInsets.all(12),
145                                     decoration: BoxDecoration(
146                                         shape: BoxShape.circle,
147                                         border: Border.all(color: ColorsManager.primaryColor, width: 2),
148                                     ),
149                                 ),
150                                 child: Icon(Icons.tune, color: ColorsManager.primaryColor),
151                             ),
152                             verticalSpace8,
153                             Text(
154                                 'ثم اختار الفصل والدروس من الشاشة التالية\nاختار المرحلة والصف والمادة',
155                                 textAlign: TextAlign.center,
156                                 style: TextStyleManager.regular14.copyWith(color: ColorsManager.secondaryText),
157                             ),
158                             verticalSpace24,
159
160                             _buildChipsSelection<CurriculumStageModel>(
161                                 label: 'اختر المرحلة الدراسية',
162                                 items: List.of(cubit.stages)..sort((a, b) {
163                                     int getWeight(String name) {
164                                         if (name.contains('الابتدائية')) return 1;
165                                         if (name.contains('المتوسطة')) return 2;
166                                         if (name.contains('الثانوية')) return 3;

```

```

168         return 4;
169     }
170     return getWeight(a.name).compareTo(getWeight(b.name));
171 },
172     selectedValue: cubit.selectedStage,
173     onSelected: (v) => cubit.updateInfo(stage: v),
174     itemLabelBuilder: (item) => item.name,
175 ),
176
177 if (cubit.selectedStage != null) ...[
178     verticalSpace24,
179     _buildChipsSelection<String>(
180         label: 'اختر الفصل الدراسي',
181         items: ['1', '2'],
182         selectedValue: cubit.selectedSemester,
183         onSelected: (v) => cubit.updateInfo(semester: v),
184         itemLabelBuilder: (item) => 'الفصل الدراسي $item',
185     ),
186 ],
187
188 if (cubit.availableTracks.isNotEmpty) ...[
189     verticalSpace24,
190     _buildChipsSelection<String>(
191         label: 'المسار',
192         items: cubit.availableTracks,
193         selectedValue: cubit.selectedTrack,
194         onSelected: (v) => cubit.updateInfo(track: v),
195         itemLabelBuilder: (item) => item,
196     ),
197 ],
198
199 if (cubit.selectedStage != null && (cubit.availableTracks.isEmpty || cubit.selectedTrack !=
null)) ...[
200     verticalSpace24,
201     _buildChipsSelection<CurriculumGradeModel>(
202         label: 'المف الدراسي',
203         items: cubit.availableGrades,
204         selectedValue: cubit.selectedGrade,
205         onSelected: (v) => cubit.updateInfo(grade: v),
206         itemLabelBuilder: (item) => item.name,
207     ),
208 ],
209
210 if (cubit.selectedGrade != null) ...[
211     verticalSpace24,
212     _buildDropdownField(
213         label: 'اختر المادة',
214         value: cubit.selectedSubject?.name ?? '',
215         onTap: () {
216             final subjects = cubit.selectedGrade!.subjects;
217             _showSelector(
218                 'اختر المادة',
219                 subjects.map((e) => e.name).toSet().toList(),
220                 (v) {
221                     final selectedSub = subjects.firstWhere((e) => e.name == v);
222                     cubit.updateInfo(subject: selectedSub);
223                 }
224             );
225         },
226     ),
227 ],
228
229 if (cubit.selectedSubject != null) ...[
230     verticalSpace24,
231     cubit.isUnitsLoading
232       ? const Center(child: CircularProgressIndicator())
233       : _buildDropdownField(
234         label: 'الوحدة',
235         value: cubit.unitName ?? '',
236         onTap: () {
237             if (cubit.availableUnits.isEmpty) {
238                 ScaffoldMessenger.of(context).showSnackBar(
239                     const SnackBar(content: Text('لا توجد وحدات متاحة لهذه المادة')),
240                 );
241                 return;

```

```

242         }
243         _showSelector(
244             'اختر الوحدة',
245             cubit.availableUnits,
246             (v) => cubit.updateInfo(unitName: v),
247         );
248     },
249 ),
250 verticalSpace24,
251 _buildTextField(
252     label: 'عنوان الاختبار',
253     hint: 'مثال: اختبار الوحدة الثانية',
254     controller: _examTitleController,
255     onChanged: (v) => cubit.updateInfo(examTitle: v),
256 ),
257 ],
258
259 verticalSpace24,
260 _buildChipsSelection<String>(
261     label: 'مستوى الاختبار',
262     items: ['easy', 'medium', 'hard'],
263     selectedValue: cubit.difficulty,
264     onSelected: (v) => cubit.updateInfo(difficulty: v),
265     itemLabelBuilder: (item) {
266         switch (item) {
267             case 'easy': return 'سهل';
268             case 'medium': return 'متوسط';
269             case 'hard': return 'صعب';
270             default: return item;
271         }
272     },
273 ),
274
275 ],
276 ),
277 ),
278 ),
279 ),
280 Container(
281     decoration: BoxDecoration(
282         color: ColorsManager.surfacePrimary,
283         boxShadow: [
284             BoxShadow(color: Colors.black.withValues(alpha: 0.05), blurRadius: 10, offset: const Offset(0,
-5)),
285         ],
286     ),
287     child: SafeArea(
288         top: false,
289         left: false,
290         right: false,
291         minimum: const EdgeInsets.all(20),
292         child: PrimaryElevatedButton(
293             text: 'متابعة لاختبار الدروس',
294             onPressed: cubit.isValid ? () {
295                 context.push(Routes.examGenerationLessons);
296             } : null,
297         ),
298     ),
299 ),
300 ],
301 );
302 }
303
304 @override
305 Widget build(BuildContext context) {
306     return Scaffold(
307         backgroundColor: ColorsManager.background,
308         body: BlocConsumer<ExamInfoCubit, ExamInfoState>(
309             listener: (context, state) {
310                 if (state is ExamInfoUpdated) {
311                     if (state.examTitle == null && _examTitleController.text.isNotEmpty) {
312                         _examTitleController.clear();
313                     }
314                 }
315             },

```

```

316     builder: (context, state) {
317         final cubit = context.read<ExamInfoCubit>();
318         return _buildBody(context, state, cubit);
319     },
320 ),
321 );
322 }
323
324 Widget _buildDropdownField({required String label, required String value, required VoidCallback onTap}) {
325     return Column(
326         crossAxisAlignment: CrossAxisAlignment.center,
327         children: [
328             Row(
329                 children: [
330                     Text(label, style: TextStylesManager.bold16.copyWith(color: ColorsManager.mainText)),
331                 ],
332             ),
333             verticalSpace12,
334             InkWell(
335                 onTap: onTap,
336                 borderRadius: BorderRadius.circular(12),
337                 child: Container(
338                     padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
339                     decoration: BoxDecoration(
340                         border: Border.all(color: ColorsManager.borderLightGray),
341                         borderRadius: BorderRadius.circular(12),
342                         color: ColorsManager.surfacePrimary,
343                     ),
344                     child: Row(
345                         mainAxisAlignment: MainAxisAlignment.spaceBetween,
346                         children: [
347                             Expanded(
348                                 child: Text(
349                                     value.isEmpty ? 'اختر...' : value,
350                                     style: value.isEmpty
351                                         ? TextStylesManager.regular14.copyWith(color: ColorsManager.secondaryText)
352                                         : TextStylesManager.bold14.copyWith(color: ColorsManager.mainText),
353                                     maxLines: 1,
354                                     overflow: TextOverflow.ellipsis,
355                                     textAlign: TextAlign.start,
356                                 ),
357                             ),
358                             Icon(Icons.keyboard_arrow_down, color: ColorsManager.mainText),
359                         ],
360                     ),
361                 ),
362             ),
363         ],
364     );
365 }
366
367 Widget _buildTextField({
368     required String label,
369     required String hint,
370     required TextEditingController controller,
371     required ValueChanged<String> onChanged,
372     bool showDropdownIcon = false,
373 }) {
374     return Column(
375         crossAxisAlignment: CrossAxisAlignment.center,
376         children: [
377             Row(
378                 children: [
379                     Text(label, style: TextStylesManager.bold16.copyWith(color: ColorsManager.mainText)),
380                 ],
381             ),
382             verticalSpace12,
383             TextFormField(
384                 controller: controller,
385                 onChanged: onChanged,
386                 textAlign: TextAlign.start,
387                 decoration: InputDecoration(
388                     hintText: hint,
389                     hintStyle: TextStylesManager.regular14.copyWith(color: ColorsManager.secondaryText),
390                     filled: true,

```

```

391         fillColor: ColorsManager.surfacePrimary,
392         contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
393         suffixIcon: showDropdownIcon ? Icon(Icons.keyboard_arrow_down, color: ColorsManager.mainText) : null,
394         border: OutlineInputBorder(
395             borderRadius: BorderRadius.circular(12),
396             borderSide: BorderSide(color: ColorsManager.borderLightGray),
397         ),
398         enabledBorder: OutlineInputBorder(
399             borderRadius: BorderRadius.circular(12),
400             borderSide: BorderSide(color: ColorsManager.borderLightGray),
401         ),
402         focusedBorder: OutlineInputBorder(
403             borderRadius: BorderRadius.circular(12),
404             borderSide: BorderSide(color: ColorsManager.primaryColor),
405         ),
406     ),
407     style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainText),
408 ),
409 ],
410 );
411 }
412
413 Widget _buildChipsSelection<T>({
414     required String label,
415     required List<T> items,
416     required T? selectedValue,
417     required Function(T) onSelected,
418     required String Function(T) itemLabelBuilder,
419 }) {
420     return Column(
421         crossAxisAlignment: CrossAxisAlignment.center,
422         children: [
423             Row(
424                 children: [
425                     Text(label, style: TextStylesManager.bold16.copyWith(color: ColorsManager.mainText)),
426                 ],
427             ),
428             verticalSpace12,
429             Wrap(
430                 spacing: 8.0,
431                 runSpacing: 12.0,
432                 alignment: WrapAlignment.center,
433                 children: items.map((item) {
434                     final isSelected = selectedValue == item;
435                     final displayText = itemLabelBuilder(item);
436                     return InkWell(
437                         onTap: () => onSelected(item),
438                         borderRadius: BorderRadius.circular(20),
439                         child: Container(
440                             padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 10),
441                             decoration: BoxDecoration(
442                                 color: isSelected ? ColorsManager.primaryColor : ColorsManager.surfacePrimary,
443                                 borderRadius: BorderRadius.circular(20),
444                                 border: Border.all(
445                                     color: isSelected ? ColorsManager.primaryColor : ColorsManager.borderLightGray,
446                                 ),
447                             ),
448                             child: Text(
449                                 displayText,
450                                 style: TextStylesManager.bold14.copyWith(
451                                     color: isSelected ? Colors.white : ColorsManager.mainText,
452                                 ),
453                             ),
454                         ),
455                     );
456                 }).toList(),
457             ),
458         ],
459     );
460 }
461 }
462

```



```
1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
7 import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
8 import 'package:moean/core/utils/constants/spacing.dart';
9 import 'package:moean/core/utils/extensions/context_extension.dart';
10 import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
11 import 'package:moean/features/exam_generation/data/models/exam_models.dart';
12 import 'package:moean/features/exam_generation/presentation/cubit/exam_preview_cubit.dart';
13 import 'package:moean/features/reports/presentation/screen/pdf_preview_screen.dart';
14 import 'package:moean/features/exam_generation/data/services/exam_pdf_service.dart';
15 import 'package:moean/core/network/local/cache_helper.dart';
16 import 'package:moean/features/exam_generation/presentation/widgets/add_manual_question_dialog.dart';
17 import 'package:moean/core/utils/constants/primary/upgrade_prompt_bottom_sheet.dart';
18
19 class ExamPreviewScreen extends StatefulWidget {
20   final ExamEntity initialExam;
21
22   const ExamPreviewScreen({super.key, required this.initialExam});
23
24   @override
25   State<ExamPreviewScreen> createState() => _ExamPreviewScreenState();
26 }
27
28 class _ExamPreviewScreenState extends State<ExamPreviewScreen> {
29   @override
30   void initState() {
31     super.initState();
32     // Load fresh exam data in case it was edited elsewhere or points need refresh
33     context.read<ExamPreviewCubit>().loadExam(widget.initialExam.id);
34   }
35
36   @override
37   Widget build(BuildContext context) {
38     return BlocListener<ExamPreviewCubit, ExamPreviewState>(
39       listener: (context, state) {
40         if (state is ExamPreviewPaymentRequired) {
41           UpgradePromptBottomSheet.show(
42             context,
43             message: state.message,
44             isQuotaExceeded: state.code == 'quota_exceeded',
45           );
46         } else if (state is ExamPreviewError) {
47           ScaffoldMessenger.of(context).showSnackBar(
48             SnackBar(content: Text(state.message), backgroundColor: ColorsManager.errorColor),
49           );
50         }
51       },
52       child: Scaffold(
53         backgroundColor: ColorsManager.background,
54         appBar: AppBar(
55           backgroundColor: ColorsManager.background,
56           elevation: 0,
57           centerTitle: true,
58           title: Text(
59             appTranslation().get('exam_preview_title'),
60             style: TextStylesManager.bold18.copyWith(color: ColorsManager.mainText),
61           ),
62         leading: IconButton(
63           icon: Icon(Icons.arrow_back_ios, color: ColorsManager.mainText),
64           onPressed: () => context.pop(),
65         ),
66         actions: [
67           BlocBuilder<ExamPreviewCubit, ExamPreviewState>(
68             builder: (context, state) {
69               if (state is ExamPreviewLoaded) {
70                 return IconButton(
71                   icon: Icon(Icons.picture_as_pdf, color: ColorsManager.primaryColor),
```

```

72         onPressed: () {
73             final teacherName = CacheHelper.getData(key: 'teacher_name') as String? ?? '';
74             Navigator.push(
75                 context,
76                 MaterialPageRoute(
77                     builder: (context) => PdfPreviewScreen(
78                         title: 'معاينة الاختبار (PDF)',
79                         buildPdf: () => ExamPdfService.generatePdf(
80                             exam: state.exam,
81                             showAnswers: state.showAnswers,
82                             teacherName: teacherName,
83                             schoolName: '',
84                             educationOffice: '',
85                         ),
86                     ),
87                 ),
88             );
89         },
90     );
91 }
92 return const SizedBox.shrink();
93 },
94 ),
95 ],
96 ),
97 body: BlocBuilder<ExamPreviewCubit, ExamPreviewState>(
98     builder: (context, state) {
99         if (state is ExamPreviewLoading || state is ExamPreviewInitial) {
100             return Center(child: CircularProgressIndicator(color: ColorsManager.primaryColor));
101         }
102
103         if (state is ExamPreviewError) {
104             return Center(
105                 child: Text(state.message, style: TextStylesManager.bold16.copyWith(color: ColorsManager.errorC
106             )),
107         );
108     }
109
110     final loadedState = state as ExamPreviewLoaded;
111     final exam = loadedState.exam;
112
113     // Sort by question_order
114     final questions = List<QuestionEntity>.from(exam.questions)
115         ..sort((a, b) => a.questionOrder.compareTo(b.questionOrder));
116
117     return Column(
118         children: [
119             // Settings Header
120             Container(
121                 padding: const EdgeInsets.all(16),
122                 color: ColorsManager.surfacePrimary,
123                 child: Column(
124                     children: [
125                         Row(
126                             mainAxisAlignment: MainAxisAlignment.spaceBetween,
127                             children: [
128                                 Text('وضع المعلم', style: TextStylesManager.bold14.copyWith(color: ColorsManager.main
129                                 Text)),
130
131                                 Switch(
132                                     value: loadedState.isTeacherMode,
133                                     activeColor: ColorsManager.primaryColor,
134                                     onChanged: (val) => context.read<ExamPreviewCubit>().toggleMode(val),
135                                 ),
136                             ],
137                         ),
138                         if (loadedState.isTeacherMode)
139                             Row(
140                                 mainAxisAlignment: MainAxisAlignment.spaceBetween,
141                                 children: [
142                                     Text('إظهار الإجابات', style: TextStylesManager.bold14.copyWith(color: ColorsManage
143                                     r.mainText)),
144
145                                     Switch(
146                                         value: loadedState.showAnswers,
147                                         activeColor: ColorsManager.primaryColor,
148                                         onChanged: (val) => context.read<ExamPreviewCubit>().toggleAnswers(val),

```

```

144         ),
145     ],
146 ),
147 ],
148 ),
149 ),
150
151 // Total points live calculator
152 Container(
153   padding: const EdgeInsets.symmetric(vertical: 8, horizontal: 16),
154   color: ColorsManager.primaryColor.withValues(alpha: 0.1),
155   child: Row(
156     mainAxisAlignment: MainAxisAlignment.spaceBetween,
157     children: [
158       Text('إجمالي الدرجات:', style: TextStyleManager.bold14.copyWith(color: ColorsManager.mai
nText)),
159       Text(
160         loadedState.provisionalTotalPoints.toStringAsFixed(2),
161         style: TextStyleManager.bold16.copyWith(color: ColorsManager.primaryColor),
162       ),
163     ],
164   ),
165 ),
166
167 Expanded(
168   child: ListView.builder(
169     padding: const EdgeInsets.all(20),
170     itemCount: questions.length,
171     itemBuilder: (context, index) {
172       final q = questions[index];
173       return _buildQuestionCard(context, q, loadedState);
174     },
175   ),
176 ),
177
178 // Save points button
179 if (exam.status == 'draft' && loadedState.pendingPointsChanges.isNotEmpty)
180   Container(
181     padding: const EdgeInsets.all(20),
182     decoration: BoxDecoration(
183       color: ColorsManager.surfacePrimary,
184       boxShadow: [
185         BoxShadow(color: Colors.black.withOpacity(0.05), blurRadius: 10, offset: const Offset
(0, -5)),
186       ],
187     ),
188     child: state is ExamPreviewSaving
189       ? const Center(child: CircularProgressIndicator())
190       : PrimaryElevatedButton(
191         text: 'حفظ الدرجات',
192         onPressed: () => context.read<ExamPreviewCubit>().savePoints(),
193       ),
194   ),
195 ],
196 );
197 },
198 ),
199 floatingActionButton: BlocBuilder<ExamPreviewCubit, ExamPreviewState>(
200   builder: (context, state) {
201     if (state is ExamPreviewLoaded && state.exam.status == 'draft') {
202       return FloatingActionButton.extended(
203         onPressed: () {
204           final uniqueLessonIds = state.exam.questions.map((q) => q.lessonId).toSet().toList();
205           showDialog(
206             context: context,
207             builder: (_) => AddManualQuestionDialog(
208               examId: state.exam.id,
209               availableLessonIds: uniqueLessonIds,
210               onSubmit: (request) {
211                 context.read<ExamPreviewCubit>().addManualQuestion(request);
212               },
213             ),
214           );
215         },
216         backgroundColor: ColorsManager.primaryColor,

```

```

217         icon: const Icon(Icons.add, color: Colors.white),
218         label: Text('إضافة سؤال', style: TextStyleManager.bold14.copyWith(color: Colors.white)),
219     );
220     }
221     return const SizedBox.shrink();
222   },
223 ),
224 ),
225 );
226 }
227
228 Widget _buildQuestionCard(BuildContext context, QuestionEntity q, ExamPreviewLoaded state) {
229   final isEnglish = ExamPdfService.isEnglishText(q.questionText);
230   final textDir = isEnglish ? TextDirection.ltr : TextDirection.rtl;
231
232   return Card(
233     margin: const EdgeInsets.only(bottom: 20),
234     elevation: 0,
235     color: ColorsManager.surfacePrimary,
236     shape: RoundedRectangleBorder(
237       borderRadius: BorderRadius.circular(12),
238       side: BorderSide(color: ColorsManager.borderLightGray),
239     ),
240     child: Directionality(
241       textDirection: textDir,
242       child: Padding(
243         padding: const EdgeInsets.all(16),
244         child: Column(
245           crossAxisAlignment: CrossAxisAlignment.start,
246           children: [
247             Row(
248               crossAxisAlignment: CrossAxisAlignment.start,
249               children: [
250                 Text('${q.questionOrder}. ', style: TextStyleManager.bold16.copyWith(color: ColorsManager.
mainText)),
251                 Expanded(
252                   child: Text(
253                     q.questionText,
254                     style: TextStyleManager.bold16.copyWith(color: ColorsManager.mainText),
255                     textDirection: textDir,
256                   ),
257                 ),
258                 if (state.exam.status == 'draft')
259                   _PointsEditor(
260                     initialPoints: state.pendingPointsChanges[q.id] ?? q.points.toDouble(),
261                     onChanged: (parsed) {
262                       context.read<ExamPreviewCubit>().updatePendingPoints(q.id, parsed);
263                     },
264                   )
265                 else
266                   Text(
267                     isEnglish ? '${q.points} point${q.points > 1 ? "s" : ""}' : '${q.points} درجة',
268                     style: TextStyleManager.bold14.copyWith(color: ColorsManager.primaryColor),
269                   ),
270               ],
271             ),
272             verticalSpace12,
273             _buildQuestionOptions(q, state.showAnswers, isEnglish),
274             if (state.isTeacherMode && state.showAnswers && q.type != 'matching' && q.type != 'mcq') ...[
275               verticalSpace12,
276               Container(
277                 padding: const EdgeInsets.all(8),
278                 decoration: BoxDecoration(
279                   color: ColorsManager.successColor.withValues(alpha: 0.1),
280                   borderRadius: BorderRadius.circular(8),
281                 ),
282                 child: Row(
283                   children: [
284                     const Icon(Icons.check_circle, color: ColorsManager.successColor, size: 16),
285                     horizontalSpace8,
286                     Expanded(
287                       child: Text(
288                         isEnglish ? 'Answer: ${q.correctAnswer}' : 'الإجابة: ${q.correctAnswer}',
289                         style: TextStyleManager.bold14.copyWith(color: ColorsManager.successColor),
290                         textDirection: textDir,

```

```

291         ),
292     ),
293 ],
294 ),
295 ),
296 ],
297 ],
298 ),
299 ),
300 ),
301 );
302 }
303
304 Widget _buildQuestionOptions(QuestionEntity q, bool showAnswers, bool isEnglish) {
305     final textDir = isEnglish ? TextDirection.ltr : TextDirection.rtl;
306
307     if (q.type == 'mcq') {
308         final options = List<String>.from(q.options ?? []);
309         return Column(
310             children: options.map((opt) {
311                 final isCorrect = opt == q.correctAnswer;
312                 return RadioGroup<String>(
313                     groupValue: showAnswers && isCorrect ? opt : null,
314                     onChanged: (_) {},
315                     child: RadioListTile<String>(
316                         value: opt,
317                         activeColor: ColorsManager.successColor,
318                         title: Text(
319                             opt,
320                             style: showAnswers && isCorrect
321                                 ? TextStylesManager.bold14.copyWith(
322                                     color: ColorsManager.successColor,
323                                 )
324                                 : TextStylesManager.regular14.copyWith(
325                                     color: ColorsManager.mainText,
326                                 ),
327                             textDirection: textDir,
328                         ),
329                     ),
330                 );
331             }).toList(),
332         );
333     } else if (q.type == 'true_false') {
334         final isTrueCorrect = showAnswers && (q.correctAnswer == 'صح' || q.correctAnswer.toLowerCase() == 'true');
335         final isFalseCorrect = showAnswers && (q.correctAnswer == 'خطأ' || q.correctAnswer.toLowerCase() == 'false');
336
337         return Row(
338             mainAxisAlignment: MainAxisAlignment.spaceEvenly,
339             children: [
340                 Row(
341                     children: [
342                         Radio<String>(
343                             value: 'صح',
344                             groupValue: isTrueCorrect ? 'صح' : null,
345                             activeColor: ColorsManager.successColor,
346                             onChanged: (_) {},
347                         ),
348                         Text(isEnglish ? 'True' : 'صح', style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainText)),
349                     ],
350                 ),
351                 Row(
352                     children: [
353                         Radio<String>(
354                             value: 'خطأ',
355                             groupValue: isFalseCorrect ? 'خطأ' : null,
356                             activeColor: ColorsManager.successColor,
357                             onChanged: (_) {},
358                         ),
359                         Text(isEnglish ? 'False' : 'خطأ', style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainText)),
360                     ],
361                 ),

```

```

362     ],
363     );
364   } else if (q.type == 'matching') {
365     List<String> columnA = [];
366     List<String> columnB = [];
367
368     if (q.options is MatchingOptionsModel) {
369       columnA = (q.options as MatchingOptionsModel).columnA;
370       columnB = (q.options as MatchingOptionsModel).columnB;
371     } else if (q.options is Map) {
372       columnA = List<String>.from(q.options['column_a'] ?? []);
373       columnB = List<String>.from(q.options['column_b'] ?? []);
374     }
375
376     return Row(
377       crossAxisAlignment: CrossAxisAlignment.start,
378       children: [
379         Expanded(
380           child: Column(
381             crossAxisAlignment: CrossAxisAlignment.start,
382             children: columnA.map((a) => Padding(
383               padding: const EdgeInsets.only(bottom: 8),
384               child: Text(a, textDirection: textDir, style: TextStyleManager.regular14.copyWith(color: Col
orsManager.mainText)),
385             )),
386           ),
387         ),
388         const SizedBox(width: 16),
389         Expanded(
390           child: Column(
391             crossAxisAlignment: CrossAxisAlignment.start,
392             children: columnB.map((b) => Padding(
393               padding: const EdgeInsets.only(bottom: 8),
394               child: Text(b, textDirection: textDir, style: TextStyleManager.regular14.copyWith(color: Col
orsManager.mainText)),
395             )),
396           ),
397         ),
398       ],
399     );
400   } else if (q.type == 'essay') {
401     return Container(
402       height: 100,
403       width: double.infinity,
404       decoration: BoxDecoration(
405         color: ColorsManager.surfacePrimary,
406         border: Border.all(color: ColorsManager.borderLightGray),
407         borderRadius: BorderRadius.circular(8),
408       ),
409     );
410   }
411   return const SizedBox.shrink();
412 }
413 }
414
415 class _PointsEditor extends StatefulWidget {
416   final double initialPoints;
417   final ValueChanged<double> onChanged;
418
419   const _PointsEditor({required this.initialPoints, required this.onChanged});
420
421   @override
422   State<_PointsEditor> createState() => _PointsEditorState();
423 }
424
425 class _PointsEditorState extends State<_PointsEditor> {
426   late TextEditingController _controller;
427
428   @override
429   void initState() {
430     super.initState();
431     _controller = TextEditingController(text: widget.initialPoints.toString());
432   }
433
434   @override

```

```

435 void didUpdateWidget(covariant _PointsEditor oldWidget) {
436     super.didUpdateWidget(oldWidget);
437     if (oldWidget.initialPoints != widget.initialPoints) {
438         if (double.tryParse(_controller.text) != widget.initialPoints) {
439             _controller.text = widget.initialPoints.toString();
440         }
441     }
442 }
443
444 @override
445 void dispose() {
446     _controller.dispose();
447     super.dispose();
448 }
449
450 void _increment() {
451     double current = double.tryParse(_controller.text) ?? widget.initialPoints;
452     current += 1.0;
453     _controller.text = current.toString();
454     widget.onChanged(current);
455 }
456
457 void _decrement() {
458     double current = double.tryParse(_controller.text) ?? widget.initialPoints;
459     if (current >= 1.0) {
460         current -= 1.0;
461         _controller.text = current.toString();
462         widget.onChanged(current);
463     } else if (current > 0) {
464         current = 0.0;
465         _controller.text = current.toString();
466         widget.onChanged(current);
467     }
468 }
469
470 @override
471 Widget build(BuildContext context) {
472     return Row(
473         mainAxisAlignment: MainAxisAlignment.min,
474         children: [
475             IconButton(
476                 icon: Icon(Icons.add_circle_outline, size: 24, color: ColorsManager.primaryColor),
477                 onPressed: _increment,
478                 padding: EdgeInsets.zero,
479                 constraints: const BoxConstraints(),
480             ),
481             const SizedBox(width: 8),
482             SizedBox(
483                 width: 80,
484                 child: PrimaryTextField(
485                     controller: _controller,
486                     hint: 'الدرجة',
487                     keyboardType: const TextInputType.numberWithOptions(decimal: true),
488                     onChanged: (val) {
489                         final parsed = double.tryParse(val);
490                         if (parsed != null && parsed >= 0) {
491                             widget.onChanged(parsed);
492                         }
493                     },
494                 ),
495             ),
496             const SizedBox(width: 8),
497             IconButton(
498                 icon: Icon(Icons.remove_circle_outline, size: 24, color: ColorsManager.primaryColor),
499                 onPressed: _decrement,
500                 padding: EdgeInsets.zero,
501                 constraints: const BoxConstraints(),
502             ),
503         ],
504     );
505 }
506 }
507

```

```
1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
7 import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
8 import 'package:moean/core/utils/constants/routes.dart';
9 import 'package:moean/core/utils/constants/spacing.dart';
10 import 'package:moean/core/utils/extensions/context_extension.dart';
11 import 'package:moean/features/exam_generation/presentation/cubit/exam_info_cubit.dart';
12 import 'package:moean/features/exam_generation/presentation/cubit/lesson_selection_cubit.dart';
13
14 class LessonSelectionScreen extends StatefulWidget {
15   const LessonSelectionScreen({super.key});
16
17   @override
18   State<LessonSelectionScreen> createState() => _LessonSelectionScreenState();
19 }
20
21 class _LessonSelectionScreenState extends State<LessonSelectionScreen> {
22   final _searchController = TextEditingController();
23
24   @override
25   void initState() {
26     super.initState();
27     final examInfoCubit = context.read<ExamInfoCubit>();
28     final selectedSubject = examInfoCubit.selectedSubject;
29     final selectedUnit = examInfoCubit.unitName;
30     if (selectedSubject != null) {
31       context.read<LessonSelectionCubit>().loadLessonsForSubjectId(
32         selectedSubject.id,
33         unitName: selectedUnit,
34       );
35     }
36   }
37
38   @override
39   void dispose() {
40     _searchController.dispose();
41     super.dispose();
42   }
43
44   Widget _buildBody(BuildContext context, LessonSelectionState state, LessonSelectionCubit cubit) {
45     if (state is LessonSelectionLoading) {
46       return Center(child: CircularProgressIndicator(color: ColorsManager.primaryColor));
47     }
48
49     if (state is LessonSelectionError) {
50       return Center(
51         child: Column(
52           mainAxisAlignment: MainAxisAlignment.center,
53           children: [
54             Text(state.message, style: TextStylesManager.bold14.copyWith(color: ColorsManager.errorColor)),
55             verticalSpace16,
56             PrimaryElevatedButton(
57               text: 'إعادة المحاولة',
58               onPressed: () => cubit.retry(),
59             ),
60           ],
61         ),
62       );
63     }
64
65     if (state is LessonSelectionUpdated) {
66       return Column(
67         children: [
68           Padding(
69             padding: const EdgeInsets.all(20),
70             child: Column(
71               crossAxisAlignment: CrossAxisAlignment.stretch,
```



```

72         children: [
73             Text('الخطوة 2 من 3: اختيار الدروس', style: TextStyleManager.bold16.copyWith(color: ColorsMa
nager.primaryColor)),
74             verticalSpace16,
75             PrimaryTextField(
76                 controller: _searchController,
77                 hint: 'ابحث عن درس...',
78                 prefixIcon: const Icon(Icons.search),
79                 onChanged: (val) => cubit.search(val),
80                 textInputAction: TextInputAction.search,
81                 onFieldSubmitted: (val) => cubit.search(val),
82             ),
83             verticalSpace16,
84
85             Text(
86                 'دروس محددة ${state.selectedLessons.length}',
87                 style: TextStyleManager.bold14.copyWith(color: ColorsManager.primaryColor),
88             ),
89             if (state.selectedLessons.isNotEmpty) ...[
90                 verticalSpace8,
91                 Wrap(
92                     spacing: 8,
93                     runSpacing: 8,
94                     children: state.selectedLessons.map((lesson) {
95                         return Container(
96                             padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 8),
97                             decoration: BoxDecoration(
98                                 color: ColorsManager.primaryColor.withValues(alpha: 0.1),
99                                 borderRadius: BorderRadius.circular(20),
100                            ),
101                            child: Row(
102                                mainAxisAlignment: MainAxisAlignment.min,
103                                children: [
104                                    Flexible(
105                                        child: Text(
106                                            lesson.title,
107                                            style: TextStyleManager.bold12.copyWith(color: ColorsManager.primaryColor),
108                                            overflow: TextOverflow.ellipsis,
109                                        ),
110                                    ),
111                                    const SizedBox(width: 8),
112                                    InkWell(
113                                        onTap: () => cubit.removeLesson(lesson.id),
114                                        child: const Icon(Icons.close, size: 16, color: Colors.red),
115                                    ),
116                                ],
117                            ),
118                        );
119                    }).toList(),
120                ),
121            ],
122        ],
123    ),
124),
125Expanded(
126    child: ListView.builder(
127        padding: const EdgeInsets.symmetric(horizontal: 20),
128        itemCount: state.filteredChapters.length,
129        itemBuilder: (context, index) {
130            final chapter = state.filteredChapters[index];
131
132            return Column(
133                crossAxisAlignment: CrossAxisAlignment.start,
134                children: [
135                    Container(
136                        width: double.infinity,
137                        padding: const EdgeInsets.symmetric(vertical: 8, horizontal: 12),
138                        decoration: BoxDecoration(
139                            color: ColorsManager.surfacePrimary,
140                            borderRadius: BorderRadius.circular(8),
141                            border: Border.all(color: ColorsManager.borderLightGray),
142                        ),
143                        child: Text(
144                            'الفصل: ${chapter.title} (الفصل: ${chapter.semester})',
145                            style: TextStyleManager.bold14.copyWith(color: ColorsManager.primaryColor),

```

```

146         ),
147     ),
148     verticalSpace8,
149     ...chapter.lessons.map((lesson) {
150         final isSelected = state.selectedLessons.any((l) => l.id == lesson.id);
151
152         return Container(
153             margin: const EdgeInsets.only(bottom: 12),
154             decoration: BoxDecoration(
155                 color: isSelected ? ColorsManager.primaryColor.withValues(alpha: 0.1) : ColorsManager.surfacePrimary,
156                 borderRadius: BorderRadius.circular(12),
157                 border: Border.all(
158                     color: isSelected ? ColorsManager.primaryColor : ColorsManager.borderLightGray,
159                     width: isSelected ? 2 : 1,
160                 ),
161             ),
162             child: CheckboxListTile(
163                 value: isSelected,
164                 activeColor: ColorsManager.primaryColor,
165                 title: Text(
166                     lesson.title,
167                     style: isSelected ? TextStylesManager.bold14.copyWith(color: ColorsManager.mainText) : TextStylesManager.regular14.copyWith(color: ColorsManager.mainText),
168                 ),
169                 onChanged: (_) => cubit.toggleLesson(lesson),
170                 shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
171             ),
172         );
173     })),
174     verticalSpace16,
175 ],
176 );
177 },
178 ),
179 ),
180 Container(
181     decoration: BoxDecoration(
182         color: ColorsManager.surfacePrimary,
183         boxShadow: [
184             BoxShadow(color: Colors.black.withValues(alpha: 0.05), blurRadius: 10, offset: const Offset(0, -5)),
185         ],
186     ),
187     child: SafeArea(
188         top: false,
189         left: false,
190         right: false,
191         minimum: const EdgeInsets.all(20),
192         child: PrimaryElevatedButton(
193             text: appTranslation().get('chose_app_continue'),
194             onPressed: cubit.hasSelection ? () {
195                 context.push(Routes.examGenerationCounts);
196             } : null,
197         ),
198     ),
199 ),
200 ],
201 );
202 }
203
204 return const SizedBox();
205 }
206
207 @override
208 Widget build(BuildContext context) {
209     return Scaffold(
210         backgroundColor: ColorsManager.background,
211         appBar: AppBar(
212             backgroundColor: ColorsManager.background,
213             elevation: 0,
214             centerTitle: true,
215             title: Text(
216                 appTranslation().get('exam_lesson_selection'),
217                 style: TextStylesManager.bold18.copyWith(color: ColorsManager.mainText),

```

```

218     ),
219     leading: IconButton(
220       icon: Icon(Icons.arrow_back_ios, color: ColorsManager.mainText),
221       onPressed: () => context.pop(),
222     ),
223   ),
224   body: BlocBuilder<LessonSelectionCubit, LessonSelectionState>(
225     builder: (context, state) {
226       final cubit = context.read<LessonSelectionCubit>();
227       return _buildBody(context, state, cubit);
228     },
229   ),
230 );
231 }
232 }
233

```

Lines • 253
Dart

Feature / Exam #187 -
Generation

lib/features/exam_generation/presentation/screens/manual_exam_screen.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7  import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
8  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
9  import 'package:moean/core/utils/constants/primary/upgrade_prompt_bottom_sheet.dart';
10 import 'package:moean/core/utils/extensions/context_extension.dart';
11 import 'package:moean/features/exam_generation/presentation/cubit/manual_exam_cubit.dart';
12 import 'package:moean/features/exam_generation/presentation/cubit/manual_exam_state.dart';
13 import 'package:moean/features/exam_generation/presentation/widgets/manual_exam_type_selector.dart';
14 import 'package:moean/features/exam_generation/presentation/widgets/manual_exam_dynamic_form.dart';
15 import 'package:moean/core/di/injections.dart';
16
17 class ManualExamScreenArgs {
18   final int examId;
19   final Map<int, String> lessons; // lesson_id -> lesson_name
20
21   ManualExamScreenArgs({
22     required this.examId,
23     required this.lessons,
24   });
25 }
26
27 class ManualExamScreen extends StatelessWidget {
28   const ManualExamScreen({super.key});
29
30   @override
31   Widget build(BuildContext context) {
32     // If arguments are missing, fallback for testing
33     final args = ModalRoute.of(context)?.settings.arguments as ManualExamScreenArgs? ??
34       ManualExamScreenArgs(examId: 1, lessons: {1: 'Lesson 1', 2: 'Lesson 2'});
35
36     return BlocProvider(
37       create: (context) => sl<ManualExamCubit>(),
38       child: Directionality(
39         textDirection: TextDirection.rtl,
40         child: Scaffold(
41           backgroundColor: ColorsManager.background,
42           appBar: AppBar(
43             backgroundColor: Colors.transparent,
44             elevation: 0,
45             centerTitle: true,
46             leading: IconButton(
47               icon: Icon(
48                 Icons.close,
49                 color: ColorsManager.primaryColor,
50               ),
51               onPressed: () => context.pop(),
52             ),
53           title: Text(

```

```

54      appTranslation().get('manual_exam_title'),
55      style: TextStyleManager.bold18.copyWith(
56        color: ColorsManager.mainText,
57      ),
58    ),
59  ),
60  body: BlocConsumer<ManualExamCubit, ManualExamState>(
61    listener: (context, state) {
62      if (state is ManualExamSuccess) {
63        ScaffoldMessenger.of(context).showSnackBar(
64          SnackBar(
65            content: Text(appTranslation().get('question_added_success')),
66            backgroundColor: ColorsManager.primaryColor,
67          ),
68        );
69        context.pop(true); // Return true to indicate success
70      } else if (state is ManualExamPaymentRequired) {
71        UpgradePromptBottomSheet.show(
72          context,
73          message: state.message,
74          isQuotaExceeded: state.code == 'quota_exceeded',
75        );
76      } else if (state is ManualExamError) {
77        ScaffoldMessenger.of(context).showSnackBar(
78          SnackBar(
79            content: Text(state.error),
80            backgroundColor: Colors.red,
81          ),
82        );
83      }
84    },
85    builder: (context, state) {
86      final cubit = ManualExamCubit.get(context);
87
88      return Stack(
89        children: [
90          SingleChildScrollView(
91            padding: const EdgeInsets.all(16.0),
92            child: Column(
93              crossAxisAlignment: CrossAxisAlignment.start,
94              children: [
95                ManualExamTypeSelector(
96                  selectedType: cubit.selectedQuestionType,
97                  onTypeSelected: cubit.changeQuestionType,
98                ),
99                verticalSpace24,
100              Row(
101                crossAxisAlignment: CrossAxisAlignment.start,
102                children: [
103                  Expanded(
104                    flex: 1,
105                    child: Column(
106                      crossAxisAlignment: CrossAxisAlignment.start,
107                      children: [
108                        Text(
109                          appTranslation().get('grade'),
110                          style: TextStyleManager.bold14.copyWith(
111                            color: ColorsManager.mainText,
112                          ),
113                        ),
114                        verticalSpace8,
115                        PrimaryTextField(
116                          controller: cubit.gradeController,
117                          keyboardType: TextInputType.number,
118                          hint: '',
119                        ),
120                      ],
121                    ),
122                  ),
123                  horizontalSpace16,
124                  Expanded(
125                    flex: 3,
126                    child: Column(
127                      crossAxisAlignment: CrossAxisAlignment.start,
128                      children: [

```

```

129         Text(
130             appTranslation().get('related_lesson'),
131             style: TextStylesManager.bold14.copyWith(
132                 color: ColorsManager.mainText,
133             ),
134         ),
135         verticalSpace8,
136         Container(
137             padding: const EdgeInsets.symmetric(horizontal: 16),
138             decoration: BoxDecoration(
139                 border: Border.all(color: ColorsManager.borderColor),
140                 borderRadius: BorderRadius.circular(12),
141                 color: ColorsManager.background,
142             ),
143             child: DropdownButtonHideUnderline(
144                 child: DropdownButton<int>(
145                     value: cubit.selectedLessonId,
146                     hint: Text(
147                         'اختر الدرس...',
148                         style: TextStylesManager.regular14.copyWith(color: ColorsManager.se
condaryText),
149                     ),
150                     isExpanded: true,
151                     icon: Icon(Icons.keyboard_arrow_down_rounded, color: ColorsManager.ma
inText),
152                     items: args.lessons.entries.map((entry) {
153                         return DropdownMenuItem<int>(
154                             value: entry.key,
155                             child: Text(
156                                 entry.value,
157                                 style: TextStylesManager.regular14.copyWith(color: ColorsManag
e
r.mainText),
158                             overflow: TextOverflow.ellipsis,
159                         ),
160                     );
161                 }).toList(),
162                 onChanged: (val) {
163                     if (val != null) cubit.updateLesson(val);
164                 },
165             ),
166         ),
167     ),
168 ],
169 ),
170 ),
171 ],
172 ),
173 verticalSpace24,
174 Text(
175     appTranslation().get('question_text'),
176     style: TextStylesManager.bold14.copyWith(
177         color: ColorsManager.mainText,
178     ),
179 ),
180 verticalSpace8,
181 PrimaryTextField(
182     controller: cubit.questionTextController,
183     hint: appTranslation().get('write_question_text'),
184     maxLines: 4,
185 ),
186 verticalSpace24,
187 const ManualExamDynamicForm(),
188 verticalSpace64, // Space for bottom buttons
189 verticalSpace24,
190 ],
191 ),
192 ),
193
194 // Bottom Buttons
195 Positioned(
196     bottom: 0,
197     left: 0,
198     right: 0,
199     child: Container(
200         padding: const EdgeInsets.all(16),

```

```

201         decoration: BoxDecoration(
202             color: ColorsManager.background,
203             boxShadow: [
204                 BoxShadow(
205                     color: Colors.black.withValues(alpha: 0.05),
206                     blurRadius: 10,
207                     offset: const Offset(0, -5),
208                 ),
209             ],
210         ),
211         child: Row(
212             children: [
213                 Expanded(
214                     child: PrimaryElevatedButton(
215                         onPressed: () => cubit.submitQuestion(args.examId),
216                         text: appTranslation().get('save_question'),
217                         icon: const Icon(Icons.check_circle_outline, color: Colors.white),
218                         isLoading: state is ManualExamLoading,
219                     ),
220                 ),
221                 horizontalSpace16,
222                 Expanded(
223                     child: OutlinedButton(
224                         onPressed: () => context.pop(),
225                         style: OutlinedButton.styleFrom(
226                             padding: const EdgeInsets.symmetric(vertical: 14),
227                             side: BorderSide(color: ColorsManager.borderColor),
228                             shape: RoundedRectangleBorder(
229                                 borderRadius: BorderRadius.circular(12),
230                             ),
231                         ),
232                         child: Text(
233                             appTranslation().get('cancel'),
234                             style: TextStylesManager.bold16.copyWith(
235                                 color: ColorsManager.secondaryText,
236                             ),
237                         ),
238                     ),
239                 ),
240             ],
241         ),
242     ),
243 ),
244 ],
245 );
246 },
247 ),
248 ),
249 ),
250 );
251 }
252 }
253

```

Lines • 282
Dart

Feature / Exam #188 -

Generation lib/features/exam_generation/presentation/screens/my_exams_screen.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/primary/conditional_builder.dart';
7  import 'package:moean/core/utils/constants/spacing.dart';
8  import 'package:moean/core/utils/constants/routes.dart';
9  import 'package:moean/core/utils/extensions/context_extension.dart';
10 import 'package:moean/features/exam_generation/presentation/cubit/my_exams_cubit.dart';
11 import 'package:moean/features/exam_generation/presentation/cubit/my_exams_state.dart';
12 import 'package:moean/features/exam_generation/presentation/widgets/exam_list_item_widget.dart';
13 import 'package:moean/features/exam_generation/presentation/widgets/my_exams_header_widget.dart';
14 import 'package:moean/features/exam_generation/domain/usecases/get_exam_usecase.dart';
15 import 'package:moean/core/di/injections.dart';
16

```

```

17 class MyExamsScreen extends StatefulWidget {
18   const MyExamsScreen({super.key});
19
20   @override
21   State<MyExamsScreen> createState() => _MyExamsScreenState();
22 }
23
24 class _MyExamsScreenState extends State<MyExamsScreen> {
25   final ScrollController _horizontalController = ScrollController();
26   final ScrollController _verticalController = ScrollController();
27
28   @override
29   void initState() {
30     super.initState();
31     context.read<MyExamsCubit>().fetchExams(isRefresh: true);
32   }
33
34   @override
35   void dispose() {
36     _horizontalController.dispose();
37     _verticalController.dispose();
38     super.dispose();
39   }
40
41   @override
42   Widget build(BuildContext context) {
43     return Scaffold(
44       backgroundColor: ColorsManager.background,
45       appBar:
46         // AppBar(
47         //   title: Text(
48         //     appTranslation().get('my_exam'),
49         //     style: TextStylesManager.bold18,
50         //   ),
51         //   backgroundColor: ColorsManager.background,
52         //   elevation: 0,
53         // ),
54       AppBar(
55         backgroundColor: ColorsManager.background,
56         elevation: 0,
57         centerTitle: true,
58         title: Text(
59           "الاختبارات السابقة",
60           style: TextStylesManager.bold18,
61         ),
62       leading: IconButton(
63         icon: Icon(Icons.arrow_back_ios, color: ColorsManager.mainText),
64         onPressed: () => context.pop(),
65       ),
66       actions: [
67         IconButton(
68           icon: Icon(Icons.quiz_outlined, color: ColorsManager.mainText),
69           onPressed: () {
70             context.push(Routes.customQuestions);
71           },
72         ),
73       ],
74     ),
75     body: Directionality(
76       textDirection: TextDirection.rtl,
77       child: BlocConsumer<MyExamsCubit, MyExamsState>(
78         buildWhen: (previous, current) {
79           return current is MyExamsInitial ||
80             current is MyExamsLoading ||
81             current is MyExamsLoaded ||
82             current is MyExamsError;
83         },
84         listener: (context, state) {
85           if (state is MyExamsActionSuccess) {
86             ScaffoldMessenger.of(context).showSnackBar(
87               SnackBar(content: Text(state.message), backgroundColor: Colors.green),
88             );
89           } else if (state is MyExamsActionError) {
90             ScaffoldMessenger.of(context).showSnackBar(
91               SnackBar(content: Text(state.message), backgroundColor: Colors.red),

```

```

92         );
93     } else if (state is MyExamsError) {
94         ScaffoldMessenger.of(context).showSnackBar(
95             SnackBar(content: Text(state.message), backgroundColor: Colors.red),
96         );
97     }
98 },
99 builder: (context, state) {
100     if (state is MyExamsInitial || state is MyExamsLoading) {
101         return Center(child: CircularProgressIndicator(color: ColorsManager.primaryColor));
102     }
103
104     if (state is MyExamsLoaded) {
105         final loadedState = state;
106
107         const double colTitle = 260;
108         const double colStatus = 110;
109         const double colQuestions = 110;
110         const double colGrade = 110;
111         const double colDate = 130;
112         const double colActions = 280;
113         const double totalWidth = colTitle + colStatus + colQuestions + colGrade + colDate + colAction
s;
114
115         return Padding(
116             padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
117             child: Column(
118                 children: [
119                     MyExamsHeaderWidget(
120                         selectedTab: loadedState.selectedTab,
121                         onTabChanged: (tab) => context.read<MyExamsCubit>().changeTab(tab),
122                         onSearchChanged: (query) => context.read<MyExamsCubit>().searchExams(query),
123                     ),
124                     verticalSpace24,
125                     Expanded(
126                         child: ConditionalBuilder(
127                             loadingState: false,
128                             emptyState: loadedState.exams.data.isEmpty,
129                             successBuilder: (context) => Container(
130                                 decoration: BoxDecoration(
131                                     color: ColorsManager.surfacePrimary,
132                                     borderRadius: BorderRadius.circular(16),
133                                     border: Border.all(color: ColorsManager.borderLightGray),
134                                 ),
135                                 child: Scrollbar(
136                                     controller: _horizontalController,
137                                     thumbVisibility: true,
138                                     child: SingleChildScrollView(
139                                         controller: _horizontalController,
140                                         scrollDirection: Axis.horizontal,
141                                         child: SizedBox(
142                                             width: totalWidth,
143                                             child: Column(
144                                                 children: [
145                                                     _buildTableHeader(colTitle, colStatus, colQuestions, colGrade, colDate, c
olActions),
146
147                                                     Expanded(
148                                                         child: ListView.separated(
149                                                             controller: _verticalController,
150                                                             physics: const AlwaysScrollableScrollPhysics(),
151                                                             itemCount: loadedState.exams.data.length,
152                                                             separatorBuilder: (_, _) => Divider(color: ColorsManager.borderLightG
ray, height: 1),
153
154                                                             itemBuilder: (context, index) {
155                                                                 final exam = loadedState.exams.data[index];
156                                                                 return ExamListItemWidget(
157                                                                     exam: exam,
158                                                                     colTitle: colTitle,
159                                                                     colStatus: colStatus,
160                                                                     colQuestions: colQuestions,
161                                                                     colGrade: colGrade,
162                                                                     colDate: colDate,
163                                                                     colActions: colActions,
164                                                                     onPublish: () => context.read<MyExamsCubit>().publishExam(exam.i
d),

```



```

163         onDelete: () => _confirmDelete(context, exam.id),
164         onView: () => _navigateToExamPreview(context, exam.id),
165         onTap: () => _navigateToExamPreview(context, exam.id),
166     );
167   },
168 ),
169 ),
170 ],
171 ),
172 ),
173 ),
174 ),
175 ),
176   emptyBuilder: (context) => Center(
177     child: Text(
178       appTranslation().get('no_exams_found'),
179       style: TextStylesManager.bold16.copyWith(color: ColorsManager.placeholder),
180     ),
181   ),
182 ),
183 ],
184 ],
185 ),
186 );
187 }
188
189   return const SizedBox.shrink();
190 },
191 ),
192 ),
193 );
194 }
195
196 Widget _buildTableHeader(double c1, double c2, double c3, double c4, double c5, double c6) {
197   final isDark = ColorsManager.isDark;
198   final headerColor = isDark ? ColorsManager.backgroundDark : Colors.grey.withValues(alpha: 0.1);
199
200   return Container(
201     height: 50,
202     decoration: BoxDecoration(
203       color: headerColor,
204       borderRadius: const BorderRadius.only(topLeft: Radius.circular(16), topRight: Radius.circular(16)),
205       border: Border(bottom: BorderSide(color: ColorsManager.borderLightGray)),
206     ),
207     child: Row(
208       children: [
209         _buildHeaderCell(appTranslation().get('title'), c1),
210         _buildHeaderCell(appTranslation().get('status'), c2, alignCenter: true),
211         _buildHeaderCell(appTranslation().get('questions_count'), c3, alignCenter: true),
212         _buildHeaderCell(appTranslation().get('grade'), c4, alignCenter: true),
213         _buildHeaderCell(appTranslation().get('last_update'), c5, alignCenter: true),
214         _buildHeaderCell(appTranslation().get('actions'), c6, alignCenter: true),
215       ],
216     ),
217   );
218 }
219
220 Widget _buildHeaderCell(String text, double width, {bool alignCenter = false}) {
221   return SizedBox(
222     width: width,
223     child: Padding(
224       padding: const EdgeInsets.symmetric(horizontal: 16.0),
225       child: Align(
226         alignment: alignCenter ? Alignment.center : AlignmentDirectional.centerStart,
227         child: Text(text, style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainText)),
228       ),
229     ),
230   );
231 }
232
233 void _confirmDelete(BuildContext context, int examId) {
234   showDialog(
235     context: context,
236     builder: (ctx) => AlertDialog(
237       title: Text(appTranslation().get('confirm_delete'), style: TextStylesManager.bold16),

```

```

238         content: Text(appTranslation().get('delete_exam_msg'), style: TextStylesManager.medium14),
239         actions: [
240             TextButton(
241                 onPressed: () => Navigator.pop(ctx),
242                 child: Text(appTranslation().get('cancel'), style: TextStylesManager.medium14),
243             ),
244             TextButton(
245                 onPressed: () {
246                     Navigator.pop(ctx);
247                     context.read<MyExamsCubit>().deleteExam(examId);
248                 },
249                 child: Text(appTranslation().get('delete'), style: TextStylesManager.bold14.copyWith(color: Color
s.red)),
250             ),
251         ],
252     ),
253 );
254 }
255
256 void _navigateToExamPreview(BuildContext context, int examId) async {
257     // Show loading
258     showDialog(context: context, barrierDismissible: false, builder: (_) => Center(child: CircularProgressIndicator(color: ColorsManager.primaryColor)));
259
260     // Fetch full exam details to pass to ExamPreviewScreen
261     final getExamUseCase = sl<GetExamUseCase>();
262     final result = await getExamUseCase.execute(examId);
263
264     if (context.mounted) {
265         Navigator.pop(context); // Close loading
266
267         result.fold(
268             (failure) {
269                 ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(failure.message), backgroundColor: Colors.red));
270             },
271             (exam) {
272                 Navigator.pushNamed(context, Routes.examGenerationPreview, arguments: exam).then((_) {
273                     if (context.mounted) {
274                         context.read<MyExamsCubit>().fetchExams(isRefresh: true);
275                     }
276                 });
277             },
278         );
279     }
280 }
281 }
282

```

Lines • 486
Dart

Feature / Exam #189 –

Generation lib/features/exam_generation/presentation/screens/question_counts_screen.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
7  import 'package:moean/core/utils/constants/routes.dart';
8  import 'package:moean/core/utils/constants/spacing.dart';
9  import 'package:moean/core/utils/extensions/context_extension.dart';
10 import 'package:moean/features/exam_generation/presentation/cubit/exam_info_cubit.dart';
11 import 'package:moean/features/exam_generation/presentation/cubit/generate_exam_cubit.dart';
12 import 'package:moean/features/exam_generation/presentation/cubit/lesson_selection_cubit.dart';
13 import 'package:moean/features/exam_generation/presentation/cubit/question_count_cubit.dart';
14 import 'package:moean/core/utils/constants/primary/upgrade_prompt_bottom_sheet.dart';
15
16 class QuestionCountsScreen extends StatefulWidget {
17     const QuestionCountsScreen({super.key});
18
19     @override
20     State<QuestionCountsScreen> createState() => _QuestionCountsScreenState();
21 }

```

```

22
23 class _QuestionCountsScreenState extends State<QuestionCountsScreen> {
24   @override
25   void initState() {
26     super.initState();
27     final selectedLessons = context.read<LessonSelectionCubit>().selectedLessons;
28     context.read<QuestionCountCubit>().initForLessons(selectedLessons);
29   }
30
31   void _handleGenerate() {
32     final infoCubit = context.read<ExamInfoCubit>();
33     final lessonCubit = context.read<LessonSelectionCubit>();
34     final countCubit = context.read<QuestionCountCubit>();
35
36     final hasBankQuestions = infoCubit.selectedBankQuestionIds.isNotEmpty;
37
38     if (countCubit.totalRequestedQuestions == 0 && !hasBankQuestions) {
39       ScaffoldMessenger.of(context).showSnackBar(
40         SnackBar(
41           content: Text('يجب اختيار سؤال واحد على الأقل', style: TextStyleManager.bold14),
42           backgroundColor: ColorsManager.errorColor,
43         ),
44       );
45       return;
46     }
47
48     context.read<GenerateExamCubit>().generate(
49       title: infoCubit.examTitle,
50       grade: infoCubit.selectedGrade?.name ?? '',
51       subject: infoCubit.selectedSubject?.name ?? '',
52       lessons: lessonCubit.selectedLessons.map((l) => {'id': l.id, 'name': l.title}).toList(),
53       counts: countCubit.counts,
54       selectedQuestionIds: infoCubit.selectedBankQuestionIds,
55     );
56   }
57
58   @override
59   Widget build(BuildContext context) {
60     return BlocListener<GenerateExamCubit, GenerateExamState>(
61       listener: (context, state) {
62         if (state is GenerateExamError) {
63           ScaffoldMessenger.of(context).showSnackBar(
64             SnackBar(
65               content: Text(state.message, style: TextStyleManager.bold14),
66               backgroundColor: ColorsManager.errorColor,
67             ),
68           );
69         } else if (state is GenerateExamPaymentRequired) {
70           UpgradePromptBottomSheet.show(
71             context,
72             message: state.message,
73             isQuotaExceeded: state.code == 'quota_exceeded',
74           );
75         } else if (state is GenerateExamSuccess) {
76           // Success, navigate to preview screen passing the exam
77           context.push(Routes.examGenerationPreview, arguments: state.exam);
78         }
79       },
80       child: Scaffold(
81         backgroundColor: ColorsManager.background,
82         appBar: AppBar(
83           backgroundColor: ColorsManager.background,
84           elevation: 0,
85           centerTitle: true,
86           title: Text(
87             appTranslation().get('exam_question_counts'),
88             style: TextStyleManager.bold18.copyWith(color: ColorsManager.mainText),
89           ),
90         leading: BlocBuilder<GenerateExamCubit, GenerateExamState>(
91           builder: (context, state) {
92             // Disable back button while generating
93             if (state is GenerateExamLoading) return const SizedBox.shrink();
94             return IconButton(
95               icon: Icon(Icons.arrow_back_ios, color: ColorsManager.mainText),
96               onPressed: () => context.pop(),

```

```

97     );
98   },
99   ),
100 ),
101 body: BlocBuilder<GenerateExamCubit, GenerateExamState>(
102   builder: (context, generateState) {
103     final isGenerating = generateState is GenerateExamLoading;
104
105     return Stack(
106       children: [
107         BlocBuilder<QuestionCountCubit, QuestionCountState>(
108           builder: (context, state) {
109             if (state is! QuestionCountUpdated) {
110               return Center(child: CircularProgressIndicator(color: ColorsManager.primaryColor));
111             }
112
113             final selectedLessons = context.read<LessonSelectionCubit>().selectedLessons;
114
115             return Column(
116               children: [
117                 Expanded(
118                   child: ListView(
119                     padding: const EdgeInsets.symmetric(horizontal: 16),
120                     children: [
121                       verticalSpace16,
122                       Text(
123                         'الخطوة 2 من 3: تحديد الأسئلة',
124                         style: TextStyleManager.bold16.copyWith(color: ColorsManager.primaryColor),
125                       ),
126                       verticalSpace16,
127                       Container(
128                         padding: const EdgeInsets.all(4),
129                         decoration: BoxDecoration(
130                           color: ColorsManager.surfacePrimary,
131                           borderRadius: BorderRadius.circular(12),
132                           border: Border.all(color: ColorsManager.borderLightGray),
133                         ),
134                         child: Row(
135                           children: [
136                             Expanded(
137                               child: GestureDetector(
138                                 onTap: () => context.push(Routes.examBankQuestions),
139                                 child: Container(
140                                   padding: const EdgeInsets.symmetric(vertical: 12),
141                                   decoration: BoxDecoration(
142                                     color: ColorsManager.surfacePrimary,
143                                     borderRadius: BorderRadius.circular(8),
144                                   ),
145                                   alignment: Alignment.center,
146                                   child: Text(
147                                     'من بنك الأسئلة',
148                                     style: TextStyleManager.bold14.copyWith(color: ColorsManager.secondaryText),
149                                   ),
150                                 ),
151                               ),
152                             ),
153                             Expanded(
154                               child: Container(
155                                 padding: const EdgeInsets.symmetric(vertical: 12),
156                                 decoration: BoxDecoration(
157                                   color: ColorsManager.primaryColor,
158                                   borderRadius: BorderRadius.circular(8),
159                                 ),
160                                 alignment: Alignment.center,
161                                 child: Text(
162                                   'إنشاء تلقائي',
163                                   style: TextStyleManager.bold14.copyWith(color: Colors.white),
164                                 ),
165                               ),
166                             ),
167                           ],
168                         ),
169                       ),
170                       verticalSpace20,

```

```

171 Text(
172   'الدروس المختارة',
173   style: TextStyleManager.bold16.copyWith(color: ColorsManager.mainText),
174 ),
175 verticalSpace12,
176 Wrap(
177   spacing: 8,
178   runSpacing: 8,
179   children: selectedLessons.asMap().entries.map((entry) {
180     final index = entry.key;
181     final lesson = entry.value;
182     return Container(
183       padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
184       decoration: BoxDecoration(
185         color: ColorsManager.surfacePrimary,
186         borderRadius: BorderRadius.circular(20),
187         border: Border.all(color: ColorsManager.borderLightGray),
188       ),
189       child: Row(
190         mainAxisAlignment: MainAxisAlignment.min,
191         children: [
192           Container(
193             width: 22,
194             height: 22,
195             decoration: BoxDecoration(
196               color: ColorsManager.primaryColor.withValues(alpha: 0.1),
197               shape: BoxShape.circle,
198             ),
199             alignment: Alignment.center,
200             child: Text(
201               '${index + 1}',
202               style: TextStyleManager.bold12.copyWith(color: ColorsManager.pri
maryColor),
203             ),
204           ),
205           horizontalSpace6,
206           ConstrainedBox(
207             constraints: BoxConstraints(
208               maxWidth: MediaQuery.of(context).size.width * 0.6,
209             ),
210             child: Text(
211               lesson.title,
212               style: TextStyleManager.bold14.copyWith(color: ColorsManager.mai
nText),
213             overflow: TextOverflow.ellipsis,
214           ),
215         ],
216       ),
217     );
218   }).toList(),
219 ),
220 verticalSpace16,
221 if (context.read<ExamInfoCubit>().selectedBankQuestionIds.isNotEmpty) ...[
222   Container(
223     padding: const EdgeInsets.all(12),
224     decoration: BoxDecoration(
225       color: ColorsManager.primaryColor.withValues(alpha: 0.1),
226       borderRadius: BorderRadius.circular(12),
227     ),
228     child: Text(
229       'سؤال من بنك الأسئلة. سيتم دمجها مع الأرقام المحددة بالأسفل',
230       style: TextStyleManager.bold12.copyWith(color: ColorsManager.primaryColo
r),
231     ),
232   ),
233   verticalSpace16,
234 ],
235 Row(
236   mainAxisAlignment: MainAxisAlignment.spaceBetween,
237   children: [
238     Expanded(
239       child: Text(
240         'إجمالي الدروس: ${state.totalLessons}',
241

```

```

242         style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainTex
t),
243     ),
244 ),
245 Text(
246     'إجمالي الأسئلة: ${state.totalQuestions}',
247     style: TextStylesManager.bold14.copyWith(color: ColorsManager.primaryColo
r),
248 ),
249 ],
250 ),
251 verticalSpace16,
252 ...selectedLessons.map((lesson) {
253     final lessonId = lesson.id;
254     final counts = state.counts[lessonId];
255     if (counts == null) return const SizedBox.shrink();
256
257     return Card(
258         margin: const EdgeInsets.only(bottom: 16),
259         elevation: 0,
260         color: ColorsManager.surfacePrimary,
261         shape: RoundedRectangleBorder(
262             borderRadius: BorderRadius.circular(16),
263             side: BorderSide(color: ColorsManager.borderLightGray),
264         ),
265         child: Padding(
266             padding: const EdgeInsets.all(14),
267             child: Column(
268                 crossAxisAlignment: CrossAxisAlignment.start,
269                 children: [
270                     Row(
271                         mainAxisAlignment: MainAxisAlignment.spaceBetween,
272                         children: [
273                             Expanded(
274                                 child: Column(
275                                     crossAxisAlignment: CrossAxisAlignment.start,
276                                     children: [
277                                         Text(
278                                             lesson.title,
279                                             style: TextStylesManager.bold16.copyWith(color: ColorsMan
ager.mainText),
280                                             maxLines: 2,
281                                             overflow: TextOverflow.ellipsis,
282                                         ),
283                                         if (context.read<ExamInfoCubit>().selectedBankQuestionIds[1
essonId] != null &&
284                                             context.read<ExamInfoCubit>().selectedBankQuestionIds[1
essonId]!.isNotEmpty) ...[
285                                             verticalSpace4,
286                                             Text(
287                                                 '${context.read<ExamInfoCubit>().selectedBankQuestionId
s[lessonId]!.length} سؤال محدد من البنك',
288                                                 style: TextStylesManager.bold12.copyWith(color: ColorsM
anager.primaryColor),
289                                             ),
290                                         ],
291                                     ],
292                                 ),
293                             ),
294                             IconButton(
295                                 icon: const Icon(Icons.delete_outline, color: ColorsManager.err
orColor),
296                                 onPressed: () {
297                                     context.read<LessonSelectionCubit>().removeLesson(lessonId);
298                                     context.read<QuestionCountCubit>().initForLessons(
299                                         context.read<LessonSelectionCubit>().selectedLessons,
300                                     );
301                                 },
302                             ),
303                         ],
304                     ),
305                     const Divider(),
306                     verticalSpace8,
307                     Text(
308                         'أنواع الأسئلة والدرجات',

```

```

309         style: TextStyleManager.bold16.copyWith(color: ColorsManager.mainT
ext),
310     ),
311     verticalSpace12,
312     _buildCounterRow(context, lessonId, 'اختيار من متعدد', 'mcq', counts.
mcq),
313     _buildCounterRow(context, lessonId, 'صح أو خطأ', 'true_false', count
s.trueFalse),
314     _buildCounterRow(context, lessonId, 'أكمل الفراغ', 'fill_blank', coun
ts.fillBlank),
315     _buildCounterRow(context, lessonId, 'سؤال مقالي', 'essay', counts.ess
ay),
316     _buildCounterRow(context, lessonId, 'مطابقة', 'matching', counts.matc
hing),
317     ],
318   ),
319 ),
320 );
321 },
322   verticalSpace8,
323 ],
324 ),
325 ),
326   Container(
327     padding: const EdgeInsets.all(16),
328     decoration: BoxDecoration(
329       color: ColorsManager.surfacePrimary,
330       boxShadow: [
331         BoxShadow(color: Colors.black.withValues(alpha: 0.05), blurRadius: 10, offset:
const Offset(0, -5)),
332       ],
333     ),
334     child: PrimaryElevatedButton(
335       text: appTranslation().get('create_exam'),
336       onPressed: isGenerating ? null : _handleGenerate,
337     ),
338   ),
339 ],
340 );
341 },
342 ),
343
344 // Full screen loading
345 if (isGenerating)
346   Container(
347     color: ColorsManager.background.withValues(alpha: 0.9),
348     child: Center(
349       child: Column(
350         mainAxisAlignment: MainAxisAlignment.center,
351         children: [
352           CircularProgressIndicator(color: ColorsManager.primaryColor),
353           verticalSpace24,
354           Text(
355             '... جاري الانشاء الاختبار',
356             style: TextStyleManager.bold16.copyWith(color: ColorsManager.mainText),
357           ),
358           verticalSpace8,
359           Text(
360             'قد يستغرق هذا الإجراء دقيقة',
361             style: TextStyleManager.regular14.copyWith(color: ColorsManager.secondaryText),
362           ),
363         ],
364       ),
365     ),
366   ),
367 ],
368 );
369 },
370 ),
371 ),
372 );
373 }
374
375 Widget _buildCounterRow(BuildContext context, int lessonId, String label, String type, int value) {
376   const int defaultGrade = 1;

```

```

377
378 return Container(
379   margin: const EdgeInsets.only(bottom: 10),
380   padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 10),
381   decoration: BoxDecoration(
382     color: ColorsManager.surfacePrimary,
383     borderRadius: BorderRadius.circular(16),
384     border: Border.all(color: ColorsManager.borderLightGray),
385   ),
386   child: Row(
387     mainAxisAlignment: MainAxisAlignment.spaceBetween,
388     children: [
389       Expanded(
390         child: Column(
391           crossAxisAlignment: CrossAxisAlignment.start,
392           mainAxisAlignment: MainAxisAlignment.min,
393           children: [
394             Text(
395               label,
396               style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainText),
397               maxLines: 1,
398               overflow: TextOverflow.ellipsis,
399             ),
400             verticalSpace4,
401             Text(
402               'عدد الأسئلة',
403               style: TextStylesManager.regular12.copyWith(color: ColorsManager.secondaryText),
404             ),
405           ],
406         ),
407       ),
408       horizontalSpace8,
409       Row(
410         mainAxisAlignment: MainAxisAlignment.min,
411         children: [
412           _CounterButton(
413             icon: Icons.add,
414             onTap: () => context.read<QuestionCountCubit>().updateCount(lessonId, type, 1),
415             enabled: true,
416             isAdd: true,
417           ),
418           horizontalSpace6,
419           SizedBox(
420             width: 22,
421             child: Text(
422               value.toString(),
423               textAlign: TextAlign.center,
424               style: TextStylesManager.bold14.copyWith(color: ColorsManager.primaryColor),
425             ),
426           ),
427           horizontalSpace6,
428           _CounterButton(
429             icon: Icons.remove,
430             onTap: () => context.read<QuestionCountCubit>().updateCount(lessonId, type, -1),
431             enabled: value > 0,
432             isAdd: false,
433           ),
434           horizontalSpace8,
435           Container(
436             padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 6),
437             decoration: BoxDecoration(
438               color: ColorsManager.primaryColor.withValues(alpha: 0.15),
439               borderRadius: BorderRadius.circular(8),
440             ),
441             child: Text(
442               '$defaultGrade درجة',
443               style: TextStylesManager.bold12.copyWith(color: ColorsManager.primaryColor),
444             ),
445           ),
446         ],
447       ),
448     ],
449   ),
450 );
451 }

```



```

452 }
453
454 class _CounterButton extends StatelessWidget {
455   final IconData icon;
456   final VoidCallback onTap;
457   final bool enabled;
458   final bool isAdd;
459
460   const _CounterButton({required this.icon, required this.onTap, required this.enabled, this.isAdd = false});
461
462   @override
463   Widget build(BuildContext context) {
464     return InkWell(
465       onTap: enabled ? onTap : null,
466       borderRadius: BorderRadius.circular(8),
467       child: Container(
468         padding: const EdgeInsets.all(4),
469         decoration: BoxDecoration(
470           color: isAdd
471             ? ColorsManager.primaryColor
472             : ColorsManager.primaryColor.withValues(alpha: 0.1),
473           borderRadius: BorderRadius.circular(8),
474         ),
475       child: Icon(
476         icon,
477         size: 18,
478         color: isAdd
479           ? Colors.white
480           : (enabled ? ColorsManager.primaryColor : Colors.grey),
481       ),
482     ),
483   );
484 }
485 }
486

```

Lines • 374
Dart

Feature / Exam #190 -

Generation lib/features/exam_generation/presentation/widgets/add_manual_question_dialog.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6  import 'package:moean/core/utils/extensions/context_extension.dart';
7
8  class AddManualQuestionDialog extends StatefulWidget {
9    final int examId;
10    final List<int> availableLessonIds;
11    final Function(Map<String, dynamic> request) onSubmit;
12
13    const AddManualQuestionDialog({
14      super.key,
15      required this.examId,
16      required this.availableLessonIds,
17      required this.onSubmit,
18    });
19
20    @override
21    State<AddManualQuestionDialog> createState() => _AddManualQuestionDialogState();
22  }
23
24  class _AddManualQuestionDialogState extends State<AddManualQuestionDialog> {
25    final _formKey = GlobalKey<FormState>();
26    String _selectedType = 'mcq';
27    int? _selectedLessonId;
28    final TextEditingController _pointsController = TextEditingController(text: '1');
29    final TextEditingController _questionTextController = TextEditingController();
30    final TextEditingController _optionsController = TextEditingController();
31    final TextEditingController _columnAController = TextEditingController();
32    final TextEditingController _columnBController = TextEditingController();
33    final TextEditingController _correctAnswerController = TextEditingController();
34

```

```

35 final Map<String, String> _questionTypes = {
36   'mcq': 'اختيار من متعدد',
37   'true_false': 'صح أو خطأ',
38   'fill_blank': 'أكمل الفراغ',
39   'essay': 'مقالتي',
40   'matching': 'مطابقة',
41 };
42
43 @override
44 void initState() {
45   super.initState();
46   if (widget.availableLessonIds.isNotEmpty) {
47     _selectedLessonId = widget.availableLessonIds.first;
48   }
49 }
50
51 @override
52 void dispose() {
53   _pointsController.dispose();
54   _questionTextController.dispose();
55   _optionsController.dispose();
56   _columnAController.dispose();
57   _columnBController.dispose();
58   _correctAnswerController.dispose();
59   super.dispose();
60 }
61
62 void _submit() {
63   if (_formKey.currentState!.validate()) {
64     if (_selectedLessonId == null) {
65       ScaffoldMessenger.of(context).showSnackBar(const SnackBar(content: Text('الرجاء اختيار الدرس')));
66       return;
67     }
68
69     final request = <String, dynamic>{
70       'exam_id': widget.examId,
71       'lesson_id': _selectedLessonId,
72       'type': _selectedType,
73       'difficulty': 'medium',
74       'points': double.tryParse(_pointsController.text) ?? 1.0,
75       'question_text': _questionTextController.text,
76     };
77
78     if (_selectedType == 'mcq') {
79       final options = _optionsController.text.split('\n').map((e) => e.trim()).where((e) => e.isNotEmpty).toList();
80       if (options.length < 2) {
81         ScaffoldMessenger.of(context).showSnackBar(const SnackBar(content: Text('الرجاء إدخال خيارين على الأقل')));
82         return;
83       }
84       request['options'] = options;
85       request['correct_answer'] = _correctAnswerController.text;
86     } else if (_selectedType == 'true_false') {
87       request['correct_answer'] = _correctAnswerController.text; // Expected "صح" or "خطأ"
88     } else if (_selectedType == 'fill_blank') {
89       request['correct_answer'] = _correctAnswerController.text;
90     } else if (_selectedType == 'essay') {
91       request['correct_answer'] = _correctAnswerController.text;
92     } else if (_selectedType == 'matching') {
93       final columnALines = _columnAController.text.split('\n').map((e) => e.trim()).where((e) => e.isNotEmpty).toList();
94       final columnBLines = _columnBController.text.split('\n').map((e) => e.trim()).where((e) => e.isNotEmpty).toList();
95       request['options'] = {
96         'column_a': columnALines,
97         'column_b': columnBLines,
98       };
99       request['correct_answer'] = _correctAnswerController.text;
100     }
101
102     widget.onSubmit(request);
103     context.pop();
104   }
105 }

```

```

106 }
107
108 @override
109 Widget build(BuildContext context) {
110   return Dialog(
111     backgroundColor: ColorsManager.surfacePrimary,
112     shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
113     insetPadding: const EdgeInsets.all(16),
114     child: Container(
115       width: double.infinity,
116       padding: const EdgeInsets.all(20),
117       decoration: BoxDecoration(
118         color: ColorsManager.surfacePrimary,
119         borderRadius: BorderRadius.circular(16),
120       ),
121       child: Form(
122         key: _formKey,
123         child: Column(
124           mainAxisAlignment: MainAxisAlignment.min,
125           crossAxisAlignment: CrossAxisAlignment.stretch,
126           children: [
127             Row(
128               mainAxisAlignment: MainAxisAlignment.spaceBetween,
129               children: [
130                 Column(
131                   crossAxisAlignment: CrossAxisAlignment.start,
132                   children: [
133                     Text('إضافة سؤال جديد', style: TextStyleManager.bold12.copyWith(color: ColorsManager.p
primaryColor)),
134                     Text('سؤال يدوي داخل نفس الاختبار', style: TextStyleManager.bold18.copyWith(color: Colo
rsManager.mainText)),
135                   ],
136                 ),
137                 IconButton(
138                   icon: Icon(Icons.close, color: ColorsManager.secondaryText),
139                   onPressed: () => context.pop(),
140                 ),
141               ],
142             ),
143             verticalSpace16,
144             Flexible(
145               child: SingleChildScrollView(
146                 child: Column(
147                   crossAxisAlignment: CrossAxisAlignment.stretch,
148                   children: [
149                     Text('نوع السؤال', style: TextStyleManager.bold14.copyWith(color: ColorsManager.mainTe
xt)),
150                     verticalSpace8,
151                     Wrap(
152                       spacing: 8,
153                       runSpacing: 8,
154                       children: _questionTypes.entries.map((entry) {
155                         final isSelected = _selectedType == entry.key;
156                         return ChoiceChip(
157                           label: Text(entry.value, style: TextStyleManager.bold12.copyWith(color: isSelect
ed ? Colors.white : ColorsManager.mainText)),
158                           selected: isSelected,
159                           selectedColor: ColorsManager.primaryColor,
160                           backgroundColor: ColorsManager.surfacePrimary,
161                           side: BorderSide(color: isSelected ? ColorsManager.primaryColor : ColorsManager.b
orderLightGray),
162                           onSelect: (selected) {
163                             if (selected) setState(() => _selectedType = entry.key);
164                           },
165                         );
166                       }).toList(),
167                     ),
168                     verticalSpace16,
169                     Row(
170                       children: [
171                         Expanded(
172                           flex: 1,
173                           child: Column(
174                             crossAxisAlignment: CrossAxisAlignment.start,
175                             children: [

```

```

176 Text('الدرجة', style: TextStylesManager.bold14.copyWith(color: ColorsManager.
mainText)),
177 verticalSpace8,
178 TextFormField(
179   controller: _pointsController,
180   style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainText),
181   keyboardType: const TextInputType.numberWithOptions(decimal: true),
182   decoration: InputDecoration(
183     filled: true,
184     fillColor: ColorsManager.surfacePrimary,
185     border: OutlineInputBorder(borderRadius: BorderRadius.circular(12), borde
rSide: BorderSide(color: ColorsManager.borderLightGray)),
186     enabledBorder: OutlineInputBorder(borderRadius: BorderRadius.circular(1
2), borderSide: BorderSide(color: ColorsManager.borderLightGray)),
187     isDense: true,
188   ),
189   validator: (value) => value!.isEmpty ? 'مطلوب' : null,
190 ),
191 ],
192 ),
193 ),
194 horizontalSpace16,
195 Expanded(
196   flex: 2,
197   child: Column(
198     crossAxisAlignment: CrossAxisAlignment.start,
199     children: [
200       Text('الدرس المرتبط بالسؤال', style: TextStylesManager.bold14.copyWith(color:
ColorsManager.mainText)),
201       verticalSpace8,
202       DropdownButtonFormField<int>(
203         value: _selectedLessonId,
204         dropdownColor: ColorsManager.surfacePrimary,
205         decoration: InputDecoration(
206           filled: true,
207           fillColor: ColorsManager.surfacePrimary,
208           border: OutlineInputBorder(borderRadius: BorderRadius.circular(12), borde
rSide: BorderSide(color: ColorsManager.borderLightGray)),
209           enabledBorder: OutlineInputBorder(borderRadius: BorderRadius.circular(1
2), borderSide: BorderSide(color: ColorsManager.borderLightGray)),
210           isDense: true,
211         ),
212         items: widget.availableLessonIds.map((id) {
213           return DropdownMenuItem<int>(
214             value: id,
215             child: Text('الدرس $id', style: TextStylesManager.regular14.copyWith(co
lor: ColorsManager.mainText)),
216           );
217         }).toList(),
218         onChanged: (val) {
219           setState(() => _selectedLessonId = val);
220         },
221         validator: (value) => value == null ? 'مطلوب' : null,
222       ),
223     ],
224   ),
225 ),
226 ],
227 ),
228 verticalSpace16,
229 Text('نص السؤال', style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainTex
t)),
230 verticalSpace8,
231 TextFormField(
232   controller: _questionTextController,
233   style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainText),
234   maxLines: 3,
235   decoration: InputDecoration(
236     hintText: 'اكتب نص السؤال...',
237     hintStyle: TextStylesManager.regular14.copyWith(color: ColorsManager.secondaryTex
t),
238     filled: true,
239     fillColor: ColorsManager.surfacePrimary,
240     border: OutlineInputBorder(borderRadius: BorderRadius.circular(12), borderSide: Bor
derSide(color: ColorsManager.borderLightGray)),

```

```

241         enabledBorder: OutlineInputBorder(borderRadius: BorderRadius.circular(12), borderSi
de: BorderSide(color: ColorsManager.borderLightGray)),
242     ),
243     validator: (value) => value!.isEmpty ? 'مطلوب' : null,
244 ),
245     verticalSpace16,
246     if (_selectedType == 'mcq') ...[
247         Text('الاختيارات - كل اختيار في سطر', style: TextStylesManager.bold14.copyWith(color:
ColorsManager.mainText)),
248         verticalSpace8,
249         TextFormField(
250             controller: _optionsController,
251             style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainText),
252             maxLines: 4,
253             decoration: InputDecoration(
254                 hintText: 'الاختيار الرابع\nالاختيار الثالث\nالاختيار الثاني\nالاختيار الأول',
255                 hintStyle: TextStylesManager.regular14.copyWith(color: ColorsManager.secondaryTex
t),
256                 filled: true,
257                 fillColor: ColorsManager.surfacePrimary,
258                 border: OutlineInputBorder(borderRadius: BorderRadius.circular(12), borderSide: B
orderSide(color: ColorsManager.borderLightGray)),
259                 enabledBorder: OutlineInputBorder(borderRadius: BorderRadius.circular(12), border
Side: BorderSide(color: ColorsManager.borderLightGray)),
260             ),
261         ),
262         verticalSpace16,
263     ],
264     if (_selectedType == 'matching') ...[
265         Row(
266             children: [
267                 Expanded(
268                     child: Column(
269                         crossAxisAlignment: CrossAxisAlignment.start,
270                         children: [
271                             Text('العمود أ - كل عنصر في سطر', style: TextStylesManager.bold14.copyWith
(color: ColorsManager.mainText)),
272                             verticalSpace8,
273                             TextFormField(
274                                 controller: _columnAController,
275                                 style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainText),
276                                 maxLines: 4,
277                                 decoration: InputDecoration(
278                                     hintText: 'المفهوم الثاني\n2. المفهوم الأول',
279                                     hintStyle: TextStylesManager.regular14.copyWith(color: ColorsManager.se
condaryText),
280                                     filled: true,
281                                     fillColor: ColorsManager.surfacePrimary,
282                                     border: OutlineInputBorder(borderRadius: BorderRadius.circular(12), bor
derSide: BorderSide(color: ColorsManager.borderLightGray)),
283                                     enabledBorder: OutlineInputBorder(borderRadius: BorderRadius.circular(1
2), borderSide: BorderSide(color: ColorsManager.borderLightGray)),
284                                 ),
285                             ),
286                         ],
287                     ),
288                 ),
289                 horizontalSpace16,
290                 Expanded(
291                     child: Column(
292                         crossAxisAlignment: CrossAxisAlignment.start,
293                         children: [
294                             Text('العمود ب - كل عنصر في سطر', style: TextStylesManager.bold14.copyWith
(color: ColorsManager.mainText)),
295                             verticalSpace8,
296                             TextFormField(
297                                 controller: _columnBController,
298                                 style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainText),
299                                 maxLines: 4,
300                                 decoration: InputDecoration(
301                                     hintText: 'ج. مشتت إضافي\nب. الإجابة الثانية\nأ. الإجابة الأولى',
302                                     hintStyle: TextStylesManager.regular14.copyWith(color: ColorsManager.se
condaryText),
303                                     filled: true,
304                                     fillColor: ColorsManager.surfacePrimary,

```

```

305         border: OutlineInputBorder(borderRadius: BorderRadius.circular(12), bor
derSide: BorderSide(color: ColorsManager.borderLightGray)),
306         enabledBorder: OutlineInputBorder(borderRadius: BorderRadius.circular(1
2), borderSide: BorderSide(color: ColorsManager.borderLightGray)),
307     ),
308 ),
309 ],
310 ),
311 ),
312 ],
313 ),
314     verticalSpace16,
315 ],
316     if (_selectedType == 'true_false') ...[
317         Text('الإجابة الصحيحة', style: TextStylesManager.bold14.copyWith(color: ColorsManag
e.r.mainText)),
318         verticalSpace8,
319         DropdownButtonFormField<String>(
320             value: _correctAnswerController.text.isEmpty ? null : _correctAnswerController.te
xt,
321             dropdownColor: ColorsManager.surfacePrimary,
322             decoration: InputDecoration(
323                 filled: true,
324                 fillColor: ColorsManager.surfacePrimary,
325                 border: OutlineInputBorder(borderRadius: BorderRadius.circular(12), borderSide:
BorderSide(color: ColorsManager.borderLightGray)),
326                 enabledBorder: OutlineInputBorder(borderRadius: BorderRadius.circular(12), bord
erSide: BorderSide(color: ColorsManager.borderLightGray)),
327                 isDense: true,
328             ),
329             items: [
330                 DropdownMenuItem(value: 'صح', child: Text('صح', style: TextStylesManager.regula
r14.copyWith(color: ColorsManager.mainText))),
331                 DropdownMenuItem(value: 'خطأ', child: Text('خطأ', style: TextStylesManager.regu
lar14.copyWith(color: ColorsManager.mainText))),
332             ],
333             onChanged: (val) {
334                 setState(() => _correctAnswerController.text = val ?? '');
335             },
336             validator: (value) => value == null ? 'مطلوب' : null,
337         ),
338         verticalSpace16,
339     ] else ...[
340         Text('الإجابة الصحيحة / النموذجية', style: TextStylesManager.bold14.copyWith(color: Co
lorsManager.mainText)),
341         verticalSpace8,
342         TextFormField(
343             controller: _correctAnswerController,
344             style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainText),
345             maxLines: 2,
346             decoration: InputDecoration(
347                 hintText: _selectedType == 'matching' ? 'اكتب الإجابة الصحيح' : 'ب-2، أ-1 مثال:
...',
348                 hintStyle: TextStylesManager.regular14.copyWith(color: ColorsManager.secondaryTex
t),
349                 filled: true,
350                 fillColor: ColorsManager.surfacePrimary,
351                 border: OutlineInputBorder(borderRadius: BorderRadius.circular(12), borderSide: B
orderSide(color: ColorsManager.borderLightGray)),
352                 enabledBorder: OutlineInputBorder(borderRadius: BorderRadius.circular(12), border
Side: BorderSide(color: ColorsManager.borderLightGray)),
353             ),
354             validator: (value) => value!.isEmpty ? 'مطلوب' : null,
355         ),
356         verticalSpace16,
357     ],
358 ],
359 ),
360 ),
361 ),
362     verticalSpace16,
363     PrimaryElevatedButton(
364         text: 'إضافة السؤال',
365         onPressed: _submit,
366     ),

```

```

367         ],
368     ),
369 ),
370 ),
371 );
372 }
373 }
374

```

Lines • 209
Dart

Feature / Exam #191 -

Generation lib/features/exam_generation/presentation/widgets/exam_list_item_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6  import 'package:moean/features/exam_generation/domain/entities/exam_entities.dart';
7
8  class ExamListItemWidget extends StatelessWidget {
9      final ExamListEntity exam;
10     final double colTitle;
11     final double colStatus;
12     final double colQuestions;
13     final double colGrade;
14     final double colDate;
15     final double colActions;
16     final VoidCallback onPublish;
17     final VoidCallback onDelete;
18     final VoidCallback onView;
19     final VoidCallback? onTap;
20
21     const ExamListItemWidget({
22         super.key,
23         required this.exam,
24         required this.colTitle,
25         required this.colStatus,
26         required this.colQuestions,
27         required this.colGrade,
28         required this.colDate,
29         required this.colActions,
30         required this.onPublish,
31         required this.onDelete,
32         required this.onView,
33         this.onTap,
34     });
35
36     @override
37     Widget build(BuildContext context) {
38         return InkWell(
39             onTap: onTap ?? onView,
40             child: Container(
41                 height: 60,
42                 padding: const EdgeInsets.symmetric(horizontal: 0, vertical: 0),
43                 child: Row(
44                     children: [
45                         SizedBox(
46                             width: colTitle,
47                             child: Padding(
48                                 padding: const EdgeInsets.symmetric(horizontal: 16.0),
49                                 child: Align(
50                                     alignment: AlignmentDirectional.centerStart,
51                                     child: Text(
52                                         exam.title,
53                                         style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainText),
54                                         overflow: TextOverflow.ellipsis,
55                                     ),
56                                 ),
57                             ),
58                         ),
59                         SizedBox(
60                             width: colStatus,
61                             child: Padding(

```

```

62         padding: const EdgeInsets.symmetric(horizontal: 16.0),
63         child: Center(
64             child: _buildStatusBadge(),
65         ),
66     ),
67 ),
68 SizedBox(
69     width: colQuestions,
70     child: Padding(
71         padding: const EdgeInsets.symmetric(horizontal: 16.0),
72         child: Center(
73             child: Text(
74                 '${exam.questionsCount}',
75                 style: TextStyleManager.medium14.copyWith(color: ColorsManager.mainText),
76             ),
77         ),
78     ),
79 ),
80 SizedBox(
81     width: colGrade,
82     child: Padding(
83         padding: const EdgeInsets.symmetric(horizontal: 16.0),
84         child: Center(
85             child: Text(
86                 '${exam.totalPoints}',
87                 style: TextStyleManager.medium14.copyWith(color: ColorsManager.mainText),
88             ),
89         ),
90     ),
91 ),
92 SizedBox(
93     width: colDate,
94     child: Padding(
95         padding: const EdgeInsets.symmetric(horizontal: 16.0),
96         child: Center(
97             child: Text(
98                 _formatDate(exam.updatedAt),
99                 style: TextStyleManager.medium14.copyWith(color: ColorsManager.secondaryText),
100             ),
101         ),
102     ),
103 ),
104 SizedBox(
105     width: colActions,
106     child: Padding(
107         padding: const EdgeInsets.symmetric(horizontal: 8.0),
108         child: Row(
109             mainAxisAlignment: MainAxisAlignment.center,
110             children: [
111                 _buildActionButton(
112                     title: appTranslation().get('view'),
113                     icon: Icons.visibility_outlined,
114                     textColor: ColorsManager.primaryColor,
115                     iconColor: ColorsManager.primaryColor,
116                     backgroundColor: ColorsManager.primaryColor.withValues(alpha: 0.06),
117                     border: Border.all(color: ColorsManager.primaryColor.withValues(alpha: 0.35)),
118                     onTap: onView,
119                 ),
120                 if (exam.status == 'draft') ...[
121                     horizontalSpace6,
122                     _buildActionButton(
123                         title: appTranslation().get('publish'),
124                         icon: Icons.check_circle_outline,
125                         textColor: Colors.white,
126                         iconColor: Colors.white,
127                         backgroundColor: ColorsManager.primaryColor,
128                         onTap: onPublish,
129                     ),
130                 ],
131                 horizontalSpace6,
132                 _buildActionButton(
133                     title: appTranslation().get('delete'),
134                     icon: Icons.delete_outline,
135                     textColor: Colors.red,
136                     iconColor: Colors.red,

```



```

137         backgroundColor: Colors.red.withValues(alpha: 0.08),
138         onTap: onDelete,
139     ),
140 ],
141 ),
142 ),
143 ),
144 ],
145 ),
146 ),
147 );
148 }
149
150 Widget _buildStatusBadge() {
151     final isPublished = exam.status == 'published';
152     final color = isPublished ? ColorsManager.primaryColor : ColorsManager.secondaryText;
153     final text = appTranslation().get(isPublished ? 'published' : 'draft');
154
155     return Container(
156         padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
157         decoration: BoxDecoration(
158             color: color.withValues(alpha: 0.1),
159             borderRadius: BorderRadius.circular(20),
160         ),
161         child: Text(
162             text,
163             style: TextStylesManager.medium12.copyWith(color: color),
164             textAlign: TextAlign.center,
165         ),
166     );
167 }
168
169 Widget _buildActionButton({
170     required String title,
171     required IconData icon,
172     required Color textColor,
173     required Color iconColor,
174     required Color backgroundColor,
175     Border? border,
176     required VoidCallback onTap,
177 }) {
178     return GestureDetector(
179         onTap: onTap,
180         child: Container(
181             padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 6),
182             decoration: BoxDecoration(
183                 color: backgroundColor,
184                 borderRadius: BorderRadius.circular(20),
185                 border: border,
186             ),
187             child: Row(
188                 mainAxisAlignment: MainAxisAlignment.min,
189                 children: [
190                     Text(title, style: TextStylesManager.bold12.copyWith(color: textColor)),
191                     horizontalSpace4,
192                     Icon(icon, size: 16, color: iconColor),
193                 ],
194             ),
195         ),
196     );
197 }
198
199 String _formatDate(String dateStr) {
200     if (dateStr.isEmpty) return '';
201     try {
202         final date = DateTime.parse(dateStr);
203         return '${date.year}/${date.month}/${date.day}';
204     } catch (_) {
205         return dateStr;
206     }
207 }
208 }
209

```

```
1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/spacing.dart';
6 import 'package:moean/features/exam_generation/presentation/cubit/exam_info_cubit.dart';
7
8 class GenerationSourceSelector extends StatelessWidget {
9   const GenerationSourceSelector({super.key});
10
11   @override
12   Widget build(BuildContext context) {
13     return BlocBuilder<ExamInfoCubit, ExamInfoState>(
14       builder: (context, state) {
15         final cubit = context.read<ExamInfoCubit>();
16         return Column(
17           crossAxisAlignment: CrossAxisAlignment.start,
18           children: [
19             Text('طريقة بناء الاختبار', style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainText)),
20             verticalSpace8,
21             Row(
22               children: [
23                 Expanded(
24                   child: _SourceOptionCard(
25                     title: 'انشاء تلقائي',
26                     value: GenerationSource.aiOnly,
27                     groupValue: cubit.generationSource,
28                     onChanged: (val) => cubit.updateGenerationSource(val!),
29                   ),
30                 ),
31                 horizontalSpace8,
32                 Expanded(
33                   child: _SourceOptionCard(
34                     title: 'اختبار من بنك الأسئلة أولاً',
35                     value: GenerationSource.mixed,
36                     groupValue: cubit.generationSource,
37                     onChanged: (val) => cubit.updateGenerationSource(val!),
38                   ),
39                 ),
40               ],
41             ),
42           ],
43         );
44       },
45     );
46   }
47 }
48
49 class _SourceOptionCard extends StatelessWidget {
50   final String title;
51   final GenerationSource value;
52   final GenerationSource groupValue;
53   final ValueChanged<GenerationSource?> onChanged;
54
55   const _SourceOptionCard({
56     required this.title,
57     required this.value,
58     required this.groupValue,
59     required this.onChanged,
60   });
61
62   @override
63   Widget build(BuildContext context) {
64     final isSelected = value == groupValue;
65     return InkWell(
66       onTap: () => onChanged(value),
67       borderRadius: BorderRadius.circular(12),
68       child: Container(
69         padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 12),
70         decoration: BoxDecoration(
```



```

39         children: [
40             Expanded(
41                 child: PrimaryTextField(
42                     controller: cubit.optionsControllers[i],
43                     hint: appTranslation().get('option_1').replaceAll('1', '${i + 1}'),
44                 ),
45             ),
46             if (cubit.optionsControllers.length > 2) ...[
47                 horizontalSpace8,
48                 IconButton(
49                     icon: const Icon(Icons.close, color: Colors.red),
50                     onPressed: () => cubit.removeOption(i),
51                 ),
52             ],
53         ],
54     ),
55 ),
56 TextButton.icon(
57     onPressed: cubit.addOption,
58     icon: Icon(Icons.add, color: ColorsManager.primaryColor),
59     label: Text(
60         appTranslation().get('add_option'),
61         style: TextStyleManager.medium14.copyWith(color: ColorsManager.primaryColor),
62     ),
63 ),
64 ],
65 ),
66 ),
67 verticalSpace16,
68 ],
69
70 if (cubit.selectedQuestionType == 'matching') ...[
71     Row(
72         crossAxisAlignment: CrossAxisAlignment.start,
73         children: [
74             Expanded(
75                 child: Column(
76                     crossAxisAlignment: CrossAxisAlignment.start,
77                     children: [
78                         _buildSectionTitle(appTranslation().get('column_b_one_per_line')),
79                         verticalSpace12,
80                         Container(
81                             padding: const EdgeInsets.all(16),
82                             decoration: BoxDecoration(
83                                 border: Border.all(color: ColorsManager.borderColor),
84                                 borderRadius: BorderRadius.circular(12),
85                                 color: ColorsManager.background,
86                             ),
87                             child: Column(
88                                 children: [
89                                     for (int i = 0; i < cubit.columnBControllers.length; i++)
90                                         Padding(
91                                             padding: const EdgeInsets.only(bottom: 12),
92                                             child: Row(
93                                                 children: [
94                                                     Expanded(
95                                                         child: PrimaryTextField(
96                                                             controller: cubit.columnBControllers[i],
97                                                             hint: _getMatchingHint(i, true),
98                                                         ),
99                                                     ),
100                                                 ],
101                                             ),
102                                         ),
103                                     TextButton.icon(
104                                         onPressed: cubit.addMatchingItem,
105                                         icon: Icon(Icons.add, color: ColorsManager.primaryColor),
106                                         label: Text(
107                                             appTranslation().get('add_matching'),
108                                             style: TextStyleManager.medium14.copyWith(color: ColorsManager.primaryColor),
109                                         ),
110                                     ),
111                                 ],
112                             ),

```

```

113         ),
114     ],
115 ),
116 ),
117 horizontalSpace16,
118 Expanded(
119   child: Column(
120     crossAxisAlignment: CrossAxisAlignment.start,
121     children: [
122       _buildSectionTitle(appTranslation().get('column_a_one_per_line')),
123       verticalSpace12,
124       Container(
125         padding: const EdgeInsets.all(16),
126         decoration: BoxDecoration(
127           border: Border.all(color: ColorsManager.borderColor),
128           borderRadius: BorderRadius.circular(12),
129           color: ColorsManager.background,
130         ),
131         child: Column(
132           children: [
133             for (int i = 0; i < cubit.columnAControllers.length; i++)
134               Padding(
135                 padding: const EdgeInsets.only(bottom: 12),
136                 child: Row(
137                   children: [
138                     Expanded(
139                       child: PrimaryTextField(
140                         controller: cubit.columnAControllers[i],
141                         hint: _getMatchingHint(i, false),
142                       ),
143                     ),
144                     if (cubit.columnAControllers.length > 2) ...[
145                       horizontalSpace8,
146                       IconButton(
147                         icon: const Icon(Icons.close, color: Colors.red),
148                         onPressed: () => cubit.removeMatchingItem(i),
149                       ),
150                     ],
151                   ],
152                 ),
153             ),
154             // Invisible button to align with Column B
155             Opacity(
156               opacity: 0,
157               child: TextButton.icon(
158                 onPressed: null,
159                 icon: const Icon(Icons.add),
160                 label: const Text('Hidden'),
161               ),
162             ),
163           ],
164         ),
165       ),
166     ],
167   ),
168 ),
169 ],
170 ),
171 verticalSpace16,
172 ],
173
174 // Correct Answer Section
175 _buildSectionTitle(appTranslation().get('correct_answer_model')),
176 verticalSpace12,
177 _buildCorrectAnswerInput(cubit),
178 ],
179 );
180 },
181 );
182 }
183
184 Widget _buildSectionTitle(String title) {
185   return Text(
186     title,
187     style: TextStylesManager.bold14.copyWith(

```

```

188         color: ColorsManager.mainText,
189     ),
190 );
191 }
192
193 String _getMatchingHint(int index, bool isColumnB) {
194     if (isColumnB) {
195         if (index == 0) return appTranslation().get('answer_a');
196         if (index == 1) return appTranslation().get('answer_b');
197         return appTranslation().get('distractor_c');
198     } else {
199         if (index == 0) return appTranslation().get('concept_1');
200         if (index == 1) return appTranslation().get('concept_2');
201         return '${index + 1}. المفهوم';
202     }
203 }
204
205 Widget _buildCorrectAnswerInput(ManualExamCubit cubit) {
206     if (cubit.selectedQuestionType == 'true_false') {
207         return Container(
208             decoration: BoxDecoration(
209                 border: Border.all(color: ColorsManager.borderColor),
210                 borderRadius: BorderRadius.circular(12),
211                 color: ColorsManager.background,
212             ),
213             padding: const EdgeInsets.symmetric(horizontal: 16),
214             child: DropdownButtonHideUnderline(
215                 child: DropdownButton<String>(
216                     value: cubit.correctAnswerController.text.isEmpty ? null : cubit.correctAnswerController.text,
217                     hint: Text(
218                         appTranslation().get('write_correct_answer'),
219                         style: TextStylesManager.regular14.copyWith(color: ColorsManager.secondaryText),
220                     ),
221                     isExpanded: true,
222                     icon: Icon(Icons.keyboard_arrow_down_rounded, color: ColorsManager.mainText),
223                     items: [
224                         DropdownMenuItem(value: 'صح', child: Text(appTranslation().get('true_text'))),
225                         DropdownMenuItem(value: 'خطأ', child: Text(appTranslation().get('false_text'))),
226                     ],
227                     onChanged: (val) {
228                         if (val != null) {
229                             cubit.correctAnswerController.text = val;
230                             // Force rebuild
231                             // ignore: invalid_use_of_visible_for_testing_member, invalid_use_of_protected_member
232                             cubit.emit(ManualExamFormUpdated());
233                         }
234                     },
235                 ),
236             ),
237         );
238     }
239
240     String hintText = appTranslation().get('write_correct_answer');
241     if (cubit.selectedQuestionType == 'matching') {
242         hintText = appTranslation().get('matching_example');
243     }
244
245     return PrimaryTextField(
246         controller: cubit.correctAnswerController,
247         hint: hintText,
248         maxLines: cubit.selectedQuestionType == 'essay' ? 4 : 1,
249     );
250 }
251 }
252

```

Lines • 72
Dart

Feature / Exam #194 –
Generation

lib/features/exam_generation/presentation/widgets/manual_exam_type_selector.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';

```

```

5 import 'package:moean/core/utils/constants/spacing.dart';
6
7 class ManualExamTypeSelector extends StatelessWidget {
8   final String selectedType;
9   final ValueChanged<String> onTypeSelected;
10
11   const ManualExamTypeSelector({
12     super.key,
13     required this.selectedType,
14     required this.onTypeSelected,
15   });
16
17   @override
18   Widget build(BuildContext context) {
19     final types = [
20       {'id': 'mcq', 'label': appTranslation().get('mcq')},
21       {'id': 'true_false', 'label': appTranslation().get('true_false')},
22       {'id': 'fill_blank', 'label': appTranslation().get('fill_blank')},
23       {'id': 'essay', 'label': appTranslation().get('essay')},
24       {'id': 'matching', 'label': appTranslation().get('matching')},
25     ];
26
27     return Column(
28       crossAxisAlignment: CrossAxisAlignment.start,
29       children: [
30         Text(
31           appTranslation().get('question_type'),
32           style: TextStylesManager.bold14.copyWith(
33             color: ColorsManager.mainText,
34           ),
35         ),
36         verticalSpace8,
37         Wrap(
38           spacing: 8.0,
39           runSpacing: 8.0,
40           children: types.map((type) {
41             final isSelected = selectedType == type['id'];
42             return GestureDetector(
43               onTap: () => onTypeSelected(type['id']!),
44               child: AnimatedContainer(
45                 duration: const Duration(milliseconds: 200),
46                 padding: const EdgeInsets.symmetric(
47                   horizontal: 16.0,
48                   vertical: 10.0,
49                 ),
50                 decoration: BoxDecoration(
51                   color: isSelected ? ColorsManager.primaryColor : Colors.transparent,
52                   border: Border.all(
53                     color: isSelected ? ColorsManager.primaryColor : ColorsManager.borderColor,
54                     width: 1,
55                   ),
56                   borderRadius: BorderRadius.circular(12),
57                 ),
58                 child: Text(
59                   type['label']!,
60                   style: TextStylesManager.medium14.copyWith(
61                     color: isSelected ? Colors.white : ColorsManager.secondaryText,
62                   ),
63                 ),
64               ),
65             );
66           }).toList(),
67         ),
68       ],
69     );
70   }
71 }
72

```

```
1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/spacing.dart';
6
7 class MyExamsHeaderWidget extends StatelessWidget {
8   final String selectedTab;
9   final Function(String) onTabChanged;
10  final Function(String) onSearchChanged;
11
12  const MyExamsHeaderWidget({
13    super.key,
14    required this.selectedTab,
15    required this.onTabChanged,
16    required this.onSearchChanged,
17  });
18
19  @override
20  Widget build(BuildContext context) {
21    return Container(
22      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
23      decoration: BoxDecoration(
24        color: ColorsManager.surfacePrimary,
25        borderRadius: BorderRadius.circular(20),
26        border: Border.all(color: ColorsManager.borderLightGray),
27      ),
28      child: Column(
29        crossAxisAlignment: CrossAxisAlignment.start,
30        children: [
31          Container(
32            height: 48,
33            width: double.infinity,
34            decoration: BoxDecoration(
35              // color: ColorsManager.borderLightGray.withValues(alpha: 0.2),
36              borderRadius: BorderRadius.circular(12),
37            ),
38            child: TextField(
39              onChanged: onSearchChanged,
40              style: TextStyleManager.bold14.copyWith(color: ColorsManager.mainText),
41              decoration: InputDecoration(
42                hintText: appTranslation().get('search_exam_title'),
43                hintStyle: TextStyleManager.regular14.copyWith(color: ColorsManager.secondaryText),
44                prefixIcon: Icon(Icons.search, color: ColorsManager.secondaryText),
45                border: InputBorder.none,
46                contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
47              ),
48            ),
49          ),
50          verticalSpace16,
51          SingleChildScrollView(
52            scrollDirection: Axis.horizontal,
53            child: Row(
54              mainAxisAlignment: MainAxisAlignment.min,
55              children: [
56                _buildTab('all', appTranslation().get('all')),
57                horizontalSpace8,
58                _buildTab('draft', appTranslation().get('draft')),
59                horizontalSpace8,
60                _buildTab('published', appTranslation().get('published')),
61              ],
62            ),
63          ),
64        ],
65      ),
66    );
67  }
68
69  Widget _buildTab(String tabKey, String label) {
70    final isSelected = selectedTab == tabKey;
71    return GestureDetector(
```



```

72     onTap: () => onTabChanged(tabKey),
73     child: Container(
74       padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 12),
75       decoration: BoxDecoration(
76         color: isSelected ? ColorsManager.primaryColor : ColorsManager.surfacePrimary,
77         borderRadius: BorderRadius.circular(20),
78         border: Border.all(
79           color: isSelected ? ColorsManager.primaryColor : ColorsManager.borderLightGray,
80         ),
81       ),
82       child: Text(
83         label,
84         style: isSelected
85           ? TextStyleManager.bold14.copyWith(color: Colors.white)
86           : TextStyleManager.medium14.copyWith(color: ColorsManager.mainText),
87       ),
88     ),
89   );
90 }
91 }
92

```

Lines • 89
Dart

Feature / Forgot #196 -

Password [lib/features/forgot_password/presentation/cubit/forgot_password_cubit.dart](#)

```

1  import 'dart:async';
2  import 'package:flutter/material.dart';
3  import 'package:flutter_bloc/flutter_bloc.dart';
4  import 'package:moean/core/network/remote/api_service.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/features/forgot_password/presentation/cubit/forgot_password_state.dart';
7
8  class ForgotPasswordCubit extends Cubit<ForgotPasswordState> {
9    ForgotPasswordCubit() : super(const ForgotPasswordInitialState());
10
11    static ForgotPasswordCubit get(BuildContext context) =>
12      BlocProvider.of(context);
13
14    final GlobalKey<FormState> formKey = GlobalKey<FormState>();
15    final TextEditingController emailController = TextEditingController();
16
17    int cooldownSeconds = 0;
18    Timer? _cooldownTimer;
19    bool isSent = false;
20    bool isLoading = false;
21
22    void startCooldown([int seconds = 60]) {
23      cooldownSeconds = seconds;
24      _cooldownTimer?.cancel();
25      if (!isClosed) {
26        emit(ForgotPasswordCooldownTickState(cooldownSeconds: cooldownSeconds));
27      }
28      _cooldownTimer = Timer.periodic(const Duration(seconds: 1), (timer) {
29        if (cooldownSeconds <= 1) {
30          timer.cancel();
31          cooldownSeconds = 0;
32          if (!isClosed) {
33            emit(const ForgotPasswordCooldownTickState(cooldownSeconds: 0));
34          }
35        } else {
36          cooldownSeconds--;
37          if (!isClosed) {
38            emit(ForgotPasswordCooldownTickState(
39              cooldownSeconds: cooldownSeconds));
40          }
41        }
42      });
43    }
44
45    Future<void> sendResetLink() async {
46      if (formKey.currentState != null && !formKey.currentState!.validate()) {
47        return;
48      }
49

```

```

49   if (emailController.text.trim().isEmpty) return;
50   if (isLoading || cooldownSeconds > 0) return;
51
52   isLoading = true;
53   if (!isClosed) emit(const ForgotPasswordLoadingState());
54
55   try {
56     final result =
57       await ApiService.forgotPassword(emailController.text.trim());
58     if (isClosed) return;
59
60     result.fold(
61       (error) {
62         isLoading = false;
63         if (!isClosed) emit(ForgotPasswordErrorState(message: error));
64       },
65       (data) {
66         isLoading = false;
67         isSent = true;
68         startCooldown(60);
69         final message = data['message'] as String? ??
70           appTranslation().get('forgot_password_sent_title');
71         if (!isClosed) {
72           emit(ForgotPasswordSentSuccessState(message: message));
73         }
74       },
75     );
76   } catch (e) {
77     isLoading = false;
78     if (!isClosed) emit(ForgotPasswordErrorState(message: e.toString()));
79   }
80 }
81
82 @override
83 Future<void> close() {
84   _cooldownTimer?.cancel();
85   emailController.dispose();
86   return super.close();
87 }
88 }
89

```

Lines • 27
Dart

Feature / Forgot Password #197 -

lib/features/forgot_password/presentation/cubit/forgot_password_state.dart

```

1  abstract class ForgotPasswordState {
2    const ForgotPasswordState();
3  }
4
5  class ForgotPasswordInitialState extends ForgotPasswordState {
6    const ForgotPasswordInitialState();
7  }
8
9  class ForgotPasswordLoadingState extends ForgotPasswordState {
10   const ForgotPasswordLoadingState();
11 }
12
13 class ForgotPasswordSentSuccessState extends ForgotPasswordState {
14   final String message;
15   const ForgotPasswordSentSuccessState({required this.message});
16 }
17
18 class ForgotPasswordCooldownTickState extends ForgotPasswordState {
19   final int cooldownSeconds;
20   const ForgotPasswordCooldownTickState({required this.cooldownSeconds});
21 }
22
23 class ForgotPasswordErrorState extends ForgotPasswordState {
24   final String message;
25   const ForgotPasswordErrorState({required this.message});
26 }
27

```

```
1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/network/remote/api_service.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/features/forgot_password/presentation/cubit/reset_password_state.dart';
6
7 class ResetPasswordCubit extends Cubit<ResetPasswordState> {
8   final String email;
9   final String token;
10
11   ResetPasswordCubit({
12     required this.email,
13     required this.token,
14   }) : super(const ResetPasswordInitialState());
15
16   static ResetPasswordCubit get(BuildContext context) =>
17     BlocProvider.of(context);
18
19   final GlobalKey<FormState> formKey = GlobalKey<FormState>();
20   final TextEditingController passwordController = TextEditingController();
21   final TextEditingController confirmPasswordController =
22     TextEditingController();
23
24   bool obscurePassword = true;
25   bool obscureConfirmPassword = true;
26   bool isLoading = false;
27   bool isSuccess = false;
28
29   void togglePasswordVisibility() {
30     obscurePassword = !obscurePassword;
31     if (!isClosed) {
32       emit(ResetPasswordObscureToggledState(
33         obscurePassword: obscurePassword,
34         obscureConfirmPassword: obscureConfirmPassword,
35       ));
36     }
37   }
38
39   void toggleConfirmPasswordVisibility() {
40     obscureConfirmPassword = !obscureConfirmPassword;
41     if (!isClosed) {
42       emit(ResetPasswordObscureToggledState(
43         obscurePassword: obscurePassword,
44         obscureConfirmPassword: obscureConfirmPassword,
45       ));
46     }
47   }
48
49   Future<void> resetPassword() async {
50     if (!(formKey.currentState?.validate() ?? false)) return;
51     if (isLoading) return;
52
53     isLoading = true;
54     if (!isClosed) emit(const ResetPasswordLoadingState());
55
56     try {
57       final result = await ApiService.resetPassword(
58         email: email,
59         token: token,
60         password: passwordController.text,
61         passwordConfirmation: confirmPasswordController.text,
62       );
63       if (isClosed) return;
64
65       result.fold(
66         (error) {
67           isLoading = false;
68           if (!isClosed) emit(ResetPasswordErrorState(message: error));
69         },
70         (data) {
71           isLoading = false;
```

```

72         isSuccess = true;
73         final message = data['message'] as String? ??
74             appTranslation().get('reset_password_success_title');
75         if (!isClosed) {
76             emit(ResetPasswordSuccessState(message: message));
77         }
78     },
79 );
80 } catch (e) {
81     isLoading = false;
82     if (!isClosed) emit(ResetPasswordErrorState(message: e.toString()));
83 }
84 }
85
86 @override
87 Future<void> close() {
88     passwordController.dispose();
89     confirmPasswordController.dispose();
90     return super.close();
91 }
92 }
93

```

Lines • 31
Dart

Feature / Forgot #199 –

Password lib/features/forgot_password/presentation/cubit/reset_password_state.dart

```

1  abstract class ResetPasswordState {
2      const ResetPasswordState();
3  }
4
5  class ResetPasswordInitialState extends ResetPasswordState {
6      const ResetPasswordInitialState();
7  }
8
9  class ResetPasswordLoadingState extends ResetPasswordState {
10     const ResetPasswordLoadingState();
11 }
12
13 class ResetPasswordObscureToggledState extends ResetPasswordState {
14     final bool obscurePassword;
15     final bool obscureConfirmPassword;
16     const ResetPasswordObscureToggledState({
17         required this.obscurePassword,
18         required this.obscureConfirmPassword,
19     });
20 }
21
22 class ResetPasswordSuccessState extends ResetPasswordState {
23     final String message;
24     const ResetPasswordSuccessState({required this.message});
25 }
26
27 class ResetPasswordErrorState extends ResetPasswordState {
28     final String message;
29     const ResetPasswordErrorState({required this.message});
30 }
31

```

Lines • 90
Dart

Feature / Forgot #200 –

Password lib/features/forgot_password/presentation/screen/forgot_password_screen.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/utils/constants/spacing.dart';
5  import 'package:moean/features/forgot_password/presentation/cubit/forgot_password_cubit.dart';
6  import 'package:moean/features/forgot_password/presentation/cubit/forgot_password_state.dart';
7  import 'package:moean/features/forgot_password/presentation/widgets/forgot_password_badge_widget.dart';
8  import 'package:moean/features/forgot_password/presentation/widgets/forgot_password_form_widget.dart';
9  import 'package:moean/features/forgot_password/presentation/widgets/forgot_password_header_widget.dart';
10 import 'package:moean/features/forgot_password/presentation/widgets/forgot_password_sent_widget.dart';
11

```

```

12 class ForgotPasswordScreen extends StatelessWidget {
13   const ForgotPasswordScreen({super.key});
14
15   @override
16   Widget build(BuildContext context) {
17     return BlocProvider(
18       create: (_) => ForgotPasswordCubit(),
19       child: BlocListener<ForgotPasswordCubit, ForgotPasswordState>(
20         listener: _onStateChanged,
21         child: Scaffold(
22           backgroundColor: ColorsManager.background,
23           appBar: AppBar(
24             backgroundColor: Colors.transparent,
25             elevation: 0,
26             leading: IconButton(
27               onPressed: () => Navigator.pop(context),
28               icon: Icon(
29                 Icons.arrow_forward_ios_rounded,
30                 size: 16,
31                 color: ColorsManager.primaryColor,
32               ),
33             ),
34           ),
35           body: SafeArea(
36             child: Center(
37               child: SingleChildScrollView(
38                 physics: const BouncingScrollPhysics(),
39                 padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
40                 child: BlocBuilder<ForgotPasswordCubit, ForgotPasswordState>(
41                   buildWhen: (previous, current) =>
42                     current is ForgotPasswordSentSuccessState ||
43                     previous is ForgotPasswordSentSuccessState,
44                   builder: (context, state) {
45                     final cubit = ForgotPasswordCubit.get(context);
46                     if (cubit.isSent || state is ForgotPasswordSentSuccessState) {
47                       return const ForgotPasswordSentWidget();
48                     }
49
50                     return Column(
51                       crossAxisAlignment: CrossAxisAlignment.center,
52                       children: [
53                         const ForgotPasswordBadgeWidget(),
54                         verticalSpace20,
55                         const ForgotPasswordHeaderWidget(),
56                         verticalSpace24,
57                         const ForgotPasswordFormWidget(),
58                         verticalSpace20,
59                       ],
60                     );
61                   },
62                 ),
63               ),
64             ),
65           ),
66         ),
67       ),
68     );
69   }
70
71   void _onStateChanged(BuildContext context, ForgotPasswordState state) {
72     if (state is ForgotPasswordErrorState) {
73       ScaffoldMessenger.of(context).showSnackBar(
74         SnackBar(
75           content: Text(state.message),
76           backgroundColor: ColorsManager.errorColor,
77         ),
78       );
79     }
80     if (state is ForgotPasswordSentSuccessState) {
81       ScaffoldMessenger.of(context).showSnackBar(
82         SnackBar(
83           content: Text(state.message),
84           backgroundColor: ColorsManager.primaryColor,
85         ),
86       );

```

```
87     }
88   }
89 }
90
```

Lines • 111
Dart

Feature / Forgot #201 -

Password lib/features/forgot_password/presentation/screen/reset_password_screen.dart

```
1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/spacing.dart';
6 import 'package:moean/features/forgot_password/presentation/cubit/reset_password_cubit.dart';
7 import 'package:moean/features/forgot_password/presentation/cubit/reset_password_state.dart';
8 import 'package:moean/features/forgot_password/presentation/widgets/reset_password_form_widget.dart';
9 import 'package:moean/features/forgot_password/presentation/widgets/reset_password_header_widget.dart';
10 import 'package:moean/features/forgot_password/presentation/widgets/reset_password_success_widget.dart';
11
12 class ResetPasswordScreen extends StatelessWidget {
13   final String token;
14   final String email;
15
16   const ResetPasswordScreen({
17     super.key,
18     required this.token,
19     required this.email,
20   });
21
22   @override
23   Widget build(BuildContext context) {
24     return BlocProvider(
25       create: (_) => ResetPasswordCubit(token: token, email: email),
26       child: BlocListener<ResetPasswordCubit, ResetPasswordState>(
27         listener: _onStateChanged,
28         child: Scaffold(
29           backgroundColor: ColorsManager.background,
30           appBar: AppBar(
31             backgroundColor: Colors.transparent,
32             elevation: 0,
33             title: Text(
34               appTranslation().get('reset_password_appbar'),
35               style: const TextStyle(fontWeight: FontWeight.bold),
36             ),
37             centerTitle: true,
38             leading: BlocBuilder<ResetPasswordCubit, ResetPasswordState>(
39               buildWhen: (previous, current) =>
40                 current is ResetPasswordSuccessState ||
41                 previous is ResetPasswordSuccessState,
42               builder: (context, state) {
43                 final cubit = ResetPasswordCubit.get(context);
44                 if (cubit.isSuccess || state is ResetPasswordSuccessState) {
45                   return const SizedBox.shrink();
46                 }
47                 return IconButton(
48                   onPressed: () => Navigator.pop(context),
49                   icon: Icon(
50                     Icons.arrow_forward_ios_rounded,
51                     size: 16,
52                     color: ColorsManager.primaryColor,
53                   ),
54                 );
55               },
56             ),
57           ),
58           body: SafeArea(
59             child: Center(
60               child: SingleChildScrollView(
61                 physics: const BouncingScrollPhysics(),
62                 padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
63                 child: BlocBuilder<ResetPasswordCubit, ResetPasswordState>(
64                   buildWhen: (previous, current) =>
65                     current is ResetPasswordSuccessState ||
```

```

66         previous is ResetPasswordSuccessState,
67     builder: (context, state) {
68         final cubit = ResetPasswordCubit.get(context);
69         if (cubit.isSuccess || state is ResetPasswordSuccessState) {
70             return const ResetPasswordSuccessWidget();
71         }
72
73         return Column(
74             crossAxisAlignment: CrossAxisAlignment.center,
75             children: [
76                 ResetPasswordHeaderWidget(email: email),
77                 verticalSpace24,
78                 const ResetPasswordFormWidget(),
79                 verticalSpace20,
80             ],
81         );
82     },
83 ),
84 ),
85 ),
86 ),
87 ),
88 ),
89 );
90 }
91
92 void _onStateChanged(BuildContext context, ResetPasswordState state) {
93     if (state is ResetPasswordErrorState) {
94         ScaffoldMessenger.of(context).showSnackBar(
95             SnackBar(
96                 content: Text(state.message),
97                 backgroundColor: ColorsManager.errorColor,
98             ),
99         );
100     }
101     if (state is ResetPasswordSuccessState) {
102         ScaffoldMessenger.of(context).showSnackBar(
103             SnackBar(
104                 content: Text(state.message),
105                 backgroundColor: ColorsManager.primaryColor,
106             ),
107         );
108     }
109 }
110 }
111

```

Lines • 44
Dart

Feature / Forgot Password #202 -

lib/features/forgot_password/presentation/widgets/forgot_password_badge_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6
7  class ForgotPasswordBadgeWidget extends StatelessWidget {
8      const ForgotPasswordBadgeWidget({super.key});
9
10     @override
11     Widget build(BuildContext context) {
12         return Container(
13             padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 6),
14             decoration: BoxDecoration(
15                 color: ColorsManager.verifyMailBadgeBg,
16                 borderRadius: BorderRadius.circular(20),
17                 border: Border.all(
18                     color: ColorsManager.isDark
19                         ? const Color(0xFF5A481E)
20                         : const Color(0xFFFDE68A),
21                     width: 1,
22                 ),
23             ),

```

```

24     child: Row(
25       mainAxisAlignment: MainAxisAlignment.min,
26       children: [
27         const Icon(
28           Icons.vpn_key_outlined,
29           size: 16,
30           color: ColorsManager.verifyMailBadgeText,
31         ),
32         horizontalSpace6,
33         Text(
34           appTranslation().get('forgot_password_badge'),
35           style: TextStylesManager.bold13.copyWith(
36             color: ColorsManager.verifyMailBadgeText,
37           ),
38         ),
39       ],
40     ),
41   );
42 }
43 }
44

```

Lines • 79
Dart

Feature / Forgot #203 -

Password [lib/features/forgot_password/presentation/widgets/forgot_password_form_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
7  import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
8  import 'package:moean/core/utils/constants/spacing.dart';
9  import 'package:moean/features/forgot_password/presentation/cubit/forgot_password_cubit.dart';
10 import 'package:moean/features/forgot_password/presentation/cubit/forgot_password_state.dart';
11
12 class ForgotPasswordFormWidget extends StatelessWidget {
13   const ForgotPasswordFormWidget({super.key});
14
15   @override
16   Widget build(BuildContext context) {
17     final cubit = ForgotPasswordCubit.get(context);
18
19     return Form(
20       key: cubit.formKey,
21       child: Column(
22         crossAxisAlignment: CrossAxisAlignment.stretch,
23         children: [
24           Align(
25             alignment: AlignmentDirectional.centerStart,
26             child: Text(
27               appTranslation().get('forgot_password_email_label'),
28               style: TextStylesManager.bold14.copyWith(
29                 color: ColorsManager.textPrimary,
30               ),
31             ),
32           ),
33           verticalSpace8,
34           PrimaryTextField(
35             controller: cubit.emailController,
36             hint: appTranslation().get('forgot_password_email_hint'),
37             keyboardType: TextInputType.emailAddress,
38             textInputAction: TextInputAction.done,
39             suffixIcon: Icon(
40               Icons.mail_outline_rounded,
41               color: ColorsManager.placeholder,
42             ),
43             validator: (value) {
44               if (value == null || value.trim().isEmpty) {
45                 return appTranslation().get('forgot_password_email_required');
46               }
47               final emailRegex = RegExp(r'^[\\w-\\.]+@[\\w-]+\\.([\\w-]{2,4})$');
48               if (!emailRegex.hasMatch(value.trim())) {

```



```

49         return appTranslation().get('email_invalid');
50     }
51     return null;
52 },
53 onFieldSubmitted: (_) => cubit.sendResetLink(),
54 ),
55 verticalSpace24,
56 BlocBuilder<ForgotPasswordCubit, ForgotPasswordState>(
57   buildWhen: (previous, current) =>
58     current is ForgotPasswordLoadingState ||
59     current is ForgotPasswordErrorState ||
60     current is ForgotPasswordSentSuccessState,
61   builder: (context, state) {
62     return PrimaryElevatedButton(
63       text: appTranslation().get('forgot_password_send_btn'),
64       icon: const Icon(
65         Icons.vpn_key_outlined,
66         color: ColorsManager.white,
67         size: 20,
68       ),
69       isLoading: state is ForgotPasswordLoadingState,
70       onPressed: cubit.sendResetLink,
71     );
72   },
73 ),
74 ],
75 ),
76 );
77 }
78 }
79

```

Lines • 35
Dart

Feature / Forgot Password #204 -

[lib/features/forgot_password/presentation/widgets/forgot_password_header_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6
7  class ForgotPasswordHeaderWidget extends StatelessWidget {
8    const ForgotPasswordHeaderWidget({super.key});
9
10   @override
11   Widget build(BuildContext context) {
12     return Column(
13       crossAxisAlignment: CrossAxisAlignment.center,
14       children: [
15         Text(
16           appTranslation().get('forgot_password_title'),
17           style: TextStylesManager.bold26.copyWith(
18             color: ColorsManager.primaryColor,
19           ),
20         ),
21         Text(
22           appTranslation().get('forgot_password_subtitle'),
23           style: TextStylesManager.regular14.copyWith(
24             color: ColorsManager.placeholder,
25             height: 1.5,
26           ),
27         ),
28       ],
29     );
30   }
31 }
32
33
34
35

```

```
1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/spacing.dart';
7 import 'package:moean/features/forgot_password/presentation/cubit/forgot_password_cubit.dart';
8 import 'package:moean/features/forgot_password/presentation/cubit/forgot_password_state.dart';
9
10 class ForgotPasswordSentWidget extends StatelessWidget {
11   const ForgotPasswordSentWidget({super.key});
12
13   @override
14   Widget build(BuildContext context) {
15     final cubit = ForgotPasswordCubit.get(context);
16
17     return Column(
18       crossAxisAlignment: CrossAxisAlignment.center,
19       children: [
20         Icon(
21           Icons.mark_email_read_rounded,
22           size: 72,
23           color: ColorsManager.primaryColor,
24         ),
25         verticalSpace16,
26         Text(
27           appTranslation().get('forgot_password_sent_title'),
28           style: TextStylesManager.bold22.copyWith(
29             color: ColorsManager.primaryColor,
30           ),
31           textAlign: TextAlign.center,
32         ),
33         verticalSpace12,
34         Text(
35           '${appTranslation().get('forgot_password_sent_desc_prefix')}${cubit.emailController.text}${appTrans
lation().get('forgot_password_sent_desc_suffix')}',
36           style: TextStylesManager.regular14.copyWith(
37             color: ColorsManager.placeholder,
38             height: 1.5,
39           ),
40           textAlign: TextAlign.center,
41         ),
42         verticalSpace24,
43         BlocBuilder<ForgotPasswordCubit, ForgotPasswordState>(
44           buildWhen: (previous, current) =>
45             current is ForgotPasswordCooldownTickState ||
46             current is ForgotPasswordLoadingState,
47           builder: (context, state) {
48             final cooldown = cubit.cooldownSeconds;
49             final isCooldown = cooldown > 0;
50             final label = isCooldown
51               ? appTranslation()
52                 .get('forgot_password_resend_cooldown')
53                 .replaceAll('{seconds}', '$cooldown')
54               : appTranslation().get('forgot_password_resend_btn');
55
56             return SizedBox(
57               width: double.infinity,
58               height: 52,
59               child: OutlinedButton.icon(
60                 style: OutlinedButton.styleFrom(
61                   side: BorderSide(
62                     color: isCooldown
63                       ? ColorsManager.borderColor
64                       : ColorsManager.primaryColor,
65                   ),
66                   shape: RoundedRectangleBorder(
67                     borderRadius: BorderRadius.circular(24),
68                   ),
69                 ),
70                 icon: Icon(
```

```

71         Icons.refresh_rounded,
72         color: isCooldown
73         ? ColorsManager.placeholder
74         : ColorsManager.primaryColor,
75     ),
76     label: Text(
77       label,
78       style: TextStylesManager.bold14.copyWith(
79         color: isCooldown
80         ? ColorsManager.placeholder
81         : ColorsManager.primaryColor,
82       ),
83     ),
84     onPressed:
85       (isCooldown || cubit.isLoading) ? null : cubit.sendResetLink,
86   ),
87 );
88 },
89 ),
90 verticalSpace12,
91 TextButton(
92   onPressed: () => Navigator.pop(context),
93   child: Text(
94     appTranslation().get('forgot_password_back_to_login'),
95     style: TextStylesManager.medium14.copyWith(
96       color: ColorsManager.placeholder,
97     ),
98   ),
99 ),
100 ],
101 );
102 }
103 }
104

```

Lines • 141
Dart

Feature / Forgot Password #206 –

lib/features/forgot_password/presentation/widgets/reset_password_form_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
7  import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
8  import 'package:moean/core/utils/constants/spacing.dart';
9  import 'package:moean/features/forgot_password/presentation/cubit/reset_password_cubit.dart';
10 import 'package:moean/features/forgot_password/presentation/cubit/reset_password_state.dart';
11
12 class ResetPasswordFormWidget extends StatelessWidget {
13   const ResetPasswordFormWidget({super.key});
14
15   @override
16   Widget build(BuildContext context) {
17     final cubit = ResetPasswordCubit.get(context);
18
19     return Form(
20       key: cubit.formKey,
21       child: Column(
22         crossAxisAlignment: CrossAxisAlignment.stretch,
23         children: [
24           Align(
25             alignment: AlignmentDirectional.centerStart,
26             child: Text(
27               appTranslation().get('reset_password_new_label'),
28               style: TextStylesManager.bold14.copyWith(
29                 color: ColorsManager.textPrimary,
30               ),
31             ),
32           ),
33           verticalSpace8,
34           BlocBuilder<ResetPasswordCubit, ResetPasswordState>(
35             buildWhen: (previous, current) =>

```

```

36         current is ResetPasswordObscureToggledState,
37     builder: (context, state) {
38         return PrimaryTextField(
39             controller: cubit.passwordController,
40             hint: appTranslation().get('reset_password_new_hint'),
41             isPassword: cubit.obscurePassword,
42             textInputAction: TextInputAction.next,
43             prefixIcon: const Icon(
44                 Icons.lock_outline_rounded,
45                 size: 20,
46             ),
47             suffixIcon: IconButton(
48                 icon: Icon(
49                     cubit.obscurePassword
50                     ? Icons.visibility_off_outlined
51                     : Icons.visibility_outlined,
52                     color: ColorsManager.placeholder,
53                     size: 20,
54                 ),
55                 onPressed: cubit.togglePasswordVisibility,
56             ),
57             validator: (value) {
58                 if (value == null || value.trim().isEmpty) {
59                     return appTranslation().get('password_required');
60                 }
61                 if (value.length < 8) {
62                     return appTranslation().get('reset_password_min_length');
63                 }
64                 return null;
65             },
66         );
67     },
68 ),
69 verticalSpace16,
70 Align(
71     alignment: AlignmentDirectional.centerStart,
72     child: Text(
73         appTranslation().get('reset_password_confirm_label'),
74         style: TextStylesManager.bold14.copyWith(
75             color: ColorsManager.textPrimary,
76         ),
77     ),
78 ),
79 verticalSpace8,
80 BlocBuilder<ResetPasswordCubit, ResetPasswordState>(
81     buildWhen: (previous, current) =>
82         current is ResetPasswordObscureToggledState,
83     builder: (context, state) {
84         return PrimaryTextField(
85             controller: cubit.confirmPasswordController,
86             hint: appTranslation().get('reset_password_confirm_hint'),
87             isPassword: cubit.obscureConfirmPassword,
88             textInputAction: TextInputAction.done,
89             prefixIcon: const Icon(
90                 Icons.lock_outline_rounded,
91                 size: 20,
92             ),
93             suffixIcon: IconButton(
94                 icon: Icon(
95                     cubit.obscureConfirmPassword
96                     ? Icons.visibility_off_outlined
97                     : Icons.visibility_outlined,
98                     color: ColorsManager.placeholder,
99                     size: 20,
100                ),
101                onPressed: cubit.toggleConfirmPasswordVisibility,
102            ),
103            validator: (value) {
104                if (value == null || value.trim().isEmpty) {
105                    return appTranslation()
106                        .get('confirm_password_required');
107                }
108                if (value != cubit.passwordController.text) {
109                    return appTranslation().get('passwords_not_match');
110                }

```

```

111         return null;
112     },
113     onFieldSubmitted: (_) => cubit.resetPassword(),
114     );
115 },
116 ),
117 verticalSpace24,
118 BlocBuilder<ResetPasswordCubit, ResetPasswordState>(
119   buildWhen: (previous, current) =>
120     current is ResetPasswordLoadingState ||
121     current is ResetPasswordErrorState ||
122     current is ResetPasswordSuccessState,
123   builder: (context, state) {
124     return PrimaryElevatedButton(
125       text: appTranslation().get('reset_password_submit_btn'),
126       icon: const Icon(
127         Icons.vpn_key_outlined,
128         color: ColorsManager.white,
129         size: 20,
130       ),
131       isLoading: state is ResetPasswordLoadingState,
132       onPressed: cubit.resetPassword,
133     );
134   },
135 ),
136 ],
137 ),
138 );
139 }
140 }
141

```

Lines • 40
Dart

Feature / Forgot Password #207 -

lib/features/forgot_password/presentation/widgets/reset_password_header_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6
7  class ResetPasswordHeaderWidget extends StatelessWidget {
8    final String email;
9
10    const ResetPasswordHeaderWidget({
11      super.key,
12      required this.email,
13    });
14
15    @override
16    Widget build(BuildContext context) {
17      return Column(
18        crossAxisAlignment: CrossAxisAlignment.center,
19        children: [
20          Text(
21            appTranslation().get('reset_password_title'),
22            style: TextStylesManager.bold26.copyWith(
23              color: ColorsManager.primaryColor,
24            ),
25            textAlign: TextAlign.center,
26          ),
27          verticalSpace8,
28          Text(
29            '${appTranslation().get('reset_password_subtitle_prefix')} $email',
30            style: TextStylesManager.regular14.copyWith(
31              color: ColorsManager.placeholder,
32              height: 1.5,
33            ),
34            textAlign: TextAlign.center,
35          ),
36        ],
37      );
38    }

```

```
39 }
40
```

Lines • 51
Dart

Feature / Forgot

#208 -

Password

lib/features/forgot_password/presentation/widgets/reset_password_success_widget.dart

```
1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
6 import 'package:moean/core/utils/constants/routes.dart';
7 import 'package:moean/core/utils/constants/spacing.dart';
8 import 'package:moean/core/utils/extensions/context_extension.dart';
9
10 class ResetPasswordSuccessWidget extends StatelessWidget {
11   const ResetPasswordSuccessWidget({super.key});
12
13   @override
14   Widget build(BuildContext context) {
15     return Column(
16       crossAxisAlignment: CrossAxisAlignment.center,
17       children: [
18         Icon(
19           Icons.check_circle_rounded,
20           size: 80,
21           color: ColorsManager.primaryColor,
22         ),
23         verticalSpace16,
24         Text(
25           appTranslation().get('reset_password_success_title'),
26           style: TextStylesManager.bold22.copyWith(
27             color: ColorsManager.primaryColor,
28           ),
29           textAlign: TextAlign.center,
30         ),
31         verticalSpace12,
32         Text(
33           appTranslation().get('reset_password_success_subtitle'),
34           style: TextStylesManager.regular14.copyWith(
35             color: ColorsManager.placeholder,
36             height: 1.5,
37           ),
38           textAlign: TextAlign.center,
39         ),
40         verticalSpace28,
41         PrimaryElevatedButton(
42           text: appTranslation().get('reset_password_go_to_login'),
43           onPressed: () {
44             context.pushNamedAndRemoveUntil(Routes.login, (route) => false);
45           },
46         ),
47       ],
48     );
49   }
50 }
51
```

Lines • Dart 105

Feature / Guest

#209 - lib/features/guest/presentation/screen/guest_screen.dart

```
1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/assets_helper.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
7 import 'package:moean/core/utils/constants/routes.dart';
8 import 'package:moean/core/utils/constants/spacing.dart';
9 import 'package:moean/core/utils/extensions/context_extension.dart';
10
11 class GuestScreen extends StatelessWidget {
12   const GuestScreen({super.key});
13
14   @override
15   Widget build(BuildContext context) {
16     return Column(
17       crossAxisAlignment: CrossAxisAlignment.center,
18       children: [
19         Image.asset(AssetsHelper.guestScreenImage),
20         verticalSpace16,
21         Text(
22           appTranslation().get('guest_screen_title'),
23           style: TextStylesManager.bold22.copyWith(
24             color: ColorsManager.placeholder,
25           ),
26           textAlign: TextAlign.center,
27         ),
28         verticalSpace12,
29         Text(
30           appTranslation().get('guest_screen_subtitle'),
31           style: TextStylesManager.regular14.copyWith(
32             color: ColorsManager.placeholder,
33             height: 1.5,
34           ),
35           textAlign: TextAlign.center,
36         ),
37         verticalSpace28,
38         PrimaryElevatedButton(
39           text: appTranslation().get('guest_screen_go_to_login'),
40           onPressed: () {
41             context.pushNamedAndRemoveUntil(Routes.login, (route) => false);
42           },
43         ),
44       ],
45     );
46   }
47 }
48
```

```

13
14 @override
15 Widget build(BuildContext context) {
16   return Scaffold(
17     backgroundColor: ColorsManager.background,
18     body: SafeArea(
19       child: Padding(
20         padding: const EdgeInsets.symmetric(horizontal: 20),
21         child: Column(
22           crossAxisAlignment: CrossAxisAlignment.center,
23           children: [
24             verticalSpace80,
25             Text(
26               appTranslation().get('app_name'),
27               style: TextStylesManager.bold40.copyWith(
28                 color: ColorsManager.textPrimary,
29               ),
30             ),
31             verticalSpace30,
32             Expanded(
33               child: Center(
34                 child: ClipRRect(
35                   borderRadius: BorderRadius.circular(24),
36                   child: Image.asset(
37                     AssetsHelper.img1,
38                     width: double.infinity,
39                     fit: BoxFit.cover,
40                   ),
41                 ),
42               ),
43             ),
44             verticalSpace24,
45             Text(
46               appTranslation().get('login_welcome_title'),
47               style: TextStylesManager.bold26.copyWith(
48                 color: ColorsManager.textPrimary,
49               ),
50               textAlign: TextAlign.center,
51             ),
52             verticalSpace8,
53             Text(
54               appTranslation().get('login_welcome_subtitle'),
55               style: TextStylesManager.regular16.copyWith(
56                 color: ColorsManager.textSecondary,
57               ),
58               textAlign: TextAlign.center,
59             ),
60             verticalSpace30,
61             PrimaryElevatedButton(
62               text: appTranslation().get('login'),
63               textStyle: TextStylesManager.bold18.copyWith(
64                 color: Colors.white,
65               ),
66               radius: 24,
67               onPressed: () {
68                 context.push<void>(Routes.login);
69               },
70             ),
71             verticalSpace16,
72             SizedBox(
73               width: double.infinity,
74               height: 52,
75               child: OutlinedButton(
76                 style: OutlinedButton.styleFrom(
77                   side: BorderSide(
78                     color: ColorsManager.primaryAction,
79                     width: 1.5,
80                   ),
81                   shape: RoundedRectangleBorder(
82                     borderRadius: BorderRadius.circular(24),
83                   ),
84                 ),
85                 onPressed: () {
86                   context.push<void>(Routes.register);
87                 },

```

```

88         child: Text(
89             appTranslation().get('create_account'),
90             style: TextStyleManager.bold18.copyWith(
91                 color: ColorsManager.primaryAction,
92             ),
93         ),
94     ),
95 ),
96
97     verticalSpace20,
98 ],
99 ),
100 ),
101 ),
102 );
103 }
104 }
105

```

Lines • Dart 165

Feature / Home

#210 – lib/features/home/presentation/cubit/extension_install_cubit.dart

```

1  import 'dart:io';
2
3  import 'package:android_intent_plus/android_intent.dart';
4  import 'package:android_intent_plus/flag.dart';
5  import 'package:flutter/material.dart';
6  import 'package:flutter/services.dart';
7  import 'package:flutter_bloc/flutter_bloc.dart';
8  import 'package:url_launcher/url_launcher.dart';
9
10 part 'extension_install_state.dart';
11
12 /// Quetta Browser Android package identifier.
13 const String _quettaPackageName = 'net.quetta.browser';
14
15 /// Haddher Chrome Web Store URL.
16 const String _extensionUrl =
17     'https://chromewebstore.google.com/detail/khpdhfpnnjimoeffjecnfpiokemdlabn';
18
19 /// Madrasati website URL.
20 const String _madrasatiUrl = 'https://schools.madrasati.sa/';
21
22 /// Play Store URL for Quetta Browser.
23 const String _quettaPlayStoreUrl =
24     'https://play.google.com/store/apps/details?id=$_quettaPackageName';
25
26 class ExtensionInstallCubit extends Cubit<ExtensionInstallState>
27     with WidgetsBindingObserver {
28     ExtensionInstallCubit() : super(const ExtensionInstallReady(isQuettaInstalled: false)) {
29         WidgetsBinding.instance.addObserver(this);
30         _checkQuettaStatus();
31     }
32
33     static ExtensionInstallCubit get(BuildContext context) =>
34         BlocProvider.of(context);
35
36     // _____
37     // Lifecycle observer
38     // _____
39
40     /// Called by Flutter whenever the app lifecycle changes.
41     /// On [AppLifecycleState.resumed] we check if Quetta was just installed
42     /// so the UI can dynamically update the "Copy Link" to "Open in Quetta".
43     @override
44     void didChangeAppLifecycleState(AppLifecycleState state) {
45         if (state == AppLifecycleState.resumed) {
46             _checkQuettaStatus();
47         }
48     }
49
50     // _____
51     // Package check (reliable)
52     // _____

```



```

53
54 Future<void> _checkQuettaStatus() async {
55     if (Platform.isIOS) {
56         if (!isClosed) emit(const ExtensionInstallIosNotSupported(isQuettaInstalled: false));
57         return;
58     }
59
60     try {
61         final AndroidIntent checkIntent = const AndroidIntent(
62             action: 'android.intent.action.MAIN',
63             category: 'android.intent.category.LAUNCHER',
64             package: _quettaPackageName,
65         );
66         final bool isInstalled = await checkIntent.canResolveActivity() ?? false;
67         if (!isClosed) emit(ExtensionInstallReady(isQuettaInstalled: isInstalled));
68     } catch (_) {
69         if (!isClosed) emit(const ExtensionInstallReady(isQuettaInstalled: false));
70     }
71 }
72
73 // _____
74 // Public API
75 // _____
76
77 /// Step 1: Open Play Store for Quetta Browser.
78 Future<void> downloadQuetta() async {
79     try {
80         final AndroidIntent intent = const AndroidIntent(
81             action: 'action_view',
82             data: 'market://details?id=$_quettaPackageName',
83             package: 'com.android.vending',
84             flags: <int>[Flag.FLAG_ACTIVITY_NEW_TASK],
85         );
86
87         final bool canResolve = await intent.canResolveActivity() ?? false;
88         if (canResolve) {
89             await intent.launch();
90         } else {
91             // Fallback to https web URL if Play Store app is missing
92             final Uri webUri = Uri.parse(_quettaPlayStoreUrl);
93             if (await canLaunchUrl(webUri)) {
94                 await launchUrl(webUri, mode: LaunchMode.externalApplication);
95             } else {
96                 if (!isClosed) {
97                     emit(ExtensionInstallError(
98                         message: 'Could not open Play Store',
99                         isQuettaInstalled: state.isQuettaInstalled,
100                     ));
101                 }
102             }
103         }
104     } catch (_) {
105         if (!isClosed) {
106             emit(ExtensionInstallError(
107                 message: 'Could not open Play Store',
108                 isQuettaInstalled: state.isQuettaInstalled,
109             ));
110         }
111     }
112 }
113
114 /// Step 2 (Option A): Copy extension link to clipboard.
115 Future<void> copyExtensionLink() async {
116     await Clipboard.setData(const ClipboardData(text: _extensionUrl));
117 }
118
119 /// Step 2 (Option B): Launch the extension URL inside Quetta explicitly.
120 Future<void> openExtensionInQuetta() async {
121     try {
122         final AndroidIntent intent = const AndroidIntent(
123             action: 'action_view',
124             data: _extensionUrl,
125             package: _quettaPackageName,
126             flags: <int>[Flag.FLAG_ACTIVITY_NEW_TASK],
127         );

```

```

128     await intent.launch();
129   } catch (_) {
130     if (!isClosed) {
131       emit(ExtensionInstallError(
132         message: 'Could not launch Quetta',
133         isQuettaInstalled: state.isQuettaInstalled,
134       ));
135     }
136   }
137 }
138
139 /// Step 3: Open Madrasati website inside Quetta browser.
140 Future<void> openMadrasatiInQuetta() async {
141   try {
142     final AndroidIntent intent = const AndroidIntent(
143       action: 'action_view',
144       data: _madrasatiUrl,
145       package: _quettaPackageName,
146       flags: <int>[Flag.FLAG_ACTIVITY_NEW_TASK],
147     );
148     await intent.launch();
149   } catch (_) {
150     if (!isClosed) {
151       emit(ExtensionInstallError(
152         message: 'Could not launch Quetta',
153         isQuettaInstalled: state.isQuettaInstalled,
154       ));
155     }
156   }
157 }
158
159 @override
160 Future<void> close() {
161   WidgetsBinding.instance.removeObserver(this);
162   return super.close();
163 }
164 }
165

```

Lines • Dart 26

Feature / Home

#211 – lib/features/home/presentation/cubit/extension_install_state.dart

```

1 part of 'extension_install_cubit.dart';
2
3 abstract class ExtensionInstallState {
4   final bool isQuettaInstalled;
5   const ExtensionInstallState({this.isQuettaInstalled = false});
6 }
7
8 /// Default state tracking whether Quetta is installed.
9 class ExtensionInstallReady extends ExtensionInstallState {
10   const ExtensionInstallReady({required super.isQuettaInstalled});
11 }
12
13 /// Running on iOS – show the "desktop-only" dialog.
14 class ExtensionInstallIosNotSupported extends ExtensionInstallState {
15   const ExtensionInstallIosNotSupported({super.isQuettaInstalled});
16 }
17
18 /// Unexpected error during the flow.
19 class ExtensionInstallError extends ExtensionInstallState {
20   final String message;
21   const ExtensionInstallError({
22     required this.message,
23     required super.isQuettaInstalled,
24   });
25 }
26

```

Lines • Dart 68

Feature / Home

#212 – lib/features/home/presentation/cubit/home_cubit.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';

```

```

3 import 'package:moean/core/network/remote/api_service.dart';
4 import 'package:moean/core/network/local/cache_helper.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6
7 part 'home_state.dart';
8
9 class HomeCubit extends Cubit<HomeState> {
10   HomeCubit() : super(HomeInitial());
11
12   static HomeCubit get(BuildContext context) => BlocProvider.of(context);
13
14   int selectedCategoryIndex = 0;
15   final TextEditingController searchController = TextEditingController();
16
17   bool isAdmin = false;
18   bool isLoadingRole = true;
19
20   void onSearchChanged(String query) {
21     if (!isClosed) emit(HomeSearchChanged(query: query));
22   }
23
24   void onCategorySelected(int index) {
25     selectedCategoryIndex = index;
26     if (!isClosed) emit(HomeCategoryChanged(selectedIndex: index));
27   }
28
29   Future<void> checkRole() async {
30     if (token != null && token!.isNotEmpty) {
31       final cachedIsAdmin = CacheHelper.getData(key: 'isAdmin');
32       final cachedToken = CacheHelper.getData(key: 'admin_token');
33       if (cachedIsAdmin != null && cachedIsAdmin is bool && cachedToken == token) {
34         isAdmin = cachedIsAdmin;
35         isLoadingRole = false;
36         if (!isClosed) emit(HomeRoleChecked(isAdmin: isAdmin));
37         return;
38       }
39
40       final result = await ApiService.getProfile();
41       if (isClosed) return;
42       result.fold(
43         (error) {
44           isLoadingRole = false;
45           if (!isClosed) emit(HomeRoleChecked(isAdmin: isAdmin)); // emit with current/default
46         },
47         (profile) {
48           final admin = profile.role == 'admin' || profile.user.email == 'admin@moeen.sa';
49           CacheHelper.saveData(key: 'isAdmin', value: admin);
50           CacheHelper.saveData(key: 'admin_token', value: token);
51           isAdmin = admin;
52           isLoadingRole = false;
53           if (!isClosed) emit(HomeRoleChecked(isAdmin: isAdmin));
54         },
55       );
56     } else {
57       isLoadingRole = false;
58       if (!isClosed) emit(HomeRoleChecked(isAdmin: false));
59     }
60   }
61
62   @override
63   Future<void> close() {
64     searchController.dispose();
65     return super.close();
66   }
67 }
68

```

```

1 part of 'home_cubit.dart';
2
3 abstract class HomeState {}
4

```

```

5 class HomeInitial extends HomeState {}
6
7 class HomeSearchChanged extends HomeState {
8   final String query;
9   HomeSearchChanged({required this.query});
10 }
11
12 class HomeCategoryChanged extends HomeState {
13   final int selectedIndex;
14   HomeCategoryChanged({required this.selectedIndex});
15 }
16
17 class HomeRoleChecked extends HomeState {
18   final bool isAdmin;
19   HomeRoleChecked({required this.isAdmin});
20 }
21

```

Lines • Dart 322

Feature / Home

#214 – lib/features/home/presentation/screen/home_screen.dart

```

1 import 'package:flutter/foundation.dart';
2 import 'package:flutter/material.dart';
3 import 'package:flutter_bloc/flutter_bloc.dart';
4 import 'package:moean/core/di/injections.dart';
5 import 'package:moean/core/models/madrasati_session_data.dart';
6 import 'package:moean/core/network/local/secure_storage_helper.dart';
7 import 'package:moean/core/network/remote/api_endpoints.dart';
8 import 'package:moean/core/network/remote/dio_helper.dart';
9 import 'package:moean/core/theme/colors.dart';
10 import 'package:moean/core/utils/constants/spacing.dart';
11 import 'package:moean/core/utils/cubit/theme/theme_cubit.dart';
12 import 'package:moean/core/utils/cubit/theme/theme_state.dart';
13 import 'package:moean/core/widgets/dot_gridPainter.dart';
14 import 'package:moean/core/widgets/session_expired_dialog.dart';
15 import 'package:moean/features/home/presentation/cubit/home_cubit.dart';
16 import 'package:moean/features/home/presentation/widgets/home_app_bar_widget.dart';
17 import 'package:moean/features/home/presentation/widgets/home_hero_banner_widget.dart';
18 import 'package:moean/features/home/presentation/widgets/home_tip_of_day_widget.dart';
19 import 'package:moean/features/home/presentation/widgets/home_features_section_widget.dart';
20
21 class HomeScreen extends StatelessWidget {
22   const HomeScreen({super.key});
23
24   @override
25   Widget build(BuildContext context) {
26     return BlocProvider(
27       create: (_) => HomeCubit()..checkRole(),
28       child: BlocBuilder<ThemeCubit, ThemeState>(
29         builder: (context, themeState) {
30           return Scaffold(
31             backgroundColor: ColorsManager.background,
32             body: Directionality(
33               textDirection: TextDirection.rtl,
34               child: Stack(
35                 children: [
36                   Positioned(
37                     right: 20,
38                     top: 200,
39                     height: 150,
40                     width: 140,
41                     child: CustomPaint(
42                       painter: DotGridPainter(
43                         color: ColorsManager.primaryColor.withValues(alpha: 0.15),
44                         spacing: 16.0,
45                       ),
46                     ),
47                   ),
48                   SafeArea(
49                     child: SingleChildScrollView(
50                       physics: const BouncingScrollPhysics(),
51                       child: Padding(
52                         padding: const EdgeInsets.symmetric(horizontal: 8.0),
53                       child: Column(

```

```

54         crossAxisAlignment: CrossAxisAlignment.stretch,
55         children: [
56             // ignore: prefer_const_constructors
57             HomeAppBarWidget(),
58             verticalSpace24,
59             // ignore: prefer_const_constructors
60             HomeHeroBannerWidget(),
61             verticalSpace40,
62             // ignore: prefer_const_constructors
63             HomeFeaturesSectionWidget(),
64             verticalSpace40,
65             // ignore: prefer_const_constructors
66             HomeTipOfDayWidget(),
67             // — DEBUG ONLY: Session Expiry Tester —————
68             // Visible ONLY in debug mode. Safe to leave in code.
69             if (kDebugMode) ...[
70                 verticalSpace24,
71                 _DebugSessionExpireButton(),
72                 verticalSpace24,
73             ],
74         ],
75     ),
76 ),
77 ),
78 ),
79 ],
80 ),
81 ),
82 );
83 },
84 ),
85 );
86 }
87 }
88
89 /// DEBUG ONLY — directly calls /madrasati/refresh-session using saved credentials.
90 /// Only rendered when [kDebugMode] is true (never in production builds).
91 class _DebugSessionExpireButton extends StatefulWidget {
92   @override
93   State<_DebugSessionExpireButton> createState() =>
94     _DebugSessionExpireButtonState();
95 }
96
97 class _DebugSessionExpireButtonState extends State<_DebugSessionExpireButton> {
98   bool _loading = false;
99
100   Future<void> _triggerRefresh() async {
101     setState(() => _loading = true);
102
103     debugPrint('—————');
104     debugPrint('🔧 [DEBUG] Manual refresh-session triggered from HomeScreen');
105
106     // — Step 1: اقرأ الجلسة المحفوظة —————
107     final savedSession = await sl<SecureStorageHelper>().getMadrasatiSession();
108     final refreshToken = savedSession?.refreshToken;
109     final schoolId = savedSession?.schoolId;
110
111     debugPrint('📁 Saved credentials:');
112     debugPrint('  schoolId      : ${schoolId ?? '❌ null'}');
113     debugPrint('  refreshToken : ${refreshToken != null && refreshToken.isNotEmpty ? '✅ "${refreshToken.substring(0, refreshToken.length.clamp(0, 30))}"' : '❌ null or empty'}');
114
115     // — Step 2: connect محفوظة → دايبلوج لازم تعمل session لو مفيش —————
116     if (refreshToken == null || refreshToken.isEmpty ||
117         schoolId == null || schoolId.isEmpty) {
118       debugPrint('⚠️ [DEBUG] No local session — showing SessionExpiredDialog (must re-connect)');
119       debugPrint('—————');
120
121       setState(() => _loading = false);
122       if (mounted) SessionExpiredDialog.show(context);
123       return;
124     }
125
126     // — Step 3: session نادی /refresh-session —————

```

```

126 try {
127     debugPrint('🐞 [DEBUG] POST $refreshMadrasatiSessionApi');
128     final response = await DioHelper.postData(
129         url: refreshMadrasatiSessionApi,
130         data: {
131             'refresh_token': refreshToken,
132             'madrasati_school_id': schoolId,
133         },
134     );
135
136     response.fold(
137         // — Left: خطأ من DioHelper —————
138         (error) {
139             debugPrint('🔴 [DEBUG] Request failed (LEFT): $error');
140             debugPrint('—————');
141
142             // (منتهية session) خطأ مصادقة vs نفذ: خطأ شبكة (انقطاع)
143             final errLow = error.toLowerCase();
144             final isNetworkError = errLow.contains('socket') ||
145                 errLow.contains('connection') ||
146                 errLow.contains('timeout') ||
147                 errLow.contains('network') ||
148                 errLow.contains('unreachable') ||
149                 errLow.contains('reset');
150
151             if (isNetworkError) {
152                 // 🚫 انقطع الإنترنت - مش نبوع الدايالوج
153                 debugPrint('🔴 [DEBUG] Network error detected - NOT showing dialog');
154                 if (mounted) {
155                     ScaffoldMessenger.of(context).showSnackBar(
156                         const SnackBar(
157                             content: Text('🔴 تحقق من اتصال الإنترنت وحاول مجدداً'),
158                             backgroundColor: Colors.orange,
159                             duration: Duration(seconds: 4),
160                         ),
161                     );
162                 }
163             } else {
164                 // ❌ من أول connect خطأ مصادقة - لازم يعمل
165                 debugPrint('🔴 [DEBUG] Auth error - showing SessionExpiredDialog');
166                 if (mounted) SessionExpiredDialog.show(context);
167             }
168         },
169     );
170
171     // — Right: جه رد من الباك —————
172     (res) async {
173         debugPrint('');
174         debugPrint('📦 /refresh-session RESPONSE - Full Breakdown');
175         debugPrint('');
176         debugPrint('HTTP statusCode : ${res.statusCode}');
177         debugPrint('');
178         debugPrint(' [top-level]');
179         debugPrint('    success : ${res.data['success']}');
180         debugPrint('    message : ${res.data['message']}');
181         debugPrint('');
182         debugPrint(' [data] object:');
183
184         final dataObj = res.data['data'];
185         if (dataObj == null) {
186             debugPrint('⚠️ data is NULL - backend returned no data object');
187         } else {
188             final d = dataObj as Map;
189             final cookie = d['session_cookie'] as String?;
190             debugPrint('session_cookie : ${cookie != null ? '✅' : '❌'} "${cookie?.substring(0, cookie?.length.clamp(0, 30))}" : '❌ null'');
191             final newRT = d['new_refresh_token'] as String?;
192             debugPrint('new_refresh_token : ${newRT != null ? '✅' : '❌'} "${newRT?.substring(0, newRT?.length.clamp(0, 30))}" : '❌ null - old token reused'');
193             final rt = d['refresh_token'] as String?;
194             debugPrint('refresh_token : ${rt != null ? '✅' : '❌'} "${rt?.substring(0, rt?.length.clamp(0, 30))}" : '❌ null'');
195             final exp = d['expires_at'];
196             debugPrint('expires_at : ${exp ?? '❌ null'}');

```

```

197         final known = {'session_cookie', 'new_refresh_token', 'refresh_token', 'expires_at'};
198         final extras = d.keys.where((k) => !known.contains(k)).toList();
199         if (extras.isNotEmpty) debugPrint('    [extra fields] : $extras');
200     }
201     debugPrint('_____');
202     debugPrint('_____');
203 ');
204
205     final success = res.data['success'] == true;
206     final message = res.data['message'] as String? ?? '';
207
208     if (success) {
209         // نجحت - نحفظ الجلسة الجديدة ✓
210         final d = res.data['data'] as Map<String, dynamic>;
211         if (d != null) {
212             final newCookie = d['session_cookie'] as String?;
213             final newRT = (d['new_refresh_token'] ?? d['refresh_token']) as String?;
214             final expiresRaw = d['expires_at'] as String?;
215             final expiry = expiresRaw != null
216                 ? DateTime.tryParse(expiresRaw.replaceAll(' ', 'T'))
217                 : DateTime.now().add(const Duration(days: 30));
218
219             if (newCookie != null && newCookie.isNotEmpty) {
220                 await sl<SecureStorageHelper>().saveMadrasatiSession(
221                     MadrasatiSessionData(
222                         sessionCookie: newCookie,
223                         schoolId: schoolId,
224                         expiresAt: expiry,
225                         refreshToken: newRT ?? refreshToken,
226                     ),
227                 );
228                 debugPrint('📁 [DEBUG] New session saved to SecureStorage ✓');
229             }
230             if (mounted) {
231                 ScaffoldMessenger.of(context).showSnackBar(
232                     SnackBar(
233                         content: Text('✓ تم تجديد الجلسة $message'),
234                         backgroundColor: Colors.green,
235                         duration: const Duration(seconds: 4),
236                     ),
237                 );
238             }
239         } else {
240             // من أول connect الباك رفض - لازم يعمل ✗
241             debugPrint('🚫 [DEBUG] success=false - showing SessionExpiredDialog');
242             if (mounted) SessionExpiredDialog.show(context);
243         }
244     },
245 );
246 } catch (e, st) {
247     // خطأ غير متوقع - نفترق شبكة من غيره
248     final isNetworkError = e.toString().toLowerCase().contains('socket') ||
249         e.toString().toLowerCase().contains('connection') ||
250         e.toString().toLowerCase().contains('timeout');
251     debugPrint('✖ [DEBUG] Exception: $e');
252     debugPrint('    StackTrace: $st');
253     debugPrint('_____');
254 ');
255
256     if (mounted) {
257         if (isNetworkError) {
258             ScaffoldMessenger.of(context).showSnackBar(
259                 const SnackBar(
260                     content: Text('🌐 اتصال الإنترنت وحاول مجدداً'),
261                     backgroundColor: Colors.orange,
262                     duration: Duration(seconds: 4),
263                 ),
264             );
265         } else {
266             SessionExpiredDialog.show(context);
267         }
268     }
269 } finally {
270     if (mounted) setState(() => _loading = false);
271 }

```

```

270 }
271
272 @override
273 Widget build(BuildContext context) {
274   return Container(
275     margin: const EdgeInsets.symmetric(horizontal: 16),
276     decoration: BoxDecoration(
277       border: Border.all(color: Colors.red.withValues(alpha: 0.5), width: 1.5),
278       borderRadius: BorderRadius.circular(12),
279       color: Colors.red.withValues(alpha: 0.06),
280     ),
281     child: Material(
282       color: Colors.transparent,
283       child: InkWell(
284         borderRadius: BorderRadius.circular(12),
285         onTap: _loading ? null : _triggerRefresh,
286         child: Padding(
287           padding: const EdgeInsets.symmetric(vertical: 12, horizontal: 16),
288           child: _loading
289             ? const Center(
290                 child: SizedBox(
291                   height: 20,
292                   width: 20,
293                   child: CircularProgressIndicator(
294                     strokeWidth: 2,
295                     color: Colors.red,
296                   ),
297                 ),
298             )
299             : Row(
300                 mainAxisAlignment: MainAxisAlignment.center,
301                 children: [
302                   const Icon(Icons.refresh_rounded,
303                     color: Colors.red, size: 20),
304                   const SizedBox(width: 8),
305                   Text(
306                     'اختبار تجديد جلسة مدرستي - DEBUG',
307                     style: TextStyle(
308                       color: Colors.red.shade700,
309                       fontWeight: FontWeight.bold,
310                       fontSize: 13,
311                     ),
312                   ),
313                 ],
314             ),
315         ),
316       ),
317     );
318   };
319 }
320 }
321
322

```

Lines • Dart 79

Feature / Home

#215 – lib/features/home/presentation/widgets/chose_app_header.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/spacing.dart';
5
6  class ChoseAppHeader extends StatelessWidget {
7    final String title;
8    final String subtitle;
9
10   const ChoseAppHeader({
11     super.key,
12     required this.title,
13     required this.subtitle,
14   });
15
16   @override
17   Widget build(BuildContext context) {

```



```

18   return Column(
19     crossAxisAlignment: CrossAxisAlignment.center,
20     children: [
21       // Top decorative pill
22       Container(
23         width: 44,
24         height: 5,
25         decoration: BoxDecoration(
26           color: ColorsManager.primaryColor.withValues(alpha: 0.25),
27           borderRadius: BorderRadius.circular(10),
28         ),
29       ),
30       verticalSpace28,
31       // Icon badge
32       Container(
33         width: 72,
34         height: 72,
35         decoration: BoxDecoration(
36           gradient: LinearGradient(
37             begin: Alignment.topLeft,
38             end: Alignment.bottomRight,
39             colors: [
40               ColorsManager.primaryColor,
41               ColorsManager.primaryColor.withValues(alpha: 0.70),
42             ],
43           ),
44           borderRadius: BorderRadius.circular(22),
45           boxShadow: [
46             BoxShadow(
47               color: ColorsManager.primaryColor.withValues(alpha: 0.30),
48               blurRadius: 20,
49               offset: const Offset(0, 8),
50             ),
51           ],
52         ),
53         child: const Icon(
54           Icons.tune_rounded,
55           color: Colors.white,
56           size: 34,
57         ),
58       ),
59       verticalSpace20,
60       Text(
61         title,
62         textAlign: TextAlign.center,
63         style: TextStylesManager.bold22.copyWith(
64           color: ColorsManager.mainText,
65         ),
66       ),
67       verticalSpace8,
68       Text(
69         subtitle,
70         textAlign: TextAlign.center,
71         style: TextStylesManager.regular14.copyWith(
72           color: ColorsManager.secondaryText,
73         ),
74       ),
75     ],
76   );
77 }
78 }
79

```

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/spacing.dart';
5
6  class ChoseAppOptionCard extends StatelessWidget {
7    final IconData icon;
8    final String title;

```

```

9     final String subtitle;
10    final String badge;
11    final bool badgeActive;
12    final bool isSelected;
13    final bool isDisabled;
14    final VoidCallback? onTap;
15
16    const ChoseAppOptionCard({
17      super.key,
18      required this.icon,
19      required this.title,
20      required this.subtitle,
21      required this.badge,
22      required this.badgeActive,
23      required this.isSelected,
24      this.isDisabled = false,
25      this.onTap,
26    });
27
28    @override
29    Widget build(BuildContext context) {
30      final Color borderColor = isSelected
31        ? ColorsManager.primaryColor
32        : ColorsManager.borderColor;
33
34      final Color iconBg = isSelected
35        ? ColorsManager.primaryColor.withValues(alpha: 0.12)
36        : ColorsManager.surfacePrimary;
37
38      final Color iconColor = isSelected
39        ? ColorsManager.primaryColor
40        : ColorsManager.placeholder;
41
42      return AnimatedContainer(
43        duration: const Duration(milliseconds: 240),
44        curve: Curves.easeInOut,
45        decoration: BoxDecoration(
46          color: ColorsManager.surfacePrimary,
47          borderRadius: BorderRadius.circular(20),
48          border: Border.all(
49            color: borderColor,
50            width: isSelected ? 2 : 1,
51          ),
52          boxShadow: [
53            BoxShadow(
54              color: isSelected
55                ? ColorsManager.primaryColor.withValues(alpha: 0.10)
56                : Colors.black.withValues(alpha: 0.04),
57              blurRadius: isSelected ? 24 : 10,
58              offset: const Offset(0, 6),
59            ),
60          ],
61        ),
62      child: Material(
63        color: Colors.transparent,
64        borderRadius: BorderRadius.circular(20),
65        child: InkWell(
66          onTap: isDisabled ? null : onTap,
67          borderRadius: BorderRadius.circular(20),
68          splashColor: ColorsManager.primaryColor.withValues(alpha: 0.08),
69          highlightColor: ColorsManager.primaryColor.withValues(alpha: 0.04),
70          child: Padding(
71            padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 18),
72            child: Row(
73              crossAxisAlignment: CrossAxisAlignment.start,
74              children: [
75                // Icon container
76                AnimatedContainer(
77                  duration: const Duration(milliseconds: 240),
78                  width: 50,
79                  height: 50,
80                  decoration: BoxDecoration(
81                    color: iconBg,
82                    borderRadius: BorderRadius.circular(14),
83

```

```

84         child: Icon(
85             icon,
86             color: iconColor,
87             size: 26,
88         ),
89     ),
90     horizontalSpace12,
91     // Text content
92     Expanded(
93         child: Column(
94             crossAxisAlignment: CrossAxisAlignment.start,
95             children: [
96                 Wrap(
97                     crossAxisAlignment: WrapCrossAlignment.center,
98                     spacing: 8,
99                     runSpacing: 4,
100                    children: [
101                        Text(
102                            title,
103                            style: TextStylesManager.bold16.copyWith(
104                                color: isDisabled
105                                    ? ColorsManager.placeholder
106                                    : ColorsManager.mainText,
107                            ),
108                        ),
109                        _BadgeChip(
110                            label: badge,
111                            active: badgeActive,
112                        ),
113                    ],
114                ),
115                verticalSpace6,
116                Text(
117                    subtitle,
118                    style: TextStylesManager.regular13.copyWith(
119                        color: ColorsManager.secondaryText,
120                        height: 1.5,
121                    ),
122                ),
123            ],
124        ),
125    ),
126    horizontalSpace10,
127    // Radio indicator
128    AnimatedContainer(
129        duration: const Duration(milliseconds: 200),
130        width: 22,
131        height: 22,
132        decoration: BoxDecoration(
133            shape: BoxShape.circle,
134            border: Border.all(
135                color: isSelected
136                    ? ColorsManager.primaryColor
137                    : ColorsManager.borderColor,
138                width: isSelected ? 6 : 2,
139            ),
140        ),
141    ),
142 ],
143 ),
144 ),
145 ),
146 ),
147 );
148 }
149 }
150
151 class _BadgeChip extends StatelessWidget {
152     final String label;
153     final bool active;
154
155     const _BadgeChip({required this.label, required this.active});
156
157     @override
158     Widget build(BuildContext context) {

```

```

159     return Container(
160       padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 3),
161       decoration: BoxDecoration(
162         color: active
163           ? ColorsManager.primaryColor.withValues(alpha: 0.12)
164           : ColorsManager.secondaryColor.withValues(alpha: 0.14),
165         borderRadius: BorderRadius.circular(20),
166       ),
167       child: Text(
168         label,
169         style: TextStyleManager.bold10.copyWith(
170           color: active
171             ? ColorsManager.primaryColor
172             : ColorsManager.secondaryColor,
173         ),
174       ),
175     );
176   }
177 }
178

```

Lines • Dart 103

Feature / Home

#217 – lib/features/home/presentation/widgets/chose_app_usage_card.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/routes.dart';
6
7  class ChoseAppUsageCard extends StatelessWidget {
8    const ChoseAppUsageCard({super.key});
9
10   @override
11   Widget build(BuildContext context) {
12     return GestureDetector(
13       onTap: () => Navigator.pushNamed(context, Routes.extensionUsage),
14       child: Container(
15         decoration: BoxDecoration(
16           gradient: LinearGradient(
17             begin: Alignment.topRight,
18             end: Alignment.bottomLeft,
19             colors: [
20               ColorsManager.primaryColor.withValues(alpha: 0.12),
21               ColorsManager.secondaryColor.withValues(alpha: 0.08),
22             ],
23           ),
24           borderRadius: BorderRadius.circular(20),
25           border: Border.all(
26             color: ColorsManager.primaryColor.withValues(alpha: 0.25),
27             width: 1.2,
28           ),
29           boxShadow: [
30             BoxShadow(
31               color: ColorsManager.primaryColor.withValues(alpha: 0.08),
32               blurRadius: 20,
33               offset: const Offset(0, 8),
34             ),
35           ],
36         ),
37         padding: const EdgeInsets.all(20),
38         child: Row(
39           children: [
40             // — Icon Container —————
41             Container(
42               width: 54,
43               height: 54,
44               decoration: BoxDecoration(
45                 color: ColorsManager.primaryColor.withValues(alpha: 0.15),
46                 borderRadius: BorderRadius.circular(16),
47                 border: Border.all(
48                   color: ColorsManager.primaryColor.withValues(alpha: 0.3),
49                 ),
50             ),

```

```

51         child: Icon(
52             Icons.play_circle_outline_rounded,
53             color: ColorsManager.primaryColor,
54             size: 30,
55         ),
56     ),
57
58     const SizedBox(width: 16),
59
60     // — Text Column —————
61     Expanded(
62         child: Column(
63             crossAxisAlignment: CrossAxisAlignment.start,
64             children: [
65                 Text(
66                     appTranslation().get('usage_card_title'),
67                     style: TextStylesManager.bold16.copyWith(
68                         color: ColorsManager.mainText,
69                     ),
70                 ),
71                 const SizedBox(height: 4),
72                 Text(
73                     appTranslation().get('usage_card_subtitle'),
74                     style: TextStylesManager.regular13.copyWith(
75                         color: ColorsManager.secondaryText,
76                         height: 1.5,
77                     ),
78                 ),
79             ],
80         ),
81     ),
82
83     // — Arrow —————
84     Container(
85         width: 36,
86         height: 36,
87         decoration: BoxDecoration(
88             color: ColorsManager.primaryColor.withValues(alpha: 0.1),
89             shape: BoxShape.circle,
90         ),
91         child: Icon(
92             Icons.arrow_back_ios_new_rounded,
93             color: ColorsManager.primaryColor,
94             size: 16,
95         ),
96     ),
97 ],
98 ),
99 ),
100 );
101 }
102 }
103

```

Lines • Dart 270

Feature / Home

#218 – lib/features/home/presentation/widgets/chose_app.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/utils/constants/assets_helper.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
6  import 'package:moean/core/utils/constants/routes.dart';
7  import 'package:moean/core/utils/constants/spacing.dart';
8  import 'package:moean/core/widgets/dot_gridPainter.dart';
9  import 'package:moean/features/home/presentation/widgets/chose_app_header.dart';
10 import 'package:moean/features/home/presentation/widgets/chose_app_option_card.dart';
11 import 'package:moean/features/home/presentation/widgets/chose_app_usage_card.dart';
12
13 class ChoseApp extends StatefulWidget {
14     const ChoseApp({super.key});
15
16     @override
17     State<ChoseApp> createState() => _ChoseAppState();

```

```

18 }
19
20 class _ChoseAppState extends State<ChoseApp>
21   with SingleTickerProviderStateMixin {
22   // — Local selection state —————
23   // 0 = Browser Extension (active), 1 = Mobile App (coming soon / disabled)
24   int _selected = 0;
25
26   // — Animation —————
27   late final AnimationController _controller;
28
29   // Stagger config: 4 items → header, card1, card2, button
30   static const int _itemCount = 5;
31   static const double _itemDuration = 0.60; // fraction of total per item
32   static const double _stagger = 0.14;      // fraction offset between items
33
34   late final List<Animation<Offset>> _slides;
35   late final List<Animation<double>> _fades;
36
37   @override
38   void initState() {
39     super.initState();
40
41     _controller = AnimationController(
42       vsync: this,
43       duration: const Duration(milliseconds: 750),
44     );
45
46     _slides = List.generate(_itemCount, (i) {
47       final start = (i * _stagger).clamp(0.0, 1.0);
48       final end = (start + _itemDuration).clamp(0.0, 1.0);
49       return Tween<Offset>(
50         begin: const Offset(0, 0.18),
51         end: Offset.zero,
52       ).animate(CurvedAnimation(
53         parent: _controller,
54         curve: Interval(start, end, curve: Curves.easeOutCubic),
55       ));
56     });
57
58     _fades = List.generate(_itemCount, (i) {
59       final start = (i * _stagger).clamp(0.0, 1.0);
60       final end = (start + _itemDuration * 0.85).clamp(0.0, 1.0);
61       return Tween<double>(begin: 0.0, end: 1.0).animate(CurvedAnimation(
62         parent: _controller,
63         curve: Interval(start, end, curve: Curves.easeOut),
64       ));
65     });
66
67     _controller.forward();
68   }
69
70   @override
71   void dispose() {
72     _controller.dispose();
73     super.dispose();
74   }
75
76   // — Helpers —————
77   Widget _animated(int index, Widget child) {
78     return SlideTransition(
79       position: _slides[index],
80       child: FadeTransition(
81         opacity: _fades[index],
82         child: child,
83       ),
84     );
85   }
86
87   void _onContinue() {
88     if (_selected == 0) {
89       Navigator.pushNamed(context, Routes.addextention);
90     }
91   }
92

```

```

93 // — Build —————
94 @override
95 Widget build(BuildContext context) {
96   return Directionality(
97     textDirection: TextDirection.rtl,
98     child: Scaffold(
99       backgroundColor: ColorsManager.background,
100       extendBodyBehindAppBar: true,
101       appBar: AppBar(
102         backgroundColor: Colors.transparent,
103         elevation: 0,
104         leading: IconButton(
105           icon: Icon(
106             Icons.arrow_back_ios_rounded,
107             color: ColorsManager.mainText,
108           ),
109           onPressed: () => Navigator.pop(context),
110         ),
111         actions: [
112           Padding(
113             padding: const EdgeInsets.only(left: 12),
114             child: Image.asset(
115               AssetsHelper.icon,
116               width: 55,
117               height: 55,
118               fit: BoxFit.contain,
119             ),
120           ),
121         ],
122       ),
123       body: Stack(
124         children: [
125           // — Decorative: dot grid (top-right) —————
126           Positioned(
127             right: 16,
128             top: 80,
129             height: 160,
130             width: 150,
131             child: CustomPaint(
132               painter: DotGridPainter(
133                 color: ColorsManager.primaryColor.withValues(alpha: 0.13),
134                 spacing: 16,
135               ),
136             ),
137           ),
138           // — Decorative: dot grid (bottom-left) —————
139           Positioned(
140             left: 12,
141             bottom: 120,
142             height: 120,
143             width: 110,
144             child: CustomPaint(
145               painter: DotGridPainter(
146                 color: ColorsManager.secondaryColor.withValues(alpha: 0.12),
147                 spacing: 14,
148               ),
149             ),
150           ),
151           // — Decorative: glowing circle top-left —————
152           Positioned(
153             top: -60,
154             left: -60,
155             child: Container(
156               width: 200,
157               height: 200,
158               decoration: BoxDecoration(
159                 shape: BoxShape.circle,
160                 color: ColorsManager.primaryColor.withValues(alpha: 0.07),
161               ),
162             ),
163           ),
164           // — Decorative: glowing circle bottom-right —————
165           Positioned(
166             bottom: -60,
167             right: -60,
168             child: Container(
169               width: 200,
170               height: 200,
171               decoration: BoxDecoration(
172                 shape: BoxShape.circle,
173                 color: ColorsManager.primaryColor.withValues(alpha: 0.07),
174               ),
175             ),
176           ),
177         ],
178       ),
179     ),
180   );
181 }

```

```

168     Positioned(
169       bottom: -80,
170       right: -60,
171       child: Container(
172         width: 220,
173         height: 220,
174         decoration: BoxDecoration(
175           shape: BoxShape.circle,
176           color: ColorsManager.secondaryColor.withValues(alpha: 0.08),
177         ),
178       ),
179     ),
180
181     // — Main content —————
182     SafeArea(
183       child: SingleChildScrollView(
184         physics: const BouncingScrollPhysics(),
185         padding: const EdgeInsets.symmetric(
186           horizontal: 24,
187           vertical: 32,
188         ),
189         child: Column(
190           crossAxisAlignment: CrossAxisAlignment.stretch,
191           children: [
192             // Item 0 — Header
193             _animated(
194               0,
195               ChoseAppHeader(
196                 title: appTranslation().get('chose_app_title'),
197                 subtitle: appTranslation().get('chose_app_subtitle'),
198               ),
199             ),
200
201             verticalSpace40,
202
203             // Item 1 — Extension card
204             _animated(
205               1,
206               ChoseAppOptionCard(
207                 icon: Icons.extension_rounded,
208                 title: appTranslation()
209                   .get('chose_app_extension_title'),
210                 subtitle: appTranslation()
211                   .get('chose_app_extension_subtitle'),
212                 badge: appTranslation()
213                   .get('chose_app_extension_badge'),
214                 badgeActive: true,
215                 isSelected: _selected == 0,
216                 onTap: () => setState(() => _selected = 0),
217               ),
218             ),
219
220             verticalSpace16,
221
222             // Item 2 — Mobile app card (coming soon)
223             _animated(
224               2,
225               ChoseAppOptionCard(
226                 icon: Icons.phone_iphone_rounded,
227                 title: appTranslation().get('chose_app_app_title'),
228                 subtitle:
229                   appTranslation().get('chose_app_app_subtitle'),
230                 badge: appTranslation().get('chose_app_app_badge'),
231                 badgeActive: false,
232                 isSelected: false,
233                 isDisabled: true,
234                 onTap: null,
235               ),
236             ),
237
238             verticalSpace24,
239
240             // Item 3 — How to use card
241             _animated(
242               3,

```



```

243         const ChoseAppUsageCard(),
244     ),
245
246     verticalSpace24,
247
248     // Item 4 – Continue button
249     _animated(
250         4,
251         PrimaryElevatedButton(
252             onPressed: _onContinue,
253             text: appTranslation().get('chose_app_continue'),
254             icon: const Icon(
255                 Icons.arrow_back_rounded,
256                 color: Colors.white,
257             ),
258         ),
259     ),
260     // verticalSpace16,
261 ],
262 ),
263 ),
264 ),
265 ],
266 ),
267 ),
268 );
269 }
270 }

```

Lines • Dart 263

Feature / Home

#219 – lib/features/home/presentation/widgets/download_extention.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/assets_helper.dart';
6  import 'package:moean/core/utils/constants/constants.dart';
7  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
8  import 'package:moean/core/utils/constants/spacing.dart';
9  import 'package:moean/core/utils/cubit/theme/theme_cubit.dart';
10 import 'package:moean/core/utils/cubit/theme/theme_state.dart';
11 import 'package:moean/features/home/presentation/cubit/extension_install_cubit.dart';
12 import 'package:moean/features/home/presentation/widgets/extension_ios_dialog.dart';
13
14 class DownloadExtention extends StatelessWidget {
15     const DownloadExtention({super.key});
16
17     @override
18     Widget build(BuildContext context) {
19         return BlocProvider(
20             create: (_) => ExtensionInstallCubit(),
21             child: const _DownloadExtentionView(),
22         );
23     }
24 }
25
26 class _DownloadExtentionView extends StatelessWidget {
27     const _DownloadExtentionView();
28
29     @override
30     Widget build(BuildContext context) {
31         return BlocListener<ExtensionInstallCubit, ExtensionInstallState>(
32             listener: (context, state) {
33                 if (state is ExtensionInstallIosNotSupported) {
34                     ExtensionIosDialog.show(context);
35                 } else if (state is ExtensionInstallError) {
36                     ScaffoldMessenger.of(context).showSnackBar(
37                         SnackBar(content: Text(state.message)),
38                     );
39                 }
40             },
41             child: BlocBuilder<ThemeCubit, ThemeState>(
42                 builder: (context, themeState) => Directionality(

```



```

118         text: appTranslation().get('ext_quetta_btn_open'),
119         icon: const Icon(Icons.rocket_launch_rounded,
120             color: Colors.white),
121     );
122   } else {
123     return PrimaryElevatedButton(
124       onPressed: () {
125 showDialog(
126   context: context,
127   builder: (context) => AlertDialog(
128     title: Text(appTranslation().get('attention')),
129     content: Text(appTranslation().get('should_download')),
130     actions: [
131       TextButton(
132         onPressed: () {
133           Navigator.pop(context);
134         },
135         child: Text(appTranslation().get('cancel')),
136       ),
137     ],
138   ),
139 );
140   };
141
142   },
143   text: appTranslation().get('ext_quetta_btn_open'),
144   icon: const Icon(Icons.rocket_launch_rounded,
145       color: Colors.white),
146   );})
147   },
148   ),
149 ),
150 verticalSpace20,
151
152 // --- Step 3 ---
153 _StepCard(
154   title: appTranslation().get('ext_quetta_step3_title'),
155   body: appTranslation().get('ext_quetta_step3_body'),
156   button: BlocBuilder<ExtensionInstallCubit, ExtensionInstallState>(
157     builder: (context, state) {
158       return PrimaryElevatedButton(
159         onPressed: () {
160           if (state.isQuettaInstalled) {
161             ExtensionInstallCubit.get(context)
162               .openMadrasatiInQuetta();
163           } else {
164             showDialog(
165               context: context,
166               builder: (context) => AlertDialog(
167                 title: Text(appTranslation().get('attention')),
168                 content: Text(
169                   appTranslation().get('should_download')),
170                 actions: [
171                   TextButton(
172                     onPressed: () => Navigator.pop(context),
173                     child: Text(appTranslation().get('cancel')),
174                   ),
175                 ],
176               ),
177             );
178           }
179         },
180         text: appTranslation().get('ext_quetta_btn_open_madrasati'),
181         icon: const Icon(Icons.open_in_browser_rounded,
182             color: Colors.white),
183       );
184     },
185   ),
186 ),
187 verticalSpace24,
188
189 ],
190 ),
191 ),
192

```

```

193         ),
194     ),
195 ),
196 ),
197 ),
198 );
199 }
200 }
201
202 /// A card representing a single step in the flow.
203 class _StepCard extends StatelessWidget {
204     final String title;
205     final String body;
206     final Widget? button;
207
208     const _StepCard({
209         required this.title,
210         required this.body,
211         this.button,
212     });
213
214     @override
215     Widget build(BuildContext context) {
216         return Container(
217             decoration: BoxDecoration(
218                 color: ColorsManager.surfacePrimary,
219                 borderRadius: BorderRadius.circular(20),
220                 border: Border.all(
221                     color: ColorsManager.primaryColor.withValues(alpha: 0.08),
222                 ),
223                 boxShadow: [
224                     BoxShadow(
225                         color: ColorsManager.primaryColor.withValues(alpha: 0.04),
226                         blurRadius: 16,
227                         offset: const Offset(0, 6),
228                     ),
229                 ],
230             ),
231             padding: const EdgeInsets.all(20),
232             child: Column(
233                 crossAxisAlignment: CrossAxisAlignment.stretch,
234                 children: [
235                     Text(
236                         title,
237                         style: TextStylesManager.bold16.copyWith(
238                             color: ColorsManager.primaryColor,
239                         ),
240                     ),
241                     verticalSpace12,
242                     Text(
243                         body,
244                         style: TextStylesManager.bold14.copyWith(
245                             color: ColorsManager.mainText,
246                             height: 1.6,
247                         ),
248                     ),
249                     if (button != null) ...[
250                         verticalSpace20,
251                         SizedBox(
252                             width: double.infinity,
253                             height: 50,
254                             child: button!,
255                         ),
256                     ],
257                 ],
258             ),
259         );
260     }
261 }
262
263

```

```
1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
6 import 'package:moean/core/utils/constants/spacing.dart';
7
8 /// Dialog displayed on iOS to inform the user that mobile extensions
9 /// are only supported on Android.
10 class ExtensionIosDialog extends StatelessWidget {
11   const ExtensionIosDialog({super.key});
12
13   static Future<void> show(BuildContext context) {
14     return showDialog(
15       context: context,
16       barrierDismissible: true,
17       builder: (_) => const ExtensionIosDialog(),
18     );
19   }
20
21   @override
22   Widget build(BuildContext context) {
23     return Dialog(
24       backgroundColor: ColorsManager.surfacePrimary,
25       shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(24)),
26       child: Directionality(
27         textDirection: TextDirection.rtl,
28         child: Padding(
29           padding: const EdgeInsets.all(28),
30           child: Column(
31             mainAxisAlignment: MainAxisAlignment.min,
32             children: [
33               // Icon badge
34               Container(
35                 width: 64,
36                 height: 64,
37                 decoration: BoxDecoration(
38                   gradient: LinearGradient(
39                     colors: [
40                       ColorsManager.primaryColor.withValues(alpha: 0.15),
41                       ColorsManager.primaryColor.withValues(alpha: 0.05),
42                     ],
43                     begin: Alignment.topLeft,
44                     end: Alignment.bottomRight,
45                   ),
46                 shape: BoxShape.circle,
47               ),
48               child: Icon(
49                 Icons.desktop_mac_rounded,
50                 color: ColorsManager.primaryColor,
51                 size: 32,
52               ),
53             ],
54             verticalSpace20,
55
56             // Title
57             Text(
58               appTranslation().get('ext_ios_title'),
59               style: TextStylesManager.bold18.copyWith(
60                 color: ColorsManager.mainText,
61               ),
62               textAlign: TextAlign.center,
63             ),
64             verticalSpace12,
65
66             // Body
67             Text(
68               appTranslation().get('ext_ios_body'),
69               style: TextStylesManager.regular14.copyWith(
70                 color: ColorsManager.secondaryText,
71                 height: 1.7,
72             ),
```

```

73         textAlign: TextAlign.center,
74     ),
75     verticalSpace28,
76
77     // Dismiss button
78     PrimaryElevatedButton(
79         text: appTranslation().get('ext_ios_ok'),
80         onPressed: () => Navigator.of(context).pop(),
81     ),
82 ],
83 ),
84 ),
85 ),
86 );
87 }
88 }
89

```

Lines • 293
Dart

Feature / #221 -

Home [lib/features/home/presentation/widgets/extension_kiwi_missing_bottom_sheet.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7  import 'package:moean/features/home/presentation/cubit/extension_install_cubit.dart';
8
9  /// Bottom sheet displayed when Kiwi Browser is not installed on the device.
10 ///
11 /// Explains the direct APK download flow cleanly using a vertical timeline.
12 class ExtensionKiwiMissingBottomSheet extends StatelessWidget {
13     final ExtensionInstallCubit cubit;
14
15     const ExtensionKiwiMissingBottomSheet({
16         super.key,
17         required this.cubit,
18     });
19
20     static Future<void> show(
21         BuildContext context,
22         ExtensionInstallCubit cubit,
23     ) {
24         return showModalBottomSheet(
25             context: context,
26             isScrollControlled: true,
27             backgroundColor: Colors.transparent,
28             builder: (_) => ExtensionKiwiMissingBottomSheet(cubit: cubit),
29         );
30     }
31
32     @override
33     Widget build(BuildContext context) {
34         return Container(
35             decoration: BoxDecoration(
36                 color: ColorsManager.surfacePrimary,
37                 borderRadius: const BorderRadius.vertical(top: Radius.circular(32)),
38                 boxShadow: [
39                     BoxShadow(
40                         color: Colors.black.withValues(alpha: 0.15),
41                         blurRadius: 30,
42                         offset: const Offset(0, -5),
43                     ),
44                 ],
45             ),
46             padding: EdgeInsets.fromLTRB(
47                 24,
48                 16,
49                 24,
50                 MediaQuery.of(context).viewInsets.bottom + 36,
51             ),
52             child: Directionality(

```

```

53     textDirection: TextDirection.rtl,
54     child: Column(
55       mainAxisAlignment: MainAxisAlignment.min,
56       crossAxisAlignment: CrossAxisAlignment.stretch,
57       children: [
58         // Premium Drag handle
59         Center(
60           child: Container(
61             width: 48,
62             height: 5,
63             decoration: BoxDecoration(
64               color: ColorsManager.borderColor,
65               borderRadius: BorderRadius.circular(10),
66             ),
67           ),
68         ),
69         verticalSpace24,
70
71         // Magic Header Icon
72         Center(
73           child: Stack(
74             alignment: Alignment.center,
75             children: [
76               Container(
77                 width: 86,
78                 height: 86,
79                 decoration: BoxDecoration(
80                   shape: BoxShape.circle,
81                   color: ColorsManager.goldDark.withValues(alpha: 0.1),
82                 ),
83               ),
84               Container(
85                 width: 66,
86                 height: 66,
87                 decoration: BoxDecoration(
88                   gradient: LinearGradient(
89                     colors: [
90                       ColorsManager.primaryColor,
91                       ColorsManager.primaryColor.withValues(alpha: 0.8),
92                     ],
93                     begin: Alignment.topLeft,
94                     end: Alignment.bottomRight,
95                   ),
96                   shape: BoxShape.circle,
97                   boxShadow: [
98                     BoxShadow(
99                       color: ColorsManager.primaryColor.withValues(alpha: 0.35),
100                       blurRadius: 16,
101                       offset: const Offset(0, 6),
102                     ),
103                   ],
104                 ),
105               child: const Icon(
106                 Icons.auto_awesome_rounded,
107                 color: Colors.white,
108                 size: 32,
109               ),
110             ],
111           ),
112         ),
113         verticalSpace20,
114
115         // Title
116         Text(
117           appTranslation().get('ext_kiwi_needed_title'),
118           style: TextStylesManager.bold22.copyWith(
119             color: ColorsManager.mainText,
120           ),
121           textAlign: TextAlign.center,
122         ),
123         verticalSpace10,
124
125         // Subtitle
126         Text(

```

```

128         appTranslation().get('ext_kiwi_needed_body'),
129         style: TextStyleManager.regular14.copyWith(
130             color: ColorsManager.secondaryText,
131             height: 1.6,
132         ),
133         textAlign: TextAlign.center,
134     ),
135     verticalSpace32,
136
137     // Vertical Timeline Steps
138     const _VerticalStep(
139         icon: Icons.cloud_download_rounded,
140         number: '1',
141         isLast: false,
142         translationKey: 'ext_kiwi_step1',
143     ),
144     const _VerticalStep(
145         icon: Icons.replay_circle_filled_rounded,
146         number: '2',
147         isLast: true,
148         translationKey: 'ext_kiwi_step2',
149     ),
150
151     verticalSpace32,
152
153     // Primary CTA - Download APK
154     PrimaryElevatedButton(
155         text: appTranslation().get('ext_install_kiwi'),
156         icon: const Icon(
157             Icons.android_rounded,
158             color: Colors.white,
159             size: 22,
160         ),
161         onPressed: () {
162             Navigator.of(context).pop();
163             cubit.downloadQuetta();
164         },
165     ),
166     verticalSpace12,
167
168     // Cancel
169     SizedBox(
170         height: 50,
171         child: TextButton(
172             onPressed: () => Navigator.of(context).pop(),
173             style: TextButton.styleFrom(
174                 foregroundColor: ColorsManager.secondaryText,
175                 shape: RoundedRectangleBorder(
176                     borderRadius: BorderRadius.circular(16),
177                 ),
178             ),
179             child: Text(
180                 appTranslation().get('ext_cancel'),
181                 style: TextStyleManager.medium16.copyWith(
182                     color: ColorsManager.secondaryText,
183                 ),
184             ),
185         ),
186     ),
187 ],
188 ),
189 ),
190 );
191 }
192 }
193
194 /// A premium vertical timeline step item.
195 class _VerticalStep extends StatelessWidget {
196     final IconData icon;
197     final String number;
198     final bool isLast;
199     final String translationKey;
200
201     const _VerticalStep({
202         required this.icon,

```



```

203     required this.number,
204     required this.isLast,
205     required this.translationKey,
206   });
207
208   @override
209   Widget build(BuildContext context) {
210     return Row(
211       crossAxisAlignment: CrossAxisAlignment.start,
212       children: [
213         // Timeline Column
214         Column(
215           children: [
216             Container(
217               width: 32,
218               height: 32,
219               decoration: BoxDecoration(
220                 color: ColorsManager.surfacePrimary,
221                 shape: BoxShape.circle,
222                 border: Border.all(
223                   color: ColorsManager.primaryColor,
224                   width: 2,
225                 ),
226                 boxShadow: [
227                   BoxShadow(
228                     color: ColorsManager.primaryColor.withValues(alpha: 0.2),
229                     blurRadius: 8,
230                     offset: const Offset(0, 2),
231                   ),
232                 ],
233             ),
234             child: Center(
235               child: Text(
236                 number,
237                 style: TextStylesManager.bold14.copyWith(
238                   color: ColorsManager.primaryColor,
239                 ),
240             ),
241           ),
242         ),
243         if (!isLast)
244           Container(
245             width: 2,
246             height: 24,
247             margin: const EdgeInsets.symmetric(vertical: 4),
248             decoration: BoxDecoration(
249               color: ColorsManager.primaryColor.withValues(alpha: 0.3),
250               borderRadius: BorderRadius.circular(2),
251             ),
252           ),
253       ],
254     ),
255     horizontalSpace16,
256
257     // Content
258     Expanded(
259       child: Container(
260         margin: EdgeInsets.only(bottom: isLast ? 0 : 20),
261         padding: const EdgeInsets.all(16),
262         decoration: BoxDecoration(
263           color: ColorsManager.primaryColor.withValues(alpha: 0.04),
264           borderRadius: BorderRadius.circular(16),
265           border: Border.all(
266             color: ColorsManager.primaryColor.withValues(alpha: 0.08),
267           ),
268         ),
269         child: Row(
270           children: [
271             Icon(
272               icon,
273               color: ColorsManager.primaryColor.withValues(alpha: 0.8),
274               size: 24,
275             ),
276             horizontalSpace12,
277             Expanded(

```

```

278         child: Text(
279             appTranslation().get(translationKey),
280             style: TextStylesManager.bold14.copyWith(
281                 color: ColorsManager.mainText,
282             ),
283         ),
284     ),
285 ],
286 ),
287 ),
288 ),
289 ],
290 );
291 }
292 }
293

```

Lines • Dart 384

Feature / Home

#222 – lib/features/home/presentation/widgets/extension_usage_slider.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter/services.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/assets_helper.dart';
6  import 'package:moean/core/utils/constants/constants.dart';
7  import 'package:moean/core/utils/constants/spacing.dart';
8
9  class ExtensionUsageSlider extends StatefulWidget {
10     const ExtensionUsageSlider({super.key});
11
12     @override
13     State<ExtensionUsageSlider> createState() => _ExtensionUsageSliderState();
14 }
15
16 class _ExtensionUsageSliderState extends State<ExtensionUsageSlider>
17     with SingleTickerProviderStateMixin {
18     late final PageController _pageController;
19     int _currentPage = 0;
20
21     static const List<String> _images = [
22         AssetsHelper.screen1,
23         AssetsHelper.screen2,
24         AssetsHelper.screen3,
25     ];
26
27
28     late final AnimationController _overlayController;
29     late final Animation<double> _overlayFade;
30
31     @override
32     void initState() {
33         super.initState();
34         _pageController = PageController();
35
36         _overlayController = AnimationController(
37             vsync: this,
38             duration: const Duration(milliseconds: 500),
39         );
40         _overlayFade = CurvedAnimation(
41             parent: _overlayController,
42             curve: Curves.easeOut,
43         );
44         _overlayController.forward();
45
46         SystemChrome.setEnabledSystemUIMode(SystemUiMode.immersiveSticky);
47     }
48
49     @override
50     void dispose() {
51         _pageController.dispose();
52         _overlayController.dispose();
53         SystemChrome.setEnabledSystemUIMode(SystemUiMode.edgeToEdge);
54         super.dispose();

```

```

55 }
56
57 void _onPageChanged(int index) {
58   setState(() => _currentPage = index);
59 }
60
61 void _goNext() {
62   if (_currentPage < _images.length - 1) {
63     _pageController.nextPage(
64       duration: const Duration(milliseconds: 380),
65       curve: Curves.easeInOutCubic,
66     );
67   } else {
68     Navigator.pop(context);
69   }
70 }
71
72 void _goPrev() {
73   if (_currentPage > 0) {
74     _pageController.previousPage(
75       duration: const Duration(milliseconds: 380),
76       curve: Curves.easeInOutCubic,
77     );
78   }
79 }
80
81 @override
82 Widget build(BuildContext context) {
83   final isLast = _currentPage == _images.length - 1;
84
85   return Directionality(
86     textDirection: TextDirection.rtl,
87     child: Scaffold(
88       backgroundColor: Colors.black,
89       body: Stack(
90         fit: StackFit.expand,
91         children: [
92           // — Full-screen page view —————
93           PageView.builder(
94             controller: _pageController,
95             itemCount: _images.length,
96             onPageChanged: _onPageChanged,
97             itemBuilder: (context, index) {
98               return _SliderPage(imagePath: _images[index]);
99             },
100           ),
101
102           // — Top bar with close button only —————
103           Positioned(
104             top: 0,
105             left: 0,
106             right: 0,
107             child: FadeTransition(
108               opacity: _overlayFade,
109               child: Container(
110                 decoration: BoxDecoration(
111                   gradient: LinearGradient(
112                     begin: Alignment.topCenter,
113                     end: Alignment.bottomCenter,
114                     colors: [
115                       Colors.black.withValues(alpha: 0.72),
116                       Colors.transparent,
117                     ],
118                   ),
119                 ),
120                 child: SafeArea(
121                   child: Padding(
122                     padding: const EdgeInsets.symmetric(
123                       horizontal: 16,
124                       vertical: 12,
125                     ),
126                     child: Align(
127                       alignment: AlignmentDirectional.centerEnd,
128                       child: GestureDetector(
129                         onTap: () => Navigator.pop(context),

```

```

130         child: Container(
131           width: 40,
132           height: 40,
133           decoration: BoxDecoration(
134             color: Colors.white.withValues(alpha: 0.18),
135             shape: BoxShape.circle,
136             border: Border.all(
137               color: Colors.white.withValues(alpha: 0.3),
138             ),
139           ),
140           child: const Icon(
141             Icons.close_rounded,
142             color: Colors.white,
143             size: 20,
144           ),
145         ),
146       ),
147     ),
148   ),
149 ),
150 ),
151 ),
152 ),
153
154 // — Bottom bar: counter + dots + navigation —————
155 Positioned(
156   bottom: 0,
157   left: 0,
158   right: 0,
159   child: FadeTransition(
160     opacity: _overlayFade,
161     child: Container(
162       decoration: BoxDecoration(
163         gradient: LinearGradient(
164           begin: Alignment.bottomCenter,
165           end: Alignment.topCenter,
166           colors: [
167             Colors.black.withValues(alpha: 0.85),
168             Colors.transparent,
169           ],
170         ),
171       ),
172       child: SafeArea(
173         child: Padding(
174           padding: const EdgeInsets.fromLTRB(20, 12, 20, 20),
175           child: Column(
176             mainAxisAlignment: MainAxisAlignment.min,
177             children: [
178               // Counter chip — centered above nav row
179               Container(
180                 padding: const EdgeInsets.symmetric(
181                   horizontal: 16,
182                   vertical: 5,
183                 ),
184                 decoration: BoxDecoration(
185                   color: Colors.white.withValues(alpha: 0.15),
186                   borderRadius: BorderRadius.circular(20),
187                   border: Border.all(
188                     color: Colors.white.withValues(alpha: 0.25),
189                   ),
190                 ),
191                 child: Text(
192                   '${_currentPage + 1} / ${_images.length}',
193                   style: TextStyleManager.bold14.copyWith(
194                     color: Colors.white,
195                   ),
196                 ),
197               ),
198             ],
199           verticalSpace14,
200
201           // Nav row: prev | dots | next
202           Row(
203             mainAxisAlignment: MainAxisAlignment.spaceBetween,
204             children: [

```

```

205         // Prev button
206         _NavButton(
207             icon: Icons.arrow_forward_ios_rounded,
208             onTap: _goPrev,
209             enabled: _currentPage > 0,
210         ),
211
212         // Dot indicators
213         Flexible(
214             child: Row(
215                 mainAxisAlignment: MainAxisAlignment.min,
216                 mainAxisAlignment: MainAxisAlignment.center,
217                 children: List.generate(
218                     _images.length,
219                     (i) => _Dot(isActive: i == _currentPage),
220                 ),
221             ),
222         ),
223
224         // Next / Done button
225         _NavButton(
226             icon: isLast
227                 ? Icons.check_rounded
228                 : Icons.arrow_back_ios_rounded,
229             onTap: _goNext,
230             enabled: true,
231             isPrimary: isLast,
232         ),
233     ],
234 ),
235 ],
236 ),
237 ),
238 ),
239 ),
240 ),
241 ),
242
243 // — Swipe hint label (first page only) —————
244 if (_currentPage == 0)
245     Positioned(
246         bottom: 100,
247         left: 0,
248         right: 0,
249         child: FadeTransition(
250             opacity: _overlayFade,
251             child: Center(
252                 child: Container(
253                     padding: const EdgeInsets.symmetric(
254                         horizontal: 16,
255                         vertical: 8,
256                     ),
257                     decoration: BoxDecoration(
258                         color: Colors.black.withValues(alpha: 0.55),
259                         borderRadius: BorderRadius.circular(20),
260                     ),
261                     child: Row(
262                         mainAxisAlignment: MainAxisAlignment.min,
263                         children: [
264                             const Icon(
265                                 Icons.swipe_left_alt_rounded,
266                                 color: Colors.white70,
267                                 size: 18,
268                             ),
269                             horizontalSpace6,
270                             Text(
271                                 appTranslation().get('slider_swipe_hint'),
272                                 style: TextStylesManager.regular12.copyWith(
273                                     color: Colors.white70,
274                                 ),
275                             ),
276                         ],
277                     ),
278                 ),
279             ),

```

```

280         ),
281     ),
282 ],
283 ),
284 ),
285 );
286 }
287 }
288
289 // — Single page with image filling the screen —————
290 class _SliderPage extends StatelessWidget {
291     final String imagePath;
292
293     const _SliderPage({required this.imagePath});
294
295     @override
296     Widget build(BuildContext context) {
297         return InteractiveViewer(
298             minScale: 1.0,
299             maxScale: 4.0,
300             child: Image.asset(
301                 imagePath,
302                 fit: BoxFit.contain,
303                 width: double.infinity,
304                 height: double.infinity,
305                 errorBuilder: (context, error, stack) => Center(
306                     child: Icon(
307                         Icons.broken_image_rounded,
308                         color: ColorsManager.secondaryText,
309                         size: 64,
310                     ),
311                 ),
312             ),
313         );
314     }
315 }
316
317 // — Navigation arrow button —————
318 class _NavButton extends StatelessWidget {
319     final IconData icon;
320     final VoidCallback onTap;
321     final bool enabled;
322     final bool isPrimary;
323
324     const _NavButton({
325         required this.icon,
326         required this.onTap,
327         required this.enabled,
328         this.isPrimary = false,
329     });
330
331     @override
332     Widget build(BuildContext context) {
333         return GestureDetector(
334             onTap: enabled ? onTap : null,
335             child: AnimatedContainer(
336                 duration: const Duration(milliseconds: 250),
337                 width: 48,
338                 height: 48,
339                 decoration: BoxDecoration(
340                     shape: BoxShape.circle,
341                     color: isPrimary
342                         ? ColorsManager.primaryColor
343                         : enabled
344                             ? Colors.white.withValues(alpha: 0.22)
345                             : Colors.white.withValues(alpha: 0.08),
346                     border: Border.all(
347                         color: isPrimary
348                             ? ColorsManager.primaryColor
349                             : Colors.white.withValues(alpha: enabled ? 0.35 : 0.12),
350                     ),
351                 ),
352             child: Icon(
353                 icon,
354                 color: enabled ? Colors.white : Colors.white38,

```

```

355         size: 20,
356       ),
357     ),
358   );
359 }
360 }
361
362 // — Page dot indicator —————
363 class _Dot extends StatelessWidget {
364   final bool isActive;
365
366   const _Dot({required this.isActive});
367
368   @override
369   Widget build(BuildContext context) {
370     return AnimatedContainer(
371       duration: const Duration(milliseconds: 300),
372       margin: const EdgeInsets.symmetric(horizontal: 3),
373       width: isActive ? 20 : 6,
374       height: 6,
375       decoration: BoxDecoration(
376         color: isActive
377           ? ColorsManager.primaryColor
378           : Colors.white.withValues(alpha: 0.35),
379         borderRadius: BorderRadius.circular(3),
380       ),
381     );
382   }
383 }
384

```

Lines • Dart 99

Feature / Home

#223 — lib/features/home/presentation/widgets/home_action_chip_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/spacing.dart';
5
6  class HomeActionChipWidget extends StatefulWidget {
7    final IconData icon;
8    final String title;
9    final VoidCallback? onTap;
10
11    const HomeActionChipWidget({
12      super.key,
13      required this.icon,
14      required this.title,
15      this.onTap,
16    });
17
18    @override
19    State<HomeActionChipWidget> createState() => _HomeActionChipWidgetState();
20  }
21
22  class _HomeActionChipWidgetState extends State<HomeActionChipWidget>
23    with SingleTickerProviderStateMixin {
24    late AnimationController _controller;
25    late Animation<double> _animation;
26
27    @override
28    void initState() {
29      super.initState();
30      _controller = AnimationController(
31        vsync: this,
32        duration: const Duration(milliseconds: 1500),
33      )..repeat(reverse: true);
34
35      _animation = Tween<double>(begin: 0.0, end: 1.0).animate(
36        CurvedAnimation(parent: _controller, curve: Curves.easeInOut),
37      );
38    }
39
40    @override

```

```

41 void dispose() {
42   _controller.dispose();
43   super.dispose();
44 }
45
46 @override
47 Widget build(BuildContext context) {
48   return AnimatedBuilder(
49     animation: _animation,
50     builder: (context, child) {
51       final bool isClickable = widget.onTap != null;
52       return Material(
53         color: Colors.transparent,
54         child: InkWell(
55           onTap: widget.onTap,
56           borderRadius: BorderRadius.circular(30),
57           child: Container(
58             padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 16),
59             decoration: BoxDecoration(
60               color: ColorsManager.surfacePrimary,
61               borderRadius: BorderRadius.circular(30),
62               border: Border.all(
63                 color: isClickable
64                   ? ColorsManager.goldDark.withValues(alpha: 0.1 + (_animation.value * 0.4))
65                   : Colors.transparent,
66                 width: 1.5,
67               ),
68               boxShadow: [
69                 BoxShadow(
70                   color: ColorsManager.primaryColor.withValues(
71                     alpha: isClickable ? (0.05 + (_animation.value * 0.1)) : 0.05,
72                   ),
73                   blurRadius: isClickable ? (8 + (_animation.value * 6)) : 10,
74                   spreadRadius: isClickable ? (_animation.value * 1.5) : 0,
75                   offset: const Offset(0, 2),
76                 ),
77               ],
78             ),
79             child: Row(
80               mainAxisAlignment: MainAxisAlignment.min,
81               children: [
82                 Icon(widget.icon, color: ColorsManager.primaryColor, size: 20),
83                 horizontalSpace8,
84                 Text(
85                   widget.title,
86                   style: TextStylesManager.bold14.copyWith(
87                     color: ColorsManager.primaryColor,
88                   ),
89                 ),
90               ],
91             ),
92           ),
93         ),
94       );
95     },
96   );
97 }
98 }
99

```

Lines • Dart 352

Feature / Home

#224 – lib/features/home/presentation/widgets/home_app_bar_widget.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/assets_helper.dart';
6 import 'package:moean/core/utils/constants/constants.dart';
7 import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
8 import 'package:moean/core/utils/constants/routes.dart';
9 import 'package:moean/core/utils/constants/spacing.dart';
10 import 'package:moean/core/utils/extensions/context_extension.dart';
11 import 'package:moean/features/home/presentation/cubit/home_cubit.dart';

```



```

12 import 'package:moean/features/profile/presentation/widgets/logout_dialog.dart';
13 import 'package:moean/core/di/injections.dart';
14 import 'package:moean/features/profile/presentation/cubit/profile_cubit.dart';
15 import 'package:moean/features/profile/presentation/cubit/profile_state.dart';
16 import 'package:moean/features/payment/presentation/cubit/subscription_cubit.dart';
17 import 'package:moean/features/payment/presentation/widgets/subscription_status_dialog.dart';
18
19 class HomeAppBarWidget extends StatelessWidget {
20   const HomeAppBarWidget({super.key});
21
22   @override
23   Widget build(BuildContext context) {
24     return BlocBuilder<HomeCubit, HomeState>(
25       builder: (context, state) {
26         final bool isAdmin = context.read<HomeCubit>().isAdmin;
27         final bool isLoggedIn = token != null && token!.isNotEmpty;
28
29         return Padding(
30           padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 12),
31           child: Row(
32             mainAxisAlignment: MainAxisAlignment.spaceBetween,
33             children: [
34               Row(
35                 children: [
36                   Image.asset(
37                     AssetsHelper.logo,
38                     width: 45,
39                     height: 45,
40                     fit: BoxFit.contain,
41                   ),
42                 ],
43               ),
44               if (isLoggedIn)
45                 Row(
46                   children: [
47                     // Phone icon button - contact (Hidden for Admin)
48                     if (!isAdmin) _ContactIconButton(isAdmin: isAdmin),
49                     if (isAdmin) ...[
50                       // Logout button - admin only
51                       _AdminLogoutButton(),
52                     ],
53                     if (!isAdmin) ...[
54                       // Subscription button - non-admin only (hidden when subscribed)
55                       _SubscriptionButton(),
56                     ],
57                   ],
58                 ),
59               else
60                 _GuestMenuButton(),
61             ],
62           ),
63         );
64       },
65     );
66   }
67 }
68
69 class _ContactIconButton extends StatefulWidget {
70   final bool isAdmin;
71
72   const _ContactIconButton({required this.isAdmin});
73
74   @override
75   State<_ContactIconButton> createState() => _ContactIconButtonState();
76 }
77
78 class _ContactIconButtonState extends State<_ContactIconButton>
79   with SingleTickerProviderStateMixin {
80   late AnimationController _controller;
81   late Animation<double> _animation;
82
83   @override
84   void initState() {
85     super.initState();
86     _controller = AnimationController(

```

```

87     vsync: this,
88     duration: const Duration(milliseconds: 300),
89   )..repeat(reverse: true);
90
91   _animation = Tween<double>(begin: -0.08, end: 0.08).animate(
92     CurvedAnimation(parent: _controller, curve: Curves.easeInOut),
93   );
94 }
95
96 @override
97 void dispose() {
98   _controller.dispose();
99   super.dispose();
100 }
101
102 @override
103 Widget build(BuildContext context) {
104   return Tooltip(
105     message: appTranslation().get('contact_support'),
106     child: Material(
107       color: Colors.transparent,
108       child: InkWell(
109         onTap: () {
110           if (widget.isAdmin) {
111             context.push(Routes.adminContact);
112           } else {
113             context.push(Routes.contact);
114           }
115         },
116         borderRadius: BorderRadius.circular(50),
117         child: Container(
118           width: 42,
119           height: 42,
120           decoration: BoxDecoration(
121             shape: BoxShape.circle,
122             border: Border.all(color: ColorsManager.goldLight, width: 1.5),
123             color: ColorsManager.surfacePrimary,
124           ),
125           child: RotationTransition(
126             turns: _animation,
127             child: Icon(
128               Icons.phone_outlined,
129               size: 20,
130               color: ColorsManager.primaryColor,
131             ),
132           ),
133         ),
134       ),
135     ),
136   );
137 }
138 }
139
140 class _AdminLogoutButton extends StatelessWidget {
141   @override
142   Widget build(BuildContext context) {
143     return BlocListener<ProfileCubit, ProfileState>(
144       bloc: sl<ProfileCubit>(),
145       listener: (context, state) {
146         if (state is ProfileLogoutSuccessState) {
147           Navigator.of(context).pushNamedAndRemoveUntil(
148             Routes.login,
149             (route) => false,
150           );
151         }
152       },
153       child: PrimaryElevatedButton(
154         icon: const Icon(
155           Icons.logout,
156           size: 18,
157           color: ColorsManager.white,
158         ),
159         text: appTranslation().get('logout'),
160         textStyle: TextStylesManager.bold13.copyWith(
161           color: ColorsManager.white,

```

```

162     ),
163     onPressed: () {
164       showDialog(
165         context: context,
166         builder: (dialogContext) => LogoutDialog(
167           onConfirm: () => sl<ProfileCubit>().logout(),
168         ),
169       );
170     },
171     width: 140,
172     height: 42,
173     radius: 24,
174   ),
175 );
176 }
177 }
178
179 class _SubscriptionButton extends StatefulWidget {
180   @override
181   State<_SubscriptionButton> createState() => _SubscriptionButtonState();
182 }
183
184 class _SubscriptionButtonState extends State<_SubscriptionButton> {
185   @override
186   void initState() {
187     super.initState();
188     final cubit = sl<SubscriptionCubit>();
189     if (cubit.currentSubscription == null && cubit.lastError == null) {
190       cubit.fetchCurrentSubscription();
191     }
192   }
193
194   @override
195   Widget build(BuildContext context) {
196     return BlocBuilder<SubscriptionCubit, dynamic>(
197       bloc: sl<SubscriptionCubit>(),
198       builder: (context, state) {
199         final current = sl<SubscriptionCubit>().currentSubscription;
200         final teacher = sl<ProfileCubit>().profileModel?.teacher;
201
202         final bool isSubscribed = current?.isSubscribed ?? teacher?.isSubscribed ?? false;
203         final bool isInTrial = current?.isInTrial ?? teacher?.isInTrial ?? false;
204         final DateTime? trialEndsAt = current?.trialEndsAt ?? teacher?.trialEndsAt;
205         final DateTime? subscriptionEndsAt = current?.subscriptionEndsAt ?? teacher?.subscriptionEndsAt;
206         final DateTime? effectiveEndsAt = isSubscribed
207           ? subscriptionEndsAt
208           : (isInTrial ? trialEndsAt : (subscriptionEndsAt ?? trialEndsAt));
209
210         int daysRemaining = 0;
211         if (effectiveEndsAt != null) {
212           final now = DateTime.now();
213           if (effectiveEndsAt.isAfter(now)) {
214             daysRemaining = (effectiveEndsAt.difference(now).inHours / 24).ceil();
215           }
216         } else {
217           daysRemaining = current?.trialDaysRemaining ?? teacher?.trialDaysRemaining ?? 0;
218         }
219
220         final bool isExpired;
221         if (isSubscribed) {
222           isExpired = subscriptionEndsAt != null && subscriptionEndsAt.isBefore(DateTime.now());
223         } else if (isInTrial) {
224           isExpired = (trialEndsAt != null && trialEndsAt.isBefore(DateTime.now())) || daysRemaining <= 0;
225         } else {
226           isExpired = true;
227         }
228
229         Color buttonColor;
230         String buttonText;
231         IconData buttonIcon;
232
233         if (isSubscribed && !isExpired) {
234           // Subscribed & Active
235           buttonColor = const Color(0xFF0E7A5E);
236           buttonText = 'اشترالك نشط';

```

```

237     buttonIcon = Icons.verified_user_outlined;
238   } else if (isExpired) {
239     // Expired
240     buttonColor = const Color(0xFFDC2626);
241     buttonText = 'انتهى الاشتراك';
242     buttonIcon = Icons.lock_clock_outlined;
243   } else if (isInTrial) {
244     // Trial Active
245     if (daysRemaining <= 3) {
246       buttonColor = const Color(0xFFFF97316); // Warning Orange
247       buttonText = daysRemaining <= 1 ? 'متبقي' : 'آخر يوم تجربة' $daysRemaining 'أيام';
248       buttonIcon = Icons.hourglass_top_rounded;
249     } else {
250       buttonColor = const Color(0xFFD97706); // Amber Gold
251       buttonText = 'تجربة مجانية';
252       buttonIcon = Icons.auto_awesome;
253     }
254   } else {
255     // Default fallback
256     buttonColor = const Color(0xFF0E7A5E);
257     buttonText = 'الاشتراك';
258     buttonIcon = Icons.workspace_premium_outlined;
259   }
260
261   return Padding(
262     padding: const EdgeInsets.only(right: 8.0),
263     child: PrimaryElevatedButton(
264       icon: Icon(
265         buttonIcon,
266         size: 18,
267         color: ColorsManager.white,
268       ),
269       text: buttonText,
270       backgroundColor: buttonColor,
271       textStyle: TextStylesManager.bold13.copyWith(
272         color: ColorsManager.white,
273       ),
274       onPressed: () => SubscriptionStatusDialog.show(context),
275       width: 140,
276       height: 42,
277       radius: 24,
278     ),
279   );
280 },
281 );
282 }
283 }
284
285 class _GuestMenuButton extends StatelessWidget {
286   @override
287   Widget build(BuildContext context) {
288     return PopupMenuButton<String>(
289       onSelect: (value) {
290         if (value == 'register') {
291           context.push(Routes.register);
292         } else if (value == 'login') {
293           context.push(Routes.login);
294         }
295       },
296       shape: RoundedRectangleBorder(
297         borderRadius: BorderRadius.circular(12),
298       ),
299       itemBuilder: (context) => [
300         PopupMenuItem<String>(
301           value: 'register',
302           child: Row(
303             children: [
304               const Icon(Icons.person_add_alt_1_outlined, size: 20),
305               horizontalSpace10,
306               Text(
307                 appTranslation().get('create_account'),
308                 style: TextStylesManager.bold14,
309               ),
310             ],
311           ),

```

```

312     ),
313     PopupMenuItem<String>(
314       value: 'login',
315       child: Row(
316         children: [
317           const Icon(Icons.login_outlined, size: 20),
318           horizontalSpace10,
319           Text(
320             appTranslation().get('login'),
321             style: TextStylesManager.bold14,
322           ),
323         ],
324       ),
325     ),
326   ],
327   child: Container(
328     padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 10),
329     decoration: BoxDecoration(
330       color: ColorsManager.primaryColor,
331       borderRadius: BorderRadius.circular(10),
332     ),
333     child: Row(
334       mainAxisAlignment: MainAxisAlignment.min,
335       children: [
336         const Icon(Icons.person_add_alt_1_outlined,
337           size: 20, color: ColorsManager.white),
338         horizontalSpace8,
339         Text(
340           appTranslation().get('create_account'),
341           style: TextStylesManager.bold14.copyWith(
342             color: ColorsManager.white),
343         ),
344         horizontalSpace6,
345         const Icon(Icons.arrow_drop_down, color: ColorsManager.white),
346       ],
347     ),
348   ),
349 );
350 }
351 }
352

```

Lines • Dart 104

Feature / Home

#225 – lib/features/home/presentation/widgets/home_feature_item_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/spacing.dart';
5
6  class HomeFeatureItemWidget extends StatelessWidget {
7    final IconData icon;
8    final Color iconColor;
9    final Color iconBgColor;
10   final String title;
11   final String subtitle;
12   final bool isHighlighted;
13   final VoidCallback? onTap;
14
15   const HomeFeatureItemWidget({
16     super.key,
17     required this.icon,
18     required this.iconColor,
19     required this.iconBgColor,
20     required this.title,
21     required this.subtitle,
22     this.isHighlighted = false,
23     this.onTap,
24   });
25
26   @override
27   Widget build(BuildContext context) {
28     return GestureDetector(
29       onTap: onTap,

```

```

30 child: AnimatedContainer(
31   duration: const Duration(milliseconds: 300),
32   padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 14),
33   decoration: BoxDecoration(
34     color: isHighlighted
35       ? ColorsManager.goldMedium.withValues(alpha: 0.04)
36       : ColorsManager.surfacePrimary,
37   border: Border.all(
38     color: isHighlighted
39       ? ColorsManager.goldMedium.withValues(alpha: 0.7)
40       : ColorsManager.primaryColor.withValues(alpha: 0.2),
41     width: isHighlighted ? 1.5 : 1,
42   ),
43   borderRadius: BorderRadius.circular(16),
44   boxShadow: [
45     BoxShadow(
46       color: isHighlighted
47         ? ColorsManager.goldMedium.withValues(alpha: 0.08)
48         : ColorsManager.primaryColor.withValues(alpha: 0.03),
49       blurRadius: 8,
50       offset: const Offset(0, 2),
51     ),
52   ],
53 ),
54 child: Column(
55   mainAxisAlignment: MainAxisAlignment.center,
56   crossAxisAlignment: CrossAxisAlignment.center,
57   children: [
58     Container(
59       padding: const EdgeInsets.all(8),
60       decoration: BoxDecoration(
61         color: isHighlighted
62           ? ColorsManager.goldMedium.withValues(alpha: 0.12)
63           : iconBgColor,
64         borderRadius: BorderRadius.circular(12),
65       ),
66       child: Icon(
67         icon,
68         color: isHighlighted ? ColorsManager.goldMedium : iconColor,
69         size: 22,
70       ),
71     ),
72     verticalSpace10,
73     Text(
74       title,
75       textAlign: TextAlign.center,
76       style: TextStyleManager.bold14.copyWith(
77         color: isHighlighted
78           ? ColorsManager.goldMedium
79           : ColorsManager.mainText,
80       ),
81       fontSize: 12,
82       fontWeight: FontWeight.bold,
83       height: 1.3,
84     ),
85     maxLines: 2,
86     overflow: TextOverflow.ellipsis,
87   ),
88   verticalSpace6,
89   Text(
90     subtitle,
91     textAlign: TextAlign.center,
92     style: TextStyleManager.regular12.copyWith(
93       color: ColorsManager.secondaryText,
94       fontSize: 10,
95       height: 1.3,
96     ),
97     maxLines: 3,
98     overflow: TextOverflow.ellipsis,
99   ),
100 ],
101 ),
102 );

```

```
103 }
104 }
```

Lines • Dart 465

Feature / Home

#226 – lib/features/home/presentation/widgets/home_features_section_widget.dart

```
1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/utils/cubit/theme/theme_cubit.dart';
4 import 'package:moean/core/utils/cubit/theme/theme_state.dart';
5 import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
6 import 'package:moean/features/home/presentation/cubit/home_cubit.dart';
7 import 'package:moean/core/theme/colors.dart';
8 import 'package:moean/core/theme/text_styles.dart';
9 import 'package:moean/core/utils/constants/constants.dart';
10 import 'package:moean/core/utils/constants/routes.dart';
11 import 'package:moean/core/utils/constants/spacing.dart';
12 import 'package:moean/core/utils/extensions/context_extension.dart';
13 import 'package:moean/features/home/presentation/widgets/home_action_chip_widget.dart';
14 import 'package:moean/features/home/presentation/widgets/home_feature_item_widget.dart';
15
16 class HomeFeaturesSectionWidget extends StatefulWidget {
17   const HomeFeaturesSectionWidget({super.key});
18
19   @override
20   State<HomeFeaturesSectionWidget> createState() => _HomeFeaturesSectionWidgetState();
21 }
22
23 class _HomeFeaturesSectionWidgetState extends State<HomeFeaturesSectionWidget> with SingleTickerProviderState
24 Mixin {
25   int _selectedFeatureIndex = 1;
26   late AnimationController _controller;
27   late Animation<double> _animation;
28
29   @override
30   void initState() {
31     super.initState();
32     _controller = AnimationController(
33       vsync: this,
34       duration: const Duration(milliseconds: 1200),
35     )..repeat(reverse: true);
36
37     _animation = Tween<double>(begin: 0.0, end: 1.0).animate(
38       CurvedAnimation(parent: _controller, curve: Curves.easeInOut),
39     );
40
41   @override
42   void dispose() {
43     _controller.dispose();
44     super.dispose();
45   }
46
47   @override
48   Widget build(BuildContext context) {
49     return BlocBuilder<ThemeCubit, ThemeState>(
50       builder: (context, themeState) {
51         return Directionality(
52           textDirection: TextDirection.rtl,
53           child: Column(
54             crossAxisAlignment: CrossAxisAlignment.stretch,
55             children: [
56               // Start Prep Button with Pulse Effect
57               Padding(
58                 padding: const EdgeInsets.symmetric(horizontal: 30),
59                 child: Align(
60                   alignment: AlignmentDirectional.centerStart,
61                   child: BlocBuilder<HomeCubit, HomeState>(
62                     builder: (context, state) {
63                       final isAdmin = context.read<HomeCubit>().isAdmin;
64                       final isLoadingRole = context.read<HomeCubit>().isLoadingRole;
65
66                       return AnimatedBuilder(
67                         animation: _animation,
```

```

68         builder: (context, child) {
69             return Container(
70                 decoration: BoxDecoration(
71                     borderRadius: BorderRadius.circular(30),
72                     boxShadow: [
73                         BoxShadow(
74                             color: ColorsManager.primaryColor.withValues(alpha: 0.2 + (_animation.value *
0.2)),
75                             blurRadius: 10 + (_animation.value * 5),
76                             spreadRadius: _animation.value * 2,
77                             offset: const Offset(0, 4),
78                         ),
79                     ],
80                 ),
81                 child: InkWell(
82                     onTap: () {
83                         if (token != null && token!.isNotEmpty) {
84                             if (isAdmin) {
85                                 context.push(Routes.adminTeachers);
86                             } else {
87                                 context.push(Routes.choseapp);
88                             }
89                         } else {
90                             context.push(Routes.login);
91                         }
92                     },
93                     borderRadius: BorderRadius.circular(30),
94                     child: Container(
95                         padding: const EdgeInsets.symmetric(horizontal: 44, vertical: 22),
96                         decoration: BoxDecoration(
97                             color: ColorsManager.primaryColor,
98                             borderRadius: BorderRadius.circular(30),
99                             border: Border.all(
100                                 color: Colors.white.withValues(alpha: _animation.value * 0.3),
101                                 width: 2,
102                             ),
103                     ),
104                     child: Row(
105                         mainAxisAlignment: MainAxisAlignment.min,
106                         children: [
107                             if (isLoadingRole)
108                                 const SizedBox(
109                                     width: 16,
110                                     height: 16,
111                                     child: CircularProgressIndicator(
112                                         strokeWidth: 2,
113                                         color: ColorsManager.white,
114                                     ),
115                                 )
116                             else
117                                 Text(
118                                     isAdmin
119                                     ? appTranslation().get('admin_dashboard')
120                                     : appTranslation().get('home_start_prep'),
121                                     style: TextStylesManager.bold16.copyWith(
122                                         color: ColorsManager.white,
123                                     ),
124                                 ),
125                             horizontalSpace8,
126                             const Icon(
127                                 Icons.arrow_forward_ios_rounded,
128                                 color: ColorsManager.white,
129                                 size: 16,
130                             ),
131                         ],
132                     ),
133                 ),
134             ),
135         );
136     },
137 );
138 },
139 ),
140 ),
141 ),

```



```

142     verticalSpace40,
143     BlocBuilder<HomeCubit, HomeState>(
144       builder: (context, state) {
145         final isAdmin = context.read<HomeCubit>().isAdmin;
146         if (isAdmin) return const SizedBox.shrink();
147         return Column(
148           children: [
149             Padding(
150               padding: const EdgeInsets.symmetric(horizontal: 24),
151               child: PrimaryTextField(
152                 controller: TextEditingController(),
153                 hint: appTranslation().get('search_title'),
154                 readOnly: true,
155                 prefixIcon: Icon(Icons.search_rounded, color: ColorsManager.textSecondary),
156                 onTap: () {
157                   if (token != null && token!.isNotEmpty) {
158                     context.push(Routes.search);
159                   } else {
160                     context.push(Routes.login);
161                   }
162                 },
163               ),
164             ),
165             verticalSpace24,
166           ],
167         );
168       },
169     ),
170     // Action Chips Row
171     BlocBuilder<HomeCubit, HomeState>(
172       builder: (context, state) {
173         final isAdmin = context.read<HomeCubit>().isAdmin;
174         return SingleChildScrollView(
175           scrollDirection: Axis.horizontal,
176           physics: const BouncingScrollPhysics(),
177           padding: const EdgeInsets.symmetric(horizontal: 15),
178           child: Row(
179             children: [
180               // Search chip - visible for all users
181               // HomeActionChipWidget(
182               //   icon: Icons.search_rounded,
183               //   title: appTranslation().get('search_title'),
184               //   onTap: () => context.push(Routes.search),
185               // ),
186               // horizontalSpace12,
187
188               if (!isAdmin) ...[
189                 HomeActionChipWidget(
190                   icon: Icons.co_present_rounded,
191                   title: appTranslation().get('presentations_title'),
192                   onTap: () {
193                     if (token != null && token!.isNotEmpty) {
194                       context.push(Routes.presentations);
195                     } else {
196                       context.push(Routes.login);
197                     }
198                   },
199                 ),
200                 horizontalSpace12,
201               ],
202               if (!isAdmin) ...[
203                 HomeActionChipWidget(
204                   icon: Icons.monitor,
205                   title: appTranslation().get('home_reports'),
206                   onTap: () {
207                     if (token != null && token!.isNotEmpty) {
208                       context.push(Routes.reports);
209                     } else {
210                       context.push(Routes.login);
211                     }
212                   },
213                 ),
214                 horizontalSpace12,
215               ],
216               HomeActionChipWidget(

```

```

217         icon: isAdmin ? Icons.assignment_turned_in_outlined : Icons.verified_outlined,
218         title: isAdmin ? "مراجعة الأسئلة" : appTranslation().get('my_exam'),
219         onTap: () {
220             if (token != null && token!.isNotEmpty) {
221                 if (isAdmin) {
222                     context.push(Routes.adminExams);
223                 } else {
224                     context.push(Routes.examGenerationInfo);
225                 }
226             } else {
227                 context.push(Routes.login);
228             }
229         },
230     ),
231     if (isAdmin) ...[
232         horizontalSpace12,
233         HomeActionChipWidget(
234             icon: Icons.check_circle_outline,
235             title: "المدفوعات",
236             onTap: () {
237                 if (token != null && token!.isNotEmpty) {
238                     context.push(Routes.adminPayments);
239                 } else {
240                     context.push(Routes.login);
241                 }
242             },
243         ),
244         horizontalSpace12,
245         HomeActionChipWidget(
246             icon: Icons.verified_user_outlined,
247             title: "تذاكر التواصل",
248             onTap: () {
249                 if (token != null && token!.isNotEmpty) {
250                     context.push(Routes.adminContact);
251                 } else {
252                     context.push(Routes.login);
253                 }
254             },
255         ),
256         horizontalSpace12,
257         HomeActionChipWidget(
258             icon: Icons.discount_outlined,
259             title: "أكواد الخصم",
260             onTap: () {
261                 if (token != null && token!.isNotEmpty) {
262                     context.push(Routes.adminPromo);
263                 } else {
264                     context.push(Routes.login);
265                 }
266             },
267         ),
268     ],
269     if (!isAdmin) ...[
270         horizontalSpace12,
271         HomeActionChipWidget(
272             icon: Icons.description_outlined,
273             title: "الاختبارات السابقة",
274             //appTranslation().get('home_tests'),
275             onTap: () {
276                 if (token != null && token!.isNotEmpty) {
277                     context.push(Routes.myExams);
278                 } else {
279                     context.push(Routes.login);
280                 }
281             },
282         ),
283     ],
284     horizontalSpace12,
285     HomeActionChipWidget(
286         icon: Icons.workspace_premium_rounded,
287         title: appTranslation().get('certificates'),
288         onTap: () {
289             if (token != null && token!.isNotEmpty) {

```

```

        context.push(Routes.certificates);
      } else {
        context.push(Routes.login);
      }
    },
  ),
  horizontalSpace12,
  HomeActionChipWidget(
    icon: Icons.people_alt_outlined,
    title: appTranslation().get('referral_chip_label'),
    onTap: () {
      if (token != null && token!.isNotEmpty) {
        context.push(Routes.referral);
      } else {
        context.push(Routes.login);
      }
    },
  ),
  horizontalSpace12,
  HomeActionChipWidget(
    icon: Icons.calendar_month_outlined,
    title: 'توزيع المناهج',
    onTap: () {
      if (token != null && token!.isNotEmpty) {
        context.push(Routes.curriculumDistribution);
      } else {
        context.push(Routes.login);
      }
    },
  ),
  horizontalSpace12,
  HomeActionChipWidget(
    icon: Icons.menu_book_rounded,
    title: 'الكتب الدراسية',
    onTap: () {
      if (token != null && token!.isNotEmpty) {
        context.push(Routes.curriculumBooks);
      } else {
        context.push(Routes.login);
      }
    },
  ),
),
],
),
),
);
),
verticalSpace24,

Padding(
  padding: const EdgeInsets.symmetric(horizontal: 16),
  child: IntrinsicHeight(
    child: Row(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        Expanded(
          child: HomeFeatureItemWidget(
            icon: Icons.menu_book_rounded,
            iconColor: ColorsManager.primaryColor,
            iconBgColor: ColorsManager.primaryColor.withValues(alpha: 0.1),
            title: appTranslation().get('feature_curriculum_title'),
            subtitle: appTranslation().get('feature_curriculum_subtitle'),
            isHighlighted: _selectedFeatureIndex == 0,
            onTap: () {
              setState(() {
                _selectedFeatureIndex = 0;
              });
            },
          ),
        ),
        Expanded(
          child: HomeFeatureItemWidget(
            icon: Icons.access_time_filled,

```

```

367         iconColor: ColorsManager.primaryColor,
368         iconBgColor: ColorsManager.primaryColor.withValues(alpha: 0.1),
369         title: appTranslation().get('feature_time_title'),
370         subtitle: appTranslation().get('feature_time_subtitle'),
371         isHighlighted: _selectedFeatureIndex == 1,
372         onTap: () {
373             setState(() {
374                 _selectedFeatureIndex = 1;
375             });
376         },
377     ),
378 ),
379 horizontalSpace8,
380
381 Expanded(
382     child: HomeFeatureItemWidget(
383         icon: Icons.shield_outlined,
384         iconColor: ColorsManager.primaryColor,
385         iconBgColor: ColorsManager.primaryColor.withValues(alpha: 0.1),
386         title: appTranslation().get('feature_security_title'),
387         subtitle: appTranslation().get('feature_security_subtitle'),
388         isHighlighted: _selectedFeatureIndex == 2,
389         onTap: () {
390             setState(() {
391                 _selectedFeatureIndex = 2;
392             });
393         },
394     ),
395 ),
396 ],
397 ),
398 ),
399 ),
400
401 // Teacher-only: Curriculum Distribution & Books
402 BlocBuilder<HomeCubit, HomeState>(
403     builder: (context, state) {
404         final isAdmin = context.read<HomeCubit>().isAdmin;
405         if (isAdmin) return const SizedBox.shrink();
406         return Column(
407             children: [
408                 verticalSpace10,
409                 Padding(
410                     padding: const EdgeInsets.symmetric(horizontal: 16),
411                     child: IntrinsicHeight(
412                         child: Row(
413                             crossAxisAlignment: CrossAxisAlignment.stretch,
414                             children: [
415                                 Expanded(
416                                     child: HomeFeatureItemWidget(
417                                         icon: Icons.calendar_month_outlined,
418                                         iconColor: ColorsManager.primaryColor,
419                                         iconBgColor: ColorsManager.primaryColor.withValues(alpha: 0.1),
420                                         title: 'توزيع المناهج',
421                                         subtitle: 'توزيع المنهج على أسابيع الفصل الدراسي',
422                                         isHighlighted: false,
423                                         onTap: () {
424                                             if (token != null && token!.isNotEmpty) {
425                                                 context.push(Routes.curriculumDistribution);
426                                             } else {
427                                                 context.push(Routes.login);
428                                             }
429                                         },
430                                     ),
431                                 ),
432                                 horizontalSpace8,
433                                 Expanded(
434                                     child: HomeFeatureItemWidget(
435                                         icon: Icons.menu_book_rounded,
436                                         iconColor: ColorsManager.primaryColor,
437                                         iconBgColor: ColorsManager.primaryColor.withValues(alpha: 0.1),
438                                         title: 'الكتب الدراسية',
439                                         subtitle: 'كتب المناهج الدراسية قابلة للتحميل',
440                                         isHighlighted: false,
441                                         onTap: () {

```

```

442         if (token != null && token!.isNotEmpty) {
443             context.push(Routes.curriculumBooks);
444         } else {
445             context.push(Routes.login);
446         }
447     },
448 ),
449 ),
450 ],
451 ),
452 ),
453 ),
454 ],
455 );
456 },
457 ),
458 ],
459 ),
460 );
461 },
462 );
463 }
464 }
465

```

Lines • Dart 118

Feature / Home

#227 – lib/features/home/presentation/widgets/home_hero_banner_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/assets_helper.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7
8  class HomeHeroBannerWidget extends StatelessWidget {
9      const HomeHeroBannerWidget({super.key});
10
11      @override
12      Widget build(BuildContext context) {
13          return Directionality(
14              textDirection: TextDirection.rtl,
15              child: Padding(
16                  padding: const EdgeInsets.only(right: 20),
17                  child: Row(
18                      mainAxisAlignment: MainAxisAlignment.spaceBetween,
19                      children: [
20                          Expanded(
21                              flex: 7,
22                              child: Padding(
23                                  padding: EdgeInsets.symmetric(vertical: 24),
24                                  child: Column(
25                                      mainAxisAlignment: MainAxisAlignment.center,
26                                      crossAxisAlignment: CrossAxisAlignment.start,
27                                      children: [
28
29                                          Stack(
30                                              clipBehavior: Clip.none,
31                                              alignment: Alignment.center,
32                                              children: [
33                                                  Positioned(
34                                                      bottom: 5,
35                                                      child: Container(
36                                                          width: 100,
37                                                          height: 10,
38                                                          decoration: BoxDecoration(
39                                                              color: ColorsManager.goldDark,
40                                                              borderRadius: BorderRadius.circular(10),
41                                                          ),
42                                                  ),
43                          ],
44                      ),
45                      Text(
46                          appTranslation().get('home_hero_title'),

```

```

47         textAlign: TextAlign.start,
48         style: TextStyleManager.bold48.copyWith(
49             color: ColorsManager.secondarytext,
50         ),
51     ),
52
53     Positioned(
54         right: -25,
55         top: 20,
56         child: Container(
57             width: 14,
58             height: 14,
59             decoration: BoxDecoration(
60                 color: ColorsManager.primaryColor,
61                 shape: BoxShape.circle,
62             ),
63         ),
64     ),
65
66     Positioned(
67         top: 0,
68         left: -20,
69         child: Container(
70             width: 16,
71             height: 16,
72             decoration: BoxDecoration(
73                 color: ColorsManager.goldDark,
74                 shape: BoxShape.circle,
75             ),
76         ),
77     ),
78 ],
79 ),
80
81         verticalSpace20,
82
83         Text(
84             appTranslation().get('home_hero_subtitle'),
85             textAlign: TextAlign.start,
86             style: TextStyleManager.bold20.copyWith(
87                 color: ColorsManager.secondarytext,
88             ),
89         ),
90
91         verticalSpace20,
92
93         Text(
94             appTranslation().get('home_hero_description'),
95             textAlign: TextAlign.start,
96             style: TextStyleManager.medium14.copyWith(
97                 color: ColorsManager.textPrimary,
98                 height: 1.6,
99             ),
100         ),
101     ],
102 ),
103 ),
104
105     Expanded(
106         flex: 5,
107         child: Image.asset(
108             AssetsHelper.img9,
109             fit: BoxFit.cover,
110             alignment: Alignment.centerRight,
111         ),
112     ),
113 ],
114 ),
115 ),
116 );
117 }
118 }

```

```
1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/routes.dart';
6 import 'package:moean/core/utils/constants/spacing.dart';
7 import 'package:moean/core/utils/extensions/context_extension.dart';
8
9 class HomeSearchSectionWidget extends StatelessWidget {
10   const HomeSearchSectionWidget({super.key});
11
12   @override
13   Widget build(BuildContext context) {
14     return Padding(
15       padding: const EdgeInsets.symmetric(horizontal: 16),
16       child: Column(
17         crossAxisAlignment: CrossAxisAlignment.stretch,
18         children: [
19           const _SearchSectionBadgeWidget(),
20           verticalSpace16,
21           const _SearchSectionHeaderWidget(),
22           verticalSpace8,
23           const _SearchSectionSubtitleWidget(),
24           verticalSpace24,
25           const _SearchPreviewFieldWidget(),
26           verticalSpace16,
27           const _SearchNowButtonWidget(),
28         ],
29       ),
30     );
31   }
32 }
33
34 class _SearchSectionBadgeWidget extends StatelessWidget {
35   const _SearchSectionBadgeWidget();
36
37   @override
38   Widget build(BuildContext context) {
39     return Align(
40       alignment: AlignmentDirectional.center,
41       child: Container(
42         padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 8),
43         decoration: BoxDecoration(
44           color: ColorsManager.surfacePrimary,
45           borderRadius: BorderRadius.circular(30),
46           border: Border.all(color: ColorsManager.borderColor),
47           boxShadow: [
48             BoxShadow(
49               color: ColorsManager.primaryColor.withValues(alpha: 0.08),
50               blurRadius: 12,
51               offset: const Offset(0, 2),
52             ),
53           ],
54         ),
55       child: Row(
56         mainAxisAlignment: MainAxisAlignment.min,
57         children: [
58           Icon(
59             Icons.auto_awesome_rounded,
60             size: 16,
61             color: ColorsManager.primaryColor,
62           ),
63           horizontalSpace8,
64           Text(
65             appTranslation().get('search_section_badge'),
66             style: TextStylesManager.bold13.copyWith(
67               color: ColorsManager.textPrimary,
68             ),
69         ),
70       ],
71     ),
72   ),
```

```

73     );
74   }
75 }
76
77 class _SearchSectionHeaderWidget extends StatelessWidget {
78   const _SearchSectionHeaderWidget();
79
80   @override
81   Widget build(BuildContext context) {
82     return Text(
83       appTranslation().get('search_section_title'),
84       style: TextStylesManager.alohaBold28.copyWith(
85         color: ColorsManager.primaryColor,
86         height: 1.4,
87       ),
88       textAlign: TextAlign.center,
89     );
90   }
91 }
92
93 class _SearchSectionSubtitleWidget extends StatelessWidget {
94   const _SearchSectionSubtitleWidget();
95
96   @override
97   Widget build(BuildContext context) {
98     return Text(
99       appTranslation().get('search_section_subtitle'),
100       style: TextStylesManager.regular14.copyWith(
101         color: ColorsManager.secondaryText,
102         height: 1.6,
103       ),
104       textAlign: TextAlign.center,
105     );
106   }
107 }
108
109 class _SearchPreviewFieldWidget extends StatelessWidget {
110   const _SearchPreviewFieldWidget();
111
112   @override
113   Widget build(BuildContext context) {
114     return GestureDetector(
115       onTap: () => context.push(Routes.search),
116       child: Container(
117         padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 16),
118         decoration: BoxDecoration(
119           color: ColorsManager.surfacePrimary,
120           borderRadius: BorderRadius.circular(16),
121           border: Border.all(color: ColorsManager.borderColor),
122           boxShadow: [
123             BoxShadow(
124               color: ColorsManager.primaryColor.withValues(alpha: 0.06),
125               blurRadius: 10,
126               offset: const Offset(0, 3),
127             ),
128           ],
129       ),
130       child: Row(
131         children: [
132           Icon(
133             Icons.search_rounded,
134             color: ColorsManager.primaryColor,
135             size: 22,
136           ),
137           horizontalSpace12,
138           Expanded(
139             child: Text(
140               appTranslation().get('search_home_hint'),
141               style: TextStylesManager.regular14.copyWith(
142                 color: ColorsManager.placeholder,
143               ),
144             ),
145           ),
146         ],
147       ),

```



```

38         style: TextStylesManager.bold14.copyWith(
39             color: ColorsManager.primaryColor,
40         ),
41     ),
42     verticalSpace6,
43     Text(
44         appTranslation().get('tip_of_day_content'),
45         textAlign: TextAlign.right,
46         style: TextStylesManager.regular12.copyWith(
47             color: ColorsManager.textPrimary,
48             height: 1.4,
49         ),
50         maxLines: 2,
51         overflow: TextOverflow.ellipsis,
52     ),
53 ],
54 ),
55 ),
56 horizontalSpace12,
57 Container(
58     width: 48,
59     height: 48,
60     decoration: BoxDecoration(
61         color: ColorsManager.surfacePrimary,
62         shape: BoxShape.circle,
63     ),
64     child: Image.asset(AssetsHelper.tips, width: 24, height: 24),
65 ),
66 ],
67 ),
68 );
69 }
70 }
71

```

Lines • Dart 38

Feature / Layout

#230 – lib/features/layout/presentation/cubit/layout_cubit.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:curved_navigation_bar/curved_navigation_bar.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/routes.dart';
6  import 'package:moean/features/home/presentation/screen/home_screen.dart';
7  import 'package:moean/features/profile/presentation/screen/profile_screen.dart';
8  import 'package:moean/features/profile/presentation/screen/settings_screen.dart';
9  import 'package:moean/features/layout/presentation/cubit/layout_state.dart';
10
11 class LayoutCubit extends Cubit<LayoutState> {
12     LayoutCubit() : super(LayoutInitial());
13
14     static LayoutCubit get(BuildContext context) => BlocProvider.of<LayoutCubit>(context);
15
16     int currentIndex = 1;
17     final GlobalKey<CurvedNavigationBarState> bottomNavigationBarKey = GlobalKey();
18
19     List<Widget> screens = [
20         const ProfileScreen(),
21         const HomeScreen(),
22         const SettingsScreen(),
23     ];
24
25     void changeBottomNav(int index, BuildContext context) {
26         if (index == 0 || index == 2) {
27             if (token == null || token!.isEmpty) {
28                 // Not logged in, prevent navigation to Profile or Settings
29                 bottomNavigationBarKey.currentState?.setPage(currentIndex);
30                 Navigator.pushNamed(context, Routes.login);
31                 return;
32             }
33         }
34         currentIndex = index;
35         emit(LayoutChangeBottomNavState());
36     }

```

```
37 }
38
```

Lines • Dart 6

Feature / Layout

#231 – lib/features/layout/presentation/cubit/layout_state.dart

```
1 abstract class LayoutState {}
2
3 class LayoutInitial extends LayoutState {}
4
5 class LayoutChangeBottomNavState extends LayoutState {}
6
```

Lines • Dart 105

Feature / Layout

#232 – lib/features/layout/presentation/screen/main_layout_screen.dart

```
1 import 'package:flutter/material.dart';
2 import 'package:flutter/services.dart';
3 import 'package:flutter_bloc/flutter_bloc.dart';
4 import 'package:curved_navigation_bar/curved_navigation_bar.dart';
5 import 'package:moean/core/di/injections.dart';
6 import 'package:moean/core/theme/colors.dart';
7 import 'package:moean/features/layout/presentation/cubit/layout_cubit.dart';
8 import 'package:moean/features/layout/presentation/cubit/layout_state.dart';
9 import 'package:moean/features/payment/presentation/cubit/subscription_cubit.dart';
10 import 'package:moean/features/payment/presentation/cubit/subscription_state.dart';
11 import 'package:moean/features/payment/presentation/widgets/trial_banner_widget.dart';
12 import 'package:moean/features/profile/presentation/cubit/profile_cubit.dart';
13 import 'package:moean/features/profile/presentation/cubit/profile_state.dart';
14 import 'package:moean/features/verify_email/presentation/widgets/email_verification_banner.dart';
15
16 class MainLayoutScreen extends StatelessWidget {
17   const MainLayoutScreen({super.key});
18
19   @override
20   Widget build(BuildContext context) {
21     return BlocProvider(
22       create: (context) => LayoutCubit(),
23       child: BlocBuilder<LayoutCubit, LayoutState>(
24         builder: (context, state) {
25           var cubit = LayoutCubit.get(context);
26
27           // Get the bottom system inset (navigation bar / gesture bar height)
28           final double bottomPadding =
29             MediaQuery.of(context).padding.bottom;
30
31           return AnnotatedRegion<SystemUiOverlayStyle>(
32             value: const SystemUiOverlayStyle(
33               statusBarColor: Colors.transparent,
34               systemNavigationBarColor: Colors.transparent,
35               systemNavigationBarDividerColor: Colors.transparent,
36             ),
37             child: Scaffold(
38               // Allow the body to extend behind the bottom nav bar
39               extendBody: true,
40               body: BlocBuilder<SubscriptionCubit, SubscriptionState>(
41                 bloc: sl<SubscriptionCubit>(),
42                 builder: (context, subState) {
43                   final bool showBanner =
44                     TrialBannerWidget.isBannerVisible(subState);
45                   return Column(
46                     children: [
47                       BlocBuilder<ProfileCubit, ProfileState>(
48                         bloc: sl<ProfileCubit>(),
49                         builder: (context, profState) {
50                           final user = sl<ProfileCubit>().profileModel?.user;
51                           if (user == null || user.isEmailVerified) {
52                             return const SizedBox.shrink();
53                           }
54                           return EmailVerificationBanner(
55                             isEmailVerified: user.isEmailVerified,
56                             email: user.email,
57                           );
58                         },
59                       ),
60                     ],
61                   );
62                 },
63               ),
64             ),
65           );
66         },
67       ),
68     );
69   }
70 }
```

```

59         ),
60         const TrialBannerWidget(),
61         Expanded(
62           child: MediaQuery.removePadding(
63             context: context,
64             removeTop: showBanner,
65             child: IndexedStack(
66               index: cubit.currentIndex,
67               children: cubit.screens,
68             ),
69           ),
70         ),
71       ],
72     );
73   },
74 ),
75 bottomNavigationBar: Padding(
76   // Push the nav bar above the system navigation area
77   padding: EdgeInsets.only(bottom: bottomPadding),
78   child: CurvedNavigationBar(
79     key: cubit.bottomNavigationKey,
80     index: cubit.currentIndex,
81     height: 60.0,
82     items: <Widget>[
83       Icon(Icons.person, size: 30, color: ColorsManager.white),
84       Icon(Icons.home, size: 30, color: ColorsManager.white),
85       Icon(Icons.settings, size: 30, color: ColorsManager.white),
86     ],
87     color: ColorsManager.primaryColor,
88     buttonBackgroundColor: ColorsManager.primaryColor,
89     backgroundColor: Colors.transparent,
90     animationCurve: Curves.easeInOut,
91     animationDuration: const Duration(milliseconds: 300),
92     onTap: (index) {
93       cubit.changeBottomNav(index, context);
94     },
95     letIndexChange: (index) => true,
96   ),
97 ),
98 ),
99 );
100 },
101 ),
102 );
103 }
104 }
105

```

Lines • Dart 130

Feature / Login

#233 – lib/features/login/presentation/cubit/login_cubit.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/features/login/presentation/cubit/login_state.dart';
4  import 'package:moean/main.dart';
5  import 'package:moean/core/network/remote/api_service.dart';
6  import 'package:moean/core/models/login_request.dart';
7  import 'package:moean/core/models/madrasati_session_data.dart';
8  import 'package:moean/core/network/local/secure_storage_helper.dart';
9  import 'package:moean/core/services/madrasati_session_service.dart';
10 import 'package:moean/core/utils/constants/constants.dart';
11 import 'package:moean/core/di/injections.dart';
12 import 'package:moean/features/payment/presentation/cubit/subscription_cubit.dart';
13 import 'package:moean/features/profile/presentation/cubit/profile_cubit.dart';
14 import 'package:moean/core/models/profile_model.dart';
15
16 LoginCubit get loginCubit => LoginCubit.get(navigatorKey.currentContext!);
17
18 class LoginCubit extends Cubit<LoginState> {
19   LoginCubit() : super(LoginInitialState());
20
21   static LoginCubit get(BuildContext context) => BlocProvider.of(context);
22
23   final TextEditingController emailController = TextEditingController();

```

```

24 final TextEditingController passwordController = TextEditingController();
25 final GlobalKey<FormState> formKey = GlobalKey<FormState>();
26
27 bool isPasswordVisible = false;
28 bool rememberMe = false;
29
30 void togglePasswordVisibility() {
31     isPasswordVisible = !isPasswordVisible;
32     emit(LoginPasswordVisibilityChangedState());
33 }
34
35 void toggleRememberMe() {
36     rememberMe = !rememberMe;
37     emit(LoginRememberMeChangedState());
38 }
39
40 Future<void> login() async {
41     if (!(formKey.currentState?.validate() ?? false)) return;
42     if (!isClosed) emit(LoginLoadingState());
43
44     final request = LoginRequest(
45         email: emailController.text.trim(),
46         password: passwordController.text,
47     );
48
49     final result = await ApiService.loginUser(request);
50     if (isClosed) return;
51
52     result.fold(
53         (error) {
54             if (!isClosed) {
55                 if (error.contains('account_suspended') ||
56                     error.contains('__403__') ||
57                     error.contains('تعليق') ||
58                     error.contains('موقوف')) {
59                     emit(LoginAccountSuspendedState(
60                         email: emailController.text.trim(),
61                         password: passwordController.text,
62                         message: error.replaceFirst('__403__:account_suspended:', ''),
63                     ));
64                 } else {
65                     emit(LoginErrorState(message: error));
66                 }
67             }
68         },
69         (response) async {
70             if (!response.user.isActive) {
71                 if (!isClosed) {
72                     emit(LoginAccountSuspendedState(
73                         email: response.user.email,
74                         userName: response.user.name,
75                     ));
76                 }
77                 return;
78             }
79
80             // Save token securely (replaces CacheHelper for sensitive data)
81             await sl<SecureStorageHelper>().saveToken(response.token);
82             token = response.token;
83
84             // If already connected to Madrasati, persist school_id locally
85             // so HeadlessRefresh can use it when session expires
86             if (response.madrasatiSchoolId != null &&
87                 response.madrasatiSchoolId!.isNotEmpty) {
88                 await sl<SecureStorageHelper>().saveMadrasatiSession(
89                     MadrasatiSessionData(
90                         sessionCookie: '', // cookies managed by WebView internally
91                         schoolId: response.madrasatiSchoolId!,
92                         expiresAt: null,
93                     ),
94                 );
95             }
96             sl<MadrasatiSessionService>().notifySessionActive();
97         }
98     );

```

```

99     if (response.teacher != null) {
100         sl<ProfileCubit>().profileModel = ProfileModel(
101             user: response.user,
102             madrasatiConnected: response.madrasatiConnected,
103             teacher: response.teacher,
104             subscriptionName: response.teacher?.subscription?.name,
105             aiQuotaRemaining: response.teacher?.aiQuotaRemaining,
106         );
107     }
108
109     // Refresh subscription & profile state in background
110     sl<SubscriptionCubit>().fetchCurrentSubscription(forceRefresh: true);
111     sl<ProfileCubit>().fetchProfile(forceRefresh: true);
112
113     if (!isClosed) {
114         emit(LoginSuccessState(
115             madrasatiConnected: response.madrasatiConnected,
116             userEmail: response.user.email,
117         ));
118     }
119 },
120 );
121 }
122
123 @override
124 Future<void> close() {
125     emailController.dispose();
126     passwordController.dispose();
127     return super.close();
128 }
129 }
130

```

Lines • Dart 37

Feature / Login

#234 – lib/features/login/presentation/cubit/login_state.dart

```

1  abstract class LoginState {}
2
3  class LoginInitialState extends LoginState {}
4
5  class LoginLoadingState extends LoginState {}
6
7  class LoginSuccessState extends LoginState {
8      final bool madrasatiConnected;
9      final String userEmail;
10     LoginSuccessState({required this.madrasatiConnected, required this.userEmail});
11 }
12
13 class LoginErrorState extends LoginState {
14     final String message;
15     LoginErrorState({required this.message});
16 }
17
18 class LoginAccountSuspendedState extends LoginState {
19     final String? email;
20     final String? password;
21     final String? message;
22     final String? userName;
23
24     LoginAccountSuspendedState({
25         this.email,
26         this.password,
27         this.message,
28         this.userName,
29     });
30 }
31
32 class LoginPasswordVisibilityChangedState extends LoginState {}
33
34 class LoginRememberMeChangedState extends LoginState {}
35
36 class LoginUpdateUIState extends LoginState {}
37

```

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/di/injections.dart';
4  import 'package:moean/core/models/madrasati_session_data.dart';
5  import 'package:moean/core/network/local/secure_storage_helper.dart';
6  import 'package:moean/core/network/remote/api_endpoints.dart';
7  import 'package:moean/core/network/remote/dio_helper.dart';
8  import 'package:moean/core/services/madrasati_session_service.dart';
9
10 abstract class MadrasatiState {}
11
12 class MadrasatiInitialState extends MadrasatiState {}
13
14 class MadrasatiLoadingState extends MadrasatiState {}
15
16 class MadrasatiSuccessState extends MadrasatiState {
17   final String message;
18   MadrasatiSuccessState(this.message);
19 }
20
21 class MadrasatiErrorState extends MadrasatiState {
22   final String message;
23   MadrasatiErrorState(this.message);
24 }
25
26 class MadrasatiCubit extends Cubit<MadrasatiState> {
27   MadrasatiCubit() : super(MadrasatiInitialState());
28
29   static MadrasatiCubit get(BuildContext context) => BlocProvider.of(context);
30
31   /// Connects the app to Madrasati by sending the session cookie.
32   ///
33   /// [sessionCookie] - The `AspNetCore.Cookies` session string extracted
34   ///   from the webview after the initial manual login.
35   /// [madrasatiSchoolId] - The school's unique ID.
36   /// [expiresAt] - Session expiry datetime string.
37   /// [refreshToken] - The Microsoft OAuth refresh_token returned by the backend
38   ///   after connecting. Used by [MadrasatiSessionInterceptor] to silently
39   ///   refresh the session in the background when it expires.
40   Future<void> connectMadrasati({
41     required String sessionCookie,
42     required String madrasatiSchoolId,
43     required String expiresAt,
44     String? refreshToken,
45   }) async {
46     // =====
47     debugPrint('-----');
48     debugPrint('🚀 [MadrasatiCubit] connectMadrasati() — START');
49     debugPrint('  schoolId   : $madrasatiSchoolId');
50     debugPrint('  expiresAt   : $expiresAt');
51     debugPrint('  refreshToken (from caller): ${refreshToken != null ? '✅' : '❌'} "${refreshToken?.substring(0, refreshToken.length.clamp(0, 20))}..." : '❌ null'');
52     debugPrint('-----');
53
54     if (!isClosed) emit(MadrasatiLoadingState());
55     debugPrint('📡 [MadrasatiCubit] State → MadrasatiLoadingState');
56
57     try {
58       debugPrint('📡 [MadrasatiCubit] Sending POST to: $connectMadrasatiApi');
59       final response = await DioHelper.postData(
60         url: connectMadrasatiApi,
61         data: {
62           'session_cookie': sessionCookie,
63           'madrasati_school_id': madrasatiSchoolId,
64           'expires_at': expiresAt,
65         },
66       );
67
68       if (isClosed) return;
69

```

```

70 response.fold(
71     (error) {
72         debugPrint('✖ [MadrasatiCubit] API returned LEFT (error): $error');
73         debugPrint('✖ [MadrasatiCubit] State → MadrasatiErrorState');
74         if (!isClosed) emit(MadrasatiErrorState(error));
75     },
76     (res) async {
77         debugPrint('👉 [MadrasatiCubit] API returned RIGHT (success path)');
78         debugPrint('    statusCode : $res.statusCode');
79         debugPrint('    statusCode : $res');
80
81         // — طباعة كل حقل بيرجع من الباك بالاسم —
82         debugPrint('');
83         debugPrint('');
84         debugPrint('📦 BACKEND RESPONSE - Full Breakdown');
85         debugPrint('');
86         debugPrint(' [top-level]');
87         debugPrint('    success : ${res.data['success']}');
88         debugPrint('    message : ${res.data['message']}');
89         debugPrint('');
90         debugPrint(' [data] object:');
91
92         final dataObj = res.data['data'];
93         debugPrint('    data : ${dataObj != null ? '✅ present' : '❌ null'}');
94         if (dataObj == null) {
95             debugPrint('    ⚠️ data is NULL - backend did not return a data object');
96         } else {
97             // session_cookie - الكوكي الجديد
98             final backendCookie = dataObj['session_cookie'] as String?;
99             debugPrint('    session_cookie : ${backendCookie != null ? '✅ "${backendCookie.substring(0, backendCookie.length.clamp(0, 30))}"' : '❌ null'}');
100
101             // refresh_token - الاسم الأول المحتمل
102             final rt1 = dataObj['refresh_token'] as String?;
103             debugPrint('    refresh_token : ${rt1 != null ? '✅ "${rt1.substring(0, rt1.length.clamp(0, 30))}"' : '❌ null'}');
104
105             // new_refresh_token - الاسم الثاني المحتمل (refresh-session مذكور فخطه)
106             final rt2 = dataObj['new_refresh_token'] as String?;
107             debugPrint('    new_refresh_token : ${rt2 != null ? '✅ "${rt2.substring(0, rt2.length.clamp(0, 30))}"' : '❌ null'}');
108
109             // expires_at - وقت انتهاء الجلسة الجديدة
110             final backendExpiry = dataObj['expires_at'];
111             debugPrint('    expires_at : ${backendExpiry ?? '❌ null'}');
112
113             // أي حقول ثانية موجودة مش متوقعة
114             final knownKeys = {'session_cookie', 'refresh_token', 'new_refresh_token', 'expires_at'};
115             final extraKeys = (dataObj as Map).keys.where((k) => !knownKeys.contains(k)).toList();
116             if (extraKeys.isNotEmpty) {
117                 debugPrint('    [extra fields] : $extraKeys');
118             }
119         }
120         debugPrint('');
121         debugPrint('');
122
123         final success = res.data['success'] ?? false;
124         final message =
125             res.data['message'] as String? ?? 'Connected successfully';
126
127         if (success) {
128             // — Extract refresh_token from API response (if available) —
129             // The backend may return a refresh_token in the connect response,
130             // or it can be passed in from the caller (e.g., WebView login flow).
131             // Per integration plan: connect endpoint → 'refresh_token'
132             //                               refresh endpoint → 'new_refresh_token'
133             final apiRefreshToken =
134                 (res.data['data']?['refresh_token'] ??
135                  res.data['data']?['new_refresh_token']) as String?;
136
137             debugPrint('');
138             debugPrint('🔑 [MadrasatiCubit] refresh token trace:');
139
140             // — End of MadrasatiCubit class —
141         }
142     }
143 );

```



```

142 |         debugPrint('    └ From API response : ${apiRefreshToken != null ? '✅' "${apiRefreshToken.substri
ng(0, apiRefreshToken.length.clamp(0, 30))}..." : '❌ null (not in API response)'}');
143 |         debugPrint('    └ From caller (param): ${refreshToken != null ? '✅' "${refreshToken.substring(0,
refreshToken.length.clamp(0, 30))}..." : '❌ null (not passed by caller)'}');
144 |
145 |         final tokenToSave = apiRefreshToken ?? refreshToken;
146 |         debugPrint('    └ Final token to save: ${tokenToSave != null ? '✅' 'will be saved 🚩' : '⚠️ NO T
OKEN — session refresh will NOT work!'}');
147 |         debugPrint('');
148 |
149 |         // — Save session securely —————
150 |         final expiry = DateTime.tryParse(expiresAt.replaceAll(' ', 'T'));
151 |         debugPrint('📁 [MadrasatiCubit] Saving session to SecureStorage...');
152 |         await sl<SecureStorageHelper>().saveMadrasatiSession(
153 |             MadrasatiSessionData(
154 |                 sessionCookie: sessionCookie,
155 |                 schoolId: madrasatiSchoolId,
156 |                 expiresAt: expiry,
157 |                 refreshToken: tokenToSave,
158 |             ),
159 |         );
160 |         debugPrint('    Session saved ✅ (expiresAt parsed: $expiry)');
161 |
162 |         // — Notify session service that session is now active —————
163 |         debugPrint('🔔 [MadrasatiCubit] Notifying MadrasatiSessionService...');
164 |         sl<MadrasatiSessionService>().notifySessionActive();
165 |         debugPrint('    notifySessionActive() called ✅');
166 |
167 |         debugPrint('✅ [MadrasatiCubit] State → MadrasatiSuccessState("$message")');
168 |         if (!isClosed) emit(MadrasatiSuccessState(message));
169 |     } else {
170 |         debugPrint('❌ [MadrasatiCubit] success=false → State → MadrasatiErrorState("$message")');
171 |         if (!isClosed) emit(MadrasatiErrorState(message));
172 |     }
173 | },
174 | );
175 | } catch (e, st) {
176 |     debugPrint('❗ [MadrasatiCubit] Exception caught: $e');
177 |     debugPrint('    StackTrace: $st');
178 |     debugPrint('❌ [MadrasatiCubit] State → MadrasatiErrorState');
179 |     if (!isClosed) emit(MadrasatiErrorState(e.toString()));
180 | }
181 |
182 | debugPrint('—————');
183 | debugPrint('🔗 [MadrasatiCubit] connectMadrasati() — END');
184 | debugPrint('—————');
185 | }
186 | }
187 |

```

Lines • Dart 116

Feature / Login

#236 – lib/features/login/presentation/screen/login_screen.dart

```

1 | import 'package:flutter/material.dart';
2 | import 'package:flutter_bloc/flutter_bloc.dart';
3 | import 'package:moean/core/theme/colors.dart';
4 | import 'package:moean/core/utils/constants/assets_helper.dart';
5 | import 'package:moean/core/utils/constants/spacing.dart';
6 | import 'package:moean/core/utils/cubit/theme/theme_cubit.dart';
7 | import 'package:moean/core/utils/cubit/theme/theme_state.dart';
8 | import 'package:moean/features/login/presentation/cubit/login_cubit.dart';
9 | import 'package:moean/features/login/presentation/cubit/login_state.dart';
10 | import 'package:moean/features/login/presentation/widgets/login_action_buttons_widget.dart';
11 | import 'package:moean/features/login/presentation/widgets/login_footer_widget.dart';
12 | import 'package:moean/features/login/presentation/widgets/login_form_widget.dart';
13 | import 'package:moean/features/login/presentation/widgets/login_header_widget.dart';
14 | import 'package:moean/features/login/presentation/widgets/login_remember_forgot_widget.dart';
15 | import 'package:moean/core/utils/constants/routes.dart';
16 | import 'package:moean/core/utils/extensions/context_extension.dart';
17 |
18 | class LoginScreen extends StatelessWidget {
19 |     const LoginScreen({super.key});

```

```

20
21 @override
22 Widget build(BuildContext context) {
23   return BlocProvider(
24     create: (_) => LoginCubit(),
25     child: BlocListener<LoginCubit, LoginState>(
26       listener: _onStateChanged,
27       child: BlocBuilder<ThemeCubit, ThemeState>(
28         builder: (context, themeState) {
29           return Scaffold(
30             backgroundColor: ColorsManager.background,
31             body: SafeArea(
32               child: SingleChildScrollView(
33                 padding: const EdgeInsets.symmetric(horizontal: 24),
34               child: Builder(
35                 builder: (context) {
36                   final cubit = LoginCubit.get(context);
37                   return Form(
38                     key: cubit.formKey,
39                     child: Column(
40                       crossAxisAlignment: CrossAxisAlignment.stretch,
41                       children: [
42                         verticalSpace16,
43                         // ignore: prefer_const_constructors
44                         LoginHeaderWidget(),
45                         Container(
46                           height: 300,
47                           width: double.infinity,
48                           decoration: BoxDecoration(
49                             color: ColorsManager.background,
50                             borderRadius: BorderRadius.circular(24),
51                           ),
52                           margin: const EdgeInsets.symmetric(vertical: 16),
53                           child: Image.asset(
54                             AssetsHelper.img6,
55                           ),
56                         ),
57                       ],
58                       // ignore: prefer_const_constructors
59                       LoginFormWidget(),
60                       verticalSpace16,
61                       // ignore: prefer_const_constructors
62                       LoginRememberForgotWidget(),
63                       verticalSpace28,
64                       // ignore: prefer_const_constructors
65                       LoginActionButtonsWidget(),
66                       verticalSpace24,
67                       // ignore: prefer_const_constructors
68                       LoginFooterWidget(),
69                       verticalSpace32,
70                     ],
71                   ),
72                 );
73               },
74             ),
75           ),
76         ),
77       );
78     },
79   ),
80 );
81 };
82 }
83
84 void _onStateChanged(BuildContext context, LoginState state) {
85   if (state is LoginErrorState) {
86     ScaffoldMessenger.of(context).showSnackBar(
87       SnackBar(
88         content: Text(state.message),
89         backgroundColor: ColorsManager.errorColor,
90       ),
91     );
92   }
93
94   if (state is LoginAccountSuspendedState) {

```

```

95     context.push(
96       Routes.accountSuspended,
97       arguments: {
98         'email': state.email,
99         'password': state.password,
100        'message': state.message,
101        'name': state.userName,
102      },
103    );
104  }
105
106  if (state is LoginSuccessState) {
107    if (state.userEmail == 'admin@moeen.sa' || state.userEmail == 'admin@moeen.com') {
108      context.pushNamedAndRemoveUntil(Routes.adminTeachers, (route) => false);
109    } else {
110      context.pushNamedAndRemoveUntil(Routes.home, (route) => false);
111    }
112  }
113 }
114
115 }
116

```

Lines • Dart 882

Feature / Login

#237 – lib/features/login/presentation/screen/microsoft_login_screen.dart

```

1  import 'package:flutter/foundation.dart' show kIsWeb;
2  import 'package:flutter/material.dart';
3  import 'package:flutter/services.dart';
4  import 'package:flutter_bloc/flutter_bloc.dart';
5  import 'package:flutter_inappwebview/flutter_inappwebview.dart';
6  import 'package:moean/core/theme/colors.dart';
7  import 'package:moean/core/utils/constants/primary/circle_indecator.dart';
8  import 'package:moean/core/utils/constants/routes.dart';
9  import 'package:moean/core/utils/extensions/context_extension.dart';
10 import 'package:moean/core/utils/platform_url/platform_url.dart';
11 import 'package:moean/features/login/presentation/cubit/madrasati_cubit.dart';
12
13 class MicrosoftLoginScreen extends StatefulWidget {
14   const MicrosoftLoginScreen({super.key});
15
16   @override
17   State<MicrosoftLoginScreen> createState() => _MicrosoftLoginScreenState();
18 }
19
20 class _MicrosoftLoginScreenState extends State<MicrosoftLoginScreen>
21   with TickerProviderStateMixin {
22   // — Mobile WebView fields —————
23   final GlobalKey webViewKey = GlobalKey();
24   InAppWebViewController? webViewController;
25   double progress = 0;
26   final String _madrasatiLoginUrl = 'https://schools.madrasati.sa/';
27   bool _isConnecting = false;
28
29   // — Web manual-entry fields —————
30   int _webStep = 0; // 0 = intro, 1 = waiting, 2 = enter cookie
31   final TextEditingController _cookieCtrl = TextEditingController();
32   final TextEditingController _schoolIdCtrl = TextEditingController();
33   bool _cookieObscured = true;
34   final _formKey = GlobalKey<FormState>();
35
36   // Animations
37   late final AnimationController _fadeCtrl;
38   late final Animation<double> _fadeAnim;
39
40   @override
41   void initState() {
42     super.initState();
43     _fadeCtrl = AnimationController(
44       vsync: this, duration: const Duration(milliseconds: 400));
45     _fadeAnim = CurvedAnimation(parent: _fadeCtrl, curve: Curves.easeIn);
46     _fadeCtrl.forward();
47   }
48

```

```

49 @override
50 void dispose() {
51   _cookieCtrl.dispose();
52   _schoolIdCtrl.dispose();
53   _fadeCtrl.dispose();
54   super.dispose();
55 }
56
57 // — School-ID extractor (mobile) —————
58 String _extractSchoolId(WebView url) {
59   final uri = Uri.tryParse(url.toString());
60   if (uri == null) return '';
61   final fromQuery = uri.queryParameters['schoolId'] ??
62     uri.queryParameters['school_id'] ??
63     uri.queryParameters['SchoolId'];
64   if (fromQuery != null && fromQuery.isNotEmpty) return fromQuery;
65   for (final segment in uri.pathSegments) {
66     if (segment.length == 32 &&
67       RegExp(r'^[a-zA-Z0-9]+$').hasMatch(segment)) {
68       return segment;
69     }
70   }
71   return '';
72 }
73
74 // — Helpers —————
75 String _buildExpiresAt() {
76   final exp = DateTime.now().add(const Duration(days: 30));
77   return '${exp.year}-${exp.month.toString().padLeft(2, '0')}-${
78     exp.day.toString().padLeft(2, '0')} '
79     '${exp.hour.toString().padLeft(2, '0')}:
80     ${exp.minute.toString().padLeft(2, '0')}:00';
81 }
82
83 void _goToStep(int step) {
84   _fadeCtrl.reverse().then(() {
85     setState(() => _webStep = step);
86     _fadeCtrl.forward();
87   });
88 }
89
90 // =====
91 // BUILD
92 // =====
93 @override
94 Widget build(BuildContext context) {
95   return BlocProvider(
96     create: (_) => MadrasatiCubit(),
97     child: BlocConsumer<MadrasatiCubit, MadrasatiState>(
98       listener: _blocListener,
99       builder: (context, state) {
100         return Scaffold(
101           backgroundColor: ColorsManager.background,
102           appBar: AppBar(
103             title: const Text('تسجيل الدخول في منصة مدرستي'),
104             backgroundColor: ColorsManager.surfacePrimary,
105             foregroundColor: ColorsManager.textPrimary,
106             elevation: 0,
107             centerTitle: true,
108           ),
109           body: SafeArea(
110             child: kIsWeb
111               ? _buildWebBody(context, state)
112               : _buildMobileBody(context, state),
113           ),
114         );
115       },
116     ),
117   );
118 }
119
120 void _blocListener(BuildContext context, MadrasatiState state) {
121   if (state is MadrasatiSuccessState) {
122     ScaffoldMessenger.of(context).showSnackBar(SnackBar(
123       content: Text(state.message),

```

```

124         backgroundColor: ColorsManager.successColor,
125     ));
126     context.pushNamedAndRemoveUntil(Routes.schedule, (r) => false);
127 } else if (state is MadrasatiErrorState) {
128     ScaffoldMessenger.of(context).showSnackBar(SnackBar(
129         content: Text(state.message),
130         backgroundColor: ColorsManager.errorColor,
131     ));
132     setState(() => _isConnecting = false);
133 }
134 }
135
136 // =====
137 // MOBILE - InAppWebView (unchanged logic)
138 // =====
139 Widget _buildMobileBody(BuildContext context, MadrasatiState state) {
140     return Stack(
141         children: [
142             Column(
143                 children: [
144                     if (progress < 1.0)
145                         LinearProgressIndicator(
146                             value: progress,
147                             color: ColorsManager.primaryColor,
148                             backgroundColor:
149                                 ColorsManager.primaryColor.withValues(alpha: 0.2),
150                         ),
151                     Expanded(
152                         child: InAppWebView(
153                             key: webViewKey,
154                             initialUrlRequest:
155                                 URLRequest(url: Uri(_madrasatiLoginUrl)),
156                             initialSettings: InAppWebViewSettings(
157                                 javaScriptEnabled: true,
158                                 domStorageEnabled: true,
159                                 databaseEnabled: true,
160                                 transparentBackground: true,
161                                 safeBrowsingEnabled: false,
162                             ),
163                             onWebViewCreated: (controller) {
164                                 webViewController = controller;
165                                 debugPrint('WebView Created');
166                             },
167                             onLoadStart: (controller, url) {
168                                 debugPrint('LOAD START - URL: $url');
169                             },
170                             onLoadStop: (controller, url) async {
171                                 setState(() => progress = 1.0);
172                                 final urlStr = url?.toString().toLowerCase() ?? '';
173                                 final isInsideMadrasati = urlStr.contains('teacher') ||
174                                     urlStr.contains('schoolschedule') ||
175                                     urlStr.contains('schoolmanagment');
176                                 if (!isInsideMadrasati || _isConnecting) {
177                                     debugPrint('Waiting for login... URL: $urlStr');
178                                     return;
179                                 }
180                                 setState(() => _isConnecting = true);
181                                 debugPrint('🔍 Login Detected! Extracting cookies...');
182                                 final cookieManager = CookieManager.instance();
183                                 final madrasatiCookies = await cookieManager.getCookies(
184                                     url: Uri('https://schools.madrasati.sa/'));
185                                 final msCookies = await cookieManager.getCookies(
186                                     url: Uri('https://login.microsoftonline.com/'));
187                                 final allCookies = [...madrasatiCookies, ...msCookies];
188                                 final sessionCookieStr =
189                                     allCookies.map((c) => '${c.name}=${c.value}').join('; ');
190                                 final schoolId = _extractSchoolId(url!);
191                                 // ignore: use_build_context_synchronously
192                                 MadrasatiCubit.get(context).connectMadrasati(
193                                     sessionCookie: sessionCookieStr,
194                                     madrasatiSchoolId: schoolId,
195                                     expiresAt: _buildExpiresAt(),
196                                 );
197                             },
198                             onReceivedError: (controller, request, error) {

```

```

199         debugPrint('WEBVIEW ERROR - ${error.description}');
200     },
201     onReceivedHttpError: (controller, request, response) {
202         debugPrint('HTTP ERROR - ${response.statusCode}');
203     },
204     onUpdateVisitedHistory: (controller, url, isReload) {
205         debugPrint('VISITED URL: $url');
206     },
207     onTitleChanged: (controller, title) {
208         debugPrint('PAGE TITLE: $title');
209     },
210     onProgressChanged: (controller, p) {
211         setState(() => progress = p / 100);
212     },
213   ),
214 ),
215 ],
216 ),
217 if (state is MadrasatiLoadingState)
218   Container(
219     color: Colors.black.withValues(alpha: 0.5),
220     child: const Center(
221       child: DiscreteCircle(
222         size: 60,
223         color: Color(0xFFD61F69),
224         secondCircleColor: Color(0xFFE2AD3B),
225         thirdCircleColor: Color(0xFF0E7A5E),
226       ),
227     ),
228   ),
229 ],
230 );
231 }
232
233 // =====
234 // WEB - step-by-step manual cookie flow
235 // =====
236 Widget _buildWebBody(BuildContext context, MadrasatiState state) {
237   return Stack(
238     children: [
239       Center(
240         child: SingleChildScrollView(
241           padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 32),
242           child: FadeTransition(
243             opacity: _fadeAnim,
244             child: ConstrainedBox(
245               constraints: const BoxConstraints(maxWidth: 600),
246               child: Column(
247                 crossAxisAlignment: CrossAxisAlignment.stretch,
248                 children: [
249                   _buildWebHeader(),
250                   const SizedBox(height: 32),
251                   _buildStepIndicator(),
252                   const SizedBox(height: 32),
253                   AnimatedSwitcher(
254                     duration: const Duration(milliseconds: 350),
255                     child: _webStep == 0
256                       ? _buildWebStep0(context)
257                       : _webStep == 1
258                         ? _buildWebStep1(context)
259                         : _buildWebStep2(context, state),
260                   ),
261                 ],
262               ),
263             ),
264           ),
265         ),
266       ),
267       if (state is MadrasatiLoadingState)
268         Container(
269           color: Colors.black.withValues(alpha: 0.5),
270           child: const Center(
271             child: DiscreteCircle(
272               size: 60,
273               color: Color(0xFFD61F69),

```

```

274         secondCircleColor: Color(0xFFE2AD3B),
275         thirdCircleColor: Color(0xFF0E7A5E),
276     ),
277 ),
278 ),
279 ],
280 );
281 }
282
283 // — Web header —————
284 Widget _buildWebHeader() {
285     return Container(
286         padding: const EdgeInsets.all(24),
287         decoration: BoxDecoration(
288             gradient: LinearGradient(
289                 colors: [
290                     ColorsManager.primaryColor.withValues(alpha: 0.12),
291                     ColorsManager.primaryColor.withValues(alpha: 0.04),
292                 ],
293                 begin: Alignment.topLeft,
294                 end: Alignment.bottomRight,
295             ),
296             borderRadius: BorderRadius.circular(20),
297             border: Border.all(
298                 color: ColorsManager.primaryColor.withValues(alpha: 0.25)),
299         ),
300         child: Row(
301             children: [
302                 Container(
303                     width: 56,
304                     height: 56,
305                     decoration: BoxDecoration(
306                         color: ColorsManager.primaryColor.withValues(alpha: 0.15),
307                         shape: BoxShape.circle,
308                     ),
309                     child: Icon(Icons.language_rounded,
310                         color: ColorsManager.primaryColor, size: 28),
311                 ),
312                 const SizedBox(width: 16),
313                 Expanded(
314                     child: Column(
315                         crossAxisAlignment: CrossAxisAlignment.start,
316                         children: [
317                             Text(
318                                 'تسجيل الدخول عبر الويب',
319                                 style: TextStyle(
320                                     fontSize: 17,
321                                     fontWeight: FontWeight.bold,
322                                     color: ColorsManager.textPrimary,
323                                 ),
324                             ),
325                             const SizedBox(height: 4),
326                             Text(
327                                 'اتبع الخطوات أدناه للاتصال بحسابك\nموقع مدرستي لا يسمح بالتضمين داخل التطبيق على الويب',
328                                 style: TextStyle(
329                                     fontSize: 13,
330                                     color: ColorsManager.textPrimary.withValues(alpha: 0.65),
331                                     height: 1.5,
332                                 ),
333                             ),
334                         ],
335                     ),
336                 ),
337             ],
338         ),
339     );
340 }
341
342 // — Step indicator —————
343 Widget _buildStepIndicator() {
344     const steps = ['فتح الموقع', 'تسجيل الدخول', 'ربط الحساب'];
345     return Row(
346         children: List.generate(steps.length * 2 - 1, (i) {
347             if (i.isOdd) {
348                 return Expanded(

```

```

349         child: Container(
350             height: 2,
351             color: i ~/ 2 < _webStep
352                 ? ColorsManager.primaryColor
353                 : ColorsManager.primaryColor.withValues(alpha: 0.2),
354         ),
355     );
356 }
357 final stepIdx = i ~/ 2;
358 final done = stepIdx < _webStep;
359 final active = stepIdx == _webStep;
360 return Column(
361     children: [
362         AnimatedContainer(
363             duration: const Duration(milliseconds: 300),
364             width: 36,
365             height: 36,
366             decoration: BoxDecoration(
367                 shape: BoxShape.circle,
368                 color: done || active
369                     ? ColorsManager.primaryColor
370                     : ColorsManager.primaryColor.withValues(alpha: 0.12),
371                 border: Border.all(
372                     color: ColorsManager.primaryColor,
373                     width: active ? 2.5 : 0,
374                 ),
375             ),
376             child: Center(
377                 child: done
378                     ? const Icon(Icons.check_rounded,
379                         color: Colors.white, size: 18)
380                     : Text(
381                         '${stepIdx + 1}',
382                         style: TextStyle(
383                             color: active
384                                 ? Colors.white
385                                 : ColorsManager.primaryColor,
386                             fontWeight: FontWeight.bold,
387                             fontSize: 14,
388                         ),
389                     ),
390             ),
391         ),
392         const SizedBox(height: 6),
393         Text(
394             steps[stepIdx],
395             style: TextStyle(
396                 fontSize: 11,
397                 fontWeight:
398                     active ? FontWeight.bold : FontWeight.normal,
399                 color: active
400                     ? ColorsManager.primaryColor
401                     : ColorsManager.textPrimary.withValues(alpha: 0.55),
402             ),
403         ),
404     ],
405 );
406 }),
407 );
408 }
409
410 // — Step 0 - Intro —————
411 Widget _buildWebStep0(BuildContext context) {
412     return _stepCard(
413         key: const ValueKey(0),
414         child: Column(
415             crossAxisAlignment: CrossAxisAlignment.stretch,
416             children: [
417                 _infoTile(
418                     icon: Icons.open_in_new_rounded,
419                     color: const Color(0xFF2196F3),
420                     title: 'افتح موقع مدرستي',
421                     subtitle:
422                         'سيُفتح الموقع في تبويب جديد في المتصفح بشكل مستقل',
423                 ),

```



```

424     const SizedBox(height: 12),
425     _infoTile(
426       icon: Icons.login_rounded,
427       color: const Color(0xFF4CAF50),
428       title: 'سجل الدخول بحساب Microsoft',
429       subtitle: 'أكمل عملية تسجيل الدخول كاملاً حتى تصل للصفحة الرئيسية',
430     ),
431     const SizedBox(height: 12),
432     _infoTile(
433       icon: Icons.link_rounded,
434       color: ColorsManager.primaryColor,
435       title: 'ارجع هنا وأدخل بيانات الجلسة',
436       subtitle: 'سنوجهك خطوة بخطوة لنسخ بيانات الجلسة من المتصفح',
437     ),
438     const SizedBox(height: 28),
439     _primaryButton(
440       label: 'فتح موقع مدرستي',
441       icon: Icons.open_in_new_rounded,
442       onTap: () {
443         openUrlInNewTab(_madrasatiLoginUrl);
444         _goToStep(1);
445       },
446     ),
447   ],
448 ),
449 );
450 }
451
452 // — Step 1 - Waiting —————
453 Widget _buildWebStep1(BuildContext context) {
454   return _stepCard(
455     key: const ValueKey(1),
456     child: Column(
457       crossAxisAlignment: CrossAxisAlignment.stretch,
458       children: [
459         Center(
460           child: Container(
461             width: 80,
462             height: 80,
463             decoration: BoxDecoration(
464               color: const Color(0xFF4CAF50).withValues(alpha: 0.12),
465               shape: BoxShape.circle,
466             ),
467           child: const Icon(Icons.hourglass_top_rounded,
468             color: Color(0xFF4CAF50), size: 38),
469         ),
470       ],
471       const SizedBox(height: 20),
472       Text(
473         'في انتظار تسجيل الدخول',
474         textAlign: TextAlign.center,
475         style: TextStyle(
476           fontSize: 18,
477           fontWeight: FontWeight.bold,
478           color: ColorsManager.textPrimary,
479         ),
480       ),
481       const SizedBox(height: 8),
482       Text(
483         'بعد الوصول للصفحة الرئيسية لمدرستي، غُد هنا.\nإُرجى إتمام تسجيل الدخول في التبويب المفتوح',
484         textAlign: TextAlign.center,
485         style: TextStyle(
486           fontSize: 14,
487           color: ColorsManager.textPrimary.withValues(alpha: 0.65),
488           height: 1.6,
489         ),
490       ),
491       const SizedBox(height: 28),
492       _primaryButton(
493         label: 'لقد سجلت الدخول، تابع',
494         icon: Icons.arrow_forward_ios_rounded,
495         onTap: () => _goToStep(2),
496       ),
497       const SizedBox(height: 12),
498       _secondaryButton(

```

```

499         label: 'إعادة فتح الموقع',
500         icon: Icons.refresh_rounded,
501         onTap: () => openUrlInNewTab(_madrasatiLoginUrl),
502     ),
503 ],
504 ),
505 );
506 }
507
508 // — Step 2 - Cookie entry —————
509 Widget _buildWebStep2(BuildContext context, MadrasatiState state) {
510     return _stepCard(
511         key: const ValueKey(2),
512         child: Form(
513             key: _formKey,
514             child: Column(
515                 crossAxisAlignment: CrossAxisAlignment.stretch,
516                 children: [
517                     // How to get cookie - collapsible guide
518                     _cookieGuideCard(),
519                     const SizedBox(height: 24),
520
521                     // Cookie field
522                     Text(
523                         'كوكي الجلسة (.AspNetCore.Cookies)',
524                         style: TextStyle(
525                             fontSize: 13,
526                             fontWeight: FontWeight.w600,
527                             color: ColorsManager.textPrimary,
528                         ),
529                     ),
530                     const SizedBox(height: 8),
531                     TextFormField(
532                         controller: _cookieCtrl,
533                         obscureText: _cookieObscured,
534                         maxLines: 1,
535                         style: TextStyle(
536                             fontSize: 12,
537                             fontFamily: 'monospace',
538                             color: ColorsManager.textPrimary,
539                         ),
540                         decoration: _fieldDecoration(
541                             hint: 'الصق قيمة الكوكي هنا',
542                             suffixIcon: IconButton(
543                                 icon: Icon(
544                                     _cookieObscured
545                                         ? Icons.visibility_rounded
546                                         : Icons.visibility_off_rounded,
547                                     color: ColorsManager.primaryColor,
548                                     size: 20,
549                                 ),
550                                 onPressed: () =>
551                                     setState(() => _cookieObscured = !_cookieObscured),
552                             ),
553                     ),
554                     validator: (v) {
555                         if (v == null || v.trim().isEmpty) {
556                             return 'يُرجى إدخال قيمة الكوكي';
557                         }
558                         return null;
559                     },
560                 ),
561
562                     const SizedBox(height: 16),
563
564                     // School ID field
565                     Text(
566                         'معزف المدرسة (اختياري)',
567                         style: TextStyle(
568                             fontSize: 13,
569                             fontWeight: FontWeight.w600,
570                             color: ColorsManager.textPrimary,
571                         ),
572                     ),
573                     const SizedBox(height: 4),

```

```

574 Text(
575   'بعد تسجيل الدخول (32 حرفاً) URL يُمكن الحصول عليه من رابط',
576   style: TextStyle(
577     fontSize: 11,
578     color: ColorsManager.textPrimary.withValues(alpha: 0.55),
579   ),
580 ),
581 const SizedBox(height: 8),
582 TextFormField(
583   controller: _schoolIdCtrl,
584   style: TextStyle(
585     fontSize: 12,
586     fontFamily: 'monospace',
587     color: ColorsManager.textPrimary,
588   ),
589   decoration: _fieldDecoration(
590     hint: 'مثال: alb2c3d4e5f6... (32 حرف)',
591   ),
592 ),
593
594 const SizedBox(height: 28),
595
596 _primaryButton(
597   label: 'ربط الحساب بمدرستي',
598   icon: Icons.link_rounded,
599   onTap: state is MadrasatiLoadingState
600     ? null
601     : () {
602       if (_formKey.currentState!.validate()) {
603         MadrasatiCubit.get(context).connectMadrasati(
604           sessionCookie: _cookieCtrl.text.trim(),
605           madrasatiSchoolId: _schoolIdCtrl.text.trim(),
606           expiresAt: _buildExpiresAt(),
607         );
608       }
609     },
610 ),
611 const SizedBox(height: 12),
612 _secondaryButton(
613   label: 'رجوع',
614   icon: Icons.arrow_back_ios_rounded,
615   onTap: () => _goToStep(1),
616 ),
617 ],
618 ),
619 ),
620 );
621 }
622
623 // — Cookie guide card —————
624 Widget _cookieGuideCard() {
625   return Container(
626     decoration: BoxDecoration(
627       color: ColorsManager.primaryColor.withValues(alpha: 0.06),
628       borderRadius: BorderRadius.circular(14),
629       border: Border.all(
630         color: ColorsManager.primaryColor.withValues(alpha: 0.2)),
631     ),
632     child: Theme(
633       data: Theme.of(context).copyWith(dividerColor: Colors.transparent),
634       child: ExpansionTile(
635         leading: Icon(Icons.help_outline_rounded,
636           color: ColorsManager.primaryColor, size: 22),
637         title: Text(
638           'كيف أحصل على كوكي الجلسة؟',
639           style: TextStyle(
640             fontSize: 13,
641             fontWeight: FontWeight.w600,
642             color: ColorsManager.primaryColor,
643           ),
644         ),
645         childrenPadding:
646           const EdgeInsets.only(left: 16, right: 16, bottom: 16),
647         children: [
648           _guideStep('في تبويب مدرستي، افتح أدوات المطور',

```

```

649         'أو كليك يمين ← فحص F12 اضغط' (Inspect)'),
650         _guideStep('إذهب إلى تبويب', '٢ "Application"',
651         'في الشريط العلوي لأدوات المطور'),
652         _guideStep('افتح', '٣ Cookies ← schools.madrasati.sa',
653         'Storage في القائمة اليسرى تحت'),
654         _guideStep('ابحث عن', '٤ .AspNetCore.Cookies',
655         'انقر عليها مرتين ثم انسخ القيمة كاملاً'),
656         const SizedBox(height: 8),
657         _cookieKeyBox('.AspNetCore.Cookies'),
658     ],
659 ),
660 ),
661 );
662 }
663
664 Widget _guideStep(String num, String title, String subtitle) {
665     return Padding(
666         padding: const EdgeInsets.only(bottom: 10),
667         child: Row(
668             crossAxisAlignment: CrossAxisAlignment.start,
669             children: [
670                 Container(
671                     width: 26,
672                     height: 26,
673                     margin: const EdgeInsets.only(top: 2, left: 10),
674                     decoration: BoxDecoration(
675                         color: ColorsManager.primaryColor,
676                         shape: BoxShape.circle,
677                     ),
678                     child: Center(
679                         child: Text(num,
680                             style: const TextStyle(
681                                 color: Colors.white,
682                                 fontSize: 11,
683                                 fontWeight: FontWeight.bold)),
684                     ),
685                 ),
686                 Expanded(
687                     child: Column(
688                         crossAxisAlignment: CrossAxisAlignment.start,
689                         children: [
690                             Text(title,
691                                 style: TextStyle(
692                                     fontSize: 13,
693                                     fontWeight: FontWeight.w600,
694                                     color: ColorsManager.textPrimary)),
695                             Text(subtitle,
696                                 style: TextStyle(
697                                     fontSize: 12,
698                                     color:
699                                     ColorsManager.textPrimary.withValues(alpha: 0.6))),
700                         ],
701                     ),
702                 ),
703             ],
704         ),
705     );
706 }
707
708 Widget _cookieKeyBox(String key) {
709     return Container(
710         padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 8),
711         decoration: BoxDecoration(
712             color: Colors.black.withValues(alpha: 0.06),
713             borderRadius: BorderRadius.circular(8),
714             border: Border.all(color: Colors.black.withValues(alpha: 0.1)),
715         ),
716         child: Row(
717             children: [
718                 Icon(Icons.key_rounded,
719                     size: 16, color: ColorsManager.primaryColor),
720                 const SizedBox(width: 8),
721                 Expanded(
722                     child: Text(
723                         key,

```

```

724         style: TextStyle(
725             fontFamily: 'monospace',
726             fontSize: 12,
727             fontWeight: FontWeight.bold,
728             color: ColorsManager.textPrimary,
729         ),
730     ),
731 ),
732 IconButton(
733     iconSize: 16,
734     padding: EdgeInsets.zero,
735     constraints: const BoxConstraints(),
736     icon: Icon(Icons.copy_rounded,
737         color: ColorsManager.primaryColor, size: 16),
738     onPressed: () =>
739         Clipboard.setData(ClipboardData(text: key)),
740 ),
741 ],
742 ),
743 );
744 }
745
746 // — Shared UI helpers —————
747 Widget _stepCard({required Widget child, required Key key}) {
748     return Container(
749         key: key,
750         padding: const EdgeInsets.all(24),
751         decoration: BoxDecoration(
752             color: ColorsManager.surfacePrimary,
753             borderRadius: BorderRadius.circular(20),
754             boxShadow: [
755                 BoxShadow(
756                     color: Colors.black.withValues(alpha: 0.06),
757                     blurRadius: 20,
758                     offset: const Offset(0, 4),
759                 ),
760             ],
761         ),
762         child: child,
763     );
764 }
765
766 Widget _infoTile({
767     required IconData icon,
768     required Color color,
769     required String title,
770     required String subtitle,
771 }) {
772     return Container(
773         padding: const EdgeInsets.all(14),
774         decoration: BoxDecoration(
775             color: color.withValues(alpha: 0.06),
776             borderRadius: BorderRadius.circular(12),
777             border: Border.all(color: color.withValues(alpha: 0.2)),
778         ),
779         child: Row(
780             children: [
781                 Icon(icon, color: color, size: 22),
782                 const SizedBox(width: 12),
783                 Expanded(
784                     child: Column(
785                         crossAxisAlignment: CrossAxisAlignment.start,
786                         children: [
787                             Text(title,
788                                 style: TextStyle(
789                                     fontSize: 14,
790                                     fontWeight: FontWeight.w600,
791                                     color: ColorsManager.textPrimary)),
792                             const SizedBox(height: 2),
793                             Text(subtitle,
794                                 style: TextStyle(
795                                     fontSize: 12,
796                                     color:
797                                         ColorsManager.textPrimary.withValues(alpha: 0.6))),
798                         ],

```

```

799         ),
800     ),
801 ],
802 ),
803 );
804 }
805
806 Widget _primaryButton({
807     required String label,
808     required IconData icon,
809     required VoidCallback? onTap,
810 }) {
811     return ElevatedButton.icon(
812         onPressed: onTap,
813         style: ElevatedButton.styleFrom(
814             backgroundColor: ColorsManager.primaryColor,
815             foregroundColor: Colors.white,
816             padding: const EdgeInsets.symmetric(vertical: 16),
817             shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(14)),
818             elevation: 0,
819         ),
820         icon: Icon(icon, size: 18),
821         label: Text(label,
822             style: const TextStyle(fontSize: 15, fontWeight: FontWeight.w600)),
823     );
824 }
825
826 Widget _secondaryButton({
827     required String label,
828     required IconData icon,
829     required VoidCallback? onTap,
830 }) {
831     return OutlinedButton.icon(
832         onPressed: onTap,
833         style: OutlinedButton.styleFrom(
834             foregroundColor: ColorsManager.primaryColor,
835             padding: const EdgeInsets.symmetric(vertical: 14),
836             side: BorderSide(
837                 color: ColorsManager.primaryColor.withValues(alpha: 0.4)),
838             shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(14)),
839         ),
840         icon: Icon(icon, size: 16),
841         label: Text(label,
842             style: const TextStyle(fontSize: 14, fontWeight: FontWeight.w500)),
843     );
844 }
845
846 InputDecoration _fieldDecoration({
847     required String hint,
848     Widget? suffixIcon,
849 }) {
850     return InputDecoration(
851         hintText: hint,
852         hintStyle: TextStyle(
853             fontSize: 12,
854             color: ColorsManager.textPrimary.withValues(alpha: 0.4),
855         ),
856         contentPadding:
857             const EdgeInsets.symmetric(horizontal: 14, vertical: 14),
858         filled: true,
859         fillColor: ColorsManager.primaryColor.withValues(alpha: 0.04),
860         border: OutlineInputBorder(
861             borderRadius: BorderRadius.circular(12),
862             borderSide:
863                 BorderSide(color: ColorsManager.primaryColor.withValues(alpha: 0.2)),
864         ),
865         enabledBorder: OutlineInputBorder(
866             borderRadius: BorderRadius.circular(12),
867             borderSide:
868                 BorderSide(color: ColorsManager.primaryColor.withValues(alpha: 0.2)),
869         ),
870         focusedBorder: OutlineInputBorder(
871             borderRadius: BorderRadius.circular(12),
872             borderSide:
873                 BorderSide(color: ColorsManager.primaryColor, width: 1.5),

```

```

874     ),
875     errorBorder: OutlineInputBorder(
876       borderRadius: BorderRadius.circular(12),
877       borderSide: BorderSide(color: ColorsManager.errorColor),
878     ),
879     suffixIcon: suffixIcon,
880   );
881 }
882 }

```

Lines • Dart 83

Feature / Login

#238 – lib/features/login/presentation/widgets/login_action_buttons_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7  import 'package:moean/features/login/presentation/cubit/login_cubit.dart';
8  import 'package:moean/features/login/presentation/cubit/login_state.dart';
9  import 'package:rounded_loading_button_plus/rounded_loading_button.dart';
10
11  class LoginActionButtonsWidget extends StatefulWidget {
12    const LoginActionButtonsWidget({super.key});
13
14    @override
15    State<LoginActionButtonsWidget> createState() => _LoginActionButtonsWidgetState();
16  }
17
18  class _LoginActionButtonsWidgetState extends State<LoginActionButtonsWidget> {
19    final RoundedLoadingButtonController _btnController = RoundedLoadingButtonController();
20
21    @override
22    Widget build(BuildContext context) {
23      final cubit = LoginCubit.get(context);
24
25      return Column(
26        children: [
27          BlocListener<LoginCubit, LoginState>(
28            listenWhen: (prev, curr) =>
29              curr is LoginLoadingState ||
30              curr is LoginSuccessState ||
31              curr is LoginErrorState ||
32              curr is LoginInitialState,
33            listener: (context, state) {
34              if (state is LoginSuccessState) {
35                _btnController.success();
36              } else if (state is LoginErrorState) {
37                _btnController.error();
38                Future.delayed(const Duration(seconds: 1), () {
39                  if (mounted) {
40                    _btnController.reset();
41                  }
42                });
43              } else if (state is LoginInitialState) {
44                _btnController.reset();
45              }
46            },
47            child: RoundedLoadingButton(
48              controller: _btnController,
49              onPressed: () {
50                cubit.login();
51              },
52              color: ColorsManager.primaryColor,
53              successColor: Colors.green,
54              errorColor: ColorsManager.errorColor,
55              width: MediaQuery.of(context).size.width,
56              height: 52,
57              borderRadius: 24,
58              elevation: 0,
59              child: Row(
60                mainAxisAlignment: MainAxisAlignment.center,
61                children: [

```

```

62         const Icon(
63             Icons.auto_awesome,
64             color: Colors.white,
65             size: 18,
66         ),
67         horizontalSpace8,
68         Text(
69             appTranslation().get('login'),
70             style: TextStylesManager.bold16.copyWith(
71                 color: ColorsManager.white,
72             ),
73         ),
74     ],
75 ),
76 ),
77 ),
78 ],
79 );
80 }
81 }
82
83

```

Lines • Dart 36

Feature / Login

#239 – lib/features/login/presentation/widgets/login_footer_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/routes.dart';
6
7  class LoginFooterWidget extends StatelessWidget {
8      const LoginFooterWidget({super.key});
9
10     @override
11     Widget build(BuildContext context) {
12         return Row(
13             mainAxisAlignment: MainAxisAlignment.center,
14             children: [
15                 GestureDetector(
16                     onTap: () => Navigator.of(context).pushNamed(Routes.register),
17                     child: Text(
18                         appTranslation().get('create_account'),
19                         style: TextStylesManager.medium14.copyWith(
20                             color: ColorsManager.primaryColor,
21                             decoration: TextDecoration.underline,
22                             decorationColor: ColorsManager.primaryColor,
23                         ),
24                     ),
25                 ),
26                 Text(
27                     appTranslation().get('no_account'),
28                     style: TextStylesManager.regular14.copyWith(
29                         color: ColorsManager.textBody,
30                     ),
31                 ),
32             ],
33         );
34     }
35 }
36

```

Lines • Dart 109

Feature / Login

#240 – lib/features/login/presentation/widgets/login_form_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
7  import 'package:moean/core/utils/constants/spacing.dart';
8  import 'package:moean/features/login/presentation/cubit/login_cubit.dart';

```



```

9 import 'package:moean/features/login/presentation/cubit/login_state.dart';
10
11 class LoginFormWidget extends StatelessWidget {
12   const LoginFormWidget({super.key});
13
14   @override
15   Widget build(BuildContext context) {
16     final cubit = LoginCubit.get(context);
17
18     return Column(
19       // crossAxisAlignment: CrossAxisAlignment.end,
20       children: [
21         Text(
22           appTranslation().get('welcome_back'),
23           style: TextStylesManager.bold26.copyWith(
24             color: ColorsManager.textSecondaryDark,
25           ),
26         ),
27         verticalSpace8,
28         Text(
29           appTranslation().get('login_subtitle'),
30           maxLines: 1,
31           style: TextStylesManager.regular14.copyWith(
32             color: ColorsManager.mainText,
33           ),
34           textAlign: TextAlign.center,
35         ),
36         verticalSpace28,
37         Row(
38           mainAxisAlignment: MainAxisAlignment.start,
39
40           children: [
41             Text(
42               appTranslation().get('email'),
43               style: TextStylesManager.medium14.copyWith(
44                 color: ColorsManager.mainText,
45               ),
46             ),
47           ],
48         ),
49         verticalSpace8,
50         PrimaryTextField(
51           controller: cubit.emailController,
52           hint: appTranslation().get('email_hint'),
53           keyboardType: TextInputType.emailAddress,
54           textInputAction: TextInputAction.next,
55           prefixIcon: const Icon(Icons.mail_outline_rounded),
56           validator: (value) {
57             if (value == null || value.trim().isEmpty) {
58               return appTranslation().get('email');
59             }
60             return null;
61           },
62         ),
63         verticalSpace20,
64         Row(
65           mainAxisAlignment: MainAxisAlignment.start,
66
67           children: [
68             Text(
69               appTranslation().get('password'),
70               style: TextStylesManager.medium14.copyWith(
71                 color: ColorsManager.mainText,
72               ),
73             ),
74           ],
75         ),
76         verticalSpace8,
77         BlocBuilder<LoginCubit, LoginState>(
78           buildWhen: (prev, curr) =>
79             curr is LoginPasswordVisibilityChangedState,
80           builder: (context, state) {
81             return PrimaryTextField(
82               controller: cubit.passwordController,
83               hint: appTranslation().get('password_hint'),

```

```

84         isPassword: !cubit.isPasswordVisible,
85         textInputAction: TextInputAction.done,
86         prefixIcon: const Icon(Icons.lock_outline_rounded),
87         suffixIcon: GestureDetector(
88           onTap: cubit.togglePasswordVisibility,
89           child: Icon(
90             cubit.isPasswordVisible
91               ? Icons.visibility_off_outlined
92               : Icons.visibility_outlined,
93             color: ColorsManager.mainText,
94           ),
95         ),
96         validator: (value) {
97           if (value == null || value.isEmpty) {
98             return appTranslation().get('password');
99           }
100           return null;
101         },
102       );
103     },
104   ),
105 ],
106 );
107 }
108 }
109

```

Lines • Dart 82

Feature / Login

#241 – lib/features/login/presentation/widgets/login_header_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/assets_helper.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/routes.dart';
7  import 'package:moean/core/utils/constants/spacing.dart';
8  import 'package:moean/core/utils/extensions/context_extension.dart';
9
10 class LoginHeaderWidget extends StatelessWidget {
11   const LoginHeaderWidget({super.key});
12
13   @override
14   Widget build(BuildContext context) {
15     return Row(
16       mainAxisAlignment: MainAxisAlignment.spaceBetween,
17       children: [
18         Expanded(
19           child: Row(
20             children: [
21               ClipRRect(
22                 borderRadius: BorderRadius.circular(8),
23                 child: Image.asset(
24                   AssetsHelper.icon,
25                   width: 50,
26                   height: 50,
27                   fit: BoxFit.cover,
28                 ),
29             ),
30             horizontalSpace8,
31             Expanded(
32               child: Column(
33                 crossAxisAlignment: CrossAxisAlignment.start,
34                 children: [
35                   Text(
36                     appTranslation().get('app_name'),
37                     maxLines: 1,
38                     overflow: TextOverflow.ellipsis,
39                     style: TextStylesManager.bold22.copyWith(
40                       color: ColorsManager.textSecondaryDark,
41                     ),
42                   ),
43                   Text(
44                     appTranslation().get('smart_assistant'),

```

```

45         maxLines: 1,
46         overflow: TextOverflow.ellipsis,
47         style: TextStylesManager.regular10.copyWith(
48             color: ColorsManager.mainText,
49         ),
50     ),
51 ],
52 ),
53 ),
54 ],
55 ),
56 ),
57   IconButton.icon(
58     style: IconButton.styleFrom(
59       padding: const EdgeInsets.symmetric(horizontal: 4, vertical: 4),
60       minimumSize: Size.zero,
61       tapTargetSize: MaterialTapTargetSize.shrinkWrap,
62     ),
63     onPressed: () {
64       context.push(Routes.home);
65     },
66     icon: Icon(
67       Icons.home_outlined,
68       size: 20,
69       color: ColorsManager.primaryColor,
70     ),
71     label: Text(
72       appTranslation().get('return_home'),
73       style: TextStylesManager.medium14.copyWith(
74         color: ColorsManager.primaryColor,
75       ),
76     ),
77   ),
78 ],
79 );
80 }
81 }
82

```

Lines • Dart 70

Feature / Login

#242 – lib/features/login/presentation/widgets/login_illustration_widget.dart

```

1  // import 'package:flutter/material.dart';
2  // import 'package:moean/core/utils/constants/assets_helper.dart';
3
4  // class LoginIllustrationWidget extends StatelessWidget {
5  //   const LoginIllustrationWidget({super.key});
6
7  //   @override
8  //   Widget build(BuildContext context) {
9  //     return SizedBox(
10 //       height: 220,
11 //       width: double.infinity,
12 //       child: Stack(
13 //         alignment: Alignment.center,
14 //         children: [
15 //           // Background decorative circle
16 //           Positioned(
17 //             top: 10,
18 //             child: Container(
19 //               width: 200,
20 //               height: 200,
21 //               decoration: BoxDecoration(
22 //                 shape: BoxShape.circle,
23 //                 color: const Color(0xFF0E7A5E).withValues(alpha: 0.06),
24 //               ),
25 //             ),
26 //           ),
27 //           // Sparkle decoration top-left
28 //           const Positioned(
29 //             top: 18,
30 //             left: 40,
31 //             child: _SparkleIcon(size: 16),
32 //           ),

```

```

33 //          // Sparkle decoration top-right
34 //          const Positioned(
35 //              top: 30,
36 //              right: 50,
37 //              child: _SparkleIcon(size: 12),
38 //          ),
39 //          // Sparkle decoration bottom-right
40 //          const Positioned(
41 //              bottom: 20,
42 //              right: 30,
43 //              child: _SparkleIcon(size: 10),
44 //          ),
45 //          // Main illustration image
46 //          Image.asset(
47 //              AssetsHelper.img6,
48 //              height: 180,
49 //              fit: BoxFit.contain,
50 //          ),
51 //      ],
52 //  ),
53 //  );
54 //  }
55 //  }
56
57 // class _SparkleIcon extends StatelessWidget {
58 //   final double size;
59 //   const _SparkleIcon({required this.size});
60
61 //   @override
62 //   Widget build(BuildContext context) {
63 //     return Icon(
64 //       Icons.auto_awesome,
65 //       size: size,
66 //       color: const Color(0xFFE2AD3B),
67 //     );
68 //   }
69 // }
70

```

Lines • Dart 69

Feature / Login

#243 – lib/features/login/presentation/widgets/login_remember_forgot_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/routes.dart';
6  import 'package:moean/core/utils/extensions/context_extension.dart';
7
8  class LoginRememberForgotWidget extends StatelessWidget {
9    const LoginRememberForgotWidget({super.key});
10
11    @override
12    Widget build(BuildContext context) {
13      return Row(
14        mainAxisAlignment: MainAxisAlignment.spaceBetween,
15        children: [
16          TextButton(
17            onPressed: () {
18              context.push(Routes.forgotPassword);
19            },
20            style: TextButton.styleFrom(
21              padding: EdgeInsets.zero,
22              minimumSize: Size.zero,
23              tapTargetSize: MaterialTapTargetSize.shrinkWrap,
24            ),
25            child: Text(
26              appTranslation().get('forgot_password'),
27              style: TextStylesManager.regular14.copyWith(
28                color: ColorsManager.primaryColor,
29                decoration: TextDecoration.underline,
30                decorationColor: ColorsManager.primaryColor,
31              ),
32            ),

```

```

33     ),
34     // BlocBuilder<LoginCubit, LoginState>(
35     //   buildWhen: (prev, curr) => curr is LoginRememberMeChangedState,
36     //   builder: (context, state) {
37     //     return Row(
38     //       children: [
39     //         Text(
40     //           appTranslation().get('remember_me'),
41     //           style: TextStylesManager.regular14.copyWith(
42     //             color: ColorsManager.textPrimaryLight,
43     //           ),
44     //         ),
45     //         Transform.scale(
46     //           scale: 0.9,
47     //           child: Checkbox(
48     //             value: cubit.rememberMe,
49     //             onChanged: (_) => cubit.toggleRememberMe(),
50     //             activeColor: ColorsManager.primaryColor,
51     //             shape: RoundedRectangleBorder(
52     //               borderRadius: BorderRadius.circular(4),
53     //             ),
54     //             side: BorderSide(
55     //               color: ColorsManager.borderColor,
56     //               width: 1.5,
57     //             ),
58     //           ),
59     //       ],
60     //     );
61     //   },
62     // ),
63     // ),
64
65   ],
66 );
67 }
68 }
69

```

Lines • Dart 34 Feature / Payment

#244 – lib/features/payment/data/models/order_model.dart

```

1  import 'package:moean/features/payment/data/models/subscription_plan_model.dart';
2
3  class OrderModel {
4    final int id;
5    final int serviceId;
6    final String amount;
7    final String currency;
8    final String status;
9    final SubscriptionPlanModel? service;
10
11    const OrderModel({
12      required this.id,
13      required this.serviceId,
14      required this.amount,
15      required this.currency,
16      required this.status,
17      this.service,
18    });
19
20    factory OrderModel.fromJson(Map<String, dynamic> json) {
21      return OrderModel(
22        id: json['id'] as int,
23        serviceId: json['service_id'] as int? ?? 0,
24        amount: json['amount']?.toString() ?? '0',
25        currency: json['currency']?.toString() ?? 'SAR',
26        status: json['status']?.toString() ?? 'pending',
27        service: json['service'] != null
28          ? SubscriptionPlanModel.fromJson(
29            json['service'] as Map<String, dynamic>
30          ) : null,
31      );
32    }
33

```

```
33 }
34
```

Lines • Dart 82 Feature / Payment

#245 – lib/features/payment/data/models/payment_model.dart

```
1 import 'package:moean/features/payment/data/models/order_model.dart';
2
3 class PaymentUserModel {
4   final int id;
5   final String name;
6   final String email;
7
8   const PaymentUserModel({
9     required this.id,
10    required this.name,
11    required this.email,
12  });
13
14  factory PaymentUserModel.fromJson(Map<String, dynamic> json) {
15    return PaymentUserModel(
16      id: json['id'] as int? ?? 0,
17      name: json['name'] as String? ?? '',
18      email: json['email'] as String? ?? '',
19    );
20  }
21 }
22
23 class PaymentModel {
24   final int id;
25   final int orderId;
26   final String paymentMethod;
27   final String? gateway;
28   final String? moyasarPaymentId;
29   final String amount;
30   final String currency;
31   final String status;
32   final String? paidAt;
33   final String? verifiedAt;
34   final String? receiptPath;
35   final String? receiptUrl;
36   final String? createdAt;
37   final OrderModel? order;
38   final PaymentUserModel? user;
39
40   const PaymentModel({
41     required this.id,
42     required this.orderId,
43     required this.paymentMethod,
44     this.gateway,
45     this.moyasarPaymentId,
46     required this.amount,
47     required this.currency,
48     required this.status,
49     this.paidAt,
50     this.verifiedAt,
51     this.receiptPath,
52     this.receiptUrl,
53     this.createdAt,
54     this.order,
55     this.user,
56  });
57
58  factory PaymentModel.fromJson(Map<String, dynamic> json) {
59    return PaymentModel(
60      id: json['id'] as int,
61      orderId: json['order_id'] as int? ?? 0,
62      paymentMethod: json['payment_method'] as String? ?? '',
63      gateway: json['gateway'] as String?,
64      moyasarPaymentId: json['moyasar_payment_id'] as String?,
65      amount: json['amount'] as String? ?? '0',
66      currency: json['currency'] as String? ?? 'SAR',
67      status: json['status'] as String? ?? 'pending',
68      paidAt: json['paid_at'] as String?,
```

```

69     verifiedAt: json['verified_at'] as String?,
70     receiptPath: json['receipt_path'] as String?,
71     receiptUrl: json['receipt_url'] as String?,
72     createdAt: json['created_at'] as String?,
73     order: json['order'] != null
74         ? OrderModel.fromJson(json['order'] as Map<String, dynamic>)
75         : null,
76     user: json['user'] != null
77         ? PaymentUserModel.fromJson(json['user'] as Map<String, dynamic>)
78         : null,
79   );
80 }
81 }
82

```

Lines • Dart 112

Feature / Payment

#246 – lib/features/payment/data/models/subscription_current_model.dart

```

1  import 'package:moean/features/payment/data/models/subscription_plan_model.dart';
2
3  class SubscriptionCurrentModel {
4    final SubscriptionPlanModel? plan;
5    final bool isInTrial;
6    final bool isSubscribed;
7    final DateTime? trialEndsAt;
8    final DateTime? subscriptionEndsAt;
9    final int trialDaysRemaining;
10   final SubscriptionUsageModel usage;
11
12   SubscriptionCurrentModel({
13     this.plan,
14     required this.isInTrial,
15     required this.isSubscribed,
16     this.trialEndsAt,
17     this.subscriptionEndsAt,
18     required this.trialDaysRemaining,
19     required this.usage,
20   });
21
22   factory SubscriptionCurrentModel.fromJson(Map<String, dynamic> json) {
23     return SubscriptionCurrentModel(
24       plan: json['plan'] != null ? SubscriptionPlanModel.fromJson(json['plan']) : null,
25       isInTrial: json['is_in_trial'] as bool? ?? false,
26       isSubscribed: json['is_subscribed'] as bool? ?? false,
27       trialEndsAt: json['trial_ends_at'] != null ? DateTime.tryParse(json['trial_ends_at']) : null,
28       subscriptionEndsAt: json['subscription_ends_at'] != null ? DateTime.tryParse(json['subscription_ends_a
t']) : null,
29       trialDaysRemaining: json['trial_days_remaining'] as int? ?? 0,
30       usage: SubscriptionUsageModel.fromJson(
31         json['usage'] is Map<String, dynamic> ? json['usage'] as Map<String, dynamic> : {},
32       ),
33     );
34   }
35
36   /// تاريخ انتهاء الصلاحية الفعلي (سواء للاشتراك المدفوع أو التجربة)
37   DateTime? get effectiveEndsAt => isSubscribed ? subscriptionEndsAt : trialEndsAt;
38
39   int get dynamicTrialDaysRemaining => dynamicDaysRemaining;
40
41   /// حساب الأيام المتبقية ديناميكياً
42   int get dynamicDaysRemaining {
43     final endsAt = effectiveEndsAt;
44     if (endsAt == null) return 0;
45     final now = DateTime.now();
46     if (endsAt.isBefore(now)) return 0;
47     return (endsAt.difference(now).inHours / 24).ceil();
48   }
49
50   bool get isLastTrialDay => isInTrial && dynamicTrialDaysRemaining <= 1;
51
52   bool get isSubscriptionExpired {
53     if (isSubscribed && subscriptionEndsAt != null) {
54       return subscriptionEndsAt!.isBefore(DateTime.now());
55     }
56   }
57

```

```

56     return false;
57 }
58
59 /// هل انتهت الصلاحية تماماً؟
60 bool get isExpired {
61     if (isSubscribed) {
62         return subscriptionEndsAt != null && subscriptionEndsAt!.isBefore(DateTime.now());
63     }
64     return !isInTrial || dynamicTrialDaysRemaining <= 0;
65 }
66
67 /// عنوان الخطة الحالي
68 String get planTitle {
69     if (isSubscribed) return plan?.name ?? 'اشتراك مدفوع';
70     if (isInTrial) return 'تجربة مجانية (7 أيام)';
71     return 'انتهت التجربة';
72 }
73
74 /// نص موعد الانتهاء
75 String get expirationSubtitle {
76     final endsAt = effectiveEndsAt;
77     if (endsAt == null) return 'بدون تاريخ انتهاء';
78     final formattedDate = "${endsAt.year}/${endsAt.month.toString().padLeft(2, '0')}/${endsAt.day.toString().padLeft(2, '0')}";
79
80     if (isSubscribed) {
81         return 'يوم $dynamicDaysRemaining متبقي $formattedDate ينتهي في';
82     }
83     if (isInTrial) {
84         return 'يوم " : "أيا " ? 1 == $dynamicDaysRemaining متبقي $formattedDate تنتهي في'
85     }
86     return 'انتهت الصلاحية في $formattedDate';
87 }
88 }
89
90 class SubscriptionUsageModel {
91     final int aiUsedThisMonth;
92     final int lessonsPreparedToday;
93     final int aiRemaining;
94     final int lessonsRemainingToday;
95
96     SubscriptionUsageModel({
97         required this.aiUsedThisMonth,
98         required this.lessonsPreparedToday,
99         required this.aiRemaining,
100         required this.lessonsRemainingToday,
101     });
102
103     factory SubscriptionUsageModel.fromJson(Map<String, dynamic> json) {
104         return SubscriptionUsageModel(
105             aiUsedThisMonth: json['ai_used_this_month'] as int? ?? 0,
106             lessonsPreparedToday: json['lessons_prepared_today'] as int? ?? 0,
107             aiRemaining: json['ai_remaining'] as int? ?? 0,
108             lessonsRemainingToday: json['lessons_remaining_today'] as int? ?? 0,
109         );
110     }
111 }
112

```

```

1 class SubscriptionPlanModel {
2     final int id;
3     final String name;
4     final String slug;
5     final String price;
6     final int aiQuotaPerMonth;
7     final int lessonLimitPerDay;
8     final Map<String, dynamic> features;
9
10     const SubscriptionPlanModel({
11         required this.id,

```



```

12     required this.name,
13     required this.slug,
14     required this.price,
15     required this.aiQuotaPerMonth,
16     required this.lessonLimitPerDay,
17     required this.features,
18   });
19
20   factory SubscriptionPlanModel.fromJson(Map<String, dynamic> json) {
21     return SubscriptionPlanModel(
22       id: json['id'] as int,
23       name: json['name'] as String? ?? '',
24       slug: json['slug'] as String? ?? '',
25       price: json['price']?.toString() ?? '0',
26       aiQuotaPerMonth: json['ai_quota_per_month'] as int? ?? 0,
27       lessonLimitPerDay: json['lesson_limit_per_day'] as int? ?? 0,
28       features: json['features'] as Map<String, dynamic>? ?? {},
29     );
30   }
31 }
32

```

Lines • Dart 259

Feature / Payment

#248 – lib/features/payment/presentation/cubit/payment_cubit.dart

```

1  import 'package:flutter/widgets.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/network/remote/api_service.dart';
4  import 'package:moean/features/payment/data/models/order_model.dart';
5  import 'package:moean/features/payment/data/models/payment_model.dart';
6  import 'package:moean/features/payment/data/models/subscription_plan_model.dart';
7  import 'package:moean/features/payment/presentation/cubit/payment_state.dart';
8  import 'package:moean/core/di/injections.dart';
9  import 'package:moean/features/payment/presentation/cubit/subscription_cubit.dart';
10
11  class PaymentCubit extends Cubit<PaymentState> {
12    PaymentCubit() : super(PaymentInitial());
13
14    static PaymentCubit get(BuildContext context) => BlocProvider.of(context);
15
16    List<SubscriptionPlanModel> plans = [];
17    OrderModel? currentOrder;
18    Map<String, dynamic>? bankInfo;
19    List<PaymentModel> paymentHistory = [];
20    int selectedPlanIndex = 0;
21    int selectedMethodIndex = 0;
22    String? appliedPromoCode;
23    Map<String, dynamic>? promoValidation;
24
25    Future<void> loadPlans() async {
26      if (!isClosed) emit(PlansLoading());
27      final result = await ApiService.getSubscriptions();
28      if (isClosed) return;
29      result.fold(
30        (error) {
31          if (!isClosed) emit(PlansError(error));
32        },
33        (data) {
34          plans = data
35            .map((e) =>
36              SubscriptionPlanModel.fromJson(e as Map<String, dynamic>))
37            .where((p) => double.tryParse(p.price) != null &&
38              double.parse(p.price) > 0 && double.parse(p.price) != 1)
39            .toList();
40          if (!isClosed) emit(PlansLoaded(plans));
41        },
42      );
43    }
44
45    void selectPlan(int index) {
46      selectedPlanIndex = index;
47      if (!isClosed) emit(PlansLoaded(plans));
48    }
49

```

```

50 void selectMethod(int index) {
51     selectedMethodIndex = index;
52     if (!isClosed) emit(PlansLoaded(plans));
53 }
54
55 Future<void> createOrder() async {
56     if (plans.isEmpty) return;
57     if (!isClosed) emit(OrderCreating());
58     final serviceId = plans[selectedPlanIndex].id;
59     final result = await ApiService.createOrder(
60         serviceId,
61         promoCode: appliedPromoCode,
62     );
63     if (isClosed) return;
64     result.fold(
65         (error) {
66             if (!isClosed) emit(OrderError(error));
67         },
68         (data) {
69             currentOrder = OrderModel.fromJson(
70                 data['order'] as Map<String, dynamic>;
71             if (!isClosed) emit(OrderCreated(currentOrder!));
72         },
73     );
74 }
75
76 Future<void> validatePromo(String code) async {
77     if (plans.isEmpty || code.trim().isEmpty) return;
78     final planSlug = plans[selectedPlanIndex].slug;
79     if (!isClosed) emit(PromoValidating());
80     final result = await ApiService.validatePromoCode(
81         code: code.trim(),
82         planSlug: planSlug,
83     );
84     if (isClosed) return;
85     result.fold(
86         (error) {
87             appliedPromoCode = null;
88             promoValidation = null;
89             if (!isClosed) emit(PromoError(error));
90         },
91         (data) {
92             appliedPromoCode = code.trim();
93             promoValidation = data;
94             if (!isClosed) emit(PromoValidated(data));
95         },
96     );
97 }
98
99 void clearPromo() {
100     appliedPromoCode = null;
101     promoValidation = null;
102     if (!isClosed) emit(PromoCleared());
103 }
104
105 Future<void> loadBankInfo() async {
106     if (!isClosed) emit(BankInfoLoading());
107     final result = await ApiService.getBankTransferInfo();
108     if (isClosed) return;
109     result.fold(
110         (error) {
111             if (!isClosed) emit(BankInfoError(error));
112         },
113         (data) {
114             bankInfo = data;
115             if (!isClosed) emit(BankInfoLoaded(data));
116         },
117     );
118 }
119
120 // Moyasar Flow
121 Future<void> getMoyasarConfig(int orderId) async {
122     if (!isClosed) emit(MoyasarConfigLoading());
123     final result = await ApiService.getOrderCheckout(orderId);
124     if (isClosed) return;

```

```

125     result.fold(
126         (error) {
127             if (!isClosed) emit(MoyasarConfigError(error));
128         },
129         (data) {
130             if (!isClosed) emit(MoyasarConfigLoaded(data));
131         },
132     );
133 }
134
135 // MyFatoorah Flow
136 Future<void> initMyfatoorahPayment(int orderId) async {
137     if (!isClosed) emit(MyfatoorahSessionLoading());
138     final result = await ApiService.createMyfatoorahSession(orderId);
139     if (isClosed) return;
140     result.fold(
141         (error) {
142             if (!isClosed) emit(MyfatoorahSessionError(error));
143         },
144         (data) {
145             if (!isClosed) emit(MyfatoorahSessionLoaded(data));
146         },
147     );
148 }
149
150 Future<void> executeMyfatoorahPayment(int orderId, String sessionId) async {
151     if (!isClosed) emit(MyfatoorahExecuteLoading());
152     final result = await ApiService.executeMyfatoorahPayment(orderId: orderId, sessionId: sessionId);
153     if (isClosed) return;
154     result.fold(
155         (error) {
156             if (!isClosed) emit(MyfatoorahExecuteError(error));
157         },
158         (data) {
159             sl<SubscriptionCubit>().fetchCurrentSubscription();
160             if (!isClosed) emit(MyfatoorahExecuteLoaded(data));
161         },
162     );
163 }
164
165 Future<void> verifyMyfatoorahPayment(String paymentKey) async {
166     if (!isClosed) emit(PaymentVerifying());
167     final result = await ApiService.verifyMyfatoorahPayment(
168         key: paymentKey,
169     );
170     if (isClosed) return;
171     result.fold(
172         (error) {
173             if (!isClosed) emit(PaymentVerifyError(error));
174         },
175         (data) {
176             final payment = PaymentModel.fromJson(
177                 data['payment'] as Map<String, dynamic>;
178
179             // Refresh subscription state globally
180             sl<SubscriptionCubit>().fetchCurrentSubscription();
181
182             if (!isClosed) emit(PaymentVerified(payment));
183         },
184     );
185 }
186
187 Future<void> verifyPayment(String paymentId, {int? orderId}) async {
188     if (!isClosed) emit(PaymentVerifying());
189
190     // نقوم بحفظ المرجع أولاً (حسب المستند) إذا كان لدينا
191     if (orderId != null) {
192         await ApiService.savePaymentReference(
193             orderId: orderId,
194             moyasarPaymentId: paymentId,
195         );
196     }
197
198     final result = await ApiService.verifyPayment(paymentId);
199     if (isClosed) return;

```

```

200     result.fold(
201       (error) {
202         if (!isClosed) emit(PaymentVerifyError(error));
203       },
204       (data) {
205         final payment = PaymentModel.fromJson(
206           data['payment'] as Map<String, dynamic>);
207
208         // Refresh subscription state globally
209         sl<SubscriptionCubit>().fetchCurrentSubscription();
210
211         if (!isClosed) emit(PaymentVerified(payment));
212       },
213     );
214   }
215
216   Future<void> uploadReceiptForOrder({
217     required int orderId,
218     required String filePath,
219     required String fileName,
220   }) async {
221     if (!isClosed) emit(ReceiptUploading());
222     final result = await ApiService.uploadManualReceipt(
223       orderId: orderId,
224       filePath: filePath,
225       fileName: fileName,
226     );
227     if (isClosed) return;
228     result.fold(
229       (error) {
230         if (!isClosed) emit(ReceiptUploadError(error));
231       },
232       (data) {
233         final payment = PaymentModel.fromJson(
234           data['payment'] as Map<String, dynamic>);
235         if (!isClosed) emit(ReceiptUploaded(payment));
236       },
237     );
238   }
239
240   Future<void> loadHistory() async {
241     if (!isClosed) emit(HistoryLoading());
242     final result = await ApiService.getPaymentHistory();
243     if (isClosed) return;
244     result.fold(
245       (error) {
246         if (!isClosed) emit(HistoryError(error));
247       },
248       (data) {
249         final list = (data['data'] as List<dynamic>? ?? [])
250           .map((e) =>
251             PaymentModel.fromJson(e as Map<String, dynamic>))
252           .toList();
253         paymentHistory = list;
254         if (!isClosed) emit(HistoryLoaded(list));
255       },
256     );
257   }
258 }
259

```

Lines • Dart 118

Feature / Payment

#249 – lib/features/payment/presentation/cubit/payment_state.dart

```

1  import 'package:moean/features/payment/data/models/order_model.dart';
2  import 'package:moean/features/payment/data/models/payment_model.dart';
3  import 'package:moean/features/payment/data/models/subscription_plan_model.dart';
4
5  abstract class PaymentState {}
6
7  class PaymentInitial extends PaymentState {}
8
9  // Plans
10 class PlansLoading extends PaymentState {}

```

```
11 class PlansLoaded extends PaymentState {
12     final List<SubscriptionPlanModel> plans;
13     PlansLoaded(this.plans);
14 }
15 class PlansError extends PaymentState {
16     final String error;
17     PlansError(this.error);
18 }
19
20 // Order
21 class OrderCreating extends PaymentState {}
22 class OrderCreated extends PaymentState {
23     final OrderModel order;
24     OrderCreated(this.order);
25 }
26 class OrderError extends PaymentState {
27     final String error;
28     OrderError(this.error);
29 }
30
31 // Bank Info
32 class BankInfoLoading extends PaymentState {}
33 class BankInfoLoaded extends PaymentState {
34     final Map<String, dynamic> data;
35     BankInfoLoaded(this.data);
36 }
37 class BankInfoError extends PaymentState {
38     final String error;
39     BankInfoError(this.error);
40 }
41
42 // Receipt Upload
43 class ReceiptUploading extends PaymentState {}
44 class ReceiptUploaded extends PaymentState {
45     final PaymentModel payment;
46     ReceiptUploaded(this.payment);
47 }
48 class ReceiptUploadError extends PaymentState {
49     final String error;
50     ReceiptUploadError(this.error);
51 }
52
53 // Moyasar Config
54 class MoyasarConfigLoading extends PaymentState {}
55 class MoyasarConfigLoaded extends PaymentState {
56     final Map<String, dynamic> config;
57     MoyasarConfigLoaded(this.config);
58 }
59 class MoyasarConfigError extends PaymentState {
60     final String error;
61     MoyasarConfigError(this.error);
62 }
63
64 // Payment Verification
65 class PaymentVerifying extends PaymentState {}
66 class PaymentVerified extends PaymentState {
67     final PaymentModel payment;
68     PaymentVerified(this.payment);
69 }
70 class PaymentVerifyError extends PaymentState {
71     final String error;
72     PaymentVerifyError(this.error);
73 }
74
75 // History
76 class HistoryLoading extends PaymentState {}
77 class HistoryLoaded extends PaymentState {
78     final List<PaymentModel> history;
79     HistoryLoaded(this.history);
80 }
81 class HistoryError extends PaymentState {
82     final String error;
83     HistoryError(this.error);
84 }
85
```

```

86 // MyFatoorah
87 class MyfatoorahSessionLoading extends PaymentState {}
88 class MyfatoorahSessionLoaded extends PaymentState {
89   final Map<String, dynamic> session;
90   MyfatoorahSessionLoaded(this.session);
91 }
92 class MyfatoorahSessionError extends PaymentState {
93   final String error;
94   MyfatoorahSessionError(this.error);
95 }
96
97 class MyfatoorahExecuteLoading extends PaymentState {}
98 class MyfatoorahExecuteLoaded extends PaymentState {
99   final Map<String, dynamic> paymentData;
100   MyfatoorahExecuteLoaded(this.paymentData);
101 }
102 class MyfatoorahExecuteError extends PaymentState {
103   final String error;
104   MyfatoorahExecuteError(this.error);
105 }
106
107 // Promo Code Validation
108 class PromoValidating extends PaymentState {}
109 class PromoValidated extends PaymentState {
110   final Map<String, dynamic> data;
111   PromoValidated(this.data);
112 }
113 class PromoError extends PaymentState {
114   final String error;
115   PromoError(this.error);
116 }
117 class PromoCleared extends PaymentState {}
118

```

Lines • Dart 75

Feature / Payment

#250 – lib/features/payment/presentation/cubit/subscription_cubit.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/network/remote/api_service.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/features/payment/presentation/cubit/subscription_state.dart';
6 import 'package:moean/features/payment/data/models/subscription_current_model.dart';
7
8 class SubscriptionCubit extends Cubit<SubscriptionState> {
9   SubscriptionCubit() : super(SubscriptionInitial());
10
11   static SubscriptionCubit get(BuildContext context) => BlocProvider.of(context);
12
13   SubscriptionCurrentModel? currentSubscription;
14   String? lastError;
15   bool _isLoading = false;
16
17   Future<void> fetchCurrentSubscription({bool forceRefresh = false}) async {
18     if (token == null || token!.isEmpty) return;
19     if (_isLoading) return;
20
21     if (!forceRefresh && (currentSubscription != null || lastError != null)) {
22       if (currentSubscription != null && state is! SubscriptionLoaded) {
23         if (!isClosed) emit(SubscriptionLoaded(currentSubscription!));
24       } else if (lastError != null && state is! SubscriptionError) {
25         if (!isClosed) emit(SubscriptionError(lastError!));
26       }
27       return;
28     }
29
30     _isLoading = true;
31     if (currentSubscription == null && lastError == null && !isClosed) {
32       emit(SubscriptionLoading());
33     }
34
35     final result = await ApiService.getCurrentSubscription();
36     _isLoading = false;
37     if (isClosed) return;

```

```

38
39     result.fold(
40       (error) {
41         lastError = error;
42         if (!isClosed) emit(SubscriptionError(error));
43       },
44       (data) {
45         lastError = null;
46         final current = SubscriptionCurrentModel.fromJson(data);
47         currentSubscription = current;
48         if (!isClosed) emit(SubscriptionLoaded(current));
49       },
50     );
51   }
52
53   void clearData() {
54     currentSubscription = null;
55     lastError = null;
56     _isLoading = false;
57     if (!isClosed) emit(SubscriptionInitial());
58   }
59
60   Future<void> upgradeSubscription({required String planSlug, String? promoCode}) async {
61     if (!isClosed) emit(SubscriptionUpgradeLoading());
62     final result = await ApiService.upgradeSubscription(planSlug: planSlug, promoCode: promoCode);
63     if (isClosed) return;
64     result.fold(
65       (error) {
66         if (!isClosed) emit(SubscriptionUpgradeError(error));
67       },
68       (data) {
69         if (!isClosed) emit(SubscriptionUpgradeSuccess(data['message'] ?? 'تم الترقية بنجاح'));
70         fetchCurrentSubscription(forceRefresh: true); // Refresh after upgrade
71       },
72     );
73   }
74 }
75

```

Lines • Dart 37

Feature / Payment

#251 – lib/features/payment/presentation/cubit/subscription_state.dart

```

1  import 'package:moean/features/payment/data/models/subscription_current_model.dart';
2  import 'package:moean/features/payment/data/models/subscription_plan_model.dart';
3
4  abstract class SubscriptionState {}
5
6  class SubscriptionInitial extends SubscriptionState {}
7  class SubscriptionLoading extends SubscriptionState {}
8  class SubscriptionLoaded extends SubscriptionState {
9    final SubscriptionCurrentModel current;
10    SubscriptionLoaded(this.current);
11  }
12  class SubscriptionError extends SubscriptionState {
13    final String error;
14    SubscriptionError(this.error);
15  }
16
17  class SubscriptionUpgradeLoading extends SubscriptionState {}
18  class SubscriptionUpgradeSuccess extends SubscriptionState {
19    final String message;
20    SubscriptionUpgradeSuccess(this.message);
21  }
22  class SubscriptionUpgradeError extends SubscriptionState {
23    final String error;
24    SubscriptionUpgradeError(this.error);
25  }
26
27  // Available plans
28  class SubscriptionPlansLoading extends SubscriptionState {}
29  class SubscriptionPlansLoaded extends SubscriptionState {
30    final List<SubscriptionPlanModel> plans;
31    SubscriptionPlansLoaded(this.plans);
32  }

```

```

33 class SubscriptionPlansError extends SubscriptionState {
34   final String error;
35   SubscriptionPlansError(this.error);
36 }
37

```

Lines • Dart 201

Feature / Payment

#252 – lib/features/payment/presentation/screen/bank_transfer_screen.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/primary/conditional_builder.dart';
7  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
8  import 'package:moean/core/utils/constants/routes.dart';
9  import 'package:moean/core/utils/constants/spacing.dart';
10 import 'package:moean/features/payment/presentation/cubit/payment_cubit.dart';
11 import 'package:moean/features/payment/presentation/cubit/payment_state.dart';
12 import 'package:moean/features/payment/presentation/widgets/bank_info_widget.dart';
13 import 'package:moean/features/payment/presentation/widgets/order_summary_widget.dart';
14 import 'package:moean/features/payment/presentation/widgets/receipt_upload_widget.dart';
15
16 class BankTransferScreen extends StatefulWidget {
17   const BankTransferScreen({super.key});
18
19   @override
20   State<BankTransferScreen> createState() => _BankTransferScreenState();
21 }
22
23 class _BankTransferScreenState extends State<BankTransferScreen> {
24   String? _filePath;
25   String? _fileName;
26   late int _orderId;
27   late String _amount;
28   late String _planName;
29
30   @override
31   void didChangeDependencies() {
32     super.didChangeDependencies();
33     final args = ModalRoute.of(context)?.settings.arguments
34       as Map<String, dynamic>?;
35     _orderId = args?['orderId'] as int? ?? 0;
36     _amount = args?['amount'] as String? ?? '';
37     _planName = args?['planName'] as String? ?? '';
38     PaymentCubit.get(context).loadBankInfo();
39   }
40
41   @override
42   Widget build(BuildContext context) {
43     return Scaffold(
44       backgroundColor: ColorsManager.background,
45       appBar: AppBar(
46         backgroundColor: ColorsManager.surfacePrimary,
47         elevation: 0,
48         centerTitle: true,
49         title: Text(
50           appTranslation().get('pay_bank_transfer_title'),
51           style: TextStylesManager.bold16.copyWith(
52             color: ColorsManager.textPrimary,
53           ),
54       ),
55       leading: IconButton(
56         icon: Icon(
57           Icons.arrow_back_ios_new_rounded,
58           color: ColorsManager.textPrimary,
59           size: 20,
60         ),
61         onPressed: () => Navigator.pop(context),
62       ),
63       bottom: PreferredSize(
64         preferredSize: const Size.fromHeight(1),
65         child: Divider(height: 1, color: ColorsManager.borderColor),
66       ),
67     );
68   }
69 }

```



```

66     ),
67   ),
68   body: BlocConsumer<PaymentCubit, PaymentState>(
69     listenWhen: (_, s) =>
70       s is ReceiptUploaded || s is ReceiptUploadError,
71     listener: (context, state) {
72       if (state is ReceiptUploadError) {
73         ScaffoldMessenger.of(context).showSnackBar(
74           SnackBar(
75             content: Text(state.error, textAlign: TextAlign.center),
76             backgroundColor: ColorsManager.errorColor,
77             behavior: SnackBarBehavior.floating,
78             shape: RoundedRectangleBorder(
79               borderRadius: BorderRadius.circular(12)),
80           ),
81         );
82       } else if (state is ReceiptUploaded) {
83         Navigator.pushReplacementNamed(
84           context,
85           Routes.paymentResult,
86           arguments: {
87             'status': state.payment.status,
88             'from': 'bank',
89           },
90         );
91       }
92     },
93     buildWhen: (_, s) =>
94       s is BankInfoLoading ||
95       s is BankInfoLoaded ||
96       s is BankInfoError ||
97       s is ReceiptUploading ||
98       s is ReceiptUploaded ||
99       s is ReceiptUploadError, // تم إضافة الحالات النهائية هنا
100     builder: (context, state) {
101       final cubit = PaymentCubit.get(context);
102       return ConditionalBuilder(
103         loadingState: state is BankInfoLoading,
104         errorState: state is BankInfoError,
105         errorBuilder: (_) => Center(
106           child: Padding(
107             padding: const EdgeInsets.all(24),
108             child: Text(
109               (state as BankInfoError).error,
110               style: TextStyleManager.regular14
111                 .copyWith(color: ColorsManager.errorColor),
112               textAlign: TextAlign.center,
113             ),
114           ),
115         ),
116         successBuilder: (_) => SingleChildScrollView(
117           padding: const EdgeInsets.all(20),
118           child: Column(
119             crossAxisAlignment: CrossAxisAlignment.end,
120             children: [
121               if (_planName.isNotEmpty || _amount.isNotEmpty)
122                 OrderSummaryWidget(
123                   planName: _planName,
124                   amount: _amount,
125                 ),
126               verticalSpace16,
127               if (cubit.bankInfo != null)
128                 BankInfoWidget(bankInfo: cubit.bankInfo!),
129               verticalSpace16,
130               _UploadSection(
131                 onFileSelected: (path, name) {
132                   _filePath = path;
133                   _fileName = name;
134                 },
135               ),
136               verticalSpace24,
137               PrimaryElevatedButton(
138                 text: appTranslation().get('pay_send_receipt'),
139                 icon: Icon(
140                   Icons.check_circle_outline,

```

```

141         size: 20,
142         color: ColorsManager.white,
143     ),
144     isLoading: state is ReceiptUploading,
145     onPressed: state is ReceiptUploading
146       ? null
147       : () {
148         if (_filePath == null || _fileName == null) {
149           ScaffoldMessenger.of(context).showSnackBar(
150             SnackBar(
151               content: Text(
152                 appTranslation().get(
153                   'pay_please_choose_receipt'),
154                 textAlign: TextAlign.center,
155               ),
156               backgroundColor: ColorsManager.errorColor,
157               behavior: SnackBarBehavior.floating,
158               shape: RoundedRectangleBorder(
159                 borderRadius: BorderRadius.circular(12),
160               ),
161             ),
162           );
163           return;
164         }
165         cubit.uploadReceiptForOrder(
166           orderId: _orderId,
167           filePath: _filePath!,
168           fileName: _fileName!,
169         );
170       },
171     ),
172     verticalSpace16,
173   ],
174 ),
175 ),
176 );
177 },
178 ),
179 );
180 }
181 }
182
183 class _UploadSection extends StatelessWidget {
184   final void Function(String, String) onFileSelected;
185
186   const _UploadSection({required this.onFileSelected});
187
188   @override
189   Widget build(BuildContext context) {
190     return Container(
191       padding: const EdgeInsets.all(16),
192       decoration: BoxDecoration(
193         color: ColorsManager.surfacePrimary,
194         borderRadius: BorderRadius.circular(16),
195         border: Border.all(color: ColorsManager.borderColor),
196       ),
197       child: ReceiptUploadWidget(onFileSelected: onFileSelected),
198     );
199   }
200 }
201

```

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/primary/conditional_builder.dart';
7 import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
8 import 'package:moean/core/utils/constants/routes.dart';
9 import 'package:moean/core/utils/constants/spacing.dart';

```

```

10 import 'package:moean/core/utils/extensions/context_extension.dart';
11 import 'package:moean/features/payment/presentation/cubit/payment_cubit.dart';
12 import 'package:moean/features/payment/presentation/cubit/payment_state.dart';
13 import 'package:moean/features/payment/presentation/widgets/plan_card_widget.dart';
14 import 'package:moean/features/payment/presentation/widgets/promo_code_section_widget.dart';
15
16 class CheckoutScreen extends StatefulWidget {
17   const CheckoutScreen({super.key});
18
19   @override
20   State<CheckoutScreen> createState() => _CheckoutScreenState();
21 }
22
23 class _CheckoutScreenState extends State<CheckoutScreen> {
24   @override
25   void initState() {
26     super.initState();
27     PaymentCubit.get(context).loadPlans();
28   }
29
30   @override
31   Widget build(BuildContext context) {
32     return Scaffold(
33       backgroundColor: ColorsManager.background,
34       appBar: AppBar(
35         backgroundColor: ColorsManager.background,
36         elevation: 0,
37         centerTitle: true,
38         title: Text(
39           appTranslation().get('payment'),
40           style: TextStyleManager.bold18.copyWith(color: ColorsManager.primaryColor),
41         ),
42         leading: IconButton(
43           icon: Icon(Icons.arrow_back_ios, color: ColorsManager.mainText),
44           onPressed: () => context.pop(),
45         ),
46       ),
47
48       body: SafeArea(
49         child: Column(
50           children: [
51             // _CheckoutHeader(),
52             Expanded(
53               child: BlocConsumer<PaymentCubit, PaymentState>(
54                 listenWhen: (_, s) =>
55                   s is OrderCreated || s is OrderError,
56                 listener: (context, state) {
57                   if (state is OrderError) {
58                     ScaffoldMessenger.of(context).showSnackBar(
59                       SnackBar(
60                         content: Text(state.error, textAlign: TextAlign.center),
61                         backgroundColor: ColorsManager.errorColor,
62                         behavior: SnackBarBehavior.floating,
63                         shape: RoundedRectangleBorder(
64                           borderRadius: BorderRadius.circular(12)),
65                       ),
66                     );
67                   } else if (state is OrderCreated) {
68                     final cubit = PaymentCubit.get(context);
69                     if (cubit.selectedMethodIndex == 0) {
70                       context.push<Map<String, dynamic>>(
71                         Routes.myfatoorahPayment,
72                         arguments: {
73                           'orderId': state.order.id,
74                           'amount': state.order.amount,
75                         },
76                       );
77                     } else {
78                       context.push<Map<String, dynamic>>(
79                         Routes.bankTransfer,
80                         arguments: {
81                           'orderId': state.order.id,
82                           'amount': state.order.amount,
83                           'planName': state.order.service?.name ?? '',
84

```

```

85         },
86       );
87     }
88   }
89 },
90   buildWhen: (_, s) =>
91     s is PlansLoading ||
92     s is PlansLoaded ||
93     s is PlansError ||
94     s is OrderCreating ||
95     s is OrderCreated ||
96     s is OrderError ||
97     s is PromoValidating ||
98     s is PromoValidated ||
99     s is PromoError ||
100    s is PromoCleared,
101   builder: (context, state) {
102     final cubit = PaymentCubit.get(context);
103     return ConditionalBuilder(
104       loadingState: state is PlansLoading,
105       errorState: state is PlansError,
106       emptyState: state is PlansLoaded && cubit.plans.isEmpty,
107       errorBuilder: (_) => Center(
108         child: Text(
109           (state as PlansError).error,
110           style: TextStyleManager.regular14.copyWith(
111             color: ColorsManager.errorColor),
112           textAlign: TextAlign.center,
113         ),
114       ),
115       emptyBuilder: (_) => Center(
116         child: Text(
117           appTranslation().get('pay_no_plans'),
118           style: TextStyleManager.regular14,
119         ),
120       ),
121       successBuilder: (_) =>
122         _CheckoutBody(isSubmitting: state is OrderCreating),
123     );
124   },
125 ),
126 ),
127 ],
128 ),
129 ),
130 );
131 }
132 }
133
134
135
136 class _CheckoutBody extends StatelessWidget {
137   final bool isSubmitting;
138
139   const _CheckoutBody({required this.isSubmitting});
140
141   @override
142   Widget build(BuildContext context) {
143     return BlocBuilder<PaymentCubit, PaymentState>(
144       buildWhen: (_, s) =>
145         s is PlansLoaded || s is OrderCreating || s is OrderCreated || s is OrderError,
146       builder: (context, state) {
147         final cubit = PaymentCubit.get(context);
148         final plans = cubit.plans;
149         final selectedPlan = plans.isNotEmpty ? plans[cubit.selectedPlanIndex] : null;
150
151         return SingleChildScrollView(
152           padding: const EdgeInsets.all(20),
153           child: Column(
154             crossAxisAlignment: CrossAxisAlignment.start,
155             children: [
156               _SectionCard(
157                 title: appTranslation().get('pay_plan_label'),
158                 child: Column(
159                   children: List.generate(

```

```

160         plans.length,
161         (i) => Padding(
162             padding: EdgeInsets.only(bottom: i < plans.length - 1 ? 10 : 0),
163             child: PlanCardWidget(
164                 plan: plans[i],
165                 isSelected: cubit.selectedPlanIndex == i,
166                 onTap: () => cubit.selectPlan(i),
167             ),
168         ),
169     ),
170 ),
171 ),
172 verticalSpace16,
173 const PromoCodeSectionWidget(),
174 verticalSpace16,
175 _SectionCard(
176     title: appTranslation().get('pay_method_label'),
177     child: Column(
178         children: [
179             _PaymentMethodTile(
180                 icon: Icons.credit_card_outlined,
181                 title: appTranslation().get('pay_method_online'),
182                 subtitle: 'MyFatoorah - ${appTranslation().get('pay_method_online_subtitle')}',
183                 isSelected: cubit.selectedMethodIndex == 0,
184                 onTap: () => cubit.selectMethod(0),
185             ),
186             // verticalSpace10,
187             // _PaymentMethodTile(
188             //     icon: Icons.account_balance_outlined,
189             //     title: appTranslation().get('pay_method_bank'),
190             //     subtitle: appTranslation().get('pay_method_bank_subtitle'),
191             //     isSelected: cubit.selectedMethodIndex == 1,
192             //     onTap: () => cubit.selectMethod(1),
193             // ),
194         ],
195     ),
196 ),
197 if (selectedPlan != null) ...[
198     verticalSpace16,
199     _SectionCard(
200         title: appTranslation().get('pay_order_summary'),
201         child: Column(
202             crossAxisAlignment: CrossAxisAlignment.start,
203             children: [
204                 Row(
205                     mainAxisAlignment: MainAxisAlignment.spaceBetween,
206                     children: [
207                         Container(
208                             padding: const EdgeInsets.symmetric(
209                                 horizontal: 10, vertical: 5),
210                             decoration: BoxDecoration(
211                                 color: const Color(0xFFFF5F0E0),
212                                 borderRadius: BorderRadius.circular(8),
213                             ),
214                             child: Text(
215                                 '${selectedPlan.price} ₪.ر',
216                                 style: TextStylesManager.bold14.copyWith(
217                                     color: const Color(0xFF8B6914)),
218                             ),
219                         ),
220                         Text(
221                             selectedPlan.name,
222                             style: TextStylesManager.bold16.copyWith(
223                                 color: ColorsManager.textPrimary),
224                         ),
225                     ],
226                 ),
227                 verticalSpace10,
228                 Row(
229                     mainAxisAlignment: MainAxisAlignment.start,
230                     children: [
231                         Icon(Icons.verified_outlined,
232                             size: 15, color: ColorsManager.primaryColor),
233                         horizontalSpace6,
234                         Flexible(

```

```

235         child: Text(
236             appTranslation().get('pay_server_amount_note'),
237             style: TextStyleManager.regular12.copyWith(
238                 color: ColorsManager.secondaryText),
239             textAlign: TextAlign.start,
240         ),
241     ),
242 ],
243 ),
244 ],
245 ),
246 ),
247 ],
248 verticalSpace24,
249 PrimaryElevatedButton(
250     text: cubit.selectedMethodIndex == 0
251         ? appTranslation().get('pay_proceed_online')
252         : appTranslation().get('pay_proceed_bank'),
253     icon: Icon(
254         cubit.selectedMethodIndex == 0
255             ? Icons.credit_card_outlined
256             : Icons.account_balance_outlined,
257         size: 20,
258         color: ColorsManager.white,
259     ),
260     isLoading: isSubmitting,
261     onPressed: isSubmitting ? null : () => cubit.createOrder(),
262 ),
263 verticalSpace16,
264 ],
265 ),
266 );
267 },
268 );
269 }
270 }
271
272 class _SectionCard extends StatelessWidget {
273     final String title;
274     final Widget child;
275
276     const _SectionCard({required this.title, required this.child});
277
278     @override
279     Widget build(BuildContext context) {
280         return Container(
281             width: double.infinity,
282             padding: const EdgeInsets.all(16),
283             decoration: BoxDecoration(
284                 color: ColorsManager.surfacePrimary,
285                 borderRadius: BorderRadius.circular(16),
286                 border: Border.all(color: ColorsManager.borderColor),
287             ),
288             child: Column(
289                 crossAxisAlignment: CrossAxisAlignment.start,
290                 children: [
291                     Text(
292                         title,
293                         style: TextStyleManager.bold16.copyWith(
294                             color: ColorsManager.textPrimary),
295                     ),
296                     verticalSpace12,
297                     child,
298                 ],
299             ),
300         );
301     }
302 }
303
304 class _PaymentMethodTile extends StatelessWidget {
305     final IconData icon;
306     final String title;
307     final String subtitle;
308     final bool isSelected;
309     final VoidCallback onTap;

```

```

310
311 const _PaymentMethodTile({
312   required this.icon,
313   required this.title,
314   required this.subtitle,
315   required this.isSelected,
316   required this.onTap,
317 });
318
319 @override
320 Widget build(BuildContext context) {
321   return GestureDetector(
322     onTap: onTap,
323     child: AnimatedContainer(
324       duration: const Duration(milliseconds: 180),
325       padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
326       decoration: BoxDecoration(
327         color: isSelected
328           ? ColorsManager.primaryColor.withValues(alpha: 0.08)
329           : Colors.transparent,
330         borderRadius: BorderRadius.circular(12),
331         border: Border.all(
332           color: isSelected
333             ? ColorsManager.primaryColor
334             : ColorsManager.borderColor,
335           width: isSelected ? 1.5 : 1,
336         ),
337       ),
338       child: Row(
339         children: [
340           AnimatedContainer(
341             duration: const Duration(milliseconds: 180),
342             width: 20,
343             height: 20,
344             decoration: BoxDecoration(
345               shape: BoxShape.circle,
346               border: Border.all(
347                 color: isSelected
348                   ? ColorsManager.primaryColor
349                   : ColorsManager.borderColor,
350                 width: 2,
351               ),
352             ),
353             child: isSelected
354               ? Center(
355                 child: Container(
356                   width: 10,
357                   height: 10,
358                   decoration: BoxDecoration(
359                     shape: BoxShape.circle,
360                     color: ColorsManager.primaryColor,
361                   ),
362                 ),
363               )
364               : null,
365           ),
366           horizontalSpace12,
367           Expanded(
368             child: Column(
369               crossAxisAlignment: CrossAxisAlignment.start,
370               children: [
371                 Text(title,
372                   style: TextStyleManager.medium14
373                     .copyWith(color: ColorsManager.textPrimary)),
374                 verticalSpace2,
375                 Text(subtitle,
376                   style: TextStyleManager.regular12
377                     .copyWith(color: ColorsManager.secondaryText)),
378               ],
379             ),
380           ),
381           horizontalSpace10,
382           Icon(icon,
383             size: 20,
384             color: isSelected

```

```

385         ? ColorsManager.primaryColor
386         : ColorsManager.secondaryText),
387     ],
388   ),
389 ),
390 );
391 }
392 }
393

```

Lines • Dart 255

Feature / Payment

#254 – lib/features/payment/presentation/screen/myfatoorah_payment_screen.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
6  import 'package:moean/core/utils/constants/primary/conditional_builder.dart';
7  import 'package:moean/core/utils/constants/routes.dart';
8  import 'package:moean/core/utils/constants/spacing.dart';
9  import 'package:moean/features/payment/presentation/cubit/payment_cubit.dart';
10 import 'package:moean/features/payment/presentation/cubit/payment_state.dart';
11 import 'package:moean/features/payment/presentation/widgets/visa_card_display.dart';
12 import 'package:moean/core/utils/constants/secrets.dart';
13 import 'package:myfatoorah_flutter/myfatoorah_flutter.dart';
14
15 class MyfatoorahPaymentScreen extends StatefulWidget {
16   const MyfatoorahPaymentScreen({super.key});
17
18   @override
19   State<MyfatoorahPaymentScreen> createState() => _MyfatoorahPaymentScreenState();
20 }
21
22 class _MyfatoorahPaymentScreenState extends State<MyfatoorahPaymentScreen> {
23   late String _amount;
24   late int _orderId;
25   String? _sessionId;
26
27   late MFCardPaymentView mfCardView;
28   bool _isProcessing = false;
29
30   @override
31   void initState() {
32     super.initState();
33     MFSDK.init(Secrets.myfatoorahApiKey, MFCountry.SAUDIARABIA, MFEnvironment.LIVE);
34
35     // المدمجة بأفضل شكل ممكن MyFatoorah تصميم حقول
36     MFCardViewStyle cardViewStyle = MFCardViewStyle();
37     cardViewStyle.cardHeight = 300;
38     cardViewStyle.hideCardIcons = false;
39
40     // ستايل الحقول
41     cardViewStyle.input?.inputMargin = 12;
42
43     // ستايل العناوين
44     cardViewStyle.label?.display = true;
45     cardViewStyle.label?.fontWeight = MFFontWeight.Bold;
46
47     mfCardView = MFCardPaymentView(cardViewStyle: cardViewStyle);
48   }
49
50   bool _isInitialized = false;
51
52   @override
53   void didChangeDependencies() {
54     super.didChangeDependencies();
55     if (!_isInitialized) {
56       _isInitialized = true;
57       final args = ModalRoute.of(context)?.settings.arguments as Map<String, dynamic>;
58       _amount = args?['amount'] as String? ?? '0';
59       _orderId = args?['orderId'] as int? ?? 0;
60
61       PaymentCubit.get(context).initMyfatoorahPayment(_orderId);

```



```

62     }
63 }
64
65 void _setupMyfatoorahSDK(String sessionId, String portalHost) {
66     _sessionId = sessionId;
67     final initiateSessionResponse = MFInitiateSessionResponse(
68         sessionId: sessionId,
69         countryCode: MFCountry.SAUDIARABIA,
70     );
71
72     Future.delayed(const Duration(milliseconds: 500), () {
73         if (mounted) {
74             mfCardView.load(initiateSessionResponse, null);
75         }
76     });
77 }
78
79 void _executePayment() {
80     if (_sessionId != null) {
81         setState(() { _isProcessing = true; });
82         final invoiceAmount = double.tryParse(_amount) ?? 0.0;
83         final executeRequest = MFExecutePaymentRequest(
84             invoiceValue: invoiceAmount,
85             sessionId: _sessionId,
86         );
87
88         mfCardView.pay(executeRequest, MFLanguage.ARABIC, (String invoiceId) {
89             // Callback
90         }).then((response) {
91             if (!mounted) return;
92             setState(() { _isProcessing = false; });
93             if (response.invoiceStatus == "Paid") {
94                 final paymentKey = (response.invoiceTransactions != null && response.invoiceTransactions!.isNotEmpty)
95                     ? (response.invoiceTransactions!.first.paymentId ?? response.invoiceId?.toString()) ?? ""
96                     : (response.invoiceId?.toString()) ?? "";
97                 PaymentCubit.get(context).verifyMyfatoorahPayment(paymentKey);
98             } else {
99                 ScaffoldMessenger.of(context).showSnackBar(
100                     const SnackBar(content: Text("فشلت عملية الدفع")),
101                 );
102             }
103         }).catchError((error) {
104             if (!mounted) return;
105             setState(() { _isProcessing = false; });
106             String errorMsg = error.toString();
107             if (error is MFError) {
108                 errorMsg = error.message ?? errorMsg;
109             }
110             ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(errorMsg)));
111         });
112     }
113 }
114
115 @override
116 Widget build(BuildContext context) {
117     return BlocConsumer<PaymentCubit, PaymentState>(
118         listenWhen: (_, s) =>
119             s is MyfatoorahSessionLoaded ||
120             s is MyfatoorahSessionError ||
121             s is MyfatoorahExecuteLoaded ||
122             s is MyfatoorahExecuteError ||
123             s is PaymentVerified ||
124             s is PaymentVerifyError,
125         listener: (context, state) {
126             if (state is MyfatoorahSessionError) {
127                 ScaffoldMessenger.of(context).showSnackBar(
128                     SnackBar(content: Text(state.error), backgroundColor: ColorsManager.errorColor),
129                 );
130             } else if (state is PaymentVerifyError) {
131                 ScaffoldMessenger.of(context).showSnackBar(
132                     SnackBar(content: Text(state.error), backgroundColor: ColorsManager.errorColor),
133                 );
134             } else if (state is MyfatoorahSessionLoaded) {
135                 final session = state.session['session'];

```

```

136         if (session != null) {
137             setState(() {
138                 if (session['amount'] != null) {
139                     _amount = session['amount'].toString();
140                 }
141                 if (session['order_id'] != null) {
142                     _orderId = session['order_id'] is int
143                         ? session['order_id'] as int
144                         : (int.tryParse(session['order_id'].toString()) ?? _orderId);
145                 }
146                 _setupMyfatoorahSDK(session['session_id'], session['portal_host']);
147             });
148         }
149     } else if (state is PaymentVerified) {
150         Navigator.pushReplacementNamed(
151             context,
152             Routes.paymentResult,
153             arguments: {'status': state.payment.status, 'from': 'myfatoorah'},
154         );
155     }
156 },
157 builder: (context, state) {
158     final isLoadingSession = state is MyfatoorahSessionLoading;
159     final isExecuting = state is MyfatoorahExecuteLoading || state is PaymentVerifying;
160     final isError = state is MyfatoorahSessionError;
161
162     return Scaffold(
163         backgroundColor: ColorsManager.background,
164         appBar: AppBar(
165             backgroundColor: ColorsManager.surfacePrimary,
166             elevation: 0,
167             title: Text(
168                 appTranslation().get('electronic_payment'),
169                 style: TextStyle(color: ColorsManager.textPrimary, fontSize: 18, fontWeight: FontWeight.bold),
170             ),
171             centerTitle: true,
172             leading: IconButton(
173                 icon: Icon(Icons.arrow_back_ios_new_rounded, color: ColorsManager.textPrimary, size: 20),
174                 onPressed: () => Navigator.pop(context),
175             ),
176         ),
177         body: SafeArea(
178             child: SingleChildScrollView(
179                 padding: const EdgeInsets.all(20),
180                 child: Column(
181                     crossAxisAlignment: CrossAxisAlignment.stretch,
182                     children: [
183                         const VisaCardDisplay(
184                             cardNumber: '',
185                             cardHolder: '',
186                             expiryDate: '',
187                         ),
188                         verticalSpace24,
189                         ConditionalBuilder(
190                             loadingState: isLoadingSession,
191                             errorState: isError,
192                             errorBuilder: (context) {
193                                 final errorMsg = state is MyfatoorahSessionError ? state.error : '';
194                                 return Column(
195                                     children: [
196                                         Text('${appTranslation().get('session_error')} $errorMsg', style: const TextStyle(co
lor: Colors.red)),
197                                         verticalSpace16,
198                                         PrimaryElevatedButton(
199                                             text: appTranslation().get('retry'),
200                                             onPressed: () => PaymentCubit.get(context).initMyfatoorahPayment(_orderId),
201                                         ),
202                                     ],
203                                 );
204                             },
205                             successBuilder: (context) {
206                                 if (_sessionId == null) return const SizedBox.shrink();
207                                 return Column(
208                                     crossAxisAlignment: CrossAxisAlignment.stretch,
209                                     children: [

```

```

210         Text(
211             appTranslation().get('card_details'),
212             style: const TextStyle(fontWeight: FontWeight.bold, fontSize: 16),
213         ),
214         verticalSpace16,
215
216         Container(
217             decoration: BoxDecoration(
218                 color: Colors.white,
219                 borderRadius: BorderRadius.circular(16),
220                 boxShadow: [
221                     BoxShadow(
222                         color: Colors.black.withValues(alpha: 0.03),
223                         blurRadius: 10,
224                         offset: const Offset(0, 4),
225                     ),
226                 ],
227             ),
228             padding: const EdgeInsets.all(16),
229             child: SizedBox(
230                 height: 300,
231                 child: mfCardView,
232             ),
233         ),
234
235         verticalSpace32,
236         PrimaryElevatedButton(
237             text: '${appTranslation().get('pay_btn_prefix')} $_amount ${appTranslation().get
('currency_sar')}',
238             isLoading: _isProcessing || isExecuting,
239             onPressed: _executePayment,
240             backgroundColor: const Color(0xFF1B49A8),
241         ),
242     ],
243 );
244 }
245 ),
246 ],
247 ),
248 ),
249 ),
250 );
251 },
252 );
253 }
254 }
255

```

Lines • Dart 102

Feature / Payment

#255 – lib/features/payment/presentation/screen/payment_history_screen.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/primary/conditional_builder.dart';
7  import 'package:moean/core/utils/constants/spacing.dart';
8  import 'package:moean/features/payment/presentation/cubit/payment_cubit.dart';
9  import 'package:moean/features/payment/presentation/cubit/payment_state.dart';
10 import 'package:moean/features/payment/presentation/widgets/payment_history_item_widget.dart';
11
12 class PaymentHistoryScreen extends StatefulWidget {
13     const PaymentHistoryScreen({super.key});
14
15     @override
16     State<PaymentHistoryScreen> createState() => _PaymentHistoryScreenState();
17 }
18
19 class _PaymentHistoryScreenState extends State<PaymentHistoryScreen> {
20     @override
21     void initState() {
22         super.initState();
23         PaymentCubit.get(context).loadHistory();

```

```

24 }
25
26 @override
27 Widget build(BuildContext context) {
28   return Scaffold(
29     backgroundColor: ColorsManager.background,
30     appBar: AppBar(
31       backgroundColor: ColorsManager.surfacePrimary,
32       elevation: 0,
33       centerTitle: true,
34       title: Text(
35         appTranslation().get('pay_history_title'),
36         style: TextStylesManager.bold16.copyWith(
37           color: ColorsManager.textPrimary,
38         ),
39     ),
40     leading: IconButton(
41       icon: Icon(Icons.arrow_back_ios_new_rounded,
42         color: ColorsManager.textPrimary, size: 20),
43       onPressed: () => Navigator.pop(context),
44     ),
45     bottom: PreferredSize(
46       preferredSize: const Size.fromHeight(1),
47       child: Divider(height: 1, color: ColorsManager.borderColor),
48     ),
49   ),
50   body: BlocBuilder<PaymentCubit, PaymentState>(
51     buildWhen: (_, s) =>
52       s is HistoryLoading || s is HistoryLoaded || s is HistoryError,
53     builder: (context, state) {
54       final cubit = PaymentCubit.get(context);
55       return ConditionalBuilder(
56         loadingState: state is HistoryLoading,
57         errorState: state is HistoryError,
58         emptyState:
59           state is HistoryLoaded && cubit.paymentHistory.isEmpty,
60         errorBuilder: (_) => Center(
61           child: Text(
62             (state as HistoryError).error,
63             style: TextStylesManager.regular14
64               .copyWith(color: ColorsManager.errorColor),
65             textAlign: TextAlign.center,
66           ),
67         ),
68         emptyBuilder: (_) => Center(
69           child: Column(
70             mainAxisAlignment: MainAxisAlignment.min,
71             children: [
72               Icon(Icons.receipt_long_outlined,
73                 size: 60, color: ColorsManager.secondaryText),
74               verticalSpace16,
75               Text(
76                 appTranslation().get('pay_no_history'),
77                 style: TextStylesManager.regular14.copyWith(
78                   color: ColorsManager.secondaryText,
79                 ),
80             ),
81           ],
82         ),
83       ),
84     successBuilder: (_) => RefreshIndicator(
85       color: ColorsManager.primaryColor,
86       onRefresh: () => cubit.loadHistory(),
87       child: ListView.separated(
88         padding: const EdgeInsets.all(20),
89         itemCount: cubit.paymentHistory.length,
90         separatorBuilder: (_, __) => verticalSpace12,
91         itemBuilder: (_, i) => PaymentHistoryItemWidget(
92           payment: cubit.paymentHistory[i],
93         ),
94       ),
95     ),
96   );
97 },
98 ),

```

```
99     );
100   }
101 }
102
```

Lines • Dart 136

Feature / Payment

#256 – lib/features/payment/presentation/screen/payment_result_screen.dart

```
1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
6  import 'package:moean/core/utils/constants/routes.dart';
7  import 'package:moean/core/utils/constants/spacing.dart';
8
9  class PaymentResultScreen extends StatelessWidget {
10    const PaymentResultScreen({super.key});
11
12    @override
13    Widget build(BuildContext context) {
14      final args =
15        ModalRoute.of(context)?.settings.arguments as Map<String, dynamic>;
16      final status = args?['status'] as String? ?? 'waiting_verification';
17      final from = args?['from'] as String? ?? 'bank';
18
19      final info = _resultInfo(status, from);
20
21      return Scaffold(
22        backgroundColor: ColorsManager.background,
23        body: SafeArea(
24          child: Padding(
25            padding: const EdgeInsets.all(24),
26            child: Column(
27              mainAxisAlignment: MainAxisAlignment.center,
28              children: [
29                Container(
30                  width: 100,
31                  height: 100,
32                  decoration: BoxDecoration(
33                    color: info.iconBgColor,
34                    shape: BoxShape.circle,
35                  ),
36                  child: Icon(
37                    info.icon,
38                    size: 50,
39                    color: info.iconColor,
40                  ),
41                ),
42                verticalSpace24,
43                Text(
44                  info.title,
45                  style: TextStylesManager.bold22.copyWith(
46                    color: ColorsManager.textPrimary,
47                  ),
48                  textAlign: TextAlign.center,
49                ),
50                verticalSpace12,
51                Text(
52                  info.subtitle,
53                  style: TextStylesManager.regular14.copyWith(
54                    color: ColorsManager.secondaryText,
55                  ),
56                  textAlign: TextAlign.center,
57                ),
58                verticalSpace40,
59                PrimaryElevatedButton(
60                  text: appTranslation().get('pay_go_home'),
61                  icon: Icon(Icons.home_outlined, size: 20,
62                    color: ColorsManager.white),
63                  onPressed: () => Navigator.pushNamedAndRemoveUntil(
64                    context, Routes.home, (_) => false),
65                ),
66                verticalSpace12,
```

```

67         PrimaryElevatedButton(
68             text: appTranslation().get('pay_view_history'),
69             icon: Icon(Icons.history_outlined, size: 20,
70                 color: ColorsManager.primaryColor),
71             backgroundColor: Colors.transparent,
72             borderSide: BorderSide(color: ColorsManager.primaryColor),
73             textStyle: TextStylesManager.bold14.copyWith(
74                 color: ColorsManager.primaryColor,
75             ),
76             onPressed: () => Navigator.pushReplacementNamed(
77                 context, Routes.paymentHistory),
78         ),
79     ],
80 ),
81 ),
82 ),
83 );
84 }
85
86 _ResultInfo _resultInfo(String status, String from) {
87     switch (status) {
88         case 'paid':
89             return _ResultInfo(
90                 icon: Icons.check_circle_rounded,
91                 iconColor: ColorsManager.successColor,
92                 iconBgColor: const Color(0xFFDCFCF7),
93                 title: appTranslation().get('pay_result_paid_title'),
94                 subtitle: appTranslation().get('pay_result_paid_subtitle'),
95             );
96         case 'failed':
97         case 'rejected':
98             return _ResultInfo(
99                 icon: Icons.cancel_rounded,
100                 iconColor: ColorsManager.errorColor,
101                 iconBgColor: const Color(0xFFFE2E2E),
102                 title: appTranslation().get('pay_result_failed_title'),
103                 subtitle: appTranslation().get('pay_result_failed_subtitle'),
104             );
105         default:
106             return _ResultInfo(
107                 icon: Icons.hourglass_top_rounded,
108                 iconColor: const Color(0xFF854D0E),
109                 iconBgColor: const Color(0xFFFEF9C3),
110                 title: from == 'bank'
111                     ? appTranslation().get('pay_result_pending_bank_title')
112                     : appTranslation().get('pay_result_pending_title'),
113                 subtitle: from == 'bank'
114                     ? appTranslation().get('pay_result_pending_bank_subtitle')
115                     : appTranslation().get('pay_result_pending_subtitle'),
116             );
117     }
118 }
119 }
120
121 class _ResultInfo {
122     final IconData icon;
123     final Color iconColor;
124     final Color iconBgColor;
125     final String title;
126     final String subtitle;
127
128     const _ResultInfo({
129         required this.icon,
130         required this.iconColor,
131         required this.iconBgColor,
132         required this.title,
133         required this.subtitle,
134     });
135 }
136

```

```
1 import 'package:flutter/material.dart';
2 import 'package:flutter/services.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/spacing.dart';
7
8 class BankInfoWidget extends StatelessWidget {
9   final Map<String, dynamic> bankInfo;
10
11   const BankInfoWidget({super.key, required this.bankInfo});
12
13   @override
14   Widget build(BuildContext context) {
15     final bankName = bankInfo['bank_name'] as String? ?? '';
16     final iban = bankInfo['iban'] as String? ?? '';
17     final holder = bankInfo['account_holder'] as String? ?? '';
18     final instructions = bankInfo['instructions'] as String? ?? '';
19
20     return Container(
21       padding: const EdgeInsets.all(16),
22       decoration: BoxDecoration(
23         color: ColorsManager.surfacePrimary,
24         borderRadius: BorderRadius.circular(16),
25         border: Border.all(color: ColorsManager.borderColor),
26       ),
27       child: Column(
28         crossAxisAlignment: CrossAxisAlignment.end,
29         children: [
30           Row(
31             mainAxisAlignment: MainAxisAlignment.end,
32             children: [
33               Text(
34                 appTranslation().get('pay_bank_details'),
35                 style: TextStylesManager.bold16.copyWith(
36                   color: ColorsManager.textPrimary,
37                 ),
38             ),
39             horizontalSpace8,
40             Icon(
41               Icons.account_balance_outlined,
42               color: ColorsManager.secondaryColor,
43               size: 20,
44             ),
45           ],
46         ),
47         verticalSpace4,
48         Text(
49           appTranslation().get('pay_upload_after_transfer'),
50           style: TextStylesManager.regular12.copyWith(
51             color: ColorsManager.secondaryText,
52           ),
53         ),
54         verticalSpace16,
55         _BankInfoRow(
56           label: appTranslation().get('pay_bank_name'),
57           value: bankName,
58         ),
59         verticalSpace10,
60         _BankInfoRow(
61           label: appTranslation().get('pay_account_holder'),
62           value: holder,
63         ),
64         verticalSpace10,
65         _IbanRow(iban: iban),
66         if (instructions.isNotEmpty) ...[
67           verticalSpace12,
68           Text(
69             instructions,
70             style: TextStylesManager.regular13.copyWith(
71               color: ColorsManager.secondaryText,
72             ),
73         ],
```

```

73         textAlign: TextAlign.end,
74     ),
75 ],
76 ],
77 ),
78 );
79 }
80 }
81
82 class _BankInfoRow extends StatelessWidget {
83     final String label;
84     final String value;
85
86     const _BankInfoRow({required this.label, required this.value});
87
88     @override
89     Widget build(BuildContext context) {
90         return Row(
91             mainAxisAlignment: MainAxisAlignment.spaceBetween,
92             children: [
93                 Text(
94                     value,
95                     style: TextStylesManager.bold14.copyWith(
96                         color: ColorsManager.textPrimary,
97                     ),
98                 ),
99                 Text(
100                     label,
101                     style: TextStylesManager.regular13.copyWith(
102                         color: ColorsManager.secondaryText,
103                     ),
104                 ),
105             ],
106         );
107     }
108 }
109
110 class _IbanRow extends StatelessWidget {
111     final String iban;
112
113     const _IbanRow({required this.iban});
114
115     @override
116     Widget build(BuildContext context) {
117         return Row(
118             mainAxisAlignment: MainAxisAlignment.spaceBetween,
119             children: [
120                 GestureDetector(
121                     onTap: () {
122                         Clipboard.setData(ClipboardData(text: iban));
123                         ScaffoldMessenger.of(context).showSnackBar(
124                             SnackBar(
125                                 content: Text(
126                                     appTranslation().get('pay_iban_copied'),
127                                     textAlign: TextAlign.center,
128                                 ),
129                                 duration: const Duration(seconds: 2),
130                                 backgroundColor: ColorsManager.primaryColor,
131                                 behavior: SnackBarBehavior.floating,
132                                 shape: RoundedRectangleBorder(
133                                     borderRadius: BorderRadius.circular(12),
134                                 ),
135                             ),
136                 );
137             ],
138             child: Row(
139                 children: [
140                     Icon(
141                         Icons.copy_rounded,
142                         size: 16,
143                         color: ColorsManager.primaryColor,
144                     ),
145                     horizontalSpace4,
146                     Text(
147                         iban,

```



```

148         style: TextStyleManager.bold13.copyWith(
149             color: ColorsManager.primaryColor,
150             letterSpacing: 1,
151         ),
152     ),
153 ],
154 ),
155 ),
156 Text(
157     'IBAN',
158     style: TextStyleManager.regular13.copyWith(
159         color: ColorsManager.secondaryText,
160     ),
161 ),
162 ],
163 );
164 }
165 }
166

```

```
Payment lib/features/payment/presentation/widgets/moyasar_card_details_widget.dart
```

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/spacing.dart';
6
7 class MoyasarCardDetailsWidget extends StatelessWidget {
8   final TextEditingController nameController;
9   final TextEditingController numberController;
10  final TextEditingController expiryController;
11  final TextEditingController cvcController;
12
13  const MoyasarCardDetailsWidget({
14    super.key,
15    required this.nameController,
16    required this.numberController,
17    required this.expiryController,
18    required this.cvcController,
19  });
20
21  @override
22  Widget build(BuildContext context) {
23    return Column(
24      crossAxisAlignment: CrossAxisAlignment.start,
25      children: [
26        Text(
27          'اسم حامل البطاقة',
28          style: TextStyleManager.bold14.copyWith(
29            color: ColorsManager.primaryColor,
30          ),
31        ),
32        verticalSpace8,
33        Container(
34          decoration: BoxDecoration(
35            color: ColorsManager.surfacePrimary,
36            borderRadius: BorderRadius.circular(12),
37            border: Border.all(color: ColorsManager.borderColor),
38          ),
39          child: Padding(
40            padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),
41            child: TextFormField(
42              controller: nameController,
43              keyboardType: TextInputType.name,
44              style: TextStyleManager.medium16,
45              decoration: InputDecoration(
46                hintText: 'الاسم كما هو مطبوع على البطاقة',
47                hintStyle: TextStyleManager.regular16.copyWith(
48                  color: ColorsManager.textSecondary.withValues(alpha: 0.5),
49                ),
50                border: InputBorder.none,

```

```

51         enabledBorder: InputBorder.none,
52         focusedBorder: InputBorder.none,
53     ),
54 ),
55 ),
56 ),
57 verticalSpace16,
58 Text(
59   'رقم البطاقة',
60   style: TextStyleManager.bold14.copyWith(
61     color: ColorsManager.primaryColor,
62   ),
63 ),
64 verticalSpace8,
65 Container(
66   decoration: BoxDecoration(
67     color: ColorsManager.surfacePrimary,
68     borderRadius: BorderRadius.circular(12),
69     border: Border.all(color: ColorsManager.borderColor),
70   ),
71   child: Padding(
72     padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),
73     child: Row(
74       children: [
75         Expanded(
76           child: Directionality(
77             textDirection: TextDirection.ltr,
78             child: TextFormField(
79               controller: numberController,
80               keyboardType: TextInputType.number,
81               style: TextStyleManager.medium16,
82               decoration: InputDecoration(
83                 hintText: '1234 5678 9101 1121',
84                 hintStyle: TextStyleManager.regular16.copyWith(
85                   color: ColorsManager.textSecondary.withValues(alpha: 0.5),
86                 ),
87                 border: InputBorder.none,
88                 enabledBorder: InputBorder.none,
89                 focusedBorder: InputBorder.none,
90               ),
91             ),
92           ),
93         ),
94         horizontalSpace8,
95         // Payment Icons
96         Row(
97           children: [
98             _buildBrandIcon('VISA', Colors.blue),
99             horizontalSpace4,
100            _buildBrandIcon('MC', Colors.red),
101            horizontalSpace4,
102            _buildBrandIcon('mada', Colors.green),
103          ],
104        ),
105      ],
106    ),
107  ),
108 ),
109 verticalSpace16,
110 Row(
111   children: [
112     Expanded(
113       child: Text(
114         'تاريخ الانتهاء',
115         style: TextStyleManager.bold14.copyWith(
116           color: ColorsManager.primaryColor,
117         ),
118       ),
119     ),
120     horizontalSpace16,
121     Expanded(
122       child: Text(
123         'رمز الأمان',
124         style: TextStyleManager.bold14.copyWith(
125           color: ColorsManager.primaryColor,

```

```

126         ),
127     ),
128 ),
129 ],
130 ),
131 verticalSpace8,
132 Row(
133     children: [
134         Expanded(
135             child: Container(
136                 decoration: BoxDecoration(
137                     color: ColorsManager.surfacePrimary,
138                     borderRadius: BorderRadius.circular(12),
139                     border: Border.all(color: ColorsManager.borderColor),
140                 ),
141                 child: Padding(
142                     padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),
143                     child: Directionality(
144                         textDirection: TextDirection.ltr,
145                         child: TextFormField(
146                             controller: expiryController,
147                             keyboardType: TextInputType.datetime,
148                             style: TextStylesManager.medium16,
149                             decoration: InputDecoration(
150                                 hintText: 'MM/YY',
151                                 hintStyle: TextStylesManager.regular16.copyWith(
152                                     color: ColorsManager.textSecondary.withValues(alpha: 0.5),
153                                 ),
154                                 border: InputBorder.none,
155                                 enabledBorder: InputBorder.none,
156                                 focusedBorder: InputBorder.none,
157                             ),
158                         ),
159                     ),
160                 ),
161             ),
162         ),
163         horizontalSpace16,
164         Expanded(
165             child: Container(
166                 decoration: BoxDecoration(
167                     color: ColorsManager.surfacePrimary,
168                     borderRadius: BorderRadius.circular(12),
169                     border: Border.all(color: ColorsManager.borderColor),
170                 ),
171                 child: Padding(
172                     padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),
173                     child: Directionality(
174                         textDirection: TextDirection.ltr,
175                         child: TextFormField(
176                             controller: cvcController,
177                             keyboardType: TextInputType.number,
178                             style: TextStylesManager.medium16,
179                             decoration: InputDecoration(
180                                 hintText: 'CVC',
181                                 hintStyle: TextStylesManager.regular16.copyWith(
182                                     color: ColorsManager.textSecondary.withValues(alpha: 0.5),
183                                 ),
184                                 border: InputBorder.none,
185                                 enabledBorder: InputBorder.none,
186                                 focusedBorder: InputBorder.none,
187                             ),
188                         ),
189                     ),
190                 ),
191             ),
192         ),
193     ],
194 ),
195 ],
196 );
197 }
198
199 Widget _buildBrandIcon(String label, Color color) {
200     return Container(

```

```

201 padding: const EdgeInsets.symmetric(horizontal: 4, vertical: 2),
202 decoration: BoxDecoration(
203   border: Border.all(color: ColorsManager.borderColor),
204   borderRadius: BorderRadius.circular(4),
205 ),
206 child: Text(
207   label,
208   style: TextStyle(
209     fontSize: 8,
210     fontWeight: FontWeight.bold,
211     color: color,
212   ),
213 ),
214 );
215 }
216 }
217
218

```

Lines • Dart 51

Feature / Payment

#259 – lib/features/payment/presentation/widgets/moyasar_card_name_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7
8  class MoyasarCardNameWidget extends StatelessWidget {
9    final TextEditingController controller;
10    const MoyasarCardNameWidget({super.key, required this.controller});
11
12    @override
13    Widget build(BuildContext context) {
14      return Column(
15        crossAxisAlignment: CrossAxisAlignment.start,
16        children: [
17          Row(
18            mainAxisAlignment: MainAxisAlignment.spaceBetween,
19            children: [
20              Text(
21                appTranslation().get('pay_name_on_card'),
22                style: TextStylesManager.bold14.copyWith(
23                  color: ColorsManager.mainText,
24                ),
25              ),
26              Text(
27                appTranslation().get('field_required'),
28                style: TextStylesManager.medium12.copyWith(
29                  color: ColorsManager.errorColor,
30                ),
31              ),
32            ],
33          ),
34          verticalSpace8,
35          PrimaryTextField(
36            controller: controller,
37            hint: appTranslation().get('pay_name_on_card'),
38            keyboardType: TextInputType.name,
39            fillColor: ColorsManager.surfacePrimary,
40            validator: (value) {
41              if (value == null || value.isEmpty) {
42                return appTranslation().get('field_required');
43              }
44              return null;
45            },
46          ),
47        ],
48      );
49    }
50  }
51

```

```
1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/spacing.dart';
6
7 class MoyasarFooterWidget extends StatelessWidget {
8   const MoyasarFooterWidget({super.key});
9
10  @override
11  Widget build(BuildContext context) {
12    return Column(
13      children: [
14        Text(
15          appTranslation().get('pay_service_provided_by'),
16          style: TextStylesManager.medium12.copyWith(
17            color: ColorsManager.textSecondary,
18          ),
19        ),
20        verticalSpace8,
21        Text(
22          appTranslation().get('pay_test_mode_warning'),
23          textAlign: TextAlign.center,
24          style: TextStylesManager.medium12.copyWith(
25            color: ColorsManager.errorColor,
26          ),
27        ),
28      ],
29    );
30  }
31 }
32
```

```
1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/spacing.dart';
6
7 class MoyasarHeaderWidget extends StatelessWidget {
8   const MoyasarHeaderWidget({super.key});
9
10  @override
11  Widget build(BuildContext context) {
12    return Row(
13      children: [
14        Expanded(
15          child: Column(
16            crossAxisAlignment: CrossAxisAlignment.start,
17            children: [
18              Text(
19                appTranslation().get('pay_online_title'),
20                style: TextStylesManager.bold20.copyWith(
21                  color: ColorsManager.primaryColor,
22                ),
23              ),
24              verticalSpace4,
25              Text(
26                appTranslation().get('pay_online_subtitle'),
27                style: TextStylesManager.medium12.copyWith(
28                  color: ColorsManager.textSecondary,
29                ),
30            ],
31          ),
32        ),
33        Container(
34          padding: const EdgeInsets.all(8),
35          decoration: BoxDecoration(
36
```

```

37         color: ColorsManager.primaryColor.withValues(alpha: 0.1),
38         borderRadius: BorderRadius.circular(12),
39     ),
40     child: Icon(
41       Icons.credit_card_rounded,
42       color: ColorsManager.primaryColor,
43       size: 32,
44     ),
45   ),
46 ],
47 );
48 }
49 }
50

```

Lines • Dart 91

Feature / Payment

#262 – lib/features/payment/presentation/widgets/order_summary_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6
7  class OrderSummaryWidget extends StatelessWidget {
8    final String planName;
9    final String amount;
10
11    const OrderSummaryWidget({
12      super.key,
13      required this.planName,
14      required this.amount,
15    });
16
17    @override
18    Widget build(BuildContext context) {
19      return Container(
20        padding: const EdgeInsets.all(16),
21        decoration: BoxDecoration(
22          color: ColorsManager.surfacePrimary,
23          borderRadius: BorderRadius.circular(16),
24          border: Border.all(color: ColorsManager.borderColor),
25        ),
26        child: Column(
27          crossAxisAlignment: CrossAxisAlignment.end,
28          children: [
29            Text(
30              appTranslation().get('pay_order_summary'),
31              style: TextStylesManager.bold14.copyWith(
32                color: ColorsManager.textPrimary,
33              ),
34            ),
35            verticalSpace12,
36            Row(
37              mainAxisAlignment: MainAxisAlignment.spaceBetween,
38              children: [
39                Container(
40                  padding:
41                    const EdgeInsets.symmetric(horizontal: 10, vertical: 5),
42                  decoration: BoxDecoration(
43                    color: const Color(0xFFFF5F0E0),
44                    borderRadius: BorderRadius.circular(8),
45                  ),
46                  child: Text(
47                    '$amount ₪.ر',
48                    style: TextStylesManager.bold14.copyWith(
49                      color: const Color(0xFF8B6914),
50                    ),
51                  ),
52                ),
53                Expanded(
54                  child: Text(
55                    planName,
56                    style: TextStylesManager.bold16.copyWith(

```

```

57         color: ColorsManager.textPrimary,
58     ),
59     textAlign: TextAlign.end,
60     overflow: TextOverflow.ellipsis,
61 ),
62 ),
63 ],
64 ),
65 verticalSpace12,
66 Row(
67   mainAxisAlignment: MainAxisAlignment.end,
68   children: [
69     Icon(
70       Icons.verified_outlined,
71       size: 16,
72       color: ColorsManager.primaryColor,
73     ),
74     horizontalSpace6,
75     Flexible(
76       child: Text(
77         appTranslation().get('pay_server_amount_note'),
78         style: TextStylesManager.regular12.copyWith(
79           color: ColorsManager.secondaryText,
80         ),
81         textAlign: TextAlign.end,
82       ),
83     ),
84   ],
85 ),
86 ],
87 ),
88 );
89 }
90 }
91

```

Lines • 181
Dart

Feature / #263 -

Payment [lib/features/payment/presentation/widgets/payment_history_item_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6  import 'package:moean/features/payment/data/models/payment_model.dart';
7
8  class PaymentHistoryItemWidget extends StatelessWidget {
9    final PaymentModel payment;
10
11    const PaymentHistoryItemWidget({super.key, required this.payment});
12
13    @override
14    Widget build(BuildContext context) {
15      final statusInfo = _statusInfo(payment.status);
16      final planName = payment.order?.service?.name ?? '';
17      final date = _formatDate(payment.createdAt);
18
19      return Container(
20        padding: const EdgeInsets.all(16),
21        decoration: BoxDecoration(
22          color: ColorsManager.surfacePrimary,
23          borderRadius: BorderRadius.circular(16),
24          border: Border.all(color: ColorsManager.borderColor),
25        ),
26        child: Column(
27          crossAxisAlignment: CrossAxisAlignment.end,
28          children: [
29            Row(
30              mainAxisAlignment: MainAxisAlignment.spaceBetween,
31              children: [
32                Container(
33                  padding:
34                    const EdgeInsets.symmetric(horizontal: 10, vertical: 4),

```

```

35         decoration: BoxDecoration(
36             color: statusInfo.bgColor,
37             borderRadius: BorderRadius.circular(20),
38         ),
39         child: Text(
40             statusInfo.label,
41             style: TextStylesManager.bold12.copyWith(
42                 color: statusInfo.textColor,
43             ),
44         ),
45     ),
46     if (planName.isNotEmpty)
47     Flexible(
48         child: Text(
49             planName,
50             style: TextStylesManager.bold14.copyWith(
51                 color: ColorsManager.textPrimary,
52             ),
53             textAlign: TextAlign.end,
54             overflow: TextOverflow.ellipsis,
55         ),
56     ),
57 ],
58 ),
59 verticalSpace10,
60 Row(
61     mainAxisAlignment: MainAxisAlignment.spaceBetween,
62     children: [
63         Text(
64             date,
65             style: TextStylesManager.regular12.copyWith(
66                 color: ColorsManager.secondaryText,
67             ),
68         ),
69         Text(
70             '${payment.amount} ر.س',
71             style: TextStylesManager.bold16.copyWith(
72                 color: ColorsManager.primaryColor,
73             ),
74         ),
75     ],
76 ),
77 verticalSpace6,
78 Row(
79     mainAxisAlignment: MainAxisAlignment.end,
80     children: [
81         Text(
82             _methodLabel(payment.paymentMethod),
83             style: TextStylesManager.regular12.copyWith(
84                 color: ColorsManager.secondaryText,
85             ),
86         ),
87         horizontalSpace6,
88         Icon(
89             _methodIcon(payment.paymentMethod),
90             size: 15,
91             color: ColorsManager.secondaryText,
92         ),
93     ],
94 ),
95 ],
96 ),
97 );
98 }
99
100 _StatusInfo _statusInfo(String status) {
101     switch (status) {
102         case 'paid':
103             return _StatusInfo(
104                 label: appTranslation().get('pay_status_paid'),
105                 bgColor: const Color(0xFFDCFCF7),
106                 textColor: const Color(0xFF15803D),
107             );
108         case 'processing':
109             return _StatusInfo(

```



```

110         label: appTranslation().get('pay_status_processing'),
111         bgColor: const Color(0xFFE0F2FE),
112         textColor: const Color(0xFF0369A1),
113     );
114     case 'waiting_verification':
115         return _StatusInfo(
116             label: appTranslation().get('pay_status_waiting_verification'),
117             bgColor: const Color(0xFFFFEF9C3),
118             textColor: const Color(0xFF854D0E),
119         );
120     case 'failed':
121         return _StatusInfo(
122             label: appTranslation().get('pay_status_failed'),
123             bgColor: const Color(0xFFFFEE2E2),
124             textColor: const Color(0xFFB91C1C),
125         );
126     case 'rejected':
127         return _StatusInfo(
128             label: appTranslation().get('pay_status_rejected'),
129             bgColor: const Color(0xFFFFEE2E2),
130             textColor: const Color(0xFFB91C1C),
131         );
132     case 'cancelled':
133         return _StatusInfo(
134             label: appTranslation().get('pay_status_cancelled'),
135             bgColor: const Color(0xFFFF1F5F9),
136             textColor: const Color(0xFF64748B),
137         );
138     default:
139         return _StatusInfo(
140             label: appTranslation().get('pay_status_pending'),
141             bgColor: const Color(0xFFFFEF9C3),
142             textColor: const Color(0xFF854D0E),
143         );
144     }
145 }
146
147 String _methodLabel(String method) {
148     if (method == 'moyasar') {
149         return appTranslation().get('pay_method_online');
150     }
151     return appTranslation().get('pay_method_bank');
152 }
153
154 IconData _methodIcon(String method) {
155     return method == 'moyasar'
156         ? Icons.credit_card_outlined
157         : Icons.account_balance_outlined;
158 }
159
160 String _formatDate(String? dateStr) {
161     if (dateStr == null) return '';
162     try {
163         final dt = DateTime.parse(dateStr).toLocal();
164         return '${dt.year}-${dt.month.toString().padLeft(2, '0')}-${dt.day.toString().padLeft(2, '0')}';
165     } catch (_) {
166         return dateStr;
167     }
168 }
169 }
170
171 class _StatusInfo {
172     final String label;
173     final Color bgColor;
174     final Color textColor;
175     const _StatusInfo({
176         required this.label,
177         required this.bgColor,
178         required this.textColor,
179     });
180 }
181

```

```
1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/spacing.dart';
5
6 class PaymentMethodTileWidget extends StatelessWidget {
7   final IconData icon;
8   final String title;
9   final String subtitle;
10  final bool isSelected;
11  final VoidCallback onTap;
12
13  const PaymentMethodTileWidget({
14    super.key,
15    required this.icon,
16    required this.title,
17    required this.subtitle,
18    required this.isSelected,
19    required this.onTap,
20  });
21
22  @override
23  Widget build(BuildContext context) {
24    return GestureDetector(
25      onTap: onTap,
26      child: AnimatedContainer(
27        duration: const Duration(milliseconds: 200),
28        padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
29        decoration: BoxDecoration(
30          color: isSelected
31            ? ColorsManager.primaryColor.withValues(alpha: 0.08)
32            : ColorsManager.surfacePrimary,
33          borderRadius: BorderRadius.circular(12),
34          border: Border.all(
35            color: isSelected
36              ? ColorsManager.primaryColor
37              : ColorsManager.borderColor,
38            width: isSelected ? 1.5 : 1,
39          ),
40        ),
41      child: Row(
42        children: [
43          AnimatedContainer(
44            duration: const Duration(milliseconds: 200),
45            width: 20,
46            height: 20,
47            decoration: BoxDecoration(
48              shape: BoxShape.circle,
49              border: Border.all(
50                color: isSelected
51                  ? ColorsManager.primaryColor
52                  : ColorsManager.borderColor,
53                width: 2,
54              ),
55            ),
56          child: isSelected
57            ? Center(
58              child: Container(
59                width: 10,
60                height: 10,
61                decoration: BoxDecoration(
62                  shape: BoxShape.circle,
63                  color: ColorsManager.primaryColor,
64                ),
65              ),
66            )
67            : null,
68        ),
69        horizontalSpace12,
70        Expanded(
71          child: Column(
```



```

36     ),
37   ),
38   child: Row(
39     mainAxisAlignment: MainAxisAlignment.spaceBetween,
40     children: [
41       Text(
42         plan.name,
43         style: TextStylesManager.medium14.copyWith(
44           color: ColorsManager.textPrimary,
45         ),
46       ),
47       Row(
48         children: [
49           Text(
50             plan.price,
51             style: TextStylesManager.bold14.copyWith(
52               color: isSelected
53                 ? ColorsManager.primaryColor
54                 : ColorsManager.textPrimary,
55             ),
56           ),
57           horizontalSpace4,
58           Text(
59             'ر.س.',
60             style: TextStylesManager.regular12.copyWith(
61               color: ColorsManager.secondaryText,
62             ),
63           ),
64         ],
65       ),
66     ],
67   ),
68 );
69 }
70 }
71 }
72

```

Lines • Dart 240

Feature / Payment

#266 – lib/features/payment/presentation/widgets/promo_code_section_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7  import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
8  import 'package:moean/features/payment/presentation/cubit/payment_cubit.dart';
9  import 'package:moean/features/payment/presentation/cubit/payment_state.dart';
10
11  class PromoCodeSectionWidget extends StatefulWidget {
12    const PromoCodeSectionWidget({super.key});
13
14    @override
15    State<PromoCodeSectionWidget> createState() => _PromoCodeSectionWidgetState();
16  }
17
18  class _PromoCodeSectionWidgetState extends State<PromoCodeSectionWidget> {
19    late final TextEditingController _controller;
20
21    @override
22    void initState() {
23      super.initState();
24      _controller = TextEditingController();
25    }
26
27    @override
28    void dispose() {
29      _controller.dispose();
30      super.dispose();
31    }
32
33    void _applyPromo(BuildContext context) {

```

```

34   final cubit = PaymentCubit.get(context);
35   cubit.validatePromo(_controller.text);
36 }
37
38 void _clearPromo(BuildContext context) {
39   _controller.clear();
40   PaymentCubit.get(context).clearPromo();
41 }
42
43 @override
44 Widget build(BuildContext context) {
45   return BlocBuilder<PaymentCubit, PaymentState>(
46     buildWhen: (_, s) =>
47       s is PromoValidating ||
48       s is PromoValidated ||
49       s is PromoError ||
50       s is PromoCleared,
51     builder: (context, state) {
52       final cubit = PaymentCubit.get(context);
53       final isValidated = state is PromoValidated ||
54         (cubit.promoValidation != null && state is! PromoCleared);
55       final isLoading = state is PromoValidating;
56
57       return Container(
58         width: double.infinity,
59         padding: const EdgeInsets.all(16),
60         decoration: BoxDecoration(
61           color: ColorsManager.surfacePrimary,
62           borderRadius: BorderRadius.circular(16),
63           border: Border.all(color: ColorsManager.borderColor),
64         ),
65         child: Column(
66           crossAxisAlignment: CrossAxisAlignment.start,
67           children: [
68             Text(
69               appTranslation().get('promo_section_title'),
70               style: TextStyleManager.bold16
71                 .copyWith(color: ColorsManager.textPrimary),
72             ),
73             verticalSpace12,
74             // Input row
75             Row(
76               children: [
77                 // Apply / Remove button
78                 AnimatedSwitcher(
79                   duration: const Duration(milliseconds: 200),
80                   child: isValidated
81                     ? GestureDetector(
82                         key: const ValueKey('remove'),
83                         onTap: () => _clearPromo(context),
84                         child: Container(
85                           padding: const EdgeInsets.symmetric(
86                             horizontal: 16, vertical: 14),
87                           decoration: BoxDecoration(
88                             color: ColorsManager.errorColor
89                               .withValues(alpha: 0.1),
90                             borderRadius: BorderRadius.circular(24),
91                             border: Border.all(
92                               color: ColorsManager.errorColor
93                                 .withValues(alpha: 0.5)),
94                           ),
95                           child: Text(
96                             appTranslation().get('promo_remove'),
97                             style: TextStyleManager.medium14.copyWith(
98                               color: ColorsManager.errorColor),
99                           ),
100                         ),
101                     )
102                 : GestureDetector(
103                     key: const ValueKey('apply'),
104                     onTap: isLoading ? null : () => _applyPromo(context),
105                     child: Container(
106                       padding: const EdgeInsets.symmetric(
107                         horizontal: 16, vertical: 14),
108                       decoration: BoxDecoration(

```

```

109         color: ColorsManager.primaryColor,
110         borderRadius: BorderRadius.circular(24),
111     ),
112     child: isLoading
113       ? const SizedBox(
114         width: 18,
115         height: 18,
116         child: CircularProgressIndicator(
117           strokeWidth: 2,
118           color: Colors.white,
119         ),
120       )
121       : Text(
122         appTranslation().get('promo_apply'),
123         style: TextStylesManager.medium14
124           .copyWith(color: Colors.white),
125       ),
126   ),
127 ),
128 ),
129 horizontalSpace10,
130 // Text field
131 Expanded(
132   child: PrimaryTextField(
133     controller: _controller,
134     hint: appTranslation().get('promo_hint'),
135     enabled: !isValidated && !isLoading,
136     textInputAction: TextInputAction.done,
137     onFieldSubmitted: (_) => _applyPromo(context),
138   ),
139 ),
140 ],
141 ),
142 // Feedback row
143 if (state is PromoValidated) ...[
144   verticalSpace10,
145   _PromoSuccessBanner(data: state.data),
146 ] else if (state is PromoError) ...[
147   verticalSpace10,
148   _PromoErrorBanner(message: state.error),
149 ],
150 ],
151 ),
152 );
153 },
154 );
155 }
156 }
157
158 class _PromoSuccessBanner extends StatelessWidget {
159   final Map<String, dynamic> data;
160
161   const _PromoSuccessBanner({required this.data});
162
163   @override
164   Widget build(BuildContext context) {
165     final discount = data['discount'] as Map<String, dynamic>;
166     final discountType = discount?['type'] as String? ?? 'percentage';
167     final discountValue = (discount?['value'] as num?).toDouble() ?? 0.0;
168     final finalAmount = (data['final_amount'] as num?).toDouble();
169
170     final discountDisplay = discountType == 'percentage'
171       ? '${discountValue.toStringAsFixed(0)}%'
172       : '${discountValue.toStringAsFixed(2)} ر.س.';
173
174     return Container(
175       width: double.infinity,
176       padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 10),
177       decoration: BoxDecoration(
178         color: const Color(0xFF22C55E).withValues(alpha: 0.08),
179         borderRadius: BorderRadius.circular(12),
180         border: Border.all(
181           color: const Color(0xFF22C55E).withValues(alpha: 0.3)),
182       ),
183       child: Row(

```

```

184     children: [
185       if (finalAmount != null)
186         Text(
187           '$finalAmount ر.س',
188           style: TextStylesManager.bold14
189             .copyWith(color: const Color(0xFF16A34A)),
190         ),
191       const Spacer(),
192       Text(
193         '${appTranslation().get('promo_discount_label')}: $discountDisplay',
194         style: TextStylesManager.medium14
195           .copyWith(color: const Color(0xFF16A34A)),
196       ),
197       horizontalSpace8,
198       const Icon(Icons.check_circle_rounded,
199         color: Color(0xFF22C55E), size: 18),
200     ],
201   ),
202 );
203 }
204 }
205
206 class _PromoErrorBanner extends StatelessWidget {
207   final String message;
208
209   const _PromoErrorBanner({required this.message});
210
211   @override
212   Widget build(BuildContext context) {
213     return Container(
214       width: double.infinity,
215       padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 10),
216       decoration: BoxDecoration(
217         color: ColorsManager.errorColor.withValues(alpha: 0.08),
218         borderRadius: BorderRadius.circular(12),
219         border: Border.all(
220           color: ColorsManager.errorColor.withValues(alpha: 0.3)),
221       ),
222       child: Row(
223         children: [
224           Icon(Icons.error_outline_rounded,
225             color: ColorsManager.errorColor, size: 18),
226           horizontalSpace8,
227           Expanded(
228             child: Text(
229               message,
230               style: TextStylesManager.regular13
231                 .copyWith(color: ColorsManager.errorColor),
232               textAlign: TextAlign.start,
233             ),
234           ),
235         ],
236       ),
237     );
238   }
239 }
240

```

Lines • Dart 108

Feature / Payment

#267 – lib/features/payment/presentation/widgets/receipt_upload_widget.dart

```

1
2 import 'package:file_picker/file_picker.dart';
3 import 'package:flutter/material.dart';
4 import 'package:moean/core/theme/colors.dart';
5 import 'package:moean/core/theme/text_styles.dart';
6 import 'package:moean/core/utils/constants/constants.dart';
7 import 'package:moean/core/utils/constants/spacing.dart';
8
9 class ReceiptUploadWidget extends StatefulWidget {
10   final void Function(String filePath, String fileName) onFileSelected;
11
12   const ReceiptUploadWidget({super.key, required this.onFileSelected});
13

```

```

14   @override
15   State<ReceiptUploadWidget> createState() => _ReceiptUploadWidgetState();
16 }
17
18 class _ReceiptUploadWidgetState extends State<ReceiptUploadWidget> {
19   String? _selectedFileName;
20
21   Future<void> _pickFile() async {
22     final result = await FilePicker.platform.pickFiles(
23       type: FileType.custom,
24       allowedExtensions: ['jpg', 'jpeg', 'png', 'pdf'],
25       allowMultiple: false,
26     );
27     if (result == null || result.files.isEmpty) return;
28     final file = result.files.single;
29     if (file.path == null) return;
30     final sizeInMb = (file.size / (1024 * 1024));
31     if (sizeInMb > 5) {
32       if (mounted) {
33         ScaffoldMessenger.of(context).showSnackBar(
34           SnackBar(
35             content: Text(
36               appTranslation().get('pay_file_too_large'),
37               textAlign: TextAlign.center,
38             ),
39             backgroundColor: ColorsManager.errorColor,
40             behavior: SnackBarBehavior.floating,
41             shape: RoundedRectangleBorder(
42               borderRadius: BorderRadius.circular(12),
43             ),
44           ),
45         );
46       }
47       return;
48     }
49     setState(() => _selectedFileName = file.name);
50     widget.onFileSelected(file.path!, file.name);
51   }
52
53   @override
54   Widget build(BuildContext context) {
55     return GestureDetector(
56       onTap: _pickFile,
57       child: Container(
58         width: double.infinity,
59         padding: const EdgeInsets.symmetric(vertical: 28, horizontal: 16),
60         decoration: BoxDecoration(
61           color: ColorsManager.surfacePrimary,
62           borderRadius: BorderRadius.circular(12),
63           border: Border.all(
64             color: _selectedFileName != null
65               ? ColorsManager.primaryColor
66               : ColorsManager.borderColor,
67             style: BorderStyle.solid,
68           ),
69         ),
70       child: Column(
71         mainAxisAlignment: MainAxisAlignment.min,
72         children: [
73           Icon(
74             _selectedFileName != null
75               ? Icons.check_circle_outline
76               : Icons.upload_file_outlined,
77             size: 36,
78             color: _selectedFileName != null
79               ? ColorsManager.primaryColor
80               : ColorsManager.secondaryText,
81           ),
82           verticalSpace10,
83           Text(
84             _selectedFileName ??
85               appTranslation().get('pay_choose_receipt'),
86             style: TextStylesManager.medium14.copyWith(
87               color: _selectedFileName != null
88                 ? ColorsManager.primaryColor

```



```

89         : ColorsManager.textPrimary,
90     ),
91     textAlign: TextAlign.center,
92 ),
93 if (_selectedFileName == null) ...[
94     verticalSpace4,
95     Text(
96         appTranslation().get('pay_receipt_formats'),
97         style: TextStylesManager.regular12.copyWith(
98             color: ColorsManager.secondaryText,
99         ),
100     ),
101 ],
102 ],
103 ),
104 ),
105 );
106 }
107 }
108

```

Lines • 511
Dart

Feature / #268 -

Payment [lib/features/payment/presentation/widgets/subscription_status_dialog.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/di/injections.dart';
4  import 'package:moean/core/theme/colors.dart';
5  import 'package:moean/core/theme/text_styles.dart';
6  import 'package:moean/core/utils/constants/routes.dart';
7  import 'package:moean/features/payment/presentation/cubit/subscription_cubit.dart';
8  import 'package:moean/features/payment/presentation/cubit/subscription_state.dart';
9  import 'package:moean/features/profile/presentation/cubit/profile_cubit.dart';
10
11  class SubscriptionStatusDialog extends StatefulWidget {
12    const SubscriptionStatusDialog({super.key});
13
14    static Future<void> show(BuildContext context) {
15      // Always force-refresh subscription data when dialog opens
16      final subCubit = sl<SubscriptionCubit>();
17      subCubit.fetchCurrentSubscription(forceRefresh: true);
18
19      return showDialog(
20        context: context,
21        barrierDismissible: true,
22        builder: (context) => const SubscriptionStatusDialog(),
23      );
24    }
25
26    @override
27    State<SubscriptionStatusDialog> createState() => _SubscriptionStatusDialogState();
28  }
29
30  class _SubscriptionStatusDialogState extends State<SubscriptionStatusDialog> {
31    @override
32    void initState() {
33      super.initState();
34      // forceRefresh already called in show(), no double call needed
35    }
36
37    static String _formatArabicDate(DateTime? date) {
38      if (date == null) return 'غير محدد';
39      const arabicMonths = [
40        'يناير',
41        'فبراير',
42        'مارس',
43        'أبريل',
44        'مايو',
45        'يونيو',
46        'يوليو',
47        'أغسطس',
48        'سبتمبر',
49        'أكتوبر',

```

```

50     'نوفمبر',
51     'ديسمبر'
52 ];
53 final monthName = (date.month >= 1 && date.month <= 12)
54     ? arabicMonths[date.month - 1]
55     : '${date.month}';
56 return '${date.day} $monthName ${date.year}';
57 }
58
59 static String _formatRemainingDaysText(int days) {
60     if (days <= 0) return 'انتهت الصلاحية';
61     if (days == 1) return 'يوم واحد';
62     if (days == 2) return 'يومان';
63     if (days >= 3 && days <= 10) return '$days أيام';
64     return '$days يوم';
65 }
66
67 @override
68 Widget build(BuildContext context) {
69     return Directionality(
70         textDirection: TextDirection.rtl,
71         child: Dialog(
72             backgroundColor: Colors.transparent,
73             insetPadding: const EdgeInsets.symmetric(horizontal: 20, vertical: 24),
74             elevation: 0,
75             child: Container(
76                 decoration: BoxDecoration(
77                     color: ColorsManager.isDark ? ColorsManager.surfaceDark : Colors.white,
78                     borderRadius: BorderRadius.circular(24),
79                     border: Border.all(
80                         color: ColorsManager.isDark
81                             ? Colors.white.withValues(alpha: 0.1)
82                             : const Color(0xFFE5E7EB),
83                         width: 1,
84                     ),
85                     boxShadow: [
86                         BoxShadow(
87                             color: Colors.black.withValues(alpha: 0.08),
88                             blurRadius: 24,
89                             offset: const Offset(0, 8),
90                         ),
91                     ],
92                 ),
93                 padding: const EdgeInsets.all(16),
94                 child: BlocBuilder<SubscriptionCubit, SubscriptionState>(
95                     bloc: sl<SubscriptionCubit>(),
96                     builder: (context, subState) {
97                         final current = sl<SubscriptionCubit>().currentSubscription;
98                         final teacher = sl<ProfileCubit>().profileModel?.teacher;
99
100                         // — DEBUG —————
101                         debugPrint('===== SubscriptionStatusDialog DATA =====');
102                         debugPrint('subState      : $subState');
103                         debugPrint('current      : $current');
104                         debugPrint('isSubscribed : ${current?.isSubscribed}');
105                         debugPrint('isInTrial   : ${current?.isInTrial}');
106                         debugPrint('isExpired   : ${current?.isSubscriptionExpired}');
107                         debugPrint('trialEndsAt : ${current?.trialEndsAt}');
108                         debugPrint('subscriptionEnd : ${current?.subscriptionEndsAt}');
109                         debugPrint('planName    : ${current?.plan?.name}');
110                         debugPrint('trialDaysLeft : ${current?.trialDaysRemaining}');
111                         debugPrint('dynamicDaysLeft : ${current?.dynamicDaysRemaining}');
112                         debugPrint('teacher subs : ${teacher?.isSubscribed}');
113                         debugPrint('teacher trial : ${teacher?.isInTrial}');
114                         debugPrint('=====');
115                         // —————
116
117                         // Determine plan attributes
118                         final bool isSubscribed = current?.isSubscribed ?? teacher?.isSubscribed ?? false;
119                         final bool isInTrial = current?.isInTrial ?? teacher?.isInTrial ?? false;
120                         final DateTime? trialEndsAt = current?.trialEndsAt ?? teacher?.trialEndsAt;
121                         final DateTime? subscriptionEndsAt = current?.subscriptionEndsAt ?? teacher?.subscriptionEndsAt;
122
123                         final DateTime? effectiveEndsAt = isSubscribed
124                             ? subscriptionEndsAt

```

```

124         : (isInTrial ? trialEndsAt : (subscriptionEndsAt ?? trialEndsAt));
125
126     // Calculate days
127     int daysRemaining = 0;
128     if (effectiveEndsAt != null) {
129         final now = DateTime.now();
130         if (effectiveEndsAt.isAfter(now)) {
131             daysRemaining = (effectiveEndsAt.difference(now).inHours / 24).ceil();
132         }
133     } else {
134         daysRemaining = current?.trialDaysRemaining ?? teacher?.trialDaysRemaining ?? 0;
135     }
136
137     // Determine expiration
138     final bool isExpired;
139     if (isSubscribed) {
140         isExpired = subscriptionEndsAt != null && subscriptionEndsAt.isBefore(DateTime.now());
141     } else if (isInTrial) {
142         isExpired = (trialEndsAt != null && trialEndsAt.isBefore(DateTime.now())) || daysRemaining <=
0;
143     } else {
144         isExpired = true;
145     }
146
147     // Determine exact state mode:
148     // 1 = Subscribed Active
149     // 2 = Expired
150     // 3 = Trial Active
151     final bool isModeSubscribed = isSubscribed && !isExpired;
152     final bool isModeExpired = isExpired;
153
154     // Plan title
155     final String planTitle;
156     if (isModeSubscribed) {
157         planTitle = current?.plan?.name ?? teacher?.subscription?.name ?? 'فصل دراسي واحد';
158     } else {
159         planTitle = 'مجاني';
160     }
161
162     // Badge configuration
163     final String badgeText;
164     final Color badgeBg;
165     final Color badgeTextColor;
166     final IconData headerIcon;
167     final Color headerIconBg;
168     final Color headerIconColor;
169     final String dateLabel;
170     final String remainingValue;
171     final String buttonText;
172     final bool showUsage;
173
174     if (isModeSubscribed) {
175         badgeText = 'اشترك نشط';
176         badgeBg = const Color(0xFFD1FAE5);
177         badgeTextColor = const Color(0xFF065F46);
178         headerIcon = Icons.gpp_good_outlined;
179         headerIconBg = const Color(0xFFD1FAE5);
180         headerIconColor = const Color(0xFF0E7A5E);
181         dateLabel = 'تاريخ انتهاء الاشتراك';
182         remainingValue = _formatRemainingDaysText(daysRemaining);
183         buttonText = 'ترقية أو تجديد الاشتراك';
184         showUsage = true;
185     } else if (isModeExpired) {
186         badgeText = 'منتهي';
187         badgeBg = const Color(0xFFFFEE2E2);
188         badgeTextColor = const Color(0xFFDC2626);
189         headerIcon = Icons.error_outline_rounded;
190         headerIconBg = const Color(0xFFFFEE2E2);
191         headerIconColor = const Color(0xFFDC2626);
192         dateLabel = 'تاريخ انتهاء الاشتراك': 'تاريخ انتهاء التجربة';
193         remainingValue = 'انتهت الصلاحية';
194         buttonText = 'اشترك في باقة مدفوعة';
195         showUsage = false;
196     } else {
197         // Trial Active

```

```

198 badgeText = 'تجربة مجانية';
199 badgeBg = const Color(0xFFFEF3C7);
200 badgeTextColor = const Color(0xFF92400E);
201 headerIcon = Icons.auto_awesome;
202 headerIconBg = const Color(0xFFFEF3C7);
203 headerIconColor = const Color(0xFFD97706);
204 dateLabel = 'تاريخ انتهاء التجربة';
205 remainingValue = _formatRemainingDaysText(daysRemaining);
206 buttonText = 'اشترك في باقة مدفوعة';
207 showUsage = true;
208 }
209
210 // Usage statistics
211 final int lessonsRemaining = current?.usage.lessonsRemainingToday ??
212     current?.plan?.lessonLimitPerDay ??
213     teacher?.subscription?.lessonLimitPerDay ??
214     15;
215 final int aiRemaining = current?.usage.aiRemaining ??
216     teacher?.aiQuotaRemaining ??
217     current?.plan?.aiQuotaPerMonth ??
218     teacher?.subscription?.aiQuotaPerMonth ??
219     200;
220
221 return Column(
222   mainAxisAlignment: MainAxisAlignment.min,
223   crossAxisAlignment: CrossAxisAlignment.stretch,
224   children: [
225     // Header Row: [Badge] ... [Title Column + Icon]
226     Row(
227       mainAxisAlignment: MainAxisAlignment.spaceBetween,
228       crossAxisAlignment: CrossAxisAlignment.center,
229       children: [
230         // Status Badge
231         Container(
232           padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
233           decoration: BoxDecoration(
234             color: badgeBg,
235             borderRadius: BorderRadius.circular(20),
236           ),
237           child: Text(
238             badgeText,
239             style: TextStyleManager.bold13.copyWith(
240               color: badgeTextColor,
241             ),
242           ),
243       ],
244       const SizedBox(width: 8),
245
246       // Plan Name & Icon Row
247       Expanded(
248         child: Row(
249           mainAxisAlignment: MainAxisAlignment.end,
250           mainAxisAlignment: MainAxisAlignment.min,
251           children: [
252             Flexible(
253               child: Column(
254                 crossAxisAlignment: CrossAxisAlignment.end,
255                 children: [
256                   Text(
257                     'خطه الاشتراك الحالية',
258                     style: TextStyleManager.medium12.copyWith(
259                       color: ColorsManager.isDark
260                         ? ColorsManager.mutedDark
261                         : const Color(0xFF6B7280),
262                   ),
263                   maxLines: 1,
264                   overflow: TextOverflow.ellipsis,
265                 ],
266                 const SizedBox(height: 2),
267                 Text(
268                   planTitle,
269                   style: TextStyleManager.bold16.copyWith(
270                     color: const Color(0xFF0E7A5E),
271                   ),
272                   maxLines: 1,

```

```

273         overflow: TextOverflow.ellipsis,
274     ),
275 ],
276 ),
277 ),
278     const SizedBox(width: 10),
279     Container(
280       width: 42,
281       height: 42,
282       decoration: BoxDecoration(
283         color: headerIconBg,
284         shape: BoxShape.circle,
285       ),
286       child: Icon(
287         headerIcon,
288         color: headerIconColor,
289         size: 22,
290       ),
291     ),
292 ],
293 ),
294 ),
295 ],
296 ),
297
298 const SizedBox(height: 16),
299 Divider(
300   color: ColorsManager.isDark
301     ? Colors.white.withValues(alpha: 0.08)
302     : const Color(0xFFFF1F5F9),
303   thickness: 1.2,
304   height: 1,
305 ),
306 const SizedBox(height: 16),
307
308 // Expiration Date Card
309 Container(
310   padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
311   decoration: BoxDecoration(
312     color: ColorsManager.isDark
313       ? ColorsManager.surfaceDark.withValues(alpha: 0.5)
314       : const Color(0xFFF7FAF9),
315     borderRadius: BorderRadius.circular(16),
316   ),
317   child: Row(
318     mainAxisAlignment: MainAxisAlignment.spaceBetween,
319     children: [
320       Expanded(
321         child: Row(
322           mainAxisAlignment: MainAxisAlignment.min,
323           children: [
324             const Icon(
325               Icons.calendar_today_outlined,
326               size: 16,
327               color: Color(0xFF0E7A5E),
328             ),
329             const SizedBox(width: 6),
330             Flexible(
331               child: Text(
332                 dateLabel,
333                 style: TextStylesManager.medium13.copyWith(
334                   color: ColorsManager.isDark
335                     ? ColorsManager.mutedDark
336                     : const Color(0xFF4B5563),
337                 ),
338                 overflow: TextOverflow.ellipsis,
339               ),
340             ),
341           ],
342         ),
343       ),
344       const SizedBox(width: 6),
345       Text(
346         _formatArabicDate(effectiveEndsAt),
347         style: TextStylesManager.bold13.copyWith(

```

```

348         color: const Color(0xFF0E7A5E),
349     ),
350 ),
351 ],
352 ),
353 ),
354
355 const SizedBox(height: 10),
356
357 // Remaining Duration Card
358 Container(
359   padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
360   decoration: BoxDecoration(
361     color: ColorsManager.isDark
362       ? ColorsManager.surfaceDark.withValues(alpha: 0.5)
363       : const Color(0xFFF7FAF9),
364     borderRadius: BorderRadius.circular(16),
365   ),
366   child: Row(
367     mainAxisAlignment: MainAxisAlignment.spaceBetween,
368     children: [
369       Expanded(
370         child: Text(
371           'المدة المتبقية:',
372           style: TextStylesManager.medium13.copyWith(
373             color: ColorsManager.isDark
374               ? ColorsManager.mutedDark
375               : const Color(0xFF4B5563),
376           ),
377         ),
378       ),
379       const SizedBox(width: 6),
380       Text(
381         remainingValue,
382         style: TextStylesManager.bold13.copyWith(
383           color: const Color(0xFF0E7A5E),
384         ),
385       ),
386     ],
387   ),
388 ),
389
390 // Usage Card (shown when trial or subscribed is active)
391 if (showUsage) ...[
392   const SizedBox(height: 12),
393   Container(
394     padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
395     decoration: BoxDecoration(
396       color: ColorsManager.isDark
397         ? ColorsManager.surfaceDark
398         : Colors.white,
399     borderRadius: BorderRadius.circular(16),
400     border: Border.all(
401       color: ColorsManager.isDark
402         ? Colors.white.withValues(alpha: 0.1)
403         : const Color(0xFFE2E8F0),
404     width: 1.2,
405   ),
406   ),
407   child: Column(
408     children: [
409     Row(
410       mainAxisAlignment: MainAxisAlignment.spaceBetween,
411       children: [
412         Expanded(
413           child: Text(
414             'تحفيزات اليوم المتبقية:',
415             style: TextStylesManager.bold13.copyWith(
416               color: ColorsManager.isDark
417                 ? ColorsManager.textPrimaryDark
418                 : const Color(0xFF475569),
419             ),
420           ),
421         ),
422         const SizedBox(width: 6),

```

```

423         Text(
424             '$lessonsRemaining',
425             style: TextStylesManager.bold13.copyWith(
426                 color: const Color(0xFF0E7A5E),
427             ),
428         ),
429     ],
430 ),
431 // const SizedBox(height: 10),
432 // Row(
433 //     mainAxisAlignment: MainAxisAlignment.spaceBetween,
434 //     children: [
435 //         Expanded(
436 //             child: Text(
437 //                 'رصيد الذكاء الاصطناعي المتبقي:',
438 //                 style: TextStylesManager.bold13.copyWith(
439 //                     color: ColorsManager.isDark
440 //                         ? ColorsManager.textPrimaryDark
441 //                         : const Color(0xFF475569),
442 //                 ),
443 //             ),
444 //         ),
445 //         const SizedBox(width: 6),
446 //         Text(
447 //             '$aiRemaining',
448 //             style: TextStylesManager.bold13.copyWith(
449 //                 color: const Color(0xFF0E7A5E),
450 //             ),
451 //         ),
452 //     ],
453 // ),
454 ],
455 ),
456 ),
457 ],
458 ],
459
460 const SizedBox(height: 18),
461
462 // Upgrade / Subscribe Action Button
463 SizedBox(
464     height: 48,
465     child: ElevatedButton(
466         onPressed: () {
467             Navigator.pop(context);
468             Navigator.pushNamed(context, Routes.checkout);
469         },
470         style: ElevatedButton.styleFrom(
471             backgroundColor: const Color(0xFF0E7A5E),
472             foregroundColor: Colors.white,
473             elevation: 0,
474             shape: RoundedRectangleBorder(
475                 borderRadius: BorderRadius.circular(25),
476             ),
477             padding: const EdgeInsets.symmetric(horizontal: 16),
478         ),
479         child: Row(
480             mainAxisAlignment: MainAxisAlignment.center,
481             children: [
482                 Flexible(
483                     child: Text(
484                         buttonText,
485                         style: TextStylesManager.bold16.copyWith(
486                             color: Colors.white,
487                         ),
488                     maxLines: 1,
489                     overflow: TextOverflow.ellipsis,
490                 ),
491                 const SizedBox(width: 8),
492                 const Icon(
493                     Icons.bolt_rounded,
494                     size: 22,
495                     color: Colors.white,
496                 ),
497             ],

```

```

498         ],
499     ),
500 ),
501 ),
502 ],
503 );
504 },
505 ),
506 ),
507 ),
508 );
509 }
510 }
511

```

Lines • Dart 188

Feature / Payment

#269 – lib/features/payment/presentation/widgets/trial_banner_widget copy.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/di/injections.dart';
4  import 'package:moean/core/theme/colors.dart';
5  import 'package:moean/core/theme/text_styles.dart';
6  import 'package:moean/core/utils/constants/routes.dart';
7  import 'package:moean/features/payment/presentation/cubit/subscription_cubit.dart';
8  import 'package:moean/features/payment/presentation/cubit/subscription_state.dart';
9
10 class TrialBannerWidget extends StatefulWidget {
11   const TrialBannerWidget({super.key});
12
13   static bool isVisible(SubscriptionState state) {
14     if (state is SubscriptionLoaded) {
15       final current = state.current;
16       if (current.isSubscribed && !current.isSubscriptionExpired) {
17         return false;
18       }
19       return true;
20     } else if (state is SubscriptionError) {
21       if (state.error.contains('__402__') ||
22         state.error.contains('trial_expired') ||
23         state.error.contains('انتهت')) {
24         return true;
25       }
26     } else {
27       final current = sl<SubscriptionCubit>().currentSubscription;
28       if (current != null) {
29         if (current.isSubscribed && !current.isSubscriptionExpired) {
30           return false;
31         }
32         return true;
33       }
34     }
35     return false;
36   }
37
38   @override
39   State<TrialBannerWidget> createState() => _TrialBannerWidgetState();
40 }
41
42 class _TrialBannerWidgetState extends State<TrialBannerWidget> {
43   @override
44   void initState() {
45     super.initState();
46     final cubit = sl<SubscriptionCubit>();
47     if (cubit.currentSubscription == null && cubit.lastError == null && cubit.state is SubscriptionInitial) {
48       cubit.fetchCurrentSubscription();
49     }
50   }
51
52   @override
53   Widget build(BuildContext context) {
54     return BlocBuilder<SubscriptionCubit, SubscriptionState>(
55       bloc: sl<SubscriptionCubit>(),
56       builder: (context, state) {

```



```

57     if (state is SubscriptionLoaded) {
58         final current = state.current;
59
60         if (current.isSubscribed && !current.isSubscriptionExpired) {
61             // Subscribed and Active -> Hidden
62             return const SizedBox.shrink();
63         }
64
65         if (current.isSubscribed && current.isSubscriptionExpired) {
66             // Subscribed but expired
67             return _buildBanner(
68                 context,
69                 icon: Icons.alarm,
70                 text: '⚠️.انتهى اشتراكك المدفوع.',
71                 buttonText: 'جدد اشتراكك',
72                 color: ColorsManager.errorColor,
73                 isExpired: true,
74             );
75         }
76
77         if (current.isInTrial) {
78             final days = current.dynamicDaysRemaining;
79             if (days <= 0) {
80                 // Trial expired
81                 return _buildBanner(
82                     context,
83                     icon: Icons.alarm,
84                     text: '⚠️.انتهت فترتك التجريبية المجانية.',
85                     buttonText: 'اشترك الآن',
86                     color: ColorsManager.errorColor,
87                     isExpired: true,
88                 );
89             } else if (days == 1) {
90                 return _buildBanner(
91                     context,
92                     icon: Icons.warning_amber_rounded,
93                     text: '⚠️!آخر يوم في تجربتك المجانية',
94                     buttonText: 'اشترك الآن',
95                     color: ColorsManager.errorColor,
96                     isExpired: false,
97                 );
98             } else {
99                 return _buildBanner(
100                     context,
101                     icon: Icons.celebration,
102                     text: '🎉.أيام على التجربة المجانية $days متبقي',
103                     buttonText: 'اشترك الآن',
104                     color: const Color(0xFFD97706),
105                     // ColorsManager.primaryColor,
106                     isExpired: false,
107                 );
108             }
109         }
110
111         // Not in trial and not subscribed -> Trial Expired
112         return _buildBanner(
113             context,
114             icon: Icons.alarm,
115             text: '⚠️.انتهت فترتك التجريبية المجانية.',
116             buttonText: 'اشترك الآن',
117             color: ColorsManager.errorColor,
118             isExpired: true,
119         );
120     } else if (state is SubscriptionError) {
121         if (state.error.contains('__402__') ||
122             state.error.contains('trial_expired') ||
123             state.error.contains('انتهت')) {
124             return _buildBanner(
125                 context,
126                 icon: Icons.alarm,
127                 text: '⚠️.انتهت فترتك التجريبية المجانية.',
128                 buttonText: 'اشترك الآن',
129                 color: ColorsManager.errorColor,
130                 isExpired: true,
131             );

```

```

132     }
133   }
134   return const SizedBox.shrink();
135 },
136 );
137 }
138
139 Widget _buildBanner(BuildContext context, {
140   required IconData icon,
141   required String text,
142   required String buttonText,
143   required Color color,
144   required bool isExpired,
145 }) {
146   return Container(
147     width: double.infinity,
148     color: color,
149     child: SafeArea(
150       bottom: false,
151       child: Padding(
152         padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
153         child: Row(
154           children: [
155             Expanded(
156               child: Text(
157                 text,
158                 style: TextStyleManager.bold13.copyWith(color: Colors.white),
159               ),
160             ),
161             const SizedBox(width: 8),
162             ElevatedButton(
163               onPressed: () {
164                 Navigator.pushNamed(context, Routes.checkout);
165               },
166               style: ElevatedButton.styleFrom(
167                 backgroundColor: Colors.white,
168                 foregroundColor: color,
169                 padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 4),
170                 minimumSize: const Size(0, 32),
171                 shape: RoundedRectangleBorder(
172                   borderRadius: BorderRadius.circular(16),
173                 ),
174               ),
175               child: Text(
176                 buttonText,
177                 style: TextStyleManager.bold12,
178               ),
179             ),
180           ],
181         ),
182       ),
183     ),
184   );
185 };
186 }
187 }
188

```

Lines • Dart 186

Feature / Payment

#270 – lib/features/payment/presentation/widgets/trial_banner_widget.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/di/injections.dart';
4 import 'package:moean/core/theme/colors.dart';
5 import 'package:moean/core/theme/text_styles.dart';
6 import 'package:moean/core/utils/constants/routes.dart';
7 import 'package:moean/features/payment/presentation/cubit/subscription_cubit.dart';
8 import 'package:moean/features/payment/presentation/cubit/subscription_state.dart';
9
10 class TrialBannerWidget extends StatefulWidget {
11   const TrialBannerWidget({super.key});
12
13   static bool isBannerVisible(SubscriptionState state) {

```

```

14     if (state is SubscriptionLoaded) {
15         final current = state.current;
16         if (current.isSubscribed && !current.isSubscriptionExpired) {
17             return false;
18         }
19         return true;
20     } else if (state is SubscriptionError) {
21         if (state.error.contains('__402__') ||
22             state.error.contains('trial_expired') ||
23             state.error.contains('انتهت')) {
24             return true;
25         }
26     } else {
27         final current = sl<SubscriptionCubit>().currentSubscription;
28         if (current != null) {
29             if (current.isSubscribed && !current.isSubscriptionExpired) {
30                 return false;
31             }
32             return true;
33         }
34     }
35     return false;
36 }
37
38 @override
39 State<TrialBannerWidget> createState() => _TrialBannerWidgetState();
40 }
41
42 class _TrialBannerWidgetState extends State<TrialBannerWidget> {
43     @override
44     void initState() {
45         super.initState();
46         // Always force-refresh to pick up payment status changes immediately
47         sl<SubscriptionCubit>().fetchCurrentSubscription(forceRefresh: true);
48     }
49
50     @override
51     Widget build(BuildContext context) {
52         return BlocBuilder<SubscriptionCubit, SubscriptionState>(
53             bloc: sl<SubscriptionCubit>(),
54             builder: (context, state) {
55                 if (state is SubscriptionLoaded) {
56                     final current = state.current;
57
58                     if (current.isSubscribed && !current.isSubscriptionExpired) {
59                         // Subscribed and Active -> Hidden
60                         return const SizedBox.shrink();
61                     }
62
63                     if (current.isSubscribed && current.isSubscriptionExpired) {
64                         // Subscribed but expired
65                         return _buildBanner(
66                             context,
67                             icon: Icons.alarm,
68                             text: 'انتهى اشتراكك المدفوع 🚨',
69                             buttonText: 'جدد اشتراكك',
70                             color: ColorsManager.errorColor,
71                             isExpired: true,
72                         );
73                     }
74
75                     if (current.isInTrial) {
76                         final days = current.dynamicDaysRemaining;
77                         if (days <= 0) {
78                             // Trial expired
79                             return _buildBanner(
80                                 context,
81                                 icon: Icons.alarm,
82                                 text: 'انتهت فترتك التجريبية المجانية 🚨',
83                                 buttonText: 'اشترك الآن',
84                                 color: ColorsManager.errorColor,
85                                 isExpired: true,
86                             );
87                         } else if (days == 1) {
88                             return _buildBanner(

```

```

89         context,
90         icon: Icons.warning_amber_rounded,
91         text: '⚠️ أآخر يوم في تجربتك المجانية',
92         buttonText: 'اشترك الآن',
93         color: ColorsManager.errorColor,
94         isExpired: false,
95     );
96 } else {
97     return _buildBanner(
98         context,
99         icon: Icons.celebration,
100        text: '🎉 أيام على التجربة المجانية $days متبقى',
101        buttonText: 'اشترك الآن',
102        color: const Color(0xFFD97706),
103        // ColorsManager.primaryColor,
104        isExpired: false,
105    );
106 }
107 }
108
109 // Not in trial and not subscribed -> Trial Expired
110 return _buildBanner(
111     context,
112     icon: Icons.alarm,
113     text: '⏰ انتهت فترتك التجريبية المجانية.',
114     buttonText: 'اشترك الآن',
115     color: ColorsManager.errorColor,
116     isExpired: true,
117 );
118 } else if (state is SubscriptionError) {
119     if (state.error.contains('__402__') ||
120         state.error.contains('trial_expired') ||
121         state.error.contains('انتهت')) {
122         return _buildBanner(
123             context,
124             icon: Icons.alarm,
125             text: '⏰ انتهت فترتك التجريبية المجانية.',
126             buttonText: 'اشترك الآن',
127             color: ColorsManager.errorColor,
128             isExpired: true,
129         );
130     }
131 }
132 return const SizedBox.shrink();
133 },
134 );
135 }
136
137 Widget _buildBanner(BuildContext context, {
138     required IconData icon,
139     required String text,
140     required String buttonText,
141     required Color color,
142     required bool isExpired,
143 }) {
144     return Container(
145         width: double.infinity,
146         color: color,
147         child: SafeArea(
148             bottom: false,
149             child: Padding(
150                 padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
151                 child: Row(
152                     children: [
153                         Expanded(
154                             child: Text(
155                                 text,
156                                 style: TextStylesManager.bold13.copyWith(color: Colors.white),
157                             ),
158                         ),
159                         const SizedBox(width: 8),
160                         ElevatedButton(
161                             onPressed: () {
162                                 Navigator.pushNamed(context, Routes.checkout);
163                             },

```

```

164         style: ElevatedButton.styleFrom(
165             backgroundColor: Colors.white,
166             foregroundColor: color,
167             padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 4),
168             minimumSize: const Size(0, 32),
169             shape: RoundedRectangleBorder(
170                 borderRadius: BorderRadius.circular(16),
171             ),
172         ),
173         child: Text(
174             buttonText,
175             style: TextStyleManager.bold12,
176         ),
177     ),
178 ],
179 ),
180
181 ),
182 ),
183 );
184 }
185 }
186

```

Lines • Dart 125

Feature / Payment

#271 – lib/features/payment/presentation/widgets/visa_card_display.dart

```

1  import 'package:flutter/material.dart';
2
3  class VisaCardDisplay extends StatelessWidget {
4      final String cardNumber;
5      final String cardHolder;
6      final String expiryDate;
7
8      const VisaCardDisplay({
9          super.key,
10         required this.cardNumber,
11         required this.cardHolder,
12         required this.expiryDate,
13     });
14
15     @override
16     Widget build(BuildContext context) {
17         String formattedNumber = cardNumber.replaceAll(' ', '').padRight(16, '*');
18         if (formattedNumber.length > 16) formattedNumber = formattedNumber.substring(0, 16);
19         formattedNumber = '${formattedNumber.substring(0,4)} ${formattedNumber.substring(4,8)} ${formattedNumber.
substring(8,12)} ${formattedNumber.substring(12,16)}';
20
21         return Container(
22             width: double.infinity,
23             height: 220,
24             padding: const EdgeInsets.all(24),
25             decoration: BoxDecoration(
26                 borderRadius: BorderRadius.circular(20),
27                 gradient: const LinearGradient(
28                     colors: [Color(0xFF0F2027), Color(0xFF203A43), Color(0xFF2C5364)],
29                     begin: Alignment.topLeft,
30                     end: Alignment.bottomRight,
31                 ),
32                 boxShadow: [
33                     BoxShadow(
34                         color: Colors.black.withValues(alpha: 0.3),
35                         blurRadius: 15,
36                         offset: const Offset(0, 8),
37                     ),
38                 ],
39             ),
40             child: Column(
41                 crossAxisAlignment: CrossAxisAlignment.start,
42                 mainAxisAlignment: MainAxisAlignment.spaceBetween,
43                 children: [
44                     Row(
45                         mainAxisAlignment: MainAxisAlignment.spaceBetween,
46                         children: [

```

```

47         const Text(
48             'VISA',
49             style: TextStyle(
50                 color: Colors.white,
51                 fontSize: 28,
52                 fontWeight: FontWeight.w900,
53                 fontStyle: FontStyle.italic,
54                 letterSpacing: 2,
55             ),
56         ),
57         Container(
58             width: 45,
59             height: 35,
60             decoration: BoxDecoration(
61                 borderRadius: BorderRadius.circular(6),
62                 color: Colors.amber.shade300,
63                 gradient: LinearGradient(
64                     colors: [Colors.amber.shade200, Colors.amber.shade400],
65                     begin: Alignment.topLeft,
66                     end: Alignment.bottomRight,
67                 ),
68             ),
69         ),
70     ],
71 ),
72 Directionality(
73     textDirection: TextDirection.ltr,
74     child: Text(
75         formattedNumber,
76         style: const TextStyle(
77             color: Colors.white,
78             fontSize: 24,
79             letterSpacing: 4,
80             fontFamily: 'monospace',
81         ),
82     ),
83 ),
84 Row(
85     mainAxisAlignment: MainAxisAlignment.spaceBetween,
86     children: [
87         Column(
88             crossAxisAlignment: CrossAxisAlignment.start,
89             children: [
90                 const Text(
91                     'EXPIRES',
92                     style: TextStyle(color: Colors.white70, fontSize: 10, letterSpacing: 1),
93                 ),
94                 const SizedBox(height: 4),
95                 Directionality(
96                     textDirection: TextDirection.ltr,
97                     child: Text(
98                         expiryDate.isEmpty ? 'MM/YY' : expiryDate,
99                         style: const TextStyle(color: Colors.white, fontSize: 16, fontWeight: FontWeight.bold,
fontFamily: 'monospace'),
100                     ),
101                 ),
102             ],
103         ),
104         Column(
105             crossAxisAlignment: CrossAxisAlignment.end,
106             children: [
107                 const Text(
108                     'CARD HOLDER',
109                     style: TextStyle(color: Colors.white70, fontSize: 10, letterSpacing: 1),
110                 ),
111                 const SizedBox(height: 4),
112                 Text(
113                     cardHolder.isEmpty ? 'NAME ON CARD' : cardHolder.toUpperCase(),
114                     style: const TextStyle(color: Colors.white, fontSize: 16, fontWeight: FontWeight.bold),
115                 ),
116             ],
117         ),
118     ],
119 ),
120 ],

```

```

121     ),
122     );
123   }
124 }
125

```

Lines • Dart 79

Feature / Presentations

#272 – lib/features/presentations/data/models/presentation_models.dart

```

1  class PresentationModel {
2    final int id;
3    final int lessonId;
4    final String status;
5    final String? templateId;
6    final String? filePath;
7    final int? slideCount;
8    final String? generatedAt;
9    final String? generationError;
10   final List<PresentationSlideModel> slides;
11
12   PresentationModel({
13     required this.id,
14     required this.lessonId,
15     required this.status,
16     this.templateId,
17     this.filePath,
18     this.slideCount,
19     this.generatedAt,
20     this.generationError,
21     this.slides = const [],
22   });
23
24   factory PresentationModel.fromJson(Map<String, dynamic> json) {
25     // Some endpoints wrap the model in a "presentation" key, while others might return it directly.
26     final data = json.containsKey('presentation') ? (json['presentation'] as Map<String, dynamic>) : json;
27
28     return PresentationModel(
29       id: data['id'] as int? ?? 0,
30       lessonId: data['lesson_id'] as int? ?? 0,
31       status: data['status'] as String? ?? json['status'] as String? ?? 'pending',
32       templateId: data['template_id'] as String?,
33       filePath: data['file_path'] as String?,
34       slideCount: data['slide_count'] as int?,
35       generatedAt: data['generated_at'] as String?,
36       generationError: data['generation_error'] as String?,
37       slides: (data['slides'] as List<dynamic>?)
38         ?.map((e) => PresentationSlideModel.fromJson(e as Map<String, dynamic>))
39         .toList() ??
40       [],
41     );
42   }
43 }
44
45 class PresentationSlideModel {
46   final int id;
47   final int slideOrder;
48   final String slideType;
49   final String title;
50   final String bodyText;
51   final String? iconKeyword;
52   final String? iconId;
53   final String? iconUrl;
54
55   PresentationSlideModel({
56     required this.id,
57     required this.slideOrder,
58     required this.slideType,
59     required this.title,
60     required this.bodyText,
61     this.iconKeyword,
62     this.iconId,
63     this.iconUrl,
64   });
65

```

```

66 factory PresentationSlideModel.fromJson(Map<String, dynamic> json) {
67   return PresentationSlideModel(
68     id: json['id'] as int,
69     slideOrder: json['slide_order'] as int,
70     slideType: json['slide_type'] as String,
71     title: json['title'] as String,
72     bodyText: json['body_text'] as String,
73     iconKeyword: json['icon_keyword'] as String?,
74     iconId: json['icon_id'] as String?,
75     iconUrl: json['icon_url'] as String?,
76   );
77 }
78 }
79

```

Lines • 51
Dart

Feature / #273 -
Presentations lib/features/presentations/data/repositories/presentations_repository.dart

```

1  import 'package:dartz/dartz.dart';
2  import 'package:moean/core/network/remote/api_endpoints.dart';
3  import 'package:moean/core/network/remote/dio_helper.dart';
4  import 'package:moean/features/presentations/data/models/presentation_models.dart';
5
6  class PresentationsRepository {
7    Future<Either<String, PresentationModel>> generatePresentation({
8      required int lessonId,
9      required Map<String, dynamic> payload,
10   }) async {
11     final response = await DioHelper.postData(
12       url: lessonPresentationGenerateApi(lessonId),
13       data: payload,
14     );
15
16     return response.fold(
17       (error) => Left(error),
18       (res) {
19         try {
20           return Right(PresentationModel.fromJson(res.data));
21         } catch (e) {
22           return Left(e.toString());
23         }
24       },
25     );
26   }
27
28   Future<Either<String, PresentationModel>> getPresentation({
29     required int lessonId,
30     required String templateId,
31   }) async {
32     final response = await DioHelper.getData(
33       url: lessonPresentationApi(lessonId),
34       query: {
35         'template_id': templateId,
36       },
37     );
38
39     return response.fold(
40       (error) => Left(error),
41       (res) {
42         try {
43           return Right(PresentationModel.fromJson(res.data));
44         } catch (e) {
45           return Left(e.toString());
46         }
47       },
48     );
49   }
50 }
51

```



```
1 import 'dart:async';
2 import 'dart:convert';
3 import 'dart:io';
4 import 'dart:math';
5 import 'dart:typed_data';
6
7 import 'package:flutter/services.dart' show rootBundle;
8 import 'package:flutter_inappwebview/flutter_inappwebview.dart';
9 import 'package:moean/features/presentations/data/models/presentation_models.dart';
10 import 'package:path_provider/path_provider.dart';
11 import 'package:moean/core/utils/constants/assets_helper.dart';
12
13 class PptxGeneratorService {
14   static Future<String> generatePresentation(
15     PresentationModel presentation,
16     String title, {
17     String? templateId,
18     String subjectName = '',
19     String gradeName = '',
20     String unitName = '',
21   }) async {
22     final completer = Completer<String>();
23
24     // Read html and js
25     final rawHtml = await rootBundle.loadString('assets/html/pptx_generator.html');
26     final rawJs = await rootBundle.loadString('assets/js/pptxgen.bundle.js');
27
28     // Inline JS
29     final combinedHtml = rawHtml.replaceFirst('<script src="../../js/pptxgen.bundle.js"></script>', '<script>\n
$rawJs\n</script>');
30
31     final randomImages = [
32       AssetsHelper.scr1, AssetsHelper.scr2, AssetsHelper.scr3, AssetsHelper.scr4,
33       AssetsHelper.scr5, AssetsHelper.scr6, AssetsHelper.scr7, AssetsHelper.scr8,
34       AssetsHelper.scr9, AssetsHelper.scr10, AssetsHelper.scr11, AssetsHelper.scr12,
35       AssetsHelper.scr13, AssetsHelper.scr14, AssetsHelper.scr15,
36     ];
37     final selectedImage = randomImages[Random().nextInt(randomImages.length)];
38
39     String coverImageBase64 = '';
40     try {
41       final ByteData data = await rootBundle.load(selectedImage);
42       final Uint8List bytes = data.buffer.asUint8List();
43       final base64Str = base64Encode(bytes);
44       coverImageBase64 = 'data:image/jpeg;base64,$base64Str';
45     } catch (e) {
46       print('Could not load random cover image: $e');
47     }
48
49     String openingImageBase64 = '';
50     try {
51       final ByteData data = await rootBundle.load(AssetsHelper.presentation1);
52       final Uint8List bytes = data.buffer.asUint8List();
53       openingImageBase64 = 'data:image/jpeg;base64,${base64Encode(bytes)}';
54     } catch (e) {
55       print('Could not load opening image: $e');
56     }
57
58     String greetingImageBase64 = '';
59     try {
60       final ByteData data = await rootBundle.load(AssetsHelper.presentation2);
61       final Uint8List bytes = data.buffer.asUint8List();
62       greetingImageBase64 = 'data:image/jpeg;base64,${base64Encode(bytes)}';
63     } catch (e) {
64       print('Could not load greeting image: $e');
65     }
66
67     String duaImageBase64 = '';
68     try {
69       final ByteData data = await rootBundle.load(AssetsHelper.presentation3);
70       final Uint8List bytes = data.buffer.asUint8List();
```

```

71     duaImageBase64 = 'data:image/jpeg;base64,${base64Encode(bytes)}';
72 } catch (e) {
73     print('Could not load dua image: $e');
74 }
75
76 // Build context
77 final jsonData = jsonEncode({
78     'presentation': {
79         'template_id': templateId ?? presentation.templateId ?? 'emerald-green',
80         'slides': presentation.slides.map((s) => {
81             'title': s.title,
82             'type': s.slideType,
83             'icon_keyword': s.iconKeyword,
84             'icon_id': s.iconId,
85             'bodyText': s.bodyText,
86             'order': 0, // Fallback order, array position will be used
87         }).toList(),
88     },
89     'context': {
90         'lessonTitle': title,
91         'subjectName': subjectName,
92         'gradeName': gradeName,
93         'unitName': unitName,
94         'coverImageBase64': coverImageBase64,
95         'openingImageBase64': openingImageBase64,
96         'greetingImageBase64': greetingImageBase64,
97         'duaImageBase64': duaImageBase64,
98     }
99 });
100
101 HeadlessInAppWebView? headlessWebView;
102
103 headlessWebView = HeadlessInAppWebView(
104     initialData: InAppWebViewInitialData(data: combinedHtml),
105     initialSettings: InAppWebViewSettings(
106         javaScriptEnabled: true,
107     ),
108     onLoadStop: (controller, url) async {
109         try {
110             final result = await controller.callAsyncJavaScript(
111                 functionBody: "return await window.generatePptxBase64(data);",
112                 arguments: {"data": jsonData}
113             );
114
115             if (result?.value != null) {
116                 final res = result!.value as Map;
117                 if (res['success'] == true) {
118                     final base64String = res['base64'] as String;
119                     final bytes = base64Decode(base64String);
120
121                     final tempDir = await getTemporaryDirectory();
122                     final sanitizedTitle = title.replaceAll(RegExp(r'[\\/:?*"<>|]'), '');
123                     final filePath = '${tempDir.path}/${sanitizedTitle}.pptx';
124                     final file = File(filePath);
125
126                     await file.writeAsBytes(bytes);
127                     completer.complete(filePath);
128                 } else {
129                     completer.completeError("JS Error: ${res['error']}");
130                 }
131             } else {
132                 completer.completeError("No result from JS");
133             }
134         } catch (e) {
135             if (!completer.isCompleted) completer.completeError(e.toString());
136         } finally {
137             headlessWebView?.dispose();
138         }
139     },
140     onReceivedError: (controller, request, error) {
141         if (!completer.isCompleted) completer.completeError("WebView Error: ${error.description}");
142         headlessWebView?.dispose();
143     },
144 );
145

```

```

146     await headlessWebView.run();
147
148     return completer.future;
149   }
150 }
151

```

Lines • 304
Dart

Feature / #275 –
Presentations [lib/features/presentations/presentation/cubit/presentations_cubit.dart](#)

```

1  import 'dart:async';
2  import 'dart:convert';
3  import 'dart:developer';
4  import 'package:flutter/foundation.dart';
5  import 'package:flutter_bloc/flutter_bloc.dart';
6  import 'package:moean/core/network/remote/api_service.dart';
7  import 'package:moean/features/exam_generation/data/models/curriculum_models.dart';
8  import 'package:moean/features/presentations/data/models/presentation_models.dart';
9  import 'package:moean/features/presentations/data/repositories/presentations_repository.dart';
10 import 'package:moean/features/presentations/domain/services/pptx_generator_service.dart';
11 import 'package:share_plus/share_plus.dart';
12 import 'presentations_state.dart';
13
14 class PresentationsCubit extends Cubit<PresentationsState> {
15   final PresentationsRepository repository;
16
17   PresentationsCubit(this.repository) : super(PresentationsInitial()) {
18     _loadSubjects();
19   }
20
21   CurriculumStageModel? selectedStage;
22   String? selectedTrack;
23   CurriculumGradeModel? selectedGrade;
24   CurriculumSubjectModel? selectedSubject;
25   CurriculumChapterModel? selectedUnit;
26   CurriculumLessonModel? selectedLesson;
27
28   List<CurriculumStageModel> stages = [];
29   SubjectDetailsModel? subjectDetails;
30
31   List<String> get availableTracks {
32     if (selectedStage == null) return [];
33     return selectedStage!.grades
34       .map((g) => g.track)
35       .whereType<String>()
36       .where((t) => t.isNotEmpty)
37       .toSet()
38       .toList();
39   }
40
41   List<CurriculumGradeModel> get availableGrades {
42     if (selectedStage == null) return [];
43     if (availableTracks.isNotEmpty) {
44       if (selectedTrack == null) return [];
45       return selectedStage!.grades.where((g) => g.track == selectedTrack).toList();
46     }
47     return selectedStage!.grades;
48   }
49
50   bool isLoading = false;
51   String? dataErrorMessage;
52
53   String selectedTemplate = 'emerald-green';
54   String selectedSlidesCount = '8';
55
56   bool get canCreate =>
57     selectedStage != null &&
58     selectedGrade != null &&
59     selectedSubject != null &&
60     selectedUnit != null &&
61     selectedLesson != null;
62
63   PresentationModel? generatedPresentation;

```

```

64
65 Future<void> _loadSubjects() async {
66     isDataLoading = true;
67     dataErrorMessage = null;
68     if (!isClosed) emit(PresentationsInitial());
69
70     final result = await ApiService.getSubjects();
71     if (isClosed) return;
72     result.fold(
73         (failure) => dataErrorMessage = failure.message,
74         (data) => stages = data,
75     );
76
77     isDataLoading = false;
78     if (!isClosed) emit(PresentationsInitial());
79 }
80
81 Future<void> _loadSubjectLessons(int subjectId) async {
82     isDataLoading = true;
83     dataErrorMessage = null;
84     if (!isClosed) emit(PresentationsInitial());
85
86     final result = await ApiService.getSubjectLessons(subjectId);
87     if (isClosed) return;
88     result.fold(
89         (failure) => dataErrorMessage = failure.message,
90         (data) => subjectDetails = data,
91     );
92
93     isDataLoading = false;
94     if (!isClosed) emit(PresentationsInitial());
95 }
96
97 void updateSelection({
98     CurriculumStageModel? stage,
99     String? track,
100     CurriculumGradeModel? grade,
101     CurriculumSubjectModel? subject,
102     CurriculumChapterModel? unit,
103     CurriculumLessonModel? lesson,
104     String? template,
105     String? slidesCount,
106 }) {
107     if (stage != null) {
108         selectedStage = stage;
109         selectedTrack = null;
110         selectedGrade = null;
111         selectedSubject = null;
112         selectedUnit = null;
113         selectedLesson = null;
114         subjectDetails = null;
115     }
116     if (track != null) {
117         selectedTrack = track;
118         selectedGrade = null;
119         selectedSubject = null;
120         selectedUnit = null;
121         selectedLesson = null;
122         subjectDetails = null;
123     }
124     if (grade != null) {
125         selectedGrade = grade;
126         selectedSubject = null;
127         selectedUnit = null;
128         selectedLesson = null;
129         subjectDetails = null;
130     }
131     if (subject != null) {
132         selectedSubject = subject;
133         selectedUnit = null;
134         selectedLesson = null;
135         subjectDetails = null;
136         _loadSubjectLessons(subject.id);
137     }
138     if (unit != null) {

```

```

139     selectedUnit = unit;
140     selectedLesson = null;
141 }
142 if (lesson != null) selectedLesson = lesson;
143 if (template != null) selectedTemplate = template;
144 if (slidesCount != null) selectedSlidesCount = slidesCount;
145
146 if (!isClosed) emit(PresentationsInitial());
147 }
148
149 Future<void> createPresentation() async {
150     if (!canCreate) return;
151
152     if (!isClosed) emit(PresentationsLoading());
153
154     int lessonId = selectedLesson!.id;
155
156     final payload = {
157         "lesson_id": lessonId,
158         "template_id": selectedTemplate,
159         "slides_count": int.tryParse(selectedSlidesCount.replaceAll(RegExp(r'^0-9'), '')) ?? 8,
160         "slide_count": int.tryParse(selectedSlidesCount.replaceAll(RegExp(r'^0-9'), '')) ?? 8,
161         "stage": selectedStage!.name,
162         "track": selectedTrack ?? selectedGrade!.track ?? "",
163         "grade": selectedGrade!.name,
164         "semester": selectedLesson!.semester,
165         "subject": selectedSubject!.name,
166         "unit": selectedUnit!.title,
167         "chapter": selectedLesson!.title,
168         "lesson_title": selectedLesson!.title,
169     };
170
171     final prettyJsonPayload = const JsonEncoder.withIndent(' ').convert(payload);
172     log('===== PRESENTATION PAYLOAD JSON =====');
173     log(prettyJsonPayload);
174     log('=====');
175     debugPrint('PRESENTATION PAYLOAD JSON:\n$prettyJsonPayload');
176
177     final result = await repository.generatePresentation(
178         lessonId: lessonId,
179         payload: payload,
180     );
181
182     if (isClosed) return;
183
184     result.fold(
185         (error) {
186             if (!isClosed) {
187                 if (error.startsWith('__402__')) {
188                     final parts = error.split(':');
189                     final code = parts.length > 1 ? parts[1] : 'quota_exceeded';
190                     final msg = parts.length > 2 ? parts.sublist(2).join(':') : 'ترقية الحساب المطلوبة';
191                     emit(PresentationsPaymentRequired(msg, code));
192                 } else if (error.contains('Webhook returned HTTP 200') || error.contains('not valid JSON')) {
193                     // The n8n webhook might have started generation in the background but failed to return a JSON re
194                     // sponse.
195                     // Treat this as 'pending' and start polling.
196                     _pollPresentation(lessonId, selectedTemplate);
197                 } else {
198                     emit(PresentationsError(error));
199                 }
200             }
201         },
202         (presentation) {
203             if (!isClosed) {
204                 if (presentation.status == 'ready') {
205                     generatedPresentation = presentation;
206                     emit(PresentationsSuccess(presentation));
207                 } else if (presentation.status == 'failed') {
208                     emit(PresentationsError(presentation.generationError ?? 'فشل في إنشاء العرض'));
209                 } else {
210                     // Poll
211                     _pollPresentation(lessonId, selectedTemplate);
212                 }
213             }
214         }
215     );

```

```

213     },
214     );
215 }
216
217 Future<void> downloadPresentation({List<PresentationSlideModel>? extraSlides}) async {
218     if (generatedPresentation == null) return;
219     try {
220         final title = selectedLesson?.title ?? 'Presentation';
221
222         final presentationToDownload = extraSlides != null && extraSlides.isNotEmpty
223             ? PresentationModel(
224                 id: generatedPresentation!.id,
225                 lessonId: generatedPresentation!.lessonId,
226                 status: generatedPresentation!.status,
227                 templateId: generatedPresentation!.templateId,
228                 filePath: generatedPresentation!.filePath,
229                 slideCount: generatedPresentation!.slideCount,
230                 generatedAt: generatedPresentation!.generatedAt,
231                 generationError: generatedPresentation!.generationError,
232                 slides: [...generatedPresentation!.slides, ...extraSlides],
233             )
234             : generatedPresentation!;
235
236         final path = await PptxGeneratorService.generatePresentation(
237             presentationToDownload,
238             title,
239             templateId: generatedPresentation!.templateId ?? selectedTemplate,
240             subjectName: selectedSubject?.name ?? '',
241             gradeName: selectedGrade?.name ?? '',
242             unitName: selectedUnit?.title ?? '',
243         );
244         await Share.shareXFiles([XFile(path)], text: 'Presentation: $title');
245     } catch (e) {
246         // emit an error if needed, but since it's a download action we might just print or show a toast via UI
247         print('Error generating PPTX: $e');
248     }
249 }
250
251 void _pollPresentation(int lessonId, String templateId) async {
252     int attempts = 0;
253     while (attempts < 20) {
254         await Future.delayed(const Duration(seconds: 3));
255         if (isClosed) return;
256         attempts++;
257
258         final result = await repository.getPresentation(
259             lessonId: lessonId,
260             templateId: templateId,
261         );
262         if (isClosed) return;
263
264         bool stopPolling = false;
265         result.fold(
266             (error) {
267                 if (!isClosed) emit(PresentationsError(error));
268                 stopPolling = true;
269             },
270             (presentation) {
271                 if (presentation.status == 'ready') {
272                     generatedPresentation = presentation;
273                     if (!isClosed) emit(PresentationsSuccess(presentation));
274                     stopPolling = true;
275                 } else if (presentation.status == 'failed') {
276                     if (!isClosed) emit(PresentationsError(presentation.generationError ?? 'إنشاء العرض فشل في إنشائه'));
277                     stopPolling = true;
278                 }
279             },
280         );
281
282         if (stopPolling) return;
283     }
284
285     if (!isClosed) {
286         emit(PresentationsError('استغرق إنشاء العرض وقتاً أطول من المعتاد. يرجى المحاولة لاحقاً'));
287     }

```

```

288     }
289
290     void reset() {
291         selectedStage = null;
292         selectedTrack = null;
293         selectedGrade = null;
294         selectedSubject = null;
295         selectedUnit = null;
296         selectedLesson = null;
297         subjectDetails = null;
298         selectedTemplate = 'emerald-green';
299         selectedSlidesCount = '8';
300         generatedPresentation = null;
301         if (!isClosed) emit(PresentationsInitial());
302     }
303 }
304

```

Lines • 24
Dart

Feature / #276 –
Presentations [lib/features/presentations/presentation/cubit/presentations_state.dart](#)

```

1  import 'package:moean/features/presentations/data/models/presentation_models.dart';
2
3  abstract class PresentationsState {}
4
5  class PresentationsInitial extends PresentationsState {}
6
7  class PresentationsLoading extends PresentationsState {}
8
9  class PresentationsSuccess extends PresentationsState {
10     final PresentationModel presentation;
11     PresentationsSuccess(this.presentation);
12 }
13
14 class PresentationsError extends PresentationsState {
15     final String message;
16     PresentationsError(this.message);
17 }
18
19 class PresentationsPaymentRequired extends PresentationsState {
20     final String message;
21     final String code;
22     PresentationsPaymentRequired(this.message, this.code);
23 }
24

```

Lines • 1111
Dart

Feature / #277 –
Presentations [lib/features/presentations/presentation/screens/presentation_preview_screen.dart](#)

```

1  import 'dart:async';
2  import 'dart:math';
3  import 'package:flutter/material.dart';
4  import 'package:flutter_bloc/flutter_bloc.dart';
5  import 'package:moean/core/theme/colors.dart';
6  import 'package:moean/core/theme/text_styles.dart';
7  import 'package:moean/core/utils/constants/constants.dart';
8  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
9  import 'package:moean/core/utils/constants/spacing.dart';
10 import 'package:moean/core/utils/extensions/context_extension.dart';
11 import 'package:moean/features/presentations/data/models/presentation_models.dart';
12 import 'package:moean/features/presentations/presentation/cubit/presentations_cubit.dart';
13 import 'package:luclide_icons_flutter/luclide_icons.dart';
14 import 'package:moean/core/utils/constants/assets_helper.dart';
15 class PresentationThemeData {
16     final Color primaryAccent;
17     final Color darkAccent;
18     final Color softContainer;
19     final Color backgroundColor;
20     final Color textColor;
21     final Color borderColor;
22
23     const PresentationThemeData({

```

```

24     required this.primaryAccent,
25     required this.darkAccent,
26     required this.softContainer,
27     required this.backgroundColor,
28     required this.textColor,
29     required this.borderColor,
30   });
31
32   factory PresentationThemeData.fromTemplateId(String? templateId) {
33     switch (templateId) {
34       case 'academic-blue':
35         return const PresentationThemeData(
36           primaryAccent: Color(0xFF1976D2),
37           darkAccent: Color(0xFF173E61),
38           softContainer: Color(0xFFEAF3FB),
39           backgroundColor: Color(0xFFFF7FAFE),
40           textColor: Color(0xFF20343F),
41           borderColor: Color(0xFFD4E4F7),
42         );
43       case 'warm-orange':
44         return const PresentationThemeData(
45           primaryAccent: Color(0xFFD89C22),
46           darkAccent: Color(0xFF865806),
47           softContainer: Color(0xFFFFF4D8),
48           backgroundColor: Color(0xFFFFF5CF),
49           textColor: Color(0xFF4E4028),
50           borderColor: Color(0xFFFB8C3),
51         );
52       case 'emerald-green':
53       default:
54         return const PresentationThemeData(
55           primaryAccent: Color(0xFF0E7A5E),
56           darkAccent: Color(0xFF075244),
57           softContainer: Color(0xFFEAF7F2),
58           backgroundColor: Color(0xFFFF7FCFA),
59           textColor: Color(0xFF24443D),
60           borderColor: Color(0xFFEAF7F2),
61         );
62     }
63   }
64 }
65
66 class PresentationPreviewScreen extends StatefulWidget {
67   final PresentationModel presentation;
68   final PresentationsCubit cubit;
69
70   const PresentationPreviewScreen({
71     super.key,
72     required this.presentation,
73     required this.cubit,
74   });
75
76   @override
77   State<PresentationPreviewScreen> createState() => _PresentationPreviewScreenState();
78 }
79
80 class _PresentationPreviewScreenState extends State<PresentationPreviewScreen> {
81   bool _isRandomPickerEnabled = false;
82   final TextEditingController _namesController = TextEditingController();
83
84   @override
85   void dispose() {
86     _namesController.dispose();
87     super.dispose();
88   }
89
90   @override
91   Widget build(BuildContext context) {
92     final bool hasTitleSlide = widget.presentation.slides.any((s) => s.slideType == 'title');
93     final List<PresentationSlideModel> displaySlides = [];
94
95     if (!hasTitleSlide) {
96       displaySlides.add(
97         PresentationSlideModel(
98           id: -1,

```



```

99         slideOrder: 0,
100         slideType: 'title',
101         title: widget.cubit.selectedLesson?.title ?? 'عرض تعليمي',
102         bodyText: [
103           if (widget.cubit.selectedSubject?.name != null) widget.cubit.selectedSubject!.name,
104           if (widget.cubit.selectedGrade?.name != null) widget.cubit.selectedGrade!.name,
105           if (widget.cubit.selectedUnit?.title != null) widget.cubit.selectedUnit!.title,
106         ].where((e) => e.isNotEmpty).join('\n'),
107         iconKeyword: 'graduation_cap',
108       ),
109     );
110   }
111   displaySlides.addAll(widget.presentation.slides);
112
113   PresentationSlideModel? randomPickerSlide;
114   if (_isRandomPickerEnabled && _namesController.text.trim().isNotEmpty) {
115     final names = _namesController.text.split(RegExp(r'[,\\n]+')).map((e) => e.trim()).where((e) => e.isNotEmpty).toList();
116     if (names.isNotEmpty) {
117       randomPickerSlide = PresentationSlideModel(
118         id: -2,
119         slideOrder: 999,
120         slideType: 'random_picker',
121         title: 'اختبار عشوائي',
122         bodyText: names.join('\n'),
123         iconKeyword: 'users',
124       );
125       displaySlides.add(randomPickerSlide);
126     }
127   }
128
129   final String selectedTemplate = widget.presentation.templateId ?? widget.cubit.selectedTemplate;
130   final theme = PresentationThemeData.fromTemplateId(selectedTemplate);
131
132   return BlocProvider.value(
133     value: widget.cubit,
134     child: SafeArea(
135       child: Scaffold(
136         backgroundColor: ColorsManager.background,
137         appBar: AppBar(
138           backgroundColor: ColorsManager.background,
139           elevation: 0,
140           centerTitle: true,
141           title: Text(
142             appTranslation().get('presentations_title'),
143             style: TextStylesManager.bold18.copyWith(color: ColorsManager.mainText),
144           ),
145           leading: IconButton(
146             icon: Icon(Icons.arrow_back_ios, color: ColorsManager.mainText),
147             onPressed: () => context.pop(),
148           ),
149           actions: [
150             IconButton(
151               icon: Icon(Icons.download, color: theme.primaryAccent),
152               onPressed: () => widget.cubit.downloadPresentation(extraSlides: randomPickerSlide != null ? [randomPickerSlide] : null),
153             ),
154           ],
155         ),
156         body: Column(
157           children: [
158             Container(
159               color: Colors.white,
160               padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 10),
161               child: Column(
162                 children: [
163                   SwitchListTile(
164                     title: Text(
165                       'إضافة شريحة الاختبار العشوائي للطلاب',
166                       style: TextStylesManager.bold14.copyWith(color: theme.darkAccent),
167                     ),
168                     value: _isRandomPickerEnabled,
169                     activeColor: theme.primaryAccent,
170                     onChanged: (val) {
171                       setState(() {

```

```

172         _isRandomPickerEnabled = val;
173     });
174 },
175     contentPadding: EdgeInsets.zero,
176 ),
177 if (_isRandomPickerEnabled)
178     Padding(
179         padding: const EdgeInsets.only(top: 8.0, bottom: 8.0),
180         child: TextField(
181             controller: _namesController,
182             maxLines: 3,
183             decoration: InputDecoration(
184                 hintText: 'أدخل أسماء الطلاب (مفصولين بفاصلة أو سطر جديد)',
185                 hintStyle: TextStyleManager.regular14.copyWith(color: Colors.grey),
186                 border: OutlineInputBorder(
187                     borderRadius: BorderRadius.circular(8),
188                     borderSide: BorderSide(color: theme.borderColor),
189                 ),
190                 focusedBorder: OutlineInputBorder(
191                     borderRadius: BorderRadius.circular(8),
192                     borderSide: BorderSide(color: theme.primaryAccent),
193                 ),
194                 contentPadding: const EdgeInsets.all(12),
195             ),
196             onChanged: (_) => setState(() {}),
197         ),
198     ),
199 ],
200 ),
201 ),
202 Expanded(
203     child: ListView.builder(
204         padding: const EdgeInsets.all(20),
205         itemCount: displaySlides.length,
206         itemBuilder: (context, index) {
207             final slide = displaySlides[index];
208             return _buildSlideCard(slide, index + 1, theme);
209         },
210     ),
211 ),
212 Padding(
213     padding: const EdgeInsets.all(20.0),
214     child: PrimaryElevatedButton(
215         text: 'حفظ وتحميل (PPTX)',
216         icon: const Icon(Icons.download),
217         onPressed: () => widget.cubit.downloadPresentation(extraSlides: randomPickerSlide != null ?
[randomPickerSlide] : null),
218     ),
219 ),
220 ],
221 ),
222 ),
223 ),
224 );
225 }
226
227 Widget _buildCoverSlideCard(PresentationSlideModel slide, int slideNumber, PresentationThemeData theme) {
228     final randomImages = [
229         AssetsHelper.scr1, AssetsHelper.scr2, AssetsHelper.scr3, AssetsHelper.scr4,
230         AssetsHelper.scr5, AssetsHelper.scr6, AssetsHelper.scr7, AssetsHelper.scr8,
231         AssetsHelper.scr9, AssetsHelper.scr10, AssetsHelper.scr11, AssetsHelper.scr12,
232         AssetsHelper.scr13, AssetsHelper.scr14, AssetsHelper.scr15,
233     ];
234     final stableIndex = slide.title.length % randomImages.length;
235     final coverImageAsset = randomImages[stableIndex];
236
237     return Container(
238         margin: const EdgeInsets.only(bottom: 24),
239         decoration: BoxDecoration(
240             color: theme.backgroundColor,
241             borderRadius: BorderRadius.circular(12),
242             boxShadow: [
243                 BoxShadow(
244                     color: Colors.black.withValues(alpha: 0.08),
245                     blurRadius: 15,

```

```

246         offset: const Offset(0, 5),
247     ),
248 ],
249 ),
250 child: ClipRRect(
251     borderRadius: BorderRadius.circular(12),
252     child: Column(
253         crossAxisAlignment: CrossAxisAlignment.stretch,
254         children: [
255             Container(height: 4, color: theme.primaryAccent),
256
257             // Slide Number & Label (Header)
258             Padding(
259                 padding: const EdgeInsets.fromLTRB(20, 20, 20, 0),
260                 child: Row(
261                     mainAxisAlignment: MainAxisAlignment.spaceBetween,
262                     children: [
263                         Container(
264                             padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 8),
265                             decoration: BoxDecoration(
266                                 color: theme.primaryAccent,
267                                 borderRadius: BorderRadius.circular(8),
268                             ),
269                             child: Text(
270                                 '$slideNumber',
271                                 style: TextStyleManager.bold14.copyWith(color: Colors.white),
272                             ),
273                         ),
274                         const SizedBox(width: 12),
275                         Expanded(
276                             child: Text(
277                                 slide.title,
278                                 style: TextStyleManager.bold14.copyWith(color: Colors.white),
279                             ),
280                         ),
281                         const SizedBox(width: 12),
282                         Text(
283                             'عنوان الدرس',
284                             style: TextStyleManager.bold12.copyWith(color: theme.primaryAccent),
285                         ),
286                     ],
287                 ),
288             ),
289
290             // Image
291             Padding(
292                 padding: const EdgeInsets.all(20),
293                 child: Center(
294                     child: ClipRRect(
295                         borderRadius: BorderRadius.circular(8),
296                         child: Image.asset(
297                             coverImageAsset,
298                             height: 120,
299                             fit: BoxFit.contain,
300                         ),
297                     ),
301                 ),
302             ),
303         ],
304     ),
305
306     // Title
307     Padding(
308         padding: const EdgeInsets.symmetric(horizontal: 20),
309         child: Text(
310             slide.title,
311             textAlign: TextAlign.center,
312             style: TextStyleManager.bold18.copyWith(
313                 color: theme.darkAccent,
314                 height: 1.4,
315             ),
316         ),
317     ),
318
319     verticalSpace16,
320
321     // Body text

```

```

321         Padding(
322             padding: const EdgeInsets.fromLTRB(20, 0, 20, 20),
323             child: _buildSlideContent(slide, theme),
324         ),
325     ],
326 ),
327 ),
328 );
329 }
330
331 Widget _buildSlideCard(PresentationSlideModel slide, int slideNumber, PresentationThemeData theme) {
332     if (slide.slideType == 'title') {
333         return _buildCoverSlideCard(slide, slideNumber, theme);
334     }
335
336     if (['static_opening', 'static_greeting', 'static_dua'].contains(slide.slideType)) {
337         return _buildSpecialSlideCard(slide, slideNumber, theme);
338     }
339
340     // Determine the type label
341     String typeLabel = _getSlideLabel(slide.slideType);
342
343     return Container(
344         margin: const EdgeInsets.only(bottom: 24),
345         decoration: BoxDecoration(
346             color: theme.backgroundColor,
347             borderRadius: BorderRadius.circular(12),
348             boxShadow: [
349                 BoxShadow(
350                     color: Colors.black.withValues(alpha: 0.08),
351                     blurRadius: 15,
352                     offset: const Offset(0, 5),
353                 ),
354             ],
355         ),
356         child: ClipRRect(
357             borderRadius: BorderRadius.circular(12),
358             child: Column(
359                 crossAxisAlignment: CrossAxisAlignment.stretch,
360                 children: [
361                     // Top Accent Line
362                     Container(
363                         height: 4,
364                         color: theme.primaryAccent,
365                     ),
366
367                     // Header Section
368                     Padding(
369                         padding: const EdgeInsets.fromLTRB(20, 20, 20, 16),
370                         child: Row(
371                             crossAxisAlignment: CrossAxisAlignment.start,
372                             children: [
373                                 // Slide Number
374                                 Container(
375                                     padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 8),
376                                     decoration: BoxDecoration(
377                                         color: theme.primaryAccent,
378                                         borderRadius: BorderRadius.circular(8),
379                                     ),
380                                     child: Text(
381                                         '$slideNumber',
382                                         style: TextStyleManager.bold14.copyWith(color: Colors.white),
383                                     ),
384                                 ),
385                                 horizontalSpace12,
386                                 // Title
387                                 Expanded(
388                                     child: Text(
389                                         slide.title,
390                                         style: TextStyleManager.bold18.copyWith(
391                                             color: theme.darkAccent,
392                                             height: 1.4,
393                                         ),
394                                 ),
395                             ],
396                         ),
397                     ),
398                 ],
399             ),
400         ),
401     );
402 }

```

```

396         horizontalSpace12,
397         // Type Label
398         Padding(
399             padding: const EdgeInsets.only(top: 4),
400             child: Text(
401                 typeLabel,
402                 style: TextStyleManager.bold12.copyWith(color: theme.primaryAccent),
403             ),
404         ),
405         horizontalSpace12,
406         // Icon
407         Container(
408             padding: const EdgeInsets.all(8),
409             decoration: BoxDecoration(
410                 color: theme.primaryAccent,
411                 borderRadius: BorderRadius.circular(8),
412             ),
413             child: Icon(
414                 _getIconData(slide.iconKeyword),
415                 color: Colors.white,
416                 size: 20,
417             ),
418         ),
419     ],
420 ),
421 ),
422
423 // Content Section
424 Padding(
425     padding: const EdgeInsets.fromLTRB(20, 0, 20, 20),
426     child: _buildSlideContent(slide, theme),
427 ),
428 ],
429 ),
430 ),
431 );
432 }
433
434 Widget _buildSlideContent(PresentationSlideModel slide, PresentationThemeData theme) {
435     // Split bullet points by new line or bullet character
436     final lines = slide.bodyText.split('\n').where((l) => l.trim().isNotEmpty).toList();
437
438     if (slide.slideType == 'random_picker') {
439         return RandomPickerWidget(names: lines, theme: theme);
440     }
441
442     return Column(
443         crossAxisAlignment: CrossAxisAlignment.stretch,
444         children: lines.map((line) {
445             String text = line.trim();
446             if (text.startsWith('-') || text.startsWith('•')) {
447                 text = text.substring(1).trim();
448             }
449
450             return Container(
451                 margin: const EdgeInsets.only(bottom: 12),
452                 padding: const EdgeInsets.all(16),
453                 decoration: BoxDecoration(
454                     color: Colors.white,
455                     borderRadius: BorderRadius.circular(8),
456                     border: Border.all(color: theme.borderColor, width: 1.5),
457                 ),
458                 child: Row(
459                     crossAxisAlignment: CrossAxisAlignment.start,
460                     children: [
461                         Container(
462                             margin: const EdgeInsets.only(top: 4, left: 12), // RTL Support
463                             width: 8,
464                             height: 8,
465                             decoration: BoxDecoration(
466                                 color: theme.primaryAccent,
467                                 shape: BoxShape.circle,
468                             ),
469                         ),
470                         Expanded(

```

```

471         child: Text(
472           text,
473           style: TextStylesManager.bold14.copyWith(
474             color: theme.textColor,
475             height: 1.5,
476           ),
477         ),
478       ),
479     ],
480   ),
481 );
482   }).toList(),
483 );
484 }
485
486 Widget _buildSpecialSlideCard(PresentationSlideModel slide, int slideNumber, PresentationThemeData theme) {
487   String typeLabel = _getSlideLabel(slide.slideType);
488   String imagePath;
489   if (slide.slideType == 'static_opening') {
490     imagePath = AssetsHelper.presentation1;
491   } else if (slide.slideType == 'static_greeting') {
492     imagePath = AssetsHelper.presentation2;
493   } else {
494     imagePath = AssetsHelper.presentation3;
495   }
496
497   return Container(
498     margin: const EdgeInsets.only(bottom: 24),
499     padding: const EdgeInsets.all(16),
500     decoration: BoxDecoration(
501       color: theme.backgroundColor,
502       borderRadius: BorderRadius.circular(24),
503     ),
504     child: Column(
505       crossAxisAlignment: CrossAxisAlignment.stretch,
506       children: [
507         // Header
508         Row(
509           crossAxisAlignment: CrossAxisAlignment.start,
510           children: [
511             // Slide Number
512             Container(
513               width: 32,
514               height: 32,
515               decoration: BoxDecoration(
516                 color: theme.darkAccent,
517                 shape: BoxShape.circle,
518               ),
519               alignment: Alignment.center,
520               child: Text(
521                 '$slideNumber',
522                 style: TextStylesManager.bold14.copyWith(color: Colors.white),
523               ),
524             ),
525             const Spacer(),
526             // Subtitle & Title
527             Column(
528               crossAxisAlignment: CrossAxisAlignment.end,
529               children: [
530                 Text(
531                   typeLabel,
532                   style: TextStylesManager.bold12.copyWith(color: theme.primaryAccent),
533                 ),
534                 Text(
535                   slide.title,
536                   style: TextStylesManager.bold20.copyWith(color: theme.darkAccent),
537                 ),
538               ],
539             ),
540             horizontalSpace12,
541             // Icon
542             Container(
543               padding: const EdgeInsets.all(8),
544               decoration: BoxDecoration(
545                 color: theme.darkAccent,

```

```

546         borderRadius: BorderRadius.circular(12),
547     ),
548     child: Icon(
549         _getIconData(slide.iconKeyword),
550         color: Colors.white,
551         size: 20,
552     ),
553 ),
554 ],
555 ),
556
557 verticalSpace24,
558
559 // Body (Image + Overlapping Card)
560 SizedBox(
561     height: slide.slideType == 'static_dua' ? 240 : 250,
562     child: LayoutBuilder(
563         builder: (context, constraints) {
564             return Stack(
565                 children: [
566                     // Image on the Left (end in RTL)
567                     PositionedDirectional(
568                         end: 0,
569                         top: 0,
570                         bottom: slide.slideType == 'static_greeting' ? 40 : (slide.slideType == 'static_dua' ?
24 : 0),
571                         width: constraints.maxWidth * 0.65,
572                         child: ClipRRect(
573                             borderRadius: BorderRadius.circular(16),
574                             child: Image.asset(imagePath, fit: BoxFit.cover),
575                         ),
576                     ),
577
578                     // Dots for dua
579                     if (slide.slideType == 'static_dua')
580                         PositionedDirectional(
581                             end: 0,
582                             bottom: 0,
583                             height: 24,
584                             width: constraints.maxWidth * 0.65,
585                             child: Row(
586                                 mainAxisAlignment: MainAxisAlignment.center,
587                                 children: List.generate(12, (index) => Container(
588                                     margin: const EdgeInsets.symmetric(horizontal: 3),
589                                     width: 6,
590                                     height: 6,
591                                     decoration: BoxDecoration(
592                                         color: index == 0 ? theme.primaryAccent : theme.primaryAccent.withOpacity(0.3),
593                                         shape: BoxShape.circle,
594                                     ),
595                                 )),
596                     ),
597
598                     // White Card on the Right (start in RTL)
599                     PositionedDirectional(
600                         start: 0,
601                         top: slide.slideType == 'static_dua' ? 40 : 24,
602                         bottom: slide.slideType == 'static_dua' ? 40 : (slide.slideType == 'static_greeting' ?
64 : 24),
604                         width: constraints.maxWidth * 0.55,
605                         child: Container(
606                             decoration: BoxDecoration(
607                                 color: Colors.white,
608                                 borderRadius: BorderRadius.circular(16),
609                                 boxShadow: [
610                                     BoxShadow(
611                                         color: Colors.black.withOpacity(0.05),
612                                         blurRadius: 10,
613                                         offset: const Offset(0, 4),
614                                     ),
615                                 ],
616                             ),
617                             child: _buildSpecialSlideWhiteCardContent(slide, theme),
618                         ),

```

```

619         ),
620
621         // Chips for Greeting
622         if (slide.slideType == 'static_greeting')
623             PositionedDirectional(
624                 start: 0,
625                 bottom: 0,
626                 width: constraints.maxWidth * 0.55,
627                 child: Directionality(
628                     textDirection: TextDirection.rtl,
629                     child: Wrap(
630                         spacing: 8,
631                         runSpacing: 8,
632                         alignment: WrapAlignment.start,
633                         children: [
634                             if (widget.cubit.selectedSubject?.name != null)
635                                 _buildChip(widget.cubit.selectedSubject!.name, theme),
636                             if (widget.cubit.selectedGrade?.name != null)
637                                 _buildChip(widget.cubit.selectedGrade!.name, theme),
638                             if (widget.cubit.selectedUnit?.title != null)
639                                 _buildChip(widget.cubit.selectedUnit!.title, theme),
640                         ],
641                     ),
642                 ),
643             ),
644         ],
645     );
646 },
647 ),
648 ),
649 ],
650 ),
651 );
652 }
653
654 Widget _buildChip(String label, PresentationThemeData theme) {
655     if (label.isEmpty) return const SizedBox.shrink();
656     return Container(
657         padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
658         decoration: BoxDecoration(
659             color: Colors.white,
660             borderRadius: BorderRadius.circular(20),
661             border: Border.all(color: theme.borderColor),
662         ),
663         child: Text(
664             label,
665             style: TextStyleManager.bold12.copyWith(color: theme.darkAccent),
666         ),
667     );
668 }
669
670 Widget _buildSpecialSlideWhiteCardContent(PresentationSlideModel slide, PresentationThemeData theme) {
671     final lines = slide.bodyText.split('\n').where((l) => l.trim().isNotEmpty).toList();
672
673     if (slide.slideType == 'static_opening') {
674         return Padding(
675             padding: const EdgeInsets.all(16),
676             child: Column(
677                 crossAxisAlignment: CrossAxisAlignment.start,
678                 children: [
679                     Align(
680                         alignment: Alignment.centerRight,
681                         child: Text(
682                             'نبدأ باسم الله',
683                             style: TextStyleManager.bold14.copyWith(color: theme.primaryAccent),
684                         ),
685                     ),
686                     const SizedBox(height: 12),
687                     Expanded(
688                         child: ListView.builder(
689                             physics: const NeverScrollableScrollPhysics(),
690                             itemCount: lines.length,
691                             itemBuilder: (context, index) {
692                                 String lineText = lines[index].trim();
693                                 if (lineText.startsWith('-') || lineText.startsWith('.')) {

```



```

694         lineText = lineText.substring(1).trim();
695     }
696     return Container(
697         margin: const EdgeInsets.only(bottom: 8),
698         padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 12),
699         decoration: BoxDecoration(
700             border: Border.all(color: theme.borderColor),
701             borderRadius: BorderRadius.circular(12),
702         ),
703         child: Row(
704             children: [
705                 Expanded(
706                     child: Text(
707                         lineText,
708                         style: TextStyleManager.bold14.copyWith(color: theme.darkAccent),
709                     ),
710                 ),
711                 const SizedBox(width: 8),
712                 Container(
713                     width: 8,
714                     height: 8,
715                     decoration: BoxDecoration(
716                         color: theme.primaryAccent,
717                         shape: BoxShape.circle,
718                     ),
719                 ),
720             ],
721         ),
722     );
723 },
724 ),
725 ),
726 ],
727 ),
728 );
729 } else if (slide.slideType == 'static_greeting') {
730     return Padding(
731         padding: const EdgeInsets.all(16),
732         child: Column(
733             mainAxisAlignment: MainAxisAlignment.center,
734             children: lines.asMap().entries.map((entry) {
735                 int index = entry.key;
736                 String lineText = entry.value.trim();
737                 if (lineText.startsWith('-') || lineText.startsWith('•')) {
738                     lineText = lineText.substring(1).trim();
739                 }
740                 return Container(
741                     margin: const EdgeInsets.only(bottom: 12),
742                     padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 12),
743                     decoration: BoxDecoration(
744                         border: Border.all(color: theme.borderColor),
745                         borderRadius: BorderRadius.circular(12),
746                     ),
747                     child: Row(
748                         children: [
749                             Container(
750                                 width: 24,
751                                 height: 24,
752                                 decoration: BoxDecoration(
753                                     color: theme.primaryAccent,
754                                     shape: BoxShape.circle,
755                                 ),
756                                 alignment: Alignment.center,
757                                 child: Text(
758                                     '${index + 1}',
759                                     style: TextStyleManager.bold12.copyWith(color: Colors.white),
760                                 ),
761                             ),
762                             const SizedBox(width: 12),
763                             Expanded(
764                                 child: Text(
765                                     lineText,
766                                     style: TextStyleManager.bold14.copyWith(color: theme.darkAccent),
767                                 ),
768                             ),

```

```

769         ],
770     ),
771 );
772 }).toList(),
773 ),
774 );
775 } else if (slide.slideType == 'static_dua') {
776     return Padding(
777         padding: const EdgeInsets.all(20),
778         child: SingleChildScrollView(
779             child: Column(
780                 crossAxisAlignment: CrossAxisAlignment.start,
781                 mainAxisAlignment: MainAxisAlignment.center,
782                 children: [
783                     Align(
784                         alignment: Alignment.centerRight,
785                         child: Text(
786                             'دعاء قبل التعلم',
787                             style: TextStyleManager.bold14.copyWith(color: theme.primaryAccent),
788                         ),
789                     ),
790                     const SizedBox(height: 16),
791                     Text(
792                         slide.bodyText.replaceAll('-', ' ').replaceAll('•', ' '),
793                         style: TextStyleManager.bold16.copyWith(
794                             color: theme.darkAccent,
795                             height: 1.8,
796                         ),
797                         textAlign: TextAlign.center,
798                     ),
799                 ],
800             ),
801         ),
802     );
803 }
804 return const SizedBox.shrink();
805 }
806
807 String _getSlideLabel(String? type) {
808     try {
809         final Map<String, String> labels = {
810             'title': 'عنوان الدرس',
811             'static_opening': 'افتتاحية',
812             'static_greeting': 'ترحيب',
813             'static_dua': 'دعاء اليوم',
814             'static_classroom_rules': 'قواعد الصف',
815             'static_success_criteria': 'محطات النجاح',
816             'lesson_info': 'معلومات الدرس',
817             'kwl_open': 'ماذا نعرف',
818             'vocabulary': 'مفردات الدرس',
819             'hook': 'تمهيد الدرس',
820             'random_picker': 'اختيار عشوائي',
821             'content': 'شرح الدرس',
822             'cross_curricular': 'الربط بالمواد الأخرى',
823             'reading_stages': 'مراحل القراءة',
824             'closing_strategy': 'استراتيجية الإغلاق',
825             'worksheet': 'ورقة عمل',
826             'kwl_close': 'ماذا تعلمنا',
827             'homework': 'الواجب المنزلي',
828             'objectives': 'أهداف التعلم',
829             'example': 'مثال تطبيقي',
830             'summary': 'ملخص الدرس',
831             'quiz_prompt': 'أسئلة ومناقشة',
832         };
833         return labels[type] ?? 'محتوى تعليمي';
834     } catch (e) {
835         print('Error in _getSlideLabel for type $type: $e');
836         return 'محتوى تعليمي';
837     }
838 }
839
840 IconData _getIconData(String? keyword) {
841     if (keyword == null) return LucideIcons.layoutTemplate;
842     final k = keyword.toLowerCase().replaceAll(' ', '_').replaceAll('-', '_');
843     switch (k) {

```

```

844     case 'lightbulb': return LucideIcons.lightbulb;
845     case 'book':
846     case 'book_open': return LucideIcons.bookOpen;
847     case 'target': return LucideIcons.target;
848     case 'users': return LucideIcons.users;
849     case 'message_circle': return LucideIcons.messageCircle;
850     case 'check_circle': return LucideIcons.checkCircle;
851     case 'puzzle': return LucideIcons.puzzle;
852     case 'activity': return LucideIcons.activity;
853     case 'star': return LucideIcons.star;
854     case 'award': return LucideIcons.award;
855     case 'file_text': return LucideIcons.fileText;
856     case 'play_circle': return LucideIcons.playCircle;
857     case 'help_circle': return LucideIcons.helpCircle;
858     case 'info': return LucideIcons.info;
859     case 'video': return LucideIcons.video;
860     case 'mic': return LucideIcons.mic;
861     case 'ear': return LucideIcons.ear;
862     case 'eye': return LucideIcons.eye;
863     case 'pen_tool': return LucideIcons.penTool;
864     case 'graduation_cap': return LucideIcons.graduationCap;
865     case 'sparkles': return LucideIcons.sparkles;
866     default: return LucideIcons.layoutTemplate;
867   }
868 }
869 }
870
871 class RandomPickerWidget extends StatefulWidget {
872   final List<String> names;
873   final PresentationThemeData theme;
874
875   const RandomPickerWidget({
876     Key? key,
877     required this.names,
878     required this.theme,
879   }) : super(key: key);
880
881   @override
882   State<RandomPickerWidget> createState() => _RandomPickerWidgetState();
883 }
884
885 class _RandomPickerWidgetState extends State<RandomPickerWidget> {
886   int? _highlightedIndex;
887   int? _selectedIndex;
888   bool _isSpinning = false;
889
890   @override
891   void initState() {
892     super.initState();
893     Future.delayed(const Duration(milliseconds: 500), () {
894       if (mounted) _spin();
895     });
896   }
897
898   void _spin() async {
899     if (_isSpinning || widget.names.isEmpty) return;
900     setState(() {
901       _isSpinning = true;
902       _selectedIndex = null;
903       _highlightedIndex = null;
904     });
905
906     final random = Random();
907     final targetIndex = random.nextInt(widget.names.length);
908
909     // Total steps to take
910     int steps = widget.names.length * 3 + targetIndex;
911
912     for (int i = 0; i < steps; i++) {
913       if (!mounted) return;
914       setState(() {
915         _highlightedIndex = i % widget.names.length;
916       });
917
918       // Calculate delay: starts fast (50ms), slows down near the end (up to 400ms)

```

```

919     int delay = 50 + (i / steps * 200).toInt();
920     if (i > steps - 5) delay += 100; // Extra slow at the very end
921
922     await Future.delayed(Duration(milliseconds: delay));
923   }
924
925   if (!mounted) return;
926   setState(() {
927     _isSpinning = false;
928     _selectedIndex = targetIndex;
929     _highlightedIndex = targetIndex;
930   });
931 }
932
933 @override
934 Widget build(BuildContext context) {
935   return Column(
936     crossAxisAlignment: CrossAxisAlignment.stretch,
937     children: [
938       Wrap(
939         spacing: 8,
940         runSpacing: 8,
941         children: List.generate(widget.names.length, (index) {
942           final name = widget.names[index];
943           final isHighlighted = _highlightedIndex == index;
944           final isSelected = _selectedIndex == index;
945
946           return AnimatedContainer(
947             duration: const Duration(milliseconds: 200),
948             padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
949             decoration: BoxDecoration(
950               color: isSelected
951                 ? widget.theme.primaryAccent
952                 : isHighlighted
953                   ? widget.theme.primaryAccent.withOpacity(0.5)
954                   : widget.theme.softContainer,
955               borderRadius: BorderRadius.circular(8),
956               border: Border.all(
957                 color: isHighlighted || isSelected
958                   ? widget.theme.primaryAccent
959                   : widget.theme.primaryAccent.withOpacity(0.3),
960                 width: isHighlighted || isSelected ? 2 : 1,
961               ),
962               boxShadow: isSelected ? [
963                 BoxShadow(
964                   color: widget.theme.primaryAccent.withOpacity(0.4),
965                   blurRadius: 8,
966                   spreadRadius: 1,
967                 )
968               ] : null,
969             ),
970             child: Text(
971               name.trim(),
972               style: TextStyleManager.bold14.copyWith(
973                 color: isSelected || isHighlighted ? Colors.white : widget.theme.darkAccent,
974               ),
975             ),
976           );
977         }],
978       ),
979     ],
980   );
981 }
982
983
984 class GreetingWidget extends StatelessWidget {
985   final String text;
986   final PresentationThemeData theme;
987   final PresentationsCubit cubit;
988
989   const GreetingWidget({
990     Key? key,
991     required this.text,
992     required this.theme,
993     required this.cubit,

```

```

994    }) : super(key: key);
995
996    @override
997    Widget build(BuildContext context) {
998      final lines = text.isEmpty
999        ? [
1000          'نبدأ يومنا بطاقة إيجابية وحماس للتعليم',
1001          'نتمنى لكم درسًا ممتعًا ومليئًا بالإنجاز.'
1002        ]
1003        : text.split('\n').where((l) => l.trim().isNotEmpty).toList();
1004
1005      return Column(
1006        crossAxisAlignment: CrossAxisAlignment.stretch,
1007        children: [
1008          // Left side in mockup: Image
1009          Container(
1010            height: 180,
1011            decoration: BoxDecoration(
1012              borderRadius: BorderRadius.circular(12),
1013              border: Border.all(color: theme.primaryAccent, width: 2),
1014              image: const DecorationImage(
1015                image: AssetImage(AssetsHelper.scr16),
1016                fit: BoxFit.cover,
1017              ),
1018            ),
1019            alignment: Alignment.bottomCenter,
1020            child: Container(
1021              width: double.infinity,
1022              padding: const EdgeInsets.symmetric(vertical: 12),
1023              decoration: BoxDecoration(
1024                borderRadius: const BorderRadius.vertical(bottom: Radius.circular(10)),
1025                color: theme.primaryAccent.withValues(alpha: 0.8),
1026              ),
1027              child: Column(
1028                mainAxisAlignment: MainAxisAlignment.min,
1029                children: [
1030                  Text(
1031                    'التعليم في المملكة العربية السعودية',
1032                    style: TextStyleManager.bold14.copyWith(color: Colors.white),
1033                  ),
1034                ],
1035              ),
1036            ),
1037          ),
1038          const SizedBox(height: 16),
1039          // Right side in mockup: Bullets
1040          ...List.generate(lines.length, (index) {
1041            String lineText = lines[index].trim();
1042            if (lineText.startsWith('-') || lineText.startsWith('•')) {
1043              lineText = lineText.substring(1).trim();
1044            }
1045            return Container(
1046              margin: const EdgeInsets.only(bottom: 12),
1047              padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
1048              decoration: BoxDecoration(
1049                color: Colors.white,
1050                borderRadius: BorderRadius.circular(12),
1051                border: Border.all(color: theme.borderColor, width: 1.5),
1052              ),
1053              child: Row(
1054                children: [
1055                  Container(
1056                    width: 24,
1057                    height: 24,
1058                    decoration: BoxDecoration(
1059                      color: theme.primaryAccent,
1060                      shape: BoxShape.circle,
1061                    ),
1062                    alignment: Alignment.center,
1063                    child: Text(
1064                      '${index + 1}',
1065                      style: TextStyleManager.bold12.copyWith(color: Colors.white),
1066                    ),
1067                  ),
1068                  const SizedBox(width: 12),

```

```

1069         Expanded(
1070             child: Text(
1071                 lineHeight,
1072                 style: TextStyleManager.bold14.copyWith(color: theme.darkAccent, height: 1.5),
1073             ),
1074         ),
1075     ],
1076 ),
1077 );
1078 },
1079 const SizedBox(height: 8),
1080 // Bottom tags
1081 Wrap(
1082     spacing: 8,
1083     runSpacing: 8,
1084     alignment: WrapAlignment.center,
1085     children: [
1086         if (cubit.selectedSubject?.name != null) _buildTag(cubit.selectedSubject!.name, theme),
1087         if (cubit.selectedGrade?.name != null) _buildTag(cubit.selectedGrade!.name, theme),
1088         if (cubit.selectedUnit?.title != null) _buildTag(cubit.selectedUnit!.title, theme),
1089     ],
1090 ),
1091 ],
1092 );
1093 }
1094
1095 Widget _buildTag(String label, PresentationThemeData theme) {
1096     if (label.isEmpty) return const SizedBox.shrink();
1097     return Container(
1098         padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
1099         decoration: BoxDecoration(
1100             color: Colors.white,
1101             borderRadius: BorderRadius.circular(16),
1102             border: Border.all(color: theme.borderColor),
1103         ),
1104         child: Text(
1105             label,
1106             style: TextStyleManager.bold12.copyWith(color: theme.primaryAccent),
1107         ),
1108     );
1109 }
1110 }
1111

```

Lines • 532
Dart

Feature / #278 –
Presentations [lib/features/presentations/presentation/screens/presentations_screen.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
7  import 'package:moean/core/utils/constants/spacing.dart';
8
9  import 'package:moean/features/presentations/presentation/cubit/presentations_cubit.dart';
10 import 'package:moean/features/presentations/presentation/cubit/presentations_state.dart';
11 import 'package:moean/features/reports/presentation/widgets/selection_bottom_sheet.dart';
12
13 import 'package:moean/core/utils/constants/primary/upgrade_prompt_bottom_sheet.dart';
14 import 'presentation_preview_screen.dart';
15
16 class PresentationsScreen extends StatefulWidget {
17     const PresentationsScreen({super.key});
18
19     @override
20     State<PresentationsScreen> createState() => _PresentationsScreenState();
21 }
22
23 class _PresentationsScreenState extends State<PresentationsScreen> {
24     void _showSelector(String title, List<String> items, Function(String) onSelect) {
25         SelectionBottomSheet.show(
26             context: context,

```

```

27     title: title,
28     items: items,
29     initialSelectedItems: [],
30     isMultiSelect: false,
31     onSelectionConfirmed: (selected) {
32       if (selected.isNotEmpty) {
33         onSelected(selected.first);
34       }
35     },
36   );
37 }
38
39 @override
40 Widget build(BuildContext context) {
41   return Scaffold(
42     backgroundColor: ColorsManager.background,
43     appBar: AppBar(
44       backgroundColor: ColorsManager.background,
45       elevation: 0,
46       centerTitle: true,
47       title: Text(
48         appTranslation().get('presentations_title'),
49         style: TextStylesManager.bold18.copyWith(color: ColorsManager.mainText),
50       ),
51       leading: IconButton(
52         icon: Icon(Icons.arrow_back_ios, color: ColorsManager.mainText),
53         onPressed: () => Navigator.of(context).pop(),
54       ),
55     ),
56     body: BlocConsumer<PresentationsCubit, PresentationsState>(
57       listener: (context, state) {
58         if (state is PresentationsSuccess) {
59           final cubit = context.read<PresentationsCubit>();
60           Navigator.push(
61             context,
62             MaterialPageRoute(
63               builder: (context) => PresentationPreviewScreen(
64                 presentation: state.presentation,
65                 cubit: cubit,
66               ),
67             ),
68           );
69         } else if (state is PresentationsPaymentRequired) {
70           UpgradePromptBottomSheet.show(
71             context,
72             message: state.message,
73             isQuotaExceeded: state.code == 'quota_exceeded',
74           );
75         } else if (state is PresentationsError) {
76           ScaffoldMessenger.of(context).showSnackBar(
77             SnackBar(content: Text(state.message), backgroundColor: Colors.red),
78           );
79         }
80       },
81       builder: (context, state) {
82         final cubit = context.read<PresentationsCubit>();
83         final isWide = MediaQuery.of(context).size.width > 800;
84
85         return SingleChildScrollView(
86           padding: const EdgeInsets.all(24.0),
87           child: Column(
88             crossAxisAlignment: CrossAxisAlignment.stretch,
89             children: [
90               _buildHeader(),
91               verticalSpace32,
92               if (isWide)
93                 Row(
94                   crossAxisAlignment: CrossAxisAlignment.start,
95                   children: [
96                     Expanded(
97                       flex: 5,
98                       child: _buildForm(cubit),
99                     ),
100                    horizontalSpace24,
101                    Expanded(

```

```

102         flex: 7,
103         child: _buildPreviewArea(state, cubit),
104       ),
105     ],
106   )
107   else ...[
108     _buildForm(cubit),
109     verticalSpace32,
110     _buildPreviewArea(state, cubit),
111   ],
112 ],
113 ),
114 );
115 },
116 ),
117 );
118 }
119
120 Widget _buildHeader() {
121   return Column(
122     crossAxisAlignment: CrossAxisAlignment.center,
123     children: [
124       Text(
125         appTranslation().get('presentations_subtitle'),
126         style: TextStyleManager.bold26.copyWith(color: ColorsManager.primaryColor),
127         textAlign: TextAlign.center,
128       ),
129       verticalSpace12,
130       Text(
131         appTranslation().get('presentations_desc'),
132         style: TextStyleManager.regular16.copyWith(color: ColorsManager.secondaryText),
133         textAlign: TextAlign.center,
134       ),
135     ],
136   );
137 }
138
139 Widget _buildForm(PresentationsCubit cubit) {
140   return Container(
141     padding: const EdgeInsets.all(24),
142     decoration: BoxDecoration(
143       color: ColorsManager.surfacePrimary,
144       borderRadius: BorderRadius.circular(24),
145       border: Border.all(color: ColorsManager.primaryColor.withValues(alpha: 0.1)),
146     ),
147     child: Column(
148       crossAxisAlignment: CrossAxisAlignment.start,
149       children: [
150         Row(
151           mainAxisAlignment: MainAxisAlignment.spaceBetween,
152           children: [
153             Column(
154               crossAxisAlignment: CrossAxisAlignment.start,
155               children: [
156                 Text(
157                   appTranslation().get('presentations_data'),
158                   style: TextStyleManager.bold12.copyWith(color: ColorsManager.secondaryText),
159                 ),
160                 Text(
161                   appTranslation().get('presentations_choose_lesson_design'),
162                   style: TextStyleManager.bold20.copyWith(color: ColorsManager.primaryColor),
163                 ),
164               ],
165             ),
166             Container(
167               padding: const EdgeInsets.all(8),
168               decoration: BoxDecoration(
169                 color: ColorsManager.primaryColor.withValues(alpha: 0.1),
170                 shape: BoxShape.circle,
171               ),
172               child: Icon(Icons.tune, color: ColorsManager.primaryColor, size: 24),
173             ),
174           ],
175         ),
176         verticalSpace24,

```



```

177     _buildDropdownField(
178       label: appTranslation().get('presentations_stage'),
179       value: cubit.selectedStage?.name,
180       onTap: () {
181         if (cubit.stages.isEmpty) return;
182         final sortedStages = List.of(cubit.stages)..sort((a, b) {
183           int getWeight(String name) {
184             if (name.contains('الابتدائية')) return 1;
185             if (name.contains('المتوسطة')) return 2;
186             if (name.contains('الثانوية')) return 3;
187             return 4;
188           }
189           return getWeight(a.name).compareTo(getWeight(b.name));
190         });
191         _showSelector(
192           appTranslation().get('presentations_stage'),
193           sortedStages.map((e) => e.name).toList(),
194           (v) {
195             final selected = sortedStages.firstWhere((e) => e.name == v);
196             cubit.updateSelection(stage: selected);
197           },
198         );
199       },
200     ),
201     if (cubit.availableTracks.isNotEmpty) ...[
202       verticalSpacel6,
203       _buildDropdownField(
204         label: 'المسار',
205         value: cubit.selectedTrack,
206         onTap: () {
207           _showSelector(
208             'المسار',
209             cubit.availableTracks,
210             (v) {
211               cubit.updateSelection(track: v);
212             },
213           );
214         },
215       ),
216     ],
217     if (cubit.selectedStage != null && (cubit.availableTracks.isEmpty || cubit.selectedTrack != null))
218     ...[
219       verticalSpacel6,
220       _buildDropdownField(
221         label: appTranslation().get('presentations_grade'),
222         value: cubit.selectedGrade?.name,
223         onTap: () {
224           if (cubit.selectedStage == null) return;
225           _showSelector(
226             appTranslation().get('presentations_grade'),
227             cubit.availableGrades.map((e) => e.name).toList(),
228             (v) {
229               final selected = cubit.availableGrades.firstWhere((e) => e.name == v);
230               cubit.updateSelection(grade: selected);
231             },
232           );
233         },
234       ],
235       verticalSpacel6,
236       _buildDropdownField(
237         label: appTranslation().get('presentations_subject'),
238         value: cubit.selectedSubject?.name,
239         onTap: () {
240           if (cubit.selectedGrade == null) return;
241           _showSelector(
242             appTranslation().get('presentations_subject'),
243             cubit.selectedGrade!.subjects.map((e) => e.name).toList(),
244             (v) {
245               final selected = cubit.selectedGrade!.subjects.firstWhere((e) => e.name == v);
246               cubit.updateSelection(subject: selected);
247             },
248           );
249         },
250       ),

```

```

251     verticalSpace16,
252     _buildDropdownField(
253       label: appTranslation().get('presentations_unit'),
254       value: cubit.selectedUnit?.title,
255       onTap: () {
256         if (cubit.subjectDetails == null) return;
257         _showSelector(
258           appTranslation().get('presentations_unit'),
259           cubit.subjectDetails!.chapters.map((e) => e.title).toList(),
260           (v) {
261             final selected = cubit.subjectDetails!.chapters.firstWhere((e) => e.title == v);
262             cubit.updateSelection(unit: selected);
263           },
264         );
265       },
266     ),
267     verticalSpace16,
268     _buildDropdownField(
269       label: appTranslation().get('presentations_lesson'),
270       value: cubit.selectedLesson?.title,
271       onTap: () {
272         if (cubit.selectedUnit == null) return;
273         _showSelector(
274           appTranslation().get('presentations_lesson'),
275           cubit.selectedUnit!.lessons.map((e) => e.title).toList(),
276           (v) {
277             final selected = cubit.selectedUnit!.lessons.firstWhere((e) => e.title == v);
278             cubit.updateSelection(lesson: selected);
279           },
280         );
281       },
282     ),
283     verticalSpace24,
284     Text(
285       appTranslation().get('presentations_template'),
286       style: TextStyleManager.bold16.copyWith(color: ColorsManager.mainText),
287     ),
288     verticalSpace12,
289     _buildTemplateOption(
290       cubit,
291       title: appTranslation().get('presentations_classic'),
292       desc: appTranslation().get('presentations_classic_desc'),
293       color: Colors.teal,
294       isSelected: cubit.selectedTemplate == 'emerald-green',
295       onTap: () => cubit.updateSelection(template: 'emerald-green'),
296     ),
297     verticalSpace8,
298     _buildTemplateOption(
299       cubit,
300       title: appTranslation().get('presentations_academic'),
301       desc: appTranslation().get('presentations_academic_desc'),
302       color: Colors.blue.shade700,
303       isSelected: cubit.selectedTemplate == 'academic-blue',
304       onTap: () => cubit.updateSelection(template: 'academic-blue'),
305     ),
306     verticalSpace8,
307     _buildTemplateOption(
308       cubit,
309       title: appTranslation().get('presentations_warm'),
310       desc: appTranslation().get('presentations_warm_desc'),
311       color: Colors.orange.shade400,
312       isSelected: cubit.selectedTemplate == 'warm-orange',
313       onTap: () => cubit.updateSelection(template: 'warm-orange'),
314     ),
315     verticalSpace24,
316     Text(
317       appTranslation().get('presentations_slides_count'),
318       style: TextStyleManager.bold16.copyWith(color: ColorsManager.mainText),
319     ),
320     verticalSpace12,
321     Row(
322       children: [
323         Expanded(
324           child: _buildSlideCountOption(
325             cubit,

```

```

326         count: 'عدد الشرائح تلقائي',
327         isSelected: cubit.selectedSlidesCount == '10',
328         onTap: () => cubit.updateSelection(slidesCount: '10'),
329     ),
330 ),
331 ],
332 ),
333 verticalSpace32,
334 PrimaryElevatedButton(
335     text: appTranslation().get('presentations_create'),
336     onPressed: cubit.canCreate ? () => cubit.createPresentation() : null,
337     icon: const Icon(Icons.auto_awesome),
338     isLoading: cubit.state is PresentationsLoading,
339 ),
340 ],
341 ),
342 );
343 }
344
345 Widget _buildDropdownField({required String label, required String? value, required VoidCallback onTap}) {
346     return Column(
347         crossAxisAlignment: CrossAxisAlignment.start,
348         children: [
349             Text(label, style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainText)),
350             verticalSpace8,
351             InkWell(
352                 onTap: onTap,
353                 borderRadius: BorderRadius.circular(12),
354                 child: Container(
355                     padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
356                     decoration: BoxDecoration(
357                         border: Border.all(color: ColorsManager.borderColor),
358                         borderRadius: BorderRadius.circular(12),
359                         color: ColorsManager.surfacePrimary,
360                     ),
361                     child: Row(
362                         mainAxisAlignment: MainAxisAlignment.spaceBetween,
363                         children: [
364                             Expanded(
365                                 child: Text(
366                                     value ?? 'اختر...',
367                                     style: value == null
368                                         ? TextStylesManager.regular14.copyWith(color: ColorsManager.secondaryText)
369                                         : TextStylesManager.bold14.copyWith(color: ColorsManager.mainText),
370                                 ),
371                             ),
372                             Icon(Icons.keyboard_arrow_down, color: ColorsManager.mainText),
373                         ],
374                     ),
375                 ),
376             ),
377         ],
378     );
379 }
380
381 Widget _buildTemplateOption(PresentationsCubit cubit, {required String title, required String desc, require
d Color color, required bool isSelected, required VoidCallback onTap}) {
382     return InkWell(
383         onTap: onTap,
384         borderRadius: BorderRadius.circular(16),
385         child: Container(
386             padding: const EdgeInsets.all(16),
387             decoration: BoxDecoration(
388                 color: isSelected ? color.withValues(alpha: 0.1) : Colors.transparent,
389                 border: Border.all(color: isSelected ? color : ColorsManager.borderColor, width: isSelected ? 2 :
1),
390                 borderRadius: BorderRadius.circular(16),
391             ),
392             child: Row(
393                 children: [
394                     if (isSelected)
395                         Icon(Icons.check_circle, color: color)
396                     else
397                         const SizedBox(width: 24, height: 24),
398                     horizontalSpace12,

```

```

399         Expanded(
400             child: Column(
401                 crossAxisAlignment: CrossAxisAlignment.start,
402                 children: [
403                     Text(title, style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainText)),
404                     verticalSpace4,
405                     Text(desc, style: TextStylesManager.regular12.copyWith(color: ColorsManager.secondaryTex
t)),
406                 ],
407             ),
408         ),
409         Container(
410             width: 50,
411             height: 36,
412             decoration: BoxDecoration(
413                 color: color,
414                 borderRadius: BorderRadius.circular(8),
415             ),
416             child: const Center(
417                 child: Icon(Icons.horizontal_split, color: Colors.white, size: 20),
418             ),
419         ),
420     ],
421 ),
422 ),
423 );
424 }
425
426 Widget _buildSlideCountOption(PresentationsCubit cubit, {required String count, required bool isSelected, r
equired VoidCallback onTap}) {
427     return InkWell(
428         onTap: onTap,
429         borderRadius: BorderRadius.circular(12),
430         child: Container(
431             padding: const EdgeInsets.symmetric(vertical: 14),
432             decoration: BoxDecoration(
433                 color: isSelected ? ColorsManager.primaryColor : ColorsManager.surfacePrimary,
434                 border: Border.all(color: isSelected ? ColorsManager.primaryColor : ColorsManager.borderColor),
435                 borderRadius: BorderRadius.circular(12),
436             ),
437             alignment: Alignment.center,
438             child: Text(
439                 count,
440                 style: TextStylesManager.bold14.copyWith(color: isSelected ? Colors.white : ColorsManager.mainTex
t),
441             ),
442         ),
443     );
444 }
445 Widget _buildPreviewArea(PresentationsState state, PresentationsCubit cubit) {
446     if (state is PresentationsSuccess) {
447         return const SizedBox.shrink();
448     } else if (state is PresentationsLoading) {
449         return Container(
450             height: 300,
451             decoration: BoxDecoration(
452                 color: ColorsManager.surfacePrimary,
453                 borderRadius: BorderRadius.circular(32),
454                 border: Border.all(color: ColorsManager.primaryColor.withValues(alpha: 0.1)),
455             ),
456             child: const Center(
457                 child: CircularProgressIndicator(),
458             ),
459         );
460     } else {
461         final selectedTemplateName = cubit.selectedTemplate == 'academic-blue'
462             ? appTranslation().get('presentations_academic')
463             : cubit.selectedTemplate == 'warm-orange'
464             ? appTranslation().get('presentations_warm')
465             : appTranslation().get('presentations_classic');
466
467         final details = [
468             if (cubit.selectedSubject != null) cubit.selectedSubject!.name,
469             if (cubit.selectedGrade != null) cubit.selectedGrade!.name,
470             selectedTemplateName,

```

```

471 ];
472 final detailsText = details.isNotEmpty ? details.join(' • ') : 'الرياضيات • الصف الأول المتوسط • حضر الكلاسيسيكي';
473
474 return Container(
475   padding: const EdgeInsets.all(40),
476   decoration: BoxDecoration(
477     color: ColorsManager.surfacePrimary,
478     borderRadius: BorderRadius.circular(32),
479     border: Border.all(color: ColorsManager.primaryColor.withValues(alpha: 0.1)),
480   ),
481   child: Column(
482     children: [
483       Align(
484         alignment: Alignment.topRight,
485         child: Column(
486           crossAxisAlignment: CrossAxisAlignment.start,
487           children: [
488             Text(
489               appTranslation().get('presentations_preview'),
490               style: TextStylesManager.bold12.copyWith(color: ColorsManager.secondaryText),
491             ),
492             Text(
493               cubit.selectedLesson?.title ?? 'التهيئة',
494               style: TextStylesManager.bold24.copyWith(color: ColorsManager.primaryColor),
495             ),
496             Text(
497               detailsText,
498               style: TextStylesManager.regular12.copyWith(color: ColorsManager.secondaryText),
499             ),
500           ],
501         ),
502       ),
503       verticalSpace40,
504       Container(
505         padding: const EdgeInsets.all(24),
506         decoration: BoxDecoration(
507           color: ColorsManager.primaryColor.withValues(alpha: 0.05),
508           shape: BoxShape.circle,
509         ),
510         child: Icon(Icons.co_present, color: ColorsManager.primaryColor, size: 40),
511       ),
512       verticalSpace24,
513       Text(
514         appTranslation().get('presentations_no_saved'),
515         style: TextStylesManager.bold20.copyWith(color: ColorsManager.primaryColor),
516         textAlign: TextAlign.center,
517       ),
518       verticalSpace12,
519       Text(
520         appTranslation().get('presentations_create_to_preview'),
521         style: TextStylesManager.regular14.copyWith(color: ColorsManager.secondaryText),
522         textAlign: TextAlign.center,
523       ),
524       verticalSpace40,
525     ],
526   ),
527 );
528 }
529 }
530
531 }
532

```

Línes • 225
Dart

Feature / Privacy **#279** —
Policy

lib/features/privacy_policy/presentation/cubit/privacy_policy_cubit.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter/services.dart';
3 import 'package:flutter_bloc/flutter_bloc.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/features/privacy_policy/presentation/cubit/privacy_policy_state.dart';
6 import 'package:pdf/pdf.dart';

```

```

7 import 'package:pdf/widgets.dart' as pw;
8 import 'package:printing/printing.dart';
9 import 'package:share_plus/share_plus.dart';
10
11 class PrivacyPolicyCubit extends Cubit<PrivacyPolicyState> {
12   PrivacyPolicyCubit() : super(const PrivacyPolicyInitial());
13
14   static PrivacyPolicyCubit get(BuildContext context) =>
15     BlocProvider.of(context);
16
17   final ScrollController scrollController = ScrollController();
18   final TextEditingController searchController = TextEditingController();
19
20   final Map<int, GlobalKey> sectionKeys = {
21     0: GlobalKey(),
22     1: GlobalKey(),
23     2: GlobalKey(),
24     3: GlobalKey(),
25     4: GlobalKey(),
26     5: GlobalKey(),
27     6: GlobalKey(),
28     7: GlobalKey(),
29     8: GlobalKey(),
30   };
31
32   int? activeSection = 0;
33   String searchQuery = '';
34
35   void setActiveSection(int index) {
36     activeSection = index;
37     if (!isClosed) {
38       emit(PrivacyPolicyStateUpdated(
39         activeSection: activeSection,
40         searchQuery: searchQuery,
41       ));
42     }
43   }
44
45   void onSearchChanged(String query) {
46     searchQuery = query.trim().toLowerCase();
47     if (!isClosed) {
48       emit(PrivacyPolicyStateUpdated(
49         activeSection: activeSection,
50         searchQuery: searchQuery,
51       ));
52     }
53   }
54
55   void scrollToSection(int index) {
56     setActiveSection(index);
57     final key = sectionKeys[index];
58     if (key?.currentContext != null) {
59       Scrollable.ensureVisible(
60         key!.currentContext!,
61         duration: const Duration(milliseconds: 450),
62         curve: Curves.easeInOutCubic,
63       );
64     }
65   }
66
67   Future<void> shareLink() async {
68     final text = appTranslation().get('privacy_share_text');
69     await SharePlus.instance.share(ShareParams(text: text));
70   }
71
72   Future<void> printPdf() async {
73     if (!isClosed) emit(const PrivacyPolicyPdfGenerating());
74     try {
75       final doc = pw.Document();
76       final fontData =
77         await rootBundle.load('assets/fonts/Tajawal-Regular.ttf');
78       final boldFontData =
79         await rootBundle.load('assets/fonts/Tajawal-Bold.ttf');
80       final font = pw.Font.ttf(fontData);
81       final boldFont = pw.Font.ttf(boldFontData);

```

```

82
83     final title = appTranslation().get('privacy_hero_title');
84     final desc = appTranslation().get('privacy_hero_desc');
85     final updated = appTranslation().get('privacy_last_updated');
86
87     final sections = [
88         (
89             appTranslation().get('privacy_sec1_title'),
90             '${appTranslation().get('privacy_sec1_p1')}\n\n${appTranslation().get('privacy_sec1_p2')}\n\n${appT
ranslation().get('privacy_sec1_highlight')}'
91         ),
92         (
93             appTranslation().get('privacy_sec2_title'),
94             '${appTranslation().get('privacy_sec2_intro')}\n- ${appTranslation().get('privacy_sec2_card1_titl
e')}: ${appTranslation().get('privacy_sec2_card1_desc')}\n- ${appTranslation().get('privacy_sec2_card2_title')}: ${a
ppTranslation().get('privacy_sec2_card2_desc')}\n- ${appTranslation().get('privacy_sec2_card3_title')}: ${appTransla
tion().get('privacy_sec2_card3_desc')}\n- ${appTranslation().get('privacy_sec2_card4_title')}: ${appTranslation().ge
t('privacy_sec2_card4_desc')}'
95         ),
96         (
97             appTranslation().get('privacy_sec3_title'),
98             '${appTranslation().get('privacy_sec3_intro')}\n- ${appTranslation().get('privacy_sec3_check1')}\n-
${appTranslation().get('privacy_sec3_check2')}\n- ${appTranslation().get('privacy_sec3_check3')}'
99         ),
100         (
101             appTranslation().get('privacy_sec4_title'),
102             '${appTranslation().get('privacy_sec4_intro')}\n- ${appTranslation().get('privacy_sec4_bullet1')}\n
- ${appTranslation().get('privacy_sec4_bullet2')}\n- ${appTranslation().get('privacy_sec4_bullet3')}\n- ${appTransla
tion().get('privacy_sec4_bullet4')}'
103         ),
104         (
105             appTranslation().get('privacy_sec5_title'),
106             '${appTranslation().get('privacy_sec5_intro')}\n- ${appTranslation().get('privacy_sec5_card1_titl
e')}: ${appTranslation().get('privacy_sec5_card1_desc')}\n- ${appTranslation().get('privacy_sec5_card2_title')}: ${a
ppTranslation().get('privacy_sec5_card2_desc')}\n- ${appTranslation().get('privacy_sec5_card3_title')}: ${appTransla
tion().get('privacy_sec5_card3_desc')}'
107         ),
108         (
109             appTranslation().get('privacy_sec6_title'),
110             '${appTranslation().get('privacy_sec6_alert_title')}: ${appTranslation().get('privacy_sec6_alert_de
sc')}\n\n${appTranslation().get('privacy_sec6_p1')}'
111         ),
112         (
113             appTranslation().get('privacy_sec7_title'),
114             '${appTranslation().get('privacy_sec7_intro')}\n1. ${appTranslation().get('privacy_sec7_item1')}\n
2. ${appTranslation().get('privacy_sec7_item2')}\n3. ${appTranslation().get('privacy_sec7_item3')}'
115         ),
116         (
117             appTranslation().get('privacy_sec8_title'),
118             '${appTranslation().get('privacy_sec8_intro')}\n- ${appTranslation().get('privacy_sec8_item1')}\n-
${appTranslation().get('privacy_sec8_item2')}\n- ${appTranslation().get('privacy_sec8_item3')}'
119         ),
120         (
121             appTranslation().get('privacy_sec9_title'),
122             '${appTranslation().get('privacy_sec9_p1')}\n\n${appTranslation().get('privacy_sec9_box_title')}:
${appTranslation().get('privacy_sec9_box_desc')}\n\n${appTranslation().get('privacy_sec9_email')}'
123         ),
124     ];
125
126     doc.addPage(
127         pw.MultiPage(
128             pageFormat: PdfPageFormat.a4,
129             textDirection: pw.TextDirection.rtl,
130             theme: pw.ThemeData.withFont(
131                 base: font,
132                 bold: boldFont,
133             ),
134             margin: const pw.EdgeInsets.all(32),
135             build: (context) => [
136                 pw.Container(
137                     margin: const pw.EdgeInsets.only(bottom: 16),
138                     child: pw.Column(
139                         crossAxisAlignment: pw.CrossAxisAlignment.start,
140                         children: [
141                             pw.Text(

```

```

142         title,
143         style: pw.TextStyle(
144             font: boldFont,
145             fontSize: 20,
146             color: PdfColors.teal800,
147         ),
148     ),
149     pw.SizedBox(height: 8),
150     pw.Text(
151         desc,
152         style: pw.TextStyle(
153             font: font,
154             fontSize: 11,
155             color: PdfColors.grey700,
156         ),
157     ),
158     pw.SizedBox(height: 4),
159     pw.Text(
160         updated,
161         style: pw.TextStyle(
162             font: font,
163             fontSize: 10,
164             color: PdfColors.grey600,
165         ),
166     ),
167     pw.SizedBox(height: 8),
168     pw.Divider(thickness: 1, color: PdfColors.grey300),
169 ],
170 ),
171 ),
172 ...sections.map(
173 (s) => pw.Container(
174     margin: const pw.EdgeInsets.only(bottom: 16),
175     child: pw.Column(
176         crossAxisAlignment: pw.CrossAxisAlignment.start,
177         children: [
178             pw.Text(
179                 s.$1,
180                 style: pw.TextStyle(
181                     font: boldFont,
182                     fontSize: 14,
183                     color: PdfColors.teal900,
184                 ),
185             ),
186             pw.SizedBox(height: 6),
187             pw.Text(
188                 s.$2,
189                 style: pw.TextStyle(
190                     font: font,
191                     fontSize: 10,
192                     lineHeight: 2,
193                     color: PdfColors.grey800,
194                 ),
195             ),
196             pw.SizedBox(height: 10),
197             pw.Divider(thickness: 0.5, color: PdfColors.grey200),
198         ],
199     ),
200 ),
201 ),
202 ],
203 ),
204 );
205
206 await Printing.layoutPdf(
207     onLayout: (format) async => doc.save(),
208     name: 'privacy_policy',
209     format: PdfPageFormat.a4,
210 );
211
212 if (!isClosed) emit(const PrivacyPolicyPdfGenerated());
213 } catch (e) {
214     if (!isClosed) emit(PrivacyPolicyPdfError(e.toString()));
215 }
216 }

```



```

217
218 @override
219 Future<void> close() {
220     scrollController.dispose();
221     searchController.dispose();
222     return super.close();
223 }
224 }
225

```

Lines • 43

Dart

Feature / Privacy #280 –

Policy [lib/features/privacy_policy/presentation/cubit/privacy_policy_state.dart](#)

```

1 import 'package:equatable/equatable.dart';
2
3 abstract class PrivacyPolicyState extends Equatable {
4     const PrivacyPolicyState();
5
6     @override
7     List<Object?> get props => [];
8 }
9
10 class PrivacyPolicyInitial extends PrivacyPolicyState {
11     const PrivacyPolicyInitial();
12 }
13
14 class PrivacyPolicyStateUpdated extends PrivacyPolicyState {
15     final int? activeSection;
16     final String searchQuery;
17
18     const PrivacyPolicyStateUpdated({
19         this.activeSection,
20         this.searchQuery = '',
21     });
22
23     @override
24     List<Object?> get props => [activeSection, searchQuery];
25 }
26
27 class PrivacyPolicyPdfGenerating extends PrivacyPolicyState {
28     const PrivacyPolicyPdfGenerating();
29 }
30
31 class PrivacyPolicyPdfGenerated extends PrivacyPolicyState {
32     const PrivacyPolicyPdfGenerated();
33 }
34
35 class PrivacyPolicyPdfError extends PrivacyPolicyState {
36     final String message;
37
38     const PrivacyPolicyPdfError(this.message);
39
40     @override
41     List<Object?> get props => [message];
42 }
43

```

Lines • 130

Dart

Feature / Privacy #281 –

Policy [lib/features/privacy_policy/presentation/screen/privacy_policy_screen.dart](#)

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/spacing.dart';
7 import 'package:moean/core/utils/extensions/context_extension.dart';
8 import 'package:moean/features/privacy_policy/presentation/cubit/privacy_policy_cubit.dart';
9 import 'package:moean/features/privacy_policy/presentation/cubit/privacy_policy_state.dart';
10 import 'package:moean/features/privacy_policy/presentation/widgets/privacy_header_card_widget.dart';
11 import 'package:moean/features/privacy_policy/presentation/widgets/privacy_section_ai_usage_widget.dart';
12 import 'package:moean/features/privacy_policy/presentation/widgets/privacy_section_cookies_widget.dart';

```

```

13 import 'package:moean/features/privacy_policy/presentation/widgets/privacy_section_data_collected_widget.dar
t';
14 import 'package:moean/features/privacy_policy/presentation/widgets/privacy_section_encryption_widget.dart';
15 import 'package:moean/features/privacy_policy/presentation/widgets/privacy_section_intro_widget.dart';
16 import 'package:moean/features/privacy_policy/presentation/widgets/privacy_section_madrasati_widget.dart';
17 import 'package:moean/features/privacy_policy/presentation/widgets/privacy_section_no_selling_widget.dart';
18 import 'package:moean/features/privacy_policy/presentation/widgets/privacy_section_teacher_rights_widget.dar
t';
19 import 'package:moean/features/privacy_policy/presentation/widgets/privacy_section_updates_contact_widget.dar
t';
20 import 'package:moean/features/privacy_policy/presentation/widgets/privacy_support_card_widget.dart';
21 import 'package:moean/features/privacy_policy/presentation/widgets/privacy_toc_widget.dart';
22
23 class PrivacyPolicyScreen extends StatelessWidget {
24   const PrivacyPolicyScreen({super.key});
25
26   @override
27   Widget build(BuildContext context) {
28     final cubit = PrivacyPolicyCubit.get(context);
29
30     return Directionality(
31       textDirection: TextDirection.rtl,
32       child: Scaffold(
33         backgroundColor: ColorsManager.background,
34         appBar: AppBar(
35           backgroundColor: ColorsManager.background,
36           elevation: 0,
37           centerTitle: true,
38           title: Text(
39             appTranslation().get('privacy_policy_title'),
40             style: TextStyleManager.bold18.copyWith(
41               color: ColorsManager.primaryColor,
42             ),
43           ),
44           leading: IconButton(
45             icon: Icon(
46               Icons.arrow_back_ios_rounded,
47               color: ColorsManager.mainText,
48               size: 20,
49             ),
50             onPressed: () => context.pop(),
51           ),
52         ), body: SafeArea(
53           child: BlocListener<PrivacyPolicyCubit, PrivacyPolicyState>(
54             listener: (context, state) {
55               if (state is PrivacyPolicyPdfError) {
56                 ScaffoldMessenger.of(context).showSnackBar(
57                   SnackBar(
58                     content: Text(state.message),
59                     backgroundColor: ColorsManager.errorColor,
60                   ),
61               );
62             }
63           ),
64           child: SingleChildScrollView(
65             controller: cubit.scrollController,
66             padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
67             child: Column(
68               crossAxisAlignment: CrossAxisAlignment.stretch,
69               children: [
70                 const PrivacyHeaderCardWidget(),
71                 verticalSpacel6,
72                 const PrivacyTocWidget(),
73                 verticalSpacel6,
74                 const PrivacySupportCardWidget(),
75                 verticalSpacel6,
76                 Container(
77                   key: cubit.sectionKeys[0],
78                   child: const PrivacySectionIntroWidget(),
79                 ),
80                 verticalSpacel6,
81                 Container(
82                   key: cubit.sectionKeys[1],
83                   child: const PrivacySectionDataCollectedWidget(),
84                 ),

```

```

85         verticalSpace16,
86         Container(
87             key: cubit.sectionKeys[2],
88             child: const PrivacySectionAiUsageWidget(),
89         ),
90         verticalSpace16,
91         Container(
92             key: cubit.sectionKeys[3],
93             child: const PrivacySectionMadrasatiWidget(),
94         ),
95         verticalSpace16,
96         Container(
97             key: cubit.sectionKeys[4],
98             child: const PrivacySectionEncryptionWidget(),
99         ),
100        verticalSpace16,
101        Container(
102            key: cubit.sectionKeys[5],
103            child: const PrivacySectionNoSellingWidget(),
104        ),
105        verticalSpace16,
106        Container(
107            key: cubit.sectionKeys[6],
108            child: const PrivacySectionTeacherRightsWidget(),
109        ),
110        verticalSpace16,
111        Container(
112            key: cubit.sectionKeys[7],
113            child: const PrivacySectionCookiesWidget(),
114        ),
115        verticalSpace16,
116        Container(
117            key: cubit.sectionKeys[8],
118            child: const PrivacySectionUpdatesContactWidget(),
119        ),
120        verticalSpace32,
121    ],
122    ),
123    ),
124    ),
125    ),
126    ),
127    );
128 }
129 }
130

```

Lines • 39
Dart

Feature / Privacy #282 -

Policy lib/features/privacy_policy/presentation/widgets/privacy_ai_check_item_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/spacing.dart';
5
6  class PrivacyAiCheckItemWidget extends StatelessWidget {
7      final String text;
8
9      const PrivacyAiCheckItemWidget({
10         super.key,
11         required this.text,
12     });
13
14     @override
15     Widget build(BuildContext context) {
16         return Row(
17             crossAxisAlignment: CrossAxisAlignment.start,
18             children: [
19                 Icon(
20                     Icons.check_circle_outline_rounded,
21                     color: ColorsManager.primaryColor,
22                     size: 20,
23                 ),

```

```

24     horizontalSpace10,
25     Expanded(
26       child: Text(
27         text,
28         style: TextStyleManager.regular13.copyWith(
29           color: ColorsManager.mainText,
30           height: 1.5,
31         ),
32         textAlign: TextAlign.right,
33       ),
34     ),
35   ],
36 );
37 }
38 }
39

```

Lines • 43
Dart

Feature / Privacy #283 –

Policy lib/features/privacy_policy/presentation/widgets/privacy_cookie_bullet_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/spacing.dart';
5
6  class PrivacyCookieBulletWidget extends StatelessWidget {
7    final String text;
8
9    const PrivacyCookieBulletWidget({
10      super.key,
11      required this.text,
12    });
13
14    @override
15    Widget build(BuildContext context) {
16      return Row(
17        crossAxisAlignment: CrossAxisAlignment.start,
18        children: [
19          Expanded(
20            child: Text(
21              text,
22              style: TextStyleManager.regular13.copyWith(
23                color: ColorsManager.mainText,
24                height: 1.5,
25              ),
26              textAlign: TextAlign.right,
27            ),
28          ),
29          horizontalSpace10,
30          Container(
31            margin: const EdgeInsets.only(top: 8),
32            width: 6,
33            height: 6,
34            decoration: BoxDecoration(
35              color: ColorsManager.primaryColor,
36              shape: BoxShape.circle,
37            ),
38          ),
39        ],
40      );
41    }
42  }
43

```

Lines 63
• Dart

Feature / #284 –

Privacy Policy lib/features/privacy_policy/presentation/widgets/privacy_data_collected_card_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/spacing.dart';
5

```

```

6 class PrivacyDataCollectedCardWidget extends StatelessWidget {
7   final String title;
8   final String description;
9   final IconData icon;
10
11   const PrivacyDataCollectedCardWidget({
12     super.key,
13     required this.title,
14     required this.description,
15     required this.icon,
16   });
17
18   @override
19   Widget build(BuildContext context) {
20     return Container(
21       padding: const EdgeInsets.all(14),
22       decoration: BoxDecoration(
23         color: ColorsManager.surfacePrimary,
24         borderRadius: BorderRadius.circular(14),
25         border: Border.all(color: ColorsManager.borderColor),
26       ),
27       child: Column(
28         crossAxisAlignment: CrossAxisAlignment.stretch,
29         children: [
30           Row(
31             children: [
32               Icon(
33                 icon,
34                 color: ColorsManager.primaryColor,
35                 size: 20,
36               ),
37               horizontalSpace8,
38               Expanded(
39                 child: Text(
40                   title,
41                   style: TextStyleManager.bold14.copyWith(
42                     color: ColorsManager.primaryColor,
43                   ),
44                   textAlign: TextAlign.right,
45                 ),
46               ),
47             ],
48           ),
49           verticalSpace8,
50           Text(
51             description,
52             style: TextStyleManager.regular12.copyWith(
53               color: ColorsManager.placeholder,
54               height: 1.5,
55             ),
56             textAlign: TextAlign.right,
57           ),
58         ],
59       ),
60     );
61   }
62 }
63

```

Lines • 63
Dart

Feature / Privacy #285 -
Policy

lib/features/privacy_policy/presentation/widgets/privacy_encryption_card_widget.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/spacing.dart';
5
6 class PrivacyEncryptionCardWidget extends StatelessWidget {
7   final String title;
8   final String description;
9   final IconData icon;
10
11   const PrivacyEncryptionCardWidget({

```

```

12     super.key,
13     required this.title,
14     required this.description,
15     required this.icon,
16   });
17
18   @override
19   Widget build(BuildContext context) {
20     return Container(
21       padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 20),
22       decoration: BoxDecoration(
23         color: ColorsManager.surfacePrimary,
24         borderRadius: BorderRadius.circular(16),
25         border: Border.all(color: ColorsManager.borderColor),
26       ),
27       child: Column(
28         crossAxisAlignment: CrossAxisAlignment.center,
29         children: [
30           Container(
31             padding: const EdgeInsets.all(12),
32             decoration: BoxDecoration(
33               color: ColorsManager.primaryColor.withValues(alpha: 0.1),
34               borderRadius: BorderRadius.circular(12),
35             ),
36             child: Icon(
37               icon,
38               color: ColorsManager.primaryColor,
39               size: 24,
40             ),
41           ),
42           verticalSpace12,
43           Text(
44             title,
45             style: TextStylesManager.bold14.copyWith(
46               color: ColorsManager.primaryColor,
47             ),
48             textAlign: TextAlign.center,
49           ),
50           verticalSpace6,
51           Text(
52             description,
53             style: TextStylesManager.regular12.copyWith(
54               color: ColorsManager.placeholder,
55             ),
56             textAlign: TextAlign.center,
57           ),
58         ],
59       ),
60     );
61   }
62 }
63

```

Lines • 157
Dart

Feature / Privacy #286 –

Policy lib/features/privacy_policy/presentation/widgets/privacy_header_card_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7  import 'package:moean/features/privacy_policy/presentation/cubit/privacy_policy_cubit.dart';
8  import 'package:moean/features/privacy_policy/presentation/cubit/privacy_policy_state.dart';
9
10 class PrivacyHeaderCardWidget extends StatelessWidget {
11   const PrivacyHeaderCardWidget({super.key});
12
13   @override
14   Widget build(BuildContext context) {
15     final cubit = PrivacyPolicyCubit.get(context);
16
17     return Container(

```

```

18 padding: const EdgeInsets.all(20),
19 decoration: BoxDecoration(
20   color: ColorsManager.surfacePrimary,
21   borderRadius: BorderRadius.circular(20),
22   border: Border.all(color: ColorsManager.borderColor),
23 ),
24 child: Column(
25   crossAxisAlignment: CrossAxisAlignment.center,
26   children: [
27     Container(
28       padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 6),
29       decoration: BoxDecoration(
30         color: ColorsManager.primaryColor.withValues(alpha: 0.1),
31         borderRadius: BorderRadius.circular(24),
32       ),
33       child: Row(
34         mainAxisAlignment: MainAxisAlignment.min,
35         children: [
36           Icon(
37             Icons.shield_outlined,
38             size: 16,
39             color: ColorsManager.primaryColor,
40           ),
41           horizontalSpace6,
42           Text(
43             appTranslation().get('privacy_official_document_badge'),
44             style: TextStylesManager.bold12.copyWith(
45               color: ColorsManager.primaryColor,
46             ),
47           ),
48         ],
49       ),
50     ),
51     verticalSpace16,
52     Text(
53       appTranslation().get('privacy_hero_title'),
54       style: TextStylesManager.bold22.copyWith(
55         color: ColorsManager.primaryColor,
56       ),
57       textAlign: TextAlign.center,
58     ),
59     verticalSpace12,
60     Text(
61       appTranslation().get('privacy_hero_desc'),
62       style: TextStylesManager.regular13.copyWith(
63         color: ColorsManager.placeholder,
64         height: 1.5,
65       ),
66       textAlign: TextAlign.center,
67     ),
68     verticalSpace12,
69     Row(
70       mainAxisAlignment: MainAxisAlignment.center,
71       children: [
72         Icon(
73           Icons.access_time_rounded,
74           size: 16,
75           color: ColorsManager.placeholder,
76         ),
77         horizontalSpace6,
78         Text(
79           appTranslation().get('privacy_last_updated'),
80           style: TextStylesManager.regular12.copyWith(
81             color: ColorsManager.placeholder,
82           ),
83         ),
84       ],
85     ),
86     verticalSpace20,
87     Row(
88       children: [
89         Expanded(
90           child: OutlinedButton.icon(
91             onPressed: cubit.shareLink,
92             icon: Icon(

```

```

93         Icons.share_outlined,
94         size: 18,
95         color: ColorsManager.primaryColor,
96     ),
97     label: Text(
98       appTranslation().get('privacy_share_link'),
99       style: TextStylesManager.bold13.copyWith(
100         color: ColorsManager.primaryColor,
101       ),
102     ),
103     style: OutlinedButton.styleFrom(
104       side: BorderSide(color: ColorsManager.borderColor),
105       shape: RoundedRectangleBorder(
106         borderRadius: BorderRadius.circular(24),
107       ),
108       padding: const EdgeInsets.symmetric(vertical: 12),
109     ),
110   ),
111 ),
112 horizontalSpace12,
113 BlocBuilder<PrivacyPolicyCubit, PrivacyPolicyState>(
114   builder: (context, state) {
115     final isGenerating = state is PrivacyPolicyPdfGenerating;
116     return Expanded(
117       child: OutlinedButton.icon(
118         onPressed: isGenerating ? null : cubit.printPdf,
119         icon: isGenerating
120           ? SizedBox(
121             width: 18,
122             height: 18,
123             child: CircularProgressIndicator(
124               strokeWidth: 2,
125               color: ColorsManager.primaryColor,
126             ),
127           )
128         : Icon(
129           Icons.print_outlined,
130           size: 18,
131           color: ColorsManager.primaryColor,
132         ),
133         label: Text(
134           appTranslation().get('privacy_print_pdf'),
135           style: TextStylesManager.bold13.copyWith(
136             color: ColorsManager.primaryColor,
137           ),
138         ),
139         style: OutlinedButton.styleFrom(
140           side: BorderSide(color: ColorsManager.borderColor),
141           shape: RoundedRectangleBorder(
142             borderRadius: BorderRadius.circular(24),
143           ),
144           padding: const EdgeInsets.symmetric(vertical: 12),
145         ),
146       ),
147     );
148   },
149 ),
150 ],
151 ),
152 ],
153 ),
154 );
155 }
156 }
157 }

```

40
Lines
•
Dart

Feature /
Privacy
Policy

#287 -

[lib/features/privacy_policy/presentation/widgets/privacy_madrasati_guarantee_item_widget.dart](#)

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';

```



```

3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/spacing.dart';
5
6 class PrivacyMadrasatiGuaranteeItemWidget extends StatelessWidget {
7   final String text;
8   final String emoji;
9
10  const PrivacyMadrasatiGuaranteeItemWidget({
11    super.key,
12    required this.text,
13    required this.emoji,
14  });
15
16  @override
17  Widget build(BuildContext context) {
18    return Row(
19      crossAxisAlignment: CrossAxisAlignment.start,
20      children: [
21        Text(
22          emoji,
23          style: const TextStyle(fontSize: 16),
24        ),
25        horizontalSpace8,
26        Expanded(
27          child: Text(
28            text,
29            style: TextStylesManager.regular12.copyWith(
30              color: ColorsManager.mainText,
31              height: 1.5,
32            ),
33            textAlign: TextAlign.right,
34          ),
35        ),
36      ],
37    );
38  }
39 }
40

```

Lines • 56
Dart

Feature / #288 –
Privacy Policy [lib/features/privacy_policy/presentation/widgets/privacy_section_ai_usage_widget.dart](#)

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/spacing.dart';
6 import 'package:moean/features/privacy_policy/presentation/widgets/privacy_ai_check_item_widget.dart';
7 import 'package:moean/features/privacy_policy/presentation/widgets/privacy_section_header_widget.dart';
8
9 class PrivacySectionAiUsageWidget extends StatelessWidget {
10   const PrivacySectionAiUsageWidget({super.key});
11
12   @override
13   Widget build(BuildContext context) {
14     return Container(
15       padding: const EdgeInsets.all(20),
16       decoration: BoxDecoration(
17         color: ColorsManager.surfacePrimary,
18         borderRadius: BorderRadius.circular(20),
19         border: Border.all(color: ColorsManager.borderColor),
20       ),
21       child: Column(
22         crossAxisAlignment: CrossAxisAlignment.stretch,
23         children: [
24           PrivacySectionHeaderWidget(
25             title: appTranslation().get('privacy_sec3_title'),
26             subtitle: appTranslation().get('privacy_sec3_subtitle'),
27             icon: Icons.memory_rounded,
28             tag: appTranslation().get('privacy_sec3_tag'),
29           ),
30           verticalSpace16,
31           Text(

```

```

32         appTranslation().get('privacy_sec3_intro'),
33         style: TextStylesManager.regular13.copyWith(
34             color: ColorsManager.mainText,
35             height: 1.6,
36         ),
37         textAlign: TextAlign.right,
38     ),
39     verticalSpacel6,
40     PrivacyAiCheckItemWidget(
41         text: appTranslation().get('privacy_sec3_check1'),
42     ),
43     verticalSpacel2,
44     PrivacyAiCheckItemWidget(
45         text: appTranslation().get('privacy_sec3_check2'),
46     ),
47     verticalSpacel2,
48     PrivacyAiCheckItemWidget(
49         text: appTranslation().get('privacy_sec3_check3'),
50     ),
51 ],
52 ),
53 );
54 }
55 }
56

```

Lines • 56

Dart

Feature / Privacy #289 –

Policy

[lib/features/privacy_policy/presentation/widgets/privacy_section_cookies_widget.dart](#)

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/spacing.dart';
6 import 'package:moean/features/privacy_policy/presentation/widgets/privacy_cookie_bullet_widget.dart';
7 import 'package:moean/features/privacy_policy/presentation/widgets/privacy_section_header_widget.dart';
8
9 class PrivacySectionCookiesWidget extends StatelessWidget {
10     const PrivacySectionCookiesWidget({super.key});
11
12     @override
13     Widget build(BuildContext context) {
14         return Container(
15             padding: const EdgeInsets.all(20),
16             decoration: BoxDecoration(
17                 color: ColorsManager.surfacePrimary,
18                 borderRadius: BorderRadius.circular(20),
19                 border: Border.all(color: ColorsManager.borderColor),
20             ),
21             child: Column(
22                 crossAxisAlignment: CrossAxisAlignment.stretch,
23                 children: [
24                     PrivacySectionHeaderWidget(
25                         title: appTranslation().get('privacy_sec8_title'),
26                         subtitle: appTranslation().get('privacy_sec8_subtitle'),
27                         icon: Icons.cookie_outlined,
28                         tag: appTranslation().get('privacy_sec8_tag'),
29                     ),
30                     verticalSpacel6,
31                     Text(
32                         appTranslation().get('privacy_sec8_intro'),
33                         style: TextStylesManager.regular13.copyWith(
34                             color: ColorsManager.mainText,
35                             height: 1.6,
36                         ),
37                         textAlign: TextAlign.right,
38                     ),
39                     verticalSpacel4,
40                     PrivacyCookieBulletWidget(
41                         text: appTranslation().get('privacy_sec8_item1'),
42                     ),
43                     verticalSpacel0,
44                     PrivacyCookieBulletWidget(

```

```

45         text: appTranslation().get('privacy_sec8_item2'),
46     ),
47     verticalSpace10,
48     PrivacyCookieBulletWidget(
49         text: appTranslation().get('privacy_sec8_item3'),
50     ),
51 ],
52 ),
53 );
54 }
55 }
56

```

68
Lines •
Dart

Feature /
Privacy
Policy

#290 –

[lib/features/privacy_policy/presentation/widgets/privacy_section_data_collected_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6  import 'package:moean/features/privacy_policy/presentation/widgets/privacy_data_collected_card_widget.dart';
7  import 'package:moean/features/privacy_policy/presentation/widgets/privacy_section_header_widget.dart';
8
9  class PrivacySectionDataCollectedWidget extends StatelessWidget {
10     const PrivacySectionDataCollectedWidget({super.key});
11
12     @override
13     Widget build(BuildContext context) {
14         return Container(
15             padding: const EdgeInsets.all(20),
16             decoration: BoxDecoration(
17                 color: ColorsManager.surfacePrimary,
18                 borderRadius: BorderRadius.circular(20),
19                 border: Border.all(color: ColorsManager.borderColor),
20             ),
21             child: Column(
22                 crossAxisAlignment: CrossAxisAlignment.stretch,
23                 children: [
24                     PrivacySectionHeaderWidget(
25                         title: appTranslation().get('privacy_sec2_title'),
26                         subtitle: appTranslation().get('privacy_sec2_subtitle'),
27                         icon: Icons.storage_rounded,
28                         tag: appTranslation().get('privacy_sec2_tag'),
29                     ),
30                     verticalSpace16,
31                     Text(
32                         appTranslation().get('privacy_sec2_intro'),
33                         style: TextStylesManager.regular13.copyWith(
34                             color: ColorsManager.mainText,
35                             height: 1.6,
36                         ),
37                         textAlign: TextAlign.right,
38                     ),
39                     verticalSpace14,
40                     PrivacyDataCollectedCardWidget(
41                         title: appTranslation().get('privacy_sec2_card1_title'),
42                         description: appTranslation().get('privacy_sec2_card1_desc'),
43                         icon: Icons.person_outline_rounded,
44                     ),
45                     verticalSpace10,
46                     PrivacyDataCollectedCardWidget(
47                         title: appTranslation().get('privacy_sec2_card2_title'),
48                         description: appTranslation().get('privacy_sec2_card2_desc'),
49                         icon: Icons.description_outlined,
50                     ),
51                     verticalSpace10,
52                     PrivacyDataCollectedCardWidget(
53                         title: appTranslation().get('privacy_sec2_card3_title'),
54                         description: appTranslation().get('privacy_sec2_card3_desc'),
55                         icon: Icons.language_rounded,
56                     ),

```

```

57         verticalSpace10,
58         PrivacyDataCollectedCardWidget(
59             title: appTranslation().get('privacy_sec2_card4_title'),
60             description: appTranslation().get('privacy_sec2_card4_desc'),
61             icon: Icons.memory_rounded,
62         ),
63     ],
64 ),
65 );
66 }
67 }
68

```

Lines 62
• Dart

Feature / #291 -
Privacy Policy

[lib/features/privacy_policy/presentation/widgets/privacy_section_encryption_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6  import 'package:moean/features/privacy_policy/presentation/widgets/privacy_encryption_card_widget.dart';
7  import 'package:moean/features/privacy_policy/presentation/widgets/privacy_section_header_widget.dart';
8
9  class PrivacySectionEncryptionWidget extends StatelessWidget {
10     const PrivacySectionEncryptionWidget({super.key});
11
12     @override
13     Widget build(BuildContext context) {
14         return Container(
15             padding: const EdgeInsets.all(20),
16             decoration: BoxDecoration(
17                 color: ColorsManager.surfacePrimary,
18                 borderRadius: BorderRadius.circular(20),
19                 border: Border.all(color: ColorsManager.borderColor),
20             ),
21             child: Column(
22                 crossAxisAlignment: CrossAxisAlignment.stretch,
23                 children: [
24                     PrivacySectionHeaderWidget(
25                         title: appTranslation().get('privacy_sec5_title'),
26                         subtitle: appTranslation().get('privacy_sec5_subtitle'),
27                         icon: Icons.lock_outline_rounded,
28                         tag: appTranslation().get('privacy_sec5_tag'),
29                     ),
30                     verticalSpace16,
31                     Text(
32                         appTranslation().get('privacy_sec5_intro'),
33                         style: TextStylesManager.regular13.copyWith(
34                             color: ColorsManager.mainText,
35                             height: 1.6,
36                         ),
37                         textAlign: TextAlign.right,
38                     ),
39                     verticalSpace16,
40                     PrivacyEncryptionCardWidget(
41                         title: appTranslation().get('privacy_sec5_card1_title'),
42                         description: appTranslation().get('privacy_sec5_card1_desc'),
43                         icon: Icons.lock_outline_rounded,
44                     ),
45                     verticalSpace12,
46                     PrivacyEncryptionCardWidget(
47                         title: appTranslation().get('privacy_sec5_card2_title'),
48                         description: appTranslation().get('privacy_sec5_card2_desc'),
49                         icon: Icons.storage_rounded,
50                     ),
51                     verticalSpace12,
52                     PrivacyEncryptionCardWidget(
53                         title: appTranslation().get('privacy_sec5_card3_title'),
54                         description: appTranslation().get('privacy_sec5_card3_desc'),
55                         icon: Icons.verified_user_outlined,
56                     ),
57                 ],

```

```
58     ),
59     );
60   }
61 }
62
```

Lines • 78
Dart

Feature / Privacy #292 -

Policy [lib/features/privacy_policy/presentation/widgets/privacy_section_header_widget.dart](#)

```
1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/spacing.dart';
5  import 'package:moean/features/privacy_policy/presentation/widgets/privacy_tag_badge_widget.dart';
6
7  class PrivacySectionHeaderWidget extends StatelessWidget {
8    final String title;
9    final String subtitle;
10   final IconData icon;
11   final String tag;
12
13   const PrivacySectionHeaderWidget({
14     super.key,
15     required this.title,
16     required this.subtitle,
17     required this.icon,
18     required this.tag,
19   });
20
21   @override
22   Widget build(BuildContext context) {
23     return Column(
24       crossAxisAlignment: CrossAxisAlignment.start,
25       children: [
26         Row(
27           crossAxisAlignment: CrossAxisAlignment.start,
28           children: [
29             Expanded(
30               child: Column(
31                 crossAxisAlignment: CrossAxisAlignment.start,
32                 children: [
33                   Text(
34                     title,
35                     style: TextStylesManager.bold18.copyWith(
36                       color: ColorsManager.primaryColor,
37                     ),
38                   ),
39                   verticalSpace4,
40                   Text(
41                     subtitle,
42                     style: TextStylesManager.regular12.copyWith(
43                       color: ColorsManager.placeholder,
44                     ),
45                   ),
46                 ],
47             ),
48           ),
49           horizontalSpace8,
50           Container(
51             padding: const EdgeInsets.all(10),
52             decoration: BoxDecoration(
53               color: ColorsManager.primaryColor.withValues(alpha: 0.1),
54               borderRadius: BorderRadius.circular(12),
55             ),
56             child: Icon(
57               icon,
58               color: ColorsManager.primaryColor,
59               size: 22,
60             ),
61           ),
62         ],
63       ),
64       verticalSpace12,
```

```

65     Align(
66         alignment: Alignment.centerLeft,
67         child: PrivacyTagBadgeWidget(label: tag),
68     ),
69     verticalSpace12,
70     Divider(
71         color: ColorsManager.borderColor,
72         thickness: 1,
73     ),
74 ],
75 );
76 }
77 }
78

```

Lines • 71
Dart

Feature / Privacy #293 -

Policy [lib/features/privacy_policy/presentation/widgets/privacy_section_intro_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6  import 'package:moean/features/privacy_policy/presentation/widgets/privacy_section_header_widget.dart';
7
8  class PrivacySectionIntroWidget extends StatelessWidget {
9      const PrivacySectionIntroWidget({super.key});
10
11     @override
12     Widget build(BuildContext context) {
13         return Container(
14             padding: const EdgeInsets.all(20),
15             decoration: BoxDecoration(
16                 color: ColorsManager.surfacePrimary,
17                 borderRadius: BorderRadius.circular(20),
18                 border: Border.all(color: ColorsManager.borderColor),
19             ),
20             child: Column(
21                 crossAxisAlignment: CrossAxisAlignment.stretch,
22                 children: [
23                     PrivacySectionHeaderWidget(
24                         title: appTranslation().get('privacy_sec1_title'),
25                         subtitle: appTranslation().get('privacy_sec1_subtitle'),
26                         icon: Icons.shield_outlined,
27                         tag: appTranslation().get('privacy_sec1_tag'),
28                     ),
29                     verticalSpace16,
30                     Text(
31                         appTranslation().get('privacy_sec1_p1'),
32                         style: TextStylesManager.regular13.copyWith(
33                             color: ColorsManager.mainText,
34                             height: 1.6,
35                         ),
36                         textAlign: TextAlign.right,
37                     ),
38                     verticalSpace14,
39                     Text(
40                         appTranslation().get('privacy_sec1_p2'),
41                         style: TextStylesManager.regular13.copyWith(
42                             color: ColorsManager.mainText,
43                             height: 1.6,
44                         ),
45                         textAlign: TextAlign.right,
46                     ),
47                     verticalSpace16,
48                     Container(
49                         padding: const EdgeInsets.all(14),
50                         decoration: BoxDecoration(
51                             color: ColorsManager.primaryColor.withValues(alpha: 0.08),
52                             borderRadius: BorderRadius.circular(14),
53                             border: Border.all(
54                                 color: ColorsManager.primaryColor.withValues(alpha: 0.18),
55                             ),

```

```

56         ),
57         child: Text(
58           '💡 ${appTranslation().get('privacy_sec1_highlight')}',
59           style: TextStylesManager.bold12.copyWith(
60             color: ColorsManager.primaryColor,
61             height: 1.5,
62           ),
63           textAlign: TextAlign.right,
64         ),
65       ),
66     ],
67   ),
68 );
69 }
70 }
71

```

Lines • 85
Dart

Feature / #294 –
Privacy Policy [lib/features/privacy_policy/presentation/widgets/privacy_section_madrasati_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6  import 'package:moean/features/privacy_policy/presentation/widgets/privacy_madrasati_guarantee_item_widget.dart';
7
8  import 'package:moean/features/privacy_policy/presentation/widgets/privacy_section_header_widget.dart';
9
10 class PrivacySectionMadrasatiWidget extends StatelessWidget {
11   const PrivacySectionMadrasatiWidget({super.key});
12
13   @override
14   Widget build(BuildContext context) {
15     return Container(
16       padding: const EdgeInsets.all(20),
17       decoration: BoxDecoration(
18         color: ColorsManager.surfacePrimary,
19         borderRadius: BorderRadius.circular(20),
20         border: Border.all(color: ColorsManager.borderColor),
21       ),
22       child: Column(
23         crossAxisAlignment: CrossAxisAlignment.stretch,
24         children: [
25           PrivacySectionHeaderWidget(
26             title: appTranslation().get('privacy_sec4_title'),
27             subtitle: appTranslation().get('privacy_sec4_subtitle'),
28             icon: Icons.language_rounded,
29             tag: appTranslation().get('privacy_sec4_tag'),
30           ),
31           verticalSpace16,
32           Text(
33             appTranslation().get('privacy_sec4_intro'),
34             style: TextStylesManager.regular13.copyWith(
35               color: ColorsManager.mainText,
36               height: 1.6,
37             ),
38             textAlign: TextAlign.right,
39           ),
40           verticalSpace16,
41           Container(
42             padding: const EdgeInsets.all(16),
43             decoration: BoxDecoration(
44               color: ColorsManager.background,
45               borderRadius: BorderRadius.circular(16),
46               border: Border.all(color: ColorsManager.borderColor),
47             ),
48             child: Column(
49               crossAxisAlignment: CrossAxisAlignment.stretch,
50               children: [
51                 Text(
52                   appTranslation().get('privacy_sec4_box_title'),
53                   style: TextStylesManager.bold14.copyWith(

```

```

53         color: ColorsManager.primaryColor,
54     ),
55     textAlign: TextAlign.right,
56 ),
57 verticalSpace14,
58 PrivacyMadrasatiGuaranteeItemWidget(
59     emoji: '🔒',
60     text: appTranslation().get('privacy_sec4_bullet1'),
61 ),
62 verticalSpace10,
63 PrivacyMadrasatiGuaranteeItemWidget(
64     emoji: '🔑',
65     text: appTranslation().get('privacy_sec4_bullet2'),
66 ),
67 verticalSpace10,
68 PrivacyMadrasatiGuaranteeItemWidget(
69     emoji: '🔗',
70     text: appTranslation().get('privacy_sec4_bullet3'),
71 ),
72 verticalSpace10,
73 PrivacyMadrasatiGuaranteeItemWidget(
74     emoji: '👉',
75     text: appTranslation().get('privacy_sec4_bullet4'),
76 ),
77 ],
78 ),
79 ),
80 ],
81 ),
82 );
83 }
84 }
85

```

Lines 87

• Dart

Feature / #295 -

Privacy Policy lib/features/privacy_policy/presentation/widgets/privacy_section_no_selling_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6  import 'package:moean/features/privacy_policy/presentation/widgets/privacy_section_header_widget.dart';
7
8  class PrivacySectionNoSellingWidget extends StatelessWidget {
9      const PrivacySectionNoSellingWidget({super.key});
10
11      @override
12      Widget build(BuildContext context) {
13          return Container(
14              padding: const EdgeInsets.all(20),
15              decoration: BoxDecoration(
16                  color: ColorsManager.surfacePrimary,
17                  borderRadius: BorderRadius.circular(20),
18                  border: Border.all(color: ColorsManager.borderColor),
19              ),
20              child: Column(
21                  crossAxisAlignment: CrossAxisAlignment.stretch,
22                  children: [
23                      PrivacySectionHeaderWidget(
24                          title: appTranslation().get('privacy_sec6_title'),
25                          subtitle: appTranslation().get('privacy_sec6_subtitle'),
26                          icon: Icons.share_rounded,
27                          tag: appTranslation().get('privacy_sec6_tag'),
28                      ),
29                      verticalSpace16,
30                      Container(
31                          padding: const EdgeInsets.all(16),
32                          decoration: BoxDecoration(
33                              color: ColorsManager.brandGold.withValues(alpha: 0.08),
34                              borderRadius: BorderRadius.circular(16),
35                              border: Border.all(
36                                  color: ColorsManager.brandGold.withValues(alpha: 0.3),

```



```

37         ),
38     ),
39     child: Column(
40       crossAxisAlignment: CrossAxisAlignment.stretch,
41       children: [
42         Row(
43           children: [
44             Icon(
45               Icons.block_rounded,
46               color: ColorsManager.errorColor,
47               size: 20,
48             ),
49             horizontalSpace8,
50             Expanded(
51               child: Text(
52                 appTranslation().get('privacy_sec6_alert_title'),
53                 style: TextStylesManager.bold14.copyWith(
54                   color: ColorsManager.primaryColor,
55                 ),
56                 textAlign: TextAlign.right,
57               ),
58             ),
59           ],
60         ),
61         verticalSpace10,
62         Text(
63           appTranslation().get('privacy_sec6_alert_desc'),
64           style: TextStylesManager.regular13.copyWith(
65             color: ColorsManager.mainText,
66             height: 1.6,
67           ),
68           textAlign: TextAlign.right,
69         ),
70       ],
71     ),
72   ),
73   verticalSpace16,
74   Text(
75     appTranslation().get('privacy_sec6_p1'),
76     style: TextStylesManager.regular13.copyWith(
77       color: ColorsManager.placeholder,
78       height: 1.6,
79     ),
80     textAlign: TextAlign.right,
81   ),
82 ],
83 ),
84 );
85 }
86 }
87

```

59
Lines •
Dart

Feature /
Privacy
Policy

#296 –

[lib/features/privacy_policy/presentation/widgets/privacy_section_teacher_rights_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6  import 'package:moean/features/privacy_policy/presentation/widgets/privacy_section_header_widget.dart';
7  import 'package:moean/features/privacy_policy/presentation/widgets/privacy_teacher_right_item_widget.dart';
8
9  class PrivacySectionTeacherRightsWidget extends StatelessWidget {
10     const PrivacySectionTeacherRightsWidget({super.key});
11
12     @override
13     Widget build(BuildContext context) {
14       return Container(
15         padding: const EdgeInsets.all(20),
16         decoration: BoxDecoration(
17           color: ColorsManager.surfacePrimary,

```

```

18     borderRadius: BorderRadius.circular(20),
19     border: Border.all(color: ColorsManager.borderColor),
20   ),
21   child: Column(
22     crossAxisAlignment: CrossAxisAlignment.stretch,
23     children: [
24       PrivacySectionHeaderWidget(
25         title: appTranslation().get('privacy_sec7_title'),
26         subtitle: appTranslation().get('privacy_sec7_subtitle'),
27         icon: Icons.person_outline_rounded,
28         tag: appTranslation().get('privacy_sec7_tag'),
29       ),
30       verticalSpacel6,
31       Text(
32         appTranslation().get('privacy_sec7_intro'),
33         style: TextStylesManager.regularl3.copyWith(
34           color: ColorsManager.mainText,
35           height: 1.6,
36         ),
37         textAlign: TextAlign.right,
38       ),
39       verticalSpacel4,
40       PrivacyTeacherRightItemWidget(
41         number: 1,
42         text: appTranslation().get('privacy_sec7_item1'),
43       ),
44       verticalSpacel0,
45       PrivacyTeacherRightItemWidget(
46         number: 2,
47         text: appTranslation().get('privacy_sec7_item2'),
48       ),
49       verticalSpacel0,
50       PrivacyTeacherRightItemWidget(
51         number: 3,
52         text: appTranslation().get('privacy_sec7_item3'),
53       ),
54     ],
55   ),
56 );
57 }
58 }
59

```

177
Lines •
Dart

Feature /
Privacy
Policy

#297 -

lib/features/privacy_policy/presentation/widgets/privacy_section_updates_contact_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/routes.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7  import 'package:moean/core/utils/extensions/context_extension.dart';
8  import 'package:moean/features/privacy_policy/presentation/widgets/privacy_section_header_widget.dart';
9  import 'package:url_launcher/url_launcher.dart';
10
11  class PrivacySectionUpdatesContactWidget extends StatelessWidget {
12    const PrivacySectionUpdatesContactWidget({super.key});
13
14    Future<void> _launchEmail(String email) async {
15      final Uri uri = Uri(
16        scheme: 'mailto',
17        path: email,
18      );
19      try {
20        final launched =
21          await launchUrl(uri, mode: LaunchMode.externalApplication);
22        if (!launched) {
23          await launchUrl(uri);
24        }
25      } catch (_) {
26        try {

```

```

27         await launchUrl(uri);
28     } catch (_) {}
29 }
30 }
31
32 @override
33 Widget build(BuildContext context) {
34     final email = appTranslation().get('privacy_sec9_email');
35     return Container(
36         padding: const EdgeInsets.all(20),
37         decoration: BoxDecoration(
38             color: ColorsManager.surfacePrimary,
39             borderRadius: BorderRadius.circular(20),
40             border: Border.all(color: ColorsManager.borderColor),
41         ),
42         child: Column(
43             crossAxisAlignment: CrossAxisAlignment.stretch,
44             children: [
45                 PrivacySectionHeaderWidget(
46                     title: appTranslation().get('privacy_sec9_title'),
47                     subtitle: appTranslation().get('privacy_sec9_subtitle'),
48                     icon: Icons.mail_outline_rounded,
49                     tag: appTranslation().get('privacy_sec9_tag'),
50                 ),
51                 verticalSpace16,
52                 Text(
53                     appTranslation().get('privacy_sec9_p1'),
54                     style: TextStyleManager.regular13.copyWith(
55                         color: ColorsManager.mainText,
56                         height: 1.6,
57                     ),
58                     textAlign: TextAlign.right,
59                 ),
60                 verticalSpace16,
61                 Container(
62                     padding: const EdgeInsets.all(16),
63                     decoration: BoxDecoration(
64                         color: ColorsManager.primaryColor.withValues(alpha: 0.06),
65                         borderRadius: BorderRadius.circular(16),
66                         border: Border.all(
67                             color: ColorsManager.primaryColor.withValues(alpha: 0.16),
68                         ),
69                     ),
70                     child: Column(
71                         crossAxisAlignment: CrossAxisAlignment.stretch,
72                         children: [
73                             Row(
74                                 children: [
75                                     Icon(
76                                         Icons.mail_outline_rounded,
77                                         color: ColorsManager.primaryColor,
78                                         size: 20,
79                                     ),
80                                     horizontalSpace8,
81                                     Expanded(
82                                         child: Text(
83                                             appTranslation().get('privacy_sec9_box_title'),
84                                             style: TextStyleManager.bold14.copyWith(
85                                                 color: ColorsManager.primaryColor,
86                                             ),
87                                             textAlign: TextAlign.right,
88                                         ),
89                                     ),
90                                 ],
91                             ),
92                             verticalSpace8,
93                             Text(
94                                 appTranslation().get('privacy_sec9_box_desc'),
95                                 style: TextStyleManager.regular12.copyWith(
96                                     color: ColorsManager.placeholder,
97                                     height: 1.5,
98                                 ),
99                                 textAlign: TextAlign.right,
100                             ),
101                             verticalSpace16,

```

```

102         Wrap(
103             spacing: 12,
104             runSpacing: 10,
105             alignment: WrapAlignment.center,
106             children: [
107                 ElevatedButton.icon(
108                     onPressed: () => context.push(Routes.createTicket),
109                     icon: Icon(
110                         Icons.mail_outline_rounded,
111                         color: ColorsManager.white,
112                         size: 18,
113                     ),
114                     label: Text(
115                         appTranslation().get('privacy_sec9_contact_btn'),
116                         style: TextStylesManager.bold13.copyWith(
117                             color: ColorsManager.white,
118                         ),
119                     ),
120                     style: ElevatedButton.styleFrom(
121                         backgroundColor: ColorsManager.primaryColor,
122                         shape: RoundedRectangleBorder(
123                             borderRadius: BorderRadius.circular(20),
124                         ),
125                         padding: const EdgeInsets.symmetric(
126                             horizontal: 20,
127                             vertical: 10,
128                         ),
129                         elevation: 0,
130                     ),
131                 ),
132                 Material(
133                     color: ColorsManager.surfacePrimary,
134                     borderRadius: BorderRadius.circular(20),
135                     child: InkWell(
136                         onTap: () => _launchEmail(email),
137                         borderRadius: BorderRadius.circular(20),
138                         child: Container(
139                             padding: const EdgeInsets.symmetric(
140                                 horizontal: 16,
141                                 vertical: 10,
142                             ),
143                             decoration: BoxDecoration(
144                                 borderRadius: BorderRadius.circular(20),
145                                 border: Border.all(color: ColorsManager.borderColor),
146                             ),
147                             child: Row(
148                                 mainAxisAlignment: MainAxisAlignment.min,
149                                 children: [
150                                     Icon(
151                                         Icons.email_outlined,
152                                         size: 16,
153                                         color: ColorsManager.primaryColor,
154                                     ),
155                                     horizontalSpace8,
156                                     Text(
157                                         email,
158                                         style: TextStylesManager.bold13.copyWith(
159                                             color: ColorsManager.primaryColor,
160                                         ),
161                                     ),
162                                 ],
163                             ),
164                         ),
165                     ),
166                 ),
167             ],
168         ),
169     ],
170 ),
171 ),
172 ],
173 ),
174 );
175 }

```

```
176 }
177
```

Lines • 79
Dart

Feature / Privacy #298 –

Policy [lib/features/privacy_policy/presentation/widgets/privacy_support_card_widget.dart](#)

```
1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/routes.dart';
6 import 'package:moean/core/utils/constants/spacing.dart';
7 import 'package:moean/core/utils/extensions/context_extension.dart';
8
9 class PrivacySupportCardWidget extends StatelessWidget {
10   const PrivacySupportCardWidget({super.key});
11
12   @override
13   Widget build(BuildContext context) {
14     return Container(
15       padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 24),
16       decoration: BoxDecoration(
17         color: ColorsManager.primaryColor.withValues(alpha: 0.05),
18         borderRadius: BorderRadius.circular(20),
19         border: Border.all(
20           color: ColorsManager.primaryColor.withValues(alpha: 0.15),
21         ),
22       ),
23       child: Column(
24         children: [
25           Container(
26             padding: const EdgeInsets.all(12),
27             decoration: BoxDecoration(
28               color: ColorsManager.primaryColor.withValues(alpha: 0.12),
29               shape: BoxShape.circle,
30             ),
31             child: Icon(
32               Icons.help_outline_rounded,
33               color: ColorsManager.primaryColor,
34               size: 28,
35             ),
36           ),
37           verticalSpace12,
38           Text(
39             appTranslation().get('privacy_ask_extra_title'),
40             style: TextStylesManager.bold16.copyWith(
41               color: ColorsManager.mainText,
42             ),
43             textAlign: TextAlign.center,
44           ),
45           verticalSpace6,
46           Text(
47             appTranslation().get('privacy_ask_extra_desc'),
48             style: TextStylesManager.regular12.copyWith(
49               color: ColorsManager.placeholder,
50             ),
51             textAlign: TextAlign.center,
52           ),
53           verticalSpace16,
54           ElevatedButton(
55             onPressed: () => context.push(Routes.createTicket),
56             style: ElevatedButton.styleFrom(
57               backgroundColor: ColorsManager.primaryColor,
58               shape: RoundedRectangleBorder(
59                 borderRadius: BorderRadius.circular(24),
60               ),
61               padding: const EdgeInsets.symmetric(
62                 horizontal: 28,
63                 vertical: 12,
64               ),
65               elevation: 0,
66             ),
67             child: Text(
```

```

68         appTranslation().get('privacy_contact_support_btn'),
69         style: TextStylesManager.bold14.copyWith(
70             color: ColorsManager.white,
71         ),
72     ),
73 ),
74 ],
75 ),
76 );
77 }
78 }
79

```

Lines • 30
Dart

Feature / Privacy #299 -

Policy lib/features/privacy_policy/presentation/widgets/privacy_tag_badge_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4
5  class PrivacyTagBadgeWidget extends StatelessWidget {
6      final String label;
7
8      const PrivacyTagBadgeWidget({
9          super.key,
10         required this.label,
11     });
12
13     @override
14     Widget build(BuildContext context) {
15         return Container(
16             padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 4),
17             decoration: BoxDecoration(
18                 color: ColorsManager.primaryColor.withValues(alpha: 0.1),
19                 borderRadius: BorderRadius.circular(20),
20             ),
21             child: Text(
22                 label,
23                 style: TextStylesManager.bold12.copyWith(
24                     color: ColorsManager.primaryColor,
25                 ),
26             ),
27         );
28     }
29 }
30

```

Lines 58
• Dart

Feature / #300 -

Privacy Policy lib/features/privacy_policy/presentation/widgets/privacy_teacher_right_item_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/spacing.dart';
5
6  class PrivacyTeacherRightItemWidget extends StatelessWidget {
7      final int number;
8      final String text;
9
10     const PrivacyTeacherRightItemWidget({
11         super.key,
12         required this.number,
13         required this.text,
14     });
15
16     @override
17     Widget build(BuildContext context) {
18         return Container(
19             padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
20             decoration: BoxDecoration(
21                 color: ColorsManager.surfacePrimary,
22                 borderRadius: BorderRadius.circular(14),

```

```

23     border: Border.all(color: ColorsManager.borderColor),
24   ),
25   child: Row(
26     children: [
27       Expanded(
28         child: Text(
29           text,
30           style: TextStylesManager.regular13.copyWith(
31             color: ColorsManager.mainText,
32             height: 1.5,
33           ),
34           textAlign: TextAlign.right,
35         ),
36       ),
37       horizontalSpace12,
38       Container(
39         width: 32,
40         height: 32,
41         alignment: Alignment.center,
42         decoration: BoxDecoration(
43           color: ColorsManager.primaryColor.withValues(alpha: 0.1),
44           borderRadius: BorderRadius.circular(8),
45         ),
46         child: Text(
47           '$number',
48           style: TextStylesManager.bold14.copyWith(
49             color: ColorsManager.primaryColor,
50           ),
51         ),
52       ),
53     ],
54   ),
55 );
56 }
57 }
58

```

Lines • 70
Dart

Feature / Privacy #301 -

Policy lib/features/privacy_policy/presentation/widgets/privacy_toc_item_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/spacing.dart';
5
6  class PrivacyTocItemWidget extends StatelessWidget {
7    final int index;
8    final String title;
9    final IconData icon;
10   final bool isSelected;
11   final VoidCallback onTap;
12
13   const PrivacyTocItemWidget({
14     super.key,
15     required this.index,
16     required this.title,
17     required this.icon,
18     required this.isSelected,
19     required this.onTap,
20   });
21
22   @override
23   Widget build(BuildContext context) {
24     final bgColor =
25       isSelected ? ColorsManager.primaryColor : Colors.transparent;
26     final textColor =
27       isSelected ? ColorsManager.white : ColorsManager.mainText;
28     final iconColor =
29       isSelected ? ColorsManager.white : ColorsManager.placeholder;
30
31     return InkWell(
32       onTap: onTap,
33       borderRadius: BorderRadius.circular(12),

```

```

34     child: AnimatedContainer(
35       duration: const Duration(milliseconds: 200),
36       padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
37       decoration: BoxDecoration(
38         color: bgColor,
39         borderRadius: BorderRadius.circular(12),
40       ),
41       child: Row(
42         children: [
43           Icon(
44             icon,
45             size: 20,
46             color: iconColor,
47           ),
48           horizontalSpace10,
49           Expanded(
50             child: Text(
51               title,
52               style: TextStylesManager.bold14.copyWith(
53                 color: textColor,
54               ),
55               textAlign: TextAlign.right,
56             ),
57           ),
58           horizontalSpace8,
59           Icon(
60             Icons.arrow_back_ios_rounded,
61             size: 14,
62             color: iconColor,
63           ),
64         ],
65       ),
66     ),
67   );
68 }
69 }
70

```

Lines - 133
Dart

Feature / Privacy #302 -

Policy [lib/features/privacy_policy/presentation/widgets/privacy_toc_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7  import 'package:moean/features/privacy_policy/presentation/cubit/privacy_policy_cubit.dart';
8  import 'package:moean/features/privacy_policy/presentation/cubit/privacy_policy_state.dart';
9  import 'package:moean/features/privacy_policy/presentation/widgets/privacy_toc_item_widget.dart';
10
11  class PrivacyTocWidget extends StatelessWidget {
12    const PrivacyTocWidget({super.key});
13
14    static final List<({int index, String key, IconData icon})> _tocItems = [
15      (index: 0, key: 'privacy_sec1_toc', icon: Icons.shield_outlined),
16      (index: 1, key: 'privacy_sec2_toc', icon: Icons.storage_rounded),
17      (index: 2, key: 'privacy_sec3_toc', icon: Icons.memory_rounded),
18      (index: 3, key: 'privacy_sec4_toc', icon: Icons.language_rounded),
19      (index: 4, key: 'privacy_sec5_toc', icon: Icons.lock_outline_rounded),
20      (index: 5, key: 'privacy_sec6_toc', icon: Icons.share_rounded),
21      (index: 6, key: 'privacy_sec7_toc', icon: Icons.person_outline_rounded),
22      (index: 7, key: 'privacy_sec8_toc', icon: Icons.cookie_outlined),
23      (index: 8, key: 'privacy_sec9_toc', icon: Icons.mail_outline_rounded),
24    ];
25
26    @override
27    Widget build(BuildContext context) {
28      final cubit = PrivacyPolicyCubit.get(context);
29
30      return Container(
31        padding: const EdgeInsets.all(16),
32        decoration: BoxDecoration(

```



```

33     color: ColorsManager.surfacePrimary,
34     borderRadius: BorderRadius.circular(20),
35     border: Border.all(color: ColorsManager.borderColor),
36   ),
37   child: Column(
38     crossAxisAlignment: CrossAxisAlignment.stretch,
39     children: [
40       Row(
41         children: [
42           Icon(
43             Icons.article_outlined,
44             color: ColorsManager.primaryColor,
45             size: 22,
46           ),
47           horizontalSpace8,
48           Expanded(
49             child: Text(
50               appTranslation().get('privacy_toc_title'),
51               style: TextStylesManager.bold16.copyWith(
52                 color: ColorsManager.primaryColor,
53               ),
54               textAlign: TextAlign.right,
55             ),
56           ),
57         ],
58       ),
59       verticalSpace16,
60       TextField(
61         controller: cubit.searchController,
62         onChanged: cubit.onSearchChanged,
63         textAlign: TextAlign.right,
64         style: TextStylesManager.regular14.copyWith(
65           color: ColorsManager.mainText,
66         ),
67         decoration: InputDecoration(
68           hintText: appTranslation().get('privacy_search_hint'),
69           hintStyle: TextStylesManager.regular13.copyWith(
70             color: ColorsManager.placeholder,
71           ),
72           suffixIcon: Icon(
73             Icons.search_rounded,
74             color: ColorsManager.placeholder,
75             size: 20,
76           ),
77           filled: true,
78           fillColor: ColorsManager.background,
79           contentPadding: const EdgeInsets.symmetric(
80             horizontal: 16,
81             vertical: 10,
82           ),
83           border: OutlineInputBorder(
84             borderRadius: BorderRadius.circular(24),
85             borderSide: BorderSide(color: ColorsManager.borderColor),
86           ),
87           enabledBorder: OutlineInputBorder(
88             borderRadius: BorderRadius.circular(24),
89             borderSide: BorderSide(color: ColorsManager.borderColor),
90           ),
91           focusedBorder: OutlineInputBorder(
92             borderRadius: BorderRadius.circular(24),
93             borderSide: BorderSide(color: ColorsManager.primaryColor),
94           ),
95         ),
96       ),
97       verticalSpace12,
98       BlocBuilder<PrivacyPolicyCubit, PrivacyPolicyState>(
99         buildWhen: (prev, curr) => curr is PrivacyPolicyStateUpdated,
100        builder: (context, state) {
101          final activeIndex = cubit.activeSection;
102          final query = cubit.searchQuery;
103
104          final filteredList = _tocItems.where((item) {
105            if (query.isEmpty) return true;
106            final title = appTranslation().get(item.key).toLowerCase();
107            return title.contains(query);

```

```

108         }).toList();
109
110         return ListView.separated(
111             shrinkWrap: true,
112             physics: const NeverScrollableScrollPhysics(),
113             itemCount: filteredList.length,
114             separatorBuilder: (context, index) => verticalSpace4,
115             itemBuilder: (context, i) {
116                 final item = filteredList[i];
117                 return PrivacyTocItemWidget(
118                     index: item.index,
119                     title: appTranslation().get(item.key),
120                     icon: item.icon,
121                     isSelected: activeIndex == item.index,
122                     onTap: () => cubit.scrollToSection(item.index),
123                 );
124             },
125         );
126     },
127 ),
128 ],
129 ),
130 );
131 }
132 }
133

```

Lines • Dart 35

Feature / Profile

#303 – lib/features/profile/presentation/cubit/change_password_cubit.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/network/remote/api_service.dart';
4  import 'package:moean/features/profile/presentation/cubit/change_password_state.dart';
5
6  class ChangePasswordCubit extends Cubit<ChangePasswordState> {
7      ChangePasswordCubit() : super(ChangePasswordInitial());
8
9      static ChangePasswordCubit get(BuildContext context) => BlocProvider.of(context);
10
11      Future<void> changePassword({
12          required String currentPassword,
13          required String password,
14          required String passwordConfirmation,
15      }) async {
16          if (!isClosed) emit(ChangePasswordLoading());
17
18          final result = await ApiService.changePassword(
19              currentPassword: currentPassword,
20              password: password,
21              passwordConfirmation: passwordConfirmation,
22          );
23          if (isClosed) return;
24
25          result.fold(
26              (error) {
27                  if (!isClosed) emit(ChangePasswordFailure(error));
28              },
29              (success) {
30                  if (!isClosed) emit(ChangePasswordSuccess());
31              },
32          );
33      }
34  }
35

```

Lines • Dart 14

Feature / Profile

#304 – lib/features/profile/presentation/cubit/change_password_state.dart

```

1  abstract class ChangePasswordState {}
2
3  class ChangePasswordInitial extends ChangePasswordState {}
4
5  class ChangePasswordLoading extends ChangePasswordState {}

```

```

6
7 class ChangePasswordSuccess extends ChangePasswordState {}
8
9 class ChangePasswordFailure extends ChangePasswordState {
10   final String message;
11
12   ChangePasswordFailure(this.message);
13 }
14

```

Lines • Dart 82

Feature / Profile

#305 – lib/features/profile/presentation/cubit/profile_cubit.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/network/local/secure_storage_helper.dart';
4 import 'package:moean/core/network/remote/api_service.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/features/profile/presentation/cubit/profile_state.dart';
7 import 'package:moean/core/models/profile_model.dart';
8 import 'package:moean/core/di/injections.dart';
9 import 'package:moean/core/services/madrasati_session_service.dart';
10 import 'package:moean/features/payment/presentation/cubit/subscription_cubit.dart';
11
12 class ProfileCubit extends Cubit<ProfileState> {
13   ProfileCubit() : super(ProfileInitialState());
14
15   static ProfileCubit get(BuildContext context) => BlocProvider.of(context);
16
17   ProfileModel? profileModel;
18   bool _isLoading = false;
19
20   Future<void> fetchProfile({bool forceRefresh = false}) async {
21     if (token == null || token!.isEmpty) return;
22     if (_isLoading) return;
23
24     if (!forceRefresh && profileModel != null) {
25       if (!isClosed) emit(ProfileLoadedState(profile: profileModel!));
26       return;
27     }
28
29     _isLoading = true;
30     if (profileModel == null && !isClosed) {
31       emit(ProfileLoadingState());
32     }
33
34     final result = await ApiService.getProfile();
35     _isLoading = false;
36     if (isClosed) return;
37
38     result.fold(
39       (error) {
40         if (!isClosed) emit(ProfileErrorState(message: error));
41       },
42       (profile) {
43         profileModel = profile;
44         if (!isClosed) emit(ProfileLoadedState(profile: profile));
45       },
46     );
47   }
48
49   Future<void> logout() async {
50     if (!isClosed) emit(ProfileLogoutLoadingState());
51
52     try {
53       // 1. مسح التوكن فوراً من الذاكرة والـ 1.
54       final secureStorage = sl<SecureStorageHelper>();
55       await secureStorage.deleteToken();
56       token = null;
57       clearData();
58       sl<SubscriptionCubit>().clearData();
59
60       // 2. إرسال حالة النجاح فوراً لكي تلتقطها الشاشة قبل أي Rebuild
61       if (!isClosed) emit(ProfileLogoutSuccessState());

```

```

62
63 // 3. تصفير جلسة مدرستي وطلب السيرفر في الخلفية
64 sl<MadrasatiSessionService>().reset();
65 ApiService.logout();
66
67 } catch (e) {
68   debugPrint('Logout error: $e');
69   // في حال حدوث خطأ، نضمن أيضاً إرسال النجاح لتسجيل الخروج محلياً
70   token = null;
71   clearData();
72   sl<SubscriptionCubit>().clearData();
73   if (!isClosed) emit(ProfileLogoutSuccessState());
74 }
75 }
76
77 void clearData() {
78   profileModel = null;
79   if (!isClosed) emit(ProfileInitialState());
80 }
81 }
82

```

Lines • Dart 27

Feature / Profile

#306 – lib/features/profile/presentation/cubit/profile_state.dart

```

1 import 'package:moean/core/models/profile_model.dart';
2
3 abstract class ProfileState {}
4
5 class ProfileInitialState extends ProfileState {}
6
7 class ProfileLoadingState extends ProfileState {}
8
9 class ProfileLoadedState extends ProfileState {
10   final ProfileModel profile;
11   ProfileLoadedState({required this.profile});
12 }
13
14 class ProfileErrorState extends ProfileState {
15   final String message;
16   ProfileErrorState({required this.message});
17 }
18
19 class ProfileLogoutLoadingState extends ProfileState {}
20
21 class ProfileLogoutSuccessState extends ProfileState {}
22
23 class ProfileLogoutErrorState extends ProfileState {
24   final String message;
25   ProfileLogoutErrorState({required this.message});
26 }
27

```

Lines • Dart 256

Feature / Profile

#307 – lib/features/profile/presentation/screen/change_password_screen.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
7 import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
8 import 'package:moean/features/profile/presentation/cubit/change_password_cubit.dart';
9 import 'package:moean/features/profile/presentation/cubit/change_password_state.dart';
10
11 class ChangePasswordScreen extends StatelessWidget {
12   const ChangePasswordScreen({super.key});
13
14   @override
15   Widget build(BuildContext context) {
16     return BlocProvider(
17       create: (context) => ChangePasswordCubit(),
18       child: const _ChangePasswordScreenBody(),
19     );
20   }
21 }
22

```

```

19     );
20   }
21 }
22
23 class _ChangePasswordScreenBody extends StatefulWidget {
24   const _ChangePasswordScreenBody();
25
26   @override
27   State<_ChangePasswordScreenBody> createState() => _ChangePasswordScreenBodyState();
28 }
29
30 class _ChangePasswordScreenBodyState extends State<_ChangePasswordScreenBody> {
31   final _formKey = GlobalKey<FormState>();
32   final _currentPasswordController = TextEditingController();
33   final _newPasswordController = TextEditingController();
34   final _confirmPasswordController = TextEditingController();
35
36   bool _isCurrentPasswordVisible = false;
37   bool _isNewPasswordVisible = false;
38   bool _isConfirmPasswordVisible = false;
39
40   @override
41   void dispose() {
42     _currentPasswordController.dispose();
43     _newPasswordController.dispose();
44     _confirmPasswordController.dispose();
45     super.dispose();
46   }
47
48   @override
49   Widget build(BuildContext context) {
50     return Scaffold(
51       backgroundColor: ColorsManager.background,
52       appBar: AppBar(
53         backgroundColor: ColorsManager.surfacePrimary,
54         elevation: 0,
55         centerTitle: true,
56         leading: IconButton(
57           icon: Icon(Icons.arrow_back_ios, color: ColorsManager.textPrimary, size: 20),
58           onPressed: () => Navigator.pop(context),
59         ),
60         title: Text(
61           appTranslation().get('change_password'),
62           style: TextStylesManager.bold18.copyWith(
63             color: ColorsManager.textPrimary,
64           ),
65         ),
66       ),
67       body: BlocConsumer<ChangePasswordCubit, ChangePasswordState>(
68         listener: (context, state) {
69           if (state is ChangePasswordSuccess) {
70             ScaffoldMessenger.of(context).showSnackBar(
71               SnackBar(
72                 content: Text(
73                   appTranslation().get('save_changes'),
74                   style: TextStylesManager.medium14.copyWith(color: Colors.white),
75                 ),
76                 backgroundColor: Colors.green,
77               ),
78             );
79             Navigator.pop(context);
80           } else if (state is ChangePasswordFailure) {
81             ScaffoldMessenger.of(context).showSnackBar(
82               SnackBar(
83                 content: Text(
84                   state.message,
85                   style: TextStylesManager.medium14.copyWith(color: Colors.white),
86                 ),
87                 backgroundColor: ColorsManager.errorColor,
88               ),
89             );
90           }
91         },
92         builder: (context, state) {
93           return SingleChildScrollView(

```

```

94     padding: const EdgeInsets.all(24.0),
95     child: Form(
96       key: _formKey,
97       child: Column(
98         crossAxisAlignment: CrossAxisAlignment.center,
99         children: [
100           const SizedBox(height: 16),
101           Container(
102             padding: const EdgeInsets.all(24),
103             decoration: BoxDecoration(
104               color: ColorsManager.primaryColor.withValues(alpha: 0.1),
105               shape: BoxShape.circle,
106             ),
107             child: Icon(
108               Icons.lock_person_outlined,
109               size: 64,
110               color: ColorsManager.primaryColor,
111             ),
112           ),
113           const SizedBox(height: 32),
114           Text(
115             appTranslation().get('about_password'),
116             textAlign: TextAlign.center,
117             style: TextStylesManager.regular14.copyWith(
118               color: ColorsManager.textSecondary,
119             ),
120           ),
121           const SizedBox(height: 32),
122           Container(
123             padding: const EdgeInsets.all(20),
124             decoration: BoxDecoration(
125               color: ColorsManager.surfacePrimary,
126               borderRadius: BorderRadius.circular(16),
127               border: Border.all(
128                 color: ColorsManager.borderColor,
129               ),
130             boxShadow: [
131               BoxShadow(
132                 color: Colors.black.withValues(alpha: 0.02),
133                 blurRadius: 10,
134                 offset: const Offset(0, 4),
135               ),
136             ],
137           ),
138           child: Column(
139             crossAxisAlignment: CrossAxisAlignment.start,
140             children: [
141               _buildLabel('current_password'),
142               const SizedBox(height: 8),
143               PrimaryTextField(
144                 controller: _currentPasswordController,
145                 hint: appTranslation().get('current_password'),
146                 isPassword: !_isCurrentPasswordVisible,
147                 prefixIcon: const Icon(Icons.lock_outline),
148                 suffixIcon: IconButton(
149                   icon: Icon(
150                     _isCurrentPasswordVisible ? Icons.visibility : Icons.visibility_off,
151                     color: ColorsManager.textSecondary,
152                   ),
153                   onPressed: () {
154                     setState(() {
155                       _isCurrentPasswordVisible = !_isCurrentPasswordVisible;
156                     });
157                   },
158                 ),
159               validator: (value) {
160                 if (value == null || value.isEmpty) {
161                   return appTranslation().get('password_required');
162                 }
163                 return null;
164               },
165             ),
166             const SizedBox(height: 20),
167             _buildLabel('new_password'),
168             const SizedBox(height: 8),

```

```

169         PrimaryTextField(
170             controller: _newPasswordController,
171             hint: appTranslation().get('new_password'),
172             isPassword: !_isNewPasswordVisible,
173             prefixIcon: const Icon(Icons.lock_outline),
174             suffixIcon: IconButton(
175                 icon: Icon(
176                     _isNewPasswordVisible ? Icons.visibility : Icons.visibility_off,
177                     color: ColorsManager.textSecondary,
178                 ),
179                 onPressed: () {
180                     setState(() {
181                         _isNewPasswordVisible = !_isNewPasswordVisible;
182                     });
183                 },
184             ),
185             validator: (value) {
186                 if (value == null || value.isEmpty) {
187                     return appTranslation().get('password_required');
188                 }
189                 return null;
190             },
191         ),
192         const SizedBox(height: 20),
193         _buildLabel('confirm_new_password'),
194         const SizedBox(height: 8),
195         PrimaryTextField(
196             controller: _confirmPasswordController,
197             hint: appTranslation().get('confirm_new_password'),
198             isPassword: !_isConfirmPasswordVisible,
199             prefixIcon: const Icon(Icons.lock_outline),
200             suffixIcon: IconButton(
201                 icon: Icon(
202                     _isConfirmPasswordVisible ? Icons.visibility : Icons.visibility_off,
203                     color: ColorsManager.textSecondary,
204                 ),
205                 onPressed: () {
206                     setState(() {
207                         _isConfirmPasswordVisible = !_isConfirmPasswordVisible;
208                     });
209                 },
210             ),
211             validator: (value) {
212                 if (value == null || value.isEmpty) {
213                     return appTranslation().get('confirm_password_required');
214                 }
215                 if (value != _newPasswordController.text) {
216                     return appTranslation().get('passwords_not_match');
217                 }
218                 return null;
219             },
220         ),
221     ],
222 ),
223 ),
224 const SizedBox(height: 40),
225 PrimaryElevatedButton(
226     text: appTranslation().get('save_changes'),
227     isLoading: state is ChangePasswordLoading,
228     onPressed: () {
229         if (_formKey.currentState!.validate()) {
230             ChangePasswordCubit.get(context).changePassword(
231                 currentPassword: _currentPasswordController.text,
232                 password: _newPasswordController.text,
233                 passwordConfirmation: _confirmPasswordController.text,
234             );
235         }
236     },
237 ),
238 ],
239 ),
240 ),
241 );
242 },
243 ),

```

```

244     );
245   }
246
247   Widget _buildLabel(String key) {
248     return Text(
249       appTranslation().get(key),
250       style: TextStylesManager.medium14.copyWith(
251         color: ColorsManager.textPrimary,
252       ),
253     );
254   }
255 }
256

```

Lines • Dart 212

Feature / Profile

#308 – lib/features/profile/presentation/screen/profile_screen.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/routes.dart';
7  import 'package:moean/core/utils/constants/primary/conditional_builder.dart';
8  import 'package:moean/core/utils/cubit/theme/theme_cubit.dart';
9  import 'package:moean/core/utils/cubit/theme/theme_state.dart';
10 import 'package:moean/features/profile/presentation/cubit/profile_cubit.dart';
11 import 'package:moean/features/profile/presentation/cubit/profile_state.dart';
12 import 'package:moean/features/profile/presentation/widgets/profile_info_card.dart';
13 import 'package:moean/features/profile/presentation/widgets/subscription_info_card.dart';
14 import 'package:moean/features/payment/presentation/cubit/subscription_cubit.dart';
15 import 'package:moean/features/payment/presentation/cubit/subscription_state.dart';
16 import 'package:moean/core/di/injections.dart';
17
18 class ProfileScreen extends StatefulWidget {
19   const ProfileScreen({super.key});
20
21   @override
22   State<ProfileScreen> createState() => _ProfileScreenState();
23 }
24
25 class _ProfileScreenState extends State<ProfileScreen> {
26   @override
27   void initState() {
28     super.initState();
29     final cubit = sl<ProfileCubit>();
30     // Force refresh profile so subscription_ends_at is always up to date
31     cubit.fetchProfile(forceRefresh: true);
32     sl<SubscriptionCubit>().fetchCurrentSubscription(forceRefresh: true);
33   }
34
35   @override
36   Widget build(BuildContext context) {
37     return BlocConsumer<ProfileCubit, ProfileState>(
38       bloc: sl<ProfileCubit>(),
39       listener: (context, state) {
40         if (state is ProfileLogoutSuccessState) {
41           Navigator.pushNamedAndRemoveUntil(
42             context,
43             Routes.login,
44             (route) => false,
45           );
46         }
47       },
48       builder: (context, state) {
49         final cubit = sl<ProfileCubit>();
50         return BlocBuilder<ThemeCubit, ThemeState>(
51           builder: (context, themeState) {
52             return Scaffold(
53               backgroundColor: ColorsManager.background,
54               appBar: AppBar(
55                 backgroundColor: ColorsManager.surfacePrimary,
56                 elevation: 0,
57                 centerTitle: true,

```



```

133         style: TextStylesManager.bold20.copyWith(color: ColorsManager.textPrimary),
134     ),
135 ),
136 const SizedBox(height: 32),
137 ProfileInfoCard(
138     icon: Icons.email_outlined,
139     title: appTranslation().get('email'),
140     value: profile.user.email,
141 ),
142 if (profile.role != null)
143     ProfileInfoCard(
144         icon: Icons.work_outline,
145         title: appTranslation().get('role'),
146         value: profile.role!,
147     ),
148 if (profile.phone != null)
149     ProfileInfoCard(
150         icon: Icons.phone_outlined,
151         title: appTranslation().get('phone_number'),
152         value: profile.phone!,
153     ),
154 if (profile.teacher != null)
155     SubscriptionInfoCard(
156         teacher: profile.teacher!,
157         onUpgradeTap: () => Navigator.pushNamed(context, Routes.checkout),
158     )
159 else if (profile.subscriptionName != null)
160     ProfileInfoCard(
161         icon: Icons.star_border,
162         title: appTranslation().get('subscription'),
163         value: profile.subscriptionName!,
164         iconColor: ColorsManager.secondaryColor,
165     ),
166 BlocBuilder<SubscriptionCubit, SubscriptionState>(
167     bloc: sl<SubscriptionCubit>(),
168     builder: (context, subState) {
169         if (subState is SubscriptionLoaded) {
170             final usage = subState.current.usage;
171             return Column(
172                 children: [
173                     // ProfileInfoCard(
174                     //     icon: Icons.auto_awesome_outlined,
175                     //     title: 'رصيد الذكاء الاصطناعي',
176                     //     value: usage.aiRemaining.toString(),
177                     //     iconColor: ColorsManager.themePink,
178                     // ),
179                     ProfileInfoCard(
180                         icon: Icons.chrome_reader_mode_outlined,
181                         title: 'الدروس المتبقية لليوم',
182                         value: usage.lessonsRemainingToday.toString(),
183                         iconColor: ColorsManager.primaryColor,
184                     ),
185                 ],
186             );
187         }
188         return const SizedBox.shrink();
189     },
190 ),
191 ProfileInfoCard(
192     icon: profile.madrasatiConnected ? Icons.link : Icons.link_off,
193     title: appTranslation().get('madrasati_status'),
194     value: profile.madrasatiConnected
195         ? appTranslation().get('connected')
196         : appTranslation().get('not_connected'),
197     iconColor: profile.madrasatiConnected ? ColorsManager.successColor : ColorsManage
r.errorColor,
198 ),
199 ],
200 ),
201 ),
202 );
203 },
204 ),
205 );
206 },

```

```

207     );
208   },
209 );
210 }
211 }
212

```

Lines • Dart 181

Feature / Profile

#309 – lib/features/profile/presentation/screen/settings_screen.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/routes.dart';
7  import 'package:moean/core/utils/cubit/theme/theme_cubit.dart';
8  import 'package:moean/core/utils/cubit/theme/theme_state.dart';
9  import 'package:moean/features/profile/presentation/cubit/profile_cubit.dart';
10 import 'package:moean/features/profile/presentation/cubit/profile_state.dart';
11 import 'package:moean/core/di/injections.dart';
12 import 'package:moean/features/profile/presentation/widgets/logout_dialog.dart';
13
14 class SettingsScreen extends StatelessWidget {
15   const SettingsScreen({super.key});
16
17   @override
18   Widget build(BuildContext context) {
19     return BlocConsumer<ProfileCubit, ProfileState>(
20       bloc: sl<ProfileCubit>(),
21       listener: (context, state) {
22         if (state is ProfileLogoutSuccessState) {
23           Navigator.pushNamedAndRemoveUntil(
24             context,
25             Routes.login,
26             (route) => false,
27           );
28         }
29       },
30       builder: (context, state) {
31         final profileCubit = sl<ProfileCubit>();
32         return BlocBuilder<ThemeCubit, ThemeState>(
33           builder: (context, themeState) {
34             final themeCubit = ThemeCubit.get(context);
35             return Scaffold(
36               backgroundColor: ColorsManager.background,
37               appBar: AppBar(
38                 backgroundColor: ColorsManager.surfacePrimary,
39                 elevation: 0,
40                 // automaticallyImplyLeading :false,
41                 centerTitle: true,
42                 title: Text(
43                   appTranslation().get('settings'),
44                   style: TextStylesManager.bold18.copyWith(
45                     color: ColorsManager.textPrimary,
46                   ),
47                 ),
48               ),
49               body: ListView(
50                 padding: const EdgeInsets.all(24.0),
51                 children: [
52                   // Dark Mode Toggle
53                   ListTile(
54                     contentPadding: EdgeInsets.zero,
55                     leading: Icon(
56                       themeCubit.isDarkMode ? Icons.dark_mode : Icons.light_mode,
57                       color: ColorsManager.primaryColor,
58                     ),
59                     title: Text(
60                       appTranslation().get('dark_mode'),
61                       style: TextStylesManager.medium16.copyWith(
62                         color: ColorsManager.textPrimary,
63                       ),
64                   ),

```

```

65         trailing: Switch(
66             value: themeCubit.isDarkMode,
67             onChanged: (value) {
68                 themeCubit.changeTheme();
69             },
70             activeThumbColor: ColorsManager.primaryColor,
71         ),
72     ),
73     const Divider(),
74     // Change Password
75     ListTile(
76         contentPadding: EdgeInsets.zero,
77         leading: Icon(
78             Icons.lock_outline,
79             color: ColorsManager.primaryColor,
80         ),
81         title: Text(
82             appTranslation().get('change_password'),
83             style: TextStyleManager.medium16.copyWith(
84                 color: ColorsManager.textPrimary,
85             ),
86         ),
87         trailing: Icon(
88             Icons.arrow_forward_ios,
89             size: 16,
90             color: ColorsManager.textSecondary,
91         ),
92         onTap: () {
93             Navigator.pushNamed(context, Routes.changePassword);
94         },
95     ),
96     const Divider(),
97     // Language Toggle
98     ListTile(
99         contentPadding: EdgeInsets.zero,
100         leading: Icon(
101             Icons.language,
102             color: ColorsManager.primaryColor,
103         ),
104         title: Text(
105             appTranslation().get('language'),
106             style: TextStyleManager.medium16.copyWith(
107                 color: ColorsManager.textPrimary,
108             ),
109         ),
110         trailing: Container(
111             decoration: BoxDecoration(
112                 color: ColorsManager.primaryColor.withValues(alpha: 0.1),
113                 borderRadius: BorderRadius.circular(8),
114             ),
115             padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 4),
116             child: DropdownButtonHideUnderline(
117                 child: DropdownButton<bool>(
118                     value: themeCubit.isArabicLang,
119                     icon: Icon(Icons.arrow_drop_down, color: ColorsManager.primaryColor),
120                     onChanged: (bool? newValue) {
121                         if (newValue != null && newValue != themeCubit.isArabicLang) {
122                             themeCubit.changeLanguage();
123                         }
124                     },
125                     items: [
126                         DropdownMenuItem(
127                             value: true,
128                             child: Text(
129                                 appTranslation().get('arabic'),
130                                 style: TextStyleManager.medium14.copyWith(
131                                     color: ColorsManager.primaryColor,
132                                 ),
133                             ),
134                         ),
135                         DropdownMenuItem(
136                             value: false,
137                             child: Text(
138                                 appTranslation().get('english'),
139                                 style: TextStyleManager.medium14.copyWith(

```

```

140             color: ColorsManager.primaryColor,
141         ),
142     ),
143 ),
144 ],
145 ),
146 ),
147 ),
148 ),
149 const Divider(),
150 // Logout Button
151 ListTile(
152   contentPadding: EdgeInsets.zero,
153   leading: const Icon(
154     Icons.logout,
155     color: ColorsManager.errorColor,
156   ),
157   title: Text(
158     appTranslation().get('logout'),
159     style: TextStyleManager.medium16.copyWith(
160       color: ColorsManager.errorColor,
161     ),
162   ),
163   onTap: () {
164     showDialog(
165       context: context,
166       builder: (dialogContext) => LogoutDialog(
167         onConfirm: () => profileCubit.logout(),
168       ),
169     );
170   },
171 ),
172 ],
173 ),
174 );
175 },
176 );
177 },
178 );
179 }
180 }
181

```

Lines • Dart 62

Feature / Profile

#310 – lib/features/profile/presentation/widgets/logout_dialog.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5
6  class LogoutDialog extends StatelessWidget {
7    final VoidCallback onConfirm;
8
9    const LogoutDialog({super.key, required this.onConfirm});
10
11    @override
12    Widget build(BuildContext context) {
13      return AlertDialog(
14        backgroundColor: ColorsManager.surfacePrimary,
15        shape: RoundedRectangleBorder(
16          borderRadius: BorderRadius.circular(16),
17        ),
18        title: Text(
19          appTranslation().get('logout'),
20          style: TextStyleManager.bold18.copyWith(
21            color: ColorsManager.textPrimary,
22          ),
23        ),
24        content: Text(
25          appTranslation().get('logout_desc'),
26          style: TextStyleManager.regular14.copyWith(
27            color: ColorsManager.textSecondary,
28          ),

```

```

29     ),
30     actions: [
31       TextButton(
32         onPressed: () => Navigator.pop(context),
33         child: Text(
34           appTranslation().get('return_home'),
35           style: TextStylesManager.bold14.copyWith(
36             color: ColorsManager.textSecondary,
37           ),
38         ),
39       ),
40       ElevatedButton(
41         onPressed: () {
42           Navigator.pop(context);
43           onConfirm();
44         },
45         style: ElevatedButton.styleFrom(
46           backgroundColor: ColorsManager.errorColor,
47           foregroundColor: ColorsManager.white,
48           shape: RoundedRectangleBorder(
49             borderRadius: BorderRadius.circular(8),
50           ),
51           padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
52         ),
53         child: Text(
54           appTranslation().get('logout'),
55           style: TextStylesManager.bold14,
56         ),
57       ),
58     ],
59   );
60 }
61 }
62

```

Lines • Dart 69

Feature / Profile

#311 – lib/features/profile/presentation/widgets/profile_info_card.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4
5  class ProfileInfoCard extends StatelessWidget {
6    final IconData icon;
7    final String title;
8    final String value;
9    final Color? iconColor;
10
11    const ProfileInfoCard({
12      super.key,
13      required this.icon,
14      required this.title,
15      required this.value,
16      this.iconColor,
17    });
18
19    @override
20    Widget build(BuildContext context) {
21      return Container(
22        margin: const EdgeInsets.only(bottom: 16),
23        padding: const EdgeInsets.all(16),
24        decoration: BoxDecoration(
25          color: ColorsManager.surfacePrimary,
26          borderRadius: BorderRadius.circular(12),
27          border: Border.all(color: ColorsManager.borderColor),
28        ),
29        child: Row(
30          children: [
31            Container(
32              padding: const EdgeInsets.all(10),
33              decoration: BoxDecoration(
34                color: (iconColor ?? ColorsManager.primaryColor).withValues(alpha: 0.1),
35                shape: BoxShape.circle,
36              ),

```

```

37         child: Icon(
38             icon,
39             color: iconColor ?? ColorsManager.primaryColor,
40             size: 24,
41         ),
42     ),
43     const SizedBox(width: 16),
44     Expanded(
45         child: Column(
46             crossAxisAlignment: CrossAxisAlignment.start,
47             children: [
48                 Text(
49                     title,
50                     style: TextStylesManager.regular14.copyWith(
51                         color: ColorsManager.textSecondary,
52                     ),
53                 ),
54                 const SizedBox(height: 4),
55                 Text(
56                     value,
57                     style: TextStylesManager.bold16.copyWith(
58                         color: ColorsManager.textPrimary,
59                     ),
60                 ),
61             ],
62         ),
63     ),
64 ],
65 ),
66 );
67 }
68 }
69

```

Lines • Dart 129

Feature / Profile

#312 – lib/features/profile/presentation/widgets/subscription_info_card.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/models/teacher_model.dart';
3  import 'package:moean/core/theme/colors.dart';
4
5  class SubscriptionInfoCard extends StatelessWidget {
6      final TeacherModel teacher;
7      final VoidCallback onUpgradeTap;
8
9      const SubscriptionInfoCard({
10         super.key,
11         required this.teacher,
12         required this.onUpgradeTap,
13     });
14
15     @override
16     Widget build(BuildContext context) {
17         final isSubscribed = teacher.isSubscribed;
18         final isInTrial = teacher.isInTrial && !teacher.isExpired;
19
20         final Color badgeBg = isSubscribed
21             ? const Color(0xFFEAF7F2)
22             : (isInTrial ? const Color(0xFFFEF3C7) : const Color(0xFFFFEE2E2));
23         final Color badgeText = isSubscribed
24             ? const Color(0xFF0E7A5E)
25             : (isInTrial ? const Color(0xFFB45309) : const Color(0xFFDC2626));
26
27         return Container(
28             padding: const EdgeInsets.all(16),
29             margin: const EdgeInsets.symmetric(vertical: 8),
30             decoration: BoxDecoration(
31                 color: ColorsManager.surfacePrimary,
32                 borderRadius: BorderRadius.circular(20),
33                 border: Border.all(color: const Color(0xFFDDEEE8)),
34                 boxShadow: [
35                     BoxShadow(
36                         color: Colors.black.withValues(alpha: 0.04),
37                         blurRadius: 12,

```

```

38         offset: const Offset(0, 4),
39     ),
40 ],
41 ),
42 child: Column(
43   crossAxisAlignment: CrossAxisAlignment.start,
44   children: [
45     Row(
46       mainAxisAlignment: MainAxisAlignment.spaceBetween,
47       children: [
48         Row(
49           children: [
50             Icon(
51               isSubscribed
52                 ? Icons.bolt
53                 : (isInTrial ? Icons.hourglass_top : Icons.lock_clock),
54               color: badgeText,
55               size: 20,
56             ),
57             const SizedBox(width: 8),
58             Text(
59               teacher.planTitle,
60               style: TextStyle(
61                 fontWeight: FontWeight.bold,
62                 fontSize: 16,
63                 color: ColorsManager.textPrimary,
64               ),
65             ),
66           ],
67         ),
68         Container(
69           padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
70           decoration: BoxDecoration(
71             color: badgeBg,
72             borderRadius: BorderRadius.circular(12),
73           ),
74           child: Text(
75             isSubscribed ? 'نشط' : (isInTrial ? 'منتهي' : 'تجربة'),
76             style: TextStyle(
77               color: badgeText,
78               fontWeight: FontWeight.bold,
79               fontSize: 12,
80             ),
81           ),
82         ),
83       ],
84     ),
85     const SizedBox(height: 10),
86     Row(
87       children: [
88         Icon(Icons.calendar_today_outlined,
89           size: 14, color: Colors.grey.shade600),
90         const SizedBox(width: 6),
91         Expanded(
92           child: Text(
93             teacher.expirationSubtitle,
94             style: TextStyle(
95               color: ColorsManager.textSecondary,
96               fontSize: 13,
97               fontWeight: FontWeight.w500,
98             ),
99           ),
100         ),
101       ],
102     ),
103     if (!isSubscribed) ...[
104       const SizedBox(height: 14),
105       SizedBox(
106         width: double.infinity,
107         child: ElevatedButton(
108           onPressed: onUpgradeTap,
109           style: ElevatedButton.styleFrom(
110             backgroundColor: ColorsManager.primaryColor,
111             foregroundColor: Colors.white,
112             shape: RoundedRectangleBorder(

```



```

113         borderRadius: BorderRadius.circular(12),
114     ),
115     padding: const EdgeInsets.symmetric(vertical: 10),
116 ),
117 child: Text(
118     isInTrial ? 'اشترك لمتابعة الاستخدام' : '⚡ ترقية الخطة الآن ⚡',
119     style: const TextStyle(fontWeight: FontWeight.bold),
120 ),
121 ),
122 ),
123 ],
124 ],
125 ),
126 );
127 }
128 }
129

```

Lines • Dart 93

Feature / Referral

#313 – lib/features/referral/data/models/referral_dashboard_model.dart

```

1  class ReferralRewardModel {
2      final int id;
3      final String code;
4      final double discountValue;
5      final String discountType;
6      final String? expiresAt;
7
8      const ReferralRewardModel({
9          required this.id,
10         required this.code,
11         required this.discountValue,
12         required this.discountType,
13         this.expiresAt,
14     });
15
16     factory ReferralRewardModel.fromJson(Map<String, dynamic> json) {
17         return ReferralRewardModel(
18             id: json['id'] as int? ?? 0,
19             code: json['code'] as String? ?? '',
20             discountValue: (json['discount_value'] as num?).toDouble() ?? 0.0,
21             discountType: json['discount_type'] as String? ?? 'percentage',
22             expiresAt: json['expires_at'] as String?,
23         );
24     }
25 }
26
27 class ReferralHistoryItemModel {
28     final int id;
29     final String referredName;
30     final String status;
31     final String registeredAt;
32     final String? qualifiedAt;
33
34     const ReferralHistoryItemModel({
35         required this.id,
36         required this.referredName,
37         required this.status,
38         required this.registeredAt,
39         this.qualifiedAt,
40     });
41
42     factory ReferralHistoryItemModel.fromJson(Map<String, dynamic> json) {
43         return ReferralHistoryItemModel(
44             id: json['id'] as int? ?? 0,
45             referredName: json['referred_name'] as String? ?? '',
46             status: json['status'] as String? ?? 'pending',
47             registeredAt: json['registered_at'] as String? ?? '',
48             qualifiedAt: json['qualified_at'] as String?,
49         );
50     }
51 }
52
53 class ReferralDashboardModel {

```

```

54 final String referralCode;
55 final String referralLink;
56 final int maxReferrals;
57 final int qualifiedReferrals;
58 final int remainingReferrals;
59 final int rewardsAvailableCount;
60 final List<ReferralRewardModel> rewards;
61 final List<ReferralHistoryItemModel> history;
62
63 const ReferralDashboardModel({
64   required this.referralCode,
65   required this.referralLink,
66   required this.maxReferrals,
67   required this.qualifiedReferrals,
68   required this.remainingReferrals,
69   required this.rewardsAvailableCount,
70   required this.rewards,
71   required this.history,
72 });
73
74 factory ReferralDashboardModel.fromJson(Map<String, dynamic> json) {
75   final rewardsList = (json['rewards'] as List<dynamic>? ?? [])
76     .map((e) => ReferralRewardModel.fromJson(e as Map<String, dynamic>))
77     .toList();
78   final historyList = (json['history'] as List<dynamic>? ?? [])
79     .map((e) => ReferralHistoryItemModel.fromJson(e as Map<String, dynamic>))
80     .toList();
81   return ReferralDashboardModel(
82     referralCode: json['referral_code'] as String? ?? '',
83     referralLink: json['referral_link'] as String? ?? '',
84     maxReferrals: json['max_referrals'] as int? ?? 5,
85     qualifiedReferrals: json['qualified_referrals'] as int? ?? 0,
86     remainingReferrals: json['remaining_referrals'] as int? ?? 5,
87     rewardsAvailableCount: json['rewards_available_count'] as int? ?? 0,
88     rewards: rewardsList,
89     history: historyList,
90   );
91 }
92 }
93

```

Lines • Dart 31

Feature / Referral

#314 – lib/features/referral/presentation/cubit/referral_cubit.dart

```

1 import 'package:flutter/widgets.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/network/remote/api_service.dart';
4 import 'package:moean/features/referral/data/models/referral_dashboard_model.dart';
5 import 'package:moean/features/referral/presentation/cubit/referral_state.dart';
6
7 class ReferralCubit extends Cubit<ReferralState> {
8   ReferralCubit() : super(ReferralInitial());
9
10  static ReferralCubit get(BuildContext context) => BlocProvider.of(context);
11
12  ReferralDashboardModel? dashboard;
13
14  Future<void> loadDashboard() async {
15    if (!isClosed) emit(ReferralLoading());
16    final result = await ApiService.getReferralDashboard();
17    if (isClosed) return;
18    result.fold(
19      (error) {
20        if (!isClosed) emit(ReferralError(error));
21      },
22      (data) {
23        dashboard = ReferralDashboardModel.fromJson(data);
24        if (!isClosed) emit(ReferralLoaded(dashboard!));
25      },
26    );
27  }
28
29  Future<void> refresh() => loadDashboard();

```

```
30 }
31
```

Lines • Dart 18

Feature / Referral

#315 – lib/features/referral/presentation/cubit/referral_state.dart

```
1 import 'package:moean/features/referral/data/models/referral_dashboard_model.dart';
2
3 abstract class ReferralState {}
4
5 class ReferralInitial extends ReferralState {}
6
7 class ReferralLoading extends ReferralState {}
8
9 class ReferralLoaded extends ReferralState {
10   final ReferralDashboardModel dashboard;
11   ReferralLoaded(this.dashboard);
12 }
13
14 class ReferralError extends ReferralState {
15   final String error;
16   ReferralError(this.error);
17 }
18
```

Lines • Dart 97

Feature / Referral

#316 – lib/features/referral/presentation/screen/referral_screen.dart

```
1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/primary/conditional_builder.dart';
7 import 'package:moean/core/utils/extensions/context_extension.dart';
8 import 'package:moean/features/referral/presentation/cubit/referral_cubit.dart';
9 import 'package:moean/features/referral/presentation/cubit/referral_state.dart';
10 import 'package:moean/features/referral/presentation/widgets/referral_body_widget.dart';
11
12 class ReferralScreen extends StatefulWidget {
13   const ReferralScreen({super.key});
14
15   @override
16   State<ReferralScreen> createState() => _ReferralScreenState();
17 }
18
19 class _ReferralScreenState extends State<ReferralScreen> {
20   @override
21   void initState() {
22     super.initState();
23     ReferralCubit.get(context).loadDashboard();
24   }
25
26   @override
27   Widget build(BuildContext context) {
28     return Directionality(
29       textDirection: TextDirection.rtl,
30       child: Scaffold(
31         backgroundColor: ColorsManager.background,
32         appBar: AppBar(
33           backgroundColor: ColorsManager.background,
34           elevation: 0,
35           centerTitle: true,
36           title: Text(
37             appTranslation().get('referral_chip_label'),
38             style: TextStylesManager.bold18
39               .copyWith(color: ColorsManager.primaryColor),
40           ),
41           leading: IconButton(
42             icon: Icon(Icons.arrow_back_ios,
43               color: ColorsManager.mainText),
44             onPressed: () => context.pop(),
45           ),
46         ),

```

```

47   body: SafeArea(
48     child: BlocBuilder<ReferralCubit, ReferralState>(
49       buildWhen: (_, s) =>
50         s is ReferralLoading ||
51         s is ReferralLoaded ||
52         s is ReferralError,
53       builder: (context, state) {
54         return ConditionalBuilder(
55           loadingState: state is ReferralLoading,
56           errorState: state is ReferralError,
57           emptyState: false,
58           errorBuilder: (_) => Center(
59             child: Column(
60               mainAxisAlignment: MainAxisAlignment.min,
61               children: [
62                 Icon(Icons.cloud_off_outlined,
63                   size: 48,
64                   color: ColorsManager.textSecondary
65                     .withValues(alpha: 0.4)),
66                 const SizedBox(height: 12),
67                 Text(
68                   appTranslation().get('referral_loading_error'),
69                   style: TextStylesManager.regular14.copyWith(
70                     color: ColorsManager.textSecondary),
71                   textAlign: TextAlign.center,
72                 ),
73                 const SizedBox(height: 16),
74                 TextButton.icon(
75                   onPressed: () =>
76                     ReferralCubit.get(context).loadDashboard(),
77                   icon: const Icon(Icons.refresh_rounded),
78                   label:
79                     Text(appTranslation().get('referral_refresh')),
80                 ),
81               ],
82             ),
83           ),
84           successBuilder: (_) {
85             final cubit = ReferralCubit.get(context);
86             if (cubit.dashboard == null) return const SizedBox.shrink();
87             return ReferralBodyWidget(dashboard: cubit.dashboard!);
88           },
89         );
90       },
91     ),
92   ),
93 );
94 }
95 }
96 }
97

```

Lines • Dart 161

Feature / Referral

#317 – lib/features/referral/presentation/widgets/referral_body_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7  import 'package:moean/features/referral/data/models/referral_dashboard_model.dart';
8  import 'package:moean/features/referral/presentation/cubit/referral_cubit.dart';
9  import 'package:moean/features/referral/presentation/cubit/referral_state.dart';
10 import 'package:moean/features/referral/presentation/widgets/referral_empty_history_widget.dart';
11 import 'package:moean/features/referral/presentation/widgets/referral_footer_widget.dart';
12 import 'package:moean/features/referral/presentation/widgets/referral_hero_card_widget.dart';
13 import 'package:moean/features/referral/presentation/widgets/referral_history_item_widget.dart';
14 import 'package:moean/features/referral/presentation/widgets/referral_stat_card_widget.dart';
15
16 class ReferralBodyWidget extends StatelessWidget {
17   final ReferralDashboardModel dashboard;
18
19   const ReferralBodyWidget({super.key, required this.dashboard});

```

```

20
21 @override
22 Widget build(BuildContext context) {
23   return Directionality(
24     textDirection: TextDirection.rtl,
25     child: SingleChildScrollView(
26       padding: const EdgeInsets.all(20),
27       child: Column(
28         crossAxisAlignment: CrossAxisAlignment.start,
29         children: [
30           // Hero card
31           ReferralHeroCardWidget(referralLink: dashboard.referralLink),
32           verticalSpace16,
33           // Stats Grid (Row 1)
34           IntrinsicHeight(
35             child: Row(
36               crossAxisAlignment: CrossAxisAlignment.stretch,
37               children: [
38                 Expanded(
39                   child: ReferralStatCardWidget(
40                     label: appTranslation().get('referral_discounts'),
41                     value: '15% × ${dashboard.rewardsAvailableCount}',
42                     hint: appTranslation().get('referral_discounts_hint'),
43                     icon: Icons.redeem_outlined,
44                     iconBgColor: const Color(0xFFFFF3E0),
45                     iconColor: const Color(0xFFFF8C00),
46                   ),
47                 ),
48                 horizontalSpace12,
49                 Expanded(
50                   child: ReferralStatCardWidget(
51                     label: appTranslation().get('referral_qualified'),
52                     value: '${dashboard.qualifiedReferrals} / ${dashboard.maxReferrals}',
53                     icon: Icons.people_alt_outlined,
54                     iconBgColor: ColorsManager.primaryColor.withValues(alpha: 0.1),
55                     iconColor: ColorsManager.primaryColor,
56                   ),
57                 ),
58               ],
59             ),
60           ),
61           verticalSpace12,
62           // Stats Row 2 (Remaining quota)
63           ReferralStatCardWidget(
64             label: appTranslation().get('referral_remaining'),
65             value:
66               '${dashboard.remainingReferrals} ${appTranslation().get('referral_remaining_suffix')}',
67             hint: appTranslation().get('referral_max_note'),
68             icon: Icons.workspace_premium_outlined,
69             iconBgColor: const Color(0xFFEDE7F6),
70             iconColor: const Color(0xFF7B1FA2),
71           ),
72           verticalSpace20,
73           // History section
74           Container(
75             width: double.infinity,
76             padding: const EdgeInsets.all(16),
77             decoration: BoxDecoration(
78               color: ColorsManager.surfacePrimary,
79               borderRadius: BorderRadius.circular(16),
80               border: Border.all(color: ColorsManager.borderColor),
81             ),
82             child: Column(
83               crossAxisAlignment: CrossAxisAlignment.start,
84               children: [
85                 // Header row
86                 Row(
87                   mainAxisAlignment: MainAxisAlignment.spaceBetween,
88                   children: [
89                     // Title
90                     Text(
91                       appTranslation().get('referral_history_title'),
92                       style: TextStylesManager.bold16.copyWith(
93                         color: ColorsManager.textPrimary,
94                       ),

```

```

95         ),
96         // Refresh button
97         BlocBuilder<ReferralCubit, ReferralState>(
98             buildWhen: (_, s) => s is ReferralLoading || s is ReferralLoaded,
99             builder: (context, state) {
100                 final isRefreshing = state is ReferralLoading;
101                 return GestureDetector(
102                     onTap: isRefreshing
103                         ? null
104                         : () => ReferralCubit.get(context).refresh(),
105                     child: Row(
106                         mainAxisAlignment: MainAxisAlignment.min,
107                         children: [
108                             Text(
109                                 appTranslation().get('referral_refresh'),
110                                 style: TextStylesManager.medium14.copyWith(
111                                     color: ColorsManager.primaryColor,
112                                 ),
113                             ),
114                             horizontalSpace4,
115                             if (isRefreshing)
116                                 SizedBox(
117                                     width: 14,
118                                     height: 14,
119                                     child: CircularProgressIndicator(
120                                         strokeWidth: 2,
121                                         color: ColorsManager.primaryColor,
122                                     ),
123                                 )
124                             else
125                                 Icon(Icons.refresh_rounded,
126                                     size: 18,
127                                     color: ColorsManager.primaryColor),
128                         ],
129                     ),
130                 );
131             },
132         ),
133     ],
134 ),
135 verticalSpace16,
136 // History list or empty state
137 if (dashboard.history.isEmpty)
138     const ReferralEmptyHistoryWidget()
139 else
140     ListView.separated(
141         shrinkWrap: true,
142         physics: const NeverScrollableScrollPhysics(),
143         itemCount: dashboard.history.length,
144         separatorBuilder: (_, __) => verticalSpace8,
145         itemBuilder: (_, i) =>
146             ReferralHistoryItemWidget(item: dashboard.history[i]),
147     ),
148 ],
149 ),
150 ),
151 verticalSpace20,
152 // Footer CTA
153 const ReferralFooterWidget(),
154 verticalSpace24,
155 ],
156 ),
157 ),
158 );
159 }
160 }
161

```

Lines • 43
Dart

Feature / #318 –
Referral

lib/features/referral/presentation/widgets/referral_empty_history_widget.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';

```

```

3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/spacing.dart';
6
7 class ReferralEmptyHistoryWidget extends StatelessWidget {
8   const ReferralEmptyHistoryWidget({super.key});
9
10  @override
11  Widget build(BuildContext context) {
12    return Padding(
13      padding: const EdgeInsets.symmetric(vertical: 24),
14      child: Column(
15        children: [
16          Icon(
17            Icons.people_outline_rounded,
18            size: 56,
19            color: ColorsManager.textSecondary.withValues(alpha: 0.4),
20          ),
21          verticalSpace12,
22          Text(
23            appTranslation().get('referral_empty_title'),
24            style: TextStylesManager.medium14.copyWith(
25              color: ColorsManager.textPrimary,
26            ),
27            textAlign: TextAlign.center,
28          ),
29          verticalSpace6,
30          Text(
31            appTranslation().get('referral_empty_subtitle'),
32            style: TextStylesManager.regular12.copyWith(
33              color: ColorsManager.textSecondary,
34              height: 1.5,
35            ),
36            textAlign: TextAlign.center,
37          ),
38        ],
39      ),
40    );
41  }
42 }
43

```

Lines • Dart 51

Feature / Referral

#319 – lib/features/referral/presentation/widgets/referral_footer_widget.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/routes.dart';
6 import 'package:moean/core/utils/constants/spacing.dart';
7 import 'package:moean/core/utils/extensions/context_extension.dart';
8 import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
9
10 class ReferralFooterWidget extends StatelessWidget {
11   const ReferralFooterWidget({super.key});
12
13   @override
14   Widget build(BuildContext context) {
15     return Container(
16       padding: const EdgeInsets.all(20),
17       decoration: BoxDecoration(
18         color: ColorsManager.surfacePrimary,
19         borderRadius: BorderRadius.circular(16),
20         border: Border.all(color: ColorsManager.borderColor),
21       ),
22       child: Column(
23         crossAxisAlignment: CrossAxisAlignment.stretch,
24         children: [
25           Text(
26             appTranslation().get('referral_footer_title'),
27             style: TextStylesManager.bold14.copyWith(
28               color: ColorsManager.textPrimary,
29             ),
30           ),
31         ],
32       ),
33     );
34   }
35 }
36

```

```

30         textAlign: TextAlign.center,
31     ),
32     verticalSpace4,
33     Text(
34         appTranslation().get('referral_footer_subtitle'),
35         style: TextStyleManager.regular12.copyWith(
36             color: ColorsManager.textSecondary,
37             height: 1.4,
38         ),
39         textAlign: TextAlign.center,
40     ),
41     verticalSpace16,
42     PrimaryElevatedButton(
43         text: appTranslation().get('referral_footer_btn'),
44         onPressed: () => context.push(Routes.checkout),
45     ),
46 ],
47 ),
48 );
49 }
50 }
51

```

Lines • 213
Dart

Feature / #320 -

Referral lib/features/referral/presentation/widgets/referral_hero_card_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter/services.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7  import 'package:url_launcher/url_launcher.dart';
8
9  class ReferralHeroCardWidget extends StatelessWidget {
10     final String referralLink;
11
12     const ReferralHeroCardWidget({
13         super.key,
14         required this.referralLink,
15     });
16
17     void _copyLink(BuildContext context) {
18         Clipboard.setData(ClipboardData(text: referralLink));
19         ScaffoldMessenger.of(context).showSnackBar(
20             SnackBar(
21                 content: Text(
22                     appTranslation().get('referral_copy_success'),
23                     textAlign: TextAlign.center,
24                 ),
25                 backgroundColor: ColorsManager.primaryColor,
26                 behavior: SnackBarBehavior.floating,
27                 shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
28                 duration: const Duration(seconds: 2),
29             ),
30         );
31     }
32
33     Future<void> _shareWhatsapp() async {
34         final text = Uri.encodeComponent(
35             '${appTranslation().get('referral_screen_title')}\n$referralLink',
36         );
37         final uri = Uri.parse('https://wa.me/?text=$text');
38         if (await canLaunchUrl(uri)) {
39             await launchUrl(uri, mode: LaunchMode.externalApplication);
40         }
41     }
42
43     @override
44     Widget build(BuildContext context) {
45         return Container(
46             width: double.infinity,
47             decoration: BoxDecoration(

```



```

48     gradient: LinearGradient(
49       begin: Alignment.topRight,
50       end: Alignment.bottomLeft,
51       colors: [
52         ColorsManager.primaryColor,
53         ColorsManager.primaryColor.withValues(alpha: 0.85),
54       ],
55     ),
56     borderRadius: BorderRadius.circular(20),
57     boxShadow: [
58       BoxShadow(
59         color: ColorsManager.primaryColor.withValues(alpha: 0.3),
60         blurRadius: 16,
61         offset: const Offset(0, 6),
62       ),
63     ],
64   ),
65   child: Padding(
66     padding: const EdgeInsets.all(16),
67     child: Column(
68       crossAxisAlignment: CrossAxisAlignment.start,
69       children: [
70         // Badge
71         Container(
72           padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
73           decoration: BoxDecoration(
74             color: Colors.white.withValues(alpha: 0.15),
75             borderRadius: BorderRadius.circular(20),
76           ),
77           child: Row(
78             mainAxisAlignment: MainAxisAlignment.min,
79             children: [
80               Text(
81                 appTranslation().get('referral_screen_badge'),
82                 style: TextStylesManager.medium12.copyWith(
83                   color: Colors.white,
84                 ),
85               ),
86               horizontalSpace6,
87               const Icon(Icons.people_alt_outlined,
88                 color: Colors.white, size: 14),
89             ],
90           ),
91         ),
92         verticalSpace8,
93         // Title
94         Text(
95           appTranslation().get('referral_screen_title'),
96           style: TextStylesManager.bold16.copyWith(
97             color: Colors.white,
98             height: 1.3,
99           ),
100         textAlign: TextAlign.end,
101       ),
102       verticalSpace10,
103       // Subtitle
104       Text(
105         appTranslation().get('referral_screen_subtitle'),
106         style: TextStylesManager.regular12.copyWith(
107           color: Colors.white.withValues(alpha: 0.9),
108           height: 1.3,
109         ),
110         textAlign: TextAlign.end,
111       ),
112       verticalSpace10,
113       // Link label
114       Text(
115         appTranslation().get('referral_link_label'),
116         style: TextStylesManager.medium12.copyWith(
117           color: Colors.white.withValues(alpha: 0.8),
118         ),
119       ),
120       verticalSpace8,
121       // Link field
122       Container(

```

```

123     decoration: BoxDecoration(
124       color: Colors.white.withValues(alpha: 0.15),
125       borderRadius: BorderRadius.circular(12),
126       border: Border.all(
127         color: Colors.white.withValues(alpha: 0.3),
128       ),
129     ),
130     child: Row(
131       children: [
132         // Link text
133         Expanded(
134           child: Padding(
135             padding: const EdgeInsets.symmetric(horizontal: 12),
136             child: Text(
137               referralLink,
138               style: TextStyleManager.regular12.copyWith(
139                 color: Colors.white,
140               ),
141               overflow: TextOverflow.ellipsis,
142               textDirection: TextDirection.ltr,
143             ),
144           ),
145         ),
146         // Copy button
147         GestureDetector(
148           onTap: () => _copyLink(context),
149           child: Container(
150             padding: const EdgeInsets.symmetric(
151               horizontal: 12, vertical: 10),
152             decoration: BoxDecoration(
153               color: Colors.white,
154               borderRadius: const BorderRadius.only(
155                 topLeft: Radius.circular(12),
156                 bottomLeft: Radius.circular(12),
157               ),
158             ),
159             child: Row(
160               mainAxisAlignment: MainAxisAlignment.min,
161               children: [
162                 Icon(Icons.copy_rounded,
163                   size: 16, color: ColorsManager.primaryColor),
164                 horizontalSpace4,
165                 Text(
166                   appTranslation().get('referral_copy'),
167                   style: TextStyleManager.medium12.copyWith(
168                     color: ColorsManager.primaryColor,
169                   ),
170                 ),
171               ],
172             ),
173           ),
174         ),
175       ],
176     ),
177   ),
178   verticalSpace12,
179   // WhatsApp share button
180   SizedBox(
181     width: double.infinity,
182     child: GestureDetector(
183       onTap: _shareWhatsapp,
184       child: Container(
185         padding: const EdgeInsets.symmetric(vertical: 12),
186         decoration: BoxDecoration(
187           color: const Color(0xFF25D366),
188           borderRadius: BorderRadius.circular(12),
189         ),
190         child: Row(
191           mainAxisAlignment: MainAxisAlignment.center,
192           children: [
193             Text(
194               appTranslation().get('referral_share_whatsapp'),
195               style: TextStyleManager.medium14.copyWith(
196                 color: Colors.white,
197             ),

```

```

198         ),
199         horizontalSpace8,
200         const Icon(Icons.share_rounded,
201             color: Colors.white, size: 18),
202     ],
203 ),
204 ),
205 ),
206 ),
207 ],
208 ),
209 ),
210 );
211 }
212 }
213

```

Lines • 120
Dart

Feature / #321 -

Referral lib/features/referral/presentation/widgets/referral_history_item_widget.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/spacing.dart';
6 import 'package:moean/features/referral/data/models/referral_dashboard_model.dart';
7
8 class ReferralHistoryItemWidget extends StatelessWidget {
9   final ReferralHistoryItemModel item;
10
11   const ReferralHistoryItemWidget({super.key, required this.item});
12
13   Color _statusColor(String status) {
14     switch (status) {
15       case 'rewarded':
16         return const Color(0xFF22C55E);
17       case 'qualified':
18         return const Color(0xFF3B82F6);
19       case 'pending':
20         return const Color(0xFFFF59E0B);
21       case 'cancelled':
22       case 'expired':
23         return ColorsManager.errorColor;
24       default:
25         return ColorsManager.textSecondary;
26     }
27   }
28
29   String _statusLabel(String status) {
30     final keyMap = {
31       'rewarded': 'referral_status_rewarded',
32       'qualified': 'referral_status_qualified',
33       'pending': 'referral_status_pending',
34       'cancelled': 'referral_status_cancelled',
35       'expired': 'referral_status_expired',
36     };
37     return appTranslation().get(keyMap[status] ?? 'referral_status_pending');
38   }
39
40   @override
41   Widget build(BuildContext context) {
42     return Container(
43       padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
44       decoration: BoxDecoration(
45         color: ColorsManager.background,
46         borderRadius: BorderRadius.circular(12),
47         border: Border.all(color: ColorsManager.borderColor),
48       ),
49       child: Row(
50         children: [
51           // Avatar
52           Container(
53             width: 40,

```

```

54         height: 40,
55         decoration: BoxDecoration(
56           color: ColorsManager.primaryColor.withValues(alpha: 0.1),
57           shape: BoxShape.circle,
58         ),
59         child: Center(
60           child: Text(
61             item.referredName.isNotEmpty
62               ? item.referredName[0].toUpperCase()
63               : '?',
64             style: TextStylesManager.bold16.copyWith(
65               color: ColorsManager.primaryColor,
66             ),
67           ),
68         ),
69       ),
70       horizontalSpace12,
71       // Name
72       Column(
73         crossAxisAlignment: CrossAxisAlignment.start,
74         children: [
75           Text(
76             item.referredName,
77             style: TextStylesManager.medium14.copyWith(
78               color: ColorsManager.textPrimary,
79             ),
80           ),
81           verticalSpace2,
82           Row(
83             mainAxisAlignment: MainAxisAlignment.min,
84             children: [
85               Text(
86                 item.registeredAt.length >= 10
87                   ? item.registeredAt.substring(0, 10)
88                   : item.registeredAt,
89                 style: TextStylesManager.regular12.copyWith(
90                   color: ColorsManager.textSecondary,
91                 ),
92               ),
93               horizontalSpace4,
94               Icon(Icons.person_add_alt_1_outlined,
95                 size: 13, color: ColorsManager.textSecondary),
96             ],
97           ),
98         ],
99       ),
100     const Spacer(),
101     // Status badge
102     Container(
103       padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
104       decoration: BoxDecoration(
105         color: _statusColor(item.status).withValues(alpha: 0.12),
106         borderRadius: BorderRadius.circular(20),
107       ),
108       child: Text(
109         _statusLabel(item.status),
110         style: TextStylesManager.medium12.copyWith(
111           color: _statusColor(item.status),
112         ),
113       ),
114     ),
115   ],
116 ),
117 );
118 }
119 }
120

```

Lines • Dart 91 Feature / Referral #322 – lib/features/referral/presentation/widgets/referral_stat_card_widget.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';

```

```

4 import 'package:moean/core/utils/constants/spacing.dart';
5
6 class ReferralStatCardWidget extends StatelessWidget {
7   final String label;
8   final String value;
9   final String? hint;
10  final IconData icon;
11  final Color iconBgColor;
12  final Color iconColor;
13
14  const ReferralStatCardWidget({
15    super.key,
16    required this.label,
17    required this.value,
18    this.hint,
19    required this.icon,
20    required this.iconBgColor,
21    required this.iconColor,
22  });
23
24  @override
25  Widget build(BuildContext context) {
26    return Directionality(
27      textDirection: TextDirection.rtl,
28      child: Container(
29        width: double.infinity,
30        padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 14),
31        decoration: BoxDecoration(
32          color: ColorsManager.surfacePrimary,
33          borderRadius: BorderRadius.circular(16),
34          border: Border.all(color: ColorsManager.borderColor),
35        ),
36        child: Column(
37          crossAxisAlignment: CrossAxisAlignment.start,
38          mainAxisAlignment: MainAxisAlignment.center,
39          children: [
40            Row(
41              mainAxisAlignment: MainAxisAlignment.spaceBetween,
42              crossAxisAlignment: CrossAxisAlignment.start,
43              children: [
44                Container(
45                  width: 36,
46                  height: 36,
47                  decoration: BoxDecoration(
48                    color: iconBgColor,
49                    borderRadius: BorderRadius.circular(10),
50                  ),
51                  child: Icon(icon, color: iconColor, size: 18),
52                ),
53                horizontalSpace8,
54                Expanded(
55                  child: Text(
56                    label,
57                    style: TextStylesManager.regular12.copyWith(
58                      color: ColorsManager.textSecondary,
59                      height: 1.2,
60                    ),
61                    textAlign: TextAlign.start,
62                  ),
63                ),
64              ],
65            ),
66            verticalSpace12,
67            Text(
68              value,
69              style: TextStylesManager.bold16.copyWith(
70                color: ColorsManager.textPrimary,
71              ),
72              textAlign: TextAlign.start,
73            ),
74            if (hint != null) ...[
75              verticalSpace4,
76              Text(
77                hint!,
78                style: TextStylesManager.regular12.copyWith(

```

```

79         color: ColorsManager.textSecondary,
80         fontSize: 10,
81     ),
82     textAlign: TextAlign.start,
83 ),
84 ],
85 ],
86 ),
87 ),
88 );
89 }
90 }
91

```

Lines • Dart 116

Feature / Register

#323 – lib/features/register/presentation/cubit/register_cubit.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/features/register/presentation/cubit/register_state.dart';
4  import 'package:moean/core/network/remote/api_service.dart';
5  import 'package:moean/core/models/register_request.dart';
6  import 'package:moean/core/network/local/cache_helper.dart';
7  import 'package:moean/core/network/local/secure_storage_helper.dart';
8  import 'package:moean/core/utils/constants/constants.dart';
9  import 'package:moean/core/utils/formatters/saudi_phone_input_formatter.dart';
10 import 'package:moean/core/di/injections.dart';
11 import 'package:moean/features/payment/presentation/cubit/subscription_cubit.dart';
12
13 enum AccountType { teacher, supervisor }
14
15 class RegisterCubit extends Cubit<RegisterState> {
16   RegisterCubit() : super(RegisterInitialState());
17
18   static RegisterCubit get(BuildContext context) => BlocProvider.of(context);
19
20   final TextEditingController fullNameController = TextEditingController();
21   final TextEditingController emailController = TextEditingController();
22   final TextEditingController phoneController = TextEditingController();
23   final TextEditingController passwordController = TextEditingController();
24   final TextEditingController confirmPasswordController =
25     TextEditingController();
26   final GlobalKey<FormState> formKey = GlobalKey<FormState>();
27
28   bool isPasswordVisible = false;
29   bool isConfirmPasswordVisible = false;
30   bool agreeToTerms = false;
31   AccountType selectedAccountType = AccountType.teacher;
32
33   void togglePasswordVisibility() {
34     isPasswordVisible = !isPasswordVisible;
35     emit(RegisterPasswordVisibilityChangedState());
36   }
37
38   void toggleConfirmPasswordVisibility() {
39     isConfirmPasswordVisible = !isConfirmPasswordVisible;
40     emit(RegisterConfirmPasswordVisibilityChangedState());
41   }
42
43   void toggleTerms() {
44     agreeToTerms = !agreeToTerms;
45     emit(RegisterTermsChangedState());
46   }
47
48   void setAccountType(AccountType type) {
49     selectedAccountType = type;
50     emit(RegisterAccountTypeChangedState());
51   }
52
53   Future<void> register() async {
54     if (!(formKey.currentState?.validate() ?? false)) {
55       if (!isClosed) emit(RegisterInitialState());
56       return;
57     }

```

```

58   if (!agreeToTerms) {
59     if (!isClosed) emit(RegisterErrorState(message: 'يجب الموافقة على الشروط والأحكام'));
60     return;
61   }
62   if (!isClosed) emit(RegisterLoadingState());
63
64   try {
65     final referralCode = CacheHelper.getData(key: 'referral_code')?.toString();
66
67     final request = RegisterRequest(
68       name: fullNameController.text.trim(),
69       email: emailController.text.trim(),
70       phone: SaudiPhoneInputFormatter.normalizePhoneNumber(phoneController.text.trim()),
71       password: passwordController.text,
72       passwordConfirmation: confirmPasswordController.text,
73       referralCode: referralCode,
74     );
75
76     final result = await ApiService.registerUser(request);
77     if (isClosed) return;
78
79     result.fold(
80       (error) {
81         if (!isClosed) emit(RegisterErrorState(message: error));
82         debugPrint('Register Error: $error');
83       },
84       (response) async {
85         await CacheHelper.saveData(key: 'auth_token', value: response.token);
86         await sl<SecureStorageHelper>().saveToken(response.token);
87
88         // Clear the referral code after successful registration
89         if (referralCode != null && referralCode.isNotEmpty) {
90           await CacheHelper.removeData(key: 'referral_code');
91           debugPrint('Cleared locally stored referral code after successful registration.');

```

Lines • Dart 24

Feature / Register

#324 – lib/features/register/presentation/cubit/register_state.dart

```

1  abstract class RegisterState {}
2
3  class RegisterInitialState extends RegisterState {}
4
5  class RegisterLoadingState extends RegisterState {}
6
7  class RegisterSuccessState extends RegisterState {
8    final String email;
9    RegisterSuccessState({required this.email});
10 }
11

```

```

12 class RegisterErrorState extends RegisterState {
13   final String message;
14   RegisterErrorState({required this.message});
15 }
16
17 class RegisterPasswordVisibilityChangedState extends RegisterState {}
18
19 class RegisterConfirmPasswordVisibilityChangedState extends RegisterState {}
20
21 class RegisterAccountTypeChangedState extends RegisterState {}
22
23 class RegisterTermsChangedState extends RegisterState {}
24

```

Lines • Dart 129

Feature / Register

#325 – lib/features/register/presentation/screen/register_screen.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/utils/constants/spacing.dart';
5  import 'package:moean/features/register/presentation/cubit/register_cubit.dart';
6  import 'package:moean/features/register/presentation/cubit/register_state.dart';
7  import 'package:moean/features/register/presentation/widgets/register_account_type_widget.dart';
8  import 'package:moean/features/register/presentation/widgets/register_action_buttons_widget.dart';
9  import 'package:moean/features/register/presentation/widgets/register_footer_widget.dart';
10 import 'package:moean/features/register/presentation/widgets/register_form_widget.dart';
11 import 'package:moean/features/register/presentation/widgets/register_illustration_widget.dart';
12 import 'package:moean/features/register/presentation/widgets/register_terms_widget.dart';
13 import 'package:moean/core/utils/cubit/theme/theme_cubit.dart';
14 import 'package:moean/core/utils/cubit/theme/theme_state.dart';
15 import 'package:moean/core/utils/constants/routes.dart';
16 import 'package:moean/core/utils/extensions/context_extension.dart';
17 import 'package:moean/core/theme/text_styles.dart';
18
19 class RegisterScreen extends StatelessWidget {
20   const RegisterScreen({super.key});
21
22   @override
23   Widget build(BuildContext context) {
24     return BlocProvider(
25       create: (_) => RegisterCubit(),
26       child: BlocListener<RegisterCubit, RegisterState>(
27         listener: _onStateChanged,
28         child: BlocBuilder<ThemeCubit, ThemeState>(
29           builder: (context, themeState) {
30             return Scaffold(
31               appBar: AppBar(
32                 leading: IconButton(
33                   onPressed: () {
34                     Navigator.pop(context);
35                   },
36                   icon: Icon(
37                     Icons.arrow_forward_ios_rounded,
38                     size: 16,
39                     color: ColorsManager.primaryColor,
40                   ),
41                 ),
42               ),
43               backgroundColor: ColorsManager.background,
44               body: SafeArea(
45                 child: SingleChildScrollView(
46                   padding: const EdgeInsets.symmetric(horizontal: 24),
47                   child: Builder(
48                     builder: (context) {
49                       final cubit = RegisterCubit.get(context);
50                       return Form(
51                         key: cubit.formKey,
52                         child: Column(
53                           crossAxisAlignment: CrossAxisAlignment.stretch,
54                           children: [
55                             verticalSpace16,
56                             // ignore: prefer_const_constructors
57                             RegisterHeaderWidget(),

```



```

58         verticalSpace16,
59         // ignore: prefer_const_constructors
60         RegisterIllustrationWidget(),
61         verticalSpace16,
62         Container(
63             padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
64             decoration: BoxDecoration(
65                 color: ColorsManager.primaryColor.withValues(alpha: 0.1),
66                 borderRadius: BorderRadius.circular(12),
67                 border: Border.all(color: ColorsManager.primaryColor.withValues(alpha: 0.2)),
68             ),
69             child: Row(
70                 children: [
71                     Icon(Icons.star_rounded, color: ColorsManager.primaryColor, size: 24),
72                     horizontalSpace8,
73                     Expanded(
74                         child: Text(
75                             'سجل الآن واحصل على تجربة مجانية كاملة لمدة 7 أيام بدون أي رسوم',
76                             style: TextStyleManager.bold12.copyWith(color: ColorsManager.primaryCo
lor),
77                         ),
78                     ),
79                 ],
80             ),
81         ),
82         verticalSpace24,
83         // ignore: prefer_const_constructors
84         RegisterFormWidget(),
85         verticalSpace20,
86         // ignore: prefer_const_constructors
87         RegisterAccountTypeWidget(),
88         verticalSpace16,
89         // ignore: prefer_const_constructors
90         RegisterTermsWidget(),
91         verticalSpace28,
92         // ignore: prefer_const_constructors
93         RegisterActionButtonWidget(),
94         verticalSpace24,
95         // ignore: prefer_const_constructors
96         RegisterFooterWidget(),
97         verticalSpace32,
98     ],
99 ),
100 );
101 },
102 ),
103 ),
104 ),
105 );
106 },
107 ),
108 ),
109 );
110 }
111
112 void _onStateChanged(BuildContext context, RegisterState state) {
113     if (state is RegisterErrorState) {
114         ScaffoldMessenger.of(context).showSnackBar(
115             SnackBar(
116                 content: Text(state.message),
117                 backgroundColor: ColorsManager.errorColor,
118             ),
119         );
120     }
121     if (state is RegisterSuccessState) {
122         context.pushReplacement(
123             Routes.verifyEmail,
124             arguments: state.email,
125         );
126     }
127 }
128 }
129

```

```
1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/spacing.dart';
7 import 'package:moean/features/register/presentation/cubit/register_cubit.dart';
8 import 'package:moean/features/register/presentation/cubit/register_state.dart';
9
10 class RegisterAccountTypeWidget extends StatelessWidget {
11   const RegisterAccountTypeWidget({super.key});
12
13   @override
14   Widget build(BuildContext context) {
15     final cubit = RegisterCubit.get(context);
16
17     return Column(
18       crossAxisAlignment: CrossAxisAlignment.end,
19       children: [
20         Text(
21           appTranslation().get('account_type'),
22           style: TextStylesManager.medium14.copyWith(
23             color: ColorsManager.mainText,
24           ),
25         ),
26         verticalSpace8,
27         BlocBuilder<RegisterCubit, RegisterState>{
28           buildWhen: (prev, curr) => curr is RegisterAccountTypeChangedState,
29           builder: (context, state) {
30             return Row(
31               children: [
32
33                 _AccountTypeOptionWidget(
34                   label: appTranslation().get('account_type_teacher'),
35                   icon: Icons.person_outline_rounded,
36                   isSelected: cubit.selectedAccountType == AccountType.teacher,
37                   onTap: () => cubit.setAccountType(AccountType.teacher),
38                 ),
39                 horizontalSpace12,
40
41                 _AccountTypeOptionWidget(
42                   label: appTranslation().get('account_type_supervisor'),
43                   icon: Icons.manage_accounts_outlined,
44                   isSelected:
45                     cubit.selectedAccountType == AccountType.supervisor,
46                   onTap: () => cubit.setAccountType(AccountType.supervisor),
47                 ),
48               ],
49             );
50           },
51         ),
52       ],
53     );
54   }
55 }
56
57 class _AccountTypeOptionWidget extends StatelessWidget {
58   final String label;
59   final IconData icon;
60   final bool isSelected;
61   final VoidCallback onTap;
62
63   const _AccountTypeOptionWidget({
64     required this.label,
65     required this.icon,
66     required this.isSelected,
67     required this.onTap,
68   });
69
70   @override
71   Widget build(BuildContext context) {
```

```

72     return Expanded(
73       child: GestureDetector(
74         onTap: onTap,
75         child: AnimatedContainer(
76           duration: const Duration(milliseconds: 200),
77           height: 52,
78           decoration: BoxDecoration(
79             color: isSelected
80               ? ColorsManager.primaryColor
81               : ColorsManager.surfacePrimary,
82             borderRadius: BorderRadius.circular(24),
83             border: Border.all(
84               color: isSelected
85                 ? ColorsManager.primaryColor
86                 : ColorsManager.borderColor,
87               width: isSelected ? 1.5 : 1.0,
88             ),
89           ),
90       child: Row(
91         mainAxisAlignment: MainAxisAlignment.center,
92         children: [
93           if (isSelected) ...[
94             Icon(
95               Icons.check_circle_rounded,
96               size: 18,
97               color: ColorsManager.white,
98             ),
99             horizontalSpace6,
100          ] else ...[
101            Icon(
102              icon,
103              size: 18,
104              color: ColorsManager.mainText,
105            ),
106            horizontalSpace6,
107          ],
108          Text(
109            label,
110            style: TextStylesManager.medium14.copyWith(
111              color: isSelected
112                ? ColorsManager.white
113                : ColorsManager.mainText,
114            ),
115          ),
116        ],
117      ),
118    ),
119  ),
120 );
121 }
122 }
123

```

Lines • 82
Dart

Feature / #327 -

Register lib/features/register/presentation/widgets/register_action_buttons_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/features/register/presentation/cubit/register_cubit.dart';
7  import 'package:moean/features/register/presentation/cubit/register_state.dart';
8  import 'package:rounded_loading_button_plus/rounded_loading_button.dart';
9
10 class RegisterActionButtonsWidget extends StatefulWidget {
11   const RegisterActionButtonsWidget({super.key});
12
13   @override
14   State<RegisterActionButtonsWidget> createState() => _RegisterActionButtonsWidgetState();
15 }
16
17 class _RegisterActionButtonsWidgetState extends State<RegisterActionButtonsWidget> {

```

```

18   final RoundedLoadingButtonController _btnController = RoundedLoadingButtonController();
19
20   @override
21   Widget build(BuildContext context) {
22     final cubit = RegisterCubit.get(context);
23
24     return Column(
25       children: [
26         BlocListener<RegisterCubit, RegisterState>(
27           listenWhen: (prev, curr) =>
28             curr is RegisterLoadingState ||
29             curr is RegisterSuccessState ||
30             curr is RegisterErrorState ||
31             curr is RegisterInitialState,
32           listener: (context, state) {
33             if (state is RegisterSuccessState) {
34               _btnController.success();
35             } else if (state is RegisterErrorState) {
36               _btnController.error();
37               Future.delayed(const Duration(seconds: 1), () {
38                 if (mounted) {
39                   _btnController.reset();
40                 }
41               });
42             } else if (state is RegisterInitialState) {
43               _btnController.reset();
44             }
45           },
46         child: RoundedLoadingButton(
47           controller: _btnController,
48           onPressed: () {
49             cubit.register();
50           },
51           color: ColorsManager.primaryColor,
52           successColor: Colors.green,
53           errorColor: ColorsManager.errorColor,
54           width: MediaQuery.of(context).size.width,
55           height: 52,
56           borderRadius: 24,
57           elevation: 0,
58           child: Row(
59             mainAxisAlignment: MainAxisAlignment.center,
60             children: [
61               const Icon(
62                 Icons.auto_awesome,
63                 color: Colors.white,
64                 size: 18,
65               ),
66               const SizedBox(width: 8),
67               Text(
68                 appTranslation().get('register'),
69                 style: TextStylesManager.bold16.copyWith(
70                   color: ColorsManager.white,
71                 ),
72             ),
73           ],
74         ),
75       ],
76     ),
77   ],
78 );
79 }
80 }
81
82

```

```

1   import 'package:flutter/material.dart';
2   import 'package:moean/core/theme/colors.dart';
3   import 'package:moean/core/theme/text_styles.dart';
4   import 'package:moean/core/utils/constants/constants.dart';
5   import 'package:moean/core/utils/constants/routes.dart';

```

```

6
7 class RegisterFooterWidget extends StatelessWidget {
8   const RegisterFooterWidget({super.key});
9
10  @override
11  Widget build(BuildContext context) {
12    return Row(
13      mainAxisAlignment: MainAxisAlignment.center,
14      children: [
15        GestureDetector(
16          onTap: () => Navigator.of(context).pushNamed(Routes.login),
17          child: Text(
18            appTranslation().get('login_link'),
19            style: TextStylesManager.medium14.copyWith(
20              color: ColorsManager.primaryColor,
21              decoration: TextDecoration.underline,
22              decorationColor: ColorsManager.primaryColor,
23            ),
24          ),
25        ),
26        Text(
27          appTranslation().get('have_account'),
28          style: TextStylesManager.regular14.copyWith(
29            color: ColorsManager.textBody,
30          ),
31        ),
32      ],
33    );
34  }
35 }
36

```

Lines • Dart 212

Feature / Register

#329 – lib/features/register/presentation/widgets/register_form_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
7  import 'package:moean/core/utils/constants/spacing.dart';
8  import 'package:moean/core/utils/formatters/saudi_phone_input_formatter.dart';
9  import 'package:moean/features/register/presentation/cubit/register_cubit.dart';
10 import 'package:moean/features/register/presentation/cubit/register_state.dart';
11
12 class RegisterFormWidget extends StatelessWidget {
13   const RegisterFormWidget({super.key});
14
15   @override
16   Widget build(BuildContext context) {
17     final cubit = RegisterCubit.get(context);
18
19     return Column(
20       crossAxisAlignment: CrossAxisAlignment.start,
21       children: [
22         Text(
23           appTranslation().get('full_name'),
24           style: TextStylesManager.medium14.copyWith(
25             color: ColorsManager.mainText,
26           ),
27         ),
28         verticalSpace8,
29         PrimaryTextField(
30           controller: cubit.fullNameController,
31           hint: appTranslation().get('full_name_hint'),
32           keyboardType: TextInputType.name,
33           textInputAction: TextInputAction.next,
34           prefixIcon: const Icon(Icons.person_outline_rounded),
35           validator: (value) {
36             if (value == null || value.trim().isEmpty) {
37               return appTranslation().get('full_name_required');
38             }
39             return null;
40           },
41         ),
42       ],
43     );
44   }
45 }
46

```

```

40      ),
41    ),
42    verticalSpace20,
43    Text(
44      appTranslation().get('email'),
45      style: TextStylesManager.medium14.copyWith(
46        color: ColorsManager.mainText,
47      ),
48    ),
49    verticalSpace8,
50    PrimaryTextField(
51      controller: cubit.emailController,
52      hint: appTranslation().get('email_hint'),
53      keyboardType: TextInputType.emailAddress,
54      textInputAction: TextInputAction.next,
55      prefixIcon: const Icon(Icons.mail_outline_rounded),
56      validator: (value) {
57        if (value == null || value.trim().isEmpty) {
58          return appTranslation().get('email_required');
59        }
60        final emailRegex = RegExp(r'^[\w-.\+@([\w-]+\.)+[\w-]{2,4}$');
61        if (!emailRegex.hasMatch(value.trim())) {
62          return appTranslation().get('email_invalid');
63        }
64        return null;
65      },
66    ),
67    verticalSpace20,
68    Text(
69      appTranslation().get('phone'),
70      style: TextStylesManager.medium14.copyWith(
71        color: ColorsManager.mainText,
72      ),
73    ),
74    verticalSpace8,
75    _PhoneFieldWidget(cubit: cubit),
76    verticalSpace20,
77    Text(
78      appTranslation().get('password'),
79      style: TextStylesManager.medium14.copyWith(
80        color: ColorsManager.mainText,
81      ),
82    ),
83    verticalSpace8,
84    BlocBuilder<RegisterCubit, RegisterState>(
85      buildWhen: (prev, curr) =>
86        curr is RegisterPasswordVisibilityChangedState,
87      builder: (context, state) {
88        return PrimaryTextField(
89          controller: cubit.passwordController,
90          hint: appTranslation().get('password_hint'),
91          isPassword: !cubit.isPasswordVisible,
92          textInputAction: TextInputAction.next,
93          prefixIcon: const Icon(Icons.lock_outline_rounded),
94          suffixIcon: GestureDetector(
95            onTap: cubit.togglePasswordVisibility,
96            child: Icon(
97              cubit.isPasswordVisible
98                ? Icons.visibility_off_outlined
99                : Icons.visibility_outlined,
100              color: ColorsManager.textBody,
101            ),
102          ),
103          validator: (value) {
104            if (value == null || value.isEmpty) {
105              return appTranslation().get('password_required');
106            }
107            return null;
108          },
109        );
110      },
111    ),
112    verticalSpace20,
113    Text(
114      appTranslation().get('confirm_password'),

```

```

115         style: TextStylesManager.medium14.copyWith(
116             color: ColorsManager.mainText,
117         ),
118     ),
119     verticalSpace8,
120     BlocBuilder<RegisterCubit, RegisterState>(
121         buildWhen: (prev, curr) =>
122             curr is RegisterConfirmPasswordVisibilityChangedState,
123         builder: (context, state) {
124             return PrimaryTextField(
125                 controller: cubit.confirmPasswordController,
126                 hint: appTranslation().get('confirm_password_hint'),
127                 isPassword: !cubit.isConfirmPasswordVisible,
128                 textInputAction: TextInputAction.done,
129                 prefixIcon: const Icon(Icons.lock_outline_rounded),
130                 suffixIcon: GestureDetector(
131                     onTap: cubit.toggleConfirmPasswordVisibility,
132                     child: Icon(
133                         cubit.isConfirmPasswordVisible
134                             ? Icons.visibility_off_outlined
135                             : Icons.visibility_outlined,
136                         color: ColorsManager.textBody,
137                     ),
138                 ),
139                 validator: (value) {
140                     if (value == null || value.isEmpty) {
141                         return appTranslation().get('confirm_password_required');
142                     }
143                     if (value != cubit.passwordController.text) {
144                         return appTranslation().get('passwords_not_match');
145                     }
146                     return null;
147                 },
148             );
149         },
150     ),
151 ],
152 );
153 }
154 }
155
156 class _PhoneFieldWidget extends StatelessWidget {
157     final RegisterCubit cubit;
158
159     const _PhoneFieldWidget({required this.cubit});
160
161     @override
162     Widget build(BuildContext context) {
163         return PrimaryTextField(
164             controller: cubit.phoneController,
165             hint: appTranslation().get('phone_hint'),
166             keyboardType: TextInputType.phone,
167             textInputAction: TextInputAction.next,
168             inputFormatters: [
169                 SaudiPhoneInputFormatter(),
170             ],
171             prefixIconConstraints: const BoxConstraints(minWidth: 0, minHeight: 0),
172             prefixIcon: Padding(
173                 padding: const EdgeInsetsDirectional.only(start: 14, end: 10),
174                 child: Row(
175                     mainAxisAlignment: MainAxisAlignment.min,
176                     children: [
177                         const Text(
178                             'SA',
179                             style: TextStyle(fontSize: 18),
180                         ),
181                         const SizedBox(width: 6),
182                         Text(
183                             '+966',
184                             textDirection: TextDirection.ltr,
185                             style: TextStylesManager.bold14.copyWith(
186                                 color: ColorsManager.textPrimary,
187                             ),
188                         ),
189                         const SizedBox(width: 8),

```

```

190         Container(
191             height: 20,
192             width: 1.5,
193             color: ColorsManager.borderColor,
194         ),
195     ],
196 ),
197 ),
198 validator: (value) {
199     final text = value?.trim() ?? '';
200     if (text.isEmpty) {
201         return appTranslation().get('phone_required');
202     }
203     final saudiPhoneRegex = RegExp(r'^(05\d{8}|5\d{8})$');
204     if (!saudiPhoneRegex.hasMatch(text)) {
205         return appTranslation().get('phone_invalid');
206     }
207     return null;
208 },
209 );
210 }
211 }
212

```

Lines • Dart 85

Feature / Register

#330 – lib/features/register/presentation/widgets/register_header_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/assets_helper.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7
8  class RegisterHeaderWidget extends StatelessWidget {
9      const RegisterHeaderWidget({super.key});
10
11     @override
12     Widget build(BuildContext context) {
13         return Row(
14             children: [
15                 Expanded(
16                     child: Column(
17                         crossAxisAlignment: CrossAxisAlignment.start,
18                         children: [
19                             Row(
20                                 children: [
21                                     ClipRRect(
22                                         borderRadius: BorderRadius.circular(8),
23                                         child: Image.asset(
24                                             AssetsHelper.icon,
25                                             width: 60,
26                                             height: 60,
27                                             fit: BoxFit.cover,
28                                         ),
29                                     ),
30                                     horizontalSpace8,
31                                     Expanded(
32                                         child: Column(
33                                             crossAxisAlignment: CrossAxisAlignment.start,
34                                             children: [
35                                                 Text(
36                                                     appTranslation().get('app_name'),
37                                                     maxLines: 1,
38                                                     overflow: TextOverflow.ellipsis,
39                                                     style: TextStylesManager.bold22.copyWith(
40                                                         color: ColorsManager.textSecondaryDark,
41                                                     ),
42                                                 ),
43                                                 Text(
44                                                     appTranslation().get('smart_assistant'),
45                                                     maxLines: 2,
46                                                     overflow: TextOverflow.ellipsis,
47                                                     style: TextStylesManager.medium10.copyWith(

```



```

48         color: ColorsManager.textSecondaryDark,
49         height: 1.2,
50     ),
51 ),
52 ],
53 ),
54 ),
55 ],
56 ),
57
58
59
60     Text(
61         appTranslation().get('register_title'),
62         maxLines: 2,
63         overflow: TextOverflow.ellipsis,
64         style: TextStylesManager.bold26.copyWith(
65             color: ColorsManager.textSecondaryDark,
66         ),
67     ),
68
69     verticalSpace8,
70
71     Text(
72         appTranslation().get('register_subtitle'),
73         maxLines: 2,
74         overflow: TextOverflow.ellipsis,
75         style: TextStylesManager.regular14.copyWith(
76             color: ColorsManager.mainText,
77         ),
78     ),
79 ],
80 ),
81 ),
82 ],
83 );
84 }
85 }

```

Lines • 40
Dart

Feature / #331 -
Register

[lib/features/register/presentation/widgets/register_illustration_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/utils/constants/assets_helper.dart';
3  import 'package:moean/core/utils/constants/spacing.dart';
4  import 'package:moean/features/register/presentation/widgets/register_header_widget.dart';
5
6  class RegisterIllustrationWidget extends StatelessWidget {
7      const RegisterIllustrationWidget({super.key});
8
9      @override
10     Widget build(BuildContext context) {
11         return Row(
12             crossAxisAlignment: CrossAxisAlignment.center,
13             children: [
14                 Expanded(
15                     flex: 5,
16                     child:
17                         RegisterHeaderWidget(),
18                 ),
19             ],
20             horizontalSpace16,
21
22             Expanded(
23                 flex: 4,
24                 child: Container(
25                     height: 180,
26                     decoration: BoxDecoration(
27                         borderRadius: BorderRadius.circular(8),
28                     ),
29                     child: Image.asset(
30                         AssetsHelper.img2,
31                         fit: BoxFit.contain,

```

```

32         ),
33     ),
34 ),
35
36 ],
37 );
38 }
39 }
40

```

Lines • Dart 88

Feature / Register

#332 – lib/features/register/presentation/widgets/register_terms_widget.dart

```

1  import 'package:flutter/gestures.dart';
2  import 'package:flutter/material.dart';
3  import 'package:flutter_bloc/flutter_bloc.dart';
4  import 'package:moean/core/theme/colors.dart';
5  import 'package:moean/core/theme/text_styles.dart';
6  import 'package:moean/core/utils/constants/constants.dart';
7  import 'package:moean/core/utils/constants/routes.dart';
8  import 'package:moean/core/utils/extensions/context_extension.dart';
9  import 'package:moean/features/register/presentation/cubit/register_cubit.dart';
10 import 'package:moean/features/register/presentation/cubit/register_state.dart';
11
12 class RegisterTermsWidget extends StatelessWidget {
13   const RegisterTermsWidget({super.key});
14
15   @override
16   Widget build(BuildContext context) {
17     final cubit = RegisterCubit.get(context);
18
19     return BlocBuilder<RegisterCubit, RegisterState>(
20       buildWhen: (prev, curr) => curr is RegisterTermsChangedState,
21       builder: (context, state) {
22         return Row(
23           mainAxisAlignment: MainAxisAlignment.end,
24           children: [
25             Expanded(
26               child: Text.rich(
27                 TextSpan(
28                   children: [
29                     TextSpan(
30                       text: appTranslation().get('terms_agree_prefix'),
31                       style: TextStylesManager.regular14.copyWith(
32                         color: ColorsManager.mainText,
33                       ),
34                       recognizer: TapGestureRecognizer()
35                         ..onTap = cubit.toggleTerms,
36                     ),
37                     TextSpan(
38                       text: appTranslation().get('terms_and_conditions'),
39                       style: TextStylesManager.regular14.copyWith(
40                         color: ColorsManager.mainText,
41                       ),
42                       recognizer: TapGestureRecognizer()
43                         ..onTap = cubit.toggleTerms,
44                     ),
45                     TextSpan(
46                       text: appTranslation().get('terms_and'),
47                       style: TextStylesManager.regular14.copyWith(
48                         color: ColorsManager.mainText,
49                       ),
50                       recognizer: TapGestureRecognizer()
51                         ..onTap = cubit.toggleTerms,
52                     ),
53                     TextSpan(
54                       text: appTranslation().get('privacy_policy'),
55                       style: TextStylesManager.bold14.copyWith(
56                         color: ColorsManager.primaryColor,
57                         decoration: TextDecoration.underline,
58                       ),
59                       recognizer: TapGestureRecognizer()
60                         ..onTap = () => context.push(Routes.privacyPolicy),
61                     ),

```

```

62         ],
63       ),
64       textAlign: TextAlign.right,
65     ),
66   ),
67   Transform.scale(
68     scale: 0.9,
69     child: Checkbox(
70       value: cubit.agreeToTerms,
71       onChanged: (_) => cubit.toggleTerms(),
72       activeColor: ColorsManager.primaryColor,
73       shape: RoundedRectangleBorder(
74         borderRadius: BorderRadius.circular(4),
75       ),
76       side: BorderSide(
77         color: ColorsManager.borderColor,
78         width: 1.5,
79       ),
80     ),
81   ),
82 ],
83 );
84 },
85 );
86 }
87 }
88

```

Lines • Dart 551

Feature / Reports

#333 – lib/features/reports/data/report_pdf_service.dart

```

1  import 'dart:typed_data';
2  import 'package:flutter/services.dart' show rootBundle;
3  import 'package:pdf/pdf.dart';
4  import 'package:pdf/widgets.dart' as pw;
5
6  class ReportPdfService {
7    static final PdfColor _primaryColor = PdfColor.fromHex('073F49');
8    static final PdfColor _headerBg = PdfColor.fromHex('0B4A45');
9    static final PdfColor _subHeaderBg = PdfColor.fromHex('E8F5F1');
10   static final PdfColor _lightGray = PdfColor.fromHex('F8FAFC');
11   static final PdfColor _borderColor = PdfColor.fromHex('E2E8F0');
12   static final PdfColor _textDark = PdfColor.fromHex('1E293B');
13   static final PdfColor _textMuted = PdfColor.fromHex('64748B');
14
15   static String _fixArabic(String text) {
16     if (text.isEmpty) return text;
17     return text
18       .replaceAll('مسار', 'إداري مسار\u00A0إداري')
19       .replaceAll('مسار', 'إداري مسار\u00A0إداري')
20       .replaceAll('مسار', 'إداري مسار\u00A0إداري')
21       .replaceAll('مسار', 'إداري مسار\u00A0إداري');
22   }
23
24
25   static Future<Uint8List> generatePdf({
26     required Map<String, dynamic> reportData,
27     required String teacherName,
28     required String grade,
29     required String subject,
30     required String unit,
31     required String semester,
32     required String schoolName,
33     required String educationOffice,
34     required String reportType,
35     required String reportDate,
36     required List<String> selectedLessons,
37   }) async {
38     final ByteData fontData = await rootBundle.load('assets/fonts/Tajawal-Regular.ttf');
39     final pw.Font ttf = pw.Font.ttf(fontData);
40
41     final ByteData boldFontData = await rootBundle.load('assets/fonts/Tajawal-Bold.ttf');
42     final pw.Font ttfBold = pw.Font.ttf(boldFontData);
43

```

```

44 final ByteData ministryImageData = await rootBundle.load('assets/images/ministry.jpg');
45 final pw.MemoryImage ministryImage = pw.MemoryImage(ministryImageData.buffer.asUint8List());
46
47 final ByteData logoImageData = await rootBundle.load('assets/images/logo_icon-removebg-preview.png');
48 final pw.MemoryImage logoImage = pw.MemoryImage(logoImageData.buffer.asUint8List());
49
50 final pdf = pw.Document(
51   theme: pw.ThemeData.withFont(
52     base: ttf,
53     bold: ttfBold,
54   ),
55 );
56
57 final List<Map<String, dynamic>> achievementRows =
58   List<Map<String, dynamic>>.from(reportData['achievementRows'] ?? []);
59 final List<Map<String, dynamic>> lessonPlanRows =
60   List<Map<String, dynamic>>.from(reportData['lessonPlanRows'] ?? []);
61 final List<Map<String, dynamic>> goalRows =
62   List<Map<String, dynamic>>.from(reportData['goalRows'] ?? []);
63 final List<Map<String, dynamic>> challengeRows =
64   List<Map<String, dynamic>>.from(reportData['challengeRows'] ?? []);
65 final List<Map<String, dynamic>> checkUnderstandingRows =
66   List<Map<String, dynamic>>.from(reportData['checkUnderstandingRows'] ?? []);
67 final List<Map<String, dynamic>> modelingRows =
68   List<Map<String, dynamic>>.from(reportData['modelingRows'] ?? []);
69 final List<Map<String, dynamic>> strategySections =
70   List<Map<String, dynamic>>.from(reportData['strategySections'] ?? []);
71
72 final pw.TextStyle arabicRegular = pw.TextStyle(
73   fontSize: 9,
74   color: _textDark,
75 );
76 final pw.TextStyle arabicBold = pw.TextStyle(
77   fontSize: 9,
78   fontWeight: pw.FontWeight.bold,
79   color: _textDark,
80 );
81 final pw.TextStyle headerStyle = pw.TextStyle(
82   fontSize: 10,
83   fontWeight: pw.FontWeight.bold,
84   color: PdfColors.white,
85 );
86 final pw.TextStyle titleStyle = pw.TextStyle(
87   fontSize: 13,
88   fontWeight: pw.FontWeight.bold,
89   color: PdfColors.white,
90 );
91
92 // Build pages
93 pdf.addPage(
94   pw.MultiPage(
95     pageFormat: PdfPageFormat.a4,
96     textDirection: pw.TextDirection.rtl,
97     margin: const pw.EdgeInsets.all(24),
98     build: (pw.Context context) {
99       return [
100         // — Page Header —
101         _buildPageHeader(
102           teacherName: _fixArabic(teacherName),
103           grade: _fixArabic(grade),
104           subject: _fixArabic(subject),
105           unit: unit,
106           semester: semester,
107           schoolName: schoolName,
108           educationOffice: educationOffice,
109           reportType: reportType,
110           reportDate: reportDate,
111           lessonsCount: selectedLessons.length.toString(),
112           titleStyle: titleStyle,
113           headerStyle: headerStyle,
114           arabicRegular: arabicRegular,
115           arabicBold: arabicBold,
116           ministryImage: ministryImage,
117           logoImage: logoImage,
118         ),

```

```

119         pw.SizedBox(height: 12),
120
121         // — Achievement Rows Table —
122         if (achievementRows.isNotEmpty) ...[
123             _buildSectionTitle('بيانات الإنجاز التعليمي', headerStyle),
124             pw.SizedBox(height: 6),
125             _buildAchievementTable(achievementRows, arabicRegular, arabicBold),
126             pw.SizedBox(height: 12),
127         ],
128
129         // — Lesson Plan Table —
130         if (lessonPlanRows.isNotEmpty) ...[
131             _buildSectionTitle('خطة الدرس', headerStyle),
132             pw.SizedBox(height: 6),
133             _buildLessonPlanTable(lessonPlanRows, arabicRegular, arabicBold),
134             pw.SizedBox(height: 12),
135         ],
136
137         // — Goals Table —
138         if (goalRows.isNotEmpty) ...[
139             _buildSectionTitle('الأهداف', headerStyle),
140             pw.SizedBox(height: 6),
141             _buildGoalTable(goalRows, arabicRegular, arabicBold),
142             pw.SizedBox(height: 12),
143         ],
144
145         // — Challenges Table —
146         if (challengeRows.isNotEmpty) ...[
147             _buildSectionTitle('التحديات والخطط المقترحة', headerStyle),
148             pw.SizedBox(height: 6),
149             _buildChallengeTable(challengeRows, arabicRegular, arabicBold),
150             pw.SizedBox(height: 12),
151         ],
152
153         // — Check Understanding Table —
154         if (checkUnderstandingRows.isNotEmpty) ...[
155             _buildSectionTitle('التحقق من الفهم', headerStyle),
156             pw.SizedBox(height: 6),
157             _buildCheckUnderstandingTable(
158                 checkUnderstandingRows, arabicRegular, arabicBold),
159             pw.SizedBox(height: 12),
160         ],
161
162         // — Modeling Rows Table —
163         if (modelingRows.isNotEmpty) ...[
164             _buildSectionTitle('النمذجة', headerStyle),
165             pw.SizedBox(height: 6),
166             _buildModelingTable(modelingRows, arabicRegular, arabicBold),
167             pw.SizedBox(height: 12),
168         ],
169
170         // — Strategy Sections —
171         if (strategySections.isNotEmpty) ...[
172             _buildSectionTitle('مقاطع الاستراتيجيات', headerStyle),
173             pw.SizedBox(height: 6),
174             _buildStrategyTable(strategySections, arabicRegular, arabicBold),
175             pw.SizedBox(height: 12),
176         ],
177
178         // — Footer —
179         _buildFooter(arabicRegular),
180     ];
181 },
182 ),
183 );
184
185 return pdf.save();
186 }
187
188 // =====
189 // Header
190 // =====
191 static pw.Widget _buildPageHeader({
192     required String teacherName,
193     required String grade,

```



```

267     color: PdfColors.white,
268     shape: pw.BoxShape.circle,
269 ),
270 child: pw.ClipOval(
271   child: pw.Padding(
272     padding: const pw.EdgeInsets.all(4),
273     child: pw.Image(
274       ministryImage,
275       fit: pw.BoxFit.cover,
276     ),
277   ),
278 ),
279 ),
280   ],
281 ),
282 ],
283 ),
284 ),
285 pw.SizedBox(height: 16),
286
287 // Titles
288 pw.Text('تقرير $reportType', style: titleStyle.copyWith(color: _primaryColor, fontSize: 16)),
289 if (reportType != 'خطأ') pw.SizedBox(height: 4),
290 if (reportType != 'خطأ') pw.Text('بيانات التقرير الأساسية', style: arabicRegular.copyWith(color: _text
Muted)),
291
292 pw.SizedBox(height: 12),
293 // Basic Info Table
294 if (reportType != 'خطأ') _buildBasicInfoTable(
295   teacherName: teacherName,
296   grade: grade,
297   subject: subject,
298   semester: semester,
299   reportDate: reportDate,
300   lessonsCount: lessonsCount,
301   regular: arabicRegular,
302   bold: arabicBold,
303 ),
304 ],
305 );
306 }
307
308 static pw.Widget _buildBasicInfoTable({
309   required String teacherName,
310   required String grade,
311   required String subject,
312   required String semester,
313   required String reportDate,
314   required String lessonsCount,
315   required pw.TextStyle regular,
316   required pw.TextStyle bold,
317 }) {
318   return pw.TableHelper.fromTextArray(
319     headers: ['اسم المعلم', 'الصف', 'المادة', 'الفصل', 'تاريخ التقرير', 'عدد الحصص'],
320     data: [
321       [teacherName, grade, subject, semester, reportDate, lessonsCount]
322     ],
323     border: pw.TableBorder.all(color: _borderColor, width: 0.5),
324     headerStyle: bold.copyWith(color: PdfColors.white),
325     headerDecoration: pw.BoxDecoration(color: _primaryColor),
326     cellStyle: regular,
327     cellAlignments: {
328       0: pw.Alignment.center,
329       1: pw.Alignment.center,
330       2: pw.Alignment.center,
331       3: pw.Alignment.center,
332       4: pw.Alignment.center,
333       5: pw.Alignment.center,
334     },
335   );
336 }
337
338 static pw.Widget _buildSectionTitle(String title, pw.TextStyle headerStyle) {
339   return pw.Center(
340     child: pw.Text(

```

```

341         title,
342         style: headerStyle.copyWith(color: _primaryColor, fontSize: 14),
343     ),
344 );
345 }
346
347 // =====
348 // Tables
349 // =====
350
351 static pw.Widget _buildAchievementTable(
352     List<Map<String, dynamic>> rows,
353     pw.TextStyle regular,
354     pw.TextStyle bold,
355 ) {
356     return pw.TableHelper.fromTextArray(
357         context: null,
358         headers: ['الدرس', 'وصف الإنجاز', 'المشاركة', 'مستوى الفهم'],
359         data: rows
360             .map((r) => [
361                 _fixArabic(r['lesson'] ?? ''),
362                 _fixArabic(r['description'] ?? ''),
363                 _fixArabic(r['participation'] ?? ''),
364                 _fixArabic(r['understanding'] ?? ''),
365             ])
366             .toList(),
367         border: pw.TableBorder.all(color: _borderColor, width: 0.5),
368         headerStyle: bold.copyWith(color: PdfColors.white),
369         headerDecoration: pw.BoxDecoration(color: _primaryColor),
370         cellStyle: regular,
371         cellAlignments: {
372             0: pw.Alignment.center,
373             1: pw.Alignment.centerRight,
374             2: pw.Alignment.center,
375             3: pw.Alignment.center,
376         },
377         oddRowDecoration: pw.BoxDecoration(color: _lightGray),
378         columnWidths: {
379             0: const pw.FlexColumnWidth(1.5),
380             1: const pw.FlexColumnWidth(4),
381             2: const pw.FlexColumnWidth(1.2),
382             3: const pw.FlexColumnWidth(1.2),
383         },
384     );
385 }
386
387 static pw.Widget _buildLessonPlanTable(
388     List<Map<String, dynamic>> rows,
389     pw.TextStyle regular,
390     pw.TextStyle bold,
391 ) {
392     return pw.TableHelper.fromTextArray(
393         headers: ['الدرس', 'المفاهيم', 'الأهداف'],
394         data: rows
395             .map((r) => [
396                 _fixArabic(r['lesson'] ?? ''),
397                 _fixArabic(r['concepts'] ?? ''),
398                 _fixArabic(r['objectives'] ?? ''),
399             ])
400             .toList(),
401         border: pw.TableBorder.all(color: _borderColor, width: 0.5),
402         headerStyle: bold.copyWith(color: PdfColors.white),
403         headerDecoration: pw.BoxDecoration(color: _primaryColor),
404         cellStyle: regular,
405         oddRowDecoration: pw.BoxDecoration(color: _lightGray),
406         columnWidths: {
407             0: const pw.FlexColumnWidth(1.5),
408             1: const pw.FlexColumnWidth(3),
409             2: const pw.FlexColumnWidth(3),
410         },
411     );
412 }
413
414 static pw.Widget _buildGoalTable(
415     List<Map<String, dynamic>> rows,

```



```

416 pw.TextStyle regular,
417 pw.TextStyle bold,
418 ) {
419 return pw.TableHelper.fromTextArray(
420   headers: ['الدرس', 'الهدف', 'مستوى الفهم المستهدف'],
421   data: rows
422     .map((r) => [
423       _fixArabic(r['lesson'] ?? ''),
424       _fixArabic(r['goal'] ?? ''),
425       _fixArabic(r['targetUnderstanding'] ?? ''),
426     ])
427     .toList(),
428   border: pw.TableBorder.all(color: _borderColor, width: 0.5),
429   headerStyle: bold.copyWith(color: PdfColors.white),
430   headerDecoration: pw.BoxDecoration(color: _primaryColor),
431   cellStyle: regular,
432   oddRowDecoration: pw.BoxDecoration(color: _lightGray),
433 );
434 }
435
436 static pw.Widget _buildChallengeTable(
437   List<Map<String, dynamic>> rows,
438   pw.TextStyle regular,
439   pw.TextStyle bold,
440 ) {
441 return pw.TableHelper.fromTextArray(
442   headers: ['الدرس', 'التحدي', 'الخطوة المقترحة'],
443   data: rows
444     .map((r) => [
445       _fixArabic(r['lesson'] ?? ''),
446       _fixArabic(r['challenge'] ?? ''),
447       _fixArabic(r['plan'] ?? ''),
448     ])
449     .toList(),
450   border: pw.TableBorder.all(color: _borderColor, width: 0.5),
451   headerStyle: bold.copyWith(color: PdfColors.white),
452   headerDecoration: pw.BoxDecoration(color: _primaryColor),
453   cellStyle: regular,
454   oddRowDecoration: pw.BoxDecoration(color: _lightGray),
455 );
456 }
457
458 static pw.Widget _buildCheckUnderstandingTable(
459   List<Map<String, dynamic>> rows,
460   pw.TextStyle regular,
461   pw.TextStyle bold,
462 ) {
463 return pw.TableHelper.fromTextArray(
464   headers: ['الدرس', 'أسلوب التحقق', 'النتيجة'],
465   data: rows
466     .map((r) => [
467       _fixArabic(r['lesson'] ?? ''),
468       _fixArabic(r['method'] ?? ''),
469       _fixArabic(r['result'] ?? ''),
470     ])
471     .toList(),
472   border: pw.TableBorder.all(color: _borderColor, width: 0.5),
473   headerStyle: bold.copyWith(color: PdfColors.white),
474   headerDecoration: pw.BoxDecoration(color: _primaryColor),
475   cellStyle: regular,
476   oddRowDecoration: pw.BoxDecoration(color: _lightGray),
477 );
478 }
479
480 static pw.Widget _buildModelingTable(
481   List<Map<String, dynamic>> rows,
482   pw.TextStyle regular,
483   pw.TextStyle bold,
484 ) {
485 return pw.TableHelper.fromTextArray(
486   headers: ['الدرس', 'أسلوب النمذجة', 'الملاحظات'],
487   data: rows
488     .map((r) => [
489       _fixArabic(r['lesson'] ?? ''),
490       _fixArabic(r['modelingMethod'] ?? ''),

```

```

491         _fixArabic(r['notes'] ?? ''),
492     ])
493     .toList(),
494     border: pw.TableBorder.all(color: _borderColor, width: 0.5),
495     headerStyle: bold.copyWith(color: PdfColors.white),
496     headerDecoration: pw.BoxDecoration(color: _primaryColor),
497     cellStyle: regular,
498     oddRowDecoration: pw.BoxDecoration(color: _lightGray),
499 );
500 }
501
502 static pw.Widget _buildStrategyTable(
503     List<Map<String, dynamic>> rows,
504     pw.TextStyle regular,
505     pw.TextStyle bold,
506 ) {
507     return pw.TableHelper.fromTextArray(
508         headers: ['الاستراتيجية', 'الوصف', 'الدروس المطبقة'],
509         data: rows
510             .map((r) => [
511                 _fixArabic(r['strategy'] ?? ''),
512                 _fixArabic(r['description'] ?? ''),
513                 _fixArabic(r['appliedLessons'] ?? ''),
514             ])
515             .toList(),
516         border: pw.TableBorder.all(color: _borderColor, width: 0.5),
517         headerStyle: bold.copyWith(color: PdfColors.white),
518         headerDecoration: pw.BoxDecoration(color: _primaryColor),
519         cellStyle: regular,
520         oddRowDecoration: pw.BoxDecoration(color: _lightGray),
521     );
522 }
523
524 // _____
525 // Footer
526 // _____
527 static pw.Widget _buildFooter(pw.TextStyle regular) {
528     return pw.Container(
529         margin: const pw.EdgeInsets.only(top: 8),
530         padding: const pw.EdgeInsets.all(8),
531         decoration: pw.BoxDecoration(
532             color: _subHeaderBg,
533             borderRadius: const pw.BorderRadius.all(pw.Radius.circular(6)),
534         ),
535         child: pw.Row(
536             mainAxisAlignment: pw.MainAxisAlignment.spaceBetween,
537             children: [
538                 pw.Text(
539                     'تم انشاء بواسطة نظام معين التعليمي',
540                     style: regular.copyWith(color: _textMuted, fontSize: 8),
541                 ),
542                 pw.Text(
543                     DateTime.now().toLocal().toString().substring(0, 10),
544                     style: regular.copyWith(color: _textMuted, fontSize: 8),
545                 ),
546             ],
547         ),
548     );
549 }
550 }
551

```

```

1 class SavedReportModel {
2     final int id;
3     final String reportType;
4     final String grade;
5     final dynamic subject; // Can be String or List
6     final List<String> selectedLessons;
7     final Map<String, dynamic> reportData;
8     final String createdAt;
9

```

```

10 SavedReportModel({
11     required this.id,
12     required this.reportType,
13     required this.grade,
14     required this.subject,
15     required this.selectedLessons,
16     required this.reportData,
17     required this.createdAt,
18 });
19
20 factory SavedReportModel.fromJson(Map<String, dynamic> json) {
21     List<String> lessons = [];
22     if (json['selected_lessons'] != null) {
23         if (json['selected_lessons'] is List) {
24             lessons = List<String>.from(json['selected_lessons'].map((e) => e.toString()));
25         }
26     }
27
28     Map<String, dynamic> rData = {};
29     if (json['report_data'] is Map<String, dynamic>) {
30         rData = json['report_data'];
31     }
32
33     return SavedReportModel(
34         id: json['id'] is int ? json['id'] : int.tryParse(json['id']?.toString() ?? '0') ?? 0,
35         reportType: json['report_type']?.toString() ?? '',
36         grade: json['grade']?.toString() ?? '',
37         subject: json['subject'] ?? '',
38         selectedLessons: lessons,
39         reportData: rData,
40         createdAt: json['created_at']?.toString() ?? '',
41     );
42 }
43
44 String get displaySubject {
45     if (subject is List) {
46         return (subject as List).join(', ');
47     }
48     return subject?.toString() ?? '';
49 }
50 }
51
52 class SavedReportsMeta {
53     final int currentPage;
54     final int lastPage;
55     final int perPage;
56     final int total;
57
58     SavedReportsMeta({
59         required this.currentPage,
60         required this.lastPage,
61         required this.perPage,
62         required this.total,
63     });
64
65     factory SavedReportsMeta.fromJson(Map<String, dynamic> json) {
66         return SavedReportsMeta(
67             currentPage: json['current_page'] ?? 1,
68             lastPage: json['last_page'] ?? 1,
69             perPage: json['per_page'] ?? 15,
70             total: json['total'] ?? 0,
71         );
72     }
73 }
74

```

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/network/remote/api_service.dart';
4 import 'package:moean/features/reports/presentation/cubit/report_state.dart';
5

```

```

6 class ReportCubit extends Cubit<ReportState> {
7   ReportCubit() : super(const ReportInitial());
8
9   static ReportCubit get(BuildContext context) => BlocProvider.of(context);
10
11   Future<void> generateReport({
12     required String reportType,
13     required String grade,
14     required dynamic subject,
15     required List<String> selectedLessons,
16   }) async {
17     if (!isClosed) emit(const ReportLoading());
18
19     final result = await ApiService.generateEducationalReport(
20       reportType: reportType,
21       grade: grade,
22       subject: subject,
23       selectedLessons: selectedLessons,
24     );
25     if (isClosed) return;
26
27     result.fold(
28       (error) {
29         if (!isClosed) {
30           if (error.startsWith('__402__')) {
31             final parts = error.split(':');
32             final code = parts.length > 1 ? parts[1] : 'quota_exceeded';
33             final msg = parts.length > 2 ? parts.sublist(2).join(':') : 'ترقية الحساب المطلوبة';
34             emit(ReportPaymentRequired(message: msg, code: code));
35           } else {
36             emit(ReportError(message: error));
37           }
38         }
39       },
40       (data) {
41         if (!isClosed) emit(ReportSuccess(data: data));
42       },
43     );
44   }
45
46   void reset() {
47     if (!isClosed) emit(const ReportInitial());
48   }
49 }
50

```

Lines • Dart 45

Feature / Reports

#336 – lib/features/reports/presentation/cubit/report_state.dart

```

1 import 'package:equatable/equatable.dart';
2
3 abstract class ReportState extends Equatable {
4   const ReportState();
5
6   @override
7   List<Object?> get props => [];
8 }
9
10 class ReportInitial extends ReportState {
11   const ReportInitial();
12 }
13
14 class ReportLoading extends ReportState {
15   const ReportLoading();
16 }
17
18 class ReportSuccess extends ReportState {
19   final Map<String, dynamic> data;
20
21   const ReportSuccess({required this.data});
22
23   @override
24   List<Object?> get props => [data];
25 }

```

```

26
27 class ReportError extends ReportState {
28   final String message;
29
30   const ReportError({required this.message});
31
32   @override
33   List<Object?> get props => [message];
34 }
35
36 class ReportPaymentRequired extends ReportState {
37   final String message;
38   final String code;
39
40   const ReportPaymentRequired({required this.message, required this.code});
41
42   @override
43   List<Object?> get props => [message, code];
44 }
45

```

Lines • Dart 55

Feature / Reports

#337 – lib/features/reports/presentation/cubit/saved_reports_cubit.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/network/remote/api_service.dart';
4  import 'package:moean/features/reports/data/saved_report_model.dart';
5  import 'package:moean/features/reports/presentation/cubit/saved_reports_state.dart';
6
7  class SavedReportsCubit extends Cubit<SavedReportsState> {
8    SavedReportsCubit() : super(const SavedReportsInitial());
9
10   static SavedReportsCubit get(BuildContext context) => BlocProvider.of(context);
11
12   String _currentFilter = 'الكل';
13
14   String get currentFilter => _currentFilter;
15
16   Future<void> fetchReports({String? filter}) async {
17     if (filter != null) {
18       _currentFilter = filter;
19     }
20
21     if (!isClosed) emit(const SavedReportsLoading());
22
23     final String? apiType = (_currentFilter == 'الكل') ? null : _currentFilter;
24
25     final result = await ApiService.getSavedEducationalReports(
26       reportType: apiType,
27       perPage: 30,
28     );
29     if (isClosed) return;
30
31     result.fold(
32       (error) {
33         if (!isClosed) emit(SavedReportsError(message: error));
34       },
35       (response) {
36         try {
37           final List<dynamic> listData = response['data'] is List ? response['data'] : [];
38           final reports = listData.map((e) => SavedReportModel.fromJson(e as Map<String, dynamic>)).toList();
39           final meta = SavedReportsMeta.fromJson((response['meta'] as Map<String, dynamic>?) ?? {});
40
41           if (!isClosed) {
42             emit(SavedReportsSuccess(
43               reports: reports,
44               meta: meta,
45               selectedFilter: _currentFilter,
46             ));
47           }
48         } catch (e) {
49           if (!isClosed) emit(SavedReportsError(message: e.toString()));
50         }
51       }
52     );
53   }
54 }
55

```

```
51     },
52   );
53 }
54 }
55
```

Lines • Dart 42

Feature / Reports

#338 – lib/features/reports/presentation/cubit/saved_reports_state.dart

```
1 import 'package:equatable/equatable.dart';
2 import 'package:moean/features/reports/data/saved_report_model.dart';
3
4 abstract class SavedReportsState extends Equatable {
5   const SavedReportsState();
6
7   @override
8   List<Object?> get props => [];
9 }
10
11 class SavedReportsInitial extends SavedReportsState {
12   const SavedReportsInitial();
13 }
14
15 class SavedReportsLoading extends SavedReportsState {
16   const SavedReportsLoading();
17 }
18
19 class SavedReportsSuccess extends SavedReportsState {
20   final List<SavedReportModel> reports;
21   final SavedReportsMeta meta;
22   final String selectedFilter;
23
24   const SavedReportsSuccess({
25     required this.reports,
26     required this.meta,
27     required this.selectedFilter,
28   });
29
30   @override
31   List<Object?> get props => [reports, meta, selectedFilter];
32 }
33
34 class SavedReportsError extends SavedReportsState {
35   final String message;
36
37   const SavedReportsError({required this.message});
38
39   @override
40   List<Object?> get props => [message];
41 }
42
```

Lines • Dart 70

Feature / Reports

#339 – lib/features/reports/presentation/screen/pdf_preview_screen.dart

```
1 import 'dart:typed_data';
2 import 'package:flutter/material.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/extensions/context_extension.dart';
6 import 'package:printing/printing.dart';
7
8 class PdfPreviewScreen extends StatelessWidget {
9   final Future<Uint8List> Function() buildPdf;
10   final String title;
11
12   const PdfPreviewScreen({
13     super.key,
14     required this.buildPdf,
15     this.title = 'معاينة التقرير',
16   });
17
18   @override
19   Widget build(BuildContext context) {
```

```

20   return Directionality(
21     textDirection: TextDirection.rtl,
22     child: Scaffold(
23       backgroundColor: ColorsManager.background,
24       appBar: AppBar(
25         backgroundColor: ColorsManager.background,
26         elevation: 0,
27         centerTitle: true,
28         title: Text(
29           title,
30           style: TextStylesManager.bold18.copyWith(
31             color: ColorsManager.mainText,
32           ),
33         ),
34         leading: IconButton(
35           icon: Icon(Icons.arrow_forward, color: ColorsManager.mainText),
36           onPressed: () => context.pop(),
37         ),
38       ),
39       body: PdfPreview(
40         build: (format) => buildPdf(),
41         allowPrinting: true,
42         allowSharing: true,
43         canChangeOrientation: false,
44         canChangePageFormat: false,
45         canDebug: false,
46         pdfFileName: 'report.pdf',
47         loadingWidget: Center(
48           child: CircularProgressIndicator(
49             color: ColorsManager.primaryColor,
50           ),
51         ),
52         scrollViewDecoration: BoxDecoration(
53           color: ColorsManager.surfacePrimary,
54         ),
55         pdfPreviewPageDecoration: BoxDecoration(
56           color: Colors.white,
57           boxShadow: [
58             BoxShadow(
59               color: Colors.black.withValues(alpha: 0.1),
60               blurRadius: 10,
61               offset: const Offset(0, 5),
62             ),
63           ],
64         ),
65       ),
66     ),
67   );
68 }
69 }
70

```

Lines • Dart 484

Feature / Reports

#340 – lib/features/reports/presentation/screen/report_screen.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/network/local/cache_helper.dart';
4  import 'package:moean/core/network/remote/api_service.dart';
5  import 'package:moean/core/theme/colors.dart';
6  import 'package:moean/core/theme/text_styles.dart';
7  import 'package:moean/core/utils/constants/constants.dart';
8  import 'package:moean/core/utils/constants/spacing.dart';
9  import 'package:moean/core/utils/extensions/context_extension.dart';
10 import 'package:moean/features/reports/data/report_pdf_service.dart';
11 import 'package:moean/features/reports/presentation/cubit/report_cubit.dart';
12 import 'package:moean/features/reports/presentation/cubit/report_state.dart';
13 import 'package:moean/features/reports/presentation/screen/pdf_preview_screen.dart';
14 import 'package:moean/features/reports/presentation/screen/saved_reports_screen.dart';
15 import 'package:moean/features/reports/presentation/widgets/report_form_widget.dart';
16 import 'package:moean/features/reports/presentation/widgets/report_result_section_widget.dart';
17 import 'package:moean/features/payment/presentation/widgets/trial_banner_widget.dart';
18 import 'package:moean/core/utils/constants/primary/upgrade_prompt_bottom_sheet.dart';
19

```

```

20 class ReportScreen extends StatefulWidget {
21   const ReportScreen({super.key});
22
23   @override
24   State<ReportScreen> createState() => _ReportScreenState();
25 }
26
27 class _ReportScreenState extends State<ReportScreen> {
28   String _teacherName = '';
29
30   // These are stored after submission to pass to PDF generator
31   String _lastGrade = '';
32   String _lastSubject = '';
33   String _lastUnit = '';
34   String _lastSemester = '';
35   String _lastSchool = '';
36   String _lastOffice = '';
37   String _lastReportType = '';
38   String _lastReportDate = '';
39   List<String> _lastLessons = [];
40
41   @override
42   void initState() {
43     super.initState();
44     _loadTeacherName();
45   }
46
47   Future<void> _loadTeacherName() async {
48     final cached = CacheHelper.getData(key: 'teacher_name');
49     if (cached is String && cached.isNotEmpty) {
50       setState(() => _teacherName = cached);
51       return;
52     }
53     final result = await ApiService.getProfile();
54     result.fold(
55       (_) {},
56       (profile) {
57         final name = profile.user.name;
58         CacheHelper.saveData(key: 'teacher_name', value: name);
59         if (mounted) setState(() => _teacherName = name);
60       },
61     );
62   }
63
64   void _handlePrint(Map<String, dynamic> reportData) {
65     Navigator.of(context).push(
66       MaterialPageRoute(
67         builder: (context) => PdfPreviewScreen(
68           buildPdf: () => ReportPdfService.generatePdf(
69             reportData: reportData,
70             teacherName: _teacherName,
71             grade: _lastGrade,
72             subject: _lastSubject,
73             unit: _lastUnit,
74             semester: _lastSemester,
75             schoolName: _lastSchool,
76             educationOffice: _lastOffice,
77             reportType: _lastReportType,
78             reportDate: _lastReportDate,
79             selectedLessons: _lastLessons,
80           ),
81         ),
82       ),
83     );
84   }
85
86   @override
87   Widget build(BuildContext context) {
88     return BlocConsumer<ReportCubit, ReportState>(
89       listener: (context, state) {
90         if (state is ReportError) {
91           ScaffoldMessenger.of(context).showSnackBar(
92             SnackBar(
93               content: Text(state.message, style: TextStylesManager.bold14),
94               backgroundColor: ColorsManager.errorColor,

```



```

95         behavior: SnackBarBehavior.floating,
96     ),
97 );
98 } else if (state is ReportPaymentRequired) {
99     UpgradePromptBottomSheet.show(
100         context,
101         message: state.message,
102         isQuotaExceeded: state.code == 'quota_exceeded',
103     );
104 }
105 },
106 builder: (context, state) {
107     return Directionality(
108         textDirection: TextDirection.rtl,
109         child: Scaffold(
110             backgroundColor: ColorsManager.background,
111             appBar: AppBar(
112                 backgroundColor: ColorsManager.background,
113                 elevation: 0,
114                 centerTitle: true,
115                 title: Text(
116                     appTranslation().get('report_title'),
117                     style: TextStylesManager.bold18.copyWith(
118                         color: ColorsManager.mainText,
119                     ),
120                 ),
121             leading: IconButton(
122                 icon: Icon(Icons.arrow_back_ios),
123                 onPressed: () => context.pop(),
124             ),
125             actions: [
126                 IconButton(
127                     tooltip: appTranslation().get('saved_reports_subtitle'),
128                     icon: Icon(
129                         Icons.inventory_2_outlined,
130                         color: ColorsManager.primaryColor,
131                     ),
132                     onPressed: () {
133                         Navigator.of(context).push(
134                             MaterialPageRoute(
135                                 builder: (context) => SavedReportsScreen(teacherName: _teacherName),
136                             ),
137                         );
138                     },
139                 ),
140             if (state is ReportSuccess)
141                 IconButton(
142                     tooltip: appTranslation().get('report_new'),
143                     icon: Icon(
144                         Icons.refresh_rounded,
145                         color: ColorsManager.primaryColor,
146                     ),
147                     onPressed: () => ReportCubit.get(context).reset(),
148                 ),
149             ],
150         ),
151     body: SafeArea(
152         child: Column(
153             children: [
154                 const TrialBannerWidget(),
155                 Expanded(
156                     child: SingleChildScrollView(
157                         padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 8),
158                         child: Column(
159                             children: [
160                                 // Banner link to Saved Reports above the form/results
161                                 InkWell(
162                                     onTap: () {
163                                         Navigator.of(context).push(
164                                             MaterialPageRoute(
165                                                 builder: (context) => SavedReportsScreen(teacherName: _teacherName),
166                                             ),
167                                         );
168                                     },
169                                 borderRadius: BorderRadius.circular(16),

```

```

170         child: Container(
171           padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
172           decoration: BoxDecoration(
173             color: ColorsManager.primaryColor.withValues(alpha: 0.08),
174             borderRadius: BorderRadius.circular(16),
175             border: Border.all(
176               color: ColorsManager.primaryColor.withValues(alpha: 0.2),
177             ),
178           ),
179         child: Row(
180           children: [
181             Container(
182               padding: const EdgeInsets.all(6),
183               decoration: BoxDecoration(
184                 color: ColorsManager.primaryColor.withValues(alpha: 0.12),
185                 shape: BoxShape.circle,
186               ),
187               child: Icon(
188                 Icons.inventory_2_outlined,
189                 color: ColorsManager.primaryColor,
190                 size: 18,
191               ),
192             ),
193             horizontalSpace10,
194             Expanded(
195               child: Column(
196                 crossAxisAlignment: CrossAxisAlignment.start,
197                 children: [
198                   Text(
199                     appTranslation().get('saved_reports_subtitle'),
200                     style: TextStyleManager.bold14.copyWith(
201                       color: ColorsManager.primaryColor,
202                     ),
203                   ),
204                   Text(
205                     appTranslation().get('saved_reports_desc'),
206                     style: TextStyleManager.regular12.copyWith(
207                       color: ColorsManager.secondaryText,
208                     ),
209                   ),
210                 ],
211             ),
212             horizontalSpace8,
213             Icon(
214               Icons.arrow_back_ios_new,
215               size: 16,
216               color: ColorsManager.primaryColor,
217             ),
218           ],
219         ),
220       ),
221     ),
222   ),
223   verticalSpace16,
224
225   state is ReportSuccess
226     ? _ReportResultView(
227       data: state.data,
228       teacherName: _teacherName,
229       onPrint: () => _handlePrint(state.data),
230     )
231     : ReportFormWidget(
232       teacherName: _teacherName,
233       isLoading: state is ReportLoading,
234       onSubmit: ({
235         required reportType,
236         required grade,
237         required subject,
238         required unit,
239         required semester,
240         required schoolName,
241         required educationOffice,
242         required reportDate,
243         required selectedLessons,
244       }) {

```

```

245         _lastGrade = grade;
246         _lastSubject = subject;
247         _lastUnit = unit;
248         _lastSemester = semester;
249         _lastSchool = schoolName;
250         _lastOffice = educationOffice;
251         _lastReportType = reportType;
252         _lastReportDate = reportDate;
253         _lastLessons = selectedLessons;
254
255         // For monthly reports, send units as list. For weekly, combine subject and unit.
t.
256         final dynamic apiSubject = reportType == 'شهري'
257             ? unit.split('.').map((e) => e.trim()).where((e) => e.isNotEmpty()).toList()
258             : '$subject - $unit';
259
260         ReportCubit.get(context).generateReport(
261             reportType: reportType,
262             grade: grade,
263             subject: apiSubject,
264             selectedLessons: selectedLessons,
265         );
266     },
267 ),
268 ],
269 ),
270 ),
271 ),
272 ],
273 ),
274 ),
275 ),
276 );
277 },
278 );
279 }
280 }
281
282 // =====
283 // Result view - shown after successful API response
284 // =====
285
286 class _ReportResultView extends StatelessWidget {
287     final Map<String, dynamic> data;
288     final String teacherName;
289     final VoidCallback onPrint;
290
291     const _ReportResultView({
292         required this.data,
293         required this.teacherName,
294         required this.onPrint,
295     });
296
297     @override
298     Widget build(BuildContext context) {
299         final List<Map<String, dynamic>> achievementRows =
300             List<Map<String, dynamic>>.from(data['achievementRows'] ?? []);
301         final List<Map<String, dynamic>> lessonPlanRows =
302             List<Map<String, dynamic>>.from(data['lessonPlanRows'] ?? []);
303         final List<Map<String, dynamic>> goalRows =
304             List<Map<String, dynamic>>.from(data['goalRows'] ?? []);
305         final List<Map<String, dynamic>> challengeRows =
306             List<Map<String, dynamic>>.from(data['challengeRows'] ?? []);
307         final List<Map<String, dynamic>> checkUnderstandingRows =
308             List<Map<String, dynamic>>.from(data['checkUnderstandingRows'] ?? []);
309         final List<Map<String, dynamic>> modelingRows =
310             List<Map<String, dynamic>>.from(data['modelingRows'] ?? []);
311         final List<Map<String, dynamic>> strategySections =
312             List<Map<String, dynamic>>.from(data['strategySections'] ?? []);
313
314         return Column(
315             crossAxisAlignment: CrossAxisAlignment.stretch,
316             children: [
317                 // Success header
318                 _ReportSuccessBanner(teacherName: teacherName),

```

```

319     verticalSpace16,
320
321     // Print button
322     ReportPrintBarWidget(
323         onPrint: onPrint,
324     ),
325     verticalSpace20,
326
327     // Result tables
328     if (achievementRows.isNotEmpty)
329         ReportResultSectionWidget(
330             sectionTitle: appTranslation().get('report_achievement_rows'),
331             rows: achievementRows,
332             columnKeys: const ['lesson', 'description', 'participation', 'understanding'],
333             columnLabels: [
334                 appTranslation().get('report_col_lesson'),
335                 appTranslation().get('report_col_description'),
336                 appTranslation().get('report_col_participation'),
337                 appTranslation().get('report_col_understanding'),
338             ],
339             initiallyExpanded: true,
340         ),
341
342     if (lessonPlanRows.isNotEmpty)
343         ReportResultSectionWidget(
344             sectionTitle: appTranslation().get('report_lesson_plan_rows'),
345             rows: lessonPlanRows,
346             columnKeys: const ['lesson', 'concepts', 'objectives'],
347             columnLabels: [
348                 appTranslation().get('report_col_lesson'),
349                 appTranslation().get('report_col_concepts'),
350                 appTranslation().get('report_col_objectives'),
351             ],
352         ),
353
354     if (goalRows.isNotEmpty)
355         ReportResultSectionWidget(
356             sectionTitle: appTranslation().get('report_goal_rows'),
357             rows: goalRows,
358             columnKeys: const ['lesson', 'goal', 'targetUnderstanding'],
359             columnLabels: [
360                 appTranslation().get('report_col_lesson'),
361                 appTranslation().get('report_col_goal'),
362                 appTranslation().get('report_col_target'),
363             ],
364         ),
365
366     if (challengeRows.isNotEmpty)
367         ReportResultSectionWidget(
368             sectionTitle: appTranslation().get('report_challenge_rows'),
369             rows: challengeRows,
370             columnKeys: const ['lesson', 'challenge', 'plan'],
371             columnLabels: [
372                 appTranslation().get('report_col_lesson'),
373                 appTranslation().get('report_col_challenge'),
374                 appTranslation().get('report_col_plan'),
375             ],
376         ),
377
378     if (checkUnderstandingRows.isNotEmpty)
379         ReportResultSectionWidget(
380             sectionTitle: appTranslation().get('report_check_understanding_rows'),
381             rows: checkUnderstandingRows,
382             columnKeys: const ['lesson', 'method', 'result'],
383             columnLabels: [
384                 appTranslation().get('report_col_lesson'),
385                 appTranslation().get('report_col_method'),
386                 appTranslation().get('report_col_result'),
387             ],
388         ),
389
390     if (modelingRows.isNotEmpty)
391         ReportResultSectionWidget(
392             sectionTitle: appTranslation().get('report_modeling_rows'),
393             rows: modelingRows,

```

```

394         columnKeys: const ['lesson', 'modelingMethod', 'notes'],
395         columnLabels: [
396             appTranslation().get('report_col_lesson'),
397             appTranslation().get('report_col_modeling_method'),
398             appTranslation().get('report_col_notes'),
399         ],
400     ),
401
402     if (strategySections.isNotEmpty)
403         ReportResultSectionWidget(
404             sectionTitle: appTranslation().get('report_strategy_sections'),
405             rows: strategySections,
406             columnKeys: const ['strategy', 'description', 'appliedLessons'],
407             columnLabels: [
408                 appTranslation().get('report_col_strategy'),
409                 appTranslation().get('report_col_description'),
410                 appTranslation().get('report_col_applied_lessons'),
411             ],
412         ),
413
414     verticalSpace32,
415 ],
416 );
417 }
418 }
419
420 class _ReportSuccessBanner extends StatelessWidget {
421     final String teacherName;
422
423     const _ReportSuccessBanner({required this.teacherName});
424
425     @override
426     Widget build(BuildContext context) {
427         return Container(
428             padding: const EdgeInsets.all(16),
429             decoration: BoxDecoration(
430                 gradient: LinearGradient(
431                     colors: [
432                         ColorsManager.primaryColor.withValues(alpha: 0.12),
433                         ColorsManager.primaryColor.withValues(alpha: 0.04),
434                     ],
435                     begin: Alignment.topRight,
436                     end: Alignment.bottomLeft,
437                 ),
438                 borderRadius: BorderRadius.circular(16),
439                 border: Border.all(
440                     color: ColorsManager.primaryColor.withValues(alpha: 0.3),
441                 ),
442             ),
443             child: Row(
444                 children: [
445                     Container(
446                         width: 44,
447                         height: 44,
448                         decoration: BoxDecoration(
449                             color: ColorsManager.successColor.withValues(alpha: 0.15),
450                             shape: BoxShape.circle,
451                         ),
452                         child: const Icon(
453                             Icons.check_circle_rounded,
454                             color: ColorsManager.successColor,
455                             size: 24,
456                         ),
457                     ),
458                     horizontalSpace12,
459                     Expanded(
460                         child: Column(
461                             crossAxisAlignment: CrossAxisAlignment.start,
462                             children: [
463                                 Text(
464                                     appTranslation().get('report_success_title'),
465                                     style: TextStylesManager.bold14.copyWith(
466                                         color: ColorsManager.mainText,
467                                     ),
468                                 ),

```

```

469         verticalSpace2,
470         Text(
471             appTranslation().get('report_success_subtitle'),
472             style: TextStyleManager.regular12.copyWith(
473                 color: ColorsManager.secondaryText,
474             ),
475         ),
476     ],
477 ),
478 ),
479 ],
480 ),
481 );
482 }
483 }
484

```

Lines • Dart 187

Feature / #341 -

Reports [lib/features/reports/presentation/screen/saved_report_details_screen.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6  import 'package:moean/core/utils/extensions/context_extension.dart';
7  import 'package:moean/features/reports/data/report_pdf_service.dart';
8  import 'package:moean/features/reports/data/saved_report_model.dart';
9  import 'package:moean/features/reports/presentation/screen/pdf_preview_screen.dart';
10 import 'package:moean/features/reports/presentation/widgets/report_result_section_widget.dart';
11
12 class SavedReportDetailsScreen extends StatelessWidget {
13     final SavedReportModel report;
14     final String teacherName;
15
16     const SavedReportDetailsScreen({
17         super.key,
18         required this.report,
19         required this.teacherName,
20     });
21
22     void _handlePrint(BuildContext context) {
23         Navigator.of(context).push(
24             MaterialPageRoute(
25                 builder: (context) => PdfPreviewScreen(
26                     buildPdf: () => ReportPdfService.generatePdf(
27                         reportData: report.reportData,
28                         teacherName: teacherName,
29                         grade: report.grade,
30                         subject: report.displaySubject,
31                         unit: '',
32                         semester: '',
33                         schoolName: '',
34                         educationOffice: '',
35                         reportType: report.reportType,
36                         reportDate: report.createdAt.split('T').first,
37                         selectedLessons: report.selectedLessons,
38                     ),
39                 ),
40             ),
41         );
42     }
43
44     @override
45     Widget build(BuildContext context) {
46         final data = report.reportData;
47         final List<Map<String, dynamic>> achievementRows =
48             List<Map<String, dynamic>>.from(data['achievementRows'] ?? []);
49         final List<Map<String, dynamic>> lessonPlanRows =
50             List<Map<String, dynamic>>.from(data['lessonPlanRows'] ?? []);
51         final List<Map<String, dynamic>> goalRows =
52             List<Map<String, dynamic>>.from(data['goalRows'] ?? []);
53         final List<Map<String, dynamic>> challengeRows =

```

```

54     List<Map<String, dynamic>>.from(data['challengeRows'] ?? []);
55 final List<Map<String, dynamic>> checkUnderstandingRows =
56     List<Map<String, dynamic>>.from(data['checkUnderstandingRows'] ?? []);
57 final List<Map<String, dynamic>> modelingRows =
58     List<Map<String, dynamic>>.from(data['modelingRows'] ?? []);
59 final List<Map<String, dynamic>> strategySections =
60     List<Map<String, dynamic>>.from(data['strategySections'] ?? []);
61
62 return Directionality(
63     textDirection: TextDirection.rtl,
64     child: Scaffold(
65         backgroundColor: ColorsManager.background,
66         appBar: AppBar(
67             backgroundColor: ColorsManager.background,
68             elevation: 0,
69             centerTitle: true,
70             title: Text(
71                 report.displaySubject,
72                 style: TextStylesManager.bold16.copyWith(
73                     color: ColorsManager.mainText,
74                 ),
75             ),
76             leading: IconButton(
77                 icon: const Icon(Icons.arrow_back_ios),
78                 onPressed: () => context.pop(),
79             ),
80         ),
81         body: SafeArea(
82             child: SingleChildScrollView(
83                 padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 12),
84                 child: Column(
85                     crossAxisAlignment: CrossAxisAlignment.stretch,
86                     children: [
87                         ReportPrintBarWidget(
88                             onPrint: () => _handlePrint(context),
89                         ),
90                         verticalSpace20,
91
92                         if (achievementRows.isNotEmpty)
93                             ReportResultSectionWidget(
94                                 sectionTitle: appTranslation().get('report_achievement_rows'),
95                                 rows: achievementRows,
96                                 columnKeys: const ['lesson', 'description', 'participation', 'understanding'],
97                                 columnLabels: [
98                                     appTranslation().get('report_col_lesson'),
99                                     appTranslation().get('report_col_description'),
100                                    appTranslation().get('report_col_participation'),
101                                    appTranslation().get('report_col_understanding'),
102                                ],
103                                 initiallyExpanded: true,
104                             ),
105
106                         if (lessonPlanRows.isNotEmpty)
107                             ReportResultSectionWidget(
108                                 sectionTitle: appTranslation().get('report_lesson_plan_rows'),
109                                 rows: lessonPlanRows,
110                                 columnKeys: const ['lesson', 'concepts', 'objectives'],
111                                 columnLabels: [
112                                     appTranslation().get('report_col_lesson'),
113                                     appTranslation().get('report_col_concepts'),
114                                     appTranslation().get('report_col_objectives'),
115                                ],
116                             ),
117
118                         if (goalRows.isNotEmpty)
119                             ReportResultSectionWidget(
120                                 sectionTitle: appTranslation().get('report_goal_rows'),
121                                 rows: goalRows,
122                                 columnKeys: const ['lesson', 'goal', 'targetUnderstanding'],
123                                 columnLabels: [
124                                     appTranslation().get('report_col_lesson'),
125                                     appTranslation().get('report_col_goal'),
126                                     appTranslation().get('report_col_target'),
127                                ],
128                             ),

```

```

129
130     if (challengeRows.isNotEmpty)
131       ReportResultSectionWidget(
132         sectionTitle: appTranslation().get('report_challenge_rows'),
133         rows: challengeRows,
134         columnKeys: const ['lesson', 'challenge', 'plan'],
135         columnLabels: [
136           appTranslation().get('report_col_lesson'),
137           appTranslation().get('report_col_challenge'),
138           appTranslation().get('report_col_plan'),
139         ],
140       ),
141
142     if (checkUnderstandingRows.isNotEmpty)
143       ReportResultSectionWidget(
144         sectionTitle: appTranslation().get('report_check_understanding_rows'),
145         rows: checkUnderstandingRows,
146         columnKeys: const ['lesson', 'method', 'result'],
147         columnLabels: [
148           appTranslation().get('report_col_lesson'),
149           appTranslation().get('report_col_method'),
150           appTranslation().get('report_col_result'),
151         ],
152       ),
153
154     if (modelingRows.isNotEmpty)
155       ReportResultSectionWidget(
156         sectionTitle: appTranslation().get('report_modeling_rows'),
157         rows: modelingRows,
158         columnKeys: const ['lesson', 'modelingMethod', 'notes'],
159         columnLabels: [
160           appTranslation().get('report_col_lesson'),
161           appTranslation().get('report_col_modeling_method'),
162           appTranslation().get('report_col_notes'),
163         ],
164       ),
165
166     if (strategySections.isNotEmpty)
167       ReportResultSectionWidget(
168         sectionTitle: appTranslation().get('report_strategy_sections'),
169         rows: strategySections,
170         columnKeys: const ['strategy', 'description', 'appliedLessons'],
171         columnLabels: [
172           appTranslation().get('report_col_strategy'),
173           appTranslation().get('report_col_description'),
174           appTranslation().get('report_col_applied_lessons'),
175         ],
176       ),
177
178     verticalSpace32,
179   ],
180 ),
181 ),
182 ),
183 ),
184 );
185 }
186 }
187

```

Lines • Dart 153

Feature / Reports

#342 – lib/features/reports/presentation/screen/saved_reports_screen.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/primary/conditional_builder.dart';
7  import 'package:moean/core/utils/constants/spacing.dart';
8  import 'package:moean/features/reports/presentation/cubit/saved_reports_cubit.dart';
9  import 'package:moean/features/reports/presentation/cubit/saved_reports_state.dart';
10 import 'package:moean/features/reports/presentation/screen/saved_report_details_screen.dart';
11 import 'package:moean/features/reports/presentation/widgets/saved_report_card_widget.dart';

```



```

12 import 'package:moean/features/reports/presentation/widgets/saved_reports_filter_widget.dart';
13 import 'package:moean/features/reports/presentation/widgets/saved_reports_header_widget.dart';
14
15 class SavedReportsScreen extends StatelessWidget {
16   final String teacherName;
17
18   const SavedReportsScreen({
19     super.key,
20     required this.teacherName,
21   });
22
23   @override
24   Widget build(BuildContext context) {
25     return BlocProvider(
26       create: (context) => SavedReportsCubit()..fetchReports(),
27       child: Directionality(
28         textDirection: TextDirection.rtl,
29         child: Scaffold(
30           backgroundColor: ColorsManager.background,
31           body: SafeArea(
32             child: BlocConsumer<SavedReportsCubit, SavedReportsState>(
33               listener: (context, state) {
34                 if (state is SavedReportsError) {
35                   ScaffoldMessenger.of(context).showSnackBar(
36                     SnackBar(
37                       content: Text(state.message, style: TextStylesManager.bold14),
38                       backgroundColor: ColorsManager.errorColor,
39                       behavior: SnackBarBehavior.floating,
40                     ),
41                 );
42               },
43             ),
44             builder: (context, state) {
45               final cubit = SavedReportsCubit.get(context);
46               final reports = state is SavedReportsSuccess ? state.reports : [];
47
48               return SingleChildScrollView(
49                 padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 16),
50                 child: Column(
51                   crossAxisAlignment: CrossAxisAlignment.start,
52                   children: [
53                     SavedReportsHeaderWidget(
54                       onRefresh: () => cubit.fetchReports(),
55                     ),
56                     verticalSpace24,
57                     SavedReportsFilterWidget(
58                       selectedFilter: cubit.currentFilter,
59                       onFilterChanged: (filter) => cubit.fetchReports(filter: filter),
60                     ),
61                     verticalSpace20,
62                     ConditionalBuilder(
63                       loadingState: state is SavedReportsLoading,
64                       errorState: state is SavedReportsError,
65                       emptyState: state is SavedReportsSuccess && reports.isEmpty,
66                       emptyBuilder: (context) => Padding(
67                         padding: const EdgeInsets.symmetric(vertical: 40),
68                         child: Center(
69                           child: Text(
70                             appTranslation().get('no_saved_reports'),
71                             style: TextStylesManager.regular16.copyWith(
72                               color: ColorsManager.secondaryText,
73                             ),
74                         ),
75                       ),
76                     ),
77                     errorBuilder: (context) => Padding(
78                       padding: const EdgeInsets.symmetric(vertical: 40),
79                       child: Center(
80                         child: Column(
81                           children: [
82                             Text(
83                               (state as SavedReportsError).message,
84                               style: TextStylesManager.bold14.copyWith(
85                                 color: ColorsManager.errorColor,
86                               ),

```

```

87         ),
88         verticalSpace12,
89         ElevatedButton(
90             onPressed: () => cubit.fetchReports(),
91             child: Text(appTranslation().get('retry')),
92         ),
93     ],
94 ),
95 ),
96 ),
97 successBuilder: (context) => LayoutBuilder(
98     builder: (context, constraints) {
99         final isWide = constraints.maxWidth > 600;
100         if (isWide) {
101             return GridView.builder(
102                 shrinkWrap: true,
103                 physics: const NeverScrollableScrollPhysics(),
104                 gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
105                     crossAxisCount: 2,
106                     crossAxisSpacing: 16,
107                     mainAxisSpacing: 16,
108                     mainAxisExtent: 260,
109                 ),
110                 itemCount: reports.length,
111                 itemBuilder: (context, index) => SavedReportCardWidget(
112                     report: reports[index],
113                     onOpen: () => _openDetails(context, reports[index]),
114                 ),
115             );
116         }
117         return ListView.separated(
118             shrinkWrap: true,
119             physics: const NeverScrollableScrollPhysics(),
120             itemCount: reports.length,
121             separatorBuilder: (context, index) => verticalSpace16,
122             itemBuilder: (context, index) => SavedReportCardWidget(
123                 report: reports[index],
124                 onOpen: () => _openDetails(context, reports[index]),
125             ),
126         );
127     },
128 ),
129 ),
130 verticalSpace32,
131 ],
132 ),
133 );
134 },
135 ),
136 ),
137 ),
138 ),
139 );
140 }
141
142 void _openDetails(BuildContext context, dynamic report) {
143     Navigator.of(context).push(
144         MaterialPageRoute(
145             builder: (context) => SavedReportDetailsScreen(
146                 report: report,
147                 teacherName: teacherName,
148             ),
149         ),
150     );
151 }
152 }
153

```

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';

```

```

4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
6 import 'package:moean/core/utils/constants/primary/primary_text_field.dart';
7 import 'package:moean/core/utils/constants/spacing.dart';
8 import 'package:moean/core/network/remote/api_service.dart';
9 import 'package:moean/features/exam_generation/data/models/curriculum_models.dart';
10 import 'package:moean/features/reports/presentation/widgets/report_lessons_widget.dart';
11 import 'package:moean/features/reports/presentation/widgets/report_type_selector_widget.dart';
12 import 'package:moean/features/reports/presentation/widgets/selection_bottom_sheet.dart';
13
14 class ReportFormWidget extends StatefulWidget {
15   final String teacherName;
16   final void Function({
17     required String reportType,
18     required String grade,
19     required String subject,
20     required String unit,
21     required String semester,
22     required String schoolName,
23     required String educationOffice,
24     required String reportDate,
25     required List<String> selectedLessons,
26   }) onSubmit;
27   final bool isLoading;
28
29   const ReportFormWidget({
30     super.key,
31     required this.teacherName,
32     required this.onSubmit,
33     required this.isLoading,
34   });
35
36   @override
37   State<ReportFormWidget> createState() => _ReportFormWidgetState();
38 }
39
40 class _ReportFormWidgetState extends State<ReportFormWidget> {
41   final _formKey = GlobalKey<FormState>();
42   final _stageController = TextEditingController();
43   final _gradeController = TextEditingController();
44   final _subjectController = TextEditingController();
45   final _unitController = TextEditingController();
46   final _semesterController = TextEditingController();
47   final _schoolController = TextEditingController();
48   final _educationOfficeController = TextEditingController();
49   final _reportDateController = TextEditingController();
50
51   String _selectedType = 'اسبوعي';
52   List<String> _selectedLessons = [];
53
54   bool _isLoadingSubjects = false;
55   List<CurriculumStageModel> _stages = [];
56
57   CurriculumStageModel? _selectedStage;
58   CurriculumGradeModel? _selectedGrade;
59   CurriculumSubjectModel? _selectedSubject;
60
61   bool _isLoadingLessons = false;
62   SubjectDetailsModel? _subjectDetails;
63
64   @override
65   void initState() {
66     super.initState();
67     _fetchSubjects();
68   }
69
70   Future<void> _fetchSubjects() async {
71     setState(() => _isLoadingSubjects = true);
72     final result = await ApiService.getSubjects();
73     if (mounted) {
74       result.fold(
75         (failure) {
76           setState(() => _isLoadingSubjects = false);
77         },
78         (stagesData) {

```

```

79         setState(() {
80             stagesData.sort((a, b) {
81                 int getWeight(String name) {
82                     if (name.contains('الابتدائية')) return 1;
83                     if (name.contains('المتوسطة')) return 2;
84                     if (name.contains('الثانوية')) return 3;
85                     return 4;
86                 }
87                 return getWeight(a.name).compareTo(getWeight(b.name));
88             });
89             _stages = stagesData;
90             _isLoadingSubjects = false;
91         });
92     },
93 );
94 }
95 }
96
97 void _showStageSelector() {
98     if (_isLoadingSubjects) {
99         ScaffoldMessenger.of(context).showSnackBar(
100             SnackBar(
101                 content: Text('جاري التحميل...', style: TextStylesManager.bold14),
102                 backgroundColor: ColorsManager.primaryColor,
103             ),
104         );
105         return;
106     }
107
108     final stageNames = _stages.map((e) => e.name).toList();
109
110     SelectionBottomSheet.show(
111         context: context,
112         title: 'اختر المرحلة الدراسية',
113         items: stageNames,
114         initialSelectedItem: _stageController.text.isNotEmpty ? [_stageController.text] : [],
115         isMultiSelect: false,
116         onSelectionConfirmed: (selected) {
117             if (selected.isNotEmpty) {
118                 setState(() {
119                     _stageController.text = selected.first;
120                     _selectedStage = _stages.firstWhere((e) => e.name == selected.first);
121
122                     _gradeController.clear();
123                     _selectedGrade = null;
124
125                     _subjectController.clear();
126                     _selectedSubject = null;
127
128                     _unitController.clear();
129                     _selectedLessons.clear();
130                     _subjectDetails = null;
131                 });
132             }
133         },
134     );
135 }
136
137 void _showGradeSelector() {
138     if (_selectedStage == null) {
139         ScaffoldMessenger.of(context).showSnackBar(
140             SnackBar(
141                 content: Text('الرجاء اختيار المرحلة أولاً', style: TextStylesManager.bold14),
142                 backgroundColor: ColorsManager.errorColor,
143             ),
144         );
145         return;
146     }
147
148     final gradeNames = _selectedStage!.grades.map((e) => e.name).toList();
149
150     SelectionBottomSheet.show(
151         context: context,
152         title: 'اختر الصف',
153         items: gradeNames,

```

```

154     initialSelectedItems: _gradeController.text.isNotEmpty ? [_gradeController.text] : [],
155     isMultiSelect: false,
156     onSelectionConfirmed: (selected) {
157         if (selected.isNotEmpty) {
158             setState(() {
159                 _gradeController.text = selected.first;
160                 _selectedGrade = _selectedStage!.grades.firstWhere((e) => e.name == selected.first);
161
162                 _subjectController.clear();
163                 _selectedSubject = null;
164
165                 _unitController.clear();
166                 _selectedLessons.clear();
167                 _subjectDetails = null;
168             });
169         }
170     },
171 );
172 }
173
174 void _showSubjectSelector() {
175     if (_selectedGrade == null) {
176         ScaffoldMessenger.of(context).showSnackBar(
177             SnackBar(
178                 content: Text('الرجاء اختيار الصف أولاً', style: TextStyleManager.bold14),
179                 backgroundColor: ColorsManager.errorColor,
180             ),
181         );
182         return;
183     }
184
185     final subjectNames = _selectedGrade!.subjects.map((e) => e.name).toList();
186
187     SelectionBottomSheet.show(
188         context: context,
189         title: 'اختر المادة',
190         items: subjectNames,
191         initialSelectedItems: _subjectController.text.isNotEmpty ? [_subjectController.text] : [],
192         isMultiSelect: false,
193         onSelectionConfirmed: (selected) {
194             if (selected.isNotEmpty) {
195                 setState(() {
196                     _subjectController.text = selected.first;
197                     _selectedSubject = _selectedGrade!.subjects.firstWhere((e) => e.name == selected.first);
198
199                     _unitController.clear();
200                     _selectedLessons.clear();
201                     _subjectDetails = null;
202
203                     _fetchSubjectDetails(_selectedSubject!.id);
204                 });
205             }
206         },
207     );
208 }
209
210 Future<void> _fetchSubjectDetails(int subjectId) async {
211     setState(() => _isLoadingLessons = true);
212     final result = await ApiService.getSubjectLessons(subjectId);
213     if (mounted) {
214         result.fold(
215             (failure) {
216                 setState(() => _isLoadingLessons = false);
217             },
218             (details) {
219                 setState(() {
220                     _subjectDetails = details;
221                     _isLoadingLessons = false;
222                 });
223             },
224         );
225     }
226 }
227
228 void _showUnitSelector() {

```

```

229     if (_selectedSubject == null || _subjectDetails == null) {
230         ScaffoldMessenger.of(context).showSnackBar(
231             SnackBar(
232                 content: Text('الرجاء اختيار المادة والانتظار للتحميل', style: TextStylesManager.bold14),
233                 backgroundColor: ColorsManager.errorColor,
234             ),
235         );
236         return;
237     }
238
239     final isMonthly = _selectedType == 'شهري';
240     final chapterTitles = _subjectDetails!.chapters.map((e) => e.title).toSet().toList();
241
242     SelectionBottomSheet.show(
243         context: context,
244         title: isMonthly ? 'اختر الفصول' : 'اختر الفصل',
245         items: chapterTitles,
246         initialSelectedItems: _unitController.text.isNotEmpty
247             ? _unitController.text.split(' ').map((e) => e.trim()).toList()
248             : [],
249         isMultiSelect: isMonthly,
250         onSelectionConfirmed: (selected) {
251             setState(() {
252                 _unitController.text = selected.join(' ');
253                 _selectedLessons.clear();
254             });
255         },
256     );
257 }
258
259 void _showLessonsSelector() {
260     if (_unitController.text.isEmpty) {
261         ScaffoldMessenger.of(context).showSnackBar(
262             SnackBar(
263                 content: Text('الرجاء اختيار الفصل أولاً', style: TextStylesManager.bold14),
264                 backgroundColor: ColorsManager.errorColor,
265             ),
266         );
267         return;
268     }
269
270     if (_selectedType == 'شهري' || _subjectDetails == null) return;
271
272     final selectedChapterTitle = _unitController.text;
273     final chapterModel = _subjectDetails!.chapters.cast<CurriculumChapterModel?>().firstWhere(
274         (e) => e?.title == selectedChapterTitle,
275         orElse: () => null,
276     );
277
278     if (chapterModel == null) return;
279
280     // Auto-fill semester based on selected chapter
281     _semesterController.text = chapterModel.semester;
282
283     final availableLessons = chapterModel.lessons;
284
285     if (availableLessons.isEmpty) {
286         ScaffoldMessenger.of(context).showSnackBar(
287             SnackBar(
288                 content: Text('لا توجد دروس متاحة لهذا الفصل', style: TextStylesManager.bold14),
289                 backgroundColor: ColorsManager.errorColor,
290             ),
291         );
292         return;
293     }
294
295     SelectionBottomSheet.show(
296         context: context,
297         title: 'اختر الدروس',
298         items: availableLessons.map((e) => e.title).toList(),
299         initialSelectedItems: _selectedLessons,
300         isMultiSelect: true,
301         onSelectionConfirmed: (selected) {
302             setState(() {
303                 _selectedLessons = selected;

```

```

304     });
305     },
306     );
307 }
308
309 Future<void> _selectReportDate() async {
310     DateTime initialDate = DateTime.now();
311     if (_reportDateController.text.isNotEmpty) {
312         final parsed = DateTime.tryParse(_reportDateController.text);
313         if (parsed != null) {
314             initialDate = parsed;
315         }
316     }
317
318     final DateTime? picked = await showDatePicker(
319         context: context,
320         initialDate: initialDate,
321         firstDate: DateTime(2020),
322         lastDate: DateTime(2030),
323     );
324     if (picked != null) {
325         setState(() {
326             _reportDateController.text =
327                 '${picked.year}-${picked.month.toString().padLeft(2, '0')}-${picked.day.toString().padLeft(2,
'0')}';
328         });
329     }
330 }
331
332 void _submit() {
333     if (!_formKey.currentState!.validate()) return;
334     if (_selectedType == 'اسبوعي' && _selectedLessons.isEmpty) {
335         ScaffoldMessenger.of(context).showSnackBar(
336             SnackBar(
337                 content: Text(
338                     appTranslation().get('report_lessons_required'),
339                     style: TextStylesManager.bold14,
340                 ),
341                 backgroundColor: ColorsManager.errorColor,
342             ),
343         );
344         return;
345     }
346     widget.onSubmit(
347         reportType: _selectedType,
348         grade: _gradeController.text.trim(),
349         subject: _subjectController.text.trim(),
350         unit: _unitController.text.trim(),
351         semester: _semesterController.text.trim(),
352         schoolName: _schoolController.text.trim(),
353         educationOffice: _educationOfficeController.text.trim(),
354         reportDate: _reportDateController.text.trim(),
355         selectedLessons: _selectedType == 'اسبوعي' ? _selectedLessons : [],
356     );
357 }
358
359 @override
360 void dispose() {
361     _stageController.dispose();
362     _subjectController.dispose();
363     _unitController.dispose();
364     _gradeController.dispose();
365     _semesterController.dispose();
366     _schoolController.dispose();
367     _educationOfficeController.dispose();
368     _reportDateController.dispose();
369     super.dispose();
370 }
371
372 @override
373 Widget build(BuildContext context) {
374     return Form(
375         key: _formKey,
376         child: Column(
377             crossAxisAlignment: CrossAxisAlignment.stretch,

```

```

378     children: [
379       _ReportInfoCard(teacherName: widget.teacherName),
380       verticalSpace20,
381
382       // Report Type
383       _FormLabel(label: appTranslation().get('report_type')),
384       verticalSpace8,
385       ReportTypeSelectorWidget(
386         selectedType: _selectedType,
387         onChanged: (v) => setState(() => _selectedType = v),
388       ),
389       verticalSpace16,
390
391       // Stage
392       _FormLabel(label: 'المرحلة الدراسية'),
393       verticalSpace8,
394       GestureDetector(
395         onTap: _showStageSelector,
396         child: AbsorbPointer(
397           child: PrimaryTextField(
398             controller: _stageController,
399             hint: _isLoadingSubjects ? 'اختر المرحلة' : 'جاري التحميل...',
400             prefixIcon: _isLoadingSubjects
401               ? const Padding(
402                 padding: EdgeInsets.all(12.0),
403                 child: SizedBox(width: 16, height: 16, child: CircularProgressIndicator(strokeWidth:
2)),
404               )
405               : const Icon(Icons.school_outlined),
406             validator: (v) => (v == null || v.isEmpty)
407               ? appTranslation().get('field_required')
408               : null,
409           ),
410         ),
411       ),
412       verticalSpace16,
413
414       // Grade
415       _FormLabel(label: appTranslation().get('report_grade')),
416       verticalSpace8,
417       GestureDetector(
418         onTap: _showGradeSelector,
419         child: AbsorbPointer(
420           child: PrimaryTextField(
421             controller: _gradeController,
422             hint: 'اختر الصف',
423             prefixIcon: const Icon(Icons.school_outlined),
424             validator: (v) => (v == null || v.isEmpty)
425               ? appTranslation().get('field_required')
426               : null,
427           ),
428         ),
429       ),
430       verticalSpace16,
431
432       // Subject
433       _FormLabel(label: appTranslation().get('report_subject')),
434       verticalSpace8,
435       GestureDetector(
436         onTap: _showSubjectSelector,
437         child: AbsorbPointer(
438           child: PrimaryTextField(
439             controller: _subjectController,
440             hint: 'اختر المادة',
441             prefixIcon: const Icon(Icons.book_outlined),
442             validator: (v) => (v == null || v.isEmpty)
443               ? appTranslation().get('field_required')
444               : null,
445           ),
446         ),
447       ),
448       verticalSpace16,
449
450       // Unit/Chapter
451       _FormLabel(label: appTranslation().get('report_unit')),

```



```

452     verticalSpace8,
453     GestureDetector(
454       onTap: _showUnitSelector,
455       child: AbsorbPointer(
456         child: PrimaryTextField(
457           controller: _unitController,
458           hint: _selectedType == 'شهري'
459             ? 'اختر الفصول'
460             : 'اختر الفصل',
461           prefixIcon: _isLoadingLessons
462             ? const Padding(
463               padding: EdgeInsets.all(12.0),
464               child: SizedBox(width: 16, height: 16, child: CircularProgressIndicator(strokeWidth:
2)),
465             )
466             : const Icon(Icons.layers_outlined),
467           validator: (v) => (v == null || v.isEmpty)
468             ? appTranslation().get('field_required')
469             : null,
470         ),
471       ),
472     ),
473     verticalSpace16,
474
475     // Semester
476     _FormLabel(label: appTranslation().get('report_semester')),
477     verticalSpace8,
478     PrimaryTextField(
479       controller: _semesterController,
480       hint: appTranslation().get('report_semester_hint'),
481       prefixIcon: const Icon(Icons.calendar_view_week_outlined),
482       validator: (v) => (v == null || v.isEmpty)
483         ? appTranslation().get('field_required')
484         : null,
485     ),
486     verticalSpace16,
487
488     // Report Date
489     _FormLabel(label: appTranslation().get('report_date')),
490     verticalSpace8,
491     GestureDetector(
492       onTap: _selectReportDate,
493       child: AbsorbPointer(
494         child: PrimaryTextField(
495           controller: _reportDateController,
496           readOnly: true,
497           hint: appTranslation().get('report_date_hint'),
498           prefixIcon: const Icon(Icons.date_range_outlined),
499           validator: (v) => (v == null || v.isEmpty)
500             ? appTranslation().get('field_required')
501             : null,
502         ),
503       ),
504     ),
505     verticalSpace16,
506
507     // School Name
508     _FormLabel(label: appTranslation().get('report_school')),
509     verticalSpace8,
510     PrimaryTextField(
511       controller: _schoolController,
512       hint: appTranslation().get('report_school_hint'),
513       prefixIcon: const Icon(Icons.home_work_outlined),
514       validator: (v) => (v == null || v.isEmpty)
515         ? appTranslation().get('field_required')
516         : null,
517     ),
518     verticalSpace16,
519
520     // Education Office
521     _FormLabel(label: appTranslation().get('report_education_office')),
522     verticalSpace8,
523     PrimaryTextField(
524       controller: _educationOfficeController,
525       hint: appTranslation().get('report_education_office_hint'),

```

```

526         prefixIcon: const Icon(Icons.account_balance_outlined),
527         validator: (v) => (v == null || v.isEmpty)
528             ? appTranslation().get('field_required')
529             : null,
530     ),
531     verticalSpace24,
532
533     // Lessons (weekly only)
534     if (_selectedType == 'اسبوعي') ...[
535         ReportLessonsWidget(
536             lessons: _selectedLessons,
537             onRemove: (index) {
538                 setState(() => _selectedLessons.removeAt(index));
539             },
540             onAdd: _showLessonsSelector,
541         ),
542         verticalSpace24,
543     ],
544
545     // Submit Button
546     PrimaryElevatedButton(
547         text: appTranslation().get('report_generate'),
548         isLoading: widget.isLoading,
549         onPressed: widget.isLoading ? null : _submit,
550         icon: widget.isLoading
551             ? null
552             : const Icon(Icons.auto_awesome, color: Colors.white, size: 20),
553     ),
554     verticalSpace8,
555     Text(
556         appTranslation().get('report_generate_hint'),
557         style: TextStylesManager.regular12.copyWith(
558             color: ColorsManager.secondaryText,
559         ),
560         textAlign: TextAlign.center,
561     ),
562     verticalSpace16,
563 ],
564 ),
565 );
566 }
567 }
568
569 // — Helper sub-widgets —————
570
571 class _FormLabel extends StatelessWidget {
572     final String label;
573     const _FormLabel({required this.label});
574
575     @override
576     Widget build(BuildContext context) {
577         return Text(
578             label,
579             style: TextStylesManager.bold14.copyWith(color: ColorsManager.mainText),
580         );
581     }
582 }
583
584 class _ReportInfoCard extends StatelessWidget {
585     final String teacherName;
586     const _ReportInfoCard({required this.teacherName});
587
588     @override
589     Widget build(BuildContext context) {
590         return Container(
591             padding: const EdgeInsets.all(16),
592             decoration: BoxDecoration(
593                 gradient: LinearGradient(
594                     colors: [
595                         ColorsManager.primaryColor.withValues(alpha: 0.12),
596                         ColorsManager.primaryColor.withValues(alpha: 0.04),
597                     ],
598                     begin: Alignment.topRight,
599                     end: Alignment.bottomLeft,
600                 ),

```

```

601     borderRadius: BorderRadius.circular(16),
602     border: Border.all(
603       color: ColorsManager.primaryColor.withValues(alpha: 0.3),
604     ),
605   ),
606   child: Row(
607     children: [
608       Container(
609         width: 44,
610         height: 44,
611         decoration: BoxDecoration(
612           color: ColorsManager.primaryColor.withValues(alpha: 0.15),
613           shape: BoxShape.circle,
614         ),
615         child: Icon(
616           Icons.person_rounded,
617           color: ColorsManager.primaryColor,
618           size: 22,
619         ),
620       ),
621       horizontalSpace12,
622       Expanded(
623         child: Column(
624           crossAxisAlignment: CrossAxisAlignment.start,
625           children: [
626             Text(
627               appTranslation().get('report_teacher_label'),
628               style: TextStylesManager.regular12.copyWith(
629                 color: ColorsManager.secondaryText,
630               ),
631             ),
632             verticalSpace2,
633             Text(
634               teacherName,
635               style: TextStylesManager.bold14.copyWith(
636                 color: ColorsManager.primaryColor,
637               ),
638             ),
639           ],
640         ),
641       ),
642       Icon(
643         Icons.verified_rounded,
644         color: ColorsManager.primaryColor,
645         size: 20,
646       ),
647     ],
648   ),
649 );
650 }
651 }
652

```

Lines • Dart 173

Feature / Reports

#344 – lib/features/reports/presentation/widgets/report_lessons_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6
7  class ReportLessonsWidget extends StatelessWidget {
8    final List<String> lessons;
9    final ValueChanged<int> onRemove;
10   final VoidCallback onAdd;
11
12   const ReportLessonsWidget({
13     super.key,
14     required this.lessons,
15     required this.onRemove,
16     required this.onAdd,
17   });
18

```

```

19 @override
20 Widget build(BuildContext context) {
21   return Column(
22     crossAxisAlignment: CrossAxisAlignment.stretch,
23     children: [
24       Text(
25         appTranslation().get('report_lessons'),
26         style: TextStyleManager.bold14.copyWith(color: ColorsManager.mainText),
27       ),
28       verticalSpace4,
29       Text(
30         appTranslation().get('report_lessons_hint'),
31         style: TextStyleManager.regular12.copyWith(color: ColorsManager.secondaryText),
32       ),
33       verticalSpace10,
34       // Existing lessons list
35       if (lessons.isNotEmpty) ...[
36         Container(
37           decoration: BoxDecoration(
38             color: ColorsManager.surfacePrimary,
39             borderRadius: BorderRadius.circular(16),
40             border: Border.all(color: ColorsManager.borderColor),
41           ),
42           child: ListView.separated(
43             physics: const NeverScrollableScrollPhysics(),
44             shrinkWrap: true,
45             itemCount: lessons.length,
46             separatorBuilder: (context2, index2) => Divider(
47               height: 1,
48               color: ColorsManager.borderColor,
49               indent: 16,
50               endIndent: 16,
51             ),
52             itemBuilder: (context, index) {
53               return _LessonTile(
54                 lesson: lessons[index],
55                 index: index,
56                 onRemove: () => onRemove(index),
57               );
58             },
59           ),
60         ),
61         verticalSpace10,
62       ],
63       // Select button
64       GestureDetector(
65         onTap: onAdd,
66         child: Container(
67           padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
68           decoration: BoxDecoration(
69             color: ColorsManager.surfacePrimary,
70             borderRadius: BorderRadius.circular(24),
71             border: Border.all(color: ColorsManager.borderColor),
72           ),
73           child: Row(
74             children: [
75               Icon(
76                 Icons.checklist_rtl_rounded,
77                 color: ColorsManager.primaryColor,
78                 size: 20,
79               ),
80               horizontalSpace12,
81               Expanded(
82                 child: Text(
83                   appTranslation().get('report_add_lesson_hint'),
84                   style: TextStyleManager.regular14.copyWith(
85                     color: ColorsManager.mainText,
86                   ),
87                 ),
88               ),
89               Container(
90                 width: 32,
91                 height: 32,
92                 decoration: BoxDecoration(
93                   color: ColorsManager.primaryColor,

```

```

94         shape: BoxShape.circle,
95     ),
96     child: const Icon(
97         Icons.add_rounded,
98         color: Colors.white,
99         size: 18,
100     ),
101 ),
102 ],
103 ),
104 ),
105 ),
106 ],
107 );
108 }
109 }
110
111 class _LessonTile extends StatelessWidget {
112     final String lesson;
113     final int index;
114     final VoidCallback onRemove;
115
116     const _LessonTile({
117         required this.lesson,
118         required this.index,
119         required this.onRemove,
120     });
121
122     @override
123     Widget build(BuildContext context) {
124         return Padding(
125             padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 10),
126             child: Row(
127                 children: [
128                     Container(
129                         width: 26,
130                         height: 26,
131                         decoration: BoxDecoration(
132                             color: ColorsManager.primaryColor.withValues(alpha: 0.1),
133                             shape: BoxShape.circle,
134                         ),
135                         child: Center(
136                             child: Text(
137                                 '${index + 1}',
138                                 style: TextStylesManager.bold12.copyWith(
139                                     color: ColorsManager.primaryColor,
140                                 ),
141                             ),
142                         ),
143                     ),
144                     horizontalSpace10,
145                     Expanded(
146                         child: Text(
147                             lesson,
148                             style: TextStylesManager.regular14.copyWith(
149                                 color: ColorsManager.mainText,
150                             ),
151                         ),
152                     ),
153                     GestureDetector(
154                         onTap: onRemove,
155                         child: Container(
156                             padding: const EdgeInsets.all(4),
157                             decoration: BoxDecoration(
158                                 color: ColorsManager.errorColor.withValues(alpha: 0.1),
159                                 shape: BoxShape.circle,
160                             ),
161                             child: Icon(
162                                 Icons.close_rounded,
163                                 size: 16,
164                                 color: ColorsManager.errorColor,
165                             ),
166                         ),
167                     ),
168                 ],

```

```
169     ),
170     );
171   }
172 }
173
```

Lines • 198
Dart

Feature / #345 -

Reports [lib/features/reports/presentation/widgets/report_result_section_widget.dart](#)

```
1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6
7  class ReportResultSectionWidget extends StatelessWidget {
8    final String sectionTitle;
9    final List<Map<String, dynamic>> rows;
10   final List<String> columnKeys;
11   final List<String> columnLabels;
12   final bool initiallyExpanded;
13
14   const ReportResultSectionWidget({
15     super.key,
16     required this.sectionTitle,
17     required this.rows,
18     required this.columnKeys,
19     required this.columnLabels,
20     this.initiallyExpanded = false,
21   });
22
23   @override
24   Widget build(BuildContext context) {
25     if (rows.isEmpty) return const SizedBox.shrink();
26     return Container(
27       margin: const EdgeInsets.only(bottom: 12),
28       decoration: BoxDecoration(
29         color: ColorsManager.surfacePrimary,
30         borderRadius: BorderRadius.circular(16),
31         border: Border.all(color: ColorsManager.borderColor),
32         boxShadow: [
33           BoxShadow(
34             color: Colors.black.withValues(alpha: 0.04),
35             blurRadius: 8,
36             offset: const Offset(0, 2),
37           ),
38         ],
39     ),
40     child: ClipRRect(
41       borderRadius: BorderRadius.circular(16),
42       child: Theme(
43         data: Theme.of(context).copyWith(dividerColor: Colors.transparent),
44         child: ExpansionTile(
45           initiallyExpanded: initiallyExpanded,
46           tilePadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),
47           leading: Container(
48             width: 36,
49             height: 36,
50             decoration: BoxDecoration(
51               color: ColorsManager.primaryColor.withValues(alpha: 0.1),
52               borderRadius: BorderRadius.circular(10),
53             ),
54             child: Icon(
55               Icons.table_chart_outlined,
56               color: ColorsManager.primaryColor,
57               size: 18,
58             ),
59           ),
60           title: Text(
61             sectionTitle,
62             style: TextStylesManager.bold14.copyWith(
63               color: ColorsManager.mainText,
64             ),

```

```

65     ),
66     subtitle: Text(
67       '${rows.length} ${appTranslation().get('report_rows_count')}',
68       style: TextStyleManager.regular12.copyWith(
69         color: ColorsManager.secondaryText,
70       ),
71     ),
72     children: [
73       Divider(height: 1, color: ColorsManager.borderColor),
74       SingleChildScrollView(
75         scrollDirection: Axis.horizontal,
76         padding: const EdgeInsets.all(12),
77         child: _buildTable(),
78       ),
79     ],
80   ),
81 ),
82 ),
83 );
84 }
85
86 Widget _buildTable() {
87   return Table(
88     border: TableBorder.all(
89       color: ColorsManager.borderColor,
90       borderRadius: BorderRadius.circular(8),
91     ),
92     defaultColumnWidth: const IntrinsicColumnWidth(),
93     children: [
94       // Header row
95       TableRow(
96         decoration: BoxDecoration(
97           color: ColorsManager.primaryColor,
98         ),
99         children: columnLabels
100           .map(
101             (label) => Padding(
102               padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 10),
103               child: Text(
104                 label,
105                 style: TextStyleManager.bold12.copyWith(
106                   color: Colors.white,
107                 ),
108                 textAlign: TextAlign.center,
109               ),
110             ),
111           )
112           .toList(),
113     ),
114     // Data rows
115     ...rows.asMap().entries.map((entry) {
116       final index = entry.key;
117       final row = entry.value;
118       final isEven = index % 2 == 0;
119       return TableRow(
120         decoration: BoxDecoration(
121           color: isEven
122             ? ColorsManager.background
123             : ColorsManager.surfacePrimary,
124         ),
125         children: columnKeys
126           .map(
127             (key) => Padding(
128               padding: const EdgeInsets.symmetric(
129                 horizontal: 12, vertical: 10),
130               child: Text(
131                 (row[key] ?? '').toString(),
132                 style: TextStyleManager.regular12.copyWith(
133                   color: ColorsManager.mainText,
134                 ),
135                 textDirection: TextDirection.rtl,
136               ),
137             ),
138           )
139           .toList(),

```

```

140         );
141     }},
142     ],
143 );
144 }
145 }
146
147 // -----
148 // Print / PDF action bar
149 // -----
150
151 class ReportPrintBarWidget extends StatelessWidget {
152     final VoidCallback onPrint;
153
154     const ReportPrintBarWidget({
155         super.key,
156         required this.onPrint,
157     });
158
159     @override
160     Widget build(BuildContext context) {
161         return Container(
162             padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 14),
163             decoration: BoxDecoration(
164                 gradient: LinearGradient(
165                     colors: [
166                         ColorsManager.primaryColor,
167                         ColorsManager.primaryColor.withValues(alpha: 0.85),
168                     ],
169                     begin: Alignment.topRight,
170                     end: Alignment.bottomLeft,
171                 ),
172                 borderRadius: BorderRadius.circular(20),
173                 boxShadow: [
174                     BoxShadow(
175                         color: ColorsManager.primaryColor.withValues(alpha: 0.35),
176                         blurRadius: 14,
177                         offset: const Offset(0, 5),
178                     ),
179                 ],
180             ),
181             child: GestureDetector(
182                 onTap: onPrint,
183                 child: Row(
184                     mainAxisAlignment: MainAxisAlignment.center,
185                     children: [
186                         const Icon(Icons.print_rounded, color: Colors.white, size: 22),
187                         horizontalSpace10,
188                         Text(
189                             appTranslation().get('report_print'),
190                             style: TextStylesManager.bold16.copyWith(color: Colors.white),
191                         ),
192                     ],
193                 ),
194             ),
195         );
196     }
197 }
198

```

Lines • 97
Dart

Feature / #346 –

Reports [lib/features/reports/presentation/widgets/report_type_selector_widget.dart](#)

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6
7  class ReportTypeSelectorWidget extends StatelessWidget {
8      final String selectedType;
9      final ValueChanged<String> onChanged;
10

```



```

11  const ReportTypeSelectorWidget({
12      super.key,
13      required this.selectedType,
14      required this.onChanged,
15  });
16
17  @override
18  Widget build(BuildContext context) {
19      return Row(
20          children: [
21              _ReportTypeChip(
22                  label: appTranslation().get('report_weekly'),
23                  value: 'اسبوعي',
24                  selectedValue: selectedType,
25                  onTap: onChanged,
26              ),
27              horizontalSpace12,
28              _ReportTypeChip(
29                  label: appTranslation().get('report_monthly'),
30                  value: 'شهري',
31                  selectedValue: selectedType,
32                  onTap: onChanged,
33              ),
34          ],
35      );
36  }
37
38
39  class _ReportTypeChip extends StatelessWidget {
40      final String label;
41      final String value;
42      final String selectedValue;
43      final ValueChanged<String> onTap;
44
45      const _ReportTypeChip({
46          required this.label,
47          required this.value,
48          required this.selectedValue,
49          required this.onTap,
50      });
51
52      @override
53      Widget build(BuildContext context) {
54          final bool isSelected = value == selectedValue;
55          return GestureDetector(
56              onTap: () => onTap(value),
57              child: AnimatedContainer(
58                  duration: const Duration(milliseconds: 200),
59                  padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 12),
60                  decoration: BoxDecoration(
61                      color: isSelected ? ColorsManager.primaryColor : ColorsManager.surfacePrimary,
62                      borderRadius: BorderRadius.circular(24),
63                      border: Border.all(
64                          color: isSelected ? ColorsManager.primaryColor : ColorsManager.borderColor,
65                      ),
66                      boxShadow: isSelected
67                          ? [
68                              BoxShadow(
69                                  color: ColorsManager.primaryColor.withValues(alpha: 0.2),
70                                  blurRadius: 8,
71                                  offset: const Offset(0, 3),
72                              ),
73                          ]
74                          : [],
75                  ),
76                  child: Row(
77                      mainAxisAlignment: MainAxisAlignment.min,
78                      children: [
79                          Icon(
80                              isSelected ? Icons.check_circle_rounded : Icons.radio_button_unchecked,
81                              size: 16,
82                              color: isSelected ? ColorsManager.white : ColorsManager.placeholder,
83                          ),
84                          horizontalSpace6,
85                          Text(

```

```

86         label,
87         style: TextStyleManager.bold14.copyWith(
88             color: isSelected ? ColorsManager.white : ColorsManager.mainText,
89         ),
90     ),
91 ],
92 ),
93 ),
94 );
95 }
96 }
97

```

Lines • Dart 186

Feature / Reports

#347 – lib/features/reports/presentation/widgets/saved_report_card_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6  import 'package:moean/features/reports/data/saved_report_model.dart';
7
8  class SavedReportCardWidget extends StatelessWidget {
9      final SavedReportModel report;
10     final VoidCallback onOpen;
11
12     const SavedReportCardWidget({
13         super.key,
14         required this.report,
15         required this.onOpen,
16     });
17
18     String _formatDate(String dateStr) {
19         if (dateStr.isEmpty) return '';
20         try {
21             final dateTime = DateTime.parse(dateStr);
22             final months = [
23                 'يناير', 'فبراير', 'مارس', 'أبريل', 'مايو', 'يونيو',
24                 'يوليو', 'أغسطس', 'سبتمبر', 'أكتوبر', 'نوفمبر', 'ديسمبر'
25             ];
26             return '${dateTime.day} ${months[dateTime.month - 1]} ${dateTime.year}';
27         } catch (_) {
28             return dateStr.split('T').first;
29         }
30     }
31
32     @override
33     Widget build(BuildContext context) {
34         final formattedDate = _formatDate(report.createdAt);
35
36         return Container(
37             padding: const EdgeInsets.all(16),
38             decoration: BoxDecoration(
39                 color: ColorsManager.surfacePrimary,
40                 borderRadius: BorderRadius.circular(20),
41                 border: Border.all(
42                     color: ColorsManager.borderColor.withValues(alpha: 0.6),
43                 ),
44             ),
45             child: Column(
46                 crossAxisAlignment: CrossAxisAlignment.start,
47                 children: [
48                     // Top Row: File icon & Report type badge
49                     Row(
50                         mainAxisAlignment: MainAxisAlignment.spaceBetween,
51                         children: [
52                             Container(
53                                 padding: const EdgeInsets.all(12),
54                                 decoration: BoxDecoration(
55                                     color: const Color(0xFFFFF7E6),
56                                     borderRadius: BorderRadius.circular(16),
57                             ),
58                             child: const Icon(

```

```

59 Icons.description_outlined,
60 color: Color(0xFFD97706),
61 size: 24,
62 ),
63 ),
64 Container(
65 padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 6),
66 decoration: BoxDecoration(
67 color: ColorsManager.primaryColor.withValues(alpha: 0.1),
68 borderRadius: BorderRadius.circular(20),
69 ),
70 child: Text(
71 report.reportType,
72 style: TextStyleManager.bold12.copyWith(
73 color: ColorsManager.primaryColor,
74 ),
75 ),
76 ),
77 ],
78 ),
79 verticalSpace16,
80
81 // Report Title
82 Text(
83 report.displaySubject,
84 maxLines: 2,
85 overflow: TextOverflow.ellipsis,
86 style: TextStyleManager.bold16.copyWith(
87 color: ColorsManager.mainText,
88 height: 1.3,
89 ),
90 ),
91 verticalSpace6,
92
93 // Report Subtitle (Grade)
94 Text(
95 report.grade,
96 maxLines: 1,
97 overflow: TextOverflow.ellipsis,
98 style: TextStyleManager.regular14.copyWith(
99 color: ColorsManager.secondaryText,
100 ),
101 ),
102 verticalSpace16,
103
104 // Meta Row (Lessons count & Date)
105 Row(
106 children: [
107 Container(
108 padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
109 decoration: BoxDecoration(
110 color: ColorsManager.background,
111 borderRadius: BorderRadius.circular(8),
112 ),
113 child: Text(
114 '${report.selectedLessons.length} درس',
115 style: TextStyleManager.bold12.copyWith(
116 color: ColorsManager.secondaryText,
117 ),
118 ),
119 ),
120 horizontalSpace8,
121 if (formattedDate.isNotEmpty)
122 Container(
123 padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
124 decoration: BoxDecoration(
125 color: ColorsManager.background,
126 borderRadius: BorderRadius.circular(8),
127 ),
128 child: Row(
129 children: [
130 Icon(
131 Icons.calendar_month_outlined,
132 size: 14,
133 color: ColorsManager.secondaryText,

```

```

134         ),
135         horizontalSpace4,
136         Text(
137             formattedDate,
138             style: TextStylesManager.bold12.copyWith(
139                 color: ColorsManager.secondaryText,
140             ),
141         ),
142     ],
143 ),
144 ),
145 ],
146 ),
147 verticalSpace16,
148
149 // Open Report Action Button
150 SizedBox(
151     width: double.infinity,
152     height: 48,
153     child: ElevatedButton(
154         onPressed: onOpen,
155         style: ElevatedButton.styleFrom(
156             backgroundColor: ColorsManager.primaryColor,
157             elevation: 0,
158             shape: RoundedRectangleBorder(
159                 borderRadius: BorderRadius.circular(16),
160             ),
161         ),
162         child: Row(
163             mainAxisAlignment: MainAxisAlignment.center,
164             children: [
165                 const Icon(
166                     Icons.visibility_outlined,
167                     color: Colors.white,
168                     size: 20,
169                 ),
170                 horizontalSpace8,
171                 Text(
172                     appTranslation().get('open_report_btn'),
173                     style: TextStylesManager.bold14.copyWith(
174                         color: Colors.white,
175                     ),
176                 ),
177             ],
178         ),
179     ),
180 ),
181 ],
182 ),
183 );
184 }
185 }
186

```

Lines • 77
Dart

Feature / #348 -

Reports lib/features/reports/presentation/widgets/saved_reports_filter_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5
6  class SavedReportsFilterWidget extends StatelessWidget {
7      final String selectedFilter;
8      final ValueChanged<String> onFilterChanged;
9
10     const SavedReportsFilterWidget({
11         super.key,
12         required this.selectedFilter,
13         required this.onFilterChanged,
14     });
15
16     @override

```

```

17 Widget build(BuildContext context) {
18   final filters = [
19     {'label': appTranslation().get('filter_all_reports'), 'value': 'الكل'},
20     {'label': appTranslation().get('report_weekly'), 'value': 'أسبوعي'},
21     {'label': appTranslation().get('report_monthly'), 'value': 'شهري'},
22   ];
23
24   return Row(
25     mainAxisAlignment: MainAxisAlignment.end,
26     children: filters.map((filter) {
27       final isSelected = selectedFilter == filter['value'];
28       return Padding(
29         padding: const EdgeInsets.only(left: 8.0),
30         child: _FilterChip(
31           label: filter['label']!,
32           isSelected: isSelected,
33           onTap: () => onFilterChanged(filter['value']!),
34         ),
35       );
36     }).toList(),
37   );
38 }
39
40
41 class _FilterChip extends StatelessWidget {
42   final String label;
43   final bool isSelected;
44   final VoidCallback onTap;
45
46   const _FilterChip({
47     required this.label,
48     required this.isSelected,
49     required this.onTap,
50   });
51
52   @override
53   Widget build(BuildContext context) {
54     return InkWell(
55       onTap: onTap,
56       borderRadius: BorderRadius.circular(20),
57       child: AnimatedContainer(
58         duration: const Duration(milliseconds: 200),
59         padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 10),
60         decoration: BoxDecoration(
61           color: isSelected ? ColorsManager.primaryColor : ColorsManager.surfacePrimary,
62           borderRadius: BorderRadius.circular(25),
63           border: Border.all(
64             color: isSelected ? ColorsManager.primaryColor : ColorsManager.borderColor.withValues(alpha: 0.
65             ),
66           ),
67           child: Text(
68             label,
69             style: TextStylesManager.bold14.copyWith(
70               color: isSelected ? Colors.white : ColorsManager.mainText,
71             ),
72           ),
73         ),
74       );
75   }
76 }
77

```

Lines • 116
Dart

Feature / #349 -

Reports [lib/features/reports/presentation/widgets/saved_reports_header_widget.dart](#)

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/spacing.dart';
6 import 'package:moean/core/utils/extensions/context_extension.dart';
7

```

```

8 class SavedReportsHeaderWidget extends StatelessWidget {
9   final VoidCallback onRefresh;
10
11   const SavedReportsHeaderWidget({
12     super.key,
13     required this.onRefresh,
14   });
15
16   @override
17   Widget build(BuildContext context) {
18     return Column(
19       crossAxisAlignment: CrossAxisAlignment.start,
20       children: [
21         Row(
22           mainAxisAlignment: MainAxisAlignment.spaceBetween,
23           children: [
24             // Top action buttons (Close X and Refresh)
25             Row(
26               children: [
27                 _HeaderIconButton(
28                   icon: Icons.close_rounded,
29                   onTap: () => context.pop(),
30                 ),
31                 horizontalSpace8,
32                 _HeaderIconButton(
33                   icon: Icons.refresh_rounded,
34                   onTap: onRefresh,
35                 ),
36               ],
37             ),
38             // Saved Reports Badge
39             Container(
40               padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
41               decoration: BoxDecoration(
42                 color: ColorsManager.primaryColor.withValues(alpha: 0.1),
43                 borderRadius: BorderRadius.circular(20),
44               ),
45               child: Row(
46                 mainAxisAlignment: MainAxisAlignment.min,
47                 children: [
48                   Text(
49                     appTranslation().get('saved_reports_title'),
50                     style: TextStylesManager.bold12.copyWith(
51                       color: ColorsManager.primaryColor,
52                     ),
53                   ),
54                   horizontalSpace6,
55                   Icon(
56                     Icons.inventory_2_outlined,
57                     size: 16,
58                     color: ColorsManager.primaryColor,
59                   ),
60                 ],
61               ),
62             ),
63           ],
64         ),
65         verticalSpace16,
66         Text(
67           appTranslation().get('saved_reports_subtitle'),
68           style: TextStylesManager.bold24.copyWith(
69             color: ColorsManager.mainText,
70           ),
71         ),
72         verticalSpace6,
73         Text(
74           appTranslation().get('saved_reports_desc'),
75           style: TextStylesManager.regular14.copyWith(
76             color: ColorsManager.secondaryText,
77           ),
78         ),
79       ],
80     );
81   }
82 }

```

```

83
84 class _HeaderIconButton extends StatelessWidget {
85   final IconData icon;
86   final VoidCallback onTap;
87
88   const _HeaderIconButton({
89     required this.icon,
90     required this.onTap,
91   });
92
93   @override
94   Widget build(BuildContext context) {
95     return InkWell(
96       onTap: onTap,
97       borderRadius: BorderRadius.circular(20),
98       child: Container(
99         padding: const EdgeInsets.all(8),
100        decoration: BoxDecoration(
101          color: ColorsManager.surfacePrimary,
102          shape: BoxShape.circle,
103          border: Border.all(
104            color: ColorsManager.borderColor.withValues(alpha: 0.5),
105          ),
106        ),
107        child: Icon(
108          icon,
109          size: 20,
110          color: ColorsManager.mainText,
111        ),
112      ),
113    );
114  }
115 }
116

```

Lines • Dart 263

Feature / Reports

#350 – lib/features/reports/presentation/widgets/selection_bottom_sheet.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7  import 'package:moean/core/utils/extensions/context_extension.dart';
8
9  class SelectionBottomSheet extends StatefulWidget {
10    final String title;
11    final List<String> items;
12    final List<String> initialSelectedItems;
13    final bool isMultiSelect;
14    final ValueChanged<List<String>> onSelectionConfirmed;
15
16    const SelectionBottomSheet({
17      super.key,
18      required this.title,
19      required this.items,
20      this.initialSelectedItems = const [],
21      this.isMultiSelect = true,
22      required this.onSelectionConfirmed,
23    });
24
25    static Future<void> show({
26      required BuildContext context,
27      required String title,
28      required List<String> items,
29      List<String> initialSelectedItems = const [],
30      bool isMultiSelect = true,
31      required ValueChanged<List<String>> onSelectionConfirmed,
32    }) {
33      return showModalBottomSheet(
34        context: context,
35        isScrollControlled: true,
36        backgroundColor: Colors.transparent,

```

```

37     builder: (context) => SelectionBottomSheet(
38         title: title,
39         items: items,
40         initialSelectedItems: initialSelectedItems,
41         isMultiSelect: isMultiSelect,
42         onSelectionConfirmed: onSelectionConfirmed,
43     ),
44 );
45 }
46
47 @override
48 State<SelectionBottomSheet> createState() => _SelectionBottomSheetState();
49 }
50
51 class _SelectionBottomSheetState extends State<SelectionBottomSheet> {
52     late List<String> _selectedItems;
53     List<String> _filteredItems = [];
54     final TextEditingController _searchController = TextEditingController();
55
56     @override
57     void initState() {
58         super.initState();
59         _selectedItems = List.from(widget.initialSelectedItems);
60         _filteredItems = List.from(widget.items);
61         _searchController.addListener(_onSearchChanged);
62     }
63
64     @override
65     void dispose() {
66         _searchController.dispose();
67         super.dispose();
68     }
69
70     void _onSearchChanged() {
71         final query = _searchController.text.trim().toLowerCase();
72         setState(() {
73             if (query.isEmpty) {
74                 _filteredItems = List.from(widget.items);
75             } else {
76                 _filteredItems = widget.items
77                     .where((item) => item.toLowerCase().contains(query))
78                     .toList();
79             }
80         });
81     }
82
83     void _toggleItem(String item) {
84         setState(() {
85             if (widget.isMultiSelect) {
86                 if (_selectedItems.contains(item)) {
87                     _selectedItems.remove(item);
88                 } else {
89                     _selectedItems.add(item);
90                 }
91             } else {
92                 _selectedItems = [item];
93             }
94         });
95     }
96
97     @override
98     Widget build(BuildContext context) {
99         return Directionality(
100             textDirection: TextDirection.rtl,
101             child: SafeArea(
102                 child: Container(
103                     height: MediaQuery.of(context).size.height * 0.7,
104                     decoration: BoxDecoration(
105                         color: ColorsManager.background,
106                         borderRadius: const BorderRadius.vertical(top: Radius.circular(24)),
107                     ),
108                     child: Column(
109                         children: [
110                             // Handle
111                             Center(

```



```

112         child: Container(
113             margin: const EdgeInsets.only(top: 12, bottom: 8),
114             width: 40,
115             height: 4,
116             decoration: BoxDecoration(
117                 color: ColorsManager.borderColor,
118                 borderRadius: BorderRadius.circular(4),
119             ),
120         ),
121     ),
122     // Header
123     Padding(
124         padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 8),
125         child: Row(
126             children: [
127                 Expanded(
128                     child: Column(
129                         crossAxisAlignment: CrossAxisAlignment.start,
130                         children: [
131                             Text(
132                                 widget.title,
133                                 style: TextStylesManager.bold16.copyWith(
134                                     color: ColorsManager.mainText,
135                                 ),
136                             ),
137                             if (widget.isMultiSelect) ...[
138                                 verticalSpace2,
139                                 Text(
140                                     'تم اختيار ${_selectedItems.length} من ${widget.items.length}',
141                                     style: TextStylesManager.regular12.copyWith(
142                                         color: ColorsManager.primaryColor,
143                                     ),
144                                 ),
145                             ],
146                         ],
147                     ),
148                 ),
149                 IconButton(
150                     onPressed: () => context.pop(),
151                     icon: Icon(
152                         Icons.close_rounded,
153                         color: ColorsManager.secondaryText,
154                     ),
155                 ),
156             ],
157         ),
158     ),
159     const Divider(height: 1),
160     // Search Box
161     Padding(
162         padding: const EdgeInsets.all(16),
163         child: Container(
164             decoration: BoxDecoration(
165                 color: ColorsManager.surfacePrimary,
166                 borderRadius: BorderRadius.circular(12),
167                 border: Border.all(color: ColorsManager.borderColor),
168             ),
169             child: TextField(
170                 controller: _searchController,
171                 decoration: InputDecoration(
172                     hintText: appTranslation().get('search_hint'),
173                     hintStyle: TextStylesManager.regular14.copyWith(
174                         color: ColorsManager.secondaryText,
175                     ),
176                     prefixIcon: Icon(
177                         Icons.search,
178                         color: ColorsManager.secondaryText,
179                     ),
180                     border: InputBorder.none,
181                     contentPadding: const EdgeInsets.symmetric(vertical: 12),
182                 ),
183             ),
184         ),
185     ),
186     // List

```

```

187 Expanded(
188   child: ListView.separated(
189     padding: const EdgeInsets.symmetric(horizontal: 16),
190     itemCount: _filteredItems.length,
191     separatorBuilder: (context2, index2) => verticalSpace8,
192     itemBuilder: (context, index) {
193       final item = _filteredItems[index];
194       final isSelected = _selectedItems.contains(item);
195       return GestureDetector(
196         onTap: () => _toggleItem(item),
197         child: Container(
198           padding: const EdgeInsets.symmetric(
199             horizontal: 16,
200             vertical: 14,
201           ),
202           decoration: BoxDecoration(
203             color: isSelected
204               ? ColorsManager.primaryColor.withValues(alpha: 0.1)
205               : ColorsManager.surfacePrimary,
206             borderRadius: BorderRadius.circular(12),
207             border: Border.all(
208               color: isSelected
209                 ? ColorsManager.primaryColor
210                 : ColorsManager.borderColor,
211             ),
212           ),
213           child: Row(
214             children: [
215               Icon(
216                 widget.isMultiSelect
217                   ? (isSelected
218                       ? Icons.check_box_rounded
219                       : Icons.check_box_outline_blank_rounded)
220                   : (isSelected
221                       ? Icons.radio_button_checked
222                       : Icons.radio_button_unchecked),
223                 color: isSelected
224                   ? ColorsManager.primaryColor
225                   : ColorsManager.placeholder,
226               ),
227               horizontalSpace12,
228               Expanded(
229                 child: Text(
230                   item,
231                   style: isSelected
232                     ? TextStylesManager.bold14.copyWith(
233                       color: ColorsManager.primaryColor)
234                     : TextStylesManager.regular14.copyWith(
235                       color: ColorsManager.mainText),
236                 ),
237               ),
238             ],
239           ),
240         ),
241       );
242     },
243   ),
244 ),
245 // Bottom Action
246 Padding(
247   padding: const EdgeInsets.all(20),
248   child: PrimaryElevatedButton(
249     text: 'تأكيد',
250     onPressed: () {
251       widget.onSelectionConfirmed(_selectedItems);
252       context.pop();
253     },
254   ),
255 ),
256 ],
257 ),
258 ),
259 ),
260 );
261 }

```

```
262 }
263
```

Lines • Dart 68

Feature / Root

#351 – lib/features/root/presentation/screen/root_screen.dart

```
1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/features/home/presentation/screen/home_screen.dart';
4 import 'package:moean/features/root/presentation/widgets/custom_bottom_nav_bar.dart';
5
6 class RootScreen extends StatefulWidget {
7   final int initialIndex;
8   const RootScreen({super.key, this.initialIndex = 0});
9
10  @override
11  State<RootScreen> createState() => _RootScreenState();
12 }
13
14 class _RootScreenState extends State<RootScreen> {
15   late int _currentIndex;
16
17   @override
18   void initState() {
19     super.initState();
20     _currentIndex = widget.initialIndex;
21   }
22
23   static const List<Widget> _screens = [
24     HomeScreen(),
25     _PlaceholderScreen(index: 1),
26     _PlaceholderScreen(index: 2),
27     _PlaceholderScreen(index: 3),
28   ];
29
30   @override
31   Widget build(BuildContext context) {
32     return Scaffold(
33       extendBody: true,
34       backgroundColor: ColorsManager.background,
35       body: IndexedStack(
36         index: _currentIndex,
37         children: _screens,
38       ),
39       bottomNavigationBar: CustomBottomNavBar(
40         currentIndex: _currentIndex,
41         onTap: (index) {
42           setState(() {
43             _currentIndex = index;
44           });
45         },
46       ),
47     );
48   }
49 }
50
51 class _PlaceholderScreen extends StatelessWidget {
52   final int index;
53   const _PlaceholderScreen({required this.index});
54
55   @override
56   Widget build(BuildContext context) {
57     return Scaffold(
58       backgroundColor: ColorsManager.background,
59       body: Center(
60         child: Text(
61           'Tab $index',
62           style: const TextStyle(fontSize: 20),
63         ),
64       ),
65     );
66   }
67 }
68
```

```
1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5
6 class CustomBottomNavBar extends StatelessWidget {
7   final int currentIndex;
8   final Function(int) onTap;
9
10  const CustomBottomNavBar({
11    super.key,
12    required this.currentIndex,
13    required this.onTap,
14  });
15
16  static const List<Map<String, dynamic>> _navItems = [
17    {'labelKey': 'home_nav', 'icon': Icons.home_rounded},
18    {'labelKey': 'library_nav', 'icon': Icons.auto_stories_rounded},
19    {'labelKey': 'tools_nav', 'icon': Icons.construction_rounded},
20    {'labelKey': 'my_account_nav', 'icon': Icons.person_rounded},
21  ];
22
23  @override
24  Widget build(BuildContext context) {
25    final bottomPadding = MediaQuery.of(context).padding.bottom;
26
27    return SizedBox(
28      height: 90 + bottomPadding,
29      child: Stack(
30        clipBehavior: Clip.none,
31        alignment: Alignment.bottomCenter,
32        children: [
33          Positioned(
34            left: 0,
35            right: 0,
36            bottom: 0,
37            child: Container(
38              height: 70 + bottomPadding,
39              decoration: BoxDecoration(
40                color: ColorsManager.primaryColor,
41                borderRadius: const BorderRadius.vertical(
42                  top: Radius.circular(30),
43                ),
44                boxShadow: [
45                  BoxShadow(
46                    color: ColorsManager.primaryColor.withValues(alpha: 0.25),
47                    blurRadius: 20,
48                    offset: const Offset(0, -4),
49                  ),
50                ],
51              ),
52            ),
53          ),
54          Positioned(
55            left: 0,
56            right: 0,
57            bottom: bottomPadding,
58            height: 90,
59            child: Row(
60              mainAxisAlignment: MainAxisAlignment.spaceEvenly,
61              crossAxisAlignment: CrossAxisAlignment.end,
62              children: List.generate(
63                _navItems.length,
64                (index) => _NavItem(
65                  index: index,
66                  currentIndex: currentIndex,
67                  icon: _navItems[index]['icon'] as IconData,
68                  label: appTranslation().get(
69                    _navItems[index]['labelKey'] as String,
70                  ),
71                  onTap: onTap,
72                  isCenter: index == 2,
```

```

73         ),
74     ),
75 ),
76 ),
77 ],
78 ),
79 );
80 }
81 }
82
83 class _NavItem extends StatelessWidget {
84   final int index;
85   final int currentIndex;
86   final IconData icon;
87   final String label;
88   final Function(int) onTap;
89   final bool isCenter;
90
91   const _NavItem({
92     required this.index,
93     required this.currentIndex,
94     required this.icon,
95     required this.label,
96     required this.onTap,
97     required this.isCenter,
98   });
99
100   @override
101   Widget build(BuildContext context) {
102     final bool isActive = currentIndex == index;
103     final Color itemColor =
104       isActive ? ColorsManager.white : ColorsManager.secondaryColor;
105
106     if (isCenter) {
107       return Expanded(
108         child: GestureDetector(
109           onTap: () => onTap(index),
110           behavior: HitTestBehavior.opaque,
111           child: SizedBox(
112             height: 90,
113             child: Stack(
114               alignment: Alignment.bottomCenter,
115               clipBehavior: Clip.none,
116               children: [
117                 Positioned(
118                   left: 0,
119                   right: 0,
120                   bottom: 12,
121                   child: Align(
122                     alignment: Alignment.bottomCenter,
123                     child: FittedBox(
124                       fit: BoxFit.scaleDown,
125                       child: Text(
126                         label,
127                         style: TextStyleManager.bold10.copyWith(
128                           color: itemColor,
129                         ),
130                       ),
131                     ),
132                   ),
133                 ),
134                 Positioned(
135                   left: 0,
136                   right: 0,
137                   bottom: 0,
138                   top: 0,
139                   child: Align(
140                     alignment: Alignment.bottomCenter,
141                     child: Padding(
142                       padding: const EdgeInsets.only(bottom: 40),
143                       child: Container(
144                         width: 56,
145                         height: 56,
146                         alignment: Alignment.center,
147                         decoration: BoxDecoration(

```

```

148         color: ColorsManager.secondaryColor,
149         shape: BoxShape.circle,
150         border: Border.all(
151             color: ColorsManager.backgroundColorLight,
152             width: 3,
153         ),
154         boxShadow: [
155             BoxShadow(
156                 color: ColorsManager.secondaryColor
157                     .withValues(alpha: 0.4),
158                 blurRadius: 12,
159                 offset: const Offset(0, 4),
160             ),
161         ],
162     ),
163     child: Icon(icon, color: Colors.white, size: 26),
164 ),
165 ),
166 ),
167 ),
168 ],
169 ),
170 ),
171 ),
172 );
173 }
174
175 return Expanded(
176     child: GestureDetector(
177         onTap: () => onTap(index),
178         behavior: HitTestBehavior.opaque,
179         child: SizedBox(
180             height: 90,
181             child: Stack(
182                 alignment: Alignment.bottomCenter,
183                 clipBehavior: Clip.none,
184                 children: [
185                     Positioned(
186                         left: 0,
187                         right: 0,
188                         bottom: 12,
189                         child: Align(
190                             alignment: Alignment.bottomCenter,
191                             child: FittedBox(
192                                 fit: BoxFit.scaleDown,
193                                 child: Text(
194                                     label,
195                                     style: TextStylesManager.bold10.copyWith(
196                                         color: itemColor,
197                                     ),
198                                 ),
199                             ),
200                         ),
201                     Positioned(
202                         left: 0,
203                         right: 0,
204                         bottom: 34,
205                         child: Align(
206                             alignment: Alignment.bottomCenter,
207                             child: AnimatedSwitcher(
208                                 duration: const Duration(milliseconds: 200),
209                                 child: Icon(
210                                     icon,
211                                     key: ValueKey<bool>(isActive),
212                                     color: itemColor,
213                                     size: 24,
214                                 ),
215                             ),
216                         ),
217                     ),
218                 ],
219             ),
220         ),
221     ),
222 );

```

```
223     );
224   }
225 }
226
```

Lines • Dart 139

Feature / Schedule

#353 – lib/features/schedule/data/models/schedule_models.dart

```
1  enum ClassStatus {
2    activity,
3    notPrepared,
4    prepared,
5    waiting,
6  }
7
8  class DayModel {
9    final String nameKey;
10   final String date;
11   final int dayOfWeek;
12
13   const DayModel({
14     required this.nameKey,
15     required this.date,
16     required this.dayOfWeek,
17   });
18 }
19
20 class ClassModel {
21   final String id;
22   final int periodNumber;
23   final String? lessonTitle;
24   final String classroomId;
25   final ClassStatus status;
26   final int dayOfWeek;
27   final String realSchoolId;
28   final String timeTaleId;
29   final int subjectId;
30   final int lessonId;
31   final String time;
32   final String date;
33   final String encryptedToken;
34   final String subjectName;
35   final String classroomName;
36   final String madrasatiStatus;
37
38   const ClassModel({
39     required this.id,
40     required this.periodNumber,
41     this.lessonTitle,
42     required this.classroomId,
43     required this.status,
44     required this.dayOfWeek,
45     this.realSchoolId = '',
46     this.timeTaleId = '',
47     this.subjectId = 0,
48     this.lessonId = 0,
49     this.time = '',
50     this.date = '',
51     this.encryptedToken = '',
52     this.subjectName = '',
53     this.classroomName = '',
54     this.madrasatiStatus = '',
55   });
56
57   factory ClassModel.fromJson(Map<String, dynamic> json) {
58     final lessonTitle = json['subject_name'] as String? ?? json['lesson_title'] as String?;
59
60     ClassStatus parsedStatus = ClassStatus.notPrepared;
61     final statusStr = json['status'] as String?;
62
63     if (statusStr == 'تم إعداد الدرس') {
64       parsedStatus = ClassStatus.prepared;
65     } else if (statusStr == 'نشاط') {
66       parsedStatus = ClassStatus.activity;
```

```

67     } else if (statusStr == 'بإعداد الدرس' || statusStr == 'بانتظار إعداد الدرس') {
68         parsedStatus = ClassStatus.waiting;
69     } else if (json.containsKey('is_prepared')) {
70         parsedStatus = (json['is_prepared'] == true)
71             ? ClassStatus.prepared
72             : ClassStatus.notPrepared;
73     } else if (statusStr != null) {
74         // Fallback for any other unexpected string
75         parsedStatus = ClassStatus.notPrepared;
76     }
77
78     int pNum = json['period_number'] as int? ?? 0;
79     if (pNum == 0) {
80         final periodStr = json['period'] as String? ?? '';
81
82         if (periodStr.contains('الأولى')) {
83             pNum = 1;
84         } else if (periodStr.contains('الثانية')) {
85             pNum = 2;
86         } else if (periodStr.contains('الثالثة')) {
87             pNum = 3;
88         } else if (periodStr.contains('الرابعة')) {
89             pNum = 4;
90         } else if (periodStr.contains('الخامسة')) {
91             pNum = 5;
92         } else if (periodStr.contains('السادسة')) {
93             pNum = 6;
94         } else if (periodStr.contains('السابعة')) {
95             pNum = 7;
96         }
97     }
98
99     int dNum = json['day_of_week'] as int? ?? 1;
100     final dayStr = json['day'] as String?;
101     if (dayStr != null) {
102         if (dayStr == 'الأحد') {
103             dNum = 1;
104         } else if (dayStr == 'الاثنين') {
105             dNum = 2;
106         } else if (dayStr == 'الثلاثاء') {
107             dNum = 3;
108         } else if (dayStr == 'الأربعاء') {
109             dNum = 4;
110         } else if (dayStr == 'الخميس') {
111             dNum = 5;
112         }
113     }
114
115     return ClassModel(
116         id: json['lesson_id']?.toString() ?? json['id']?.toString() ?? '',
117         periodNumber: pNum,
118         lessonTitle: lessonTitle,
119         classroomId: json['grade'] as String? ?? json['classroom_id'] as String? ?? '',
120         status: parsedStatus,
121         dayOfWeek: dNum,
122         realSchoolId: json['real_school_id']?.toString() ?? '',
123         timeTableId: json['time_table_id']?.toString() ?? '',
124         subjectId: int.tryParse(json['subject_id']?.toString() ?? '') ?? 0,
125         lessonId: int.tryParse(json['lesson_id']?.toString() ?? '') ??
126             int.tryParse(json['lesson_madrasati_id']?.toString() ?? '') ?? 0,
127         time: json['time'] as String? ?? '',
128         date: json['date'] as String? ?? '',
129         encryptedToken: json['encrypted_token'] as String? ?? '',
130         subjectName: json['subject_name'] as String? ?? '',
131         classroomName: json['classroom_name'] as String? ?? '',
132         madrasatiStatus: json['madrasati_status'] as String? ?? '',
133     );
134 }
135
136 /// Returns a translation key like 'class_1', 'class_2', etc.
137 String get periodKey => 'class_$periodNumber';
138 }
139

```



```

1  import 'dart:async';
2  import 'package:flutter/material.dart';
3  import 'package:flutter_bloc/flutter_bloc.dart';
4  import 'package:moean/core/network/remote/api_service.dart';
5  import 'package:moean/features/schedule/data/models/schedule_models.dart';
6  import 'package:moean/features/schedule/presentation/cubit/schedule_state.dart';
7  import 'package:moean/core/di/injections.dart';
8  import 'package:moean/core/services/madrasati_session_service.dart';
9
10 class ScheduleCubit extends Cubit<ScheduleState> {
11   StreamSubscription? _sessionActiveSubscription;
12
13   ScheduleCubit() : super(ScheduleInitial()) {
14     _sessionActiveSubscription = sl<MadrasatiSessionService>().onSessionActive.listen((_) {
15       // Re-fetch schedule when session is refreshed
16       if (state is ScheduleError) {
17         getSchedule();
18       }
19     });
20   }
21
22   static ScheduleCubit get(BuildContext context) =>
23     BlocProvider.of<ScheduleCubit>(context);
24
25   @override
26   void onChange(Change<ScheduleState> change) {
27     super.onChange(change);
28     debugPrint('📡 ScheduleCubit State Changed: ${change.currentState.runtimeType} -> ${change.nextState.runtimeType}');
29   }
30   Future<void> getSchedule() async {
31     final weekDate = _currentWeekDate();
32
33     // Emit loading state initially
34     if (!isClosed) emit(ScheduleLoading());
35
36     // 1. Fetch from /api/schedule
37     final result = await ApiService.getSchedule(weekDate: weekDate);
38
39     if (isClosed) return;
40
41     result.fold(
42       (error) {
43         debugPrint('❌ Schedule Error Response: $error');
44         if (!isClosed) emit(ScheduleError(error));
45       },
46       (data) {
47         _handleData(data, weekDate);
48         // 2. After successfully displaying (or handling empty), sync in background
49         _syncScheduleBackground(weekDate);
50       },
51     );
52   }
53
54   Future<void> _syncScheduleBackground(String weekDate) async {
55     // Sync from madrasati in the background
56     final syncResult = await ApiService.syncSchedule(weekDate: weekDate);
57
58     if (isClosed) return;
59
60     syncResult.fold(
61       (error) {
62         debugPrint('❌ Background Sync Error: $error');
63       },
64       (syncData) async {
65         debugPrint('✅ Background Sync Success, re-fetching schedule...');
66         // 3. Once synced, fetch from /api/schedule again to reflect in UI
67         final freshResult = await ApiService.getSchedule(weekDate: weekDate);
68         if (isClosed) return;
69
70         freshResult.fold(
71           (error) {

```

```

72         debugPrint('✖ Fresh Schedule Error: $error');
73     },
74     (data) {
75         _handleData(data, weekDate);
76     },
77 );
78 },
79 );
80 }
81
82 Future<void> refreshSchedule() async {
83     if (!isClosed) emit(ScheduleLoading());
84     final weekDate = _currentWeekDate();
85
86     // When user explicitly clicks refresh, we trigger the sync process
87     final syncResult = await ApiService.syncSchedule(weekDate: weekDate);
88
89     if (isClosed) return;
90
91     syncResult.fold(
92         (error) {
93             debugPrint('✖ Refresh Sync Error: $error');
94             if (!isClosed) emit(ScheduleError(error));
95         },
96         (syncData) async {
97             // Then get from /api/schedule
98             final freshResult = await ApiService.getSchedule(weekDate: weekDate);
99
100             if (isClosed) return;
101
102             freshResult.fold(
103                 (error) {
104                     debugPrint('✖ Fresh Schedule Error: $error');
105                     if (!isClosed) emit(ScheduleError(error));
106                 },
107                 (data) {
108                     _handleData(data, weekDate);
109                 },
110             );
111         },
112     );
113 }
114
115 Future<void> syncSchedule() async {
116     final weekDate = _currentWeekDate();
117     await _syncScheduleBackground(weekDate);
118 }
119
120 Future<void> prepareLesson({
121     required ClassModel classModel,
122     required List<String> selectedModules,
123 }) async {
124     debugPrint('🚀 Starting Lesson Preparation for: ${classModel.lessonTitle}');
125     debugPrint('    - Lesson ID: ${classModel.lessonId}');
126     debugPrint('    - Subject ID: ${classModel.subjectId}');
127     debugPrint('    - Classroom ID: ${classModel.classroomId}');
128     debugPrint('    - Modules: $selectedModules');
129
130     // emit(SchedulePrepareLoading()); // Could emit a specific loading state if needed.
131
132     final result = await ApiService.prepareLesson(
133         lessonId: classModel.lessonId,
134         subjectId: classModel.subjectId,
135         classroomId: classModel.classroomId,
136         schoolMadrasatiId: classModel.realSchoolId,
137         timeTableId: classModel.timeTaleId,
138         selectedModules: selectedModules,
139         encryptedToken: classModel.encryptedToken,
140     );
141
142     if (isClosed) return;
143
144     result.fold(
145         (error) {
146             debugPrint('✖ Lesson Preparation Error: $error');

```

```

147         if (!isClosed && error.startsWith('__402__')) {
148             final parts = error.split(':');
149             final code = parts.length > 1 ? parts[1] : 'quota_exceeded';
150             final msg = parts.length > 2 ? parts.sublist(2).join(':') : 'ترقية الحساب المطلوبة';
151             emit(SchedulePaymentRequired(msg, code));
152         }
153         // emit(SchedulePrepareError(error)); // Handle error state if UI needs to react
154     },
155     (data) {
156         debugPrint('✅ Lesson Preparation Success!');
157         debugPrint('    - Response: $data');
158
159         final preparationId = int.tryParse(data['preparation_id']?.toString() ?? '');
160         if (preparationId != null) {
161             _startPolling(preparationId);
162         }
163     },
164 );
165 }
166
167 Timer? _pollingTimer;
168
169 void _startPolling(int preparationId) {
170     _pollingTimer?.cancel();
171     _pollingTimer = Timer.periodic(const Duration(seconds: 3), (timer) async {
172         if (isClosed) {
173             timer.cancel();
174             return;
175         }
176         debugPrint('🕒 Polling preparation status for ID: $preparationId');
177         final result = await ApiService.checkPreparationStatus(preparationId: preparationId);
178         if (isClosed) {
179             timer.cancel();
180             return;
181         }
182
183         result.fold(
184             (error) {
185                 debugPrint('❌ Polling Error: $error');
186                 timer.cancel();
187             },
188             (data) {
189                 debugPrint('✅ Polling Response: $data');
190                 final status = data['status'] as String?;
191                 if (status == 'completed' || status == 'success' || status == 'failed' || status == 'error') {
192                     timer.cancel();
193                     if (status == 'completed' || status == 'success') {
194                         refreshSchedule();
195                     }
196                 }
197             },
198         );
199     });
200 }
201
202 @override
203 Future<void> close() {
204     _pollingTimer?.cancel();
205     _sessionActiveSubscription?.cancel();
206     return super.close();
207 }
208
209 void selectDay(int index) {
210     if (state is! ScheduleLoaded) return;
211     final current = state as ScheduleLoaded;
212     final selected = current.days[index];
213     if (!isClosed) {
214         emit(ScheduleLoaded(
215             days: current.days,
216             classes: _classesForDay(current.allClasses, selected.dayOfWeek),
217             allClasses: current.allClasses,
218             selectedDayIndex: index,
219             availableLessons: current.availableLessons,
220         ));
221     }

```

```

222     }
223
224     Future<void> fetchAvailableLessonsIfNeeded() async {
225         if (state is! ScheduleLoaded) return;
226         final current = state as ScheduleLoaded;
227         if (current.availableLessons != null && current.availableLessons!.isNotEmpty) return;
228
229         final weekDate = _currentWeekDate();
230         final result = await ApiService.getAvailableLessons(weekDate: weekDate);
231         if (isClosed) return;
232         result.fold(
233             (error) => debugPrint('✖ Available Lessons Error: $error'),
234             (data) {
235                 if (!isClosed) {
236                     emit(ScheduleLoaded(
237                         days: current.days,
238                         classes: current.classes,
239                         allClasses: current.allClasses,
240                         selectedDayIndex: current.selectedDayIndex,
241                         availableLessons: data,
242                     ));
243                 }
244             },
245         );
246     }
247
248     void _handleData(Map<String, dynamic> data, String weekDate) {
249         debugPrint('✅ ScheduleCubit Success Full Response: $data');
250
251         final weekStart = data['week_start'] as String? ?? weekDate;
252         final days = _parseDays(weekStart, <String, dynamic>{}); // mock doesn't need rawSchedule for days
253         final allClasses = <ClassModel>[];
254
255         final daysList = data['days'] is List
256             ? data['days'] as List
257             : (data['schedule'] is List ? data['schedule'] as List : null);
258         final fakeData = data['data'] is List ? data['data'] as List : null;
259
260         if (daysList != null) {
261             for (final dayItem in daysList) {
262                 if (dayItem is Map && dayItem['periods'] is List) {
263                     final int dayOfWeekValue = dayItem['day_of_week'] as int? ?? 0;
264                     final int normalizedDayOfWeek = dayOfWeekValue + 1; // 0 -> 1 (Sunday)
265                     for (final periodItem in dayItem['periods']) {
266                         if (periodItem is Map) {
267                             final p = Map<String, dynamic>.from(periodItem);
268                             p['day_of_week'] = normalizedDayOfWeek;
269                             allClasses.add(ClassModel.fromJson(p));
270                         }
271                     }
272                 }
273             }
274         } else if (fakeData != null) {
275             // Parse fake API structure
276             for (final item in fakeData) {
277                 if (item is Map) {
278                     allClasses.add(ClassModel.fromJson(Map<String, dynamic>.from(item)));
279                 }
280             }
281         } else {
282             // Parse old structure
283             final scheduleRaw = data['schedule'] ?? data['data'];
284             final rawSchedule = scheduleRaw is Map
285                 ? Map<String, dynamic>.from(scheduleRaw)
286                 : <String, dynamic>{};
287             allClasses.addAll(_parseAllClasses(rawSchedule));
288         }
289
290         allClasses.sort((a, b) => a.periodNumber.compareTo(b.periodNumber));
291
292         final firstDayClasses = days.isNotEmpty
293             ? _classesForDay(allClasses, days.first.dayOfWeek)
294             : <ClassModel>[];
295
296         final newState = ScheduleLoaded(

```

```

297     days: days,
298     classes: firstDayClasses,
299     allClasses: allClasses,
300     selectedDayIndex: 0,
301     availableLessons: null,
302   );
303
304   debugPrint('✅ ScheduleCubit Success: week_start=$weekStart | days=${days.length} | classes=${allClasses.
length}');
305
306   if (!isClosed) {
307     emit(newState);
308   }
309 }
310
311
312
313 DateTime? _currentWeekStart;
314
315 String _currentWeekDate() {
316   if (_currentWeekStart == null) {
317     final now = DateTime.now();
318     // Dart weekday: Monday=1 ... Sunday=7
319     final daysFromSunday = now.weekday == 7 ? 0 : now.weekday;
320     _currentWeekStart = now.subtract(Duration(days: daysFromSunday));
321   }
322   return '${_currentWeekStart!.year}-${
323     '${_currentWeekStart!.month.toString().padLeft(2, '0')}'-
324     '${_currentWeekStart!.day.toString().padLeft(2, '0')}'';
325 }
326
327 void nextWeek() {
328   _currentWeekDate();
329   _currentWeekStart = _currentWeekStart!.add(const Duration(days: 7));
330   getSchedule();
331 }
332
333 void previousWeek() {
334   _currentWeekDate();
335   _currentWeekStart = _currentWeekStart!.subtract(const Duration(days: 7));
336   getSchedule();
337 }
338
339 // -----
340 // Private - parse helpers (no Widget returned - compliant with rules.md)
341 // -----
342
343 List<DayModel> _parseDays(
344   String weekStart,
345   Map<String, dynamic> rawSchedule,
346 ) {
347   final startDate = DateTime.parse(weekStart);
348   // Always show Sunday (1) to Thursday (5)
349   final sortedKeys = ['1', '2', '3', '4', '5'];
350
351   const dayNameKeys = <String, String>{
352     '1': 'sunday',
353     '2': 'monday',
354     '3': 'tuesday',
355     '4': 'wednesday',
356     '5': 'thursday',
357   };
358
359   return sortedKeys.map((key) {
360     final dayOfWeek = int.parse(key);
361     final dayDate = startDate.add(Duration(days: dayOfWeek - 1));
362     return DayModel(
363       nameKey: dayNameKeys[key] ?? 'day_$key',
364       date: '${dayDate.day} ${_arabicMonth(dayDate.month)}',
365       dayOfWeek: dayOfWeek,
366     );
367   }).toList();
368 }
369
370 List<ClassModel> _parseAllClasses(Map<String, dynamic> rawSchedule) {

```

```

371     final classes = <ClassModel>[];
372     rawSchedule.forEach((_, dayList) {
373         if (dayList is List) {
374             for (final item in dayList) {
375                 if (item is Map) {
376                     classes.add(ClassModel.fromJson(Map<String, dynamic>.from(item)));
377                 }
378             }
379         }
380     });
381     classes.sort((a, b) => a.periodNumber.compareTo(b.periodNumber));
382     return classes;
383 }
384
385 List<ClassModel> _classesForDay(
386     List<ClassModel> allClasses,
387     int dayOfWeek,
388 ) {
389     final dayClasses =
390         allClasses.where((c) => c.dayOfWeek == dayOfWeek).toList();
391     if (dayClasses.isEmpty) return [];
392
393     // Fixed to 7 periods as standard
394     const maxPeriod = 7;
395
396     final filledClasses = <ClassModel>[];
397     for (int i = 1; i <= maxPeriod; i++) {
398         final existing = dayClasses.where((c) => c.periodNumber == i).toList();
399         if (existing.isNotEmpty) {
400             filledClasses.addAll(existing);
401         } else {
402             filledClasses.add(
403                 ClassModel(
404                     id: 'empty_${dayOfWeek}_$i',
405                     periodNumber: i,
406                     classroomId: '',
407                     status: ClassStatus.notPrepared,
408                     dayOfWeek: dayOfWeek,
409                     lessonTitle: null,
410                 ),
411             );
412         }
413     }
414     return filledClasses;
415 }
416
417 /// Arabic month names - data utility, not UI label.
418 String _arabicMonth(int month) {
419     const names = [
420         '',
421         'يناير',
422         'فبراير',
423         'مارس',
424         'أبريل',
425         'مايو',
426         'يونيو',
427         'يوليو',
428         'أغسطس',
429         'سبتمبر',
430         'أكتوبر',
431         'نوفمبر',
432         'ديسمبر',
433     ];
434     return names[month];
435 }
436 }
437

```

```

1 import 'package:moean/features/schedule/data/models/schedule_models.dart';
2
3 abstract class ScheduleState {}

```

```

4
5 class ScheduleInitial extends ScheduleState {}
6
7 class ScheduleLoading extends ScheduleState {}
8
9 class ScheduleLoaded extends ScheduleState {
10   final List<DayModel> days;
11   final List<ClassModel> classes;
12   final List<ClassModel> allClasses;
13   final int selectedDayIndex;
14   final Map<String, dynamic>? availableLessons;
15
16   ScheduleLoaded({
17     required this.days,
18     required this.classes,
19     required this.allClasses,
20     required this.selectedDayIndex,
21     this.availableLessons,
22   });
23 }
24
25 class ScheduleError extends ScheduleState {
26   final String message;
27
28   ScheduleError(this.message);
29 }
30
31 class SchedulePaymentRequired extends ScheduleState {
32   final String message;
33   final String code;
34
35   SchedulePaymentRequired(this.message, this.code);
36 }
37

```

Lines • Dart 209

Feature / Schedule

#356 – lib/features/schedule/presentation/screen/schedule_screen.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter_bloc/flutter_bloc.dart';
3 import 'package:moean/core/theme/colors.dart';
4 import 'package:moean/core/theme/text_styles.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/primary/conditional_builder.dart';
7 import 'package:moean/core/utils/constants/spacing.dart';
8 import 'package:moean/core/utils/cubit/theme/theme_cubit.dart';
9 import 'package:moean/core/utils/cubit/theme/theme_state.dart';
10 import 'package:moean/features/schedule/presentation/cubit/schedule_cubit.dart';
11 import 'package:moean/features/schedule/presentation/cubit/schedule_state.dart';
12 import 'package:moean/features/schedule/presentation/widgets/class_card.dart';
13 import 'package:moean/features/schedule/presentation/widgets/day_tabs_list.dart';
14 import 'package:moean/features/schedule/presentation/widgets/schedule_app_bar.dart';
15 import 'package:moean/features/schedule/presentation/widgets/status_strip.dart';
16 import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
17 import 'package:moean/features/payment/presentation/widgets/trial_banner_widget.dart';
18 import 'package:moean/core/utils/constants/primary/upgrade_prompt_bottom_sheet.dart';
19
20 class ScheduleScreen extends StatelessWidget {
21   const ScheduleScreen({super.key});
22
23   @override
24   Widget build(BuildContext context) {
25     return BlocProvider(
26       create: (context) => ScheduleCubit().getSchedule(),
27       child: BlocBuilder<ThemeCubit, ThemeState>(
28         builder: (context, themeState) {
29           return BlocListener<ScheduleCubit, ScheduleState>(
30             listener: (context, state) {
31               if (state is SchedulePaymentRequired) {
32                 UpgradePromptBottomSheet.show(
33                   context,
34                   message: state.message,
35                   isQuotaExceeded: state.code == 'quota_exceeded',
36                 );
37             }
38           );
39         },
40       ),
41     );
42   }
43 }
44

```



```

109         ? _ScheduleEmptyView(message: appTranslation().get('no_classes'))
110         : ListView.builder(
111             itemCount: loaded.classes.length,
112             itemBuilder: (context, index) =>
113                 ClassCard(
114                     key: ValueKey(loaded.classes[index].id),
115                     classModel: loaded.classes[index],
116                 ),
117             ),
118         ),
119     ],
120 ),
121 );
122 },
123 );
124 },
125 ),
126 ),
127 ],
128 ),
129 ),
130 ),
131 );
132 },
133 ),
134 );
135 }
136 }
137
138
139 class _ScheduleErrorView extends StatelessWidget {
140     final String message;
141
142     const _ScheduleErrorView({required this.message});
143
144     @override
145     Widget build(BuildContext context) {
146         return Center(
147             child: Padding(
148                 padding: const EdgeInsets.symmetric(horizontal: 32),
149                 child: Column(
150                     mainAxisAlignment: MainAxisAlignment.min,
151                     children: [
152                         Icon(
153                             Icons.wifi_off_rounded,
154                             size: 64,
155                             color: ColorsManager.secondaryText.withValues(alpha: 0.5),
156                         ),
157                         verticalSpace16,
158                         Text(
159                             appTranslation().get('schedule_error_title'),
160                             style: TextStylesManager.bold16
161                                 .copyWith(color: ColorsManager.mainText),
162                             textAlign: TextAlign.center,
163                         ),
164                         verticalSpace8,
165                         Text(
166                             message,
167                             style: TextStylesManager.regular12
168                                 .copyWith(color: ColorsManager.secondaryText),
169                             textAlign: TextAlign.center,
170                         ),
171                     ],
172                 ),
173             ),
174         );
175     }
176 }
177
178 class _ScheduleEmptyView extends StatelessWidget {
179     final String? message;
180     const _ScheduleEmptyView({this.message});
181
182     @override
183     Widget build(BuildContext context) {

```

```

184     return Center(
185       child: Padding(
186         padding: const EdgeInsets.symmetric(horizontal: 32),
187         child: Column(
188           mainAxisAlignment: MainAxisAlignment.min,
189           children: [
190             Icon(
191               Icons.calendar_today_outlined,
192               size: 64,
193               color: ColorsManager.primaryDark,
194             ),
195             verticalSpace16,
196             Text(
197               message ?? appTranslation().get('schedule_empty_title'),
198               style: TextStylesManager.bold16
199                 .copyWith(color: ColorsManager.mainText),
200               textAlign: TextAlign.center,
201             ),
202             verticalSpace8,
203           ],
204         ),
205       ),
206     );
207   }
208 }
209

```

Lines • Dart 57

Feature / Schedule

#357 – lib/features/schedule/presentation/widgets/action_button_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6
7  class ActionButtonWidget extends StatelessWidget {
8    final String titleKey;
9    final IconData icon;
10   final Color color;
11   final Color? backgroundColor;
12   final bool isOutlined;
13   final VoidCallback? onTap;
14
15   const ActionButtonWidget({
16     super.key,
17     required this.titleKey,
18     required this.icon,
19     required this.color,
20     this.backgroundColor,
21     this.isOutlined = false,
22     this.onTap,
23   });
24
25   @override
26   Widget build(BuildContext context) {
27     return GestureDetector(
28       onTap: onTap,
29       child: Container(
30         width: 72,
31         height: 72,
32         decoration: BoxDecoration(
33           color: backgroundColor ?? ColorsManager.surfacePrimary,
34           borderRadius: BorderRadius.circular(12),
35           border: Border.all(
36             color: isOutlined ? color : (backgroundColor == null ? ColorsManager.borderLightGray : Colors.transparent),
37           ),
38         ),
39         child: Column(
40           mainAxisAlignment: MainAxisAlignment.center,
41           children: [
42             Icon(icon, color: color, size: 24),
43             verticalSpace4,

```

```

44         Text(
45             appTranslation().get(titleKey),
46             style: TextStylesManager.regular10.copyWith(
47                 color: color,
48             ),
49             textAlign: TextAlign.center,
50         ),
51     ],
52 ),
53 ),
54 );
55 }
56 }
57

```

Lines • Dart 526

Feature / Schedule

#358 – lib/features/schedule/presentation/widgets/class_card.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6  import 'package:moean/features/schedule/data/models/schedule_models.dart';
7  import 'package:moean/features/schedule/presentation/widgets/status_strip.dart';
8  import 'package:flutter_bloc/flutter_bloc.dart';
9  import 'package:moean/features/schedule/presentation/cubit/schedule_cubit.dart';
10 import 'package:moean/features/schedule/presentation/cubit/schedule_state.dart';
11 import 'package:moean/features/schedule/presentation/widgets/schedule_bottom_sheet.dart';
12
13 class ClassCard extends StatefulWidget {
14     final ClassModel classModel;
15
16     const ClassCard({super.key, required this.classModel});
17
18     @override
19     State<ClassCard> createState() => _ClassCardState();
20 }
21
22 class _ClassCardState extends State<ClassCard> {
23     bool _isExpanded = false;
24     Map<String, dynamic>? _selectedLessonMap;
25
26     final List<String> _fallbackLessons = [
27         'lesson_2',
28         'lesson_1',
29         'review',
30         'applied_activity',
31     ];
32
33     @override
34     void initState() {
35         super.initState();
36         _selectedLessonMap = null;
37     }
38
39     void _toggleExpand() {
40         setState(() {
41             _isExpanded = !_isExpanded;
42         });
43         if (_isExpanded) {
44             ScheduleCubit.get(context).fetchAvailableLessonsIfNeeded();
45         }
46     }
47
48     void _showActions(BuildContext context) {
49         final cubit = ScheduleCubit.get(context);
50
51         ClassModel activeClass = widget.classModel;
52         if (_selectedLessonMap != null) {
53             final selectedId = int.tryParse(_selectedLessonMap!['lesson_id']?.toString() ?? '') ??
54                 int.tryParse(_selectedLessonMap!['id']?.toString() ?? '') ?? widget.classModel.lesso
55 nId;
56             final selectedTitle = _selectedLessonMap!['title']?.toString() ??

```

```

56         _selectedLessonMap!['lesson_title']?.toString() ??
57         _selectedLessonMap!['name']?.toString() ??
58         _selectedLessonMap!['value']?.toString() ?? widget.classModel.lessonTitle;
59
60     activeClass = ClassModel(
61       id: widget.classModel.id,
62       periodNumber: widget.classModel.periodNumber,
63       lessonTitle: selectedTitle,
64       classroomId: widget.classModel.classroomId,
65       status: widget.classModel.status,
66       dayOfWeek: widget.classModel.dayOfWeek,
67       realSchoolId: widget.classModel.realSchoolId,
68       timeTableId: widget.classModel.timeTableId,
69       subjectId: widget.classModel.subjectId,
70       lessonId: selectedId,
71       time: widget.classModel.time,
72       date: widget.classModel.date,
73       encryptedToken: widget.classModel.encryptedToken,
74       subjectName: widget.classModel.subjectName,
75       classroomName: widget.classModel.classroomName,
76       madrasatiStatus: widget.classModel.madrasatiStatus,
77     );
78   }
79
80   showModalBottomSheet(
81     context: context,
82     isScrollControlled: true,
83     backgroundColor: Colors.transparent,
84     builder: (context) => BlocProvider.value(
85       value: cubit,
86       child: ScheduleBottomSheet(classModel: activeClass),
87     ),
88   );
89 }
90
91 ClassStatus _getMappedStatus(ClassModel classModel) {
92   final madrasatiStatus = classModel.madrasatiStatus.toLowerCase();
93   if (madrasatiStatus == 'expired') return ClassStatus.notPrepared;
94   if (madrasatiStatus == 'waiting') return ClassStatus.waiting;
95   if (madrasatiStatus == 'ready') return ClassStatus.prepared;
96
97   return classModel.status;
98 }
99
100 Color _getStatusColor(ClassModel classModel) {
101   final status = _getMappedStatus(classModel);
102   switch (status) {
103     case ClassStatus.waiting:
104       return ColorsManager.statusWaiting;
105     case ClassStatus.prepared:
106       return ColorsManager.statusSuccess;
107     case ClassStatus.notPrepared:
108       return ColorsManager.statusWarning;
109     case ClassStatus.activity:
110       return ColorsManager.statusActivity;
111   }
112 }
113
114 List<Map<String, dynamic>> _getAvailableLessonsForThisClass() {
115   final state = ScheduleCubit.get(context).state;
116   if (state is ScheduleLoaded && state.availableLessons != null) {
117     final available = state.availableLessons!;
118     try {
119       final timeTableId = widget.classModel.timeTableId;
120       final subjectId = widget.classModel.subjectId.toString();
121
122       dynamic targetList;
123
124       if (timeTableId.isNotEmpty && available.containsKey(timeTableId)) {
125         targetList = available[timeTableId];
126       } else if (subjectId != '0' && available.containsKey(subjectId)) {
127         targetList = available[subjectId];
128       } else if (available.containsKey('data')) {
129         final dataObj = available['data'];
130

```

```

131         if (dataObj is List) {
132             for (var item in dataObj) {
133                 if (item is Map && item['time_table_id']?.toString() == timeTableId) {
134                     targetList = item['lessons'] ?? item['available_lessons'];
135                     break;
136                 }
137             }
138             if (targetList == null && dataObj.isNotEmpty && dataObj.first is Map) {
139                 if (dataObj.first.containsKey('lesson_id') || dataObj.first.containsKey('id')) {
140                     targetList = dataObj;
141                 }
142             }
143             } else if (dataObj is Map) {
144                 if (dataObj.containsKey(timeTableId)) {
145                     targetList = dataObj[timeTableId];
146                 } else if (dataObj.containsKey(subjectId)) {
147                     targetList = dataObj[subjectId];
148                 } else if (dataObj.values.length == 1 && dataObj.values.first is List) {
149                     targetList = dataObj.values.first;
150                 }
151             }
152             } else if (available.containsKey('periods')) {
153                 final periodsList = available['periods'];
154                 if (periodsList is List) {
155                     for (var period in periodsList) {
156                         if (period is Map && (period['time_table_id']?.toString() == timeTableId || period['subject_id']?.toString() == subjectId)) {
157                             targetList = period['available_lessons'] ?? period['lessons'];
158                             if (targetList != null && (targetList as List).isEmpty) break;
159                         }
160                     }
161                 }
162             } else if (available.containsKey('lessons')) {
163                 targetList = available['lessons'];
164             } else {
165                 var lists = available.values.whereType<List>().toList();
166                 if (lists.isNotEmpty) {
167                     targetList = lists.first;
168                 }
169             }
170
171             if (targetList is List) {
172                 return targetList.map((e) => e is Map ? Map<String, dynamic>.from(e) : {'value': e.toString()}).toList();
173             }
174             } catch (e) {
175                 debugPrint('Error parsing available lessons: $e');
176             }
177         }
178         return [];
179     }
180
181     String _getLessonDisplayName(Map<String, dynamic> lesson) {
182         return lesson['title']?.toString() ??
183             lesson['lesson_title']?.toString() ??
184             lesson['name']?.toString() ??
185             lesson['value']?.toString() ?? 'Lesson';
186     }
187
188     @override
189     Widget build(BuildContext context) {
190         final bool hasLesson = widget.classModel.lessonTitle != null;
191         final bool isActivity = widget.classModel.status == ClassStatus.activity;
192         // final bool isNotAssigned = widget.classModel.lessonId == 0;
193
194         return GestureDetector(
195             onTap: (isActivity || !hasLesson) ? null : () => _showActions(context),
196             child: Stack(
197                 children: [
198                     AnimatedContainer(
199                         duration: const Duration(milliseconds: 300),
200                         margin: const EdgeInsets.only(bottom: 16),
201                         padding: const EdgeInsets.all(16),
202                         decoration: BoxDecoration(
203                             color: _isExpanded

```

```

204         ? ColorsManager.brandMint.withValues(alpha: 0.2)
205         : ColorsManager.surfacePrimary,
206         borderRadius: BorderRadius.circular(16),
207     ),
208     child: Column(
209         children: [
210             Row(
211                 crossAxisAlignment: CrossAxisAlignment.start,
212                 mainAxisAlignment: MainAxisAlignment.spaceBetween,
213                 children: [
214                     Expanded(
215                         child: Column(
216                             crossAxisAlignment: CrossAxisAlignment.start,
217                             children: [
218                                 Text(
219                                     appTranslation().get(widget.classModel.periodKey),
220                                     style: TextStylesManager.regular12
221                                     .copyWith(color: ColorsManager.mainText),
222                                 ),
223                                 verticalSpace4,
224                                 Text(
225                                     _selectedLessonMap != null
226                                     ? _getLessonDisplayName(_selectedLessonMap!)
227                                     : (widget.classModel.subjectName.isNotEmpty
228                                     ? widget.classModel.subjectName
229                                     : (widget.classModel.lessonTitle ?? appTranslation().get('no_classes'))),
230                                     style: TextStylesManager.bold18
231                                     .copyWith(color: ColorsManager.mainText),
232                                     maxLines: 1,
233                                     overflow: TextOverflow.ellipsis,
234                                 ),
235                                 verticalSpace4,
236                                 if (hasLesson)
237                                 Row(
238                                     children: [
239                                         Icon(
240                                             Icons.school,
241                                             size: 14,
242                                             color: ColorsManager.primaryColor,
243                                         ),
244                                         horizontalSpace4,
245                                         Text(
246                                             widget.classModel.classroomName.isNotEmpty
247                                             ? widget.classModel.classroomName
248                                             : widget.classModel.classroomId,
249                                             style: TextStylesManager.regular12
250                                             .copyWith(color: ColorsManager.secondaryText),
251                                         ),
252                                     ],
253                                 ),
254                             ],
255                         ),
256                     ),
257                     Column(
258                         crossAxisAlignment: CrossAxisAlignment.end,
259                         children: [
260                             if (widget.classModel.time.isNotEmpty)
261                             Row(
262                                 mainAxisAlignment: MainAxisAlignment.min,
263                                 children: [
264                                     Icon(
265                                         Icons.access_time,
266                                         size: 14,
267                                         color: ColorsManager.secondaryText,
268                                     ),
269                                     horizontalSpace4,
270                                     Text(
271                                         widget.classModel.time,
272                                         style: TextStylesManager.regular12
273                                         .copyWith(color: ColorsManager.secondaryText),
274                                         textDirection: TextDirection.ltr,
275                                     ),
276                                 ],
277                             ),
278                             verticalSpace16,

```

[illegible]

```

353         .copyWith(color: ColorsManager.mainText),
354         maxLines: 1,
355         overflow: TextOverflow.ellipsis,
356     ),
357 ),
358 ],
359 ),
360 onChanged: (String? newValue) {
361     if (newValue != null && newValue != 'header') {
362         setState(() {
363             if (useDynamic) {
364                 _selectedLessonMap = dynamicLessons.firstWhere(
365                     (l) => _getLessonDisplayName(l) == newValue,
366                     orElse: () => {'value': newValue},
367                 );
368             } else {
369                 _selectedLessonMap = {'value': newValue};
370             }
371         });
372     }
373 },
374 items: [
375     DropdownMenuItem<String>(
376         enabled: false,
377         value: 'header',
378         child: Row(
379             children: [
380                 Container(
381                     padding: const EdgeInsets.all(6),
382                     decoration: BoxDecoration(
383                         color: ColorsManager.brandMint.withValues(alpha: 0.2),
384                         shape: BoxShape.circle,
385                     ),
386                     child: Text(
387                         '4',
388                         style: TextStylesManager.bold18.copyWith(
389                             color: ColorsManager.primaryColor,
390                         ),
391                     ),
392                 ),
393                 horizontalSpace12,
394                 Text(
395                     appTranslation().get('choose_lesson'),
396                     style: TextStylesManager.bold18.copyWith(
397                         color: ColorsManager.primaryColor,
398                     ),
399                 ),
400             ],
401         ),
402     ),
403     if (useDynamic)
404         ...dynamicLessons.map<DropdownMenuItem<String>>((Map<String, dynamic> lessonMap)
405 {
406         String value = _getLessonDisplayName(lessonMap);
407         bool isSelected = _selectedLessonMap != null && _getLessonDisplayName(_selected
LessonMap!) == value;
408         return DropdownMenuItem<String>(
409             value: value,
410             child: Container(
411                 width: double.infinity,
412                 padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
413                 decoration: BoxDecoration(
414                     color: isSelected ? ColorsManager.primaryColor : Colors.transparent,
415                     borderRadius: BorderRadius.circular(8),
416                 ),
417                 child: Row(
418                     children: [
419                         if (isSelected) ...[
420                             Icon(Icons.check_circle_outline, color: ColorsManager.surfacePrimary,
size: 20),
421                             horizontalSpace8,
422                         ],
423                         Expanded(
424                             child: Text(
425                                 value,

```



```

425         style: TextStyleManager.regular12.copyWith(
426             color: isSelected ? ColorsManager.surfacePrimary : ColorsManager.
mainText,
427             fontWeight: isSelected ? FontWeight.bold : FontWeight.normal,
428         ),
429         maxLines: 1,
430         overflow: TextOverflow.ellipsis,
431     ),
432 ),
433 ],
434 ),
435 ),
436 );
437 })
438 else
439     ..._fallbackLessons.map<DropdownMenuItem<String>>((String value) {
440         String translatedValue = appTranslation().get(value);
441         bool isSelected = _selectedLessonMap != null && _selectedLessonMap!['value'] ==
value;
442         return DropdownMenuItem<String>(
443             value: value,
444             child: Container(
445                 width: double.infinity,
446                 padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
447                 decoration: BoxDecoration(
448                     color: isSelected ? ColorsManager.primaryColor : Colors.transparent,
449                     borderRadius: BorderRadius.circular(8),
450                 ),
451                 child: Row(
452                     children: [
453                         if (isSelected) ...[
454                             Icon(Icons.check_circle_outline, color: ColorsManager.surfacePrimary,
size: 20),
455                             horizontalSpace8,
456                         ],
457                         Text(
458                             translatedValue,
459                             style: TextStyleManager.regular12.copyWith(
460                                 color: isSelected ? ColorsManager.surfacePrimary : ColorsManager.ma
inText,
461                                 fontWeight: isSelected ? FontWeight.bold : FontWeight.normal,
462                             ),
463                         ),
464                     ],
465                 ),
466             ),
467         );
468     }),
469 ],
470     selectedItemBuilder: (BuildContext context) {
471         return ['header', if (useDynamic) ...dynamicLessons.map((e) => _getLessonDisplayNam
e(e)) else ..._fallbackLessons].map<Widget>((String value) {
472             return Row(
473                 children: [
474                     Icon(
475                         Icons.bookmark_border,
476                         color: ColorsManager.primaryColor,
477                         size: 20,
478                     ),
479                     horizontalSpace8,
480                     Expanded(
481                         child: Text(
482                             _selectedLessonMap != null
483                                 ? _getLessonDisplayName(_selectedLessonMap!)
484                                 : widget.classModel.lessonTitle!,
485                             style: TextStyleManager.regular12
486                                 .copyWith(color: ColorsManager.mainText),
487                             maxLines: 1,
488                             overflow: TextOverflow.ellipsis,
489                         ),
490                     ),
491                 ],
492             );
493         }).toList();
494     },

```



```

39         borderRadius: BorderRadius.circular(3),
40     ),
41 ),
42 ),
43 ),
44
45 Container(
46   height: 72,
47   width: double.infinity,
48   decoration: BoxDecoration(
49     color: isSelected ? ColorsManager.primaryColor : Colors.transparent,
50     borderRadius: BorderRadius.circular(16),
51   ),
52   child: Column(
53     mainAxisAlignment: MainAxisAlignment.center,
54     children: [
55       FittedBox(
56         fit: BoxFit.scaleDown,
57         child: Text(
58           appTranslation().get(day.nameKey),
59           style: isSelected
60             ? TextStylesManager.bold14.copyWith(color: ColorsManager.white)
61             : TextStylesManager.regular14.copyWith(color: ColorsManager.secondaryText),
62         ),
63       ),
64       verticalSpace4,
65       FittedBox(
66         fit: BoxFit.scaleDown,
67         child: Text(
68           day.date,
69           style: isSelected
70             ? TextStylesManager.regular12.copyWith(color: ColorsManager.white)
71             : TextStylesManager.regular12.copyWith(color: ColorsManager.secondaryText),
72         ),
73       ),
74     ],
75   ),
76 ),
77 ],
78 ),
79 );
80 }
81 }

```

Lines • Dart 89

Feature / Schedule

#360 – lib/features/schedule/presentation/widgets/day_tabs_list.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/features/schedule/data/models/schedule_models.dart';
4  import 'package:moean/features/schedule/presentation/widgets/day_tab_item.dart';
5  import 'package:moean/features/schedule/presentation/cubit/schedule_cubit.dart';
6
7  class DayTabsList extends StatelessWidget {
8    final List<DayModel> days;
9    final int selectedIndex;
10    final Function(int) onDaySelected;
11
12    const DayTabsList({
13      super.key,
14      required this.days,
15      required this.selectedIndex,
16      required this.onDaySelected,
17    });
18
19    @override
20    Widget build(BuildContext context) {
21      return Row(
22        children: [
23
24          // IconButton(
25          //   padding: EdgeInsets.zero,
26          //   constraints: const BoxConstraints(), // هذا السطر يلغي المساحة الافتراضية حول الأيقونة
27          //   icon: const Icon(

```

```

28 // Icons.keyboard_arrow_right,
29 // size: 24,
30 // color: ColorsManager.primaryDark,
31 // ),
32 // onPressed: () => ScheduleCubit.get(context).previousWeek(),
33 // ),
34 GestureDetector(
35   onTap: () => ScheduleCubit.get(context).previousWeek(),
36   child: const Icon(
37     Icons.keyboard_arrow_right,
38     size: 24,
39     color: ColorsManager.primaryDark,
40   ),
41 ),
42
43 Expanded(
44   child: Container(
45     height: 88,
46     width: double.infinity,
47     padding: const EdgeInsets.symmetric(vertical: 8, horizontal: 8),
48     decoration: BoxDecoration(
49       color: ColorsManager.surfacePrimary,
50       borderRadius: BorderRadius.circular(24),
51       border: Border.all(color: ColorsManager.borderColor),
52     ),
53     child: Row(
54       children: List.generate(days.length, (index) {
55         return Expanded(
56           child: DayTabItem(
57             day: days[index],
58             isSelected: index == selectedIndex,
59             onTap: () => onDaySelected(index),
60           ),
61         );
62       }),
63     ),
64   ),
65 ),
66
67 // IconButton(
68 //   padding: EdgeInsets.zero,
69 //   constraints: const BoxConstraints(), // هذا السطر يلغي المساحة الافتراضية حول الأيقونة
70 //   icon: const Icon(
71 //     Icons.keyboard_arrow_left,
72 //     size: 24,
73 //     color: ColorsManager.primaryDark,
74 //   ),
75 //   onPressed: () => ScheduleCubit.get(context).nextWeek(),
76 // ),
77 GestureDetector(
78   onTap: () => ScheduleCubit.get(context).nextWeek(),
79   child: const Icon(
80     Icons.keyboard_arrow_left,
81     size: 24,
82     color: ColorsManager.primaryDark,
83   ),
84 ),
85 ],
86 );
87 }
88 }
89 }

```

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/assets_helper.dart';
5 import 'package:moean/core/utils/constants/constants.dart';
6 import 'package:moean/core/utils/constants/routes.dart';
7 import 'package:moean/core/utils/extensions/context_extension.dart';
8

```

```

9 class ScheduleAppBar extends StatelessWidget {
10   const ScheduleAppBar({super.key});
11
12   @override
13   Widget build(BuildContext context) {
14     return Padding(
15       padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
16       child: Row(
17         mainAxisAlignment: MainAxisAlignment.spaceBetween,
18         children: [
19           GestureDetector(
20             onTap: () {
21               context.pop();
22             },
23             child: Icon(Icons.arrow_back_ios, color: ColorsManager.primaryColor),
24           ),
25           Image.asset(
26             AssetsHelper.logo,
27             width: 55,
28             height: 55,
29             fit: BoxFit.cover,
30           ),
31
32           Text(
33             appTranslation().get('schedule_title'),
34             style: TextStylesManager.bold18.copyWith(color: ColorsManager.mainText),
35           ),
36           GestureDetector(
37             onTap: () {
38               Navigator.pushNamed(context, Routes.profile);
39             },
40             child: Icon(Icons.person, color: ColorsManager.primaryColor, size: 28),
41           ),
42         ],
43       ),
44     );
45   }
46 }
47

```

Lines • Dart 197

Feature / Schedule

#362 – lib/features/schedule/presentation/widgets/schedule_bottom_sheet.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/spacing.dart';
6 import 'package:moean/features/schedule/data/models/schedule_models.dart';
7 import 'package:moean/features/schedule/presentation/widgets/status_strip.dart';
8 import 'package:moean/features/schedule/presentation/widgets/action_button_widget.dart';
9 import 'package:moean/features/schedule/presentation/cubit/schedule_cubit.dart';
10
11 class ScheduleBottomSheet extends StatefulWidget {
12   final ClassModel classModel;
13
14   const ScheduleBottomSheet({super.key, required this.classModel});
15
16   @override
17   State<ScheduleBottomSheet> createState() => _ScheduleBottomSheetState();
18 }
19
20 class _ScheduleBottomSheetState extends State<ScheduleBottomSheet> {
21   final List<String> _selectedModules = [];
22
23   void _toggleModule(String module) {
24     setState(() {
25       if (_selectedModules.contains(module)) {
26         _selectedModules.remove(module);
27       } else {
28         _selectedModules.add(module);
29       }
30     });
31   }
32 }
33

```

```

32
33 @override
34 Widget build(BuildContext context) {
35   return Container(
36     padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
37     decoration: BoxDecoration(
38       color: ColorsManager.surfacePrimary,
39       borderRadius: const BorderRadius.vertical(top: Radius.circular(24)),
40     ),
41     child: Column(
42       mainAxisAlignment: MainAxisAlignment.min,
43       children: [
44         Container(
45           width: 40,
46           height: 4,
47           decoration: BoxDecoration(
48             color: ColorsManager.borderLightGray,
49             borderRadius: BorderRadius.circular(2),
50           ),
51         ),
52         verticalSpace24,
53         Row(
54           children: [
55             Container(
56               padding: const EdgeInsets.all(16),
57               decoration: BoxDecoration(
58                 color: ColorsManager.brandMint.withValues(alpha: 0.3),
59                 borderRadius: BorderRadius.circular(16),
60               ),
61               child: Icon(
62                 Icons.menu_book,
63                 color: ColorsManager.primaryColor,
64                 size: 32,
65               ),
66             ),
67             horizontalSpace16,
68             Expanded(
69               child: Column(
70                 crossAxisAlignment: CrossAxisAlignment.start,
71                 children: [
72                   if (widget.classModel.lessonTitle != null)
73                     Text(
74                       widget.classModel.lessonTitle!,
75                       style: TextStylesManager.bold14
76                         .copyWith(color: ColorsManager.mainText),
77                     ),
78                   verticalSpace8,
79                   Row(
80                     children: [
81                       Text(
82                         widget.classModel.classroomId,
83                         style: TextStylesManager.regular12
84                           .copyWith(color: ColorsManager.secondaryText),
85                       ),
86                       const Spacer(),
87                       StatusIcon(status: widget.classModel.status, size: 24),
88                     ],
89                   ),
90                 ],
91             ),
92             horizontalSpace16,
93             Column(
94               crossAxisAlignment: CrossAxisAlignment.end,
95               children: [
96                 Text(
97                   appTranslation().get(widget.classModel.periodKey),
98                   style: TextStylesManager.regular12
99                     .copyWith(color: ColorsManager.primaryColor),
100                 ),
101                 Text(
102                   '${appTranslation().get('period_label')} ${widget.classModel.periodNumber}',
103                   style: TextStylesManager.bold16
104                     .copyWith(color: ColorsManager.mainText),
105                 ),
106               ],

```

```

107         Row(
108             children: [
109                 Text(
110                     widget.classModel.classroomId,
111                     style: TextStylesManager.regular12
112                     .copyWith(color: ColorsManager.secondaryText),
113                 ),
114                 horizontalSpace4,
115                 Icon(
116                     Icons.school,
117                     size: 14,
118                     color: ColorsManager.primaryColor,
119                 ),
120             ],
121         ),
122     ],
123 ),
124 ],
125 ),
126 verticalSpace24,
127 Text(
128     appTranslation().get('action_choose_prep'),
129     style: TextStylesManager.regular14
130     .copyWith(color: ColorsManager.mainText),
131 ),
132 verticalSpace16,
133 Row(
134     mainAxisAlignment: MainAxisAlignment.spaceEvenly,
135     children: [
136         ActionButtonWidget(
137             titleKey: 'action_enrichment',
138             icon: Icons.star_border,
139             color: _selectedModules.contains('enrichment') ? ColorsManager.white : ColorsManager.brandGold
140             backgroundColor: _selectedModules.contains('enrichment') ? ColorsManager.brandGold : null,
141             onTap: () => _toggleModule('enrichment'),
142         ),
143         ActionButtonWidget(
144             titleKey: 'action_exam',
145             icon: Icons.assignment_outlined,
146             color: _selectedModules.contains('exam') ? ColorsManager.white : ColorsManager.statusSuccess,
147             backgroundColor: _selectedModules.contains('exam') ? ColorsManager.statusSuccess : null,
148             onTap: () => _toggleModule('exam'),
149         ),
150         ActionButtonWidget(
151             titleKey: 'action_activity',
152             icon: Icons.directions_walk_outlined,
153             color: _selectedModules.contains('activity') ? ColorsManager.white : ColorsManager.statusActivity,
154             backgroundColor: _selectedModules.contains('activity') ? ColorsManager.statusActivity : null,
155             onTap: () => _toggleModule('activity'),
156         ),
157         ActionButtonWidget(
158             titleKey: 'action_homework',
159             icon: Icons.home_work_outlined,
160             color: _selectedModules.contains('homework') ? ColorsManager.white : ColorsManager.statusWaiting,
161             backgroundColor: _selectedModules.contains('homework') ? ColorsManager.statusWaiting : null,
162             onTap: () => _toggleModule('homework'),
163         ),
164     ],
165 ),
166 verticalSpace24,
167 SizedBox(
168     width: double.infinity,
169     height: 52,
170     child: ElevatedButton(
171         style: ElevatedButton.styleFrom(
172             backgroundColor: ColorsManager.primaryColor,
173             shape: RoundedRectangleBorder(
174                 borderRadius: BorderRadius.circular(16),
175             ),
176             elevation: 0,
177         ),
178         onPressed: () {

```

```

179         ScheduleCubit.get(context).prepareLesson(
180             classModel: widget.classModel,
181             selectedModules: _selectedModules,
182         );
183         Navigator.pop(context);
184     },
185     child: Text(
186         appTranslation().get('action_prepare_moean'),
187         style: TextStylesManager.bold16.copyWith(color: ColorsManager.white),
188     ),
189 ),
190 ),
191     verticalSpace24,
192 ],
193 ),
194 );
195 }
196 }
197

```

Lines • Dart 32

Feature / Schedule

#363 – lib/features/schedule/presentation/widgets/status_item_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/text_styles.dart';
3  import 'package:moean/core/utils/constants/constants.dart';
4  import 'package:moean/core/utils/constants/spacing.dart';
5  import 'package:moean/features/schedule/data/models/schedule_models.dart';
6  import 'package:moean/features/schedule/presentation/widgets/status_strip.dart';
7
8  class StatusItemWidget extends StatelessWidget {
9      final ClassStatus status;
10     final String titleKey;
11
12     const StatusItemWidget({
13         super.key,
14         required this.status,
15         required this.titleKey,
16     });
17
18     @override
19     Widget build(BuildContext context) {
20         return Row(
21             children: [
22                 StatusIcon(status: status, size: 24),
23                 horizontalSpace8,
24                 Text(
25                     appTranslation().get(titleKey),
26                     style: TextStylesManager.regular10,
27                 ),
28             ],
29         );
30     }
31 }
32

```

Lines • Dart 77

Feature / Schedule

#364 – lib/features/schedule/presentation/widgets/status_strip.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/utils/constants/spacing.dart';
4  import 'package:moean/features/schedule/data/models/schedule_models.dart';
5  import 'package:moean/features/schedule/presentation/widgets/status_item_widget.dart';
6
7  class StatusStrip extends StatelessWidget {
8      const StatusStrip({super.key});
9
10     @override
11     Widget build(BuildContext context) {
12         return SingleChildScrollView(
13             scrollDirection: Axis.horizontal,
14             child: Row(
15                 mainAxisAlignment: MainAxisAlignment.center,

```



```

16     children: [
17       StatusItemWidget(status: ClassStatus.waiting, titleKey: 'status_waiting'),
18       horizontalSpace16,
19       StatusItemWidget(status: ClassStatus.prepared, titleKey: 'status_prepared'),
20       horizontalSpace16,
21       StatusItemWidget(status: ClassStatus.notPrepared, titleKey: 'status_not_prepared'),
22       horizontalSpace16,
23       StatusItemWidget(status: ClassStatus.activity, titleKey: 'status_activity'),
24     ],
25   ),
26 );
27 }
28 }
29
30 class StatusIcon extends StatelessWidget {
31   final ClassStatus status;
32   final double size;
33
34   const StatusIcon({super.key, required this.status, this.size = 24});
35
36   @override
37   Widget build(BuildContext context) {
38     Color color;
39     IconData icon;
40
41     switch (status) {
42       case ClassStatus.waiting:
43         color = ColorsManager.statusWaiting;
44         icon = Icons.hourglass_top_rounded;
45         break;
46       case ClassStatus.prepared:
47         color = ColorsManager.statusSuccess;
48         icon = Icons.check;
49         break;
50       case ClassStatus.notPrepared:
51         color = ColorsManager.statusWarning;
52         icon = Icons.priority_high;
53         break;
54       case ClassStatus.activity:
55         color = ColorsManager.statusActivity;
56         icon = Icons.directions_walk;
57         break;
58     }
59
60     return Container(
61       width: size,
62       height: size,
63       decoration: BoxDecoration(
64         color: color.withValues(alpha: 0.1),
65         shape: BoxShape.circle,
66       ),
67       child: Center(
68         child: Icon(
69           icon,
70           color: color,
71           size: size * 0.6,
72         ),
73       ),
74     );
75   }
76 }
77

```

Lines • Dart 56

Feature / Search

#365 – lib/features/search/data/models/search_result_model.dart

```

1 class SearchResultModel {
2   final int? id;
3   final String type;
4   final String title;
5   final String? subtitle;
6   final String? description;
7   final String? difficulty;
8   final String? questionType;

```

```

9   final List<String>? options;
10  final String? correctAnswer;
11  final Map<String, dynamic> raw;
12
13  const SearchResultModel({
14    this.id,
15    required this.type,
16    required this.title,
17    this.subtitle,
18    this.description,
19    this.difficulty,
20    this.questionType,
21    this.options,
22    this.correctAnswer,
23    required this.raw,
24  });
25
26  factory SearchResultModel.fromJson(Map<String, dynamic> json, String type) {
27    List<String>? options;
28    if (json['options'] is List) {
29      options = (json['options'] as List).map((e) => e.toString()).toList();
30    }
31
32    return SearchResultModel(
33      id: json['id'] as int?,
34      type: type,
35      title: (json['title'] ?? json['question_text'] ?? json['name'] ?? '').toString(),
36      subtitle: (json['subject_name'] ?? json['lesson_name'] ?? '').toString().isEmpty
37        ? (json['subject_name'] ?? json['lesson_name'] ?? '').toString()
38        : null,
39      description: (json['description'] ?? '').toString().isEmpty
40        ? json['description'].toString()
41        : null,
42      difficulty: (json['difficulty'] ?? '').toString().isEmpty
43        ? json['difficulty'].toString()
44        : null,
45      questionType: (json['question_type'] ?? '').toString().isEmpty
46        ? json['question_type'].toString()
47        : null,
48      options: options,
49      correctAnswer: (json['correct_answer'] ?? '').toString().isEmpty
50        ? json['correct_answer'].toString()
51        : null,
52      raw: json,
53    );
54  }
55 }
56

```

Lines • Dart 68

Feature / Search

#366 – lib/features/search/data/repositories/search_repository.dart

```

1  import 'package:dartz/dartz.dart';
2  import 'package:moean/core/network/remote/api_endpoints.dart';
3  import 'package:moean/core/network/remote/dio_helper.dart';
4  import 'package:moean/features/search/data/models/search_result_model.dart';
5
6  class SearchRepository {
7    Future<Either<String, List<SearchResultModel>>> search({
8      required String query,
9      String? type,
10     int limit = 20,
11   }) async {
12     final response = await DioHelper.getData(
13       url: searchApi,
14       query: {
15         'query': query,
16         if (type != null && type.isNotEmpty) 'type': type,
17         'limit': limit,
18       },
19     );
20
21     return response.fold(
22       (error) => Left(error),
23

```

```

23 (res) {
24   try {
25     final data = res.data;
26     final List<SearchResultModel> results = [];
27
28     if (data is Map<String, dynamic>) {
29       final dataBody = data['data'];
30
31       if (dataBody is Map<String, dynamic>) {
32         // Grouped response: { questions: [...], resources: [...], ... }
33         final typeKeys = {
34           'questions': 'questions',
35           'resources': 'resources',
36           'exams': 'exams',
37           'lessons': 'lessons',
38         };
39         typeKeys.forEach((key, label) {
40           final items = dataBody[key];
41           if (items is List) {
42             for (final item in items) {
43               if (item is Map<String, dynamic>) {
44                 results.add(SearchResultModel.fromJson(item, label));
45               }
46             }
47           }
48         });
49       } else if (dataBody is List) {
50         // Flat list response
51         for (final item in dataBody) {
52           if (item is Map<String, dynamic>) {
53             final itemType = (item['type'] ?? type ?? 'resources').toString();
54             results.add(SearchResultModel.fromJson(item, itemType));
55           }
56         }
57       }
58     }
59
60     return Right(results);
61   } catch (_) {
62     return const Left('حدث خطأ أثناء معالجة البيانات');
63   }
64 },
65 );
66 }
67 }
68

```

Lines • Dart 74

Feature / Search

#367 – lib/features/search/presentation/cubit/search_cubit.dart

```

1 import 'package:flutter_bloc/flutter_bloc.dart';
2 import 'package:moean/features/search/data/repositories/search_repository.dart';
3 import 'package:moean/features/search/presentation/cubit/search_state.dart';
4
5 class SearchCubit extends Cubit<SearchState> {
6   SearchCubit() : super(const SearchInitial());
7
8   final SearchRepository _repository = SearchRepository();
9
10  String _activeFilter = 'all';
11  String _lastQuery = '';
12
13  String get activeFilter => _activeFilter;
14
15  static const Map<String, String?> _filterTypeMap = {
16    'all': null,
17    'questions': 'questions',
18    'resources': 'resources',
19    'exams': 'exams',
20    'lessons': 'lessons',
21  };
22
23  Future<void> search(String query) async {
24    if (query.trim().isEmpty) return;
25

```

```

25     _lastQuery = query.trim();
26     if (!isClosed) emit(const SearchLoading());
27
28     final typeParam = _filterTypeMap[_activeFilter];
29
30     final result = await _repository.search(
31       query: _lastQuery,
32       type: typeParam,
33       limit: 20,
34     );
35     if (isClosed) return;
36
37     result.fold(
38       (error) {
39         if (!isClosed) emit(SearchError(message: error));
40       },
41       (items) {
42         if (!isClosed) {
43           if (items.isEmpty) {
44             emit(SearchEmpty(query: _lastQuery));
45           } else {
46             emit(SearchLoaded(
47               results: items,
48               query: _lastQuery,
49               activeFilter: _activeFilter,
50             ));
51           }
52         }
53       },
54     );
55   }
56
57   Future<void> changeFilter(String filter) async {
58     if (_activeFilter == filter) return;
59     _activeFilter = filter;
60
61     if (_lastQuery.isNotEmpty) {
62       await search(_lastQuery);
63     } else {
64       if (!isClosed) emit(const SearchInitial());
65     }
66   }
67
68   void reset() {
69     _lastQuery = '';
70     _activeFilter = 'all';
71     if (!isClosed) emit(const SearchInitial());
72   }
73 }
74

```

Lines • Dart 38

Feature / Search

#368 – lib/features/search/presentation/cubit/search_state.dart

```

1  import 'package:moean/features/search/data/models/search_result_model.dart';
2
3  abstract class SearchState {
4    const SearchState();
5  }
6
7  class SearchInitial extends SearchState {
8    const SearchInitial();
9  }
10
11 class SearchLoading extends SearchState {
12   const SearchLoading();
13 }
14
15 class SearchLoaded extends SearchState {
16   final List<SearchResultModel> results;
17   final String query;
18   final String activeFilter;
19
20   const SearchLoaded({

```

```

21     required this.results,
22     required this.query,
23     required this.activeFilter,
24   });
25 }
26
27 class SearchEmpty extends SearchState {
28   final String query;
29
30   const SearchEmpty({required this.query});
31 }
32
33 class SearchError extends SearchState {
34   final String message;
35
36   const SearchError({required this.message});
37 }
38

```

Lines • Dart 90

Feature / Search

#369 – lib/features/search/presentation/screen/search_screen.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7  import 'package:moean/core/utils/cubit/theme/theme_cubit.dart';
8  import 'package:moean/core/utils/cubit/theme/theme_state.dart';
9  import 'package:moean/features/search/presentation/cubit/search_cubit.dart';
10 import 'package:moean/features/search/presentation/widgets/search_filter_chips_widget.dart';
11 import 'package:moean/features/search/presentation/widgets/search_input_widget.dart';
12 import 'package:moean/features/search/presentation/widgets/search_results_body_widget.dart';
13
14 class SearchScreen extends StatelessWidget {
15   const SearchScreen({super.key});
16
17   @override
18   Widget build(BuildContext context) {
19     return BlocBuilder<ThemeCubit, ThemeState>(
20       builder: (context, themeState) {
21         return Directionality(
22           textDirection: TextDirection.rtl,
23           child: Scaffold(
24             backgroundColor: ColorsManager.background,
25             body: SafeArea(
26               child: Column(
27                 crossAxisAlignment: CrossAxisAlignment.stretch,
28                 children: [
29                   _SearchHeaderWidget(),
30                   verticalSpacel6,
31                   Padding(
32                     padding: const EdgeInsets.symmetric(horizontal: 16),
33                     child: SearchInputWidget(
34                       onSearch: (query) =>
35                         context.read<SearchCubit>().search(query),
36                       onClear: () => context.read<SearchCubit>().reset(),
37                     ),
38                   ),
39                   verticalSpacel6,
40                   const SearchFilterChipsWidget(),
41                   verticalSpace8,
42                   const Divider(height: 1),
43                   verticalSpace8,
44                   const Expanded(
45                     child: SingleChildScrollView(
46                       physics: BouncingScrollPhysics(),
47                       child: SearchResultsBodyWidget(),
48                     ),
49                   ),
50                 ],
51               ),
52             ),
53           ),
54         ),
55       ),
56     );
57   }
58 }
59

```

```

53     ),
54     );
55   },
56   );
57 }
58 }
59
60 class _SearchHeaderWidget extends StatelessWidget {
61   @override
62   Widget build(BuildContext context) {
63     return Container(
64       padding: const EdgeInsets.fromLTRB(8, 8, 16, 0),
65       child: Row(
66         children: [
67           IconButton(
68             onPressed: () => Navigator.of(context).pop(),
69             icon: Icon(
70               Icons.arrow_back_ios_rounded,
71               color: ColorsManager.textPrimary,
72               size: 20,
73             ),
74           ),
75           Expanded(
76             child: Text(
77               appTranslation().get('search_title'),
78               style: TextStyleManager.bold20.copyWith(
79                 color: ColorsManager.textPrimary,
80               ),
81               textAlign: TextAlign.center,
82             ),
83           ),
84           const SizedBox(width: 44),
85         ],
86       ),
87     );
88   }
89 }
90

```

Lines • Dart 63

Feature / Search

#370 – lib/features/search/presentation/widgets/search_empty_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6
7  class SearchEmptyWidget extends StatelessWidget {
8    final String query;
9
10    const SearchEmptyWidget({super.key, required this.query});
11
12    @override
13    Widget build(BuildContext context) {
14      return Center(
15        child: Padding(
16          padding: const EdgeInsets.symmetric(horizontal: 32, vertical: 48),
17          child: Column(
18            mainAxisAlignment: MainAxisAlignment.min,
19            children: [
20              Container(
21                width: 96,
22                height: 96,
23                decoration: BoxDecoration(
24                  color: ColorsManager.primaryColor.withValues(alpha: 0.08),
25                  shape: BoxShape.circle,
26                ),
27                child: Icon(
28                  Icons.search_off_rounded,
29                  size: 48,
30                  color: ColorsManager.primaryColor.withValues(alpha: 0.5),
31                ),
32              ),

```

```

33     verticalSpace24,
34     Text(
35       appTranslation().get('search_empty_title'),
36       style: TextStyleManager.bold18.copyWith(
37         color: ColorsManager.textPrimary,
38       ),
39       textAlign: TextAlign.center,
40     ),
41     verticalSpace8,
42     Text(
43       appTranslation().get('search_empty_subtitle'),
44       style: TextStyleManager.regular14.copyWith(
45         color: ColorsManager.secondaryText,
46       ),
47       textAlign: TextAlign.center,
48     ),
49     verticalSpace8,
50     Text(
51       "$query",
52       style: TextStyleManager.bold14.copyWith(
53         color: ColorsManager.primaryColor,
54       ),
55       textAlign: TextAlign.center,
56     ),
57   ],
58 ),
59 ),
60 );
61 }
62 }
63

```

Lines • Dart 99

Feature / Search

#371 — lib/features/search/presentation/widgets/search_filter_chips_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/features/search/presentation/cubit/search_cubit.dart';
7  import 'package:moean/features/search/presentation/cubit/search_state.dart';
8
9  class SearchFilterChipsWidget extends StatelessWidget {
10     const SearchFilterChipsWidget({super.key});
11
12     static const List<Map<String, String>> _filters = [
13       {'key': 'all', 'labelKey': 'search_filter_all'},
14       {'key': 'questions', 'labelKey': 'search_filter_questions'},
15       {'key': 'resources', 'labelKey': 'search_filter_resources'},
16       {'key': 'exams', 'labelKey': 'search_filter_exams'},
17       {'key': 'lessons', 'labelKey': 'search_filter_lessons'},
18     ];
19
20     @override
21     Widget build(BuildContext context) {
22       return BlocBuilder<SearchCubit, SearchState>(
23         builder: (context, state) {
24           final cubit = context.read<SearchCubit>();
25           final activeFilter = cubit.activeFilter;
26           return SingleChildScrollView(
27             scrollDirection: Axis.horizontal,
28             physics: const BouncingScrollPhysics(),
29             padding: const EdgeInsets.symmetric(horizontal: 16),
30             child: Row(
31               children: _filters.map((filter) {
32                 final isActive = activeFilter == filter['key'];
33                 return Padding(
34                   padding: const EdgeInsetsDirectional.only(end: 8),
35                   child: _FilterChipItem(
36                     label: appTranslation().get(filter['labelKey']!),
37                     isActive: isActive,
38                     onTap: () => cubit.changeFilter(filter['key']!),
39                   ),

```

```

40         );
41         }).toList(),
42     ),
43 );
44 },
45 );
46 }
47 }
48
49 class _FilterChipItem extends StatelessWidget {
50   final String label;
51   final bool isActive;
52   final VoidCallback onTap;
53
54   const _FilterChipItem({
55     required this.label,
56     required this.isActive,
57     required this.onTap,
58   });
59
60   @override
61   Widget build(BuildContext context) {
62     return GestureDetector(
63       onTap: onTap,
64       child: AnimatedContainer(
65         duration: const Duration(milliseconds: 220),
66         curve: Curves.easeInOut,
67         padding: const EdgeInsets.symmetric(horizontal: 18, vertical: 10),
68         decoration: BoxDecoration(
69           color: isActive
70             ? ColorsManager.primaryColor
71             : ColorsManager.surfacePrimary,
72           borderRadius: BorderRadius.circular(30),
73           border: Border.all(
74             color: isActive
75               ? ColorsManager.primaryColor
76               : ColorsManager.borderColor,
77           ),
78           boxShadow: isActive
79             ? [
80               BoxShadow(
81                 color:
82                   ColorsManager.primaryColor.withValues(alpha: 0.25),
83                 blurRadius: 8,
84                 offset: const Offset(0, 3),
85               ),
86             ]
87             : null,
88         ),
89       child: Text(
90         label,
91         style: TextStyleManager.bold13.copyWith(
92           color: isActive ? ColorsManager.white : ColorsManager.textPrimary,
93         ),
94       ),
95     ),
96   );
97 }
98 }
99

```

Lines • Dart 97

Feature / Search

#372 – lib/features/search/presentation/widgets/search_input_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5
6  class SearchInputWidget extends StatefulWidget {
7    final ValueChanged<String> onSearch;
8    final VoidCallback? onClear;
9
10   const SearchInputWidget({

```



```

11     super.key,
12     required this.onSearch,
13     this.onClear,
14   });
15
16   @override
17   State<SearchInputWidget> createState() => _SearchInputWidgetState();
18 }
19
20 class _SearchInputWidgetState extends State<SearchInputWidget> {
21   final TextEditingController _controller = TextEditingController();
22   bool _hasText = false;
23
24   @override
25   void initState() {
26     super.initState();
27     _controller.addListener(_onTextChanged);
28   }
29
30   void _onTextChanged() {
31     final hasText = _controller.text.isNotEmpty;
32     if (hasText != _hasText) {
33       setState(() => _hasText = hasText);
34     }
35   }
36
37   void _submit() {
38     final query = _controller.text.trim();
39     if (query.isNotEmpty) widget.onSearch(query);
40   }
41
42   void _clear() {
43     _controller.clear();
44     widget.onClear?.call();
45   }
46
47   @override
48   void dispose() {
49     _controller.removeListener(_onTextChanged);
50     _controller.dispose();
51     super.dispose();
52   }
53
54   @override
55   Widget build(BuildContext context) {
56     return TextField(
57       controller: _controller,
58       textDirection: TextDirection.rtl,
59       style: TextStyleManager.regular14.copyWith(
60         color: ColorsManager.textPrimary,
61       ),
62       decoration: InputDecoration(
63         hintText: appTranslation().get('search_hint'),
64         hintStyle: TextStyleManager.regular14.copyWith(
65           color: ColorsManager.placeholder,
66         ),
67         hintTextDirection: TextDirection.rtl,
68         prefixIcon: GestureDetector(
69           onTap: _submit,
70           child: Icon(
71             Icons.search_rounded,
72             color: ColorsManager.primaryColor,
73             size: 22,
74           ),
75         ),
76         suffixIcon: _hasText
77           ? GestureDetector(
78             onTap: _clear,
79             child: Icon(
80               Icons.close_rounded,
81               color: ColorsManager.placeholder,
82               size: 20,
83             ),
84           )
85         : null,

```

```

86         border: InputBorder.none,
87         contentPadding: const EdgeInsets.symmetric(
88             horizontal: 16,
89             vertical: 16,
90         ),
91     ),
92     textInputAction: TextInputAction.search,
93     onSubmitted: (_) => _submit(),
94 );
95 }
96 }
97

```

Lines • Dart 182

Feature / Search

#373 – lib/features/search/presentation/widgets/search_result_item_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6  import 'package:moean/features/search/data/models/search_result_model.dart';
7
8  class SearchResultItemWidget extends StatelessWidget {
9      final SearchResultModel result;
10
11      const SearchResultItemWidget({super.key, required this.result});
12
13      @override
14      Widget build(BuildContext context) {
15          return Container(
16              margin: const EdgeInsets.only(bottom: 12),
17              decoration: BoxDecoration(
18                  color: ColorsManager.surfacePrimary,
19                  borderRadius: BorderRadius.circular(16),
20                  border: Border.all(color: ColorsManager.borderColor),
21                  boxShadow: [
22                      BoxShadow(
23                          color: ColorsManager.primaryColor.withValues(alpha: 0.04),
24                          blurRadius: 8,
25                          offset: const Offset(0, 2),
26                      ),
27                  ],
28              ),
29              child: Padding(
30                  padding: const EdgeInsets.all(16),
31                  child: Column(
32                      crossAxisAlignment: CrossAxisAlignment.stretch,
33                      children: [
34                          Row(
35                              children: [
36                                  _TypeBadgeWidget(type: result.type),
37                                  const Spacer(),
38                                  if (result.difficulty != null && result.difficulty!.isNotEmpty)
39                                      _DifficultyBadgeWidget(difficulty: result.difficulty!),
40                              ],
41                          ),
42                          verticalSpace12,
43                          Text(
44                              result.title,
45                              style: TextStylesManager.bold14.copyWith(
46                                  color: ColorsManager.textPrimary,
47                              ),
48                          ),
49                          if (result.subtitle != null) ...[
50                              verticalSpace4,
51                              Text(
52                                  result.subtitle!,
53                                  style: TextStylesManager.regular12.copyWith(
54                                      color: ColorsManager.secondaryText,
55                                  ),
56                              ),
57                          ],
58                          if (result.options != null && result.options!.isNotEmpty) ...[

```

```

59         verticalSpace12,
60         ...result.options!.map(
61             (option) => _OptionWidget(
62                 option: option,
63                 isCorrect: result.correctAnswer != null &&
64                     option.contains(result.correctAnswer!),
65             ),
66         ),
67     ],
68 ],
69 ),
70 ),
71 );
72 }
73 }
74
75 class _TypeBadgeWidget extends StatelessWidget {
76     final String type;
77
78     const _TypeBadgeWidget({required this.type});
79
80     String _getLabel(String type) {
81         switch (type) {
82             case 'questions':
83                 return appTranslation().get('search_filter_questions');
84             case 'resources':
85                 return appTranslation().get('search_filter_resources');
86             case 'exams':
87                 return appTranslation().get('search_filter_exams');
88             case 'lessons':
89                 return appTranslation().get('search_filter_lessons');
90             default:
91                 return type;
92         }
93     }
94
95     @override
96     Widget build(BuildContext context) {
97         return Container(
98             padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
99             decoration: BoxDecoration(
100                 color: ColorsManager.primaryColor.withValues(alpha: 0.1),
101                 borderRadius: BorderRadius.circular(20),
102             ),
103             child: Text(
104                 _getLabel(type),
105                 style: TextStylesManager.medium10.copyWith(
106                     color: ColorsManager.primaryColor,
107                 ),
108             ),
109         );
110     }
111 }
112
113 class _DifficultyBadgeWidget extends StatelessWidget {
114     final String difficulty;
115
116     const _DifficultyBadgeWidget({required this.difficulty});
117
118     @override
119     Widget build(BuildContext context) {
120         return Container(
121             padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
122             decoration: BoxDecoration(
123                 color: ColorsManager.secondaryColor.withValues(alpha: 0.12),
124                 borderRadius: BorderRadius.circular(20),
125             ),
126             child: Text(
127                 difficulty,
128                 style: TextStylesManager.medium10.copyWith(
129                     color: ColorsManager.secondaryColor,
130                 ),
131             ),
132         );
133     }

```

```

134 }
135
136 class _OptionWidget extends StatelessWidget {
137   final String option;
138   final bool isCorrect;
139
140   const _OptionWidget({required this.option, required this.isCorrect});
141
142   @override
143   Widget build(BuildContext context) {
144     return Container(
145       margin: const EdgeInsets.only(bottom: 6),
146       padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 10),
147       decoration: BoxDecoration(
148         color: isCorrect
149           ? ColorsManager.primaryColor.withValues(alpha: 0.08)
150           : ColorsManager.background,
151         borderRadius: BorderRadius.circular(10),
152         border: Border.all(
153           color: isCorrect
154             ? ColorsManager.primaryColor.withValues(alpha: 0.4)
155             : ColorsManager.borderColor,
156         ),
157       ),
158       child: Row(
159         children: [
160           Expanded(
161             child: Text(
162               option,
163               style: TextStylesManager.regular13.copyWith(
164                 color: isCorrect
165                   ? ColorsManager.primaryColor
166                   : ColorsManager.textPrimary,
167             ),
168           ),
169         ],
170         if (isCorrect)
171           Text(
172             '${appTranslation().get('search_correct_answer')}',
173             style: TextStylesManager.bold10.copyWith(
174               color: ColorsManager.primaryColor,
175             ),
176           ),
177       ],
178     ),
179   );
180 }
181
182

```

Lines • Dart 186 Feature / Search #374 — lib/features/search/presentation/widgets/search_results_body_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/primary/conditional_builder.dart';
7  import 'package:moean/core/utils/constants/spacing.dart';
8  import 'package:moean/features/search/presentation/cubit/search_cubit.dart';
9  import 'package:moean/features/search/presentation/cubit/search_state.dart';
10 import 'package:moean/features/search/presentation/widgets/search_empty_widget.dart';
11 import 'package:moean/features/search/presentation/widgets/search_result_item_widget.dart';
12
13 class SearchResultsBodyWidget extends StatelessWidget {
14   const SearchResultsBodyWidget({super.key});
15
16   @override
17   Widget build(BuildContext context) {
18     return BlocBuilder<SearchCubit, SearchState>(
19       builder: (context, state) {
20         final isLoading = state is SearchLoading;
21         final isError = state is SearchError;

```

```

22     final isEmpty = state is SearchEmpty;
23     final isLoading = state is SearchLoaded;
24     final isInitial = state is SearchInitial;
25
26     if (isInitial) {
27         return _SearchInitialHintWidget();
28     }
29
30     return ConditionalBuilder(
31         loadingState: isLoading,
32         errorState: isError,
33         emptyState: isEmpty,
34         errorBuilder: (ctx) => _SearchErrorWidget(
35             message: (state as SearchError).message,
36         ),
37         emptyBuilder: (ctx) => SearchEmptyWidget(
38             query: (state as SearchEmpty).query,
39         ),
40         successBuilder: (ctx) {
41             if (!isLoading) return const SizedBox.shrink();
42             final loaded = state as SearchLoaded;
43             return Padding(
44                 padding: const EdgeInsets.symmetric(horizontal: 16),
45                 child: Column(
46                     crossAxisAlignment: CrossAxisAlignment.stretch,
47                     children: [
48                         verticalSpace12,
49                         _SearchResultsCountWidget(
50                             count: loaded.results.length,
51                             query: loaded.query,
52                         ),
53                         verticalSpace16,
54                         ListView.builder(
55                             shrinkWrap: true,
56                             physics: const NeverScrollableScrollPhysics(),
57                             itemCount: loaded.results.length,
58                             itemBuilder: (context, index) => SearchResultItemWidget(
59                                 result: loaded.results[index],
60                             ),
61                         ),
62                     ],
63                 ),
64             );
65         },
66     );
67 },
68 );
69 }
70 }
71
72 class _SearchInitialHintWidget extends StatelessWidget {
73     @override
74     Widget build(BuildContext context) {
75         return Center(
76             child: Padding(
77                 padding: const EdgeInsets.only(top: 72),
78                 child: Column(
79                     mainAxisAlignment: MainAxisAlignment.min,
80                     children: [
81                         Icon(
82                             Icons.manage_search_rounded,
83                             size: 72,
84                             color: ColorsManager.primaryColor.withValues(alpha: 0.25),
85                         ),
86                         verticalSpace16,
87                         Text(
88                             appTranslation().get('search_hint_prompt'),
89                             style: TextStylesManager.regular14.copyWith(
90                                 color: ColorsManager.secondaryText,
91                             ),
92                             textAlign: TextAlign.center,
93                         ),
94                     ],
95                 ),
96             ),

```

```

97     );
98   }
99 }
100
101 class _SearchErrorWidget extends StatelessWidget {
102   final String message;
103
104   const _SearchErrorWidget({required this.message});
105
106   @override
107   Widget build(BuildContext context) {
108     return Center(
109       child: Padding(
110         padding: const EdgeInsets.all(32),
111         child: Column(
112           mainAxisAlignment: MainAxisAlignment.min,
113           children: [
114             Icon(
115               Icons.error_outline_rounded,
116               size: 56,
117               color: ColorsManager.errorColor.withValues(alpha: 0.6),
118             ),
119             verticalSpace16,
120             Text(
121               appTranslation().get('search_error_title'),
122               style: TextStylesManager.bold16.copyWith(
123                 color: ColorsManager.textPrimary,
124               ),
125             ),
126             verticalSpace8,
127             Text(
128               message,
129               style: TextStylesManager.regular13.copyWith(
130                 color: ColorsManager.secondaryText,
131               ),
132               textAlign: TextAlign.center,
133             ),
134           ],
135         ),
136       ),
137     );
138   }
139 }
140
141 class _SearchResultsCountWidget extends StatelessWidget {
142   final int count;
143   final String query;
144
145   const _SearchResultsCountWidget({
146     required this.count,
147     required this.query,
148   });
149
150   @override
151   Widget build(BuildContext context) {
152     return Row(
153       children: [
154         Container(
155           padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 5),
156           decoration: BoxDecoration(
157             color: ColorsManager.primaryColor,
158             borderRadius: BorderRadius.circular(20),
159           ),
160           child: Text(
161             '($count)',
162             style: TextStylesManager.bold13.copyWith(
163               color: ColorsManager.white,
164             ),
165           ),
166         ),
167         horizontalSpace8,
168         Expanded(
169           child: Text(
170             appTranslation().get('search_results_for'),
171             style: TextStylesManager.regular13.copyWith(

```

```

172         color: ColorsManager.secondaryText,
173     ),
174 ),
175 ),
176 Text(
177     "$query",
178     style: TextStylesManager.bold13.copyWith(
179         color: ColorsManager.primaryColor,
180     ),
181 ),
182 ],
183 );
184 }
185 }
186

```

Lines • Dart 162

Feature / Verify Email

#375 – lib/features/verify_email/presentation/cubit/verify_email_cubit.dart

```

1  import 'dart:async';
2  import 'package:flutter/material.dart';
3  import 'package:flutter_bloc/flutter_bloc.dart';
4  import 'package:moean/core/network/remote/api_service.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/features/verify_email/presentation/cubit/verify_email_state.dart';
7
8  class VerifyEmailCubit extends Cubit<VerifyEmailState> with WidgetsBindingObserver {
9      final String email;
10
11      VerifyEmailCubit({required this.email}) : super(VerifyEmailInitialState()) {
12          WidgetsBinding.instance.addObserver(this);
13          _startPolling();
14      }
15
16      static VerifyEmailCubit get(BuildContext context) => BlocProvider.of(context);
17
18      int cooldownSeconds = 0;
19      Timer? _cooldownTimer;
20      Timer? _pollingTimer;
21      bool isResending = false;
22      bool isChecking = false;
23
24      void _startPolling() {
25          _pollingTimer?.cancel();
26          _pollingTimer = Timer.periodic(const Duration(seconds: 4), (timer) {
27              if (state is VerifyEmailSuccessState) {
28                  timer.cancel();
29              } else {
30                  checkVerificationStatus(isSilent: true);
31              }
32          });
33      }
34
35      @override
36      void didChangeAppLifecycleState(AppLifecycleState state) {
37          if (state == AppLifecycleState.resumed && this.state is! VerifyEmailSuccessState) {
38              checkVerificationStatus(isSilent: true);
39          }
40      }
41
42      void startCooldown([int seconds = 60]) {
43          cooldownSeconds = seconds;
44          _cooldownTimer?.cancel();
45          if (!isClosed) emit(VerifyEmailCooldownTickState(cooldownSeconds: cooldownSeconds));
46          _cooldownTimer = Timer.periodic(const Duration(seconds: 1), (timer) {
47              if (cooldownSeconds <= 1) {
48                  timer.cancel();
49                  cooldownSeconds = 0;
50                  if (!isClosed) emit(VerifyEmailCooldownTickState(cooldownSeconds: 0));
51              } else {
52                  cooldownSeconds--;
53                  if (!isClosed) emit(VerifyEmailCooldownTickState(cooldownSeconds: cooldownSeconds));
54              }
55          });
56      }
57

```

```

56     }
57
58     Future<void> resendVerificationEmail() async {
59         if (cooldownSeconds > 0 || isResending) return;
60
61         isResending = true;
62         if (!isClosed) emit(VerifyEmailResendLoadingState());
63
64         try {
65             final result = await ApiService.resendEmailVerification(email: email);
66             if (isClosed) return;
67
68             result.fold(
69                 (error) {
70                     isResending = false;
71                     if (!isClosed) emit(VerifyEmailErrorState(message: error));
72                 },
73                 (data) {
74                     isResending = false;
75                     if (data['already_verified'] == true) {
76                         _pollingTimer?.cancel();
77                         if (!isClosed) emit(VerifyEmailSuccessState());
78                         return;
79                     }
80                     startCooldown(60);
81                     final message = data['message'] as String? ?? appTranslation().get('resend_verification_sent');
82                     if (!isClosed) emit(VerifyEmailResendSuccessState(message: message));
83                 },
84             );
85         } catch (e) {
86             isResending = false;
87             if (!isClosed) emit(VerifyEmailErrorState(message: e.toString()));
88         }
89     }
90
91     Future<void> checkVerificationStatus({bool isSilent = false}) async {
92         if (isChecking || state is VerifyEmailSuccessState) return;
93         isChecking = true;
94
95         if (!isSilent && !isClosed) emit(VerifyEmailLoadingState());
96
97         try {
98             final result = await ApiService.getMe();
99             isChecking = false;
100             if (isClosed) return;
101
102             result.fold(
103                 (error) {
104                     if (!isSilent && !isClosed) emit(VerifyEmailErrorState(message: error));
105                 },
106                 (user) {
107                     if (user.isEmailVerified) {
108                         _pollingTimer?.cancel();
109                         if (!isClosed) emit(VerifyEmailSuccessState());
110                     } else {
111                         if (!isSilent && !isClosed) {
112                             emit(VerifyEmailErrorState(message: appTranslation().get('email_unverified_banner')));
113                         }
114                     }
115                 },
116             );
117         } catch (e) {
118             isChecking = false;
119             if (!isSilent && !isClosed) emit(VerifyEmailErrorState(message: e.toString()));
120         }
121     }
122
123     Future<void> verifyEmailToken({
124         required String id,
125         required String hash,
126         required String expires,
127         required String signature,
128     }) async {
129         if (!isClosed) emit(VerifyEmailLoadingState());
130

```



```

131     try {
132       final result = await ApiService.verifyEmailToken(
133         id: id,
134         hash: hash,
135         expires: expires,
136         signature: signature,
137       );
138       if (isClosed) return;
139
140       result.fold(
141         (error) {
142           if (!isClosed) emit(VerifyEmailErrorState(message: error));
143         },
144         (user) {
145           _pollingTimer?.cancel();
146           if (!isClosed) emit(VerifyEmailSuccessState());
147         },
148       );
149     } catch (e) {
150       if (!isClosed) emit(VerifyEmailErrorState(message: e.toString()));
151     }
152   }
153
154   @override
155   Future<void> close() {
156     WidgetsBinding.instance.removeObserver(this);
157     _cooldownTimer?.cancel();
158     _pollingTimer?.cancel();
159     return super.close();
160   }
161 }
162

```

Lines • Dart 25

Feature / Verify Email

#376 – lib/features/verify_email/presentation/cubit/verify_email_state.dart

```

1  abstract class VerifyEmailState {}
2
3  class VerifyEmailInitialState extends VerifyEmailState {}
4
5  class VerifyEmailLoadingState extends VerifyEmailState {}
6
7  class VerifyEmailResendLoadingState extends VerifyEmailState {}
8
9  class VerifyEmailResendSuccessState extends VerifyEmailState {
10     final String message;
11     VerifyEmailResendSuccessState({required this.message});
12   }
13
14  class VerifyEmailCooldownTickState extends VerifyEmailState {
15     final int cooldownSeconds;
16     VerifyEmailCooldownTickState({required this.cooldownSeconds});
17   }
18
19  class VerifyEmailSuccessState extends VerifyEmailState {}
20
21  class VerifyEmailErrorState extends VerifyEmailState {
22     final String message;
23     VerifyEmailErrorState({required this.message});
24   }
25

```

Lines • Dart 115

Feature / Verify Email

#377 – lib/features/verify_email/presentation/screen/verify_email_screen.dart

```

1
2  import 'package:flutter/material.dart';
3  import 'package:flutter_bloc/flutter_bloc.dart';
4  import 'package:moean/core/theme/colors.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7  import 'package:moean/features/verify_email/presentation/cubit/verify_email_cubit.dart';
8  import 'package:moean/features/verify_email/presentation/cubit/verify_email_state.dart';
9  import 'package:moean/features/verify_email/presentation/widgets/verify_email_actions_widget.dart';

```

```

10 import 'package:moean/features/verify_email/presentation/widgets/verify_email_badge_widget.dart';
11 import 'package:moean/features/verify_email/presentation/widgets/verify_email_header_widget.dart';
12 import 'package:moean/features/verify_email/presentation/widgets/verify_email_steps_widget.dart';
13 import 'package:moean/features/verify_email/presentation/widgets/verify_email_success_widget.dart';
14
15 class VerifyEmailScreen extends StatelessWidget {
16   final String email;
17
18   const VerifyEmailScreen({
19     super.key,
20     required this.email,
21   });
22
23   @override
24   Widget build(BuildContext context) {
25     return BlocProvider(
26       create: (_) => VerifyEmailCubit(email: email),
27       child: BlocListener<VerifyEmailCubit, VerifyEmailState>(
28         listener: _onStateChanged,
29         child: Scaffold(
30           backgroundColor: ColorsManager.background,
31           appBar: AppBar(
32             backgroundColor: Colors.transparent,
33             elevation: 0,
34             leading: BlocBuilder<VerifyEmailCubit, VerifyEmailState>(
35               buildWhen: (previous, current) =>
36                 current is VerifyEmailSuccessState ||
37                 previous is VerifyEmailSuccessState,
38               builder: (context, state) {
39                 if (state is VerifyEmailSuccessState) {
40                   return const SizedBox.shrink();
41                 }
42                 return IconButton(
43                   onPressed: () {
44                     Navigator.pop(context);
45                   },
46                   icon: Icon(
47                     Icons.arrow_forward_ios_rounded,
48                     size: 16,
49                     color: ColorsManager.primaryColor,
50                   ),
51                 );
52               },
53             ),
54           ),
55           body: SafeArea(
56             child: Center(
57               child: SingleChildScrollView(
58                 physics: const BouncingScrollPhysics(),
59                 padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
60                 child: BlocBuilder<VerifyEmailCubit, VerifyEmailState>(
61                   buildWhen: (previous, current) =>
62                     current is VerifyEmailSuccessState ||
63                     current is VerifyEmailInitialState,
64                   builder: (context, state) {
65                     if (state is VerifyEmailSuccessState) {
66                       return const VerifyEmailSuccessWidget();
67                     }
68
69                     return Column(
70                       crossAxisAlignment: CrossAxisAlignment.center,
71                       children: [
72                         const VerifyEmailBadgeWidget(),
73                         verticalSpace20,
74                         VerifyEmailHeaderWidget(email: email),
75                         verticalSpace24,
76                         const VerifyEmailStepsWidget(),
77                         verticalSpace28,
78                         const VerifyEmailActionsWidget(),
79                         verticalSpace20,
80                       ],
81                     );
82                   },
83                 ),
84               ),

```

```

85         ),
86     ),
87 ),
88 ),
89 );
90 }
91
92 void _onStateChanged(BuildContext context, VerifyEmailState state) {
93   if (state is VerifyEmailErrorState) {
94     ScaffoldMessenger.of(context).showSnackBar(
95       SnackBar(
96         content: Text(state.message),
97         backgroundColor: ColorsManager.errorColor,
98       ),
99     );
100   }
101   if (state is VerifyEmailResendSuccessState) {
102     ScaffoldMessenger.of(context).showSnackBar(
103       SnackBar(
104         content: Text(
105           state.message.isNotEmpty
106             ? state.message
107             : appTranslation().get('resend_verification_sent'),
108         ),
109         backgroundColor: ColorsManager.primaryColor,
110       ),
111     );
112   }
113 }
114 }
115

```

Lines • 137
Dart

Feature / Verify #378 -

Email lib/features/verify_email/presentation/widgets/email_verification_banner.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/network/remote/api_service.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/spacing.dart';
7
8  class EmailVerificationBanner extends StatefulWidget {
9    final bool isEmailVerified;
10    final String? email;
11
12    const EmailVerificationBanner({
13      super.key,
14      required this.isEmailVerified,
15      this.email,
16    });
17
18    @override
19    State<EmailVerificationBanner> createState() =>
20      _EmailVerificationBannerState();
21  }
22
23  class _EmailVerificationBannerState extends State<EmailVerificationBanner> {
24    bool _dismissed = false;
25    bool _isResending = false;
26
27    Future<void> _handleResend() async {
28      if (_isResending) return;
29      setState(() => _isResending = true);
30
31      final result =
32        await ApiService.resendEmailVerification(email: widget.email);
33      if (!mounted) return;
34      setState(() => _isResending = false);
35
36      result.fold(
37        (error) {
38          ScaffoldMessenger.of(context).showSnackBar(

```

```

39         SnackBar(
40             content: Text(error),
41             backgroundColor: ColorsManager.errorColor,
42         ),
43     );
44 },
45 (data) {
46     final message = data['message'] as String? ??
47       appTranslation().get('resend_verification_sent');
48     ScaffoldMessenger.of(context).showSnackBar(
49       SnackBar(
50         content: Text(message),
51         backgroundColor: ColorsManager.primaryColor,
52       ),
53     );
54   },
55 );
56 }
57
58 @override
59 Widget build(BuildContext context) {
60   if (widget.isEmailVerified || !_dismissed) return const SizedBox.shrink();
61
62   return Container(
63     width: double.infinity,
64     padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 8),
65     decoration: const BoxDecoration(
66       color: Color(0xFFFFF8E7),
67       border: Border(
68         bottom: BorderSide(
69           color: Color(0xFFF4E3B2),
70         ),
71     ),
72   ),
73   child: SafeArea(
74     bottom: false,
75     child: Row(
76       children: [
77         const Icon(
78           Icons.warning_amber_rounded,
79           color: Color(0xFFC98D14),
80           size: 18,
81         ),
82         horizontalSpace8,
83         Expanded(
84           child: Text(
85             appTranslation().get('email_unverified_banner'),
86             style: TextStyleManager.bold12.copyWith(
87               color: const Color(0xFF8A5807),
88             ),
89         ),
90       ],
91     InkWell(
92       onTap: _isResending ? null : _handleResend,
93       borderRadius: BorderRadius.circular(6),
94       child: Container(
95         padding: const EdgeInsets.symmetric(
96           horizontal: 10,
97           vertical: 5,
98         ),
99         decoration: BoxDecoration(
100           color: const Color(0xFFE2AD3B),
101           borderRadius: BorderRadius.circular(6),
102         ),
103         child: _isResending
104           ? const SizedBox(
105             width: 12,
106             height: 12,
107             child: CircularProgressIndicator(
108               strokeWidth: 2,
109               color: Colors.white,
110             ),
111           )
112           : Text(
113             appTranslation().get('resend'),

```

```

114         style: TextStylesManager.bold10.copyWith(
115             color: Colors.white,
116         ),
117     ),
118 ),
119 ),
120 horizontalSpace4,
121 IconButton(
122     icon: const Icon(
123         Icons.close,
124         size: 16,
125         color: Color(0xFF8A5807),
126     ),
127     onPressed: () => setState(() => _dismissed = true),
128     padding: EdgeInsets.zero,
129     constraints: const BoxConstraints(),
130 ),
131 ],
132 ),
133 ),
134 );
135 }
136 }
137

```

Lines • 110
Dart

Feature / Verify #379 -

Email lib/features/verify_email/presentation/widgets/verify_email_actions_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_bloc/flutter_bloc.dart';
3  import 'package:moean/core/theme/colors.dart';
4  import 'package:moean/core/theme/text_styles.dart';
5  import 'package:moean/core/utils/constants/constants.dart';
6  import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
7  import 'package:moean/core/utils/constants/spacing.dart';
8  import 'package:moean/features/verify_email/presentation/cubit/verify_email_cubit.dart';
9  import 'package:moean/features/verify_email/presentation/cubit/verify_email_state.dart';
10
11  class VerifyEmailActionsWidget extends StatelessWidget {
12      const VerifyEmailActionsWidget({super.key});
13
14      @override
15      Widget build(BuildContext context) {
16          return BlocBuilder<VerifyEmailCubit, VerifyEmailState>(
17              buildWhen: (previous, current) =>
18                  current is VerifyEmailCooldownTickState ||
19                  current is VerifyEmailResendLoadingState ||
20                  current is VerifyEmailResendSuccessState ||
21                  current is VerifyEmailErrorState,
22              builder: (context, state) {
23                  final cubit = VerifyEmailCubit.get(context);
24                  final bool isCooldown = cubit.cooldownSeconds > 0;
25                  final bool isLoading = cubit.isResending;
26
27                  final String resendLabel = isCooldown
28                      ? appTranslation()
29                        .get('resend_verification_cooldown')
30                        .replaceAll('{seconds}', cubit.cooldownSeconds.toString())
31                      : appTranslation().get('resend_verification_link');
32
33                  return Column(
34                      children: [
35                          SizedBox(
36                              width: double.infinity,
37                              height: 52,
38                              child: OutlinedButton(
39                                  onPressed: isCooldown || isLoading
40                                      ? null
41                                      : () {
42                                          cubit.resendVerificationEmail();
43                                      },
44                              style: OutlinedButton.styleFrom(
45                                  side: BorderSide(

```

```

46         color: isCooldown
47         ? ColorsManager.placeholder.withValues(alpha: 0.5)
48         : ColorsManager.primaryColor,
49         width: 1.5,
50       ),
51       shape: RoundedRectangleBorder(
52         borderRadius: BorderRadius.circular(24),
53       ),
54       backgroundColor: Colors.transparent,
55     ),
56     child: isLoading
57       ? SizedBox(
58         width: 20,
59         height: 20,
60         child: CircularProgressIndicator(
61           strokeWidth: 2,
62           color: ColorsManager.primaryColor,
63         ),
64       )
65       : Row(
66         mainAxisAlignment: MainAxisAlignment.center,
67         children: [
68           Icon(
69             Icons.refresh_rounded,
70             size: 20,
71             color: isCooldown
72               ? ColorsManager.placeholder
73               : ColorsManager.primaryColor,
74           ),
75           horizontalSpace8,
76           Text(
77             resendLabel,
78             style: TextStylesManager.bold14.copyWith(
79               color: isCooldown
80                 ? ColorsManager.placeholder
81                 : ColorsManager.primaryColor,
82             ),
83           ),
84         ],
85       ),
86     ),
87   ),
88   verticalSpace14,
89   PrimaryElevatedButton(
90     text: appTranslation().get('continue_to_platform'),
91     height: 52,
92     radius: 24,
93     isLoading: cubit.isChecking,
94     backgroundColor: ColorsManager.primaryColor,
95     icon: const Icon(
96       Icons.arrow_back,
97       color: Colors.white,
98       size: 20,
99     ),
100    onPressed: () {
101      cubit.checkVerificationStatus();
102    },
103  ),
104 ],
105 );
106 },
107 );
108 }
109 }
110

```

Lines • 66
Dart

Feature / Verify #380 -

Email lib/features/verify_email/presentation/widgets/verify_email_badge_widget.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';

```

```

5 import 'package:moean/core/utils/constants/spacing.dart';
6
7 class VerifyEmailBadgeWidget extends StatelessWidget {
8   const VerifyEmailBadgeWidget({super.key});
9
10  @override
11  Widget build(BuildContext context) {
12    return Column(
13      mainAxisAlignment: MainAxisAlignment.min,
14      children: [
15        Container(
16          width: 100,
17          height: 100,
18          decoration: BoxDecoration(
19            shape: BoxShape.circle,
20            color: ColorsManager.verifyMailIconBg,
21            boxShadow: [
22              BoxShadow(
23                color: ColorsManager.brandGold.withValues(alpha: 0.2),
24                blurRadius: 32,
25                spreadRadius: 8,
26              ),
27            ],
28          ),
29          child: const Center(
30            child: Icon(
31              Icons.mark_email_read_outlined,
32              size: 48,
33              color: ColorsManager.verifyMailIcon,
34            ),
35          ),
36        ),
37        verticalSpace16,
38        Container(
39          padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 6),
40          decoration: BoxDecoration(
41            color: ColorsManager.verifyMailBadgeBg,
42            borderRadius: BorderRadius.circular(20),
43          ),
44          child: Row(
45            mainAxisAlignment: MainAxisAlignment.min,
46            children: [
47              const Icon(
48                Icons.mail_outline_rounded,
49                size: 16,
50                color: ColorsManager.verifyMailBadgeText,
51              ),
52              horizontalSpace6,
53              Text(
54                appTranslation().get('verify_email_badge'),
55                style: TextStyleManager.bold12.copyWith(
56                  color: ColorsManager.verifyMailBadgeText,
57                ),
58              ),
59            ],
60          ),
61        ),
62      ],
63    );
64  }
65 }
66

```

Lines • 59
Dart

Feature / Verify #381 –
Email

[lib/features/verify_email/presentation/widgets/verify_email_header_widget.dart](#)

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/spacing.dart';
6
7 class VerifyEmailHeaderWidget extends StatelessWidget {

```

```

8   final String email;
9
10  const VerifyEmailHeaderWidget({
11    super.key,
12    required this.email,
13  });
14
15  @override
16  Widget build(BuildContext context) {
17    return Column(
18      mainAxisAlignment: MainAxisAlignment.min,
19      crossAxisAlignment: CrossAxisAlignment.center,
20      children: [
21        Text(
22          appTranslation().get('verify_email_title'),
23          style: TextStylesManager.bold26.copyWith(
24            color: ColorsManager.verifySuccessDark,
25          ),
26          textAlign: TextAlign.center,
27        ),
28        verticalSpace8,
29        Text(
30          appTranslation().get('verify_email_subtitle'),
31          style: TextStylesManager.regular14.copyWith(
32            color: ColorsManager.placeholder,
33            height: 1.4,
34          ),
35          textAlign: TextAlign.center,
36        ),
37        verticalSpace12,
38        Container(
39          padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 10),
40          decoration: BoxDecoration(
41            color: ColorsManager.verifyEmailContainerBg,
42            borderRadius: BorderRadius.circular(12),
43            border: Border.all(
44              color: ColorsManager.verifyEmailContainerBorder,
45            ),
46          ),
47          child: Text(
48            email,
49            style: TextStylesManager.bold16.copyWith(
50              color: ColorsManager.primaryColor,
51            ),
52            textAlign: TextAlign.center,
53          ),
54        ),
55      ],
56    );
57  }
58 }
59

```

Lines • 68
Dart

Feature / Verify #382 -

Email lib/features/verify_email/presentation/widgets/verify_email_steps_widget.dart

```

1  import 'package:flutter/material.dart';
2  import 'package:moean/core/theme/colors.dart';
3  import 'package:moean/core/theme/text_styles.dart';
4  import 'package:moean/core/utils/constants/constants.dart';
5  import 'package:moean/core/utils/constants/spacing.dart';
6
7  class VerifyEmailStepsWidget extends StatelessWidget {
8    const VerifyEmailStepsWidget({super.key});
9
10   @override
11   Widget build(BuildContext context) {
12     return Container(
13       width: double.infinity,
14       padding: const EdgeInsets.all(18),
15       decoration: BoxDecoration(
16         color: ColorsManager.surfacePrimary,
17         borderRadius: BorderRadius.circular(16),

```



```

18     border: Border.all(
19       color: ColorsManager.verifyStepBorder,
20     ),
21   ),
22   child: Column(
23     crossAxisAlignment: CrossAxisAlignment.start,
24     children: [
25       Text(
26         appTranslation().get('verify_email_steps_title'),
27         style: TextStylesManager.bold14.copyWith(
28           color: ColorsManager.verifySuccessDark,
29         ),
30       ),
31       verticalSpace10,
32       Text(
33         appTranslation().get('verify_email_step_1'),
34         style: TextStylesManager.regular13.copyWith(
35           color: ColorsManager.verifyStepText,
36           height: 1.5,
37         ),
38       ),
39       verticalSpace8,
40       Text(
41         appTranslation().get('verify_email_step_2'),
42         style: TextStylesManager.regular13.copyWith(
43           color: ColorsManager.verifyStepText,
44           height: 1.5,
45         ),
46       ),
47       verticalSpace8,
48       Text(
49         appTranslation().get('verify_email_step_3'),
50         style: TextStylesManager.regular13.copyWith(
51           color: ColorsManager.verifyStepText,
52           height: 1.5,
53         ),
54       ),
55       verticalSpace8,
56       Text(
57         appTranslation().get('verify_email_step_4'),
58         style: TextStylesManager.regular13.copyWith(
59           color: ColorsManager.verifyStepText,
60           height: 1.5,
61         ),
62       ),
63     ],
64   ),
65 );
66 }
67 }
68

```

Lines • 134
Dart

Feature / Verify #383 -

Email lib/features/verify_email/presentation/widgets/verify_email_success_widget.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:moean/core/theme/colors.dart';
3 import 'package:moean/core/theme/text_styles.dart';
4 import 'package:moean/core/utils/constants/constants.dart';
5 import 'package:moean/core/utils/constants/primary/primary_elevated_button.dart';
6 import 'package:moean/core/utils/constants/routes.dart';
7 import 'package:moean/core/utils/constants/spacing.dart';
8
9 class VerifyEmailSuccessWidget extends StatelessWidget {
10   const VerifyEmailSuccessWidget({super.key});
11
12   @override
13   Widget build(BuildContext context) {
14     return Column(
15       mainAxisAlignment: MainAxisAlignment.min,
16       crossAxisAlignment: CrossAxisAlignment.center,
17       children: [
18         verticalSpace20,

```

```

19     Container(
20       width: 76,
21       height: 76,
22       decoration: BoxDecoration(
23         shape: BoxShape.circle,
24         color: ColorsManager.verifyBadgeBg,
25       ),
26       child: Center(
27         child: Container(
28           width: 38,
29           height: 38,
30           decoration: BoxDecoration(
31             shape: BoxShape.circle,
32             border: Border.all(
33               color: ColorsManager.primaryColor,
34               width: 2.5,
35             ),
36           ),
37           child: Center(
38             child: Icon(
39               Icons.check_rounded,
40               size: 22,
41               color: ColorsManager.primaryColor,
42             ),
43           ),
44         ),
45       ),
46     ),
47     verticalSpace20,
48     Container(
49       padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 6),
50       decoration: BoxDecoration(
51         color: ColorsManager.verifyBadgeBg,
52         borderRadius: BorderRadius.circular(20),
53       ),
54       child: Row(
55         mainAxisAlignment: MainAxisAlignment.min,
56         children: [
57           Icon(
58             Icons.auto_awesome_rounded,
59             size: 16,
60             color: ColorsManager.primaryColor,
61           ),
62           horizontalSpace6,
63           Text(
64             appTranslation().get('email_verified_badge'),
65             style: TextStyleManager.bold13.copyWith(
66               color: ColorsManager.primaryColor,
67             ),
68           ),
69         ],
70       ),
71     ),
72     verticalSpace16,
73     Text(
74       appTranslation().get('email_verified_success_title'),
75       style: TextStyleManager.bold24.copyWith(
76         color: ColorsManager.verifySuccessDark,
77       ),
78       textAlign: TextAlign.center,
79     ),
80     verticalSpace12,
81     Text(
82       appTranslation().get('email_verified_success_desc'),
83       style: TextStyleManager.regular14.copyWith(
84         color: ColorsManager.placeholder,
85         height: 1.6,
86       ),
87       textAlign: TextAlign.center,
88     ),
89     verticalSpace32,
90     PrimaryElevatedButton(
91       text: appTranslation().get('continue_to_platform'),
92       height: 52,
93       radius: 24,

```

```

94         backgroundColor: ColorsManager.primaryColor,
95         icon: const Icon(
96             Icons.arrow_back,
97             color: Colors.white,
98             size: 20,
99         ),
100         onPressed: () {
101             Navigator.pushNamedAndRemoveUntil(
102                 context,
103                 Routes.home,
104                 (route) => false,
105             );
106         },
107     ),
108     verticalSpace12,
109     PrimaryElevatedButton(
110         text: appTranslation().get('back_to_home'),
111         height: 52,
112         radius: 24,
113         backgroundColor: ColorsManager.surfacePrimary,
114         borderSide: BorderSide(
115             color: ColorsManager.borderColor,
116             width: 1.5,
117         ),
118         textStyle: TextStylesManager.bold14.copyWith(
119             color: ColorsManager.placeholder,
120         ),
121         onPressed: () {
122             Navigator.pushNamedAndRemoveUntil(
123                 context,
124                 Routes.home,
125                 (route) => false,
126             );
127         },
128     ),
129     verticalSpace20,
130 ],
131 );
132 }
133 }
134

```